

C/C++言語入門

第 0.6 版

Copyright © 2026, Katsunori Nakamura

中村勝則

2026 年 9 月 23 日

免責事項

本書の内容は参考資料であり、掲載したプログラムリストは全て試作品である。本書の使用に伴って発生した不利益、損害の一切の責任を筆者ならびに IDEJ 出版は負わない。

目次

第1部 C言語の基礎	1
1 はじめに	2
1.1 C言語の歴史	2
1.1.1 C++	2
1.2 プログラミングに関する前提知識	2
1.2.1 機械語, アセンブリ言語, コンパイラ	2
1.2.2 インタプリタ, 仮想機械	3
1.3 本書で使用するC言語処理系	3
1.3.1 GNU Project について	3
1.3.2 Linux と GCC	4
1.3.3 本書で使用するツール	4
2 C言語プログラムの基本構造	5
2.1 main 関数の型, 戻り値	6
2.2 プリプロセッサ	6
2.3 プログラムのコンパイルと実行	6
2.4 関数定義の順序とプロトタイプ宣言	7
2.5 関数への引数の受け渡しについて	8
2.6 リテラル	8
2.6.1 数値リテラルのサフィックス	9
2.6.2 数値リテラルのプレフィックス	9
2.6.3 文字列リテラル	11
2.6.4 定数のマクロ定義	11
2.7 識別子の命名規則	12
3 変数, 型, 数値	13
3.1 変数の宣言	13
3.2 値の代入	13
3.2.1 定数としての変数	13
3.2.2 複合代入	14
3.3 基本的な型	14
3.4 変数のスコープと記憶期間	15
3.4.1 静的変数	16
3.4.2 グローバル変数	17
3.5 数値	18
3.5.1 整数が扱える値の範囲	18
3.5.1.1 最大幅整数型	20
3.5.2 浮動小数点数が扱える値の範囲	20
3.5.2.1 特殊な値	21
3.5.3 算術演算	23
3.5.3.1 インクリメント, デクリメント	23
3.5.4 浮動小数点数の誤差	24
3.5.5 キャストによる型の変換	25
3.5.6 数学関数	26
3.5.6.1 丸めモードの設定	26
3.5.6.2 乱数の生成	27

3.5.7	複素数型	29
3.5.7.1	虚数単位	29
3.5.7.2	実部と虚部の取得	29
3.5.7.3	複素数用の数学関数	30
3.5.7.4	複素数の出力	30
3.6	配列	31
3.6.1	配列宣言時の初期化	32
3.6.2	配列全体を複製する方法	33
3.7	文字と文字列	33
3.7.1	文字コードのリテラル表現	34
3.7.2	文字列の長さ	34
3.8	書式文字列と変換指定子	35
3.8.1	エスケープシーケンス	37
3.9	型の別名の定義	38
4	制御構造	39
4.1	条件分岐	39
4.1.1	if 文	39
4.1.1.1	条件判定式	40
4.1.1.2	_Bool 型と bool	41
4.1.1.3	浮動小数点数の判定	42
4.1.1.4	複素数の等値性の判定	44
4.1.2	条件演算子による条件式	44
4.1.3	switch 文	45
4.2	反復	47
4.2.1	while 文	47
4.2.1.1	do 文	48
4.2.2	for 文	48
4.2.2.1	コンマ演算子	49
4.3	無条件ジャンプ	50
5	構造体	51
5.1	構造体の様々な定義方法	52
5.1.1	構造体型の別名の定義	53
5.1.2	構造体をメンバとして持つ構造体	53
5.1.3	関数内で定義するローカルな構造体型	54
5.2	構造体の代入と複製	54
5.2.1	構造体のメンバとしての配列の扱い	55
5.2.2	関数呼び出しに伴う構造体の複製	55
6	メモリとポインタ	57
6.1	ポインタ	58
6.2	ポインタを介した値の受け渡し	59
6.2.1	関数間での変数の間接的な共有	61
6.3	配列のポインタとしての扱い	62
6.3.1	配列名とポインタ	63
6.3.2	ポインタの加算	63
6.3.3	配列とポインタの違い	64
6.3.3.1	関数定義の仮引数の書き方	64

6.4	ヌルポインタと NULL	65
7	入出力	67
7.1	ストリーム：入出力の対象	67
7.2	標準出力, 標準エラー出力	67
7.2.1	出力のリダイレクト	68
7.3	標準入力	69
7.3.1	入力のリダイレクト	70
7.3.2	入力用の様々な関数	70
7.3.2.1	文字列の読み込み	70
7.3.2.2	書式指定の読み込み：scanf	73
7.4	ファイル入出力	74
7.4.1	ファイルのオープンとクローズ	74
7.4.2	テキストファイルとバイナリファイル	74
7.4.3	ファイルへの出力	75
7.4.3.1	出力バッファの強制書き出し：fflush	76
7.4.4	ファイルからの入力	77
7.4.5	バイナリデータの扱い	79
7.4.6	既存のファイルの変更	81
7.4.7	ファイルのアクセス位置の操作	83
7.4.7.1	ファイルのアクセス位置の移動：fseek	83
7.4.7.2	アクセス位置の取得：ftell	84
7.4.7.3	ファイル先頭への移動：rewind	85
7.4.8	ファイルアクセスの際のエラーの確認：ferror	87
7.5	低水準入出力	88
7.5.1	低水準入出力の基本概念	88
7.5.2	ファイルのオープン：open	89
7.5.2.1	フラグとモード	89
7.5.3	書き込み：write	90
7.5.3.1	出力の確定処理	91
7.5.4	読み込み：read	91
7.5.5	ファイルのクローズ：close	92
7.5.6	入出力の効率化	93
8	分割コンパイルとプリプロセッサ	94
8.1	分割コンパイルのための基礎事項	94
8.1.1	extern による共有	96
8.1.2	static 宣言による共有範囲の制限	97
8.2	プリプロセッサ	98
8.2.1	ヘッダファイルの読み込み（#include）	98
8.2.2	定数マクロ（#define）	98
8.2.2.1	条件コンパイルへの応用	99
8.2.2.2	インクルードガード	100
8.2.2.3	処理環境に関する定数	101
9	実用的なプログラミングに必要な事柄	102
9.1	関数の再帰的定義	102
9.1.1	関数呼び出しにおけるメモリの管理の仕組み	103
9.1.2	関数の再帰呼び出しに関する問題点	104

9.1.2.1	スタックのサイズに関すること	104
9.1.2.2	計算規模に関すること	105
9.1.3	複雑な問題を再帰的関数で解決する例	107
9.2	関数定義の高度な方法	109
9.2.1	可変長の引数	109
9.2.1.1	受け取った可変長引数の個数について	111
9.2.1.2	受け取った可変長引数の型について	111
9.2.2	関数のポインタ	111
9.2.2.1	関数ポインタの配列	112
9.3	ビット演算	114
9.3.1	ビット演算の実用的な応用	115
9.3.2	ビットフィールド	117
9.3.2.1	C 言語の標準として使用できるビットフィールドの型	118
9.3.2.2	ビットフィールド使用上の注意	120
9.4	共用体	121
9.5	データのバイト順 (バイトオーダー) に関すること	124
9.5.1	バイト順の反転	124
9.5.2	ホスト/ネットワークバイト順変換関数	125
9.6	文字列操作関数	127
9.6.1	文字列の長さの取得: strlen	127
9.6.2	文字列の複製: strcpy	127
9.6.3	文字列の比較: strcmp	127
9.6.4	文字列の連結: strcat	127
9.6.5	文字の検索: strchr	127
9.6.6	文字列の検索: strstr	128
9.6.7	書式整形結果の文字列の作成: sprintf/snprintf	128
9.7	文字列解析関数	130
9.7.1	文字列からの値の読み取り: sscanf	130
9.7.2	文字列を整数値に変換する関数: strtol / strtoll / strtoul / strtoull	130
9.7.3	文字列を浮動小数点数に変換する関数: strtod / strtod / strtold	131
9.8	メモリの柔軟な取扱い	134
9.8.1	メモリの初期化	135
9.8.2	メモリの複製 (コピー)	136
9.8.3	メモリの比較	138
9.9	プログラム実行のためのメモリモデル	140
9.9.1	ヒープ領域の利用	141
9.10	列挙型: 名前付き整数の扱い	144
9.10.1	列挙子の値の割当てに関する注意点	144
9.10.2	列挙型を使用する目的に関すること	145
9.11	シグナル処理とジャンプ	147
9.11.1	シグナル処理の仕組みと流れ	147
9.11.2	シグナル処理関数: signal	148
9.11.3	シグナル処理のための変数	148
9.11.4	ハンドラからの強制的な制御移動	150
9.11.5	シグナルを他のプログラムに通知する方法	152
9.11.6	使用頻度の高いシグナル	152
9.12	日付, 時刻の扱い	154
9.12.1	UNIX 時間	154

9.12.2	時間を取り扱うための基本的な機能	154
9.12.2.1	日付, 時刻の書式整形	155
9.12.2.2	日付, 時刻の文字列からの読取り	156
9.12.3	高精度の時間計測	158
9.13	プログラムの一時停止	159
9.13.1	POSIX 環境での方法	159
9.13.2	Windows 環境での方法	160
9.13.3	待ち時間を実現する際の推奨されない方法	161
9.14	コマンドライン引数	163
9.14.1	main 関数の引数	163
9.14.2	コマンドラインの記述に関すること	163
9.15	プログラムの終了処理	166
9.15.1	exit 関数によるプログラムの終了	166
9.15.2	abort 関数によるプログラムの異常終了	168
第 2 部	C++	170
10	C++とは	171
11	C++の基礎	173
11.1	main 関数の記述の形式	173
11.2	クラスとオブジェクトに関する予備知識	173
11.3	C++の基本的なデータ型	173
11.3.1	変数宣言時の初期化	174
11.3.2	C++での複素数の扱い	174
11.4	入出力	175
11.4.1	iostream	175
11.4.2	書式制御の方法 (C 言語の変換指定子との違い)	175
11.4.2.1	桁数, 精度の設定	175
11.4.2.2	左右のアラインメントと埋め文字	176
11.4.3	ファイルストリーム	177
11.4.3.1	出力ファイルストリーム (ofstream)	177
11.4.3.2	入力ファイルストリーム (ifstream)	178
11.4.3.3	入出力ファイルストリーム (fstream)	179
11.4.3.4	バイナリファイルの扱い	179
11.4.3.5	入出力位置の制御	180
11.4.3.6	ファイルの一括読み込み	183
11.4.4	ストリームに対する各種の関数	184
11.4.4.1	ストリーム状態の検査	184
11.4.4.2	ストリーム状態の操作	185
11.4.4.3	入力補助関数	186
11.4.4.4	バイナリ入出力に関わる関数	187
11.4.4.5	その他の機能	188
11.4.5	C 言語の stdio の利用	189
11.5	名前空間	190
11.5.1	名前空間の指定	190
11.5.2	主な名前空間	190
11.5.3	独自の名前空間の定義	191
11.6	C++における型の考え方	192

11.6.1	論理型 (bool 型)	192
11.6.1.1	bool 値の入出力時の形式	193
11.6.2	特殊な数値	195
11.6.3	バイトを表す型 (byte 型)	196
11.6.4	C++の標準文字列クラス	197
11.6.4.1	string オブジェクトとして扱えるデータサイズ	197
11.6.4.2	文字列へ出力するストリーム: ostream	198
11.6.4.3	文字列から入力するストリーム: istream	199
11.6.5	明示的なヌルポインタ: nullptr	201
11.6.6	型名としての構造体名	202
11.6.7	型変換	202
11.6.7.1	static_cast	202
11.6.7.2	const_cast	203
11.6.7.3	reinterpret_cast	203
11.6.7.4	型で意味を表す設計	204
11.7	参照	204
11.7.1	関数呼び出しにおける参照の応用	204
11.7.1.1	構造体を参照で渡すケース	206
11.7.1.2	参照を返す関数	207
11.7.2	一時オブジェクトと const 参照	209
11.7.3	参照とポインタの違い	210
11.8	関数のオーバーロード	211
11.8.1	オーバーロードの設計上の注意	212
11.8.1.1	引数が参照である場合	213
11.8.2	デフォルト引数	213
11.9	auto による型推論	215
11.10	範囲に基づく for 文	217
11.10.1	要素を参照として受け取る場合	218
11.11	コンパイル時定数: constexpr	219
11.12	値のカテゴリ	220
11.13	例外処理	221
11.13.1	例外として扱われる事象	221
11.13.2	例外のハンドリング	222
11.13.3	例外の送出	222
11.13.4	noexcept による例外伝播の抑止	224
11.13.5	回復不能な異常と終了処理	225
11.14	日付や時間に関する機能	228
11.14.1	C 言語由来の日付, 時刻の処理	228
11.14.2	日付, 時刻の文字列変換	229
11.14.3	時刻点と時間間隔	230
11.15	C 言語標準ライブラリの利用	233
11.15.1	C 言語由来の標準ライブラリ	233
11.15.2	C++用ヘッダの読み込み	233
11.15.3	std 名前空間からの利用	234
11.15.4	拡張子.hを持つヘッダとの違い	235
11.15.5	C++で利用する際の注意	235

12 クラスとオブジェクト	237
12.1 クラスの定義	237
12.2 オブジェクトの独立性	238
12.3 アクセス指定	239
12.3.1 public と private	239
12.4 コンストラクタ	239
12.4.1 コンストラクタのオーバーロード	241
12.4.2 初期化の際の丸括弧と波括弧	241
12.4.3 暗黙の型変換を禁止する explicit 宣言	243
12.4.4 コピーコンストラクタ	243
12.4.5 単純なバイトコピーが可能な型: trivially copyable	244
12.5 デストラクタ	245
12.6 this ポインタ	246
12.7 const メンバ関数と mutable 指定	247
12.8 演算子のオーバーロード	248
12.8.1 複合代入演算子のオーバーロード	249
12.8.2 異なる型の値との演算とその実装	250
12.8.2.1 friend 関数	251
12.8.3 関数のように呼び出せるオブジェクト: operator()	251
12.9 カプセル化	253
12.9.1 不変条件 (インバリアント) の維持	253
12.9.2 インターフェースと実装の分離	254
12.10 メンバの静的宣言	255
12.11 friend によるクラス間の特別なアクセス	257
13 継承とポリモーフィズム	259
13.1 継承とは何か	259
13.2 継承の基本構文	260
13.3 基底クラスと派生クラス	261
13.4 メンバの継承とアクセス制御	261
13.5 コンストラクタとデストラクタの呼び出し順序	263
13.6 関数の再定義と名前の隠蔽	266
13.7 仮想関数と動的束縛	267
13.8 ポリモーフィズム	268
13.8.1 オブジェクトスライシング	270
13.9 抽象クラスと純粋仮想関数	271
13.10 仮想デストラクタ	272
13.11 継承設計に関する注意	276
13.12 C++におけるオブジェクトとオブジェクト指向プログラミング	276
14 オブジェクトの寿命と所有権	278
14.1 オブジェクトの寿命	278
14.1.1 オブジェクトの生成と破棄	278
14.1.2 スコープと寿命	279
14.1.3 関数から戻る際のオブジェクトの破棄	279
14.1.4 一時オブジェクトの寿命	279
14.1.5 静的記憶期間を持つオブジェクト	280
14.1.6 動的に生成されたオブジェクトの寿命	280

14.1.7	寿命を終えたオブジェクトへのアクセス	280
14.2	オブジェクトの所有権	282
14.2.1	所有と非所有	282
14.2.1.1	ポインタと所有権	282
14.2.1.2	所有権が不明確なポインタ	283
14.2.1.3	参照による非所有の利用	283
14.2.1.4	関数の引数と所有権	284
14.2.2	単独所有と共有所有	284
14.2.2.1	単独所有	284
14.2.2.2	共有所有	285
14.2.2.3	単独所有と共有所有の使い分け	285
14.2.3	所有権の移動	285
14.2.3.1	通常のポインタによる所有権の移動	286
14.2.3.2	コピーと移動の違い	286
14.2.3.3	移動後のオブジェクト	286
14.2.3.4	所有権の移動と単独所有	287
14.3	RAII	287
14.3.1	RAII による資源管理	287
14.3.2	複数の終了経路と RAII	288
14.3.3	ファイルの管理と RAII	288
14.3.4	標準ライブラリと RAII	289
14.3.5	RAII を利用する際の基本方針	289
14.4	スマートポインタ	290
14.4.1	単独所有のための unique_ptr	290
14.4.1.1	unique_ptr による所有権の移動	291
14.4.1.2	unique_ptr を関数へ渡す方法	291
14.4.2	共有所有のための shared_ptr	293
14.4.2.1	shared_ptr による所有権の解放	293
14.4.3	非所有の参照を表す weak_ptr	295
14.4.4	共有所有における循環参照	297
14.4.5	スマートポインタを選ぶ際の基本方針	298
14.5	Rule of Three/Five/Zero	298
14.5.1	特殊メンバ関数	298
14.5.2	Rule of Three	299
14.5.3	コピーを禁止する方法	301
14.5.4	Rule of Five	301
14.5.5	Rule of Zero	305
14.5.6	Rule of Three/Five/Zero の使い分け	305
15	クラス設計の実践	306
15.1	基本的なクラスの構成	306
15.2	小さなデータを表すクラス (値オブジェクト)	308
15.3	リソース管理のためのクラス設計	309
15.4	抽象クラスによるインターフェース設計	311
15.5	ポリモーフィズムを前提とした設計	313
15.6	コンポジションによる設計	315
15.7	よくある設計上の問題	317
15.8	設計指針のまとめ	318

16 型を抽象化するための機能	322
16.1 関数テンプレート	322
16.1.1 テンプレートにおける転送参照	324
16.2 クラステンプレート	326
16.3 値を抽象化するテンプレート	329
16.3.1 型の別名を定義する方法	331
16.4 変数テンプレート	334
16.5 型安全な列挙型：enum class	335
17 ラムダ式	337
17.1 ラムダ式の基本	337
17.1.1 ラムダ式の即時呼び出し	338
17.2 外部の変数の利用：キャプチャ	339
17.3 ラムダ式の実体	340
18 並行処理の基礎	342
18.1 並行処理の実行単位：thread	342
18.2 スレッドの終了待ち：join	343
18.3 スレッドの切り離し：detach	345
18.4 スレッドの休止：sleep_for	346
18.5 共有データとデータ競合	347
18.5.1 データ競合が発生する例	348
18.5.2 mutex による排他制御	350
18.5.2.1 lock_guard による安全なロック管理	352
18.5.3 atomic による単純な共有値の操作	353
19 プログラムの構成と分割	357
19.1 クラスの宣言とメンバ関数の定義	357
19.1.1 class 文の外部におけるメンバ関数の定義	358
19.1.2 宣言と定義の一致	358
19.2 クラス定義と実装のファイル分割	360
19.2.1 クラス定義とメンバ関数定義の分離	360
19.2.2 ヘッダファイルを記述する際の C++固有の注意	362
19.3 名前空間に属する関数やメンバ関数の定義	363
19.4 ヘッダファイルに定義を記述する場合	365
19.4.1 inline 関数	365
19.4.2 クラス定義内で定義されたメンバ関数	366
19.4.3 inline 変数	367
19.4.4 定義の重複に関する注意	367
19.5 静的データメンバの定義	369
19.6 テンプレートの定義とファイル分割	372
19.6.1 テンプレートの定義を利用側から参照できる必要性	372
19.6.2 テンプレート定義の別ファイル化	373
19.6.3 明示的インスタンス化	374
19.7 C 言語のプログラムとの連携	377
19.7.1 C と C++のリンケージの違い	377
19.7.2 extern "C"	378
19.7.3 C と C++の双方から利用する宣言	378
19.7.4 C と C++のオブジェクトファイルのリンク	379

20 STLの基礎	384
20.1 要素数を変更できる配列：vector	384
20.1.1 生成時の初期化	385
20.2 イテレータ	386
20.2.1 要素を書き換えないイテレータ：const_iterator	388
20.3 要素数が固定された配列：array	390
20.4 双方向リスト：list	391
20.5 両端キューのコンテナ：deque	393
20.6 キーと値の対応を管理するコンテナ：map	394
20.6.1 順序を管理しない高速な連想コンテナ：unordered_map	396
20.7 重複のない値を管理するコンテナ：set	397
20.7.1 順序を管理しない高速な集合コンテナ：unordered_set	398
20.8 主なコンテナに対して使用できるメンバ関数	400
20.8.1 STL が提供するアルゴリズム	401
20.8.2 コンテナ間の変換処理	403
21 string による文字列処理	407
21.1 文字と文字列の検索	407
21.2 文字列の分割	409
21.3 文字の種類の判定と変換	410
21.4 日本語文字列と文字コードに関する注意	412
21.5 正規表現による文字列処理	414
21.5.1 正規表現の基本的な要素	416
21.5.2 正規表現に一致する箇所の全検索	417
22 数値データの処理	419
22.1 乱数	419
22.1.1 乱数エンジン	419
22.1.2 一様乱数	422
22.1.2.1 整数の一様乱数	422
22.1.2.2 浮動小数点数の一様乱数	423
22.1.3 正規分布に従う乱数	424
22.1.4 様々な乱数分布	425
22.2 数値列の集計：numeric	426
22.2.1 総和と平均	426
22.2.2 分散と標準偏差	427
22.2.3 内積と累積和	429
22.2.4 連番の生成	431
22.3 複素数の扱い：complex	433
22.3.1 実部、虚部	433
22.3.2 基本的な関数	434
22.3.3 複素数に対応する数学関数	435
22.4 多倍長精度演算	437
22.4.1 GMP	438
22.4.1.1 GMP の主な型	438
22.4.1.2 GMP の主な関数	439
22.4.1.3 多倍長浮動小数点数の精度の設定と初期化	442

22.4.1.4	mpf_t の数学的な演算のための関数	443
22.4.1.5	比較, 判定のための関数	444
22.4.1.6	mpf_t の値の近さの判定	446
22.4.1.7	書式整形と出力	447
22.4.1.8	サンプルプログラム (1): 多倍長整数	449
22.4.1.9	サンプルプログラム (2): 多倍長有理数	450
22.4.1.10	サンプルプログラム (3): 多倍長浮動小数点数	451
22.4.1.11	mpz_t 型の乱数	451
22.4.1.12	C++における GMP の利用	454
22.4.2	MPFR	463
22.4.2.1	MPFR の主な型	463
22.4.2.2	オブジェクトの管理のための関数	465
22.4.2.3	値の設定と取得のための関数	466
22.4.2.4	演算のための関数 (1): 基本的な演算	468
22.4.2.5	比較, 判定のための関数	469
22.4.2.6	mpfr_t の値の近さの判定	470
22.4.2.7	整数値への丸めに関する関数	473
22.4.2.8	演算のための関数 (2): 累乗・平方根など	474
22.4.2.9	演算のための関数 (3): 指数関数, 対数関数	476
22.4.2.10	演算のための関数 (4): 三角関数	477
22.4.2.11	演算のための関数 (5): 双曲線関数	479
22.4.2.12	演算のための関数 (6): 特殊関数, 定数	480
22.4.2.13	MPFR のキャッシュの解放	482
22.4.2.14	mpfr_t の特殊な値	483
22.4.2.15	書式整形や入出力のための関数	484
22.4.2.16	mpfr_t 型の乱数	486
22.4.2.17	10 進桁数による精度の指定	489
22.4.3	MPC	491
22.4.3.1	MPC の型とオブジェクトの管理	492
22.4.3.2	値の設定と取得	493
22.4.3.3	MPC の丸め方法	495
22.4.3.4	実部, 虚部, 絶対値, 偏角	495
22.4.3.5	基本的な演算	496
22.4.3.6	累乗, 指数関数, 対数関数	498
22.4.3.7	三角関数, 双曲線関数	499
22.4.3.8	書式整形と入出力	501
22.4.3.9	サンプルプログラム: 1 の n 乗根	503
第 4 部 付録		505
A 定義済みマクロ		506
A.1	言語モードと規格に関するマクロ	506
A.2	コンパイラとターゲット環境に関するマクロ	506
A.3	ホスト実装とフリースタANDING実装	507
B サンプルプログラム		511
B.1	整数を 2 進数表記の文字列に変換する機能	511
B.2	高精度の時間計測 (C 言語用)	514

C	コンパイラと開発環境	517
C.1	GCC	517
C.1.1	基本的なコマンド書式	517
C.1.2	主なコンパイラオプション	518
C.1.3	最適化水準	519
C.1.4	文字のエンコーディングの指定	519
C.1.5	実行時のスタックサイズの指定	520
C.2	Clang / LLVM	522
C.2.1	LLVM IR とコンパイラの構成	522
C.2.2	clang と clang++ の基本的な使用方法	523
C.2.3	主なコンパイラオプション	524
C.3	MSVC (Microsoft Visual C++)	526
C.3.1	必要となるツールの導入	526
C.3.2	MSVC を使用するためのコマンド環境の起動	526
C.3.3	cl コマンド	527
C.3.4	主なコンパイラオプション	528
C.3.5	文字のエンコーディングの指定	529
C.3.6	実行時のスタックサイズの指定	530
C.3.7	開発ツールのアップデート	531
C.4	MSYS2 による Windows 開発環境	532
C.4.1	MSYS2 のインストール	532
C.4.2	MSYS2 の起動	532
C.4.3	開発ツールのインストール	533
C.4.4	パッケージの管理	534
D	ソフトウェア構築の支援ツール	535
D.1	make	537
D.1.1	基本的な使用方法	537
D.1.2	ソースファイルの変更に対する make の働き	540
D.1.3	擬似ターゲット	541
D.2	CMake	542
D.2.1	基本的な使用方法	543
D.2.2	生成物の格納位置の設定	547
D.3	Ninja	548
D.3.1	CMake による build.ninja ファイルの生成	549
D.3.2	Ninja による構築	549
E	動的ライブラリの構築	551
E.1	動的ライブラリの基本的な仕組み	551
E.1.1	公開シンボルと利用側プログラムとの結合	552
E.1.2	起動時の読み込みと実行中の読み込み	552
E.2	公開インターフェースの設計	552
E.2.1	公開用ヘッダファイルと C ABI	553
E.2.2	公開指定のためのマクロ	553
E.3	例) 数値集計ライブラリの実装	554
E.3.1	C によるライブラリの実装	555
E.3.2	C++ によるライブラリの実装	556
E.3.3	ライブラリを利用するプログラム	557

E.4	環境別の動的ライブラリ構築	558
E.4.1	Linux で GCC を使用する場合	558
E.4.2	Linux で Clang を使用する場合	559
E.4.3	macOS で Clang を使用する場合	559
E.4.4	Windows の MSYS2 UCRT64 環境で GCC を使用する場合	560
E.4.5	Windows の MSYS2 CLANG64 環境で Clang を使用する場合	560
E.4.6	Windows で MSVC を使用する場合	561
E.5	公開インターフェース設計上の注意	562
E.5.1	ABI 互換性の維持	562
E.5.2	メモリの管理とオブジェクトの寿命	562
E.5.3	C++ 例外の扱い	563
F	gnuplot	564
F.1	データファイルの形式	565
F.1.1	データファイルの区切り文字の指定	565
F.2	描画に関する各種のコマンド	567
F.2.1	グラフの色の指定	568
F.2.2	線の太さの指定	569
F.2.3	線種の指定	569
F.3	描画結果の画像ファイルへの保存	570
F.4	ファイルに記述されたコマンドの実行	571
F.5	グラフに日本語を使用する際の注意事項	572
F.6	各種のグラフ	573
F.6.1	マーカープロット (点グラフ)	573
F.6.2	棒グラフ	574
F.6.3	散布図	576
F.6.4	ヒストグラム	578
F.6.5	3次元プロット (ワイヤーフレーム)	581
F.6.6	ヒートマップ	583
F.6.7	等高線	585

第1部

C言語の基礎

1 はじめに

1.1 C 言語の歴史

C 言語は、1960 年代後半から 70 年代にかけて米 AT&T がオペレーティングシステム (UNIX) を開発する中で生まれた。開発の初期段階ではアセンブリ言語でプログラムを記述していたが、後に、記述の効率を高めるために C 言語が生まれ、その後 UNIX の大部分が C 言語によって記述された。

初期の C 言語は、ベル研究所の Dennis Ritchie によって、Ken Thompson が開発した B 言語などを基礎として開発された。その後、Brian Kernighan と Dennis Ritchie は、C 言語を体系的に解説した書籍「The C Programming Language」を 1978 年に出版した。この書籍は著者名の頭文字から「K&R」と呼ばれ、当時の C 言語の事実上の基準として広く用いられた。更にその後、C 言語の仕様が整備され、ANSI X3.159-1989 として標準化された。この C 言語の仕様は「ANSI C」や「C89」と呼ばれて普及した。またこの規格はその後、ISO/IEC 9899:1990 として国際標準化され、「C90」と呼ばれた。その後 C 言語は、「C95」、「C99」、「C11」、「C17」、「C23」とアップデートされた。

1.1.1 C++

ベル研究所の Bjarne Stroustrup は、1979 年ごろから C 言語にクラスなどの機能を導入した C with Classes の開発を始めた。これを更に発展させた言語が、1983 年に C++ と名づけられた。その後、C++ は 1998 年に ISO/IEC 14882:1998 として国際標準化された。これは「C++98」と呼ばれる。その後 C++ は「C++03」、「C++11」、「C++14」、「C++17」、「C++20」、「C++23」とアップデートされた。

C++ は C 言語の基盤の上に成り立っているが、単純な上位互換ではない。言語仕様や型体系、抽象化の機構において大きな拡張が加えられており、現在では C 言語とは独立した別の言語として位置づけられる。

1.2 プログラミングに関する前提知識

C 言語は機械語へ翻訳される言語である。その理解のためには、まずコンピュータがどのようにプログラムを実行しているのかを確認しておく必要がある。

1.2.1 機械語、アセンブリ言語、コンパイラ

コンピュータの CPU (中央演算処理装置) は、それが解釈できる機械語で記述されたプログラムに沿って動作する。CPU の内部構造は、その各部の仕様と役割 (アーキテクチャ) によって理解されるものであり、1 つのアーキテクチャは 1 揃いの機械語の体系 (命令セット) を持つ。

CPU のアーキテクチャには、x86-64 や Arm など複数の種類が存在しており、それぞれに対応する命令セットが定められている。機械語の個々の命令 (インストラクション) は 2 進数の値として表される。ただし、人間が機械語の命令に対応する処理を記述する場合には、通常、2 進数を直接記述するのではなく、**ニモニック**と呼ばれる略記号を用いる。このようなニモニックを用いて記述する言語を**アセンブリ言語**と呼ぶ。アセンブリ言語で記述されたプログラムは、**アセンブラ**と呼ばれるソフトウェアによって 2 進数表現の機械語プログラムに変換され、当該 CPU で実行することが可能になる。

2 進数表現の機械語やアセンブリ言語 (いわゆる**低水準言語**¹) は、人間にとって学習と運用のための労力が大きくなるので、現実的には各種の**高水準言語**でプログラム (ソースプログラム) を記述して、**コンパイラ**と呼ばれるソフトウェアでそれを機械語のプログラムに変換 (翻訳) して実行する。この変換はプログラムの実行前に完了しており、生成された機械語プログラムは CPU によって直接実行される。

高水準言語は、人間にとって理解しやすい言語構造 (英単語由来) を持ち、テキスト形式 (人間に可読な文字列) で記述する。

¹本書では、機械語に逐一对応する命令の記述体系という意味で**低水準言語**という用語を用いる。

1.2.2 インタプリタ, 仮想機械

インタプリタとは、高水準言語で記述されたソースプログラムを入力として受け取り、その意味を逐次解釈しながら処理を進めるソフトウェアである。ここでいう「逐次解釈」とは、あらかじめ全体を機械語へ翻訳することなく、処理の進行に応じて意味を評価して動作することを指す。例えば、Python 言語で記述されたプログラムを CPython で実行する場合、ソースプログラムは、まず CPython が扱うバイトコードを含む内部形式に変換される。その後、CPython の実行機構がその命令を順次処理する。このため、CPython は一般にインタプリタ型の言語処理系に分類される。OS 配下で実際に動いているのは CPython 処理系 (python.exe) であり、厳密には Python で記述されたプログラムが OS 配下で直接実行されているわけではない。

プログラミング言語の実行に用いられる**仮想機械**は、特定の物理 CPU に直接対応するものではない独自の命令体系を定め、その命令を実行するソフトウェア上の処理系である。例えば、Java 言語で記述したソースプログラムを javac コマンドで翻訳すると、JVM (Java 仮想機械) が実行できるバイトコードに変換される。そのバイトコードは java コマンドによって JVM 上で実行される。この場合、実際に OS 上で動作しているのは JVM 処理系であり、Java プログラムそのものが直接 CPU で実行されているわけではない²。

以上の整理を踏まえると、C 言語の位置づけが明確になる。本書で扱う一般的な C/C++ 処理系では、ソースプログラムは、CPU が直接実行できるネイティブ機械語を含む実行形式へ変換される。従って、通常は、実行時にインタプリタや仮想機械を介在させることなく、生成された実行形式ファイルを OS がメモリへロードし、CPU による実行を開始する。

1.3 本書で使用する C 言語処理系

本書では、移植性と実用性を重視し、標準規格に準拠したコンパイラを用いる。本書で前提とする C/C++ コンパイラは GCC (GNU Compiler Collection) である。GCC は各種 CPU のアーキテクチャに対応しており、高度に最適化された機械語を生成する。また、このコンパイラは各種 OS に移植されており、マルチプラットフォームに対応している。

GCC の最初の版は 1987 年に公開された。当初の名称は GNU C Compiler であり、C 言語専用のコンパイラであった。ここから「GCC」という略称が生まれた。GCC は、ソースコードが公開され、多様なハードウェアへ移植でき、無償で利用できるコンパイラとして大きな役割を果たした。また、実用的な最適化機能を備えていたことから、広く普及した。

やがて GCC は C だけでなく、他の言語にも対応するようになった。これに伴い、名称は「GNU C Compiler」から「GNU Compiler Collection」に改められた。現在の GCC は単一言語のコンパイラではなく、複数言語を扱う統合的なコンパイラ群である。

◎本書では、C 言語の解説は C17、C++ の解説は C++17 の規格に基づく。

1.3.1 GNU Project について

GCC の開発は、Richard Stallman が 1983 年に構想を発表した GNU Project の活動に端を発する。当時、MIT 人工知能研究所に所属していた Stallman は、ソフトウェアが企業によって非公開化され、改変や共有が制限されていく状況に強い危機感を抱いた。特に、研究コミュニティにおいてソースコードの共有文化が失われつつあったことが直接の動機となった。

彼は「自由 (free) なソフトウェア」によって構成された完全な UNIX 互換 OS を作ることを決意する。これが GNU Project である。GNU とは "GNU's Not Unix" の略であり、「UNIX ではないが UNIX と互換である」という意味を持つ再帰的頭字語である。

1985 年には Free Software Foundation (FSF) が設立され、GNU Project を法的、財政的に支える体制が整えられた。ここで提唱された「フリーソフトウェア」の理念は、後のオープンソース運動にも大きな影響を与えることになる。

²参考) JVM の実装によっては、実行中にバイトコードの一部をネイティブ機械語へ変換して実行することもある。

1.3.2 Linux と GCC

1991 年に Linus Torvalds が開発を開始した Linux は、GNU のツール群（特に GCC）を用いてビルドされた。この結果、Linux カーネルと、GCC を含む GNU のツール群などが組み合わせられ、現在一般に「Linux」と呼ばれる OS 環境³ が広く利用されるようになった。GNU Project は当初、独自カーネル（Hurd）を開発していたが、現在では、オープンソースの UNIX 系 OS のカーネルは Linux が主流である。こうして GCC は、Linux の発展とともに世界的に普及した。また GCC は、単なるコンパイラではなく、現代の UNIX 系ソフトウェア基盤を支える中核的存在となった。

1.3.3 本書で使用するツール

本書では、基本的に GCC を用いてサンプルプログラムのコンパイルと実行を行うが、扱う内容の多くは標準規格に基づいているため、Clang や MSVC（Microsoft Visual C++）など、他の C/C++ コンパイラを使用して試すこともできる。

ただし、コンパイラごとに、対応している言語規格や拡張機能、警告の内容、コマンドラインオプション、標準ライブラリの実装などに違いがある。このため、同じソースプログラムであっても、使用するコンパイラや実行環境によっては、修正や設定の変更が必要となる場合があるので注意すること。

各コンパイラの特徴や基本的な使用方法については、巻末付録「C コンパイラと開発環境」（p.517～）を参照されたい。また、MSVC の基本的な導入方法や使用方法については、「C.3 MSVC（Microsoft Visual C++）」（p.526～）で解説する。

³GNU/Linux と呼ばれることがある。

2 C言語プログラムの基本構造

C 言語では、プログラムを関数という単位で記述して定義する。関数内には文を書き並べる。式はその評価の結果として値をもたらすものであり、他の計算式や条件判定式の中を含むことができる。

例として、次のようなプログラム test001.c を例示する。

プログラム：test001.c

```

1 #include      <stdio.h>           // 標準入出力関連の定義ファイルの読み込み
2
3 int main(void) {                  // これがOSから起動される関数
4     printf("Hello, world.\n");    // 標準出力（ターミナル）への出力
5     return 0;                    // この関数の戻り値
6 }

```

これは、ターミナルウィンドウに "Hello, world." と出力するだけのプログラムである。プログラム中に `main` という関数が定義されており、このプログラムが起動された際にこれが実行される。`main` 関数の中には、標準出力に対して書式指定出力を行うための `printf` 関数の呼び出しが記述されており、これが、ターミナルウィンドウに対する出力処理 (`\n` は改行処理を行うエスケープシーケンス⁴) を行う。このように、1つの関数から他の関数が次々と呼び出される形でプログラムが実行される。

■ プログラム中のコメント

プログラム中にはコメントを書くことができ、この部分はプログラムとは解釈されない。コメントは説明などを記述するために用いられる。ソースプログラム中にスラッシュを2つ続けて `//` と書くと、それ以降は行末までがコメントとなる。また、`/* ... */` で括った部分もコメントとなる。この場合は `/*` から `*/` までの間に改行を含めることができ、複数行に渡るコメントを記述することができる。ただし、引用符 `"..."` で括られた範囲はデータとしての文字列であり、この中に `//`、`/*`、`*/` を含めても、それらはコメントではなく、文字列として扱われることに注意すること。また、`/* ... */` のネスト（コメントの入れ子）はできない点にも注意すること。

C 言語のソースプログラムでは、インデント（字下げ）を適用して可読性を高めることが一般的である。ただし、インデントはプログラムとしての意味を持たない⁵。また、式や文の意味を壊さない範囲で空白や改行も自由に挿入することができる。

■ ソースプログラムのファイル名

C 言語のソースプログラムのファイル名については特別な規定はないが、拡張子 *.c を付ける習慣がある。

■ 文の終端子, ブロック

基本的には、プログラムの各文の終端にはセミコロン「;」（終端子：terminator）を書き⁶，文は記述した順番に（上から順に）実行される．また，文のまとまりはブロックとして { } で括る．ブロックの終端にはセミコロンは書かない．何も文を書かずにセミコロンのみを書いた場合，それは空文とみなされ，何も実行されない．

プログラムの最終盤には `return` 文が記述されており、`main` 関数の終了時に呼び出し元（通常は OS）に 0 という値が返される。関数定義の形式は次の通り。

《関数定義》

```
書き方: 戻り値の型 関数名 ( 型 1 引数 1, 型 2 引数 2, ... ) {
                                     (処理の記述)
                                     return 戻り値;
}
```

「関数名」の関数を定義する．この関数を呼び出す際には，呼び出し側で**実引数**の並び（arguments）を与える．またそれらは，この関数の**仮引数**「引数 1」，「引数 2」…（parameters）に受け取られる．引数の型は「型 1」，「型 2」，… で指定される．

⁴後の「3.8.1 エスケープシーケンス」(p.37)で解説する。

⁵参考) Python や Ruby ではプログラムのインデントが文法的な意味を持つ。

⁶セミコロンを終端に書かない構文もある。

関数は終了時に「戻り値の型」の「戻り値」を呼び出し元に返す。戻り値を返さない関数を定義する場合は、「戻り値の型」は `void` 型とする。 `void` 型の関数内では `return` 文を記述する必要はないが、記述する場合は `'return;'` の形で値を書かない `return` 文とする。また、 `void` 型以外の関数内に `return` 文の記述が無い場合、終了時の戻り値は不定となる。(未定義動作)

本書では、関数の「戻り値の型」のことを単に**関数の型**と呼ぶことがある。

引数を取らない関数を定義する場合、仮引数の括弧内に何も書かず「**関数名 ()**」と記述することもできる。しかしこの書き方は「引数が未指定」であることを意味する。引数を取らないことを明示するには「**関数名 (void)**」と記述するのが正しい形式である。これは `main` 関数においても同様である⁷。

2.1 main 関数の型, 戻り値

プログラムの実行の起点となる `main` 関数の戻り値の型は `int` (整数型) と決められている。 `main` 関数は外部から (OS から) 実行されるものであり、当該プログラムの終了状態を呼び出し元 (OS) に対して `return` 文で通知する。当該プログラムの終了状態が**正常終了**の場合に `0` を、それ以外の場合は `0` 以外の値を呼び出し元に通知する。この場合、プログラムの終了状態は、そのプログラムの仕様として開発者が策定するものである。

`main` 関数には `return` 文を書かないことも多い⁸。その場合、 `main` 関数の末尾まで到達すると自動的に `0` が返される。

2.2 プリプロセッサ

プログラム `test001.c` の冒頭部分に「`#include <stdio.h>`」という記述がある。これは、用意された別のファイル (この例では `'stdio.h'` というファイル) を読み込んで、この位置に挿入するという処理を意味するものである。

C 言語のコンパイラには **プリプロセッサ**と呼ばれる機構があり、機械語への翻訳処理の前にソースプログラムに変換処理を施す。プリプロセッサには、今回の例のように、別のファイルを読み込んで挿入する機能の他に、定義された名前を具体的な値やプログラムに展開する機能などがある。今回の例では、 `printf` 関数の宣言 (引数や戻り値の型に関する情報など) をソースプログラム `test001.c` に展開して使用するために、C 標準ライブラリのヘッダファイル `'stdio.h'` を冒頭の位置に読み込んでいる。このように、様々な宣言や定義を記述したファイルを、プログラムの冒頭部分で読み込むことが多いため、それらは**ヘッダファイル**とも呼ばれ、ファイル名の拡張子に `.h` を付けることが習慣となっている。ヘッダファイルは、ユーザが独自に作成して使用することもできる。

2.3 プログラムのコンパイルと実行

C 言語で記述された**ソースプログラム**は、人間にとって可読なテキストファイルとして作成される。これを**コンパイル**して実行可能な**機械語**のプログラムに翻訳する。本書では、Linux や macOS、および Windows の MSYS2 環境において、GCC によってこの処理を行う。先に示した `test001.c` をターミナルウィンドウでコンパイルする例 (gcc コマンド) を次に示す。

例. gcc コマンドによるコンパイル

```
$ gcc test001.c 
```

「\$」は POSIX 系 OS の Bourne 系シェルなどで用いられるコマンドプロンプトであり、これに続いて gcc コマンドを入力している。この結果、 `test001.c` がコンパイルされ、 `a.out` というファイル (Windows の MSYS2 環境下では `a.exe`) が作成される。これはコンパイルの結果として生成されたものであり、実行可能な機械語プログラムを格納した**実行形式ファイル**である。これを実行するには、コマンドプロンプトに続いて `./a.out` と入力 (Windows 環境では `./a` と入力) する。

例. Linux, macOS の場合の実行

```
$ ./a.out   
Hello, world.      ←処理結果の出力
```

例. Windows の MSYS2 の場合の実行

```
$ ./a   
Hello, world.      ←処理結果の出力
```

⁷本書で示すサンプルプログラムでは `main()` と、引数を取らない場合は `void` を省略することが多いことを了解いただきたい。

⁸本書で示すサンプルプログラムでは、 `main` 関数が正常に終了することを前提として `return` 文を書かないことが多いことを了解いただきたい。

実用的な使用方法としては、gcc コマンドによる翻訳時に「-o 出力ファイル名」オプションを与え、生成する実行形式ファイルの名前を指定する。(次の例)

例. コンパイル時の出力ファイルの指定

```
$ gcc test001.c -o test001 
```

こうすることで、出力される実行用ファイルが「test001」(Windows の MSYS2 環境では test001.exe) となる。

2.4 関数定義の順序とプロトタイプ宣言

複数の関数を定義する例を次の test002.c に示す。

プログラム：test002.c

```
1 #include <stdio.h>
2
3 int kasan(int x, int y) {
4     return x+y;
5 }
6
7 int main() {
8     int a=2, b=3, c; // 変数の宣言
9     c = kasan(a,b);
10    printf("%d + %d = %d\n",a,b,c);
11 }
```

プログラム：test002-1.c (良くない例)

```
1 #include <stdio.h>
2
3 int main() {
4     int a=2, b=3, c; // 変数の宣言
5     c = kasan(a,b);
6     printf("%d + %d = %d\n",a,b,c);
7 }
8
9 int kasan(int x, int y) {
10     return x+y;
11 }
```

プログラム test002.c では、加算を実行する関数 kasan が定義されており、これは、受け取った 2 つの整数値を加算し、その結果を return 文で返す。また、main 関数は kasan 関数を呼び出して結果を取得し、printf 関数で処理結果を出力している。

例. test002.c のコンパイルと実行

```
$ gcc test002.c -o test002  ←コンパイル
$ ./test002  ←実行
2 + 3 = 5 ←処理結果の出力
```

test002.c の 10 行目に printf 関数の呼び出しがあり、第 1 引数の文字列には「%」記号で始まる**変換指定子** (conversion specifier)⁹ が含まれている。これは、後続の引数の値を適切に変換してその部分に埋め込むためのものである。また、printf の第 1 引数に与える文字列 (変換指定子を含んだもの) を**書式文字列** (format string) と呼ぶ。

test002.c では、kasan 関数を main 関数より上に記述している。これは、コンパイラがソースプログラムを上から順番に解析するため、main 関数が kasan 関数を使用するに先立って、kasan 関数の名前、引数の型、戻り値の型をコンパイラに知らせておく必要がある。例えば、test002-1.c のように、kasan 関数の定義を main 関数の定義より後に記述すると、kasan 関数の型に関する情報を main 関数が参照できず、コンパイルの段階でエラー¹⁰ となる。(次の例)

例. test002-1.c のコンパイル

```
$ gcc test002-1.c -o test002-1  ←コンパイルの試み
test002-1.c: In function 'main':          ←↓ エラーとなる
test002-1.c:5:13: error: implicit declaration of function 'kasan'
                    [-Wimplicit-function-declaration]
5 |         c = kasan(a,b);
```

これを解決するためには、関数呼び出しの記述より前の位置に、その関数の名前、引数の型、戻り値の型を示す**プロトタイプ宣言**を記述しておく。そのような記述形式で先のプログラムを書き換えたものを test002-2.c に示す。

プログラム：test002-2.c

```
1 #include <stdio.h>
2
3 int kasan(int x, int y); // プロトタイプ宣言 (型宣言のみ)
4
```

⁹書式指定子と呼ばれることもある。変換指定子については後の「3.8 書式文字列と変換指定子」(p.35) で解説する。

¹⁰GCC の版やコンパイラオプションによっては、エラーではなく警告として報告される場合もある。

```

5 | int main() {
6 |     int a = 2, b = 3, c;
7 |     c = kasan(a,b);
8 |     printf("%d + %d = %d\n",a,b,c);
9 | }
10 |
11 | int kasan(int x, int y) {    // 本体の定義はこちら
12 |     return x+y;
13 | }

```

2.5 関数への引数の受け渡しについて

C 言語では、関数内で宣言された変数は、原則としてその関数内でのみ使用できる。これに関しては後の「3.4 変数のスコープと記憶期間」(p.15) で詳しく解説する。関数定義に記述された仮引数も、その関数内で使用される変数として扱われ、仮引数そのものを関数内で変更しても、呼び出し元の実引数には影響しない。このことを次のプログラム test002-5.c で確認する。

プログラム：test002-5.c

```

1 | #include <stdio.h>
2 |
3 | void f( int x, int y ) {
4 |     printf("[inside f]\t: x=%d, y=%d\n", x,y);    // 受取り直後の引数の確認
5 |     x = -123; y = -456;                          // この関数内で変更
6 |     printf("[inside f]\t: x and y are changed\n");
7 |     printf("[inside f]\t: x=%d, y=%d\n", x,y);    // 内容確認
8 | }
9 |
10 | int main() {
11 |     int x, y;
12 |     x = 67; y = 89;                                // 値の設定
13 |     printf("[inside main]\t: x=%d, y=%d\n", x,y);  // 内容確認
14 |     f( x, y );                                      // f の呼び出し
15 |     printf("[inside main]\t: x=%d, y=%d\n", x,y);  // 内容を再度確認
16 | }

```

このプログラムでは、main 関数内で変数 x, y が宣言されて値が設定されており、これらの変数の値を別の関数 f に渡している。関数 f 内では、仮引数の値 x, y を変更して printf で出力している。しかし、main と f は異なる関数なので、f 内での x, y は main 内の x, y とは別物であり、f 内での値の変更は main とは無関係である。

このプログラムをコンパイルして実行する例を次に示す。

例. test002-5.c のコンパイルと実行

```

$ gcc test002-5.c -o test002-5  [Enter]    ←コンパイル
$ ./test002-5  [Enter]    ←実行
[inside main]    : x=67, y=89                ← main での最初の x, y の値
[inside f]       : x=67, y=89                ←関数 f が受け取った x, y の値
[inside f]       : x and y are changed
[inside f]       : x=-123, y=-456            ←関数内で x, y が変更された結果
[inside main]    : x=67, y=89                ← main の x, y は影響を受けていない

```

この実行結果から、main 側の x, y と f 側の x, y は別のものであることがわかる。

重要) C 言語の関数呼び出しでは、呼び出し元の**実引数**の値が、呼び出された関数の**仮引数**に複製（コピー）される。この受け渡し方法を**値渡し**と呼ぶ。

2.6 リテラル

プログラム中に直接書かれている値を **リテラル** (literal) という。例えば次の式

```
x = 10 + 3;
```

の 10 と 3 がリテラルである。これに対して x は変数であり、その値はプログラムの実行中に変化する可能性がある。

C 言語では、例えば次のようなものがリテラルとして用いられる。

10 : 整数リテラル
 3.14 : 浮動小数点リテラル
 'A' : 文字定数（リテラルに準ずる）
 "ABC" : 文字列リテラル

2.6.1 数値リテラルのサフィックス

数値リテラルの末尾には、その値の型を明示するための文字を付けることができる。このような付加文字を**サフィックス**という。サフィックスを付けることで、整数リテラルや浮動小数点数リテラルの型を明示できる。

例えば、「123L」は long 型の整数リテラルを表し、「3.14f」は float 型の浮動小数点数リテラルを表す。

整数リテラルに付ける主なサフィックスを表 1 に示す。

表 1: 整数リテラルの主なサフィックス

サフィックス	主に指定される型	サフィックス	主に指定される型
U	unsigned 系の整数型	L	long 型
LL	long long 型	UL, LU	unsigned long 型
ULL, LLU	unsigned long long 型		

※サフィックスは小文字で書いても良い。

整数リテラルの実際の型は、その値が各整数型で表現できるかどうかによっても決まる。表 1 に示した型は、サフィックスによって指定される基本的な型を表している。

浮動小数点数リテラルに付ける主なサフィックスを表 2 に示す。

表 2: 浮動小数点数リテラルの主なサフィックス

サフィックス	主に指定される型	サフィックス	主に指定される型	サフィックス	主に指定される型
なし	double 型	F, f	float 型	L, l	long double 型

例えば次のようになる。

123 int 型の整数リテラル, 123U unsigned int 型の整数リテラル,
 123L long 型の整数リテラル, 123LL long long 型の整数リテラル,
 3.14 double 型の浮動小数点数リテラル, 3.14f float 型の浮動小数点数リテラル,
 3.14L long double 型の浮動小数点数リテラル

なお、これらのサフィックスは常に必須というわけではないが、型を明示したい場合に用いられる。例えば、float 型の値を扱う際に 3.14f のように記述すれば、その値が float 型であることを明確に示すことができる。また、123L のように書けば、long 型の整数リテラルであることを明示できる¹¹。

なお、l のような小文字を使える場合もあるが、数字の 1 と見分けにくいいため、通常は L のように大文字を用いる方がわかりやすい。

2.6.2 数値リテラルのプレフィックス

整数リテラルでは、先頭に特定の文字を付けることで、数値の表記方法（基数）を指定することができる。このような先頭に付ける文字を**プレフィックス**という。

整数リテラルで用いられる主なプレフィックスを表 3 に示す。

表 3: 整数リテラルのプレフィックス

プレフィックス	基 数	例	プレフィックス	基 数	例
なし	10 進数	123	0	8 進数	075
0x, 0X	16 進数	0x9A	0b, 0B	2 進数 (注)	0b1010

(注) 2 進数に関しては C23 から標準化。それ以前は処理系依存

例えば次のようになる。

¹¹他の型のリテラルに関するサフィックスもあるが、割愛する。

123 10 進数, 075 8 進数 (10 進数では 61),
0x9A 16 進数 (10 進数では 154), 0b1010 2 進数 (10 進数では 10)

16 進数では, A~F (または a~f) を用いて, 10~15 を表す. 例えば, 0xFF は 16 進数であり, 10 進数では 255 を表す.

注意) 整数リテラルの先頭に 0 を付けると, 8 進数として扱われる. 例えば, 次のような記述はエラーとなる.

例. 誤った 8 進数表記

```
int x = 088;  
int y = 099;
```

理由は, 8 進数では使用できる数字が 0~7 に限られるためである.

また, 次のような場合も注意が必要である.

例. 見間違いしやすいケース

```
int x = 012;
```

この場合, 012 は 10 進数の 12 ではなく, 8 進数として扱われるため, 10 進数では 10 を表す. このような誤りは見ただけでは分かりにくいので, 注意が必要である.

■ 2 進数表記 (0b) について

整数リテラルでは, 0b を用いて 2 進数を表すことができる. 例えば, 0b1010 は 2 進数の 1010 を表し, 10 進数では 10 となる.

ただし, この表記は古い C 規格では標準ではなく, GCC や Clang などの処理系が拡張機能として提供していたものである. 比較的新しい C 規格 (C23) では, この表記が正式に標準化された. そのため, 処理系やコンパイラの設定によっては, 0b 表記が利用できない場合もあるので注意が必要である.

■ 16 進浮動小数点数

C 言語 (C99 以降) では, 整数だけでなく浮動小数点数についても次のような 16 進数表記を用いることができる.

0x[整数部].[小数部]p[指数]

この「整数部」, 「小数部」共に 16 進数で, これらによって仮数部を構成する.

例. 16 進数表現の浮動小数点数

```
0x1.8p1
```

この表記は, 16 進数の仮数部と 2 の指数部を組み合わせたものであり, 次のような意味を持つ.

$$0x1.8p1 = 1.5_{(16)} \times 2^1 = 3.0_{(10)}$$

このように, 16 進浮動小数点数では, p の後に 2 の指数を記述する. これは, 通常の浮動小数点数の e (10 の指数) とは異なる点である.

例. 2 の指数による表現

$$1.5e1 \rightarrow 1.5 \times 10^1$$
$$0x1.8p1 \rightarrow 1.5 \times 2^1$$

この表記は, 仮数部を 16 進数, 指数部を 2 の累乗として記述するため, 2 進浮動小数点数の値を正確に記述したい場合に適している. このため, 数値計算やシステムプログラミングなどで用いられることがある. ただし, 一般的なプログラムではあまり使用されない.

■ サンプルプログラム

各種基数のプレフィックスを用いたサンプルプログラムを示す.

プログラム: test002-7.c

```
1 #include <stdio.h>  
2  
3 int main() {  
4     int a;  
5     a = 0xff;          // 16 進数リテラル  
6     printf("(dec)%3d = (hex)%03x = (oct)%3o, \t ", a, a, a);
```

```

7      a = 0777;          // 8進数リテラル
8      printf("(dec)%3d = (hex)%3x = (oct)%03o\n", a,a,a);
9      a = 0b1111;       // 2進数リテラル (C17ではGCCの拡張機能)
10     printf("(dec)%3d = (hex)%3x = (oct)%03o,\t ", a,a,a);
11
12     double x = 0xffff.0p-8;    // 16進数浮動小数点数リテラル
13     printf("(dec)%f = (hex)%a\n", x,x);
14 }

```

これをコンパイルして実行する例を次に示す。

例. test002-7.c のコンパイルと実行

```

$ gcc test002-7.c -o test002-7  ←コンパイル
$ ./test002-7  ←実行
(dec)255 = (hex)0ff = (oct)377, (dec)511 = (hex)1ff = (oct)777
(dec) 15 = (hex) f = (oct)017, (dec)255.996094 = (hex)0x1.fffep+7

```

注意) %a による 16 進浮動小数点数の出力は、正規化された形式で表示されるため、入力した表記とは異なる形になる場合がある。

考察) 0xffff.0p-8 は 2 進数表現ではどうなるか考えよ。

2.6.3 文字列リテラル

文字列をダブルクォート文字「"」で囲んで記述したものを**文字列リテラル** (string literal) という。例えば、次に示すものは文字列リテラルである。

```
"Hello"    "abc123"    "This is a string."
```

C 言語では、文字列リテラルは**文字の配列**として扱われ、その末尾には文字列の終端を表すヌル文字 '\0' が自動的に配置される¹²。また、C 言語では、複数の文字列リテラルを空白文字や改行を挟んで続けて記述すると、それらは自動的に連結され、1つの文字列リテラルとして扱われる。例えば、

```
"abc" "def"
は,
"abcdef"
```

と同じ文字列リテラルとして扱われる。この際、文字列リテラルの間に空白文字が自動的に挿入されることはない。長い文字列リテラルをソースコード上で複数行に分けて記述する場合などに便利である。

次のプログラムは、隣接して記述した文字列リテラルが自動的に連結されることを確認するものである。

プログラム: test002-8.c

```

1  #include <stdio.h>
2
3  int main() {
4      printf("abc" "def" "\n");          // 文字列リテラルの連結(1)
5
6      const char *s = "This is " "one string."; // 文字列リテラルの連結(2)
7      printf("%s\n", s);
8  }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test002-8.c のコンパイルと実行

```

$ gcc test002-8.c -o test002-8  ←コンパイル
$ ./test002-8  ←実行
abcdef          ← 1つの文字列に
This is one string.    なっている

```

2.6.4 定数のマクロ定義

ソースプログラム中に数値のリテラルを直接記述すると、その意味がわかりにくくなることがあり、可読性の低下の原因となる。このような場合は、「#define」によるマクロ定義が役立つ。

¹²配列についての詳細は後の「3.6 配列」(p.31～)で、文字列についての詳細は後の「3.7 文字と文字列」(p.33～)で解説する。

書き方： #define 定義名 具体的な記述

「定義名」を「具体的な記述」を表すマクロとして定義する。これにより「具体的な記述」の代りに「定義名」をソースプログラム中に記述することができる。例えば、次に示すプログラムでは、円周率を表すマクロ PI を定義しており、円周率の値のリテラルの代りにこれを使用している。

プログラム：test002-3.c

```
1 #include <stdio.h>
2 #include <math.h>    // 数学関数に関するヘッダファイル
3
4 #define PI          3.1415926535897932    // 円周率を意味するマクロ
5
6 int main() {
7     double y1, y2;
8     y1 = sin(PI);
9     y2 = cos(PI);
10    printf("sin(PI) = %12.9f\n", y1);
11    printf("cos(PI) = %12.9f\n", y2);
12 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test002-3.c のコンパイルと実行

```
$ gcc test002-3.c -o test002-3 -lm  ←コンパイル13
$ ./test002-3  ←実行
sin(PI) = 0.000000000
cos(PI) = -1.000000000
```

#define によるマクロ定義に関しては、後の「8.2 プリプロセッサ」(p.98) で詳しく解説する。

2.7 識別子の命名規則

変数名や関数名、ラベル名など、プログラム中で名前として用いるものは**識別子**と呼ばれる。識別子には次のような規則がある。

- ・英字 (a-z, A-Z), 数字 (0-9), アンダースコア (_) が使用できる
- ・先頭は英字またはアンダースコアでなければならない。また、数字で始めることはできない
- ・大文字と小文字は区別される
- ・予約語は使用できない

■ 命名に関する注意

識別子には自由に名前を付けることができるが、実用的なプログラミングにおいては可読性を高めるため、意味が分かる名前を付けることが望ましい。例えば、a や x1 のような名前ではなく、count や total_sum のように役割が分かる名前を付けることが推奨される。(本書は入門書につき、素朴な識別子を採用することが多い)

¹³-lm は数学関数ライブラリをリンクするためのオプション。環境によってはこれを指定しなくても数学関数をリンクできる場合がある。

3 変数, 型, 数値

3.1 変数の宣言

C 言語の変数宣言には、いくつかの**宣言指定子**を用いる。宣言指定子には、**記憶クラス**を指定する**記憶クラス指定子**、**型**を指定する**型指定子**、および型を修飾する**型修飾子**がある。宣言指定子の詳細に関しては後で解説する。

宣言指定子の後に、具体的に使用する変数名を記述する。複数の宣言指定子を記述する場合、それらの記述順序は文法上は任意であるが、一般には

記憶クラス指定子 型修飾子 型指定子 宣言子

の順序で記述するのが慣例となっている。変数名も「宣言子」の1つである。また、このような宣言を記述する際、複数の宣言子（変数名など）をコンマ「,」で区切って並べることができ、同様の特性を持つ複数の変数を一括して宣言することもできる。

3.2 値の代入

変数にはイコール記号「=」を用いて値を**代入**することができる。代入の記述はそれ自体が**式**であり、代入後の値がその代入式の評価結果となる。このことは、代入の記述を条件判定の際の条件判定式として用いたり、代入の連鎖を実現するといったことに応用できる。

例. 代入の連鎖

```
a = b = 3;
```

この例は、変数 a, b に 3 を代入するものであるが、厳密には、まず「b = 3」が実行（評価）され、その結果の値である 3 が a に代入される。すなわち、「a = (b = 3);」という解釈である。

変数は、宣言と同時に初期値を設定することができる。これを**初期化**という。

例. 変数宣言時の初期値の設定

```
int x = 3;
```

このような記述により、変数 x が宣言され、初期値 3 が設定される。

■ 初期化されていない変数

自動記憶期間を持つ変数¹⁴に初期値を設定しなかった場合、その変数は不定の値を持つ。その値を通常の演算や出力に使用すると、プログラムの動作は保証されない。次に示すプログラム test002-4.c は、初期化されていない変数を参照する良くない例である。

プログラム：test002-4.c

```
1 #include <stdio.h>
2
3 int main() {
4     int a;
5     double x;
6     printf("a = %d, x = %f\n", a, x);
7 }
```

実行例.

```
$ gcc test002-4.c -o test002-4  ←コンパイル
```

```
$ ./test002-4  ←実行
```

```
a = 32758, x = 0.000000 ←意図しない値
```

この結果から、初期化していない変数の値は保証されておらず、意図しないものになっていることがわかる。

このように、初期化されていない変数には**不定値**が入っており、その値は処理系や実行状況によって変化する。したがって、変数はその値が参照される前に、必ず値を設定しておかなければならない。

3.2.1 定数としての変数

型修飾子 const を用いると、宣言後に値を変更できない変数を宣言することができる。このような変数を、本書では**定数**として扱う。

例. 定数としての変数

```
const int c = 3;
```

¹⁴後の「3.4 変数のスコープと記憶期間」(p.15)で解説する。

この例では、定数としての変数 `c` が宣言され、初期値 3 が設定されるが、この `c` の値は変更することができず、その後に入力を行うことはできない。

3.2.2 複合代入

既存の変数に、現行の値を変化させる形での代入の方法（複合代入）がある。例えば、整数の変数 `x` に 3 が代入されているとする。この場合、

```
x = x + 1
```

とすると、現行の `x` の値 3 に 1 を加えた値を、`x` に上書き（更新）する形で代入する。

これを簡単に実行する代入の形式

```
x += 1
```

があり、これが複合代入¹⁵である。加算以外の算術演算¹⁶ `-`, `*`, `/`, `%` に対しても同様の複合代入の演算子がある。

例. `x *= 3` → `x = x * 3` と同じ結果となる。

3.3 基本的な型

C 言語で扱う代表的な数値型には、整数型と浮動小数点型がある。表 4 に代表的な数値型とそのサイズの例¹⁷を示す。上段が整数型、下段が浮動小数点型のカテゴリである。

表 4: 変数の型とメモリサイズの例（単位：バイト）

型	32bit x86 (GCC)	64bit Linux (x86-64 GCC)	64bit MSYS2 (MINGW64/UCRT64)	64bit MSVC	Apple Silicon (macOS)
char	1	1	1	1	1
short	2	2	2	2	2
int	4	4	4	4	4
long	4	8	<u>4</u>	<u>4</u>	8
long long	8	8	8	8	8
float	4	4	4	4	4
double	8	8	8	8	8
long double	12 または 16	16	16	<u>8</u>	<u>8</u>

※ 型のサイズは処理系に依存する。特に 64bit Windows の long 型と、MSVC および Apple Silicon macOS の long double 型のサイズに注意。

注意) 表 4 は代表的な例であり、実際には使用するコンパイラによっても異なる場合があるので注意すること。

更に整数型は符号の有無の指定が可能で、型の前に

signed : 符号あり

unsigned : 符号なし

を記述する。short, int, long, long long では、signed と unsigned のどちらも記述しない場合、符号付き整数型となる。一方、単なる char が符号付きとして扱われるか符号なしとして扱われるかは、処理系によって異なる。

signed, unsigned も型指定子であり、他の型指定子と組み合わせて整数型を指定する。

C 言語では、型の具体的なサイズは一律には定められておらず、CPU, OS, ABI, コンパイラなどの処理環境によって異なる場合がある。これに対して `stdint.h` には、利用可能な処理系において、ビット幅を明確に定めた整数型が定義されているので、データサイズを明確にしたプログラミングの際にはこれを「`#include <stdint.h>`」として読み込んで使用する。`stdint.h` に定義された整数型を表 5 に示す。

表 5 の型は、対応する幅の整数型を処理系が提供できる場合に定義される。本書で扱う一般的な計算機環境では、通常、表の型をすべて利用できる。

¹⁵ 累算代入、拡張代入などと呼ばれることもある。

¹⁶ 後の「3.5.3 算術演算」(p.23) で解説。

¹⁷ 1 バイトが 8 ビットである代表的な計算機での例である。

表 5: stdint.h に定義されている固定サイズ整数型

符号あり	符号なし	説 明
int8_t	uint8_t	8 ビット整数
int16_t	uint16_t	16 ビット整数
int32_t	uint32_t	32 ビット整数
int64_t	uint64_t	64 ビット整数

■ データサイズを求める sizeof 演算子

sizeof 演算子を用いると、型のサイズを調べることができる。

書き方 (1): sizeof 式

書き方 (2): sizeof(型)

「式」の型のサイズや、指定した「型」のサイズ (バイト数) を返す。戻り値は、符号なし整数型 size_t¹⁸ の値である。

次に示すプログラム test003-9.c は、値や型のサイズを調べる例である。

プログラム: test003-9.c

```

1  #include    <stdio.h>
2
3  int main() {
4      int a;
5      double b;
6      printf("[size check] (bytes)\n");
7      printf(" int           : %zu\n",sizeof a);           // 変数 a の型のサイズ
8      printf(" double        : %zu\n",sizeof(b));          // 変数 b の型のサイズ
9      printf(" long double   : %zu\n",sizeof(long double)); // 型 long double のサイズ
10 }
```

参考) このプログラムでは、printf 関数に与える書式文字列が「%zu」という変換指定子を含んでいる。これは size_t 型の値を埋め込むためのものである。

プログラム中に sizeof(b) とあるように、値のサイズを調べる際に括弧を記述しても良い。このプログラムの実行例を次に示す。

例. test003-9.c のコンパイルと実行 (long double が 16 バイトの環境)

```

$ gcc test003-9.c -o test003-9  ←コンパイル
$ ./test003-9  ←実行
[size check] (bytes)
 int           : 4
 double        : 8
 long double   : 16
```

参考) sizeof は関数ではなく演算子であり、多くの場合その値はコンパイル時に求められる。

3.4 変数のスコープと記憶期間

変数には、プログラム中でその名前を使用できる範囲である**スコープ**と、その変数のための記憶領域が存在する期間である**記憶期間**がある。これについて例を挙げて解説する。次に示すプログラム test003.c は main 関数が別の関数 dbl を呼び出すものである。dbl は仮引数 x に受け取った値を 2 倍し、その結果を関数内で宣言した変数 r に格納して返す関数である。

main 内でも変数 r を宣言しているが、これは dbl の変数 r とは別のものである。プログラムの実行によってそのことが確認できる。

¹⁸size_t は処理系によって選ばれる符号なし整数型である。ソースプログラム中で型名 size_t を直接使用する場合は、通常、stdint.h など、この型を定義するヘッダを読み込む。

プログラム：test003.c

```
1 #include <stdio.h>
2
3 int dbl( int x ) {
4     int r = 2*x;
5     return r;
6 }
7
8 int main() {
9     int a = 3, a2, r = 99;
10    a2 = dbl( a );
11    printf("2*%d = %d\n",a,a2);
12    printf("r = %d\n",r);
13 }
```

実行例.

```
$ gcc test003.c -o test003  ←コンパイル
$ ./test003  ←実行
2*3 = 6      ← dbl からの戻り値が得られている
r = 99       ← main の変数 r の値
```

この結果から、dbl 内の変数 r と、main 内の変数 r が別のものであることがわかる。

このプログラムでは、両方の関数の中で変数 r を宣言して使用しているが、それぞれの変数 r のスコープは、その宣言位置から、当該変数が宣言されたブロックの終端までである。

次のプログラム test003-1.c で変数のスコープがブロック内に限定されることが確認できる。このプログラムでは、同じ変数名 r が宣言されているが、属しているブロックが異なるため、それぞれ別の変数となる。

プログラム：test003-1.c

```
1 #include <stdio.h>
2
3 int main() {
4     // block1
5     int r = 5;
6     { // block2
7         int r = 9;
8         printf("block2: r = %d\n",r);
9     }
10    printf("block1: r = %d\n",r);
11 }
```

実行例.

```
$ gcc test003-1.c -o test003-1 
$ ./test003-1  ←実行 ↑コンパイル
block2: r = 9      ←同じ変数名 r でもスコープ
block1: r = 5      ←が異なると別のものとなる
```

このプログラムでは、内側のブロックで宣言された r が、そのブロック内だけ外側の r を隠すことになる。これを **名前の隠蔽（シャドーイング）** という。

ここまでの例のように、関数内のブロックで記憶クラス指定子を付けずに変数を宣言した場合、その変数は通常、**自動記憶期間**を持つ。これは、記憶クラス指定子 **auto** を付けて宣言した場合と同じである。

auto の変数（自動記憶期間を持つ変数）は、当該ブロックの実行中だけ存在する。このような変数は**自動変数**（automatic variable）と呼ばれる。

自動変数であることを

```
auto int x;
```

などと記述して宣言することもできるが、多くの場合 auto の記述は省略する。

結論）ブロック内で宣言された変数のスコープは、ブロックの入れ子構造に従ってネストされる。

重要）関数内で宣言される auto 変数は、その関数が呼び出されるたびに新たに生成される。従って、関数が再帰的に呼び出された場合¹⁹でも、各呼び出しごとに別の変数として確保される。

3.4.1 静的変数

static の記憶クラスの変数は、プログラム全体が終了するまで存在し続ける。このことを次のプログラム test003-2.c の実行によって確認する。このプログラムでは、main 関数から count 関数が3回呼び出される。count 関数は静的変数 c を持ち、初期値が0に設定される。この変数 c は静的変数であり、count 関数の実行が終了しても破棄されずに存在し続ける。

¹⁹詳しくは、後の「9.1 関数の再帰的定義」（p.102）で解説する。

プログラム：test003-2.c

```
1 #include <stdio.h>
2
3 void count(void) {
4     static int c = 0;
5     c = c + 1;
6     printf("count: c = %d\n",c);
7 }
8
9 int main() {
10     int c = -99;
11     count(); count(); count();
12     printf("main: c = %d\n",c);
13 }
```

実行例.

```
$ gcc test003-2.c -o test003-2 Enter
$ ./test003-2 Enter ←実行 ↑コンパイル
count: c = 1 ←変数 c は count 関数が
count: c = 2 終了しても引き続き
count: c = 3 存在していることがわかる
main: c = -99 ← main 関数の変数 c は別物
```

プログラムが終了すると、静的変数を含むプログラムの記憶領域も実行環境によって解放される。このとき count 関数の変数 c も破棄される。

重要) 静的記憶期間を持つ変数の初期化は、プログラムの実行開始前に一度だけ行われる。上記プログラムの関数 count 内に static 変数 c の初期化の記述があるが、count 関数が呼び出されるたびに初期化されることはない。

重要) 関数内で宣言された静的変数は、その関数が何回呼び出されても同一の変数である。従って、関数が再帰的に呼び出された場合²⁰でも、その静的変数は全ての呼び出しで共有される。

3.4.2 グローバル変数

関数定義の外側で宣言された変数は、一般に**グローバル変数**と呼ばれる。その変数名は、宣言位置からソースファイルの末尾まで有効²¹となり、その範囲にある複数の関数から同じ変数を参照できる。

プログラム：test003-3.c

```
1 #include <stdio.h>
2
3 int g = 99; // グローバル変数
4
5 void f1(void) {
6     printf("f1 : g = %d\n",g);
7 }
8
9 int main() {
10     printf("main: g = %d\n",g);
11     f1();
12 }
```

実行例.

```
$ gcc test003-3.c -o test003-3 Enter
$ ./test003-3 Enter ←実行 ↑コンパイル
main: g = 99 ← main 関数から参照できる
f1 : g = 99 ←関数 f1 からも参照できる
```

グローバル変数 g が main, f1 両方の関数から同じものとして参照できている。

ブロック内で変数を宣言すると、その変数はローカル変数となる。その変数名がグローバル変数と同じ場合、そのブロック内ではローカル変数が優先され、グローバル変数はその名前では参照できなくなる。このことが次のプログラム test003-4.c で確認できる。

プログラム：test003-4.c

```
1 #include <stdio.h>
2
3 int g = 99; // グローバル変数
4
5 void f1(void) {
6     int g = 3; // ローカル変数
7     printf("f1 : g = %d\n",g);
8 }
9
10 int main() {
11     printf("main: g = %d\n",g);
12     f1();
13     printf("main: g = %d\n",g);
14 }
```

実行例.

```
$ gcc test003-4.c -o test003-4 Enter
$ ./test003-4 Enter ←実行 ↑コンパイル
main: g = 99 ←グローバルの g を参照している
f1 : g = 3 ← f1 ではローカルの g を参照している
main: g = 99 ←グローバルの g を参照している
```

同じ g という変数名でも、グローバル変数とローカル変数では別々の変数として扱われることがわかる。

注意) グローバル変数を多用すると、プログラム内での変数の一貫性の管理が難しくなるため、その使用は必要最小限に留めることが望ましい。

²⁰詳しくは、後の「9.1 関数の再帰的定義」(p.102)で解説する。

²¹このようなスコープをファイルスコープと呼ぶ。

3.5 数値

C 言語では、**整数**と**浮動小数点数**を扱うことができる。

整数の例. -123, 0, 7, 255

浮動小数点数の例. -4.5, 0.0, 3.14159, 9.1093837e-31

この例のように、浮動小数点数のリテラルとして

仮数部 e 指数部

という**指数表現**を用いることもできる。例えば 9.1093837e-31 は $9.1093837 \times 10^{-31}$ を意味する。また指数部直前の e は大文字で E と書いても良い。

3.5.1 整数が扱える値の範囲

現在のほとんどのコンピュータでは、符号付き整数は **2 の補数** (two's complement) によって表現される。C17 の規格では、符号付き整数の内部表現は 2 の補数だけに限定されていない²² が、実際の処理系では 2 の補数がほぼ標準となっている。各整数型のサイズは処理系ごとに定まっており、保持できる値には限界 (表 6) がある。

表 6: 典型的な固定長整数の扱える値の範囲

バイト長	符号付き	符号なし
1	-128 ~ 127	0 ~ 255
2	-32,768 ~ 32,767	0 ~ 65,535
4	-2,147,483,648 ~ 2,147,483,647	0 ~ 4,294,967,295
8	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807	0 ~ 18,446,744,073,709,551,615

※ 1 バイトを 8 ビットとし、符号付き整数を 2 の補数で表現する典型的な環境における値の範囲

実際に、コンパイラが扱える int 型整数の範囲を表す INT_MIN, INT_MAX, および unsigned int 型整数の最大値を表す UINT_MAX は、limits.h に定義されている。固定幅整数型が処理系によって提供される場合、それぞれの最小値、最大値を表すマクロも stdint.h に定義 (表 7) される。

表 7: 固定長の整数型の最小値と最大値 (stdint.h)

型	最小値	最大値	型	最大値
int8_t	INT8_MIN	INT8_MAX	uint8_t	UINT8_MAX
int16_t	INT16_MIN	INT16_MAX	uint16_t	UINT16_MAX
int32_t	INT32_MIN	INT32_MAX	uint32_t	UINT32_MAX
int64_t	INT64_MIN	INT64_MAX	uint64_t	UINT64_MAX

※これらのマクロは、対応する固定幅整数型が処理系によって提供される場合に定義される。

int 型および unsigned int 型の最大値、最小値の付近で演算を行った場合の違いを、次のプログラム test003-8.c で確認する。ただし、このプログラムの符号付き整数に対する演算には未定義動作が含まれており、実際のプログラムでは行ってはならない。これは、最大値に 1 を加える、あるいは最小値から 1 を引くとうなるかの試みである。

プログラム: test003-8.c (未定義動作を含む良くない例)

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 int main() {
5     // 符号付き整数に関する試み
6     int a = INT_MAX;    // int の最大値
7     printf("a = %d, a+1 = %d\n", a, a+1);    // 最大値に1を加える試み
8     a = INT_MIN;       // int の最小値
9     printf("a = %d, a-1 = %d\n", a, a-1);    // 最小値から1を引く試み
10
11     // 符号なし整数に関する試み
```

²²C23 では、符号付き整数の表現が 2 の補数に統一された。

```

12     unsigned int u = UINT_MAX; // この型の最大値
13     printf("u = %u, u+1 = %u\n", u, u+1); // 最大値に1を加える試み
14     u = 0; // この型の最小値
15     printf("u = %u, u-1 = %u\n", u, u-1); // 最小値から1を引く試み
16 }

```

このプログラムをコンパイルして実行した例を次に示す。

例. test003-8.c のコンパイルと実行（未定義動作を含む良くない例）

```

$ gcc test003-8.c -o test003-8  ←コンパイル
$ ./test003-8  ←実行
a = 2147483647, a+1 = -2147483648 ←出力：この環境によるものであり、これと
a = -2147483648, a-1 = 2147483647   同じ出力を期待してはならない
u = 4294967295, u+1 = 0
u = 0, u-1 = 4294967295

```

補足) Windows MSYS2 の GCC, 最適化なしで得られた一例

出力の前半の符号付き整数に関する 2 行は、未定義動作を含む演算を実行した結果であり、同じ結果が得られることは保証されない。これに対して、後半の符号なし整数に関する 2 行は、符号なし整数演算の規則に基づく結果である。

符号なし整数の演算では、表現可能な範囲を超えた結果は、その型で表現できる値の個数を法 (*mod*) として処理される。例えば、32 ビットの `unsigned int` 型では、`UINT_MAX + 1` は 0 となり、0 から 1 を引いた結果は `UINT_MAX` となる。

これに対して、符号付き整数の演算結果がその型で表現できる範囲を超えた場合、動作は**未定義**となる。2 の補数を採用する多くの実際の処理系では、単純な実行結果として値が循環したように見える場合もあるが、C 言語のプログラムとしてその動作を前提にしてはならない。

8 ビットに簡便化し、符号なし整数で保証される循環と、符号付き整数で循環したように見える場合との違いを図 1 に示す。

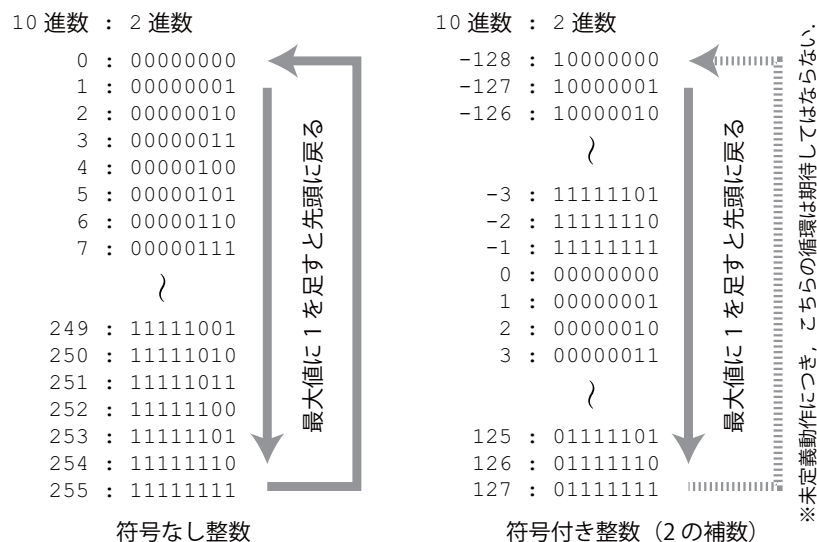


図 1: 8 ビット符号なし整数の循環と、符号付き整数の範囲外演算

▲重要▲

test003-8.c の符号付き整数に関する演算は未定義動作であり、その結果は保証されない。処理系、コンパイラの版、最適化の有無などによって結果が変わる可能性がある。実際のプログラミングでは、符号付き整数の演算結果が、その型で表現できる範囲を超えないようにしなければならない。一方、符号なし整数の演算は、その型で表現できる値の個数を法とする演算として定義されている。

注意) 整数の 0 による除算は**未定義動作**であり、多くの場合、プログラムは異常終了する。

3.5.1.1 最大幅整数型

当該環境で利用できる**最大幅整数型**として、`intmax_t` と `uintmax_t` がある。`intmax_t` は、処理系が提供する全ての符号付き整数型の値を表現できる符号付き整数型である。また、`uintmax_t` は、処理系が提供する全ての符号なし整数型の値を表現できる符号なし整数型である。

表 8: 最大幅整数型 (`stdint.h`)

型 (符号付き)	最小値	最大値	型 (符号なし)	最大値
<code>intmax_t</code>	<code>INTMAX_MIN</code>	<code>INTMAX_MAX</code>	<code>uintmax_t</code>	<code>UINTMAX_MAX</code>

`intmax_t`, `uintmax_t` のサイズと取り得る値の範囲を調べるプログラム `test003-g.c` を次に示す。

プログラム: `test003-g.c`

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <inttypes.h>
4
5 int main() {
6     intmax_t    imin = INTMAX_MIN, imax = INTMAX_MAX;
7     uintmax_t   umax = UINTMAX_MAX;
8     size_t      s = sizeof(intmax_t);
9     // 型のサイズと取り得る値の範囲を表示
10    printf("sizeof(intmax_t)=%zu\n", s);
11    printf("intmax_t:  %jd ... %jd\n", imin, imax);
12    printf("uintmax_t: 0 ... %ju\n", umax);
13 }
```

※変換指定子 `%jd`, `%ju` はそれぞれ `intmax_t`, `uintmax_t` のためのものである。

このプログラムをコンパイルして実行する例を次に示す。

例. `test003-g.c` のコンパイルと実行

```
$ gcc test003-g.c -o test003-g  ←コンパイル
$ ./test003-g  ←実行
sizeof(intmax_t)=8
intmax_t:  -9223372036854775808 ... 9223372036854775807
uintmax_t:  0 ... 18446744073709551615
```

実行結果は、環境 (OS, コンパイラなど) 毎に異なる。

注意) `intmax_t`, `uintmax_t` は最大幅整数型であるため、常に最も効率よく演算できる型であるとは限らない。必要とする値の範囲や用途に応じて、適切な整数型を選ぶことが望ましい。

3.5.2 浮動小数点数が扱える値の範囲

浮動小数点数が扱える数について、各種の限界値がヘッダファイル `float.h` に定義されている。その内の重要なものを表 9 に挙げる。

表 9: 浮動小数点数の各型の限界値

値	float	定 義 double	long double	備 考
最 大 値	<code>FLT_MAX</code>	<code>DBL_MAX</code>	<code>LDBL_MAX</code>	
正の最小値	<code>FLT_MIN</code> <code>FLT_TRUE_MIN</code>	<code>DBL_MIN</code> <code>DBL_TRUE_MIN</code>	<code>LDBL_MIN</code> <code>LDBL_TRUE_MIN</code>	正規化数の正の最小値 真の正の最小値
機械イプシロン	<code>FLT_EPSILON</code>	<code>DBL_EPSILON</code>	<code>LDBL_EPSILON</code>	$1.0 + \epsilon \neq 1.0$ となる最小の ϵ ^注

注) 正確には、「1 と、1 より大きい最小の表現可能値との差」

各型で表現できる最小の有限値は、最大有限値の符号を負にした値である。一方、最も小さい正の値は `FLT_TRUE_MIN` などで表される。表 9 の具体的な値を確認するプログラム `test003-a.c` を次に示す。

プログラム：test003-a.c

```
1  #include    <stdio.h>
2  #include    <float.h>
3
4  int main() {
5      printf("[MAX]\n");
6      printf("FLT_MAX    : %.17e\n", FLT_MAX);
7      printf("DBL_MAX    : %.17e\n", DBL_MAX);
8      printf("LDBL_MAX   : %.35Le\n", LDBL_MAX);
9      printf("[POSITIVE MIN]\n");
10     printf("FLT_MIN    : %.17e\n", FLT_MIN);
11     printf("DBL_MIN    : %.17e\n", DBL_MIN);
12     printf("LDBL_MIN   : %.35Le\n", LDBL_MIN);
13     printf("- TRUE POSITIVE MIN -\n");
14     printf("FLT_TRUE_MIN : %.17e\n", FLT_TRUE_MIN);
15     printf("DBL_TRUE_MIN : %.17e\n", DBL_TRUE_MIN);
16     printf("LDBL_TRUE_MIN : %.35Le\n", LDBL_TRUE_MIN);
17     printf("[EPSILON]\n");
18     printf("FLT_EPSILON  : %.17e\n", FLT_EPSILON);
19     printf("DBL_EPSILON  : %.17e\n", DBL_EPSILON);
20     printf("LDBL_EPSILON : %.35Le\n", LDBL_EPSILON);
21 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test003-a.c のコンパイルと実行

```
$ gcc test003-a.c -o test003-a  ←コンパイル
$ ./test003-a  ←実行
[MAX]
FLT_MAX    : 3.40282346638528860e+38
DBL_MAX    : 1.79769313486231571e+308
LDBL_MAX   : 1.18973149535723176502126385303097021e+4932
[POSITIVE MIN]
FLT_MIN    : 1.17549435082228751e-38
DBL_MIN    : 2.22507385850720138e-308
LDBL_MIN   : 3.36210314311209350626267781732175260e-4932
- TRUE POSITIVE MIN -
FLT_TRUE_MIN : 1.40129846432481707e-45
DBL_TRUE_MIN : 4.94065645841246544e-324
LDBL_TRUE_MIN : 3.64519953188247460252840593361941982e-4951
[EPSILON]
FLT_EPSILON  : 1.19209289550781250e-07
DBL_EPSILON  : 2.22044604925031308e-16
LDBL_EPSILON : 1.08420217248550443400745280086994171e-19
```

補足) 64bit Linux の GCC での一例

■ アンダーフロー

浮動小数点演算の結果の絶対値が小さ過ぎて、正規化数として表現できなくなる現象をアンダーフローという。この場合、結果は非正規化数として表現されるか、さらに小さい場合には 0 に丸められることがある。

正規化数として表現できない小さな非ゼロ値は、処理系が対応している場合、非正規化数として表現される。非正規化数では、値が 0 に近づくにつれて有効桁数が減少する。また、一部の環境では、非正規化数の演算によって処理性能が低下することがある。

3.5.2.1 特殊な値

通常の形式で表現される有限の非ゼロの浮動小数点数を**正規化数**という。正規化数では、仮数部の桁を有効に使って値を表現するため、その型に応じた通常の精度が得られる。一方、正規化数として表現できる範囲よりも絶対値が小さく、0 ではない値は、処理系が対応している場合、**非正規化数**として表現される。非正規化数では、値が 0 に近づくにつれて表現できる有効桁数が減少する。

浮動小数点数には、正規化数や非正規化数のほかに、正のゼロと負のゼロ、無限大、非数などの特殊な値がある。IEEE 754 型の浮動小数点環境では、**無限大**を表す INFINITY と、**非数**を表す NAN が、ヘッダファイル math.h にマ

クロとして定義されている。正の無限大は、有限のいかなる浮動小数点数よりも大きい値として扱われる。また非数は、数値として解釈できない演算結果などを表すために用いられる。これらを printf 関数で書式出力すると、通常は inf, nan などと表示される²³。

IEEE 754 に準拠し、浮動小数点例外をトラップしない一般的な処理系では、正の有限値を 0.0 で除算すると、正の無限大が得られる場合がある。ただし、C 言語の基本的な規則では 0 による除算の動作は未定義であり、この結果を全ての処理系で前提にしてはならない。また非数は、無限大から無限大を引いた場合などに現れる。

正規化数かどうかを判定するマクロ isnormal がある。

書き方： isnormal(値)

「値」が正規化数の場合は非 0 を、それ以外の場合は 0 を整数値として返す。このマクロはヘッダファイル math.h 内に定義されている。

■ 浮動小数点数の分類

浮動小数点数の分類を表 10 に示す。

表 10: 浮動小数点数の分類

分類	例	備 考
正規化数	1.0 0.5 2.31e5	通常の形式で表現される有限の非ゼロ値
ゼロ	0.0 -0.0	正のゼロと負のゼロがある
非正規化数	DBL_MIN/2	0 に近い有限の非ゼロ値。有効桁数が少なくなる
無限大	INFINITY -INFINITY	正の無限大と負の無限大がある
非数	NAN	数値としての大小関係を持たない特殊な値

次に示すプログラム test003-b.c は、非正規化数の作成とその検査、無限大と非数の作成などに関するサンプルである。

プログラム：test003-b.c (IEEE 754 型の浮動小数点環境を前提とする例)

```

1  #include <stdio.h>
2  #include <float.h>
3  #include <math.h>
4
5  int main() {
6      double maxp1, min03, min07;
7      // 値の作成
8      maxp1 = DBL_MAX + 1.0; // 1.0を加えても値が変化しない例
9      min03 = DBL_MIN * 0.3; // 非正規化数
10     min07 = DBL_MIN * 0.7; // 非正規化数
11     // 出力
12     printf("maxp1: %.17e + 1 = %.17e\n", DBL_MAX, maxp1);
13     printf("min03: %.17e * 0.3 = %.17e\n", DBL_MIN, min03);
14     printf("min07: %.17e * 0.7 = %.17e\n", DBL_MIN, min07);
15     // 正規化数かどうかを調べる
16     printf("[isnormal] DBL_MAX=>%d, DBL_MIN=>%d\n",
17            isnormal(DBL_MAX), isnormal(DBL_MIN));
18     printf("[isnormal] maxp1=>%d, 1.0=>%d, min03=>%d, min07=>%d\n",
19            isnormal(maxp1), isnormal(1.0), isnormal(min03), isnormal(min07));
20     // 無限大
21     double x = 1.0 / 0.0;
22     printf("1.0 / 0.0 = '%f'\n", x);
23     printf("1.0 / INFINITY = '%f'\n", 1.0/INFINITY);
24     printf("INFINITY is ");
25     if ( DBL_MAX < INFINITY ) {
26         printf("greater than DBL_MAX.\n");
27     } else {
28         printf("not greater than DBL_MAX.\n");
29     }
30     // 非数
31     x = INFINITY - INFINITY;
32     printf("INFINITY - INFINITY is '%f'\n", x);

```

²³処理環境によって表記が異なる場合もある。

このプログラムで使用している if … else … 文は後の「4.1 条件分岐」(p.39) で解説する。
このプログラムをコンパイルして実行する例を次に示す。

例. test003-b.c のコンパイルと実行

```
$ gcc test003-b.c -o test003-b Enter    ←コンパイル
$ ./test003-b Enter    ←実行
maxp1:  1.79769313486231571e+308 + 1 = 1.79769313486231571e+308
min03:  2.22507385850720138e-308 * 0.3 = 6.67522157552160514e-309
min07:  2.22507385850720138e-308 * 0.7 = 1.55755170095504087e-308
[isnormal] DBL_MAX=>1, DBL_MIN=>1
[isnormal] maxp1=>1, 1.0=>1, min03=>0, min07=>0
1.0 / 0.0 = 'inf'
1.0 / INFINITY = 0.000000
INFINITY is greater than DBL_MAX.
INFINITY - INFINITY is 'nan'.
```

この例では、DBL_MAX に 1.0 を加えても値が変化していない。これは、DBL_MAX に比べて 1.0 が小さ過ぎ、両者の差を double 型の精度で表現できないためである。これはオーバーフローではなく、浮動小数点数の有限の精度による現象である。また、min03 と min07 は 0 ではないが正規化数ではないため、isnormal は 0 を返している。

この環境では、1.0 を 0.0 で除算した結果として正の無限大が得られ、無限大から無限大を引いた結果として非数が得られている。これらの動作は、前述した IEEE 754 型の浮動小数点環境によるものである。

3.5.3 算術演算

表 11 に挙げる演算子は**算術演算**のためのものである。算術演算は数値の計算の最も基本的なものである。

表 11: 算術演算子

演算子	記述例	解説
+	$x + y$ +x	x と y の和 x の値をそのまま得る単項プラス演算子
-	$x - y$ -x	x から y を引いた値. 差 x の符号を反転した値を得る単項マイナス演算子
*	$x * y$	x と y の積
/	x / y	x を y で割った値
%	$x \% y$	x を y で割ったときの余り. 整数の剰余. 浮動小数点数には使えない

注意) 整数同士の除算では、小数部分は 0 の方向に切り捨てられる。例えば、7/3 は 2、-7/3 は-2 となる。

また、整数 x, y に対して x/y と $x\%y$ が定義される場合、

$$x == (x/y)*y + x\%y$$

の関係が成り立つ。

3.5.3.1 インクリメント、デクリメント

実際のプログラミングでは、変数の値を 1 増やす（インクリメントする）、あるいは 1 減らす（デクリメントする）処理を記述する頻度が多い。そのような処理を簡潔に記述するための演算子「++」、「--」があり、対象の変数の前、あるいは後に記述する。変数の前後のどちらにこれらの演算子を書くかによって、式が値を与える時点と、変数の値が変化する時点との関係が異なる。次のプログラム test003-d.c でこのことを確認する。

プログラム：test003-d.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x;
5     // 試み：その 1
6     x = 3;
7     printf("value of x : %d\n",x);
```



```

8      printf("value of x++ : %d\n",x++);  // 変更前の値を渡した後で増加
9      printf("value of x : %d\n",x);
10     printf("-----\n");
11     // 試み：その2
12     x = 3;
13     printf("value of x : %d\n",x);
14     printf("value of ++x : %d\n",++x);  // 増加後の値を渡す
15     printf("value of x : %d\n",x);
16 }

```

このプログラムは、変数 x のインクリメントの効果とタイミングを示すものである。このプログラムをコンパイルして実行する例を次に示す。

例. test003-d.c のコンパイルと実行

```

$ gcc test003-d.c -o test003-d  ←コンパイル
$ ./test003-d  ←実行
value of x : 3
value of x++ : 3      ← x++ の式の値は、変更前の3
value of x : 4      ←変数 x 自体は1増えている
-----
value of x : 3
value of ++x : 4      ←先に x を1増やし、増加後の4を式の値とする
value of x : 4

```

この例からわかるように、 $x++$ は変更前の x の値を式の値とし、その後で x の値を1増やす。これに対して、 $++x$ は先に x の値を1増やし、増加後の値を式の値とする。デクリメントについても、同様に前置と後置で式の値が異なる。

参考) $x = x++$ のように、同じ変数を一つの式の中で複数回変更すると、未定義動作になる場合がある。
このような記述をしてはならない。

3.5.4 浮動小数点数の誤差

C 言語で扱う浮動小数点数は有限個のビットで表現されるため、表現できる値の精度（有効桁数）には限界がある。また、10 進数では有限桁で表現できる小数であっても、2 進数では循環小数となり、有限のビット数では正確に表現できない場合がある。

一般的な double 型は、10 進数でおおむね 15~16 桁の精度を持つ。一方、保持されている近似値を 10 進数で詳しく観察するには、17 桁程度を表示する必要がある。次のプログラム test003-6.c は、double 型の浮動小数点数が誤差を含んでいることを確認するものである。

プログラム：test003-6.c

```

1  #include <stdio.h>
2
3  int main() {
4      double a = 1.0, b = 3.0, r;
5      r = a / b;
6      printf("%f / %f = %f\n",a,b,r);          // デフォルト（小数点以下6桁）
7      printf("detail: %.17g / %.17g = %.17g\n",a,b,r);    // 有効数字17桁で出力
8
9      a = 0.1;    b = 0.2;
10     r = a + b;
11     printf("%f + %f = %f\n",a,b,r);          // デフォルト（小数点以下6桁）
12     printf("detail: %.17g + %.17g = %.17g\n",a,b,r);    // 有効数字17桁で出力
13 }

```

このプログラムは2つのことを示す。1つは、 $1.0/3.0$ の正確な結果が有限桁では表現できず、近似値として保持されることである。もう1つは、10 進数では有限桁で表現できる値であっても、2 進数では正確に表現できない場合があることである。

このプログラムをコンパイルして実行する例を次に示す。

例. test003-6.c のコンパイルと実行

```
$ gcc test003-6.c -o test003-6 Enter ←コンパイル
$ ./test003-6 Enter ←実行
1.000000 / 3.000000 = 0.333333 ←正しく計算出来ているように見えるが…
detail: 1 / 3 = 0.33333333333333331 ←誤差がある
0.100000 + 0.200000 = 0.300000 ←正しく計算出来ているように見えるが…
detail: 0.100000000000000001 + 0.20000000000000001 = 0.30000000000000004 ←誤差がある
```

重要) 浮動小数点数は、多くの場合、実数の値を近似的に表現したものである。従って、計算結果には丸め誤差が含まれる可能性があり、値の比較や多数回の演算を行う場合には、誤差の影響を考慮しなければならない。

3.5.5 キャストによる型の変換

異なる型の値を織り交ぜて計算する場合、C 言語では計算結果を適切な型に自動的に調整する。この際に注意すべきことについて、次のプログラム test003-7.c を例に挙げて考える。

プログラム：test003-7.c

```
1  #include <stdio.h>
2
3  int main() {
4      int a = 3, b = 2;
5      double r;
6
7      // 除算での検証
8      r = a / b; // キャストなし
9      printf("no cast: %d / %d = %f\n", a, b, r);
10     r = (double)a / b; // キャストあり
11     printf("using cast: %d / %d = %f\n", a, b, r);
12
13     // オーバーフローの検証
14     a = 1000000;
15     b = 1000000;
16     r = a * b; // キャストなし
17     printf("no cast: %d * %d = %f\n", a, b, r);
18     r = (double)a * b; // キャストあり
19     printf("using cast: %d * %d = %f\n", a, b, r);
20 }
```

このプログラムの 8 行目では、int 型整数の除算を行った結果を double 型の変数 r に代入している。この場合、右辺の値が int 型として得られた後、それが double 型に変換されて r に代入される。int 型同士の除算では、演算結果の小数部分は捨てられ、0 の方向へ丸められる。

10 行目には「(double)a」という部分があるが、これは**キャスト**と呼ばれるもので、

(変換先の型) 式

と書く。これにより、「式」の値を「変換先の型」に変換する。従って、上のプログラムでは、int 変数 a の値が double 型に変換される。これにより、double 型の値を int 型の b で除算することになる。この場合、通常の算術型変換 (usual arithmetic conversions) によって b の値も double 型に変換され、double 型同士の除算が行われる。従って、演算結果も double 型となる。

注) 先の (double)a は、変数 a 自体の型を変更するものではなく、この式で用いる a の値を一時的に double 型へ変換するものである。

14~16 行目では、int 型同士の乗算結果が int 型の上限を超えるため、未定義動作となる。その後で結果を double 型の変数に代入しても、右辺の演算で既に未定義動作が発生しているため、正しい値は得られない。これに対して 18 行目ではキャストを用いて a の値を double に変換しているため、乗算が double 型の演算として行われ、この例では正しい値が得られる。

このプログラムをコンパイルして実行する例を次に示す。

例. test003-7.c のコンパイルと実行 (未定義動作となる例)

```
$ gcc test003-7.c -o test003-7  ←コンパイル
$ ./test003-7  ←実行
no cast:    3 / 2 = 1.000000    ←小数点以下が切り捨てられている
using cast: 3 / 2 = 1.500000    ← double として計算されるため、結果が小数点以下を含んでいる
no cast:    1000000 * 1000000 = -727379968.000000 ← int 同士の計算のため、正しくない結果となる
using cast: 1000000 * 1000000 = 1000000000000.000000 ← double として計算 (正しい結果)
```

重要) キャストによって式中の値の型を明示的に変換すると、通常の算術型変換によって、その演算が行われる型を変えることができる。

考察. test003-7.c の 10 行目を「`r = (double)(a / b);`」とするとどのような結果になるか確かめ、原因について考えよ。

3.5.6 数学関数

ヘッダファイル `math.h` にプロトタイプ宣言されている関数の中で、使用頻度の高いものを表 12 に挙げる。表に挙げた関数は `double` 型の引数を取り、戻り値も `double` 型である。

表 12: 各種の `double` 型の数学関数 (一部)

関数	解説	関数	解説	関数	解説
<code>sin(x)</code>	正弦関数	<code>cos(x)</code>	余弦関数	<code>tan(x)</code>	正接関数
<code>asin(x)</code>	逆正弦関数	<code>acos(x)</code>	逆余弦関数	<code>atan(x)</code>	逆正接関数
<code>atan2(y,x)</code>	座標 (x,y) の偏角*	<code>pow(x,y)</code>	x^y	<code>sqrt(x)</code>	$\sqrt{x}$
<code>exp(x)</code>	e^x	<code>log(x)</code>	$\ln x$	<code>log10(x)</code>	$\log_{10} x$
<code>sinh(x)</code>	双曲線正弦関数	<code>cosh(x)</code>	双曲線余弦関数	<code>tanh(x)</code>	双曲線正接関数
<code>asinh(x)</code>	逆双曲線正弦関数	<code>acosh(x)</code>	逆双曲線余弦関数	<code>atanh(x)</code>	逆双曲線正接関数
<code>fmax(x,y)</code>	x, y の大きい方	<code>fmin(x,y)</code>	x, y の小さい方	<code>hypot(x,y)</code>	$\sqrt{x^2 + y^2}$
<code>fabs(x)</code>	絶対値	<code>floor(x)</code>	x 以下の最大の整数値	<code>ceil(x)</code>	x 以上の最小の整数値
<code>round(x)</code>	最近接整数へ丸める	<code>rint(x)</code>	現在の丸め方向に従う	<code>fmod(x,y)</code>	x/y の浮動小数点剰余

※三角関数における角度は弧度法 (単位: ラジアン)。atan2 は引数の順序に注意

※ rint 関数の丸め方向に関しては後述する。

`round` 関数は、最も近い整数位置の浮動小数点数を返す。ちょうど中間の場合は、0 から遠い方の整数位置を返す。

`float` 型に対応する関数では、関数名の末尾に `f` を付ける。また、`long double` 型に対応する関数では、末尾に `l` (小文字のエル) を付ける。(表 13)

表 13: 型毎の関数名の例: `sin` の場合

float	double	long double
<code>sinf</code>	<code>sin</code>	<code>sinl</code>

これらの関数を使用するには、ソースプログラムに `math.h` をインクルードする。また、処理環境によっては、コンパイル時に「`-lm`」オプションを付ける必要がある場合がある。

参考) 巻末付録「F gnuplot」(p.564〜) で、三角関数などの計算結果を可視化する例を挙げている。

3.5.6.1 丸めモードの設定

浮動小数点数を整数位置へ丸める際の方向は、ヘッダファイル `fenv.h` に定義されている**丸めモード**によって指定できる。主な丸めモードを表 14 に示す。

表 14: 浮動小数点数の主な丸めモード

マクロ	丸める方向	マクロ	丸める方向
<code>FE_TONEAREST</code>	最も近い値へ丸める	<code>FE_DOWNWARD</code>	負の無限大の方向へ丸める
<code>FE_UPWARD</code>	正の無限大の方向へ丸める	<code>FE_TOWARDZERO</code>	0 の方向へ丸める

丸めモードを設定するには、fesetround 関数を用いる。

書き方： fesetround(丸めモード)

設定に成功した場合は 0 を返し、失敗した場合は非 0 を返す。

丸めモードに従う（あるいは従わない）関数を表 15 に挙げる。

表 15: 丸めモードに関連する主な関数

関数	戻り値の型	丸めモード	解 説
rint	double	従う	整数位置へ丸める
nearbyint	double	従う	rint とほぼ同様だが、不正確例外を発生させない
lrint	long int	従う	丸めた結果を整数型で返す
llrint	long long int	従う	丸めた結果を整数型で返す
round	double	従わない	中間値を 0 から遠い方向へ丸める
lround	long int	従わない	round と同じ規則で整数型の値を返す
llround	long long int	従わない	round と同じ規則で整数型の値を返す

rint, nearbyint, lrint, llrint は、現在設定されている丸めモードに従う。これに対して、round, lround, llround は丸めモードには従わず、ちょうど二つの整数の間にある値を 0 から遠い方向へ丸める。例えば、

```
fesetround(FE_DOWNWARD);
```

とすると、rint 関数は負の無限大の方向へ丸める。この場合、rint(2.7) は 2.0、rint(-2.3) は -3.0 となる。

プログラム開始時の丸めモードは通常 FE_TONEAREST であり、多くの処理系では、二つの整数のちょうど中間にある値を偶数側へ丸める。

注意) 全ての処理系が、表 14 に挙げた全ての丸めモードをサポートするとは限らない。また、丸めモードの変更は、その後に行われる浮動小数点演算にも影響する場合がある。現在の丸めモードは fegetround 関数で取得できるため、必要に応じて変更前の設定を保存し、処理後に元へ戻すことが望ましい。

例. 丸めモードの取得と復元

```
int old_mode = fegetround(); // この時点の丸めモードを取得
if (fesetround(FE_DOWNWARD) == 0) {
    /* 必要な処理 */
    fesetround(old_mode); // 丸めモードを old_mod に戻す
}
```

3.5.6.2 乱数の生成

C 言語の標準のライブラリには、**乱数**を生成するための rand 関数が提供されている。この関数のプロトタイプは stdlib.h にあり、この関数を使用する際はこれをインクルードする。

プロトタイプ： int rand(void)

rand は 0 から RAND_MAX まで（両端を含む）の**疑似乱数**²⁴ を整数値として生成して返す。RAND_MAX は stdlib.h に定義されている²⁵。rand 関数が乱数列を生成する具体的なアルゴリズムは処理系に依存するが、生成される値は真の乱数ではなく、内部状態から計算によって生成される疑似乱数である。

rand 関数による疑似乱数列の生成は、**種** (seed) によって初期化される。すなわち、同じ種を設定すると、同じ処理系では、それ以降に同じ疑似乱数列が生成される。種の設定は srand 関数で行う。

プロトタイプ： void srand(unsigned int seed)

次のプログラム test003-e.c は、種の設定を省略した場合の乱数列と、同じ種を設定すると同じ乱数列が生成されることを示すものである。なお、srand を一度も呼び出さずに rand を使用した場合は、種として 1 を指定した場合と同じ乱数列が生成される。

²⁴一様乱数であることは保証されていない。

²⁵Windows の MSYS2 などの環境では、RAND_MAX は 32767 ($2^{15} - 1$) となる場合がある。一方、Linux や macOS の Xcode などの環境では 2147483647 ($2^{31} - 1$) となることが多い。このように、rand 関数の生成する乱数の範囲は処理系に依存する。

プログラム：test003-e.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int c;
6     // 種の指定なし（デフォルトの種1）で乱数生成
7     printf("[no seed setting]\n");
8     for ( c=0; c<10; c++ ) printf("%d, ",rand());
9     // 種3で乱数生成
10    srand(3);
11    printf("\n[seed:3]\n");
12    for ( c=0; c<10; c++ ) printf("%d, ",rand());
13    // 再度種3を設定して乱数生成
14    srand(3);
15    printf("\n[seed:3]\n");
16    for ( c=0; c<10; c++ ) printf("%d, ",rand());
17    printf("\n");
18 }
```

※このプログラムでは反復処理のための for 文を使っている。詳しくは後の「4.2 反復」(p.47) で解説する。

このプログラムをコンパイルして実行する例を次に示す。

例. test003-e.c のコンパイルと実行

```
$ gcc test003-e.c -o test003-e  ←コンパイル
$ ./test003-e  ←実行
[no seed setting]           ↓種 1 で初期化された乱数列
41, 18467, 6334, 26500, 19169, 15724, 11478, 29358, 26962, 24464,
[seed:3]                   ↓種「3」で初期化された乱数列
48, 7196, 9294, 9091, 7031, 23577, 17702, 23503, 27217, 12168,
[seed:3]                   ↓同じ種からは同じ乱数列ができる
48, 7196, 9294, 9091, 7031, 23577, 17702, 23503, 27217, 12168,
```

注意) rand 関数が生成する疑似乱数は安全性を目的として設計されたものではなく、予測される可能性があるため、暗号、パスワード、秘密鍵など、安全性が要求される用途には使用してはならない。

■ 種の設定の実用的な方法

乱数生成の種には、現在の時刻の情報を与える方法がよく用いられる。時々刻々変化する値を種にすることで、プログラムを実行するたびに異なる疑似乱数列を生成しやすくなるためである。具体的には、時刻を取得する time 関数²⁶（プロトタイプは time.h に宣言されている）を使用する。

注意) 短い時間内に time 関数を繰り返し実行すると同じ時刻の値が得られ、これを種にすると、同じ疑似乱数列が生成される場合があるので注意が必要である。

この方法で乱数生成を初期化する例を次のプログラム test003-f.c に示す。

プログラム：test003-f.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main() {
6     unsigned int seed;
7
8     // 現在時刻の情報で種を設定
9     seed = (unsigned int)time(NULL);
10    srand(seed);
11
12    printf("[seed:%u]\n", seed);
13    for (int c=0; c<10; c++) printf("%d, ", rand());
14    printf("\n");
15 }
```

※このプログラムでは反復処理のための for 文を使っている。詳しくは後の「4.2 反復」(p.47) で解説する。

²⁶後の「9.12 日付、時刻の扱い」(p.154～) で解説する。

このプログラムをコンパイルして実行する例を次に示す。

例. test003-f.c のコンパイルと実行

```
$ gcc test003-f.c -o test003-f  ←コンパイル
$ ./test003-f  ←実行
[seed:1773983663]           ↓種「1773983663」で初期化された乱数列
29511, 25774, 19669, 27645, 8605, 14759, 12343, 5166, 17984, 14168,
$ ./test003-f  ←再度実行すると
[seed:1773985308]           ↓種「1773985308」で初期化された乱数列（上とは異なる）
2115, 12182, 13335, 28052, 2256, 18933, 20517, 6695, 14878, 25909,
```

■ rand 関数の品質について

rand 関数が生成する乱数は完全にランダムな値ではなく、ある規則に従って計算された疑似乱数である。C 言語の規格では、rand 関数が乱数を生成する具体的な方法や、生成される値の統計的な品質は規定されていない。処理系によっては比較的単純な生成法が用いられており、周期が十分に長くなかったり、下位ビットの性質が良くなかったりする場合がある。そのため、簡単な利用には便利であるが、高い品質の乱数が求められる数値実験や統計的処理では、より高品質な乱数生成法²⁷を用いる必要がある。

3.5.7 複素数型

C 言語では、C99 以降の言語規格において、複素数を扱うための型と演算機能が標準で用意されている。複素数とは、実数 a 、 b と虚数単位 i を用いて、

$$a + bi$$

の形で表される数である。このとき、 a を実部、 b を虚部と呼ぶ。また、虚数単位 i は、 $i^2 = -1$ を満たす数である。

C 言語の複素数型は、整数型や浮動小数点型と同様に、算術型の一種である。従って、複素数型の値に対しても、加算、減算、乗算、除算などの算術演算を行うことができる。また、標準ヘッダファイル `complex.h` には、実部や虚部の取得、絶対値や偏角の計算、複素共役、平方根、指数関数、対数関数、三角関数など、複素数を扱うための機能が用意されている。

C 言語では、使用する浮動小数点数の精度に対応して、次の 3 種類の複素数型が用意されている。

型	実部と虚部に用いられる型
<code>float _Complex</code>	<code>float</code>
<code>double _Complex</code>	<code>double</code>
<code>long double _Complex</code>	<code>long double</code>

通常は、`complex.h` でマクロとして定義される `complex` を用いて、`double complex` などと記述することができる。この `complex` は、C 言語に組み込まれたキーワード `_Complex` に展開される。

3.5.7.1 虚数単位

`complex.h` には、虚数単位 i を表すマクロ `I` が定義されている。これを用いると、実部が a 、虚部が b である複素数 z を、次のように記述することができる。

```
double complex z = a + b * I;
```

例えば、実部が 1.0、虚部が 2.0 である複素数 z は、次のように記述する。

```
double complex z = 1.0 + 2.0 * I;
```

ここで、`I` は虚数単位そのものを表し、`2.0 * I` が虚部 $2.0i$ に対応する。

3.5.7.2 実部と虚部の取得

複素数の実部と虚部は、それぞれ `creal` 関数、`cimag` 関数を用いて取り出すことができる。例えば、先に例示した z から実部、虚部を取得するには次のようにする。

²⁷C++ の場合の乱数生成法について、「22.1 乱数」(p.419～)で解説する。


```
double re = creal(z);    /* 実部 */
double im = cimag(z);    /* 虚部 */
```

この例では、re の値は 1.0、im の値は 2.0 となる。

3.5.7.3 複素数用の数学関数

標準ヘッダファイル `complex.h` には、複素数を引数として受け取る数学関数が用意されている。代表的な関数については、`math.h` で宣言される実数用関数の名前の先頭に `c` を付けたものが、複素数用の関数名となる。例えば、実数用の `sqrt`, `exp`, `log` に対応する複素数用関数は、それぞれ `csqrt`, `cexp`, `clog` である。

ただし、`math.h` で宣言される全ての関数に、先頭へ `c` を付けた複素数版が存在するわけではない。例えば、値を整数方向へ丸める `floor`, `ceil`, `round`, `trunc` に対応する `cfloor`, `cceil`, `cround`, `ctrunc` は用意されていない。また、剰余を求める `fmod`, `remainder` に対応する `cfmod`, `cremainder` も用意されていない。

さらに、浮動小数点数を仮数部と指数部に分解する `frexp`, 2 の整数乗を乗じる `ldexp`, 整数部と小数部に分ける `modf` に対応する `cfrexp`, `cldexp`, `cmodf` も、標準 C には存在しない。これらの処理が必要な場合は、処理の目的に応じて実部と虚部を個別に処理するか、通常の実数演算を組み合わせて実現する。

一方、`complex.h` には、実数用関数の名前の先頭に `c` を付けるという単純な対応によらない、複素数を扱うための関数も用意されている。主なものを表 16 に示す。

表 16: 複素数を扱う主な関数（一部）

関数	機 能	戻り値の型
<code>creal</code>	実部を取り出す	実数型
<code>cimag</code>	虚部を取り出す	実数型
<code>cabs</code>	絶対値を求める	実数型
<code>carg</code>	偏角を求める	実数型
<code>conj</code>	複素共役を求める	複素数型
<code>cproj</code>	リーマン球面への射影を求める	複素数型

■ 型に応じた関数

`math.h` の関数と同様に、複素数用の関数にも、型すなわち演算精度に応じたものが用意されている。`float _Complex` 用の関数名の末尾には `f` が、`long double _Complex` 用の関数名の末尾には `l` が付く。その一例を表 17 に示す。

表 17: 型毎の関数名の例：`creal`, `cimag` の場合

目的	<code>float _Complex</code>	<code>double _Complex</code>	<code>long double _Complex</code>
実部の取得	<code>crealf</code>	<code>creal</code>	<code>creall</code>
虚部の取得	<code>cimagf</code>	<code>cimag</code>	<code>cimagl</code>

3.5.7.4 複素数の出力

`printf` には複素数専用の変換指定子は用意されていない。複素数を出力する場合は、`creal` と `cimag` を用いて実部と虚部を取り出し、それぞれを通常の実数として出力する。

■ サンプルプログラム

次のプログラム `test003-h.c` では、複素数用の数学関数を用いた計算例を示す。最初に、関数 `csqrt` を用いて負の実数 -2.0 の平方根を複素数として求める。次に、関数 `cabs` を用いて複素数 $3 + 4i$ の絶対値を求める。最後に、関数 `carg` を用いて複素数 $1 + i$ の偏角を求め、ラジアンと度数法の両方で表示する。複素数として得られた平方根は、関数 `creal` と `cimag` を用いて実部と虚部に分けて出力している。

プログラム：test003-h.c

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <complex.h>
4
5 int main() {
6     double complex z1 = csqrt( -2.0 ); //  $\sqrt{-2}$ 
```

```

7   printf("sqrt(-2.0) = %f%fi\n", creal(z1), cimag(z1));
8
9   double complex z2 = 3.0 + 4.0 * I;    // 3+4i
10  double r2 = cabs(z2);                // |3+4i|
11  printf("abs(3.0+4.0i) = %f\n", r2);
12
13  double complex z3 = 1.0 + 1.0 * I;    // 1+i
14  double r3 = carg(z3);                // 偏角
15  double pi = acos(-1.0);              // π
16  printf("arg(1.0+1.0i) = %f rad = %f deg\n", r3, r3/pi*180.0);
17 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test003-h.c のコンパイルと実行

```

$ gcc test003-h.c -o test003-h -lm  ←コンパイル
$ ./test003-h  ←実行
sqrt(-2.0) = 0.000000+1.414214i    ←  $\sqrt{-2} = \sqrt{2}i$ 
abs(3.0+4.0i) = 5.000000          ←  $|3+4i| = 5$ 
arg(1.0+1.0i) = 0.785398 rad = 45.000000 deg ←  $1+i$  の偏角は  $\pi/4=45^\circ$ 

```

3.6 配列

配列は、同じ型の要素を複数格納できるデータ領域である。変数宣言の際に

型名 変数名 [要素数]

と書くと、指定した「要素数」の要素を持つ配列が確保される。このように、配列には [...] という添字を付けて使用する。また、添字には、格納位置を意味する番号であるインデックスを指定する。次に示すプログラム test003-5.c は 3 つの整数の要素を持つ配列を宣言して使用するものである。

重要) n 要素の配列のインデックスの範囲は 0 から n-1 までである。また、配列はメモリ上で連続した領域に確保される。

プログラム：test003-5.c

```

1  #include <stdio.h>
2
3  int main() {
4      int a[3];
5      a[0] = 1;
6      a[1] = 2;
7      a[2] = 3;
8      printf("%d,%d,%d\n",
9              a[0], a[1], a[2]);
10 }

```

実行例.

```

$ gcc test003-5.c -o test003-5 
$ ./test003-5  ←実行    ↑コンパイル
1,2,3 ←配列の 3 つの要素の値

```

C 言語の配列は一次元の考え方を基本として、これを拡張して多次元の配列を実現する。言い換えると、「配列を要素とする配列を宣言する」ことにより、多次元の配列を実現する。

例えば「2 行 3 列」の二次元配列を作成するには、

```
int a[2][3];
```

などとして宣言する。これは「3 要素の配列を 2 セット」作成したことになる。この配列にアクセスする際は、

a[行番号][列番号] (番号は 0 から始まる)

と記述する。同様に、高次元の配列を作成するには、

```
int a[n1][n2][n3]...
```

などと記述する。これも、 $n2 \times n3 \times \dots$ の配列を $n1$ セット確保するという考え方である。

■ “添字”という用語について

配列の要素を使用する際に、配列名の右側に付ける [...] は、厳密には添字演算子という。また、添字演算子の角括弧の中に記述する式を添字 (subscript) と呼ぶ。

本書では、配列要素の位置を表す整数値という意味で、添字をインデックスと呼ぶ場合もある。

3.6.1 配列宣言時の初期化

配列は宣言の際に初期値を与えることができる。初期値は { ... } で記述する。これは、プログラムのブロックと似た形式ではあるが、初期化の場合はブロックではなく**初期化子**と呼ばれるものである。個々の初期値はコンマ「,」で区切る。配列宣言時の初期化の例を次のプログラム test003-5-2.c で示す。

プログラム：test003-5-2.c

```
1 #include <stdio.h>
2
3 int main() {
4     int a[3] = {1, 2, 3};           // 「初期化子」で初期化
5     int b[] = {4, 5, 6};           // サイズを自動取得
6     // 「初期化子」中に「指定子」で要素を指定すると任意の記述順序となる
7     int c[3] = {[1]=8,[0]=7,[2]=9};
8
9     printf("a:%d,%d,%d, ", a[0],a[1],a[2]);
10    printf("b:%d,%d,%d, ", b[0],b[1],b[2]);
11    printf("c:%d,%d,%d\n", c[0],c[1],c[2]);
12 }
```

このプログラムの配列 b の宣言にあるように、配列の要素数を省略して初期化子を与えると、要素数が初期化子の内容から自動的に決定される。また、配列 c の初期化の記述のように、初期化子の中には対象要素（配列の添字）を明示するための**指示子**（designator）²⁸を使用することができる。これを用いると、任意の順序で要素の初期値を記述することができる。

このプログラムをコンパイルして実行する例を次に示す。

例. test003-5-2.c のコンパイルと実行

```
$ gcc test003-5-2.c -o test003-5-2  ←コンパイル
$ ./test003-5-2  ←実行
a:1,2,3, b:4,5,6, c:7,8,9 ←結果
```

重要） 初期化子は配列の宣言時にだけ使用できる。宣言後の配列全体に、{...} を用いて一括して値を代入することはできない。

初期化子で配列を初期化する際、整数配列の場合に初期化を省略した要素の値は 0 となる。このことを次のプログラム test003-5-3.c で示す。

プログラム：test003-5-3.c

```
1 #include <stdio.h>
2
3 int main() {
4     int a[3] = {1, 2};             // 末尾の要素の初期化を省略
5     int b[] = {4};                 // この場合は要素の個数は1となる
6
7     // 「指定子」を用いた初期化の際、初期値設定の省略があるケース
8     int c[3] = {[1]=8,[0]=7};     // 最終要素の初期化を省略
9
10    printf("a:%d,%d,%d, ", a[0],a[1],a[2]);
11    printf("b:%d, ", b[0]);
12    printf("c:%d,%d,%d\n", c[0],c[1],c[2]);
13 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test003-5-3.c のコンパイルと実行

```
$ gcc test003-5-3.c -o test003-5-3  ←コンパイル
$ ./test003-5-3  ←実行
a:1,2,0, b:4, c:7,8,0 ←結果
```

初期化子による値の指定を省略した要素の値が 0 になっていることがわかる。

²⁸ 指定子と呼ぶ場合もある。

3.6.2 配列全体を複製する方法

配列全体をイコール「=」による代入で複製することはできない。配列は、そのままでは配列全体を1つの値として代入することができず、C言語では配列自体を代入演算子の左辺に置くことが認められていないためである。

例. 許されない方法

```
int a[5] = {1,2,3,4,5};
int b[5];
b = a;           // これはコンパイルエラー
```

従って、配列を複製するには、複写元の各要素を、対応する複写先の要素へ個別に代入する必要がある。

例. 配列の複製（先の例の続き）

```
b[0] = a[0]; b[1] = a[1]; b[2] = a[2]; b[3] = a[3]; b[4] = a[4];
```

この方法は配列の要素数が増えると煩雑になる。従って、後に解説する反復制御の構文を用いて全要素の代入を実行する²⁹と良い。あるいは、後の「5.2.1 構造体のメンバとしての配列の扱い」(p.55)で解説するように、配列を構造体のメンバに組み込んで取り扱う方法もある。

重要) 配列はそのままでは、配列全体を1つの値として「=」で代入することができない。

3.7 文字と文字列

文字は各種のエンコーディング³⁰に基づいて「数値」として表現される。例えば、半角英数文字のエンコーディングとして最も広く用いられているものに ASCII³¹がある。ASCIIでは、英大文字「A」に65という文字コードが割り当てられている。従って、ASCII互換の実行文字集合を用いる一般的な環境では、文字「A」は数値65として扱われる。

参考) 実際には、使用する計算機環境やコンパイラによって採用されるエンコーディングが決まる。

現在の多くの環境では ASCII 互換のエンコーディングが用いられている。

ASCIIの1文字を表すコードは7ビットで表現されるため、一般的な8ビットの1バイトに格納できる。従って、ASCII文字などの1バイト文字は、char型の1つの値として扱うことができる。また、文字列はchar型の配列として取り扱う。

文字と文字列を扱うプログラム test004.c を次に示す。このプログラムでは、単独の文字（キャラクタ）を変数 c に、文字列を s1, s2 に格納している。

プログラム：test004.c

```
1 #include <stdio.h>
2
3 int main() {
4     char c    = 'A';           // 単独のキャラクタ
5     char s1[6] = "ABCDE";      // 明にサイズ指定して宣言
6     char s2[]  = "FGHIJ";      // サイズを自動取得
7     printf("'%c' = %d\n", c, c); // キャラクタとそのコード
8     printf("s1:%s, s2:%s\n", s1, s2); // 文字列として出力
9     printf("s1:%d,%d,%d,%d,%d,%d\n", // s1の全要素のコードを調べる
10         s1[0], s1[1], s1[2], s1[3], s1[4], s1[5]);
11 }
```

変数 c に 'A' を代入している。このような単引用符による表記は、文字コードの数値を表すリテラルである。文字列のリテラルは二重引用符で表記する。また、s2 の配列を宣言する際には要素数を省略しているが、初期値から自動的に決定される。この場合、5 個の文字と末尾のヌル文字を格納するため、要素数は 6 となる。

このプログラムの 7 行目では、printf 関数に変換指定子「%c」を使用して、文字コードに対応する文字（キャラクタ）を出力している。また、文字列を出力するには「%s」を使用する。

このプログラムをコンパイルして実行すると、次のような出力が得られる。

²⁹ただしその場合は、複写先に十分な領域がなければならない。

³⁰個々の文字を数値に対応づける規則（取り決め）として様々なものが標準化されている。

³¹現実的に普及している計算機環境で半角英数文字を扱う際に一般的に用いられている。

例. test004.c のコンパイルと実行

```
$ gcc test004.c -o test004  ←コンパイル
$ ./test004  ←実行
'A' = 65 ←文字と文字コードの出力
s1:ABCDE, s2:FGHIJ ←文字列の出力
s1:65,66,67,68,69,0 ←文字列 s1 の内容を敢えて数値として出力 (末尾が 0 になっている)
```

C 言語では、値 0 を持つヌル文字を文字列の終端に置く。s1 の各要素を数値として確認すると、末尾の要素が 0 になっていることがわかる。char 型の配列を文字列として扱う場合は、末尾にヌル文字を置く必要がある。

重要) C 言語では、文字列の終端を、値 0 を持つ**ヌル文字** '\0' で表す。ヌル文字は、数字を表す文字 '0' とは異なる。

3.7.1 文字コードのリテラル表現

文字コードの数値を文字として表すには、単引用符の中に「\」と 8 進数（もしくは 16 進数）を組み合わせた表記を記述する。このような表記は**エスケープシーケンス**³² の一種である。また、単引用符を含む表記全体を**文字定数** (character constant) と呼ぶ。例えば、ASCII における文字 'A' の文字コードは 65 である。これを 8 進数で表すと 101 であるため、文字定数として '\101' と記述できる。また、16 進数では 41 であるため、'\x41' と記述できる。

3.7.2 文字列の長さ

strlen 関数を用いると、文字列の長さを求めることができる。

プロトタイプ: `size_t strlen( const char *s );`

文字列 s の長さを返す。この関数のプロトタイプはヘッダファイル string.h に宣言されている。

重要) ヌル文字で正しく終端された文字列では、文字列の長さは、それを格納する char 配列の要素数よりも必ず小さくなる。

この関数を用いて文字列の長さを求めるプログラム test004-4.c を次に示す。

プログラム: test004-4.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char s1[] = "abc";
6     char s2[] = "This is a pen.";
7     char s3[] = ""; // 空文字列
8     printf("%s:(%zu)\n", s1, strlen(s1));
9     printf("%s:(%zu)\n", s2, strlen(s2));
10    printf("%s:(%zu)\n", s3, strlen(s3));
11 }
```

実行例.

コンパイル↓

```
$ gcc test004-4.c -o test004-4 
$ ./test004-4  ←実行
abc:(3) ← 3 文字の長さ
This is a pen.:(14) ← 14 文字の長さ
:(0) ← 0 文字の長さ (空文字列)
```

strlen 関数は、文字列の先頭から**ヌル文字**が現れるまでの char 型の要素数を求める。この場合、ヌル文字は個数には含めない。ASCII 文字だけからなる今回の例では、この値は表示される文字数と一致する。

重要) strlen 関数が返す値は、文字列の先頭から終端のヌル文字までに格納されている char 型の要素数であり、表示される文字の個数とは限らない。例えば、UTF-8 などのマルチバイト文字を含む文字列では、1 文字が複数の char 型の要素で表現されるため、strlen 関数の戻り値は文字数より大きくなる場合がある。

³²後の「3.8.1 エスケープシーケンス」(p.37) で解説する。

3.8 書式文字列と変換指定子

書式文字列は変換指定子を含む文字列であり、printf 関数の第一引数にこれを与える。この際に、変換指定子に対応する値を後続の引数から埋め込む。代表的な変換指定子には表 18 に挙げるようなものがある。

表 18: 代表的な変換指定子

変換指定子	意味	対応する代表的な型
%d, %u	10 進整数	前者は int, 後者は unsigned int (符号なし)
%hd, %hu	10 進整数	前者は short, 後者は unsigned short (符号なし)
%ld, %lu	10 進整数	前者は long, 後者は unsigned long (符号なし)
%lld, %llu	10 進整数	前者は long long, 後者は unsigned long long (符号なし)
%jd, %ju	10 進整数	前者は intmax_t, 後者は uintmax_t (符号なし)
%o, %lo, %llo	8 進整数	それぞれ unsigned int, unsigned long, unsigned long long に対応
%x, %lx, %llx	16 進整数(小文字)	それぞれ unsigned int, unsigned long, unsigned long long に対応
%X, %lX, %lX	16 進整数(大文字)	それぞれ unsigned int, unsigned long, unsigned long long に対応
%f, %lf, %Lf %e, %le, %Le %g, %lg, %Lg	浮動小数点数	前者 2 つは double (float も可), 後者は long double 指数表現 指数表現か否かを自動選択
%a, %la, %La %A, %lA, %LA	16 進浮動小数点数 (指数表現)	前者 2 つは double (float も可), 後者は long double 上は小文字 (例: ff), 下は大文字 (例: FF)
%c %s	文字 文字列	char char 配列, char ポインタ
%zu, %zd	サイズ整数	前者は size_t, 後者はそれに対応する符号付き整数型
%p	アドレス値	void ポインタ. ポインタをアドレス値表現 (通常は 16 進数) にする. (形式の詳細は処理環境に依存する)

注) ssize_t は POSIX 系環境で定義される型であり, C 言語標準ではない。

注意) 後の「7.3.2.2 書式指定の読み込み: scanf」(p.73) で解説する scanf 関数では、書式指定子の記述が厳格である。例えば、%f, %lf, %Lf などに対応する型に応じて正しく使い分けなければならない。これは、scanf がポインタを通じてメモリに値を書き込むためである。

注意) ポインタ値を %lu などで出力すると、一見正しく表示される場合があるが、これは C 言語として保証された方法ではない。ポインタの値を出力する場合は、void * 型に変換して%p を用いる。

ポインタとは、変数などの実体が配置されているメモリ上の**アドレス**（メモリ上の番地、すなわち在り処）を保持するものである。「ポインタ」と「アドレス」については、後の「6 メモリとポインタ」(p.57) で詳しく解説する。

重要) printf 関数では、変換指定子と対応する引数の型を正しく一致させる必要がある。型が一致しない場合、正しい出力は保証されず、未定義動作となることがある。

「%」の後には、出力する値を配置する領域の幅を指定できる。指定した幅より表示内容が短い場合は、標準では右寄せとなる。左寄せにするには、幅の前にマイナス記号「-」を付ける。表示幅の指定例を表 19 に挙げる。

表 19: 表示幅と精度の指定例

例	解 説	例	解 説
%4d	幅 4 の 10 進整数	%04d	幅 4 の 10 進整数 (前に 0 を充填)
%-4d	幅 4 の 10 進整数 (左寄せ)	%12.6f	幅 12 の浮動小数点数 (小数点以下は 6 桁)
%32s	幅 32 の文字列 (右寄せ)	%-32s	幅 32 の文字列 (左寄せ)
%32.4s	幅 32 の文字列 (右寄せ) ただし、文字列の先頭から最大 4 バイトを配置する (ASCII 文字列では 4 文字)	%-32.4s	幅 32 の文字列 (左寄せ)

変換指定子を様々な形で応用した例をプログラム test004-1.c に示す。

プログラム：test004-1.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int      x      = 1234;
5      double   pi     = 3.141592653589793;
6      char     s[]    = "ABCD";
7      printf("-----1-----2-----3-----\n");
8      printf("%10d%10s : %-10s\n",x,s,"%10d%10s");
9      printf("%010d%10s : %-10s\n",x,s,"%010d%10s");
10     printf("%-10d%-10s : %-10s\n",x,s,"%-10d%-10s");
11     printf("%10.2s%10s : %-10s\n",s,s,"%10.2s%10s");
12     printf("%-10.2s%-10s : %-10s\n",s,s,"%-10.2s%-10s");
13     printf("%10.8f%010.5f : %-10s\n",pi,pi,"%10.8f%010.5f");
14     printf("%-10.2f%010.2f : %-10s\n",pi,pi,"%-10.2f%010.2f");
15     printf("-----1-----2-----3-----\n");
16 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test004-1.c のコンパイルと実行

\$ gcc test004-1.c -o test004-1 ←コンパイル

\$./test004-1 ←実行

```
-----1-----2-----3-----
      1234      ABCD : %10d%10s
0000001234      ABCD : %010d%10s
1234      ABCD      : %-10d%-10s
      AB      ABCD : %10.2s%10s
AB      ABCD      : %-10.2s%-10s
3.141592650003.14159 : %10.8f%010.5f
3.14      0000003.14 : %-10.2f%010.2f
-----1-----2-----3-----
```

■ 2 進数の変換指定子：%b

C17 までの規格には、整数を 2 進数として出力する標準の変換指定子は存在しない。ただし、一部の処理系では、拡張機能として %b を利用できる。また、C23 では %b が標準の変換指定子として追加された。

C17 以前の処理系拡張として提供される %b の詳細な機能は、処理環境によって異なる場合がある。このような挙動も処理系に依存するため、注意が必要である。

%b は GCC では利用できる環境がある一方で、他の処理系や標準準拠モードではエラーとなる場合がある。可搬性の高いプログラムを作成する場合には、%b に依存しないよう注意が必要である。

次の test004-5.c は、%b に対応した処理環境で、整数を 2 進数形式で出力する例である。

プログラム：test004-5.c

```
1  #include    <stdio.h>
2
3  int main() {
4      printf("[digit]\n01234567\n");
5      printf("-----\n");
6      printf("%b\n", 63);
7      printf("%8b\n", 63);
8      printf("%08b\n", 63);
9  }
```

実行例.

コンパイル↓

\$ gcc test004-5.c -o test004-5

\$./test004-5 ←実行

```
[digit]
01234567
-----
111111
  111111
00111111
```

3.8.1 エスケープシーケンス

文字列リテラルや文字定数には**エスケープシーケンス**を含めることができる。これは、バックスラッシュ「\」に続けて文字を書くことで表現する。エスケープシーケンスによって、通常の表記では記述しにくい文字や、改行、水平タブなどの制御文字を表現できる。printf 関数の書式文字列でも、これらを利用できる。

注意) 日本語環境では、バックスラッシュ「\」³³ が、フォントや端末の表示によって日本の通貨である「円」の記号「¥」の形で表示されることがある。この場合も、C 言語のソースコード上ではバックスラッシュとして扱われる。

代表的なエスケープシーケンスを表 20 に示す。

表 20: 代表的なエスケープシーケンス			
表記	意 味	表記	意 味
\n	改行	\t	水平タブ
\"	二重引用符 "	\'	単一引用符 '
\\	バックスラッシュ \	\0	文字列の終端に用いる ヌル文字

注) 水平タブは次のタブ位置まで表示位置を進める。タブ位置の間隔は端末や設定によって異なり、一般には 8 文字ごと、または 4 文字ごとに設定されることが多い。

エスケープシーケンスを応用したサンプルプログラム test004-2.c を次に示す。

プログラム：test004-2.c

```
1 #include <stdio.h>
2
3 int main() {
4     printf("----+----1----+----2----+----\n");
5     printf("ABC\tDEF\tGHI\t8-tab\n");           // TAB
6     printf("the first line.\nthe second line.\n"); // 改行
7     printf("double quotation: \" \"\n");         // "
8     printf("back slash:      \\ \n");           // ¥もしくは\
9 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test004-2.c のコンパイルと実行（水平タブが 8 文字間隔の端末の場合）

```
$ gcc test004-2.c -o test004-2  ←コンパイル
$ ./test004-2  ←実行
----+----1----+----2----+----
ABC    DEF    GHI    8-tab
the first line.
the second line.
double quotation: "
back slash:      \
```

³³文字コードは 10 進数で 92, 16 進数で 0x5c である。

3.9 型の別名の定義

typedef を用いた宣言によって、型の別名を定義できる。

《型の別名の定義》

書き方： `typedef 既存の型 新しい型名;`

「既存の型」で指定した型に対して、「新しい型名」で示す別名を与える。

既存の整数型 `int` に新たな型名（別名）`seisu` を与える例を `test004-3.c` に示す。

プログラム：`test004-3.c`

```
1 #include <stdio.h>
2
3 typedef int seisu;
4
5 int main() {
6     seisu a = 2, b = 3;
7     printf("%d + %d = %d\n", a, b, a+b);
8 }
```

実行例.

```
$ gcc test004-3.c -o test004-3 
$ ./test004-3  ←実行 ↑コンパイル
2 + 3 = 5 ←結果
```

この例だけでは、`typedef` によって型に別名を付けることの有用性はわかりにくいですが、後に解説する**構造体**³⁴の使用において有用性が明らかになる。

³⁴ 「5 構造体」(p.51) で解説する。

4 制御構造

ここでは反復や条件分岐といった、プログラムの実行の流れを制御するための方法について解説する。

4.1 条件分岐

プログラムでは、通常、記述された式や文が上から順に実行される。しかし、処理の途中で条件を判定し、その結果に応じて、実行する処理を選択したい場合がある。このように、条件の判定結果によってプログラムの実行の流れを変える仕組みを条件分岐という。

4.1.1 if 文

最も代表的な条件分岐構造に if 文がある。

《if 文》

書き方： if (条件判定式) {
 (「条件判定式」が真の場合の処理)
 } else {
 (「条件判定式」が偽の場合の処理)
 }

「条件判定式」が真の場合または偽の場合に実行する処理が1つの文だけである場合は、ブロックを表す { ... } を省略できる。ただし、処理の範囲を明確にするため、ブロックを省略しない書き方が一般に安全である。また、
「条件判定式」が偽の場合の処理
を記述しない場合は else 以降を省略することができる。

注意) 本書では、後に説明する条件式と区別するために条件判定式という用語を用いる。

if 文で条件判定を行う素朴なプログラム test005.c を次に示す。

プログラム：test005.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 7; // この値を変えて試す
5     if ( x > 5 ) {
6         printf("%d > 5\n", x);
7     } else {
8         printf("5 >= %d\n", x);
9     }
10 }
```

実行例.

```
$ gcc test005.c -o test005 
$ ./test005  ←実行      ↑コンパイル
7 > 5      ←結果
```

このプログラムの変数 x の初期値を 3 に変更し、コンパイルして実行すると、

5 >= 3

と出力される。

■ 複数の条件による多段階の判定

if 文の else 句には更に if 文を書くことができ、

```
if ( 条件判定式 1 ) {
    処理 1
} else if ( 条件判定式 2 ) {
    処理 2
    :
} else {
    上記のどの条件も満たさない場合の処理
}
```

のように、複数の条件による多段階の判定処理ができる。条件判定は上から順に行われ、最初に真となった箇所の処理だけが実行される。

4.1.1.1 条件判定式

2つの値を比較する際の演算子（比較演算子）を表 21 に挙げる。

表 21: 比較演算子

演算子	解 説	演算子	解 説
<code>x == y</code>	x と y が等しい	<code>x != y</code>	x と y が等しくない
<code>x < y</code>	x は y より小さい	<code>x > y</code>	x は y より大きい
<code>x <= y</code>	x は y 以下	<code>x >= y</code>	x は y 以上

比較演算子は基本的に、x、y の等値関係、あるいは順序関係に基づいて判定する。ポインタの値は「==」、「!=」によって比較できる。一方、順序関係の比較は、原則として同一配列内の要素を指すポインタ同士など、順序関係が定められている場合に限られる。それ以外のポインタ同士の順序比較は、未定義動作となる場合がある。

複素数の等値関係は判定できるが、複素数の値には単純な順序関係はないので、大小の判定はできないことに注意すること。

条件判定式の連言、選言、否定を表 22 に挙げる。

表 22: 条件判定式の論理的な扱い

記述	解説	備 考
<code>!x</code>	x の否定	真→偽、偽→真
<code>x && y</code>	x かつ y	$p1 \ \&\& \ p2 \ \&\& \ p3 \rightarrow (p1 \ \&\& \ p2) \ \&\& \ p3$ (左結合)
<code>x y</code>	x または y	$p1 \ \ p2 \ \ p3 \rightarrow (p1 \ \ p2) \ \ p3$ (左結合)

表 22 の論理的な記述を複数組み合わせた場合の優先順位は

`! > && > ||`

であり、! がまず先に評価され、次に &&、その次に || が評価される。

■ 条件判定式がもたらす値

条件判定式の判定結果は真もしくは偽である。C 言語では、0 を偽、0 以外を真とする。ただし、表 21、表 22 に挙げた比較演算子と論理演算子の評価結果は、偽の場合は 0、真の場合は 1 となる。

条件判定式の評価結果の値を確認するサンプルを test005-1.c に示す。

プログラム：test005-1.c

```
1 #include <stdio.h>
2
3 int main() {
4     int p1, p2, p3;
5
6     p1 = 2==2; // 「2 と 2 は等しい」の値
7     p2 = 2==3; // 「2 と 3 は等しい」の値
8     p3 = !(2==3); // 「『2 と 3 は等しい』の否定」の値
9     printf("p1:2==2 = %d, ", p1);
10    printf("p2:2==3 = %d, ", p2);
11    printf("p3:!(2==3) = %d\n", p3);
12    // 連言と選言の評価値
13    printf("p1 && p2 = %d, ", p1 && p2);
14    printf("p1 || p2 = %d\n", p1 || p2);
15 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test005-1.c のコンパイルと実行

```
$ gcc test005-1.c -o test005-1  ←コンパイル
$ ./test005-1  ←実行
p1:2==2 = 1, p2:2==3 = 0, p3:!(2==3) = 1 ←比較演算の結果の値, 否定の値
p1 && p2 = 0, p1 || p2 = 1 ←連言, 選言の結果の値
```

真が 1, 偽が 0 として得られている。

応用) 整数が条件判定式として使用できるので, 0 の判定を簡単な形で実現することができる。(次の例)

プログラム: test005-3.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 3;
5     if ( x ) printf("%d is not 0.\n",x);
6     if ( !x ) printf("%d is 0\n",x);
7 }
```

実行例.

```
$ gcc test005-3.c -o test005-3 [Enter]
$ ./test005-3 [Enter] ←実行 ↑コンパイル
3 is not 0. ←このような結果となる
```

このプログラムの 4 行目の x の値を 0 に変更し, コンパイルして実行すると,

0 is 0.

と出力される。

▲注意▲

表 21 に挙げた比較演算子を, 数学における連鎖比較のように記述してはならない。このことを次のプログラム test005-2.c で考える。

プログラム: test005-2.c

```
1 #include <stdio.h>
2
3 int main() {
4     int p;
5     p = 9>8>7;
6     printf("9>8>7 : %d\n",p);
7 }
```

実行例.

```
$ gcc test005-2.c -o test005-2 [Enter]
$ ./test005-2 [Enter] ←実行 ↑コンパイル
9>8>7 : 0 ←このような結果となる
```

このプログラムはコンパイル時, 実行時ともにエラーにはならないが, 上のような実行結果となる。一見すると, 「9>8>7」は正しいものに見えるが, 期待に反した結果となっている。

関係演算子は左結合であるため, 「9>8>7」は「(9>8)>7」と解釈される。まず左側の「9>8」の評価結果が真を表す値 1 となり, 結果的に「1>7」が評価されるため, 最終的な結果は偽を表す値 0 となる。

課題. test005-2.c のプログラムを改造し, 9>8>7 を正しく判定するプログラム test005-2-2.c を作れ。

■ 条件判定式として扱うことができるもの

if 文などの条件判定式には, **スカラ型**の値を使用することができる。スカラ型とは, 整数型, 浮動小数点型, 複素数型などの**算術型**と, **ポインタ型**³⁵の総称である。従って, **構造体**や**共用体**³⁶の値を, 条件判定式に直接使用することはできない。配列型もスカラ型ではないが, 多くの式の中では, 配列はその先頭要素を指すポインタに自動的に変換される。従って, 配列名を条件判定式として記述することはできる。ただし, 実在する配列の先頭を指すポインタはヌルポインタではないため, このような判定は「真」となる。

算術型の値は, その型におけるゼロの値が, 条件判定式における「偽」となり, それ以外の値は「真」となる。例えば, 浮動小数点数では, 0.0 と -0.0 が「偽」となり, それ以外の値は「真」となる。また, ポインタ型では**ヌルポインタ**が「偽」となり, それ以外の値は「真」となる。

複素数型では, 実部と虚部が共にゼロである場合に「偽」となり, 少なくとも一方がゼロでなければ「真」となる。符号付きゼロを持つ一般的な浮動小数点環境では, 実部と虚部の正負の符号の組み合わせによって 4 通りの複素ゼロが表現されるが, これらはいずれも条件判定では「偽」となる。

4.1.1.2 _Bool 型と bool

C99 以降の C 言語には, 真偽値を表すための基本型として **_Bool** 型が用意されている。**_Bool** 型の変数が保持する値は, 偽を表す 0 と, 真を表す 1 のいずれかである。例えば次のように変数を宣言できる。

³⁵後の「6 メモリとポインタ」(p.57)で解説する。

³⁶それぞれ, 後の「5 構造体」(p.51), 「9.4 共用体」(p.121)で解説する。


```
_Bool found = 0;
```

_Bool 型の変数に 0 を代入すると、その値は 0 になる。0 以外の値を代入すると、その値は 1 に変換される。

```
_Bool a = 0;      /* a の値は 0 */
_Bool b = 10;     /* b の値は 1 */
_Bool c = -5;     /* c の値は 1 */
```

ただし、C 言語では、真偽値を表すために int 型を使用することも一般的であり、実用上は int 型で十分な場合が多い。_Bool 型を使用する主な目的は、int 型では実現できない特別な処理を行うことではなく、その変数が個数や番号などの数値ではなく、真または偽の二つの状態を表すことを明確に示すことである。たとえば、次の二つの変数を考える。

```
int count = 0;
_Bool found = 0;
```

count は個数を保持する変数であり、found は「何かが見つかったかどうか」を表す変数であることが、型からも読み取りやすくなる。このように、型を用いて変数の役割や値の意味を示すと、プログラムを読む人に対して意図が伝わりやすくなり、**可読性**が高まる。

stdbool.h を読み込むと、_Bool の代わりに bool と記述できる。また、真と偽を表すために、true と false も使用できる。

例. bool, true, false の使用

```
bool found = false;
if (/* 条件 */) {
    found = true;
}
```

C17 では、bool, true, false は stdbool.h で定義されるマクロであり、bool は _Bool を表す。したがって、_Bool 型と bool 型という二つの異なる型が存在するわけではない。一般のプログラムでは、_Bool と直接記述するよりも、bool, true, false を用いる方が、真偽値を扱っていることが分かりやすい。

真偽値を表す型は、処理状態を保持する変数、真偽を判定する関数の戻り値、構造体に格納する設定値などに利用できる。ただし、本書の C 言語編では、真偽値を表すために必ずしも _Bool 型や bool を使用することは求めない。C 言語では、int 型による真偽値の表現も広く用いられており、プログラムの目的や読みやすさに応じて使い分ければよい。

_Bool は int の別名ではなく、独立した整数型である。ただし、算術演算など、整数拡張が行われる式の中では、_Bool 型の値は int 型に変換される。このため、式の中では int 型に近い振る舞いをすることがあるが、両者は同じ型ではない。

参考) C++ では、真偽値を表す bool が言語の基本型として用意されており、変数がどのような意味の値を保持するかを、型や表記によって明確にすることがより重視される。C 言語で _Bool 型や bool を用いる考え方は、C++ におけるこのような型の使い方を理解するための手がかりにもなる。

4.1.1.3 浮動小数点数の判定

浮動小数点数も先に解説した比較演算子を使えるが、**等値の判定**には注意が必要である。これに関して、次のプログラム test005-4.c を例に挙げて考える。

プログラム：test005-4.c

```
1  #include    <stdio.h>
2  #include    <math.h>
3
4  int main() {
5      double x = 0.1 + 0.2;
6      double y = 0.3;
7      const double d = 1e-12;      // 近似判定の許容値
8      // 厳密な比較
9      if ( x == y ) {
10         printf("%.17f and %.17f are same.\n",x,y);
11     } else {
```

```

12     printf("%.17f and %.17f are not same.\n",x,y);
13 }
14 // 近似比較
15 if ( fabs(x-y) < d ) {
16     printf("%.17f and %.17f are near. (delta:%e)\n",x,y,d);
17 } else {
18     printf("%.17f and %.17f are not near. (delta:%e)\n",x,y,d);
19 }
20 }

```

このプログラムの最初の if 文は $0.1+0.2$ の値が 0.3 に等しいかどうかを判定しているが、浮動小数点数の扱いは誤差を伴うため、意図した動きに反することがある。実際にこのプログラムをコンパイルして実行すると次のようになる。

例. test005-4.c のコンパイルと実行

```

$ gcc test005-4.c -o test005-4  ←コンパイル
$ ./test005-4  ←実行
0.30000000000000004 and 0.29999999999999999 are not same.      ←等しくないという判定
0.30000000000000004 and 0.29999999999999999 are near. (delta:1.000000e-12) ←しかし近い

```

このプログラムでは、 0.1 、 0.2 、 0.3 が浮動小数点数として近似的に表現されるため、 x と y は厳密には同じ値にならず、「 $=$ 」による比較では偽となる。これに対して 2 つ目の if 文では、 x 、 y の値の差の絶対値が、ある定めた許容範囲 d よりも小さいかどうかを判定している。このプログラムでは、 10^{-12} を許容範囲としており、 x 、 y の差の絶対値がこれ未満の場合は「近い」と判定している。このような比較方法を**近似比較**と呼ぶことがある。近似比較の際の許容範囲は**許容誤差**と呼ぶことがある。

注意) 適切な許容誤差は、扱う値の大きさや計算の目的によって異なる。非常に大きな値や非常に小さな値を比較する場合は、値の大きさも考慮した判定が必要となることがある。

■ 非数に関する判定

非数 NAN は通常の数値のように比較することができず、「 $\text{NAN}==\text{NAN}$ 」の判定も真にならない。このため、値が非数であるかどうかを調べるには関数 `isnan` を用いる。(この関数のプロトタイプは `math.h` に定義されている)

注意) 実際の計算処理では、非数が発生するとプログラムの動作が意図しないものになることがある。そのため、プログラム中では必要に応じて `isnan` 関数を用いて非数を検出することが望ましい。

■ 無限大に関する判定

無限大 INFINITY は「 $\text{INFINITY}==\text{INFINITY}$ 」による判定が真となる。また、値が無限大であるかどうかを調べるには関数 `isinf` を用いる。(この関数のプロトタイプは `math.h` に定義されている)

`isnan` 関数と `isinf` 関数は、判定結果が真の場合に 0 以外、偽の場合に 0 を返す。次に示すプログラムは、非数と無限大に関する判定結果を示すものである。

プログラム：test005-5.c

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      printf("---[NAN]-----\n");
6      printf("NAN==NAN : %d\n",NAN==NAN);
7      printf("isnan(NAN) : %d\n",isnan(NAN));
8      printf("isnan(1.2) : %d\n",isnan(1.2));
9      printf("isnan(INFINITY) : %d\n",isnan(INFINITY));
10     printf("---[INFINITY]---\n");
11     printf("INFINITY==INFINITY : %d\n",INFINITY==INFINITY);
12     printf("isinf(INFINITY) : %d\n",isinf(INFINITY));
13     printf("isinf(1.2) : %d\n",isinf(1.2));
14     printf("isinf(NAN) : %d\n",isinf(NAN));
15 }

```

例. test005-5.c のコンパイルと実行

```
$ gcc test005-5.c -o test005-5 Enter ←コンパイル
$ ./test005-5 Enter ←実行
---[NAN]-----
NAN==NAN : 0          ←これは真にはならない
isnan(NAN) : 1        ←非数が判定出来ている
isnan(1.2) : 0        ←有限の値は非数ではない
isnan(INFINITY) : 0   ←無限大は非数ではない
---[INFINITY]---
INFINITY==INFINITY : 1 ←これは真となる
isinf(INFINITY) : 1    ←無限大の判定ができています
isinf(1.2) : 0        ←有限の値は無限大ではない
isinf(NAN) : 0        ←非数は無限大ではない
```

この実行例では、isnan 関数と isinf 関数が「真」の値として 1 を返しているが、これは保証されたものではないことに注意すること。

4.1.1.4 複素数の等値性の判定

複素数の値の等値性も ==, != を用いて判定することができるが、先に解説した浮動小数点数同士の近似判定と同様の問題がある。複素数においても、許容誤差に基づく近似比較の考え方が重要である。

2つの複素数 z_1, z_2 が近いかどうかは、それらの差 $z_1 - z_2$ の絶対値（大きさ）を調べることで判定できる。許容誤差を d とすると、

$$|z_1 - z_2| < d$$

を満たす場合に、2つの複素数を近似的に等しいとみなす。C 言語では、複素数の絶対値を求める cabs 関数を用いて、次のように記述できる。

```
if ( cabs(z1-z2) < d ) {
    ...
}
```

cabs 関数のプロトタイプはヘッダファイル complex.h に宣言されている。許容誤差 d は、扱う値の大きさや計算の目的に応じて定める。

4.1.2 条件演算子による条件式

条件判定の結果によって、式の値を選択するための条件演算子を使える。

書き方： 条件判定式 ? 値1 : 値2

「条件判定式」が真の場合は「値1」が、偽の場合は「値2」が、条件式全体の値となる。「?」と「:」を組み合わせた演算子を条件演算子という。また、この演算子を用いた式を条件式という。

条件式は式であるため、他の式の中に組み込むことができる。このとき、「値1」と「値2」の両方が評価されるのではなく、条件判定の結果に応じて選択された一方だけが評価される。

条件式を用いたプログラム test005-6.c を次に示す。

プログラム：test005-6.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int x = 3;
5      printf("%dは%s.\n", x, x>5 ? "5より大きい" : "5より大きくない" );
6  }
```

このプログラムでは、printf 関数の引数に条件式を組み込んでいる。このプログラムをコンパイルして実行する例を次に示す。

例. test005-6.c のコンパイルと実行

```
$ gcc test005-6.c -o test005-6 Enter ←コンパイル
$ ./test005-6 Enter ←実行
3は5より大きくない。 ←結果
```

例えば、変数 x の値に 7 を設定してコンパイル、実行すると、

7 は 5 より大きい。

と出力される。

4.1.3 switch 文

switch 文は、式の値に基づいて、処理を開始する位置を選択する。if 文が条件判定式の真偽によって処理を選択するのに対し、switch 文では、式の値と各 case ラベルの値が一致するかどうかによって開始位置を選択する。

《switch 文》

書き方： `switch ( 式 ) {`
 `case 値 1:` `// 「式」が「値 1」の場合にここから実行する`
 `処理 1`
 `case 値 2:` `// 「式」が「値 2」の場合にここから実行する`
 `処理 2`
 `⋮`
 `default:` `// 「式」が上のどの値でもない場合にここから実行する`
 `処理 x`
 `}`

「式」を評価した値と一致する case ラベルがあれば、その位置から処理を開始する。case ラベルに指定する「値」は**整数定数式**（後述）でなければならない。一致する case ラベルがない場合は default ラベルの位置から処理を開始する。default ラベルもない場合は、switch 文の内部を実行しない。処理は、break 文などによって中断されない限り、後続の case ラベルを越えて switch 文の末尾まで続く。この動作を**フォールスルー**（fallthrough）という。なお、default ラベルは不要な場合には省略できる。

switch 文を用いたプログラム test006-1.c を次に示す。これは、変数 x の値によって、switch 文の中の開始位置を選択する例である。

プログラム：test006-1.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int x = 2;
5
6      switch ( x ) {
7          case 1:
8              printf("step-1,");
9          case 2:
10             printf("step-2,");
11          case 3:
12             printf("step-3,");
13          default:
14             printf("last step\n");
15      }
16 }
```

実行例。

```
$ gcc test006-1.c -o test006-1 
$ ./test006-1  ←実行      ↑コンパイル
step-2,step-3,last step ←このような結果となる
```

変数 x の値を 3 にすると次のような出力となる。

```
step-3,last step
```

x に他の値を設定して試されたい。

switch 文は、文の中の開始位置を選択するものであるが、break 文を用いることで、次の case ラベルの位置に到達する前に処理を終了することができる。このことを、次のプログラム で示す。

プログラム：test006-2.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 2;
5
6     switch ( x ) {
7         case 1:
8             printf("step-1\n");
9             break;
10        case 2:
11            printf("step-2\n");
12            break;
13        case 3:
14            printf("step-3\n");
15            break;
16        default:
17            printf("last step\n");
18    }
19 }
```

実行例.

```
$ gcc test006-2.c -o test006-2 [Enter]
$ ./test006-2 [Enter] ←実行      ↑コンパイル
step-2 ←このような結果となる
```

変数 x の値を 3 にすると次のような出力となる.

step-3

x に他の値を設定して試されたい.

このように break 文を用いる書き方は、switch 文の中の特定の部分のみを実行するという形の処理を実現する.

■ break 文

break 文は、構造を持つ文の実行を中断するためのものであり、後に解説する反復構造の中断にも利用できる.

■ 整数定数式

整数定数式とは、プログラムの実行前に値が確定する（コンパイル時に決定される）整数値の式である.（整数リテラルもこれに含まれる）

例. 10, 'A', 1+2, sizeof(int)

重要) 変数の値など、実行時に決まる値は case ラベルには記述できない.

4.2 反復

同じ処理を複数回繰り返して実行する仕組みを**反復**という。反復を利用すると、同じ文を何度も記述することなく、条件が成立している間や、指定した回数だけ処理を繰り返すことができる。C 言語には、反復を行うための構文として、while 文、do 文、for 文が用意されている。

4.2.1 while 文

while 文を用いると、文やブロックの実行を反復することができる。

《while 文》

書き方： while (条件判定式) {
 処理
 }

「条件判定式」が真である (0 でない) 間「処理」を反復する。反復する処理が 1 つの文だけである場合は、ブロックを表す { ... } を省略できる。「条件判定式」の評価は「処理」の実行前に行われる。従って、最初の判定で偽となった場合、「処理」は一度も実行されない。

while 文による処理の反復の例を次の test007-1.c に示す。

プログラム：test007-1.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int n = 3;
5      while ( n > 0 ) {
6          printf("%d, ",n);
7          n--;
8      }
9      printf("%d\n",n);
10 }
```

実行例.

```
$ gcc test007-1.c -o test007-1 
$ ./test007-1  ←実行      ↑コンパイル
3, 2, 1, 0 ←このような結果となる
```

末尾の 0 の出力は、while 文終了後の printf 関数による。

while 文は break 文で中断することができる。break 文を応用して test007-1.c と同様の処理を test007-2.c のようにして実現することもできる。

プログラム：test007-2.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int n = 3;
5      while ( 1 ) {
6          printf("%d, ",n);
7          if ( --n <= 0 ) break;
8      }
9      printf("%d\n",n);
10 }
```

実行例.

```
$ gcc test007-2.c -o test007-2 
$ ./test007-2  ←実行      ↑コンパイル
3, 2, 1, 0 ←このような結果となる
```

末尾の 0 の出力は、while 文終了後の printf 関数による。

このプログラムにあるように、「while (1) { ... }」として**無限ループ**を構成し、反復処理の内部で終了条件を判定して break 文で終了する形式も、しばしば用いられる。

■ 反復処理のスキップ

while 文の中で continue 文を実行すると、その反復における continue 文より後の処理を実行せず、次の反復のための条件判定へ移る。これを次のプログラム test007-3.c で示す。

プログラム：test007-3.c

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 0;
5     while ( n < 21 ) {
6         n++;
7         if ( n % 3 ) continue;
8         printf("%d, ",n);
9     }
10    printf("\n");
11 }
```

実行例.

```
$ gcc test007-3.c -o test007-3 
$ ./test007-3  ←実行      ↑コンパイル
3, 6, 9, 12, 15, 18, 21, ←このような結果となる
```

n が 3 の倍数でない場合は、continue 文によって、それより後にある printf 関数を実行せずに次の条件判定へ移る。その結果、3 の倍数だけが出力される。

4.2.1.1 do 文

while 文では、反復対象の処理を実行する前に反復条件を評価する。これに対して do 文では、反復対象部分を一度実行した後で、次回実行するか否かを判定する。

《do 文》

書き方： do {
 処理
 } while (条件判定式);

「処理」を実行した後で「条件判定式」を評価し、これが真 (0 でない) の場合、再度「処理」の部分を実行する。また偽の場合は do 文を終了する。反復する処理が 1 つの文だけである場合は、ブロックを表す { ... } を省略できる。結果として、「処理」は少なくとも 1 度は実行される。処理の中で break 文、continue 文を使用できる。continue 文が実行されると、その反復における残りの処理を実行せず、文末の while にある条件判定式の評価へ移る。

注意) ブロックの部分は do 文の一部であり、while (...) までを含めて 1 つの文となる。ブロック { ... } で終了する形式ではないため、末尾にセミコロン「;」を書く必要がある。

do 文を用いたプログラム test007-4.c を次に示す。

プログラム：test007-4.c

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 0;
5     do {
6         printf("%d, ",n);
7         n++;
8     } while ( n < 10 );
9     printf("%d\n",n);
10 }
```

実行例.

```
$ gcc test007-4.c -o test007-4 
$ ./test007-4  ←実行      ↑コンパイル
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ←出力
```

末尾の 10 の出力は、do 文終了後の printf 関数による。

4.2.2 for 文

初期化、条件判定、更新をまとめて記述できる反復制御の構文に for 文がある。

《for 文》

書き方： for (初期化; 条件判定式; 更新) {
 処理
 }

for 文では最初に「初期化」を 1 度実行し、その後「条件判定式」を評価する。条件が真の間「処理」を実行し、各回の終了時に「更新」を実行した後、再び条件判定式を評価する。while 文と同様に break 文、continue 文が使えるが、continue 文が実行されると、その反復における残りの処理を実行せず、「更新」を実行した後で、次の「条件判定式」の評価へ移る。break で処理を中断した場合は、更新処理は実行されない。

「初期化」の部分には式だけでなく、変数の宣言を記述することもできる。最初の「条件判定式」の評価で偽となった場合は、「処理」は一度も実行されない。

for 文を用いたプログラム test007-5.c を次に示す。

プログラム：test007-5.c

```
1 #include <stdio.h>
2
3 int main() {
4     int c;
5     for ( c = 0; c < 4; c++ ) {
6         printf("%d, ", c);
7     }
8     printf("%d\n", c);
9 }
```

実行例.

```
$ gcc test007-5.c -o test007-5 
$ ./test007-5  ←実行      ↑コンパイル
0, 1, 2, 3, 4 ←出力
```

末尾の 4 の出力は、for 文終了後の printf 関数による。

このプログラムでは for 文の中の「c = 0」で、反復処理に先立って初期化処理を実行する。そして「c < 4」の条件を満たす間ブロック内のプログラムを実行する。反復実行の各回の終了時に「c++」という更新処理を行い、変数 c の値を 1 増やしている。

for 文の初期化と更新の部分には、**コンマ演算子**（後述）を用いて複数の式を記述することができる。次のプログラム test007-6.c はそれを応用した例である。

プログラム：test007-6.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x, y;
5     for ( x=0,y=4; x < 4; x++,y-- ) {
6         printf("(%d,%d), ", x,y);
7     }
8     printf("(%d,%d)\n", x,y);
9 }
```

実行例.

コンパイル↓

```
$ gcc test007-6.c -o test007-6 
$ ./test007-6  ←実行
(0,4), (1,3), (2,2), (3,1), (4,0) ←出力
```

末尾の (4,0) の出力は、for 文終了後の printf 関数による。

このプログラムでは、for 文の初期化部分で「x=0,y=4」と 2 つの代入式を実行し、更新部分で「x++,y--」と 2 つの式（インクリメントとデクリメント）を実行している。

4.2.2.1 コンマ演算子

コンマ演算子を使用すると、複数の式をコンマで連結して 1 つの式にすることができる。この場合、個々の式は左側から順番に実行（評価）され、最後の式の評価値が最終的な値となる。

プログラム：test007-7.c

```
1 #include <stdio.h>
2
3 int main() {
4     int x = 1, y;
5     // コンマ演算子
6     y = (x*=2,x*=3,x*=4,x*=5);
7     printf("y = %d\n", y);
8 }
```

実行例.

```
$ gcc test007-7.c -o test007-7 
$ ./test007-7  ←実行      ↑コンパイル
y = 120 ←出力
```

コンマ演算子を代入式の右辺全体として用いる場合は、この例のように丸括弧で囲む。

参考) for 文による無限ループの構成

for 文の条件判定式を省略すると無限ループが構成できる。

注意) 関数の引数や変数の宣言を区切るコンマは、ここでのコンマ演算子とは異なる。

4.3 無条件ジャンプ

プログラム中にはラベルを設置することができ、goto 文によって、指定したラベルの位置にプログラムの実行を移すことができる。ラベルとは、プログラム中の文に付ける名前であり、goto 文を用いると、指定したラベルの付いた文へ処理を無条件に移すことができる。

ラベルを設置する際には、ラベル名の後ろにコロン「:」を置く。goto 文で移動先を指定する際には、コロンは記述しない。

プログラム：test007-8.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int      n = 3;
5
6      lb1: // ラベル1
7      if ( n > 0 ) {
8          printf("%d, ",n);
9      } else {
10         goto lb2;
11     }
12     n--;
13     goto lb1;
14
15     lb2: // ラベル2
16     printf("%d\n",n);
17 }
```

実行例.

```
$ gcc test007-8.c -o test007-8 
$ ./test007-8  ←実行      ↑コンパイル
3, 2, 1, 0      ←出力
```

末尾の 0 は、ラベル lb2 の後の printf による出力である。

このプログラムは、ラベルと goto 文によって反復構造を実現した例である。

▲重要▲

goto 文による無条件ジャンプは、同一の関数定義内でのみ可能である。ある関数から別の関数の内部に goto 文で移動することはできない。しかし、プログラム test007-8.c からわかるように、同一関数内であれば、ブロック { ... } の範囲を超えて goto 文で移動することができる場合がある。ただしその場合は、変数の宣言や初期化を飛び越える移動には注意が必要である。また、可変長配列などを含むブロックへ外側から移動することはできない。

▲注意▲

goto 文によるプログラムの流れの制御は、可読性を低下させるだけでなく、不必要にプログラムを複雑化することにもつながるので一般には使用を避けることが望ましい。実際に goto 文を全く使用せずに、while, do, for などの反復構文や、if, switch などの条件分岐構文を用いてプログラムを記述することは可能³⁷ である。

³⁷ 処理を順次実行する構造、条件分岐構造、反復構造などを組み合わせて、プログラムの流れを明確に構成する考え方を構造化プログラミングという。goto 文への依存を避けることも、その重要な方針の 1 つである。

5 構造体

構造体は、様々な型の複数の値をまとめて格納するためのものである。

《構造体の定義》

書き方： `struct 構造体名 {`
 `型1 メンバ1;`
 `型2 メンバ2;`
 `⋮`
 `};`

指定した「型」の「メンバ」（構成要素）をまとめた構造体型を定義する。この定義の後には、「`struct 構造体名`」を型として用いて、構造体変数（記憶上の実体）を作成することができる。

《構造体変数の宣言》

書き方： `struct 構造体名 変数名;`

先に定義された構造体型の変数を「変数名」で宣言する。これによって、その構造体型の値を格納する記憶上の実体が作られる。

作成された構造体の各メンバには、変数名とメンバ名をドット「`.`」でつないでアクセスする。

書き方： `変数名.メンバ名`

構造体のメンバに値を与える方法について、次のサンプル `test008-1.c` を例にして解説する。このプログラムでは、3つのメンバ `age`（年齢）、`weight`（体重）、`height`（身長）を持つ構造体型 `struct Human` が定義されており、この構造体型の変数 `p1`, `p2`, `p3`, `p4` がそれぞれ宣言されている。

プログラム：`test008-1.c`

```
1  #include    <stdio.h>
2
3  struct Human {           // 構造体の定義
4      int     age;
5      double  weight, height;
6  };
7
8  int main() {
9      struct Human p1, p2 = {24, 59.4, 164.7};    // p2は初期化済み
10     struct Human p3, p4;
11
12     // ドット表記で構造体に値を設定
13     p1.age = 21;
14     p1.weight = 65.3;
15     p1.height = 172.5;
16     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
17         "p1", p1.age, p1.weight, p1.height );
18
19     // p2も同様に参照できる
20     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
21         "p2", p2.age, p2.weight, p2.height );
22
23     // 複合リテラルを用いて宣言後に一括設定
24     p3 = (struct Human){23, 80.8, 185.2};
25     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
26         "p3", p3.age, p3.weight, p3.height );
27
28     // 指示子付きの複合リテラル（メンバを任意の順序で指定可能）
29     p4 = (struct Human){.height=170.0, .age=20, .weight=60.0};
30     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
31         "p4", p4.age, p4.weight, p4.height );
32 }
```

構造体変数 `p1` はドット表記で各メンバの値が設定されている。また、`p2` のように宣言の段階で初期値を与えることもできる。

注意) test008-1.c では、p2 の初期化に { ... } を記述しているが、これはブロックではなく、構造体の初期化子であり、複数のメンバをまとめて初期化する。

p3 のように、変数として宣言した後で、値を一括して設定する場合は、キャストと同様の表記で型を明示³⁸ する。このように、型名と初期化子を組み合わせで表した、名前を持たない構造体の値を複合リテラルという。複合リテラルは式であるため、この例のように代入式の右边などで使用できる。(注：構造体変数宣言時の初期化子は複合リテラルではない)

p4 のように、複合リテラルを与える際に「.age」などと、メンバ名の指定子を使うことができる。これにより、どのメンバに値を設定するのが明確になり、メンバを指定する順序も自由になる。

プログラム test008-1.c をコンパイルして実行する例を次に示す。

例. test008-1.c のコンパイルと実行

```
$ gcc test008-1.c -o test008-1  ←コンパイル
$ ./test008-1  ←実行
p1: age=21, weight=65.3, height=172.5
p2: age=24, weight=59.4, height=164.7
p3: age=23, weight=80.8, height=185.2
p4: age=20, weight=60.0, height=170.0
```

構造体変数への値の設定と参照ができていることがわかる。

5.1 構造体の様々な定義方法

構造体の定義と同時に、その変数を宣言することも可能である。

書き方： struct 構造体名 { メンバの定義 } 変数 1, 変数 2, … 変数 n;

この書き方では「構造体名」を省略できる。ただし、構造体名を省略すると、後からその構造体型を構造体名によって指定し、別の変数を宣言することはできない。

次のプログラム test008-2.c はこの書き方を例示するものである。

プログラム：test008-2.c

```
1 #include <stdio.h>
2
3 struct Human {
4     int age;
5     double weight, height;
6 } p1 = {24, 59.4, 164.7}; // 構造体定義と同時に変数宣言が可能
7
8 int main() {
9     printf("s: age=%d, weight=%.1f, height=%.1f\n",
10         "p1", p1.age, p1.weight, p1.height );
11 }
```

このプログラムでは、構造体型 struct Human を定義しており、それと同時にその変数 p1 を宣言している。このプログラムをコンパイルして実行する例を次に示す。

例. test008-2.c のコンパイルと実行

```
$ gcc test008-2.c -o test008-2  ←コンパイル
$ ./test008-2  ←実行
p1: age=24, weight=59.4, height=164.7 ←結果
```

test008-2.c では、変数 p1 はグローバル変数である。先の「3.4.2 グローバル変数」(p.17) でも述べたように、グローバル変数の使用は必要な場合に限定することが望ましい。

³⁸厳密には、この場合の型指定はキャストではなく、複合リテラルの構文の一部である。

5.1.1 構造体型の別名の定義

構造体型に `typedef` を用いて別名を与えると、その別名が「型」の名前として使用できる。これにより、変数宣言などで `struct` を付ける必要がなくなり、より簡潔に記述できる。実際のプログラムでも、このような形で構造体型を扱うことが多い。`typedef` を用いた宣言³⁹ によって構造体型に別名を与える方法を、次のプログラム `test008-3.c` で示す。

プログラム：test008-3.c

```
1 #include <stdio.h>
2
3 typedef struct {
4     int     age;
5     double  weight, height;
6 } Human;    // 構造体型に Human という別名を定義
7
8 int main() {
9     Human p1 = {24, 59.4, 164.7};    // struct を付けずに宣言できる
10    printf("s: age=%d, weight=%.1f, height=%.1f\n",
11           "p1", p1.age, p1.weight, p1.height );
12 }
```

このプログラムでは、匿名の構造体型に `Human` という別名（型名）が与えられている。そのため、変数 `p1` は「`Human p1`」という簡潔な形で宣言できる。これをコンパイルして実行する例を次に示す。

例. test008-3.c のコンパイルと実行

```
$ gcc test008-3.c -o test008-3  ←コンパイル
$ ./test008-3  ←実行
p1: age=24, weight=59.4, height=164.7 ←結果
```

参考) C++ では、名前を付けて構造体を宣言すると、`typedef` を使わなくても構造体名をそのまま型名として使用できる。

5.1.2 構造体をメンバとして持つ構造体

構造体は、そのメンバとして構造体を持つことができる。次のプログラム `test008-4.c` はそれを例示するものである。

プログラム：test008-4.c

```
1 #include <stdio.h>
2
3 // 日付を表す構造体
4 typedef struct {
5     int     year;
6     int     month;
7     int     day;
8 } Date;
9 // 学生情報を表す構造体
10 typedef struct {
11     char    name[50];
12     int     id;
13     Date    birthday;    // 上で定義された構造体をメンバとして持つ
14 } Student;
15
16 int main() {
17     Student s1 = {"Suzuki Taro", 1001, {2007, 5, 2}};
18     printf("name: %s, id: %d\n", s1.name, s1.id);
19     printf("birthday: %d/%d/%d\n",
20           s1.birthday.year, s1.birthday.month, s1.birthday.day);
21 }
```

このプログラムでは、構造体型 `Student` が、別の構造体型 `Date` の値をメンバ `birthday` として持っている。初期化子の内側にある「`{2007, 5, 2}`」は、このメンバの初期値を表す。また、「`s1.birthday.year`」のようにドット演算子を続けて記述することで、内部の構造体のメンバにアクセスできる。

これをコンパイルして実行する例を次に示す。

³⁹ 「3.9 型の別名の定義」（p.38）で解説。

例. test008-4.c のコンパイルと実行

```
$ gcc test008-4.c -o test008-4  ←コンパイル
$ ./test008-4  ←実行
name: Suzuki Taro, id: 1001
birthday: 2007/5/2 ←結果
```

5.1.3 関数内で定義するローカルな構造体型

構造体型の定義を、関数内のブロックに限定することも可能である。次に示すプログラム test008-5.c では、関数 f のブロック内でのみ使用できる構造体型 struct Pos が定義されている。この構造体型の定義と同時に、変数 p が宣言され、初期値として {3, 5} が与えられている。

プログラム：test008-5.c

```
1 #include <stdio.h>
2
3 void f( void ) {
4     struct Pos {           // この関数内でのみ使用できる構造体
5         int x, y;
6     } p = {3, 5};
7     printf("p: (x=%d,y=%d)\n", p.x, p.y);
8 }
9
10 int main() {
11     f();
12 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test008-5.c のコンパイルと実行

```
$ gcc test008-5.c -o test008-5  ←コンパイル
$ ./test008-5  ←実行
p: (x=3,y=5) ←関数 f による出力
```

5.2 構造体の代入と複製

構造体の内容は、代入演算子「=」を用いて、同じ構造体型の別の変数に代入することができる。このとき、構造体の全メンバの値が、代入先の構造体変数に複製（コピー）される。

重要) 配列は代入によるコピーができないが、構造体は代入によって内容を複製できるという違いがある。

注意) 構造体が大きい場合には、このようなコピー処理の負荷が大きくなることがある。

代入による構造体のコピーの動作を次のプログラム test008-6.c で確認する。

プログラム：test008-6.c

```
1 #include <stdio.h>
2
3 typedef struct {
4     int age;
5     double weight, height;
6 } Human; // 構造体型に Human という別名を定義
7
8 int main() {
9     Human p1 = {24, 59.4, 164.7}, p2;
10    p2 = p1; // 代入による複製
11    // 内容確認
12    printf("s: age=%d, weight=%.1f, height=%.1f\n",
13        "p1", p1.age, p1.weight, p1.height );
14    printf("s: age=%d, weight=%.1f, height=%.1f\n",
15        "p2", p2.age, p2.weight, p2.height );
16    // p1の内容を変更
17    p1.age = 50;
18    p1.weight = 66.1;
19    p1.height = 161.2;
20    // 再度確認
```

```

21     printf("p1 is changed.\n");
22     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
23           "p1", p1.age, p1.weight, p1.height );
24     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
25           "p2", p2.age, p2.weight, p2.height );
26 }

```

このプログラムでは、構造体変数 p1 の内容を p2 に代入する処理を行う。当然のことではあるが、代入直後は、p1 と p2 の各メンバが同じ値になっている。しかし、p1, p2 は別の実体であり、代入後に p1 の内容を変更しても p2 の内容はその影響を受けない。

このプログラムをコンパイルして実行する例を次に示す。p1, p2 の値の推移がわかる。

例. test008-6.c のコンパイルと実行

```

$ gcc test008-6.c -o test008-6  ←コンパイル
$ ./test008-6  ←実行
p1: age=24, weight=59.4, height=164.7      ←この内容が代入によって
p2: age=24, weight=59.4, height=164.7      ←ここに複製されている
p1 is changed.
p1: age=50, weight=66.1, height=161.2      ← p1 が変更されても
p2: age=24, weight=59.4, height=164.7      ← p2 は別物なので影響なし

```

まとめ) 代入演算子による構造体の代入は、各メンバに対する代入をまとめて行う操作である。

5.2.1 構造体のメンバとしての配列の扱い

配列は、その内容全てをまとめて代入によって複製することができない。例えば、

```

int a[5] = {1,2,3,4,5};
int b[5];
b = a;      // これはエラー

```

のように、配列同士を代入することは許されていない。このため、配列の内容を複製するには、要素ごとに代入するなどの方法を用いる必要がある。

一方、構造体は1つの値として扱われるため、代入によって全メンバの値を複製することができる。さらに、構造体は配列をメンバとして持つことができるため、配列を含むデータ全体を、1つの構造体の値として扱うことが可能となる。

例えば、次のような構造体を考える。

```

typedef struct {
    int x[5];
} Data;

```

このとき、構造体変数同士の代入は次のように書くことができる。

```

Data a = {{1,2,3,4,5}};
Data b;
b = a;      // 構造体全体のコピー

```

この代入により、メンバとして含まれる配列 x の各要素も含めて、構造体の全メンバの値が複製される。すなわち、配列を直接代入することはできないが、構造体のメンバとして扱うことで、構造体の代入によって配列の内容を一括して複製することができる。

このように、構造体の代入は、配列を含むデータを簡潔にコピーできる有用な方法である。(ただし、含まれる配列が大きい場合には、コピー処理の負荷も大きくなることもある) 特に、複数の配列や関連するデータをまとめて扱う場合には、構造体として定義しておくことで、データの複製や受け渡しを容易に記述できる。

5.2.2 関数呼び出しに伴う構造体の複製

構造体は配列とは異なり、それ自体が1つの値として扱われるため、関数の引数として値渡しすることができる。この場合、関数の仮引数として別の実体が作られ、構造体の全メンバの値がそこに複製される。従って、構造体を値として受け取った関数内でその内容を変更しても、呼び出し元の構造体には影響しない。このことを次に示すプログラ

↳ test008-7.c で確認する.

このプログラムでは、構造体型 DS が定義されており、main 関数内でその型の変数 d が宣言され、初期化されている。変数 d を関数 f に引数として渡すと、その全メンバの値が仮引数 d に複製される。

そして、f から復帰した後で、main 側で d の内容を確認（関数 printDS）している。

プログラム：test008-7.c

```
1  #include    <stdio.h>
2  #include    <math.h>
3  #define     DSIZE      8
4
5  typedef struct {          // 構造体の定義
6      int      x[DSIZE];
7      double   y[DSIZE];
8  } DS;
9
10 void printDS( DS d ) {    // 構造体の内容表示
11     int      c;
12     for ( c = 0; c < DSIZE; c++ ) printf("%d\t",d.x[c]);
13     printf("\n");
14     for ( c = 0; c < DSIZE; c++ ) printf("%.4f\t",d.y[c]);
15     printf("\n\n");
16 }
17
18 void f( DS d ) {          // 構造体の内容の書き換え
19     int      c;
20     for ( c = 0; c < DSIZE; c++ ) {
21         d.x[c] = c;        d.y[c] = sqrt( (double)c );
22     }
23     printDS(d);
24 }
25
26 int main() {
27     DS      d;
28     int      c;
29
30     // 構造体の初期化
31     for ( c = 0; c < DSIZE; c++ ) {
32         d.x[c] = 0;        d.y[c] = 0.0;
33     }
34
35     f(d);                  // この呼び出しの後で
36     printDS(d);           // 構造体はどうなるか
37 }
```

関数 f の仮引数 d は、呼び出し元の構造体そのものではなく、その複製である。そのため、関数 f の内部で d.x や d.y の各要素を書き換えても、main 関数側の構造体 d には影響しない。

このプログラムをコンパイルして実行する例を次に示す。

例. test008-7.c のコンパイルと実行

```
$ gcc test008-7.c -o test008-7 -lm  ←コンパイル
$ ./test008-7  ←実行
```

0	1	2	3	4	5	6	7	
0.0000	1.0000	1.4142	1.7321	2.0000	2.2361	2.4495	2.6458	←関数 f 内での構造体の内容
0	0	0	0	0	0	0	0	←復帰後の main での構造体の内容
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	(初期化時から変わっていない)

関数 f から復帰した後は、main 側の d は変化していないことがわかる。なお、表示用の関数 printDS も構造体を値として受け取っているため、printDS の呼び出し時にも構造体の複製が作られる。大きな構造体を扱う場合には、このような複製が処理効率に影響することがある。

6 メモリとポインタ

C 言語の変数は、具体的には計算機環境の記憶装置（メモリ）に割り当てられている。メモリはバイト単位で値を保持する領域が1次元的に並んだものとして扱われる。本書で扱う一般的な計算機環境では、1バイトは8ビットである。例えば、

```
double x;
```

として変数 x を宣言すると、プログラムの実行時に x の値を格納する領域がメモリ上に割り当てられる。(図2)

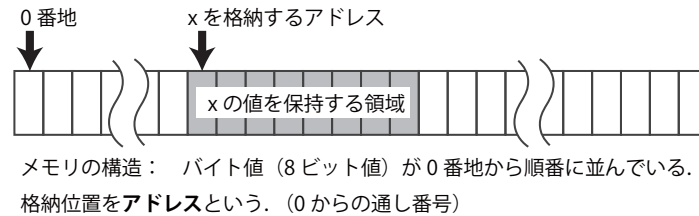


図 2: double 型の変数 x がメモリ上の連続した領域に配置される様子

図2は、double 型のオブジェクトの大きさが64ビット（8バイト）である環境でのメモリ上のイメージであり、 x の値を格納するための連続した8バイトの領域が確保されている様子を示している。

変数が格納されているメモリ上のアドレスを取得するための**アドレス演算子「&」**がある。

書き方: &変数

「変数」が格納されている場所を指すポインタ値が得られる。

変数に代入されている値とそのアドレスを確認するプログラム test009-1.c を次に示す。

プログラム: test009-1.c

```
1 #include <stdio.h>
2 #include <stdint.h> // uintptr_t, uintmax_t
3
4 int main() {
5     double x = 3.141592653589793;
6
7     printf("x = %.15f\n", x);
8     printf("address of x = %p\n", (void *)&x); // %p で編集・出力
9     printf("address of x = %ju (dec)\n",      // 10進数表現で出力
10            (uintmax_t)(uintptr_t)&x);
11 }
```

このプログラムでは、double 型の変数 x が宣言されており、そこに 3.141592653589793 という値が代入されている。C 言語では、変数の値だけでなく、その変数が割り当てられているメモリ上のアドレスを知ることができる。

■ アドレスの型

プログラム test009-1.c では、printf 関数の変換指定子「%p」を用いて変数 x のアドレスを出力している。アドレスを表すポインタ値は、変数 x に格納されている値そのものとは異なる型の値である。「%p」は void * 型の値を対象とするため、& x を (void *) に変換して渡している。これによって、特定のオブジェクト型を指定しない汎用ポインタとして出力できる。

また、アドレスを整数として表示する例では、ポインタ値を uintptr_t 型に変換し、さらに変換指定子「%ju」に対応する uintmax_t 型に変換している⁴⁰。

次のようにして、test009-1.c をコンパイルして実行することで、変数 x の値とそのアドレスの両方を確認することができる。

⁴⁰uintptr_t は、ポインタ値を整数として表現するために用意される符号なし整数型である。また、uintmax_t は、処理系で使用できる符号なし整数型のすべての値を表現できる型である。

例. test009-1.c のコンパイルと実行

```
$ gcc test009-1.c -o test009-1  ←コンパイル
$ ./test009-1  ←実行
x = 3.141592653589793
address of x = 0000002a32fffc88 ←変数 x のアドレス
address of x = 181244263608 (dec) ←10 進数の形の x のアドレス
```

宣言した変数がメモリ上のどこに割り当てられるかは、実行環境によって決まり、実行ごとに変わる場合がある。また、「%p」によるアドレスの具体的な表示形式は処理系によって異なる。

例. 再度実行（先の例の続き）

```
$ ./test009-1  ←再度実行
x = 3.141592653589793
address of x = 00000050425ffa88 ←再実行時には別のアドレスとなる場合がある
address of x = 344710969992 (dec) ←その 10 進数表現
```

このように、プログラムを再度実行すると、変数 x が異なるアドレスに配置される場合がある。ただし、実行するたびに必ず異なるアドレスになるとは限らない。

6.1 ポインタ

C 言語では、オブジェクトのアドレスを値として保持する変数である**ポインタ**を使用できる。ポインタ変数を宣言するには、変数名を含む宣言子に**アスタリスク「*」**を付ける。

書き方： 型 *変数名

この宣言により、「型」のオブジェクトを指すポインタ変数を宣言する。ただし、宣言だけでは、ポインタの値は不定であり、有効なアドレスは与えられていない。有効なアドレスを代入する前に、ポインタの値を利用したり、「*変数名」によって指す先を参照したりしてはならない。

アドレス演算子「&」で得た、対応する型のオブジェクトのアドレスを代入すると、その変数を指すようになる。また、式の中で用いる「*」は**間接演算子**と呼ばれ、「*変数名」と記述すると、ポインタが指している先のオブジェクトを参照できる。

重要) p はアドレスを保持するポインタであり、*p は p が指している先のオブジェクトを表す。

ポインタを宣言する際には、

型 *変数

のほかに、

型* 変数

のように記述することもできる。ただし、複数の変数を同時に宣言する場合には注意が必要である。

型* 変数 1, 変数 2, …

と宣言すると、「変数 1」だけがポインタとなり、「変数 2」以降は通常の変数となる。

例えば、「double *p1, f1, f2;」と宣言すると、p1 だけがポインタとなり、f1, f2 は通常の double 型変数となる。複数のポインタを宣言する場合は、「double *p1, *p2;」のように記述する。

重要) ポインタを宣言する際のアスタリスク「*」は、各変数名を含む宣言子に個別に記述する必要がある。

次に示すプログラム test009-2.c は、ポインタがアドレスを保持し、間接演算子「*」を用いることで、ポインタが指す先のオブジェクトを参照したり、その値を変更したりできることを示す例である。

プログラム：test009-2.c

```
1 #include <stdio.h>
2
3 int main() {
4     double x = 3.141592653589793;
5     double y = 2.718281828459045;
6     double *p; // double型のオブジェクトを指すポインタ
7
8     p = &x; // xのアドレスを代入
9     printf("[p = &x]\n");
```

```

10     printf(" p = %p\n", (void *)p); // アドレスを出力
11     printf("*p = %.15f\n", *p);      // 格納されている値を出力
12     *p = 12.34;                      // xに値を代入するのと同じ処理
13     printf("[*p = 12.34]\n");
14     printf(" x = %.2f\n", x);         // xの値を確認
15
16     p = &y;                          // yのアドレスを代入
17     printf("[p = &y]\n");
18     printf(" p = %p\n", (void *)p); // アドレスを出力
19     printf("*p = %.15f\n", *p);      // 格納されている値を出力
20     *p = 45.67;                      // yに値を代入するのと同じ処理
21     printf("[*p = 45.67]\n");
22     printf(" y = %.2f\n", y);        // yの値を確認
23 }

```

このプログラムでは、double 型の変数 x, y が宣言され、値が設定されている。これとは別に、double 型のオブジェクトを指すポインタ変数 p が宣言されている。p は double 型の値そのものを保持するのではなく、double 型のオブジェクトが格納されている位置を指すポインタ値を保持する。

まず x のアドレスを p に代入することで、p は x を指すようになる。この状態で「*p」と記述すると x を参照でき、「*p = 12.34;」と記述すると x に値を代入できる。次に y のアドレスを p に代入すると、p は y を指すようになり、「*p」は y を表すようになる。

このプログラムをコンパイルして実行する例を次に示す。

例. test009-2.c のコンパイルと実行

```

$ gcc test009-2.c -o test009-2  ←コンパイル
$ ./test009-2  ←実行
[p = &x]
p = 0000008db29ff9b0          ← p には x のアドレスが設定されているので
*p = 3.141592653589793        ← *p は x と同じ扱いができる
[*p = 12.34]                  ← *p を介して x に値を代入することもできる
x = 12.34                     ← x を参照すると、そのことが確認できる
[p = &y]
p = 0000008db29ff9a8          ← p には y のアドレスが設定されているので
*p = 2.718281828459045        ← *p は y と同じ扱いができる
[*p = 45.67]                  ← *p を介して y に値を代入することもできる
y = 45.67                     ← y を参照すると、そのことが確認できる

```

この例では、ポインタ p が他の変数のアドレスを持つことで、その変数の値を読み出したり、書き換えたりできることを示している。特に重要なのは、変数 p 自体が保持しているのはアドレスであり、p が指す先に格納された値とは別のものであるという点である。

test009-2.c は、ポインタが他の変数の代理となることが可能であることを示したが、他にも有用な応用が可能である。以下にいくつかの応用事例を示す。

6.2 ポインタを介した値の受け渡し

ポインタはメモリ上のアドレス値を保持する変数であり、メモリ上のデータの「在り処」を直接扱うために利用できる。このことを応用すると、データの受け渡しの様々な局面で利便性をもたらす。

関数呼び出しの際に、引数に構造体を渡すことについて例を挙げて考える。まずは、構造体を値として渡す例として test009-3.c を示す。

プログラム：test009-3.c (値渡しの例：呼び出し元には変更が反映されない)

```

1  #include <stdio.h>
2
3  typedef struct {
4      int     age;
5      double  weight, height;
6  } Human;    // 構造体型に Human という別名を定義
7
8  // 構造体の複製を受け取る関数 f
9  void f( Human p ) {    // 構造体の全メンバの値が仮引数 p に複製される

```



```

10 // 引数に受け取った直後の構造体を出力
11 printf("[inside f]\n");
12 printf("  p.age=%d, p.weight=%.1f, p.height=%.1f\n",
13        p.age, p.weight, p.height);
14 // 構造体の内容を変更
15 p.age = 48; p.weight = 65.2; p.height = 162.1;
16 // f内で変更した構造体を出力
17 printf("  --- p is changed ---\n");
18 printf("  p.age=%d, p.weight=%.1f, p.height=%.1f\n",
19        p.age, p.weight, p.height);
20 }
21
22 int main() {
23     Human p1 = {24, 59.4, 164.7};
24     // fを呼び出す前に構造体を出力
25     printf("[inside main]\n");
26     printf("  p1.age=%d, p1.weight=%.1f, p1.height=%.1f\n",
27           p1.age, p1.weight, p1.height );
28     f( p1 ); // 構造体自体を渡す
29     // fを呼び出した後で構造体を出力
30     printf("[inside main]\n");
31     printf("  p1.age=%d, p1.weight=%.1f, p1.height=%.1f\n",
32           p1.age, p1.weight, p1.height );
33 }

```

このプログラムでは、main 内で作成された構造体型 Human の変数 p1 を、関数 f に値として渡している。関数 f の仮引数 p には、p1 の全メンバの値が複製される⁴¹。関数 f では、この複製を出力した後、内容を変更して再度出力している。仮引数 p は p1 とは別の実体であるため、f 内で変更しても、main 側の p1 には影響しない。

test009-3.c をコンパイルして実行する例を次に示す。

例. test009-3.c のコンパイルと実行

```

$ gcc test009-3.c -o test009-3  ←コンパイル
$ ./test009-3  ←実行
[inside main]
  p1.age=24, p1.weight=59.4, p1.height=164.7    ← main 内で作成された構造体
[inside f]
  p.age=24, p.weight=59.4, p.height=164.7      ← fが受け取った構造体（複製）
  --- p is changed ---
  p.age=48, p.weight=65.2, p.height=162.1      ←それをf内で変更
[inside main]
  p1.age=24, p1.weight=59.4, p1.height=164.7    ← main 側の構造体には影響なし

```

この結果から、main で作成した構造体 p1 と f が受け取った構造体 p は別のものであることがわかる。すなわち、f を呼び出したときに p1 の複製 p が作成されて f がそれを使用したことがわかる。

このことから、構造体を値渡しする場合には、次の性質があることがわかる。

- 1) 関数呼び出しの際に、構造体の全メンバの値が仮引数へ複製される。
- 2) 関数内で仮引数の構造体を変更しても、呼び出し元の構造体には反映されない。

サイズの大きな構造体では、1) の複製が処理時間や記憶領域の使用量に影響することがある。また、呼び出し元の構造体を関数内で変更したい場合には、2) の値渡しでは目的を実現できない。しかし、関数呼び出し時に構造体のアドレスを渡すことで、構造体全体の複製を避けながら、呼び出し元の構造体そのものを関数内で操作できる。

関数呼び出しの際に構造体のアドレスを渡す形にしたプログラム test009-4.c を次に示す。

プログラム：test009-4.c

```

1 #include <stdio.h>
2
3 typedef struct {
4     int     age;
5     double  weight, height;
6 } Human;    // 構造体型に Human という別名を定義
7

```

⁴¹この複製は、関数の引数として値が渡されることによって作られる。

```

8 // 構造体のアドレスを受け取る関数 f
9 void f( Human *p ) { // 構造体のアドレスをポインタ p に受け取る
10 // 受け取った直後の構造体を出力
11 printf("[inside f]\n");
12 printf(" p->age=%d, p->weight=%.1f, p->height=%.1f\n",
13        p->age, p->weight, p->height);
14 // 構造体の内容を変更
15 p->age = 48; p->weight = 65.2; p->height = 162.1;
16 // f 内で変更した構造体を出力
17 printf(" --- p1 is changed through p ---\n");
18 printf(" p->age=%d, p->weight=%.1f, p->height=%.1f\n",
19        p->age, p->weight, p->height);
20 }
21
22 int main() {
23     Human p1 = {24, 59.4, 164.7};
24     // f を呼び出す前に構造体を出力
25     printf("[inside main]\n");
26     printf(" p1.age=%d, p1.weight=%.1f, p1.height=%.1f\n",
27            p1.age, p1.weight, p1.height );
28     f( &p1 ); // 構造体のアドレスを渡す
29     // f を呼び出した後で構造体を出力
30     printf("[inside main]\n");
31     printf(" p1.age=%d, p1.weight=%.1f, p1.height=%.1f\n",
32            p1.age, p1.weight, p1.height );
33 }

```

このプログラムでは、関数 `f` が構造体型 `Human` のオブジェクトを指すポインタを受け取る形となっている。main から構造体変数 `p1` のアドレスを渡すと、仮引数 `p` にはそのアドレス値が複製される⁴²。構造体の全メンバの値は複製されず、関数 `f` と `main` は同じ構造体変数 `p1` を参照することになる。

■ 構造体のアドレスからメンバにアクセスする方法

構造体変数のメンバにはドット演算子「`.`」を用いてアクセスする。一方、構造体を指すポインタを介してメンバにアクセスする場合は、アロー演算子「`->`」を用いる。

書き方： 構造体を指すポインタ→メンバ

「`p->age`」は「`(*p).age`」と同じ意味である。間接演算子「`*`」よりドット演算子「`.`」の方が優先順位が高いため、後者では丸括弧が必要である。

`test009-4.c` をコンパイルして実行する例を次に示す。

例. `test009-4.c` のコンパイルと実行

```

$ gcc test009-4.c -o test009-4  ←コンパイル
$ ./test009-4  ←実行
[inside main]
  p1.age=24, p1.weight=59.4, p1.height=164.7    ← main 内で作成された構造体
[inside f]
  p->age=24, p->weight=59.4, p->height=164.7    ← f が受け取ったアドレスが指し示す構造体 (main で作成したもの)
  --- p1 is changed through p ---
  p->age=48, p->weight=65.2, p->height=162.1    ←それを f 内で変更
[inside main]
  p1.age=48, p1.weight=65.2, p1.height=162.1    ← main 側の構造体に反映されている

```

まとめ) ポインタを関数へ渡すことで、構造体全体を複製することなく、呼び出し元と呼び出された関数が同じ構造体を参照し、その内容を読み書きできる。

6.2.1 関数間での変数の間接的な共有

通常、関数内で宣言されたローカル変数は、別の関数からその変数名を用いて直接操作することはできない。また、関数へ引数として値を渡すと、その値は仮引数に複製されるため、仮引数を変更しても、呼び出し元の変数には反映されない。これについては、先の「2.5 関数への引数の受け渡しについて」(p.8) で解説した。

⁴²アドレス値の複製は行われるが、これに要するコストは、構造体の内容全体の複製にかかるコストと比べて、一般的に小さい。

一方、変数のアドレスをポインタとして関数へ渡すことで、呼び出された関数から、呼び出し元の変数を間接的に操作できる。このことを次のプログラム test009-9.c で確認する。

このプログラムでは、main 内で宣言された変数 x のアドレスを、関数 f の仮引数 p に渡している。仮引数 p にはアドレス値が複製され、p は x を指すようになる。そのため、f では「*p」を通じて x の値を読み出したり、変更したりできる。

プログラム：test009-9.c

```
1 #include <stdio.h>
2
3 void f( int *p ) {
4     printf("[f] : p=%p\n", (void*)p);
5     printf("[f] : *p=%d\n", *p);
6     *p = 456;
7     printf("[f] : *p is set to 456.\n");
8     printf("[f] : *p=%d\n", *p);
9 }
10
11 int main() {
12     int x;
13
14     x = 123;
15     printf("[main]: x=%d\n", x);
16     printf("[main]: &x=%p\n", (void*)&x);
17     printf("[main]: execution of f(&x)\n");
18     f( &x );
19     printf("[main]: x=%d\n", x);
20 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test009-9.c のコンパイルと実行

```
$ gcc test009-9.c -o test009-9  ←コンパイル
$ ./test009-9  ←実行
[main]: x=123                      ←変数 x の値
[main]: &x=0000007d1c9ff99c        ←変数 x のアドレス
[main]: execution of f(&x)
[f] : p=0000007d1c9ff99c          ←受け取ったアドレス（上と同じ）
[f] : *p=123                      ←受け取ったアドレス位置にある int の値（main 側 x と同じ）
[f] : *p is set to 456.
[f] : *p=456                      ←関数 f 側で値を変更
[main]: x=456                    ←main の変数 x の値に変更が反映されている
```

このように、変数のアドレスをポインタとして関数へ渡すことで、呼び出された関数は、ポインタを介して呼び出し元の変数の値を読み出したり、変更したりできる。

6.3 配列のポインタとしての扱い

配列は、同じ型の複数の要素を、メモリ上の連続した領域に並べて保持するためのもの⁴³である。例えば、

```
int a[6];
```

として配列 a を宣言すると、int 型の要素が 6 個、図 3 のようにメモリ上の連続した領域に配置される。

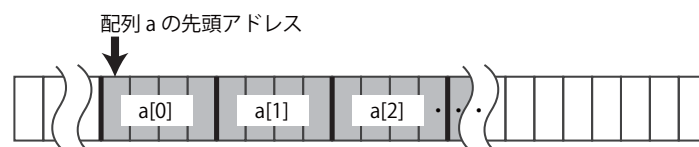


図 3: int 型の配列 a の各要素がメモリ上の連続した領域に配置される様子

この図は、int 型の大きさが 32 ビット（4 バイト）である環境を前提としたものであり、各要素に対応する 4 バイトの領域が連続して並んでいる様子を示している。

⁴³ 「3.6 配列」(p.31) で解説。

6.3.1 配列名とポインタ

配列名は、式の中では通常、配列の先頭要素を指すポインタに変換される⁴⁴。例えば「`int a[6];`」と宣言した場合、この配列名 `a` は、多くの式の中では、先頭要素 `a[0]` を指す `int *` 型のポインタに変換される。例えば、

```
int a[6], *p;
p = a;
```

とすると、配列名 `a` は先頭要素 `a[0]` のアドレスに変換され、その値がポインタ `p` に代入される。従って、`p` は配列 `a` の先頭要素を指す。この状態では、「`p[0]`、`p[1]`、…」によって、「`a[0]`、`a[1]`、…」と同じ要素にアクセスできる。また、「`*p`」は「`a[0]`」を表す。

6.3.2 ポインタの加算

ポインタは、通常の整数とは異なった性質を持つ。例えば、

```
int a[6], *p;
p = a;
```

とした場合、「`p+1`、`p+2`、…」は、それぞれ「`a[1]`、`a[2]`、…」のアドレス、すなわち「`&a[1]`、`&a[2]`、…」と同じ値になる。一般に、「`p+n`」は、`p` が指す要素から `n` 個先の要素を指すポインタとなる。(図 4)

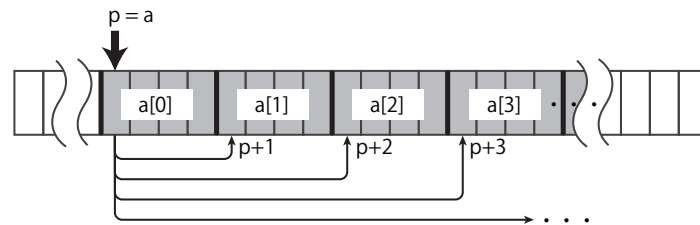


図 4: ポインタ `p` に対する加算 `p+n` が、`n` 個先の要素のアドレスを指す様子

実際のメモリ上では、ポインタが指す型の大きさに `n` を掛けたバイト数だけ先の位置となるが、プログラム上では型の大きさを明示的に掛ける必要はない。

注意) ポインタの加算は、同じ配列の要素を指す範囲で行う。配列の末尾の 1 つ先を指すポインタを作ることができるが、その位置にある値を間接演算子「`*`」で読み書きしてはならない。

■ サンプルプログラム

次に挙げるプログラム `test009-5.c` は `int` 型の配列 `a` とポインタ `p` の関係を示すものである。先に解説した内容を、処理結果として具体的に示す。

プログラム：test009-5.c

```
1  #include    <stdio.h>
2  #define    AR_SIZE    6          // 配列の要素数の定義
3
4  int main() {
5      int    a[AR_SIZE], c;
6      int    *p;
7
8      for (c=0; c<AR_SIZE; c++) a[c] = c*c;    // 配列の内容の設定（平方数）
9
10     // 配列内容の確認(1)
11     printf("Ex.1: ");
12     for (c=0; c<AR_SIZE; c++) printf("a[%d]=%d,      ", c, a[c] );
13     printf("\n");
14
15     // 配列内容の確認(2)：ポインタに添字を付ける例
16     p = a;    // 配列名 a は先頭要素のアドレスとして扱われる
17     printf("Ex.2: ");
18     for (c=0; c<AR_SIZE; c++) printf("p[%d]=%d,      ", c, p[c] );
19     printf("\n");
```

⁴⁴厳密には、配列名そのものはポインタではない。多くの式の中では先頭要素を指すポインタに変換されるが、`sizeof` 演算子やアドレス演算子「`&`」の対象とする場合などには、この変換は行われない。

```

20
21 // 配列内容の確認(3)：ポインタが指し示すメモリの値
22 printf("Ex.3: ");
23 for (c=0; c<AR_SIZE; c++) printf("(p+%d)=%d, ", c, *(p+c) );
24 printf("\n");
25
26 // 配列内容の確認(4)：配列の各要素のアドレスを使う例
27 printf("Ex.4: ");
28 for (c=0; c<AR_SIZE; c++) {
29     p = &a[c]; // 各要素のアドレス
30     printf("(a[%d])=%d, ", c, *p );
31 }
32 printf("\n");
33 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test009-5.c のコンパイルと実行

```

$ gcc test009-5.c -o test009-5  ←コンパイル
$ ./test009-5  ←実行
Ex.1: a[0]=0,      a[1]=1,      a[2]=4,      a[3]=9,      a[4]=16,      a[5]=25,
Ex.2: p[0]=0,      p[1]=1,      p[2]=4,      p[3]=9,      p[4]=16,      p[5]=25,
Ex.3: *(p+0)=0,    *(p+1)=1,    *(p+2)=4,    *(p+3)=9,    *(p+4)=16,    *(p+5)=25,
Ex.4: *(&a[0])=0,  *(&a[1])=1,  *(&a[2])=4,  *(&a[3])=9,  *(&a[4])=16,  *(&a[5])=25,

```

この結果から、「p[c]」と「*(p+c)」が同じ要素を表すことも確認できる。また、「*(&a[c])」は、a[c] のアドレスを取得した後、その位置の値を間接演算子で読み出す表記であり、結果は「a[c]」と同じになる。

6.3.3 配列とポインタの違い

多くの場合において、配列とポインタは取り扱いが似ているが、厳密には両者は異なる。例えば、

```

int a[100], *p;
p = a;

```

とした場合、sizeof 演算子によって得られる大きさが両者で異なる。「sizeof a」は配列 a 全体の大きさとなる⁴⁵ が、「sizeof p」はポインタ変数 p 自体の大きさとなる。また、「*p」は「a[0]」を表すため、「sizeof *p」は int 型の大きさとなる。

更に、上のケースでは、「p++」によって p が次の配列要素を指すように変更することはできるが、a は配列名であり、別のアドレスを保持するように変更できないため、「a++」とは記述できない。

6.3.3.1 関数定義の仮引数の書き方

配列の要素を関数内で扱うために、配列の先頭要素を指すポインタを引数として受け取ることができる。仮引数は、通常のポインタ形式で次のように記述できる。

例. int 型のオブジェクトを指すポインタを受け取る関数

```
void f( int *p ) { (定義内容) }
```

また、配列を受け取れることを意識しやすいように、仮引数を配列形式で記述することもできる。

例. 配列形式の仮引数

```
void f( int a[] ) { (定義内容) }
```

関数の仮引数に「int a[]」と記述した場合、仮引数 a の型は「int *」に調整される。従って、上の 2 つの関数定義における仮引数は同じ型である。また、「int a[100]」のように要素数を記述しても、100 要素の配列だけを受け取るという制約にはならない。

なお、配列形式で宣言した仮引数に対する「sizeof a」で得られるのは、配列全体の大きさではなく、ポインタの大きさである。ただし、実際のプログラミングでは、この記述は「配列全体の大きさを得ようとした間違いである可能性が高い」ため、コンパイラが警告を出すことがある。

配列とポインタの違いを示すサンプルプログラム test009-6.c を次に示す。

⁴⁵この例では、「sizeof a」は「100 * sizeof(int)」と同じ大きさになる。

プログラム：test009-6.c

```

1  #include    <stdio.h>
2  #define     AR_SIZE      100      // 配列の要素数の定義
3
4  void f1( int *p ) {
5      printf("[inside f1( int *p )]:  sizeof p  = %zu\n", sizeof p);
6      printf("[inside f1( int *p )]:  sizeof *p = %zu\n", sizeof *p);
7  }
8
9  // 次の行は f2( int *a ) と同じ
10 void f2( int a[] ) {      // 関数の仮引数にa[]を書くと、aはポインタと見なされる
11     printf("[inside f2( int a[] )]: sizeof a  = %zu\n", sizeof a);
12     printf("[inside f2( int a[] )]: sizeof *a = %zu\n", sizeof *a);
13 }
14
15 int main() {
16     int      a[AR_SIZE], *p, c;
17
18     for (c=0; c<AR_SIZE; c++) a[c] = c*c;    // 配列の内容の設定（平方数）
19     p = a;                                   // pは配列aの先頭要素を指す
20
21     printf("sizeof a      = %zu\n", sizeof a );           // 配列全体のサイズ
22     printf("sizeof a[0] = %zu\n", sizeof a[0] );          // 配列の先頭要素のサイズ
23     printf("sizeof *p    = %zu\n", sizeof *p );           // *pはa[0]と同じ
24     printf("sizeof p     = %zu\n", sizeof p );            // ポインタのサイズ
25
26     f1( a );
27     f2( a );
28 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test009-6.c のコンパイルと実行

```

$ gcc test009-6.c -o test009-6     ←コンパイル
(警告メッセージが出ることがある)

$ ./test009-6     ←実行
sizeof a      = 400                                ←配列 a 全体のサイズ
sizeof a[0] = 4                                   ←配列 a の先頭要素のサイズ
sizeof *p     = 4                                  ← (同上)
sizeof p      = 8                                  ←アドレス値のデータサイズ
[inside f1( int *p )]:  sizeof p  = 8               ←アドレス値のデータサイズ
[inside f1( int *p )]:  sizeof *p = 4               ←p が指し示す配列の先頭要素のサイズ
[inside f2( int a[] )]: sizeof a  = 8               ←f1 の場合と同じ
[inside f2( int a[] )]: sizeof *a = 4               ←f1 の場合と同じ

```

参考) 警告を出さない書き方

通常のプログラムでは、配列形式の仮引数に対して「sizeof a」を使用することは避ける。この例のようにポインタ自体の大きさを表示する場合は、仮引数を最初からポインタ形式で記述すると、意図が明確になり、この警告も出ない。

```

void f2( int *a ) {
    printf("sizeof a = %zu\n", sizeof a);
}

```

この場合も仮引数 a の型は int * であり、表示される値は、配列形式で記述した場合と同じである。

6.4 ヌルポインタと NULL

ポインタ変数を宣言する際、直ちに有効なオブジェクトのアドレスを設定できない場合は、NULL⁴⁶ で初期化しておくことが望ましい。NULL は、どのオブジェクトも指していないヌルポインタを表すために用いられる。

初期化されていない自動変数のポインタは不定な値を持つため、その値をポインタとして使用してはならない。例えば次のプログラム test009-7.c では、ポインタ p が初期化されていない状態で使用されている。このような処理の

⁴⁶NULL は stddef.h や stdio.h など、複数の標準ヘッダで定義されるマクロであり、ヌルポインタ定数を表す。処理系によって「0」や「(void*)0」などと定義される。NULL を使用するプログラムでは、NULL を定義する標準ヘッダを明示的にインクルードする。

動作は保証されず、不正なメモリアクセスによって異常終了したり、意味不明な結果が生じたりする可能性がある。

プログラム：test009-7.c

```
1 #include <stdio.h>
2 #define AR_SIZE 100
3
4 int main() {
5     int a[AR_SIZE], *p, c;
6     // p = a;
7     for (c=0; c<AR_SIZE; c++) {
8         p[c] = c*c;
9         printf("%d, ", a[c]);
10    }
11 }
```

例. コンパイルと実行

```
$ gcc test009-7.c -o test009-7 
$ ./test009-7  ←実行      ↑コンパイル
Segmentation fault      ./test009-7
```

(あるいは、意味不明な出力が得られる場合もある)

この例では、不正なメモリアクセスによってプログラムが異常終了している。ただし、未初期化ポインタを使用した場合の動作は保証されないため、常に同じメッセージが表示されるとは限らない。

このプログラムの6行目のコメント「//」を外すと正常に動作するので確認されたい。

これに対して、次のプログラム test009-8.c では、ポインタを宣言する際に NULL で初期化しており、更に、ポインタが有効な対象を指していない状態を検出する仕組みになっている。

プログラム：test009-8.c

```
1 #include <stdio.h>
2 #include <stddef.h>
3 #define AR_SIZE 100
4
5 int main() {
6     int a[AR_SIZE], *p = NULL, c;
7
8     // p = a;      // ポインタの設定
9
10    if ( p ) {      // ポインタが設定されている際の動作
11        for (c=0; c<AR_SIZE; c++) {
12            p[c] = c*c;
13            printf("%d, ", a[c]);
14        }
15    } else {        // ポインタ未設定時の動作
16        printf("Pointer p has not been set!\n");
17        printf("Nothing has been done...\n");
18    }
19 }
```

このプログラムでは、ポインタ p を NULL で初期化し、更に、ポインタを使用する前に「if (p)」で状態を検査している。条件式「if (p)」は「if (p != NULL)」と同じ意味であり、p がヌルポインタでない場合に真となる。そのため、p がどのオブジェクトも指していない状態では、p を介したメモリアクセスを行わない仕組みになっている。

このプログラムをコンパイルして実行する例を次に示す。

例. test009-8.c のコンパイルと実行

```
$ gcc test009-8.c -o test009-8       ←コンパイル
$ ./test009-8       ←実行
Pointer p has not been set!      ←ポインタ未設定時の処理
Nothing has been done...
```

test009-8.c の8行目のコメント「//」を外すと、ポインタ p が配列 a を指し示すので、その場合の動作を確認されたい。

重要) NULL は、ポインタがどのオブジェクトも指していないことを表すために用いられる。NULL で初期化しておくことで、ポインタが有効な対象を指しているかどうかを検査できる。ただし、ヌルポインタを間接参照してはならず、使用前に「p != NULL」などの条件によって確認する必要がある。

7 入出力

ここでは端末装置（ターミナル環境）やファイルに対する入出力の方法について解説する。

7.1 ストリーム：入出力の対象

ストリームとは、データの入出力を連続した流れとして扱うための抽象的な仕組みである。プログラムは、データの入出力をストレージデバイス（ファイルなど）や端末装置（ディスプレイ、キーボード）などの個別の装置に対してではなく、抽象化されたストリームに対して行う。

ストリームは抽象的な概念であり、C 言語では、ストリームに関する情報が **FILE 型オブジェクト**⁴⁷ によって管理される。プログラムからストリームを操作する際には、通常、そのオブジェクトを指す FILE * 型のポインタを用いる。

重要 FILE * はストリームそのものではなく、ストリームを操作するために用いられる FILE 型オブジェクトへのポインタである。

以下に、重要なストリームとして標準入出力、標準エラー出力、ファイル入出力について解説する。

7.2 標準出力、標準エラー出力

標準出力は出力用のストリームであり、通常は端末の画面に接続されている⁴⁸。標準出力を表す stdout は、ヘッダファイル stdio.h で定義されている。printf 関数は、この stdout に対して出力を行う。

出力用のストリームとしては標準エラー出力（stderr）も存在しており、これも通常は端末の画面に接続されている⁴⁹。stderr も stdio.h に定義されており、stdout と同様の扱いが可能である。

標準出力がプログラムの通常の出力に用いられるのに対し、標準エラー出力は、エラーメッセージや警告メッセージなどに用いられる。どちらも出力用のストリームであるが、用途に応じて使い分ける。

printf 以外にも標準出力に対する出力用の関数がある。

■ 1 バイトの出力：putchar

プロトタイプ： `int putchar( int c );`

引数 c の値を unsigned char に変換し、その 1 バイトを標準出力に出力する。正常に出力された場合は、出力したバイトの値を unsigned char から int に変換した値として返し、エラー時には EOF を返す。

■ 文字列の出力：puts

プロトタイプ： `int puts( const char *s );`

文字列 s を標準出力に出力する。文字列の終端のヌル文字 '\0' までを出力し（ヌル文字は出力しない）、出力後に改行文字 '\n' を自動的に付加する。正常に出力された場合は非負の値を返し、エラー時には EOF を返す。

■ サンプルプログラム

printf, putchar, puts それぞれで文字列を標準出力に出力するプログラム test010-1.c を次に示す。

プログラム：test010-1.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     size_t c, len;
6     char s1[] = "1) printfで出力";
7     char s2[] = "2) putcharで出力\n"; // 文字列に改行コードを含める
8     char s3[] = "3) putsで出力";
9
10    printf("%s\n", s1);
11
12    len = strlen(s2);
```

⁴⁷構造体として実装されていることが多い。

⁴⁸標準出力はリダイレクトにより、端末の画面以外の出力先（ファイルなど）に接続することも可能である。

⁴⁹後の「7.2.1 出力のリダイレクト」（p.68）で実例を示す。

```

13     for ( c=0; c<len; c++ ) putchar( (unsigned char)s2[c] );    // 1バイトずつ出力
14
15     puts(s3);           // 出力後に自動改行
16 }

```

このプログラムでは、putcharによって文字列を文字単位ではなく、1バイトずつ順番に出力している。日本語の文字は複数バイトで表現される場合があるが、すべてのバイトを元の順序で出力することで、文字列全体が表示される。

このプログラムをコンパイルして実行する例を次に示す。

例. test010-1.c のコンパイルと実行

```

$ gcc test010-1.c -o test010-1  ←コンパイル
$ ./test010-1  ←実行
1) printf で出力
2) putchar で出力
3) puts で出力

```

7.2.1 出力のリダイレクト

通常、標準出力と標準エラー出力は端末の画面に接続されている。シェルのリダイレクト機能を用いることで、これらの出力先をファイルなどへ切り替えることができる。

重要) リダイレクトはC言語の機能ではなく、プログラムを起動するシェルが提供する機能である。

プログラムからの出力を、リダイレクトでファイルに保存する処理を、次のプログラム test010-2.c を用いて確認する。

プログラム：test010-2.c

```

1  #include <stdio.h>
2
3  int main() {
4      printf("This message is for stdout.\n");
5      // 出力先ストリームを指定して書式出力する関数
6      fprintf(stderr,"This message is for stderr.\n");
7  }

```

このプログラムは、標準出力と標準エラー出力に文字列を出力するものである。プログラム中の fprintf 関数は、printf と同様の書式で出力を行うが、第1引数として出力先のストリームを指定する点が異なる。

このプログラムをコンパイルして実行する例を次に示す。

例. test010-2.c のコンパイルと実行

```

$ gcc test010-2.c -o test010-2  ←コンパイル
$ ./test010-2  ←実行
This message is for stdout.    ←標準出力
This message is for stderr.    ←標準エラー出力

```

この例から、デフォルトでは標準出力、標準エラー出力ともに端末の画面に接続されていることがわかる。次に、同じプログラムの出力先をファイルにリダイレクトする例を示す。プログラムを実行する際に、プログラム名に続いて

> 出力先ファイル

とすると、標準出力の内容が「出力先ファイル」にリダイレクトされて保存される。また、

2> 出力先ファイル

とすると、標準エラー出力の内容が「出力先ファイル」にリダイレクトされて保存される。このように、標準出力と標準エラー出力は独立したストリームであるため、それぞれ別々にリダイレクトすることができる。

例. 標準出力、標準エラー出力のリダイレクト（先の例の続き）

```

$ ./test010-2 > test010-2_stdout.txt 2> test010-2_stderr.txt  ←出力先をファイルにリダイレクト
$ ls -l test010-2*.txt  ←出力ファイルを確認
-rw-r--r-- 1 katsu admin 29  3月 22 18:18 test010-2_stderr.txt ←標準エラー出力の内容
-rw-r--r-- 1 katsu admin 29  3月 22 18:18 test010-2_stdout.txt  ←標準出力の内容

```

出力内容がディスプレイに表示されず、ファイル test010-2_stdout.txt, test010-2_stderr.txt が作成されている。次に、これらのファイルの内容をディスプレイに表示して確認する。

例. 出力されたファイルの内容の確認（先の例の続き）

```
$ cat test010-2_stderr.txt  ←ファイルの内容の確認
This message is for stderr.    ←標準エラー出力の内容
$ cat test010-2_stdout.txt  ←ファイルの内容の確認
This message is for stdout.    ←標準出力の内容
```

標準出力が test010-2_stdout.txt、標準エラー出力が test010-2_stderr.txt にリダイレクトされて保存されていることがわかる。

7.3 標準入力

標準入力は入力用のストリームであり、ヘッダファイル stdio.h に stdin として定義されている。これは、通常は端末のキーボードからの入力に接続されている⁵⁰。

■ 1バイトの入力：getchar

プロトタイプ： `int getchar( void );`

標準入力から1バイトを読み取る。読み取った値は `unsigned char` として解釈された後、`int` 型に変換されて⁵¹返される。入力が終端に達した場合や、読み取りエラーが発生した場合には EOF⁵² を返す。従って、入力の終了は、この関数が EOF を返すことで識別する。

`getchar` は1回の呼び出しで1バイトを読み取る関数であるが、端末のキーボードから入力する場合は通常、 キーが押されるまで入力内容がプログラムへ渡されない。これは `getchar` が行単位で読み取るためではなく、通常の端末が入力を行単位でプログラムへ渡すように動作するためである。すなわち、入力された文字は内部で一時的に蓄えられ、 キーが押された時点でプログラムに渡される。

一方、ファイルからリダイレクトされた標準入力では、キーボード操作を待つことなく、ファイルの内容が終端まで順次読み取られる。

■ キーボード入力を EOF として終了する方法

端末から標準入力の終了を通知するには、Windows のコマンドプロンプトなどでは + に続けて を入力する。macOS や Linux などの端末では、入力行が空の状態では + を入力する。

`getchar` を使用したサンプルプログラムを test010-3.c に示す。このプログラムは標準入力から1文字入力し、それをそのまま標準出力に送るものである。

プログラム：test010-3.c

```
1  #include    <stdio.h>
2
3  int main() {
4      int      c;                // 入力受け付け用の変数
5
6      while ( 1 ) {              // EOFが来るまで繰り返すループ
7          c = getchar();          // 標準入力から1文字受け取る
8          if ( c != EOF ) {       // EOFでなければ
9              putchar(c);         // それを標準出力に送り
10         } else {                // EOFならば
11             break;              // ループを終了
12         }
13     }
14 }
```

このプログラムをコンパイルして実行する例を次に示す。

⁵⁰標準入力はリダイレクトにより、キーボード以外の入力元（ファイルなど）に接続することも可能である。

⁵¹戻り値が `int` 型であるのは、すべての `unsigned char` の値に加えて、特殊な値 EOF も表せるようにするためである。

⁵²EOF は文字コードではなく、入力の終端や読み取りエラーを表すための特殊な値として stdio.h に定義されている。

例. test010-3.c のコンパイルと実行

```
$ gcc test010-3.c -o test010-3  ←コンパイル
$ ./test010-3  ←実行
a  ←キーボード入力
a ←それが標準出力に送られて表示される
b  ←キーボード入力
b ←それが標準出力に送られて表示される
^Z  ←キーボード入力 (CTRL+Z など：入力終了)
```

この例ではわかりにくいですが、 キーによる入力も改行文字「\n」として標準入力から読み取られ、その改行文字が標準出力へ送られることで、表示が改行されている。

7.3.1 入力のリダイレクト

通常、標準入力は端末のキーボードからの入力に接続されている。シェルのリダイレクト機能を用いることで、標準入力の入力元をファイルへ切り替えることができる。具体的には、プログラムを実行する際に、プログラム名に続いて

< 入力用ファイル

とすると、そのファイルの内容がプログラムから標準入力として読み込まれる。

このことを先のプログラム test010-3.c で確認する。同じディレクトリにある次のようなファイル test010-3.in.txt の内容を標準入力に与える例を示す。

ファイル：test010-3.in.txt

```
1 The first line.
2 The second line.
```

例. 標準入力をリダイレクトして実行（先の例の続き）

```
$ ./test010-3 < test010-3.in.txt  ←標準入力をファイルにリダイレクト
The first line.      ←ファイル test010-3.in.txt の内容が
The second line.    ←読み込まれて出力されている
```

ファイルの内容が標準入力として読み込まれ、そのまま標準出力へ送られていることがわかる。

7.3.2 入力用の様々な関数

標準入力からデータを読み込む方法には、1 バイトずつ読み込む方法のほかに、文字列としてまとめて読み込む方法や、書式を指定して数値などを読み込む方法がある。ここでは、入力に用いられる代表的な関数について解説する。

7.3.2.1 文字列の読み込み

標準入力から文字列を読み込む関数に `fgets` がある。

プロトタイプ： `char *fgets( char *s, int n, FILE *stream );`

`s` には読み込んだ文字列を格納するバッファ（`char` 配列）を指定する。`n` にはバッファのサイズ（バイト数）を指定する。実際に読み込まれるデータは最大で `n-1` バイトであり、残りの 1 バイトに終端文字 '\0' が格納される。`stream` には入力元のストリームを指定（標準入力から読み込む場合は、`stdin` を指定）する。

この関数は、データの読み込みに成功した場合はバッファ `s` の先頭アドレスを返す。エラーが発生した場合、または入力終端に達するまでに 1 バイトも読み込めなかった場合は `NULL` を返す。

`fgets` の 1 回の実行が終わる条件は、

- 1) `n-1` バイトを読み込んだ時
- 2) 入力に改行 '\n' が来た時
- 3) 入力終端 (EOF) に達した時

のどれかである。改行 '\n' が入力された場合は、その改行文字もバッファに含めて格納される。

`fgets` を用いたサンプルプログラム test010-4.c を次に示す。このプログラムでは、標準入力からのデータを、1 回につき最大で `BUFSIZE-1` バイトずつ `fgets` で読み込み、それを標準出力へ出力する処理を繰り返す。入力が終端に達

した場合、または読み込みエラーが発生した場合に処理を終了する。

プログラム：test010-4.c

```
1  #include    <stdio.h>
2  #define     BUFSIZE      8
3
4  int main() {
5      char     buf[BUFSIZE];    // 読み込み用バッファ
6
7      while ( 1 ) {    // 読み込みが全て終わるまで繰り返すループ
8          if ( fgets( buf, BUFSIZE, stdin ) ) {    // データの読み込みができた場合は
9              printf("%s",buf);                    // それを出力し
10             } else {                                // 入力終端に達した場合、または読み込みエラーの場合
11                 break;                            // ループを終了する
12             }
13     }
14 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test010-4.c のコンパイルと実行

```
$ gcc test010-4.c -o test010-4  ←コンパイル
$ ./test010-4  ←実行
abcdefghijklmn  ←キーボード入力
abcdefghijklmn ←それが出力される
opqrstuvwxyz  ←キーボード入力
opqrstuvwxyz ←それが出力される
^Z  ←キーボード入力 (  +  など：入力終了)
```

入力の終端まで読み込みと出力を繰り返している様子がわかる。また、fgets は改行文字も含めて読み込むため、printf("%s", buf) のようにそのまま出力すると、入力と同じ形で表示される。

この実行例ではわかりにくい、「abcdefghijklmn」に続けて改行を入力した場合、fgets による読み込みは 3 回に分かれる。1 回目には「abcdefg」と終端文字「\0」、2 回目には「hijklmn」と終端文字「\0」、3 回目には改行文字「\n」と終端文字「\0」が buf に格納される。各回で読み込まれた内容が直ちに出力されるため、画面上では元の入力行と同じ形に見える。

このプログラムの標準入力をリダイレクトして次のファイル poem01.txt から読み込む例を示す。

ファイル：poem01.txt

```
1  Measure twice.
2  Code once.
3  Debug forever.
```

例. 標準入力をリダイレクトして実行（先の例の続き）

```
$ ./test010-4 < poem01.txt  ←標準入力をファイルにリダイレクト
Measure twice.    ← BUFSIZE-1 毎に fgets で
Code once.        繰り返し読み込まれて
Debug forever.    出力される
```

重要) fgets は、第 3 引数で指定した入力ストリームから文字列を読み込む関数である。stdin を指定すれば標準入力から読み込み、fopen 関数（後述）などで開いたファイルのストリームを指定すれば、そのファイルから読み込むことができる。

■ 廃止された gets 関数

古い C 言語の教材や Web 上のサンプル、過去に作成されたプログラムなどでは、gets 関数を用いたコードを見かけることがある。しかし、gets には安全に使用方法がないため、C11 で標準ライブラリから削除された。従って、現在の C 言語のプログラムでは使用してはならない。

以前の規格におけるプロトタイプは、次のものであった。

旧プロトタイプ： char *gets(char *s);

s には、標準入力から読み込んだ文字列を格納するバッファ (char 配列) を指定する。入力された改行文字「\n」はバッファには格納されず、その位置には文字列の終端を表すヌル文字「\0」が格納される。

この関数の重大な問題は、読み込み先となるバッファの大きさを指定する引数がないことである。そのため、入力された文字列がバッファに収まるかどうかを、gets 側で確認することができない。バッファより長い文字列が入力されると、配列の範囲を超えてデータが書き込まれ、未定義動作となる。その結果、別のデータが破壊されたり、プログラムが異常終了したりすることがある。しかし、一見正常に動作したように見えることもある。

次に示すプログラム test010-5.c は、古い教材などで見られる gets の使用例を再現したものである。このプログラムは、gets の危険性を確認するための例であり、実用プログラムとして使用してはならない。

プログラム：test010-5.c

```
1  #include    <stdio.h>
2  #define     BUFSIZE      8
3
4  int main() {
5      char     buf[BUFSIZE];    // 読み込み用バッファ
6
7      while ( 1 ) {    // 読み込みが全て終わるまで繰り返すループ
8          if ( gets( buf ) ) {    // データの読み込みができた場合は
9              printf("%s\n",buf);    // それを出力し
10             } else {    // EOF に達し、かつ読み込むデータが無い場合
11                 break;    // ループを終了する
12             }
13         }
14     }
```

このプログラムをコンパイルして実行する例を次に示す。なお、C17 では gets は標準関数ではないため、処理系によってはコンパイル時やリンク時に警告またはエラーが出る場合がある。

例. test010-5.c のコンパイルと実行

```
$ gcc test010-5.c -o test010-5  ←コンパイル
(警告メッセージが出ることもある)
$ ./test010-5  ←実行
abcdefg  ←キーボード入力
abcdefg ←それが出力される
~Z  ←キーボード入力 (CTRL+Z など：入力終了)
```

この例では、プログラムは正常に実行されたように見える。しかし、BUFSIZE は 8 であるため、終端文字「\0」を含めて安全に格納できる入力は最大 7 バイトである。これを超える長さの文字列が入力されると、バッファの外側のメモリへ書き込みが行われる。

次に、バッファに収まらない長い文字列を入力した例を示す。

例. バッファサイズを超える入力によって未定義動作が生じる例 (先の例の続き)

```
$ ./test010-5  ←実行
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx  ←長い入力
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx ←それが出力されるが…
~Z  ←キーボード入力 (CTRL+Z など：入力終了)
Segmentation fault ./test010-5 ←エラー
```

この例では異常終了しているが、これは未定義動作の結果の一例にすぎない。同じプログラムでも、実行環境やコンパイル条件によっては、異常終了しない場合や、別のデータを破壊する場合もある。従って、「短い入力だけを与えれば安全」という使い方はしてはならない。

重要) 古い教材や既存のプログラムで gets を見かけても、新しいコードでは使用してはならない。標準入力から文字列を読み込む場合は、読み込み先のバッファサイズを指定できる fgets を使用する。

7.3.2.2 書式指定の読み込み：scanf

標準入力からのデータを、書式文字列に沿って解釈した形で値を読み取る scanf 関数がある。

プロトタイプ： `int scanf( const char *format, ... );`

第1引数 format には、入力データの形式を指定するための**書式文字列**を与える。書式文字列には、`%d` や `%f` などの**変換指定子**⁵³ を記述することで、入力された値をどのような型として解釈するかを指定する。

重要) scanf の書式文字列には、printf と似た形の変換指定子を用いるが、対応する型や意味が異なる場合がある。

scanf の第2引数以降には、書式文字列に対応する値を格納する変数の**アドレス**を指定する。これにより、入力された値を変数に書き込むことができる。

scanf の戻り値は、正常に読み取って変数へ格納できた項目数である。途中で入力が指定した形式に合わなくなった場合は、それまでに格納できた項目数を返す。例えば、最初の項目から形式に合わなかった場合は 0 を返す。最初の項目を格納する前に入力終端または読み込みエラーに達した場合は EOF を返す。

scanf を用いたプログラム test010-6.c を次に示す。

プログラム：test010-6.c

```
1 #include <stdio.h>
2
3 int main() {
4     int a, r;
5     double x;
6
7     r = scanf( "%d %lf", &a, &x);
8     printf("a=%d x=%f\n", a, x);
9     printf("return value of scanf: %d\n", r);
10 }
```

例.

コンパイル↓

```
$ gcc test010-6.c -o test010-6 
$ ./test010-6  ←実行
123 3.141592  ← scanf に対する入力
a=123 x=3.141592 ← printf による出力
return value of scanf: 2
2 個の値を読み取った ↑
```

これは、scanf で変数 a と x の値を読み込む例である。scanf がそれらの変数へ値を書き込めるように、アドレス演算子「&」を用いて、各変数のアドレスを引数として渡している。このプログラムでは、scanf の戻り値が 2 未満の場合のチェックはしていないが、実際のプログラミングにおいては意識する必要がある。

▲重要▲

scanf では、各変換指定子に対応する型を厳格に守らなければならない。例えば、printf では float 型の値も可変長引数へ渡される際に double 型へ変換されるため、float、double のどちらにも「%f」を用いる。一方、scanf では、float * に対して「%f」、double * に対して「%lf」を使い分ける必要がある。これは、scanf が渡されたアドレスを通じて、変数の型に応じた大きさと形式で値を書き込むためである。

⁵³ 「表 18：代表的な変換指定子」(p.35) を参照のこと。

7.4 ファイル入出力

ファイルは、データを外部記憶装置などに保存し、プログラムの終了後も保持するために用いられる。

C 言語では、ファイルも標準入力や標準出力と同様に、ストリームを通して扱う。データを読み取るストリームを**入力ストリーム**、データを書き込むストリームを**出力ストリーム**という。そのため、`fgets` や `fprintf` など、これまでに扱ったストリーム入出力関数をファイルにも利用できる。

ファイルを扱うには、まずファイルを開き、処理が終わった後に閉じる必要がある。

7.4.1 ファイルのオープンとクローズ

ファイルをストリームとして扱うためには、まずファイルを開く必要がある。このために用いる関数が `fopen` である。

プロトタイプ: `FILE *fopen( const char *filename, const char *mode );`

第1引数 `filename` には開くファイルの名前（パス）を指定する。第2引数 `mode` には、ファイルの開き方（モード）を指定する文字列を与える。

`mode` には次のようなものがある。

"r" : 読み込み用を開く（ファイルが存在しない場合は失敗）

"w" : 書き込み用を開く（存在しなければ作成し、存在する場合は内容を消去）

"a" : 追記用を開く（存在しなければ作成し、書き込みはファイル末尾に行く）

注意) "w" で既存のファイルを開くと、それまでの内容は失われる。

バイナリファイル（後述）を扱う場合には、"rb" や "wb" のように "b" を付ける。更に、読み書き両用にするために "+" を付けること⁵⁴ ができる。

この関数は、ファイルのオープンに成功すると、ストリームを操作するための `FILE *` 型のポインタを返し、失敗した場合には `NULL` を返す。従って、`fopen` を用いる際には、戻り値が `NULL` でないかを確認する必要がある。

ファイルの使用が終了したら、開いたストリームを閉じる必要がある。このために用いる関数が `fclose` である。

プロトタイプ: `int fclose( FILE *stream );`

引数 `stream` には、`fopen` によって取得したストリームを指定する。この関数は、ストリームに関連付けられたファイルを閉じるとともに、バッファに残っているデータを書き出すなどの後処理を行う。

`fclose` は、正常に終了した場合には 0 を返し、エラーが発生した場合には EOF を返す。特にファイルへ重要なデータを書き込んだ場合は、`fclose` の戻り値も確認することが望ましい。

ファイルを開いたままにしておくと、データが正しく書き込まれない場合や、使用できるシステム資源を消費し続ける原因となる。プログラムの終了時には開かれているストリームも処理系によって閉じられるが、ファイルを使い終えた時点で、明示的に `fclose` によって閉じることを習慣にすべきである。

7.4.2 テキストファイルとバイナリファイル

ファイルは本質的には**バイト列（バイナリデータ）**として保存されるものである。このため、全てのファイルは広い意味ではバイナリファイルであると考えることができる。

その中で、文字コードに基づいて文字列として解釈できるように構成されたファイルを**テキストファイル**という。テキストファイルは、テキストエディタなどで内容を直接閲覧することができ、改行などによって構造が表現される。

一方、数値や構造体のメモリ上の表現、画像、音声などを、文字列としてではなくバイト列として記録したファイルは、通常**バイナリファイル**と呼ばれる。これらは人間が直接読むことは難しいが、プログラムが効率的に扱うことができる。

C 言語では、いずれもストリームとして扱われる。ファイルをテキストモードで開いた場合は、処理系によって改行コードなどの変換が行われることがある。一方、バイナリモードでは、ファイル中のバイト列が原則としてそのま

⁵⁴後の「7.4.6 既存のファイルの変更」(p.81)で解説する。

ま読み書きされる。

バイナリファイルを扱う場合には、"rb" や "wb" のようにモードを指定してファイルを開くのが一般的である。

7.4.3 ファイルへの出力

■ 指定した出力ストリームへの1バイト出力：fputc

標準出力への1文字出力には putchar 関数を用いたが、出力先を指定して1バイトを出力する関数として fputc がある。

プロトタイプ： int fputc(int c, FILE *stream);

第1引数 c には出力する1バイトの値を指定し、第2引数 stream には出力先のストリームを指定する。この関数は、指定したストリームに1バイトを書き込む。

引数 c は int 型であるが、実際には unsigned char に変換された値が出力される。出力に成功した場合は、出力した unsigned char の値を int 型に変換した値を返し、エラーが発生した場合は EOF を返す。

参考) 標準出力に対しては、次のように stdout を指定することで出力できる。

```
fputc( 'A', stdout );
```

これは、標準出力に対して putchar を用いる場合と同様の結果となる。

■ 指定した出力ストリームへの文字列出力：fputs

文字列を出力する関数として fputs がある。この関数は、出力先のストリームを指定して文字列を出力することができる。

プロトタイプ： int fputs(const char *s, FILE *stream);

第1引数 s には出力する文字列を指定し、第2引数 stream には出力先のストリームを指定する。この関数は、文字列 s の終端文字 '\0' までの内容をストリームに書き込む。

出力に成功した場合は非負の値を返し、エラーが発生した場合は EOF を返す。

参考) 標準出力に対しては、次のように stdout を指定することで出力できる。

```
fputs( "Hello", stdout );
```

なお、puts は文字列を標準出力に出力し、末尾に改行文字「\n」を自動的に付加する。一方、fputs は改行文字を自動的に付加しない。

■ 指定した出力ストリームへの書式付き出力：fprintf

書式指定に従ってデータを出力する関数として fprintf がある。この関数は、出力先のストリームを指定して書式付き出力を行うことができる。

プロトタイプ： int fprintf(FILE *stream, const char *format, ...);

第1引数 stream には出力先のストリームを指定し、第2引数 format には書式文字列を指定する。第3引数以降には、書式文字列に対応する値を指定する。

fprintf の書式指定は、printf 関数と同様であり、%d、%f などの変換指定子を用いて値を出力する。出力に成功した場合は、出力された文字数を返し、エラーが発生した場合は負の値を返す。

参考) 標準出力に対しては、次のように stdout を指定することで出力できる。

```
fprintf( stdout, "a=%d x=%f\n", a, x );
```

この場合は printf と同様の結果となる。また、ファイルに対して書式付き出力を行う場合は、fopen により取得したストリームを指定する。

■ サンプルプログラム

各種の関数でファイルにテキストデータを出力するプログラム test010-7.c を次に示す。

プログラム：test010-7.c

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *f;
5     char buf[] = "This is a string.";
6     int c;
```

```

7
8     f = fopen("test010-7_out.txt","w");        // 出力用にファイルを開く
9     if ( f ) {                                // ファイルを正常に開くことができた場合
10         // fputcによるファイル出力
11         for ( c=0; buf[c]; c++ ) {            // 文字列buf[c]が0になるまでのループ
12             fputc(buf[c],f);
13         }
14         fputc('\n',f);
15         // fputsによるファイル出力
16         fputs("fputsによる出力\n",f);
17         // fprintfによるファイル出力
18         fprintf(f,"%s\n","fprintfによる出力");
19         fclose(f); // 処理の最後にファイルを閉じる
20     } else { // ファイルを開くことができなかった場合
21         fprintf(stderr,"ファイルが開けませんでした.\n");
22         return 1; // 失敗の通知
23     }
24 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test010-7.c のコンパイルと実行

```

$ gcc test010-7.c -o test010-7  ←コンパイル
$ ./test010-7  ←実行

```

このプログラムは正常に処理が終了すると、端末装置のディスプレイには何も出力しないが、プログラムを実行したディレクトリにファイル test010-7_out.txt が作成される。それを確認する例を次に示す。

例. ファイルの存在確認と中身の確認（先の例の続き）

```

$ ls -l test010-7_out.txt  ← test010-7_out.txt の存在確認
-rw-r--r-- 1 katsu admin 65  3月 24 14:34 test010-7_out.txt ←存在する(ファイルサイズは環境依存)
$ cat test010-7_out.txt  ← test010-7_out.txt の内容確認
This is a string. ←以下, test010-7_out.txt の内容
fputs による出力
fprintf による出力

```

注意) 実際のプログラミングでは、fclose の戻り値の確認と対応も実装すべきである。

7.4.3.1 出力バッファの強制書き出し：fflush

これまで解説してきた printf, fprintf, fputs などの出力関数では、出力されたデータは直ちにファイルや端末に書き込まれるとは限らない。実際には、バッファと呼ばれるメモリ領域に一時的に蓄えられ、一定の条件でまとめて書き出される仕組みになっている。

このように出力を一時的に蓄えてからまとめて書き出す仕組みを **バッファリング**という。バッファリングを行うことで、ディスクや端末へのアクセス回数が減り、プログラムの実行効率が向上する。

しかし、この仕組みにより、出力関数を呼び出しても、すぐには出力先に反映されない場合がある。このような場合に、バッファに蓄えられたデータを強制的に書き出す関数として fflush がある。

プロトタイプ: int fflush(FILE *stream);

引数 stream には対象となる出力ストリームを指定する。この関数を呼び出すと、バッファに蓄えられている出力データが、ストリームに対応する出力先へ強制的に送り出される⁵⁵。

fflush 関数は、正常に終了した場合は 0 を返し、エラーが発生した場合は EOF を返す。

通常、出力バッファの内容は、次のような場合に自動的に書き出される。

- 1) バッファが満たされた時
- 2) 改行文字「\n」が出力された時（行バッファリングされている場合⁵⁶）
- 3) fclose が呼び出された時
- 4) プログラムが正常終了した時

⁵⁵ fflush は C 言語処理系の出力バッファを書き出すものであり、データが物理的な記憶装置へ書き込まれたことまで保証するものではない。

⁵⁶ 端末に接続された標準出力 stdout は、一般に行単位でバッファリングされる。

しかし、途中経過をすぐに出力したい場合などには、`fflush` を用いて明示的に書き出す必要がある。

■ 全ての出力ストリームに対する `fflush`

`fflush` 関数の引数に `NULL` を指定すると、書出し可能な全ての出力ストリームに対して、バッファの書き出しを行う。

注意) `fflush` は出力ストリームに対して用いる。標準入力の内容を消去する目的で `fflush(stdin)` と記述してはならない。このような使い方は C 言語規格では保証されていない。

■ 応用例 標準入出力におけるプロンプト表示と入力の受け付け

標準入力から値を受け付ける際には、入力を促すためのメッセージ（プロンプト）を標準出力に表示することがある。次の例では、`printf` による出力の末尾に改行文字「`\n`」を付けていないため、環境によってはプロンプトが出力バッファに残り、画面にすぐ表示されない場合がある。

```
int n;
printf("Input value: ");
fflush(stdout);
scanf("%d", &n);
```

この例では、`scanf` による入力を受け付ける前に `fflush(stdout)` を実行して標準出力のバッファを強制的に書き出している。これにより、プロンプト「Input value: 」を確実に表示した後、利用者からの入力を待つことができる。

7.4.4 ファイルからの入力

先の「7.3.2.1 文字列の読み込み」(p.70) で解説した通り、ファイルからの入りに `fgets` 関数を使える他、様々な関数が存在する。

■ 指定した入力ストリームからの 1 文字入力： `fgetc`

`getchar` 関数は、入力が `stdin`（標準入力）に限られているが、`fgetc` 関数は入力元を指定できる。

プロトタイプ： `int fgetc( FILE *stream );`

ストリーム `stream` から 1 バイトを読み取る。読み取った文字は `unsigned char` として受け取るが、`int` 型に変換して返す。入力が終端に達した場合やエラー時には `EOF` を返す。

参考) `fgetc` の引数に `stdin` を指定すると、標準入力から 1 バイトを読み取ることができ、`getchar` を用いた場合と同様の結果となる。

■ 指定した入力ストリームからの書式付き入力： `fscanf`

書式文字列に従って入力データを読み込む関数として `fscanf` がある。この関数は、入力元のストリームを指定して書式付き入力を行うことができる。

プロトタイプ： `int fscanf( FILE *stream, const char *format, ... );`

第 1 引数 `stream` には入力元のストリームを指定し、第 2 引数 `format` には書式文字列を指定する。第 3 引数以降には、読み込んだ値を格納する変数のアドレスを指定する。

`fscanf` の書式指定は、`scanf` 関数と同様であり、`%d`、`%f` などの変換指定子を用いて入力データを解釈する。

この関数は、書式文字列に従って入力データを読み取り、正常に読み取って変数へ格納できた項目数を返す。途中で入力が指定した形式に合わなくなった場合は、それまでに格納できた項目数を返す。最初の項目を格納する前に入力終端または読み込みエラーに達した場合は `EOF` を返す。

参考) `fscanf` の第 1 引数に `stdin` を指定すると、標準入力から書式付き入力を行うことができ、`scanf` を用いた場合と同様の結果となる。

■ ファイル終端の確認： `feof`

ファイルの入力が終端（`EOF`）に達したかどうかを確認する関数として `feof` がある。

プロトタイプ： `int feof( FILE *stream );`

引数 `stream` には、対象となる入力ストリームを指定する。この関数は、指定したストリームにファイル終端を示す状態が設定されている場合には 0 以外の値を返し、そうでない場合には 0 を返す。

なお、ファイル終端（EOF）は、最後のデータを読み込んだ時点ではなく、その後さらに読み込みを試みて失敗した時点で検出される。そのため、feof を用いてファイル終端を判定する場合には注意が必要である。

例えば、fread や fscanf などの入力関数の戻り値を確認し、読み込みが終了した後に feof や ferror⁵⁷ を用いることで、ファイル終端によるものか、読み込みエラーによるものかを判定することができる。

重要) feof を用いて、データを読み込む前にファイル終端を判定することはできない。まず入力関数を実行し、その戻り値によって読み込みの成否を判定するのが基本である。

■ サンプルプログラム

fgetc, fgets, fscanf を使って入力用のファイル test010-8.in.txt の内容を読み取って出力するプログラム test010-8.c を次に示す。このプログラムでは、入力ファイルの最初の 1 行を fgetc の反復実行で読み取り、2 行目を fgets で、3 行目以降の内容を fscanf の反復実行で読み取る。（ここでは、各行は読み込み用バッファに収まる長さであるものとする）

fscanf の反復実行では、単語毎の読み取りである。

プログラム：test010-8.c

```
1  #include    <stdio.h>
2  #define    BUFSIZE    256
3
4  int main() {
5      FILE    *f;
6      char    buf[BUFSIZE];
7      int     c, ch;
8
9      f = fopen("test010-8.in.txt","r"); // 入力用ファイルを開く
10     if ( f ) { // 正常にオープンできた場合の処理
11         // 1バイトずつfgetcで読み込むループ
12         for ( c=0; c < BUFSIZE-1 &&
13             (ch=fgetc(f)) != '\n' && ch != EOF; c++ ) {
14             buf[c] = (char)ch;
15         }
16         buf[c] = '\0'; // 文字列の終端処理
17         printf("%s\n",buf); // fgetcの繰り返しで読み取った文字列を出力
18         // fgetsによる1行の読み込み
19         if ( fgets(buf,BUFSIZE,f) ) {
20             printf("%s",buf); // 読み取った文字列を出力
21         }
22         // fscanfで単語を1つずつ読み込んで出力するループ
23         while ( fscanf(f,"%255s",buf) == 1 ) {
24             printf("%s ",buf);
25         }
26         printf("\n");
27         // fscanfによる読み込みが終了した理由を確認
28         if ( feof(f) ) {
29             // ファイル終端に達した
30         } else if ( ferror(f) ) {
31             fprintf(stderr,"ファイルの読み込み中にエラーが発生しました.\n");
32         }
33         fclose(f);
34     } else { // 入力ファイルをオープンできなかった場合の処理
35         fprintf(stderr,"ファイルが開けませんでした.\n");
36         return 1; // 失敗の通知
37     }
38 }
```

ファイル：test010-8.in.txt

```
1 The first line.
2 The second line.
3 The third line.
```

プログラム test010-8.c をコンパイルして実行する例を次に示す。

⁵⁷後の「7.4.8 ファイルアクセスの際のエラーの確認：ferror」(p.87)で解説する。

例. test010-8.c のコンパイルと実行

```
$ gcc test010-8.c -o test010-8  ←コンパイル
$ ./test010-8  ←実行
The first line.      ←以下は出力
The second line.
The third line.
```

このプログラムを実行するディレクトリにファイル test010-8.in.txt がオープンできなければ（ファイルが無い、アクセス権が無いなど）次のような結果となる。

例. 入力ファイルがオープンできない場合

```
$ ./test010-8  ←実行
ファイルが開けませんでした。 ←結果
```

7.4.5 バイナリデータの扱い

■ バイナリデータの書き込み：fwrite

バイナリデータをまとめて書き込む関数として fwrite がある。この関数は、指定したストリームに指定したサイズのデータを書き込む。

プロトタイプ： `size_t fwrite( const void *ptr, size_t size, size_t nmemb, FILE *stream );`

第1引数 ptr には書き込むデータを格納したバッファを指定する。第2引数 size には1要素あたりのサイズ（バイト数）を指定し、第3引数 nmemb には書き込む要素数を指定する。第4引数 stream には出力先のストリームを指定する。

この関数は、1要素あたり size バイトのデータを最大 nmemb 個書き込み、正常に書き込むことができた要素数を返す。エラーが発生した場合などには、nmemb より小さい値が返される。

この関数は、構造体や配列などのデータをまとめてファイルに保存する際に用いられる。

■ バイナリデータの読み込み：fread

バイナリデータをまとめて読み込む関数として fread がある。この関数は、指定したストリームから指定したサイズのデータを読み込む。

プロトタイプ： `size_t fread( void *ptr, size_t size, size_t nmemb, FILE *stream );`

第1引数 ptr には読み込んだデータを格納するバッファを指定する。第2引数 size には1要素あたりのサイズ（バイト数）を指定し、第3引数 nmemb には読み込む要素数を指定する。第4引数 stream には入力元のストリームを指定する。

この関数は、1要素あたり size バイトのデータを最大 nmemb 個読み込み、正常に読み込むことができた要素数を返す。ファイル終端に達した場合や読み込みエラーが発生した場合には、nmemb より小さい値が返される。

■ サンプルプログラム

構造体をバイナリデータとしてファイルに保存するプログラム test010-9_w.c、保存されたバイナリファイルを読み込むプログラム test010-9_r.c を例に挙げてバイナリファイルの取扱の例を示す。

プログラム： test010-9_w.c バイナリファイルの保存

```
1  #include    <stdio.h>
2
3  typedef struct {
4      int     age;
5      double  weight, height;
6  } Human;
7
8  int main() {
9      FILE    *f;
10     Human    p1 = {24, 59.4, 164.7};
11     size_t    s, n;
12
13     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
14            "p1", p1.age, p1.weight, p1.height);
```

```

15
16 // p1をバイナリデータとしてファイル出力
17 s = sizeof(Human);
18 f = fopen("test010-9_dat.bin", "wb");
19
20 if ( f ) {
21     n = fwrite(&p1, s, 1, f);
22     if ( n != 1 ) {
23         fprintf(stderr, "ファイルへの書込みに失敗しました. \n");
24         fclose(f);
25         return 1;
26     }
27     if ( fclose(f) == EOF ) {
28         fprintf(stderr, "ファイルを正常に閉じることができませんでした. \n");
29         return 1;
30     }
31     printf("p1 is saved to file. (%zu)\n", n);
32 } else {
33     fprintf(stderr, "ファイルが開けませんでした. \n");
34     return 1; // 失敗の通知
35 }
36 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test010-9_w.c のコンパイルと実行

```

$ gcc test010-9_w.c -o test010-9_w  ←コンパイル
$ ./test010-9_w  ←実行
p1: age=24, weight=59.4, height=164.7 ←構造体の内容
p1 is saved to file.(1) ←保存処理の報告

```

この処理でバイナリファイル test010-9_dat.bin ができた。次の例はそのファイルの存在を確認するものである。

例. 作成されたバイナリファイルの存在確認

```

$ ls -l test010-9_dat.bin  ←確認
-rw-r--r-- 1 katsu admin 24 3月 24 18:06 test010-9_dat.bin ←存在している(ファイルサイズは環境依存)

```

プログラム：test010-9_r.c バイナリファイルの読み込み

```

1 #include <stdio.h>
2
3 typedef struct {
4     int age;
5     double weight, height;
6 } Human;
7
8 int main() {
9     FILE *f;
10    Human p1;
11    size_t s, n;
12
13    // ファイルからバイナリデータを読み込む
14    s = sizeof(Human);
15    f = fopen("test010-9_dat.bin", "rb");
16
17    if ( f ) {
18        n = fread(&p1, s, 1, f);
19        if ( n != 1 ) {
20            if ( feof(f) ) {
21                fprintf(stderr,
22                    "必要なデータを読み込む前にファイル終端に達しました. \n");
23            } else if ( ferror(f) ) {
24                fprintf(stderr, "ファイルの読み込み中にエラーが発生しました. \n");
25            } else {
26                fprintf(stderr, "ファイルからデータを読み込めませんでした. \n");
27            }
28            fclose(f);
29            return 1;
30        }
31        if ( fclose(f) == EOF ) {

```

```

32         fprintf(stderr, "ファイルを正常に閉じることができませんでした. \n");
33         return 1;
34     }
35     printf("p1 is loaded from file. (%zu)\n", n);
36     printf("%s: age=%d, weight=%.1f, height=%.1f\n",
37           "p1", p1.age, p1.weight, p1.height);
38 } else {
39     fprintf(stderr, "ファイルが開けませんでした. \n");
40     return 1;        // 失敗の通知
41 }
42 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test010-9_r.c のコンパイルと実行

```

$ gcc test010-9_r.c -o test010-9_r  ←コンパイル
$ ./test010-9_r  ←実行
p1 is loaded from file.(1)          ←読み込み処理の報告
p1: age=24, weight=59.4, height=164.7 ←構造体の内容

```

test010-9_w.c で作成した構造体と同じものが読み込まれていることがわかる。

このように、構造体のメモリ上の内容をバイナリデータとしてまとめて保存し、再び構造体へ読み込むことができる。ただし、この方法で保存されるのは構造体のメモリ上の表現そのものである。従って、処理系や実行環境が異なる場合には、各データ型のサイズ、構造体のパディング、エンディアン、数値の内部表現などが異なり、同じファイルを正しく読み込めない場合がある。従って、異なる環境間で共通に利用するファイル形式を作成する場合には、各データ項目の形式やサイズ、バイト順などをあらかじめ定めて保存する必要がある。

7.4.6 既存のファイルの変更

ファイルをオープンする際、モードを "w" とすると、ファイルが存在しない場合は新規に作成され、既に存在する場合はそれまでの内容が破棄される。一方、"a" (追記モード) を指定すると、既存の内容を残したままファイルの末尾に追記することができる。このことをサンプルプログラムを挙げて示す。

まず、次のプログラム test010-a-0.c でファイル test010-a_dat.txt を新規に作成する。

プログラム：test010-a-0.c

```

1  #include <stdio.h>
2
3  int main() {
4      FILE *f;
5
6      f = fopen("test010-a_dat.txt", "w");
7      if ( f ) {
8          fprintf(f, "%s\n", "abcdefghijklm");
9          fclose(f);
10     } else {
11         fprintf(stderr, "ファイルが開けませんでした. \n");
12         return 1;
13     }
14 }

```

例. test010-a-0.c のコンパイルと実行 (先の例の続き)

```

$ gcc test010-a-0.c -o test010-a-0  ←コンパイル
$ ./test010-a-0  ←実行
$ ls -l test010-a_dat.txt  ←作成されたファイルの存在確認
-rw-r--r-- 1 katsu admin 15  3月 25 13:00 test010-a_dat.txt ←出来ている(ファイルサイズは環境依存)
$ cat test010-a_dat.txt  ←作成されたファイルの内容確認
abcdefghijklm      ←ファイルの内容

```

これで、"abcdefghijklm\n" の内容を持つファイル test010-a_dat.txt ができた。次のプログラム test010-a-1.c はこのファイル test010-a_dat.txt を追記モード "a" で開き、その内容に 1 行追記する。

プログラム：test010-a-1.c

```
1  #include    <stdio.h>
2
3  int main() {
4      FILE *f;
5
6      f = fopen("test010-a_dat.txt", "a");
7      if ( f ) {
8          fprintf(f, "%s\n", "nopqrstuvwxyz");
9          fclose(f);
10     } else {
11         fprintf(stderr, "ファイルが開けませんでした. \n");
12         return 1;
13     }
14 }
```

例. test010-a-1.c のコンパイルと実行（先の例の続き）

```
$ gcc test010-a-1.c -o test010-a-1  ←コンパイル
$ ./test010-a-1  ←実行
$ ls -l test010-a_dat.txt  ←追記されたファイルの確認
-rw-r--r-- 1 katsu admin 30  3月 25 13:00 test010-a_dat.txt ←サイズが増えている
$ cat test010-a_dat.txt  ←追記されたファイルの内容確認    (↑このサイズも環境依存)
abcdefghijklm      ←先の内容
nopqrstuvwxyz     ←追加された内容
```

ファイル test010-a_dat.txt が更に 1 行追記され、2 行の内容になっていることがわかる。

ファイルをオープンする際のモードには、既存のファイルを読み書き可能な状態で開く "r+"（読み書きモード）がある。このモードでは、ファイルの現在のアクセス位置から読み書きが行われる。通常、ファイルを開いた直後はアクセス位置は先頭にあるため、この状態で書き込み処理を行うと、既存の内容が上書きされる。

次のプログラム test010-a-2.c はこのファイル test010-a_dat.txt を "r+" のモードで開き、その内容の 1 行目の開始部分を上書きして変更する。

プログラム：test010-a-2.c

```
1  #include    <stdio.h>
2
3  int main() {
4      FILE *f;
5
6      f = fopen("test010-a_dat.txt", "r+");
7      if ( f ) {
8          fprintf(f, "%s", "ABCDEF");
9          fclose(f);
10     } else {
11         fprintf(stderr, "ファイルが開けませんでした. \n");
12         return 1;
13     }
14 }
```

例. test010-a-2.c のコンパイルと実行

```
$ gcc test010-a-2.c -o test010-a-2  ←コンパイル
$ ./test010-a-2  ←実行
$ cat test010-a_dat.txt  ←変更されたファイルの内容確認
ABCDEFghijklm    ←変更された内容
nopqrstuvwxyz     ←先の内容
```

ファイル test010-a_dat.txt の 1 行目、"abcdefghijklm\n" の開始部分が変更され、"ABCDEFghijklm\n" となっていることがわかる。

参考) 読み書き両用で新規ファイルを作成するには "w+" のモードでファイルを開く。この場合、既存のファイルがある場合はその内容は破棄される。

注意) "r+", "w+", "a+" のような読み書き両用のモードでは、同じストリームに対して読み込みと書き込みを交互に行う際に注意が必要である。書き込みから読み込みに切り替える場合には `fflush` や `fseek` などを、読み込みから書き込みに切り替える場合には原則として `fseek` などのファイル位置を設定する処理を行う。なお、"r+" では既存のファイルを開くため、ファイルが存在しない場合はオープンに失敗する。

7.4.7 ファイルのアクセス位置の操作

これまでの解説では、ファイルの読み書きは常に先頭から順に行われるものとして扱ってきた。すなわち、読み書きの度にファイルの処理位置は自動的に前へ進み、過去の位置へ戻る操作は行わないものとして扱っていた。このように、ファイル先頭から順に読み書きする方式を **シーケンシャルアクセス** という。例えば、`fscanf` や `fgets` による読み取りでは、データを読み取る度にファイルのアクセス位置は次の位置へと進む。また、`fprintf` や `fputs` による書き込みでも、書き込みのたびにファイルのアクセス位置が前方へと進む。これまでの、このアクセス位置が自動的に前へ進むという振る舞いを暗黙の前提としていた。

しかし、実際にはファイルのアクセス位置はプログラムから自由に変更することが可能である。すなわち、任意の位置へ移動して読み書きを行うことができる。このように、ファイルの任意の位置へ移動して読み書きする操作を **シーク (seek)** と呼ぶ。シーク操作を用いると、例えば次のような処理が可能となる。

- ファイルの先頭へ戻る
- ファイルの途中の位置へ移動する
- ファイルの末尾へ移動する
- 既存のデータを上書きする
- 特定の位置のデータのみを書き換える

このように、シーク操作を用いることで、ファイル内容を先頭から順に処理するだけでなく、任意の位置へ直接移動して読み書きすることができる。このような方式は一般に **ランダムアクセス** と呼ばれる。なお、ファイルのアクセス位置の操作はテキストファイルに対しても可能であるが、通常はバイナリファイルに対して用いられることが多い。テキストファイルでは、処理系による改行コードなどの変換が行われる場合があり、ファイル上の位置を単純なバイト単位で扱えないことがあるためである。そのため、ファイルのランダムアクセスは主としてバイナリファイルに対して用いられる。

C 言語では、ファイルのアクセス位置を操作するために、`fseek`、`ftell`、`rewind` などの関数が用意されている。以下では、これらの関数について順に解説する。

7.4.7.1 ファイルのアクセス位置の移動：fseek

ファイルのアクセス位置を移動するための関数として、`fseek` が用意されている。この関数のプロトタイプは `stdio.h` にあり、使用する際はこれをインクルードする。

プロトタイプ： `int fseek( FILE *fp, long offset, int whence );`

第 1 引数 `fp` には対象となるファイルを表す `FILE` 型ポインタを指定する。第 2 引数 `offset` には移動量を指定する。第 3 引数 `whence` には、移動の基準となる位置を指定する。`fseek` は、この基準位置から `offset` だけ移動した位置へ、ファイルのアクセス位置を変更する。

第 3 引数 `whence` には、通常、次のいずれかを指定する。

- `SEEK_SET` ファイルの先頭を基準とする
- `SEEK_CUR` 現在のアクセス位置を基準とする
- `SEEK_END` ファイルの末尾を基準とする

例えば、

```
fseek( fp, 0, SEEK_SET );
```

と書くと、ファイルのアクセス位置は先頭へ移動する。また、

```
fseek( fp, 0, SEEK_END );
```

と書くと、ファイルのアクセス位置は末尾へ移動する。更に、

```
fseek( fp, 10, SEEK_SET );
```

と書くと、バイナリストリームではファイル先頭から 10 バイト後方の位置へ移動する。

fseek の返り値は、正常にアクセス位置の移動が行われた場合は 0 であり、失敗した場合は 0 以外の値である。そのため、必要に応じて返り値を調べることで、アクセス位置の移動が成功したかどうかを確認できる。

なお、fseek はテキストファイルに対しても使用できるが、テキストモードでは指定できる offset の値に制約がある。例えば、任意のバイト数を指定して移動できるとは限らない⁵⁸。このため、バイト単位で位置を指定するランダムアクセスには、通常バイナリファイルを用いる。

■ ファイルのサイズに関する注意

fseek の第 2 引数 offset には long の値（符号付き）を与える。従って、long が 32 ビットである環境では、offset に指定できる移動量が約 ±2GiB の範囲に制限される。そのため、大きなファイルの任意の位置を扱う用途では、より大きなオフセット値を扱える関数（下記）を使用する必要がある。

環境	関数のプロトタイプ	備 考
POSIX	int fseeko(FILE *fp, off_t offset, int whence);	off_t 型による大きなファイル位置を扱える
Windows	int _fseeki64(FILE *fp, __int64 offset, int whence);	__int64 型が 64 ビットの値を扱える

注) 環境や使用する API によっては sys/types.h のインクルードが必要となる場合がある。

7.4.7.2 アクセス位置の取得：ftell

ファイルの現在のアクセス位置を取得するための関数として、ftell が用意されている。この関数のプロトタイプは stdio.h にあり、使用する際はこれをインクルードする。

プロトタイプ： long ftell(FILE *fp);

引数 fp には対象となるファイルを表す FILE 型ポインタを指定する。ftell は、現在のファイルアクセス位置を表す値を返す。バイナリストリームでは、この値はファイル先頭から現在位置までのバイト数として扱うことができる。一方、テキストストリームでは、この値が単純なバイト位置を表すとは限らない。

例えば、次のように書くことで、現在のアクセス位置を変数 pos に取得できる。

```
long pos = ftell( fp );
```

ftell の返り値は、正常に取得できた場合は現在のアクセス位置を表す値となり、失敗した場合は -1L が返される。そのため、必要に応じて返り値を確認することで、アクセス位置の取得が成功したかどうかを判定することができる。

■ アクセス位置の退避と復帰

ftell は fseek と組み合わせて用いられることが多い。例えば、次のように書くことで、現在のアクセス位置を保存し、後でその位置へ戻ることができる。

例. ストリーム fp のアクセス位置の退避と復帰

```
long pos;
pos = ftell( fp );           ← 現在位置の退避
    (アクセス位置の変更が伴う処理)
fseek( fp, pos, SEEK_SET );  ← 退避した位置へ復帰
```

このように、ftell と fseek を組み合わせることで、任意の位置を記録し、その位置へ戻ることが可能となる。なお、実際のプログラミングでは ftell や fseek の戻り値を確認して、処理が成功したかどうかを確認する必要がある。

■ バイナリファイルのサイズの取得

バイナリファイルでは、fseek と組み合わせることで、ファイルサイズを取得することもできる。例えば、次のように書く。

例. ストリーム fp のファイルサイズの取得

```
fseek( fp, 0, SEEK_END );
size = ftell( fp );
```

なお、この操作を行うとアクセス位置はファイル末尾へ移動するため、必要なら事前に現在位置を退避し、処理後に復帰する。

⁵⁸テキストモードでは、処理系による改行コードなどの変換のため、ファイル上の位置を単純なバイト数として扱えない場合がある。

このように、`fseek` と `ftell` は、ファイルのアクセス位置の操作において基本となる関数である。

■ ファイルのサイズに関する注意

`ftell` は `long` 型の値を返す。従って、`long` が 32 ビットである環境では、`ftell` で表現できるファイル位置に制限があり、大きなファイルの位置を正しく取得できない場合がある。そのような用途では、より大きなファイル位置を扱える関数（下記）を使用する。

環境	関数のプロトタイプ	備 考
POSIX	<code>off_t ftello(FILE *fp);</code>	<code>off_t</code> 型で大きなファイル位置を扱える
Windows	<code>_int64 _ftelli64(FILE *fp);</code>	<code>_int64</code> 型が 64 ビットの値を扱える

注) 環境や使用する API によっては `sys/types.h` のインクルードが必要となる場合がある。

7.4.7.3 ファイル先頭への移動：rewind

ファイルのアクセス位置を先頭へ戻すための関数として、`rewind` が用意されている。この関数のプロトタイプは `stdio.h` にあり、使用する際はこれをインクルードする。

プロトタイプ： `void rewind( FILE *fp );`

引数 `fp` には対象となるファイルを表す `FILE` 型ポインタを指定する。`rewind` を実行すると、ファイルのアクセス位置は先頭へ移動する。

例えば、

```
rewind( fp );
```

と書くと、ファイルのアクセス位置は先頭に戻る。そのため、一度最後まで読み取ったファイルを、再び先頭から読み直したい場合などに用いることができる。

`rewind` は、

```
fseek( fp, 0, SEEK_SET );
```

と似た働きをするが、これに加えて、ストリームに設定されているファイル終端の状態（EOF）やエラーの状態も解除する⁵⁹。そのため、ファイルの終端まで読み込んだ後で再び先頭から読み直したい場合には、`rewind` を用いる。

なお、`rewind` の返り値は `void` 型であり、値を返さない。そのため、処理が成功したかどうかを返り値によって判定することはできない。従って、エラーの有無を戻り値で直接確認する必要がある場合は、`fseek` を用いる。

■ サンプルプログラム

ファイルに対するランダムアクセスの例としてプログラム `test010-b.c` を次に示す。このプログラムでは、ファイル `test010-b.dat` を 5 バイトの空白文字で初期化し、入力された 0~4 の数値によって指定されたファイル内の位置に、対応するアルファベットを書き込む。ファイル内の書き込み位置の指定には `fseek` を用いている。ファイルの内容を表示する関数 `fshow` の中では、`rewind` を用いてファイルの先頭に移動する処理を行っている。

プログラム：test010-b.c

```
1  #include    <stdio.h>
2  #define     FSIZE      5
3
4  // ファイル内容を表示する関数
5  void fshow( FILE *f ) {
6      int     i;
7
8      printf("index: 01234\n");    // ゲージの表示
9      printf("data : ");          // ファイル内容の表示
10     rewind(f);                  // ファイル先頭に移動
11
12     // ファイル内容の表示
13     for ( i=0; i<FSIZE; i++ ) {
14         int ch = fgetc(f);
15         if ( ch == EOF ) break;
16         printf("%c", ch);
17     }
18     printf(" [last:%ld]\n", ftell(f));
```

⁵⁹これは `clearerr( fp )` によって EOF 表示とエラー表示を解除するのと同様の効果である。

```

19     rewind(f);                // ファイル先頭に移動
20 }
21
22 int main() {
23     FILE *f;
24     int i, n;
25     char s[] = "ABCDE";        // アルファベットのデータ
26
27     // ファイルの作成と初期化
28     f = fopen("test010-b_dat", "wb");
29     if ( f ) {
30         for ( i=0; i<FSIZE; i++ ) {
31             fputc(' ', f);      // 空白文字で充填する
32         }
33         fclose(f);
34     } else {
35         fprintf(stderr, "ファイルが作成できませんでした. \n");
36         return 1;
37     }
38     // 指定したバイト位置にアルファベットを書き込むループ
39     f = fopen("test010-b_dat", "r+b");
40     if ( f ) {
41         fshow(f);               // ファイルの初期状態の表示
42         while ( 1 ) {
43             printf("input [-1:end,0-%d]> ", FSIZE-1);
44             if ( scanf("%d", &n) != 1 ) {
45                 fprintf(stderr, "数値を入力してください. \n");
46                 break;
47             }
48             if ( n < 0 ) break;   // 入力値が負なら終了
49             if ( 0 <= n && n < FSIZE ) {
50                 if ( fseek(f, n, SEEK_SET) != 0 ) {
51                     fprintf(stderr, "アクセス位置の移動に失敗しました. \n");
52                     break;
53                 }
54                 fputc(s[n], f);   // 指定位置にアルファベットを書き込む
55                 fshow(f);         // ファイル内容を表示
56             } else {
57                 printf("ignored...\n");
58             }
59         }
60         fclose(f);
61     } else {
62         fprintf(stderr, "ファイルを開くことができませんでした. \n");
63         return 1;
64     }
65 }

```

例. test010-b.c のコンパイルと実行

```

$ gcc test010-b.c -o test010-b  ←コンパイル
$ ./test010-b  ←実行
index: 01234                ←初期化済みのデータが
data :      [last:5]        表示されている
input[-1:end,0-4]> 4  ←インデックス 4 のバイト位置に書き込み
index: 01234                ←データ'E' が
data :      E [last:5]      書き込まれている
input[-1:end,0-4]> 0  ←インデックス 0 のバイト位置に書き込み
index: 01234                ←データ'A' が
data : A    E [last:5]      書き込まれている
input[-1:end,0-4]> 3  ←インデックス 3 のバイト位置に書き込み
index: 01234                ←データ'D' が
data : A DE [last:5]        書き込まれている
input[-1:end,0-4]> 1  ←インデックス 1 のバイト位置に書き込み
index: 01234                ←データ'B' が
data : AB DE [last:5]       書き込まれている
input[-1:end,0-4]> 2  ←インデックス 2 のバイト位置に書き込み
index: 01234                ←データ'C' が
data : ABCDE [last:5]       書き込まれている

```

```
input[-1:end,0-4]> 5  ←インデックス5のバイト位置に書き込もうと
ignored...          すると無視される
input[-1:end,0-4]> -1  ←負の値を入力すると終了する
$ cat test010-b_dat  ←ファイルの内容確認
ABCDE               ←内容
```

ファイル内のバイト位置0~4を自由に決めてアルファベットを書き込んでいる様子がわかる。

7.4.8 ファイルアクセスの際のエラーの確認：ferror

ファイル操作中にエラーが発生したかどうかを確認するための関数として、`ferror` が用意されている。この関数のプロトタイプは `stdio.h` にあり、使用する際はこれをインクルードする。

プロトタイプ： `int ferror( FILE *fp );`

引数 `fp` には対象となるファイルを表す `FILE` 型ポインタを指定する。`ferror` は、指定したストリームにエラーを示す状態が設定されている場合は 0 以外の値を返し、そうでない場合は 0 を返す。

なお、`ferror` は `fopen` の失敗には用いられず、ファイル操作中（オープン後の処理）のエラーを検出するための関数である。

ファイルアクセス中に起こるエラーの原因は多岐にわたり、全ての状況を列挙することは難しいが、次のようなケースが多い。

- ・ ディスク容量不足や外部装置の切断などによる入出力エラー
- ・ 読み込み専用モードで書き込みを行うなどのモード指定の誤り
- ・ ネットワークドライブなどの通信エラー

■ ファイル読み込み時に起こるエラーの扱い

ファイルの読み込み処理の後で次のように書くことで、エラーの有無を確認することができる。

例. エラー状態の確認

```
if ( ferror( fp ) ) {
    fprintf(stderr,"ファイル操作中にエラーが発生しました. \n");
}
```

このように、`ferror` を用いることで、ファイル操作中にエラーが発生したかどうかを判定することができる。

なお、ファイルの読み込みでは、ファイル終端（EOF）に達した場合とエラーが発生した場合の両方で、読み込み関数が失敗を示す値を返すことがある。このため、読み込み処理が失敗した場合には、`feof` と `ferror` を組み合わせて判定することが多い。

例. EOF とエラーの判定

```
if ( feof( fp ) ) {
    printf("ファイル終端に達しました. \n");
}
else if ( ferror( fp ) ) {
    fprintf(stderr,"ファイル読み込みエラーが発生しました. \n");
}
```

このように、`feof` と `ferror` を用いることで、読み込み終了の原因がファイル終端によるものか、エラーによるものかを区別することができる。

参考) エラー状態や EOF 状態は `clearerr` 関数によって解除することができる。

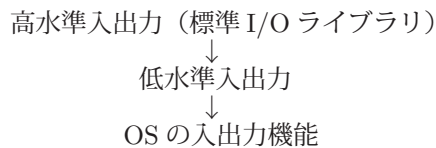
7.5 低水準入出力

これまでに `printf` や `scanf`, `fopen`, `fread`, `fwrite` などの関数を用いた入出力の方法について説明してきた。これらは、**標準入出力ライブラリ**（標準 I/O, `stdio`）⁶⁰ によって提供されるものであり、通常のプログラミングではこれらを用いるのが一般的である。これらの機能は、バッファリングなどの仕組みを内部に備えており、使いやすく効率的な入出力を実現できるように設計されている。

一方で、これらの標準 I/O ライブラリのさらに下位には、**低水準入出力**と呼ばれる仕組みが存在する。これは、オペレーティングシステム（OS）が提供する基本的な入出力機能に近いインタフェースを利用するものであり、より単純で素朴な入出力を行うことができる。

一般に、プログラムがファイルや端末装置などに対して入出力を行う場合、最終的には OS が提供する基本的な機能と呼び出すことになる。このような OS の基本機能の呼び出しは、**システムコール**と呼ばれる。標準 I/O ライブラリは、OS の基本的な入出力機能を内部で利用し、より高機能で扱いやすい入出力機能を提供している。

このように、入出力機能には次のような階層構造があると考えることができる。



通常のプログラミングでは、標準 I/O を用いる方が便利であり、バッファリングや書式変換などの機能も利用できる。しかし、より単純な入出力を行いたい場合や、標準 I/O ライブラリを使用することが適切でない状況では、低水準入出力が用いられることもある。

例えば、シグナル処理など、標準 I/O ライブラリの関数を使用することが適切でない特殊な状況では、`write` などの低水準入出力機能を用いて最小限のデータを出力すること⁶¹がある。

この節では、このような低水準入出力の基本的な考え方と、代表的な機能について説明する。なお、通常のプログラミングでは標準 I/O ライブラリを用いることが推奨されるが、低水準入出力の存在を理解しておくことは、より実践的なプログラムを作成する上で有用である。

7.5.1 低水準入出力の基本概念

低水準入出力では、ファイルや端末装置などの入出力対象を、**ファイル**として統一的に扱う。ここでいうファイルとは、ディスク上のファイルに限らず、端末装置やパイプなども含めた入出力の対象全般を意味する。このように、様々な入出力対象を統一的に扱う仕組みは、UNIX 系 OS における重要な設計思想の一つである。

低水準入出力の具体的な仕組みは OS によって異なるが、本書では、UNIX/POSIX 系 OS で用いられる低水準入出力を中心に説明する。この方式では、ファイルや端末装置などの入出力対象を、ファイルディスクリプタと呼ばれる整数値によって識別する。ファイルディスクリプタは、OS が入出力対象を識別するために割り当てる番号であり、プログラムはこの番号を用いて入出力操作を行う。

プログラムがファイルを利用する際には、まずファイルを**オープン**し、その後必要な入出力操作を行い、最後に**クローズ**するという流れになる。このとき、ファイルをオープンすると、OS によってファイルディスクリプタが割り当てられ、プログラムはこの値を用いてファイルを操作する。

この流れは、標準 I/O における `fopen`, `fread`, `fwrite`, `fclose` などの操作と基本的には同様であるが、低水準入出力ではより単純な仕組みで実現されている。すなわち、低水準入出力では、ファイルディスクリプタという整数値を直接扱い、OS の基本機能を利用して入出力を行う。

なお、プログラムが開始された時点で、既にいくつかのファイルディスクリプタが割り当てられている。UNIX/POSIX 系の環境では、プログラムの開始時点で通常、次の 3 つのファイルディスクリプタが標準入出力用として利用可能になっている。また、これらについてはヘッダファイル `unistd.h` に対応するマクロ定数が定義されている。（下記参照）

⁶⁰高水準入出力ライブラリと呼ばれることもある。

⁶¹エラーメッセージを出力して、プログラムを緊急終了させるケースなど。

ファイルディスクリプタ	意 味	マクロ定数
0	標準入力（キーボードなど）	STDIN_FILENO
1	標準出力（ディスプレイなど）	STDOUT_FILENO
2	標準エラー出力	STDERR_FILENO

UNIX/POSIX 系の環境では、これらは標準 I/O における `stdin`, `stdout`, `stderr` にそれぞれ対応している。すなわち、標準 I/O のストリームの背後では、このようなファイルディスクリプタが利用されている。

このように、低水準入出力では、入出力対象をファイルディスクリプタによって識別するという単純な仕組みを用いる。このため、低水準入出力は、より基本的な入出力の仕組みとして位置付けられる。

次節では、このような低水準入出力において用いられる基本的な関数について説明する。

7.5.2 ファイルのオープン：open

標準 I/O の場合と同様に、低水準入出力でも、ファイルを利用する際には、まずファイルをオープンする必要がある。この操作を行うための関数が `open` である。この関数は、OS の低水準入出力機能を利用するための基本的な関数の一つである。なお、`open` 関数の宣言や、各種フラグの定義は `fcntl.h` にあり、使用する際にはこのヘッダファイルをインクルードする。

プロトタイプ： `int open( const char *pathname, int flags, ... );`

この関数はファイルをオープンし、成功した場合にはファイルディスクリプタを返す。失敗した場合には `-1` を返す。

第 1 引数 `pathname` には、オープンするファイルのパスを指定する。第 2 引数 `flags` には、ファイルのオープン方法を指定するためのフラグを指定する。また、新しいファイルを作成する場合には、第 3 引数として `mode_t` 型の値 `mode`（モード）を与え、作成するファイルのアクセス権を指定する。

例えば、読み込み専用でファイルをオープンする場合には、次のようにする。

例. ファイル `data.txt` を読み込み用に開く

```
int fd;
fd = open("data.txt", O_RDONLY);
```

このとき、オープンに成功すると、`fd` にはファイルディスクリプタが格納される。この値を用いて、低水準入出力の関数によりファイル操作（読み込み）を行うことができる。第 2 引数以降のフラグとモードについては後で解説する。

ファイルのオープンに失敗した場合には `open` は `-1` を返す。従って、`open` の戻り値は必ず確認する必要がある。

このように、低水準入出力では、ファイルディスクリプタという整数値を用いて、入出力対象を扱う点が、標準 I/O との大きな違いである。

7.5.2.1 フラグとモード

`open` 関数では、第 2 引数 `flags`（フラグ）によってファイルのオープン方法を指定する。この引数には、あらかじめ定義された定数を指定する。これらの定数は `fcntl.h` に定義されている。`flags` に指定する代表的な値として、次のようなものがある。

<code>O_RDONLY</code>	読み込み専用でオープンする
<code>O_WRONLY</code>	書き込み専用でオープンする
<code>O_RDWR</code>	読み書き両用でオープンする
<code>O_CREAT</code>	ファイルが存在しない場合に新規作成する
<code>O_TRUNC</code>	既存ファイルの内容を切り詰めて長さ 0 にする
<code>O_APPEND</code>	ファイルの末尾に追加書き込みする

これらのフラグは、必要に応じてビット単位の OR 演算子「`|`」⁶² を用いて組み合わせる。例えば、書き込み専用でファイルを開き、ファイルが存在しない場合には新規作成し、既に存在する場合には内容を空にする場合は、次のように指定する。

```
O_WRONLY | O_CREAT | O_TRUNC
```

⁶²後の「9.3 ビット演算」（p.114～）で解説する。

このように、`flags` では複数の動作を組み合わせて指定することができる。これは、低水準入出力ではファイルの扱いを細かく制御できるように設計されているためである。

注意) `O_RDONLY`, `O_WRONLY`, `O_RDWR` は「|」で組み合わせてはならない。どれか1つのみを使用すること。

一方、第3引数 `mode` (モード) は、`O_CREAT` を指定した場合に、新しく作成するファイルのアクセス権を指定するために用いられる。この引数は、ファイルを新規作成しない場合には指定する必要はない。

`mode` には、通常8進数で**アクセス権**⁶³を指定する。例えば、「0644」と指定すると、所有者には読み書き、それ以外のユーザには読み込みの権限を与える指定となる。ただし、実際に設定されるアクセス権は、システムのファイル作成マスク (`umask`) の影響を受ける。

新しくファイルを作成して書き込み専用でオープンする場合は、次のようになる。

例. ファイルの新規作成

```
int fd;
fd = open("data.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

この例では、

- ・書き込み専用でオープンする
- ・ファイルが存在しない場合は新規作成する
- ・既存ファイルは内容を空にする
- ・新規作成時のアクセス権は 0644

という指定を同時に行っている。

このように、低水準入出力では、`flags` および `mode` によってファイルの扱いを細かく制御することができる。一方で、これらの指定を誤ると期待した動作にならない可能性があるため、注意が必要である。

7.5.3 書き込み：write

低水準入出力では、ファイルへの書き込みは `write` 関数を用いて行う。この関数は、ファイルディスクリプタで指定された対象に対して、指定したバイト数のデータを書き込むためのものである。`write` 関数のプロトタイプは `unistd.h` に定義されており、使用する際にはこのヘッダファイルをインクルードする。

プロトタイプ: `ssize_t write( int fd, const void *buf, size_t count );`

第1引数 `fd` には、書き込み先を表すファイルディスクリプタを指定する。この値は、`open` 関数などにより取得したものである。

第2引数 `buf` には、書き込むデータが格納されたメモリ領域の先頭アドレスを指定する。この引数は `const void *` 型であるため、任意の型のデータが格納されたメモリ領域を指定することができる。

第3引数 `count` には、書き込むバイト数を指定する。この値は、書き込みたいデータのサイズ (バイト単位) を指定する必要がある。

`write` 関数は、実際に書き込まれたバイト数を返す。エラーが発生した場合には `-1` を返す。このとき、正常に処理された場合でも、戻り値が `count` より小さくなることもある。この場合は一部のデータだけが書き込まれており、残りのデータについて再度 `write` を行う必要がある。

例えば、文字列をファイルに書き込む場合は、次のようにする。

例. 文字列の書き込み

```
int fd;
char msg[] = "Hello\n";
fd = open("data.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, msg, 6);
```

注意) ここでは `open` が成功したものとして、`write` の使用方法だけを示している。

この例では、文字列 `"Hello\n"` をファイル `data.txt` に書き込んでいる。第3引数には、書き込むバイト数 (この場合は6バイト) を指定している。

⁶³ ファイルのアクセス権 (パーミッション) の詳しい仕組みについては本書では解説しない。詳しくは OS のファイルパーミッションや `chmod` コマンドに関する資料や教材を参照のこと。

標準 I/O の `fprintf` や `fputs` などとは異なり、`write` は書式変換などを行わず、指定されたバイト数のデータをそのまま書き込む。そのため、文字列を書き込む際には、書き込むバイト数を明示的に指定する必要がある。文字列の長さを求めるには `strlen` などを用いることができる。

例. `strlen` を用いた書き込み

```
write(fd, msg, strlen(msg));
```

このように、`write` 関数では、メモリ上のデータをそのまま出力することができる。このため、文字列だけでなく、配列や構造体などが格納されたメモリ領域も書き込むことができる。ただし、構造体をそのままファイルに保存する場合には、型のサイズやパディング、エンディアンなどに注意が必要である。

`write` 関数は、ファイルだけでなく、標準出力や標準エラー出力に対しても使用することができる。例えば、標準出力に文字列を出力する場合には、次のようにする。

例. 標準出力への書き込み

```
write(1, "Hello\n", 6);
```

ここで、`1` は標準出力を表すファイルディスクリプタである。このように、低水準入出力では、ファイルだけでなく、様々な入出力対象を同じ方法で扱うことができる。

7.5.3.1 出力の確定処理

標準 I/O では、出力は内部バッファに蓄えられ、必要に応じてまとめて書き込まれる。このため、標準 I/O 側のバッファに残っている出力を送り出すには `fflush` などを用いる必要があった。一方、低水準入出力では `write` 関数を用いるため、このようなバッファリングは行われないように見える。しかし、実際には `write` を呼び出しただけでは、出力が直ちにディスクに書き込まれるとは限らない点に注意が必要である。

`write` により書き込まれたデータは、まず OS のカーネル内部のバッファに格納される。このため、`write` が成功した場合でも、まだディスクへの書き込みが完了していない可能性がある。例えば、この状態でシステムが異常停止した場合や電源が失われた場合には、データが失われる可能性がある。

このような場合に、カーネル内部のバッファの内容を実際にディスクへ書き込むための関数として `fsync` が用意されている。この関数のプロトタイプは `unistd.h` に定義されている。

プロトタイプ: `int fsync( int fd );`

第 1 引数 `fd` には、対象となるファイルディスクリプタを指定する。この関数は、対象ファイルについて OS 内部に保持されている変更内容を記憶装置へ反映するよう要求し、その処理が完了するまで待つ。処理が成功した場合は `0` を返し、失敗した場合には `-1` を返す。

例えば、確実にデータを書き込みたい場合は、次のようにする。

例. 出力の確定

```
write(fd, buf, size);
fsync(fd);
```

このように `fsync` は、`write` によって書き込まれた内容を記憶装置へ同期させるための「出力の確定処理」として用いることができる。ただし、`fsync` は実際のディスク書き込みを伴うため、処理速度が低下する可能性がある。このため、通常の用途では必ずしも必要ではなく、ログファイルや設定ファイルなどの保存を確定する必要がある場合などに使用することが多い。

7.5.4 読み込み: read

低水準入出力では、ファイルからの読み込みは `read` 関数を用いて行う。この関数は、ファイルディスクリプタで指定された対象から、指定したバイト数のデータを読み込み、メモリ上の領域に格納するためのものである。`read` 関数のプロトタイプは `unistd.h` に定義されており、使用する際にはこのヘッダファイルをインクルードする。

プロトタイプ: `ssize_t read( int fd, void *buf, size_t count );`

第 1 引数 `fd` には、読み込み元を表すファイルディスクリプタを指定する。この値は、`open` 関数などにより取得したものである。

第2引数 `buf` には、読み込んだデータを格納するメモリ領域の先頭アドレスを指定する。この領域には、あらかじめ十分なサイズのメモリを確保しておく必要がある。

第3引数 `count` には、読み込む最大バイト数を指定する。`read` 関数は、この値を上限としてデータを読み込む。

`read` 関数は、実際に読み込まれたバイト数を返す。なお、`read` は常に `count` バイトを読み込むとは限らず、それより少ないバイト数を返すこと（読み込み途中でファイル終端に達した場合や、端末やパイプなどから読み込む場合など）もある。

ファイル終端に達し、それ以上読み込むデータがない場合には 0 を返す。エラーが発生した場合には -1 を返す。

例えば、ファイルからデータを読み込む場合は、次のようにする。

例. ファイルからの読み込み

```
int fd;
char buf[100];
ssize_t n;
fd = open("data.txt", O_RDONLY);
n = read(fd, buf, sizeof(buf));
```

注意) ここでは `open` が成功したものとして、`read` の使用方法だけを示している。

この例では、ファイル `data.txt` から最大 100 バイトのデータを読み込み、配列 `buf` に格納している。読み込まれたバイト数は変数 `n` に格納される。この値を用いることで、実際に読み込まれたデータ量を知ることができる。

標準 I/O の `fgets` や `fscanf` などとは異なり、`read` はデータの内容を解釈せず、指定した最大バイト数までのデータをそのまま読み込む。このため、文字列として扱う場合には、終端文字（`'\0'`）が自動的に付加されない点に注意が必要である。例えば、文字列として扱う場合は、次のように終端文字を追加する。

例. 文字列として扱う場合

```
n = read(fd, buf, sizeof(buf)-1);
if (n > 0) buf[n] = '\0';
```

このように、`read` 関数では、読み込んだデータの扱いをプログラム側で制御する必要がある。この点が、標準 I/O との大きな違いの一つである。

また、`read` 関数は、ファイルだけでなく標準入力からの読み込みにも使用することができる。例えば、標準入力からデータを読み込む場合は、次のようにする。

例. 標準入力からの読み込み

```
read(0, buf, sizeof(buf));
```

ここで、0 は標準入力を表すファイルディスクリプタである。このように、低水準入出力では、様々な入力対象を同じ方法で扱うことができる。

7.5.5 ファイルのクローズ：close

低水準入出力では、ファイルの使用が終了した場合、`close` 関数を用いてファイルをクローズする必要がある。この関数は、ファイルディスクリプタに対応するファイルを閉じ、使用していた資源を解放するためのものである。`close` 関数のプロトタイプは `unistd.h` に定義されており、使用する際にはこのヘッダファイルをインクルードする。

プロトタイプ： `int close( int fd );`

第1引数 `fd` には、クローズするファイルディスクリプタを指定する。この値は、`open` 関数により取得したものである。

`close` 関数は、処理が成功した場合には 0 を返し、失敗した場合には -1 を返す。

例えば、ファイルの読み込みを行った後にクローズする場合は、次のようにする。

例. ファイルのクローズ

```
int fd;
char buf[100];
fd = open("data.txt", O_RDONLY);
read(fd, buf, sizeof(buf));
close(fd);
```

注意) ここでは `open`, `read` が成功したものとして、例示を簡略化している。

このように、ファイルの操作が終了したら、`close` によりファイルを閉じる。

ファイルをクローズしないままプログラムを終了した場合でも、多くの環境では OS により自動的にクローズされる。しかし、クローズされないままファイルを開き続けると、使用可能なファイルディスクリプタの数が上限に達してしまい、新たにファイルを開けなくなる可能性がある。このため、ファイルの使用が終了した場合には `close` を呼び出すべきである。

また、`close` によってファイルディスクリプタは閉じられるが、それだけで書き込んだデータが記憶装置へ確実に同期されたことが保証されるわけではない。保存内容を記憶装置へ同期させる必要がある場合には、前節で説明した `fsync` を `close` の前に用いる。

7.5.6 入出力の効率化

低水準入出力では、`read` や `write` を用いて直接データの入出力を行うことができるが、これらの関数を細かい単位で繰り返し呼び出すと、処理効率が低下する場合がある。例えば、1 文字ずつ `write` を呼び出して出力するような処理は、非常に非効率となる。これは、`read` や `write` を細かく呼び出すたびに、OS との間で入出力処理を行うためのオーバーヘッドが生じるためである。

このため、一般には、ある程度のサイズのデータをまとめて入出力するように、**バッファリング**を行うことが望ましい。例えば、数 KiB 以上のバッファ⁶⁴ を用意し、このバッファにデータを蓄えてからまとめて `write` を呼び出すことで、細かく何度も呼び出す場合に比べて効率を高めることができる。適切なバッファサイズは、OS や処理内容などによって異なる。

このようなバッファリングは、標準 I/O においては自動的に行われている。例えば、`fprintf` や `fwrite` などの関数では、内部にバッファが用意されており、一定量のデータが蓄えられた段階でまとめて出力される。このため、通常の用途では、低水準入出力を直接使用して自分でバッファリングを実装するよりも、標準 I/O を利用する方が手軽であり、効率の良い入出力を実現しやすい。また、プログラムの記述も簡潔になる。

したがって、特別な理由がない限り、入出力処理には標準 I/O を用いることが望ましい。低水準入出力は、より細かな制御が必要な場合や、ファイルディスクリプタを直接扱う必要がある場合などに使用されることが多い。例えば、後に説明するシグナル処理では、シグナルハンドラ内で安全に使用できる関数に制限があり、標準 I/O 関数を使用できない場合がある。このような場面では、`write` などの低水準入出力機能が用いられることがある。

また、パイプをプログラム内で扱ったり、ファイルディスクリプタの複製や付け替えなどを行ったりする場合にも、低水準入出力の知識が役立つ。このように、通常は標準 I/O を用いるのが一般的であるが、低水準入出力は、より柔軟で高度な入出力制御を行うための基盤として重要な役割を持っている。

⁶⁴実際には、システムのメモリページサイズに由来する 4096 バイト (4KiB) というサイズが採用されることが多い。

8 分割コンパイルとプリプロセッサ

これまで、本書では1つのソースファイルにすべてのプログラムを記述する形で解説してきた。小規模なプログラムであればこの方法でも問題ないが、実際の開発ではプログラムの規模が大きくなり、単一のソースファイルでは管理が困難になる。

例えば、多数の関数を1つのファイルにまとめて記述すると、プログラムの全体構造を把握することが難しくなり、修正や拡張の際にどの部分に影響が及ぶのか分かりにくくなる。また、複数人で開発する場合にも、1つのファイルに処理が集中していると編集作業が競合しやすくなる。このような問題を避けるため、通常はプログラムを複数のソースファイルに分割して開発する。

このように、プログラムを複数のソースファイルに分割してコンパイルする方法を**分割コンパイル**と呼ぶ。分割コンパイルには、次のような利点がある。

- 規模が大きくなってもプログラムの把握が容易になる
- 機能ごとに分割することでプログラムの構造が明確になる
- 修正したソースファイルのみ再コンパイルできるため開発効率が向上する
- 作成した関数をライブラリとして再利用できる

例えば、数値計算に関する関数をまとめたファイルや、入出力処理をまとめたファイルなど、目的ごとにソースファイルを分割することで、プログラムの見通しが良くなる。また、作成した関数を他のプログラムから利用することも容易になる。

この章では、複数のソースファイルを用いた分割コンパイルの方法と、それに関連して利用されるプリプロセッサの機能について解説する。

8.1 分割コンパイルのための基礎事項

C言語のプログラミングにおいては、使用する関数のプロトタイプ宣言や、「`#define`」による定数の定義などを記述する。これらは多くの場合、ソースプログラムの冒頭部分に記述する。また、複数のソースファイルに分割してプログラムを記述する場合、それらの宣言は複数のソースファイルで共通して必要となることが多い。このような宣言をヘッダファイル`*.h`にまとめて記述しておき、各ソースファイルの冒頭部でインクルードすると便利である。

C言語のプログラム（関数）は複数のソースファイルに分けて作成することができるが、最終的には `main` 関数を含む1つの実行形式ファイルとしてリンクされる。この場合、個々のソースファイルを個別にコンパイルし、その後でそれらを結合して1つの実行形式ファイルを作成する処理（リンク処理）を行うことになる。これまでに解説した例では、`gcc` コマンドで一度に実行形式ファイル（翻訳後の機械語プログラムを保持する）を作成していたが、そこでは、各関数のコンパイル（翻訳）とリンク（結合）が立て続けに行われていた。分割コンパイルでは、翻訳とリンクを明示的に分離した処理として実行する。またこの場合、各ソースファイルを別々にコンパイルするため、他のソースファイルで定義されている関数を使用する場合には、その関数のプロトタイプ宣言が必要になる。このような共通の宣言は、ヘッダファイルにまとめて記述する。

`gcc` では、各ソースファイルを個別に翻訳するには、オプション「`-c`」を付ける。

コマンド： `gcc -c ソースファイル`

これにより、ソースプログラムを翻訳した結果として、機械語コードやリンクに必要な情報などを保持する**オブジェクトファイル**⁶⁵が作成される。オブジェクトファイルは多くの場合、ファイル名の拡張子が「`*.o`」となる。オブジェクトファイルは実行可能な形式ではない。すなわち、必要なオブジェクトファイル群をリンク（結合）して、実行形式ファイルを作成する。複数のオブジェクトファイルをリンクし、1つの実行形式ファイルを作成する処理も「`gcc`」コマンドで行うことができる。

コマンド： `gcc オブジェクトファイル1 オブジェクトファイル2 …`

この処理で、複数のオブジェクトファイルがリンクされる。この場合、作成する実行形式ファイルを「`-o`」オプションで指定することができる。

⁶⁵オブジェクト指向プログラミング（OOP）における「オブジェクト」とは全く別の範疇の用語なので注意すること。

▲重要▲

分割コンパイルでは、各ソースファイルを個別にコンパイルしてオブジェクトファイルを作成する。このため、あるソースファイルに修正が発生した場合、すべてのソースファイルを再コンパイルする必要はなく、当該ソースファイルのみを再コンパイルすれば良い場合が多い。ただし、共通して使用しているヘッダファイルを変更した場合には、そのヘッダファイルをインクルードしている複数のソースファイルを再コンパイルする必要が生じることがある。

このような仕組みにより、大規模なプログラムでも効率的に開発を進めることができる。

1つのプログラムを複数のソースファイルに分割して作成する方法を、例を挙げて解説する。次に示す例は、ヘッダファイル `calc.h`、関数定義ファイル `calc.c`、主たる（main 関数を含む）ソースファイル `main.c` の3つから成る。この内、具体的に関数を記述しているのは `calc.c`、`main.c` の2つであり、これらを個別にコンパイルする処理を示す。

ヘッダファイル：calc.h

```
1 int add(int a, int b);
2 int sub(int a, int b);
```

プログラム：calc.c

```
1 #include "calc.h"
2
3 int add(int a, int b) {
4     return a + b;
5 }
6
7 int sub(int a, int b) {
8     return a - b;
9 }
```

プログラム：main.c

```
1 #include <stdio.h>
2 #include "calc.h"
3
4 int main( ) {
5     int x = 10, y = 5;
6
7     printf("add(%d,%d) = %d\n", x,y,
8         add(x, y));
9     printf("sub(%d,%d) = %d\n", x,y,
10        sub(x, y));
11 }
```

`calc.c`、`main.c` 共に、共通のヘッダファイル `calc.h` を読み込んで使用している。

`calc.c`、`main.c` を `gcc` コマンドでコンパイルする例を次に示す。

例. `calc.c` のコンパイル

```
$ gcc -c calc.c  ←コンパイル（リンクせず）
$ ls -l calc.o  ←オブジェクトファイルの確認
-rw-r--r-- 1 katsu admin 806  3月 26 14:16 calc.o ←オブジェクトファイルができている
```

例. `main.c` のコンパイル

```
$ gcc -c main.c  ←コンパイル（リンクせず）
$ ls -l main.o  ←オブジェクトファイルの確認
-rw-r--r-- 1 katsu admin 1085  3月 26 14:21 main.o ←オブジェクトファイルができている
```

このようにして出来たオブジェクトファイルを結合（リンク）して、実行形式ファイルを作る。次の例は、上で作成したオブジェクトファイルをリンクして `app1` という実行形式ファイルを作るものである。

例. オブジェクトファイルをリンクして実行形式ファイルを作る（先の例の続き）

```
$ gcc calc.o main.o -o app1  ←オブジェクトファイル群をリンクして app1 にする
$ ./app1  ←実行
add(10,5) = 15          ←実行結果
sub(10,5) = 5
```

実行形式ファイルが作成され、実行できている様子がわかる。

参考) 今回のサンプルプログラム `calc.c` では、自身が定義する関数のプロトタイプをヘッダファイルとして読み込んでいる。これは厳密には必要ではないが、C 言語ではこれを習慣としている。このようにしておくことで、ヘッダファイルに記述されたプロトタイプ宣言と関数定義の不一致をコンパイル時に検出することができる。

参考) 次のようにして、必要なソースファイルを一度に全てコンパイルし、リンクして実行形式ファイルを作成することも可能である。

コマンド： `gcc ソースファイル1 ソースファイル2 … -o 実行形式ファイル`

8.1.1 extern による共有

分割コンパイルを行う場合、複数のソースファイルに渡って変数や関数を共有したい場合がある。このようなときに用いられるのが `extern` 宣言である。`extern` は、変数や関数の実体をここでは定義せず、外部に定義されているものとして宣言するためのキーワードであり、分割コンパイルでは、特に他のソースファイルで定義されている変数などを参照するために用いられる。

注意) 関数定義はデフォルトで `extern` として扱われ、他のソースファイルからも参照可能である⁶⁶。一方、グローバル変数を他のソースファイルから参照する場合は、参照側のソースファイルにおいて `extern` 宣言を行う必要がある。

逆に、他のソースファイルへの共有を制限したい場合には `static` を用いる。これについては次節で解説する。

次に示す3つのファイル `test011-1.h`、`test011-1_sub.c`、`test011-1_main.c` は、グローバル変数を共有するプログラムの例である。

ヘッダファイル：test011-1.h

```
1 void f( void );
```

プログラム：test011-1_sub.c

```
1 #include <stdio.h>
2 #include "test011-1.h"
3
4 //int g; // 誤った書き方
5 extern int g; // グローバル変数(参照)
6
7 void f(void) {
8     printf("[g] : g=%d\n", g);
9 }
```

プログラム：test011-1_main.c

```
1 #include <stdio.h>
2 #include "test011-1.h"
3
4 int g = 123; // グローバル変数
5
6 int main() {
7     printf("[main]: g=%d\n", g);
8     f();
9 }
```

`test011-1_sub.c`、`test011-1_main.c` 共に、共通のヘッダファイル `test011-1.h` を読み込んで使用している。

まず、`test011-1_main.c` では、グローバル変数 `g` を定義している。

次に、`test011-1_sub.c` では、`extern` を用いてグローバル変数 `g` を外部に定義されている変数として宣言している。このように `extern` を用いることで、別のソースファイルで定義された変数を共有して利用することができる。上のプログラムをコンパイル、リンクして実行する例を次に示す。

```
$ gcc -c test011-1_main.c test011-1_sub.c [Enter] ←コンパイル（リンクせず）
$ gcc test011-1_main.o test011-1_sub.o -o test011-1 [Enter] ←オブジェクトファイル群をリンク
$ ./test011-1 [Enter] ←実行
[main]: g=123      ← test011-1_main.c で定義された g が
[g] : g=123        ← test011-1_sub.c 側で参照できている
```

この結果から、`test011-1_main.c` で定義されたグローバル変数 `g` が、`test011-1_sub.c` から共有されていることがわかる。ここで重要なのは、グローバル変数の定義は1か所だけにすることが必要であるという点である。上の例では、グローバル変数 `g` は `test011-1_main.c` でのみ定義されており、`test011-1_sub.c` では `extern` によって参照しているだけである。

このように、複数のソースファイルに渡って変数を共有する場合には、`extern` 宣言を用いる。

ここで、`test011-1_sub.c` の次の部分

```
//int g; // 誤った書き方
```

のコメント「`//`」を外して

```
int g; // 誤った書き方
```

として有効にし、次の行

```
extern int g; // グローバル変数(参照)
```

の先頭に「`//`」を挿入して

```
//extern int g; // グローバル変数(参照)
```

⁶⁶より正確には、通常の関数は外部リンケージを持つことがその理由である。

としてコメントにすると、test011-1.sub.c に記述した `int g;` は単なる参照のための宣言ではなく、`g` の**仮定義**となる。このため、test011-1.main.c にある `g` の定義と重複することになり、現在の GCC の通常の設定ではリンク処理の段階で多重定義のエラーが発生する。

まとめ) 複数のソースファイルに渡って共有するグローバル変数は

- ・ 1 か所のみで定義する
- ・ 他のソースファイルでは `extern` を用いる

というルールに従う必要がある。

重要) 関数自体の定義には、分割コンパイルの場合においても `extern` を付ける必要はない。通常関数は外部リンケージを持つため、他のソースファイルから参照可能であり、通常は `extern` を明示的に記述しない。

参考) 実際のプログラムでは、複数のソースファイルから共通して利用する `extern` 宣言は、ヘッダファイルに記述して各ソースファイルからインクルードする方法が一般的である。本例では、`int g;` と `extern int g;` の違いを明確に示すため、test011-1.sub.c に `extern` 宣言を直接記述している。

8.1.2 static 宣言による共有範囲の制限

前節では、`extern` 宣言により、複数のソースファイルに渡って変数や関数を共有できることを説明した。一方で、あるソースファイルの内部だけで使用する変数や関数は、他のソースファイルから参照できないように制限したい場合がある。このようなときに用いられるのが `static` 宣言である。

`static` をファイルスコープのグローバル変数や関数の定義に付けると、その識別子は**内部リンケージ**を持ち、同じソースファイル(より正確には同一の翻訳単位)の内部からのみ参照できるようになる。このため、他のソースファイルからは参照できなくなる。このように `static` は、識別子の共有範囲を制限するために用いられる。

次に示す例は、test011-3.h, test011-3.sub.c, test011-3.main.c の 3 つのファイルから成るプログラムである。

ヘッダファイル: test011-3.h

```
1 void f( void );
```

プログラム: test011-3.sub.c

```
1 #include <stdio.h>
2 #include "test011-3.h"
3
4 //static void f( void ) { // 共有禁止
5 void f( void ) {
6     printf("message from f\n");
7 }
```

プログラム: test011-3.main.c

```
1 #include <stdio.h>
2 #include "test011-3.h"
3
4 int main() {
5     f();
6 }
```

test011-3.sub.c, test011-3.main.c 共に、共通のヘッダファイル test011-3.h を読み込んで使用している。

これらをコンパイル、リンクして実行する例を次に示す。

例. コンパイル、リンクして実行

```
$ gcc test011-3_main.c test011-3_sub.c -o test011-3  ←コンパイルとリンク
$ ./test011-3  ←実行
message from f ←関数 f からの出力
```

この例では、test011-3.sub.c において関数 `f` を定義しているが、4 行目先頭のコメント表記「`//`」を外し、5 行目先頭に「`//`」を挿入すると、関数 `f` は `static` となり、他のソースからは参照できなくなる。その状態でコンパイルすると、ヘッダファイルでは通常の外部リンケージを持つ関数として宣言されている `f` を、`static` によって内部リンケージを持つ関数として定義しようとすることになり、宣言と定義のリンケージが矛盾するためエラーとなる。

このように `static` を用いることで、関数やグローバル変数の共有範囲をソースファイル内に限定できるので、次のようなことが可能になる。

- ・ 他のソースファイルとの名前の衝突を防ぐ
- ・ 内部処理用の関数を隠蔽できる
- ・ プログラムの構造を明確にできる

次に、`static` 宣言によって、同じ名前の関数が複数のファイルで定義できる様子を示す。

ヘッダファイル：test011-4.h

```
1 void f_sub( void );
```

プログラム：test011-4_sub.c

```
1 #include <stdio.h>
2 #include "test011-4.h"
3
4 static void f(void) { //このソース限定
5     printf("[sub]:\tmessage from f\n");
6 }
7
8 void f_sub( void ) {
9     f(); // このソースのf
10 }
```

プログラム：test011-4_main.c

```
1 #include <stdio.h>
2 #include "test011-4.h"
3
4 static void f(void) { //このソース限定
5     printf("[main]:\tmessage from f\n");
6 }
7
8 int main() {
9     f(); // このソースの関数
10    f_sub(); // 他のソースの関数
11 }
```

test011-4_sub.c, test011-4_main.c 共に、共通のヘッダファイル test011-4.h を読み込んで使用している。

test011-4_sub.c, test011-4_main.c の両方で同じ名前の関数 `f` が定義されているが、それぞれ別の関数として機能する。(なお、内部処理専用の関数 `f` は他のソースファイルから利用しないため、ヘッダファイルには宣言していない) そのことが次の実行例で確認できる。

例. コンパイル, リンクして実行

```
$ gcc test011-4_main.c test011-4_sub.c -o test011-4 [Enter] ←コンパイルとリンク
$ ./test011-4 [Enter] ←実行
[main]: message from f ← test011-4_main.c の f による出力
[sub]: message from f ← test011-4_sub.c の f による出力
```

なお, `static` は関数だけでなく, グローバル変数に対しても使用できる。この場合も, その変数は内部リンケージを持ち, 定義されたソースファイル内からのみ参照可能となる。

8.2 プリプロセッサ

C 言語では, コンパイル (翻訳) を行う前に, **プリプロセッサ**と呼ばれる処理が実行される。この処理では, ソースプログラムに対して文字列レベルの置換, ヘッダファイルの読み込み, 条件によるソースの選択などが行われる。

プリプロセッサによる命令 (**プリプロセッサディレクティブ**) は, `#` で始まる形式で記述する。例えば, これまでに使用してきた

```
#include <stdio.h>
```

も, プリプロセッサ命令の 1 つである。

プリプロセッサは主に次のような用途で使われる。

- 定数の定義 (`#define`)
- ヘッダファイルの読み込み (`#include`)
- 条件によるソースコードの切り替え (`#ifdef` など)

以下, それぞれについて解説する。

8.2.1 ヘッダファイルの読み込み (`#include`)

標準ヘッダなど, コンパイラ環境に用意されているヘッダファイルを読み込むには `< ... >` を用いて

```
#include <ヘッダファイル>
```

と書くが, 独自に作成したヘッダファイルなどを読み込む場合には, 通常, 二重引用符を用いて

```
#include "ヘッダファイル"
```

として読み込む。GCC などの一般的な環境では, この形式ではまずソースファイル付近のディレクトリからヘッダファイルを探し, 見つからなければ他のヘッダ検索パスから探す。

8.2.2 定数マクロ (`#define`)

プリプロセッサでは, `#define` を用いて定数マクロを定義することができる。(次の例)

```
#define PI 3.14159
```

このように定義すると, ソースコード中の `PI` は, コンパイル前に `3.14159` に置き換えられる。すなわち, この定義

の後で、

```
double r = 5.0, s;  
s = PI * r * r;
```

と記述すると、コンパイルされる前に、ソースプログラムの該当部分が

```
s = 3.14159 * r * r;
```

と書き換えられる。このように、`#define` はコンパイル前に、ソースプログラム中の該当する記述を別の記述に置き換えるマクロ機能として動作する。

なお、`#define` による定数は、通常、大文字で記述する習慣がある。また、`#define` による定数は型を持たない。

8.2.2.1 条件コンパイルへの応用

定数マクロは、条件コンパイルと組み合わせて用いることで、プログラムの内容を切り替える用途にも利用することができる。このような方法を **条件コンパイル** と呼ぶ。

条件コンパイルでは、`#if`、`#ifdef`、`#ifndef` などのプリプロセッサ命令を用いる。これらは、コンパイル前の段階で条件判定を行い、条件に応じてソースコードの一部を有効または無効にするものである。すなわち、ソースプログラムの中で、コンパイルの対象とする部分を選択することができる。例えば、次のようにマクロを定義する。

```
#define DEBUG
```

このようなマクロ定義では置換する内容を指定しておらず、マクロが定義されているかどうかのみを条件判定に利用することができる。このマクロを用いて、次のように条件コンパイルを行うことができる。

```
#ifdef DEBUG  
    printf("デバッグモードです\n");  
#endif
```

この例では、`DEBUG` が定義されている場合のみ、`printf` 文が有効となる。`DEBUG` の定義を削除すると、この部分のコードはコンパイル対象から除外される。

また、`#if` を用いると、数値定数を用いた条件判定も可能である。例えば、次のようにマクロ定数を定義する。

```
#define VERSION 2
```

これを用いて、次のように条件コンパイルを行うことができる。

```
#if VERSION >= 2  
    printf("新しいバージョンです\n");  
#else  
    printf("旧バージョンです\n");  
#endif
```

このように、条件コンパイルを用いることで、マクロ定数の値や定義の有無に応じて、異なるプログラムを生成することができる。

なお、`#ifndef` は、指定したマクロが定義されていない場合に有効となる。

重要) `#if` を用いて、あるマクロが定義されているかどうかを判定するには `defined` を用いて

```
#if defined(マクロ名)  
と記述する。これは #ifdef マクロ名 と同等の判定である。
```

重要) 条件コンパイルでは連言（かつ）「`&&`」、選言（または）「`||`」、否定「`!`」などの演算子を使用することができる。

条件コンパイルは、デバッグ用コードの切り替えや、環境の違いに応じた処理の分岐などに広く用いられている。例えば、OS やコンパイラの違いに応じて処理を切り替える場合にも有効である。

このように、定数マクロと条件コンパイルを組み合わせることで、柔軟なプログラム構成を実現することができる。

■ サンプルプログラム

ヘッダファイルに定義されているマクロによって異なるプログラムを作成する例を、次の `test011-2.c`、`test011-2.h` で示す。

プログラム：test011-2.c

```
1  #include    <stdio.h>
2  #include    "test011-2.h"
3
4  // マクロ定義によって4種類の異なる関数 f の内1つがコンパイルされる
5  #if defined(FLAG1) && defined(FLAG2)
6  void f(void) {
7      printf("FLAG1, FLAG2の両方が定義されたプログラム\n");
8  }
9  #elif defined(FLAG1)
10 void f(void) {
11     printf("FLAG1のみが定義されたプログラム\n");
12 }
13 #elif defined(FLAG2)
14 void f(void) {
15     printf("FLAG2のみが定義されたプログラム\n");
16 }
17 #else
18 void f(void) {
19     printf("FLAG1, FLAG2のどちらも定義されていないプログラム\n");
20 }
21 #endif
22
23 int main() {
24     f();
25 }
```

このプログラムでは、メッセージを出力する関数 `f` の定義が、条件コンパイルによって4種類用意されている。実際にどの `f` がコンパイルされるかは、読み込むヘッダファイル `test011-2.h` の内容によって異なる。このヘッダファイルは、マクロ `FLAG1`, `FLAG2` の定義を書くもので、その書き方によって異なるプログラムが作成される様子を示す。

[test011-2.h の内容毎に異なる実行結果]

● パターン 0

```
//#define    FLAG1
//#define    FLAG2
```

例. コンパイルして実行

```
$ gcc test011-2.c -o test011-2 
$ ./test011-2  ↓出力
FLAG1, FLAG2のどちらも定義されていないプログラム
```

● パターン 1

```
#define    FLAG1
#define    FLAG2
```

例. コンパイルして実行

```
$ gcc test011-2.c -o test011-2 
$ ./test011-2  ↓出力
FLAG1, FLAG2の両方が定義されたプログラム
```

● パターン 2

```
#define    FLAG1
//#define    FLAG2
```

例. コンパイルして実行

```
$ gcc test011-2.c -o test011-2 
$ ./test011-2  ↓出力
FLAG1のみが定義されたプログラム
```

● パターン 3

```
//#define    FLAG1
#define    FLAG2
```

例. コンパイルして実行

```
$ gcc test011-2.c -o test011-2 
$ ./test011-2  ↓出力
FLAG2のみが定義されたプログラム
```

上記4つのパターンは、プログラムの実行時の条件分岐ではなく、プリプロセッサによる前処理の段階で4種類の関数 `f` のうち1つだけが選択され、その結果がコンパイルされる点が重要である。

8.2.2.2 インクルードガード

条件コンパイルを用いて、マクロの定義に応じて異なるプログラムを生成する方法について解説した。このような条件コンパイルは、ヘッダファイルを扱う際にも重要な役割を果たす。

特に、規模の大きなプログラムを分割コンパイルの形で構築する場合、多数のヘッダファイルを用意することがある。このような状況では、ヘッダファイルを複数回読み込んでしまうことによって、エラーが発生する場合がある。このような問題を防ぐために用いられる方法を **インクルードガード** (include guard) と呼ぶ。

例えば、あるソースファイルが複数のヘッダファイルを読み込み、それらのヘッダファイルがさらに共通のヘッダファイルを読み込んでいる場合などには、同一の翻訳単位の中で、意図せず同じヘッダファイルが複数回読み込まれることがある。このとき、構造体や列挙型などの定義が重複し、コンパイルエラーとなる場合がある。

例えば、先に使用したヘッダファイル `calc.h` (p.95) にインクルードガードを施す場合は、次のように記述する。

```
#ifndef CALC_H
#define CALC_H

int add(int a, int b);
int sub(int a, int b);

#endif
```

最初に `calc.h` が読み込まれたときには `CALC_H` はまだ定義されていないので、`#ifndef` の条件が成立し、ヘッダファイルの内容が有効になる。その際、`#define CALC_H` によってマクロ `CALC_H` が定義される。

同じ翻訳単位の中で `calc.h` が再び読み込まれた場合には、既に `CALC_H` が定義されているため `#ifndef` の条件は成立せず、`#endif` までの内容は読み飛ばされる。

このように、ヘッダファイルが多重に読み込まれる問題を防ぐために、条件コンパイルを用いて、同一の翻訳単位の中ではヘッダファイルの内容が一度だけ有効になるようにする方法が一般的に用いられている。

8.2.2.3 処理環境に関する定数

コンパイラ環境では、処理環境やコンパイラに関する重要なマクロがあらかじめ定義されている。詳細に関しては巻末付録「A 定義済みマクロ」(p.506～)に挙げる。それらを用いると、システム環境に適したコードをコンパイルすることが可能になる。

9 実用的なプログラミングに必要な事柄

9.1 関数の再帰的定義

関数は、その内部で自分自身を呼び出すことができる。このような呼び出しを**再帰呼び出し**と呼び、そのような処理を含む関数を**再帰関数**⁶⁷と呼ぶ。例えば、0以上の整数 n の階乗 $n!$ は次のように定義できる。

$$n! = \begin{cases} n = 0 & \rightarrow 1 \\ n > 0 & \rightarrow n \times (n-1)! \end{cases}$$

この定義では、 $n!$ を求めるために $(n-1)!$ を求める必要があり、さらに $(n-1)!$ を求めるために $(n-2)!$ を求める必要がある。このように、同じ処理を繰り返しながら、問題を小さくしていく方法が再帰的定義の基本的な考え方である。

プログラム：test017-1.c

```
1 #include <stdio.h>
2
3 int fct( int n ) {
4     if ( n <= 0 ) {
5         return 1;
6     } else {
7         return n*fct(n-1);
8     }
9 }
10
11 int main() {
12     printf("10! = %d\n",fct(10));
13 }
```

上の階乗の定義を C 言語の関数 `fct` として実装した例を左のプログラム `test017-1.c` に示す。

例. コンパイルと実行

```
$ gcc test017-1.c -o test017-1  ←コンパイル
$ ./test017-1  ←実行
10! = 3628800 ←結果
```

注意) このプログラムでは簡単のため、 $n \leq 0$ の場合に 1 を返すようにしている。

関数 `fct` では、その内部で再び `fct` を呼び出している。このような関数を **再帰関数** と呼ぶ。

`fct(3)` を呼び出すと、次のような順序で関数が呼び出される。

```
fct(3)
→ 3 * fct(2)
   → 2 * fct(1)
      → 1 * fct(0)
         → 1
```

そして、最も内側の `fct(0)` が 1 を返すと、下記のように順に計算が戻っていく。

```
fct(0) = 1
fct(1) = 1 × 1 = 1
fct(2) = 2 × 1 = 2
fct(3) = 3 × 2 = 6
```

このように、再帰呼び出しでは、

- ・呼び出しが次々に深くなる
- ・条件を満たすと処理が戻ってくる

という流れで計算が行われる。この動作は、関数呼び出しが「積み重なり」、その後で逆順に戻る形となる。

再帰呼び出しを用いる場合には、必ず処理を終了させるための条件（**終了条件**）を用意する必要がある。上の $n!$ の定義では、

$n = 0$ （プログラム中では $n \leq 0$ ）

が終了条件である。この条件がなければ、関数呼び出しが際限なく繰り返され、やがてスタック領域を使い果たしてプログラムが異常終了する可能性がある。

■ 関数の再帰呼び出しを観察するための例

先のプログラムにおける関数 `fct` と同等の計算処理を行うが、`fct` の計算を一時停止して確認する形のプログラムを `test017-2.c` に示す。このプログラムをコンパイルして実行すると、`fct` の実行開始直後と、`return` によって呼び出し元へ戻る直前に、キーボードからの文字列入力力が求められ、ステップ形式で `fct` の再帰呼び出しを観察できる。

⁶⁷関数の再帰的な定義は、**数学的帰納法**とも深い関係があり、小さな場合を基礎として、より大きな場合を段階的に定めていく考え方である。

プログラム：test017-2.c

```

1  #include <stdio.h>
2
3  int fct( int n ) {
4      int r;
5      char dummy[80];
6      printf("start fct(%d)[Enter str]:",n);
7      scanf("%79s",dummy);
8      if ( n <= 0 ) {
9          r = 1;
10         printf("end fct(%d)=%d[Enter str]:",n,r);
11         scanf("%79s",dummy);
12         return r;
13     } else {
14         r = n*fct(n-1);
15         printf("end fct(%d)=%d[Enter str]:",n,r);
16         scanf("%79s",dummy);
17         return r;
18     }
19 }
20
21 int main() {
22     printf("3!=%d\n",fct(3));
23 }

```

例. コンパイルと実行 コンパイル↓

```

$ gcc test017-2.c -o test017-2 [Enter]
$ ./test017-2 [Enter] ←実行
start fct(3)[Enter str]:x [Enter]
start fct(2)[Enter str]:x [Enter]
start fct(1)[Enter str]:x [Enter]
start fct(0)[Enter str]:x [Enter]
end fct(0)=1[Enter str]:x [Enter]
end fct(1)=1[Enter str]:x [Enter]
end fct(2)=2[Enter str]:x [Enter]
end fct(3)=6[Enter str]:x [Enter]
3!=6      ←最終的に得られた結果

```

プログラムを実行すると、関数 fct の実行開始直後と return の直前に文字列入力が必要になる。それに対して何か文字を入力して [Enter] を押下すると次の処理に進む。これで、関数の再帰呼び出しが観察できる。

9.1.1 関数呼び出しにおけるメモリの管理の仕組み

先に、再帰関数呼び出しの例として、関数 fct が自分自身を何度も呼び出す様子を観察した。例えば fct(3) を呼び出すと、次のような順序で関数が呼び出される。

```

fct(3)
→ fct(2)
→ fct(1)
→ fct(0)

```

そして、fct(0) が終了すると、次のように順に処理が戻っていく。

```

fct(0) → return 1
fct(1) → return 1
fct(2) → return 2
fct(3) → return 6

```

ここで重要なことがある。例えば、fct(3) の計算の途中で、fct(2) や fct(1) が呼び出されるが、それぞれの関数の引数 n の値が全て保存されているという点が重要である。すなわち、各関数呼び出しは、独立した実行状態を持っているのである。

例えば、fct(3) の実行中に fct(2) が呼び出されると、n の値は 3 から 2 になってしまうように思われる。さらに、fct(1) や fct(0) が呼び出されると、元々の値は失われてしまうように思える。

しかし実際には、それぞれの関数呼び出しにおける変数の値は、すべて独立して保持されている。すなわち、次のように、それぞれの呼び出しにおける n の値は別々に存在している。

```

fct(3)    n = 3
fct(2)    n = 2
fct(1)    n = 1
fct(0)    n = 0

```

このように、関数が呼び出されるたびに、その関数呼び出し専用のメモリ領域が確保され、関数の状態（引数やローカル変数など）が保持される仕組みになっている。

この仕組みによって、関数の実行が終了すると、呼び出し元の関数に戻り、処理を再開することができる。すなわち、実行環境は関数の呼び出しの順序や、各関数呼び出しの状態を保持しているのである。

このような関数呼び出しの際のメモリ管理は、一般的な処理系では、**スタック**⁶⁸ (stack) と呼ばれる仕組みによって実現されている。次に、このスタックの仕組みについて説明する。

⁶⁸FILO (First-In Last-Out) あるいは、LIFO (Last-In First-Out) と呼ばれる仕組みである。

先のプログラム test017-2.c では、関数 fct は、値を受け取る引数 n、キーボードからの入力用バッファ dummy、計算途中の値を保持するローカル変数 r を作業用の記憶として持つ。これらは関数 fct が実行を開始する際にメモリ上に確保され、関数が return によって実行を終了する際に解放（廃棄）される。

一般的な C 言語処理系では、関数の実行に必要な自動記憶域期間を持つローカル変数や、引数などを保持するための領域が、関数呼び出しの際に**スタック領域**上に確保される。多くの処理系では、スタック領域はメモリアドレスの高い側から低い側へ向かって使用される（この方向は処理系によって異なる）。また、ある関数の実行中に別の関数を呼び出す際は、元の関数の呼び出し状態を保持したまま、呼び出される関数のための新しい記憶域がスタック上に追加される。

以上のような仕組みにより、プログラム test017-2.c の関数 fct が呼び出されると、図 5 のような流れで次々と記憶域が確保され、fct(0) によって再帰呼び出しが終了すると、逆の順でメモリを解放しながら戻り値を返す。

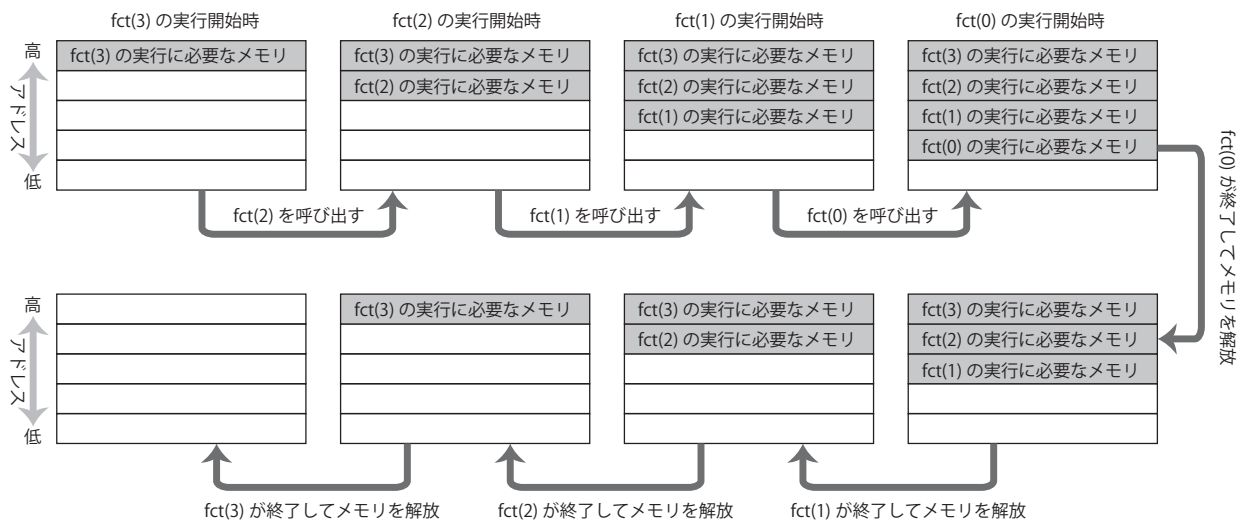


図 5: 関数 fct(3) が呼び出され、終了するまでのメモリの割り当てと解放

再帰呼び出しでは、処理は一番深いところまで進み、その後で逆順に戻ってくる。この「最後に積み上げたものから先に取り出す」という性質が、スタックの LIFO（Last-In First-Out）の仕組みとよく対応している。

このように、関数呼び出しに伴うメモリの管理は、スタック構造によって自然に実現されている。関数の再帰呼び出しは、このスタックの仕組みを最も分かりやすく観察できる例である。

9.1.2 関数の再帰呼び出しに関する問題点

9.1.2.1 スタックのサイズに関すること

関数の呼び出しと実行に用いられるスタック領域の大きさや管理方法は、OS や処理系、実行環境などによって異なる。スタックとして使用できる領域には上限があるため、関数の再帰呼び出しの深さに関しては、プログラムの設計の段階で注意する必要がある。

次に示すプログラム test017-3.c は、再帰呼び出しの深さの限界を試すことにより、スタック領域の制限が再帰呼び出しに及ぼす影響を示すものである。このプログラムで定義されている関数 count は、与えられた整数値が 0 になるまでに必要なデクリメント回数を調べるもの⁶⁹であり、自身の再帰呼び出しによってそれを実現する。

プログラム：test017-3.c

```

1  #include    <stdio.h>
2
3  //  nが0になるまでデクリメントできる回数を、再帰呼び出しで調べる関数
4  int count( int n ) {
5      if ( n <= 0 ) {
6          printf("reached to 0.\n");
7          return 0;
8      } else {
9          return 1+count(n-1);

```

⁶⁹この関数は、スタックの限界を調べるためにのみ考案されたものであり、実用上の合理性に反していることに留意されたい。

```

10     }
11 }
12
13 int main() {
14     int    n;
15
16     while ( 1 ) {
17         printf("input positive integer(0 or negative to exit): ");
18         scanf("%d",&n);          // 整数値の入力を受け付ける
19         if ( n < 1 ) break;       // 0以下なら終了
20         printf("count(%d):%d\n", n, count(n)); // count関数を呼び出す
21     }
22 }

```

注意) scanf で数値が読み取れなかった場合のチェックは省略している。

このプログラムでは、標準入力から与えた整数値を関数 count に与え、その実行を試みる反復処理を行う。このプログラムをコンパイルして実行する例を次に示す。

例. test017-3.c のコンパイルと実行

```

$ gcc test017-3.c -o test017-3  ←コンパイル
$ ./test017-3  ←実行
input positive integer(0 or negative to exit): 30000  ← 30,000 のデクリメントは
reached to 0.          ←再帰呼び出しで実行することが可能である
count(30000):30000     ←結果
input positive integer(0 or negative to exit): 40000  ← 40,000 のデクリメントも
reached to 0.          ←再帰呼び出しで実行することが可能である
count(40000):40000     ←結果
input positive integer(0 or negative to exit): 50000  ←しかし 50,000 のデクリメントは
$          ←再帰呼び出しで実行することができない (異常終了してシェルに戻った)

```

この実行例では、関数 count の約 4 万回程度の再帰呼び出しまでは実行可能であったが、5 万回程度の再帰呼び出しは失敗して、プログラムが強制的に終了（異常終了）していること⁷⁰ がわかる。これはスタックのサイズの限界を超えてメモリを確保しようとしたことによるエラー（スタックオーバーフロー）（stack overflow）である。

この試みによって、関数の再帰的呼び出しの深さには、スタックサイズによる限界があることがわかる。また、単純な再帰呼び出しではなくても、関数から別の関数を呼び出す深さにも、同様の事情による限界がある。

重要) 関数呼び出しの深さが大きくなる場合には、スタック領域の制限に注意する必要がある。

プログラムが使用するスタックに関しては、後の「9.9 プログラム実行のためのメモリモデル」(p.140)でも更に解説する。

9.1.2.2 計算規模に関すること

関数の再帰的な実装方法によっては、計算量が急激に増大することがある。次の例はフィボナッチ数 F_n を再帰的に定義したものである。

$$F_n = \begin{cases} n = 0 & \rightarrow 0 \\ n = 1 & \rightarrow 1 \\ n > 1 & \rightarrow F_{n-1} + F_{n-2} \end{cases}$$

これは C 言語のプログラムとして素朴に実装することができるが、再帰的に定義された関数の、実行時の計算規模の増大が問題となる例を示す。

次のプログラム test017-4.c は上の定義を実装した関数 fib を用いてフィボナッチ数列を作成するものである。

プログラム：test017-4.c

```

1  #include    <stdio.h>
2
3  unsigned long long fib( int n ) {
4      if ( n < 1 ) {
5          return 0;

```

⁷⁰ 再帰呼び出しの実際の深さやスタック使用量は、コンパイラの最適化方法によっても変化する場合がある。本例の結果は、使用する処理系やコンパイル条件によって異なる。具体的な限界は、環境ごとに異なるので試されたい。

```

6      } else if ( n == 1 ) {
7          return 1;
8      } else {
9          return fib(n-1) + fib(n-2);
10     }
11 }
12
13 int main() {
14     int    n, c;
15
16     while ( 1 ) {
17         printf("enter integer(negative to exit): ");
18         scanf("%d",&n);
19         if ( n < 0 ) return 0;
20         printf("Fibonacci sequence:\n");
21         for ( c = 0; c <= n; c++ ) {
22             printf("%llu(%d), ", fib(c),c);
23         }
24         printf("\n");
25     }
26 }

```

このプログラムを実行すると整数値の入力が促され、入力された整数値 n に対するフィボナッチ数列

$$F_0, F_1, \dots, F_n$$

を作成する。(負の値を入力するとプログラムは終了する)

このプログラムをコンパイルして実行する例を次に示す。

例. test017-4.c のコンパイルと実行

\$ gcc test017-4.c -o test017-4 ←コンパイル

\$./test017-4 ←実行

enter integer(negative to exit): 10 ←整数値 10 を入力

Fibonacci sequence:

0(0), 1(1), 1(2), 2(3), 3(4), 5(5), 8(6), 13(7), 21(8), 34(9), 55(10), ← $F_0 \dots F_{10}$

enter integer(negative to exit): 60 ←整数値 60 を入力

Fibonacci sequence:

0(0), 1(1), 1(2), 2(3), 3(4), 5(5), 8(6), 13(7), 21(8), 34(9), 55(10), 89(11), 144(12),
 233(13), 377(14), 610(15), 987(16), 1597(17), 2584(18), 4181(19), 6765(20), 10946(21),
 17711(22), 28657(23), 46368(24), 75025(25), 121393(26), 196418(27), 317811(28), 514229(29),
 832040(30), 1346269(31), 2178309(32), 3524578(33), 5702887(34), 9227465(35), 14930352(36),
 24157817(37), 39088169(38), 63245986(39), 102334155(40), 165580141(41), 267914296(42),
 433494437(43), 701408733(44), 1134903170(45), ... + で中断

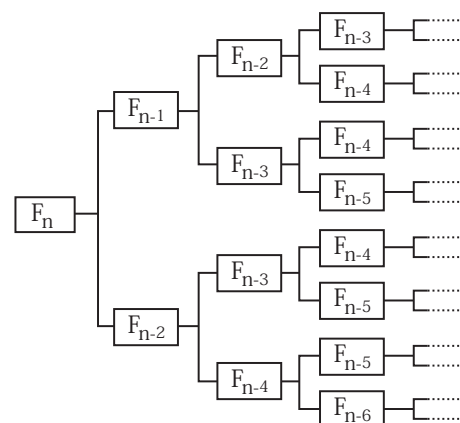
上のように実行すると、 F_{40} を超えたあたりから出力にかかる時間が急激に増大する。この例では、計算の終了を待たずに

+

でプログラムを強制終了している。

F_n の実装は単純であるが、これを算出するために F_{n-1} と F_{n-2} の計算が必要になる。このように、再帰呼び出しが枝分かれして多数発生するため、計算量が急激に増大する。

右の図は p.105 の F_n の定義に従って実際に計算の過程を展開したものである。なお、この例では再帰呼び出しの最大の深さは n に比例する程度であるが、関数呼び出しの総数は指数関数的に増加する。従って、 n が少し大きくなるだけで計算時間が急激に大きくなる。



フィボナッチ数 F_n の再帰呼び出しの展開例
 (同じ計算が何度も繰り返されることに注意)

F_n は定義が単純であり、プログラムとして実装する際には、関数の再帰的呼び出しに依らない形にすることも容易である。これによって、上の例で見た計算量の急激な増大の問題を解決することができる。

次に示す関数 fib は、再帰的呼び出しに依らない実装であり、プログラム test017-4.c の関数 fib を置き換えるものである。

関数 fib : test017-4.c の fib の定義の置き換え

```

1 unsigned long long fib( int n ) {
2     unsigned long long  a = 0, b = 1, r;
3     int                c;
4
5     if ( n < 1 ) {
6         return a;
7     } else if ( n == 1 ) {
8         return b;
9     }
10
11     for ( c = 2; c <= n; c++ ) {
12         r = a + b;
13         a = b;
14         b = r;
15     }
16     return r;
17 }

```

関数 fib をこの形にして、再度コンパイル、実行する例を次に示す。

例. test017-4.c の fib を変更してコンパイル、実行

```

$ gcc test017-4.c -o test017-4  ←コンパイル
$ ./test017-4  ←実行
enter integer(negative to exit): 10  ←整数値 10 を入力
Fibonacci sequence:
0(0), 1(1), 1(2), 2(3), 3(4), 5(5), 8(6), 13(7), 21(8), 34(9), 55(10),      ←  $F_0 \cdots F_{10}$ 
enter integer(negative to exit): 60  ←整数値 60 を入力
Fibonacci sequence:
0(0), 1(1), 1(2), 2(3), 3(4), 5(5), 8(6), 13(7), 21(8), 34(9), 55(10), 89(11), 144(12),
233(13), 377(14), 610(15), 987(16), 1597(17), 2584(18), 4181(19), 6765(20), 10946(21),
⋮
(途中省略)
⋮
86267571272(54), 139583862445(55), 225851433717(56), 365435296162(57), 591286729879(58),
956722026041(59), 1548008755920(60),      ←  $F_0 \cdots F_{60}$  全て計算できた
enter integer(negative to exit): -1  ←負の値を入力
$      ←プログラムが正常に終了してシェルに戻った

```

先の例（関数 fib を書き換える前）に比べて、格段に短い時間で計算が終了することがわかる。（試されたい）

後の方の関数 fib では、1 回の呼び出しにおいてフィボナッチ数を先頭から順に生成する（図 6）ため、素朴な再帰版のような指数関数的な計算量の増大は起こらない。関数 fib の 1 回の呼び出しに要する計算量は $O(n)$ (n に比例) である。

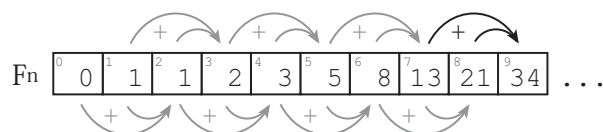


図 6: 反復処理による 1 つのフィボナッチ数の計算

重要) 関数の再帰呼び出しを用いる場合には、同じ計算が何度も繰り返されていないかを確認し、必要に応じて for や while による反復処理など、より効率的な実装方法を検討すべきである。

9.1.3 複雑な問題を再帰的関数で解決する例

前節では、再帰呼び出しはスタックの消費や計算規模の増大といった問題を引き起こす可能性があることを説明した。このため、再帰呼び出しは注意して使用する必要がある。しかし、再帰呼び出しは単に危険なだけの手法ではない。問題の中に、元の問題と同じ構造を持つ小さな問題が含まれている場合には、再帰呼び出しを用いることで、複雑な問題を非常に簡潔に記述することができる。ここではその例としてハノイの塔⁷¹を取り上げる。

⁷¹フランスの数学者 エドゥアール・リュカ (Édouard Lucas) によって 1883 年に考案された数学パズルである。

■ ハノイの塔

ハノイの塔 (Tower of Hanoi) は、再帰的アルゴリズムの代表例として広く知られている数学パズルである。この問題の設定は次の通り。

- ・ 3 本の棒 (塔) がある
- ・ 一度に動かせる円盤は 1 枚のみ
- ・ 大きな円盤を小さな円盤の上に置くことはできない
- ・ 円盤を棒以外の場所 (床など) に置くことはできない

初期状態として、 $n, n-1, \dots, 1$ の大きさの円盤が、大きいものほど下になるように 1 つの棒 (塔) に積み上げられている。

上の条件のもとで、すべての円盤を別の棒へ移動することが課題である。

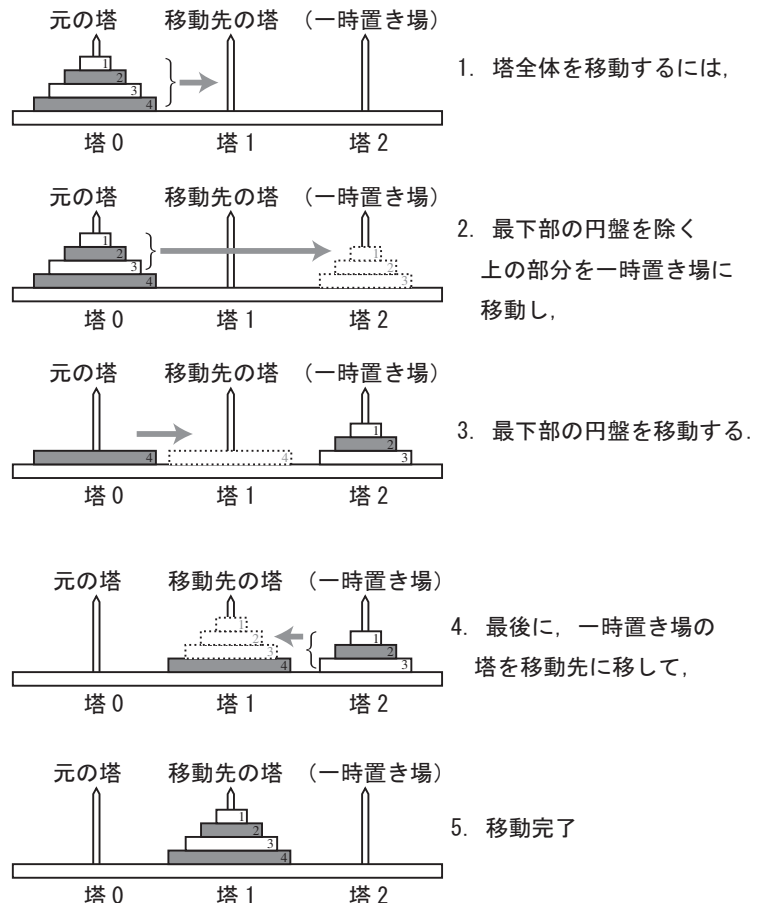
この問題を解くためには、塔全体を一度に移動しようとする (右図 1) のではなく、問題をより小さい問題に分割して考える。

n 枚の円盤を移動するには、次の手順を用いる。

- ・ 上部の $n-1$ 枚を一時置き場へ移動 (右図 2) する
- ・ 最下部の円盤を移動先へ移動 (右図 3) する
- ・ 一時置き場に移した $n-1$ 枚を移動先へ移動 (右図 4) する

ここで、「 $n-1$ 枚の円盤を移動する」という操作は、元の問題と同じ構造を持つ問題である。このような自己相似的な構造を持つ問題は、再帰呼び出しによって自然に記述することができる。

次のプログラム test017-5.c は、ハノイの塔の移動手順を表示するものである。



プログラム：test017-5.c

```

1  #include <stdio.h>
2
3  /* n枚の円盤を from から to へ、work を補助に用いて移動する */
4  void hanoi( int n, char from, char to, char work ) {
5      if ( n <= 0 ) return;
6      hanoi(n-1, from, work, to);    // 最下部の円盤に乗っているn-1枚を退避
7      printf("%c -> %c\n", from, to); // 最下部の円盤を目的の場所に移動
8      hanoi(n-1, work, to, from);    // 退避したn-1枚を目的の場所に移動
9  }
10
11 int main() {
12     int    n;
13
14     while ( 1 ) {
15         printf("number of disks(0 or negative to exit): ");
16         scanf("%d", &n);
17         if ( n <= 0 ) break;
18         printf("move sequence:\n");
19         hanoi(n, 'A', 'B', 'C');
20         printf("\n");
21     }
22 }

```

このプログラムでは、関数 `hanoi` が塔の移動を再帰的に実行する。

`hanoi`(円盤の枚数, 塔 0 のラベル (元の塔) , 塔 1 のラベル (移動先) , 塔 2 のラベル (一時置き場))
通常、「円盤の枚数」には 0 より大きい `int` 型整数を与え, 「塔 `n` のラベル」は `char` の値で与える。

このプログラムをコンパイルして実行する例を次に示す。

例. `test017-5.c` のコンパイルと実行

```
$ gcc test017-5.c -o test017-5 [Enter]
$ ./test017-5 [Enter]
number of disks(0 or negative to exit): 1 [Enter]
move sequence:
A -> B      ←円盤が 1 枚だけの場合
number of disks(0 or negative to exit): 2 [Enter]
move sequence:
A -> C      ←円盤が 2 枚の場合の
A -> B      移動手順
C -> B
number of disks(0 or negative to exit): 3 [Enter]
move sequence:
A -> B      ←円盤が 3 枚の場合の
A -> C      移動手順
B -> C
A -> B
C -> A
C -> B
A -> B
```

(右上に続く) →

→ (左下からの続き)

```
number of disks(0 or negative to exit): 4 [Enter]
move sequence:
A -> C      ←円盤が 4 枚の場合の
A -> B      移動手順
C -> B
A -> C
B -> A
B -> C
A -> C
A -> B
C -> B
C -> A
B -> A
C -> B
A -> C
A -> B
C -> B
number of disks(0 or negative to exit): 0 [Enter]
$      ←終了してシェルに戻った
```

まとめ) このように、再帰呼び出しを用いることで、複雑な問題の手順を極めて簡潔に表現することができる。

再帰呼び出しは使用に注意を要するが、適切に用いれば非常に強力なプログラミング技法となる。

9.2 関数定義の高度な方法

9.2.1 可変長の引数

これまでに説明した関数では、引数の個数は関数定義の時点で固定されていた。例えば、

```
int add( int x, int y );
```

のように定義された関数は、引数を必ず 2 つ受け取り、呼び出し側の引数 (実引数) と、関数定義の側の仮引数の位置がそのまま対応する。このような引数を、ここでは可変長引数との対比から「固定引数」と呼ぶ。

引数の個数と位置が予め決まっている通常関数に対して、引数の個数を可変にできる関数⁷² を以下で説明する。

《可変長の引数の関数定義》

書き方: 戻り値の型 関数名 (固定引数, ...) { 定義内容 }

ピリオド 3 つによる省略記号 (ellipsis) 「...」の指定により、可変長の引数を取る関数となる。「固定引数」の部分は、これまでに解説してきた関数定義における、位置と個数が決まった引数の並びである。また、可変長の引数の関数を定義する際は、少なくとも 1 つの固定引数が必要である。これは、`va_start` (後述) が可変長引数の開始位置を特定するために必要となるためである。

可変長引数の関数定義を行うためには、それに必要なマクロなどの定義が記述された `stdarg.h` というヘッダファイルを読み込む必要がある。このヘッダファイルが提供する重要な項目とその使用例を表 23 に示す。

表 23 の型宣言やマクロに関して、それらを使用するプログラムの例 `test018-1.c` を示して解説する。このプログラムでは、関数 `sum` が定義されており、この関数は可変長の引数を取り、それらの合計値を返すものである。また `sum` の第 1 引数には、与える可変長の引数の個数を指定する。

⁷²これまで使用してきた `printf` 関数は、まさに可変長の引数を取る関数の代表的な例である。

表 23: stdarg.h が提供する重要な項目と使用例

項目	例	解 説
va_list	va_list ap	可変長引数を順に参照するための情報を保持する変数 ap の宣言
va_start	va_start(ap,n)	初期化. 固定引数並びの最後の引数 n を指定して, 可変長引数を順に参照するための状態を ap に設定する. (マクロ)
va_arg	va_arg(ap,int)	引数並び ap から int 型の値を取り出す. 読み取った後は, 次に取り出す可変長引数へ読み取り位置が進む. (マクロ)
va_end	va_end(ap)	va_start で初期化した ap の使用を終了する. (マクロ)

プログラム: test018-1.c

```

1  #include <stdio.h>
2  #include <stdarg.h>
3
4  // 可変長の引数 (int) の合計を求める関数
5  // 先頭の引数は固定: 可変長引数の個数
6  int sum( int n, ... ) {
7      int r = 0;          // 合計値を初期化
8
9      va_list ap;          // 可変長引数の並びを宣言
10     va_start( ap, n );    // 可変長引数を用いる処理の初期化
11
12     for ( int i = 0; i < n; i++ ) {    // 全引数を合計するループ
13         r += va_arg( ap, int );
14     }
15
16     va_end( ap );        // 可変長引数を用いた処理の終了
17
18     return r;
19 }
20
21 int main() {
22     printf("%d\n", sum(2, 10, 20));
23     printf("%d\n", sum(5, 1, 2, 3, 4, 5));
24 }

```

このプログラムは, sum(2, 10, 20) と, sum(5, 1, 2, 3, 4, 5) を実行するもので, それぞれ, 与える引数の個数が異なる. このプログラムをコンパイルして実行する例を次に示す.

例. test018-1.c のコンパイルと実行

```

$ gcc test018-1.c -o test018-1  ←コンパイル
$ ./test018-1  ←実行
30      ←結果
15

```

以下, このプログラムの関数 sum に沿って, 解説する.

■ 可変長引数を参照するための変数の宣言: va_list

まず, 可変長引数を順に参照するための情報を保持する変数を va_list で宣言する.

書き方: va_list 変数

関数が受け取る可変長引数を順に参照するために用いる「変数」を宣言する. test018-1.c の sum 関数では ap という変数を宣言している.

■ 可変長の引数の取扱いの開始 (初期化): va_start

va_start によって, 可変長引数を順に参照するための状態を初期化する.

書き方: va_start(変数, 固定引数の最後のもの)

「固定引数の最後のもの」を指定して, 可変長引数を順に参照するための状態を「変数」に設定する. この「変数」は va_list で宣言したものである. va_arg を使用する前には, va_start による初期化が必要である. test018-1.c の sum 関数では, 固定引数 n の次からが可変長引数であり, それらを「変数」ap (va_list で宣言したもの) で扱うものとして初期化している.

■ 可変長引数の並びからの値の取り出し：va_arg

可変長の引数の並びから、実際に値を取り出す。

書き方： `va_arg( 変数, 型 )`

可変長引数の「変数」から「型」で指定した値を1つ取り出して返す。値を取り出すと、次の `va_arg` の実行では、その次の値が引数並びから取り出される。test018-1.c の `sum` 関数では、「型」に `int` を指定し、その型の値を1つずつ「変数」`ap` から取り出している。

■ 可変長の引数の取扱いの終了：va_end

`va_start` で初期化した「変数」の使用を終了する。

書き方： `va_end( 変数 )`

「変数」による可変長引数の取扱いの終了を宣言する。test018-1.c の `sum` 関数では、「変数」`ap` を指定して可変長引数の取扱いを終了している。

9.2.1.1 受け取った可変長引数の個数について

C 言語には、関数が受け取った可変長引数の個数を自動的に知るための標準的な仕組みはない。test018-1.c の `sum` 関数では、第1引数に、可変長引数の個数を受け取り、それに従って、可変長引数全てを加算している。

参考) `printf` 関数では、第1引数に受け取った書式文字列に含まれる変換指定子に対応させる形で、可変長の引数並びから値を1つずつ取り出している。

9.2.1.2 受け取った可変長引数の型について

C 言語には、関数が受け取った個々の可変長引数の型を自動的に知るための標準的な仕組みはない。test018-1.c の `sum` 関数では、専ら `int` の値を受け取るものとしている。

参考) `printf` 関数では、第1引数に受け取った書式文字列に含まれる変換指定子に従って、可変長引数から取り出す値の型を決めている。

重要) C 言語で可変長の引数を受け取る関数を定義する際は、引数の個数や型に関する情報をその関数に伝える方法を策定しなければならない。

注意) 可変長部分の引数には、整数拡張や `float` から `double` への変換などのデフォルト引数拡張が行われる。従って、`va_arg` で値を取り出す際には、実際に渡される型に注意する必要がある。

9.2.2 関数のポインタ

これまでに説明してきた関数は、プログラムの中で定義されると、その処理内容は固定されたものであった。しかし C 言語では、関数を指すポインタを変数に保持し、そのポインタを通して関数を呼び出すことができる。このように、関数を間接的に指定して呼び出すための仕組みが**関数のポインタ**である。

書き方： `型 (*ポインタ名)( 型 1, 型 2, … )`

指定した型の関数を指すポインタとして「ポインタ名」を宣言する。また、引数の型のみを「型1」、「型2」、…として記述する。

注意) 「(*ポインタ名)」の括弧は必要である。例えば、

```
int (*f)(int, int);
```

は「`int` 型の値を返す関数へのポインタ `f`」を宣言するが、括弧を付けずに

```
int *f(int, int);
```

とすると、「`int` 型へのポインタを返す関数 `f`」の宣言となり、意味が異なる。

上の宣言の後、「ポインタ名」に既存の関数の名前を代入することができる。ただしその場合、関数ポインタの型と、代入する関数の型が適合している必要がある。

次に示すプログラム test019-1.c は、宣言された関数のポインタ `f` に、具体的に定義された関数 `add`, `sub` を代入して実行する例である。

プログラム：test019-1.c

```
1  #include    <stdio.h>
2
3  int add( int x, int y ) {    // 加算の関数
4      return x + y;
5  }
6
7  int sub( int x, int y ) {    // 減算の関数
8      return x - y;
9  }
10
11 int main() {
12     int  a = 10, b = 6, c;
13     int  (*f)( int, int );    // 関数のポインタ f の宣言
14
15     f = add;                  // f に add を設定
16     c = f( a, b );            // f が add として働く
17     printf("f(%d,%d)=%d\n", a,b,c);
18
19     f = sub;                  // f に sub を設定
20     c = f( a, b );            // f が sub として働く
21     printf("f(%d,%d)=%d\n", a,b,c);
22 }
```

このプログラムでは、

```
int  (*f)( int, int );
```

として関数のポインタ `f` を宣言しており、この `f` に

```
int add( int, int );
int sub( int, int );
```

のような型の関数を代入することができる。

このプログラムをコンパイルして実行する例を次に示す。

例. test019-1.c のコンパイルと実行

```
$ gcc test019-1.c -o test019-1      ←コンパイル
$ ./test019-1      ←実行
f(10,6)=16      ← f が add (加算) として機能している
f(10,6)=4       ← f が sub (減算) として機能している
```

ここで重要なことは、`f` に `add` や `sub` の計算結果を代入しているのではなく、それらの関数を指すポインタ値を代入しているということである。

重要) 関数名は、多くの式の中ではその関数を指すポインタに変換されるため、適合する型の関数ポインタに代入することができる。

参考) ポインタを明示して「`(*f)(10,6)`」という書き方でも実行可能であるが、通常は簡潔な「`f(10,6)`」の形式で実行する。この 2 つの書き方は同じ意味である。

9.2.2.1 関数ポインタの配列

関数ポインタは、通常のポインタ変数と同様に配列の要素として並べることができる。これにより、複数の関数をまとめて管理し、インデックスによって処理を切り替えることができる。

関数ポインタの配列は、次のように宣言する。

書き方： 型 (*配列名 [要素数])(型 1, 型 2, …)

次に示すプログラム test019-2.c は、関数ポインタの配列を用いて、複数の関数をインデックス（添字）を切り替えて実行する例である。

プログラム：test019-2.c

```
1  #include    <stdio.h>
2  #define     TBLSIZE      4
3
4  int add( int x, int y ) { return x + y; }    // 4 種の
5  int sub( int x, int y ) { return x - y; }    // 異なる
```

```

6 | int mul( int x, int y ) { return x * y; }    // 関数の
7 | int dvd( int x, int y ) { return x / y; }    // 定義
8 |
9 | int main() {
10 |     int      (*f[TBLSIZE])( int, int );    // 関数のポインタの配列
11 |     int      a, b, c;
12 |
13 |     // 関数テーブルとして設定
14 |     f[0] = add;      f[1] = sub;      f[2] = mul;      f[3] = dvd;
15 |
16 |     a = 12;      b = 3;
17 |     for ( c = 0; c < TBLSIZE; c++ ) {      // cをインデックスにして
18 |         printf("f[%d](%d,%d) = %d\n",      // 関数テーブル内の機能を
19 |             c,a,b, f[c](a,b) );          // 選択し、処理を切り替える
20 |     }
21 | }

```

このプログラムではまず、

```
int (*f[TBLSIZE])( int, int );
```

として、関数ポインタの配列 `f` を宣言している。この配列の各要素には、この関数ポインタ型に適合する関数を設定することができる。プログラム中では `add`, `sub`, `mul`, `dvd` の4つの関数を配列 `f` に登録し、`for` 文のインデックス `c` を用いて順に呼び出している。具体的には、

```
f[c](a,b)
```

のように、配列のインデックスを指定することで、実行する関数を切り替えている。

このように、関数ポインタの配列を用いて複数の関数を管理する方法は、**関数テーブル**⁷³ (function table) と呼ばれ、メニュー処理やコマンド選択など、実用的なプログラムでよく用いられる。

参考) この方法は、`switch` 文による分岐の代替として用いられることもある。

上のプログラムをコンパイルして実行する例を次に示す。

例. test019-2.c のコンパイルと実行

```

$ gcc test019-2.c -o test019-2      ←コンパイル
$ ./test019-2      ←実行
f[0](12,3) = 15    ←配列のインデックスで
f[1](12,3) = 9      実行する関数を
f[2](12,3) = 36     選択している
f[3](12,3) = 4

```

⁷³C 言語規格で定められた用語ではないが、一般的に用いられる表現である。

9.3 ビット演算

C 言語の整数値は 2 進数のビット列で構成されており、システムプログラミングにおいては、整数値の各ビットを意識した演算が求められることがある。1 つの例として、8 ビットの整数値の概観を図 7 に示す。

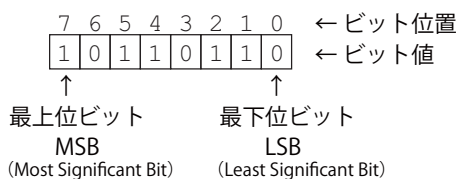


図 7: 8 ビット整数のビットの概観

これに倣って、16ビット、32ビット、64ビットの長さの2進数についても同様に考える。ビット値を下位ビットから $b_0, b_1, b_2, \dots, b_{n-1}$ とすると、 n ビット長の符号なし整数の値 x は

$$x = b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + \dots + b_{n-1} \times 2^{n-1} \quad \text{MSB : } b_{n-1}, \text{ LSB : } b_0$$

となる。また、現代の主要な処理系では、符号付き整数は一般に **2 の補数**によって表現され⁷⁴、最上位ビット (MSB) が符号に關係するビット (1:負, 0:非負) として機能する。

上記 b_0 (LSB) から b_{n-1} (MSB) のビット位置を

$bit_0, bit_1, \dots, bit_{n-1}$ あるいは 「第0ビット」, 「第1ビット」, 「第2ビット」, $\dots$, 「第 $n-1$ ビット」
と表記することも一般的である。

C 言語には、整数値に対するビット演算の機能が提供されている。最も基本的なビット演算を表 24 に示す。

表 24: 整数値のビット操作

優先度	演算	解説
1	$\sim x$	x の各ビットの反転
2	$x \ll n$ $x \gg n$	x を左に n ビットシフト (x が符号なしの場合 $x \times 2^n$)* x を右に n ビットシフト (x が符号なしの場合 $x/2^n$)*
3	$x \& y$	x と y の対応するビットの論理積
4	$x \wedge y$	x と y の対応するビットの排他的論理和
5	$x y$	x と y の対応するビットの論理和

※上のものほど優先度が高い

*範囲外に出たビットは捨てられる

注意) シフト演算では，符号付き整数，特に負の値に対する動作には注意が必要である．以下では，ビット列そのものを扱う場合には主に符号なし整数を用いる．

表 24 の演算子を用いてビットの反転, 論理積, 論理和の結果を求めるプログラム test012-1.c を次に示す.

注意) 整数値を 2 進数の形式に変換するための関数として、u2bin という関数を用いているが、これは巻末付録「B.1 整数を 2 進数表記の文字列に変換する機能」(p.511)に掲載しているものである。

プログラム：test012-1.c

```

1  #include      <stdio.h>
2  #include      <limits.h>
3  #include      "dec2bin.h"
4
5  int main() {
6      unsigned int    a = 2863311530,      b = 4294901760;
7      char            buf[CHAR_BIT * sizeof(unsigned int) + 1];
8
9      printf("%u(dec) =\n%s\n", a, u2bin(buf, a));
10     printf("----- not\n");
11     printf("%s\n\n", u2bin(buf, ~a));
12
13     printf("%s\n", u2bin(buf, b));
14     printf("%s\n", u2bin(buf, a));

```

⁷⁴C17 までの言語規格では，符号付き整数の内部表現は 2 の補数に限定されていない．本書では，現在広く用いられている 2 の補数表現を前提として説明する．

```
printf("----- and\n");
printf("%s\n\n", u2bin(buf,b&a));

printf("%s\n", u2bin(buf,b));
printf("%s\n", u2bin(buf,a));
printf("----- or\n");
printf("%s\n", u2bin(buf,b|a));
}
```

このプログラムをコンパイル，リンクして実行する例を次に示す.

例. test012-1.c のコンパイル, リンク, 実行

```
$ gcc test012-1.c dec2bin.o -o test012-1 Enter ←コンパイル, リンク
```

\$./test012-1 Enter ←実行

2863311530(dec) =

10101010101010101010101010101010 ←これを反転すると

~~01010101010101010101010101010101~~ not ←こうなる

```
11111111111111110000000000000000 ←これをビットマスクにして
10101010101010101010101010101010 ←これの対象ビット範囲を取り出すと、
```

$\frac{10101010101010101010101010101010}{1010101010101010100000000000000000}$ and ←こうなる

```
11111111111111110000000000000000 ←これで指定した範囲を
10101010101010101010101010101010 ←これに対して1で埋めると
```

----- or ←こうなる

11111111111111111010101010101010

注) これは unsigned int が 32 ビットの環境で実行したものである.

9.3.1 ビット演算の実用的な応用

ビット演算は、整数の各ビットを直接操作するための機能である。このため、ビット演算を行う際には、**符号なし整数型**を用いることが望ましい。

符号付き整数型に対してビット演算を行った場合、特にビットシフト演算において処理系に依存する動作が含まれる可能性がある。例えば、負の値に対して右シフト演算を行った場合、符号ビットを維持したままシフトされる処理系もあれば、単純にビットを右に移動して左側を 0 で埋める処理系もある。このような違いは、プログラムの移植性を損なう原因となる。

一方、符号なし整数型に対するビット演算は、符号付き整数型に比べてビット列に対する操作として扱いやすく、処理系に依存する動作を避けやすい。このため、ビットマスクやフラグ管理などの用途では、`unsigned int` などの符号なし整数型を用いるのが一般的である。

なお、ビット単位の操作で整数型のビット幅を明確にする必要がある場合には、`stdint.h` に定義される `uint32_t` や `uint64_t` などを用いる方法もある。これらを使用できる環境では、整数のビット幅を明示したプログラムを記述できる。

以下に、ビット演算の実用的な応用について解説する。

■ 全てのビットを off にする

0 を代入すると全てのビットが off になる.

■ 全てのビットを on にする

扱っている変数の型（符号なし整数）の最大値⁷⁵ を代入すると全てのビットが on になる。

■ 指定したビットの on/off を調べる

ビットシフト「>>」を用いて、調べたいビット位置が最下位ビット（LSB）に来るようにシフトし、LSB の値を調べる。LSB の値を調べるには 1（LSB のみが 1）と論理積を取る。例えば、

```
unsigned int x = 123456789;
```

の x の bit3 の状態を調べるには,

⁷⁵ 「3.5.1 整数が扱える値の範囲」(p.18)を参照のこと.

を求めると良い.

1u << 3 は bit3 のみが 1 である。これを応用する（対象の値との論理和を取る）と、特定のビットのみを on にすること（既に on ならそのまま）ができる。例えば、

の x の bit3 を on にしたければ,

とすれば良い.

~(1u << 3) は bit3 のみが 0 である。これを応用する（対象の値との論理積を取る）と、特定のビットのみを off にすること（既に off ならそのまま）ができる。例えば、

の x の bit3 を off にしたければ,

とすれば良い.

第 n ビットから、それより上位に向けて m ビット分を全て on にして、特定のビット範囲を示すビットマスクを作ることができる。これには次のような処理を行う。

例えば, $n = 4$, $m = 3$ の場合,

のようなビットマスクが生成される.

重要) 4ビット分を全て1にしたビット列は、16進数では1桁のfと表せるため、4ビット区切りのビットマスクは16進数表記を用いると簡単に記述できる。

[illegible]

重要) 上記の処理において、指定するビット位置やシフト量は、扱う整数型のビット幅の範囲内でなければならない。特に、 $((1u \ll m) - 1u) \ll n$ の形でビットマスクを生成する場合には、 m は unsigned int のビット幅未満とし、 $n+m$ がビット幅を超えないようにする必要がある。

次のプログラム test012-2.c では、指定したビットの状態を調べる関数 `chkbit`、指定したビットの on/off を設定する関数 `setbit` を定義しており、ビット状態の設定と参照を行う。ビットマスク作成の例も示す。

注) 整数値を 2 進数の形式に変換するための関数として、u2bin という関数を用いているが、これは巻末付録「B.1 整数を 2 進数表記の文字列に変換する機能」(p.511)に掲載しているものである。

```

1 #include <stdio.h>
2 #include <limits.h>
3 #include "dec2bin.h"
4
5 int chkbit( unsigned int field, int n ) {
6     return (field >> n) & 1u;
7 }

```

```

8
9 unsigned int setbit( unsigned int field, int n, int bit ) {
10     return (field & ~(1u << n)) | ((bit ? 1u : 0u) << n);
11 }
12
13 int main() {
14     unsigned int a = 0; // ビット編集用変数
15     char buf[CHAR_BIT * sizeof(unsigned int) + 1]; // 2進数変換バッファ
16
17     printf("%s\n", u2bin(buf,a)); // 初期状態の確認
18     printf("turn bit15 on\n");
19     a = setbit(a,15,1); // bit15をon
20     printf("%s\n", u2bin(buf,a)); // 状態確認
21     // bit16, bit15を確認
22     printf("bit16:%d, bit15:%d\n",chkbit(a,16),chkbit(a,15));
23     printf("-----\n");
24     printf("turn all bits on\n");
25     a = UINT_MAX; // 全ビットをon
26     printf("%s\n", u2bin(buf,a)); // 状態確認
27     printf("turn bit15 off\n");
28     a = setbit(a,15,0); // bit15をoff
29     printf("%s\n", u2bin(buf,a)); // 状態確認
30     // bit16, bit15を確認
31     printf("bit16:%d, bit15:%d\n",chkbit(a,16),chkbit(a,15));
32     // bit7から上位に向けて21ビット分を全てonにする (ビットマスク作成)
33     printf("---[bitmask]-----\n");
34     a = ((1u << 21) - 1u) << 7;
35     printf("%s\n",u2bin(buf,a));
36 }

```

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test012-2.c のコンパイル、リンク、実行

```

$ gcc test012-2.c dec2bin.o -o test012-2  ←コンパイル、リンク
$ ./test012-2  ←実行
00000000000000000000000000000000 ←はじめは全ビット off
turn bit15 on
0000000000000000000000001000000000000000 ← bit15 のみ on にした
bit16:0, bit15:1 ← bit16, bit15 の状態確認
-----
turn all bits on
11111111111111111111111111111111 ←全ビット on
turn bit15 off
11111111111111111011111111111111 ← bit15 のみ off にした
bit16:1, bit15:0 ← bit16, bit15 の状態確認
---[bitmask]-----
000011111111111111111111111100000000 ← bit7 から 21 ビット分のビットマスク

```

9.3.2 ビットフィールド

構造体を定義する際、メンバにビット単位でのサイズ指定を行うことができる。このようなメンバをビットフィールド (bit-field) と呼ぶ。

通常の構造体では、各メンバは int や char などの型単位でメモリが確保される。しかし、状態フラグのように「0 または 1」だけを表す値の場合には、1 ビットだけで十分である。このような場合、ビットフィールドを用いることで、メモリを効率よく利用できることがある。

ビットフィールドを用いた構造体を宣言するには、各メンバの記述の後ろに「:ビット数」を付ける。

例. ビットフィールドの定義

```

struct BFLD {
    unsigned char b1:1; // サイズが1ビットのメンバ
    unsigned char b2:4; // サイズが4ビットのメンバ
    :
};

```

重要) ビットフィールドでは、宣言に用いた型によってメモリ上での配置が異なることがある。この型はしばしば**基底型** (base type) と呼ばれる。ビットフィールドの配置方法は処理系に依存するため、詳細については以下で説明する。

9.3.2.1 C 言語の標準として使用できるビットフィールドの型

ビットフィールドの型として標準的⁷⁶ に使用できるものは、`Bool`, `int`, `signed int`, `unsigned int` である。このうち `int` については、符号付きとして扱われるか符号なしとして扱われるかが処理系に依存するため、符号の有無を明確にしたい場合には `signed int` または `unsigned int` を用いるのがよい。

一方、`char`, `unsigned char`, `short`, `long` など、これ以外の整数型をビットフィールドに使用できるかどうかは処理系に依存する。ただし、実際の主要なコンパイラでは、C 言語規格で必須とされている型以外についてもビットフィールドとして使用できるよう拡張されていることが多い。

例えば GCC や Clang では、`char`, `short`, `long` などを含む各種の整数型をビットフィールドに使用できる環境が多く、Microsoft Visual C++ (MSVC) でも `char` や `long` などをビットフィールドに使用できる拡張が提供されている。このため、現実のプログラムでは、例えば

```
unsigned char b:1;
```

のような記述を目にすることも多い。

実際のプログラミングでは、主要なコンパイラが提供する拡張機能を積極的に利用することも、目的や対象環境によっては十分に合理的である。ただし、その場合は使用するコンパイラの仕様や対応範囲をよく確認し、移植性への影響を理解した上で適切に利用する必要がある。

ビットフィールドを定義して使用するプログラム例 `test013-1.c` を次に示す。このプログラムでは、様々な基底型のビットフィールドをメンバとして持つ3つの構造体型 `BFLD1`, `BFLD2`, `BFLD3` が定義されている。これらはそれぞれ、1ビットのメンバを3つ持つ構造体であるが、結果的にどのようなサイズになるかを確認することができる。

プログラム: `test013-1.c`

```
1  #include    <stdio.h>
2
3  typedef struct {
4      unsigned int b1:1; // それぞれ符号なし
5      unsigned int b2:1; // 整数宣言の1ビット
6      unsigned int b3:1; // のメンバ
7  } BFLD1;
8
9  typedef struct {
10     unsigned char b1:1; // それぞれ符号なし
11     unsigned char b2:1; // char宣言の1ビット
12     unsigned char b3:1; // のメンバ
13 } BFLD2;
14
15 typedef struct {
16     unsigned char b1:1; // 符号なしcharと符号なし
17     unsigned char b2:1; // 整数が存在する1ビット
18     unsigned int b3:1; // のメンバ
19 } BFLD3;
20
21 int main() {
22     BFLD1 bf1;
23     BFLD2 bf2;
24     BFLD3 bf3;
25
26     bf1.b1 = 0;      bf1.b2 = 1;      bf1.b3 = 0;      // メンバへの値の設定
27     printf("bf1.b1=%d, bf1.b2=%d, bf1.b3=%d\n", bf1.b1, bf1.b2, bf1.b3);
28     printf("sizeof(bf1): %zu bytes\n", sizeof(bf1));
29
30     bf2.b1 = 1;      bf2.b2 = 0;      bf2.b3 = 1;      // メンバへの値の設定
31     printf("bf2.b1=%d, bf2.b2=%d, bf2.b3=%d\n", bf2.b1, bf2.b2, bf2.b3);
32     printf("sizeof(bf2): %zu byte\n", sizeof(bf2));
33
34     bf3.b1 = 1;      bf3.b2 = 1;      bf3.b3 = 0;      // メンバへの値の設定
```

⁷⁶C17 規格

```

35     printf("bf3.b1=%d, bf3.b2=%d, bf3.b3=%d\n", bf3.b1, bf3.b2, bf3.b3);
36     printf("sizeof(bf3): %zu bytes\n", sizeof(bf3));
37 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test013-1.c のコンパイルと実行

```

$ gcc test013-1.c -o test013-1  ←コンパイル
$ ./test013-1  ←実行
bf1.b1=0, bf1.b2=1, bf1.b3=0
sizeof(bf1): 4 bytes          ← 4 バイト
bf2.b1=1, bf2.b2=0, bf2.b3=1
sizeof(bf2): 1 byte          ← 1 バイト
bf3.b1=1, bf3.b2=1, bf3.b3=0
sizeof(bf3): 8 bytes          ← 8 バイト

```

この例は、Windows の MSYS2 環境下の GCC によるコンパイルと実行である。1 ビットのビットフィールドを 3 つ持つ 3 種類の構造体のサイズが互いに異なる結果となっている。これは、基底型が異なることで、構造体のメモリ上での配置が異なる (図 8)⁷⁷ ためである。

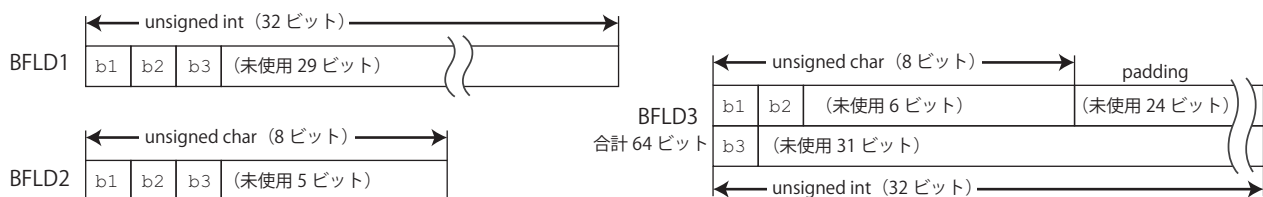


図 8: 基底型が異なることで、ビットフィールドのメモリ上の配置が異なる例

注意) ビットフィールドは通常の構造体メンバとは異なり、個々のビットフィールドのアドレスを取得することはできない。従って、ビットフィールドに対してアドレス演算子 & を用いることはできない。

次に、ビットフィールドが意図したバイトサイズに収まる可能性が高い例 test013-2.c を示す。これは、1 ビットのサイズのビットフィールドを 8 個 (多くの処理系では unsigned char と同じビット長) 定義する構造体の例である。

プログラム: test013-2.c

```

1  #include <stdio.h>
2
3  typedef struct {
4      unsigned char b1:1; // それぞれ符号なし char 宣言.
5      unsigned char b2:1; // 各1ビットのメンバが合計で
6      unsigned char b3:1; // 8つ定義されている.
7      unsigned char b4:1;
8      unsigned char b5:1; // 多くのコンパイラで,
9      unsigned char b6:1; // 全体のサイズが
10     unsigned char b7:1; // 8ビット=1バイト
11     unsigned char b8:1; // になる
12 } BFLD4;
13
14 int main() {
15     BFLD4 bf4;
16     printf("sizeof(bf4): %zu byte\n", sizeof(bf4));
17 }

```

これをコンパイルして実行する例を次に示す。

例. test013-2.c のコンパイルと実行

```

$ gcc test013-2.c -o test013-2  ←コンパイル
$ ./test013-2  ←実行
sizeof(bf4): 1 byte          ←ビットフィールドの合計長が8ビット (1 バイト) になっている

```

⁷⁷ この図はあくまで例であり、ビットフィールドのメモリ上での配置が必ずこのようになるという保証はない。

注意) 全ての環境、コンパイラで同様の結果になる保証はないので注意が必要である。ただし、多くのコンパイラでは、次のような方針でメモリ配置を行う傾向がある。

- (1) 同じ基底型のビットフィールドは、可能な限り詰めて配置する。
- (2) 基底型が変わると、新しい配置単位で配置されることがある。
- (3) アラインメントの条件を満たすようにパディングが挿入されることがある。

9.3.2.2 ビットフィールド使用上の注意

ビットフィールドはビット単位でのフラグ管理を簡潔に記述できる便利な機能である。しかし、ビットフィールドのメモリ上での配置は処理系に依存するため、構造体のサイズやメンバの配置が、OS、ビットモデル、コンパイラなどによって異なる場合がある。

また、基底型の違いやメンバの並び方によっては、アラインメントのためのパディングが挿入され、想定よりも大きなサイズになることもある。先の例でも、同じ 1 ビットのメンバを 3 つ持つ構造体であっても、基底型の違いによって異なるサイズとなる結果が確認された。

ビットフィールドと通常のメンバ（通常の変数の形式）を混在させた場合、アラインメントのためのパディングが複雑に挿入されることがあり、構造体のサイズが予想しにくくなる。このため、ビットフィールドを使用する場合は、通常のメンバとの混在を避ける方が望ましい場合もある。

ビット指定のフラグ管理を行う場合は、構造体の中に符号なし整数型のメンバを設け、その値をビット演算によって扱う方が分かりやすく、安全である場合が多い。

このように、ビットフィールドではメモリ配置を厳密に制御することが難しいため、特に次のような用途では注意が必要である。

- ・ バイナリファイルの読み書き
- ・ ネットワーク通信データの処理
- ・ ハードウェアレジスタの操作
- ・ 可搬性の高いプログラムの作成

これらの用途では、整数型の変数を用い、ビット演算によって各ビットを操作する方法の方が安全である場合が多い。従って、特段の技術的事情がない限り、ビット単位のフラグ管理には、ビットフィールドではなく、**整数のビット演算を応用する方法**⁷⁸ を用いる方が望ましい。

⁷⁸先の「9.3 ビット演算」(p.114～)で解説した内容に基づいて、整数データを 1 つのビットフラグのフィールドとして用いる方法。

9.4 共用体

複数のメンバをまとめたデータ型として、**共用体** (union) がある。共用体は構造体と同様の形式で宣言するが、各メンバが同じメモリ領域を共用する点が異なる。構造体では、各メンバのために個別のメモリ領域が確保されるが、共用体では、全てのメンバが同一のメモリ領域を共有するため、共用体のサイズは、少なくとも最も大きいメンバを格納できる大きさとなる⁷⁹。

共用体は複数のメンバが同じメモリ領域を共用するため、次のような用途に用いられる。

(1) データの分解

サイズの大きなデータを、より小さな単位 of データとして分解して解析することができる。例えば、整数値や浮動小数点数などのデータを、バイト単位の値として分解することが可能である。

(2) データの合成

複数のバイト値をまとめて、より大きなデータ (整数値や浮動小数点数など) として扱うことができる。例えば、4 個の 1 バイト値を連結して 4 バイトの整数として扱うことができる。

(3) データの再解釈

同一のメモリ領域を異なる型として解釈する用途である。例えば、浮動小数点数の内部表現を整数型として参照し、ビット単位の構造を確認することなどができる。

▲注意▲

共用体のあるメンバに値を設定した後、別の型のメンバを通してその内容を読み出した場合、その結果は型の内部表現や処理系に依存することがある。従って、この方法を用いてデータの内部表現を調べる場合には、使用する処理系の仕様にも注意する必要がある。

■ サンプルプログラム：データの分解と合成

共用体を応用してデータを分解、合成するサンプルプログラム test014-1.c を次に示す。このプログラムは、unsigned int 型の値を unsigned char の配列に対応させるものである。

プログラム：test014-1.c

```
1  #include    <stdio.h>
2
3  typedef union {
4      unsigned int    val;    // 符号なし整数と
5      unsigned char    b[4];    // 4バイトの列を共用
6  }    IUnion;
7
8  int main() {
9      IUnion    u;
10
11      // 符号なし整数のメンバに値を設定
12      u.val = (1u<<24) + (2u<<16) + (3u<<8) + 4u;    // 0x01020304
13      printf("u.val=%u(dec), 0x%08x(hex)\n", u.val, u.val);
14
15      // その内容をバイト別に出す
16      printf("u.b[0]=%u, u.b[1]=%u, u.b[2]=%u, u.b[3]=%u (dec)\n",
17              u.b[0], u.b[1], u.b[2], u.b[3]);
18
19      // 各バイトに個別に値を設定
20      u.b[0] = 1;    u.b[1] = 2;    u.b[2] = 3;    u.b[3] = 4;
21      printf("-----\n");
22      printf("u.b[0]=%u, u.b[1]=%u, u.b[2]=%u, u.b[3]=%u (dec)\n",
23              u.b[0], u.b[1], u.b[2], u.b[3]);
24      printf("u.val=%u(dec), 0x%08x(hex)\n", u.val, u.val);
25 }
```

このプログラムの前半では、整数値をバイト単位に分解して参照し、後半では各バイトに値を設定して整数値として参照している。

このプログラムをコンパイルして実行する例を次に示す。

⁷⁹アラインメントの関係で、更に大きくなる可能性もある。

例. test014-1.c のコンパイルと実行

```
$ gcc test014-1.c -o test014-1  ←コンパイル
$ ./test014-1  ←実行
u.val=16909060(dec), 0x01020304(hex) ←結果
u.b[0]=4, u.b[1]=3, u.b[2]=2, u.b[3]=1 (dec)
-----
u.b[0]=1, u.b[1]=2, u.b[2]=3, u.b[3]=4 (dec)
u.val=67305985(dec), 0x04030201(hex)
```

注) これは、unsigned int が 32 ビット、1 バイトが 8 ビットの環境で実行したものである。

この実行結果から、この実行環境における整数値のバイト配置がわかる。u.val の値 0x01020304 の下位バイトから順に、u.b[0]、u.b[1]、u.b[2]、u.b[3] に対応している。このように、値の下位バイトを低いアドレス側に格納する方式をリトルエンディアンと呼ぶ。これについては後の「9.5 データのバイト順（バイトオーダー）に関すること」(p.124～) で解説する。

■ サンプルプログラム：浮動小数点数の分解と合成

次に示すサンプルプログラム test014-2.c は double 型の値を符号部、指数部、仮数部に分解する例である。

ここでは、多くの処理系で採用されている IEEE 754 の 64 ビット倍精度形式 (binary64) を前提として説明する。この形式では、bit0～bit51 が仮数部 (fraction field)、bit52～bit62 が指数部、bit63 が符号部である。また指数部は 1023 のバイアスを持つ整数として扱われる。正規化された有限値では、指数部が 1023 の場合 $2^0 = 1$ を意味し、それより大きい場合は正の指数、小さい場合は負の指数となる。このとき値は次の式で表される。

$$\text{double の値} = (-1)^{\text{符号部}} \times \left(1 + \frac{\text{仮数部}}{2^{52}}\right) \times 2^{\text{指数部}-1023}$$

詳しくは IEEE 754 の規格を参照のこと。

プログラム：test014-2.c

```
1  #include <stdio.h>
2  #include <limits.h>
3  #include "dec2bin.h"
4
5  typedef union {
6      double          val;      // 64ビット浮動小数点数
7      unsigned long long b;      // 8バイト符号なし整数を共用
8  } DUnion;
9
10 int main() {
11     unsigned long long s, man, exp, // 符号, 仮数部, 指数部のための変数
12                     mask;          // ビットマスク
13     DUnion d;                      // 共用体
14     // 2進数文字列変換用バッファ
15     char buf[CHAR_BIT * sizeof(unsigned long long) + 1];
16
17     // double の分解
18     d.val = 6.02214076e23;          // とても大きい数：アボガドロ数
19     printf("[value] %.8e\n", d.val);
20     printf(" (bin) %s\n", ull2bin(buf, d.b));
21     mask = 1ull << 63;              // 符号ビット用のビットマスク
22     s = (d.b & mask) >> 63;         // 符号ビットの取り出し
23     mask = ((1ull << 11) - 1ull) << 52; // 指数部用のビットマスク
24     exp = (d.b & mask) >> 52;       // 指数部の取り出し
25     mask = (1ull << 52) - 1ull;     // 仮数部用のビットマスク
26     man = d.b & mask;               // 仮数部の取り出し
27     // 内容確認
28     printf("sign:%llu\n", s);
29     printf("man :%llu\n", man);
30     printf(" (bin) %s\n", ull2bin(buf, man));
31     printf("exp :%llu = %lld + 1023\n", exp, (long long)exp - 1023);
32     printf(" (bin) %s\n\n", ull2bin(buf, exp));
33
34     // double の合成
35     d.b = (923ull << 52) | 696939119748816ull; // 値の合成：電子の質量
36     printf("[value] %.8e\n", d.val);
37     printf(" (bin) %s\n", ull2bin(buf, d.b));
```


9.5 データのバイト順（バイトオーダー）に関すること

整数や浮動小数点数のように、複数バイトで表現される数値は、主記憶上では複数のバイトに分割されて格納される。このとき、各バイトをどの順序で格納するかは処理系によって異なる。この格納順序をバイト順（バイトオーダー）という。例えば、4 バイトの整数として

$$1 \times 256^3 + 2 \times 256^2 + 3 \times 256^1 + 4 \times 256^0 = 0x01020304$$

という値を考える。この値は主記憶上では 4 つのバイト値

1, 2, 3, 4

の並びとして表現される。このとき、主記憶上に格納される順序には次の 2 通りがある。

- ビッグエンディアン（Big Endian）
- リトルエンディアン（Little Endian）

ビッグエンディアンでは、上位バイトから順に格納されるため、

1 → 2 → 3 → 4

の順で記憶される。一方、リトルエンディアンでは、下位バイトから順に格納されるため、

4 → 3 → 2 → 1

の順で記憶される。この様子を図 9 に示す。

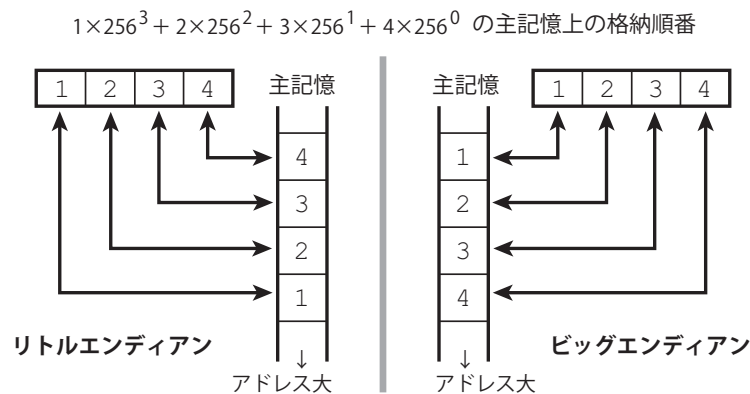


図 9: バイト順（バイトオーダー）

現在広く用いられている Intel や AMD の x86 / x86-64 系 CPU は、リトルエンディアンを採用している。一方、Motorola の MC68000 系列の CPU などではビッグエンディアンが採用されている⁸⁰。また、ネットワーク通信ではビッグエンディアンが標準的に用いられる。

このように、バイトオーダーは計算機のアーキテクチャや処理の目的によって異なるため、異なる環境間でバイナリデータをやり取りする場合には注意が必要となる。特に次のような場合にはバイトオーダーの違いが問題となることがある。

- バイナリファイルの保存と読み込み
- ネットワーク通信
- 構造体データの保存
- 共用体を用いたデータ解釈

なお、バイトオーダーの問題は、1 つの値を複数バイトに分けて表現する場合に生じる。1 バイト単位のデータでは、この問題は生じない。

9.5.1 バイト順の反転

先の「9.3 ビット演算」（p.114）で解説した方法を応用すると、バイト順を簡単に反転することができる。このことを次のプログラム test015-1.c で示す。

⁸⁰ ARM や PowerPC などの CPU では、リトルエンディアンとビッグエンディアンの両方を切り替えて使用できるものもある。このような方式をバイエンディアン（bi-endian）という。Java の class ファイルでは、多バイトデータはビッグエンディアンで格納される。

プログラム：test015-1.c

```
1  #include    <stdio.h>
2  #include    <stdint.h>
3
4  int main() {
5      uint32_t    b1 = 0x01020304, b2;
6
7      b2 =      (b1<<24)                // 最下位バイトを最上位に移動し,
8                | (b1&0x0000ff00)<<8    // 途中のバイトをビットマスクで
9                | (b1&0x00ff0000)>>8    // 取り出して移動し,
10               | (b1>>24);             // 最上位バイトを最下位に移動する
11      printf("original:%08x, reversed:%08x\n", b1, b2);
12 }
```

このプログラムでは、b1 の各バイトをビットマスクやビットシフトによって取り出して目的の位置に移動し、最終的にビット論理和で合成して b2 に結果を得ている。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test015-1.c のコンパイルと実行

```
$ gcc test015-1.c -o test015-1  ←コンパイル
$ ./test015-1  ←実行
original:01020304, reversed:04030201 ←出力
```

ビットマスク 0x0000ff00, 0x00ff0000 の作成に関しては「ビットマスクの作成」(p.116)を参照のこと。

64 ビットのデータに関してもこれと同様の方法が適用できる。更に、より長いデータや任意長のバイト列の扱いに関しては、後の「9.8 メモリの柔軟な取扱い」(p.134)で解説する方法を応用すると良い。

9.5.2 ホスト／ネットワークバイト順変換関数

バイトオーダーが異なる環境間で数値データをやり取りする場合には、バイト順を変換する必要がある。特に TCP/IP によるネットワーク通信では、ビッグエンディアンが標準（ネットワークバイトオーダー）として定められている。このため、ホスト環境のバイトオーダーとネットワークバイトオーダーの間で変換を行うための関数が用意されている。代表的なものとして、次の4つの関数がある。

- htons : 16 ビット整数をホスト → ネットワークに変換
- htonl : 32 ビット整数をホスト → ネットワークに変換
- ntohs : 16 ビット整数をネットワーク → ホストに変換
- ntohl : 32 ビット整数をネットワーク → ホストに変換

これらの関数名は次の意味を持つ。

hton : host to network, ntoh : network to host
s : short (16 ビット), l : long (32 ビット相当⁸¹)

これらの関数が扱う値は 16 ビットまたは 32 ビットの符号なし整数である。POSIX 環境では、arpa/inet.h において概ね次の形式で宣言されている。

プロトタイプ：

```
uint16_t htons( uint16_t hostshort );
uint32_t htonl( uint32_t hostlong );
uint16_t ntohs( uint16_t netshort );
uint32_t ntohl( uint32_t netlong );
```

Windows の Winsock では u_short, u_long などの型名を用いた宣言となっているが、扱う値のビット長は同様に 16 ビット、32 ビットである。

これらの関数を使用する際には、次のヘッダファイルをインクルードする。

Linux / macOS の場合 : #include <arpa/inet.h>
Windows の場合 : #include <winsock2.h>

⁸¹ 名称は歴史的なものであり、実際には 32 ビット整数を扱う。

これらの関数は、ホストバイトオーダーとネットワークバイトオーダー（ビッグエンディアン）の間で相互に変換を行う。ホスト環境がリトルエンディアンであればバイト順が入れ替えられ、ビッグエンディアンであれば実質的にそのままの値となる。

これらの関数の実行例を次のプログラム test015-2.c で示す。

プログラム：test015-2.c（Windows 環境での例）

```
1  #include    <stdio.h>
2  #include    <stdint.h>
3  // #include  <arpa/inet.h>          // POSIX環境の場合
4  #include    <winsock2.h>          // Windowsの場合
5
6  int main() {
7      uint16_t    a1 = 0x0102,      a2, a3;
8      uint32_t    b1 = 0x01020304,  b2, b3;
9
10     // 16ビットデータの変換／逆変換
11     a2 = htons(a1);      // 変換
12     a3 = ntohs(a2);      // 逆変換
13     printf("16bit: %04x -> htons -> %04x -> ntohs -> %04x\n",
14           (unsigned int)a1, (unsigned int)a2, (unsigned int)a3);
15
16     // 32ビットデータの変換／逆変換
17     b2 = htonl(b1);      // 変換
18     b3 = ntohl(b2);      // 逆変換
19     printf("32bit: %08x -> htonl -> %08x -> ntohl -> %08x\n",
20           (unsigned int)b1, (unsigned int)b2, (unsigned int)b3);
21 }
```

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test015-2.c のコンパイル、リンク、実行（Windows での例）

```
$ gcc test015-2.c -lws2_32 -o test015-2  Enter    ←コンパイル、リンク
$ ./test015-2  Enter    ←実行
16bit: 0102 -> htons -> 0201 -> ntohs -> 0102    ←結果
32bit: 01020304 -> htonl -> 04030201 -> ntohl -> 01020304
```

各関数によって、変換と逆変換が実行されている様子がわかる。

注意) リンク時のオプション「-lws2_32」は、Windows 環境で必要となる。

9.6 文字列操作関数

文字列を扱う関数が `string.h` に定義されている。ここでは、文字列用関数の中でも特に使用頻度の高いものを取り上げる。

9.6.1 文字列の長さの取得： `strlen`

文字列の長さを求める関数として、`strlen` が用意されている⁸²。

プロトタイプ： `size_t strlen( const char *s );`

引数 `s` には文字列へのポインタを指定する。`strlen` は、文字列の先頭から終端文字 `'\0'` までの文字数（終端文字は含まない）を数え、その値を返す。

9.6.2 文字列の複製： `strcpy`

文字列をコピーする関数として、`strcpy` が用意されている。

プロトタイプ： `char *strcpy( char *dest, const char *src );`

引数 `src` に指定された文字列を、`dest` にコピーする。コピーの際には、終端文字 `'\0'` も含めてコピーされる。また、戻り値として、コピー先のポインタ `dest` が返される。

注意) `dest` が示す領域には、終端文字 `'\0'` も含めて十分なサイズがなければならない。

9.6.3 文字列の比較： `strcmp`

文字列を比較する関数として、`strcmp` が用意されている。

プロトタイプ： `int strcmp( const char *s1, const char *s2 );`

引数 `s1` と `s2` に指定された文字列を比較した結果を、次のような値として返す。

- ・ 0 : 文字列が等しい場合
- ・ 負の値 : `s1` が `s2` より小さい場合
- ・ 正の値 : `s1` が `s2` より大きい場合

ここで、文字列の大小は、各文字の文字コード値に基づいて辞書順に比較される。

9.6.4 文字列の連結： `strcat`

文字列を連結する関数として、`strcat` が用意されている。

プロトタイプ： `char *strcat( char *dest, const char *src );`

引数 `dest` の文字列の末尾に、引数 `src` の文字列を連結する。連結の際には `dest` 上に、その終端文字 `'\0'` の位置から `src` の内容をコピーする。戻り値は、連結後の文字列の先頭アドレス（`dest`）である。

注意) `dest` が示す領域には、終端文字 `'\0'` も含めて十分なサイズがなければならない。

9.6.5 文字の検索： `strchr`

文字列の中から指定した文字を検索する関数として、`strchr` が用意されている。

プロトタイプ： `char *strchr( const char *s, int c );`

引数 `s` に指定された文字列の中から、文字 `c` を検索する。文字が見つかった場合は、その文字の位置を指すポインタを返し、見つからなかった場合は `NULL` を返す。

文字の検索は `s` の先頭から開始し、最初に現れる `c` を探す。また `strchr` は、見つかった文字の位置へのポインタを返すので、戻り値は、その位置からの文字列として扱うことができる。

⁸² 「3.7.2 文字列の長さ」(p.34～)でも解説した。

9.6.6 文字列の検索： strstr

文字列の中から指定した文字列を検索する関数として、 strstr が用意されている。

プロトタイプ： char *strstr(const char *s1, const char *s2);

引数 s1 に指定された文字列の中から、引数 s2 の文字列を検索する。文字列が見つかった場合は、最初に見つかった位置へのポインタを返し、見つからなかった場合は NULL を返す。

文字列の検索は s1 の先頭から開始し、最初に現れる s2 を探す。また strstr は、見つかった文字の位置へのポインタを返すので、戻り値は、その位置からの文字列として扱うことができる。

9.6.7 書式整形結果の文字列の作成： sprintf/snprintf

文字列へ書式付きで出力する関数として、 sprintf が用意されている。

プロトタイプ： int sprintf(char *str, const char *format, ...);

sprintf は、printf と同様に書式指定に従って出力を行うが、出力先が標準出力ではなく、文字列となる点異なる。戻り値は、出力された文字数（終端文字を除く）である。この関数のプロトタイプは stdio.h に定義されている。

sprintf は、出力先の配列のサイズを確認しない。したがって、出力される文字列が配列のサイズを超える場合、配列の範囲外に書き込みが行われる可能性⁸³ がある。

sprintf と同様の処理を行う関数に snprintf がある。この関数は、出力先の領域のサイズを明示的に指定するので sprintf よりも安全である。

プロトタイプ： int snprintf(char *str, size_t size, const char *format, ...);

snprintf は、引数 str が指す配列に書式整形した文字列を書き込む。このとき、引数 size に書き込み先の配列サイズを指定することで、書き込む文字数の上限を設定することができる。size が 0 より大きい場合、書き込まれる文字列は終端文字 '\0' を含めて最大 size バイトとなる。

戻り値は、出力先のサイズが十分であったと仮定した場合に書き込まれる文字数（終端文字 '\0' は含まない）である。従って、戻り値が size 以上の場合、文字列の一部が切り詰められたことがわかる。

例. snprintf の使用

```
char buf[20];
int n = snprintf(buf, sizeof(buf), "x=%d, y=%d", x, y);
```

このように、sizeof(buf) を指定しておく、配列 buf の範囲を超えて文字列が書き込まれることを防ぐことができる。

重要) snprintf を使用しても、出力結果が配列に収まらない場合には文字列が切り詰められる。必要に応じて戻り値を確認し、文字列全体が格納されたかどうかを判定する必要がある。

参考) 新規にプログラムを作成する場合には、出力先のサイズを指定できる snprintf を用いる方が安全性を高めやすい。ただし、sprintf が C 言語規格で非推奨とされているわけではない。

■ サンプルプログラム

先に示した文字列操作関数を使ったサンプルプログラム test020-1.c を示す。

プログラム： test020-1.c

```
1  #include    <stdio.h>
2  #include    <string.h>
3  #define     MAXLEN          1024
4
5  int main() {
6      // 文字列の長さを求める
7      char    s1[] = "The length of string: ";
8      size_t  len;
9      len = strlen(s1);
10     printf("%s%zu\n", s1, len);
11
12     // 文字列の複製
13     char    s2[MAXLEN];
```

⁸³未定義動作となり、プログラムの異常終了やデータ破壊などを引き起こす可能性がある。

```

14 strcpy(s2,"The copy of a string.");
15 printf("%s\n", s2);
16
17 // 文字列の比較
18 char s3[] = "abcde", s4[] = "abcde",
19 s5[] = "abcdef", s6[] = "abcd";
20 printf("strcmp(\"%s\", \"%s\")=%d\n", s3,s4,strcmp(s3,s4));
21 printf("strcmp(\"%s\", \"%s\")=%d\n", s3,s5,strcmp(s3,s5));
22 printf("strcmp(\"%s\", \"%s\")=%d\n", s3,s6,strcmp(s3,s6));
23
24 // 文字列の連結
25 char s7[MAXLEN] = "first half", s8[] = "later half";
26 strcat(s7,s8);
27 printf("%s\n",s7);
28
29 // 文字の検索
30 char s9[] = "appleorange", c = 'o', *p;
31 p = strchr(s9, c);
32 printf("strchr(\"%s\", '%c') -> %s\n", s9,c,p);
33
34 // 文字列の検索
35 char s10[] = "I am a boy. She is a girl.", s11[] = "She";
36 p = strstr(s10,s11);
37 printf("strstr(\"%s\", \"%s\") -> %s\n", s10,s11,p);
38
39 // 書式整形(1)
40 int x = 2, y = 3, r;
41 char s12[] = "a long long long long long long long long string";
42 char buf[MAXLEN];
43 puts("---sprintf---");
44 r = sprintf(buf,"x=%d, y=%d, x+y=%d\nText: \"%s\"", x,y,x+y,s12);
45 printf("%s\nreturn value = %d\n", buf,r);
46
47 // 書式整形(2)
48 puts("---snprintf with buffersize:40---");
49 r = snprintf(buf,40,"x=%d, y=%d, x+y=%d\nText: \"%s\"", x,y,x+y,s12);
50 printf("%s\nreturn value = %d\n", buf,r);
51 }

```

例. test020-1.c のコンパイルと実行

```

$ gcc test020-1.c -o test020-1 Enter ←コンパイル
$ ./test020-1 Enter ←実行
The length of string: 22 ←文字列の長さ
The copy of a string. ←文字列の複製
strcmp("abcde","abcde")=0 ←文字列の比較 (1)
strcmp("abcde","abcdef")=-1 ←文字列の比較 (2)
strcmp("abcde","abcd")=1 ←文字列の比較 (3)
first half,later half ←文字列の連結
strchr("appleorange",'o') -> orange ←文字の検索
strstr("I am a boy. She is a girl.,"She") -> She is a girl. ←文字列の検索
---sprintf---
x=2, y=3, x+y=5 ← sprintf による
Text:"a long long long long long long long long string" ←書式整形の結果
return value = 76 ← sprintf の戻り値：書式整形で得られたバイト数
---snprintf with buffersize:40---
x=2, y=3, x+y=5 ← snprintf による
Text:"a long long long ←書式整形の結果
return value = 76 ← snprintf の戻り値：書式整形が切り詰められなかった場合のバイト数

```

9.7 文字列解析関数

文字列を解析し、その内容を数値などの値として解釈するための関数群について解説する。

9.7.1 文字列からの値の読み取り：sscanf

文字列から書式指定に従ってデータを読み取る関数として、sscanf が用意されている。

プロトタイプ： `int sscanf( const char *str, const char *format, ... );`

sscanf は、scanf と同様に書式指定に従って入力を読み取るが、入力元が標準入力ではなく、文字列となる点異なる。戻り値は、正常に読み取られて変数に格納された項目数である。この関数のプロトタイプは stdio.h に定義されている。

■ サンプルプログラム

sprintf, sscanf を使用したプログラム test020-2.c を次に示す。

プログラム：test020-2.c

```
1  #include      <stdio.h>
2  #define      BUFLen          1024
3  #define      STRLEN          32
4
5  int main() {
6      int      a, n;
7      double   x;
8      char     s[STRLEN], buf[BUFLen];
9
10     // 文字列に各種値を埋め込む
11     sprintf(buf,"%d %.15f %s", 123, 3.141592653589793, "string" );
12     // 文字列から書式指定して値を取り出す
13     n = sscanf(buf,"%d %lf %31s", &a, &x, s );
14
15     printf("buf:\"%s\\", a=%d, x=%.15f, s=%s\\n", buf,a,x,s);
16     printf("items:%d\\n", n);
17 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test020-2.c のコンパイルと実行

```
$ gcc test020-2.c -o test020-2  ←コンパイル
$ ./test020-2  ←実行
buf:"123 3.141592653589793 string", a=123, x=3.141592653589793, s=string
items:3
```

9.7.2 文字列を整数値に変換する関数：strtol / strtoll / strtoul / strtoull

文字列を整数値に変換するための関数として、strtol および strtoll がある。これらの関数は、文字列を解析して整数値を生成するものであり、変換結果だけでなく、どこまで解析が行われたかという情報も取得できる点に特徴がある。プロトタイプは stdlib.h に定義されているので、使用する際はこのヘッダファイルをインクルードする。

プロトタイプ： `long strtol ( const char *nptr, char **endptr, int base );`
`long long strtoll( const char *nptr, char **endptr, int base );`
`unsigned long strtoul( const char *nptr, char **endptr, int base );`
`unsigned long long strtoull( const char *nptr, char **endptr, int base );`

nptr は変換対象の文字列へのポインタ, endptr は変換が終了した位置を受け取るポインタ（不要な場合は NULL）, base は変換のための基数（10 進数なら 10, 16 進数なら 16 など）を指定する。対応可能な基数は 2～36 であり、0 を指定して自動判定させることもできる。

base に 0 を指定した場合には、文字列の形式に応じて基数が自動的に判定される。例えば、先頭が "0x" の場合は 16 進数, "0" の場合は 8 進数, それ以外の場合は 10 進数として扱われる。

文字列の先頭に '+' や '-' がある場合には、その符号も解釈され、`strtol`、`strtoll` では符号付き整数として変換される。`strtoul`、`strtoull` でも先頭の符号は受理される⁸⁴。

`strtol` と `strtoll` は基本的な動作は同じであり、主な違いは戻り値の型である。`strtol` は `long` 型、`strtoll` は `long long` 型の値を返す。より広い範囲の整数値を扱いたい場合には `strtoll` を用いる。また、符号なし整数への変換を行う関数として `strtoul` と `strtoull` がある。これらはそれぞれ `unsigned long` 型、`unsigned long long` 型の値を返し、基本的な使用方法は `strtol`、`strtoll` と同様である。

これらの関数は、`nptr` が指す文字列の先頭から解析を行い、整数として解釈できる部分を読み取って値として返す。例えば、文字列が "123abc" の場合、先頭から整数として解釈できる部分は "123" であるため、戻り値は 123 となる。このとき、`endptr` は "abc" の先頭を指す。

`endptr` が `NULL` でない場合には、変換が終了した位置がこのポインタに格納される。すなわち、整数として解釈された部分の直後の文字を指す。

一方、文字列が "abc" のように整数として解釈できる部分を全く含まない場合、`endptr` は元の文字列 (`nptr`) と同じ位置を指す。この性質を用いることで、少なくとも 1 文字以上が整数として解釈されたかどうかを判定することができる。

■ 変換の成否の判定の例

文字列配列 `s` に格納されている内容を 10 進数の値として解釈して取り出す例を次に示す。

```
char *end;
long x = strtol(s, &end, 10);
if (s == end) {
    // 1 文字も数値として解釈できなかった場合の処理
}
```

また、少なくとも 1 文字以上が整数として解釈され、かつ文字列全体が整数として解釈されたかどうかは、次のように判定できる。(下の例)

```
if (s != end && *end == '\0') {
    // 文字列全体が整数である
}
```

重要) `strtol`、`strtoll`、`strtoul`、`strtoull` は、1 文字も数値として解釈できなかった場合に 0 を返す。この場合、読み取った値が実際に 0 である場合と区別するために、これら関数を実行した直後は、上で説明した方法で、読み取りの成否を検査しなければならない。

重要) これらの関数は部分的に解釈を行うため、入力が完全に整数であるかどうかは `endptr` を用いて確認する必要がある。また、変換結果が戻り値の型で表現可能な範囲を超えた場合には `errno` に `ERANGE` が設定されるので、厳密なエラー判定が必要な場合にはこれも確認する。

変換結果が表現可能な範囲を超えたかどうかを厳密に判定する場合には、`errno.h` をインクルードし、`strtol` などの実行前に `errno=0` とした後、実行後に `errno==ERANGE` であるかを調べる。

9.7.3 文字列を浮動小数点数に変換する関数：`strtof` / `strtod` / `strtold`

文字列を実数値に変換するための関数として、`strtof`、`strtod`、`strtold` がある。これらの関数は、文字列を解析して実数値を生成するものであり、変換結果だけでなく、どこまで解析が行われたかという情報も取得できる点に特徴がある。プロトタイプは `stdlib.h` に定義されているので、使用する際はこのヘッダファイルをインクルードする。

```
プロトタイプ： float strtof(const char *nptr, char **endptr);
                double strtod( const char *nptr, char **endptr );
                long double strtold( const char *nptr, char **endptr );
```

`nptr` は変換対象の文字列へのポインタ、`endptr` は変換が終了した位置（実数として解釈された部分の直後）を受け取るポインタ（不要な場合は `NULL`）である。

⁸⁴この場合は、読み取った値に対して符号なし整数型の演算規則に従って負号を適用した値が結果となる。例えば "-1" は、その型で表現できる最大値となる。

これらは基本的な動作は同じであり、戻り値の型がそれぞれ float, double, long double となる。より高い精度で実数値を扱う必要がある場合には strtold を用いる。

これらの関数は、nptr が指す文字列の先頭から解析を行い、実数として解釈できる部分を読み取って値として返す。例えば、文字列が "3.14abc" の場合、先頭から実数として解釈できる部分は "3.14" であるため、戻り値は 3.14 となる。この場合、endptr は "abc" の先頭を指す。

実数の表現としては、小数点を含む形式のほか、指数表記（例えば "1.23e3" など）も解釈される。

一方、文字列が "abc" のように実数として解釈できる部分を全く含まない場合、endptr は元の文字列（nptr）と同じ位置を指す。この性質を用いることで、少なくとも 1 文字以上が実数として解釈されたかどうかを判定することができる。

■ 変換の成否の判定の例

文字列配列 s に格納されている内容を実数値として解釈して取り出す例を次に示す。

```
char *end;
double x = strtod(s, &end);
if (s == end) {
    // 1 文字も実数として解釈できなかった場合の処理
}
```

また、文字列全体が実数として解釈されたかどうかは、少なくとも 1 文字以上が実数として解釈され、かつ end が文字列終端（'\0'）を指しているかどうかで判定できる。（下の例）

```
if (s != end && *end == '\0') {
    // 文字列全体が実数である
}
```

strtof, strtod, strtold は、1 文字も数値として解釈できなかった場合に 0.0 を返す。この場合、読み取った値が実際に 0.0 である場合と区別するために、これら関数を実行した直後は、上で説明した方法で、読み取りの成否を検査しなければならない。また、これらの関数は部分的に解釈を行うため、入力が完全に実数であるかどうかは endptr を用いて確認する必要がある。

参考） 変換結果が表現可能な範囲を超える場合などには errno に ERANGE が設定されることがある。厳密なエラー判定を行う場合には、関数の実行前に errno=0 とし、実行後の値も確認する。

文字列から整数値を取得する関数として atoi, atol, atoll, また浮動小数点数を取得する関数として atof がある。これらの関数は簡便に使用できるが、変換に失敗した場合の判定や、文字列のどこまでが解釈されたかを知ることができない。このため、より厳密な処理が必要な場合には、ここで解説した strtol や strtod などを用いることが望ましい。

■ サンプルプログラム

文字列から各種の型の数値を読み取るプログラムの例 test020-3.c を次に示す。

プログラム：test020-3.c

```
1  #include    <stdio.h>
2  #include    <stdlib.h>
3
4  int main() {
5      // データの文字列
6      char    s1[] = "1234567890abc",           // long用
7              s2[] = "123456789012345def",       // long long用
8              s3[] = "1.234567ghi",             // double用
9              s4[] = "3.141592653589793238jkl",  // long double用
10             *p;
11      // 読み取り用の変数
12      long    l;
13      long long ll;
14      double  d;
15      long double ld;
16
17      printf("1) '%s' からの読み取り\t: ", s1);
18      l = strtol(s1,&p,10);
```

```

19     printf("%ld,\t\t    残り:%s\n", l,p);
20
21     printf("2) '%s' からの読み取り\t: ", s2);
22     ll = strtoll(s2,&p,10);
23     printf("%lld,\t    残り:%s\n", ll,p);
24
25     printf("3) '%s' からの読み取り\t\t: ", s3);
26     d = strtod(s3,&p);
27     printf("%f,\t\t    残り:%s\n", d,p);
28
29     printf("4) '%s' からの読み取り: ", s4);
30     ld = strtold(s4,&p);
31     printf("%.18Lf,    残り:%s\n", ld,p);
32 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test020-3.c のコンパイルと実行

\$ gcc test020-3.c -o test020-3 ←コンパイル

\$./test020-3 ←実行

```

1) '1234567890abc' からの読み取り      : 1234567890,          残り:abc
2) '123456789012345def' からの読み取り : 123456789012345,      残り:def
3) '1.234567ghi' からの読み取り        : 1.234567,          残り:ghi
4) '3.141592653589793238jkl' からの読み取り: 3.141592653589793238, 残り:jkl

```

注) long double の精度は処理系によって異なるため、4) の表示結果は実行環境によって異なる場合がある。

9.8 メモリの柔軟な取扱い

C 言語では、メモリ上に格納されているデータの表現をバイト単位で扱うことができる。特に、オブジェクトへのポインタを `unsigned char *` にキャストすることで、そのオブジェクトのメモリ上の表現をバイト列として参照することができる。

`unsigned char` 型は 1 バイト単位のデータを表す型である。このため、任意のデータのアドレスを `unsigned char *` にキャストすれば、そのデータのメモリ上の表現をバイト単位で参照することができ、`p[0]`, `p[1]`, ... のように配列と同様の形式で扱うことができる。

次のプログラム `test016-1.c` では、`unsigned long long` 型の変数 `x`（この例では 8 バイトであることを前提とする）のアドレス（メモリ上での格納位置）を `unsigned char *` の `p` にキャストしている。これにより、`p` を用いて `unsigned char` の配列と同様の形式でバイト単位にアクセスできることを示す。

プログラム：test016-1.c

```
1  #include    <stdio.h>
2
3  int main() {
4      unsigned long long  x = 0x0102030405060708;    // 8 バイトのデータ（整数）
5      unsigned char      *p;
6      size_t             c, sz;
7
8      p = (unsigned char *)&x;    // 整数データのアドレスをキャスト
9      sz = sizeof(x);    // 整数データのサイズ（バイト）
10     for ( c = 0; c < sz; c++ ) {
11         printf("%02x, ", (unsigned int)p[c]);    // 配列としてバイト値を参照
12     }
13     printf("\n");
14 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test016-1.c のコンパイルと実行（リトルエンディアン環境）

```
$ gcc test016-1.c -o test016-1  [Enter]    ←コンパイル
$ ./test016-1  [Enter]    ←実行
08, 07, 06, 05, 04, 03, 02, 01,    ←結果
```

変数 `x` の内容（バイト毎の値）を `p[c]` として（配列の要素として）参照していることがわかる。

重要） 共用体の別のメンバとしてデータを再解釈する方法とは異なり、`unsigned char *` を用いる方法では、オブジェクトのメモリ上の表現そのものをバイト単位で直接参照することができる。

■ 応用例：指定したメモリ領域のバイト値の順を逆転する例

次に示すプログラム `test016-2.c` では、指定したメモリ領域のバイト値の順を逆転する関数 `bytereverse` が定義されている。これによって、`unsigned long long` 型の整数（この例では 8 バイト）のバイト順を反転する。

プログラム：test016-2.c

```
1  #include    <stdio.h>
2
3  // 指定したメモリの領域のバイト順を逆転する関数
4  // データ領域の先頭アドレスと
5  // データ領域のサイズを与える
6  void bytereverse(unsigned char *p, size_t size) {
7      size_t i;
8      unsigned char tmp;    // バイト値交換用の一時作業用変数
9
10     for (i = 0; i < size / 2; i++) {    // バイト値の交換
11         tmp = p[i];    // によって、領域の
12         p[i] = p[size - 1 - i];    // 全てのバイト値を
13         p[size - 1 - i] = tmp;    // 入れ替えるループ
14     }
15 }
16
17 int main() {
```

```

18     unsigned long long  x = 0x0102030405060708ull;  // 8バイトのデータ（整数）
19     unsigned char      *p;
20     size_t             c, sz;
21
22     p = (unsigned char *)&x;          // 整数データのアドレスをキャスト
23     sz = sizeof(x);                  // 整数データのサイズ（バイト）
24     for ( c = 0; c < sz; c++ ) {
25         printf("%02x, ", (unsigned int)p[c]);  // 配列としてバイト値を参照
26     }
27     printf("\n");
28     bytewise(p, sz);                  // xのバイト並びを反転
29     for ( c = 0; c < sz; c++ ) {
30         printf("%02x, ", (unsigned int)p[c]);  // 配列としてバイト値を参照
31     }
32     printf("\n");
33 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test016-2.c のコンパイルと実行（リトルエンディアン環境）

```

$ gcc test016-2.c -o test016-2  [Enter]  ←コンパイル
$ ./test016-2  [Enter]  ←実行
08, 07, 06, 05, 04, 03, 02, 01,  ←逆転前
01, 02, 03, 04, 05, 06, 07, 08,  ←逆転後

```

変数 `x` のバイト順が反転できていることがわかる。

参考）整数型などの固定長データのバイト順を反転する場合には、先の「9.5.1 バイト順の反転」（p.124）で解説した方法も利用できる。処理系によっては、このような固定長データに対する処理を効率よく実行できる。

9.8.1 メモリの初期化

メモリ領域をまとめて初期化するための関数として、`memset` 関数が用意されている。この関数のプロトタイプは `string.h` に定義されている。

プロトタイプ： `void *memset( void *s, int c, size_t n );`

この関数は、メモリ領域 `s` の先頭から `n` バイト分に対して、値 `c` をバイト単位で設定する関数である。このとき、`c` は `unsigned char` 型の値に変換され、各バイトに同じ値が設定される。

例えば、配列の内容をすべて 0 に初期化する場合には、次のようにする。

例. 配列を 0 で初期化する

```

int a[10];
memset(a, 0, sizeof(a));

```

この例では、配列 `a` の全ての領域に 0 が設定される。ここで、`sizeof(a)` は配列 `a` の全体のバイト数を表す。

`memset` 関数は、配列や構造体などのデータ領域をまとめて一定のバイト値で初期化⁸⁵ する際に便利である。例えば、構造体のメモリ領域の全バイトを 0 にする場合には、次のようにする。

例. 構造体のメモリ領域を 0 で初期化する

```

struct sample {
    int x;
    double y;
};

struct sample s;
memset(&s, 0, sizeof(s));

```

このように、`memset` 関数を用いることで、指定したメモリ領域をまとめて初期化することができる。

注意）`memset` はバイト単位で値を設定する関数であるため、0 以外の値で整数配列などを初期化する場合には注意が必要である。このことに関して、次のプログラム `test016-3.c` で確認する。

⁸⁵要素ごと、メンバごとではなく、バイトレベルで。

プログラム：test016-3.c

```
1  #include    <stdio.h>
2  #include    <string.h>
3  #define     ARSIZE      10
4
5  int main() {
6      short int    a[ARSIZE];
7      int          c;
8
9      // 配列 a のメモリ領域をバイト単位で全て1にする
10     memset( a, 1, sizeof(a) );
11     for ( c = 0; c < ARSIZE; c++ ) { // 配列の要素の値を確認するループ
12         printf("%04x, ", (unsigned int)a[c]); // 16進数で表示
13     }
14     printf("\n");
15 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test016-3.c のコンパイルと実行

```
$ gcc test016-3.c -o test016-3  ←コンパイル
$ ./test016-3  ←実行
0101, 0101, 0101, 0101, 0101, 0101, 0101, 0101, 0101, 0101, ←結果
```

この実行環境では、各バイトに値 1 が設定されるため、2 バイトの short 型の各要素は 0x0101 となり、配列の各要素が 1 になるわけではない。

9.8.2 メモリの複製（コピー）

メモリ領域の内容を別の領域にコピーするための関数として、memcpy 関数が用意されている。この関数のプロトタイプは string.h に定義されている。

プロトタイプ： void *memcpy(void *dest, const void *src, size_t n);

memcpy 関数は、メモリ領域 src の先頭から n バイト分を、メモリ領域 dest にコピーする関数である。この関数は、バイト単位でメモリ内容をそのままコピーする。

例えば、配列の内容を別の配列にコピーする場合には、次のようにする。

例. 配列の内容をコピーする

```
int a[10] = {0,1,2,3,4,5,6,7,8,9};
int b[10];
memcpy(b, a, sizeof(a));
```

この例では、配列 a の内容が配列 b にコピーされる。ここで、sizeof(a) はコピーするバイト数を指定している。

memcpy 関数は、構造体などの複合データ型のコピーにも使用できる。例えば、次のように構造体の内容をコピーすることができる。

例. 構造体のコピー

```
struct sample {
    int x;
    double y;
};

struct sample s1 = {10, 3.14};
struct sample s2;
memcpy(&s2, &s1, sizeof(s1));
```

このように、memcpy 関数を用いることで、任意のデータのメモリ内容をそのままコピーすることができる。（なお、同じ構造体型の変数同士では、通常の代入によって内容をコピーすることもできる）

注意） memcpy 関数では、コピー元とコピー先のメモリ領域が重なってはならない。両者が重なる場合の動作は未定義となる。このような場合には、memmove 関数を使用する。memmove 関数のプロトタイプは次の通りである。

```
void *memmove( void *dest, const void *src, size_t n );
```

memmove 関数は、コピー元とコピー先の領域が重なっている場合でも、正しくコピーすることができる。

memcpy, memmove の働きを、次のプログラム test016-4.c で示す。

プログラム：test016-4.c

```
1  #include    <stdio.h>
2  #include    <string.h>
3  #define     ARSIZE      5
4
5  int main() {
6      short int  a[ARSIZE];
7      int        b[ARSIZE];
8      int        c;
9
10     memset( a, 1, sizeof(a) );      // 配列 a をバイト単位で全て1にする
11     memset( b, 0, sizeof(b) );      // 配列 b をバイト単位で全て0にする
12     printf("a: ");                  // 配列 a の全要素を16進数で確認
13     for ( c = 0; c < ARSIZE; c++ ) printf("%04x, ", (unsigned int)a[c]);
14     printf("\n");
15     printf("b: ");                  // 配列 b の全要素を16進数で確認
16     for ( c = 0; c < ARSIZE; c++ ) printf("%08x, ", (unsigned int)b[c]);
17
18     // 配列 a の内容を全て配列 b の領域にコピー
19     printf("\na -> memcpy -> b\n");
20     memcpy( b, a, sizeof(a) );
21     printf("b: ");
22     for ( c = 0; c < ARSIZE; c++ ) printf("%08x, ", (unsigned int)b[c]);
23     printf("\n");
24
25     // 配列 b 内で重なった領域をコピー：b[0]～b[2] -> b[2]～b[4]
26     printf("b[0],b[1],b[2] -> memmove -> b[2],b[3],b[4]\n");
27     memmove(&b[2], &b[0], 3 * sizeof(b[0]));
28     printf("b: ");
29     for ( c = 0; c < ARSIZE; c++ ) printf("%08x, ", (unsigned int)b[c]);
30     printf("\n");
31 }
```

このプログラムでは、要素のバイトサイズの異なる2つの配列 a, b を用意し、a の全内容をそのまま b に（先頭位置から）memcpy によってコピーする。この実行環境では short int が2バイト、int が4バイトであるため、a の全メモリ領域（10バイト）が b の先頭部分にコピーされる。

また、配列 b の内部で互いに重なる領域の複製を memmove で行っている。この処理では、b[0], b[1], b[2] のメモリ領域を b[2], b[3], b[4] の領域にコピーする。

このプログラムをコンパイルして実行する例を次に示す。

例. test016-4.c のコンパイルと実行

```
$ gcc test016-4.c -o test016-4  ←コンパイル
```

```
$ ./test016-4  ←実行
```

```
a: 0101, 0101, 0101, 0101, 0101, ← a の内容
```

```
b: 00000000, 00000000, 00000000, 00000000, 00000000, ← b の内容
```

```
a -> memcpy -> b
```

```
b: 01010101, 01010101, 00000101, 00000000, 00000000, ← a の全内容を b にコピー
```

```
b[0],b[1],b[2] -> memmove -> b[2],b[3],b[4]
```

```
b: 01010101, 01010101, 01010101, 01010101, 00000101, ← b の先頭3要素を末尾3要素にコピー
```

memcpy で a の内容を b にコピーすると、b[2] の値が 0x00000101 となっている。これは、この実行環境がリトルエンディアンであり、int が4バイトであるため、コピーされた残り2バイトが b[2] の下位側に配置されたためである。

この例から、memcpy はメモリ内容をバイト列としてそのまま複製する関数であり、データ型の意味を考慮しないことがわかる。また、同一領域内でコピー元とコピー先が重なる可能性がある場合には、memmove を用いる必要がある。

9.8.3 メモリの比較

メモリ領域の内容を比較するための関数として、memcmp 関数が用意されている。この関数のプロトタイプは string.h に定義されている。

プロトタイプ： int memcmp(const void *s1, const void *s2, size_t n);

memcmp 関数は、メモリ領域 s1 と s2 の先頭から n バイト分を比較する関数である。比較は、各バイトを unsigned char 型の値として、先頭から順に行われる。この関数の戻り値は次の通りである。

- ・ 0 : 比較した領域の内容が等しい
- ・ 正の値 : 最初に異なるバイトについて s1 側の値が大きい
- ・ 負の値 : 最初に異なるバイトについて s1 側の値が小さい

重要) 「大きい／小さい」は、各バイトの値を先頭から順に比較し、最初に異なるバイトの値によって判定される。

2つの配列の内容が一致しているかどうかを確認する場合には、次のように使用する。

例. 配列の内容を比較する

```
unsigned char a[5] = {1,2,3,4,5};
unsigned char b[5] = {1,2,3,4,5};
if ( memcmp(a, b, sizeof(a)) == 0 ) {
    printf("一致\n");
}
```

この例では、配列 a と b の内容が一致しているため、条件が成立する。

注意) 構造体が占めるメモリ領域を memcmp で比較すること自体は可能であるが、構造体の各メンバの値が等しいかどうかを判定する目的には一般に適さない。

例. memcmp による構造体の比較（危険）

```
struct sample {
    int x;
    double y;
};

struct sample s1 = {10, 3.14};
struct sample s2 = {10, 3.14};
if ( memcmp(&s1, &s2, sizeof(s1)) == 0 ) {
    printf("一致\n");
}
```

memcmp 関数は、メモリ内容をバイト単位で比較する関数であるため、構造体の比較では注意が必要である。構造体にはアラインメントのための余分な領域（パディング）や未使用領域が含まれる場合がある。このため、各メンバの値がすべて等しい場合でも、memcmp の結果が一致しないことがある。また、浮動小数点数では、数値として等しい値であってもバイトレベルでの表現が常に一致するとは限らないため、数値としての等価性を memcmp で判定することは適切ではない。

memcmp の働きを、次のプログラム test016-5.c で示す。このプログラムは、サイズが同じで内容の異なる a1 ～ a4 の配列の比較をするものである。

プログラム：test016-5.c

```
1  #include    <stdio.h>
2  #include    <string.h>
3  #define     ARSIZE      5
4
5  int main() {
6      unsigned char    a1[] = {1,2,3,4,5},        // 基準
7                      a2[] = {1,2,3,4,5},        // 同じ
8                      a3[] = {1,2,3,4,4},        // 小さい
9                      a4[] = {1,2,3,4,6};        // 大きい
10     int            res;
11
12     res = memcmp( a1, a2, sizeof(a1) );
13     printf("memcmp(a1,a2) = %d\n",res);
14 }
```

```

15     res = memcmp( a1, a3, sizeof(a1) );
16     printf("memcmp(a1,a3) = %d\n",res);
17
18     res = memcmp( a1, a4, sizeof(a1) );
19     printf("memcmp(a1,a4) = %d\n",res);
20 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test016-5.c のコンパイルと実行

```

$ gcc test016-5.c -o test016-5  ←コンパイル
$ ./test016-5  ←実行
memcmp(a1,a2) = 0      ← a1 と a2 は等しい
memcmp(a1,a3) = 1      ← a1 は a3 より大きい
memcmp(a1,a4) = -1     ← a1 は a4 より小さい

```

注意) memcmp が 0 以外の値を返す場合, 具体的な値は定められていない.

9.9 プログラム実行のためのメモリモデル

実行形式ファイルを起動すると、OS がそのプログラムの内容を読み込んで、記憶（メモリ）の適切な位置に配置し、プロセス⁸⁶ としてその実行を開始する。実際にプログラムが、メモリ上のどの位置に配置されるかは、OS がそのプログラムの起動時に判断する。OS が、プロセスの実行のために必要なメモリを割り当てる様子を図 10 に概略的に示す。

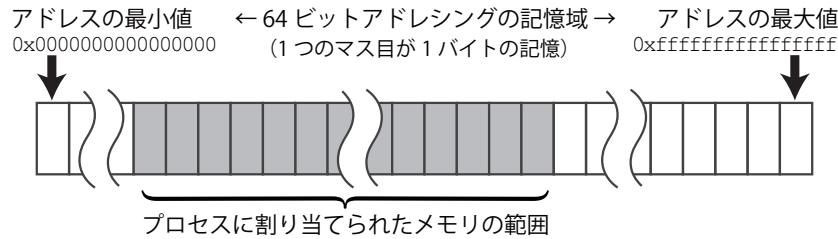
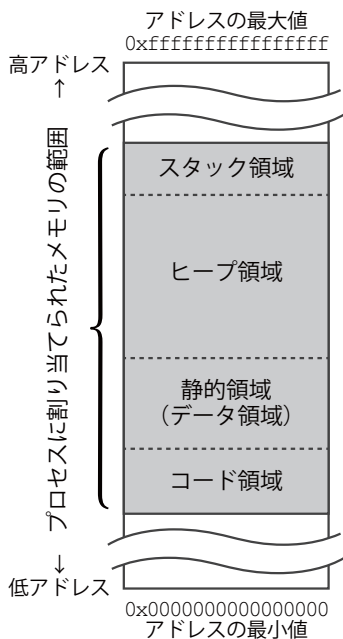
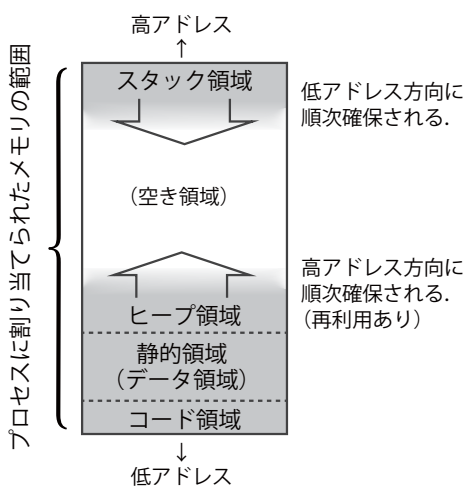


図 10: プログラムがメモリ上に配置されているイメージ



(a) プロセスに割り当てられたメモリの内訳



(b) スタック領域、ヒープ領域を確保するイメージ

図 10 は、プロセスの実行に使用されるメモリ領域を概念的に示したものである。一般的な OS では、各プロセスに仮想的なアドレス空間が与えられ、その中の必要な位置にプログラムやデータなどの領域が配置される。またこのメモリの範囲は、左図の (a) が示すように、更に複数の領域に細分化されている。

左図 (a) のコード領域は、実行形式ファイルの中の機械語プログラムを収める部分であり、この部分の内容が実際に実行される。また、プログラム中で定義されたグローバル変数や static 変数の内容は、静的領域（データ領域）の中に格納される。

C 言語のプログラムは、main 関数をはじめとする複数の関数から成り、一般的な処理系では、ある関数が呼び出されると、その関数で使用する auto 変数などのための領域が、主としてスタック領域に確保される。また、その関数の実行が終了すると、当該関数が使用していたスタックの部分は解放され、他の関数の実行のために使用可能となる。

一般的な環境では、スタック領域は比較的高いアドレス側に配置され、関数呼び出しが多重に連鎖（関数が更に他の関数を呼び出す連鎖）すると、低いアドレス側に向けて使用領域が拡大する構成が広く用いられている。

多重に呼び出された関数は、最後に呼び出された関数から順に終了し、それに沿って、スタックも、次々と、アドレスの高い方に向けて解放される。左図の (b) はそれを図示している。

このスタック領域は、関数の実行状態（auto 変数など）を保持するのが目的であるが、サイズには上限があり、デスクトップ向けの一般的な環境では、特に指定しない場合、数 MB 程度に設定されていることが多い。このため、サイズの大きな配列を auto 変数で宣言すると、スタック領域の枯渇（stack overflow）のリスクを高める。

グローバル変数や static 変数はプログラムの実行中を通して保持されるが、基本的には宣言時に必要なサイズが決まるため、実行中の必要量に応じて柔軟にサイズを変更する用途には適さない。大きいサイズのメモリを動的に確保するにはヒープ領域を利用すると良い。

後述の malloc 関数などを使用すると、プログラムの実行中に必要なサイズのメモリ領域を動的に確保し、使用が終了した時点で解放することができる。図 (b) では典型的なイメージとして、ヒープ領域が低いアドレス側から高いア

⁸⁶OS がプログラムの実行状態を管理する単位。

ドレス側へ拡大する様子を示しているが、実際の配置や確保方法は OS や処理系によって異なる。

▲注意▲

上で説明したコード領域、静的領域、ヒープ領域、スタック領域などのメモリ配置は、現在の一般的な OS および C 言語処理系で広く用いられている実装モデルを概略的に示したものである。C 言語規格そのものが、これらの領域の配置やアドレスの大小関係を規定しているわけではない。

9.9.1 ヒープ領域の利用

C 言語では、ヒープ領域からメモリを確保するための関数として、`malloc` 関数が用意されている。この関数のプロトタイプは `stdlib.h` にあり、使用する際にはこれをインクルードする。

プロトタイプ： `void *malloc( size_t size );`

引数 `size` には、確保したいメモリサイズ（バイト数）を指定する。この関数は、指定したサイズのメモリ領域を動的に確保し、その先頭アドレスを `void *` 型のポインタとして返す。メモリ領域の確保に失敗した場合は、戻り値は `NULL` となる。

参考） このように動的に確保されるメモリ領域は、一般的な処理系ではヒープ領域として管理される。

`void *` 型は、特定のデータ型を指すことに限定されない汎用的なポインタ型である。`malloc` は、確保したメモリ領域の先頭アドレスを `void *` 型で返す。C 言語では、`void *` 型のポインタは、他のオブジェクトへのポインタ型に暗黙に変換される。例えば、整数型（`int`）の配列を確保する場合は、次のように記述する。

例. `int` 型の配列を動的に作成する

```
int *p;
p = malloc( 10 * sizeof(int) );
```

この例では、`int` 型の要素を 10 個格納できる配列のためのメモリを確保している。`sizeof(int)` は `int` 型のサイズ（バイト数）を表し、これに要素数を掛けることで、必要なメモリサイズを求めている。

なお、実際のプログラミングでは、`malloc` の実行の後で、確保が成功したかどうか（`NULL` でないか）を確認するべきである。

■ メモリの解放

`malloc` 関数により確保されたメモリは自動的に解放されない。確保したメモリの使用が終了した場合には、`free` 関数を用いて明示的に解放する必要がある。この関数のプロトタイプは `stdlib.h` にあり、使用する際にはこれをインクルードする。

プロトタイプ： `void free( void *ptr );`

引数 `ptr` には、`malloc` などによって確保されたメモリ領域の先頭アドレスを指定する。`free` 関数は、指定されたメモリ領域を解放し、以後その領域をプログラムから使用しない状態にする。解放された領域は、処理系によっては後のメモリ確保に再利用されることがある。

なお、`free` に渡すことができるのは、`malloc` などのメモリ確保関数から得られたポインタ（または `NULL`）である。同じ領域を 2 回以上解放してはならない。このような操作は、プログラムの誤動作や異常終了の原因となる。

`free(NULL)` は何も行わないため、安全に呼び出すことができる。

▲注意▲

ヒープ領域に確保したメモリは、不要になった際には解放しなければならない。これを怠ると、不要になったにもかかわらず解放されないメモリ領域が残り続ける。このような状態を**メモリリーク**（memory leak）と呼ぶ。メモリリークは、長時間動作するプログラムや繰り返し処理を行うプログラムでは、ヒープ領域を枯渇させ、重大な問題を引き起こすことがあるため注意が必要である。

`free` 関数で解放されたメモリ領域は、後のメモリ確保に再利用されることがある。しかし、異なるサイズのメモリ領域の確保と解放を頻繁に繰り返すと、空き領域が細かく分散し、大きな連続領域として利用しにくくなることがある。このような状態をヒープの**フラグメンテーション**⁸⁷ と呼ぶ。

⁸⁷ 英語の **fragmentation**（断片化）に由来する語であり、ヒープ領域に細かい空き領域が分散して発生する現象を指す。

このようなフラグメンテーションを完全に防ぐことは難しいが、プログラミングの際に、メモリの確保と解放のタイミングや、確保するサイズなどを意識して設計を工夫することで、その影響を抑えることができる場合がある。特に、長時間動作するプログラムや、頻繁にメモリの確保と解放を繰り返すプログラムでは、フラグメンテーションを抑える設計を心掛けることが望ましい。

■ サンプルプログラム

次に示す test021-1.c は、大きなサイズの配列を auto で宣言する（スタック領域に確保する）試みである。

プログラム：test021-1.c

```
1 #include <stdio.h>
2 #define MAXSIZE 268435456
3
4 int main() {
5     int pool[MAXSIZE]; // 大きなサイズ（1GiB）をスタックに確保する試み
6
7     printf("start\n"); // 実行開始メッセージの出力
8     pool[0] = 1; // 先頭要素への代入
9     printf("pool[0] = %d\n", pool[0]);
10
11     pool[MAXSIZE-1] = MAXSIZE-1; // 末尾の要素への代入
12     printf("pool[%d] = %d\n", MAXSIZE-1, pool[MAXSIZE-1]);
13 }
```

このプログラムは、巨大な配列を auto で宣言する試みである。これをコンパイルする処理は正常に終了する⁸⁸が、実行時にスタックが確保できずに異常終了する。（次の例）

例. test021-1.c のコンパイルと実行

```
$ gcc test021-1.c -o test021-1  ←コンパイルは成功する
$ ./test021-1  ←実行すると…
$ ←意図した出力が得られていない（スタック領域を確保できず異常終了）
```

この実行例では、はじめの「printf("start\n");」による出力も行われないまま異常終了している。スタック領域は大きな配列を確保するには向かないことがわかる。

次に示す test021-2.c は、大きなサイズの配列を malloc でヒープ領域に確保する例である。

プログラム：test021-2.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #define MAXSIZE 268435456
5
6 int main() {
7     int *p, c, roffset = RAND_MAX/2;
8
9     p = malloc( sizeof(int)*MAXSIZE ); // 大きな配列（1GiB）の確保
10    if ( p == NULL ) {
11        printf("Can not allocate heap memory.\n"); // 確保できなかった場合
12        return 1; // ここで終了
13    }
14    printf("successfully allocated the heap memory.\n");
15    printf("sizeof(p) : %zu\n", sizeof(p));
16    printf("allocated size : %zu\n", sizeof(int) * (size_t)MAXSIZE);
17
18    srand( time(NULL) );
19    for ( c = 0; c < MAXSIZE; c++ ) { // 配列を乱数で満たす
20        p[c] = rand() - roffset;
21    }
22
23    printf("Head of p (0-23):\n");
24    for ( c = 0; c < 24; c++ ) { // 配列の先頭24要素を出力
25        printf("%10d,", p[c]);
26        if ( (c+1)%6 == 0 ) printf("\n");
27    }
```

⁸⁸処理環境によっては、コンパイルも正常に終了しないことがある。

```

28
29     free(p);                // 配列の記憶域を解放
30     printf("successfully freed the heap.\n");
31 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test021-2.c のコンパイルと実行

\$ gcc test021-2.c -o test021-2 ←コンパイル

\$./test021-2 ←実行

successfully allocated the heap memory.

sizeof(p) : 8

allocated size : 1073741824

Head of p (0-23):

-13017,	3899,	4217,	5115,	6830,	7706,
11192,	-8214,	-479,	15685,	5101,	-4245,
11133,	1234,	9380,	1132,	8013,	-1999,
10336,	242,	11981,	-6724,	2528,	-7342,

successfully freed the heap.

大きな配列のためのメモリを malloc によって動的に確保し、値を代入して出力した後、最後に free で解放している。

9.10 列挙型：名前付き整数の扱い

複数の整数値に対して名前を与え、それらをひとまとまりの集合として扱うための仕組みとして、**列挙型** (enumeration) がある。列挙型は、`enum` キーワードを用いて次の形式で定義する。

書き方： `enum 列挙型名 { 列挙子 1, 列挙子 2, ... };`

ここで列挙子 (enumerator) は、それぞれ整数値に対応付けられる識別子である。

■ 自動的に整数値が割り当てられる場合

列挙子に対して明示的な値の指定を行わない場合、先頭の列挙子に 0 が割り当てられ、以降は 1 ずつ増加する整数値が自動的に割り当てられる。例えば、

```
enum Color { RED, GREEN, BLUE };
```

と定義した場合、各列挙子には次の値が対応付けられる。

列挙子	値	列挙子	値	列挙子	値
RED	0	GREEN	1	BLUE	2

このように、特に指定しなくても連続した整数値が割り当てられる。

■ 明示的に整数値を指定する場合

各列挙子には、`=` を用いて整数値を明示的に指定することもできる。例えば、

```
enum ErrorCode { OK = 0, ERROR = -1, UNKNOWN = 100 };
```

と記述すると、それぞれの列挙子には指定した値が割り当てられる。また、列挙子に値を明示的に与えない場合は、その次の列挙子からは、直前の列挙子の値に 1 を加えた値が自動的に割り当てられる。例えば、

```
enum Number { A = 10, B, C = 20, D };
```

とすると、各列挙子の値は次のようになる。

列挙子	値	列挙子	値	列挙子	値	列挙子	値
A	10	B	11	C	20	D	21

このように、値の指定と自動割り当てを組み合わせる用いることができる。

■ 列挙型の変数宣言

定義した列挙型は、通常の型と同様に変数の型として使用することができる。先の列挙型 `enum Color` の定義の後で、

```
enum Color c;  
c = RED;
```

などと記述して、`c` に値を与えることができる。列挙子には整数値が対応付けられているため、代入や比較などの操作が可能である。

なお、C 言語では、個々の列挙型はそれぞれ別の型として定義されるが、列挙型は整数型の一種であり、整数型との間で変換が行われる⁸⁹。このため、整数値との比較や代入、演算なども可能であり、列挙型によって厳密な型安全性が保証されるわけではない点に注意が必要である。

9.10.1 列挙子の値の割当てに関する注意点

列挙子の値は、必ずしも 0 から連続して増加するとは限らない。途中で値を明示的に指定した場合、その次に値を指定しなかった列挙子には、直前の列挙子の値に 1 を加えた値が割り当てられる。例えば、

```
enum Number { A, B, C = 0, D };
```

と定義した場合、各列挙子の値は次のようになる。

列挙子	値	列挙子	値	列挙子	値	列挙子	値
A	0	B	1	C	0	D	1

このように、列挙子の値は重複することがあり、また、必ずしも単調に増加するとは限らない点に注意が必要である。

⁸⁹C17 では、列挙子そのものは `int` 型である。一方、列挙型が実際にどの整数型と互換になるかは処理系に依存する。

特に、途中で値を指定した場合には、その次の列挙子から、直前の値を基準として1ずつ増加する値が割り当てられる。

▲注意▲ 列挙型を用いる際には、各列挙子の値がどのように割り当てられるかを正確に把握する必要がある。列挙子の値の重複自体は文法上問題とはならないが、値に意味を持たせて利用する場合には注意が必要である。

9.10.2 列挙型を使用する目的に関すること

前節までで述べたように、列挙型は整数値に名前を与える仕組みである。しかし、列挙型の本質的な目的は、単に整数に別名を付けることではない。

プログラミングにおいては、特定の意味を持つ値を複数扱う場面が多く存在する。例えば、処理の状態やエラーの種類、あるいはモードの切り替えといった場面が挙げられる。これらを単なる整数値として扱っていると、プログラム中に数値リテラルの記述が散在し、その意味を読み取ることが困難になる。

このような状況は、プログラムの**可読性**を低下させるだけでなく、修正や機能追加を行う際の**保守性**にも悪影響を及ぼす。列挙型を用いると、意味のある値の集合に対して名前を与え、その意図を明確に表現することができる。これにより、プログラムの可読性が向上するだけでなく、将来的な変更に対しても分かりやすく保守しやすい構造にすることができる。

以下では、整数リテラルを直接用いた場合と、定数マクロや列挙型を用いた場合とを比較し、列挙型を使用する利点について例を挙げて解説する。

■ サンプルケースの概要

例えば、Web のサービスプログラムを実装する場合、クライアントに対して処理結果を通知するために、HTTP ステータスコードを扱う必要がある。HTTP では、処理が正常に完了した場合は 200、指定された資源が存在しない場合は 404、サーバ内部でエラーが発生した場合は 500 など、整数値によって状態が表現される。

このような値は、プログラム中では条件分岐やログ出力などの処理において頻繁に参照されることになる。しかし、これらを単なる整数値として扱っていると、コード中に現れる数値の意味を読み取ることが難しくなる。

以下では、HTTP ステータスを扱う簡単なログ出力関数を例として、整数リテラルを直接用いる場合、定数マクロを用いる場合、および列挙型を用いる場合の記述を比較し、列挙型を用いることの利点について確認する。

次に示す `putLog.1.c` は、HTTP アクセスの処理結果 `status` を受け取って、それに応じたログを標準エラー出力に対して出力する `putLog` 関数を定義するものである。

プログラム：putLog.1.c

```
1 void putLog(int status) {
2     switch (status) {
3         case 200:
4             write(2, "OK\n", 3);
5             break;
6         case 404:
7             write(2, "Not Found\n", 10);
8             break;
9         case 500:
10            write(2, "Server Error\n", 13);
11            break;
12        default:
13            write(2, "Unknown\n", 8);
14            break;
15    }
16 }
```

このコードにおける `putLog` 関数は素朴に記述されており、間違いなく動作するが、`case` ラベルの値が何を意味するかは一見して分かりづらく、`write` 関数を呼び出す記述を見なければ理解しにくい。このような場合は、`#define` によるマクロ定義が有効であり、次の `putLog.2.c` のように実装すると可読性が高まる。

プログラム：putLog_2.c

```
1  #define HTTP_OK          200
2  #define HTTP_NOT_FOUND   404
3  #define HTTP_SERVER_ERROR 500
4
5  void putLog(int status) {    // 仮引数が通常のint型
6      switch (status) {
7          case HTTP_OK:
8              write(2, "OK\n", 3);
9              break;
10         case HTTP_NOT_FOUND:
11             write(2, "Not Found\n", 10);
12             break;
13         case HTTP_SERVER_ERROR:
14             write(2, "Server Error\n", 13);
15             break;
16         default:
17             write(2, "Unknown\n", 8);
18             break;
19     }
20 }
```

このコードは先の putLog_1.c に比べると、case ラベルの記述がわかりやすい。しかし、putLog 関数の引数は何を受け取っているかは、型として明確に表現されているとは言えず（int 型は汎用的過ぎるため）、理解しにくい。

次の putLog_3.c は、enum を応用して putLog 関数を実装している。

プログラム：putLog_3.c

```
1  enum HttpStatus {        // enum型でHTTP関連処理の状態を明確に定義する
2      HTTP_OK = 200,
3      HTTP_NOT_FOUND = 404,
4      HTTP_SERVER_ERROR = 500
5  };
6
7  void putLog(enum HttpStatus status) {    // 仮引数の型が明確
8      switch (status) {
9          case HTTP_OK:
10             write(2, "OK\n", 3);
11             break;
12         case HTTP_NOT_FOUND:
13             write(2, "Not Found\n", 10);
14             break;
15         case HTTP_SERVER_ERROR:
16             write(2, "Server Error\n", 13);
17             break;
18         default:
19             write(2, "Unknown\n", 8);
20             break;
21     }
22 }
```

この記述では、putLog 関数は何を受け取る関数であるかが、列挙型 enum HttpStatus の定義と、仮引数の型として明確に表現されており、値の意味がプログラム構造として記述されている。

注意) #define によるマクロ定義は、定数の定義そのものを目的とする機能ではなく、ソースコードの記述を機械的に置換するための汎用的な仕組みである。一方、本節で解説した列挙型は、意味を持つ整数値の集合を型として表現するためのものである。両者は目的や役割が異なるため、状況に応じて適切に使い分けることが重要である。

9.11 シグナル処理とジャンプ

通常、C 言語のプログラムは上から下に向かって順に処理が実行される⁹⁰。また関数呼び出しにおいては、一旦、他の関数に処理が移るが、その関数の実行が終了すると、呼び出し元に処理が戻る。このように、通常のプログラムの実行の流れは、定められた形式と順序に従って進行する。

しかし実際のプログラムでは、外部からの通知や異常な状況の発生などにより、このような通常の制御の流れとは異なる動作が必要となる場合がある。例えば、ユーザがキーボードから `CTRL` + `C` を入力してプログラムを中断したり、例外的な状況⁹¹が発生した場合には、通常の処理の流れを中断して、別の処理を実行する必要がある。

このような場合に用いられる仕組みとして、C 言語にはシグナルと呼ばれる機構が用意されている。また、通常の関数呼び出しとは異なる方法で処理の流れを移動させる機能として、`setjmp` および `longjmp` が提供されている。

シグナルは、プログラムの実行中に発生する通知の一種であり、外部から送られる場合もあれば、実行中の異常などによって発生する場合もある。シグナルは、他のプログラム（プロセス）に対して送信することも可能である。POSIX 系の環境では、他のプロセスにシグナルを送信するための `kill` 関数⁹²が用意されている。

ここでは、シグナルの取扱いについての基本的な事柄を解説する。

9.11.1 シグナル処理の仕組みと流れ

プログラムの実行中にシグナルが発生すると、登録されたハンドラがある場合には、通常の処理の流れが一時的に中断され、シグナルハンドラ（以後、単に「ハンドラ」と略すことがある）が起動される。シグナル受信前後の処理の流れは概ね次のようなものである。

- 1) プログラムが通常通り実行されている（main 関数など）
- 2) プロセスに対してシグナルが発生する
- 3) 実行中の処理が、その時点で一時中断される
- 4) あらかじめ `signal` 関数で登録されたシグナルハンドラが呼び出される
- 5) ハンドラの処理が終了する
- 6) 中断されていた元の処理に復帰⁹³する

この一連の流れは、あたかも実行中のプログラムに「割り込み」が発生したかのように、別の処理が差し込まれると解釈する（図 11）ことができる。

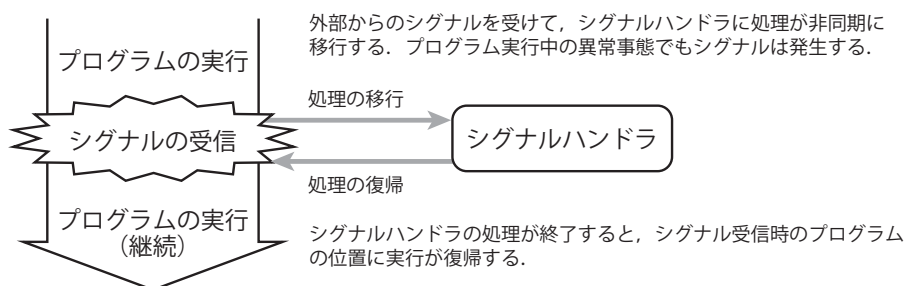


図 11: シグナルを受けてハンドラが起動される様子のイメージ

特に重要なのは、外部から非同期的に発生するシグナルの場合、シグナルハンドラはプログラム側では予測しにくいタイミングで呼び出され得るという点である。すなわち、通常の処理のさまざまな位置で、その時点で処理が中断され、シグナルハンドラに制御が移る可能性がある。

注意) シグナルハンドラの実行後は通常、元の処理に復帰するが、シグナルの発生原因や中断された処理の状態によっては、安全に実行を継続できない場合がある。このような場合には、結果として OS によってプログラムが終了されることがある。

⁹⁰もちろん、条件分岐や反復処理も含む。

⁹¹プログラムが異常終了するような状況。例えば、一般的な OS 環境では、不正なメモリアクセスや 0 による除算などが、シグナルの発生につながることもある。

⁹²これはシグナル送信用の関数であり、プロセスの強制終了のみを目的とするものではない。

⁹³通常は中断された位置の続きから実行される。ただしこれは、ハンドラから正常に復帰可能な場合に限る。

このように、シグナルハンドラは通常の関数呼び出しとは異なり、プログラムの流れの中で明示的に呼び出されるとは限らない。特に、外部から非同期的に発生するシグナルでは、予測しにくいタイミングでハンドラが起動される。このため、ハンドラ内では、実行中のプログラムの状態がどのようになっているかを前提にした、複雑な処理を書くことは危険である。

以上を踏まえると、シグナル処理は「特定のイベントが発生したことを通知する仕組み」として利用し、実際の処理は主プログラム側で行うのが基本的な設計方針となる。

9.11.2 シグナル処理関数：signal

C 言語では、シグナルを受け取った際の動作を設定するために、signal 関数が用意されている。signal 関数のプロトタイプは signal.h にあり、次のように定義されている。

プロトタイプ： `void (*signal( int sig, void (*handler)(int)) )(int);`

この宣言は複雑に見えるが、要点は単純である。第 1 引数 sig には対象とするシグナルの種類を指定し、第 2 引数 handler には、そのシグナルを受け取ったときに呼び出される関数（シグナルハンドラ）へのポインタを指定する。シグナルの種類はヘッダファイル signal.h に定義されており、使用頻度の高いものは後の「9.11.6 使用頻度の高いシグナル」（p.152）で紹介する。（signal 関数の挙動は処理系に依存する部分があるので注意すること）

signal のプロトタイプは、「関数へのポインタを受け取り、関数へのポインタを返す関数」であるため一見して複雑である。これは C 言語における「関数へのポインタ」の表記が入り組んでいるためである。現段階では、signal は「シグナルの種類と、その際の処理方法を対応付けるための関数」と解釈すると良い。

signal 関数は、正常に設定できた場合には、それまで設定されていたシグナルの処理方法を返す。以前の設定がハンドラ関数であればその関数へのポインタが返され、既定動作であれば SIG_DFL、無視する設定であれば SIG_IGN が返される。設定に失敗した場合には SIG_ERR が返される。

シグナルハンドラとしてユーザが定義する関数は、int 型の引数を 1 つ受け取り、戻り値を持たない形式でなければならない。すなわち、次のような関数である。

シグナルハンドラ： `void 関数名 (int sig);`

「関数名」は任意である。また引数 sig には、発生したシグナルを表す整数値（シグナル番号）が渡される。通常は SIGINT など、signal.h に定義されたシグナル定数と比較して判定する。

9.11.3 シグナル処理のための変数

シグナルハンドラと主プログラムの間で共有する変数には、volatile sig_atomic_t 型を用いるのが望ましい。sig_atomic_t は、シグナルハンドラとの間で単純な読み書きを安全に行うために用意された整数型である。また volatile は、その変数の値が通常の処理の流れとは独立したタイミングで変更される可能性があることをコンパイラに知らせるための指定である。

■ サンプルプログラム (1)

次のプログラム test022-1.c は、標準入力から 2 つの整数値を読み込み、加算結果を出力するものであるが、**CTRL** + **C** の操作によって発生するシグナル（SIGINT）を受けてハンドラの関数 handler を起動する。ハンドラ内では、シグナルを受信したことを示すフラグ変数 flg を 1 に設定する。主プログラム側ではこの値を確認し、while ループを終了する。

プログラム：test022-1.c

```
1  #include <stdio.h>
2  #include <signal.h>
3
4  volatile sig_atomic_t flg = 0; // 主プログラムとシグナルハンドラで共有する変数
5
6  void handler(int sig)          // シグナルハンドラ
7  {
8      flg = 1;
9  }
```



```

11 int main() {
12     signal(SIGINT, handler);    // シグナルに対するハンドラを登録
13
14     int    a, b, c;
15
16     while ( !flg ) {
17         printf("enter integer a,b: "); // 入力待ちの表示
18         fflush(stdout);                // 念のための処理
19         if ( scanf("%d %d", &a, &b) != 2 ) { // 2値の入力がなければ
20             break;                    // ループを終了
21         }
22         c = a + b;
23         printf("%d + %d = %d\n", a,b,c);
24     }
25
26     // プログラムが末尾まで到達したことを確認
27     printf("プログラムを安全に終了します. \n");
28 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test022-1.c のコンパイルと実行

```

$ gcc test022-1.c -o test022-1  [Enter]    ←コンパイル
$ ./test022-1  [Enter]          ←実行
enter integer a,b: 2 3  [Enter]    ← 2つの整数値の入力
2 + 3 = 5                ←出力
enter integer a,b:  [CTRL] + [C]    ←シグナル
プログラムを安全に終了します.    ← while ループを終了した

```

注意) 標準入出力ライブラリ (stdio) を用いた入出力待ちの最中にシグナルを受けた場合、中断された処理が終了するか、再開されるかなどの挙動は、処理系や実行環境によって異なることがある。このため、このプログラムが [CTRL]+[C] の操作直後に、上の実行例と同じように終了するとは限らない。

■ サンプルプログラム (2) ハンドラから元の処理への復帰が保証されないケース

プログラムの実行状態に重大な異常が生じたことによって発生するシグナルでは、ハンドラの終了後に主プログラムの処理を安全に継続できない場合がある。次に示すプログラム test022-2.c は、標準入力から得られた2つの整数値を除算するもので、0による整数除算を試みると未定義動作となるが、一般的な環境では SIGFPE シグナルが発生することがある。このような場合、ハンドラから元の処理へ正常に復帰できることは保証されない。

プログラム：test022-2.c

```

1  #include <stdio.h>
2  #include <unistd.h>    // POSIX前提
3  #include <signal.h>
4
5  volatile sig_atomic_t flg = 0;
6
7  void handler(int sig)
8  {
9      flg = 1;
10     write(2, "算術例外\n", 13);
11 }
12
13 int main() {
14     signal(SIGFPE, handler);    // シグナルに対するハンドラを登録
15
16     int    a, b, c;
17
18     while ( !flg ) {
19         printf("enter integer a,b: ");
20         if ( scanf("%d %d", &a, &b) != 2 ) { // 2値の入力がなければ
21             break;                    // ループを終了
22         }
23         c = a / b; // 0除算でSIGFPEの可能性
24         printf("%d / %d = %d\n", a,b,c);
25     }
26 }

```



```

27     printf("プログラムを安全に終了します. \n");
28 }

```

※本例ではハンドラ内からの出力に POSIX の write 関数を使用している。

このプログラムをコンパイルして実行する例を次に示す。

例. test022-2.c のコンパイルと実行 (Windows 上の MSYS2)

```

$ gcc test022-2.c -o test022-2  ←コンパイル
$ ./test022-2  ←実行
enter integer a,b: 6 2  ← 2つの整数値の入力
6 / 2 = 3 ←出力
enter integer a,b:  +   ←入力終了 (UNIX 系の場合は  + )
プログラムを安全に終了します. ←プログラムが正常に終了
$ ./test022-2  ←再度実行
enter integer a,b: 3 0  ← 0 による除算が起こる入力
算術例外 ←ハンドラからの出力 (そこでプログラムが強制終了)

```

この実行例では、0 による除算を契機として SIGFPE のハンドラが起動した後、主プログラムの処理には戻らず、プログラムが終了している。

なお、プログラム内のハンドラでは flg を 1 にしているが、元の処理を安全に継続できなければ while 文の条件判定まで戻ることができない。前のサンプル test022-1.c における SIGINT の例とは、この点が大きく異なる。

注意) 整数の 0 除算は C 言語では未定義動作であり、必ず SIGFPE が発生することが保証されているわけではない。また、実際の計算上の異常によって SIGFPE が発生した場合、そのハンドラから通常の処理へ復帰して実行を継続することは C 言語規格上保証されない。したがって、本例の結果は実行環境に依存する。

9.11.4 ハンドラからの強制的な制御移動

前節までに見たように、シグナルハンドラの処理が終了した後は、通常、元の実行位置に復帰する。しかし、0 による除算のような危険な事態では、ハンドラが起動されても、主プログラムの元の流れへ安全に戻れないことがある。

一方で、C 言語には、通常の実行位置とは異なる方法で制御を移動させるための機能として、setjmp および longjmp が用意されている。これらを用いると、あらかじめ記録しておいた位置へ、通常の実行位置の復帰経路をたどらずに制御（実行位置）を移すことができる。なお、これらをシグナルハンドラから使用する場合には特別な注意が必要であり、一般的なシグナル処理では安易に使用すべきではない。

ただし、このような方法は、関数呼び出しの対応関係を見失って制御を移動させるものであり、プログラムの状態が不整合になる可能性がある。従って、一般的には危険であり推奨されない。ここでは、このような仕組みが存在することを理解するために、基本的な使用方法を紹介しておく。

setjmp は、後で復帰するための実行環境を記録するためのマクロであり、longjmp は、その記録された実行環境へ制御を移す関数である。

通常、setjmp は 0 を返すが、対応する longjmp が呼び出された場合には、setjmp は 0 以外の値を返したものととして復帰する。この性質を利用すると、異常発生時に通常の制御の流れを中断し、別の処理へ移ることができる。

setjmp および longjmp の宣言は ヘッダファイル setjmp.h にあり、次のように定義されている。

```

プロトタイプ: int setjmp( jmp_buf env );
プロトタイプ: void longjmp( jmp_buf env, int val );

```

ここで、jmp_buf は、後で制御を戻すために必要な実行環境を保持するための型であり、setjmp によってその内容が設定される。その後、longjmp に同じ jmp_buf 型変数を渡すと、記録された位置へ制御が戻る。また、longjmp の第 2 引数に与えた値が、復帰を受けた setjmp の戻り値となる。ただし、この値に 0 を指定した場合には、setjmp は 1 を返したものととして動作する。

注意) setjmp を実行した関数が既に終了している場合には、そのとき記録した実行位置へ longjmp で戻るとはできない。

■ サンプルプログラム

setjmp, longjmp を用いて、ハンドラから別の実行位置へ強制的に制御を移す例をプログラム test022-3.c に示す。

プログラム：test022-3.c

```
1  #include <stdio.h>
2  #include <unistd.h>      // POSIX前提
3  #include <signal.h>
4  #include <setjmp.h>
5
6  jmp_buf      env;        // ハンドラから復帰するための情報
7
8  void handler(int sig)
9  {
10     write(2, "算術例外\n", 13);
11     longjmp(env, 1);      // setjmp(env) の部分に強制的に移る
12 }
13
14 int main() {
15     signal(SIGFPE, handler);
16
17     int a, b, c;
18
19     if (setjmp(env) != 0) { // handlerから戻ってくるポイント
20         // longjmpで戻ってきた場合
21         printf("ループを抜け、終了します. \n");
22         return 0;
23     }
24
25     while (1) {
26         printf("enter integer a,b: ");
27         if ( scanf("%d %d", &a, &b) != 2 ) { // 2値の入力がなければ
28             break;                          // ループを終了
29         }
30         c = a / b; // 0除算でSIGFPEの可能性
31         printf("%d / %d = %d\n", a, b, c);
32     }
33
34     printf("通常終了します. \n");
35 }
```

このプログラムでは、シグナル SIGFPE の受信によって handler が起動し、その後、ハンドラ内の longjmp によって、main 関数内で setjmp により記録しておいた実行環境へ強制的に制御が移る。

このプログラムをコンパイルして実行する例を次に示す。

例. test022-3.c のコンパイルと実行 (Windows 上の MSYS2)

```
$ gcc test022-3.c -o test022-3 [Enter]    ←コンパイル
$ ./test022-3 [Enter]    ←実行
enter integer a,b: 4 2 [Enter]    ← 2つの整数値の入力
4 / 2 = 2    ←出力
enter integer a,b: [CTRL] + [Z] [Enter]    ←入力終了 (UNIX 系の場合は [CTRL] + [D])
通常終了します。    ←プログラムが正常に終了
$ ./test022-3 [Enter]    ←再度実行
enter integer a,b: 1 0 [Enter]    ← 0 による除算が起こる入力
算術例外    ←ハンドラからの出力
ループを抜け、終了します。    ←復帰先の if 文のブロック内での出力処理
```

注) 0 による整数除算で SIGFPE が発生し、このように処理できることは保証されない。

▲重要▲

これは、シグナルハンドラから longjmp を用いて制御を移動できる場合があることを示すための実験的な例である。このような処理は、標準 C の範囲で一般的に安全性が保証された使用方法ではなく、実行環境やシグナル発生時の状態にも強く依存する。従って、一般的な異常処理の方法として推奨されない。

9.11.5 シグナルを他のプログラムに通知する方法

シグナルは、プログラムが外部から受信するだけでなく、他のプログラム（プロセス）に対して送信することもできる。POSIX 系の環境では、このような処理を行うための関数として、kill が用意されている。kill 関数のプロトタイプは signal.h にあり、次のように定義されている。

プロトタイプ： `int kill( pid_t pid, int sig );`

第 1 引数 pid にはシグナルを送信する対象のプロセス ID を指定し、第 2 引数 sig には送信するシグナルの種類を指定する。この関数を呼び出すと、指定したプロセスに対してシグナルが送信される。例えば、

```
kill(1234, SIGTERM);
```

を実行すると、プロセス ID が 1234 のプロセスに対して SIGTERM シグナルが送信される。

この関数は、正常に送信できた場合には 0 を返し、失敗した場合には -1 を返す。

なお、関数名 kill は「プロセスを終了させる」という処理を連想させるが、この関数は単にシグナルを送信するためのものであり、必ずしもプロセスの終了を目的とするものではない。送信するシグナルの種類によっては、受信側のプロセスは終了せずに処理を継続することもある。

また、他のプロセスへのシグナル送信には適切な権限が必要であり、一般には同一ユーザが実行しているプロセスなどに対して送信することができる。権限がない場合には kill の呼び出しは失敗する。

参考) 第 1 引数 pid に特定の負の値を指定すると、プロセスグループに属する複数のプロセスへシグナルを送信することもできる。pid の値には他にも特別な意味が定められているが、本書では詳細は割愛する。

他のプロセスに対して送信するシグナルは、その目的に応じて下記のように分類できる。

■ 終了に関するもの：SIGINT, SIGTERM, SIGKILL

プログラムの終了を指示するためのシグナルである。SIGINT および SIGTERM の既定動作はプロセスの終了であり、これらのシグナルに対する処理方法を変更していない場合には、受信するとプログラムは終了する。また、SIGKILL はハンドラで捕捉したり無視したりすることができず、受信したプロセスは強制的に終了する。

■ 実行状態の制御に関するもの：SIGSTOP, SIGCONT

プロセスの実行を一時停止したり、再開したりするためのシグナルである。SIGSTOP はハンドラで捕捉したり無視したりすることができず、受信するとプロセスの実行が強制的に停止される。一方、SIGCONT は一時停止していたプロセスを再開させるシグナルであり、ハンドラを設定している場合には、再開後に対応するハンドラが実行される。

■ ユーザ定義の通知：SIGUSR1, SIGUSR2

用途を自由に定めることができる汎用的な通知用シグナルであり、プロセス間で簡単な合図を送るために用いられる。ただし、これらのシグナルの既定動作はプロセスの終了であるため、通知用として使用する場合には受信側で適切なハンドラを設定する必要がある。

9.11.6 使用頻度の高いシグナル

実際のプログラミングで使用することの多いシグナルを、表 25、26 に挙げる。これらのシグナルは厳密に 2 種に分類できるものではないが、理解しやすくするため、主として自プロセスの実行を契機として発生するものと、外部から通知されることの多いものに分けて示す。

表 25: 主に自プロセスの実行を契機として発生するシグナル

シグナル	意味	説明
SIGSEGV	記憶保護違反	不正なメモリアクセスなどで発生
SIGFPE	算術例外	0 除算など（発生しない場合もある）
SIGABRT	異常終了	abort() により発生
SIGPIPE	パイプ異常	読み取り側が存在しないパイプなどへの書き込みで発生
SIGALRM	タイマ通知	alarm() などで設定した時間の経過により発生

表 26: 主に外部から通知されるシグナル

シグナル	意味	説明
SIGINT	割り込み	<code>CTRL</code> + <code>C</code> による中断
SIGQUIT	終了（既定動作ではコアダンプ）	<code>CTRL</code> + <code>\</code> による終了
SIGTERM	終了要求	終了を要求するための通知
SIGKILL	強制終了	捕捉・無視不可，強制終了
SIGSTOP	一時停止	捕捉・無視不可，強制停止
SIGCONT	再開	停止状態からの復帰
SIGUSR1	ユーザ定義	任意の用途の通知（既定動作は終了）
SIGUSR2	ユーザ定義	任意の用途の通知（既定動作は終了）

POSIX 系の環境では，この他にも SIGHUP（端末切断や設定再読み込みの通知など），SIGCHLD（子プロセスの状態変化の通知）などがよく用いられる．

■ 参考

POSIX 系の環境では，実用的なシグナル処理には `signal` よりも `sigaction` 関数が用いられることが多い．`sigaction` では，ハンドラ実行時の動作やシグナルマスクなどを，より明確に制御することができる．また，移植性や動作の明確さの点でも扱いやすい．

なお，`sigaction` は POSIX の機能であり，MSVC の C ランタイムでは提供されていない．Windows 環境では，`signal` や Windows 固有の例外・コンソール制御機構などを用途に応じて使用する．

9.12 日付、時刻の扱い

プログラムにおいて日付や時刻を扱うためには、「時刻とは何か」をどのように表現するかを理解する必要がある。日常生活では、「2026 年 4 月 17 日 10 時 30 分 41 秒」といった形で、年月日と時刻を組み合わせて表現するが、このような形式は人間にとっては直感的である一方で、計算機にとっては扱いにくい。

例えば、2 つの時刻の差を求める場合、年月日・時分秒の各要素を考慮しながら計算する必要がある、処理が複雑になる。このため計算機では、時刻を基準時刻からの経過時間など、数値として表現する方法が広く用いられる。

9.12.1 UNIX 時間

現在の一般的な OS 環境では、時刻を表現する方法として **UNIX 時間** が広く用いられている。これは、「1970 年 1 月 1 日 00:00:00 (UTC)」(**UNIX エポック**) を基準とし、そこからの経過時間を秒単位で表すものである。UNIX 系オペレーティングシステムをはじめ、多くの環境でこの形式の時刻表現が採用されている。

注意) C 言語規格では、時刻の基準として UNIX 時間を使用することを要求していない。ただし、現在の一般的な処理系では UNIX 時間に基づく実装が広く用いられている。

UNIX 時間では、ある時刻は単に「UNIX エポックからの経過秒数」という整数値として表される。例えば、基準時刻の 1 秒後は 1、1 日後は 86400 ($= 60 \times 60 \times 24$) となる。このような表現を用いることで、時刻の差分計算や大小比較を単純な整数演算として扱うことができる。これは計算機における時間処理を大幅に単純化するものである。

一方で、この表現は人間にとって直感的ではないため、実際のプログラムでは、この値を「年・月・日・時・分・秒」といった形式に分解して扱う必要がある。C 言語では、そのための仕組みとして専用のデータ構造および関数群が用意されている。

なお、ここで説明した UNIX 時間は秒単位の整数値として定義されるが、実際のシステムにおいては、より高い精度で時刻を扱う必要がある場合も多い。このため、環境によっては、ミリ秒、マイクロ秒、ナノ秒といった、より細かい時間単位で時刻を表現するための仕組みが提供されている。

これらは通常、秒単位の値に加えて、その端数部分を別の変数として保持する形式で実現される。例えば、「秒」と「ナノ秒」を組み合わせて時刻を表すといった方法が用いられる。このような高精度な時刻表現は、処理時間の測定や、高速なイベントの記録などにおいて重要である。

9.12.2 時間を取り扱うための基本的な機能

前節で述べたように、現在の一般的な処理系では、時刻を UNIX 時間 (UNIX エポックを基準とする秒単位の値) として扱う実装が広く用いられている。C 言語の標準ライブラリには、時刻を扱うための基本的な型や関数が用意されており、ヘッダファイル `time.h` にそれらが宣言されている。

■ `time_t` 型

時刻を表すための型として、`time_t` 型が定義されている。この型の具体的な表現方法は処理系に依存する。現在の一般的な処理系では整数型として実装され、UNIX 時間に相当する秒単位の値を保持するものが多い。

主な計算機環境における `time_t` の仕様を表 27 に挙げる。

表 27: `time_t` 型の代表的な実装例

環境	代表的な実装	備考
古い 32bit 環境	32bit 符号付き整数型	2038 年問題の対象
一般的な 64bit Unix 系環境	64bit 整数型	Linux などで広く使用
現在の MSVC (既定)	64bit 符号付き整数型	<code>_time64_t</code> 相当
POSIX 環境	整数型	UNIX エポックを基準とする秒単位の値

■ 現在時刻の取得

現在の時刻を取得するための関数として、`time` が用意されている。この関数のプロトタイプは `time.h` にあり、使用する際はこれをインクルードする。

プロトタイプ: `time_t time( time_t *t );`

この関数は、現在の暦時刻を `time_t` 型の値として返す。一般的な処理系では、この値は UNIX 時間に相当する秒単位の値として実装されている。

引数 `t` には、取得した時刻を格納するための `time_t` 型変数のアドレスを指定することができる。この場合、関数の戻り値として返される時刻と同じ値が、その変数にも書き込まれる。一方、単に現在時刻を取得したいだけであれば、引数に `NULL` を指定する。時刻を取得できない場合には `(time_t)-1` (`time_t` 型の-1) を返す。

なお、`time` 関数は一般的な実装では秒単位の時刻取得に用いられ、高精度な時刻測定には適さない。より細かい時間単位が必要な場合には、`time.h` の他の機能や、OS が提供する機能などを利用する。

■ 年月日時分秒への分解

`time_t` の値を人間にとって理解しやすい形式に変換するためには、これを「年・月・日・時・分・秒」に展開する必要がある。このために構造体 `struct tm` が用意されている。`struct tm` は、時刻を各要素に分解して保持する構造体であり、代表的なメンバを表 28 に示す。

表 28: `struct tm` の主なメンバ

メンバ名	内容	値の範囲	メンバ名	内容	値の範囲
<code>tm_year</code>	年	—	<code>tm_hour</code>	時	0~23
<code>tm_mon</code>	月	0~11	<code>tm_min</code>	分	0~59
<code>tm_mday</code>	日	1~31	<code>tm_sec</code>	秒	0~60

注) `tm_year` は西暦 1900 年からの経過年数である。

注) `tm_mon` は 0 から始まることに注意。(0 は 1 月を意味する)

注) `tm_sec` は閏秒を扱うために 60 となることがある。

`time_t` 型で表された時刻を `struct tm` 構造体の各メンバに展開するための関数として、`localtime` および `gmtime` が用意されている。これらの関数のプロトタイプは `time.h` にあり、使用する際はこれをインクルードする。

プロトタイプ: `struct tm *localtime( const time_t *t );`

プロトタイプ: `struct tm *gmtime( const time_t *t );`

これらの関数は、いずれも引数 `t` が指す `time_t` 型の時刻を `struct tm` 型のデータに分解し、そのポインタを返す。また、時刻を変換できない場合には `NULL` を返す。

`localtime` は実行環境で設定されている**タイムゾーン**⁹⁴ やサマータイムの情報に基づいて変換を行った結果を返す。一方、`gmtime` は**協定世界時 (UTC)** に基づいて変換を行う。

例えば、同じ `time_t` 型の時刻に対して `localtime` と `gmtime` を適用すると、環境のタイムゾーンによっては、両者で異なる時刻が得られることがある。

重要) `localtime`、`gmtime` が返すポインタは、処理系が管理する `static` な領域を指す。この領域は、これらの時刻変換関数を後から呼び出すことによって内容が上書きされる可能性がある。そのため、値を保持しておく必要がある場合には、構造体の内容を別の変数にコピーしておく必要がある。

9.12.2.1 日付、時刻の書式整形

`struct tm` により表された時刻を、人間が読みやすい文字列に変換するための関数として、`strftime` が用意されている。この関数のプロトタイプは `time.h` にあり、使用する際はこれをインクルードする。

プロトタイプ: `size_t strftime( char *s, size_t max, const char *format, const struct tm *tm );`

この関数は、引数 `tm` に指定された時刻情報を、書式文字列 `format` に従って文字列に変換し、その結果を文字配列 `s` に格納する。このとき、最大で `max` 文字分の領域を使用して書き込む。

注意) 出力先のバッファ `s` は十分な大きさを確保しておく必要がある。

変換結果が `max` 文字分の領域に収まった場合には、生成された文字列の長さ（終端のヌル文字を除く）を返す。収まらない場合には 0 を返す。

書式文字列 `format` には、変換指定子を用いることで、年・月・日・時・分・秒などを任意の形式で出力することができる。代表的な指定子を表 29 に示す。

⁹⁴ 当該タイムゾーンにおける時刻を**ローカル時刻**という。

表 29: strftime の主な書式指定子

指定子	内容	指定子	内容
%Y	西暦年 (通常 4 桁)	%H	時 (00~23)
%m	月 (01~12)	%M	分 (00~59)
%d	日 (01~31)	%S	秒 (00~60)

例えば, format に "%Y-%m-%d %H:%M:%S" を指定すれば, 「2026-04-17 10:30:41」のような形式の文字列を生成することができる。

参考) 表 29 に挙げたもの以外にも, 変換結果の文字列に曜日を盛り込む指定子などもある。また変換結果に「%」の文字自体を含めるには「%%」と記述する。

このように, strftime を用いることで, struct tm による時刻情報を, 用途に応じた形式の文字列として柔軟に整形することができ, タイムスタンプの文字列を作成するのに役立つ。

9.12.2.2 日付, 時刻の文字列からの読取り

文字列として表された日付や時刻を struct tm 型のデータに変換するための関数として, strptime が用意されている。ただしこの関数は, C 言語の標準ではなく, 主に POSIX (Linux, macOS など) 環境で利用できる。

この関数のプロトタイプは time.h にあり, 使用する際はこれをインクルードする。

プロトタイプ: char *strptime(const char *s, const char *format, struct tm *tm);

この関数は, 引数 s に指定された文字列を, 書式文字列 format に従って解析し, その結果を struct tm 型の変数 tm に格納する。

解析に成功した場合は, 文字列中で解析を終えた位置を指すポインタを返し, 解析できなかった場合には NULL を返す。従って, 文字列の末尾まで正常に解析できたかどうかを確認することもできる。

書式文字列 format には, strftime とよく似た変換指定子を用いることができる。これにより, 文字列中の年・月・日・時・分・秒を対応するメンバに読み取ることができる。

例えば, 文字列 "2026-04-17 10:30:41" を解析するには, 書式として "%Y-%m-%d %H:%M:%S" を指定する。

このように, strptime を用いることで, 文字列として与えられた日付・時刻を, struct tm 型のデータとして扱うことができる。

注意) strptime は, 書式文字列で指定されていない struct tm のメンバまで初期化するとは限らない。そのため, 解析先の構造体はあらかじめ 0 で初期化しておくことが安全である。

例. strptime 使用前の構造体の初期化

```
struct tm t = {0};
strptime("2026-04-18 18:30:41", "%Y-%m-%d %H:%M:%S", &t);
```

▲重要▲

strptime は標準 C には含まれず, 主に POSIX 環境で利用可能な関数である。そのため, 処理系によっては利用できない場合がある点に注意する必要がある。なお, MSVC の C ランタイムでは strptime は提供されていない。

また, 環境によっては, strptime の宣言を有効にするために, _XOPEN_SOURCE などの機能選択マクロを定義する⁹⁵必要がある。

参考) _XOPEN_SOURCE に定義する値

値	規格の世代
500	SUSv2 / POSIX.1c 世代
600	POSIX.1-2001 / SUSv3 世代
700	POSIX.1-2008 / SUSv4 世代

⁹⁵time.h をインクルードする前に定義する必要がある。

■ サンプルプログラム： 基本機能の確認

現在時刻の取得と、ローカル時刻および UTC の構造体への変換、それらの書式整形の働きを確認するプログラム test023-1.c を次に示す。

プログラム：test023-1.c

```
1 #include <stdio.h>
2 #include <time.h>
3 #define BUFSIZE 128
4
5 int main() {
6     time_t now; // 時刻を保持する変数
7     struct tm lt, gt, *tmp; // 日付・時刻情報のための構造体
8     char buf[BUFSIZE]; // 日付・時刻のための文字配列
9
10    now = time(NULL); // 現在時刻の取得
11
12    tmp = localtime(&now); // ローカル時間として日付・時刻情報を構造体に展開
13    lt = *tmp; // 内容をltに複製
14    // 構造体の各メンバを確認
15    printf("local)\tyear:%d, month:%d, day:%d,\n\thour:%d, min:%d, sec:%d\n",
16           lt.tm_year, lt.tm_mon, lt.tm_mday,
17           lt.tm_hour, lt.tm_min, lt.tm_sec);
18    strftime(buf, BUFSIZE, "%Y-%m-%d %H:%M:%S", &lt); // 書式整形(タイムスタンプ)
19    printf("local)\t%s\n", buf); // 出力
20
21    tmp = gmtime(&now); // UTCとして日付・時刻情報を構造体に展開
22    gt = *tmp; // 内容をgtに複製
23    strftime(buf, BUFSIZE, "%Y-%m-%d %H:%M:%S", &gt); // 書式整形(タイムスタンプ)
24    printf("UTC)\t%s\n", buf); // 出力
25 }
```

注意) この例では処理を分かりやすくするため、time, localtime, gmtime, strftime のエラー判定は省略している。実用的なプログラムでは、それぞれの戻り値を確認する必要がある。

これをコンパイルして実行する例を次に示す。

例. test023-1.c のコンパイルと実行

```
$ gcc test023-1.c -o test023-1  ←コンパイル
$ ./test023-1  ←実行
local) year:126, month:3, day:18, ←構造体の各メンバの値を
      hour:16, min:22, sec:29      出力 (ローカル時刻)
local) 2026-04-18 16:22:29 ←書式整形した文字列 (ローカル時刻)
UTC) 2026-04-18 07:22:29 (同, UTC)
```

これは日本国内での実行例であり、ローカル時刻と UTC との間に 9 時間の差が見られる。

注意) tm_year の値が 126 となっているが、これは西暦 1900 年からの経過年数である。また、この例の実行時は 4 月であるが、tm_mon の値は 3 となっている。これは月の値が 0 から始まるためである。実用的なプログラミングにおいては、これらのことを常に意識して、適切に変換処理しなければならない。

■ サンプルプログラム： 文字列からの日付・時刻情報の読み取り (一部環境限定)

次のプログラム test023-2.c は、タイムスタンプの文字列から年月日・時分秒の情報を読み取り、それらを struct tm のメンバに設定する例である。

プログラム：test023-2.c

```
1 #define _XOPEN_SOURCE 700
2 #include <stdio.h>
3 #include <time.h>
4
5 int main()
6 {
7     const char *str = "2026-04-18 18:30:41"; // タイムスタンプ文字列
8
9     struct tm t = {0}; // 初期化しておく
10
11     if (strptime(str, "%Y-%m-%d %H:%M:%S", &t) == NULL) {
```

```

12         printf("変換に失敗しました\n");
13         return 1;
14     }
15
16     printf("year:%d, ", t.tm_year + 1900);
17     printf("mon:%d, ", t.tm_mon + 1);
18     printf("day:%d, ", t.tm_mday);
19     printf("hour:%d, ", t.tm_hour);
20     printf("min:%d, ", t.tm_min);
21     printf("sec:%d\n", t.tm_sec);
22 }

```

これをコンパイルして実行する例を次に示す。

例. test023-2.c のコンパイルと実行 (Linux 環境下の gcc)

\$ gcc test023-2.c -o test023-2 ←コンパイル

\$./test023-2 ←実行

year:2026, mon:4, day:18, hour:18, min:30, sec:41 ←構造体の内容を反映した文字列

9.12.3 高精度の時間計測

先に解説した time 関数は、一般的な実装では秒単位の時刻取得に用いられるため、短い処理の実行時間などを高い分解能で測定する用途には適さない。しかし、POSIX や Windows などの環境では、高分解能の時計を利用するための機能が提供されており、それらを用いることで、ミリ秒、マイクロ秒、ナノ秒といった時間尺度で経過時間を扱うことができる。

巻末付録「B.2 高精度の時間計測」(p.514) には、POSIX, Windows の両方で高精度の時間計測を行うための具体的な方法を例示しているので、必要に応じて参照されたい。

9.13 プログラムの一時停止

C 言語によるプログラミングでは、指定した時間の間、プログラムを一時停止するための機能を利用することができる。このような機能は、一見すると単に処理を止めるだけのもののように見えるが、実用的なプログラムにおいては重要な役割を持つ。

例えば、入出力や通信処理では、相手側の処理速度や応答時間などに応じて、処理のタイミングを調整する必要がある場合がある。このような場合、処理の途中で一定時間待機することが、安定した動作につながる場合がある。

また、一定間隔で処理を繰り返すようなプログラムにおいても、時間の制御は重要である。例えば、ログの監視やファイルの更新確認など、周期的に状態を確認する処理では、各処理の間に適切な待機時間を設けることで、無駄な処理を抑えつつ効率的に動作させることができる場合がある。

さらに、外部資源へのアクセスに失敗した場合に、直ちに再試行を行うのではなく、一定時間待機してから再試行することで、システム全体への負荷を軽減し、安定した動作につなげることができる場合がある。

このほかにも、処理の実行頻度を制限することで CPU 資源の無駄な消費を防いだり、画面表示の更新速度を人間が認識しやすい程度に調整したりする目的でも、一時停止の機能は広く利用されている。

このように、プログラムの一時停止は、単に処理を止めるためだけのものではなく、処理のタイミングを制御するための重要な手段である。本節では、このような一時停止の機能について、その基本的な考え方と利用方法を説明する。

9.13.1 POSIX 環境での方法

POSIX 環境では、指定した秒数の間、プログラムの実行を一時停止するために、`sleep` 関数を用いることができる。この関数のプロトタイプはヘッダファイル `unistd.h` において宣言されているので、使用する際はこれをインクルードする。

プロトタイプ： `unsigned int sleep( unsigned int seconds );`

この関数は、引数 `seconds` に指定した秒数の間、呼び出し元のプログラムを一時停止する。例えば、`sleep(1)` とすると、プログラムは約 1 秒間停止する。

`sleep` 関数は、指定した時間が経過するまで処理を停止するが、シグナルなどにより途中で中断される場合がある。この場合、戻り値として残りの秒数が返される。正常に指定時間の待機が完了した場合は 0 が返される。

`sleep` 関数は、秒単位での簡単な待機処理を記述する場合に適しているが、より細かい時間指定が必要な場合には `nanosleep` 関数を用いる。この関数のプロトタイプはヘッダファイル `time.h` において宣言されているので、使用する際はこれをインクルードする。

プロトタイプ： `int nanosleep( const struct timespec *req, struct timespec *rem );`

この関数は、引数 `req` に指定した時間だけ、プログラムの実行を一時停止する。待機時間は、`struct timespec` によって秒 (`tv_sec`) とナノ秒 (`tv_nsec`) の組で指定する。また、この関数は、正常に指定時間の待機が完了した場合には 0 を返し、途中で中断された場合などには -1 を返す。シグナルによる中断時に第 2 引数 `rem` が NULL でなければ、残りの待機時間がそこに格納される。

例えば、1 秒間待機する場合には、次のように指定する。

```
struct timespec ts = {1, 0};    // 1 秒と 0 ナノ秒
nanosleep(&ts, NULL);
```

注意) `tv_nsec` には 0 以上 1000000000 未満の値を指定する。

`sleep`、`nanosleep` 関数を使用した例を次の `test025-2.c` に示す。このプログラムは、入力された整数値の回数だけカウントダウンを表示 (1 秒間隔) して、"done" と表示する。

プログラム： `test025-2.c`

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <time.h>
4
5 int main() {
6     int    n;
```

```

7
8     printf("input a number: ");      fflush(stdout);
9     if (scanf("%d", &n) != 1) {
10         printf("invalid input\n");
11         return 1;
12     }
13
14     printf("countdown start\n");
15
16     for (int i = n; i > 0; i--) {
17         printf("%d ", i);      fflush(stdout);
18
19         //--- 秒単位の待機 ---
20         sleep(1); // 1秒待機
21
22         //--- 高精度(ナノ秒単位)版 (必要に応じて切り替え) ---
23         //struct timespec ts = {1, 0}; // 1秒
24         //nanosleep(&ts, NULL);
25     }
26
27     printf("\ndone\n");
28 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test025-2.c のコンパイルと実行

```

$ gcc test025-2.c -o test025-2  ←コンパイル
$ ./test025-2  ←実行
input a number: 5  ←「5回」のカウントダウンを指定
countdown start
5 4 3 2 1          ← 1秒間隔でカウントダウンが表示される
done              ←終了

```

test025-2.c の sleep 関数の呼び出しをコメントにし、その直後にある nanosleep 用の 2 行のコメントを外すと、nanosleep による一時停止が実現できるので試されたい。

9.13.2 Windows 環境での方法

Windows 環境では、指定した時間の間、実行中のスレッドを一時停止するために、Sleep 関数を用いる。この関数のプロトタイプはヘッダファイル windows.h で宣言されているので、使用する際はこれをインクルードする。

プロトタイプ: void Sleep(DWORD dwMilliseconds);

この関数は、引数 dwMilliseconds に指定した時間 (ミリ秒単位) の間、呼び出したスレッドの実行を一時停止する。例えば、Sleep(1000) とすると、約 1 秒間停止する。

ここで用いられている DWORD は、Windows API において定義されている整数型であり、32 ビットの符号なし整数 (unsigned long に相当)⁹⁶ である。

Sleep 関数による実際の待機時間は、システムタイマの分解能やスレッドのスケジューリングなどの影響を受けるため、指定した時間と厳密に一致するとは限らない。

次に示すプログラムは、先の test025-2.c と同等の機能を Windows 環境で実現する例である。(出力も test025-2.c と同様)

プログラム: test025-1.c

```

1  #include <stdio.h>
2  #include <windows.h>
3
4  int main() {
5      int      n;
6
7      printf("input a number: ");      fflush(stdout);
8      if (scanf("%d", &n) != 1) {
9          printf("invalid input\n");

```

⁹⁶uint32_t の型が策定される以前の時代のものである。

```

10     return 1;
11 }
12
13 printf("countdown start\n");
14
15 for (int i = n; i > 0; i--) {
16     printf("%d ", i);    fflush(stdout);
17     Sleep(1000); // 1000 ミリ秒 = 1秒
18 }
19
20 printf("\ndone\n");
21 }

```

9.13.3 待ち時間を実現する際の推奨されない方法

プログラムにおいて一定時間の待機を実現する方法として、経過時間を繰り返し確認するループを用いる方法が考えられる。このような方法は実装が容易であるため、実際に用いられることも多い。

例えば、現在時刻を取得しながら一定時間が経過するまでループを回し続けることで、待機時間を実現することができる。しかし、この方法は一般には推奨されない。

次に示す例 test025-3.c は、このような方法により待機時間を実現したものである。

プログラム：test025-3.c

```

1  #include <stdio.h>
2  #include "dbl_now.h"
3
4  int main() {
5      int                n;
6      unsigned long long r = 0;
7      double             t1, t2;
8
9      printf("input a number: ");    fflush(stdout);
10     if (scanf("%d", &n) != 1) {
11         printf("invalid input\n");
12         return 1;
13     }
14
15     printf("countdown start\n");
16
17     for (int i = n; i > 0; i--) {
18         printf("%d ", i);    fflush(stdout);
19         // CPUを占有し続ける非効率な待機 (busy waiting)
20         t1 = now();          // 基準時刻
21         while ( 1 ) {
22             t2 = now();
23             if ( t2-t1 >= 1.0 ) break; // 経過時間が1秒を超えたら忙待ちループ終了
24             r++;                     // *この間もCPUは全力で動作し続ける*
25         }
26     }
27
28     printf("\ndone\ntotal wait round:%llu\n",r);
29 }
30

```

このプログラムは、巻末付録「B.2 高精度の時間計測」(p.514) に挙げたプログラム dbl_now.c の now 関数を用いたものであり、now 関数を while ループで繰り返し実行する形で1秒の経過を検査している。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test025-3.c のコンパイル、リンクと実行

```

$ gcc test025-3.c dbl_now.o -o test025-3  [Enter]    ←コンパイルとリンク
$ ./test025-3  [Enter]    ←実行
input a number: 5  [Enter]    ←「5回」のカウントダウンを指定
countdown start
5 4 3 2 1          ←約1秒間隔でカウントダウンが表示される
done              ←終了
total wait round:287813096    ←膨大な回数の無駄なループが実行されている

```


一見すると、この方法でも正しく待機が行われているように見えるが、実際には待機中もループが高速に実行され続けている。その結果、プログラムは待機している間も CPU の処理能力を消費し続けることになる。

実行結果に示されている `total wait round` の値は、待機中にループが繰り返された回数を表している。この値が非常に大きくなっていることから、待機時間の間に膨大な回数の処理が行われていることが分かる。

このような方法では、待機中も CPU が無駄に動作し続けるため、計算資源が浪費されるだけでなく、他のプログラムの実行にも悪影響を及ぼす⁹⁷。また、消費電力の増加や発熱の原因にもなる。

このように、待機時間をループによって実現する方法は、**ビジーウェイト**（busy waiting, **忙待ち**）と呼ばれ、一般には推奨されない。通常の待機処理を行う場合には、先に示したような、オペレーティングシステムが提供する待機機能を用いるべきである。

⁹⁷これは、オペレーティングシステムが本来持っている「待機中のスレッドを実行対象から外し、他の処理に CPU を割り当てる」という仕組みを利用しないためである。

9.14 コマンドライン引数

UNIX 系のオペレーティングシステムでは、**シェル**と呼ばれるプログラムを介してコマンドを入力し、それを実行するという操作形態が基本となっている。すなわち、ユーザはシェル上でコマンドを入力し、そのコマンドに応じたプログラムが起動される。

C 言語で作成されたプログラムも、このような環境では**コマンド**として実行されることが多い。コンパイルによって生成された実行ファイルは、シェルから起動される一つのコマンドとして振る舞う。

一般に、コマンドは単に起動されるだけでなく、**引数** (argument) を伴って実行されることが多い。例えば、ファイル名や数値などを引数として与えることで、コマンドの振る舞いを外部から指定することができる。

このような引数は、**コマンドライン引数** (command-line arguments) と呼ばれ、プログラムの起動時に外部から渡される情報である。実用的なプログラムを作成するためには、これらの引数を正しく受け取り、適切に処理することが重要となる。

本節では、C 言語においてコマンドライン引数を受け取る方法と、その基本的な扱い方、および利用上の注意点について説明する。

9.14.1 main 関数の引数

C 言語において、**コマンドライン引数**は main 関数の引数としてプログラムに渡される。main 関数は、次のような形式で定義することができる。

```
int main( int argc, char *argv[ ] )
```

ここで、第 1 引数 `argc` (argument count) は、`argv` に格納される文字列の個数を表す整数値であり、第 2 引数 `argv` (argument vector) は、各文字列へのポインタを要素として持つ配列である。すなわち、`argv` は「文字列へのポインタの配列」となっている。また、この記述は次のようにしても良い。

```
int main( int argc, char **argv )
```

`argv[argc]` は NULL ポインタとなることが保証されており、配列の終端を判定するために利用することができる。

コマンドラインに記述された内容は、これらの引数に次のように対応付けられる。例えば、シェル上で次のようにコマンドを実行したとする。

```
$ ./prog apple orange
```

このとき、プログラムには以下のような形で引数が渡される。

<code>argc = 3</code>	コマンドラインに記述された文字列の個数 (<code>argv</code> に格納される文字列の個数)
<code>argv[0] = "./prog"</code>	最初に記述された文字列
<code>argv[1] = "apple"</code>	次に記述された文字列
<code>argv[2] = "orange"</code>	その次に記述された文字列
<code>argv[3] = NULL</code>	(終端)

コマンドラインに指定された引数は、記述された順序のまま `argv` に格納される。すなわち、引数の並び順はそのままプログラムに渡される。ここで注意すべき点は、通常、`argv[0]` には実行されたプログラム自身の名前を表す文字列が格納されるということである。したがって、ユーザが与えた引数は `argv[1]` 以降に格納される。また、`argc` の値は、この `argv[0]` を含めた総数となる。

注意) `argv[0]` に格納される文字列の具体的な形式は実行環境に依存する。プログラムのパスを含む場合や、ファイル名のみの場合などがあり、常に同じ形式になることは保証されない。また、環境によってはプログラム名を取得できず、空文字列となる場合もある。

重要) `argv` の各要素はすべて文字列として扱われるため、数値として利用する場合には、文字列から数値への変換を行う必要がある。

9.14.2 コマンドラインの記述に関すること

本節では、主として UNIX 系環境における代表的なシェル (`bash` など) を前提としてコマンドラインの記述について説明する。シェルの種類によって細部の仕様は異なるが、基本的な動作は多くの環境で共通している。

コマンドラインでは、空白によって引数が区切られる。したがって、空白を含む文字列を1つの引数として扱う場合には、引用符で囲む必要がある。

例. `$ ./prog "hello world"`

また、バックスラッシュ (\) を用いることで、次の文字が持つ特別な意味を打ち消して、通常の文字として扱うことができる場合があり⁹⁸、空白などの特殊な文字をそのまま文字として扱うことができる。

例. `$ ./prog hello\ world`

さらに、シェルでは * や ? などのワイルドカードが展開され、複数のファイル名に置き換えられる。次のような例を考える。

例. `$ ./prog *.txt`

このようなコマンドを入力すると、カレントディレクトリに存在する、名前が .txt で終わるファイルの名前がすべて列挙され、それらが個別の引数としてプログラムに渡される。すなわち、C 言語のプログラム側からは、ワイルドカードそのものではなく、展開後の複数のファイル名が並んだコマンドライン引数として受け取られる。(これらの展開はシェルによって行われる)

注意) ワイルドカードに一致するファイルが存在しない場合の動作は、シェルの種類や設定によって異なることがある。

コマンドラインでは、文字列を引用符で囲むことにより、空白を含む内容を1つの引数として扱うことができる。引用符には、単一引用符 (') と二重引用符 (") の2種類がある。

二重引用符で囲まれた文字列では、シェル変数や環境変数などの展開が行われる。例えば、次のようにすると環境変数 HOME の内容が展開される。

例. `$ ./prog "$HOME"`

一方、単一引用符で囲まれた文字列では、内部の文字はすべてそのまま扱われ、特別な意味を持つ記号も展開されない。

例. `$ ./prog '$HOME'`

この場合はそのまま「\$HOME」という文字列がプログラムに渡される。

■ サンプルプログラム

実行時に与えられた任意の個数のコマンドライン引数を読み取り、数値に変換しながら累積加算するプログラム test024-1.c を次に示す。

プログラム：test024-1.c

```
1  #include    <stdio.h>
2  #include    <stdlib.h>
3
4  int main( int ac, char **av ) {
5      int      c;
6      double   s = 0, a;
7      char     *p;
8
9      printf("program: %s\n", av[0]);      // プログラム名を出力
10     for ( c = 1; c < ac; c++ ) {
11         a = strtod( av[c], &p );          // 引数の文字列をdouble型に変換
12         if ( av[c] != p ) {                // 数値の読み取りができた場合
13             s += a;                        // 累積加算する
14         }
15     }
16     printf("sum: %f\n", s);
17 }
```

注意) 簡単のため、コマンドライン引数として与えられた文字列全体が正しい数値表現であるかどうかの厳密な検査は行わない。main 関数の引数名は argc, argv とするのが一般的であるが、この例のように短く ac, av とすることもある..

このプログラムをコンパイルして実行する例を次に示す。

⁹⁸厳密には、エスケープシーケンスとは異なる。

例. test024-1.c のコンパイルと実行

```
$ gcc test024-1.c -o test024-1  ←コンパイル
$ ./test024-1 1 2 3  ←引数を 3 つ与えて実行
program: test024-1
sum: 6.000000
$ ./test024-1 1 2 3 4 5 6  ←引数を 6 つ与えて実行
program: test024-1
sum: 21.000000
```

この実行例では、コマンドライン引数の最初の要素（av[0] の内容）が"test024-1" となっているが、av[0] に格納される文字列の具体的な形式は実行環境によって異なることがあるので注意されたい。

9.15 プログラムの終了処理

これまでに説明したように、main 関数で return 文を実行すると、main 関数の処理が終了し、その結果としてプログラムも終了する。一方、C 言語には、exit 関数を呼び出してプログラムを終了させる方法も用意されている。

return 文は、現在実行している関数の処理を終了して、その関数の呼び出し元へ戻るための文である。これに対して exit 関数は、呼び出された位置が main 関数の中であるか、他の関数の中であるかにかかわらず、プログラム全体の終了処理を開始する。従って、main 関数の中で使用した場合には、return 文と exit 関数は、どちらもプログラムを終了させる。特に、main 関数から値を return して終了することは、その値を引数として exit 関数を呼び出すことと基本的に同等の結果となる。しかし、他の関数の中で使用した場合には動作が異なる。return 文では、その関数の呼び出し元へ処理が戻るだけであるが、exit 関数では、呼び出し元へ戻ることなく、そのままプログラム全体が終了する。

また、exit 関数による終了では、プログラムを直ちに打ち切るのではなく、終了に伴う所定の後処理が行われる。その一つとして、atexit 関数を用いてあらかじめ登録しておいた関数が呼び出される。main 関数から return 文によって終了した場合にも、同じ終了処理が行われる。

9.15.1 exit 関数によるプログラムの終了

exit 関数を使用すると、呼び出した位置からプログラム全体を終了させることができる。この関数は、ヘッダファイル stdlib.h で宣言されている。

プロトタイプ: void exit(int status);

引数 status には、プログラムの終了状態を表す整数値を指定する。この値は、プログラムの終了状態として、オペレーティングシステムなどの実行環境へ通知される。正常に終了したことを表す場合には EXIT_SUCCESS を、処理の失敗を表す場合には EXIT_FAILURE を指定できる。これらも stdlib.h で定義されている。

例えば、次のように記述すると、プログラムは正常終了として扱われる。

```
exit(EXIT_SUCCESS);
```

また、次のように記述すると、処理に失敗したことを表す終了状態でプログラムが終了する。

```
exit(EXIT_FAILURE);
```

exit 関数は呼び出し元へ戻らず、その位置からプログラム全体の終了処理を開始する。なお、exit 関数による終了では、概ね次の順序で所定の終了処理が行われる。

- atexit 関数で登録された終了処理関数を実行する
- 開いているストリームの未出力データを書き出し、ストリームを閉じる
- tmpfile() で作成した一時ファイルを削除する
- 終了状態 status を実行環境へ渡し、プログラムを終了する

■ atexit 関数による終了処理の登録

atexit 関数を使用すると、プログラムが通常の終了処理によって終了する際に呼び出す関数（終了処理関数）を、あらかじめ登録しておくことができる。atexit 関数は、ヘッダファイル stdlib.h で宣言されている。

プロトタイプ: int atexit(void (*func)(void));

引数 func には、プログラムの終了時に呼び出す関数へのポインタ⁹⁹ を指定する。登録できる関数は、引数を受け取らず、戻り値を返さない、次の形式の関数である。

```
void 関数名(void) {  
    終了時に行う処理  
}
```

atexit 関数による登録に成功すると 0 が返され、登録に失敗すると 0 以外の値が返される。

例えば、次の記述では、cleanup 関数を終了処理関数として登録している。

⁹⁹関数名をそのまま指定すると、その関数へのポインタとして扱われる。

```

void cleanup(void) {
    printf("終了処理を実行します。\\n");
}

int main() {
    if (atexit(cleanup) != 0) {
        fprintf(stderr, "終了処理を登録できませんでした。\\n");
        return EXIT_FAILURE;
    }

    printf("プログラムを実行します。\\n");
    return EXIT_SUCCESS;
}

```

このプログラムでは、main 関数から return 文によって正常終了する際に、登録されている cleanup 関数が呼び出される。また、exit 関数を呼び出して終了した場合にも、cleanup 関数が呼び出される。

atexit 関数によって複数の関数を登録することもできる。この場合、登録された関数は、登録した順序とは逆の順序で呼び出される。例えば、次のように func1、func2 の順に登録した場合には、プログラムの終了時に func2、func1 の順に実行される。

```

atexit(func1);
atexit(func2);

```

atexit 関数で登録された関数は、main 関数からの復帰や exit 関数の呼び出しによって、通常の終了処理が行われる場合に実行される。一方、abort 関数（後述）によってプログラムが異常終了する場合には、登録された関数は実行されない。

参考） C 言語規格では、atexit による終了処理関数を少なくとも 32 個登録できることが保証されている。

■ サンプルプログラム

次に示すプログラム test024-3.c では、終了処理を行う関数 cleanup を atexit 関数で登録する。入力された整数値が負の場合は関数 sub 内で、0 以上の場合は main 関数内で exit を実行する。

main 関数内から終了した場合も、別の関数内から終了した場合も、終了直前に cleanup 関数が実行される様子を見ることができる。

プログラム：test024-3.c

```

1  #include    <stdio.h>
2  #include    <stdlib.h>
3
4  void cleanup(void) {          // 終了処理のための関数
5      printf("--cleanup function\\n");
6  }
7
8  void sub(void) {
9      printf("--sub function\\n");
10     exit(EXIT_SUCCESS);      // この関数内でプログラム全体を終了
11 }
12
13 int main() {
14     int n;
15
16     atexit(cleanup);          // 終了処理用関数の登録
17
18     printf("--main function\\n");
19     printf("input integer: "); fflush(stdout);
20     scanf("%d", &n);
21
22     if (n >= 0) {              // 入力された値が0以上なら
23         exit(EXIT_SUCCESS);    // main内で終了
24     } else {                   // 負なら
25         sub();                 // 関数subを呼び出す
26     }
27 }

```

注意） この例では処理の流れを分かりやすくするため、atexit および scanf のエラー判定は省略している。

このプログラムをコンパイルして実行する例を次に示す。

例. test024-3.c のコンパイルと実行

```
$ gcc test024-3.c -o test024-3  ←コンパイル
$ ./test024-3  ←実行
--main function          ← main 関数内での実行
input integer: 1  ←正の値「1」の入力
--cleanup function       ← main 関数で exit し, cleanup 関数が実行された
$ ./test024-3  ←再度実行
--main function          ← main 関数内での実行
input integer: -1  ←負の値「-1」の入力
--sub function           ← sub 関数に実行が移った
--cleanup function       ← sub 関数で exit し, cleanup 関数が実行された
```

9.15.2 abort 関数によるプログラムの異常終了

abort 関数を使用すると、通常の終了処理を行わずに、プログラムを異常終了させることができる。この関数は、ヘッダファイル `stdlib.h` で宣言されている。

プロトタイプ: `void abort( void );`

abort 関数は引数を受け取らず、呼び出されると、その位置からプログラムを異常終了させる。この関数から呼び出し元へ処理が戻ることはない。

exit 関数による終了では、`atexit` 関数で登録された終了処理関数の呼び出しや、標準入出力ストリームに残っているデータの書き出しなど、所定の終了処理が行われる。これに対して、abort 関数による終了では、`atexit` 関数で登録された関数は呼び出されず、標準入出力ストリームに残っているデータが書き出されることも保証されない。

abort 関数は、プログラムの内部状態が壊れている場合など、通常の処理を継続することができず、通常の終了処理を行うことも困難な場合に使用される。ただし、通常のエラー処理では、まず関数の戻り値を用いて呼び出し元へ異常を通知し、可能であれば必要な後処理を行ってから終了する方が望ましい。

■ サンプルプログラム

次に示すプログラム test024-2.c では、`atexit` 関数を用いて終了処理関数 `finish` を登録し、`exit` 関数と `abort` 関数による終了の違いを確認する。入力した値に応じて、`main` 関数または関数 `f1` から `exit` 関数を実行する場合と、関数 `f2` から `abort` 関数を実行する場合に処理を分岐する。`exit` 関数によって終了した場合には、登録した終了処理関数 `finish` が呼び出されるが、`abort` 関数によって異常終了した場合には呼び出されない。

プログラム: test024-2.c

```
1  #include    <stdio.h>
2  #include    <stdlib.h>
3
4  void finish(void) {          // 終了処理関数
5      printf("termination function 'finish()' was called.\n");
6  }
7
8  void f1(void) {
9      printf("function f1: ");
10     exit(EXIT_SUCCESS);
11 }
12
13 void f2(void) {
14     printf("function f2: abnormal termination.\n");
15     abort();
16 }
17
18 int main() {
19     int      n;
20
21     if ( atexit(finish) != 0 ) {
22         printf("could not register termination function 'finish()'.\n");
23         return EXIT_FAILURE;
24     }
25 }
```

```

26     while ( -1 ) {
27         printf("0 or 1 or 2: ");
28         scanf("%d",&n);
29         if ( n == 0 ) {
30             printf("main function: ");
31             exit(EXIT_SUCCESS);
32         } else if ( n == 1 ) {
33             f1();
34         } else if ( n == 2 ) {
35             f2();
36         }
37     }
38 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test024-2.c のコンパイルと実行

```

$ gcc test024-2.c -o test024-2  ←コンパイル
$ ./test024-2  ←実行
0 or 1 or 2: 0  ←0を入力
main function: termination function 'finish()' was called. ←main 関数で exit を実行
$ ./test024-2  ←実行
0 or 1 or 2: 1  ←1を入力
function f1: termination function 'finish()' was called. ←関数 f1 で exit を実行
$ ./test024-2  ←実行
0 or 1 or 2: 2  ←2を入力
function f2: abnormal termination. ←関数 f2 で abort を実行

```

注意) abort 関数による異常終了では、出力バッファに残っているデータが書き出されないことがある。また、実行環境によっては、プログラム自身が出力した内容とは別に、異常終了を示すメッセージがシェルなどから表示される場合がある。

第2部

C++

10 C++とは

C++ は、C 言語を基盤として設計されたプログラミング言語である。C 言語が持つ低水準の制御能力や効率性を維持しつつ、より大規模で複雑なプログラムを安全かつ柔軟に構築するための機能が追加されている。

C++ は、1980 年代初頭に、AT&T Bell Laboratories（ベル研究所）の研究員であった Bjarne Stroustrup によって設計された。当初は「C with Classes」と呼ばれ、C 言語にクラス機構を導入することで、データと処理を一体として扱うことを目的としていた。その後、継承や多態性、テンプレートなどの機能が追加され、現在の C++ へと発展した。

現在の C++ は、クラスを中心とするオブジェクト指向プログラミングだけを目的とした言語ではない。C 言語と同様の手続き型プログラミングに加えて、クラスを利用したオブジェクト指向プログラミング、テンプレートを利用した汎用的なプログラミングなど、複数のプログラミング手法を組み合わせることで利用できる言語となっている。

ここで注意すべき点として、C++ は C 言語を拡張した言語ではあるが、単純な意味での上位互換ではなく、独立した別の言語である。実際、C 言語で記述されたプログラムの多くは C++ としてもコンパイル可能であるが、言語仕様の違いにより、そのままではコンパイルできない場合や、C 言語とは異なる意味として扱われる場合もある。

C++ には、C 言語にはないさまざまな機能が追加されており、より大規模なプログラムを構成しやすくなっている。C++ の重要な特徴の一つは、高度な抽象化のための機能を備えながら、実行時の性能も重視して設計されている点にある。このため、システムソフトウェア、ゲーム開発、組み込みシステム、高性能計算など、特に性能が重視される分野で広く利用されている。また、標準ライブラリも充実しており、汎用的なアプリケーション開発にも利用できる。

C 言語で学んだ知識の多くは C++ でも活用することができるが、C++ には、C 言語とは異なる、より安全あるいは簡潔な記述方法が用意されている場合も多い。本書では、C 言語で学んだ知識を基礎として、C++ に適した記述方法や基本的な機能を順に解説する。

以下に、C++ と C 言語の主な違いを示す。

■ 抽象化のための機構

C 言語では、構造体とそれを操作する関数を組み合わせてプログラムを構成することができるが、構造体のデータと関数は言語上は別々に定義される。これに対して C++ では、クラスによりデータと処理を一体として扱うことができ、より明確な構造でプログラムを記述することが可能となる。これは、大規模なプログラムの開発において特に重要である。

■ 型の表現力の向上

C++ では、C 言語にも存在する `const` 修飾子に加えて、参照など、型を利用してプログラムの意図を表現するための機能が拡充されている。これにより、プログラムの意図をより明確に記述し、誤りを防ぎやすくなる。

■ 関数や演算子の柔軟な利用

C++ では、同じ名前の関数について、引数の型や個数が異なる複数の関数を定義する関数のオーバーロードが可能である。また、クラスに対して演算子の働きを定義することもできる。これらの機能により、処理の意味に応じた自然な表現でプログラムを記述することができる。

■ 安全性と資源管理

C 言語では、メモリやファイルなどの資源の管理は、すべて開発者が明示的に行う必要がある。これに対して C++ では、コンストラクタやデストラクタを用いることで、資源の確保と解放をオブジェクトの寿命と結び付けて管理する仕組みを構築することができ、メモリの解放忘れなどの問題を起こしにくいプログラムを記述することができる。

■ 汎用的なプログラムの記述

C++ にはテンプレートと呼ばれる機能があり、扱うデータ型に依存しない汎用的な関数やクラスを記述することができる。C++ の標準ライブラリで提供されるコンテナやアルゴリズムの多くも、この仕組みを利用して実現されている。

■ 標準ライブラリの充実

C++ では、文字列や動的配列などの基本的なデータ構造や、それら进行操作するための多くの機能が標準ライブラリとして提供されている。これにより、基本的な処理を一から実装する必要がなく、効率的にプログラムを開発することが可能となる。

以上のように、C++ は C 言語の特徴を受け継ぎながら、抽象化、型、資源管理、汎用的なプログラミングなどの機能を大幅に拡張した言語である。

C++ のソースプログラムのファイル拡張子は `*.cpp` とすることが一般的である。GCC では、C++ のコンパイラのコマンドは `g++` であり、コンパイル時のオプションは概ね C 言語の場合と同様である。

11 C++の基礎

11.1 main 関数の記述の形式

C++ における main 関数は、プログラムの実行開始点であり、その基本的な記述形式は C 言語 とほぼ同様である。すなわち、次のような形式で定義する。

形式 1: `int main(void)` (void の記述は省略することが多い)

形式 2: `int main(int argc, char *argv[])`

前者はコマンド引数を受け取らない場合、後者はコマンド引数を扱う場合の典型的な形である。

■ 注意事項

- main 関数の戻り値の型は `int` でなければならない。void main() のような定義は規格上正しくない。
- main 関数では、正常終了時の `return 0;` の記述を省略することができる。main 関数の実行が末尾まで正常に到達した場合は、`return 0;` が記述されている場合と同様に扱われる。

なお、C++ では `int main()` のように、引数リスト中の void を省略する書き方も一般的であり、本書では基本的にその慣例に従う。

11.2 クラスとオブジェクトに関する予備知識

C++ では、**クラス**と呼ばれる仕組みを用いて、データとそれを操作する機能をまとめて定義することができる。また、クラスに基づいて作られた実体を**オブジェクト**と呼ぶ。これに関しては「12 クラスとオブジェクト」(p.237〜)で詳しく解説するが、それまでは「クラス」という語を「型」と読み替えると良い。同様に、「オブジェクト」という語を「データ」、「メンバ関数」という語を単に「関数」と読み替えると良い。

ただし、これらはあくまで理解のための大まかな対応関係であり、厳密に同じ意味ではない。

オブジェクトのメンバ関数を呼び出す場合には、オブジェクト名とメンバ関数名の間をドット「`.`」で結び、例えば次のように記述する。

```
obj.func();
```

これは、オブジェクト `obj` に対してメンバ関数 `func()` を呼び出す記述である。「12 クラスとオブジェクト」で解説するまでも「クラス」、「オブジェクト」、「メンバ関数」といった用語を使用するが、しばらくは以上の程度に理解しておくといよい。

11.3 C++の基本的なデータ型

C++ の基本的なデータ型は、概ね C 言語のデータ型を踏襲している。例えば、`int`, `long`, `float`, `double`, `char` などは、C 言語と同様に使用できる。また、配列、ポインタ、構造体などの考え方も、C 言語と同様である。

このため、C++ では多くの C 言語向けライブラリを利用することができる。ただし、C++ では、C 言語の標準ヘッダを利用する際に、C++ の固有のヘッダ名を用いることが多い。例えば、C 言語の `stdio.h`, `stdlib.h`, `string.h`, `math.h` に対応する C++ のヘッダとして、`cstdio`, `cstdlib`, `cstring`, `cmath` が用意されている。

従って、C 言語で用いた数学関数の多くは、C++ でも同様に使用できる。例えば、平方根を求める `sqrt`、累乗を求める `pow`、三角関数である `sin`, `cos`, `tan`、指数関数 `exp`、対数関数 `log` などは、C++ では主に `cmath` ヘッダを読み込んで使用する。ただし、C++ ではこれらの関数は標準名前空間 `std` (後で解説する) に属するため、基本的には `std::sqrt` のように名前空間を指定して使用する。本書のサンプルプログラムでは、必要に応じて `using std::sqrt` のように必要な名前だけを導入することがある。**名前空間**に関しては後の「11.5 名前空間」(p.190〜)で解説する。

一方、C++ には、C 言語とは異なる型や機能も用意されている。例えば、論理値を扱う `bool` 型、標準文字列クラス `string`、ヌルポインタを明示的に表す `nullptr` などがある。これらは後に解説する。

11.3.1 変数宣言時の初期化

C++では、変数を宣言する際に、イコール `=`、丸括弧 `()`、波括弧 `{ }` などを用いて初期値を与えることができる。例えば、`int` 型の変数を値 10 で初期化する場合には、次のように記述できる。

```
int n1 = 10;
int n2(10);
int n3{10};
```

これらはいずれも、`int` 型の変数を宣言し、その初期値を 10 とする記述である。イコールを用いる方法を**コピー初期化**、丸括弧を用いる方法を**直接初期化**、波括弧を用いる方法を**リスト初期化**という。

波括弧による初期化には、値や精度が失われる可能性のある一定の型変換を禁止するという特徴がある。例えば、次の記述を考える。

```
int n1 = 3.14;    // 整数値 3 に変換される
int n2(3.14);    // 整数値 3 に変換される
int n3{3.14};    // コンパイルエラー
```

イコールや丸括弧による初期化では、3.14 は `int` 型の値 3 に変換される。これに対して、波括弧による初期化では、小数点以下の値が失われる**縮小変換**が禁止されるため、コンパイルエラーとなる。従って、意図しない型変換を防ぐという点では、波括弧による初期化が有効である。

また、空の波括弧を用いると、基本型の変数を 0 に相当する値で初期化できる。例えば、次のように記述する。

```
int n{};          // 0 に初期化
double x{};       // 0.0 に初期化
bool b{};         // false に初期化
char c{};         // '\0' に初期化
```

これに対して、関数内で基本型のローカル変数を次のように初期値なしで宣言した場合、その値は不定となる。

```
int n;
```

不定な値を使用すると、プログラムが意図しない動作をする原因となる。そのため、基本型の変数を宣言する場合には、必要に応じて初期値を明示することが重要である。特に、初期値として 0 に相当する値を設定する場合には、空の波括弧を用いて簡潔に記述できる。

空の波括弧による初期化は、クラス型のオブジェクトにも使用できる。クラス型では、その型に応じた初期化が行われる。波括弧を用いたクラス型のオブジェクトの初期化については、後の「12 クラスとオブジェクト」(p.237～)で詳しく解説する。

11.3.2 C++での複素数の扱い

C 言語では、ヘッダファイル `complex.h` で提供される機能を用いて複素数を扱うことができる。これについては、「3.5.7 複素数型」(p.29～)で解説した。

一方、C++ の標準ライブラリでは、ヘッダファイル `complex` で提供される**クラステンプレート**¹⁰⁰ `std::complex` を用いて複素数を扱う。例えば、実部と虚部を `double` 型の値として保持する場合には、`std::complex<double>` を使用する。

C++17 には、C 言語との互換性のためのヘッダファイル `ccomplex` や `complex.h` も存在する。ただし、`ccomplex` は C++17 では非推奨とされており、C++17 の規格では、`complex.h` は `ccomplex` を読み込むものとして定められている。これらは、C 言語の複素数型を C++ でそのまま利用できるようにするものではない。また、一部の処理系では、拡張機能によって C 言語形式の複素数を使用できる場合もあるが、そのようなプログラムには移植性がない可能性がある。従って、C++ で複素数を扱う場合には、ヘッダファイル `complex` を読み込み、`std::complex` を使用することを強く推奨する。詳しい使用方法は、後の「22.3 複素数の扱い：complex」(p.433～)で解説する。

¹⁰⁰ より基本的な**クラス**の概念については、後の「12 クラスとオブジェクト」(p.237～)で解説する。また、**クラステンプレート**については、「16 型を抽象化するための機能」(p.322～)で解説する。

11.4 入出力

C++ では、入出力を**ストリーム**と呼ばれる抽象的なデータの流れとして扱う。ストリームとは、プログラムと外部との間でやり取りされるデータを、一定の方向に流れるものとして捉える考え方である。

C++ では、標準入力、標準出力、ファイル入出力などを、共通したストリームの枠組みで扱うことができる。入出力の対象が異なっている、データの読み書きそのものはよく似た方法で記述することができる。

本書では、まず標準入出力を対象として `iostream` による基本的な操作を解説し、その後、ファイルストリームを用いた入出力について解説する。

11.4.1 `iostream`

C++ では、標準的な入出力のために `iostream` が用意されている。標準出力には `cout` (**標準出力ストリーム**)、標準入力には `cin` (**標準入力ストリーム**) と呼ばれるストリームオブジェクトを用いる。また、標準エラー出力には `cerr` と `clog` が用意されている。これらをまとめて**標準ストリーム**と呼ぶ。

C 言語でも、標準入出力やファイル入出力はストリームとして扱われ、`FILE` 型を介して操作する。C++ では、これらをストリームクラスのオブジェクトとして扱い、`<<` や `>>` などを用いて入出力を記述する。なお、C++ においても、C 言語の `printf` や `scanf` を利用することは可能であるが、本書では `iostream` による入出力を基本として解説する。

C 言語の `printf` では、出力する値の型に応じて `%d`、`%f`、`%s` などの変換指定子を使い分ける必要があった。これに対して C++ の `cout` では、値を `<<` を用いて順に与えることで、文字列や数値を出力することができる。基本的な出力では、各値の型に応じた処理が自動的に行われるため、変換指定子を明示する必要がない。例えば、C 言語では、

```
printf("x = %d\n", x);
```

のように記述していた処理は、C++ では、

```
cout << "x = " << x << '\n';
```

のように記述できる。

入力についても、C 言語では `scanf` などを用いて書式を指定して値を読み取るが、C++ では `cin` と `>>` を用いて変数に値を読み込むことができる。例えば、

```
cin >> x;
```

と記述すると、標準入力から値を読み取り、変数 `x` に格納することができる。

`iostream` による入出力では、処理が正常に行われたかどうかを確認することも重要である。特に `cin` による入力では、入力されたデータが変数の型に適合しない場合などに入力処理が失敗する。ストリームの状態の検査や、処理が失敗した場合の対応については、後の「11.4.4 ストリームに対する各種の関数」(p.184) で解説する。

11.4.2 書式制御の方法 (C 言語の変換指定子との違い)

C 言語の `printf` では、出力する値の書式は、書式文字列中の変換指定子 (`%d`、`%f` など) によって指定する必要があった。例えば、桁数や小数点以下の精度なども、書式文字列の中で指定する。

これに対して C++ の `iostream` では、書式の指定は**マニピュレータ**と呼ばれる機能を用いて行う。これは、出力ストリームの状態を変更することで、以降の出力の書式を制御する仕組みである。

11.4.2.1 桁数、精度の設定

出力幅や精度の指定は、次のように記述する。

```
cout << setw(5) << x;  
cout << fixed << setprecision(2) << y;
```

ここで、`setw` は出力する値の最小幅を指定するマニピュレータであり、`setprecision` は浮動小数点数の精度を指定する。通常は有効桁数を表すが、`fixed` と併用した場合には、小数点以下の桁数を指定する。

`setw` は直後に出力する値にのみ有効であり、出力の度に使用する必要がある。また、`fixed`、`setprecision` による設定は、その設定が後々の出力に継続して有効である。`fixed` の設定は `defaultfloat` によって解除することができる。また、`setprecision` で設定した精度を既定値に戻す場合には、`setprecision(6)` とする。

11.4.2.2 左右のアラインメントと埋め文字

left, right で左寄せ, 右寄せ (アラインメント) を指定することが可能である。例えば次のようにして, 指定した出力幅 10 の範囲でアラインメントが可能である。(アラインメントのデフォルトは right である)

```
cout << left << setw(10) << s;           // 左寄せで s を出力
cout << right << setw(10) << s;          // 右寄せで s を出力
```

出力する値の幅が, 指定した出力幅よりも小さい場合, setfill によって, 余った部分に埋め文字 (fill character) を出力することができる。

```
cout << setw(5) << setfill('0') << x;
```

これは, x の出力に必要な幅が 5 に満たない場合, 空いた桁を '0' で埋める処理である。

注意) left, right, setfill の設定は効果が持続する。left からデフォルトの右寄せに戻すには, right を設定する。また, setfill のデフォルトはスペース ' ' であり, これに戻すには setfill(' ') とする。

このように, C++ では値の出力と書式の指定とを分離して記述することができ, 柔軟な書式制御が可能となる。これは, 書式を文字列として記述する printf とは異なる特徴である。

なお, setw や setprecision などのマニピュレータを使用するには, iomanip ヘッダをインクルードする必要がある。また, ここで示したもの以外にも多くのマニピュレータ (表 30) が用意されている。

表 30: 使用頻度の高いマニピュレータ (一部)

マニピュレータ	機能	備考
setw(n) setprecision(n) fixed defaultfloat	出力幅を n 桁に指定 数値の精度を指定 固定小数点形式で表示 浮動小数点の通常表示に戻す	次の出力にのみ有効 通常は有効桁数, fixed 併用時は小数桁数 setprecision と併用可能 (丸めあり) fixed 等の設定解除
setfill(c)	埋め文字を指定	例: '0'
left right	左寄せ 右寄せ	フラグとして保持 フラグとして保持
scientific	指数形式で表示	必要に応じて使用
flush endl	バッファをフラッシュ 改行とバッファのフラッシュ	即時出力が必要な場合に使用 (改行なし) 即時出力が必要な場合に使用 (改行あり)

これらのマニピュレータを組み合わせることで, 柔軟な書式制御が可能となる。これは, 出力の書式を状態として管理する ostream の特徴である。

ostream を使用して入出力を行う例を test201-1.cpp, test201-2.cpp に示す。

プログラム: test201-1.cpp

```
1 #include <iostream>
2 #include <iomanip>           // マニピュレータ使用時に必要
3 using namespace std;        // 名前空間の指定
4
5 int main() {
6     // 右寄せ (通常)
7     cout << "|" << setw(9) << 1 << "|" << setw(9) << 3.14159 << "|\n";
8     cout << "|" << setw(9) << setfill('0') << 1 << "|"
9                                     << setw(9) << 3.14159 << "|\n";
10
11     // 左寄せ
12     cout << left << setfill('*');
13     cout << "|" << setw(9) << 1 << "|" << setw(9) << 3.14159 << "|\n";
14
15     // 指数表現
16     double na = 6.02214076e23;
17     cout << right;
18     cout << "|" << scientific << setw(19) << na << "|\n";
19
20     cout << flush; // バッファのフラッシュ
21 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test201-1.cpp のコンパイルと実行

```
$ g++ test201-1.cpp -o test201-1  ←コンパイル
$ ./test201-1  ←実行
|          1|  3.14159|    ←通常の右寄せ
|000000001|003.14159|    ←右寄せ, '0' で埋める
|1*****|3.14159**|    ←左寄せ, '*' で埋める
|*****6.022141e+23|    ←右寄せ, 指数表現, '*' で埋める
```

プログラム冒頭にある名前空間の指定に関しては後述する。

次のプログラム test201-2.cpp は、標準入力ストリームから値を読み取る例である。

プログラム：test201-2.cpp

```
1  #include <iostream>
2  using namespace std;    // 名前空間の指定
3
4  int main() {
5      double  x, y;
6
7      cout << "x = ";      cin >> x;
8      cout << "y = ";      cin >> y;
9      cout << x << " + " << y << " = " << x+y << endl;
10 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test201-2.cpp のコンパイルと実行

```
$ g++ test201-2.cpp -o test201-2  ←コンパイル
$ ./test201-2  ←実行
x = 3  ←入力 (1)
y = 4  ←入力 (2)
3 + 4 = 7 ←出力
```

11.4.3 ファイルストリーム

C++ では、ファイルに対する入出力も、標準入出力用のストリーム (cin, cout など) と同様にストリームとして扱う。すなわち、ファイルは特別な対象ではなく、データの入出力先の一つとして、ストリームの枠組みの中で統一に取り扱われる。

ファイルストリームは、その用途に応じて、出力専用、入力専用、および入出力両用の 3 種類に分類される。ファイルストリームでも、標準入出力の場合と同様に、その用途に応じて << や >> といった演算子を用いてデータの入出力を行うことができるため、プログラムの記述方法を統一することができる。

以下では、これらのファイルストリームについて、それぞれの役割と基本的な使用方法を順に解説する。

11.4.3.1 出力ファイルストリーム (ofstream)

ofstream は、ファイルヘデータを書き込むための出力ファイルストリームを表すクラスである。標準出力を表す cout と同様に、<< 演算子を用いて値を出力することができる。ofstream は名前空間 std に属しており、これを使用するには、ヘッダファイル fstream をインクルードする。

書き方： ofstream ストリーム名 (出力先ファイルのパス)

「出力先ファイルのパス」を文字列で与えることで、指定したファイルをオープンし、それに対応する出力ストリームオブジェクトが「ストリーム名」として生成される。「ストリーム名」は任意に命名できる。

例. 出力用ストリームの作成と出力の実行

```
#include <fstream>
std::ofstream fout("out.txt");    ←"out.txt" を開き, それを fout とする
fout << 123 << '\n';             ←出力の実行
```

この例では、カレントディレクトリのファイル "out.txt" を開き、それを出力用のストリーム fout としている。こ

のストリームオブジェクトは、所属するスコープを抜けるまで引き続き出力処理に使用できる。スコープを抜けるとオブジェクトは破棄され、開かれていたファイルも自動的にクローズされる。

このように、ファイルへの出力も標準出力 `cout` の場合と同じ形式で記述できる。

■ 明示的にファイルをオープンする形式

出力ファイルストリームは、オブジェクト作成後にオープンすることも可能である。

例. 出力ファイルストリームの作成とオープンを別々に行う方法

```
std::ofstream fout;           ←この段階では fout を作成しただけ
fout.open("out.txt");         ←この処理で実際にファイルをオープンする
```

このように、出力ファイルストリームに対して `open` 関数を実行する。この際に、対象のファイルのパスを引数に与える。

参考) 開いているファイルは、通常はスコープを抜ける際に自動的にクローズされるが、必要に応じて

`close()` メンバ関数で明示的に閉じることもできる。

例. `fout.close();`

11.4.3.2 入力ファイルストリーム (ifstream)

`ifstream` は、ファイルからデータを読み取るための入力ファイルストリームを表すクラスである。標準入力を表す `cin` と同様に、`>>` 演算子を用いて値を読み込むことができる。`ifstream` は名前空間 `std` に属しており、これを使用するには、ヘッダファイル `fstream` をインクルードする。

書き方: `ifstream` ストリーム名 (入力元ファイルのパス)

「入力元ファイルのパス」を文字列で与えることで、指定したファイルをオープンし、それに対応する入力ストリームオブジェクトが「ストリーム名」として生成される。「ストリーム名」は任意に命名できる。

例. 入力用ストリームの作成と入力の実行

```
#include <fstream>
std::ifstream fin("data.txt");  ←"data.txt" を開き、それを fin とする
int x;
fin >> x;                       ←入力の実行
```

この例では、カレントディレクトリのファイル `"data.txt"` をオープンし、それを入力用のストリーム `fin` としている。このストリームオブジェクトは、所属するスコープを抜けるまで引き続き入力処理に使用できる。スコープを抜けるとオブジェクトは破棄され、開かれていたファイルも自動的にクローズされる。

このように、ファイルからの入力も標準入力 `cin` と同じ形式で記述できる。

■ 明示的にファイルをオープンする形式

入力ファイルストリームは、オブジェクト作成後にオープンすることも可能である。

例. 入力ファイルストリームの作成とオープンを別々に行う方法

```
std::ifstream fin;           ←この段階では fin を作成しただけ
fin.open("data.txt");        ←この処理で実際にファイルをオープンする
```

このように、入力ファイルストリームに対して、メンバ関数 `open` を実行する。この際に、対象のファイルのパスを引数に与える。

参考) 入力処理においては、ファイルの終端 (EOF) に達してそれ以上データを読み取れない場合や、値として解釈できない入力が見れた場合に、ストリームは失敗状態となる。したがって、

```
if ( fin >> x ) { ... }
```

のように、入力の成否を判定しながら処理を行うのが一般的である。

参考) 開いているファイルは、通常はスコープを抜ける際に自動的にクローズされるが、必要に応じてメンバ関数 `close()` で明示的に閉じることもできる。

11.4.3.3 入出力ファイルストリーム (fstream)

fstream は、ファイルに対して入力と出力の両方を行う入出力ファイルストリームを表すクラスである。標準入力 `cin` および標準出力 `cout` の機能を併せ持つストリームとして考えることができ、`>>` 演算子による入力と `<<` 演算子による出力の双方を同一のストリームで扱うことができる。fstream は名前空間 `std` に属しており、これを使用するには、ヘッダファイル `fstream` をインクルードする。

書き方： `fstream` ストリーム名 (入出力対象ファイルのパス)

「入出力対象ファイルのパス」を文字列で与えることで、指定したファイルをオープンし、それに対応する入出力ストリームオブジェクトが「ストリーム名」として生成される。「ストリーム名」は任意に命名できる。なお、fstream をこの形式で使用する場合、既定では既存のファイルを入出力用に開くため、対象のファイルが存在しない場合はオープンに失敗する

例. 入出力ストリームの作成と入出力の実行

```
#include <fstream>
std::fstream fs("data.txt");    ←"data.txt"をオープンし、それを fs とする
int x;
fs >> x;                        ←入力の実行
fs << x << '\n';               ←出力の実行
```

この例では、カレントディレクトリのファイル `"data.txt"` をオープンし、それを入出力用のストリーム `fs` としている。このストリームオブジェクトは、所属するスコープを抜けるまで引き続き入出力処理に使用できる。スコープを抜けるとオブジェクトは破棄され、開かれていたファイルも自動的にクローズされる。

このように、fstream を用いることで、単一のファイルに対して入力と出力の両方を行うことができる。

▲注意▲

上の例のように同一の fstream オブジェクトで入力の直後に出力を行う場合には注意が必要である。入力と出力では共通のファイル位置が使用されるため、入力後の位置から書き込みが行われ、既存のデータを意図せず上書きすることがある。また、入力と出力を切り替える際には、ファイル位置やストリームの状態を適切に管理する必要がある。従って、単純な用途においては、`ifstream` と `ofstream` を用途に応じて使い分けることが推奨される。

■ 明示的にファイルをオープンする形式

入出力ファイルストリームは、オブジェクト作成後にオープンすることも可能である。

例. 入出力ファイルストリームの作成とオープンを別々に行う方法

```
std::fstream fs;                ←この段階では fs を作成しただけ
fs.open("data.txt");            ←この処理で実際にファイルをオープンする
```

このように、入出力ファイルストリームに対してメンバ関数 `open` を実行する。この際に、対象のファイルのパスを引数に与える。

参考) 開いているファイルは、通常はスコープの終了時に自動的にクローズされるが、必要に応じてメンバ関数 `close()` で明示的に閉じることもできる。

11.4.3.4 バイナリファイルの扱い

C++ においても、ファイルをバイナリ形式で扱うことができる。バイナリ形式では、データを文字列に変換せず、バイト列として読み書きすることができる。メモリ上の内容をそのままのバイト列として読み書きする場合には、通常の書式付き入出力で用いる `<<` や `>>` ではなく、`write` および `read` を使用する。

また、バイナリファイルとして扱う場合には、ファイルをオープンする際に `ios::binary` を指定する。特に Windows 環境では、テキストモードのままでは改行コードの変換が行われるため、バイナリデータが破損する可能性がある。

以下では、出力、入力、および入出力両用の3つの場合について、それぞれの基本的な扱い方を示す。

■ バイナリファイルの出力

ファイルをオープンして `ofstream` を作成する際に、バイナリモードを指定する。

例. バイナリファイルの出力

```
#include <fstream>
std::ofstream fout("data.bin", std::ios::binary);
int x = 123;
fout.write( reinterpret_cast<const char*>(&x), sizeof(x) );
```

このように、write 関数を用いて、変数の内容をそのままファイルに書き込む。write の第1引数にはバイト列としてのポインタを与える必要があるため、reinterpret_cast による型変換¹⁰¹ を行い、メモリ内容をバイト列として扱う。

■ バイナリファイルの入力

ファイルをオープンして ifstream を作成する際に、バイナリモードを指定する。

例. バイナリファイルの入力

```
#include <fstream>
std::ifstream fin("data.bin", std::ios::binary);
int x;
fin.read( reinterpret_cast<char*>(&x), sizeof(x) );
```

read 関数を用いることで、ファイルから指定したバイト数を読み取り、変数に格納することができる。

入力においては、ファイル終端 (EOF) に到達した場合や、十分なデータが存在しない場合に読み取りが失敗してストリームは失敗状態となるため、必要に応じてストリームの状態を確認することが望ましい。これに関しては後の「11.4.4 ストリームに対する各種の関数」(p.184) で解説する。

■ 入出力両用の場合

バイナリファイルに対して読み書きの両方を行う場合には、fstream を作成する際に、入出力両用およびバイナリモードの設定を行う。

例. バイナリファイルの入出力

```
#include <fstream>
std::fstream fs("data.bin",
    std::ios::in | std::ios::out | std::ios::binary);
int x;
fs.read( reinterpret_cast<char*>(&x), sizeof(x) );
fs.write( reinterpret_cast<const char*>(&x), sizeof(x) );
```

このように、read と write の両方を使用することができる。バイナリモードなどのオープンモードを第2引数で明示する場合には、入出力両用とするため ios::in および ios::out も併せて指定する。

また、入力と出力を混在させて使用する場合には、ファイルのアクセス位置やストリームの状態に注意が必要である。特に、入力の直後に出力を行う場合などには、seekg や seekp による位置の調整¹⁰² や、clear() による状態のリセット¹⁰³ が必要となることがある。

このように、バイナリファイルは効率的なデータの保存や読み込みに適しているが、データの表現が環境に依存するため (エンディアンや型のサイズなど)、異なる環境間での互換性には注意が必要である。

11.4.3.5 入出力位置の制御

ファイルに対する入出力では、通常、データを読み書きするたびに、ファイル内のアクセス位置が自動的に進む。例えば、ファイルからデータを読み取ると、次の読み取り位置は、直前に読み取ったデータの直後へ移動する。また、ファイルへデータを書き込むと、次の書き込み位置は、直前に書き込んだデータの直後へ移動する。

このような処理は、ファイルの内容を先頭から順に読み書きする場合には自然であり、多くのプログラムでは、アクセス位置を明示的に意識しなくても処理を記述できる。しかし、実用的なプログラムでは、ファイルの途中からデータを読み取ったり、特定の位置へ戻って再び読み直したり、あるいはファイルの末尾へ移動して追記を行いたい場合

¹⁰¹ 「11.6 C++における型の考え方」(p.192) で解説する。

¹⁰² 「11.4.3.5 入出力位置の制御」(p.180) で解説する。

¹⁰³ 「11.4.4 ストリームに対する各種の関数」(p.184) で解説する。

がある。

C++ のファイルストリームでは、このような入出力位置の制御もストリームの機能として提供されている。読み取り位置は `seekg`, `tellg` によって、書き込み位置は `seekp`, `tellp` によって操作する。ここで、関数名末尾の `g` は入力側の `get position`（読み取り位置）、`p` は出力側の `put position`（書き込み位置）を表している。

読み取り位置を操作する `seekg`, `tellg` と、書き込み位置を操作する `seekp`, `tellp` がそれぞれ用意されているが、`fstream` のように同一のファイルに対して入力と出力の両方を行う場合、実際のファイル位置は入力側と出力側で共通している。従って、入力と出力を混在させる場合には、現在のファイル位置を意識し、必要に応じて `seekg` や `seekp` によって位置を明示的に設定することが重要である。

また、C 言語の `rewind` に相当する専用の関数はなく、ファイル先頭への移動も `seekg` や `seekp` によって行う（例えば、先頭へ移動するには `seekg(0, std::ios::beg)` のように指定する）。さらに、EOF に達してストリームが失敗状態となった後に再び先頭から読み直す場合などには、ストリームの状態を `clear()` によってリセットする必要がある。以下では、これらの入出力位置の制御方法について順に解説する。

■ 入出力位置を操作する関数

ファイルストリームでは、現在の読み取り位置や書き込み位置を取得したり、任意の位置へ移動したりするための関数（表 31）が用意されている。これらは、`ifstream`, `ofstream`, `fstream` などのストリームオブジェクトに対して使用するメンバ関数である。これらの機能を使用するには、「`#include <fstream>`」のようにしてヘッダファイル `fstream` をインクルードする。

表 31: ファイルのアクセス位置を設定・取得する関数

入力用		出力用	
関数	機 能	関数	機 能
<code>seekg</code>	読み取り位置を移動する。	<code>seekp</code>	書き込み位置を移動する。
<code>tellg</code>	現在の読み取り位置を取得する。	<code>tellp</code>	現在の書き込み位置を取得する。

これら関数の主な書き方は次の通りである。

- | | |
|---|---|
| (1) ストリーム名. <code>seekg</code> (位置); | (4) ストリーム名. <code>seekp</code> (位置); |
| (2) ストリーム名. <code>seekg</code> (移動量, 基準位置); | (5) ストリーム名. <code>seekp</code> (移動量, 基準位置); |
| (3) ストリーム名. <code>tellg</code> (); | (6) ストリーム名. <code>tellp</code> (); |

上記 (1), (4) の形式では、指定したファイル位置へ移動する。(2), (5) の形式では、基準位置からの相対的な移動量を指定する。`seekg`, `seekp` は元のストリームへの参照を返すため、ドット「`.`」による関数呼び出しを続けて記述すること¹⁰⁴が可能である。

基準位置には、表 32 の値を用いる。

表 32: 基準位置を指定するための値

指定	意味	指定	意味	指定	意味
<code>std::ios::beg</code>	ファイル先頭	<code>std::ios::cur</code>	現在位置	<code>std::ios::end</code>	ファイル末尾

例えば、読み取り位置をファイル先頭へ移動するには、次のように記述する。

```
fin.seekg(0, std::ios::beg);
```

また、書き込み位置をファイル末尾へ移動するには、次のように記述する。

```
fout.seekp(0, std::ios::end);
```

現在位置（ストリーム内部の位置）を調べるには、`tellg` または `tellp` を用いる。

```
auto pos1 = fin.tellg();
auto pos2 = fout.tellp();
```

これらの関数によって得られる位置は、ストリーム内で用いられる位置情報である。通常は、ファイル先頭からのバイト位置として考えてよいが、テキストモードでは改行コードの変換などが関係する場合があるため、厳密なバイト

¹⁰⁴この形式はメソッドチェーンなどと呼ばれることがある。ただし、入出力位置の制御では、可読性の観点から個別に記述することも多い。

位置として扱いたい場合には、バイナリモードでファイルをオープンするのが安全である。

■ 入出力位置の値を表す型

`tellg` や `tellp` によって得られる値、および `seekg` や `seekp` で位置や移動量を指定する際には、ストリーム用に定められた型が用いられる。具体的には、位置を表す型として `std::streampos`、移動量を表す型として `std::streamoff` が用いられる。

例えば、現在位置を取得する `tellg` や `tellp` の戻り値は `std::streampos` 型である。`std::streampos` は、単なるバイト位置を表す整数ではなく、ストリーム内の位置情報を表すための型である。一方、`std::streamoff` は、位置間の移動量を表すための符号付き整数型である。

`std::streampos` は `std::streamoff` に変換して扱うことができる。例えば、現在位置を移動量として扱える値に変換する場合には、次のように記述する。

例. `streampos` から `streamoff` への変換

```
auto pos = fin.tellg();
auto offset = static_cast<std::streamoff>(pos);
```

逆に、外部で用意した整数値を `seekg` や `seekp` の移動量として使用する場合には、`std::streamoff` 型へ明示的に変換して指定することができる。(次の例)

```
fin.seekg(static_cast<std::streamoff>(1024), std::ios::beg);
```

注意) `std::streamoff` から `int` や `long long` などの一般の整数型へさらに変換することも可能であるが、その場合には、変換先の整数型で値を十分に表現できることを確認する必要がある。

このように、入出力位置や移動量は、基本的には `std::streampos` や `std::streamoff` といった専用の型のまま扱い、一般の整数型として扱う必要がある場合にのみ、必要な型変換を行うのが安全である。

■ アクセス位置や移動量を入出力処理の外部で扱う方法

`tellg` や `tellp` によって取得した現在位置を、ストリームの外部で数値として扱いたい場合には、`std::streamoff` を経由して一般の整数型へ明示的に変換することができる。

例. 現在位置を `long long` として取得する

```
auto pos = fin.tellg();
long long n = static_cast<long long>(static_cast<std::streamoff>(pos));
```

ただし、一般の整数型へ変換する場合には、変換先の型で値を十分に表現できることを確認する必要がある。

逆に、ファイル内の位置や移動量を入出力処理とは独立に計算した場合には、最終的に `std::streamoff` 型へ明示的に変換して `seekg` や `seekp` に渡すことができる。

例. 外部で計算した位置を用いる場合

```
long long pos = 1024;
fin.seekg(static_cast<std::streamoff>(pos), std::ios::beg);
```

また、現在位置を基準として移動する場合にも、同様に移動量を計算してから `std::streamoff` に変換して用いることができる。

例. 相対位置の移動

```
auto cur = fin.tellg();
long long offset = 256;
fin.seekg(cur + static_cast<std::streamoff>(offset));
```

このように、必要に応じて計算を整数型で行い、ストリームに渡す際に専用の型へ変換することで、柔軟に入出力位置を制御することができる。

▲注意▲ この方法が安全に適用できるのは、ファイル内の位置をバイト単位の値として扱うことができる場合に限られる。すなわち、バイナリファイルに対しては、この方法により位置を正確に制御することができるが、テキストファイルに対しては注意が必要である。

テキストファイルでは、改行コードの変換などが内部で行われることがあり、ファイル内の論理的な位置と実際の

バイト位置が一致しない場合がある。このため、外部で計算した値をそのまま位置として用いると、処理系によっては意図した位置へ正しく移動できない可能性がある。

重要) 入出力位置を数値として厳密に扱う必要がある場合には、ファイルをバイナリモードでオープンし、バイト単位の位置として管理することが望ましい。テキストモードでは、外部で計算した位置を直接用いる方法は一般には推奨されない。

このように、入出力位置の計算を外部で行うこと自体は有効な手法であるが、ファイルの種類やモードによって扱い方が異なるため、その前提条件を理解した上で適切に使用することが重要である。

11.4.3.6 ファイルの一括読み込み

ファイルからデータを読み取る場合には、必要な量ずつ順に読み取って処理する方法のほか、ファイルの内容全体を一度にメモリ上へ読み込む方法もある。後者は、ファイル全体の内容をまとめて処理したい場合に便利であり、比較的小さなファイルに対しては利便性の高い方法である。ただし、ファイル全体をメモリ上に保持するため、大きなファイルを扱う場合にはメモリ使用量に注意する必要がある。

■ テキストファイルの一括読み込み

テキストファイルの内容全体を1つの **string オブジェクト**¹⁰⁵ として取り出す場合は、入力ファイルストリームの内容を文字列出力ストリームに送り、最後に **string オブジェクト** として取り出す方法が有効である。

入力ファイルストリーム **ifs** の内容全体を、**string オブジェクト text** に読み込む基本的な書き方は次のようなものである。

```
std::ostringstream oss;
oss << ifs.rdbuf();
std::string text = oss.str();
```

ここで、**rdbuf()** は、ストリームに関連付けられているストリームバッファへのポインタ¹⁰⁶ を返す関数である。これを出力ストリームへ挿入することで、入力ファイルストリームから読み取れる内容をまとめて別のストリームへ転送できる。この例では、ファイル全体の内容が文字列出力ストリーム **oss** に送られ、最後に **oss.str()** によって **string オブジェクト** として取り出される。

■ バイナリファイルの一括読み込み

バイナリファイルの場合には、入力ファイルストリームをバイナリモードでオープンし、ファイルサイズを調べてその大きさのバイト列を用意し、**read** 関数によって内容全体を読み込むことができる。例えば、入力ファイルストリーム **ifs** からファイル全体を **byte 型**¹⁰⁷ の **vector** のオブジェクト¹⁰⁸ **data** に読み込む基本的な書き方は次のようなものである。

```
ifs.seekg(0, std::ios::end);
auto size = static_cast<std::streamoff>(ifs.tellg());
ifs.seekg(0, std::ios::beg);
std::vector<std::byte> data(static_cast<std::size_t>(size));
ifs.read(reinterpret_cast<char*>(data.data()),
         static_cast<std::streamsize>(data.size()));
```

注) ここでは、ファイルのオープンや位置取得が正常に行われたものとしている。

最初に **seekg** によってファイル終端へ移動し、**tellg** によってファイルサイズを取得している。その後、アクセス位置をファイル先頭へ戻し、得られたサイズ¹⁰⁹ に合わせて **byte 型**の要素を持つ **vector オブジェクト data** を作成する。最後に **read** によってファイル全体を一括して読み込む。

read 関数が読み込み先として **char*** 型の値を受け取るため、この例では **data.data()** によって得られる **byte***

¹⁰⁵ C++ における標準的な文字列の形式であり、**char** 型の配列とは異なる。詳しくは後の「11.6.4 C++の標準文字列クラス」(p.197～)で解説する。

¹⁰⁶ **std::streambuf***

¹⁰⁷ 「11.6.3 バイトを表す型 (byte 型)」(p.196～)で詳しく解説する。

¹⁰⁸ 「20 STL の基礎」(p.384)で詳しく解説する。

¹⁰⁹ **static_cast** で型の変換をしている。**static_cast** に関しては、「11.6.7.1 static_cast」(p.202)で解説する。

型のポインタを `reinterpret_cast<char*>` によって変換¹¹⁰ している。読み込まれたデータは文字列ではなく、バイト列として `data` に保持される。

注意) バイナリファイルには画像や音声など大容量となるものも多いため、ファイル全体の一括読み込みが常に適切であるとは限らない。大きなファイルを扱う場合には、一定量ずつ読み込んで処理する方法を用いることが望ましい。

11.4.4 ストリームに対する各種の関数

これまでに見てきたように、C++ の入出力はストリームを用いて統一的に扱われる。標準入力を表す `cin`、標準出力を表す `cout`、およびファイル入出力に用いる `ifstream`、`ofstream`、`fstream` などは、いずれも同じストリームの仕組みに基づいており、入力ストリーム同士、あるいは出力ストリーム同士では、共通する多くの操作を用いて扱うことができる。

このため、これらのストリームオブジェクトに対しては、メンバ関数を用いて、入出力の制御や状態の確認を行うことができる。例えば、入力の成否を判定する関数や、ストリームの状態をリセットする関数、あるいは入力処理を補助する関数などが用意されている。入力に関する関数の多くは、標準入力に対してもファイル入力に対しても、同様に使用することができる。

実際のプログラムでは、単にデータを読み書きするだけでなく、入力が正しく行われたかどうかを確認したり、必要に応じてストリームの状態を回復したりすることが重要である。そのため、これらのメンバ関数を適切に利用することで、より安全で堅牢な入出力処理を記述することができる。

11.4.4.1 ストリーム状態の検査

ストリームに対する入力や出力の処理は、常に成功するとは限らない。例えば、ストリームにはエラーや入力の終端 (EOF) などを示す状態が設定される。このような状況に適切に対応するためには、ストリームの状態を検査することが重要である。

ストリームの状態を検査するための主な関数を、表 33 に示す。

表 33: ストリーム状態を検査するための関数	
関数	機能
<code>good()</code>	正常状態であるかを判定する。
<code>fail()</code>	入出力処理が失敗状態 *1 になったかを判定する。
<code>bad()</code>	回復が困難な入出力エラー *2 などを判定する。
<code>eof()</code>	ファイル終端に到達したかを判定する。

*1 形式エラーなど、 *2 ストリームバッファの異常など

これらの関数は、ストリームオブジェクトに対して次のように使用する。

```
ストリーム名.good();           |   ストリーム名.bad();
ストリーム名.fail();           |   ストリーム名.eof();
```

これらはいずれも真偽値 (`bool` 型) を返し、対応する状態であれば `true` (真) となる。通常の入力処理では、特に `fail()` を用いることが多く、`eof()` は補助的に用いられる。例えば、入力が成功したかどうかを判定するには、次のように記述する。

```
int x;
std::cin >> x;
if ( std::cin.fail() ) {
    // 入力失敗時の処理
}
```

また、ファイル入力において、終端に到達したかどうかを判定するには、次のように記述できる。

```
if ( fin.eof() ) {
    // ファイル終端に到達した場合の処理
}
```

¹¹⁰ 「バイナリデータの扱いと型変換」(p.188) で解説する。

ただし、ファイルからの入力では、EOF（ファイル終端）に到達した時点で直ちに処理を行うのではなく、入力処理の成否そのものを判定する方法¹¹¹ が一般的である。

■ 入力成功を判定する例

次のように記述すると、入力が成功した場合のみ処理を継続することができる。

```
int x;
while ( fin >> x ) {
    // x を用いた処理
}
```

このように、ストリームは入力や出力の結果に応じて内部状態を保持しており、これらの関数を用いることで、その状態を適切に検査し、入出力処理を適切に制御することができる。

11.4.4.2 ストリーム状態の操作

前節で述べたように、ストリームは入出力の結果に応じて内部状態を保持しており、エラーが発生すると失敗状態となる。この状態のままでは、その後の入出力処理が正常に行えない場合があるため、必要に応じてストリームの状態を操作して回復することが重要である。

ストリームの状態を操作するための主な関数を、表 34 に示す。

表 34: ストリーム状態を操作するための関数

関数	機 能
<code>clear()</code>	状態を正常状態に戻す。
<code>rdstate()</code>	現在の状態を取得する。
<code>setstate()</code>	状態フラグを設定する。

これらの関数は、ストリームオブジェクトに対して次のように使用する。

```
ストリーム名.clear();
ストリーム名.rdstate();
ストリーム名.setstate( 状態 );
```

通常のプログラムにおいて特に重要なのは `clear()` であり、入力失敗後にストリームの状態を回復するために用いられる。例えば、数値を入力する際に、数値として解釈できない文字列が入力されると、ストリームは失敗状態となり、その状態を解除するまで後続の入力処理を正常に行うことができなくなる。この場合、`clear()` を用いて状態を正常に戻し、さらに不要な入力を破棄する必要がある。

■ 入力失敗からの回復の例

```
int x;
std::cin >> x;
if ( std::cin.fail() ) {
    std::cin.clear();
    std::cin.ignore(1000, '\n');
}
```

この例では、入力に失敗した場合に `clear()` によってストリームの状態を正常に戻し、`ignore()` によって入力バッファに残っている不要なデータを破棄している。（`ignore` 関数とそれに与える値については後で解説する）

また、`rdstate()` を用いると、現在の状態フラグを取得することができる。これは、より詳細な状態判定を行う場合に用いられる。

```
auto state = std::cin.rdstate();
```

さらに、`setstate()` を用いると、任意の状態フラグを設定することができる。

```
std::cin.setstate( std::ios::failbit );
```

ただし、`rdstate()` や `setstate()` は通常の入出力処理ではあまり使用されず、ストリームの状態を詳細に制御する場合や、高度な処理において用いられることが多い。

¹¹¹`eof()` は、読み取り処理によってファイル終端が検出された後に `true` となる。

このように、ストリームの状態を適切に操作することで、入力失敗後の回復や、より柔軟な入出力処理を実現することができる。

11.4.4.3 入力補助関数

C++ のストリーム入力では、`>>` 演算子を用いた書式付き入力が基本となる。この方法は、数値や単語などを簡潔に読み取ることができる反面、入力の区切り（空白や改行）に依存するため、柔軟な入力処理を行うには不十分な場合がある。

例えば、1 行全体を文字列として読み取りたい場合や、入力バッファに残っている不要なデータを除去したい場合には、専用の補助関数を用いる必要がある。これらの関数は、書式付き入力では扱いにくい処理を補うために用いられる。

代表的な入力補助関数を表 35 に示す。

表 35: 主な入力補助関数	
関数	機能
<code>getline</code>	1 行全体を文字列として読み取る。
<code>get</code>	1 文字または文字列を読み取る。
<code>peek</code>	次に読み取る文字を取り出さずに参照する。
<code>ignore</code>	指定した文字数または区切りまで読み飛ばす。

これらの関数は、入力の粒度（行単位・文字単位）や、入力バッファの制御を行うために用いられる。以下では、それぞれの特徴と使い方について簡単に説明する。

■ 行単位の入力 (`getline`)

`getline` は、改行文字までの 1 行を文字列として読み取る関数である。

書き方： `getline( ストリーム, string 型変数 )`

「ストリーム」から 1 行分の内容（改行コードまで）を読み取り、「string 型変数」に設定する。読み取り内容に改行は含めないが、改行コード自体は読み取られてストリームから取り除かれる。すなわちバッファには残らない。（次の例）

```
std::string s;
std::getline(std::cin, s);
```

この関数を用いることで、空白を含む文字列をまとめて取得することができる。この例の `std::string` は `char` の配列ではなく、C++ の標準ライブラリ独特の型である。`char` の配列として文字列を受け取るには、ストリームのメンバ関数である `getline` を用いる。

書き方： `ストリーム.getline( char 配列, 長さ )`

「ストリーム」から「長さ」未満（バイト単位）の内容を読み込み「char 配列」に格納する。「長さ」を読み込む前に改行コードがあれば、読み込みはそこまでとする。読み取り内容に改行は含めないが、改行コード自体は読み取られてストリームから取り除かれる。（次の例）

```
char s[100];
std::cin.getline(s, 100);
```

■ 1 文字単位の入力 (`get`, `peek`)

`get` は 1 文字を読み取る関数であり、`peek` は次に読み取る文字を取り出さずに参照する関数である。

```
int c = std::cin.get();
int d = std::cin.peek();
```

これらを用いることで、入力内容を細かく制御することができる。

■ 入力の調整 (`ignore`)

`ignore` は、入力バッファに残っている不要なデータを読み飛ばすための関数である。例えば、

```
std::cin.ignore(1000, '\n');
```

とすると、改行文字が現れるまで最大 1000 文字を読み飛ばす。これは見やすさを優先したサンプルであるが、実際

には

```
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

として、ストリームで扱うことのできる最大値を指定する。

この関数は、書式付き入力と行単位入力を組み合わせる場合に、特に重要となる。

■ 書式付き入力との関係

>> による入力と `getline` を組み合わせて用いる場合には注意が必要である。>> による書式付き入力では、入力後の改行文字がストリームに残る場合がある。この状態で直後に `getline` を呼び出すと、その改行文字が直ちに読み取られ、空文字列が取得されることがある。

例. 入力の混在による問題

```
int x;
std::cin >> x;
std::string s;
std::getline(std::cin, s);
```

この場合、`s` には空文字列が格納される。これを防ぐためには、`getline` の前に `ignore` を用いて改行を取り除く。

例. 正しい処理

```
std::cin >> x;
std::cin.ignore(1000, '\n');
std::getline(std::cin, s);
```

このように、入力補助関数を適切に用いることで、ストリーム入力の挙動を細かく制御し、より柔軟な入力処理を記述することができる。

11.4.4.4 バイナリ入出力に関わる関数

C++ では、バイナリ形式でデータを読み書きするための関数として、`read` および `write` が用意されている。これらの関数の詳細については、先に解説した通り¹¹² である。本節では、それらの関数を実際に利用する際に重要となる補助的な機能について説明する。

バイナリ入出力では、テキスト入出力とは異なり、データをバイト単位でそのまま扱う。そのため、実際にどれだけのデータが読み取られたか、あるいはファイルの終端にどのように到達するかを正しく理解することが重要である。

■ 実際に読み取られたバイト数：gcount

`read` や `get` などの非書式入力では、指定したサイズのデータが常に読み取られるとは限らない。特にファイル終端 (EOF) 付近では、要求したバイト数よりも少ないデータしか読み取られない場合がある。

このような場合に、実際に読み取られたバイト数を取得するための関数が `gcount` である。

```
fin.read(buf, 100);
auto n = fin.gcount();
```

この例では、最大で 100 バイトの読み取りを試み、実際に読み取られたバイト数が `n` に格納される。

■ 読み取りとファイル終端の関係

`read` による入力では、ファイル終端に到達した場合でも、その時点で読み取ることができるデータが存在すれば、その分だけは読み取りが行われる。そのため、単純に `read` の成否だけでループを制御すると、最後のデータを正しく処理できないことがある。

例えば、次のようなコードは注意が必要である。

```
while ( fin.read(buf, size) ) {
    // データ処理
}
```

この場合、最後の読み取りがサイズ未満で終了した場合には、そのデータが処理されない可能性がある。これを正しく扱うには、`gcount` を併用して、読み取られたデータが存在するかどうかを判定する。

¹¹² 「11.4.3.4 バイナリファイルの扱い」(p.179) で解説した。

例. 安全な読み取りループ

```
while (true) {
    fin.read(buf, size);
    auto n = fin.gcount();
    if (n == 0) break;
    // n バイト分のデータ処理
}
```

このように、毎回の読み取り結果を明示的に確認することで、読み取られたデータ量を基準として安全な処理を行うことができる。

■ バイナリデータの扱いと型変換

バイナリ入出力では、変数などのメモリ上の表現をバイト列として読み書きすることができる。この際には、対象の変数のアドレスを `char*` 型として扱う必要があるため、`reinterpret_cast` を用いる。

```
int x;
fin.read(reinterpret_cast<char*>(&x), sizeof(x));
```

このようにして読み取られたデータは、メモリ上のビット列としてそのまま変数に格納される。このような方法は、基本型や単純なデータ構造など、メモリ上の表現をそのまま扱えるデータに対して用いられる。

ただし、この方法はデータの内部表現に依存するため、異なる環境間でデータをやり取りする場合には、バイト順（エンディアン）やデータ型のサイズ、内部表現などの違いによって、正しく解釈できない可能性があるため注意が必要である。

■ バイナリ入出力の特徴

バイナリモードでの入出力では、テキストモードで行われることのある改行コードの変換などは行われない。そのため、データはバイト単位でそのままファイルに保存される。この性質により、書式変換を伴わず効率的にデータを読み書きできる一方で、人間が直接内容を確認することは困難である。

このように、バイナリ入出力では、単に `read` や `write` を用いるだけでなく、実際に読み取られたデータ量やファイル終端の扱いを正しく理解することが重要である。

11.4.4.5 その他の機能

本節では、これまでに解説してきた入出力機能を補完するものとして、ストリームの挙動や性能に関わるいくつかの機能について説明する。これらは日常的に必ずしも使用するものではないが、適切に理解して用いることで、入出力処理の効率や挙動を制御することができる。

■ 入出力の同期と高速化

C++ の標準ストリーム入出力は、C 言語の標準入出力（`stdio`）との互換性を保つため、初期状態では対応する入出力が同期されている。このため、`printf` や `scanf` などと C++ の標準ストリーム入出力を混在して扱うことができる。しかし、この同期処理には一定のオーバーヘッドがあるため、大量の入出力を行う場合には性能低下の原因となることがある。この同期を解除するには、次の関数を用いる。

```
std::ios::sync_with_stdio(false);
```

この関数を呼び出すことで、C++ のストリームと C の標準入出力との同期が解除され、入出力処理が高速化される場合がある。ただし、この設定を行った場合には、`printf` や `scanf` などの関数と混在して使用すると、C++ のストリームと C の標準入出力との間で入出力の順序が意図通りにならないことがあるため注意が必要である。

■ 入力と出力の結び付け：tie

C++ では、標準入力 `cin` は、既定では標準出力 `cout` を結び付け先として持ち、入力操作が行われる直前に `cout` の出力バッファをフラッシュする（すなわち、出力を確定させる）仕組みとなっている。これにより、対話的なプログラムにおいて、入力待ちに入る前にプロンプトなどの出力が表示される。しかし、この自動フラッシュも性能低下の要因となることがあるため、必要に応じてこの結び付けを解除することができる。

```
std::cin.tie(nullptr);
```

これにより、入力のたびに出力バッファがフラッシュされることがなくなり、入出力の高速化が期待できる。ただし、

出力が遅れて表示される可能性があるため、対話的な用途では注意が必要である。

ここで用いている `nullptr` は、「何も指していないポインタ」を表す特別な値である。すなわち、`cin.tie(nullptr)` とすることで、`cin` に結び付けられている出力ストリームを「なし」に設定し、結び付けを解除することを意味する。

C 言語ではこのような値として `NULL` が用いられていたが、C++ では型安全性の観点から `nullptr` を用いることが推奨される。

■ 出力の確定：flush と endl

ストリームへの出力は、通常、内部バッファに蓄えられ、適切なタイミングでまとめて実際の出力先へ送られる。このバッファリングにより、入出力の効率が向上する。

出力を直ちに確定したい場合には、`flush` マニピュレータを用いる。

```
std::cout << std::flush;
```

また、`endl` マニピュレータは、改行を出力すると同時に、出力バッファのフラッシュを行う。

```
std::cout << std::endl;
```

`endl` は便利であるが、頻繁に使用すると不要なフラッシュが発生し、性能低下の原因となることがある。そのため、単に改行を行いたい場合には、次のように改行文字を直接出力する方法が推奨される。

```
std::cout << '\n';
```

このように、出力のタイミングを明示的に制御することで、効率と挙動のバランスを適切に取ることができる。

以上のように、ストリーム入出力には、その挙動や性能に関わる様々な機能が用意されている。これらを適切に理解した上で使用することで、より効率的で意図通りの入出力処理を実現することができる。

11.4.5 C 言語の stdio の利用

C++ においても、C 言語の標準入出力ライブラリ（`stdio`）を利用することができる。例えば、`printf` や `scanf` を用いた入出力は、C++ のプログラム中でも利用することができる。ただし、C++ では前節で述べた `iostream` による入出力が標準的に用いられており、型に応じた処理が自動的に行われるなど、より安全で柔軟な記述が可能である。このため、第2部では `iostream` を基本として解説を行う。

なお、`printf` などの関数では、書式文字列によって出力形式を指定する必要があり、引数の型と変換指定子が適合していない場合には、正しく処理されないことがあるため注意すること。

C++ では、`cstdio` をインクルードすることで、C 言語の `stdio` に由来する関数を利用することができる。これらの関数は `std` 名前空間に属する。

11.5 名前空間

大規模なプログラムでは、多数の関数、変数、型、クラス（後述）などが定義される。また、実用的なプログラムでは、自分で作成したソースファイルだけでなく、標準ライブラリや外部ライブラリも利用することが多い。このような状況では、異なる場所で同じ名前が使われてしまう可能性がある。

例えば、複数のソースファイルに分割してプログラムを作成しているとき、プログラム全体で同じ名前の関数や変数が競合することがある。また、自分が定義した名前が、後から利用するライブラリの中で使われている名前と衝突する場合もある。このような問題を**名前の衝突**という。

C 言語では、このような衝突を避けるために、関数名や型名などに長い接頭辞を付けるなどの方法がよく用いられてきた。例えば、画像処理用の関数には `img_`、文字列処理用の関数には `str_` のような接頭辞を付けることで、名前の重複を避ける方法である。しかし、この方法はあくまで命名規則による工夫であり、言語仕様として名前を整理する仕組みではない。

C++ では、この問題に対処するために、**名前空間 (namespace)** という仕組みが用意されている。名前空間を用いると、関数、変数、型、クラスなどの名前を特定の領域にまとめることができる。これにより、同じ名前であっても、異なる名前空間に属していれば別のものとして扱うことができ、名前の衝突を回避することができる。

名前空間は、特に大規模なプログラムやライブラリを利用するプログラムにおいて重要である。また、C++ の標準ライブラリで提供される多くの名前は `std` という名前空間に属している。そのため、これまでに用いてきた `cout` や `cin` は `std` 名前空間に属しており、完全な名前¹¹³ は `std::cout`、`std::cin` である。

本節では、名前空間の基本的な考え方と、`std::` の意味、および `using namespace` の使い方について解説する。

11.5.1 名前空間の指定

名前空間に属する名前を明示的に指定して使用するには、名前空間名と名前とを**スコープ解決演算子「::」**で結合して記述する。

```
std::cout << "Hello\n";
std::cin >> x;
```

これまでに用いてきた `cout` や `cin` は、実際には `std` という名前空間に属しており、厳密にはこのように記述される。

しかし、このような記述を毎回行うのは煩雑であるため、C++ では `using` を用いて名前空間の使用を簡略化することができる。

まず、特定の名前のみを導入する方法がある。

```
using std::cout;
using std::cin;
```

このように記述すると、以降は `cout` や `cin` を名前空間名を付けずに使用することができる。

また、名前空間に属する名前をまとめて名前空間名なしで参照できるようにする方法として、`using namespace` を用いることもできる。

```
using namespace std;
```

この記述により、`std` 名前空間に属する名前を、名前空間名を付けずに参照できるようになる。ただし、`using namespace` を用いると、多数の名前が名前探索の対象となるため、他の名前との衝突を引き起こす可能性がある。このため、大規模なプログラムでは、必要な名前のみを `using` によって導入する方法が望ましい。

以上のように、名前空間はスコープ解決演算子によって明示的に使用方法と、`using` を用いて簡略化する方法とがあり、状況に応じて適切に使い分けことが重要である。

11.5.2 主な名前空間

C++ の標準ライブラリでは、多くの機能が名前空間 `std` にまとめられている。本書でこれまでに扱った `cout` や `cin` も、この名前空間に属する。

¹¹³完全修飾名

実際のプログラムでは、標準ライブラリのほかにも、各種のライブラリが独自の名前空間を用いて機能を提供している。ここでは、C++ の利用において比較よく用いられる名前空間を表 36 に挙げる。

表 36: 主な標準ライブラリの名前空間

名前空間	主な内容
<code>std</code>	標準ライブラリ全般（入出力、文字列、コンテナなど）
<code>std::chrono</code>	時刻・時間に関する機能
<code>std::filesystem</code>	ファイルシステム操作
<code>std::this_thread</code>	スレッド操作
<code>std::ranges</code>	範囲ベースの処理（C++20 以降）

これらの名前空間は、必要に応じて `using` やスコープ解決演算子「`::`」を用いて参照することができる。なお、C++ 標準ライブラリの機能は基本的に `std` 名前空間、またはその内部に設けられた名前空間に属している。

11.5.3 独自の名前空間の定義

これまでに述べた名前空間は、標準ライブラリなどで定義されたものを利用する場合であったが、ユーザが独自に名前空間を定義することもできる。

名前空間は、`namespace` キーワードを用いて定義することができる。

《独自の名前空間の定義》

`namespace 空間名 { (定義内容) }`

「空間名」の名前空間を定義し、「定義内容」をその内部に含める。

例えば次のようにして、名前空間 `mynamespace` を定義して、その中に関数 `add` を定義することができる。

```
namespace mynamespace {
    int add(int x, int y) {
        return x + y;
    }
}
```

このように定義された関数は `mynamespace` に所属しており、スコープ解決演算子「`::`」を用いて、次のように呼び出すことができる。

```
int z = mynamespace::add(3, 4);
```

名前空間の内部には、変数、関数、構造体、クラス、列挙型などの宣言および定義を記述することができる。ただし、関数本体の中に記述するような実行文を、名前空間の直下に記述することはできない（変数の初期化などは記述可能）。

なお、同じ名前空間の定義は複数の場所に分けて記述することが可能であり、それらは同一の名前空間として扱われる。例えば、

```
namespace mynamespace {
    int a;
}

namespace mynamespace {
    int b;
}
```

と記述すると、`mynamespace` は `a` と `b` の両方を含む名前空間となる。

このような性質により、複数のソースファイルに分割されたプログラムにおいても、同一の名前空間に機能を追加していくことができる。ただし、意図せず既存の名前空間に内容を追加してしまう可能性があるため、名称の設計には注意が必要である。

次のプログラム `test201-3.cpp` では、名前空間 `MyNameSp1` と `MyNameSp2` が定義されており、それぞれの中で同じ名前の関数 `writeMsg` が定義されている。ただし、それぞれの関数の定義内容は異なっており、異なる内容を入力する。main 関数からは、それぞれの名前空間の関数 `writeMsg` を呼び出している。

プログラム：test201-3.cpp

```
1  #include <iostream>
2
3  namespace MyNameSp1 {
4      void writeMsg() {
5          std::cout << "こんにちは. \n";
6      }
7  }
8
9  namespace MyNameSp2 {
10     void writeMsg() {
11         std::cout << "Hello.\n";
12     }
13 }
14
15 int main() {
16     MyNameSp1::writeMsg();
17     MyNameSp2::writeMsg();
18 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test201-3.cpp のコンパイルと実行

\$ g++ test201-3.cpp -o test201-3 ←コンパイル

\$./test201-3 ←実行

こんにちは. ← MyNameSp1 の writeMsg 関数からの出力

Hello. ← MyNameSp2 の writeMsg 関数からの出力

同じ名前の関数名が、異なる名前空間に属することで衝突せずに共存できることがわかる。これは、それぞれの関数が異なる名前空間に属しており、別の名前として扱われるためである。すなわち、これらの関数は厳密には `MyNameSp1::writeMsg` および `MyNameSp2::writeMsg` という異なる名前として扱われる。

11.6 C++における型の考え方

C 言語においても変数には型があり、それに応じたデータの扱いが行われる。しかし C 言語では、異なる型の間でも暗黙の型変換が行われる場面が比較的多く、必要に応じてキャストによって柔軟に処理を記述することができた。

これに対して C++ では、型は単なるデータの分類ではなく、プログラムの意味や意図を表現するための重要な要素として位置づけられている。そのため、型をより積極的に用いて意味や意図を表現し、意図しない変換や型の混在を避ける設計が重視されている。

このような設計により、プログラムの安全性が向上するとともに、可読性や保守性の向上が期待できる。すなわち、型を明確に記述することで、プログラムがどのような値を扱い、どのような処理を意図しているのかを、コードに直接反映することができる。

本節では、このような C++ における型の考え方を踏まえた上で、まず論理型 (bool) について説明し、続いて型変換の方法、さらに型によって意味を表現する設計について解説する。

11.6.1 論理型 (bool 型)

C++ では、**論理型** (bool 型) により、真偽を明確に表現することができる。論理型は `true` と `false` の 2 値のみを取る型である。これらは論理値を表すリテラルであり、プログラムにおける条件や状態を表すための重要な役割を持っている。

C 言語では、従来から整数値を用いて真偽を表現する方法が広く用いられており、0 を偽、0 以外を真として扱う。C 言語にも論理値を扱うための型は用意されているが、整数値による真偽表現も広く用いられている。しかし、この方法では、値が「条件」を表しているのか、「数値」を表しているのかが曖昧になることがあり、プログラムの可読性の低下につながることもある。

これに対して C++ では、bool 型を用いることで、プログラムにおける意図を明確に表現することができる。例えば、2 状態を表す

`isPlaying, isValid, hasError`

のような変数名を論理型で宣言することで、「状態」や「条件」を直接的に表すことが可能となる。実際に論理型の値は、条件分岐や反復処理の判定に直接用いられる。

条件を表す式は、そのまま論理型の値として扱うことができる。例えば、次のように比較演算の結果を論理型の変数に代入することができる。

```
bool b1 = (3 < 5);
bool b2 = (x == 0);
```

これらの式はそれぞれ、条件が成り立つ場合には `true`、成り立たない場合には `false` となる。

このように、条件式の評価結果をそのまま論理型の値として扱うことができるため、条件の内容を変数として保持したり、複雑な条件を分割して扱うことが可能となる。

論理型の変数を用いてプログラムの動作を制御する例を `test201-4.cpp` に示す。

プログラム：test201-4.cpp

```
1 #include <iostream>
2 using std::cout;
3
4 int main() {
5     int n = 10;
6     bool exe = true;    // 論理型変数
7
8     while ( exe ) {      // 反復の実行条件を論理型で判定
9         cout << n << " ";
10        n--;
11        exe = (n > 0);    // 条件判定の結果を値として代入
12    }
13    cout << "\n";
14 }
```

このプログラムは、`while` による反復条件を `bool` 型の変数 `exe` で判定している。また、値を判定する条件式の結果を「`exe = (n > 0)`」として `exe` に代入し、反復条件を更新している。

このプログラムをコンパイルして実行する例を次に示す。

例. `test201-4.cpp` のコンパイルと実行

```
$ g++ test201-4.cpp -o test201-4  Enter    ←コンパイル
$ ./test201-4  Enter    ←実行
10 9 8 7 6 5 4 3 2 1    ←出力
```

このように、論理型は単なるデータ型ではなく、プログラムの意味構造を明確にするための重要な要素である。

11.6.1.1 bool 値の入出力時の形式

`bool` 値を入出力ストリームで扱う場合、標準状態では、外部表現として整数値の 1 と 0 が用いられる。すなわち `true` が 1, `false` が 0 に対応する。ただし、`std::boolalpha` を用いると「`true`」,「`false`」の形式で入出力することができる。具体的には、出力ストリームに `std::boolalpha` を送出すると、以降の `bool` 値の出力が「`true`」,「`false`」の形式となる。また、`std::noboolalpha` を送出すると、以降の `bool` 値の出力が整数値の 1 と 0 (デフォルト) になる。

次に示すプログラム `test201-5.cpp` でそれを確認することができる。

プログラム：test201-5.cpp

```
1 #include <iostream>
2 using std::cout, std::boolalpha, std::noboolalpha;
3
4 int main() {
5     bool b1 = true, b2 = false;
6
7     cout << "default:\t";
8     cout << "b1 = " << b1 << ", b2 = " << b2 << "\n";
9
10    cout << "boolalpha:\t";
11    cout << boolalpha;
```

```

12     cout << "b1 = " << b1 << ", b2 = " << b2 << "\n";
13
14     cout << "noboolalpha:\t";
15     cout << noboolalpha;
16     cout << "b1 = " << b1 << ", b2 = " << b2 << "\n";
17 }

```

これをコンパイルして実行する例を次に示す。

例. test201-5.cpp のコンパイルと実行

```

$ g++ test201-5.cpp -o test201-5  ←コンパイル
$ ./test201-5  ←実行
default:      b1 = 1, b2 = 0      ←デフォルトでは整数値の形式
boolalpha:    b1 = true, b2 = false ←表示を切り替えて出力
noboolalpha:  b1 = 1, b2 = 0      ←デフォルトに戻して出力

```

入力ストリームから bool 値を受け取る場合も `std::boolalpha`, `std::noboolalpha` によって形式を制御することができる。

書き方： 入力ストリーム >> `std::boolalpha` >> 変数

これにより、「入力ストリーム」からの "true", "false" を bool 値として「変数」に受け取ることができ、以降はこの形式による bool 値の受け取りとなる。また、

書き方： 入力ストリーム >> `std::noboolalpha` >> 変数

とすることで、1 もしくは 0 を bool 型の「変数」に受け取る設定となり、以降はこの形式による bool 値の受け取りとなる。

入力ストリームから bool 値を受け取る際に、適切でない形式の値である場合は入力処理が失敗となるので、`clear()` 関数で失敗状態を解除し、更に、不適切な値の読み飛ばしなどの回復処理をする¹¹⁴ 必要がある。

次のプログラム test201-6.cpp は、標準入力ストリームからの bool 値の読み取り形式を制御する例である。入力は 3 回行われ、最初はデフォルト（整数値形式）の入力、2 回目は「true/false」形式の入力、3 回目はデフォルトに戻した入力である。

プログラム：test201-6.cpp

```

1  #include <iostream>
2  #include <limits>
3  using std::cout, std::cin, std::boolalpha, std::noboolalpha;
4
5  bool chkInput() {
6      if ( cin.fail() ) {
7          cout << "invalid value!\n";
8          cin.clear();
9          cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
10         return false;
11     }
12     return true;
13 }
14
15 void printBool( bool b ) {
16     if ( b ) {
17         cout << "b = true\n";
18     } else {
19         cout << "b = false\n";
20     }
21 }
22
23 int main() {
24     bool b = false;
25
26     cout << "default:\ninput bool b = ";
27     cin >> b;
28     if ( chkInput() ) printBool(b);
29 }

```

¹¹⁴ 「11.4.4.2 ストリーム状態の操作」(p.185) を参照のこと。

```

30     cout << "boolalpha:\ninput bool b = ";
31     cin >> boolalpha >> b;
32     if ( chkInput() ) printBool(b);
33
34     cout << "noboolalpha:\ninput bool b = ";
35     cin >> noboolalpha >> b;
36     if ( chkInput() ) printBool(b);
37 }

```

各回の入力直後で、入力失敗した場合の回復処理を `chkInput` 関数で行っている。

このプログラムをコンパイルして実行する例を次に示す。

例. test201-6.cpp のコンパイルと実行

```

$ g++ test201-6.cpp -o test201-6  ←コンパイル
$ ./test201-6  ←実行
default:
input bool b = 1  ←デフォルトでは整数形式の入力
b = true ←読み取り結果
boolalpha:
input bool b = true  ← true/false 形式の入力
b = true ←読み取り結果
noboolalpha:
input bool b = 0  ←デフォルトに戻した（整数形式の入力）
b = false ←読み取り結果

```

11.6.2 特殊な数値

浮動小数点数では、通常の数値の他に、NaN（非数）や infinity（無限大）といった特殊な値を扱うことができる¹¹⁵。NaN は “Not a Number” の略であり、数値として定まらない結果を表すために用いられる。例えば、0.0/0.0 のような演算では、処理系によって NaN が生成されることがある。また、正の無限大、負の無限大を表す値も用意されている。

C 言語では、`math.h` ヘッダで定義されるマクロ `NAN` や `INFINITY` を用いて、NaN や無限大を表す値を得ることができた。一方、C++ では、`limits` ヘッダで提供される `numeric_limits` を用いるのが一般的¹¹⁶である。例えば、`double` 型の NaN と正の無限大は、次のようにして得ることができる。

```

非 数：  numeric_limits<double>::quiet_NaN()
無限大：  numeric_limits<double>::infinity()

```

負の無限大は、正の無限大に符号を付けて次のように表すことができる。

```

負の無限大： -numeric_limits<double>::infinity()

```

`numeric_limits` は型を指定して使用するため、`float`、`double`、`long double` など、対象とする浮動小数点型を明確にできる。この点は、C 言語のマクロ `NAN` や `INFINITY` とは異なる。

NaN や無限大であるかどうかを調べるには、`cmath` ヘッダで提供される `isnan`、`isinf`、`isfinite` を使用する。これら関数を使用したプログラム `test201-8.cpp` を次に示す。

プログラム：test201-8.cpp

```

1  #include <iostream>
2  #include <limits>
3  #include <cmath>
4  using namespace std;
5
6  int main() {
7      double x = numeric_limits<double>::quiet_NaN(); // 非数
8      double y = numeric_limits<double>::infinity(); // 無限大
9
10     cout << "x = " << x << ", y = " << y << "\n";
11
12     cout << boolalpha;

```

¹¹⁵非数や無限大は主に浮動小数点型で扱われる特殊な値であり、`int` や `long` などの整数型には存在しない。

¹¹⁶ただし、対象の型が NaN や無限大を表現できる場合に限る。

```

13     cout << "isnan(x)      : " << isnan(x) << "\n";
14     cout << "isinf(y)      : " << isinf(y) << "\n";
15     cout << "isfinite(y)   : " << isfinite(y) << "\n";
16 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test201-8.cpp のコンパイルと実行

```

$ g++ test201-8.cpp -o test201-8  ←コンパイル
$ ./test201-8  ←実行
x = nan, y = inf      ←非数と無限大
isnan(x)      : true   ←判定結果
isinf(y)      : true
isfinite(y)   : false

```

■ IEEE 754 との関係

NaN や無限大は、IEEE 754 で定められた浮動小数点数の特殊な値である。多くの処理系では、float や double は IEEE 754 に沿った形式で実装されており、C 言語の NAN, INFINITY と、C++ の `numeric_limits<double>::quiet_NaN()`, `numeric_limits<double>::infinity()` は、いずれもこの考え方に基づいている。

C++ では、型が ISO/IEC 60559 に準拠しているかどうかを調べるために、`numeric_limits<T>::is_iec559` というメンバ（値は bool 型）が用意されている。（`iec559` は IEEE 754 に対応する IEC 規格の名称に由来する）

11.6.3 バイトを表す型 (byte 型)

C++ では、1 バイトのデータを表すための型として `byte` 型が用意されている¹¹⁷。この型は、名前空間 `std` に属する。`byte` 型を使用するには `cstdint` ヘッダを読み込む。例えば、次のようにして `byte` 型の変数を宣言することができる。

```
std::byte b0x3f;
```

`byte` 型は、文字や整数としての意味を持たない「1 バイトのデータ」を表すことを目的とした型である。C 言語ではバイナリデータを `char` 型や `unsigned char` 型の配列として扱うことが多いが、これらの型は整数としての演算も可能である。これに対して `byte` 型は、データを文字や通常の整数として扱う意図がないことを型によって明示できる。

このため、`byte` 型の値に対して通常の加減乗除などの算術演算を行うことはできないが、ビット単位のデータを扱うためのビット演算を行うことができる。また、`byte` 型の値を整数値として取得するには `std::to_integer` を使用する。

例. `byte` 型の値を整数値として取得する例

```
std::byte b0x3f;
int n = std::to_integer<int>(b);
```

複数のバイトからなるデータを扱う場合は、`byte` 型の配列を使用することができる。例えば、次の宣言では 1024 バイトの領域を確保する。

```
std::byte buf[1024];
```

また、要素数を実行時に変更できるバイト列として扱う場合は、後に解説する `vector` クラス¹¹⁸ と組み合わせて `std::vector<std::byte>` とする方法が便利である。（次の例）

例. `byte` 型の要素を 1024 個持つ `vector` オブジェクト `buf` の宣言

```
std::vector<std::byte> buf(1024);
```

この例では、1024 個の `byte` 型の要素を持つ `vector` オブジェクトを生成している。ファイルから読み込んだバイナリデータや画像データ、通信データなど、大きさが固定されていないバイト列を保持する用途では、このような `vector` オブジェクトを利用することができる。

¹¹⁷C++17 以降

¹¹⁸「20 STL の基礎」(p.384～) で `vector` クラスについて解説している。

11.6.4 C++の標準文字列クラス

C 言語では、文字列は `char` 型の配列として扱われる。すなわち、文字データをメモリ上に並べ、終端文字 `'\0'` によって文字列の終わりを表す。この仕組みにおいて、`char*` は文字列そのものを表す型ではなく、文字データの先頭位置を指すポインタである。この方法は低水準で柔軟であり、C 言語由来の関数を利用する場合や、固定長のバッファを扱う場合には、C++ のプログラミングでも必要になることがある。しかし、C 形式の文字列を直接扱う場合には、文字列の長さ、終端文字、記憶領域の大きさなどをプログラマが意識して扱う必要がある。そのため、扱いを誤ると、バッファオーバーフローなどの不具合の原因になりやすい。

C++ では、文字列をより自然な形で扱うために、標準ライブラリのクラス `std::string` が用意されている。`string` は、文字列データとその記憶領域を管理する機能、および文字列操作のためのメンバ関数群を備えたクラスであり、文字列を 1 つのオブジェクトとして管理する。なお、**クラスとオブジェクト**については「12 クラスとオブジェクト」(p.237～) で改めて解説するが、ここでは、`string` を「文字列を扱うための標準的な型」として理解しておけばよい。

`string` を用いると、文字列を `char` 型配列やポインタとしてではなく、1 つの値のように扱うことができる。例えば、文字列の代入、比較、連結、長さの取得などを、通常の変数に近い方法で記述できる。これは、C 言語形式の文字列を直接扱う場合との大きな違いである。

C++ の標準ライブラリを利用するプログラムでは、文字列を `string` オブジェクトとして扱うのが一般的である。特に、文字列を STL のコンテナ¹¹⁹ の要素として扱う場合には、`string` を用いることで、文字列を自然に扱うことができる。

注意) C++ でも、`char` 型の配列として文字列を表現して扱うことは可能であり、C 言語由来の関数を利用する場合や、固定長のバッファを扱う場合には、現在でも必要となることがある。ただし、C++ における通常の文字列処理では、文字列の長さや記憶領域の大きさを `string` に管理させる方が安全で自然である。

`string` を使用するには、ヘッダファイル `string` をインクルードする。また、`string` は `std` 名前空間に属しているため、通常は `std::string` と記述する。

書き方： `string` 変数名

例えば、文字列を格納する変数 `s` は次のように宣言できる。

```
std::string s;
```

この宣言により、文字列を保持するための `string` オブジェクト `s` が作成される。この段階では、`s` は空の文字列を保持している。

`string` オブジェクトは、宣言時に文字列を与えて初期化することもできる。

例. `string` 変数の宣言と初期化

```
std::string s1 = "Hello";      ← 文字列リテラルを与える形式
std::string s2("World");      ← string コンストラクタに初期値を与える形式
```

この例では、`s1` は文字列 `"Hello"` で初期化され、`s2` は文字列 `"World"` で初期化される。

`string` オブジェクトの基本的な取扱いを表 37 に挙げる。

注意) `string` の `size()` は、見た目の文字数を返すとは限らない。特に、UTF-8 の日本語文字列では、1 文字が複数の `char` で表されるため、`size()` の値は日本語の文字数とは一致しない。

注意) `s[i]` の形式では範囲検査は行われないので、要素範囲を外れる添字を指定した場合には、未定義動作となることがある。一方、`s.at(i)` は範囲検査を行い、有効範囲外の位置を指定した場合には例外を送出する。

`string` には、文字列の一部を取り出す、文字列中を検索する、文字列を挿入・削除・置換する、数値と文字列を相互に変換する、といった実用的な機能も用意されている。表 38 に、代表的な応用的操作を示す。

11.6.4.1 string オブジェクトとして扱えるデータサイズ

固定長の `char` 型配列を文字列バッファとして扱う場合と異なり、`string` オブジェクトでは、使用に先立って文字列の最大長を決めて記憶領域を確保しておく必要はない。これは、文字列の記憶領域の管理を `string` オブジェクトが行うためである。文字列の長さに応じて、必要な記憶領域はライブラリ側で管理される。

¹¹⁹ 「20 STL の基礎」(p.384～) で解説する。

表 37: string の基本的な操作 (一部)

操作	記述例	補足
宣言 初期化 1 初期化 2	<code>std::string s;</code> <code>std::string s = "Hello";</code> <code>std::string s("Hello");</code>	文字列リテラルで初期化 コンストラクタによる初期化
代入 出力	<code>s = "World";</code> <code>std::cout << s;</code>	既存の string オブジェクトへの代入 出力ストリームへの送出
連結 末尾への追加	<code>s = s1 + s2;</code> <code>s += "abc";</code>	s1, s2 は変化しない s 自身に追加
クリア	<code>s.clear();</code> <code>s = "";</code>	文字列としての内容を消去して、空文字列にする。どちらも同じ結果となる。
比較 空文字列の判定	<code>s1 == s2</code> <code>s.empty()</code>	結果は bool 型 結果は bool 型
長さの取得	<code>s.size()</code>	格納されている char の個数 (size_type 型) が返される。
要素のアクセス	<code>s[i]</code> <code>s.at(i)</code>	範囲検査なし 範囲外では例外を送出

表 38: string の応用的操作 (一部)

操作	記述例	説明
部分文字列の取得 検索 後方検索	<code>s.substr(pos, n)</code> <code>s.find("abc")</code> <code>s.rfind("abc")</code>	位置 pos から n 個の char を取り出す 文字列 "abc" が最初に現れる位置を返す 後方から検索する
挿入 削除 置換	<code>s.insert(pos, "abc")</code> <code>s.erase(pos, n)</code> <code>s.replace(pos, n, "abc")</code>	位置 pos に文字列を挿入する 位置 pos から n 文字分を削除する 位置 pos から n 文字分を置換する
比較 C 文字列への変換 数値から文字列へ変換 文字列から整数へ変換 文字列から実数へ変換	<code>s.compare(t)</code> <code>s.c_str()</code> <code>std::to_string(x)</code> <code>std::stoi(s)</code> <code>std::stod(s)</code>	文字列 t と比較する C 言語形式の文字列へのポインタ(注)を返す 数値 x を文字列に変換する 文字列を整数値に変換する 文字列を実数値に変換する

注) const char*

ただし、string オブジェクトが扱える文字列の長さは無制限ではない。扱うことができる最大の要素数は、次のようにして確認できる。

```
std::string().max_size()
```

ここでいう要素数とは、string に格納される char の個数である。ただし、max_size() が返す値は処理系が定める理論上の上限であり、実際にその大きさの文字列を必ず作成できるわけではない。実際には、使用可能なメモリ量や実行環境の制約を受ける。

11.6.4.2 文字列へ出力するストリーム: ostreamstream

数値を文字列へ変換するには to_string() を用いることができる。しかし、to_string() では、小数点以下の桁数や表示形式を柔軟に指定することはできない。例えば、浮動小数点数を小数点以下 3 桁で表現したい場合や、固定小数点表記、指数表記などを明示的に指定したい場合には別の方法が必要になる。このような場合に利用できるのが、ostreamstream である。

ostreamstream は、文字列へ出力するためのストリームである (名前空間は std)。std::cout が標準出力ヘッダを送出するのに対して、ostreamstream は内部の文字列バッファヘッダを送出する。最後に str() を呼び出すことで、内部に蓄積された内容を string オブジェクトとして取り出すことができる。

ostreamstream では、通常の出力行ストリームと同じように << 演算子を用いて値を送出する。また、std::fixed, std::scientific, std::setprecision() などの書式指定も使用できる。従って、ostreamstream を用いると、数

値や文字列を組み合わせながら、書式を指定した文字列を作成することができる。

`ostringstream` を使用するには、ヘッダファイル `sstream` をインクルードする。また、`std::setprecision()` などの書式指定を使用する場合には、ヘッダファイル `iomanip` もインクルードする。

書き方： `ostringstream` 変数

「変数」に `ostringstream` オブジェクトが作成される。基本的な使用方法を次に例示する。

例. `ostringstream` による `string` オブジェクトの作成

```
std::ostringstream oss;           ← string 作成用ストリームを生成
oss << std::fixed << std::setprecision(3) << x;    ← 書式を設定して x をストリームに出力
std::string s = oss.str();         ← 内部に蓄積された内容を string として取得
```

この例では、変数 `x` (`double` を想定) の値を小数点以下 3 桁の固定小数点表記で `oss` に送出し、その結果を `oss.str()` によって `string` として取り出している。

文字列出力ストリームの内容、すなわち内部バッファの内容は、出力操作を行うたびに順次蓄積されていく。`oss.str("")` を実行すると、この内部バッファの内容を空にすることができる。なお、ストリームの状態も初期状態に戻したい場合には、必要に応じて `oss.clear()` も呼び出す。その他、基本的には通常の入力ストリームと同様の扱いができる。

11.6.4.3 文字列から入力するストリーム： `istringstream`

文字列に含まれるデータを読み取るには、`istringstream` を用いることができる。これは、文字列から入力するためのストリームである (名前空間は `std`)。 `std::cin` が標準入力からデータを読み取るのに対して、`istringstream` はプログラム中の文字列からデータを読み取る。

これは、C 言語の `sscanf()` に近い役割を持つ。例えば、文字列 `"123 3.14 abc"` から整数値、浮動小数点数、文字列を順に取り出したい場合、`istringstream` を用いると、通常の入力ストリームと同じように `>>` 演算子で値を読み取ることができる。

`istringstream` を使用するには、ヘッダファイル `sstream` をインクルードする。

書き方： `istringstream` 変数 (文字列)

「変数」に `istringstream` オブジェクトが作成され、指定した「文字列」からデータを読み取る入力ストリームとして利用できる。基本的な使用方法を次に例示する。

例. `istringstream` による文字列からの値の読み取り

```
int n;
double x;
std::string word;
std::string s = "123 3.14 abc";      ← 読み取り対象の文字列
std::istringstream iss(s);          ← 文字列入力ストリームを生成
iss >> n >> x >> word;              ← 文字列から値を順に読み取る
```

この例では、文字列 `s` に格納された内容から、整数値、浮動小数点数、文字列を順に読み取っている。`>>` による読み取りでは、通常の入力ストリームと同様に、空白文字を区切りとして値を順に読み取ることができる。

なお、読み取りに失敗した場合には、通常の入力ストリームと同様にストリーム状態がエラー状態になる。そのため、必要に応じて読み取り結果を条件式で確認することができる。例えば、次のように書くと、文字列から整数値と浮動小数点数を正しく読み取れた場合だけ処理を行うことができる。

```
if ( iss >> n >> x ) {
    /* 読み取り成功時の処理 */
}
```

同じ `istringstream` オブジェクトに別の文字列を設定して再利用する場合には、`iss.str("")` によって内部の文字列を設定する。また、前の読み取りによって失敗状態や終端状態になっている場合には、`iss.clear()` によってストリーム状態もリセットする。ただし、単純なプログラムでは、新しい `istringstream` オブジェクトを作成する方法をとっても良い。

■ サンプルプログラム

string オブジェクトに関する様々な処理を、次の test201-7.cpp で例示する.

プログラム: test201-7.cpp

```
1  #include    <iostream>
2  #include    <iomanip>
3  #include    <string>
4  #include    <sstream>
5  using std::cout;
6
7  int main() {
8      std::string    s1 = "Hello,", s2 = "World.", s3;
9      std::string::size_type    i;
10
11     // stringオブジェクトが扱える理論上の最大サイズの確認
12     cout << "string().max_size():" << std::string().max_size() << "\n";
13
14     s3 = s1 + s2;        // stringオブジェクトの連結 (複製を返す)
15     cout << "\"" << s1 << " " + "\"" << s2 << " = \"" << s3 << "\", ";
16     cout << "size:" << s3.size() << "\n";
17
18     cout << "----+----1\n";
19     for ( i=0; i < s1.size(); i++ ) cout << s1[i];        // 添字による参照
20     cout << "\n";
21     for ( i=0; i < s2.size(); i++ ) cout << s2.at(i);    // atによる参照
22     cout << "\n";
23
24     s1 = "ABCDEFGHJKLMNOP";
25     cout << "s1=\"" << s1 << "\", ";
26     cout << "s1.substr(3,4):\"" << s1.substr(3,4) << "\"\n";    // 部分文字列
27
28     s1.insert(3,"1234");    // stringの途中に文字列を挿入
29     cout << "s1.insert(3,\"1234\"), s1=\"" << s1 << "\"\n";
30
31     s1.replace(3,4,"wxyz"); // stringの部分を置換
32     cout << "s1.replace(3,4,\"wxyz\"), s1=\"" << s1 << "\"\n";
33
34     s1.erase(3,4);        // stringの途中を削除
35     cout << "s1.erase(3,4), s1=\"" << s1 << "\"\n";
36
37     int    v1 = 1234;
38     double v2 = 3.1415;
39     s1 = std::to_string(v1);    // intをstringに変換
40     s2 = std::to_string(v2);    // doubleをstringに変換
41     cout << "s1=\"" << s1 << "\", s2=\"" << s2 << "\"\n";    // 書式制御できず
42
43     std::ostringstream    oss;    // 文字列出力ストリーム作成
44     oss << std::fixed << std::setprecision(4) << v2;
45     s2 = oss.str();
46     cout << "s2=\"" << s2 << "\"\n";    // 書式制御できる
47
48     // 文字列からint, doubleに変換
49     v1 = std::stoi(s1);    v2 = std::stod(s2);
50     cout << "v1=" << v1 << ", v2=" << v2 << "\n";
51
52     s1 = "5678 2.71828";
53     cout << "read from \"" << s1 << "\": ";
54     std::istringstream    iss(s1);    // 文字列s1から入力するストリーム作成
55     iss >> v1 >> v2;
56     cout << "v1=" << v1 << ", v2=" << v2 << "\n";
57
58     s1 = "C++ programming language";    // この文字列の中の
59     auto pos = s1.find("program");    // "program"の位置を探す
60     cout << "in \"" << s1 << "\", ";
61     if ( pos != std::string::npos ) {
62         cout << "\"program\" found at " << pos << ".\n";
63     } else {
64         cout << "\"program\" not found\n";
65     }
66     // 見つからないケース
67     pos = s1.find("Python");
```

```

68     if ( pos == std::string::npos ) {
69         cout << "but \"Python\" is not found.\n";
70     }
71 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test201-7.cpp のコンパイルと実行

```

$ g++ test201-7.cpp -o test201-7  Enter    ←コンパイル
$ ./test201-7  Enter    ←実行
string().max_size():9223372036854775806  ← string が扱える理論上の最大サイズ(処理系により異なる)
"Hello," + "World." = "Hello,World.", size:12  ←連結処理
----+----1
Hello,                                ←添字による参照で 1 文字ずつ出力
World.                                ← at による参照で 1 文字ずつ出力
s1="ABCDEFGH IJKLMN", s1.substr(3,4):"DEFG"  ←部分文字列の取り出し
s1.insert(3,"1234"), s1="ABC1234DEFGH IJKLMN"  ←文字列の挿入
s1.replace(3,4,"wxyz"), s1="ABCwxyzDEFGH IJKLMN"  ←文字列の置換
s1.erase(3,4), s1="ABCDEFGH IJKLMN"  ←文字列の部分の削除
s1="1234", s2="3.141500"  ← to.string で数値を文字列に変換 (書式制御できず)
s2="3.1415"  ← ostringstream を用いた書式制御
v1=1234, v2=3.1415  ← stoi, stod で文字列を数値に変換
read from "5678 2.71828": v1=5678, v2=2.71828  ← istream を用いた数値の読み取り
in "C++ programming language", "program" found at 4.  ← find による文字列の検索
but "Python" is not found.  ←見つからない場合は string::npos で判定

```

発展) 後の「21 string による文字列処理」(p.407～) で、更に高度な string クラスの応用方法について解説する。

11.6.5 明示的なヌルポインタ: nullptr

C 言語では、ポインタがどのオブジェクトも指していないことを表すために、NULL や整数値 0 が用いられてきた。C++ でも、従来は同様の書き方が行われていたが、C++11 以降では、**ヌルポインタ**を表すための専用の値として `nullptr` が用意されている。

`nullptr` は、どのオブジェクトも指していないポインタ値を表すための値¹²⁰ である。例えば、ポインタ変数を初期化するときには、次のように記述できる。

```
int *p = nullptr;
```

この場合、`p` はヌルポインタとなり、どの `int` 型のオブジェクトも指していない。ポインタがヌルポインタであるかどうかを調べる場合にも、次のように `nullptr` と比較できる。

```
if ( p == nullptr ) { ... }
```

`nullptr` は条件式では偽として扱われる。従って、ポインタ変数 `p` に対して、`if (p)` と書くと、`p` がヌルポインタでない場合に真となり、`if (!p)` と書くと、`p` がヌルポインタである場合に真となる。ただし、上のように `p == nullptr` や `p != nullptr` と、明示的に比較する条件式を記述しても良い。

`nullptr` の重要な点は、整数値の 0 とは異なる専用の値として扱われることである。`nullptr` の型は `std::nullptr_t` であり¹²¹、整数型ではない。従って、`nullptr` はポインタ型に変換できるが、通常の整数値として扱うことはできない。この性質により、NULL や 0 を用いた場合に比べて、ヌルポインタであることを明確に表すことができる。

NULL は処理系によっては整数値 0 として定義されることがあり、関数のオーバーロードなどで意図しない解釈を招く場合がある。これに対して、`nullptr` はポインタに対するヌル値であることを明確に表す。従って、C++ では、ポインタが何も指していないことを表す場合には、NULL や 0 ではなく `nullptr` を用いるのが望ましい。

¹²⁰厳密には、`std::nullptr_t` 型のヌルポインタリテラルである。

¹²¹`nullptr` の使用においては、特にヘッダファイルをインクルードする必要はないが、`std::nullptr_t` の型を扱う場合はヘッダファイル `cstdint` をインクルードする。

11.6.6 型名としての構造体名

C 言語では、構造体のタグ名（構造体名）を型名として直接使用することはできず、`typedef` によって構造体型に別名を定義することで、その別名を型名として使用できることを先に説明した。これに対して C++ では、名前を付けて構造体を定義すると、その構造体名をそのまま型名として使用することができる。

例. 型名として使用できる構造体名

```
struct Point {           // 名前を付けて構造体を定義
    double x;
    double y;
};

Point p;                 // 構造体名を型名として変数 p を宣言
```

なお、C++ でも

```
struct Point p;
```

と記述することは可能である。

11.6.7 型変換

C++ においても、異なる型の値を扱う際には型変換を行う必要がある。C 言語では、

(型名) 式

の形式によるキャストを用いて型変換を記述していた。この方法は簡潔であるが、変換の意図が明確に表れにくく、不適切な型変換をコード上で識別しにくいという問題がある。

C++ においても、このような C 言語のキャストを引き続き使用することは可能である。しかし、C++ では、型変換の目的に応じて複数のキャストが用意されており、それぞれの用途に応じて使い分けことが推奨される。

11.6.7.1 `static_cast`

C++ の型変換の中で最も基本となるのが `static_cast` である。これは、比較的安全な型変換を明示的に行うための仕組みであり、次のような形式で記述する。

`static_cast<変換後の型>( 式 )`

例えば、整数値を実数値に変換するには、次のように記述する。

例. `static_cast` による型変換

```
int a = 3, b = 2;
double x = static_cast<double>(a) / b;
```

この例では、整数同士の除算をそのまま行うのではなく、`a` を実数型に変換した後で計算を行うことで、結果を実数として求めている。

同様の処理は C 言語のキャストを用いて、

```
double x = (double)a / b;
```

のように記述することもできる。この式も、結果としては `a` を実数型に変換して計算を行っている。

しかし、この `(double)` という記法は、見た目には単なる括弧付きの式と区別しにくいだけでなく、「型変換を行っている」という意図が明確に表れない。さらに、この記法では、`static_cast`、`const_cast`、`reinterpret_cast` などに相当する複数種類の型変換を同じ記法で表すことができるため、どのような変換が行われているのかをコードから読み取ることが難しい。

これに対して `static_cast` を用いると、「ここで型変換を行う」という意図が明確に示される。また `static_cast` では、許可される型変換の種類が限定されているため、C 言語形式のキャストに比べて、意図しない種類の型変換が紛れ込むことを防ぎやすい。

以上のことから、両者は同じ結果を得ることができる場合でも、コードに表れる意味や安全性が異なるため、C++ では `static_cast` を用いることが推奨される。このような記法は、プログラムの意図を明確にし、安全性と可読性の向上に寄与する。

11.6.7.2 const_cast

C++ では、変数に `const` 修飾子を付けることで、「値を書き換ええない」という意図を明示することができる。このような指定は、プログラムの安全性や可読性を高める上で重要である。しかし、特別な場合には、`const` 修飾の有無を調整する必要があることがある。そのような場合に用いるのが `const_cast` である。

`const_cast` は、`const` 修飾子の有無を変更するためのキャストであり、次のような形式で記述する。

`const_cast<変換後の型>(式)`

例えば、`const` 修飾されたポインタから `const` を取り除くには、次のように記述する。

例. `const_cast` による変換

```
const int x = 10;
const int* p = &x;
int* q = const_cast<int*>(p);
```

この例では、`const int*` 型のポインタ `p` を、`int*` 型のポインタに変換している。

注意) 元々 `const` として定義された変数に対して、`const_cast` によって `const` を取り除いて書き換えを行った場合、その動作は未定義となる。すなわち、プログラムが正しく動作する保証はなく、予測不能な結果を引き起こす可能性がある。

▲重要▲

`const_cast` によって `const` を取り除いた後に値を書き換えてよいのは、元のオブジェクト自体が `const` ではない場合に限られる。例えば、古いライブラリとの互換性のために、実際にはデータを書き換ええないことが保証されている関数であるにもかかわらず、引数型の `const` の有無が一致しないという問題を解決する場合などが、`const_cast` を利用するケースに該当する。

このように、`const_cast` は `const` 修飾子を操作するための専用のキャストであり、使用には十分な注意が必要であり、安易に用いるべきではない。

11.6.7.3 reinterpret_cast

C++ では、異なる型のデータを低水準な方法で「別の型として解釈」して扱いたい場合がある。このような型の対応関係を明示的に変更するために用いられるキャストが `reinterpret_cast` である。

`reinterpret_cast` は、主にポインタ型や参照型（後述）などに対して用いられ、低水準な型変換を明示的に行うための仕組みである。次のような形式で記述する。

`reinterpret_cast<変換後の型>(式)`

例えば、ある型のポインタを別の型のポインタとして扱うには、次のように記述する。

例. `reinterpret_cast` による `int` 型オブジェクトのバイト列としての参照

```
int x = 123;
char* p = reinterpret_cast<char*>(&x);
```

この例では、整数型の変数 `x` のアドレスを、`char*` 型のポインタとして扱っている。これにより、`x` の内容をバイト列として取り出すことができる。

また、この `p` はもともと `int` 型のオブジェクト `x` を指すアドレスから得たものであるため、次のように元の `int*` 型に戻すこともできる。

例. `reinterpret_cast` による元の `int` 型ポインタへの変換（先の例の続き）

```
int y = *reinterpret_cast<int*>(p);
```

▲注意▲

この例では、`p` はもともと `int` 型の変数 `x` のアドレスから得られたものであり、それを元の `int*` 型に戻しているため正しく扱うことができる。しかし、一般のバイト列を単に `int*` 型へ変換して読み取ってよいわけではない。アドレスのアラインメントや、その記憶領域に実際に存在するオブジェクトの型などの条件によっては、未定義動作を引き起こす可能性があるので注意すること。

■ 結論

`reinterpret_cast` による変換は、型の安全性を保証するものではない。異なる型のデータとして読み替えた場合、その結果がどのように解釈されるかは処理系や実行環境に依存することがあり、意図しない結果を引き起こす可能性があることを予め理解しておくこと。

`reinterpret_cast` による変換は、バイナリデータの入出力や、メモリの内容を直接扱うような低水準の処理において用いられる。`reinterpret_cast` は、データの「意味」を変換するものではなく、ポインタなどの型としての「見方」を低水準に変更するためのキャストである。従って、その使用は必要最小限にとどめ、用途を明確にした上で慎重に用いるべきである。

11.6.7.4 型で意味を表す設計

これまでに見てきたように、C++ では、型は単なるデータの分類ではなく、プログラムの意味や意図を表現するための重要な要素として扱われる。例えば、論理型 `bool` を用いることで「条件」や「状態」を明確に表現することができる。また、`static_cast` を用いて型変換を明示的に記述することで、処理の意図をコード上に明確に示すことができる。一方で、`const_cast` や `reinterpret_cast` は、特定の目的のために型の制約を操作したり、データの見方を変更したりするための手段であり、その使用には慎重さが求められる。

このように、C++ においては、型を適切に選択し、その意味を活かしてプログラムを記述することが重要である。型を用いてデータの性質や役割を表現することで、コードの可読性や保守性が向上し、意図しない誤りを減らすことにつながる。

実際のプログラムでは、「値」を扱うだけでなく、「その値が何を表しているのか」を明確にすることが求められる。例えば、単なる整数値であっても、「個数」、「位置」、「識別子」など、異なる意味を持つ場合がある。これらを適切な型や名前によって区別することで、プログラムの構造をより明確にすることができる。

このような考え方は、構造体や後に学ぶクラスによる設計のための指針にもなる。C++ においては、型を用いて意味を表現することが、良いプログラムを作るための基本となる。

11.7 参照

C++ では、既に存在する変数に対して、別の名前を与える仕組みとして**参照**（reference）が用意されている。参照は、ある変数の**別名**（エイリアス）として機能し、同じ対象を別の名前でも扱うことを可能にする。

参照は「&」を用いて宣言する。例えば、変数 `x` に対して参照 `r` を次のように定義することができる。

```
int x = 10;
int& r = x;
```

このとき、`r` は `x` の参照であり、`x` と `r` は同じ対象を表している。したがって、`r` を用いて値を変更すると、`x` の値も同時に変更される。

```
r = 20;
```

とすると、`x` の値も 20 に変更される。

このように、参照は新しい値を保持する別個の変数を作るものではなく、既に存在する対象に別名を与える仕組みである。したがって、参照は定義時に必ず初期化する必要があり、後から別の変数を参照するように変更することはできない。

参照は、ポインタと同様に複数の名前から同一の対象を扱うための仕組みであるが、ポインタのようにアドレスを明示的に扱う必要がなく、より直感的に使用することができる。このため、関数の引数としてオブジェクトを受け渡す場合などにおいて、分かりやすい記述を行うことができる。

参照は、C++ における重要な基本機能の一つであり、関数の引数の扱いやオブジェクトの操作など、多くの場面で用いられる。

11.7.1 関数呼び出しにおける参照の応用

参照は、単に変数に別名を与える仕組みとして理解することもできるが、関数の引数として用いることで、その有用性がより明確になる。

通常、関数の呼び出しの際に引数に値を渡すと、その値がコピーされるため、関数内で引数の値を変更しても、呼び出し元の変数には影響しない。これに対して、参照を用いると、呼び出し元の変数を関数内から直接操作することができる。

例えば、変数の値を1だけ増加させる関数を考える。

```
void increment(int& x) {  
    x++;  
}
```

この関数は、仮引数を参照として定義しているため、`x` は呼び出し元から与えられた実引数の別名として扱われる。

この関数は、引数 `x` を1だけ増加させるもので、関数内での変更は呼び出し元の変数に直接反映される。この関数を次のようにして呼び出す場合を考える。

```
int a = 10;  
increment(a);
```

この実行によって、変数 `a` の値は11に変更される。

これに対して、次のように、引数を通常の値渡しとして定義した場合を考える。

```
void increment(int x) {  
    x++;  
}
```

この関数の内部では、変更はコピーされた値に対してのみ行われるため、呼び出し元の変数には影響しない。

このような関数は、ポインタを引数として渡す形でも実現することができるが、参照の形式で引数を定義することで、より直感的なプログラムの記述が可能となる。

■ サンプルプログラム

先に示した `increment` と、2つの整数変数の値を交換する `swap` 関数を実装する例を `test202-1.cpp` に示す。

プログラム：test202-1.cpp

```
1  #include <iostream>  
2  using std::cout;  
3  
4  // インクリメント関数  
5  void increment( int& x ) {  
6      x++;  
7  }  
8  
9  // 交換関数  
10 void swap( int& a, int& b ) {  
11     int tmp = a;  
12     a = b;  
13     b = tmp;  
14 }  
15  
16 int main() {  
17     int    x = 3, y = 7;  
18  
19     cout << "x:" << x << " をインクリメントすると ";  
20     increment(x);  
21     cout << "x:" << x << " となる. \n";  
22  
23     cout << "x:" << x << ", y:" << y << " を交換すると ";  
24     swap(x,y);  
25     cout << "x:" << x << ", y:" << y << " となる. \n";  
26 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test202-1.cpp のコンパイルと実行

\$ g++ test202-1.cpp -o test202-1 ←コンパイル

\$./test202-1 ←実行

x:3 をインクリメントすると x:4 となる. ←出力

x:4, y:7 を交換すると x:7, y:4 となる.

■ 標準ライブラリの swap 関数

C++ の標準ライブラリには、2つのオブジェクトの値を交換する `std::swap` 関数が用意されている。基本的な書式は次の通りである。

```
std::swap(a, b);
```

これにより、`a` と `b` の値が交換される。`std::swap` を使用するには、ヘッダファイル `utility` をインクルードする。

先の `test202-1.cpp` における `swap` 関数は、参照の仕組みを示すために自作したものであるが、実際のプログラミングで単に2つの値を交換する場合には、通常は標準ライブラリの `std::swap` を使用の方がよい。

11.7.1.1 構造体を参照で渡すケース

通常の方法で構造体を関数の引数として値渡しした場合には、その内容が複製されるため、関数内での変更が呼び出し元に反映されない。これに対して、C++ では参照を用いることができるので、呼び出し元の構造体を、参照を介して直接操作することができる。このことを、次のプログラム `test202-2.cpp` で示す。

プログラム：test202-2.cpp

```
1  #include    <iostream>
2  #include    <iomanip>
3  #include    <cmath>
4  #define     DSIZE      8
5  using std::cout, std::setw;
6
7  typedef struct {
8      int      x[DSIZE];
9      double   y[DSIZE];
10 } DS;
11
12 // 平方根算出
13 void sqr( DS& a ) {
14     int      c;
15     for ( c = 0; c < DSIZE; c++ ) a.y[c] = sqrt( static_cast<double>(a.x[c]) );
16 }
17
18 // 内容表示
19 void printDS( const DS& a ) {
20     int      c;
21     for ( c = 0; c < DSIZE; c++ ) cout << setw(8) << a.x[c];
22     cout << "\n";
23     for ( c = 0; c < DSIZE; c++ ) cout << setw(8) << a.y[c];
24     cout << "\n";
25 }
26
27
28 int main() {
29     DS d = {{0,1,2,3,4,5,6,7},
30             {0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0}};
31     sqr(d);      // 構造体を変更する関数
32     printDS(d);  // 変更が反映されている
33 }
```

このプログラムで定義されている関数 `sqr` では、仮引数 `a` は呼び出し元から渡された構造体そのものの別名として扱われる。従って、関数内で `a` の内容を書き換えると、呼び出し元の構造体にもその変更が直接反映される。

また、仮引数 `a` は構造体 `DS` の変数の別名であることから、ドット表記「`.`」でそのメンバにアクセスすることができる。構造体のポインタを関数に渡す形での実装では、メンバには「`->`」でアクセスするので、今回の表記の方がより直感的である。

このプログラムでは、構造体 `DS` 型の変数 `d` が定義されている。また、関数 `sqr` は構造体のメンバ `x` の値に対して平方根を計算し、結果を `y` に格納するものである。

また、構造体の内容を表示する関数 `printDS` においても、同様に参照を用いて構造体を受け取っている。このようにすることで、構造体の内容を複製することなく効率的に処理を行うことができる。なお、関数 `printDS` においては、受け取った構造体の内容を変更しないので `const` を付けた参照として受け取る形にしている¹²²。この方法では、構

¹²²このケースでは、仮引数を `const` 宣言しなくても文法上は誤りではない。

造体の複製を避けながら、関数内から内容を書き換えないことも明示できる。

このプログラムを実行すると、関数 `sqr` の呼び出し後に、`main` 側の構造体 `d` の内容が更新されていることが確認できる。(次の例)

例. `test202-2.cpp` のコンパイルと実行

```
$ g++ test202-2.cpp -o test202-2  [Enter]    ←コンパイル
$ ./test202-2  [Enter]    ←実行
      0      1      2      3      4      5      6      7
      0      1 1.41421 1.73205      2 2.23607 2.44949 2.64575
```

このように、構造体を参照で渡すことにより、関数内で呼び出し元のデータを直接操作することが可能となる。また、大きな構造体を扱う場合には、複製を避けることで処理効率の向上にも寄与する。

注意) 参照を用いる場合、関数内での変更が呼び出し元に直接影響するため、意図しない値の変更を引き起こさないように注意が必要である。

11.7.1.2 参照を返す関数

通常関数では、戻り値は値として返されるため、その結果を直接書き換えることはできない。例えば、関数の戻り値に対して代入を行うことは、一般には許されていない。

これに対して、C++ では関数の戻り値として参照を返すことができる。参照を返す関数では、戻り値は既存のオブジェクトを参照するため、関数の呼び出し結果を変数と同様に扱うことが可能となる。

このことを確認するために、次のプログラム `test202-3.cpp` について考える。

プログラム：`test202-3.cpp`

```
1  #include    <iostream>
2  #include    <iomanip>
3  #define     SIZE      7
4  using std::cout, std::setw;
5
6  // 配列の中央要素の参照を返す
7  int& mid( int a[], int n ) {
8      return a[n / 2];
9  }
10
11 int main() {
12     int    a[SIZE] = {0,1,2,3,4,5,6};
13     int    i;
14
15     cout << "変更前: ";
16     for ( i = 0; i < SIZE; i++ ) cout << setw(6) << a[i];
17     cout << "\n";
18
19     mid(a, SIZE) = 100;    // 戻り値は参照なので代入が可能
20
21     cout << "変更後: ";
22     for ( i = 0; i < SIZE; i++ ) cout << setw(6) << a[i];
23     cout << "\n";
24 }
```

このプログラムでは、関数 `mid` が整数配列の中央の要素を参照として返している。従って、次の文

```
mid(a, SIZE) = 100;
```

は、関数 `mid` の戻り値を左辺として代入を行っているように見えるが、実際には配列 `a` の中央の要素そのものに対して値 `100` を代入していることになる。すなわち、関数 `mid` の呼び出し結果は、単なる値ではなく、変数と同様に扱うことができる。

一方、通常関数が値を返す場合には、その戻り値は一時的な値であり、それに対して代入を行うことはできない。例えば、関数が `int` 型の値を返す場合、

```
f() = 100;
```

のような記述は許されない。

このように、参照を返す関数を用いることで、関数の呼び出し結果を変数と同様に扱い、その内容を直接変更することが可能となる。ただし、参照を返す関数では、参照先となるオブジェクトの寿命に注意する必要がある。例えば、関数内で定義されたローカル変数への参照を返すと、関数終了後にその変数は消滅するため、返された参照は無効になる。その参照を使用すると未定義動作となる。

このプログラムをコンパイルして実行する例を次に示す。

例. test202-3.cpp のコンパイルと実行

```
$ g++ test202-3.cpp -o test202-3  ←コンパイル
$ ./test202-3  ←実行
変更前:      0      1      2      3      4      5      6
変更後:      0      1      2    100      4      5      6
```

参照を返す関数は、関数の戻り値¹²³を変数として扱うための方法を与える。

注意) 参照を返す関数では、常に有効なオブジェクトへの参照を返すように設計する必要がある。

■ 応用事例：関数ローカルの静的変数（static 変数）への外部からのアクセス

通常、関数内で定義されたローカル変数は、その関数の内部でのみ有効であり、外部から直接アクセスすることはできない。これは、変数の有効範囲を関数内部に限定するための基本的な仕組みである。

しかし、C++では関数の戻り値として**参照**を返すことができるため、関数内の変数そのものを外部に公開することが可能となる。特に、関数内で定義された static 変数は、関数の呼び出しをまたいで値を保持するため、参照として返すことで、外部からその値を共有する手段として利用することができる。

このことを確認するために、次のプログラム test202-4.cpp を考える。

プログラム：test202-4.cpp

```
1  #include <iostream>
2  using std::cout;
3
4  // 共有変数を実現する関数
5  int& sharedInt() {
6      static int  n = 0;  // この関数ローカルの静的変数を
7      return n;          // 外部に公開する
8  }
9
10 int main() {
11     cout << sharedInt() << "\n";    // 共有変数の値の確認
12     sharedInt() = 5;                // 共有変数への値の設定
13     cout << sharedInt() << "\n";    // 共有変数の値の確認
14 }
```

このプログラムでは、関数 sharedInt の内部に、関数ローカルの静的変数 n が定義されている。通常、この変数 n は関数の外部から直接アクセスすることはできないが、関数 sharedInt はこの変数への**参照**を返すため、関数の呼び出しを通じて、外部からその値を参照および変更することが可能となる。例えば、次の文

```
sharedInt() = 5;
```

は、関数の戻り値に対して代入を行っているように見えるが、実際には関数内部の静的変数 n に対して値 5 を代入していることになる。このようにして、関数を介して外部と内部の間で値を共有することができる。

このプログラムを実行すると、関数 sharedInt を通じて同一の変数 n にアクセスしているため、値が保持されることが確認できる。（次の例）

例. test202-4.cpp のコンパイルと実行

```
$ g++ test202-4.cpp -o test202-4  ←コンパイル
$ ./test202-4  ←実行
0      ←初期の値
5      ←変更された値
```

¹²³くれぐれも、厳密には「値」が返されるのではなく、「参照」が返されることに注意すること。

このような方法は、グローバル変数のように複数の箇所から共有して利用できるデータを実現する一つの手段であり、関数を通じてアクセス経路を限定する¹²⁴ ことで、直接的なグローバル変数の使用に比べて管理しやすいという利点がある。ただし、参照を返すことにより内部状態が外部から自由に変更可能となるため、設計上の制約が弱くなる点には注意が必要である。

11.7.2 一時オブジェクトと const 参照

C++では、式の中でその場で生成されるオブジェクトを**一時オブジェクト**と呼ぶ。例えば、

```
typedef struct {
    int n[4];
} DS;
```

のような構造体型の定義がある時、DS{{4,5,6,7}} のような記述によって生成されるオブジェクトは、変数として名前を持たず、通常は式の評価が終了すると直ちに破棄される。

このような一時オブジェクトは、通常の参照（非 const 参照）に束縛することはできない。しかし、**const 参照**を用いると、一時オブジェクトを関数の引数として受け取ることが可能となる。また、一時オブジェクトをローカルな const 参照変数に直接束縛した場合には、その参照の有効範囲が終了するまで一時オブジェクトの寿命が延長される。

const 参照は、このように一時オブジェクトを扱うための仕組みであると同時に、関数内での値の変更を禁止することで安全性を確保する役割も持っている。また、構造体のようにサイズの大きなデータを値渡しする場合に発生するコピーを避けることができるため、効率の面でも有用である。

これらの性質を確認するために、次のプログラム test202-5.cpp を考える。

プログラム：test202-5.cpp

```
1  #include    <iostream>
2  #include    <iomanip>
3  #define     DSIZE      4
4  using std::cout, std::setw;
5
6  typedef struct {
7      int      n[DSIZE];
8  }    DS;
9
10 //void printSQR( DS& d ) {          // この宣言では一時オブジェクトを受け取れない
11 void printSQR( const DS& d ) {
12     int      c;
13     for ( c = 0; c < DSIZE; c++ ) cout << setw(4) << d.n[c] * d.n[c];
14     cout << "\n";
15 }
16
17 int main() {
18     DS d1 = {{0,1,2,3}};
19     printSQR(d1);                      // 構造体を与える
20     printSQR( DS{{4,5,6,7}} );        // 一時オブジェクトを与える
21
22     // const参照に束縛することで、一時オブジェクトの寿命が延長される
23     const DS& d2 = DS{{8,9,10,11}};
24     // DS& d2 = DS{{8,9,10,11}};      // これは不可
25     printSQR(d2);
26 }
```

このプログラムでは、関数 printSQR が構造体 DS 型の引数を受け取り、そのメンバの値の平方値を表示している。ここで、引数は const 参照として定義されているため、関数内で値を書き換えることはできない。

まず、変数 d1 を引数として渡す場合、通常の構造体の受け渡しと同様に、関数はその内容を参照して処理を行う。次に、DS{{4,5,6,7}} のように、その場で生成された一時オブジェクトを直接関数に渡しているが、const 参照を用いているため、このようなオブジェクトも問題なく受け取ることができる。

さらに、次の文

```
const DS& d2 = DS{{8,9,10,11}};
```

¹²⁴カプセル化の 1 つの事例である。カプセル化に関しては、後の「12.9 カプセル化」(p.253～) で詳しく解説する。

では、一時オブジェクトを明示的に `const` 参照に束縛している。この場合、本来は式の終了とともに破棄されるはずの一時オブジェクトが、参照 `d2` の有効範囲まで保持されることになる。

このプログラムをコンパイルして実行する例を次に示す。

例. `test202-5.cpp` のコンパイルと実行

```
$ g++ test202-5.cpp -o test202-5  ←コンパイル
$ ./test202-5  ←実行
    0    1    4    9    ←構造体 d1 を渡した場合の出力
   16   25   36   49    ←一時オブジェクトの構造体を渡した場合の出力
   64   81  100  121    ←一時オブジェクトの構造体の参照を渡した場合の出力
```

このように、`const` 参照は、一時オブジェクトを安全に受け取るための重要な仕組みである。また、一時オブジェクトをローカルな `const` 参照変数に直接束縛した場合には、その寿命を参照の有効範囲まで延長することができる。更に、`const` 参照を関数の引数として用いることで、効率と安全性を両立したデータの受け渡しを実現することができる。

重要) 通常の非 `const` 左辺値参照では一時オブジェクトを受け取ることはできない¹²⁵が、`const` 参照を用いると受け取ることができる。

11.7.3 参照とポインタの違い

参照とポインタは、いずれも同じ対象を別の方法で扱うための仕組みであるが、その性質には本質的な違いがある。特に重要なのは、指し示す対象を後から変更できるかどうかである。

この違いを確認するために、次のプログラムを考える。

```
int a = 10;
int b = 20;

int& r = a;
r = b;

int* p = &a;
p = &b;
```

まず、参照 `r` は変数 `a` の別名として定義されている。ここで

```
r = b;
```

とすると、これは参照先を変更しているのではなく、変数 `a` に対して変数 `b` の値を代入していることになる。すなわち、参照 `r` は常に `a` の別名であり、途中で別の変数の別名となるように変更することはできない。

一方、ポインタ `p` は変数のアドレスを保持する変数である。

```
p = &b;
```

とすると、ポインタ `p` が指し示す対象は、もともとの `a` から `b` に変更される。すなわち、ポインタは指し示す対象を後から自由に変更することができる。

このように、参照は一度結び付けた対象を変更することができないのに対し、ポインタは指し示す対象を後から変更することができるという違いがある。

参照は「変数の別名」として扱われるため、ポインタに比べて直感的な記述が可能であり、主に関数の引数や戻り値として利用される。一方、ポインタは柔軟に対象を切り替えることができるため、より低水準で柔軟な処理を実現するために用いられる。

¹²⁵C++11 以降では、後に学ぶ右辺値参照によって一時オブジェクトを受け取る方法もある。

11.8 関数のオーバーロード

C 言語では、同一の名前を持つ関数は一つしか定義できず、引数の型や個数が異なる場合には、別の関数名を用いる必要があった。例えば、整数用と実数用で同様の処理を行う場合でも、`add_int`、`add_double` のように名前を変えて定義するのが一般的である。このような方法は単純ではあるが、関数名が増えやすく、可読性や保守性の面で問題が生じることがある。

これに対して C++ では、同じ名前の関数を、引数の違いに応じて複数定義することができる。この仕組みを**関数のオーバーロード** (function overloading) と呼ぶ。関数のオーバーロードは、「同一の概念に属する操作」を、型や引数の違いに応じて同一の関数名を用いて自然に表現することに用いるのが一般的である。

■ 距離を求める関数の例

点の原点からの距離を求める処理を考える。1 次元、2 次元、3 次元で、それぞれ次のように定義することができる。

```
#include <cmath>

// 1 次元
double distance(double x) {
    return std::abs(x);
}

// 2 次元
double distance(double x, double y) {
    return std::sqrt(x*x + y*y);
}

// 3 次元
double distance(double x, double y, double z) {
    return std::sqrt(x*x + y*y + z*z);
}
```

いずれも「距離を求める」という同一の概念に属する処理であるが、扱う次元によって引数の個数が異なる。C++ では、これらをすべて `distance` という同一の関数名で定義し、呼び出し時の引数に応じて、適切な関数が選択される。これにより、プログラムの意味が直感的に把握しやすくなる。

このように、C++ では関数を区別する際に、関数名だけでなく、引数の型や個数、順序などの情報が用いられる。関数名と仮引数の型の並びなど、関数を識別するための情報を**関数シグネチャ** (function signature) と呼ぶ。例えば、

```
distance(double)
distance(double, double)
distance(double, double, double)
```

は¹²⁶、それぞれ異なるシグネチャを持つ関数であり、これらは同一の関数名であっても別の関数として定義することができる。なお、関数シグネチャには戻り値の型は含まれないため、関数シグネチャが同一で戻り値の型のみが異なる関数を定義することはできない。このように関数呼び出しでは、戻り値の型ではなく、呼び出し時の引数に応じて、適切な関数が選択される。

関数のオーバーロードは、単なる記述の簡略化のための機能ではなく、同一の意味を持つ操作を統一的に表現するための重要な仕組みである。ただし、曖昧な定義はかえって可読性を損なう原因となるため、「どの関数が呼び出されるかが一意に理解できるか」を基準として、慎重に設計する必要がある。

■ サンプルプログラム

次に示すプログラム `test203-1.cpp` は、整数の合計を求める関数 `sum` を 2 通りの方法で定義し、オーバーロードとして利用する例を示す。1 つは可変長引数として与えられた整数値の合計、もう 1 つは配列の要素の合計を求めるものである。

プログラム：test203-1.cpp

```
1  #include    <iostream>
2  #include    <cstdlib>          // 可変長引数を扱う場合に必要
3  #define     N          4
4  using std::cout;
5
6  // 可変長引数版
```

¹²⁶ この表記は関数シグネチャの簡略表記として多く用いられる。

```

7 | int sum(int n, ...) {          // nは引数の個数
8 |     va_list args;
9 |     va_start(args, n);
10 |
11 |     int s = 0;
12 |     for (int i = 0; i < n; ++i) {
13 |         s += va_arg(args, int);    // 引数を1つずつ取り出して加算
14 |     }
15 |
16 |     va_end(args);
17 |     return s;
18 | }
19 |
20 | // 配列版
21 | int sum(int n, int a[]) {      // nは計算対象の要素数
22 |     int s = 0;
23 |     for (int i = 0; i < n; ++i) {
24 |         s += a[i];                // 配列要素を1つずつ加算
25 |     }
26 |     return s;
27 | }
28 |
29 | int main() {
30 |     int    a[N] = {0, -1, -2, -3};
31 |
32 |     cout << sum( 4, 0, 1, 2, 3 ) << "\n";    // 可変長引数で計算
33 |     cout << sum( N, a ) << "\n";            // 配列で計算
34 | }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test203-1.cpp のコンパイルと実行

```

$ g++ test203-1.cpp -o test203-1  ←コンパイル
$ ./test203-1  ←実行
6          ←可変長引数の sum 関数による計算結果
-6         ←配列用の sum 関数による計算結果

```

注意) 可変長引数を用いる方法は柔軟であるが、引数の個数や型を通常の仮引数のように検査できないため、型安全性の面で注意が必要である。

11.8.1 オーバーロードの設計上の注意

関数のオーバーロードでは、引数の型と完全に一致する関数が存在する場合、それが優先して選択される。例えば、次のような定義を考える。

```

void f(int);
void f(double);

```

このとき、

```

int x = 10;
f(x);

```

と呼び出すと、引数 x は `int` 型であるため、`f(int)` が完全に一致し、これが選択される。

一方、次のような定義を考える。

```

void f(long);
void f(double);

```

この場合、`int` 型の変数 x を

```

f(x);

```

と呼び出すと、`long` 型は整数というカテゴリであるので、一見すると `void f(long);` の方が呼び出されると考えがちになる。しかし、 x の型は `int` であり、`long` でも `double` でもない。すなわち、いずれの関数も完全には一致していない。このとき、`int` 型から `long` 型への変換も、`double` 型への変換もいずれも可能であるため、両方の関数が呼び出し候補となるが、どちらを優先すべきかを決定することができない。その結果、この呼び出しは曖昧となり、コンパイルエラーが発生する。

このように、「意味的には近い型」であっても、完全に一致していなければ、オーバーロードの解決においては別の型として扱われる。従って、関数を設計する際には、どの関数が呼び出されるかが一意に決定されるように注意する必要がある。

重要) オーバーロードでは、「意味的な近さ」ではなく、引数の型と仮引数の型との一致や、必要となる型変換の優先順位に基づいて呼び出す関数が選択される。

11.8.1.1 引数が参照である場合

関数のオーバーロードでは、参照と `const` 参照の違いによって、呼び出される関数が変わる場合がある。例えば、次のような2つの関数を考える。

```
void f(int& x);
void f(const int& x);
```

これらは一見すると似ているが、受け取ることのできる引数の性質が異なる。

まず、変数を引数として渡す場合を考える。

```
int a = 10;
f(a);
```

このとき、`a` は変更可能な左辺値であるため、`int&` にも `const int&` にも束縛できるが、`f(int&)` が優先して選択される。

次に、リテラルを引数として渡す場合を考える。

```
f(10);
```

この場合、`10` は右辺値であり、通常の非 `const` 左辺値参照である `int&` には束縛できない。一方、`const int&` には束縛できるため、`f(const int&)` が選択される。

このように、`int&` は変更可能な左辺値に束縛できる参照であり、`const int&` は変更可能な左辺値だけでなく、一時的な値にも束縛することができる。このように、与えた引数が左辺値か右辺値か、また各参照に束縛できるかどうかが関数の選択のポイントとなる。

以上のことから、オーバーロードを設計する際には、「どのような引数を受け取る関数であるか」を明確にし、意図しない関数が選択されないように注意する必要がある。

重要) 変更可能な変数は `int&` と `const int&` のどちらにも渡すことができるが、この場合は `int&` が優先して選択される。また、一時的な値（リテラルなど）は通常の非 `const` 左辺値参照には渡すことができないが、`const int&` には渡すことができる。

11.8.2 デフォルト引数

C++では、関数の仮引数に対して**デフォルト値**（既定値）を設定することができる。これにより、関数呼び出しの際に一部の引数を省略することが可能となる。例えば、次のような関数を考える。

```
void f(int x, int y = 0) {
    std::cout << x + y << std::endl;
}
```

この関数では、第2引数 `y` に対してデフォルト値 `0` が設定されている。このとき、関数は次のように呼び出すことができる。

```
f(5, 3);    // x=5, y=3
f(5);       // x=5, y=0 (デフォルト値が用いられる)
```

このように、デフォルト引数が指定されている場合には、引数が省略されたときに、その値が自動的に補われる。ただし、この補完は実行時ではなく、**コンパイル時**に行われることに注意が必要である。

デフォルト引数を用いると、関数の呼び出しを簡潔に記述することができるが、使用にあたってはいくつかの制約がある。まず、デフォルト値は仮引数の右側から順に設定する必要がある。例えば、

```
void g(int x = 1, int y = 2);    // 正しい
void g(int x = 1, int y);       // 誤り
```

のように、途中の引数のみを省略可能にすることはできない。これは、関数の引数が左から順に対応付けられ、呼び出し時の実引数の省略の順序との関係から、途中の引数だけを省略すると後続の引数を決定できなくなるためである。

従って、デフォルト引数は右側から連続して設定する必要がある。

デフォルト引数は、関数のプロトタイプ宣言と処理の定義のいずれかで指定することができるが、通常はプロトタイプ宣言の方で指定する。同じ仮引数に対して、後の宣言でデフォルト値を再指定することはできない。例えば、

```
void f(int x, int y = 0);           // プロトタイプ宣言
void f(int x, int y) {              // 処理の定義
    std::cout << x + y << std::endl;
}
```

のように、デフォルト値は宣言側だけに記述するのが一般的である。この場合、

```
f(5);
```

と呼び出すと、宣言で指定されたデフォルト値 0 が用いられる。デフォルト引数は関数を実行するときに関数側で補われるものではなく、関数呼び出し側で利用される情報である。従って、通常はプロトタイプ宣言などの一箇所にまとめて記述するのがよい。

さらにデフォルト引数は、関数のオーバーロードと組み合わせた場合に、曖昧な呼び出しを生じることがある。例えば、

```
void h(int x);
void h(int x, int y = 0);
```

と定義した場合、

```
h(5);
```

という呼び出しは、どちらの関数を呼び出すべきかを決定できず、曖昧となる。

このように、デフォルト引数は関数呼び出しを簡潔にする有用な機能であるが、オーバーロードとの関係を含めて慎重に設計する必要がある。

重要) デフォルト引数は、引数の省略を可能にする機能であるが、オーバーロードと併用する場合には、呼び出しが曖昧にならないように注意する必要がある。

11.9 autoによる型推論

C++ では、変数を宣言する際に、型名の代わりに `auto` を用いることができる。 `auto` を用いると、初期化式の結果から、その変数の型が自動的に推定される。この仕組みを**型推論**と呼ぶ。

ただし、ここでいう `auto` は、C 言語における自動変数の記憶クラス指定子としての `auto` とは意味合いが異なる。C 言語では `auto` は、ローカル変数が自動記憶期間を持つことを表すための記憶クラス指定子であった。しかし、通常のローカル変数はもともと自動記憶期間を持つため、C 言語において `auto` を明示的に書く機会は少ない。

これに対して、C++ における `auto` は、主として右辺の式から変数の型を推定する場合に用いられる。たとえば、次のように書くことができる。

書き方： `auto 変数名 = 式`

例. `auto` による変数宣言

```
auto x = 10;           ← x は int 型と推定される
auto y = 3.14;         ← y は double 型と推定される
auto c = 'A';          ← c は char 型と推定される
```

この例では、変数 `x`, `y`, `c` の型を明示的に書いていない。しかし、右辺の式の結果から、コンパイラはそれぞれの型を推定できる。つまり、`auto` は「型を決めなくてよい」という意味ではなく、「型をコンパイラに推定させる」ための記法である。

■ 関数の戻り値と `auto`

`auto` が有用になる場面の一つは、関数の戻り値を変数で受け取る場合である。C++ では、関数のオーバーロードにより、同じ名前の関数を、引数の型に応じて複数定義することができる。そのため、同じ関数名を用いていても、呼び出し方によって、実際に呼び出される関数が変わることがある。

ここでは、二つの値を加算する `add` 関数を例に考えてみる。

例. 2 値を加算する `add` 関数

```
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}
```

このように定義された `add` 関数は、受け取る引数の型に応じた計算結果を返す。すなわち、

```
add(10, 20)
```

では、10 と 20 がどちらも整数であるため、`int` 型の値を返し、

```
add(1.5, 2.5)
```

では、1.5 と 2.5 がどちらも浮動小数点数であるため、`double` 型の値を返す。このように、同じ `add` という名前であっても、引数の型によって呼び出される関数は異なる。

`add` 関数の戻り値を変数で受け取る場合を考える。型を明示して書く場合は、次のような記述になる。

例. 戻り値の型を明示して受け取る場合

```
int x = add(10, 20);
double y = add(1.5, 2.5);
```

このように、戻り値の型が明らかであれば、それを受け取る変数の型を明示的に書くことができる。しかし、関数のオーバーロードが使われている場合や、式が複雑な場合には、右辺の式の型を判断してプログラムを書くことが煩雑になることもある。そこで、次のようにして `auto` を用いると、右辺の式から変数の型を推定させることができる。

例. 戻り値を `auto` で受け取る場合

```
auto x = add(10, 20);
auto y = add(1.5, 2.5);
```

この場合、右辺の `add(10, 20)` の戻り値から、`x` の型が `int` 型と推定され、同様に `add(1.5, 2.5)` の戻り値から `y` は `double` 型と推定される。

ここで重要なのは、auto によって型の確認が省略されているわけではないという点である。型は、右辺の式にもとづいてコンパイル時に決定される。したがって、auto で宣言された変数も、通常の型付き変数として扱われる。

■ 動的型付けではない auto 宣言

auto を用いると、型名を書かずに変数を宣言できる。しかし、これは変数の型が自由に変化するという意味ではない。auto によって推定された型は、コンパイル時に決定され、その後は通常の型付き変数として扱われる。例えば、次の例について考える。

```
auto x = 10;
x = 20;
x = 3.14;
```

この場合、x は最初の宣言によって int 型と推定される。したがって、x = 3.14; と書いても、x の型が double 型に変わるわけではない。これは、int 型の変数に浮動小数点数を代入する処理¹²⁷ として扱われる。

■ 初期値が必要な auto 宣言

auto は、初期化式から型を推定する機能である。そのため、型を推定する材料となる初期値が必要である。

```
auto x = 10;    ← 正しい
auto y;        ← 誤り：型を推定できない
```

auto y; のように初期値を与えない場合、コンパイラは変数 y の型を決定できない¹²⁸。したがって、auto による変数宣言では、初期化を同時に行う必要がある。

■ auto による型推論の用途

auto は、型名が長くなる場合や、右辺の式から型が明らかな場合に便利である。特に、標準ライブラリを用いたプログラムでは、複雑な型名が現れることがあるため、auto を用いることでプログラムを簡潔に記述できる場合がある。

一方で、auto を多用すると、変数の型が読み取りにくくなることもある。基本的な型を学習している段階では、int、double、char などの型名を明示した方がわかりやすい場合も多い。したがって、auto は、型を隠すための記法ではなく、右辺の式から型が自然に読み取れる場面で用いる機能として理解するとよい。

本書では、基本的な型を説明する場面では型名を明示し、関数の戻り値やライブラリの機能を扱う場面が必要に応じて auto を用いることにする。

上で例示した add 関数のオーバーロードと auto による型推論の例を、実際に test203-2.cpp として示す。

プログラム：test203-2.cpp

```
1  #include <iostream>
2  using std::cout;
3
4  // 関数のオーバーロード：int用とdouble用
5  int add(int a, int b) {
6      cout << "[add for int]\t : ";
7      cout << "add(" << a << ", " << b << ")=" << a+b << "\n";
8      return a + b;
9  }
10
11 double add(double a, double b) {
12     cout << "[add for double] : ";
13     cout << "add(" << a << ", " << b << ")=" << a+b << "\n";
14     return a + b;
15 }
16
17 int main() {
18     auto x = add(10, 20);
19     auto y = add(1.2, 3.4);
20
21     cout << "x=" << x << ", " << "y=" << y << "\n";
22 }
```

¹²⁷ 小数部は 0 の方向に切り捨てられる。また、使用する環境によってはコンパイル時に警告メッセージが出力されることもある。

¹²⁸ コンパイル時にエラーとなる。C++ では、auto だけで初期値のない変数を宣言することはできない。

このプログラムをコンパイルして実行する例を次に示す。

例. test203-2.cpp のコンパイルと実行

```
$ g++ test203-2.cpp -o test203-2 Enter    ←コンパイル
$ ./test203-2 Enter    ←実行
[add for int]      : add(10,20)=30      ← int 用の add が実行されている
[add for double]   : add(1.2,3.4)=4.6    ← double 用の add が実行されている
x=30, y=4.6
```

この例では、引数の型に応じて異なる add 関数が選択され、その戻り値の型に応じて x と y の型がそれぞれ自動的に推定される。すなわち、x は int 型、y は double 型となる。

■ auto と const 参照

auto による型推論では、初期化に用いた変数が const 型であっても、単純な auto 宣言では、その const が推論後の変数の型に引き継がれない場合がある。例えば、次の例を考える。

```
const int n = 10;
auto x = n;
auto& r = n;
```

この場合、n は const int 型であるが、auto x = n; とすると、x の型は int 型と推定される。すなわち、x は n とは別の変数として初期化され、その値を書き換えることができる。例えば、

```
x = 20;
```

とすることができる。この操作によって、元の変数 n の値が変化することはない。

一方、auto& r = n; とすると、r は n への参照となる。この場合、n が const int 型であることも反映され、r の型は const int& となる。従って、

```
r = 20;    ← 誤り
```

のように、r を通して n の値を書き換えることはできない。

このように、単純な auto 宣言では値を受け取る変数の型が推定されるのに対して、auto& とすると参照として型が推定される。特に、const オブジェクトへの参照では、その const 性も保たれる。

重要) auto と auto& では型推論の結果が異なる。元の変数への参照として扱う必要がある場合には、auto& や const auto& などを用いる。

11.10 範囲に基づく for 文

C++ では、配列、文字列、コンテナ（後述する）などに含まれる要素を、先頭から順に取り出すための簡潔な for 文が用意されている。これを**範囲に基づく for 文**と呼ぶ。C 言語の for 文では、反復用の変数を用意し、その値を増減させながら、配列の各要素にアクセスすることが多い。例えば、配列 a の要素に順にアクセスする場合には、添字を用いて a[0], a[1], a[2], … のように要素を指定する。これに対して、範囲に基づく for 文では、対象となる配列やコンテナから要素を 1 つずつ取り出して処理することができる。

範囲に基づく for 文の基本的な書き方は次の通り。

書き方： for (変数宣言 : 対象) { 処理 }

ここで、「対象」には、配列、文字列、STL のコンテナ¹²⁹などを指定する。反復のたびに、「対象」に含まれる要素が先頭から順に取り出され、「変数宣言」で宣言された変数によって受け取られる。そして、その変数を用いて波括弧内の処理が実行される。

次に示すプログラム test203-4.cpp は、範囲に基づく for 文によって、string オブジェクトの char 型の要素を 1 つずつ出力する例である。

プログラム：test203-4.cpp

```
1 #include <iostream>
2 #include <string>
3 using std::cout;
```

¹²⁹ 「20 STL の基礎」(p.384～) で解説する。

```

4
5 int main() {
6     std::string word = "programming";
7
8     bool flg = true;          // 初期状態フラグ
9     for (char c : word) {     // wordからchar型の要素を1つずつcに受け取る
10         if ( flg ) {
11             flg = false;
12         } else {
13             cout << ",";
14         }
15         cout << c;
16     }
17     cout << "\n";
18 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test203-4.cpp のコンパイルと実行

```

$ g++ test203-4.cpp -o test203-4  ←コンパイル
$ ./test203-4  ←実行
p,r,o,g,r,a,m,m,i,n,g ←出力

```

test203-4.cpp の for 文は、次のように auto を用いて記述しても良い。

```
for ( auto c : word ) { ... }
```

この場合、c の型は、word から取り出される要素の型に基づいて自動的に決定され、この例では char 型となる。

11.10.1 要素を参照として受け取る場合

範囲に基づく for 文では、要素を受け取る変数を参照型として宣言することもできる。例えば、次の2つの書き方を比較する。

形式 1.

```
for (auto c : word) {
    ...
}
```

形式 2.

```
for (auto& c : word) {
    ...
}
```

形式 1 では、word から取り出した各要素の値が c にコピーされる。従って、c は元の要素とは別の変数であり、c の値を変更しても word の要素には影響しない。

これに対して、形式 2 の auto& c では、c は word の各要素への参照となる。従って、c を通して値を書き換えると、word の要素そのものが変更される。例えば、

```
std::string word = "abc";
for (auto& c : word) {
    c = 'X';
}
```

とすると、処理後の word の内容は "XXX" となる。

一方、要素の値を書き換えず、参照として受け取るだけの場合には、const 参照を用いて、

形式 3.

```
for (const auto& c : word) {
    ...
}
```

のように記述することもできる。この場合、c を通して元の要素を書き換えることはできない。特に、サイズの大きなオブジェクトを要素とするコンテナでは、値をコピーせずに要素を参照できるという利点もある。

重要) 範囲に基づく for 文で auto を用いると要素の値を受け取り、auto& を用いると元の要素への参照を受け取る。また、読み取り専用で参照する場合には const auto& を用いることができる。

11.11 コンパイル時定数：constexpr

C++ では、値を変更できない変数を宣言するために `const` を使用できる。例えば、次のように宣言した変数 `a` は、初期化された後で値を変更することができない。

```
const int a = 100;
```

この意味で、`const` は「値を変更できない」ことを表す指定である。ただし、`const` で宣言された値が、常にコンパイル時に決定されるとは限らない。例えば、次のように実行時に入力された値を `const` 変数に格納することもできる。

```
int n;
cin >> n;
const int a = n;
```

この場合、`a` は後から変更できないが、その値はプログラムの実行中に入力されて初めて決まる。従って、`a` は「変更できない値」ではあるが、「コンパイル時に決まる値」ではない。これに対して、`constexpr` は、コンパイル時に評価可能な定数や処理を表すための指定子である。例えば、次のように宣言できる。

```
constexpr int b = 100;
```

この場合、`b` は後から変更できないだけでなく、コンパイル時に値が決定できる定数として扱われる。そのため、配列の大きさやテンプレート引数など、コンパイル時に値が必要となる場面で利用できる。

表 39: `const` と `constexpr` の違い

指定	特 徴
<code>const</code>	実行時に初期値を決定できるが、値を後から変更できない。
<code>constexpr</code>	コンパイル時に値が決定できる定数であり、値を後から変更できない。

`const` は主に「変更禁止」を表し、`constexpr` は主に「コンパイル時に決定可能」を意味する。`constexpr` で宣言された変数は、コンパイル時に評価可能な定数として扱われ、後から値を変更することはできない。

次の例を見ると、両者の違いが分かりやすい。

```
int n;
cin >> n;
const int a = n;          ←これは可能
constexpr int b = n;      ←これは不可
```

`n` の値は、プログラムを実行して入力を受け取るまで決まらない。そのため、`const int a = n;` は、「実行時に決まった値を、その後変更できないようにする」宣言として成立する。一方、`constexpr int b = n;` は、`b` の値をコンパイル時に決定しようとするため、実行時にしか決まらない `n` を初期値にすることはできない。

`constexpr` は、C 言語の `#define` によるマクロ定数に近い用途で使われることもある。しかし、`#define` はプリプロセッサによる単純な文字列置換であり、型やスコープを持たない。これに対して、`constexpr` による定数は、C++ の通常の名前として扱われ、型情報とスコープを持つ。従って、`constexpr` は、型を持つ安全なコンパイル時定数を定義する方法として利用できる。例えば、次のように記述できる。

```
constexpr int SIZE = 100;
int a[SIZE];
```

この例では、`SIZE` の値がコンパイル時に決定できるため、配列の要素数として使用できる。このように、プログラム中で使用する固定的な値を表す場合には、`#define` よりも `constexpr` を用いる方が、C++ では望ましい。

`constexpr` は、変数だけでなく関数の定義にも指定することができ、その関数は、引数などの値がコンパイル時に決定できる場合に、コンパイル時に評価できる関数となる。例えば、次の関数は整数値の 2 乗を返す `constexpr` 関数である。

```
constexpr int square(int x) {
    return x * x;
}
```

この関数にコンパイル時に決定できる値を与えると、結果もコンパイル時に決定できる。

```
constexpr int n = square(5);
```

この場合、`square(5)` はコンパイル時に評価できるため、`n` は値 25 を持つコンパイル時定数となる。

注意すべきこととして、`constexpr` 関数は、常にコンパイル時に実行されるということを意味しないという点がある。実行時に決まる値を引数として与えた場合には、通常の間数と同じように実行時に評価される。例えば、次のような場合である。

```
int x;
cin >> x;
int y = square(x);
```

この場合、`x` の値はプログラムの実行時に入力されるため、`square(x)` の値も実行時に計算される。すなわち、`constexpr` 関数は、コンパイル時に評価できる条件が整っている場合にはコンパイル時に評価でき、そうでない場合には通常の間数と同様に実行時に評価される。

11.12 値のカテゴリ

これまでに見てきたように、C++では、式の値の性質によって、関数の引数の受け取り方や、オーバーロードの解決の結果が変わる場合がある。これらの振る舞いは、値の持つ性質を分類した**値のカテゴリ**によって整理することができる。

C++における値は、大きく次の2つに分類される。

- **lvalue** (左辺値)
- **rvalue** (右辺値)

lvalue とは、式の評価結果として、特定のオブジェクトを表す値のカテゴリである。通常の変数を表す式は **lvalue** となる。例えば、

```
int a = 10;
```

としたとき、変数 `a` は **lvalue** であり、

```
a = 20;
```

のように代入の左辺に用いることができる。

一方、**rvalue** とは、一時的な値や計算によって得られる値などを表す値のカテゴリであり、通常は代入の対象とはならない。例えば、数値リテラルや、通常の算術演算によって得られる値などは **rvalue** である。次に示す

```
10
a + 5
```

のような値は **rvalue** であり、これらに対して直接代入を行うことはできない。

この値のカテゴリの違いは、参照の振る舞いと密接に関係している。例えば、

```
int& r1 = a;
int& r2 = 10;    // 誤り
```

とした場合、`a` は **lvalue** であるため参照を束縛することができるが、`10` は **rvalue** であるため、変更可能な参照 (`int&`) には束縛することができない。

一方、**const 参照**を用いると、**rvalue** に対しても参照を束縛することができる。例えば、

```
const int& r3 = 10;
```

とすると、一時的な値である **rvalue** に対して参照が束縛され、その寿命は参照の有効範囲 (スコープ) まで延長される。

このように、**lvalue** と **rvalue** の違いによって、参照に束縛できるかどうかが決まり、結果として関数のオーバーロードの選択にも影響を与える。これまでに見てきた「一時オブジェクト」や「**const 参照**の振る舞い」は、この値のカテゴリの違いによって統一的に説明することができる。

本書では基本的な理解のために、この2分類で扱う。

重要) 通常の変数を表す式は **lvalue** となり、数値リテラルや算術演算によって得られる一時的な値などは **rvalue** となる。この違いが参照の扱いや関数の選択に影響を与える。

11.13 例外処理

C 言語では、プログラムの実行中に発生する異常な事態への対応として、関数の戻り値やエラーコードを調べる方法が広く用いられる。また、シグナルを受け取る方法があり、必要に応じて `setjmp`、`longjmp` により、通常の関数呼び出しとは異なる制御¹³⁰ を行う。シグナル処理や `longjmp` による制御移動は強力である一方、プログラムの流れを大きく変更するため、処理の見通しが悪くなる原因にもなる。特に、途中で確保した資源の後始末や、オブジェクトの状態の整合性を保つ処理には注意が必要である。

C++ では、このような異常な事態を扱うために、**例外処理**という仕組みが用意されている。例外処理は、通常の処理の流れでは対処しにくい異常な状態を、呼び出し元の適切な場所へ通知するための仕組みである。これによって、通常の処理と、異常時の処理を分けて記述しやすくなる。

例外処理を用いると、エラーが発生した場所で直ちに処理を完結させるのではなく、そのエラーを呼び出し元へ伝え、適切な場所で対処する処理を構造的に記述することができる。これにより、プログラム全体の構造を整理しやすくなり、異常時の処理を見通しよく記述しやすくなる。特に C++ では、クラス、コンストラクタ、デストラクタ、リソース管理など¹³¹ と組み合わせることで、異常時にも適切に後始末を行う設計が可能になる。

ただし、例外処理は慎重に行うべきであり、通常の条件分岐で十分に扱える事柄まで例外として扱うと、かえって処理の流れが分かりにくくなることがある。例外処理は、主として「通常の処理としては続行できない状態」や「その場では適切に対処できない異常」を受けて対処するために用いる。そのため、例外処理を応用すべき局面については、実装前に十分に検討しなければならない。

本節では、C++ の基本的な機能である例外処理について解説するが、**クラス**、**オブジェクト**とそれらに纏わる知識とも深く関わっている。そのため、それらの知識をまだ持たない読者は、後の「12 クラスとオブジェクト」(p.237～)、「13 継承とポリモーフィズム」(p.259～)の知識を得た後で本節の内容を学ぶことを推奨する。

11.13.1 例外として扱われる事象

C++ では、実行中に発生した異常な状態を表すために、いくつかの標準的な**例外クラス**¹³² が用意されている。例外クラスとは、エラーや異常状態の種類を表すためのものであり、例外処理では、これら例外クラスのオブジェクトを用いて異常の発生を通知する。代表的な標準例外クラスを表 40 に挙げる。

表 40: 代表的な標準例外クラス (一部)

例外クラス	ヘッダ	主な意味
<code>std::exception</code>	<code><exception></code>	標準例外クラスの基底クラス
<code>std::runtime_error</code>	<code><stdexcept></code>	実行時に発生する一般的なエラー
<code>std::logic_error</code>	<code><stdexcept></code>	プログラムの論理上の誤りを表すエラー
<code>std::invalid_argument</code>	<code><stdexcept></code>	不正な引数が渡された場合のエラー
<code>std::out_of_range</code>	<code><stdexcept></code>	範囲外アクセスを表すエラー
<code>std::range_error</code>	<code><stdexcept></code>	計算結果などが有効範囲を外れた場合のエラー*
<code>std::overflow_error</code>	<code><stdexcept></code>	数値が大きすぎる場合のエラー
<code>std::underflow_error</code>	<code><stdexcept></code>	数値が小さすぎる場合のエラー
<code>std::bad_alloc</code>	<code><new></code>	メモリ確保に失敗した場合のエラー

* 値域に関するエラー

`std::exception` は、標準例外クラスの基底クラスである。多くの標準例外クラスは `std::exception` から派生しているため、個々の例外クラスを区別せずにまとめて扱う場合には、`std::exception` 型として受け取ることができる。この点について詳しくは後に解説する。

`std::runtime_error` は、実行してみなければ分からない種類のエラー¹³³ を表すために用いられる。例えば、ファイルが開けない、通信に失敗した、外部から必要なデータが得られない、といった場合が挙げられる。これらは、プ

¹³⁰ 「9.11 シグナル処理とジャンプ」(p.147)

¹³¹ 「12 クラスとオブジェクト」以降で解説する。

¹³² 「クラス」あるいはそれに纏わる用語に関しても、詳しくは「12 クラスとオブジェクト」以降で解説する。

¹³³ 実行時の環境や外部条件などによって発生するエラー。

プログラムの文法や論理構造の誤りによるものではなく、実行時の環境や外部条件によって発生するエラーである。

`std::logic_error` は、不適切なプログラムの設計や用法に起因する不正な状態を表すために用いられる。例えば、不正な引数を渡した場合には `std::invalid_argument`、範囲外の位置を指定した場合には `std::out_of_range` が対応する。これらは `std::logic_error` の派生クラスである。

数値計算に関係する例外としては、`std::range_error`、`std::overflow_error`、`std::underflow_error` などがある。ただし、通常の整数演算や浮動小数点演算において、オーバーフローや 0 による除算が発生したときに、これらの例外が自動的に発生するとは限らない。これらは主として、ライブラリや自作の関数が数値計算上の異常を通知するために用いる例外クラスである。

`std::bad_alloc` は、動的なメモリ確保に失敗した場合に送出される例外である。大きな配列や多数のオブジェクトを動的に確保しようとしたとき、必要なメモリを確保できない場合がある。このような場合、通常の `new` によるメモリ確保では `std::bad_alloc` が送出される。

このように、例外クラスは、異常な状態の種類を表すために用いられる。どの例外クラスを用いるかは、発生した問題の性質によって決める。外部要因による実行時の失敗であれば `std::runtime_error`、不正な引数であれば `std::invalid_argument`、範囲外アクセスであれば `std::out_of_range` のように、異常の意味に応じて適切な例外クラスを選ぶことが重要である。

11.13.2 例外のハンドリング

例外が送出される可能性のある処理を実行する場合には、`try` 文と `catch` 節を用いて、例外を受け取る処理を記述する。`try` の中には、例外が発生する可能性のある処理を記述する。そして、その処理の途中で例外が送出された場合には、対応する `catch` 節に制御が移る。

《例外のハンドリング》

```
try {  
    (例外が発生する可能性のある処理)  
} catch ( 例外の型 1 引数 1 ) {  
    (例外発生時の処理 1)  
} catch ( 例外の型 2 引数 2 ) {  
    (例外発生時の処理 2)  
    :  
} catch ( ... ) {  
    (上のどの例外の型にも該当しない場合の例外処理)  
}
```

`try` ブロックの中で例外が送出されると、そのブロックの実行は中断し、発生した例外の型に対応する `catch` ブロックへ制御が移る。`catch` では、受け取る例外の型と引数を指定し、その例外に対する処理「例外発生時の処理 *n*」を記述する。また、最終の「`catch (...)`」のブロックは、記述したどの例外の型にも該当しない例外が送出された場合の処理を扱う部分である。

注意) `catch (...)` は、C++ の例外として送出されたものを受け取るための仕組みであるが、整数の 0 除算やメモリアクセス違反など、未定義動作や OS レベルの実行時異常の全てを安全に捕捉するものではない。

11.13.3 例外の送出

標準ライブラリや処理系が、異常な状態を検出して例外を送出することがある。例えば、後述の `new` による動的メモリ確保に失敗した場合には `std::bad_alloc` が送出され、後述の `std::vector` の `at` メンバ関数で範囲外の位置を指定した場合には `std::out_of_range` が送出される。さらに、`std::stoi` や `std::stod` などの文字列変換関数では、文字列を数値として解釈できない場合に `std::invalid_argument`、変換結果が表現範囲を超える場合に `std::out_of_range` が送出される。

例外は、システムやライブラリの内部で自動的に発生するものだけではない。ユーザが作成するプログラムの側で、異常な状態を検出し、明示的に例外を送出することもできる。この際に用いるのが `throw` である。

例えば、関数に不正な引数が渡された場合や、ファイルを開けなかった場合、指定された値が有効な範囲を外れている場合などには、その関数の内部で異常を検出できる。このような場合、その場で無理に処理を続けるのではなく、`throw` によって例外を送出し、呼び出し元に異常を通知することができる。

例外の送出方法は次の通り。

書き方： `throw` 例外オブジェクト

例えば、実行時に発生した一般的なエラーを通知する場合には、`std::runtime_error` を用いて、次のように記述できる。

例. 実行時エラーを例外として送出する記述

```
throw std::runtime_error("file open error");
```

この文が実行されると、その時点で通常の処理の流れは中断される。そして、その例外を受け取ることのできる `catch` 節が探され、対応する `catch` ブロックへ制御が移る。したがって、`throw` の後に書かれた同じブロック内の処理は、通常は実行されない。

このように、例外処理は、単に「どこかで発生したエラーを受け取る」ためだけの仕組みではない。プログラム自身が異常な状態を検出し、その情報を呼び出し元へ通知するための仕組みでもある。この考え方は、例外処理を用いたプログラミングにおいて重要である。例えば、事前に検出できる不正な状態については、危険な処理を実行する前にその状態を検出し、その時点で例外を送出するように設計することができる。

なお、`throw` によって送出する例外には、標準ライブラリで定義されている例外クラスのオブジェクトを用いるのが一般的である。

多くの標準例外クラスは `std::exception` から派生しているため、次のように書く¹³⁴ と、標準的な例外をまとめて受け取ることができる。

例. 様々な例外をまとめてハンドリングする記述

```
try {  
    (例外が発生する可能性のある処理)  
} catch (const std::exception& e) {  
    (標準例外が発生した場合の処理)  
} catch ( ... ) {  
    (上記で対応できない例外が発生した場合の処理)  
}
```

ここで、`e` は受け取った例外オブジェクトを参照するための引数である。`std::exception` およびその派生クラスでは、`what()` というメンバ関数を用いることで、例外に関する説明文字列を取得できる。

注意) 上の例で `catch (const std::exception& e)` と、引数を `const` 参照にしているのは、例外オブジェクトの不要なコピーを防ぐだけでなく、派生クラスのオブジェクトスライシングを防ぐためである。これに関しては後で解説する。

■ サンプルプログラム

次のプログラム `test203-3.cpp` では、存在しないファイルを入力用に開こうとし、ファイルを開けなかった場合に `std::runtime_error` 型の例外を送出している。そして、`main` 関数側でその例外を受け取り、エラーメッセージを表示している。

例外処理を施さずにファイルを読み込み用にオープンすると、対象ファイルが存在しない場合、入力ファイルストリームは失敗の状態になる。`test203-3.cpp` は、この事態に例外処理の手法を応用して対処する例である。

プログラム： `test203-3.cpp`

```
1 #include <iostream>  
2 #include <fstream>  
3 #include <stdexcept>  
4 #include <exception>  
5  
6 void open_file(const char* filename) {  
7     std::ifstream fin(filename);           // ファイルをオープンしようとして
```

¹³⁴`std::exception` クラスのオブジェクトの参照として受け取る形式

```

8      if (!fin) {                                // 失敗した場合は
9          throw std::runtime_error("file open error");    // このエラーを起こす
10     }
11     // 正常にオープンできた場合の処理
12     std::cout << "file opened successfully\n";
13 }
14
15 int main() {
16     try {
17         std::cout << "opening 'testFile.txt'...\n";
18         open_file("testFile.txt");
19         std::cout << "message after open_file\n";    // ブロックが正常に終了した場合
20     } catch (const std::exception& e) {            // open_file関数で例外発生時
21         std::cerr << "exception: " << e.what() << "\n";
22     }
23
24     std::cout << "program end\n";
25 }

```

このプログラムでは、`open_file` 関数の中で `std::ifstream` を用いてファイルを開こうとしている。指定したファイル `testFile.txt` が存在しない場合、ファイルストリーム `fin` は正常に利用できる状態にならない。そこで、「`if (!fin)`」でファイルを開けなかったことを確認し、「`throw std::runtime_error("file open error")`」によって例外を送出している。

`main` 関数では、`open_file` 関数の呼び出しを `try` ブロックの中に記述している。そのため、`open_file` 関数の中で送出された例外は「`catch (const std::exception& e)`」によって受け取られる。このとき、`e.what()` によって、例外に付けられた説明文字列を取り出している。

`open_file` 関数の中で例外が送出されると、その時点で通常の処理の流れは中断される。従って、`try` ブロック内にある「`std::cout << "message after open_file\n";`」は実行されない。その後、対応する `catch` ブロックで例外が処理され、さらにその後の処理として "program end" が表示される。

このように、例外処理を用いると、エラーが発生した場所と、そのエラーに対処する場所を分けて記述できる。ファイルを開く関数は、「ファイルを開けなかった」という事実を例外として通知し、呼び出し側は、その例外を受け取って、処理の中止やメッセージの出力といった別の処理を行う判断をすることができる。

上のプログラムをコンパイルして実行する例を次に示す。

例. `test203-3.cpp` のコンパイルと実行

```

$ g++ test203-3.cpp -o test203-3  [Enter]    ←コンパイル
$ ./test203-3  [Enter]    ←実行
opening 'testFile.txt'...                ←ファイルが存在しないので
exception: file open error                ←エラーメッセージを出力
program end
$ touch testFile.txt  [Enter]    ←ファイルを作成して
$ ./test203-3  [Enter]    ←再度実行すると
opening 'testFile.txt'...                ←ファイルが存在するので
file opened successfully                ←ファイルがオープンできた旨のメッセージ
message after open_file                ←処理が中断されずに続いている
program end
$ rm testFile.txt  [Enter]    ←ファイルを削除

```

11.13.4 noexcept による例外伝播の抑止

関数によっては、処理の途中で例外が発生しても、その例外を呼び出し元へ伝播させることが適切でない場合がある。例えば、オブジェクトの終了処理を行う関数や、プログラムの内部状態を安全に保つための処理では、例外が関数の外部へ送出されると、後続の処理を正常に継続できなくなることがある。

また、関数から例外が呼び出し元へ伝播しないことがあらかじめ分かっているならば、その性質を前提として呼び出し側の処理を組み立てることができる。このような性質を関数の仕様として明示するために、C++では `noexcept` を使用する。

`noexcept` は、関数から例外を外部へ伝播させないことを宣言するための指定である。コンパイラや標準ライブラリは、この情報を利用して、処理の最適化や、オブジェクトの移動・複製方法の選択を行うことがある。

関数が例外を外部へ伝播させないことを指定するには、関数宣言の引数リストの後に `noexcept` を記述する。

書き方： 戻り値の型 関数名 (引数リスト) `noexcept` { 関数の定義 }

`noexcept` は、戻り値の型の前や関数名の前ではなく、引数リストの直後に記述する。また、関数を宣言と定義に分けて記述する場合¹³⁵ には、宣言側と定義側の両方に `noexcept` を記述する。このとき、宣言と定義で例外仕様が一致していなければ、コンパイルエラーとなる。

■ 補足

`noexcept` を指定した関数内に `throw` 文を記述しても、通常はコンパイルエラーにはならない。ただし、送出された例外がその関数内で捕捉されず、関数の外部へ伝播しようとする、後述の `std::terminate()` が呼び出される。そのため、対応する `catch` 節を呼び出し元に記述していても、その例外を捕捉することはできない。一方、例外を関数内で捕捉して処理し、外部へ伝播させなければ、`noexcept` の指定には反しない。

11.13.5 回復不能な異常と終了処理

これまでに説明した例外処理では、`throw` 文によって送出された例外を `catch` 節で捕捉し、異常の内容に応じた処理を行った。しかし、例外が適切に捕捉されなかった場合や、例外処理を継続できない状態に陥った場合には、通常の処理へ戻ることができない。このような場合、C++では `std::terminate` 関数が呼び出され、プログラムの実行が終了する。

`std::terminate` 関数は、単に例外が発生したことを通知するための関数ではなく、例外処理を正常に継続できないことが確定した場合に呼び出される。例えば、送出された例外に適合する `catch` 節が見つからない場合や、例外を外部へ送出不いことを表す `noexcept` 指定の関数から例外が出ようとした場合などに、`std::terminate` 関数が呼び出される。

第1部の「9.15 プログラムの終了処理」(p.166～)で説明したように、C言語の `abort` 関数は、通常の終了処理を行わずにプログラムを異常終了させるための関数である。`std::terminate` 関数も最終的にはプログラムを異常終了させるが、主な役割は、C++の例外処理機構が回復不能な状態を検出した際に、あらかじめ設定されている**終了処理関数** (terminate handler) を呼び出すことにある。既定の終了処理では、`std::abort` 関数が呼び出され、プログラムが異常終了する。

また、`std::set_terminate` 関数を使用すると、`std::terminate` 関数が呼び出された際に実行する独自の終了処理関数を設定できる。ただし、この処理は、異常から回復して通常の実行を再開するためのものではない。エラーメッセージや診断情報を出力するなどの最終処理を行い、プログラムを終了させるために使用する。

■ 終了処理の設定方法

`std::terminate` 関数が呼び出された際に、既定の終了処理とは異なる独自の処理を実行させるには、あらかじめ終了処理用の関数を作成し、それを `std::set_terminate` 関数を用いて登録する。この設定は、次のような一連の手順で行う。

- (1) 終了時に実行する関数を定義する
- (2) その関数を `std::set_terminate` 関数に渡して登録する
- (3) 回復不能な異常によって `std::terminate` 関数が呼び出されると、登録した終了処理関数が実行される

ここで、`std::terminate_handler` と `std::set_terminate` は、二つの異なる設定方法を表しているのではない。前者は終了処理関数へのポインタを表す型名であり、後者が実際に終了処理関数を登録するための関数である。

まず、ユーザが独自に定義する終了処理関数 `my_terminate` を、次のように定義する (関数名は任意でよい)。

```
void my_terminate() {  
    終了時に行う処理  
}
```

`std::terminate` 関数から呼び出される終了処理関数は、引数を受け取らず、戻り値を返さない形式で定義する。従っ

¹³⁵例えば、ヘッダファイルに関数のプロトタイプ宣言を記述する場合などが典型的である。

て、この例の `my_terminate` 関数の型は、「引数を受け取らず、戻り値を返さない関数」である。

次に、この関数を `std::set_terminate` 関数に渡して登録する。

```
std::set_terminate(my_terminate);
```

この記述によって、以後 `std::terminate` 関数が呼び出された場合には、既定の終了処理に代わって `my_terminate` 関数が呼び出される。つまり、終了処理関数を実際に設定しているのは、この `std::set_terminate` 関数の呼び出しである。

このとき、引数として指定した `my_terminate` は関数名であるが、関数名をこのような位置に記述すると、その関数へのポインタとして扱われる。従って、

```
std::set_terminate(my_terminate);
```

は、`my_terminate` 関数そのものを複製したり、その場で実行したりする記述ではない。`my_terminate` 関数の所在を表す情報、すなわち関数へのポインタを `std::set_terminate` 関数へ渡している。

終了処理関数へのポインタを表すために、C++標準ライブラリでは `std::terminate_handler` という型名が用意されている。この型は、概念的には次のように定義されている。

```
using terminate_handler = void (*)();
```

ここで、`void (*)()` は、「引数を受け取らず、戻り値を返さない関数へのポインタ型」を表している。この複雑な型に `terminate_handler` という分かりやすい名前を付けたものが、`std::terminate_handler` である。

ただし、この `using` 宣言を利用者が自分で記述する必要はない。`std::terminate_handler` は、標準ライブラリの中ですでに定義されている型名である。従って、通常は次のように `std::set_terminate` 関数へ関数名を直接渡せばよい。

```
std::set_terminate(my_terminate);           (1)
```

一方、次のようにして終了処理関数へのポインタを変数に保存してから登録することもできる。

```
std::terminate_handler handler = my_terminate;    (2)-1
std::set_terminate(handler);                      (2)-2
```

(1)、(2) の二つの記述は、終了処理関数の登録という点では同じ結果になる。前者は関数名を直接渡しており、後者は一度 `std::terminate_handler` 型の変数 `handler` に保存してから渡している。従って、これらは異なる機能を利用しているのではなく、同じ終了処理関数を異なる書き方で `std::set_terminate` 関数へ渡しているだけである。ただし、通常は、次の簡単な書き方で十分である。

```
std::set_terminate(my_terminate);
```

`std::terminate_handler` 型の変数を明示的に用いるのは、終了処理関数へのポインタを保存しておきたい場合や、複数の終了処理関数を切り替えて使用したい場合などである。

また、`std::set_terminate` 関数は、新しい終了処理関数を登録すると同時に、それまで設定されていた終了処理関数へのポインタを返す。従って、次のように記述すると、以前の終了処理関数を保存しておくことができる。

```
std::terminate_handler old_handler;
old_handler = std::set_terminate(my_terminate);
```

この処理では、`my_terminate` 関数が新たな終了処理関数として登録される。同時に、それまで登録されていた終了処理関数へのポインタが返され、変数 `old_handler` に格納される。必要であれば、後で次のように以前の終了処理関数へ戻すこともできる。

```
std::set_terminate(old_handler);
```

従って、`std::terminate_handler` 型の変数は、終了処理関数を設定するために必ず必要なものではない。終了処理関数へのポインタを保存したり、後で再利用したりする場合に用いるものである。終了処理関数を単に一つ登録するだけであれば、先のように

```
std::set_terminate(my_terminate);
```

と記述するだけでよい。

登録する終了処理関数は、通常関数のように処理を終えて呼び出し元へ戻ってはならない。この関数は、エラー

メッセージや診断情報を出力するなどの最終処理を行った後、`std::abort` 関数などを用いてプログラムの実行を終了させる必要がある。

■ サンプルプログラム

次のプログラム `test203-5.cpp` は終了処理を示す例である。

プログラム：test203-5.cpp

```
1  #include    <cstdlib>
2  #include    <exception>
3  #include    <iostream>
4
5  // 終了処理関数
6  void my_terminate() {
7      std::cerr << "回復不能な異常が発生しました。\\n";
8      std::abort();    // 異常終了させる
9  }
10
11 int main() {
12     std::set_terminate(my_terminate);    // 終了処理関数の登録
13
14     std::terminate();                    // 終了処理を確認するために直接呼び出す
15 }
```

この例では、`std::set_terminate` 関数によって `my_terminate` 関数を `terminate handler`（終了処理関数）として登録している。その後、終了処理の動作を確認するために `std::terminate` 関数を直接呼び出している。これによって `my_terminate` 関数が実行され、メッセージを表示した後、この関数の中で呼び出している `std::abort` 関数によってプログラムが異常終了する。

このプログラムをコンパイルして実行する例を次に示す。

例. `test203-5.cpp` のコンパイルと実行

```
$ g++ test203-5.cpp -o test203-5      ←コンパイル
$ ./test203-5      ←実行
回復不能な異常が発生しました。    ←出力
```


11.14 日付や時間に関する機能

C++では、C言語の `time.h` に対応する `ctime` ヘッダが用意されており、これをインクルードすることで、C言語由来の日付・時刻関連の型や関数を使用することができる。従って、現在の日付や時刻の取得、年月日や時分秒への分解、日付・時刻の文字列への書式整形などは、基本的にC言語の場合と同様の方法で行うことができる。

一方、C++には、時間間隔や時刻を型として扱うための独自の機能も用意されている。これらの機能は主に `chrono` ヘッダで提供され、秒、ミリ秒、マイクロ秒などの時間間隔や、ある時点を表す時刻点、時刻を測定するためのクロックなどを扱うことができる。また、これらを利用することで、プログラム中の処理に要した時間を比較的安全で分かりやすい形で計測することができる。

本節では、まずC言語由来の日付・時刻処理について確認した後、C++の `chrono` ヘッダで提供される時間間隔、時刻点、クロックの基本的な扱い方と、それらを用いた処理時間の計測方法について解説する。

11.14.1 C言語由来の日付、時刻の処理

先の「9.12 日付、時刻の扱い」(p.154～)で、C言語による日付、時刻の取扱いについて解説した。C++でも同様のAPIを使用することができ、`ctime` ヘッダを用いる場合は、これらの型や関数を名前空間 `std` の名前として利用できる。

先に紹介したC言語のプログラム `test023-1.c` と同様の処理を行うC++のプログラム `test023-1.cpp` を示す。

プログラム：test023-1.cpp

```
1  #include <iostream>
2  #include <ctime>
3  #define BUFSIZE 128
4
5  int main() {
6      std::time_t now;           // 時刻を保持する変数
7      std::tm lt, gt, *tmp;      // 日付・時刻情報のための構造体
8      char buf[BUFSIZE];        // 日付・時刻のための文字配列
9
10     now = std::time(nullptr);   // 現在時刻の取得
11
12     tmp = std::localtime(&now); // ローカル時間として日付・時刻情報を構造体に展開
13     lt = *tmp;                  // 内容をltに複製
14
15     // 構造体の各メンバを確認
16     std::cout
17         << "local)\tyear:" << lt.tm_year << ", month:" << lt.tm_mon
18         << ", day:" << lt.tm_mday << ",\n\thour:" << lt.tm_hour
19         << ", min:" << lt.tm_min << ", sec:" << lt.tm_sec << "\n";
20
21     // 書式整形 (タイムスタンプ)
22     std::strftime(buf, BUFSIZE, "%Y-%m-%d %H:%M:%S", &lt);
23     std::cout << "local)\t" << buf << "\n";
24
25     tmp = std::gmtime(&now);    // UTCとして日付・時刻情報を構造体に展開
26     gt = *tmp;                  // 内容をgtに複製
27
28     // 書式整形 (タイムスタンプ)
29     std::strftime(buf, BUFSIZE, "%Y-%m-%d %H:%M:%S", &gt);
30     std::cout << "UTC)\t" << buf << "\n";
31 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test023-1.cpp のコンパイルと実行

\$ g++ test023-1.cpp -o test023-1 ←コンパイル

\$./test023-1 ←実行

local) year:126, month:5, day:27, ←出力
hour:12, min:12, sec:36

local) 2026-06-27 12:12:36

UTC) 2026-06-27 03:12:36

注意) tm 構造体のメンバ tm_mon は 0～11 の値を取る。従ってこの例では、6 月の日付において tm_mon の値は 5 となっていることに注意すること。また、tm_year は 1900 年からの経過年数を表すため、2026 年の場合の値は 126 となる。

11.14.2 日付、時刻の文字列変換

C++では、文字列として記述された日付や時刻を読み取るために、`iomanip` ヘッダで提供される `std::get_time()` を使用することができる。また、tm 構造体に格納された日付や時刻を書式整形して出力するために、`std::put_time()` が用意されている。

`std::get_time()` は入力ストリームから日付や時刻を読み取るための機能であり、`std::put_time()` は出力ストリームに日付や時刻を書き出すための機能である。このように、入力用の `get_time()` と出力用の `put_time()` が対になる形で用意されている点は、C++のストリーム入出力の特徴の1つである。

文字列に記述された日付や時刻を `get_time()` で読み取る場合は、文字列入力ストリーム `istringstream` と組み合わせて、次のように記述する。

```
iss >> std::get_time(&t, 書式);
```

ここで、`iss` は入力元となる文字列入力ストリーム、`t` は読み取った日付や時刻を格納する tm 構造体の変数である。また、書式には、入力文字列がどのような形式で記述されているかを指定する。例えば、

```
iss >> std::get_time(&t, "%Y-%m-%d %H:%M:%S");
```

と記述すると、2026-06-27 18:30:41 のような形式の文字列を読み取ることができる。変換に失敗した場合は入力ストリームがエラー状態になるため、`fail()` を用いて結果を確認することができる。

一方、`put_time()` は、次のように出力ストリームの演算子 `<<` と組み合わせて使用する。

```
std::cout << std::put_time(&t, 書式);
```

`get_time()` および `put_time()` の書式には、基本的に C 言語の `strftime()` などを使用するものと同様の変換指定を使用することができる。例えば、`%Y` は西暦年、`%m` は月、`%d` は日、`%H` は時、`%M` は分、`%S` は秒を表す。従って、同じ書式文字列を入力と出力の両方に利用することもできる。

先に紹介した C 言語のプログラム `test023-2.c` と同様の処理を行う C++のプログラム `test023-2.cpp` を示す。このプログラムでは、文字列として与えられた日付と時刻を `get_time()` で tm 構造体に読み込み、構造体の各メンバの値を表示する。さらに、`put_time()` を用いて、読み取った日付と時刻をタイムスタンプの形式で出力する。

プログラム：test023-2.cpp

```
1  #include <iostream>
2  #include <iomanip>
3  #include <sstream>
4  #include <ctime>
5
6  int main() {
7      const char *str = "2026-06-27 18:30:41";    // タイムスタンプ文字列
8
9      std::tm t = {};                             // 初期化しておく
10     std::istringstream iss(str);                 // 文字列入力ストリーム
11
12     iss >> std::get_time(&t, "%Y-%m-%d %H:%M:%S"); // 文字列 → tm構造体
13
14     if ( iss.fail() ) {
15         std::cout << "変換に失敗しました\n";
16         return 1;
17     }
18
19     std::cout
20         << "year:" << t.tm_year + 1900 << ", mon:" << t.tm_mon + 1
21         << ", day:" << t.tm_mday << ", hour:" << t.tm_hour
22         << ", min:" << t.tm_min << ", sec:" << t.tm_sec << "\n";
23
24     std::cout << std::put_time(&t, "%Y-%m-%d %H:%M:%S") << "\n";
25 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test023-2.cpp のコンパイルと実行

```
$ g++ test023-2.cpp -o test023-2  ←コンパイル
$ ./test023-2  ←実行
year:2026, mon:6, day:27, hour:18, min:30, sec:41 ←出力
2026-06-27 18:30:41
```

この例では、`get_time()` によって読み取られた年と月は、`tm` 構造体の `tm_year` および `tm_mon` に格納される。これらのメンバは、それぞれ 1900 年からの経過年数および 0 から始まる月番号を表すため、各値を通常の西暦年と月として表示する際には、`tm_year` に 1900 を、`tm_mon` に 1 を加えている。これに対して、`put_time()` では、`tm` 構造体の値が指定された書式に従って変換されるため、このような補正を明示的に行う必要はない。

11.14.3 時刻点と時間間隔

C++では、時刻を扱うための機能が `chrono` ヘッダに用意されている。この機能では、ある特定の時点を表す**時刻点** (time point) と、2つの時刻点の間の長さなどを表す**時間間隔** (duration) を区別して扱う。

例えば、ある処理を開始した時刻と終了した時刻は、それぞれ時刻点として表される。これら2つの時刻点の差を求めると、その処理に要した時間間隔が得られる。概念的には、次の関係が成り立つ。

時刻点 − 時刻点 = 時間間隔

また、時刻点に時間間隔を加えたり、時刻点から時間間隔を引いたりすることで、別の時刻点を求めることもできる。

時刻点 + 時間間隔 = 時刻点

時刻点 − 時間間隔 = 時刻点

C++には、時刻を測定するための複数種類の**クロック** (clock) が用意されている。それぞれのクロックは、時刻の進み方、基準点、用途などが異なる。時刻点は単独で存在するのではなく、必ずいずれかのクロックに属する値として扱われる。従って、異なるクロックから得られた時刻点は別の型となり、原則として直接比較したり、差を求めたりすることはできない。

C++では、時刻点は `std::chrono::time_point`、時間間隔は `std::chrono::duration` によって表される。これらはクラステンプレートとして定義されており、時刻点の場合は使用するクロック、時間間隔の場合は値の型や時間単位などが型の一部として管理される。

時刻点は、通常、クロックの `now()` を呼び出して取得する。例えば、`std::chrono::steady_clock::now()` は、単調に進む `steady_clock` における現在の時刻点を取得する。この関数によって得られる値の型は、次のように記述することができる。

```
std::chrono::steady_clock::time_point
```

ただし、実際のプログラムでは型名が長くなるため、`auto` を用いて受け取ることが多い。

```
auto t = std::chrono::steady_clock::now();
```

補足) C++では、`std::chrono::time_point` は時刻点を表す汎用的なクラステンプレートとして定義されている。実際には、使用するクロックを指定して型を具体化する必要がある。例えば、`steady_clock` による時刻点の型は `std::chrono::steady_clock::time_point` と表される。また、`system_clock` による時刻点の型は `std::chrono::system_clock::time_point` と表され、これらは互いに異なる型である。

時間間隔を表す `duration` には、秒、ミリ秒、マイクロ秒、ナノ秒などを単位とする型があらかじめ用意されている。代表的なものを次に示す。

<code>std::chrono::seconds</code>	秒
<code>std::chrono::milliseconds</code>	ミリ秒
<code>std::chrono::microseconds</code>	マイクロ秒
<code>std::chrono::nanoseconds</code>	ナノ秒

例えば、`std::chrono::seconds(3)` は3秒の時間間隔を表し、`std::chrono::milliseconds(500)` は500ミリ秒の時間間隔を表す。これらの型では、時間の値だけでなく単位も型の一部として管理されるため、異なる単位の時間を比較したり演算したりする際の誤りを減らすことができる。

■ 時刻点の基準について

時刻点は、それぞれのクロックに定められた基準点からの経過時間として表される。C++では、すべてのクロック

に共通する絶対的な基準点は定められておらず、基準点はクロックの種類や処理系によって異なる。従って、時刻点の内部の値だけを見ても、それが暦上のどの日時を表しているかを常に判断できるわけではない。特に `steady_clock` の基準点は任意であり、日付や時刻を表すためではなく、主として時間間隔の計測に用いられる。

■ クロックと暦の間の変換

`system_clock` は、実世界の日付や時刻に対応するクロックである。`system_clock` から得られた時刻点は、`system_clock::to_time_t()` を使用することで `time_t` 型の値に変換することができる。さらに、この値を `localtime()` または `gmtime()` に渡すことで、同じ時刻点をローカル時間または UTC の暦情報として `tm` 構造体に展開することができる。概念的には、次のような変換となる。

```
system_clock::time_point → time_t → tm
```

例えば、`system_clock` における現在の時刻点を現地時刻として表示する場合は、次のように記述することができる。

```
auto tp = std::chrono::system_clock::now();
std::time_t tt = std::chrono::system_clock::to_time_t(tp);
std::tm t = *std::localtime(&tt);
```

逆に、暦上の日付や時刻を `system_clock` の時刻点へ変換する場合は、まず `tm` 構造体に年月日や時分秒を設定し、`mktime()` を使用して `time_t` 型の値に変換する。その後、`system_clock::from_time_t()` を使用することで、`system_clock::time_point` 型の時刻点を得ることができる。

```
tm → time_t → system_clock::time_point
```

例えば、2026 年 6 月 27 日 18 時 30 分 41 秒を表す現地時刻から時刻点を生成する場合は、次のように記述する。

```
std::tm t = {};
t.tm_year = 2026 - 1900;
t.tm_mon = 6 - 1;
t.tm_mday = 27;
t.tm_hour = 18;
t.tm_min = 30;
t.tm_sec = 41;
t.tm_isdst = -1;

std::time_t tt = std::mktime(&t);
auto tp = std::chrono::system_clock::from_time_t(tt);
```

`tm.isdst` に `-1` を設定すると、夏時間が適用されるかどうかの判定を処理系に任せることができる。また、`mktime()` は `tm` 構造体に格納された日時を、実行環境の現地時刻として解釈する。従って、この方法では任意のタイムゾーンを指定して変換することはできない。

一方、`steady_clock` は主として経過時間を計測するためのクロックであり、暦上の日付や時刻との対応関係を持たない。このため、`steady_clock::time_point` 型の時刻点を `time_t` や `tm` に変換するための標準的な機能は用意されていない。従って、日付や時刻の取得には `system_clock`、処理時間などの計測には `steady_clock` を使用する。

■ クロックの分解能について

`system_clock` および `steady_clock` が時刻を表す際の時間単位や分解能は、C++ の規格によって一律には定められておらず、処理系や実行環境に依存する。主要な処理系では、ナノ秒または 100 ナノ秒程度を単位として時刻点を表す実装が多いが、すべての環境で同じであるとは限らない。

クロックが時刻点を表す際の最小単位は、それぞれのクロックに定義されている `period` によって確認することができる。例えば、`steady_clock` の時間単位は、次のように調べることができる。

```
using P = std::chrono::steady_clock::period;
std::cout << P::num << "/" << P::den << "秒\n";
```

例えば、出力が `1/1000000000 秒` であれば、そのクロックの時刻点はナノ秒単位で表現されていることを意味する。同様に、`std::chrono::system_clock::period` を調べることで、`system_clock` の時間単位を確認することができる。

ただし、`period` が示す値は、時刻点を数値として表現する際の単位であり、クロックの値が実際にその間隔で更新されることを保証するものではない。例えば、ナノ秒単位で値を表現するクロックであっても、実際の時計の更新間

隔や測定精度は、OS、ハードウェア、クロックの実装などによって、これより粗くなる場合がある。

また、時間計測では、クロック自体の分解能だけでなく、`now()` の呼び出しに要する時間、OS によるタスクの切替え、CPU の動作状況なども測定結果に影響する。従って、高精度な時間計測を行う場合は、使用するコンパイラ、標準ライブラリ、OS などの技術仕様を確認し、実際の環境におけるクロックの特性を調べる必要がある。

■ サンプルプログラム

次に示すプログラム `dbl_now_test02.cpp` では、大きな整数に対して素数判定を行い、その処理に要した時間を計測する。処理時間の計測には、`std::chrono::steady_clock` を使用する。

まず、素数判定を開始する直前と終了した直後に `std::chrono::steady_clock::now()` を呼び出し、それぞれの時刻点を取得する。終了時刻点から開始時刻点を引くと、処理に要した時間間隔が得られる。この時間間隔を `std::chrono::duration<double>` 型の変数に格納することで、経過時間を秒単位の実数として扱うことができる。また、`count()` を呼び出すことで、その値を `double` 型の数値として取り出すことができる。

このプログラムでは、出力する経過時間を小数点以下 6 桁で表示するために、`fixed` および `setprecision()` を使用している。なお、実際に得られる計測値は、使用する処理系、OS、CPU の動作状況などによって変化する。

プログラム：`dbl_now_test02.cpp`

```
1  #include    <iostream>
2  #include    <iomanip>
3  #include    <chrono>
4
5  /* unsigned long long に対する素数判定 */
6  bool is_prime_ull(unsigned long long n) {
7      unsigned long long i;
8
9      if (n < 2) return false;
10     if (n == 2) return true;
11     if (n % 2 == 0) return false;
12
13     for (i = 3; i <= n / i; i += 2) if (n % i == 0) return false;
14
15     return true;
16 }
17
18 int main() {
19     unsigned long long n;
20     // n = 4000000000000000007ULL;      // 検査候補(1)
21     n = 4611686018427387847ULL;      // 検査候補(2)
22     // n = 18446744073709551557ULL;    // 検査候補(3)
23
24     auto t1 = std::chrono::steady_clock::now();    // 時間計測開始
25
26     if (is_prime_ull(n)) {
27         std::cout << n << " is prime.\n";
28     } else {
29         std::cout << n << " is not prime.\n";
30     }
31
32     auto t2 = std::chrono::steady_clock::now();    // 時間計測終了
33     std::chrono::duration<double> elapsed = t2 - t1;    // 経過時間
34
35     std::cout << std::fixed << std::setprecision(6)
36               << "elapsed = " << elapsed.count() << " sec\n";
37 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. `dbl_now_test02.cpp` のコンパイルと実行

```
$ g++ dbl_now_test02.cpp -o dbl_now_test02  [Enter]    ←コンパイル
$ ./dbl_now_test02  [Enter]    ←実行
4611686018427387847 is prime.    ←出力
elapsed = 4.358719 sec
```


11.15 C 言語標準ライブラリの利用

C++の標準ライブラリには、C++独自の機能に加えて、C 言語の標準ライブラリに由来する関数や型も含まれている。第1部で説明した入出力、文字列処理、数学関数、日付・時刻の処理、数値変換などの多くの機能は、C++のプログラムからも引き続き使用できる。これらの機能は、単に古い言語との互換性のためだけに残されているものではなく、C++においても用途に応じて利用される標準的な機能である。

ただし、C++では、C 言語の場合とはヘッダファイルの名称が異なることがあり、関数や型が名前空間に属する点などにも注意が必要である。また、同じ目的のために C++独自の機能が用意されている場合もあり、プログラムの設計や用途に応じて使い分ける必要がある。本節では、C 言語の標準ライブラリを C++から利用するための基本的な方法と、その際の主な注意点について解説する。

11.15.1 C 言語由来の標準ライブラリ

C++の標準ライブラリには、C 言語の標準ライブラリに由来する関数、型、定数などが含まれている。従って、第1部で説明した書式整形入出力、C 形式の文字列処理、数学関数、数値変換、日付・時刻の処理などの多くは、C++のプログラムでも利用することができる。例えば、次のような処理が可能である。

```
const char str[] = "C++ programming";
auto n = std::strlen(str);
auto y = std::sqrt(2.0);
std::printf("length = %zu\n", n);
std::printf("sqrt(2) = %.6f\n", y);
```

ここで使用している `strlen`, `sqrt`, `printf` は、いずれも C 言語の標準ライブラリに由来する関数である。これらは、単に C 言語との互換性を維持するためだけに残されているものではなく、C++の標準ライブラリを構成する機能として提供されている。そのため、C++のプログラムでも、目的に応じて通常の標準ライブラリ関数として利用してよい。

ただし、C++では、C 言語の標準ヘッダに対応する C++用のヘッダが用意されており、ヘッダファイルの名称や、関数および型が宣言される名前空間などに違いがある。また、一部の機能には、C++の型や関数のオーバーロードに対応した拡張や調整が加えられている。以降では、C 言語由来の標準ライブラリを C++で利用するための記述方法について説明する。

11.15.2 C++用ヘッダの読み込み

C 言語の標準ライブラリに含まれる機能を C++で利用する場合には、通常、C++用として用意されたヘッダファイルを読み込む。C++用のヘッダファイルの名称は、多くの場合、対応する C 言語のヘッダファイル名から拡張子 `.h` を取り除き、先頭に `c` を付けた形式になっている。例えば、C 言語の `stdio.h`, `string.h`, `math.h` に対応する C++用のヘッダファイルは、それぞれ次のようになる。

```
#include <cstdio>
#include <cstring>
#include <cmath>
```

これらのヘッダを読み込むことにより、`printf`, `strlen`, `sqrt` などの C 言語由来の機能を C++のプログラムで利用できる。例えば、前項で示したコード断片に、適切なヘッダファイルの読み込みと `main` 関数を加え、実行できる形にしたものを `test206-1.cpp` に示す。

プログラム：test206-1.cpp

```
1  #include    <cstdio>
2  #include    <cstring>
3  #include    <cmath>
4
5  int main() {
6      const char str[] = "C++ programming";
7      auto n = std::strlen(str);
8      auto y = std::sqrt(2.0);
9      std::printf("length = %zu\n", n);
10     std::printf("sqrt(2) = %.6f\n", y);
11 }
```


このプログラムをコンパイルして実行する例を次に示す。

例. test206-1.cpp のコンパイルと実行

```
$ g++ test206-1.cpp -o test206-1  ←コンパイル
$ ./test206-1  ←実行
length = 15 ←出力
sqrt(2) = 1.414214
```

主な C 言語の標準ヘッダと、それに対応する C++ 用ヘッダを表 41 に示す。

表 41: 主な C 言語の標準ヘッダと C++ 用ヘッダ

C 言語のヘッダ	C++ 用ヘッダ	主な機能
stdio.h	cstdio	書式整形入出力, ファイル入出力
stdlib.h	cstdlib	数値変換, 動的メモリ管理, 終了処理など
string.h	cstring	C 形式の文字列処理, メモリ操作
math.h	cmath	数学関数
time.h	ctime	日付, 時刻の処理
ctype.h	cctype	文字種の判定, 文字の変換
limits.h	climits	整数型が扱える値の範囲
float.h	cfloat	浮動小数点型の性質
stdint.h	cstdint	固定幅整数型
errno.h	cerrno	エラー番号
assert.h	cassert	アサーション

C++ 用ヘッダの名称は, 単に C 言語のヘッダファイル名を機械的に変更したものではない. C++ の型や名前空間の仕組みに合わせて宣言が調整されている場合があり, 一部の関数には C++ 用のオーバーロードが追加されている. そのため, C++ のプログラムでは, 原則として `cstdio`, `cstdlib`, `cstring` などの C++ 用ヘッダを読み込む。

11.15.3 std 名前空間からの利用

`cstdio`, `cstring`, `cmath` などの C++ 用ヘッダを読み込んだ場合, そこで宣言される関数や型は, 基本的に `std` 名前空間に属する. 従って, C 言語由来の関数を使用する場合にも, 次のように名前空間を指定する。

```
std::printf("value = %d\n", n);
auto len = std::strlen(str);
auto y = std::sqrt(x);
auto t = std::time(nullptr);
```

この例では, `printf`, `strlen`, `sqrt`, `time` を, それぞれ `std::printf`, `std::strlen`, `std::sqrt`, `std::time` として使用している. 型についても同様であり, 例えば, `size_t` 型は次のように記述できる。

```
std::size_t len = std::strlen(str);
```

C++ 用ヘッダを読み込んだ場合でも, 処理系によっては, これらの名前を `std::` を付けずに使用できることがある. 例えば, 次の記述がコンパイルできる場合がある。

```
#include <cstdio>
// ...
printf("Hello\n");
```

しかし, `cstdio` などの C++ 用ヘッダについて確実に利用できるのは, `std` 名前空間に属する名前である. そのため, 本書では, C++ 用ヘッダから提供される関数や型には, 原則として `std::` を付けて記述する。

ただし, C 言語由来の名前の中には, 関数や型ではなく, プリプロセッサマクロとして定義されるものもある. マクロは名前空間には属さないため, 次のように `std::` を付けずに使用する。

```
assert(n >= 0);
return EXIT_SUCCESS;
```

従って、C 言語由来の機能を使用する場合には、関数や型として宣言される名前と、マクロとして定義される名前とを区別する必要がある。

11.15.4 拡張子.hを持つヘッダとの違い

C++では、`cstdio`、`cstdlib`、`cstring` などの C++用ヘッダだけでなく、C 言語と同じく拡張子 `.h` を持つ標準ヘッダも利用できる。例えば、次のような記述も C++のプログラムとして有効である。

```
#include <stdio.h>
#include <string.h>
#include <math.h>
```

これらは、それぞれ `cstdio`、`cstring`、`cmath` に対応するヘッダである。ただし、両者では、そこで宣言される名前について保証される名前空間が異なる。例えば、次の二つの形式を比較する。

```
#include <cstdio>
std::printf("Hello\n");

#include <stdio.h>
printf("Hello\n");
```

`cstdio` を読み込んだ場合、`printf` は `std` 名前空間において確実に利用できる。一方、`stdio.h` を読み込んだ場合には、`printf` はグローバル名前空間において確実に利用できる。従って、それぞれのヘッダに対応する基本的な記述は、次のようになる。

読み込むヘッダ	基本的な記述
<code>cstdio</code>	<code>std::printf(...)</code>
<code>stdio.h</code>	<code>printf(...)</code>

ただし、処理系によっては、`cstdio` を読み込んだ場合にも `printf` をグローバル名前空間から利用できることがある。反対に、`stdio.h` を読み込んだ場合にも `std::printf` を利用できることがある。しかし、これらの追加的な名前の提供は保証されていないため、処理系に依存しないプログラムを記述するには、読み込んだヘッダに対応する名前空間を使用する必要がある。

拡張子 `.h` を持つヘッダは、C 言語と C++の双方から読み込むソースコードやヘッダファイルを作成する場合など、C 言語との互換性を必要とする場面で利用される。一方、C++ だけを対象とするソースコードでは、原則として `cstdio`、`cstdlib`、`cstring` などの C++用ヘッダを読み込み、関数や型を `std` 名前空間から利用する。

11.15.5 C++で利用する際の注意

C 言語由来の標準ライブラリは、C++でも通常の標準ライブラリとして利用できるが、すべての機能が C 言語の場合と全く同じ形で提供されているとは限らない。C++の型の仕組みに合わせて関数が調整されていたり、複数の型に対応するオーバーロードが追加されていたりする場合がある。例えば、`cmath` で提供される数学関数は、次のように異なる浮動小数点型の値に対して同じ関数名を使用できる。

```
auto y1 = std::sqrt(2.0f);    ← float 型の値
auto y2 = std::sqrt(2.0);     ← double 型の値
auto y3 = std::sqrt(2.0L);    ← long double 型の値
```

このように、C++用ヘッダで提供される機能は、C 言語の関数を単にそのまま移したのではなく、C++の型に合わせて利用しやすい形に拡張されている場合がある。

また、C 言語由来の関数には、文字配列やポインタ、書式指定文字列などを直接扱うものが多い。これらを利用する場合には、C 言語の場合と同様の注意が必要である。例えば、`strlen` 関数に渡す文字列は、末尾にヌル文字を持つ C 形式の文字列でなければならない。

```
const char str[] = "C++ programming";
auto n = std::strlen(str);
```

一方、`string` 型の文字列では、格納されている `char` 型要素の個数を調べるために `size` メンバ関数を使用できる。

```
std::string str = "C++ programming";  
auto n = str.size();
```

どちらの方法も C++ で利用できるが、扱うデータの型や、既存のプログラムとの関係に応じて使い分ける必要がある。

`printf` や `scanf` などの書式整形入出力関数を使用する場合には、書式指定子と引数の型を正しく対応させる必要がある。例えば、`printf` 関数によって値を出力する場合、`int` 型の整数値には `%d`、`double` 型の値には `%f` などを使用する。書式指定子と引数の型が一致しない場合には、正しい結果が得られないだけでなく、プログラムが不正な動作をすることもある。C++ では `iostream` による入出力も利用できるため、目的や必要な出力形式に応じて選択する。

さらに、標準ライブラリの機能を使用する場合には、その機能が宣言されているヘッダを明示的に読み込む必要がある。他のヘッダを読み込んだ結果として、必要な宣言が偶然利用できる場合があっても、そのような間接的な読み込みに依存してはならない。例えば、`strlen` を使用する場合には、次のように `cstring` を明示的に読み込む。

```
#include <cstring>
```

C 言語由来の標準ライブラリには、簡潔で実用的な機能が多く、C++ においても利用価値が高い。一方、同じ目的のために `string`、`iostream` などの C++ 独自の機能が用意されている場合もある。従って、C 言語由来の機能を一律に避けたり、反対に常に優先したりするのではなく、扱うデータ、安全性、既存コードとの互換性などを考慮して選択することが重要である。なお、C 言語で作成された別のプログラムやライブラリと C++ のプログラムを連携させる方法については、後の「19.7 C 言語のプログラムとの連携」(p.377～)で解説する。

12 クラスとオブジェクト

C 言語では、データ（変数）とそれに対する操作（関数）は分離して記述するのが一般的であった。例えば、ある構造体に対する処理であっても、その操作は個別の関数として別に定義され、どの関数がどのデータを対象としているのかは、プログラム全体を見渡して把握する必要があった。

これに対して C++ では、データとその操作を一体として記述するための仕組みが導入されている。これにより、プログラム中におけるデータと操作の対応関係が明確になり、構造が整理される。具体的には、次の点が明確になる。

- どのデータに対して、どのような操作が可能であるか
- どの操作が、どのデータを対象としているか

ここで、本章で扱う基本概念を整理する。

■ オブジェクト

型を持つデータの実体

■ クラス

オブジェクトの型を定義し、そのデータと操作を一体として記述する仕組み

クラスの中では、次の要素を定義する。

- メンバ変数： オブジェクトのデータ
- メンバ関数： オブジェクトの操作

C++ は、C 言語の考え方と枠組みを受け継ぎながら、データと操作を一体として扱うための仕組みを加えて発展した言語である。本章では、この「クラス」と「オブジェクト」という概念を基礎として、データと操作を統合的に扱うための方法について解説する。

12.1 クラスの定義

クラスは、オブジェクトの型を定義するための仕組みである。クラスは `class` キーワードを用いて次のように定義する。

《class キーワードによるクラスの定義》

```
class クラス名 {  
    (メンバの定義)  
};
```

クラスは、波括弧 `{` と `}` によって囲まれた領域の中に、そのクラスに属するメンバ（メンバ変数およびメンバ関数）を記述して定義する。なお、クラス定義の末尾にはセミコロン `;` が必要であることに注意する。

■ クラス定義の例

```
#include <cmath>  
  
class Vec2D {  
public:  
    double x;  
    double y;  
  
    double norm() {  
        return std::sqrt(x*x + y*y);  
    }  
};
```

この例では、2次元ベクトルを表すクラス `Vec2D` を定義している。このクラスは、次のメンバを持つ。

- メンバ変数： `x`, `y` ベクトルの各成分
- メンバ関数： `norm()` ベクトルの長さを求める関数

このように、クラスの中にデータとそれに対する操作をまとめて記述することで、データと操作の対応関係が明確になり、プログラムの構造が整理される。ここで「`public:`」は、この後に記述されたメンバがクラスの外部からアクセス可能であることを示すものである。メンバの公開範囲を制御する仕組みについては、後で解説する。

クラスによって定義された型を用いると、次のようにしてオブジェクトを定義することができる。

オブジェクトの定義： `Vec2D v;`

ここで定義された `v` は、クラス `Vec2D` に基づくオブジェクトである¹³⁶。このオブジェクトに対しては、ドット演算子「`.`」を用いて、次のようにしてメンバにアクセスすることができる。

```
v.x = 3.0;           // オブジェクト v のメンバ x に値を代入
v.y = 4.0;           // オブジェクト v のメンバ y に値を代入
double d = v.norm(); // オブジェクト v に対して norm() を実行する
```

このように、クラスを用いることで、データと操作を一体として扱うことが可能となる。これがクラスの基本的な役割である。

12.2 オブジェクトの独立性

同一のクラスに基づいて複数のオブジェクトを定義した場合であっても、それらは互いに独立した実体として扱われる。すなわち、各オブジェクトはそれぞれ固有のメンバ変数（状態）を持ち、その値は他のオブジェクトとは共有されない。

この性質を確認するために、前節で定義したクラス `Vec2D` を用いた例 `test204-1.cpp` を次に示す。

プログラム：`test204-1.cpp`

```
1  #include <iostream>
2  #include <cmath>
3  using std::cout;
4
5  class Vec2D {
6  public:
7      double x;
8      double y;
9      double norm() {
10         return std::sqrt(x*x + y*y);
11     }
12 };
13
14 int main() {
15     Vec2D a;
16     Vec2D b;
17
18     // Pythagorean triple
19     a.x = 3.0; a.y = 4.0;
20     b.x = 5.0; b.y = 12.0;
21
22     cout << "norm of (" << a.x << ", " << a.y << ") is " << a.norm() << "\n";
23     cout << "norm of (" << b.x << ", " << b.y << ") is " << b.norm() << "\n";
24 }
```

このプログラムでは、クラス `Vec2D` に基づいて2つのオブジェクト `a` と `b` を定義している。それぞれのオブジェクトに対して異なる値を代入した後、メンバ関数 `norm()` を呼び出している。

ここで重要なのは、`a` と `b` が互いに独立したオブジェクトであるという点である。`a` に対する代入は `b` には影響せず、また `b` に対する操作も `a` の状態を変化させることはない。すなわち、各オブジェクトはそれぞれ固有の非静的メンバ変数を持ち、その値は個別に保持されている。

このプログラムをコンパイルして実行すると、それぞれのオブジェクトに対して異なる結果が得られることが確認できる。(次の例)

例. `test204-1.cpp`

```
$ g++ test204-1.cpp -o test204-1  ←コンパイル
$ ./test204-1  ←実行
norm of (3,4) is 5      ←出力
norm of (5,12) is 13
```

¹³⁶ クラスに基づいて生成されたオブジェクトは**インスタンス**と呼ばれることがある。

このように、同一のクラスに基づいて複数のオブジェクトを生成した場合であっても、それぞれのオブジェクトは独立した状態を持つ。この例のメンバ関数 `norm()` は、呼び出された対象のオブジェクトが持つメンバ変数に対して処理を行う。従って、`a.norm()` と `b.norm()` では、それぞれ異なるオブジェクトの状態が用いられる。

クラスはオブジェクトの構造と振る舞いを定義するものであり、個々のオブジェクトはその定義に基づいて生成される実体である。すなわち、同一のクラスに基づいていても、各オブジェクトは独立した状態を持つ。この「定義」と「実体」の関係を理解することが、クラスとオブジェクトの振る舞いを正しく把握するための重要な基礎となる。

12.3 アクセス指定

クラスのメンバには、ドット演算子「`.`」を用いてアクセスする。すなわち、オブジェクト `v` に対して、`v.x` のように記述することでメンバ変数にアクセスでき、また `v.norm()` のように記述することでメンバ関数を呼び出すことができる。

しかし、すべてのメンバが常に自由にアクセスできるとは限らない。クラスでは、メンバへのアクセスを制御するための仕組みが用意されており、これにより、外部からの不適切な操作を防ぐことが可能となる。

12.3.1 `public` と `private`

クラスでは、メンバへのアクセスを制御するために、**アクセス指定子**と呼ばれる仕組みが用意されている。代表的なものとして、`public` と `private` がある。

`public` に指定されたメンバは、クラスの外部から自由にアクセスすることができる。一方、`private` に指定されたメンバは、クラスの外部から直接アクセスすることができず、基本的にはそのクラスのメンバ関数の中から利用する。例えば、アクセス指定子を用いると、次のようにメンバの公開範囲を制御することができる。

■ アクセス指定子を用いたクラスの定義例

```
class Vec2D {
    private:
        double x;
        double y;

    public:
        void set(double a, double b) {
            x = a; y = b;
        }

        double norm() {
            return std::sqrt(x*x + y*y);
        }
};
```

この例では、メンバ変数 `x` および `y` を `private` として定義しているため、クラスの外部から `v.x` のように直接アクセスすることはできない。その代わりに、`set()` のようなメンバ関数を通して値を設定することができる。

このように、メンバを `private` として隠蔽し、必要な操作のみを `public` として公開することで、クラスの内部状態を適切に管理することができる。このような仕組みにより、外部からの不正な操作や意図しない状態の変更を防ぐことが可能となる。実際には、隠蔽したメンバ変数に対して値を設定するための専用の関数¹³⁷ や、値を参照するための専用の関数¹³⁸ を実装し、`public` として公開するケースが多い。

アクセス指定子を適切に用いることは、クラスの設計において重要であり、後に解説する**カプセル化**¹³⁹ の考え方とも密接に関係している。

12.4 コンストラクタ

先の例では、オブジェクトを定義した後に `set()` のようなメンバ関数を用いて、その値を設定していた。しかし、この方法では、値が設定されていない状態のオブジェクトが存在する可能性があり、そのまま使用してしまうと、意

¹³⁷セッター (setter) と呼ばれることが多い。

¹³⁸ゲッター (getter) と呼ばれることが多い。

¹³⁹「12.9 カプセル化」(p.253～) で解説する。

図しない結果を招く可能性がある。

このような問題を防ぐために、C++ では、オブジェクトの生成と同時に初期化を行うための仕組みが用意されている。これを**コンストラクタ**と呼ぶ。

コンストラクタは、クラスと同じ名前を持つ特別なメンバ関数であり、戻り値を持たない（void も記述しない）。これは、オブジェクトの生成時に自動的に呼び出されるものであり、明示的に呼び出すものではない。コンストラクタの主な目的は、メンバ変数の初期化¹⁴⁰ である。

例えば、クラス Vec2D にコンストラクタを追加すると、次のように記述できる。

■ コンストラクタを用いたクラスの定義例

```
#include <cmath>

class Vec2D {
private:
    double x;
    double y;

public:
    Vec2D(double a, double b) {
        x = a; y = b;
    }

    void set(double a, double b) {
        x = a; y = b;
    }

    double norm() {
        return std::sqrt(x*x + y*y);
    }
};
```

この例では、Vec2D(double a, double b) がコンストラクタである。オブジェクトの生成時に引数として値を渡すことで、メンバ変数 x および y に初期値を設定することができる。

このクラスを用いてオブジェクトを定義する場合は、次のように記述する。

```
Vec2D v(3.0, 4.0);
```

このように記述すると、オブジェクト v の生成と同時にコンストラクタが呼び出され、x および y がそれぞれ 3.0 および 4.0 に設定される。

コンストラクタを用いることで、オブジェクトの生成時に必要な初期設定を行わせることができる。これは、初期化忘れによる不具合を防ぐことにつながる。

■ 初期値設定を伴うコンストラクタの記述

より直接的にメンバを初期化する方法として、次のような書き方もある。

書き方： クラス名 (引数リスト) : メンバ1(値1), メンバ2(値2), ... { コンストラクタの定義 }

このように、ブロック { ... } の前に、各メンバの初期化を記述すること¹⁴¹ ができる。例えば上の Vec2D コンストラクタを

```
Vec2D(double a, double b) : x(a), y(b) { }
```

と記述することができる。

この記述形式による初期化では、対象として、基本的な型のメンバ変数だけでなく、他のクラスのオブジェクトを指定する¹⁴² ことができる。その場合は、この初期化の記述により、そのメンバのクラスのコンストラクタが呼び出される。

重要) この方法では、コンストラクタ本体が実行される前に、各メンバが指定された値によって初期化される。

¹⁴⁰ オブジェクトの初期設定に関する処理が記述されることも多い（例えば、値の検査や初期状態の設定など）。

¹⁴¹ この記述を、**メンバ初期化子リスト**（member initializer list）という。

¹⁴² クラスのメンバとして、他のクラスのオブジェクトを用いることができる。このような設計を**コンポジション**と言い、後に解説する。

12.4.1 コンストラクタのオーバーロード

コンストラクタは、関数と同様に複数定義（オーバーロード）することができる。すなわち、引数の個数や型が異なる複数のコンストラクタを用意することで、さまざまな方法でオブジェクトを初期化することが可能となる。

例えば、クラス `Vec2D` において、次のように複数のコンストラクタを定義することができる。

■ 複数のコンストラクタの定義の例 （変更部分のみ）

```
public:
    Vec2D() {
        x = 0.0; y = 0.0;
    }

    Vec2D(double a, double b) {
        x = a; y = b;
    }
```

この例では、引数を持たないコンストラクタと、2つの引数を持つコンストラクタの2種類を定義している。前者はオブジェクトを原点 (0, 0) に初期化し、後者は指定された値で初期化する。

これらのコンストラクタは、オブジェクトの定義時に次のように使い分けることができる。

```
Vec2D a;           // (0, 0) に初期化
Vec2D b(3.0, 4.0); // 指定した値で初期化
```

このように、複数のコンストラクタを定義することで、状況に応じた柔軟な初期化を行うことが可能となる。

12.4.2 初期化の際の丸括弧と波括弧

C++では、コンストラクタに引数を渡してオブジェクトを初期化する際に、丸括弧 () または波括弧 { } を使用することができる。例えば、先の `Vec2D` クラスのオブジェクトは、次の2通りの方法で初期化できる。

```
Vec2D v1(3.0, 4.0);    ←丸括弧による初期化
Vec2D v2{3.0, 4.0};    ←波括弧による初期化
```

この例では、どちらの記述でも、2つの `double` 型の引数を受け取るコンストラクタが呼び出され、`x` および `y` がそれぞれ 3.0 および 4.0 に初期化される。従って、多くの場合、丸括弧と波括弧は同じようにコンストラクタへ引数を渡すために使用できる。

丸括弧による初期化は、従来から用いられてきたコンストラクタ呼び出しの記法である。これに対して、波括弧による初期化は、C++11以降で広く使用されるようになった初期化の記法であり、基本的な型、配列、構造体、クラスなどに対して、共通した形式で使用できる。このため、波括弧による初期化は**一様初期化** (uniform initialization) と呼ばれることがある。また、C++の文法上は、この形式の初期化を**リスト初期化** (list initialization) という。

丸括弧と波括弧による初期化の基本的な書式を次に示す。

書き方 1: クラス名 オブジェクト名 (引数リスト)
書き方 2: クラス名 オブジェクト名 { 引数リスト }

どちらの書き方でも、引数の個数や型に基づいてコンストラクタが選択される。ただし、後で述べるように、波括弧を使用した場合には選択規則が異なることがある。例えば、次のようなクラスが定義されているものとする。

■ 複数のコンストラクタを持つクラスの例

```
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D() : x(0.0), y(0.0) {}
    Vec2D(double v) : x(v), y(v) {}
    Vec2D(double a, double b) : x(a), y(b) {}
};
```

このクラスのオブジェクトは、次のように初期化できる。

```
Vec2D a;           // 引数を持たないコンストラクタ
Vec2D b{};         // 引数を持たないコンストラクタ
Vec2D c(3.0);      // 1つの引数を持つコンストラクタ
Vec2D d{3.0};      // 1つの引数を持つコンストラクタ
Vec2D e(3.0, 4.0); // 2つの引数を持つコンストラクタ
Vec2D f{3.0, 4.0}; // 2つの引数を持つコンストラクタ
```

a と b は、どちらも引数を持たないコンストラクタによって初期化される。c と d は、1つの引数を持つコンストラクタによって初期化され、x と y はともに 3.0 となる。また、e と f は、2つの引数を持つコンストラクタによって初期化される。

■ 縮小変換の防止

波括弧による初期化には、**縮小変換**と呼ばれる一部の型変換を禁止するという特徴がある。例えば、次のクラスを考える。

```
class Sample {
private:
    int n;

public:
    Sample(int x) : n(x) {}
};
```

このクラスに対して、次のようにオブジェクトを初期化したとする。

```
Sample s1(3.14);    // 3.14 が整数値 3 に変換される
Sample s2{3.14};    // コンパイルエラー
```

丸括弧を使用した場合、3.14 は int 型の値 3 に変換され、その値がコンストラクタに渡される。この変換では、小数点以下の値が失われる。これに対して、波括弧による初期化では、このような縮小変換が禁止されるため、コンパイルエラーとなる。

上の Sample s1(3.14); の例における double 型から int 型への変換などを**縮小変換** (narrowing conversion) という。波括弧による初期化を使用すると、意図しない縮小変換をコンパイル時に検出しやすくなる。そのため、新しくオブジェクトを初期化する場合には、波括弧を用いる方法が選ばれることも多い。

なお、丸括弧と波括弧による初期化は、常に完全に同じ動作をするわけではない。クラスによっては、複数の値をまとめて受け取るためのコンストラクタが定義されており、波括弧を使用した場合には、丸括弧を使用した場合とは異なるコンストラクタが優先して選択されることがある。このような例については、標準ライブラリのコンテナを解説する際に改めて取り上げる。

■ 引数を持たない初期化に関する注意

引数を持たないコンストラクタによってオブジェクトを初期化する場合は、次のいずれかの方法を使用できる。

```
Vec2D v1;
Vec2D v2{};
```

これらはいずれも、Vec2D の引数を持たないコンストラクタを呼び出す。一方、次の記述は、オブジェクトの作成としては解釈されない。

```
Vec2D v3();
```

この記述は、「引数を持たず、Vec2D 型の値を返す関数 v3 の宣言」として解釈される。従って、引数を持たないコンストラクタによってオブジェクトを明示的に初期化したい場合には、空の波括弧を用いて

```
Vec2D v3{};
```

と記述するとよい。

以上のような特徴から、新しくオブジェクトを初期化する場合には、波括弧を用いる方法も広く使用されている。特に、意図しない縮小変換を防ぎたい場合には有効である。ただし、クラスによっては、波括弧を使用すると丸括弧の

場合とは異なるコンストラクタが選択されることがある。そのため、呼び出されるコンストラクタを明確に区別する必要がある場合には、丸括弧と波括弧を適切に使い分ける必要がある。

12.4.3 暗黙の型変換を禁止する explicit 宣言

1 個の引数で呼び出すことができるコンストラクタは、その引数の型からクラス型への暗黙の型変換に使用される場合がある。例えば、先の Vec2D クラスのオブジェクトの生成時に次のような記述が可能である。

```
Vec2D v = 3.0;
```

この記述では、double 型の値 3.0 から、Vec2D(double) コンストラクタによって Vec2D 型のオブジェクトが暗黙に生成され、x と y の値がともに 3.0 となる。このような暗黙の型変換を許可したくない場合は、次のように、コンストラクタの宣言に explicit を指定する。

```
explicit Vec2D(double v) : x(v), y(v) {}
```

この場合、冒頭のような暗黙の型変換を伴う形の初期化はできない。一方、コンストラクタを明示的に使用する形であれば、次のように初期化することができる。

```
Vec2D v1(3.0);
```

```
Vec2D v2{3.0};
```

このように explicit を指定すると、コンストラクタが意図しない暗黙の型変換に利用されることを防止できる。特に、1 個の引数で呼び出すことができるコンストラクタでは、その引数の型からクラス型への変換が意図したものであるかを考慮して、必要に応じて explicit を指定する。

12.4.4 コピーコンストラクタ

コンストラクタは、オブジェクトを生成するときに呼び出される特別な関数である。通常は、引数として与えた値を用いてメンバ変数を初期化する。これに対して、既に存在する同じ型のオブジェクトを元にして、新しいオブジェクトを生成する場合に呼び出されるコンストラクタをコピーコンストラクタ (copy constructor) と呼ぶ。

例えば、先の Vec2D クラスがあるときに次のように記述したとする。

```
Vec2D v1(1.0, 2.0);
```

```
Vec2D v2 = v1;
```

この場合、v1 は x に 1.0、y に 2.0 を持つ Vec2D オブジェクトである。次の行では、その v1 をもとにして、新しい Vec2D オブジェクト v2 を生成している。このとき呼び出されるのがコピーコンストラクタである。

コピーコンストラクタは、次のような形で定義できる。

```
Vec2D(const Vec2D& other) : x(other.x), y(other.y) {}
```

ここで、引数 other は、コピー元となる Vec2D オブジェクトである。例えば、「Vec2D v2 = v1;」のように記述した場合、v1 がコピー元となる。const Vec2D& として受け取っているのは、コピー元のオブジェクトを変更せず、かつ不要な複製を避けるためである。この定義では、other.x の値を新しいオブジェクトの x に、other.y の値を新しいオブジェクトの y に設定している。

ただし、この例のように、double 型のメンバ変数だけを持つ単純なクラスでは、コピーコンストラクタを明示的に定義しなくても、C++ が暗黙のコピーコンストラクタを用意する。暗黙のコピーコンストラクタは、各メンバを順にコピーする。そのため、Vec2D のような単純なクラスでは、明示的にコピーコンストラクタを書かなくても、通常は期待通りに複製できる。

一方、ポインタをメンバとして持ち、動的に確保したメモリなどを管理するクラスでは、単なるメンバごとのコピーでは不十分な場合がある。ポインタの値だけがコピーされると、複数のオブジェクトが同じメモリ領域を指してしまうことがあるためである。このような場合には、コピーコンストラクタを明示的に定義し、どのように複製するかをクラスの設計者が決める必要がある。

なお、次の 2 つの記述は、見た目は似ているが意味が異なる。

```
Vec2D v2 = v1;
```

```
v2 = v1;
```

1 つ目は、新しいオブジェクト v2 を v1 から生成している。従って、これは初期化であり、コピーコンストラクタが

関係する。これに対して、2つ目は、既に存在している v2 に v1 の内容を代入している。この場合、新しいオブジェクトを生成しているわけではないため、コピーコンストラクタではなく、**コピー代入演算子**（後述）が関係する。

つまり、コピーコンストラクタは「既存のオブジェクトをもとにして、新しいオブジェクトを作る」ためのコンストラクタである。すでに存在しているオブジェクトに値を代入する処理とは、区別して理解する必要がある。

■ コピー代入演算子

既に存在するオブジェクトに、同じ型の別のオブジェクトの内容を代入する場合には、**コピー代入演算子**（copy assignment operator）が使用される。例えば、

```
Vec2D v1(1.0, 2.0);
Vec2D v2(3.0, 4.0);
v2 = v1;
```

と記述した場合、v2 は既に存在しているため、新しいオブジェクトを生成するコピーコンストラクタではなく、コピー代入演算子による処理となる。コピー代入演算子は、例えば次のように定義できる。

```
Vec2D& operator=(const Vec2D& other) {
    x = other.x;
    y = other.y;
    return *this;
}
```

この関数では、コピー元 other の各メンバの値を、既に存在するオブジェクトのメンバに代入している。戻り値としては、自分自身 *this への参照を返す¹⁴³ のが一般的である。

なお、Vec2D のような単純なクラスでは、コピーコンストラクタの場合と同様に、コピー代入演算子を明示的に定義しなくても、C++によって暗黙のコピー代入演算子が用意される。

コピー代入演算子を含む演算子の独自定義については、後の「12.8 演算子のオーバーロード」（p.248）で詳しく解説する。

12.4.5 単純なバイトコピーが可能な型：trivially copyable

先に、オブジェクトを複製するためのコピーコンストラクタについて解説した。C++では、クラスから生成されたオブジェクトを複製する際に、そのクラスに定められたコピー処理が必要となる場合がある。一方、クラスの構造によっては、オブジェクトを構成するデータを単純なバイト列として複製するだけで、その値を正しく複製できる場合もある。このような性質を持つ型を **trivially copyable**（トリビアルにコピー可能）な型という。ここで、ある型が trivially copyable であるかどうかは、プログラマが任意に決めるものではなく、C++の規格に基づいて定まる型の性質であり、処理系によって判定される。

trivially copyable な型のオブジェクトでは、その内部表現を構成するバイト列を別の同じ型のオブジェクトに複製することで、元のオブジェクトと同じ値を持つオブジェクトを得ることができる。したがって、このような型では memcpy などによる単純なメモリ上の複製を利用することができる。これに対して、trivially copyable でない型では、オブジェクトの内部表現を単純に複製しても、正しいオブジェクトの複製になることは保証されない。

ある型がトリビアルにコピー可能かどうかを調べるには is_trivially_copyable_v を使用する。これは変数テンプレート¹⁴⁴ であり、宣言はヘッダ type_traits にある。また、名前空間 std に属する。

書き方： std::is_trivially_copyable_v<T>

指定した型 T がトリビアルにコピー可能な場合は true、そうでない場合は false となる。この機能によって、コンパイル時に型の性質を判定することができる。

次に示すプログラム test204-6.cpp は、定義された 2 つのクラス Vec2D, Person のオブジェクトが、トリビアルにコピー可能かどうかを調べる例である。

プログラム：test204-6.cpp

```
1 #include <iostream>
2 #include <string>
```

¹⁴³ 「12.6 this ポインタ」（p.246）で解説する。

¹⁴⁴ 後の「16 型を抽象化するための機能」（p.322～）で詳しく解説する。

```

3  #include    <type_traits>
4  using std::cout;
5
6  class Vec2D {
7  private:
8      double x;
9      double y;
10 public:
11     Vec2D(double a, double b) : x(a), y(b) {}
12 };
13
14 class Person {
15 private:
16     std::string name;
17 public:
18     Person(const std::string& s) : name(s) {}
19 };
20
21 int main()
22 {
23     cout << std::boolalpha;
24     cout << "Vec2D: " << std::is_trivially_copyable_v<Vec2D> << '\n';
25     cout << "Person: " << std::is_trivially_copyable_v<Person> << std::endl;
26 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test204-6.cpp のコンパイルと実行

```

$ g++ test204-6.cpp -o test204-6  ←コンパイル
$ ./test204-6  ←実行
Vec2D: true      ←トリビアルにコピー可能
Person: false    ←トリビアルにコピー可能でない

```

`std::string` のメンバを持つ `Person` クラスはトリビアルにコピー可能でないことがわかる。 `std::string` は文字列データを管理するための内部状態を持ち、そのコピーや破棄には単純なバイト列の複写だけでは済まない処理が必要となるためである。

トリビアルにコピー可能であるかどうかは、オブジェクトを `memcpy` などで単純に複製する場合や、オブジェクトの内部表現をバイト列として扱うような低水準の処理を行う場合に重要となる。例えば、バイナリデータの入出力、通信データの作成、低水準のメモリ操作などで、オブジェクトの内容をバイト列として直接扱おうとする場合には注意が必要である。

通常の代入やコピーコンストラクタを用いてオブジェクトを複製する場合には、この性質を特に意識する必要はない。ただし、トリビアルにコピー可能な型であっても、単純なバイトコピーがプログラム上の意味として適切な複製になるとは限らない。例えば、オブジェクトが別のオブジェクトを指すポインタを内部に持つ場合、単純なバイトコピーを行うとポインタの値もそのまま複写されるため、コピー元とコピー先の両方が同じオブジェクトを指す状態になる。そのポインタが動的に確保した資源の所有や管理に用いられている場合には、これが意図したコピーにならないことがあるので注意が必要である。

12.5 デストラクタ

コンストラクタは、オブジェクトの生成時に呼び出され、メンバ変数の初期化などを行う特別な関数である。これに対して、オブジェクトが破棄されるときに呼び出され、後始末を行うための関数を **デストラクタ** と呼ぶ。

デストラクタは、クラス名の前にチルダ「`~`」を付けた名前 で定義され、引数や戻り値を持たない。その基本的な書き方は次のようになる。

書き方： `~クラス名 () { 後始末に関する処理 }`

例えば、`Vec2D` クラスにデストラクタを定義すると、次のようになる。

■ デストラクタ実装の例

```
#include <iostream>
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D(double a, double b) : x(a), y(b) {}
    ~Vec2D() {
        std::cout << "Vec2D のデストラクタ\n";
    }
};
```

このように定義しておくと、Vec2D 型のオブジェクトが破棄される際に、デストラクタが自動的に呼び出される。例えば、次のように関数内でオブジェクトを定義した場合を考える。

```
void f() {
    Vec2D v(3.0, 4.0);
    (vを用いた処理)
}
```

この関数 f の実行が終了して、そのスコープを抜けると同時にオブジェクト v は破棄され、Vec2D のデストラクタが呼び出される。

このように、関数内で定義されたオブジェクトは、通常、その有効範囲（スコープ）を抜けると自動的に破棄される。このときデストラクタが自動的に呼び出されるため、オブジェクトの後始末を行うことができる。

なお、デストラクタは必ずしも自分で定義する必要はない。特別な後始末が不要な場合には、コンパイラによって自動的にデフォルトのデストラクタが生成されるため、明示的に記述しなくても問題はない。

一方で、オブジェクト自身が動的に確保したメモリなどの資源を直接管理している場合には、必要に応じてデストラクタを定義し、それらを適切に解放することが重要となる。

コンストラクタが「生成と初期化」を担うのに対し、デストラクタは「破棄と後始末」を担う。これらを適切に定義することで、オブジェクトのライフサイクルを安全に管理することが可能となる。

12.6 this ポインタ

メンバ関数の内部では、通常、メンバ変数や他のメンバ関数を直接用いることができる。しかし、場合によっては、現在処理の対象となっているオブジェクト自身を明示的に区別して参照する必要があることがある。

例えば、クラス Vec2D に次のようなメンバ関数を追加することを考える。

■ メンバ関数の内部で他のメンバ関数を利用する例

```
void show() {
    std::cout << "(" << x << "," << y << ") : "
               << this->norm() << "\n";
}
```

この show() は、オブジェクトの内部状態とその長さを表示する関数である。ここで、this->norm() と記述している点に注目する。

show() の内部で norm() を呼び出すと、呼び出し対象となっているオブジェクト自身に対して norm() が実行される。メンバ関数は特定のオブジェクトに対して呼び出されるため、その内部では、呼び出し対象となっているオブジェクト自身のメンバを操作している。

このとき、メンバ関数の内部では、そのオブジェクト自身を指す特別なポインタが自動的に用意されている。これを **this ポインタ** と呼ぶ。this は、現在処理の対象となっているオブジェクトを指しており、this->norm() と記述することで、そのオブジェクトに対して norm() を呼び出していることを明示することができる。

this はポインタであるため、メンバへのアクセスには this->x のようにアロー演算子「->」を使用する。

注意) `this` は、非静的なメンバ関数の内部で利用できる特別な名前である。

なお、多くの場合、`this->` は省略することができ、単に `norm()` と記述しても同様に動作する。しかし、`this->` を明示することで、どのオブジェクトのメンバを用いているのかを意識しやすくなる。

更に、他のオブジェクトと組み合わせた処理を考えると、`this` の役割はより明確になる。例えば、次のようなメンバ関数を考える。

■ 他のオブジェクトの値を加える例

```
void add(const Vec2D& other) {
    this->x += other.x;
    this->y += other.y;
}
```

この `add()` は、現在のオブジェクトの値に、引数 `other` として与えられた別のオブジェクトの値を加える関数である。ここで、`other.x` や `other.y` は別のオブジェクトのメンバ変数を表しているのに対し、`this->x` や `this->y` は、現在処理の対象となっているオブジェクト自身のメンバ変数を表している。

すなわち、`add()` は「自分自身の値に、他のオブジェクトの値を加える」処理であり、このような場合に `this` を用いることで、「どのオブジェクトのメンバを操作しているか」を明確に表すことができる。

重要) 上の例において、`x`、`y` は `private` メンバであるが、クラスのメンバ関数からは、同じクラスの別のオブジェクトが持つ `private` メンバにもアクセスすることができる。従って、この例では `other.x` や `other.y` を参照できる。

12.7 const メンバ関数と mutable 指定

C++では、オブジェクトの状態を変更しないメンバ関数であることを明示するために、メンバ関数に `const` を指定することができる。このようなメンバ関数を **const メンバ関数** という。const メンバ関数では、mutable 指定されたものを除き、原則としてオブジェクトのデータメンバを変更することができない。また、`const` を指定して宣言されたオブジェクトから実行できるメンバ関数も、基本的には const メンバ関数に限られる。このため、オブジェクトの状態を変更する必要のないメンバ関数には `const` を指定することが、安全なプログラムを作成するための一つの基本的な姿勢となる。

一方、オブジェクトの論理的な状態¹⁴⁵ は変更しない処理であっても、処理の実行回数や内部的な管理情報など、一部のデータメンバについては値を変更する必要がある場合がある。このような場合には、データメンバに `mutable` を指定することで、const メンバ関数からでも、そのデータメンバの値を変更できるようにすることができる。

■ メンバ関数の const 指定

const メンバ関数を定義するには、引数リストの後に `const` を記述する。

書き方： 戻り値の型 メンバ関数名 (引数リスト) `const` { 処理 }

例えば、`Vec2D` クラスに、ベクトルのノルムを求める `norm` メンバ関数を定義する場合は、次のように記述できる。

```
double norm() const {
    return std::sqrt(x*x + y*y);    // 注) cmath ヘッダが必要
}
```

この `norm` メンバ関数は、`x` や `y` の値を参照するだけで変更しないため、const メンバ関数として定義することができる。const 指定されたメンバ関数内で、特定のメンバ変数に限って値の変更を許可する場合は、対象となるメンバ変数を宣言時に `mutable` 指定する。

■ メンバ変数の mutable 指定

mutable 指定されたメンバ変数は、const メンバ関数内でも値を変更することができる。ただし、static なデータメンバは mutable 指定できないことに注意すること。

書き方： mutable 型名 変数名 1, 変数名 2, … ;

次に示すクラス定義の例は、const、mutable 指定を用いたクラス定義の例である。メンバ関数 `norm` は const 指定されており、通常のメンバ変数 `x`、`y` の値を変更することはできないが、`count` は mutable 指定されているので、`norm`

¹⁴⁵例えば、オブジェクトを外部から見たときの値や性質を決定する状態。

関数内で値を変更することができる。

[const 指定, mutable 指定を用いたクラスの定義例]

```
#include <cmath>

class Vec2D {
private:
    double x;
    double y;
    mutable int count = 0;
public:
    Vec2D(double a, double b) : x{a}, y{b} {}

    double norm() const {
        count++;
        return std::sqrt(x*x + y*y);
    }
};
```

この例では、norm 関数が実行された回数をカウントする変数 count が宣言されており、norm 関数内でこの値が変更（インクリメント）可能である。

また、const 指定されたオブジェクトからも、const メンバ関数を実行することができる。例えば、次の記述が可能である。

```
const Vec2D v{3.0, 4.0};
double r = v.norm();
```

この場合も、オブジェクト v が内部に持つ count の値は変更される。

■ mutable 指定を使用する場合の考え方

mutable は、const メンバ関数から特定のデータメンバの値を変更する必要がある場合に使用する。ただし、オブジェクトの論理的な状態を決定するデータメンバについては、原則として mutable を指定しない。このようなデータメンバの値を変更すると、const メンバ関数であるにもかかわらず、オブジェクトの意味や性質そのものが変化することになるためである。

一方、処理の実行回数、計算結果を一時的に保存するためのキャッシュ、内部的な管理情報など、オブジェクトの論理的な状態には影響しない補助的な情報については、必要に応じて mutable を指定することができる。このような情報は内部的には変更されても、オブジェクトが外部に示す値や性質が変化しないため、const メンバ関数から変更することが設計上適切な場合がある。

ただし、mutable を多用すると、const によってオブジェクトの状態を変更しないことを示す意味が不明確になる。従って、mutable は、論理的な状態には影響しない内部情報を変更する必要がある場合に限定して使用することが望ましい。

12.8 演算子のオーバーロード

C++ では、クラスのオブジェクトに対して加算や減算などの演算を、通常の算術式と同じように記述することができる。この仕組みを**演算子のオーバーロード**と呼ぶ。例えば、2次元ベクトルを表すクラス Vec2D に対して、次のように加減算を記述できると便利である。

```
Vec2D a(3.0, 4.0), b(5.0, 12.0);
Vec2D c = a + b;
Vec2D d = a - b;
```

このような記述を可能にするために、C++ では、operator に続けて演算子を記述した特別な関数を定義する。例えば、operator+ は加算演算子「+」に対応する関数を表す。

例えば、operator+ をメンバ関数として定義した場合、式 a + b は、a.operator+(b) のような関数呼び出しに対応するものと考えることができる。このように、演算子オーバーロードは、オブジェクトに対する操作を自然な記法で記述するための仕組みである。

この形式でオーバーロード可能な主な演算子を表 42 に示す。

表 42: オーバーロード可能な演算子 (一部)

演 算 子	説 明
+, -, *, /, %	算術演算子
==, !=, <, >, <=, >=	比較演算子
=	代入演算子
+=, -=, *=, /=	複合代入演算子
++, --	インクリメント／デクリメント
[]	添字演算子
()	関数呼び出し演算子
->	メンバアクセス演算子
<<, >>	シフト演算子 (ストリーム入出力にも使用)

なお、すべての演算子がオーバーロード可能であるわけではない。例えば「.」や「::」などはオーバーロードすることができない。

■ Vec2D の加算・減算演算子の定義の例 (該当部分のみ)

```
public:
    Vec2D operator+(const Vec2D& other) const {
        return Vec2D(x + other.x, y + other.y);
    }

    Vec2D operator-(const Vec2D& other) const {
        return Vec2D(x - other.x, y - other.y);
    }
```

この関数宣言では、引数リスト (...) と定義内容のブロック { ... } の間に `const` が記述されている。これは、この関数がオブジェクトの状態を変更しないことを示している。より厳密には、このメンバ関数が扱うオブジェクト (`this` が指すもの) を変更しないことを宣言している。このような関数は、オブジェクトの値を読み取るだけの処理に用いられる。

参考) このコードでは、`Vec2D` のオブジェクトを生成して戻り値として返しているように見える。C++17 以降では、このように戻り値の型と同じ型のオブジェクトを直接生成して返す場合、不要なコピーを作らずに戻り値のオブジェクトが直接構築される¹⁴⁶。

上のサンプルコードの `operator+` 関数を次のようにして、関数内で一旦変数にしてから値を返す形で実装することもできる。

■ 戻り値の生成方法の違い (先の例の変種)

```
// 一旦ローカル変数に代入してから返す
Vec2D operator+(const Vec2D& other) const {
    Vec2D tmp(x + other.x, y + other.y);
    return tmp;
}
```

この場合も、多くの処理系では **NRVO** (Named Return Value Optimization) によってコピーが省略される。ただし、先のように戻り値のオブジェクトを直接生成する場合とは異なり、このコピー省略は必ず行われるとは限らない。

12.8.1 複合代入演算子のオーバーロード

先に、`+` や `-` などの演算子をオーバーロードすることで、新しいオブジェクトを生成する演算を定義した。例えば、式「`a + b`」は、`a` と `b` の内容から新しいオブジェクトを生成して返すものである。

これに対して、`+=` のような複合代入演算子は、左辺のオブジェクト自身を直接更新する演算である。例えば、式 `a += b` は、`a` の内容を変更し、その結果を `a` に反映させる。このような複合代入演算子は、次のように定義すること

¹⁴⁶このような仕組みは一般に **RVO** (Return Value Optimization) と呼ばれる。C++17 では、この形のコピー省略は規格上保証される。

ができる。

■ 複合代入演算子 += の定義例

```
Vec2D& operator+=(const Vec2D& other) {  
    x += other.x;  
    y += other.y;  
    return *this;  
}
```

この関数では、左辺のオブジェクト自身のメンバ変数を書き換える。従って、このメンバ関数を `const` メンバ関数として定義することはできない。

また、戻り値として `*this` を返している点も重要である。これは、自分自身への参照を返していることを意味し、 $(a += b) += c$ のように、複数の演算を連続して記述することが可能となる。

このように、複合代入演算子は、新しいオブジェクトを生成して返す `+` などの演算子とは異なり、左辺のオブジェクト自身を更新する演算である。また演算子の設計においては、新しいオブジェクトを生成するのか、それとも既存のオブジェクトを更新するのかを意識することが重要である。

12.8.2 異なる型の値との演算とその実装

これまでと同様の考え方で、ベクトルとスカラーの積を考える。例えば、`Vec2D` に対して、次のようにスカラー倍を記述できると便利である。

```
Vec2D v(3.0, 4.0);  
Vec2D w = v * 2.0;
```

このような演算を実現するために、乗算演算子 `*` を次のようにオーバーロードすることができる。

■ スカラー倍を行うメンバ関数

```
Vec2D operator*(double s) const {  
    return Vec2D(x * s, y * s);  
}
```

この関数により、式「`v * 2.0`」は「`v.operator*(2.0)`」のように解釈され、ベクトルの各成分にスカラー値を掛けた結果が得られる。しかし、ここで次のような記述を考える。

```
Vec2D w = 2.0 * v;
```

この式は直感的には自然であるが、上の定義のままでは正しく動作しない。これは、左辺の `2.0` は `double` 型であり、`operator*` が `Vec2D` のメンバ関数として定義されているため、`(2.0).operator*(v)` のような呼び出しができないためである。

このような順序の演算を有効にするには、`Vec2D` のメンバ関数ではなく、クラスの外部に非メンバ関数として `operator*` を定義することができる。

■ 非メンバ関数によるスカラー倍の定義

```
class Vec2D {  
private:  
    double x;  
    double y;  
public:  
    Vec2D(double a, double b) : x(a), y(b) {}  
    Vec2D operator*(double s) const {  
        return Vec2D(x * s, y * s);  
    }  
};  
  
Vec2D operator*(double s, const Vec2D& v) {  
    return v * s;  
}
```

重要) メンバ関数として定義された演算子では、左辺がそのクラスのオブジェクトとなる。従って、上の $2.0 * v$ のように左辺が `Vec2D` ではない演算を定義する場合には、非メンバ関数として演算子を定義する。

12.8.2.1 friend 関数

通常、クラスの `private` メンバには、クラスの外部に定義された関数から直接アクセスすることはできない。しかし、特定の非メンバ関数に対して、そのクラスの `private` メンバへのアクセスを許可したい場合がある。このために使用されるのが `friend` 宣言である。これに関して、例を挙げて解説する。

■ friend 関数の例

```
class Vec2D {
private:
    double x;
    double y;

public:
    Vec2D(double a, double b) : x(a), y(b) {}
};

friend double dot(const Vec2D& a, const Vec2D& b);

double dot(const Vec2D& a, const Vec2D& b) {
    return a.x * b.x + a.y * b.y;
}
```

この `dot()` は、2つの `Vec2D` オブジェクトを受け取り、それらが表す2次元ベクトルの内積（要素同士の積の総和）を求める関数である。`dot()` は `Vec2D` のメンバ関数ではないが、クラス内で `friend` として宣言されているため、`private` メンバ `x`, `y` に直接アクセスできる。

この関数は、通常非メンバ関数として次のように呼び出すことができる。

例. 関数 `dot` の呼び出し

```
Vec2D a(1.0, 2.0);
Vec2D b(3.0, 4.0);
double d = dot(a, b);
```

この場合、`d` には $1.0 \times 3.0 + 2.0 \times 4.0 = 11.0$ が得られる。

重要) `friend` として宣言された関数は、そのクラスのメンバ関数になるわけではない。あくまで、クラスの `private` メンバなどへのアクセスが許可された非メンバ関数である。

先の $2.0 * v$ の実装¹⁴⁷ でも、非メンバの `operator*` から `private` メンバへ直接アクセスしたい場合には、その `operator*` を `friend` として宣言することができる。ただし、公開されている機能だけを用いて実装できる場合には、必ずしも `friend` を使う必要はない。

12.8.3 関数のように呼び出せるオブジェクト：operator()

C++では、`+` や `+=` などの演算子をオーバーロードすることで、独自に定義したクラスのオブジェクトに対して、数値型の値と同じような記法で演算を行うことができる。同様に、関数呼び出しの形で使用する丸括弧 `()` も演算子として扱われる。すなわち、

オブジェクト (引数)

という記法に対する動作をクラスの中で定義することができる。このために使用するのが `operator()` である。この記述は、見かけ上は関数呼び出しのように見えるが、実際には、オブジェクトに対するメンバ関数呼び出しである。すなわち、概念的には次のような呼び出しと考えることができる。

オブジェクト.operator()(引数)

このように、`operator()` は、オブジェクトを関数のように呼び出せるようにするための演算子オーバーロードである。この機能によって、単なるデータのまとまりとしてのオブジェクトだけでなく、内部状態に基づいて処理を行うオブジェクトを、関数のような形で扱うことができる。

¹⁴⁷ 「非メンバ関数によるスカラー倍の定義」(p.250) の例。

`operator()` を持ち、関数のように呼び出せるオブジェクトを**関数オブジェクト**または**ファンクタ**と呼ぶ。関数オブジェクトは、通常関数と同じような形式で呼び出せる一方で、オブジェクトであるため、内部にデータを保持することができる。この点が、通常関数との大きな違いである。

この仕組みは、後に説明するラムダ式や STL アルゴリズムとも深く関係する。特にラムダ式は、内部的には、`operator()` を持つ関数オブジェクトに近いものとして理解することができる。

`operator()` は、クラスのメンバ関数として定義する。記述形式は次の通り。

戻り値の型 `operator()` (仮引数リスト) { 処理内容 }

ここで、`operator()` が関数呼び出し演算子を表し、その後ろの丸括弧の中に受け取る仮引数を記述する。

整数値を 1 つ受け取り、その 2 乗を返す関数オブジェクトを作るサンプルプログラム `test204-3.cpp` を次に示す。

プログラム：test204-3.cpp

```
1  #include    <iostream>
2  using std::cout;
3
4  class Square {
5  public:
6      int operator()(int x) {
7          return x * x;
8      }
9  };
10
11 int main()
12 {
13     Square sq;
14
15     cout << "sq(5) : " << sq(5) << "\n";    // ← ↓ 異なる呼び出し方
16     cout << "sq.operator()(5) : " << sq.operator()(5) << "\n";
17 }
```

このプログラムでは、クラス `Square` の中で、整数値を 1 つ受け取って整数値を返す `operator()` を定義している。そのため、`Square` 型のオブジェクト `sq` は、`sq(5)` のように関数と同じ形式で呼び出すことができる。またこの記述は、実際には `sq` のメンバ関数 `operator()` を呼び出している。従って、`sq.operator()(5)` という記述と同じである。

このプログラムをコンパイルして実行する例を次に示す。

例. `test204-3.cpp` のコンパイルと実行

```
$ g++ test204-3.cpp -o test204-3  [Enter]    ←コンパイル
$ ./test204-3  [Enter]    ←実行
sq(5) : 25    ←出力
sq.operator()(5) : 25
```

この例の `Square` は内部にデータを持たない単純な関数オブジェクトであるが、メンバ変数を持たせることで、内部状態に応じて動作する関数オブジェクトを作ることにもできる。次に示すプログラム `test204-5.cpp` は、内部状態を持つ関数オブジェクトを実装する例である。

プログラム：test204-5.cpp

```
1  #include    <iostream>
2  using std::cout, std::endl;
3
4  class Counter {
5  private:
6      int count = 0;
7
8  public:
9      int operator()() {
10         return ++count;
11     }
12 };
13
14 int main() {
```

```

15     Counter c;
16
17     cout << "c() : " << c() << "\n";
18     cout << "c() : " << c() << "\n";
19     cout << "c() : " << c() << endl;
20 }

```

このプログラムでは、クラス Counter のメンバ変数 count が、関数オブジェクトの内部状態を表している。オブジェクト c に対して c() を実行すると operator() が呼び出され、count の値が 1 増加した後、その値が戻り値として返される。count の値はオブジェクト c の内部に保持されるため、次に c() を実行した際にも、前回の値を引き継いで処理が行われる。

このプログラムをコンパイルして実行する例を次に示す。

例. test204-5.cpp のコンパイルと実行

```

$ g++ test204-5.cpp -o test204-5  ←コンパイル
$ ./test204-5  ←実行
c() : 1      ←内部状態 count を持ち、
c() : 2      ←呼び出しの度に
c() : 3      ←戻り値が変わる

```

このように、関数オブジェクトは通常の関数と同じような形式で呼び出すことができるだけでなく、オブジェクトとして内部状態を保持し、その状態に応じた処理を行うことができる。

12.9 カプセル化

これまでに、クラスを用いてデータとそれに対する操作をまとめて扱う方法や、public と private によるアクセス制御について解説してきた。これらの仕組みは、単にプログラムの構造を整理するためだけでなく、オブジェクトの状態を適切に保つために重要な役割を持っている。

このように、データ（メンバ変数）とそれを操作する関数（メンバ関数）を一体として扱い、外部からのアクセスを制御する考え方を**カプセル化**と呼ぶ。

カプセル化の主な目的は、オブジェクトの内部状態へのアクセスを適切に制御して、正しい状態を保つとともに、内部の実装と外部に公開するインターフェースを分離することである。そのため、通常はメンバ変数を private として隠蔽し、必要な操作のみを public なメンバ関数として提供する。

このような設計を行うことで、外部からの不適切な操作や意図しない状態の変化を防ぐことができる。また、クラスの内部の実装を変更した場合でも、公開されたインターフェースが変わらない限り、そのクラスを利用する側のプログラムに影響を与えにくくなる。

以下では、カプセル化の考え方をより具体的に理解するために、オブジェクトの状態をどのように保つかという観点と、外部とのインターフェースの設計という観点から説明する。

12.9.1 不変条件（インバリアント）の維持

オブジェクトが常に正しい状態を保つためには、その状態が満たすべき条件を明確にしておく必要がある。このような条件を、**不変条件（インバリアント）**と呼ぶ。不変条件そのものは、C++ の言語機能によって自動的に保証されるものではなく、クラス設計において開発者が目的に応じて設定する条件である。また、カプセル化に沿ったプログラムの設計は、不変条件を保つための有効な手段となる。

プログラムの設計において、不変条件を維持する例を test205-1.cpp に示す。

プログラム：test205-1.cpp

```

1  #include <iostream>
2  using std::cout, std::cin;
3
4  class AudioPanel {
5  private:
6      int volume;    // 0 <= volume <= 100
7  public:

```



```

8      AudioPanel() : volume(50) {}      // コンストラクタ
9
10     // ボリュームを設定するメンバ関数
11     void setVolume(int v) {
12         if (v < 0) {                    // 負の値が来たら
13             volume = 0;                 // 最小値0にする
14         } else if (v > 100) {            // 大きすぎる値が来たら
15             volume = 100;               // 最大値100にする
16         } else {                        // 正常範囲の値は
17             volume = v;                 // そのまま代入
18         }
19     }
20     // 現在のボリュームを取得するメンバ関数
21     int getVolume() const {
22         return volume;
23     }
24 };
25
26 int main() {
27     bool     exe = true;
28     int      v;
29     AudioPanel ap;
30
31     while ( exe ) {
32         cout << "current:" << ap.getVolume() << "\n";
33         cout << "input volume> ";
34         cin >> v;
35         exe = (v < 999);    // 反復処理の条件を更新
36         if ( exe ) {
37             ap.setVolume(v);
38         } else {
39             cout << "shutting down...\n";
40         }
41     }
42 }

```

このプログラムは、仮想的なオーディオパネル（モック）を AudioPanel というクラスとして実現する例で、ボリュームの値の設定（setVolume 関数）と取得（getVolume 関数）の機能を持つ。また、volume は private メンバであるため、クラスの外部から直接値を書き換えることはできない。外部からの値の設定を setVolume() に限定することで、音量を 0 以上 100 以下に保つことができる。これは、オブジェクトの不変条件（インバリアント）を保つ典型的な例である。

このプログラムをコンパイルして実行する例を次に示す。

例. test205-1.cpp のコンパイルと実行

```

$ g++ test205-1.cpp -o test205-1  [Enter]    ←コンパイル
$ ./test205-1  [Enter]    ←実行
current:50                                ←ボリュームの初期値
input volume> 70  [Enter]    ←正常値の入力
current:70
input volume> -30  [Enter]    ←異常値（負の値）の入力
current:0          ←強制的に 0（最小値）に設定
input volume> 250  [Enter]    ←異常値（100 より大きい）の入力
current:100        ←強制的に 100（最大値）に設定
input volume> 999  [Enter]    ←終了の入力
shutting down...    ←プログラム終了

```

12.9.2 インターフェースと実装の分離

クラスを用いたプログラム設計では、オブジェクトの「使い方」と、その内部でどのように処理が行われているかという「実装」を分離して考えることが重要である。

クラスの外部から利用できる関数や操作の集合と、それらの利用方法をインターフェースと呼び、それらが内部でどのように実現されているかを実装と呼ぶ。先に示した AudioPanel クラスでは、setVolume や getVolume といった関数がインターフェースとして提供されている。一方で、音量をどのように制限しているかという処理はクラスの

内部に実装されており、利用側ではその詳細を意識する必要がない。

このように、インターフェースと実装を分離することで、利用者は内部の処理を意識することなく、提供された機能を安全に利用することができる。また、内部の実装を変更した場合でも、インターフェースが変わらない限り、そのクラスを利用する側のプログラムに影響を与えにくくなる。

このように外部から利用される機能の集合は、一般に **API** (Application Programming Interface) と呼ばれる。本章で扱っているインターフェースは、その基本的な考え方に対応している。

12.10 メンバの静的宣言

これまでに扱ってきたメンバ変数やメンバ関数は、基本的には個々のオブジェクトに属するものであった。例えば、Vec2D クラスに x, y というメンバ変数がある場合、それらの値は各 Vec2D オブジェクトごとに個別に保持される。

すなわち、v1 と v2 という 2 つの Vec2D オブジェクトがある場合、v1 の x, y と、v2 の x, y は別々の記憶領域に保持される。これにより、各オブジェクトは互いに独立した状態を持つことができる。これは、先に述べた「オブジェクトの独立性」の基本である。

一方、クラスに関係する情報の中には、個々のオブジェクトごとに持たせる必要がないものもある。例えば、Vec2D が 2 次元ベクトルを表すクラスであるならば、その次元数が 2 であることは、どの Vec2D オブジェクトについても共通である。このような情報を各オブジェクトに個別に持たせるのは不自然であり、無駄でもある。

C++ では、このような場合に、メンバを **static** 宣言することができる。static 宣言されたメンバは、個々のオブジェクトではなく、クラスそのものに属するメンバとして扱われる。このようなメンバを **static メンバ**、または**静的メンバ**と呼ぶ。

通常のメンバ変数は、オブジェクトごとに存在する。これに対して、static メンバ変数は、そのクラスの全オブジェクトで共有される変数として扱われる。また、通常のメンバ関数は、特定のオブジェクトに対して呼び出され、その内部では、呼び出し対象のオブジェクトを指す this ポインタを使用できる。これに対して static メンバ関数は、特定のオブジェクトに対して呼び出される関数ではないため、this ポインタを持たない。

なお、ここでいうメンバの static 宣言は、C 言語で扱った記憶クラス指定子としての static とは意味合いが異なる。C 言語における static には、ローカル変数の記憶期間を静的にする、あるいはファイル外部からその名前を参照できないようにする、といった働きがあった。これに対して、クラスのメンバに付ける static は、そのメンバが個々のオブジェクトではなく、クラスそのものに属することを表す指定である。

static 宣言された public メンバにクラスの外部からアクセスするには

クラス名::メンバ

の形式で記述する。

C++ のクラスにおける static メンバは、「オブジェクトに属するメンバ」ではなく、「クラスに属するメンバ」を定義するための仕組みである。以下では、Vec2D クラスを例に、static メンバ変数と static メンバ関数に関して例を挙げて解説する。

次に示すプログラム test204-2.cpp では、Vec2D クラスを定義しているが、このクラスは静的メンバ (count 変数、dimension 定数、getCount 関数) を持ち、Vec2D オブジェクトの個数と、Vec2D が表すベクトルの次元を管理する。また、count 変数は private であり、getCount 関数でその値を参照する。

プログラム：test204-2.cpp

```
1 #include <iostream>
2 using std::cout;
3
4 class Vec2D {
5     private:
6         double x;
7         double y;
8         // 静的メンバ (クラスに属する) : Vec2D オブジェクトの個数を管理する
9         static int count;
10
11     public:
12         // 2次元ベクトルであることを表す「クラスの定数」
```

```

13     static constexpr int dimension = 2;
14     // コンストラクタ
15     Vec2D(double x0, double y0) : x(x0), y(y0) {           // 新規作成時
16         count++;
17     }
18     Vec2D(const Vec2D& other) : x(other.x), y(other.y) {     // コピー時
19         count++;
20     }
21     // デストラクタ
22     ~Vec2D() {
23         count--;
24     }
25     // クラスに属するメンバ関数：オブジェクトの個数を返す
26     static int getCount() {
27         return count;
28     }
29     // オブジェクトに属する関数：オブジェクトの内容を出力する
30     void print() const {
31         cout << "(" << x << ", " << y << ")\n";
32     }
33 };
34
35 // staticメンバ変数の実体を定義する（初期化）
36 int Vec2D::count = 0;
37
38 int main()
39 {
40     cout << "Vec2D::dimension = " << Vec2D::dimension << "\n";
41     cout << "Vec2D::getCount() = " << Vec2D::getCount() << "\n";
42     cout << "-----\n";
43
44     Vec2D v1(1.0, 2.0);
45     cout << "after creating v1:\tcount = " << Vec2D::getCount() << "\n";
46
47     Vec2D v2 = v1;           // コピーコンストラクタが呼び出される
48     cout << "after copying v1 to v2: count = " << Vec2D::getCount() << "\n";
49
50     Vec2D* p = new Vec2D(3.0, 4.0);
51     cout << "after new Vec2D:\tcount = " << Vec2D::getCount() << "\n";
52
53     delete p;               // デストラクタが呼び出される
54     cout << "after delete:\t\tcount = " << Vec2D::getCount() << "\n";
55     cout << "-----\n";
56
57     cout << "v1 = ";        v1.print();
58     cout << "v2 = ";        v2.print();
59     cout << "-----\n";
60
61     cout << "v1.dimension = " << v1.dimension << "\n";           // この使用方法も
62     cout << "v2.getCount() = " << v2.getCount() << "\n";         // 可能
63 }

```

このプログラムでは、Vec2D オブジェクトを新規作成するコンストラクタと、既存のオブジェクトをもとに新しいオブジェクトを作るコピーコンストラクタで、count 変数をインクリメントしている。また Vec2D クラスの次元は、public の dimension 定数で表している。これは constexpr 宣言¹⁴⁸されたクラスに属する定数であり、実行中に変更することはできない。

オブジェクトの個数を管理する count 変数は、クラス定義直後の「int Vec2D::count = 0;」によって実体が定義され、初期値が 0 に設定されている。count 変数は private であり、クラスの外部から Vec2D::count のように直接アクセスすることはできない。ただし、この記述は count に外部からアクセスしているのではなく、static メンバ変数の実体をクラス外で定義しているものである。

static 宣言された public のメンバには、クラスの各オブジェクトを介してアクセスすることも可能であり、プログラムの末尾にある「v1.dimension」、「v2.getCount()」という記述はその例である。ただし、通常のメンバと静的メンバの混同を避けるためにも、この書き方はあまり推奨されない。

¹⁴⁸ 「11.11 コンパイル時定数：constexpr」(p.219) 参照。

このプログラムをコンパイルして実行する例を次に示す。

例. test204-2.cpp のコンパイルと実行

```
$ g++ test204-2.cpp -o test204-2 Enter    ←コンパイル
$ ./test204-2 Enter    ←実行
Vec2D::dimension  = 2                ← static メンバに
Vec2D::getCount() = 0                アクセスしている
-----
after creating v1:      count = 1    ←オブジェクト v1 を作成
after copying v1 to v2: count = 2    ←オブジェクト v1 の複製 v2 を作成
after new Vec2D:        count = 3    ←更にもう 1 つを作成
after delete:           count = 2    ←それをすぐに削除
-----
v1 = (1, 2)              ←オブジェクトの
v2 = (1, 2)              ←内容確認
-----
v1.dimension  = 2        ←この形式でも static メンバに
v2.getCount() = 2        ←アクセスできるが非推奨
```

12.11 friend によるクラス間の特別なアクセス

先にアクセス指定について解説したように、クラスの `private` メンバは、通常、そのクラスの外部から直接アクセスすることはできない。C++には、このような非公開のメンバへのアクセスを、特定のクラスに対して例外的に許可する `friend` という機能がある。これにより、互いに密接に関連するクラスの間で、一方のクラスの内部情報を他方のクラスから直接利用できるようにすることができる。

あるクラスに対して、別のクラスから `private` メンバへのアクセスを許可するには、クラス定義の内部に次の形式の宣言を記述する。

```
friend class クラス名;
```

この宣言によって指定されたクラスを `friend` クラス と呼ぶ。 `friend` クラスのメンバ関数からは、この宣言を記述したクラスの `private` メンバや `protected` メンバに直接アクセスすることができる。

`friend` 宣言は `class` 定義内の任意の位置に記述することができ、当該クラスのアクセス指定子の影響を受けない。

`friend` クラスを利用する例を、次のプログラム test204-4.cpp に示す。

プログラム：test204-4.cpp

```
1  #include    <iostream>
2
3  class Person {
4      private:
5          double weight;
6          double height;
7
8          friend class Doctor;    // privateを開示する相手クラス
9
10     public:
11         Person(double w, double h) : weight(w), height(h) { }
12 };
13
14 class Doctor {    // Personのprivateメンバを参照できるクラス
15     public:
16         double bmi(const Person& p) {
17             return p.weight / (p.height * p.height);
18         }
19 };
20
21 int main() {
22     Person p(59.4, 1.647);
23     Doctor d;
24
25     std::cout << d.bmi(p) << std::endl;
26 }
```

このプログラムでは、人物を表す `Person` クラスと、その人物の BMI ¹⁴⁹ を算出する `Doctor` クラスを定義している。`Person` クラスでは、`weight` と `height` を `private` メンバとして定義している。一方、クラス定義の中に

```
friend class Doctor;
```

と記述することで、`Doctor` クラスに対して、これらのメンバへの直接アクセスを特別に許可している。そのため、`Doctor` クラスの `bmi` メンバ関数では、引数として受け取った `Person` オブジェクト `p` に対して、

```
p.weight      p.height
```

のように `private` メンバを直接参照することができる。`bmi` メンバ関数では、これらの値から BMI を算出して、その結果を戻り値として返している。

このプログラムをコンパイルして実行する例を次に示す。

例. `test204-4.cpp` のコンパイルと実行

```
$ g++ test204-4.cpp -o test204-4  ←コンパイル
$ ./test204-4  ←実行
21.8977      ←出力
```

▲注意▲

`friend` によるアクセス許可は一方向である。例えば、`Person` クラスが `Doctor` クラスを `friend` として指定しても、`Person` から `Doctor` クラスの `private` メンバへアクセスできるようになるわけではない。

`friend` の関係は継承¹⁵⁰ されない。例えば、`Doctor` の派生クラスが自動的に `Person` の `private` メンバへアクセスできるようになるわけではない。

¹⁴⁹ ボディマス指数 (body mass index)

¹⁵⁰ 後の「13 継承とポリモーフィズム」(p.259～) で解説する。

13 継承とポリモーフィズム

これまでに、クラスを用いることで、データとそれに対する操作をひとまとまりにした構造を定義できることを見てきた。また、カプセル化により、オブジェクトの内部実装を隠蔽し、外部からはインターフェースを通じて利用するという設計が可能になることも示した。

しかし、実際のプログラムでは、単一のクラスだけで完結することはまれであり、複数のクラスが相互に関係を持ちながら全体を構成する。その際に問題となるのが、「似た性質を持つ複数の型を、どのように整理し、どのように扱うか」という点である。

例えば、図形を扱うプログラムを考える。円や長方形といった図形は、それぞれ異なる形状を持つが、「面積を求める」「画面に描画する」といった共通の操作を持っている。このような場合、それぞれの型に共通する性質や操作を個別に記述していくと、似た構造や処理が複数箇所に分散し、仕様の変更に対して修正箇所が増えてゆく。結果として、プログラムの構造が冗長になり、保守性の低下につながる。

また、これらの図形をひとまとめにして扱いたい場面も多い。例えば、複数の図形を格納した配列やコンテナに対して、一括して描画処理を行うような場合である。しかし、円と長方形は異なる型であるため、そのままでは同一の型の配列として扱うことはできない。すなわち、型が異なるという理由だけで、本来まとめて扱いたい対象を統一的に処理できないという問題が生じる。

このように、「共通のインターフェースを持つが、具体的な振る舞いは異なる複数の型」を扱うことは、プログラム設計において重要な課題となる。

C++では、この課題に対して、主に次の二つの仕組みが用意されている。

- ・ クラス同士の関係を定義し、ある型をより一般的な型の一種として扱う仕組み（**継承**）
- ・ 共通のインターフェースを通じて、実体に応じた処理を実行する仕組み（**ポリモーフィズム**）

継承を用いると、複数の型の間に「一般的な型」と「より具体的な型」という関係を定義することができる。これにより、共通の性質を基底となるクラスにまとめ、個別の振る舞いを派生クラスに分けて記述することが可能となる。

一方、ポリモーフィズムを用いると、これらの型を共通の型として扱いながら、実際に呼び出される処理は、同一の操作を記述した場合であっても、各オブジェクトの実体に応じて切り替えることができる。すなわち、呼び出し側は具体的な型の違いを意識することなく、統一的な操作を記述できる。

ただし、C++においては、このような動的な振る舞いは自動的に導入されるものではない。どの操作を共通化し、どの部分で実行時の切り替えを行うかは、プログラマが明示的に設計しなければならない。

本章では、これらの仕組みを通じて、複数のクラスを体系的に整理し、柔軟で拡張性の高いプログラムを設計するための方法について解説する。

13.1 継承とは何か

前節では、共通のインターフェースを持ちながら異なる振る舞いを行う複数の型を、どのように整理し扱うかという問題について述べた。このような問題に対する基本的な解決方法の一つが、**継承**と呼ばれる仕組みである。

継承とは、既存のクラスを基にして、新たなクラスを定義する仕組みであり、元となるクラスの機能を引き継ぎつつ、必要に応じて新たな機能を追加したり、振る舞いを個別に定義したりすることができる。

具体例として、人間を表す `Human` クラスを考える。このクラスは、体重と身長から BMI (Body Mass Index) を算出するメンバ関数 `bmi` を持つものとする。

ここで、`Human` クラスを基にして、日本人を表す `Japanese` クラスや、アメリカ人を表す `American` クラスを定義することを考える。これらのクラスは、いずれも「人間」の一種である¹⁵¹ ため、`Human` クラスの機能をそのまま利用することができる。

このような関係は、しばしば「`Japanese` は `Human` の一種である」といった形で表現される。すなわち、より一般的な型¹⁵² である `Human` に対して、より具体的な型として `Japanese` や `American` を定義することになる。

¹⁵¹ サブクラス、派生クラスと（文脈によっては拡張クラスとも）呼ばれる。

¹⁵² サブクラスの元になった上位のクラスを基底クラス、スーパークラス、あるいは単に上位クラスと呼ぶ。

実際に継承を用いてクラスを定義すると、Human クラスに定義された bmi 関数は、Japanese や American、さらにはそれらをさらに拡張したクラスにおいてもそのまま継承され、特別な定義を行うことなく利用することができる。

例えば、Japanese クラスをさらに拡張した Kyotoite クラスを定義した場合でも、Kyotoite のオブジェクトから bmi 関数を呼び出すことができる。これは、Kyotoite が Japanese の一種¹⁵³ であり、Japanese が Human の一種であるという関係に基づいて、最終的に Human の機能が段階的に継承されているためである。

このように、継承を用いることで、クラス間に「一般的な型」と「より具体的な型」という関係を構成し、共通の機能を上位のクラスにまとめることができる。これにより、プログラム全体の構造を整理し、同じ機能を繰り返し定義することを避けることが可能となる。

ただし、継承は単に既存のコードを再利用するための仕組みではない。重要なのは、複数の型（クラス）の間に意味のある関係を定義し、それらを体系的に整理する点にある。

なお、ここで用いた Human, Japanese, Kyotoite といった例は、継承の仕組みを理解するために単純化したものである。実際のプログラムでは、サブクラスごとにメンバ変数の構成が大きく異なるような場合も多く、そのような場面においてこそ、継承とポリモーフィズムの有効性がより明確に現れる。

13.2 継承の基本構文

前節では、既存のクラスを基にして新たなクラスを定義し、機能を引き継ぐ仕組みとして継承の概念を説明した。ここでは、C++における継承の基本的な構文について確認する。

あるクラスを基にして新しいクラスを定義する場合、クラス定義の宣言部において、基底となるクラスを次のように指定する。

《派生クラスの定義》

```
class 派生クラス名 : アクセス指定子 基底クラス名 {  
    (メンバの定義)  
};
```

「基底クラス」を継承した「派生クラス」を定義する。ここで、コロン「:」の後に続く「アクセス指定子」と「基底クラス名」によって、どのクラスを基にして新しい「派生クラス」を定義するかを明示する。

例えば、前節で扱った Human クラスを基にして Japanese クラスを定義する場合、次のように記述する。

```
class Japanese : public Human {  
public:  
    void greet() const {  
        std::cout << "こんにちは\n";  
    }  
};
```

この定義において、Japanese クラスは Human クラスを基底クラスとして継承し、その機能を引き継ぐ。その結果、Human に定義されている利用可能なメンバ関数は、Japanese のオブジェクトからも利用することができる。

コロンの後に指定している public は、継承におけるアクセス指定子であり、基底クラスのメンバが派生クラスにおいてどのように扱われるかを決定するものである。この詳細については後の節で改めて説明するが、本章では特に断りのない限り、public 継承を用いるものとする。

なお、継承関係はさらに連鎖させることができる。例えば、Japanese クラスを基にして、Kyotoite クラスを次のように定義することができる。

```
class Kyotoite : public Japanese {  
public:  
    void omotenashi() const {  
        std::cout << "よう、おこしやす\n";  
    }  
};
```

¹⁵³本例では説明を単純化するため、Japanese や Kyotoite を階層的な分類として仮定する。

この場合、`Kyotoite` は `Japanese` の機能を引き継ぎ、さらにその基底である `Human` の機能も間接的に引き継ぐことになる。したがって、`Kyotoite` のオブジェクトからは、`Human` に定義された `bmi` 関数も利用することができる。

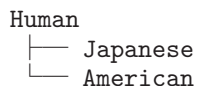
このように、C++ではクラス定義において基底クラスを明示的に指定することで、クラス間の継承関係を構成することができる。

13.3 基底クラスと派生クラス

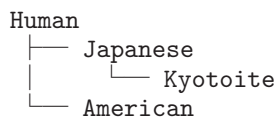
前節では、クラス定義において基底クラスを指定することで、継承関係を構成できることを示した。ここでは、継承関係におけるクラスの役割について整理する。

継承において、元となるクラスを**基底クラス** (base class) と呼び、それを基にして定義されたクラスを**派生クラス** (derived class) と呼ぶ。

例えば、`Human` クラスを基底クラスとし、それを拡張した `Japanese` や `American` を派生クラスとする場合、それぞれの関係は次のように表される。



さらに、`Japanese` クラスを基にして `Kyotoite` クラスを定義した場合には、継承関係は次のように連鎖する。



このように、継承関係は階層的な構造を持ち、より一般的な型から、より具体的な型へと段階的に細分化されていく。この関係は、「`Japanese` は `Human` の一種である」、「`Kyotoite` は `Japanese` の一種である」といった形で表現される。このように、ある型が別の型の一種であるという関係は、しばしば **is-a 関係** と呼ばれる。

基底クラスと派生クラスの関係において重要なのは、派生クラスが基底クラスの機能を引き継ぐ点である。例えば、`Human` クラスに定義されている `bmi` 関数は、`Japanese` や `American`、さらに `Kyotoite` においても利用することができる。すなわち、派生クラスのオブジェクトは、自身のクラスで定義されたメンバだけでなく、基底クラスから継承された利用可能なメンバも利用することができる。

一方で、派生クラスは基底クラスの機能を引き継ぐだけでなく、独自に新たなメンバを追加することもできる。例えば、`Japanese` クラスに `greet` 関数を定義し、日本語で挨拶を行う機能を持たせることができる。また、`Kyotoite` クラスでは、さらに独自の機能として `omotenashi` 関数を定義することができる。

このように、基底クラスは共通の性質や機能をまとめた一般的な型を表し、派生クラスはそれを拡張したより具体的な型を表す。継承を用いることで、これらの関係を明示的に構成し、プログラムの構造を体系的に整理し、可読性と保守性を高めることが可能となる。

■ 派生クラスのコンストラクタについて

継承に関して注意すべき点として、通常、基底クラスのコンストラクタは派生クラスにそのまま継承されるわけではないということが挙げられる。例えば、`Human` クラスにコンストラクタが定義されている場合でも、派生クラスである `Japanese` や `American` では、必要に応じて派生クラス自身のコンストラクタを定義する。これは、他の多くのメンバ関数とは異なり、コンストラクタがオブジェクトの生成と初期化に密接に関わる特別な関数であるためである。

なお、派生クラスのコンストラクタでは、メンバ初期化子リストを用いて、基底クラスのコンストラクタを明示的に呼び出すことができる。この点については、後の節で説明する。

13.4 メンバの継承とアクセス制御

前節では、基底クラスと派生クラスの関係について説明した。派生クラスは基底クラスの機能を引き継ぐことができるが、基底クラスに定義されたメンバを常に自由に利用できるわけではない。C++では、メンバに対するアクセスの可否が、アクセス指定によって制御される。

クラスのメンバには、主に次のようなアクセス指定を与えることができる。

- `public` : クラスの外部から利用できるメンバ
- `private` : 基本的にそのクラスの内部からのみ利用できるメンバ
- `protected` : そのクラス自身と、派生クラスの内部から利用できるメンバ

`class` 内で定義されたメンバは、アクセス指定を書かなければ既定で `private` となる。そのため、外部から利用させたいメンバ関数は明示的に `public` として定義する必要がある。

例えば、`Human` クラスを次のように定義したとする。

```
class Human {
private:
    double weight;
    double height;
public:
    Human(double w, double h) : weight(w), height(h) {}
    double bmi() const {
        return weight / (height * height);
    }
};
```

この例では、体重を表す `weight` と身長を表す `height` は `private` なメンバ変数である。したがって、クラスの外部からこれらの変数を直接読み書きすることはできない。一方、`bmi` 関数は `public` なメンバ関数であるため、クラスの外部から呼び出すことができる。

この `Human` クラスを基にして、次のように `Japanese` クラスを定義する。

```
class Japanese : public Human {
public:
    Japanese(double w, double h) : Human(w, h) {}
    void greet() const {
        std::cout << "こんにちは\n";
    }
};
```

▲注意▲ この例では、`Japanese` クラスのコンストラクタのメンバ初期化子リストで、基底クラス `Human` のコンストラクタを指定している。基底クラスのコンストラクタは、通常、派生クラスのコンストラクタとしてそのまま継承されるわけではなく、派生クラスのオブジェクトを生成する際には、まず基底クラス部分が初期化される。基底クラスに引数なしのコンストラクタが存在する場合には、それが自動的に呼び出されるが、この例のように引数なしのコンストラクタが存在しない場合には、呼び出すコンストラクタをメンバ初期化子リストで指定する必要がある。指定を省略するとコンパイルエラーとなる。

上の例では、`Japanese` は `Human` を継承しているため、`Human` に定義された `bmi` 関数を利用することができる。

例. `bmi`, `greet` 関数の呼び出し

```
Japanese taro(60.0, 1.70);      ← Japanese クラスのオブジェクト taro を作成
std::cout << taro.bmi() << "\n"; ← taro に対して bmi 関数を実行して、戻り値を出力
taro.greet();                  ← taro に対して greet 関数を実行
```

ここで重要なのは、`bmi` 関数は利用できるが、`weight` や `height` を `Japanese` クラスの外部から直接操作できるわけではない、という点である。これらのメンバ変数は `Human` クラスにおいて `private` として定義されているため、外部からは隠蔽されたままである。

また、`private` なメンバは、派生クラスの内部からも直接アクセスできない。すなわち、`Japanese` クラスは `Human` の一種 (is-a 関係) として `bmi` 関数を利用できるが、`Human` の内部にある `weight` や `height` を直接参照することはできない。

例. 許されないケース

```
class Japanese : public Human {
public:
    Japanese(double w, double h) : Human(w, h) {}
    void showWeight() const {
        std::cout << weight << "\n";    ←これはエラーとなる
    }
};
```


このように、継承によって基底クラスの機能が派生クラスに受け継がれても、基底クラスの `private` メンバまで自由に操作できるようになるわけではない。 `private` メンバは、あくまでそのクラス自身の実装の一部として隠蔽される。なお、 `private` メンバそのものが派生クラスのオブジェクトからなくなるわけではない。それらは基底クラス部分に含まれているが、派生クラスの内部から直接アクセスすることはできない。また、基底クラスが提供する `public` または `protected` なメンバ関数を通じて、これらの `private` メンバを間接的に利用することはできる。

一方、基底クラスの内部データを派生クラスから参照させたい場合には、 `protected` を用いることができる。 `protected` として定義されたメンバは、クラスの外部からは利用できないが、派生クラスの内部からは利用できる。

例. 派生クラスに公開される `protected` 宣言

```
class Human {
    protected:
        double weight;
        double height;

    public:
        Human(double w, double h) : weight(w), height(h) {}

        double bmi() const {
            return weight / (height * height);
        }
};
```

このように定義すると、 `Japanese` クラスの内部から `weight` や `height` を参照することができる。ただし、 `protected` を用いると、派生クラスが基底クラスの内部構造に依存しやすくなり、基底クラスの実装変更が派生クラスに影響を与えやすくなる。そのため、単に便利であるという理由だけで `protected` メンバ変数を多用するのは望ましくない。

重要) `protected` は派生クラスの実装から利用できるようにするためのものであり、外部インターフェースとして公開するものではなく、クラスの利用者に見せるべき機能ではない。

多くの場合、基底クラスの内部データは `private` として隠蔽し、必要な操作を `public` または `protected` なメンバ関数として提供する方が、クラスの独立性を保ちやすい。これは、前章で述べたカプセル化の考え方とも一致する。

なお、継承の指定に用いる `public` は、基底クラスの `public` メンバを派生クラスにおいても `public` な機能として扱うための指定である。本書では、特に断りのない限り、この `public` 継承を用いる。 `private` 継承や `protected` 継承も存在するが、これらは継承関係の意味を変えるため、初学者向けの基本的な設計ではまず `public` 継承を理解することが重要である。

以上のように、継承では基底クラスのメンバが派生クラスに受け継がれるが、それらをどこから利用できるかはアクセス指定によって制御される。継承はカプセル化を破壊する仕組みではなく、カプセル化を保ったまま型の関係を構成するための仕組みである。すなわち、継承によって基底クラスの内部実装が露出するわけではない。

13.5 コンストラクタとデストラクタの呼び出し順序

前節では、派生クラスが基底クラスのメンバを引き継ぐこと、およびアクセス指定によって利用できる範囲が制御されることを説明した。ここでは、派生クラスのオブジェクトを生成するとき、基底クラスと派生クラスのコンストラクタがどのような順序で呼び出されるかを確認する。

継承関係にあるクラスでは、派生クラスのオブジェクトを生成するとき、まず基底クラス部分が初期化され、その後で派生クラス自身の部分が初期化される。したがって、コンストラクタの呼び出し順序は、基底クラスのコンストラクタが先で、派生クラスのコンストラクタが後になる。

次の例では、 `Human` クラスを基底クラスとし、それを継承した `Japanese` クラスと `American` クラスを定義している。更に `Japanese` クラスを基底クラスとした `Kyotoite` クラスも定義している。各クラスのコンストラクタとデストラクタでは、呼び出し順序が分かるように、メッセージを出力している。

プログラム： `test210All.cpp`

```
1 #include <iostream>
2 using std::cout;
3
```



```

4 // 基底クラス
5 class Human {
6     private:
7         double weight;
8         double height;
9     public:
10        Human(double w, double h) : weight(w), height(h) {
11            cout << "Human のコンストラクタ\n";
12        }
13        ~Human() {
14            cout << "Human のデストラクタ\n";
15        }
16        double bmi() const {
17            return weight / (height * height);
18        }
19 };
20
21 // 派生クラス (日本人)
22 class Japanese : public Human {
23     public:
24        Japanese(double w, double h) : Human(w, h) {
25            cout << "Japanese のコンストラクタ\n";
26        }
27        ~Japanese() {
28            cout << "Japanese のデストラクタ\n";
29        }
30        void greet() const {
31            cout << "こんにちは\n";
32        }
33        void omotenashi() const {
34            cout << "ようこそ日本へ\n";
35        }
36 };
37
38 // 派生クラス (アメリカ人)
39 class American : public Human {
40     public:
41        American(double w, double h) : Human(w, h) {
42            cout << "American のコンストラクタ\n";
43        }
44        ~American() {
45            cout << "American のデストラクタ\n";
46        }
47        void greet() const {
48            cout << "Hello!\n";
49        }
50 };
51
52 // さらに派生 (京都人)
53 class Kyotoite : public Japanese {
54     public:
55        Kyotoite(double w, double h) : Japanese(w, h) {
56            cout << "Kyotoite のコンストラクタ\n";
57        }
58        ~Kyotoite() {
59            cout << "Kyotoite のデストラクタ\n";
60        }
61
62        void omotenashi() const { // 再定義によって基底クラスの関数を隠蔽
63            cout << "よう, おこしやす\n";
64        }
65 };
66
67 int main() {
68     cout << "--- taro作成 ---\n";           Japanese taro(60.0, 1.70);
69     cout << "--- john作成 ---\n";           American john(80.0, 1.80);
70     cout << "--- jiro作成 ---\n";           Kyotoite jiro(65.0, 1.75);
71
72     cout << "--- メンバ関数: bmi ---\n";
73     cout << "taro BMI = " << taro.bmi() << "\n";
74     cout << "john BMI = " << john.bmi() << "\n";
75     cout << "jiro BMI = " << jiro.bmi() << "\n";

```

```

76
77     cout << "---- メンバ関数：greet ----\n";
78     cout << "taro.greet(): ";          taro.greet();
79     cout << "john.greet(): ";          john.greet();
80
81     cout << "---- メンバ関数：omotenashi ----\n";
82     cout << "taro.omotenashi(): ";      taro.omotenashi();
83     cout << "jiro.omotenashi(): ";      jiro.omotenashi();
84
85     cout << "---- 終了 ----\n";
86 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test210All.cpp のコンパイルと実行

```

$ g++ test210All.cpp -o test210All  Enter  ←コンパイル
$ ./test210All  Enter  ←実行
--- taro 作成 ---
Human のコンストラクタ  ← Japanese の taro のコンストラクタは、その基底クラスのコンストラクタを
Japanese のコンストラクタ 実行した後で実行される
--- john 作成 ---
Human のコンストラクタ  ← American の john のコンストラクタは、その基底クラスのコンストラクタを
American のコンストラクタ 実行した後で実行される
--- jiro 作成 ---
Human のコンストラクタ  ← Kyotoite の jiro のコンストラクタは、その基底クラスのコンストラクタから
Japanese のコンストラクタ 順番に派生クラスの方に向かう順序で
Kyotoite のコンストラクタ 実行される
--- メンバ関数：bmi ---
taro BMI = 20.7612  ←基底クラス Human で定義されているメンバ関数が
john BMI = 24.6914  全ての派生クラスのオブジェクトに対して
jiro BMI = 21.2245  実行できる
--- メンバ関数：greet ---
taro.greet(): こんにちは  ←クラスごとに異なる greet 関数が
john.greet(): Hello! 正しく実行されている
--- メンバ関数：omotenashi ---
taro.omotenashi(): ようこそ日本へ  ← Japanese クラスの omotenashi 関数
jiro.omotenashi(): よう、おこしやす  ← Kyotoite クラスでは同名の omotenashi 関数が定義されている
--- 終了 ---
Kyotoite のデストラクタ  ←オブジェクトの廃棄は派生クラスから
Japanese のデストラクタ 基底クラスに向かう順序で
Human のデストラクタ 実行される
American のデストラクタ
Human のデストラクタ
Japanese のデストラクタ
Human のデストラクタ

```

Japanese 型のオブジェクト taro を生成するとき、最初に呼び出されるのは Human のコンストラクタであり、その後で Japanese のコンストラクタが呼び出される。これは、Japanese のオブジェクトの内部に、基底クラスである Human の部分が含まれていると考えると分かりやすい。

同様に、American 型のオブジェクト john を生成するときも、まず Human のコンストラクタが呼び出され、続いて American のコンストラクタが呼び出される。このように、派生クラスのオブジェクトを生成する際には、基底クラスの初期化が先に行われる。

一方、オブジェクトが破棄されるときには、生成時とは逆の順序でデストラクタが呼び出される。すなわち、まず派生クラスのデストラクタが呼び出され、その後で基底クラスのデストラクタが呼び出される。

上の例では、main 関数の中で taro が先に生成され、それに続けて john, jiro が順に生成されている。ローカル変数として生成されたオブジェクトは、スコープを抜けるときに生成とは逆の順序で破棄されるため、まず jiro が破棄され、次に john, taro が順に破棄される。そのため、出力では Kyotoite のデストラクタ、Japanese のデストラクタ、Human のデストラクタのメッセージ (jiro の破棄処理) が順に出力される。以下同様に、john, taro の破棄に伴うメッセージが出力される。

このように、継承関係にあるクラスでは、生成時には基底クラスから派生クラスへ、破棄時には派生クラスから基底クラスへという順序で処理が行われる。これは、派生クラスのオブジェクトが、基底クラスとしての部分を内部に土台として持っているためである。

なお、基底クラスに引数なしで呼び出せるコンストラクタが存在しない場合、派生クラスのコンストラクタでは、メンバ初期化子リストを用いて基底クラスのコンストラクタに必要な引数を渡す必要がある。上の例では、`Japanese` や `American` のコンストラクタにおいて、`Human(w, h)` と記述することで、基底クラス `Human` のコンストラクタを呼び出している。（`Kyotoite` クラスも同様）

この点は、派生クラスのオブジェクトを生成するとき、基底クラスの部分も同時に正しく初期化しなければならないことを意味している。派生クラスのコンストラクタは、自分自身のメンバだけでなく、基底クラス部分の初期化も考慮して記述する必要がある。

注意) 基底クラス型のポインタを介して派生クラスのオブジェクトを `delete` する可能性がある場合には、基底クラスのデストラクタを `virtual` として定義する必要がある。これを行わずに基底クラス型のポインタから派生クラスのオブジェクトを `delete` すると、動作は未定義となる。

13.6 関数の再定義と名前の隠蔽

前節では、継承関係にあるクラスにおいて、コンストラクタとデストラクタがどのような順序で呼び出されるかを確認した。本節では、派生クラスにおいて基底クラスと同名の関数を定義した場合に、どのような挙動になるかについて説明する。

先の例において、`Japanese` クラスには次のような `omotenashi` 関数が定義されている。

```
void omotenashi() const {
    std::cout << "ようこそ日本へ\n";
}
```

一方、`Kyotoite` クラスでは、同名の関数 `omotenashi` を次のように定義している。

```
void omotenashi() const {
    std::cout << "よう、おこしやす\n";
}
```

このように、派生クラスにおいて基底クラスと同名の関数を定義すると、派生クラスのスコープでは基底クラス側の同名関数が隠蔽される。例えば、`Kyotoite` 型のオブジェクト `jiro` に対して

```
jiro.omotenashi();
```

として呼び出した場合には、`Kyotoite` に定義された関数が実行される。

ここで重要なのは、この仕組みは「関数の上書き（オーバーライド）」（後述）ではなく、単に派生クラスに同名の関数を定義した結果、基底クラスの同名関数は通常の名前探索では見えなくなるという点である。すなわち、`Kyotoite` クラスにおいて `omotenashi` を定義すると、基底クラス `Japanese` に定義されている同名の関数は隠蔽される。

この挙動を**名前の隠蔽（name hiding）**と呼ぶ。名前の隠蔽が起きると、派生クラスのスコープにおいては基底クラスの同名関数をそのままでは呼び出すことができなくなる。例えば、`Kyotoite` クラスのメンバ関数の中で、基底クラス `Japanese` の `omotenashi` を呼び出したい場合には、スコープ解決演算子を用いて次のように記述する。

```
Japanese::omotenashi();
```

このように明示的に指定することで、隠蔽されている基底クラスの関数を呼び出すことができる。

なお、名前の隠蔽は関数だけでなく、変数や型名などにも同様に適用される。すなわち、派生クラスにおいて同名の識別子を定義すると、基底クラスにある同名の識別子は見えなくなる。

ここで注意すべき点として、この段階で行っているのはあくまで「同名関数の再定義」であり、ポリモーフィズムにおける動的な関数選択とは異なるということが挙げられる。このような非仮想関数では、どの関数が呼び出されるかはコンパイル時に決定され、実行時のオブジェクトの型によって切り替わることはない。

この違いは、次節で扱う「仮想関数」と「ポリモーフィズム」において重要となる。本節で説明した名前の隠蔽は、実行時の動的な関数選択とは異なる、コンパイル時の名前探索に関する仕組みであることを理解しておくことが重要

である。

以上のように、派生クラスにおける同名関数の定義は、基底クラスの関数を上書きするのではなく、その名前を隠すという形で働く。この挙動を正しく理解することは、継承を用いたプログラム設計において非常に重要である。

▲注意▲ 派生クラスで基底クラスと同名の関数を定義すると、仮引数の構成が異なる場合でも、基底クラス側の同名関数は通常の名前探索では隠蔽される。すなわち、関数のシグネチャが異なっているにもかかわらず、基底クラス側の同名関数が自動的にオーバーロードの候補に加わるわけではない。(次の例)

```
class Base {
public:
    void f(int);
    void f(double);
};

class Derived : public Base {
public:
    void f(const char*);
};
```

この場合、Derived のスコープでは `Base::f(int)` と `Base::f(double)` は隠蔽される。これは、C++ ではオーバーロード解決より先に名前探索が行われるためである。

13.7 仮想関数と動的束縛

前節では、派生クラスにおいて基底クラスと同名の関数を定義した場合、基底クラスの関数が隠蔽されること、およびどの関数が呼び出されるかはコンパイル時に決定されることを説明した。本節では、実行時に呼び出す関数を切り替える仕組みである**仮想関数と動的束縛**について説明する。

先の例において、Japanese クラスと Kyotoite クラスの `omotenashi` 関数に、それぞれ `virtual` および `override` を付加して次のように定義する。

例. 基底クラス側の変更

```
class Japanese : public Human {
public:
    virtual void omotenashi() const {
        std::cout << "ようこそ日本へ\n";
    }
};
```

例. 派生クラス側の変更

```
class Kyotoite : public Japanese {
public:
    void omotenashi() const override {
        std::cout << "よう、おこしやす\n";
    }
};
```

ここで、基底クラスにおいて関数を `virtual` として宣言すると、その関数は仮想関数となる。派生クラスで対応する関数をオーバーライドした場合には、基底クラス型のポインタや参照を通して呼び出しても、実際のオブジェクトの型に応じて実行される関数が決定される。この挙動を確認するために、基底クラス型のポインタを用いて次のように呼び出す。

```
Japanese* p = &jiro;
p->omotenashi();
```

ここで `jiro` は `Kyotoite` 型のオブジェクトである。このとき、ポインタ `p` の型は `Japanese*` であるにもかかわらず、実行されるのは `Kyotoite` に定義された `omotenashi` 関数である。

これは、ポインタの型ではなく、実際に指しているオブジェクトの型に基づいて呼び出す関数が決定されるためである。このように、実行時に関数の呼び出し先が決定される仕組みを**動的束縛** (dynamic binding) と呼ぶ。

重要) 仮想関数を基底クラス型のポインタや参照を通して呼び出す場合、「どの関数を呼ぶか」の決定が実行時に行われる。

一方、仮想関数でない通常の関数の場合には、ポインタや参照の静的な型に基づいて呼び出す関数が決定される。このような仕組みを**静的束縛**（static binding）と呼ぶ。

したがって、次のようなコードでは結果が異なる。

例. 仮想関数でない場合

```
Japanese* p = &jiro;
p->omotenashi();      ← Japanese の関数が呼び出される
```

例. 仮想関数の場合

```
Japanese* p = &jiro;
p->omotenashi();      ← Kyotoite の関数が呼び出される
```

この違いは、どの関数を呼び出すかの決定が、コンパイル時に行われるか、実行時に行われるかの違いに対応している。

なお、`override` 指定子は、基底クラスの仮想関数を、派生クラス側で正しくオーバーライドしていることをコンパイラにチェックさせるためのものである。例えば、関数名や引数の型が一致していない場合にはコンパイルエラーとなるため、記述ミス¹⁵⁴を防ぐことができる。

以上のように、仮想関数を用いることで、基底クラス型のインターフェースを通して、実際のオブジェクトの型に応じた関数を呼び出すことが可能となる。この仕組みは、**オブジェクト指向プログラミングにおけるポリモーフィズム**（多態性）を実現するための基本となるものである。すなわち、同じインターフェースを通して、異なる振る舞いを実現できる点が、ポリモーフィズムの重要な特徴である。

13.8 ポリモーフィズム

前節では、仮想関数と動的束縛を用いることで、基底クラス型のポインタや参照を通してオブジェクトを扱った場合でも、実際のオブジェクトの型に応じた関数が呼び出されることを解説した。このように、同一のインターフェースを通じて、異なる型のオブジェクトに対して適切な処理を実行できる性質を、**ポリモーフィズム**（polymorphism）と呼ぶ。

ポリモーフィズムとは「多様な形態を持つこと」を意味する語であり、プログラミングにおいては、同一の操作が、対象となるオブジェクトの型に応じて異なる振る舞いを示すことを指す。

次に示すプログラム `test210-2.cpp` は、先の `test210All.cpp` のクラス構成に倣って作り直したもので、`Human` 型のポインタを用いて、異なる型のオブジェクトをまとめて扱う例である。

プログラム：test210-2.cpp

```
1  #include    <iostream>
2  using std::cout;
3
4  // 基底クラス
5  class Human {
6  private:
7      double weight;
8      double height;
9
10 public:
11     Human(double w, double h) : weight(w), height(h) {}
12
13     virtual ~Human() = default;
14
15     double bmi() const {
16         return weight / (height * height);
17     }
18
19     virtual void greet() const {
20         cout << "Hello\n";
21     }
```

¹⁵⁴例えば、関数名は同じでも、引数の型や `const` 修飾の違いなどにより、基底クラスの仮想関数を、派生クラス側で正しく再定義できていない場合、意図しない別の関数として扱われることがある。このような場合でもコンパイルは通るが、動的束縛が働かず、期待した関数が呼び出されない原因となる。`override` を付けることで、このような誤りをコンパイル時に検出することができる。


```

22 };
23
24 // 派生クラス
25 class Japanese : public Human {
26     public:
27         Japanese(double w, double h) : Human(w, h) {}
28
29         void greet() const override {
30             cout << "こんにちは\n";
31         }
32
33         virtual void omotenashi() const {
34             cout << "ようこそ日本へ\n";
35         }
36 };
37
38 class American : public Human {
39     public:
40         American(double w, double h) : Human(w, h) {}
41
42         void greet() const override {
43             cout << "Hello!\n";
44         }
45 };
46
47 // Japaneseからさらに派生
48 class Kyotoite : public Japanese {
49     public:
50         Kyotoite(double w, double h) : Japanese(w, h) {}
51
52         void greet() const override {
53             cout << "よう, おこしやす\n";
54         }
55
56         void omotenashi() const override {
57             cout << "よう, おこしやす\n";
58         }
59 };
60
61 int main() {
62     Japanese taro(60.0, 1.70);
63     American john(80.0, 1.80);
64     Kyotoite jiro(65.0, 1.75);
65
66     Human* people[] = { &taro, &john, &jiro };
67
68     for (auto p : people) {
69         p->greet();
70     }
71 }

```

このコードでは、`people` には異なる型のオブジェクトを指すポインタが格納されているが、ループ内ではすべてを `Human` 型のポインタとして扱い、同一の `greet` 関数を呼び出している。このとき、各ポインタが指している実際のオブジェクトの型に応じて、それぞれ適切な `greet` 関数が実行される。これは、`greet` 関数が仮想関数として定義されており、動的束縛によって実行時に呼び出す関数が決定されるためである。

このプログラムをコンパイルして実行する例を次に示す。

例. test210-2.cpp のコンパイルと実行

```

$ g++ test210-2.cpp -o test210-2  [Enter]    ←コンパイル
$ ./test210-2  [Enter]             ←実行
こんにちは                          ← Japanese クラスの taro の場合
Hello!                              ← American クラスの john の場合
よう, おこしやす                    ← Kyotoite クラスの jiro の場合

```

このように、呼び出し側は具体的な型の違いを意識することなく、統一的な操作を記述することができる。この点が、ポリモーフィズムの最も重要な特徴である。また、ポリモーフィズムを用いることで、プログラムの拡張が容易になる。例えば、新たに `Kyotoite` クラスを追加した場合でも、`Human` 型として扱う限り、既存のループ処理などを変

更することなく扱うことができる。すなわち、新しい型を追加しても、その型を基底クラス型として扱うことで、既存の統一的な処理の枠組みに組み込むことができる。

このような性質は、大規模なプログラムにおいて特に重要であり、機能の追加や変更に対して柔軟に対応できる設計を可能にする。なお、ポリモーフィズムは単に「同じ名前の関数を複数定義すること」を意味するものではない。本節で扱っているのは、仮想関数と動的束縛によって、実行時に適切な関数が選択される**実行時のポリモーフィズム**である。

以上のように、継承と仮想関数を組み合わせることで、同一のインターフェースを通じて多様な型を統一的に扱うことが可能となる。これが、C++におけるポリモーフィズムの基本的な考え方である。

重要) 基底クラス型の共通インターフェースを通して同じメンバ関数を呼び出しても、実際のオブジェクトの型に応じた振る舞いが選択される点が重要である。

13.8.1 オブジェクトスライシング

これまでに、基底クラス型のポインタや参照を通して派生クラスのオブジェクトを扱うことで、実際のオブジェクトの型に応じた仮想関数が呼び出されることを説明した。しかし、派生クラスのオブジェクトを基にして、基底クラス型のオブジェクトを値として生成すると、このような動的な振る舞いが失われる。

例えば、前節のプログラム `test210-2.cpp` で定義した `Japanese` クラスと、それを継承した `Kyotoite` クラスを用いて、次のように記述した場合を考える。

```
Kyotoite jiro(65.0, 1.75);
Japanese person = jiro;
person.omotenashi();
```

`jiro` は `Kyotoite` 型のオブジェクトであるが、変数 `person` の型は `Japanese` である。この代入では、`jiro` に含まれる基底クラス `Japanese` の部分だけを用いて、新しい `Japanese` 型のオブジェクト `person` が生成される。その結果、`Kyotoite` 型としての性質は `person` には引き継がれない。

従って、`person` は `Kyotoite` 型のオブジェクトを参照しているのではなく、新たに生成された `Japanese` 型のオブジェクトである。そのため、上の `omotenashi` 関数の呼び出しでは、この関数が仮想関数として定義されていても、`Japanese` クラスの関数が実行され、次のように出力される。

ようこそ日本へ

このように、派生クラスのオブジェクトから基底クラス型のオブジェクトを生成すると、派生クラス固有の部分が切り落とされ、基底クラス型のオブジェクトだけが生成される。この現象を**オブジェクトスライシング** (object slicing) という。

上のような基底クラス型オブジェクトの生成は通常、コンパイルエラーとはならないため、意図せずオブジェクトスライシングを起こさないように注意する必要がある。

オブジェクトスライシングは、関数の引数にオブジェクトを値として渡す場合にも発生する。例えば、次の関数を考える。

```
void welcome(Japanese person) {
    person.omotenashi();
}
```

この関数に `Kyotoite` 型のオブジェクトを渡す。

```
Kyotoite jiro(65.0, 1.75);
welcome(jiro);
```

関数の呼び出しに伴って、`jiro` の `Japanese` に相当する部分だけを用いて、仮引数 `person` が生成される。従って、この場合も `Kyotoite` 型としての性質が失われ、`Japanese` クラスの `omotenashi` 関数が実行される。

オブジェクトスライシングを避け、ポリモーフィズムを維持するには、オブジェクトを値として複製するのではなく、基底クラス型の参照またはポインタを介して扱う。例えば、上の関数を参照渡しに変更すると、次のようになる。

```
void welcome(const Japanese& person) {
    person.omotenashi();
}
```

この場合、関数の呼び出しに伴うオブジェクトの複製は行われない。仮引数 `person` は、呼び出し側の `jiro` を参照しており、その実体は `Kyotoite` 型のオブジェクトのままである。従って、動的束縛によって `Kyotoite` クラスの `omotenashi` 関数が実行され、次のように出力される。

よう、おこしやす

ポインタを用いる場合も、同様にオブジェクトスライシングを避けることができる。

```
void welcome(const Japanese* person) {
    person->omotenashi();
}
```

この関数は、次のように呼び出す。

```
welcome(&jiro);
```

このように、継承関係にある複数の型を実行時のポリモーフィズムによって扱う場合には、基底クラス型の変数にオブジェクトを値として格納するのではなく、基底クラス型の参照またはポインタを介して扱う。特に、関数の引数として派生クラスのオブジェクトを値渡しする場合や、基底クラス型のオブジェクトを値として格納するコンテナを使用する場合には、オブジェクトスライシングが発生する可能性があるので注意が必要である。

先のプログラム `test210-2.cpp` の `main` 関数を次のように書き換えて実行すると、オブジェクトスライシングが発生することが確認できる。

[main 関数の書き換え]

```
1 int main() {
2     Kyotoite jiro(65.0, 1.75);
3
4     Japanese person = jiro;    // オブジェクトスライシングが発生
5
6     cout << "jiro: ";
7     jiro.omotenashi();
8
9     cout << "person: ";
10    person.omotenashi();
11 }
```

`main` 関数をこのように書き換えたものを `test210-3.cpp` として実行する例を次に示す。

例. `test210-3.cpp` のコンパイルと実行

```
$ g++ test210-3.cpp -o test210-3  [Enter]    ←コンパイル
$ ./test210-3  [Enter]    ←実行
jiro: よう、おこしやす    ← Kyotoite クラスのオブジェクトとしての omotenashi
person: ようこそ日本へ    ← Japanese クラスのオブジェクトとしての omotenashi
```

`person` は `jiro` の `Japanese` クラスに相当する部分を基に生成されたオブジェクトであり、`Japanese` クラスのオブジェクトとして機能することがわかる。

13.9 抽象クラスと純粋仮想関数

前節では、仮想関数と動的束縛を用いることで、同一のインターフェースを通して、実際のオブジェクトの型に応じた処理を実行できることを確認した。このような仕組みによってポリモーフィズムが実現される。

しかし、これまでに扱ってきた `Human` などの例は極めて単純なものであり、更に、基底クラスにおいて具体的な処理内容まで定義されていた。実際のプログラム設計においては、「共通の操作は存在するが、その具体的な内容は各クラスによって大きく異なる」という状況が多く現れる。

ここで、別の例として図形を扱う場合を考える。円、長方形、三角形といった図形は、それぞれ異なるデータ構造を持つが、「面積を求める」という操作はすべてに共通している。このような場合には、「面積を求めることができる」という共通の性質だけを持つ基底クラスを定義し、具体的な計算方法は派生クラスに委ねる設計が有効である。

各種図形を扱うための基底クラス `Shape` を次のように定義する。

例. 純粋仮想関数を持つ基底クラス

```
class Shape {
public:
    virtual double area() const = 0;
};
```

ここで、「= 0」と記述されている関数 `area` は、**純粋仮想関数**と呼ばれる。純粋仮想関数は、派生クラス側で具体的な処理を定義することを前提とした仮想関数である。従って、このような純粋仮想関数を持つクラスのオブジェクトを直接生成することはできない。また、このような「= 0」という記述は一見すると特異であるが、C++ では関数の意味を指定する重要な記法である。他にも同様に、代入形式によって関数の振る舞いを指定するもの（表 43）がある。

表 43: 代入形式による関数宣言の意味づけ

記 法	役 割	設計の意味
= 0	純粋仮想関数として宣言	インターフェースの定義 派生クラスでの実装を前提とする
= default	デフォルト実装を使用（コンパイラ生成）	標準動作
= delete	使用禁止	制約

純粋仮想関数を1つ以上持つクラスは、**抽象クラス**と呼ばれる。抽象クラスは、それ自身のオブジェクトを生成するためのものではなく、共通のインターフェースや基礎となる機能を定義するために用いられる。

例. 良くない例：抽象クラスのインスタンス化の試み

```
Shape s;           ←エラーとなる
```

抽象クラスを継承した派生クラスが純粋仮想関数を実装しない場合は、その派生クラスも抽象クラスとなり、そのクラスのオブジェクトを生成することはできない。

次に、抽象クラスを基底クラスとして、具体的な図形を表すクラスを定義する例を示す。例えば、円を表す `Circle` クラスは次のように定義できる。

例. 派生クラスでの実装

```
class Circle : public Shape {
private:
    double r;
public:
    Circle(double radius) : r(radius) {}
    double area() const override {
        return 3.14159 * r * r;
    }
};
```

このように `Circle` クラスでは、面積を求める `area` 関数を具体的に実装している。同様に、他の図形についても、それぞれの形状に応じた面積の計算方法を実装することになる。

この設計において重要なのは、`Shape` クラスが「面積を求めることができる」という共通のインターフェースのみを定義し、具体的な処理は一切記述していない点である。

このように、抽象クラスと純粋仮想関数を用いることで、具体的な実装を伴わない共通のインターフェースを定義し、多様な型を統一的に扱うための枠組みを構築することができる。これは、前節で説明したポリモーフィズムを、実際のプログラム設計において活用するための基本的な設計の枠組みである。

13.10 仮想デストラクタ

前節では、抽象クラスと純粋仮想関数を用いることで、共通のインターフェースを通して多様な型を統一的に扱うことができることを説明した。このような設計では、基底クラス型のポインタや参照を通してオブジェクトを操作する場面が頻繁に現れる。

このとき注意すべき点として、オブジェクトの**破棄**に関する問題がある。すなわち、基底クラス型のポインタを用いてオブジェクトを解放する場合には、デストラクタの定義方法に注意しなければならない。

例として、前節で導入した抽象クラス `Shape` と、その派生クラス `Circle` を用いる。

例. デストラクタの定義（不完全な例）

```
class Shape {
public:
    virtual double area() const = 0;
};

class Circle : public Shape {
private:
    double r;
public:
    Circle(double radius) : r(radius) {}
    ~Circle() {
        std::cout << "Circle のデストラクタ\n";
    }
    double area() const override {
        return 3.14159 * r * r;
    }
};
```

ここで、基底クラス型のポインタを用いてオブジェクトを生成し、解放する場合を考える。

■ new と delete

オブジェクトを動的に生成するための演算子に `new` がある。また、明示的にオブジェクトを破棄するための演算子に `delete` がある。これらの直接的な使用は慎重に行う必要があり、現代的な C++ では、より安全な資源管理手法¹⁵⁵ を用いることが推奨される。本節では、仮想デストラクタの挙動を理解するために、これらの基本的な使用方法を示す。

例. 基底クラス型ポインタによる操作

```
Shape* p = new Circle(2.0);
delete p;
```

このとき、`Shape` のデストラクタが仮想関数として定義されていない状態で、`Shape*` を通して `Circle` 型のオブジェクトを `delete` すると、プログラムの動作は未定義となる。従って、派生クラスまで含めた正しい破棄処理は保証されない。

しかし、実際に生成されているオブジェクトは `Circle` 型であり、その内部には `Circle` 固有の処理や資源が含まれている可能性がある。そのため、`Circle` のデストラクタが呼び出されない場合、正しく後始末が行われず、資源の解放漏れなどの問題を引き起こす恐れがある。

この問題を防ぐために、基底クラスのデストラクタは仮想関数として定義する必要がある。これにより、実行時に実際のオブジェクトの型に応じたデストラクタが呼び出されるようになる（動的束縛）。すなわち、次のように `virtual` を付けて宣言する。

例. 仮想デストラクタの定義

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
```

このように定義すると、基底クラス型のポインタを通して `delete` を行った場合でも、実際のオブジェクトの型に応じて、派生クラスから基底クラスへと必要なデストラクタが順に実行される。

重要 基底クラス型のポインタを通してオブジェクトを削除する可能性がある場合には、デストラクタは必ず `virtual` として定義する必要がある。特に、抽象クラスを基底クラスとして用いる場合には、仮想デストラクタの定義は実質的に必須である。

以上のように、ポリモーフィズムを用いた設計においては、オブジェクトの生成だけでなく、破棄の方法についても考慮する必要がある。仮想デストラクタは、そのための重要な仕組みであり、安全な資源管理のために不可欠である。

¹⁵⁵ `new/delete` を直接的に使用せず、オブジェクトの寿命を自動的に管理する設計（後述の RAII など）。

■ サンプルプログラム

本節で説明した仮想デストラクタの挙動を確認するためのサンプルプログラム test211All.cpp を次に示す。このプログラムでは、抽象クラスを基底クラスとし、複数の派生クラスを基底クラス型のポインタで統一的に扱っている。オブジェクトの生成時および破棄時に出力されるメッセージにより、コンストラクタとデストラクタの呼び出し順序、および基底クラス型のポインタを通して削除した場合の仮想デストラクタの働きを確認することができる。

プログラム：test211All.cpp

```
1  #include    <iostream>
2  #include    <cmath>
3  #include    <iomanip>
4  using std::cout, std::sqrt;
5
6  // 抽象クラス
7  class Shape {
8      public:
9          Shape() {
10             cout << "Shape のコンストラクタ\n";
11         }
12
13         virtual ~Shape() {
14             cout << "Shape のデストラクタ\n";
15         }
16
17         virtual double area() const = 0;
18     };
19
20 // 円
21 class Circle : public Shape {
22     private:
23         double r;
24
25     public:
26         Circle(double radius) : r(radius) {
27             cout << "Circle のコンストラクタ\n";
28         }
29
30         ~Circle() override {
31             cout << "Circle のデストラクタ\n";
32         }
33
34         double area() const override {
35             return 3.14159 * r * r;
36         }
37     };
38
39 // 三角形
40 class Triangle : public Shape {
41     private:
42         double a, b, c;
43
44     public:
45         Triangle(double x, double y, double z) : a(x), b(y), c(z) {
46             cout << "Triangle のコンストラクタ\n";
47         }
48
49         ~Triangle() override {
50             cout << "Triangle のデストラクタ\n";
51         }
52
53         double area() const override {
54             double s = (a + b + c) / 2.0;
55             return sqrt(s * (s - a) * (s - b) * (s - c));
56         }
57     };
58
59 // 長方形
60 class Rectangle : public Shape {
61     private:
62         double w, h;
```

```

63
64 public:
65     Rectangle(double width, double height) : w(width), h(height) {
66         cout << "Rectangle のコンストラクタ\n";
67     }
68
69     ~Rectangle() override {
70         cout << "Rectangle のデストラクタ\n";
71     }
72
73     double area() const override {
74         return w * h;
75     }
76 };
77
78 int main() {
79     cout << "--- Circleオブジェクト作成 ---\n";
80     Shape* p1 = new Circle(3.0);
81
82     cout << "--- Triangleオブジェクト作成 ---\n";
83     Shape* p2 = new Triangle(3.0, 4.0, 5.0);
84
85     cout << "--- Rectangleオブジェクト作成 ---\n";
86     Shape* p3 = new Rectangle(4.0, 5.0);
87
88     Shape* shapes[] = {p1, p2, p3};
89
90     cout << "--- 面積の計算 ---\n";
91     cout << std::fixed << std::setprecision(3);
92     for (auto p : shapes) {
93         cout << "area = " << p->area() << "\n";
94     }
95
96     cout << "--- オブジェクト破棄 ---\n";
97     for (auto p : shapes) {
98         delete p;
99         cout << "---\n";
100     }
101 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test211All.cpp のコンパイルと実行

```

$ g++ test211All.cpp -o test211All  ←コンパイル
$ ./test211All  ←実行
--- Circle オブジェクト作成 ---
Shape のコンストラクタ      ←基底クラスのコンストラクタが呼び出され
Circle のコンストラクタ     ←次に派生クラスのコンストラクタが呼び出される
--- Triangle オブジェクト作成 ---
Shape のコンストラクタ      ←以下同様
Triangle のコンストラクタ
--- Rectangle オブジェクト作成 ---
Shape のコンストラクタ
Rectangle のコンストラクタ
--- 面積の計算 ---
area = 28.274                ←円の面積
area = 6.000                 ←三角形の面積
area = 20.000                ←長方形の面積
--- オブジェクト破棄 ---
Circle のデストラクタ       ←派生クラスのデストラクタが呼び出され
Shape のデストラクタ        ←次に基底クラスのデストラクタが呼び出される
---
Triangle のデストラクタ     ←以下同様
Shape のデストラクタ
---
Rectangle のデストラクタ
Shape のデストラクタ
---

```

13.11 継承設計に関する注意

本章では、継承、関数の再定義と名前の隠蔽、仮想関数と動的束縛、ポリモーフィズム、さらに抽象クラスや仮想デストラクタに至るまで、クラスの階層構造を用いたプログラム設計の基本的な仕組みを一通り解説した。これらは強力な機能であるが、実際の設計においては慎重に用いる必要がある。

まず重要なのは、継承は単なるコードの再利用のための仕組みではない、という点である。継承は、クラス間に意味のある関係を定義するための仕組みであり、「ある型が別の型の一種である」という関係（is-a 関係）を表現するために用いられる。したがって、見た目が似ているという理由だけで継承関係を構成することは望ましくない。例えば、機能が似ているからといって無理に継承を用いると、本来独立しているべきクラス同士が強く結び付けられ、プログラムの柔軟性が失われることがある。

また、継承によって基底クラスの内部構造に依存する設計は、変更に対して脆くなる傾向がある。特に、`protected` で宣言されたメンバ変数を多用すると、派生クラスが基底クラスの内部実装に強く依存することになり、基底クラスの変更が派生クラスに影響を及ぼしやすくなる。このような状況は、保守性の低下を招く原因となる。

このため、多くの場合、基底クラスの内部データは `private` として隠蔽し、必要な操作のみを `public` または `protected` なメンバ関数として提供する方が、クラスの独立性を保ちやすい。これは、本章で扱ったポリモーフィズムを安全に活用するためにも重要な設計方針である。

さらに、継承を用いるよりも、オブジェクトをメンバとして持つ設計（コンポジション）の方が適切な場合も多い。継承が「～である（is-a）」という関係を表すのに対して、コンポジションは「～を持つ」（has-a）という関係を表す。

例えば、「車はエンジンを持つ」という関係は継承ではなくコンポジションで表現するのが自然である。このように、クラス間の関係の性質に応じて、継承とコンポジションを適切に使い分けることが重要である。

なお、本章の前半で用いた `Human`, `Japanese`, `Kyotoite` といった例は、継承の仕組みを理解するために単純化したものである。実際の設計では、このような分類をそのまま継承階層として表すとは限らない。

一方、本章後半で扱った `Shape` の例のように、派生クラスごとに内部構造や処理内容が大きく異なる場合には、抽象クラスと純粹仮想関数を用いて、共通のインターフェースのみを定義し、具体的な実装は各クラスに委ねる設計が有効である。さらに、このような設計では、基底クラス型のポインタを通してオブジェクトを扱うことが多くなるため、そのポインタを通してオブジェクトを削除する可能性がある場合には、基底クラスのデストラクタを仮想関数として定義する必要がある点にも注意が必要である。

以上のように、継承とポリモーフィズムの本質は、「型の関係を適切に定義し、共通のインターフェースを通じて多様な振る舞いを統一的に扱うこと」にある。これらの仕組みを適切に用いることで、拡張性と保守性に優れたプログラムを構築することが可能となる。

13.12 C++におけるオブジェクトとオブジェクト指向プログラミング

オブジェクト指向プログラミングでは、クラスに基づいて生成された個々の実体を、そのクラスのインスタンス（instance）と呼ぶことがある。この用語は、Java や Python をはじめ、多くのオブジェクト指向言語において広く用いられている。C++においても、「クラスのインスタンス」という表現が誤りというわけではない。

ただし、C++では、クラス型に基づいて生成された実体を、通常はオブジェクト（object）と呼ぶ。また、C++ではインスタンス化（instantiation）という語が、テンプレート¹⁵⁶に関する用語として用いられる。例えば、次のようなクラステンプレートを考える。

```
template<class T>
class Box {
    T value;
};
```

このテンプレートに対して、次のように記述すると、`int` 型に対応した `Box<int>` が必要となる。

```
Box<int> box;
```

`Box<int>` は、クラステンプレート `Box` に `int` を指定して得られる特殊化（specialization）である。このように、テ

¹⁵⁶後の「16 型を抽象化するための機能」（p.322～）で詳しく解説する。

ンプレートから具体的な型を作り出す処理を**インスタンス化** (instantiation) という。このため、本書では、クラス型に基づいて生成された実体を「オブジェクト」と呼び、「インスタンス化」は主としてテンプレートに関する処理を表す用語として使用する。

なお、C++は、クラス、継承、仮想関数、ポリモーフィズムなど、オブジェクト指向プログラミングを行うための機能を備えている。そのため、C++はしばしば「オブジェクト指向プログラミング言語」と呼ばれる。しかし、C++はオブジェクト指向プログラミングだけを前提とした言語ではない。C言語から受け継いだ手続き型プログラミングに加えて、テンプレートを用いたジェネリックプログラミング、関数オブジェクトやラムダ式を用いた関数型に近い記述など、複数のプログラミング様式を利用することができる。さらに、低水準のメモリ操作を含むシステムプログラミングにも対応している。

このように、複数のプログラミング上の考え方を組み合わせて利用できる言語を、**マルチパラダイム言語**と呼ぶ。従って、C++を単にオブジェクト指向言語と説明するよりも、「オブジェクト指向プログラミングを含む複数のプログラミング様式を支援する言語」と捉える方が正確である。

「オブジェクト指向プログラミング」という用語自体が不適切なわけではない。クラスや継承、ポリモーフィズムを中心としてプログラムを構成する方法は、C++においてもオブジェクト指向プログラミングと呼ばれる。ただし、それはC++で利用できるプログラミング様式の一つであり、すべてのデータや処理をクラス階層として設計しなければならないという意味ではない。

14 オブジェクトの寿命と所有権

C++では、オブジェクトが生成されてから破棄されるまでの期間を適切に管理することが重要である。特に、動的に生成されたオブジェクトや、ファイル、メモリ領域、通信接続などの資源を扱う場合には、それらがいつまで有効であり、誰が管理し、どの時点で解放するかを明確にしなければならない。

オブジェクトが存在している期間を、そのオブジェクトの**寿命**という。また、オブジェクトや資源を管理し、不要になったときに破棄または解放する責任を持つことを、本書ではそのオブジェクトや資源の**所有権**を持つという。所有権が明確でないプログラムでは、メモリリーク、二重解放、既に破棄されたオブジェクトへのアクセスなどの問題が発生しやすい。

C++では、資源の管理をオブジェクトの寿命と結び付けることによって、資源を自動的に管理することができる。この考え方は **RAII** と呼ばれ、C++における資源管理の基本原則となっている。標準ライブラリの文字列、コンテナ、入出力ストリームなども、この考え方に基づいて設計されている。

また、動的に生成されたオブジェクトの所有権を安全に管理するために、C++の標準ライブラリには**スマートポインタ**が用意されている。スマートポインタを使用すると、**単独所有**や**共有所有**といった所有関係を型によって表し、オブジェクトの破棄を自動化することができる。また、共有所有されているオブジェクトを所有せずに参照するための仕組みも用意されている。

本章では、まずオブジェクトの寿命と所有権の基本的な考え方を説明する。次に、所有権の移動と RAII について解説し、スマートポインタを用いた安全なオブジェクト管理の方法を示す。最後に、コピー、移動、破棄を考慮したクラス設計の指針である Rule of Three/Five/Zero について説明する。

14.1 オブジェクトの寿命

C++のプログラムでは、変数、配列、クラスから生成された実体などを、広い意味で**オブジェクト**という。オブジェクトはプログラムの実行中に生成され、しばらくの間使用された後、破棄される。このように、オブジェクトが生成されてから破棄されるまでの期間を、そのオブジェクトの**寿命**という。

オブジェクトは、その寿命の間だけ有効である。寿命が始まる前、または寿命が終了した後にそのオブジェクトへアクセスすると、未定義動作となる場合がある。従って、オブジェクトを扱う際には、そのオブジェクトがどの時点で生成され、いつまで存在するかを理解することが重要である。

14.1.1 オブジェクトの生成と破棄

通常のローカル変数は、その宣言が実行されたときに生成される。例えば、次の宣言では、`int` 型のオブジェクト `n` が生成され、値 10 で初期化される。

```
int n = 10;
```

クラス型のオブジェクトでは、オブジェクトの生成時にコンストラクタが呼び出される。また、そのオブジェクトが破棄される際には、デストラクタが呼び出される。例えば、次のクラスを考える。

```
class Sample {
private:
    std::string name;
public:
    explicit Sample(const std::string& s) : name(s) {
        std::cout << name << "を生成\n";
    }
    ~Sample() {
        std::cout << name << "を破棄\n";
    }
};
```

このクラス定義の後、

```
Sample obj("obj");
```

としてオブジェクト `obj` を生成すると、コンストラクタによって生成時のメッセージが

```
obj を破棄
```

と表示される。また、オブジェクトの寿命が終了すると、デストラクタによって破棄時のメッセージが

obj を破棄
と表示される。

このように、コンストラクタとデストラクタに処理を記述すると、オブジェクトの生成と破棄のタイミングに合わせて、必要な処理を実行できる。

14.1.2 スコープと寿命

関数や複文の内部で宣言された通常のローカル変数は、宣言された位置で生成され、その変数が属するブロックを抜ける際に破棄される。例えば、先の Sample クラスを用いて、次のような処理を考える。

```
void func() {  
    Sample a("a");  
    {  
        Sample b("b");  
    }  
}
```

a は関数 func の本体に入り、その宣言が実行された時点で生成される。b は内側のブロックに入って宣言が実行された時点で生成され、そのブロックを抜ける際に破棄される。その後、関数 func を抜ける際に a が破棄される。従って、この処理では次の順序でメッセージが表示される。

```
a を生成  
b を生成  
b を破棄  
a を破棄
```

同じブロック内で複数のローカルオブジェクトが生成された場合には、基本的に生成された順序とは逆の順序で破棄される。

ローカル変数では、変数名を使用できる範囲である**スコープ**と、オブジェクトが存在している期間である**寿命**が、密接に対応している。ただし、スコープと寿命は同じ概念ではない。スコープは、プログラムの記述上、その名前を使用できる範囲を表す。これに対して、寿命は、プログラムの実行中にオブジェクトが実際に存在している期間を表す。

14.1.3 関数から戻る際のオブジェクトの破棄

ローカルオブジェクトは、関数の末尾まで処理が到達した場合だけでなく、return 文によって関数から戻る場合にも破棄される。例えば、先の Sample クラスを用いて、次の関数を考える。

```
void func(bool flag) {  
    Sample obj("obj");  
    if (flag) return;  
    std::cout << "次の処理を実行\n";  
}
```

この場合でも、return 文によって関数から戻る際には、obj のデストラクタが呼び出される。この性質により、ローカルオブジェクトに後処理を担当させることができる。この仕組みは、後に説明する RAII の基礎となる。

また、例外が送出され、通常とは異なる経路でブロックから抜ける場合にも、その時点までに生成されているローカルオブジェクトは、基本的に生成された順序とは逆の順序で破棄される。従って、デストラクタを利用すると、関数の終了経路が複数存在する場合でも、後処理を自動的に実行させることができる。

14.1.4 一時オブジェクトの寿命

C++では、明示的に変数名を付けなくても、式の評価の途中で一時的なオブジェクトが生成されることがある。このようなオブジェクトを**一時オブジェクト**という。例えば、先の Sample クラスを用いて、次のように記述すると一時オブジェクトが生成される。

```
Sample("temp");
```

通常、一時オブジェクトは、そのオブジェクトを含む式の評価が完了した後に破棄される。従って、上の例で生成されたオブジェクトは、同じ文の末尾で破棄される。また、一時オブジェクトを const 参照で受け取ると、一定の条件の下で、その寿命を参照の寿命まで延長できる。

```
const Sample& ref = Sample("temp");
```

この場合、一時オブジェクトは文の末尾では破棄されず、`ref` の寿命が終了するまで存在する。一時オブジェクトと `const` 参照の関係については、既に「11.7.2 一時オブジェクトと `const` 参照」(p.209) で説明した通りである。

14.1.5 静的記憶期間を持つオブジェクト

グローバル変数や、`static` を指定して宣言された変数などは、静的記憶期間を持ち、通常のローカル変数とは異なる長い寿命を持つ。グローバルオブジェクトは、基本的に `main` 関数の実行が始まる前に生成され、プログラムの終了時に破棄される。

また、関数内で `static` を指定して宣言されたローカルオブジェクトは、その宣言が初めて実行された際に生成され、関数の呼び出しが終了しても破棄されない。例えば、先の `Sample` クラスを用いて、次のように記述する。

```
void func() {
    static Sample s("static");
}
```

この `s` は、`func` が初めて呼び出され、この宣言が実行されたときに生成される。その後、`func` が繰り返し呼び出されても同じオブジェクトが使用され、プログラムの終了時に破棄される。

このように、オブジェクトの寿命は、必ずしも変数名のスコープと一致するとは限らない。関数内の `static` 変数では、変数名を使用できる範囲は関数内に限られるが、オブジェクト自体は関数の終了後も存在し続ける。

14.1.6 動的に生成されたオブジェクトの寿命

`new` 演算子によって動的に生成されたオブジェクト（下記）は、通常のローカル変数のように、ブロックを抜けただけでは自動的に破棄されない。例えば、先の `Sample` クラスを用いて、次のように記述する。

```
Sample* p = new Sample("dynamic");
```

この処理では、`Sample` 型のオブジェクトが動的に生成され、そのアドレスが `p` に格納される。ポインタ変数 `p` 自体は、その属するブロックを抜けると消滅するが、`new` によって生成されたオブジェクトは、`delete` によって明示的に破棄されるまで存在し続ける。

```
delete p;
```

`delete` を実行すると、対象のオブジェクトのデストラクタが呼び出された後、そのオブジェクトに使用されていたメモリ領域が解放される。この場合、`delete` を実行しないまま、そのオブジェクトを指すポインタを失うと、後からそのオブジェクトを適切に破棄することができなくなる。このように、不要になったメモリ領域が解放されずに残ることをメモリリークという。

14.1.7 寿命を終えたオブジェクトへのアクセス

オブジェクトの寿命が終了した後、そのオブジェクトを指していたポインタや参照を介してアクセスしてはならない。例えば、次の処理では、`delete` を実行しても `p` 自体が自動的に `nullptr` になるわけではない。

```
int* p = new int(10);
delete p;
std::cout << *p; // 誤り
```

`p` が指していたオブジェクトは既に破棄されているため、`p` を介してそのオブジェクトへアクセスすると、未定義動作となる。このように、寿命を終えたオブジェクトを指しているポインタを**ダングリングポインタ**という。また、寿命を終えたオブジェクトを参照している参照を**ダングリング参照**という。

同様の問題は、ローカル変数へのポインタや参照を関数の外部へ返した場合にも発生する。

```
int* func() {
    int n = 10;
    return &n;    // 誤り
}
```

`n` の寿命は関数 `func` から戻る際に終了する。従って、返されたポインタは、既に寿命を終えたオブジェクトを指すダングリングポインタとなる。

オブジェクトへのポインタや参照を保持する場合には、参照先のオブジェクトが、そのポインタや参照を使用している間、確実に存在している必要がある。このような寿命の関係を適切に設計することは、C++プログラムの安全性

を保つ上で重要である。

■ サンプルプログラム

次のプログラム test212-1.cpp では、ブロック内のローカルオブジェクト、関数内の静的オブジェクト、および動的に生成されたオブジェクトについて、生成と破棄のタイミングを確認する。

プログラム：test212-1.cpp

```
1  #include    <iostream>
2  #include    <string>
3  using std::cout;
4
5  class Sample {
6  private:
7      std::string name;
8  public:
9      explicit Sample(const std::string& s) : name(s) {    // コンストラクタ
10         cout << name << "を生成\n";
11     }
12     ~Sample() {                                          // デストラクタ
13         cout << name << "を破棄\n";
14     }
15 };
16
17 void func() {
18     cout << "funcを開始\n";
19     Sample a("a");
20     {
21         Sample b("b");
22     }
23     static Sample s("static");
24     cout << "funcを終了\n";
25 }
26
27 int main() {
28     cout << "mainを開始\n";
29     Sample m("m");
30     func();
31     cout << "動的オブジェクトを生成\n";
32     Sample* p = new Sample("dynamic");
33     cout << "動的オブジェクトを破棄\n";
34     delete p;
35     cout << "mainを終了\n";
36 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test212-1.cpp のコンパイルと実行

\$ g++ test212-1.cpp -o test212-1 ←コンパイル

\$./test212-1 ←実行

main を開始 ←出力

m を生成

func を開始

a を生成

b を生成

b を破棄

static を生成

func を終了

a を破棄

動的オブジェクトを生成

dynamic を生成

動的オブジェクトを破棄

dynamic を破棄

main を終了

m を破棄

static を破棄

b は内側のブロックを抜ける際に破棄され、a は関数 func の終了時に破棄される。これに対して、静的オブジェクト s は関数 func の終了時には破棄されず、プログラムの終了時まで存在する。

また、new によって動的に生成された Sample オブジェクトは、delete p; が実行された時点で破棄される。このように、オブジェクトの生成や記憶期間の形態によって、その寿命が終了する時点や破棄の仕組みは異なる。

14.2 オブジェクトの所有権

前節では、オブジェクトが生成されてから破棄されるまでの期間である寿命について説明した。オブジェクトを安全に扱うためには、その寿命だけでなく、そのオブジェクトを誰が管理し、不要になったときに誰が破棄するかを明確にする必要がある。

オブジェクトを管理し、不要になったときにその寿命を終える責任を持つことを、本書では「そのオブジェクトの**所有権**を持つ」と表現する。また、所有権を持つものを、「そのオブジェクトの**所有者**」と表現する。一方、オブジェクトを一時的に利用するだけで、その破棄には責任を持たないものもある。従って、オブジェクトを指すポインタや参照があるからといって、それが必ずしも所有権を持つとは限らない。

所有権が明確でないと、動的に生成されたオブジェクトが適切に破棄されずに残るメモリリークや、同じオブジェクトを複数回破棄する二重解放、寿命を終えたオブジェクトへのアクセスなどの問題が発生する可能性がある。C++ では、オブジェクトを所有する場合と、所有せずに利用する場合とを区別し、所有権の関係を明確に設計することが重要である。

以下では、まず所有する場合と所有しない場合の違いについて説明する。次に、一つのオブジェクトを一つの所有者だけが管理する**単独所有**と、複数の所有者が共同で管理する**共有所有**について説明する。さらに、所有権を別の所有者へ引き渡す**所有権の移動**について解説する。

14.2.1 所有と非所有

オブジェクトを所有するものは、そのオブジェクトの寿命を管理し、最終的に破棄する責任を持つ。これに対して、既に存在するオブジェクトを一時的に利用するだけで、その破棄には責任を持たないことを、本書では**非所有**という。

通常のローカル変数として生成されたオブジェクトでは、その寿命は変数のスコープに従って自動的に管理される。例えば、次の処理では、std::string 型のローカルオブジェクト s が生成される。

```
std::string s = "Hello";
```

このオブジェクトは、s が属するブロックを抜ける際に自動的に破棄される。従って、プログラマが明示的に破棄の処理を記述する必要はない。

クラスのメンバとして保持されるオブジェクトについても同様である。例えば、次のクラスでは、Student 型のオブジェクトが、メンバ name のオブジェクトを所有する。

```
class Student {  
private:  
    std::string name;  
};
```

Student 型のオブジェクトが破棄されると、そのメンバである name も自動的に破棄される。このように、あるオブジェクトが別のオブジェクトをメンバとして直接保持する場合には、外側のオブジェクトが内側のオブジェクトを所有する関係になる。これは、先に説明した**コンポジション**における典型的な所有関係である。

14.2.1.1 ポインタと所有権

ポインタは、オブジェクトが格納されているアドレスを保持する。しかし、ポインタがオブジェクトを指していることと、そのオブジェクトを所有していることは同じではない。例えば、次の処理を考える。

```
int n = 10;  
int* p = &n;
```

この場合、p は n を指しているが、n を所有してはいない。n は通常のローカル変数であり、その属するブロックを抜ける際に自動的に破棄される。従って、次のようにして、p を用いて n を破棄してはならない。

```
delete p;    // 誤り
```

この p は、n を一時的に指して利用するための非所有ポインタである。

一方、次の処理では、new 演算子によって動的にオブジェクトが生成される。

```
int* p = new int(10);
```

この場合、生成されたオブジェクトは自動的に破棄されないため、最終的に、次のように、delete を実行する必要がある。

```
delete p;
```

このようなコードでは、p を、動的に生成されたオブジェクトを管理し、その破棄に責任を持つポインタとして扱っている。すなわち、p は所有権を表すポインタとして使用されている。ただし、通常のポインタ型そのものには、所有権を表したり、自動的に管理したりする機能はない。そのポインタが所有権を持つかどうかは、プログラムの設計や使用方法によって決まる。

14.2.1.2 所有権が不明確なポインタ

通常のポインタだけを用いて動的なオブジェクトを管理すると、どのポインタが所有者であるかが不明確になりやすい。例えば、次の処理を考える。

```
int* p = new int(10);
int* q = p;
```

p と q は、同じオブジェクトを指している。しかし、このコードだけでは、どちらがそのオブジェクトを破棄する責任を持つかは明確でない。両方で delete を実行する（下記）と、同じオブジェクトを二度解放しようとする二重解放となり、プログラムの動作は未定義となる。

```
delete p;
delete q;    // 誤り
```

一方、どちらも delete を実行しなければ、メモリリークが発生する。このように、通常のポインタでは、所有権の有無が型から判別できない。従って、所有権を持つポインタと、所有せずに指して利用するだけのポインタとを、設計上明確に区別する必要がある。

14.2.1.3 参照による非所有の利用

参照は、既に存在するオブジェクトに別の名前を与えて利用するための仕組みであり、通常は所有権を持たない。例えば、次の関数 print は、受け取った文字列を表示するだけで、その文字列を所有しない。

```
void print(const std::string& s) {
    std::cout << s << "\n";
}
```

次のようにこの関数を呼び出しても、print 関数がオブジェクト message を破棄することはない。

```
std::string message = "Hello";
print(message);
```

この場合、message は std::string 型のオブジェクトであり、print 関数はそのオブジェクトを一時的に参照しているだけである。関数呼び出しが終了した後も、message はそのまま存在する。

非所有のポインタや参照を使用する場合には、参照先のオブジェクトが、それらを利用している間、確実に存在している必要がある。例えば、次のようにローカルオブジェクトへの参照を関数の外部へ返すと、参照先の寿命が先に終了する。

```
const std::string& func() {
    std::string s = "Hello";
    return s;    // 誤り
}
```

s は関数 func の終了時に破棄されるため、返された参照は既に存在しないオブジェクトを参照するダングリング参照となる。所有権を持たないポインタや参照を保持する場合には、参照先となるオブジェクトの寿命との関係に注意しなければならない。

14.2.1.4 関数の引数と所有権

関数がオブジェクトを受け取る方法には、値、参照、ポインタなどがある。これらは、所有権に関して異なる意味を持つ場合がある。例えば、次の関数 `func1` は、値としてオブジェクトを受け取る。

```
void func1(std::string s) {  
    // s を使用する  
}
```

関数呼び出しの際には、引数として渡された値から仮引数 `s` のオブジェクトが生成される。このオブジェクトは `func1` 関数のローカルオブジェクトとして管理され、関数の終了時に自動的に破棄される。

これに対して、次の関数は参照で受け取るため、呼び出し元のオブジェクトを所有しない。

```
void func2(const std::string& s) {  
    // s を参照して使用する  
}
```

また、ポインタで受け取る場合も、通常は呼び出し元で管理されているオブジェクトを指して利用する非所有の関係として設計される。

```
void func3(const std::string* p) {  
    if (p != nullptr)  
        std::cout << *p << "\n";  
}
```

ただし、通常のポインタ型だけでは、所有権を持つかどうかを型から判断できない。従って、関数の仕様や説明において、そのポインタが所有権を受け取るのか、単に参照するだけなのかを明確にする必要がある。後に説明するスマートポインタを利用すると、所有権の関係を型によって、より明確に表すことができる。

以上のように、オブジェクトを利用するものが、必ずしもそのオブジェクトを所有しているとは限らない。所有権を持つものはオブジェクトの寿命を管理し、非所有のポインタや参照は、自身ではその寿命を管理せず、既に存在するオブジェクトを利用する。安全なプログラムを作成するためには、コードの中で所有と非所有の関係を明確にし、非所有のポインタや参照を使用している間に、参照先のオブジェクトの寿命が終了しないように設計することが重要である。

14.2.2 単独所有と共有所有

オブジェクトの所有権の持ち方には、一つのオブジェクトを一つの所有者だけが管理する場合と、複数の所有者が共同で管理する場合がある。前者を**単独所有**、後者を**共有所有**という。

単独所有では、ある時点において、一つのオブジェクトの所有者は一つだけである。従って、そのオブジェクトを最終的に破棄する責任を持つものが明確である。一方、共有所有では、複数の所有者が同じオブジェクトの寿命を共同で管理する。C++の共有所有の仕組みでは、最後の所有者が所有権を手放した時点で、そのオブジェクトが破棄される。

14.2.2.1 単独所有

単独所有では、オブジェクトを管理する所有者が一つに限定される。例えば、次のように、あるポインタ `p` が、動的に生成されたオブジェクトを所有しているものとする。

```
int* p = new int(10);
```

この場合、プログラムの設計として、`p` をそのオブジェクトの所有者とし、最終的に `delete` を実行する責任を持たせることができる。(下記)

```
delete p;
```

この設計では、オブジェクトを破棄する責任を持つ所有者が一つであるため、所有関係を理解しやすい。また、同じオブジェクトを複数回破棄してしまう問題も避けやすい。従って、オブジェクトを複数の所有者で共同管理する必要がない場合には、単独所有を基本とするのが望ましい。

ただし、通常のポインタだけでは、そのポインタが単独所有を表していることを型から判別できない。また、次のようにポインタを別のポインタへ代入すると、どちらが所有者であるかが不明確になる。

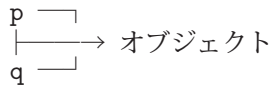
```
int* p = new int(10);  
int* q = p;
```

このとき、`q` が単にオブジェクトを指して利用しているだけなのか、それとも所有権が `p` から `q` へ移ったのかは、このコードだけでは判断できない。単独所有を安全に表すためには、所有者を一つに保ち、所有権の移動を明確に行う仕組みが必要である。この仕組みについては、次節で説明する。

14.2.2.2 共有所有

一つのオブジェクトを複数の場所から利用し、そのいずれもがオブジェクトの寿命を維持する必要がある場合には、共有所有を用いることがある。共有所有では、複数の所有者が同じオブジェクトを共同で管理する。

例えば、概念的に、あるオブジェクトを二つの所有者 `p` と `q` が共有している場合を考える。



このとき、`p` が所有権を手放しても、`q` が所有している間は、そのオブジェクトを破棄してはならない。`p` と `q` の両方が所有権を手放し、所有者が一つもなくなった時点で、そのオブジェクトが破棄される。このような管理を通常のポインタだけで実現するには、現在の所有者の数を記録し、所有者が増減するたびにその数を更新する必要がある。さらに、所有者の数が 0 になったときに、オブジェクトを破棄しなければならない。この管理を手作業で行うと、数え間違いや解放忘れなどの問題が発生しやすい。

後に説明する `std::shared_ptr` は、このような共有所有を実現するスマートポインタである。`std::shared_ptr` では、同じオブジェクトを共有所有しているスマートポインタの数が内部で管理され、最後の所有者が所有権を手放したときに、そのオブジェクトが自動的に破棄される。

14.2.2.3 単独所有と共有所有の使い分け

単独所有は、所有者が一つであるため、オブジェクトの寿命を追跡しやすく、破棄の責任も明確である。また、共有所有のように所有者の数を管理する必要がないため、仕組みも単純である。従って、複数の所有者が必要であることが明確でない場合には、単独所有を選ぶのが基本である。

共有所有は、同じオブジェクトを複数の構成要素が利用し、それぞれがそのオブジェクトの寿命を維持する必要がある場合に用いる。ただし、単に複数の場所からオブジェクトを利用するだけであれば、必ずしも共有所有にする必要はない。一つの所有者がオブジェクトを管理し、他の場所では非所有のポインタや参照を用いて利用する設計も可能である。

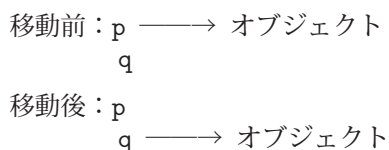
例えば、あるオブジェクトを一つの管理オブジェクトが所有し、複数の関数とそのオブジェクトを一時的に利用する場合、所有者は一つのままでよい。複数の利用者が存在することと、複数の所有者が必要であることは同じではない。

従って、所有権を設計する際には、まず単独所有が可能かどうかを検討し、オブジェクトの寿命を複数の所有者によって共同で維持する必要がある場合にのみ、共有所有を選ぶのが望ましい。

14.2.3 所有権の移動

単独所有では、ある時点において、一つのオブジェクトの所有者は一つだけである。しかし、プログラムの処理に応じて、その所有権を別の所有者へ引き渡す必要が生じることがある。このように、ある所有者が持っている所有権を別の所有者へ引き渡すことを、**所有権の移動**という。

所有権が移動すると、移動前の所有者はそのオブジェクトを所有しなくなり、移動後の所有者が、そのオブジェクトの寿命を管理して破棄する責任を持つ。従って、所有権の移動によって所有者が増えるのではなく、所有者が別のものへ交代する。以下は、所有権の移動を概念的に表したものである。



なお、この図の `p` と `q` は所有者を概念的に表したものであり、通常のポインタを単に代入するだけで所有権が自動的に移動するわけではない。

このような関係では、移動後の所有者 `q` がオブジェクトを破棄する責任を持ち、移動前の所有者 `p` は、そのオブジェクトを破棄してはならない。

14.2.3.1 通常のポインタによる所有権の移動

通常のポインタを用いて所有権を移動する場合には、所有関係をプログラマが明示的に管理しなければならない。例えば、次の処理を考える。

```
int* p = new int(10);
int* q = p;
p = nullptr;
```

この処理では、まず `p` を、動的に生成されたオブジェクトの所有者として扱っている。次に、そのアドレスを `q` に代入し、`q` を新しい所有者とする。その後、`p` に `nullptr` を代入することで、`p` がそのオブジェクトを指さない状態になっている。このように、プログラム上の取り決めとして、所有権を `p` から `q` へ移したものと扱うことができる。移動後は、`q` がオブジェクトを破棄する責任を持つ。(下記)

```
delete q;
```

ただし、この処理では、所有権が実際に移動したことをポインタ型そのものが保証しているわけではない。例えば、`p` に `nullptr` を代入し忘れると、`p` と `q` の両方が同じオブジェクトを指したままになる。

```
int* p = new int(10);
int* q = p;
```

この状態では、`p` と `q` のどちらを所有者として扱うのかがコードから分かりにくくなり、二重解放や解放忘れの原因となる。

14.2.3.2 コピーと移動の違い

オブジェクトを別の変数へ代入する場合には、コピーと移動を区別する必要がある。コピーでは、元のオブジェクトを残したまま、別のオブジェクトが生成される。例えば、次の処理では、`s1` の内容が `s2` にコピーされる。

```
std::string s1 = "Hello";
std::string s2 = s1;
```

この場合、処理の後も `s1` と `s2` の両方が存在し、それぞれ独立した `std::string` 型のオブジェクトとして扱われる。

これに対して、移動では、元のオブジェクトが管理している資源などを、新しいオブジェクトへ引き渡す。C++では、移動することを明示するために、`std::move` を使用することがある。

```
std::string s1 = "Hello";
std::string s2 = std::move(s1);
```

この処理では、`s1` が管理していた内部資源などが、`s2` へ効率的に引き継がれる場合がある。移動では、コピーのように内容を一から複製する代わりに、既に管理している資源を引き継ぐ処理が行われることがあるため、大きなオブジェクトでは効率のよい処理になることがある。

`std::move` は、それ自体がオブジェクトの内容を移動する関数ではない。`std::move` は、指定したオブジェクトを、移動の対象として扱える値に変換する。実際にどのような処理によって資源が移されるかは、その型に定義されたムーブコンストラクタやムーブ代入演算子によって決まる。

14.2.3.3 移動後のオブジェクト

オブジェクトを移動した後も、移動元のオブジェクト自体が直ちに破棄されるわけではない。移動元のオブジェクトはそのまま存在し、後にデストラクタによって破棄される。ただし、移動前と同じ値や状態を保持しているとは限らない。

例えば、先の処理における `s1` は、`std::move` の実行後も存在するが、その内容が空文字列になるとは限らない。移動後のオブジェクトは、破棄したり、新しい値を代入したりできる有効な状態に保たれるが、その具体的な値や状態は型の仕様によって異なる。従って、移動後のオブジェクトの内容を、移動前と同じであると仮定して使用してはならない。

```
std::string s1 = "Hello";
std::string s2 = std::move(s1);
s1 = "World";    // 新しい値を代入できる
```

このように、移動元のオブジェクトは、必要に応じて新しい値を代入して再利用できる。

14.2.3.4 所有権の移動と単独所有

所有権を移動する仕組みは、単独所有を維持するために重要である。例えば、動的に生成したオブジェクトを関数へ引き渡したり、クラスのメンバへ格納したりする場合には、所有権を元の場所から新しい場所へ移すことができる。

このとき、移動元は所有権を失い、移動先だけが所有者となる。この関係が型によって表現されれば、所有権を誤って複製することを防ぎ、破棄の責任を明確にできる。後に説明する `std::unique_ptr` は、このような単独所有と所有権の移動を表すスマートポインタである。

`std::unique_ptr` はコピーできないが、`std::move` を用いて所有権を別の `std::unique_ptr` へ移すことができる。

```
auto p = std::make_unique<int>(10);
auto q = std::move(p);
```

この処理の後、`q` が動的に生成された整数オブジェクトを所有し、`p` はそのオブジェクトを所有しない状態になる。このように、所有権の移動を型によって明確に表すことで、単独所有の関係を安全に維持できる。`std::unique_ptr` と `std::move` を用いた具体的な処理については、後のスマートポインタの節で詳しく説明する。

14.3 RAII

C++では、メモリ領域、ファイル、通信接続、排他制御のためのロックなど、プログラムの実行中に確保して使用し、不要になったときに解放する必要があるものを、広い意味で**資源**という。資源を使用する場合には、その獲得と解放を適切に対応させなければならない。

例えば、動的に確保したメモリ領域は、使用後に解放する必要がある。また、開いたファイルは閉じ、取得したロックは解除しなければならない。これらの解放処理を忘れると、メモリリーク、ファイル記述子の不足、デッドロックなどの問題が発生する可能性がある。

C++では、資源の獲得をオブジェクトの初期化時に行い、資源の解放をオブジェクトの破棄時に行う設計方法が広く用いられている。この設計方法を **RAII** という。RAII は、*Resource Acquisition Is Initialization* の略であり、「資源の獲得を初期化として扱う」という意味である。

RAII では、コンストラクタで資源を獲得し、デストラクタでその資源を解放する。これにより、資源の寿命を、その資源を管理するオブジェクトの寿命に結び付けることができる。オブジェクトがスコープを抜けて破棄されると、デストラクタが自動的に呼び出され、資源も自動的に解放される。

14.3.1 RAII による資源管理

RAII を用いるクラスは、コンストラクタで資源を獲得し、デストラクタで解放する。例えば、次のクラスは、動的に生成した整数オブジェクトを管理する。

```
class Number {
private:
    int* value;
public:
    explicit Number(int n) : value(new int(n)) {}
    ~Number() {
        delete value;
    }
};
```

このクラスでは、コンストラクタが実行されると、`new` 演算子によって整数オブジェクトが動的に生成される。また、`Number` 型のオブジェクトが破棄されると、デストラクタが呼び出され、`delete` によって整数オブジェクトが破棄される。

注意) この例の `Number` クラスは、RAII の基本的な仕組みを示すために簡略化したものである。このクラスのオブジェクトをそのままコピーすると、内部のポインタ値もそのままコピーされるため、複数のオブジェクトが同じメモリ領域を解放しようとする問題が生じる。資源を所有するクラスにおけるコピーや移動の設計については、後の Rule of Three/Five/Zero で説明する。

次のように関数内でローカルオブジェクトを生成すると、関数の終了時にデストラクタが自動的に呼び出される。


```
void func() {
    Number n(10);
}
```

この場合、プログラマが関数の末尾で明示的に `delete` を実行する必要はない。 `Number` 型のオブジェクト `n` が破棄される際に、内部で管理している資源も自動的に解放される。

このように、資源の解放処理をデストラクタに記述すると、資源管理をオブジェクト自身に担当させることができる。資源を使用する側は、そのオブジェクトを通常のローカル変数として生成するだけでよく、解放処理を個別に記述する必要がなくなる。

14.3.2 複数の終了経路と RAII

関数には、通常の末尾から終了する場合だけでなく、`return` 文によって途中で終了する場合や、例外が送出されてブロックから抜ける場合がある。資源の解放処理を関数の末尾に直接記述していると、終了経路ごとに解放処理を記述しなければならない。

```
void func(bool flag) {
    int* p = new int(10);
    if (flag) {
        delete p;
        return;
    }
    // 処理
    delete p;
}
```

このようなコードでは、処理の分岐が増えるたびに、解放処理の記述漏れが発生しやすくなる。また、`delete` の実行前に例外が送出された場合には、資源が解放されない可能性もある。

これに対して、RAII を用いると、ブロックを抜ける際にローカルオブジェクトのデストラクタが自動的に呼び出される。従って、`return` 文や例外によって途中で処理が終了した場合でも、デストラクタが呼び出され、資源が自動的に解放される。

```
void func(bool flag) {
    Number n(10);
    if (flag) return;
    // 処理
}
```

この場合、どの経路で関数を終了しても、`n` のデストラクタが呼び出される。この性質により、解放処理を終了経路ごとに繰り返して記述する必要がなくなり、資源の解放忘れを防ぎやすくなる。

14.3.3 ファイルの管理と RAII

RAII は、動的なメモリ領域だけでなく、様々な資源の管理に利用できる。例えば、C 言語のファイル入出力では、`fopen` 関数によって開いたファイルを、`fclose` 関数によって閉じる必要がある。この処理をクラスにまとめると、ファイルを自動的に閉じることができる。

```
class File {
private:
    std::FILE* fp;
public:
    File(const char* name, const char* mode) {
        fp = std::fopen(name, mode);
    }
    ~File() {
        if (fp != nullptr)
            std::fclose(fp);
    }
    std::FILE* get() {
        return fp;
    }
};
```


このクラスでは、コンストラクタでファイルを開き、デストラクタでファイルを閉じる。

注意 この例の `File` クラスは、RAII の基本的な仕組みを示すために簡略化したものである。先の `Number` クラスと同様に、オブジェクトをそのままコピーすると、複数のオブジェクトが同じ資源を解放しようとする問題が生じる。また、実際のプログラムでは、ファイルを開く処理に失敗した場合の処理も必要である。

次の関数では、`File` 型のオブジェクトが破棄される際に、ファイルが自動的に閉じられる。

```
void write_data() {
    File file("data.txt", "w");
    if (file.get() != nullptr)
        std::fprintf(file.get(), "Hello\n");
}
```

このように、資源の管理処理をクラス内部にまとめると、資源を使用する側では、獲得と解放の対応を個別に意識する必要が少なくなる。

14.3.4 標準ライブラリと RAII

C++ の標準ライブラリには、RAII の考え方に基づいて設計されたクラスが多く用意されている。例えば、`std::string` や `std::vector`¹⁵⁷ は、内部で使用するメモリ領域を自動的に管理する。これらのオブジェクトが破棄されると、内部で使用されていたメモリ領域も自動的に解放される。

```
void func() {
    std::string s = "Hello";
    std::vector<int> v(100);
}
```

この例では、関数の終了時に `s` と `v` が破棄され、それぞれが管理している内部資源も自動的に解放される。プログラマが、それらの内部で使用されているメモリ領域を個別に解放する必要はない。

入出力ストリームも RAII を利用している。例えば、`std::ofstream` 型のオブジェクトは、生成時にファイルを開き、破棄される際にファイルを閉じる。

```
void write_data() {
    std::ofstream ofs("data.txt");
    ofs << "Hello\n";
}
```

また、後に説明するスマートポインタは、動的に生成されたオブジェクトを RAII によって管理するためのクラスである。さらに、並行処理で使用する `std::lock_guard` は、生成時にミューテックスをロックし、破棄時にロックを解除する。

このように、C++ では、資源を直接管理する処理を各所に記述するのではなく、RAII に基づくクラスに資源管理を担当させる設計が基本となる。

14.3.5 RAII を利用する際の基本方針

RAII を利用する際には、資源を管理するオブジェクトを、できるだけ通常のローカル変数やクラスのメンバとして保持するのが望ましい。これにより、オブジェクトの寿命に応じて、資源を自動的に解放できる。

また、動的なメモリ領域を直接 `new` と `delete` で管理するよりも、`std::vector`、`std::string`、スマートポインタなど、既に RAII に対応している標準ライブラリのクラスを利用の方が安全である。

RAII の基本的な考え方は、次のようにまとめられる。

- 資源の獲得を、オブジェクトの生成時に行う。
- 資源の解放を、デストラクタによって行う。
- 資源の寿命を、管理するオブジェクトの寿命に結び付ける。
- 資源管理を、RAII に対応したクラスへまとめる。

この設計により、資源の解放忘れを防ぎ、複数の終了経路や例外が存在する場合でも、資源を安全に管理しやすくなる。

¹⁵⁷配列よりも柔軟なコンテナである。詳しくは後の「20 STL の基礎」(p.384～)で解説する。

14.4 スマートポインタ

`new` 演算子によって動的に生成されたオブジェクトは、通常のローカル変数のように自動的に破棄されない。従って、通常のポインタを用いて管理する場合には、適切な時点で `delete` を実行しなければならない。しかし、所有者が不明確であったり、処理の途中で関数から戻ったり、例外が送出されたりすると、解放忘れや二重解放などの問題が発生しやすい。

C++の標準ライブラリには、動的に生成されたオブジェクトの所有権を管理するための**スマートポインタ** が用意されている。スマートポインタは、ポインタに似た操作が可能なクラスであり、所有権の状態に応じて、管理しているオブジェクトを自動的に破棄する機能を持つ。

スマートポインタは RAII の考え方に基づいている。動的に生成されたオブジェクトをスマートポインタに管理させると、その所有権をスマートポインタによって管理できる。また、スマートポインタが所有権を失う際には、所有権の状態に応じて、管理しているオブジェクトが自動的に破棄される。従って、通常はプログラマが明示的に `delete` を実行する必要がない。スマートポインタを使用するには、ヘッダファイル `memory` を読み込む。

標準ライブラリには、所有権の関係に応じて、主に次の3種類のスマートポインタが用意されている。

- `std::unique_ptr` : 一つのオブジェクトを単独所有する。
- `std::shared_ptr` : 一つのオブジェクトを複数の所有者で共有所有する。
- `std::weak_ptr` : 共有所有されているオブジェクトを、所有せずに参照する。

これらは同じ目的で使い分けるものではなく、所有権の関係に応じて選択する必要がある。原則として、所有者を一つにできる場合には `std::unique_ptr` を使用し、複数の所有者がオブジェクトの寿命を共同で維持する必要がある場合にのみ `std::shared_ptr` を使用する。`std::weak_ptr` は、共有所有に参加せずにオブジェクトを参照する必要がある場合に使用する。

14.4.1 単独所有のための `unique_ptr`

`std::unique_ptr` は、動的に生成されたオブジェクトを単独所有するためのスマートポインタである。一つのオブジェクトを所有できる `std::unique_ptr` は、一つだけである。従って、オブジェクトを破棄する責任を持つ所有者が明確になる。

`std::unique_ptr` 型のオブジェクトは、次のように `std::make_unique` を用いて生成できる。

```
std::unique_ptr<int> p = std::make_unique<int>(10);
```

この処理では、値 10 を持つ `int` 型のオブジェクトが動的に生成され、その所有権が `p` に与えられる。`p` を介して、通常のポインタと同様に、間接参照演算子 `*` を用いて値にアクセスできる。

```
std::cout << *p << "\n";
```

`p` が属するブロックを抜けると、`p` のデストラクタが呼び出され、所有している整数オブジェクトも自動的に破棄される。従って、次のような `delete` の記述は必要ない。

```
delete p; // 誤り
```

`p` は通常のポインタではなく、`std::unique_ptr<int>` 型のオブジェクトであるため、`delete` の対象にはできない。

クラス型のオブジェクトも同様に管理できる。例えば、次の処理では、`Sample` 型のオブジェクトを動的に生成し、その所有権を `p` に持たせている。

```
auto p = std::make_unique<Sample>(10);
```

メンバへアクセスする場合には、通常のポインタと同様に、アロー演算子 `->` を使用できる。

```
p->show();
```

`std::make_unique` を使用すると、`new` を直接記述せずに、動的なオブジェクトとそれを所有する `std::unique_ptr` を生成できる。通常は、次のように `new` を直接用いるよりも、`std::make_unique` を使用の方が望ましい。

```
std::unique_ptr<int> p(new int(10));
```

後者の記述も可能であるが、所有権の受け渡しを明確にし、記述を簡潔にするため、基本的には `std::make_unique` を使用する。

14.4.1.1 unique_ptr による所有権の移動

std::unique_ptr は単独所有を表すため、通常のコピーはできない。例えば、次の処理はコンパイルエラーとなる。

```
auto p = std::make_unique<int>(10);
auto q = p;    // 誤り：コピーできない
```

このコピーを許すと、p と q の両方が同じオブジェクトの所有者となり、単独所有の関係が崩れるためである。

所有権を別の std::unique_ptr へ引き渡す場合には、std::move を使用する。

```
auto p = std::make_unique<Sample>(10);
auto q = std::move(p);
```

この処理によって、所有権が p から q へ移動する。移動後は、q が Sample 型のオブジェクトを所有し、p は何も所有しない空のスマートポインタとなる。

```
if (p == nullptr) std::cout << "p は何も所有していません\n";
```

このように、std::unique_ptr ではコピーを禁止し、移動だけを許可することによって、所有者が一つであることを型によって保証している。

14.4.1.2 unique_ptr を関数へ渡す方法

std::unique_ptr を関数へ値として渡す場合には、その関数へ所有権を移動することになる。例えば、次の関数は、Sample 型のオブジェクトの所有権を受け取る。

```
void receive(std::unique_ptr<Sample> p) {
    p->show();
}
```

この関数を呼び出す際には、std::move を使用して所有権を渡す。

```
auto p = std::make_unique<Sample>(10);
receive(std::move(p));
```

関数呼び出し後、呼び出し元の p は何も所有しない状態になる。一方、関数内の仮引数 p がオブジェクトを所有し、関数の終了時に p が破棄されると、所有しているオブジェクトも自動的に破棄される。

関数がオブジェクトを利用するだけで、所有権を受け取る必要がない場合には、std::unique_ptr 自体を値として渡す必要はない。通常の参照やポインタを用いて、所有しない形で渡すことができる。

```
void use(const Sample& obj) {
    obj.show();
}
```

呼び出し側では、std::unique_ptr が指すオブジェクトを間接参照して渡す。

```
auto p = std::make_unique<Sample>(10);
use(*p);
```

この場合、所有権は p に残ったままであり、use 関数はオブジェクトを一時的に参照するだけである。

■ サンプルプログラム：unique_ptr

unique_ptr による単独所有、所有権の移動、および関数への所有権の受け渡しを確認するサンプルプログラムを次に示す。

プログラム：test213-1.cpp

```
1  #include    <iostream>
2  #include    <memory>
3  #include    <utility>
4
5  class Sample {
6  private:
7      int value;
8
9  public:
10     explicit Sample(int n) : value(n) {
11         std::cout << "Sampleオブジェクトを生成しました\n";
12     }
13
14     ~Sample() {
```

```

15         std::cout << "Sampleオブジェクトを破棄しました\n";
16     }
17
18     void show() const {
19         std::cout << "value = " << value << "\n";
20     }
21 };
22
23 void use(const Sample& obj) {
24     std::cout << "use関数で利用します\n";
25     obj.show();
26 }
27
28 void receive(std::unique_ptr<Sample> p) {
29     std::cout << "receive関数が所有権を受け取りました\n";
30     p->show();
31 }
32
33 int main() {
34     std::cout << "1. unique_ptrによる単独所有\n";
35
36     auto p = std::make_unique<Sample>(10);
37
38     p->show();
39     use(*p);
40
41     std::cout << "\n2. 所有権の移動\n";
42
43     auto q = std::move(p);
44
45     if (p == nullptr) std::cout << "pは何も所有していません\n";
46
47     q->show();
48
49     std::cout << "\n3. 関数への所有権の受け渡し\n";
50
51     receive(std::move(q));
52
53     if (q == nullptr) std::cout << "qは何も所有していません\n";
54
55     std::cout << "\nmain関数を終了します\n";
56 }

```

このプログラムでは、整数値を保持する `Sample` クラスを定義し、オブジェクトの生成時と破棄時にメッセージを表示する。これにより、`unique_ptr` が管理しているオブジェクトが破棄される時点を確認できる。

`main` 関数では、まず `make_unique` によって `Sample` 型のオブジェクトを生成し、スマートポインタ `p` に単独所有させる。`p` からは、通常のポインタと同様に、`Sample` オブジェクトのメンバ関数を呼び出すことができる。また、`use` 関数には `Sample` オブジェクトへの参照だけを渡しているため、所有権は `p`に残ったままである。

次に、`move` を用いて、`p` が持っている所有権を `q` へ移動する。移動後は、`q` が `Sample` オブジェクトを所有し、`p` は何も所有しない空の状態となる。

最後に、`q` の所有権を `receive` 関数へ移動する。この関数では、仮引数として受け取った `unique_ptr` が `Sample` オブジェクトを所有する。関数の終了時に仮引数が破棄されると、それが所有している `Sample` オブジェクトも自動的に破棄される。従って、関数呼び出し後の `q` も、何も所有しない空の状態となる。

このプログラムをコンパイルして実行する例を次に示す。

例. test213-1.cpp のコンパイルと実行

```
$ g++ test213-1.cpp -o test213-1 Enter ←コンパイル
```

```
$ ./test213-1 Enter ←実行
```

1. unique_ptr による単独所有

Sample オブジェクトを生成しました

value = 10

use 関数で利用します

value = 10

2. 所有権の移動

p は何も所有していません

value = 10

3. 関数への所有権の受け渡し

receive 関数が所有権を受け取りました

value = 10

Sample オブジェクトを破棄しました

q は何も所有していません

main 関数を終了します

14.4.2 共有所有のための shared_ptr

std::shared_ptr は、一つのオブジェクトを複数の所有者で共有所有するためのスマートポインタである。

std::shared_ptr は、同じオブジェクトを共有所有している std::shared_ptr の数を内部で管理する。この数を参照カウントという。

std::shared_ptr 型のオブジェクトは、通常、std::make_shared を用いて生成する。

```
std::shared_ptr<int> p = std::make_shared<int>(10);
```

この処理では、値 10 を持つ整数オブジェクトが動的に生成され、p がそのオブジェクトを所有する。std::shared_ptr はコピーすることができ、コピーすると所有者が増える。

```
std::shared_ptr<int> q = p;
```

この時点では、p と q の両方が同じ整数オブジェクトを所有している。一方の std::shared_ptr が破棄されても、もう一方が所有している間は、整数オブジェクトは破棄されない。

例えば、次の処理では、内側のブロックで q が生成される。

```
auto p = std::make_shared<int>(10);
{
    auto q = p;
    std::cout << *q << "\n";
}
```

内側のブロックを抜けると q は破棄されるが、p が所有権を保持しているため、整数オブジェクトはまだ破棄されない。その後、p も破棄され、最後の所有者がなくなった時点で、整数オブジェクトが自動的に破棄される。

現在の所有者の数は、use_count メンバ関数によって確認できる。

```
std::cout << p.use_count() << "\n";
```

ただし、通常のプログラムでは、use_count の値に依存して処理を制御するべきではない。この関数は、動作の確認やデバッグなどに使用するものとするのがよい。

14.4.2.1 shared_ptr による所有権の解放

std::shared_ptr 型のオブジェクトが破棄されると、そのオブジェクトが持っていた所有権が一つ失われる。また、reset メンバ関数を使用して、明示的に所有権を手放すこともできる。

```
auto p = std::make_shared<int>(10);
auto q = p;
p.reset();
```

この処理では、p が所有権を手放すが、q が所有しているため、整数オブジェクトは破棄されない。その後、q も所有権を手放し、最後の所有者がなくなった時点で、整数オブジェクトが自動的に破棄される。

```
q.reset();
```


reset を実行したスマートポインタは、何も所有しない空の状態になる。

■ サンプルプログラム：shared_ptr

shared_ptr による共有所有、参照カウントの変化、および所有権の解放を確認するサンプルプログラム test213-2.cpp を次に示す。

プログラム：test213-2.cpp

```
1 #include <iostream>
2 #include <memory>
3
4 class Sample {
5     private:
6         int value;
7
8     public:
9         explicit Sample(int n) : value(n) {
10             std::cout << "Sampleオブジェクトを生成しました\n";
11         }
12
13         ~Sample() {
14             std::cout << "Sampleオブジェクトを破棄しました\n";
15         }
16
17         void show() const {
18             std::cout << "value = " << value << "\n";
19         }
20 };
21
22 int main() {
23     std::cout << "1. shared_ptrによる共有所有\n";
24
25     auto p = std::make_shared<Sample>(20);
26     std::cout << "所有者数：" << p.use_count() << "\n";
27     p->show();
28
29     {
30         std::cout << "\n2. shared_ptrのコピー\n";
31
32         auto q = p;
33         std::cout << "所有者数：" << p.use_count() << "\n";
34         q->show();
35
36         {
37             auto r = q;
38             std::cout << "\n3. さらに所有者を追加\n";
39             std::cout << "所有者数：" << p.use_count() << "\n";
40             r->show();
41         }
42
43         std::cout << "\nrが破棄された後\n";
44         std::cout << "所有者数：" << p.use_count() << "\n";
45
46         q.reset();
47         std::cout << "\nqが所有権を手放した後\n";
48         std::cout << "所有者数：" << p.use_count() << "\n";
49
50         if (q == nullptr) std::cout << "qは何も所有していません\n";
51     }
52
53     std::cout << "\n4. 最後の所有者による管理\n";
54     std::cout << "所有者数：" << p.use_count() << "\n";
55
56     p.reset();
57
58     if (p == nullptr) std::cout << "pは何も所有していません\n";
59
60     std::cout << "\nmain関数を終了します\n";
61 }
```

このプログラムでは、整数値を保持する `Sample` クラスを定義し、オブジェクトの生成時と破棄時にメッセージを表示する。これにより、複数の `shared_ptr` によって共有所有されているオブジェクトが破棄される時点を確認できる。

`main` 関数では、まず `make_shared` によって `Sample` 型のオブジェクトを生成し、スマートポインタ `p` に所有させる。この時点では、所有者は `p` だけであるため、参照カウントは 1 となる。

次に、`p` をコピーしてスマートポインタ `q` を生成する。これにより、`p` と `q` が同じ `Sample` オブジェクトを共有所有し、参照カウントは 2 となる。さらに、内側のブロックで `q` をコピーして `r` を生成すると、所有者が三つになり、参照カウントは 3 となる。

内側のブロックを抜けると `r` が破棄され、参照カウントは 2 に戻る。その後、`q` に対して `reset` メンバ関数を実行すると、`q` は所有権を手放し、参照カウントは 1 となる。この時点では `p` が所有権を保持しているため、`Sample` オブジェクトはまだ破棄されない。

最後に、`p` に対して `reset` メンバ関数を実行すると、最後の所有者である `p` が所有権を手放すため、所有者が一つなくなる。この時点で、共有所有されていた `Sample` オブジェクトが自動的に破棄される。`reset` を実行した `q` は、何も所有しない空のスマートポインタとなる。その後、内側のブロックを抜ける際に、`q` 自身も破棄される。また、最後に `reset` を実行した `p` も、何も所有しない空のスマートポインタとなる。

このプログラムをコンパイルして実行する例を次に示す。

例. `test213-2.cpp` のコンパイルと実行

```
$ g++ test213-2.cpp -o test213-2  ←コンパイル
```

```
$ ./test213-2  ←実行
```

```
1. shared_ptr による共有所有
Sample オブジェクトを生成しました
```

```
所有者数: 1
```

```
value = 20
```

```
2. shared_ptr のコピー
```

```
所有者数: 2
```

```
value = 20
```

```
3. さらに所有者を追加
```

```
所有者数: 3
```

```
value = 20
```

```
r が破棄された後
```

```
所有者数: 2
```

```
q が所有権を手放した後
```

```
所有者数: 1
```

```
q は何も所有していません
```

```
4. 最後の所有者による管理
```

```
所有者数: 1
```

```
Sample オブジェクトを破棄しました
```

```
p は何も所有していません
```

```
main 関数を終了します
```

14.4.3 非所有の参照を表す `weak_ptr`

`std::weak_ptr` は、`std::shared_ptr` によって共有所有されているオブジェクトを、所有権を持たずに参照するためのスマートポインタである。`std::weak_ptr` は共有所有の参照カウントを増加させないため、オブジェクトの寿命を延長しない。

例えば、次のように、`std::shared_ptr` から `std::weak_ptr` を生成できる。

```
auto p = std::make_shared<int>(10);
std::weak_ptr<int> w = p;
```

この場合、`p` は整数オブジェクトを所有するが、`w` は所有しない。従って、`p` が所有権を手放し、他に所有者が存在しなくなると、整数オブジェクトは破棄される。その後も `w` 自体は存在できるが、参照先のオブジェクトは既に存在しない状態となる。

`std::weak_ptr` は、間接参照演算子 `*` やアロー演算子 `->` を用いて直接オブジェクトへアクセスすることはできない。

い。参照先のオブジェクトを使用する場合には、lock メンバ関数を使用して、一時的に std::shared_ptr を取得する。

```
std::shared_ptr<int> q = w.lock();
if (q != nullptr)
    std::cout << *q << "\n";
```

参照先のオブジェクトがまだ存在している場合には、lock はそのオブジェクトを共有所有する std::shared_ptr を返す。この std::shared_ptr が存在している間は、一時的に所有者が一つ増える。既に破棄されている場合には、空の std::shared_ptr を返す。

参照先のオブジェクトが既に破棄されているかどうかは、expired メンバ関数でも確認できる。

```
if (w.expired())
    std::cout << "参照先は存在しない\n";
```

ただし、expired で確認した直後に、別の処理によって最後の所有者が消滅する可能性もある。従って、実際にオブジェクトを使用する場合には、expired と lock を別々に用いるよりも、lock によって取得した std::shared_ptr が空でないことを確認の方が安全である。

■ サンプルプログラム：weak_ptr

weak_ptr による非所有の参照、lock による参照先の取得、および所有者が消滅した後の動作を確認するサンプルプログラム test213-3.cpp を次に示す。

プログラム：test213-3.cpp

```
1  #include <iostream>
2  #include <memory>
3
4  class Sample {
5  private:
6      int value;
7
8  public:
9      explicit Sample(int n) : value(n) {
10         std::cout << "Sampleオブジェクトを生成しました\n";
11     }
12
13     ~Sample() {
14         std::cout << "Sampleオブジェクトを破棄しました\n";
15     }
16
17     void show() const {
18         std::cout << "value = " << value << "\n";
19     }
20 };
21
22 int main() {
23     std::weak_ptr<Sample> w;
24
25     std::cout << "1. weak_ptrによる非所有の参照\n";
26
27     {
28         auto p = std::make_shared<Sample>(30);
29         w = p;
30         std::cout << "shared_ptrの所有者数：" << p.use_count() << "\n";
31
32         if (auto q = w.lock()) {
33             std::cout << "参照先のオブジェクトを利用できます\n";
34             std::cout << "shared_ptrの所有者数：" << p.use_count() << "\n";
35             q->show();
36         }
37
38         std::cout << "lockによる一時的な所有が終了しました\n";
39         std::cout << "shared_ptrの所有者数：" << p.use_count() << "\n";
40
41         if (!w.expired()) std::cout << "参照先のオブジェクトは存在しています\n";
42     }
43 }
```

```

44     std::cout << "\n2. 所有者が消滅した後\n";
45
46     if (w.expired()) std::cout << "参照先のオブジェクトは既に存在しません\n";
47
48     if (auto q = w.lock()) {
49         q->show();
50     }
51     else {
52         std::cout << "参照先のオブジェクトを取得できません\n";
53     }
54
55     std::cout << "\nmain関数を終了します\n";
56 }

```

このプログラムでは、整数値を保持する `Sample` クラスを定義し、オブジェクトの生成時と破棄時にメッセージを表示する。これにより、`weak_ptr` が参照しているオブジェクトの寿命を延長しないことを確認できる。

`main` 関数では、まず、何も参照していない `weak_ptr` 型のスマートポインタ `w` を生成する。その後、内側のブロックで `make_shared` によって `Sample` 型のオブジェクトを生成し、`shared_ptr` 型のスマートポインタ `p` に所有させる。さらに、`p` を `w` に代入し、`w` に同じオブジェクトを非所有で指させる。

`w` は所有権を持たないため、この代入によって参照カウントは増加しない。従って、この時点での所有者は `p` だけであり、所有者数は 1 のままである。

次に、`w` の `lock` メンバ関数を用いて、参照先を所有する `shared_ptr` 型のスマートポインタ `q` を一時的に取得する。参照先のオブジェクトが存在している場合には、`lock` は空でない `shared_ptr` を返す。この間は `p` と `q` が同じオブジェクトを共有所有するため、所有者数は 2 となる。

`if` 文のブロックを抜けると `q` が破棄され、一時的な共有所有が終了する。このため、所有者数は再び 1 となる。また、`expired` メンバ関数によって、`w` の参照先がまだ存在していることを確認できる。

その後、外側のブロックを抜けると、最後の所有者である `p` が破棄され、所有者が一つもなくなる。この時点で `Sample` オブジェクトが自動的に破棄される。`w` 自体は引き続き存在するが、参照先のオブジェクトは既に存在しない状態となる。

最後に、`expired` メンバ関数によって参照先が消滅していることを確認する。この状態で `lock` メンバ関数を実行すると、空の `shared_ptr` が返されるため、参照先のオブジェクトを取得できないことが分かる。

このプログラムをコンパイルして実行する例を次に示す。

例. `test213-3.cpp` のコンパイルと実行

\$ `g++ test213-3.cpp -o test213-3` ←コンパイル

\$ `./test213-3` ←実行

```

1. weak_ptr による非所有の参照
Sample オブジェクトを生成しました
shared_ptr の所有者数: 1
参照先のオブジェクトを利用できます
shared_ptr の所有者数: 2
value = 30
lock による一時的な所有が終了しました
shared_ptr の所有者数: 1
参照先のオブジェクトは存在しています
Sample オブジェクトを破棄しました
2. 所有者が消滅した後
参照先のオブジェクトは既に存在しません
参照先のオブジェクトを取得できません
main 関数を終了します

```

14.4.4 共有所有における循環参照

`std::shared_ptr` を使用する際には、複数のオブジェクトが互いを `std::shared_ptr` によって所有すると、**循環参照** が生じることがある。例えば、二つのオブジェクト `a` と `b` が互いを所有している場合を考える。

a —所有—> b
a <—所有— b

この状態では、外部の所有者がすべて消滅しても、a と b が互いを所有しているため、それぞれの参照カウントが 0 にならない。従って、両方のオブジェクトが破棄されず、メモリリークが発生する。

循環参照を避けるには、所有関係の一方を `std::weak_ptr` にする。例えば、木構造では、親ノードが子ノードを `std::shared_ptr` で所有し、子ノードから親ノードへの参照を `std::weak_ptr` にする設計が考えられる。

```
class Node {  
public:  
    std::vector<std::shared_ptr<Node>> children;  
    std::weak_ptr<Node> parent;  
};
```

この設計では、親ノードが子ノードを所有するが、子ノードは親ノードを所有しない。従って、親子間で循環する所有関係を作らずに、双方向の関係を表すことができる。

14.4.5 スマートポインタを選ぶ際の基本方針

スマートポインタは、所有権の関係に応じて使い分ける必要がある。基本的な選択方針は、次のようにまとめることができる。

- 所有者を一つにできる場合には、`std::unique_ptr` を使用する。
- 複数の所有者が寿命を共同で管理する必要がある場合には、`std::shared_ptr` を使用する。
- 共有所有されているオブジェクトを所有せずに参照する場合には、`std::weak_ptr` を使用する。
- 所有権を持たない一時的な利用には、通常の参照やポインタを使用する。

単に複数の場所からオブジェクトを使用するという理由だけで、`std::shared_ptr` を選ぶべきではない。一つの所有者が管理し、ほかの場所では参照または非所有ポインタを用いて利用できるのであれば、単独所有の方が所有関係を理解しやすい。

また、通常のアプリケーションコードでは、動的なオブジェクトを所有するために、`new` と `delete` を直接記述するよりも、`std::make_unique` や `std::make_shared` を用いてスマートポインタに管理させる方が安全である。スマートポインタを利用すると、オブジェクトの所有関係を型によって表し、RAII によって寿命を自動的に管理できる。

14.5 Rule of Three/Five/Zero

クラスが動的なメモリ領域、ファイル、通信接続などの資源を直接所有する場合には、オブジェクトの破棄だけでなく、コピーや移動が行われたときの資源の扱いも適切に設計する必要がある。

例えば、ポインタ型のデータメンバによって動的なオブジェクトを所有しているクラスで、デストラクタだけを定義した場合を考える。コンパイラが自動的に生成するコピー処理では、ポインタが指しているオブジェクトそのものではなく、ポインタに格納されているアドレスだけがコピーされる。その結果、複数のオブジェクトが同じ資源を所有しているかのような状態となり、二重解放などの問題が生じる可能性がある。

このような資源管理に関する特殊メンバ関数の設計指針として、**Rule of Three**, **Rule of Five**, **Rule of Zero** と呼ばれる考え方が広く用いられている。これらは C++ の文法規則ではなく、安全なクラス設計のための指針である。

14.5.1 特殊メンバ関数

C++ では、オブジェクトの生成、コピー、移動、破棄などに関係する特別なメンバ関数を**特殊メンバ関数**という。特殊メンバ関数にはデフォルトコンストラクタも含まれるが、本節では資源管理との関係が特に深い、次の五つを扱う。

- コピーコンストラクタ
- コピー代入演算子
- ムーブコンストラクタ
- ムーブ代入演算子
- デストラクタ

コピーコンストラクタは、既存のオブジェクトから新しいオブジェクトを生成するときに使用される。コピー代入演算子は、既に存在しているオブジェクトへ、別のオブジェクトの内容をコピーするときに使用される。

ムーブコンストラクタとムーブ代入演算子は、別のオブジェクトが所有している資源を、コピーせずに引き継ぐために使用される。デストラクタは、オブジェクトが破棄される際に、所有している資源を解放する。

これらの関数をプログラマが定義しなかった場合には、条件に応じてコンパイラが自動的に生成する。しかし、自動生成されるコピー処理や移動処理が、資源を所有するクラスにとって適切であるとは限らない。

14.5.2 Rule of Three

Rule of Three は、次の三つの特殊メンバ関数のいずれかをプログラマが定義する必要がある場合には、残りの関数についても定義する必要があるかを検討するべきである、という設計指針である。

- コピーコンストラクタ
- コピー代入演算子
- デストラクタ

例えば、次のクラスは、動的に生成された整数オブジェクトをポインタ `value` によって所有する。

```
class Number {
private:
    int* value;
public:
    explicit Number(int n) : value(new int(n)) {}
    ~Number() {
        delete value;
    }
};
```

このクラスでは、デストラクタによって `value` が指している整数オブジェクトを破棄している。しかし、コピーコンストラクタとコピー代入演算子を定義していないため、コンパイラが自動的に生成するコピー処理が使用される。

自動生成されるコピー処理では、ポインタ `value` に格納されているアドレスだけがコピーされる。従って、次のようにオブジェクトをコピーすると、`a` と `b` が同じ整数オブジェクトを指すことになる。

```
Number a(10);
Number b = a;
```

この状態で `a` と `b` が破棄されると、両方のデストラクタが同じ整数オブジェクトに対して `delete` を実行しようとする。このような二重解放は未定義動作を引き起こす。

コピーを許可する場合には、コピー元とは別の資源を新たに生成し、内容を複製する**深いコピー**を行う必要がある。例えば、コピーコンストラクタは次のように定義できる。

```
Number(const Number& other) : value(new int(*other.value)) {}
```

この処理では、コピー元の整数値を持つ新しい整数オブジェクトを生成する。従って、コピー後の二つの `Number` オブジェクトは、互いに異なる整数オブジェクトを所有する。

コピー代入演算子では、コピー元の内容を新しい資源へ複製したうえで、代入先が既に所有している資源を解放し、所有する資源を置き換える必要がある。

```
Number& operator=(const Number& other) {
    if (this != &other) {
        int* p = new int(*other.value);
        delete value;
        value = p;
    }
    return *this;
}
```

ここでは、最初に新しい整数オブジェクトを生成してから、代入先が所有していた資源を解放している。これにより、新しい資源の生成に失敗して例外が送出された場合でも、代入先のオブジェクトが元の資源を保持した状態を維持できる。また、`this` と `&other` を比較することによって、オブジェクト自身を自分自身へ代入する**自己代入**にも対応している。

このように、資源を直接所有するクラスでコピーを許可する場合には、コピーと破棄の処理を一体として設計する必要がある。

ここで例示した Number クラスを使ったプログラム test214-1.cpp を示す。

プログラム：test214-1.cpp

```
1  #include    <iostream>
2  using std::cout;
3
4  class Number {
5  private:
6      int* value;
7
8  public:
9      explicit Number(int n) : value(new int(n)) {
10         cout << "コンストラクタ：" << *value << "\n";
11     }
12
13     Number(const Number& other) : value(new int(*other.value)) {
14         cout << "コピーコンストラクタ：" << *value << "\n";
15     }
16
17     Number& operator=(const Number& other) {
18         cout << "コピー代入演算子\n";
19         if (this != &other) {
20             int* p = new int(*other.value);
21             delete value;
22             value = p;
23         }
24         return *this;
25     }
26
27     ~Number() {
28         cout << "デストラクタ：" << *value << "\n";
29         delete value;
30     }
31
32     void set(int n) {
33         *value = n;
34     }
35
36     void show() const {
37         cout << "value = " << *value << "\n";
38     }
39 };
40
41 int main() {
42     cout << "1. オブジェクト a を生成\n";
43     Number a(10);
44
45     cout << "\n2. a から b をコピー生成\n";
46     Number b = a;
47
48     cout << "\n3. 既存の c へ a をコピー代入\n";
49     Number c(30);
50     c = a;
51
52     cout << "\n4. a の値だけを変更\n";
53     a.set(100);
54
55     cout << "a : ";
56     a.show();
57     cout << "b : ";
58     b.show();
59     cout << "c : ";
60     c.show();
61
62     cout << "\nmain関数を終了します\n";
63 }
```

このプログラムでは、`Number` クラスのコピーコンストラクタ、コピー代入演算子、デストラクタを実装し、深いコピーの動作を確認する。

`main` 関数では、まず `a` を生成し、次に `a` をコピーして新しいオブジェクト `b` を生成する。このときにはコピーコンストラクタが呼び出される。その後、既に存在しているオブジェクト `c` に `a` を代入し、コピー代入演算子の動作を確認する。

最後に、`a` が保持している値だけを 100 に変更する。`b` と `c` の値は 10 のままであることから、それぞれの `Number` オブジェクトが異なる整数オブジェクトを所有しており、深いコピーが行われていることを確認できる。

このプログラムをコンパイルして実行する例を次に示す。

例. `test214-1.cpp` のコンパイルと実行

```
$ g++ test214-1.cpp -o test214-1  ←コンパイル
$ ./test214-1  ←実行
1. オブジェクト a を生成          ←↓ 出力
   コンストラクタ：10
2. a から b をコピー生成
   コピーコンストラクタ：10
3. 既存の c へ a をコピー代入
   コンストラクタ：30
   コピー代入演算子
4. a の値だけを変更
   a : value = 100
   b : value = 10
   c : value = 10
main 関数を終了します
デストラクタ：10
デストラクタ：10
デストラクタ：100
```

14.5.3 コピーを禁止する方法

資源を所有するクラスでは、必ずしもコピーを許可する必要はない。ファイル、通信接続、排他制御のためのロックなど、単純に複製できない資源を管理する場合には、コピーを禁止する方が適切なことも多い。

コピーコンストラクタとコピー代入演算子の定義に `delete` を指定すると、その関数の使用を禁止できる。

```
Number(const Number&) = delete;
Number& operator=(const Number&) = delete;
```

このように定義したクラスのオブジェクトをコピーしようとする、コンパイルエラーとなる。

```
Number a(10);
Number b = a;    // 誤り：コピーできない
```

コピーを禁止すると、複数のオブジェクトが同じ資源を誤って所有することを防ぐことができる。ただし、所有権を別のオブジェクトへ移す必要がある場合には、ムーブコンストラクタやムーブ代入演算子を定義することを検討する。

14.5.4 Rule of Five

C++11 以降では、コピーと破棄に加えて、資源の所有権を移動するためのムーブ処理を定義できる。このため、Rule of Three を次の五つの特殊メンバ関数へ拡張した指針を **Rule of Five** という。

- コピーコンストラクタ
- コピー代入演算子
- ムーブコンストラクタ
- ムーブ代入演算子
- デストラクタ

これらのいずれかをプログラマが定義する必要がある場合には、ほかの特殊メンバ関数についても、定義、既定動作、または禁止のいずれが適切であるかを検討する必要がある。

先の `Number` クラスにムーブコンストラクタを追加すると、次のように記述できる。

```

    Number(Number&& other) noexcept : value(other.value) {
        other.value = nullptr;
    }

```

この処理では、移動元が所有していたポインタを移動先へ引き渡し、移動元のポインタを `nullptr` に変更する。従って、動的な整数オブジェクトそのものをコピーせずに、所有権だけを移動できる。

移動元のオブジェクトは、引き続き有効なオブジェクトとして存在するが、何も所有していない状態となる。この `Number` クラスでは、デストラクタで `nullptr` に対して `delete` を実行しても問題はないため、移動元のオブジェクトも通常どおり破棄できる。

ムーブ代入演算子では、代入先が現在所有している資源を解放し、移動元の資源を引き継ぐ。

```

Number& operator=(Number&& other) noexcept {
    if (this != &other) {
        delete value;
        value = other.value;
        other.value = nullptr;
    }
    return *this;
}

```

この処理でも、新しい整数オブジェクトを生成せず、ポインタの値を移動先へ引き渡す。従って、深いコピーを行う場合よりも効率よく所有権を移動できる。

ムーブ処理を導入すると、移動元には何も所有していない状態の `Number` オブジェクトが残る場合がある。従って、コピー処理についても、コピー元が何も所有していない場合を扱えるようにする必要がある。例えば、コピーコンストラクタは次のように記述できる。

```

Number(const Number& other)
    : value(other.value != nullptr
        ? new int(*other.value) : nullptr) {}

```

コピー代入演算子についても、同様に、コピー元が何も所有していない場合を扱えるようにしておく。

```

Number& operator=(const Number& other) {
    if (this != &other) {
        int* p = (other.value != nullptr)
            ? new int(*other.value) : nullptr;
        delete value;
        value = p;
    }
    return *this;
}

```

このようにすると、通常の `Number` オブジェクトだけでなく、ムーブ後の何も所有していない `Number` オブジェクトについても、安全にコピーできる。

ムーブコンストラクタとムーブ代入演算子が例外を送出しない処理として実装できる場合には、通常、`noexcept` を指定する。これにより、ムーブ処理が例外を送出しないことを表すことができ、標準ライブラリのコンテナなどがムーブ処理を利用しやすくなる。

このクラスでは、コピー時にはコピー元の状態に応じて新しい資源を生成して内容を複製し、移動時には既存の資源の所有権だけを引き渡す。また、各オブジェクトが破棄される際には、デストラクタによって、そのオブジェクトが所有している資源が解放される。

ここで例示した `Number` クラスを使ったプログラム `test214-2.cpp` を示す。

プログラム：test214-2.cpp

```

1  #include    <iostream>
2  #include    <utility>
3  using std::cout;
4
5  class Number {
6  private:
7      int* value;
8
9  public:

```

```

10     explicit Number(int n) : value(new int(n)) {
11         cout << "コンストラクタ：" << *value << "\n";
12     }
13
14     Number(const Number& other)
15         : value(other.value != nullptr
16             ? new int(*other.value) : nullptr) {
17         cout << "コピーコンストラクタ\n";
18     }
19
20     Number& operator=(const Number& other) {
21         cout << "コピー代入演算子\n";
22         if (this != &other) {
23             int* p = (other.value != nullptr)
24                 ? new int(*other.value) : nullptr;
25             delete value;
26             value = p;
27         }
28         return *this;
29     }
30
31     Number(Number&& other) noexcept : value(other.value) {
32         cout << "ムーブコンストラクタ\n";
33         other.value = nullptr;
34     }
35
36     Number& operator=(Number&& other) noexcept {
37         cout << "ムーブ代入演算子\n";
38         if (this != &other) {
39             delete value;
40             value = other.value;
41             other.value = nullptr;
42         }
43         return *this;
44     }
45
46     ~Number() {
47         if (value != nullptr) {
48             cout << "デストラクタ：" << *value << "\n";
49         } else {
50             cout << "デストラクタ：何も所有していません\n";
51         }
52         delete value;
53     }
54
55     void show() const {
56         if (value != nullptr) {
57             cout << "value = " << *value << "\n";
58         } else {
59             cout << "何も所有していません\n";
60         }
61     }
62 };
63
64 int main() {
65     cout << "1. オブジェクト a を生成\n";
66     Number a(10);
67
68     cout << "\n2. a から b をコピー生成\n";
69     Number b = a;
70     cout << "a : ";
71     a.show();
72     cout << "b : ";
73     b.show();
74
75     cout << "\n3. a から c へムーブ生成\n";
76     Number c = std::move(a);
77     cout << "a : ";
78     a.show();
79     cout << "c : ";
80     c.show();
81

```



```

82     cout << "\n4. 既存のdへbをコピー代入\n";
83     Number d(40);
84     d = b;
85     cout << "b: ";
86     b.show();
87     cout << "d: ";
88     d.show();
89
90     cout << "\n5. 既存のeへcをムーブ代入\n";
91     Number e(50);
92     e = std::move(c);
93     cout << "c: ";
94     c.show();
95     cout << "e: ";
96     e.show();
97
98     cout << "\nmain関数を終了します\n";
99 }

```

このプログラムでは、Number クラスにコピーコンストラクタ、コピー代入演算子、ムーブコンストラクタ、ムーブ代入演算子、デストラクタを実装し、コピーとムーブの違いを確認する。

main 関数では、まず a を生成し、a から b をコピー生成する。このときにはコピーコンストラクタが呼び出され、a と b はそれぞれ別の資源を所有する。

次に、std::move を用いて a から c をムーブ生成する。このときにはムーブコンストラクタが呼び出され、a が所有していた資源が c へ移動する。移動後の a は、何も所有していない状態となる。

さらに、既に存在している d へ b をコピー代入し、e へ c をムーブ代入する。コピー代入では新しい資源を生成して内容を複製するのに対して、ムーブ代入では移動元が所有していた資源を引き継ぐ。ムーブ代入後の c は、何も所有していない状態となる。

このプログラムをコンパイルして実行する例を次に示す。

例. test214-2.cpp のコンパイルと実行

\$ g++ test214-2.cpp -o test214-2 ←コンパイル

\$./test214-2 ←実行

1. オブジェクト a を生成 ←↓ 出力
コンストラクタ: 10

2. a から b をコピー生成
コピーコンストラクタ
a: value = 10
b: value = 10

3. a から c へムーブ生成
ムーブコンストラクタ
a: 何も所有していません
c: value = 10

4. 既存の d へ b をコピー代入
コンストラクタ: 40
コピー代入演算子
b: value = 10
d: value = 10

5. 既存の e へ c をムーブ代入
コンストラクタ: 50
ムーブ代入演算子
c: 何も所有していません
e: value = 10

main 関数を終了します
デストラクタ: 10
デストラクタ: 10
デストラクタ: 何も所有していません
デストラクタ: 10
デストラクタ: 何も所有していません

14.5.5 Rule of Zero

資源を所有するクラスを作成するたびに、コピー、移動、破棄の処理をすべて手作業で実装すると、コードが複雑になり、二重解放、解放忘れ、自己代入への対応漏れなどの問題が生じやすくなる。

これに対して、資源管理を標準ライブラリのクラスなどに任せ、クラス自身ではコピー、移動、破棄に関する特殊メンバ関数を定義しないという設計指針を **Rule of Zero** という。

例えば、複数の整数値を保持するクラスを、動的配列と生ポインタによって実装すると、コピー、移動、破棄の処理を自分で定義する必要がある。しかし、`std::vector` をデータメンバとして使用すれば、その処理を `std::vector` に任せることができる。

```
class Numbers {
private:
    std::vector<int> values;
public:
    explicit Numbers(std::size_t n)
        : values(n) {}
};
```

このクラスでは、コピーコンストラクタ、コピー代入演算子、ムーブコンストラクタ、ムーブ代入演算子、デストラクタを明示的に定義していない。しかし、`std::vector` 自身がコピー、移動、破棄を適切に処理するため、`Numbers` クラスでも安全なコピーと移動が行われる。

動的に生成された単独のオブジェクトを所有する場合には、生ポインタの代わりに `std::unique_ptr` をデータメンバとして使用できる。

```
class Number {
private:
    std::unique_ptr<int> value;
public:
    explicit Number(int n)
        : value(std::make_unique<int>(n)) {}
};
```

このクラスでは、デストラクタを定義しなくても、`Number` オブジェクトが破棄される際に `unique_ptr` が内部の整数オブジェクトを自動的に破棄する。また、`unique_ptr` の性質により、`Number` クラスもコピーできず、移動だけが可能なクラスとなる。

このように、データメンバとして使用するクラスが既に適切な資源管理を行っている場合には、コピー、移動、破棄の動作をデータメンバの性質に任せることができる。これにより、資源管理のためのコードを減らし、クラスの設計を簡潔で安全なものにできる。

14.5.6 Rule of Three/Five/Zero の使い分け

Rule of Three/Five/Zero は、次のように整理できる。

- 資源を直接所有し、コピーを許可する場合には、コピーと破棄を一体として設計する。
- 資源の所有権を移動できるようにする場合には、ムーブ処理についても設計する。
- コピーを許可しない場合には、コピーコンストラクタとコピー代入演算子を禁止する。
- 可能であれば、標準ライブラリのクラスに資源管理を任せ、コピー、移動、破棄に関する特殊メンバ関数を定義しない。

現代の C++ では、原則として Rule of Zero に従い、`std::vector`、`std::string`、スマートポインタなどの RAII に対応したクラスをデータメンバとして使用するのが望ましい。資源を直接管理する必要がある場合にのみ、Rule of Three または Rule of Five に基づいて、コピー、移動、破棄の処理を明示的に設計する。

15 クラス設計の実践

これまでの章では、C++ におけるクラスの基本的な機能について解説してきた。「12 クラスとオブジェクト」では、クラスの定義方法、コンストラクタやデストラクタ、アクセス制御といった基本事項を扱い、オブジェクトを作成して利用する方法を解説した。また、「13 継承とポリモーフィズム」では、継承とポリモーフィズムについて解説し、複数のクラスに共通する振る舞いを抽象化し、柔軟に扱うための仕組みを解説した。

これらの機能により、C++ では単なる関数とデータの集まりを超えて、「構造を持ったプログラム」を記述することが可能となる。しかしながら、これらの機能を理解しただけでは、実際に良質なプログラムを設計することはできない。

例えば、次のような疑問は自然に生じる。

- ・ クラスはどのような単位で分割すべきか
- ・ 継承とコンポジションはどのように使い分けるべきか
- ・ 仮想関数はどの段階で導入すべきか
- ・ データと処理の責任はどのように配置すべきか

これらはすべて、言語仕様や文法の問題ではなく、設計の問題である。

C++ の学習において重要な転換点は、ここにある。すなわち、「クラスを書く」段階から、「クラスを設計する」段階へと進むことである。

本章では、これまでに学んだクラス、継承、ポリモーフィズムといった機能を前提として、それらをどのように組み合わせ、実際のプログラム設計に応用していくかを解説する。対象とするのは、特定の文法事項ではなく、「よく用いられる設計の考え方」や「設計のパターン」¹⁵⁸ である。

まず、基本的なクラスの構成を確認したうえで、小さなデータを安全に扱うための「値オブジェクト」、外部資源を適切に管理するための「リソース管理クラス」といった、代表的な設計の型を取り上げる。さらに、抽象クラスを用いたインターフェース設計や、ポリモーフィズムを前提としたクラス構造について考察する。

また、継承に過度に依存しない設計手法として、オブジェクトの組み合わせによって機能を構成する **コンポジション** についても扱う。これは、プログラムの柔軟性と保守性を高めるために重要な考え方である。

最後に、初学者が陥りやすい設計上の誤りをいくつか取り上げ、それらを回避するための指針を整理する。

本章の目的は、単に新しい言語仕様を解説することではない。プログラムを構成する要素の「役割」と「責任」を意識し、それらを適切に配置するための考え方を身につけることである。この視点を獲得することで、読者はより大きく、より複雑なプログラムを、見通しよく構築するための基礎力を得ることができる。

15.1 基本的なクラスの構成

クラスを用いたプログラム設計において、最初に考えるべきことは、「そのクラスが何を表し、どのような責任を持つか」という点である。単にデータと関数をまとめるだけではなく、そのクラスがどのような役割を担い、どのような状態を管理するのかを明確にする必要がある。

本節では、銀行口座を表す簡単なクラスを例に取り上げ、基本的なクラスの構成について考える。まずは、素朴な実装から出発し、そこにどのような問題が潜んでいるかを確認する。

■ 素朴な実装

銀行口座の残高を管理するだけであれば、次のようなクラスを考えることができる。

```
class BankAccount {
public:
    int balance;        // 口座の残高を保持するメンバ変数
};
```

このクラスは、口座の残高を整数値として保持するだけのものである。一見すると単純でわかりやすいが、この設計には重大な問題がある。それは、残高の値が外部から自由に変更できてしまうことである。たとえば、次のような操

¹⁵⁸ より広い射程では **デザインパターン** として体系的に論じられている（文献 [8]）。本章では、そのような体系的なデザインパターンの分類には踏み込まず、C++ の入門段階で有用な基本的な設計の考え方を扱う。

作が可能となる。

```
BankAccount account;  
account.balance = -1000;    // 不正な操作
```

このように、クラスの外部から直接データを書き換えることができる場合、そのクラスが保持すべき状態の整合性を保つことができない。

クラス設計においては、「そのクラスが管理すべき状態は、そのクラス自身が責任を持って維持する」という原則が重要である。このためには、内部のデータを外部から直接操作できないようにし、適切な操作を通じてのみ変更できるようにする必要がある。

■ 改善した設計

上記の問題を解決するためには、データを隠蔽し、操作を通じて状態を変更するように設計する。その一例を次に示す。

クラス定義：銀行口座を管理するためのクラス

```
1 class BankAccount {  
2     private:                // 外部から隠蔽  
3         int balance;  
4  
5     public:  
6         explicit BankAccount(int initial) {    // コンストラクタ（初期値設定）  
7             if (initial >= 0) {  
8                 balance = initial;  
9             } else {  
10                 balance = 0;  
11             }  
12         }  
13  
14         void deposit(int amount) {            // 預け入れの処理  
15             if (amount > 0) {  
16                 balance += amount;  
17             }  
18         }  
19  
20         bool withdraw(int amount) {           // 引き出しの処理  
21             if (amount > 0 && balance >= amount) {  
22                 balance -= amount;  
23                 return true;  
24             }  
25             return false;  
26         }  
27  
28         int getBalance() const {              // 残高照会  
29             return balance;  
30         }  
31 };
```

この設計では、残高 `balance` は `private` として宣言されており、外部から直接アクセスすることはできない。その代わりに、`deposit` や `withdraw` といった関数を通じてのみ状態が変更される。

ここで重要なのは、クラスの内部において、状態の整合性が維持されている点である。たとえば、`withdraw` 関数では、残高が不足している場合には処理を行わないようにしており、不正な状態への遷移を防いでいる。これは、「残高は負にならない」という、クラスが常に維持すべき条件（インバリアント）を守るための処理である。

なお、`withdraw` 関数は、処理に成功した場合に `true` を、失敗した場合に `false` を返すようにしている。このように、操作の成否を戻り値で表す設計は、実用的なクラスではよく用いられる。

■ クラスの責任

この例からわかるように、クラス設計において重要なのは、「データをどのように保持するか」だけではなく、「そのデータをどのように扱うか」である。すなわち、クラスは単なるデータの入れ物ではなく、「状態とその操作を一体として管理する存在」である。

銀行口座の例では、「残高という状態を管理し、その整合性を保つこと」がクラスの責任であるといえる。この責任

を明確にすることで、クラスの設計は自然と定まってくる。

■ 基本構成のまとめ

一般に、クラスは次のような構成を持つ。

- ・ 管理すべき状態（メンバ変数）
- ・ 状態を操作するための関数（メンバ関数）
- ・ 状態の整合性を維持するための条件（インバリアント）

これらを適切に設計することにより、クラスは単なるデータ構造ではなく、意味を持ったプログラムの構成要素となる。本章では、この基本的な構成を出発点として、より実践的なクラス設計の方法について段階的に解説していく。

15.2 小さなデータを表すクラス（値オブジェクト）

前節では、銀行口座を表すクラスを例に取り上げ、クラスが「状態とその操作を一体として管理する存在」であることを示した。しかしながら、その実装では残高を単なる整数値（`int` 型）として扱っており、いくつかの問題が残されている。たとえば、次のような点である。

- ・ 金額であるにもかかわらず、単なる整数として扱われている
- ・ 金額としての意味が型に表れていない
- ・ 誤った値（負の値など）の生成や使用を防ぎにくい

このような問題は、「意味を持つ値」を単なる基本型で表現していることに起因する。この問題を解決するために用いられるのが、**値オブジェクト**と呼ばれる設計である。

値オブジェクトとは、ある値そのものを表現するためのクラスであり、主に次のような特徴を持つ。

- ・ 小さく単純なデータを表す
- ・ 値そのものに意味（例えば「金額」など）がある
- ・ 不変（immutable）¹⁵⁹であることが多い

ここでは、金額を表す `Money` クラスを例に取り上げる。

クラス定義：金額を表す値オブジェクト

```
1 class Money {
2     private:
3         int amount;
4
5     public:
6         Money(int value) {
7             if (value >= 0) {
8                 amount = value;
9             } else {
10                 amount = 0;
11             }
12         }
13
14         int getAmount() const {
15             return amount;
16         }
17     };
```

このクラスは、金額という値を表現するためのものであり、`amount` は外部から直接変更できない。また、負の値が設定されないようにすることで、常に正しい状態が維持される。これは、値オブジェクトにおけるインバリアントの一例である。

このように、意味を持つ値をクラスとして表現することにより、プログラムの可読性と安全性が向上する。すなわち、「金額」という概念に関する制約や意味は、`Money` クラスに集約される。

この `Money` クラスを応用して、先の銀行口座のクラスは次のように書き換えることができる。

¹⁵⁹作成後に変更ができないオブジェクトを意味する。

使用例：値オブジェクトを用いた銀行口座

```
1 class BankAccount {
2     private:
3         Money balance;
4
5     public:
6         explicit BankAccount(int initial) : balance(initial) {}
7
8         void deposit(int amount) {
9             if (amount > 0) {
10                 balance = Money(balance.getAmount() + amount);
11             }
12         }
13
14         bool withdraw(int amount) {
15             int current = balance.getAmount();
16             if (amount > 0 && current >= amount) {
17                 balance = Money(current - amount);
18                 return true;
19             }
20             return false;
21         }
22
23         int getBalance() const {
24             return balance.getAmount();
25         }
26 };
```

このように、残高を保持するメンバを単なる整数ではなく `Money` 型として扱うことで、「これは金額である」という意味がコード上で明確になる。また、値の生成処理を `Money` クラスに集約することで、不正な状態を持つオブジェクトが生成されることを防ぐことができる。すなわち、`BankAccount` のコンストラクタを

```
explicit BankAccount(int initial) : balance(initial) {} // Money クラスのコンストラクタを呼び出している
```

と定義するだけで、負値を内部状態として保持しないための処理を `Money` クラスに任せることができる。

ただし、ここで示した例では、金額に関する演算の一部がまだ `BankAccount` 側に残っている。より本格的には、これらの処理を `Money` クラスに持たせることで、責任をより明確に分離することができる。この点については、後の節で改めて扱う。

■ 値オブジェクトの役割

値オブジェクトは、大きな構造を持つクラスを設計する際に、その内部で用いられる基本的な構成要素として重要な役割を果たす。すなわち、大きなクラスを直接設計するのではなく、小さく意味のある値を組み合わせることで、全体の構造を構築する方法である。

また、値オブジェクトは不変 (immutable) として設計されることが多い。すなわち、一度生成されたオブジェクトの状態は変更せず、新しい値が必要な場合には新たなオブジェクトを生成する。この性質により、プログラムの安全性が向上し、意図しない挙動¹⁶⁰ の発生を抑制することができる。

■ 本節のまとめ

値オブジェクトは、「意味を持つ値」を安全に扱うための重要な設計手法である。基本型による表現では失われがちな意味や制約をクラスとして明示することで、より理解しやすく、安全なプログラムを構築することができる。

次節では、外部資源を扱うクラスについて考察し、値オブジェクトとは異なる性質を持つ設計について見ていく。

15.3 リソース管理のためのクラス設計

前節では、小さく意味のある値を表現するクラスとして、値オブジェクトについて解説した。値オブジェクトは、一般に小さく単純であり、不変 (immutable) として扱われることが多い。一方で、実際のプログラムでは、ファイル、メモリ、ネットワーク接続、データベース接続などの**外部資源**（システムのリソース）を扱う必要がある。

¹⁶⁰ 「バグ」(bug) と表現されることが多い。

これらの外部資源は、データとしての単なる値とは異なり、「使用開始」と「使用終了」の管理が必要となる。例えば、ファイルを使用する場合には、ファイルを開いた後、適切なタイミングで閉じなければならない。また、動的に確保したメモリについても、使用後には解放処理を行う必要がある。

このような資源の管理を適切に行うために用いられるのが、**リソースを扱うクラス**である。この種のクラスでは、「資源を確保したら、必ず後始末を行う」という責任をクラス自身に持たせることが重要となる。

ここでは、銀行口座の取引履歴をファイルに記録するクラスを例に考える。まず、素朴な実装例を示す。

■ 素朴な実装

```
1 FILE *fp = fopen("log.txt", "a");
2
3 if (fp != NULL) {
4     fprintf(fp, "deposit: 1000\n");
5 }
```

補足) ここでは、資源の「確保」と「解放」の対応関係を明確に示すため、C++ のファイルストリームではなく、敢えて C 言語の FILE を用いた例を示している。もちろん、C++ による現実的なシステム開発では、ファイルストリームを用いる方法を優先的に採用すべきである。

上のコードは、ファイルを開き、文字列を書き込む処理を行っている。しかし、このままでは問題がある。それは、ファイルを閉じる処理が存在しないことである。

ファイルを開いたまま放置すると、OS が管理している資源が解放されず、長時間動作するプログラムでは問題を引き起こす可能性がある。したがって、fopen によってファイルを開いた場合には、対応する fclose を必ず呼び出さなければならない。

■ リソース管理を行うクラス

このような問題を防ぐために、ファイルの管理そのものをクラスとして表現することができる。

クラス定義：取引履歴を書き込むクラス

```
1 class TransactionLog {
2     private:
3         FILE *fp;
4
5     public:
6         explicit TransactionLog(const char *filename) { // コンストラクタで
7             fp = fopen(filename, "a"); // ファイルを開く
8         }
9
10        ~TransactionLog() { // デストラクタによって、
11            if (fp != NULL) { // もし開かれたファイルがあれば、
12                fclose(fp); // それを閉じる
13            }
14        }
15
16        void write(const char *message) { // ログにメッセージを出力する関数
17            if (fp != NULL) {
18                fprintf(fp, "%s\n", message);
19            }
20        }
21    };
```

注意) ここで示した TransactionLog クラスは、RAII による資源管理の基本的な考え方を示すために簡略化したものである。このクラスのオブジェクトをそのままコピーすると、同じ FILE オブジェクトを複数のオブジェクトが閉じようとする問題が生じる。実際には、コピーを禁止するか、適切なコピーやムーブの設計を行う必要がある。

このクラスでは、コンストラクタでファイルを開き、デストラクタでファイルを閉じている。すなわち、オブジェクトの生成と破棄に対応して、資源の確保と解放が自動的に行われる。このように、資源の寿命をオブジェクトの寿命に結び付けて管理する方法は、先に解説した **RAII** (Resource Acquisition Is Initialization) の考え方に基づいている。

RAII を利用して、コンストラクタで資源を取得し、デストラクタで資源を解放するようにクラスを設計することで、ファイルの閉じ忘れやメモリの解放忘れなどを防ぎやすくなる。この考え方は、実用的な C++ プログラムにお

いて非常に重要である。

たとえば、このクラスは次のように使用することができる。

使用例：取引履歴の記録

```
1 void process(int d, int w) {
2     char msg[256];
3     TransactionLog log("log.txt");
4
5     if (d > 0) {
6         sprintf(msg, "deposit: %d", d);
7         log.write(msg);
8     }
9
10    if (w > 0) {
11        sprintf(msg, "withdraw: %d", w);
12        log.write(msg);
13    }
14 }
```

これは、預金額 *d*、引き出し額 *w* を受け取り、それらをログに追加出力する関数の実装例である。この例では、`process` 関数の終了時に `log` オブジェクトが破棄され、対応してデストラクタが呼び出される。その結果、ファイルは自動的に閉じられる。

このように、後始末をクラスに任せることで、「閉じ忘れ」や「解放忘れ」といった問題を防ぎやすくなる。

■ 値オブジェクトとの違い

前節で扱った `Money` クラスは、「値」を表現するためのクラスであった。それに対し、`TransactionLog` クラスは、「外部資源」を管理するためのクラスである。すなわち、値オブジェクトでは、「意味のある値の表現」が目的であったのに対し、リソース管理クラスでは、「資源の確保と後始末」が目的である。このように、目的に沿ったクラス的设计が重要である。

■ 本節のまとめ

リソースを扱うクラスでは、「資源の確保」と「資源の解放」を適切に管理することが重要である。C++ では、コンストラクタとデストラクタを利用することで、これらの処理をオブジェクトの寿命に対応付けて管理することができる。

この考え方は、実用的な C++ プログラムにおいて極めて重要であり、先に解説したスマートポインタや、標準ライブラリの各種クラスにも広く用いられている。

課題. 先の `process` 関数を、引数として `Money` クラスのオブジェクトを受け取るものとして再設計せよ。

15.4 抽象クラスによるインターフェース設計

前節までは、銀行口座を表すクラス、金額を表す `Money` クラス、取引履歴を記録する `TransactionLog` クラスなどを例に、クラスが持つべき責任について考えてきた。ここからは、異なる複数のクラスに共通する「使い方」をどのように設計するかについて考える。

銀行口座には、実際にはいくつかの異なる種類がある。例えば、普通預金口座、当座預金口座、定期預金口座などである。これらはすべて「口座」であり、残高を持ち、入金や引き出し、残高照会といった共通の操作を持つ。しかし、引き出しの条件や手数料、利息の扱いなどは、口座の種類によって異なる場合がある。

このような場合、それぞれの口座クラスを完全に別々のものとして設計してしまうと、プログラム全体の見通しが悪くなる。そこで、「口座として共通に備えるべき操作」をあらかじめ決めておき、それぞれの具体的な口座クラスがその操作を実装するように設計する方法が考えられる。このような共通の使い方を表すものを**インターフェース**と呼ぶ。

■ 抽象クラスによる共通インターフェース

C++ では、純粋仮想関数を持つクラスを用いて、共通のインターフェースを表現することができる。次の例では、銀行口座として共通に利用したい操作を `Account` クラスとして定義している。

クラス定義：口座の共通インターフェース

```
1 class Account {
2     public:
3         virtual ~Account() = default;
4
5         virtual void deposit(int amount) = 0;
6         virtual bool withdraw(int amount) = 0;
7         virtual int getBalance() const = 0;
8 };
```

この Account クラスでは、deposit, withdraw, getBalance が純粋仮想関数として定義されている。したがって、Account クラスそのもののオブジェクトを作成することはできない。このクラスは、具体的な口座を作るための「共通の約束」を表している。

また、基底クラスとして用いることを前提としているため、デストラクタを virtual として定義している。これは、派生クラスのオブジェクトを基底クラスのポインタや参照を通じて扱う場合に重要となる。

■ 具体的な口座クラスの実装

次に、この Account クラスを継承して、普通預金口座を表すクラスを定義する。

クラス定義：普通預金口座

```
1 class SavingsAccount : public Account {
2     private:
3         Money balance;
4
5     public:
6         explicit SavingsAccount(int initial) : balance(initial) {}
7
8         void deposit(int amount) override {
9             if (amount > 0) {
10                 balance = Money(balance.getAmount() + amount);
11             }
12         }
13
14         bool withdraw(int amount) override {
15             int current = balance.getAmount();
16             if (amount > 0 && current >= amount) {
17                 balance = Money(current - amount);
18                 return true;
19             }
20             return false;
21         }
22
23         int getBalance() const override {
24             return balance.getAmount();
25         }
26 };
```

このクラスは、Account クラスで定められた操作をすべて実装している。override は、基底クラスの仮想関数を派生クラスで再定義していることを明示するための指定である。これにより、関数名や引数の型を誤った場合に、コンパイラが誤りを検出しやすくなる。

■ インターフェースとして扱う

抽象クラスを用いる利点は、具体的なクラス名ではなく、共通のインターフェースを通じてオブジェクトを扱える点にある。たとえば、次の関数は、具体的な口座の種類を知らなくても、Account として扱える口座（派生クラスのオブジェクト）であれば利用できる。

使用例：口座を共通のインターフェースで扱う関数（残高出力関数）

```
1 void printBalance(const Account& account) {
2     std::cout << "balance: " << account.getBalance() << "\n";
3 }
```

この関数は、SavingsAccount 専用の関数ではない。Account のインターフェースを満たす任意の口座クラスに対して使用できる。このように、具体的な実装ではなく、共通の操作に着目して設計することで、プログラムの柔軟性

が高まる。

■ 抽象クラスの役割

抽象クラスは、それ自体で完結した具体的な部品というよりも、「この種類のオブジェクトは、少なくともこの操作を持つべきである」という約束を表すために用いられる。つまり、抽象クラスは、クラス群の共通の入口を定める役割を持つ。

銀行口座の例では、Account クラスは、「口座であれば、入金、出金、残高照会ができる」という共通の使い方を表している。普通預金口座、当座預金口座、定期預金口座などの具体的なクラスは、この共通の使い方に従って、それぞれの振る舞いを実装する。

■ 本節のまとめ

抽象クラスを用いると、複数のクラスに共通する操作をインターフェースとして定義することができる。これにより、プログラムは具体的なクラスに直接依存するのではなく、共通の使い方に基づいて設計できるようになる。

次節では、この考え方をさらに進め、抽象クラスを通じて複数種類の口座を同じ形式で扱うポリモーフィズムの設計について考察する。

15.5 ポリモーフィズムを前提とした設計

前節では、抽象クラスを用いて、複数の口座クラスに共通するインターフェースを定義する方法について解説した。しかしながら、抽象クラスを定義しただけでは、その利点は十分に発揮されない。重要なのは、「共通のインターフェース」を通じて異なる種類のオブジェクトを扱うように設計することである。

このような設計を可能にするのが、ポリモーフィズム¹⁶¹である。

銀行口座の例で考えると、普通預金口座、当座預金口座、定期預金口座などは、それぞれ内部の動作は異なるものの、「口座」として共通の操作を持っている。したがって、Account 型として扱うことができれば、個々の口座の違いを意識せずに共通の処理を記述することができる。

■ 複数種類の口座クラス

まず、前節で示した SavingsAccount に加えて、当座預金口座を表すクラスを定義してみる。ここでは、引き出し時に手数料 (fee) が発生するものとする。

クラス定義：当座預金口座

```
1 class CheckingAccount : public Account {
2     private:
3         Money balance;
4         int fee = 100;
5
6     public:
7         explicit CheckingAccount(int initial) : balance(initial) {}
8
9         void deposit(int amount) override {
10             if (amount > 0) {
11                 balance = Money(balance.getAmount() + amount);
12             }
13         }
14
15         bool withdraw(int amount) override {
16             int total = amount + fee;
17             int current = balance.getAmount();
18
19             if (amount > 0 && current >= total) {
20                 balance = Money(current - total);
21                 return true;
22             }
23             return false;
24         }
25 }
```

¹⁶¹多態性 (polymorphism)


```

26     int getBalance() const override {
27         return balance.getAmount();
28     }
29 };

```

※ここでは説明を簡単にするため、金額計算の一部には整数型 (int) をそのまま用いている。

このクラスは、SavingsAccount と同じインターフェースを持っているが、withdraw の動作が異なっている。すなわち、引き出し時に手数料を加算している。このように、ポリモーフィズムでは、「共通の使い方」と「異なる内部動作」を両立させることができる。

■ 共通インターフェースによる処理

ポリモーフィズムの利点は、複数種類のオブジェクトを、共通のインターフェースを通じてまとめて扱える点にある。例えば、次の関数は、任意の口座クラスに対して出金処理を行うことができる。

使用例：口座種別を意識しない出金処理

```

1 void processWithdraw(Account& account, int amount) {
2     if (account.withdraw(amount)) {
3         std::cout << "withdraw success\n";
4     } else {
5         std::cout << "withdraw failed\n";
6     }
7 }

```

この関数は、SavingsAccount 専用でも CheckingAccount 専用でもない。Account インターフェースを持つ任意のオブジェクトに対して使用することができる。実際には、呼び出されたオブジェクトの種類に応じて、適切な withdraw 関数が実行される。例えば、次のように異なる種類の口座を同じ関数で扱うことができる。

使用例：異なる口座クラスの共通利用

```

1 SavingsAccount sa(10000);
2 CheckingAccount ca(10000);
3
4 processWithdraw(sa, 1000);
5 processWithdraw(ca, 1000);

```

この場合、processWithdraw 関数は、口座の種類を意識する必要がない。sa に対しては SavingsAccount の withdraw が呼び出され、ca に対しては CheckingAccount の withdraw が呼び出される。これが、仮想関数による動的結合を利用したポリモーフィズムである。

■ ポリモーフィズムを前提とした設計

ポリモーフィズムを活用する場合、重要なのは、「具体的なクラス」を基準に設計するのではなく、「共通のインターフェース」を基準に設計することである。

例えば、「普通預金口座専用の処理」、「当座預金口座専用の処理」と個別に記述してしまうと、クラスの追加や変更により弱い構造となる。それに対し、「口座として必要な操作」を抽象クラスとして定義し、共通インターフェースを通じて処理を行うように設計すると、新しい種類の口座を追加しやすくなる。

この考え方は、大規模なソフトウェア設計において極めて重要である。実際、多くのライブラリやフレームワークでは、具体的な実装ではなく、共通インターフェースに基づいてオブジェクトを扱う設計が広く用いられている。

■ 本節のまとめ

ポリモーフィズムを利用すると、異なる種類のオブジェクトを、共通のインターフェースを通じて同じ形式で扱うことができる。これにより、具体的なクラスへの依存を減らし、柔軟で拡張しやすいプログラムを設計することが可能となる。

次節では、継承だけに依存しない設計方法として、オブジェクトを組み合わせで機能を構築する「コンポジション」について考察する。

15.6 コンポジションによる設計

前節では、抽象クラスとポリモーフィズムを用いて、共通インターフェースを通じて複数種類のオブジェクトを扱う設計について解説した。このような継承を利用した設計は強力であるが、実際のプログラム設計では、継承だけですべてを構成するわけではない。

多くの場合、クラスは他のクラスを「部品」として内部に保持し、それらを組み合わせることによって構築される。このように、オブジェクトを内部に組み込んで機能を構成する設計を**コンポジション**¹⁶² と呼ぶ。

実際、この章でも既にコンポジションの例を扱っている。例えば、銀行口座クラスにおける

```
Money balance;
```

という記述は、「口座」が「金額オブジェクト」を内部に保持していることを表している。すなわち、口座クラスは、Money クラスを部品として利用している。

ここでは、さらにもう一步進めて、取引履歴を記録する TransactionLog クラスを、口座クラスの内部に組み込む設計について考える。

■ 素朴な設計

まず、ログ出力処理を口座クラスの中へ直接書き込んだ例を考える。

クラス定義：ログ処理を直接記述した口座クラス（あまり良くない例）

```
1 class SavingsAccount : public Account {
2     private:
3         Money balance;
4
5     public:
6         explicit SavingsAccount(int initial) : balance(initial) {}
7
8         void deposit(int amount) override {          // 預け入れ処理とログの記録
9             if (amount > 0) {
10                 balance = Money(balance.getAmount() + amount);
11                 FILE *fp = fopen("log.txt", "a");
12                 if (fp != NULL) {
13                     fprintf(fp, "deposit: %d\n", amount);
14                     fclose(fp);
15                 }
16             }
17         }
18
19         bool withdraw(int amount) override {          // 引き出し処理とログの記録
20             int current = balance.getAmount();
21             if (amount > 0 && current >= amount) {
22                 balance = Money(current - amount);
23                 FILE *fp = fopen("log.txt", "a");
24                 if (fp != NULL) {
25                     fprintf(fp, "withdraw: %d\n", amount);
26                     fclose(fp);
27                 }
28                 return true;
29             }
30             return false;
31         }
32
33         int getBalance() const override {
34             return balance.getAmount();
35         }
36     };
```

この設計でもシステムは動作する。しかし、ログ出力処理が deposit や withdraw の内部に直接記述されており、口座管理の処理とログ処理が強く結び付いてしまっている。また、ファイル操作の記述が複数箇所に重複しており、保守性も低下している。

クラス設計では、「異なる責任を分離する」ことが重要である。銀行口座クラスの本来の責任は、「口座の状態を管理

¹⁶²composition（合成）

すること」であり、ファイル管理やログ出力処理そのものではない。従って、ログ出力の責任は、別のクラスとして分離した方が自然である。

■ コンポジションによる改善

次に、先に作成した TransactionLog クラスを、口座クラスの内部に組み込む。

クラス定義：ログ機能を組み込んだ口座クラス

```
1 class SavingsAccount : public Account {
2     private:
3         Money balance;
4         TransactionLog log;
5
6     public:
7         explicit SavingsAccount(int initial) : balance(initial), log("log.txt") {}
8
9         void deposit(int amount) override {
10             // 預金額が適正であれば入金処理とログ出力を行う
11             if (amount > 0) {
12                 balance = Money(balance.getAmount() + amount);
13                 char msg[256];
14                 sprintf(msg, "deposit: %d", amount);
15                 log.write(msg);
16             }
17         }
18
19         bool withdraw(int amount) override {
20             int current = balance.getAmount();
21             // 引き出し可能な金額であれば出金処理とログ出力を行う
22             if (amount > 0 && current >= amount) {
23                 balance = Money(current - amount);
24                 char msg[256];
25                 sprintf(msg, "withdraw: %d", amount);
26                 log.write(msg);
27                 return true;
28             }
29             return false;
30         }
31
32         int getBalance() const override {
33             return balance.getAmount();
34         }
35     };
```

この設計では、ログ出力の責任が TransactionLog クラスへ分離されている。SavingsAccount クラスは、ログ出力処理の詳細を意識する必要がなく、「必要な内容をログへ記録する」という操作だけを行えばよい。

また、ファイルのオープンやクローズといった資源管理も TransactionLog クラス側に集約されるため、コード全体の見通しが良くなる。

■ 継承とコンポジション

ここで重要なのは、「継承」と「コンポジション」は目的が異なるという点である。継承は、「～の一種である」という関係を表すために用いられる。例えば、SavingsAccount は Account の一種である。

一方、コンポジションは、「～を内部に持つ」という関係を表すために用いられる。例えば、SavingsAccount は Money や TransactionLog を内部に持っている。

実際のプログラム設計では、継承だけですべてを構成するのではなく、小さな部品となるクラスを組み合わせることとで、全体の機能を構築することが多い。

■ 本節のまとめ

コンポジションは、オブジェクトを部品として組み合わせることで、機能を構築する設計手法である。これにより、異なる責任を分離し、柔軟で保守性の高いプログラムを構築することができる。

継承とコンポジションは対立する概念ではなく、それぞれ異なる役割を持つ設計手法である。実際のソフトウェア設計では、両者を適切に使い分けることが重要となる。

15.7 よくある設計上の問題

これまでに、値オブジェクト、リソース管理クラス、抽象クラス、ポリモーフィズム、コンポジションといった、代表的なクラス設計の考え方について解説してきた。しかし実際のプログラム設計では、これらの機能を単に知っているだけでは十分ではない。重要なことは、「どのように設計すべきか」と同時に、「どのような設計を避けるべきか」を理解することである。

本節では、設計上の重要なポイントをいくつか取り上げる。

■ どのデータを public にすべきか

最も典型的な誤りの一つは、内部データを public のまま公開してしまうことである。例えば、次のような設計である。

悪い例：内部状態を直接公開したクラス

```
1 class BankAccount {
2     public:
3         int balance;
4 };
```

この設計では、外部から自由に残高を書き換えることができる。その結果、負の残高など、本来許されない状態が簡単に作れてしまう。

クラス設計では、「そのクラスが管理すべき状態は、そのクラス自身が責任を持って維持する」ことが重要である。そのため、内部状態は原則として private に隠蔽し、適切な操作を通じてのみ変更できるように設計するべきである。

■ 一つのクラスに過剰に責任を負わせていないか

次のように、口座管理、ログ出力、ファイル操作など、過剰な責任を一つのクラスに負わせる設計も好ましくない。

悪い例：責任が過剰に集中するクラス

```
1 class BankAccount {
2     private:
3         int balance;
4         FILE *fp;
5
6     public:
7         void deposit(int amount) {
8             ...
9         }
10
11        void withdraw(int amount) {
12            ...
13        }
14
15        void writeLog(const char *msg) {
16            ...
17        }
18
19        void openFile() {
20            ...
21        }
22
23        void closeFile() {
24            ...
25        }
26 };
```

このような設計では、クラスの役割が不明瞭になり、変更に弱い構造となる。例えば、ログ出力の仕様変更が、口座管理クラス全体へ影響を及ぼす可能性がある¹⁶³。

前節で解説したように、ログ出力は TransactionLog クラスとして分離し、口座クラスは「口座の状態管理」に責任を集中させた方が自然である。

¹⁶³このような設計は、巨大クラス（God Object）へ発展しがちである。

■ 適切な形での継承関係

継承は強力な機能であるが、「コードを再利用したい」という理由だけで安易に継承を利用すると、不自然なクラス構造が生じやすい。

例えば、「ログ出力機能を使いたい」という理由だけで、銀行口座クラスを `TransactionLog` から継承させる設計は不自然である。

悪い例：不適切な継承

```
1 class BankAccount : public TransactionLog {
2     ...
3 };
```

`BankAccount` は、「ログの一種」ではない。したがって、この継承関係は意味的に不自然である。このような場合には、継承ではなく、`TransactionLog` を内部に持つコンポジションを用いる方が適切である。

継承は、少なくとも「～の一種である」という関係が自然に成立する場合に利用するべきである。

■ 型の意味付け

意味を持つ値を、単なる整数や文字列として扱うのではなく、意味を持つオブジェクト（値オブジェクト）として扱う方が良い。

例えば、金額を単なる `int` 型として扱うと、「これは金額なのか、件数なのか、識別番号なのか」といった意味がコード上から読み取りにくくなる。また、不正な値の混入も起こりやすくなる。

このような場合には、先に扱った `Money` クラスのように、「意味を持つ値」を専用のクラスとして表現することで、可読性と安全性を向上させることができる。ただし、プログラミングの中で扱う全ての値を、専用のクラスのオブジェクトとして表現するのは好ましいことではない。これに関しては、「値」が担う機能から判断しなければならない。

■ 「責任の分離」に基づく設計

プログラムの設計において、「責任の分離」は十分に配慮すべきであり、これが不十分な場合、「どのクラスが、何に責任を持つのか」が曖昧になる。

クラス設計では、単に機能を実装するだけではなく、「その責任をどのクラスが担うべきか」を意識することが重要である。

例えば、本章で扱った設計では、

- `Money` : 金額という値の意味を管理する
- `TransactionLog` : ログ出力と資源管理を行う
- `Account` : 口座としての共通インターフェースを定義する
- `SavingsAccount` : 具体的な口座の振る舞いを実装する

というように、それぞれの責任が分離されている。

■ 本節のまとめ

クラス設計における多くの問題は、「責任の分離」が不十分であることに起因する。内部状態の無秩序な公開、不自然な継承、多数の責任の集中などは、いずれも保守性や拡張性を低下させる原因となる。

良いクラス設計とは、「どのクラスが、どの責任を持つべきか」を明確にし、小さな部品を適切に組み合わせることで、全体を構築する設計である。

15.8 設計指針のまとめ

本章では、クラス設計に関する代表的な考え方について解説してきた。ここで扱った内容は、特定の文法事項や個別の技法ではなく、「どのようにプログラムを構成するか」という設計上の視点に関するものである。

C++ では、クラス、継承、仮想関数、コンストラクタ、デストラクタなど、多くの強力な機能を利用することができる。しかしながら、重要なのは、それらの機能を単に知っていることではない。それぞれの機能を、「どのような責任を持たせるために用いるのか」を意識して設計することが重要である。

本章では、まず、「クラスは状態とその操作を一体として管理する存在である」という観点から、基本的なクラスの構成について考察した。また、Money クラスを例として、意味を持つ値を値オブジェクトとして表現する方法について解説した。

さらに、TransactionLog クラスを例として、リソース管理をクラスへ分離し、コンストラクタとデストラクタを利用して、資源の確保と解放をオブジェクトの寿命に対応付けて管理する方法を示した。これは、実用的な C++ プログラムにおいて極めて重要な考え方である。

また、抽象クラスとポリモーフィズムを用いることで、具体的なクラスではなく、「共通の使い方」に基づいてプログラムを構成する方法についても考察した。これにより、柔軟で拡張性の高い設計を実現できることを示した。

加えて、コンポジションを用いることで、クラスを小さな部品として組み合わせ、全体の機能を構築する設計について解説した。実際のソフトウェア設計では、継承だけに依存するのではなく、小さな責任を持つクラスを適切に組み合わせることが重要である。

本章で繰り返し現れた重要な考え方は、「責任の分離」である。すなわち、

- どのクラスが、どの状態を管理するのか
- どのクラスが、どの操作に責任を持つのか
- どの機能を分離し、どの機能を組み合わせるのか

を明確にすることが、良い設計につながる。

クラス設計に絶対的な正解は存在しない。実際のプログラム設計では、規模、性能、可読性、保守性、拡張性など、多くの要因を考慮しながら設計を行う必要がある。しかしながら、本章で扱った考え方は、そのような設計判断を行う際の重要な指針となる。

本章で学んだ内容は、今後の C++ プログラミングにおいて繰り返し登場する。特に、標準ライブラリや各種フレームワークの設計を理解する上でも、ここで扱った「責任の分離」や「インターフェースによる設計」の考え方は重要となる。

次章以降では、これらの設計思想を踏まえながら、C++ のより実践的な機能や標準ライブラリについて解説していく。

■ サンプルプログラム

最後に、本章で扱った内容を踏まえた簡単なサンプルプログラム test220All.cpp を示す。このプログラムでは、

- Money クラスによる値オブジェクトの利用
- TransactionLog クラスによるログ出力と資源管理
- Account クラスによる共通インターフェース
- SavingsAccount クラスによる具体的な口座実装

といった、本章で解説した設計要素を組み合わせている。test220All.cpp は、本章で挙げたサンプルを単純に連結したのではなく、全体を通して作り直したものである。

これまでの説明では、資源管理の流れを明確に示すため、敢えて C 言語形式のファイル操作 (FILE) を用いた例も示したが、ここでは、C++ プログラムとして設計し直し、ファイル出力ストリーム (ofstream) を用いた実装へ整理している。

プログラム：test220All.cpp

```
1  #include <cstdio>
2  #include <cstring>
3  #include <fstream>
4  #include <iostream>
5
6  using namespace std;
7
8  //-----
9  // 金額を表す値オブジェクト
10 //-----
11 class Money {
12     private:
13         int amount;
```

```

14
15 public:
16     Money(int value = 0) {          // コンストラクタは
17         if (value >= 0) {          // 正常な初期値のみ
18             amount = value;        // 受け入れる
19         } else {
20             amount = 0;
21         }
22     }
23
24     int getAmount() const {         // 整数値としての金額の取得
25         return amount;
26     }
27     // 以下はMoneyクラスを意識した処理
28     Money add(const Money& other) const {
29         return Money(amount + other.amount);
30     }
31
32     bool canSubtract(const Money& other) const {
33         return amount >= other.amount;
34     }
35
36     Money subtract(const Money& other) const {
37         return Money(amount - other.amount);
38     }
39 };
40
41 //-----
42 // ログ出力クラス
43 //-----
44 class TransactionLog {
45     private:
46         ofstream fout;          // ログ出力用ストリーム
47         char msg[256];          // ログ出力編集用バッファ
48
49     public:
50         TransactionLog(const char *filename) { // コンストラクタで
51             fout.open(filename, ios::app);    // ファイルを開く
52         }
53
54         ~TransactionLog() {          // デストラクタで
55             if (fout.is_open()) {    // オープンされているストリームを
56                 fout.close();        // 閉じる
57             }
58         }
59         // 預け入れ時のログ出力
60         void writeDeposit(const Money& amount) {
61             sprintf(msg, "deposit: %d", amount.getAmount());
62             fout << msg << endl;
63         }
64         // 引き出し時のログ出力
65         void writeWithdraw(const Money& amount) {
66             sprintf(msg, "withdraw: %d", amount.getAmount());
67             fout << msg << endl;
68         }
69 };
70
71 //-----
72 // 口座インターフェース
73 //-----
74 class Account {
75     public:
76         virtual ~Account() = default;
77
78         virtual void deposit(const Money& amount) = 0;
79         virtual bool withdraw(const Money& amount) = 0;
80         virtual Money getBalance() const = 0;
81 };
82
83 //-----
84 // 普通預金口座
85 //-----

```

```

86 class SavingsAccount : public Account {
87     private:
88         Money balance;
89         TransactionLog log;
90
91     public:
92         // コンストラクタ (口座開設)
93         SavingsAccount(const Money& initial) : balance(initial), log("log.txt") {}
94         // 預け入れ処理の実装
95         void deposit(const Money& amount) override {
96             if (amount.getAmount() > 0) {
97                 balance = balance.add(amount);
98                 log.writeDeposit(amount);
99             }
100         }
101         // 引き出し処理の実装
102         bool withdraw(const Money& amount) override {
103             // 引き出し可能な場合の処理
104             if (amount.getAmount() > 0 && balance.canSubtract(amount)) {
105                 balance = balance.subtract(amount);
106                 log.writeWithdraw(amount);
107                 return true;
108             }
109             // 引き出し不可の場合の処理
110             return false;
111         }
112         // 残高照会
113         Money getBalance() const override {
114             return balance;
115         }
116 };
117
118 //-----
119 // 共通インターフェースによる処理
120 //-----
121 void printBalance(const Account& account) {
122     cout << "balance: "
123          << account.getBalance().getAmount()
124          << endl;
125 }
126
127 //-----
128 // メイン関数
129 //-----
130 int main() {
131     SavingsAccount account(Money(10000)); // 口座開設
132
133     account.deposit(Money(3000));          // 預け入れ
134     account.withdraw(Money(2000));         // 引き出し
135
136     printBalance(account);                 // 残高確認
137 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test220All.cpp のコンパイルと実行

\$ g++ test220All.cpp -o test220All ←コンパイル

\$./test220All ←実行

balance: 11000 ←最終的な残高

\$ cat log.txt ←ログファイルの閲覧

deposit: 3000

withdraw: 2000

16 型を抽象化するための機能

これまでの章では、C++ におけるクラス、継承、ポリモーフィズムなどについて説明してきた。これらの機能を用いることで、様々な概念や対象を「型」として表現し、整理された形でプログラムを記述できるようになる。

しかし、実際にプログラムを作成していると、型は異なるが本質的には同じ処理を繰り返し記述する場面がしばしば現れる。例えば、2つの値を交換する処理や、配列中の最大値を求める処理などは、`int` 型に対しても `double` 型に対しても、論理的には同じ構造で記述できる。

このような場合に C 言語では、扱う型ごとに別々の関数を作成したり、`void` ポインタを用いて型に依存しない形へ一般化する、などといった方法が用いられてきた。しかし、これらの方法では、プログラムが冗長になったり、型安全性が失われやすくなったりする、といった問題が起こる。

C++ では、この問題を解決するために、**テンプレート** (template) と呼ばれる仕組みが導入されている。テンプレートを用いると、具体的な型を指定せずに関数やクラスの雛形を記述し、後に具体的な型を与えて利用することができる。

さらに C++ のテンプレートは、単に「型」を抽象化するだけではない。整数値などの「値」そのものを抽象化し、プログラムの構造に組み込むこともできる。例えば、要素数が異なる配列型や、固定サイズの行列などを、サイズ情報を含めた形で型として表現することが可能である。

このように C++ では、具体的な型や値に依存しない抽象的なプログラムを記述することができる。これは C 言語と比較した場合の C++ の大きな特徴の1つであり、後に解説する STL (Standard Template Library) などの基盤ともなっている。

本章では、まず関数を抽象化するための**関数テンプレート**について説明し、続いてクラスを抽象化するための**クラステンプレート**、さらに、整数値などをテンプレート引数として扱う方法について説明する。

16.1 関数テンプレート

関数テンプレート (function template) とは、扱う型を抽象化した形で関数を定義するための仕組みである。これを用いると、異なる型に対して本質的に同じ処理を行う関数を、一般化された形で1つだけ記述することができる。

例えば、2つの値を交換する処理を考える。C 言語の、型ごとに個別の関数を定義する方法では、`int` 型用、`double` 型用、構造体用、など、扱う型ごとに別々の関数を用意する必要があった。しかし、これらの関数の処理内容は、本質的にはほとんど同じである。

C++ の関数テンプレートを用いると、このような「型だけが異なる同種の処理」を、抽象的な形で統一的に記述することができる。

例. 各種の型の2値（同じ型同士）を交換する関数テンプレート

```
template <typename T>
void swap_value(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
```

この例では、`T` が「各種の型を表すための仮の型名」として用いられている。例えば、`int` 型の変数に対して呼び出した場合には、`int` 型用の交換処理として動作し、`double` 型の変数に対して呼び出した場合には、`double` 型用の交換処理として動作する。

このとき、コンパイラは関数の呼び出しに用いられた引数の型から `T` に対応する型を決定する。その結果、`int` 型の値を渡した場合には `swap_value<int>`、`double` 型の値を渡した場合には `swap_value<double>` に対応する関数が使用される。このように、テンプレートに具体的な型を当てはめて、必要な関数を具体化することを**インスタンス化** (instantiation) という。

関数テンプレートの定義では、まず `template` キーワードを記述し、続いて山括弧「`< >`」の中に**テンプレートパラメータ** (template parameter) を記述する。型を受け取るテンプレートパラメータは、具体的に決定されていない型を受け取るものであり、上の例では、`typename T` によって、「`T` という名前で型を受け取る」ことを表すテンプレ

トパラメータを記述している。

書き方： `template <テンプレートパラメータ>`

テンプレートパラメータとして型を受け取る場合には、

typename 仮の型名

の形式で記述する。この場合の「仮の型名」には、未使用の識別子を任意に割り当てることができる¹⁶⁴。

なお、同様の記述を `typename` の代わりに `class` を用いて、

`template <class T>`

のように書くことも可能である。型を受け取るテンプレートパラメータを宣言する用途においては、両者に意味の違いはない。ただし、本書では、型を受け取ることが明確に伝わる `typename` の書き方を主に用いる。

関数テンプレートは、「後から具体的な型を与えて完成させる関数の雛形」と考えることができる。実際には、関数の呼び出しに記述された引数の型に基づいて、コンパイラが適切な型の関数を具体化して使用する。

`template` キーワードで宣言された仮の型名（先の例では `T`）は、その直後に定義される関数テンプレートやクラステンプレートの中でのみ使用できる。従って、別のテンプレートを定義する場合には、その直前で改めて

`template <typename T>`

のように記述する必要がある。また、その度に同じ名前（例えば `T`）を用いてもよい。

型を受け取るテンプレートパラメータに対して、具体的に指定または推論される型のことを**テンプレート引数**（`template argument`）という。

■ サンプルプログラム

次に示すプログラム `test231-1.cpp` は、先に例示した `swap_value` を実際に使用する例である。`int`、`double` の値のみならず、構造体型の値も、1つの関数テンプレートで交換される様子が確認できる。

プログラム：test231-1.cpp

```
1  #include <iostream>
2  using std::cout;
3
4  struct Student {
5      char name[16];
6      int score;
7  };
8
9  template <typename T>
10 void swap_value(T& a, T& b)
11 {
12     T tmp = a;
13     a = b;
14     b = tmp;
15 }
16
17 int main()
18 {
19     int ix = 10, iy = 20;
20     double dx = 1.25, dy = 3.75;
21     Student s1 = {"Alice", 80}, s2 = {"Bob", 95};
22
23     cout << "int values before swap: " <<
24         "ix = " << ix << ", iy = " << iy << "\n";
25     swap_value(ix, iy);    // 交換処理
26     cout << "int values after swap: " <<
27         "ix = " << ix << ", iy = " << iy << "\n";
28
29     cout << "double values before swap: " <<
30         "dx = " << dx << ", dy = " << dy << "\n";
31     swap_value(dx, dy);    // 交換処理
32     cout << "double values after swap: " <<
33         "dx = " << dx << ", dy = " << dy << "\n";
34
35     cout << "Student values before swap: " <<
```

¹⁶⁴仮の型名を表す場合は `T` が用いられることが多い。


```

36         "s1 = {" << s1.name << ", " << s1.score << "}, " <<
37         "s2 = {" << s2.name << ", " << s2.score << "}\n";
38     swap_value(s1, s2);
39     cout << "Student values after swap: " <<
40         "s1 = {" << s1.name << ", " << s1.score << "}, " <<
41         "s2 = {" << s2.name << ", " << s2.score << "}\n";
42 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test231-1.cpp のコンパイルと実行

```

$ g++ test231-1.cpp -o test231-1  [Enter]    ←コンパイル
$ ./test231-1  [Enter]    ←実行
int values before swap:    ix = 10, iy = 20    ← int の値の
int values after swap:    ix = 20, iy = 10    交換
double values before swap: dx = 1.25, dy = 3.75 ← double の値の
double values after swap: dx = 3.75, dy = 1.25 交換
Student values before swap: s1 = {Alice, 80}, s2 = {Bob, 95} ← 構造体の
Student values after swap: s1 = {Bob, 95}, s2 = {Alice, 80}  交換

```

16.1.1 テンプレートにおける転送参照

関数テンプレートでは、テンプレートパラメータとして宣言された型に `&&` を付けて、

```

template <typename T>
void func(T&& value)

```

のように仮引数を宣言することがある。このような仮引数は、特定の条件を満たす場合に**転送参照** (forwarding reference) と呼ばれる。転送参照を用いると、`std::forward` と組み合わせることで、関数に渡された引数が左辺値であるか右辺値であるかという性質を保ったまま、別の関数へ引き渡すことができる。

ここで使用されている `T&&` は、通常の右辺値参照と同じ記法である。しかし、`T` が関数テンプレートの型推論によって決定される場合には、左辺値と右辺値のどちらも受け取ることができる特別な働きをする。

例. 転送参照を仮引数に持つ関数テンプレート

```

template <typename T>
void relay(T&& value) {
    // value を使用する処理
}

```

この例の `value` は、次のように左辺値と右辺値のどちらも受け取ることができる。

```

int n = 10;
relay(n);    // 左辺値を渡す
relay(20);   // 右辺値を渡す

```

通常の右辺値参照は左辺値を受け取ることができないが、この場合の `T&&` には左辺値である `n` も渡すことができる。これは、引数の種類に応じてテンプレートパラメータ `T` の型が特別な規則で推論されるためである。

左辺値を渡した場合、`T` は左辺値参照を含む型として推論される。例えば、`int` 型の左辺値を渡した場合、`T` は `int&` として推論される。従って、仮引数の型 `T&&` は、形式上、

```
int& &&
```

となる。C++では、このように参照が重なった場合、**参照の折り畳み** (reference collapsing) と呼ばれる規則によって、最終的な参照型が決定される。参照の折り畳みでは、左辺値参照 `&` が1つでも含まれていれば、結果は左辺値参照になる。参照型の組合せと折り畳み後の型との関係を次の表に示す。表中の `U` は、任意の型を表している。

参照を組み合わせた形	折り畳み後の型
<code>U& &</code>	<code>U&</code>
<code>U& &&</code>	<code>U&</code>
<code>U&& &</code>	<code>U&</code>
<code>U&& &&</code>	<code>U&&</code>

この規則により、`int& &&` は `int&` に折り畳まれる。従って、左辺値を渡した場合、仮引数 `value` の型は、最終的に左辺値参照型となる。一方、右辺値を渡した場合には、`T` は参照を含まない通常の型として推論される。例えば、`relay(20)` の場合、`T` は `int` となるため、仮引数の型はそのまま `int&&` となる。

渡される引数	推論される T	仮引数 T&& の最終的な型
int 型の左辺値	int&	int&
int 型の右辺値	int	int&&

このように、転送参照は、引数が左辺値であれば左辺値参照として、引数が右辺値であれば右辺値参照として受け取る。

ただし、`T&&` と記述すれば常に転送参照になるわけではない。転送参照となるのは、`T` が関数呼び出し時に型推論されるテンプレートパラメータであり、`const` や `volatile` などで修飾されておらず、その `T` に直接 `&&` を付けた形で仮引数が宣言されている場合である。例えば、次の仮引数は、`T` に `const` が付けられているため、転送参照ではなく、`const` が付いた右辺値参照である。

```
template <typename T>
void func(const T&& value);
```

また、次の仮引数も、型が既に `int` に決まっているため、通常の右辺値参照であり、転送参照ではない。

```
void func(int&& value);
```

さらに、クラステンプレートに与えたテンプレート引数によって `T` が既に決定されている場合など、関数呼び出しによって `T` が推論されない場面では、`T&&` は転送参照としては扱われない。

転送参照の主な目的は、受け取った引数の性質を保ったまま、別の関数へ引き渡すことである。しかし、仮引数として名前が付けられた変数を、その名前を用いた式として使用すると、その型が右辺値参照であっても左辺値として扱われる。例えば、

```
template <typename T>
void relay(T&& value) {
    destination(value);
}
```

と記述すると、もとの引数が右辺値であった場合でも、`destination` には左辺値として渡される。これは、`value` は名前を持つ変数であり、その名前を用いた式は左辺値となるためである。

もとの引数が左辺値であったか右辺値であったかを保ったまま引き渡すには、関数テンプレート `std::forward` を使用する。`std::forward` は `utility` ヘッダで宣言されており、次のように使用する。

```
#include <utility>

template <typename T>
void relay(T&& value) {
    destination(std::forward<T>(value));
}
```

`std::forward<T>(value)` は、`T` の推論結果に基づいて、もとの引数が左辺値であれば左辺値として、右辺値であれば右辺値として `value` を引き渡す。このように引数の性質を保って別の関数へ渡すことを、**完全転送** (perfect forwarding) と呼ぶ。

`std::forward` は、常に引数を右辺値として扱う関数ではない。左辺値として受け取った引数は左辺値のまま、右辺値として受け取った引数だけを右辺値として転送する。この点が、指定した値を無条件に右辺値として扱える形にする `std::move` との重要な違いである。

例. 左辺値と右辺値を区別して転送する関数テンプレート

```
void receive(int& value) {  
    cout << "lvalue: " << value << "\n";  
}  
  
void receive(int&& value) {  
    cout << "rvalue: " << value << "\n";  
}  
  
template <typename T>  
void relay(T&& value) {  
    receive(std::forward<T>(value));  
}
```

この relay に左辺値を渡すと receive(int&) が呼び出され、右辺値を渡すと receive(int&&) が呼び出される。

```
int n = 10;  
relay(n);           // receive(int&) を呼び出す  
relay(20);          // receive(int&&) を呼び出す
```

この仕組みは、コンストラクタの引数を別のオブジェクトへ渡す処理や、引数を保持せずに別の関数を呼び出す処理などで広く使用される。例えば、thread クラスのコンストラクタは、実行する関数とその引数を様々な型で受け取るために、転送参照を用いた次のような形で宣言されている。

```
template <class F, class... Args>  
thread(F&& f, Args&&... args);
```

ここで、F&& および Args&&... は、実行する関数や関数オブジェクトと、それらに渡す引数を、左辺値と右辺値のどちらからでも効率よく受け取るために使用されている。受け取った関数や引数は、必要に応じてコピーまたはムーブされ、thread オブジェクトの内部に保持される。従って、左辺値を渡した場合に、その変数への参照がそのままスレッドへ渡されるわけではない。変数を参照として渡す方法については、後に thread クラスを扱う箇所で説明する。

16.2 クラステンプレート

クラステンプレート (class template) とは、扱う型を抽象化した形でクラスを定義するための仕組みである。関数テンプレートが、特定の型に固定されない関数の雛形を定義するものであったのに対し、クラステンプレートは、特定の型に固定されないクラスの雛形を定義するものである。

ただし、クラステンプレートは、どのようなクラスでも無条件に一般化できる仕組みではない。有効に使えるのは、扱う型は異なっても、クラスの構造や処理の流れが本質的に同じである場合である。

例えば、値を一定の範囲内に収めて保持するクラスを考える。年齢のような整数値を扱う場合には int 型を用いることが多く、音量や比率のような値を扱う場合には double 型を用いることが多い。このとき、値の具体的な型は異なっても、「値を保持する」、「値を設定する」、「設定された値を最小値と最大値の範囲内に収める」という基本的な構造はほとんど同じである。

このような場合に、int 型用のクラス、long long 型用のクラス、double 型用のクラスなどを別々に定義すると、よく似たクラス定義を何度も記述することになる。クラステンプレートは、このような「型だけが異なる変種をたくさん作る煩わしさ」を軽減するための仕組みとして用いると分かりやすい。

C++ のクラステンプレートを用いると、このような「型だけが異なる同種のクラス」を、抽象的な形で統一的に記述することができる。

例. 値を範囲内に収めて保持するクラステンプレート

```
template <typename T>
class RangeValue {
private:
    T value;
    T min_value;
    T max_value;
public:
    RangeValue(T v, T min_v, T max_v) : min_value(min_v), max_value(max_v) {
        set(v);
    }
    void set(T v) {
        if (v < min_value) {
            value = min_value;
        } else if (v > max_value) {
            value = max_value;
        } else {
            value = v;
        }
    }
    T get() const { return value; }
};
```

この例では、T が各種の型を表すための仮の型名として用いられている。メンバ変数 `value`, `min_value`, `max_value` の型、コンストラクタの引数の型、メンバ関数 `set` の引数の型、`get` の戻り値の型が、全て T によって記述されている。

このクラステンプレートは、それ自体が具体的なクラスを表しているわけではない。実際に使用する際には、次のように山括弧「< >」の中に具体的な型を指定する。

```
RangeValue<int> age(20, 0, 120);
RangeValue<double> volume(0.75, 0.0, 1.0);
```

このように、`RangeValue<int>` と記述すると、`int` 型の値を範囲内に収めて保持するクラスとして使用できる。また、`RangeValue<double>` と記述すると、`double` 型の値を範囲内に収めて保持するクラスとして使用できる。

クラステンプレートに具体的なテンプレート引数を与えた `RangeValue<int>` や `RangeValue<double>` などを、クラステンプレートの**特殊化**¹⁶⁵ (specialization) という。また、プログラム内で必要となった特殊化に対応する具体的なクラスを、クラステンプレートから生成する処理を**インスタンス化** (instantiation) という。上の例では、`RangeValue<int>` と `RangeValue<double>` が必要となるため、これらの特殊化が暗黙的にインスタンス化される。

上の例では、`int` 型と `double` 型で扱う値の種類は異なるが、値を保持し、範囲外の値が与えられたときに最小値または最大値に強制する、という処理の構造は共通している。このような場合にクラステンプレートを用いると、型ごとに似たようなクラスを何度も定義する必要がなくなり、設計を統一的に記述することができる。

ただし、`RangeValue<T>` がどのような型に対しても使えるわけではない。上の例では、`set` メンバ関数の中で

```
v < min_value
v > max_value
value = v
```

のような処理を行っている。従って、T として指定できるのは、少なくとも大小比較や値の代入など、このクラスの内部で用いられている操作が可能な型である。このように、クラステンプレートは「どのような型でも受け取れる」仕組みではなく、クラス定義の内部で必要とされる操作が成立する型に対して使用できる仕組みである。

関数テンプレートの場合には、関数呼び出し時の引数から型が推定されることが多い。これに対してクラステンプレートでは、通常は `RangeValue<int>` や `RangeValue<double>` のように、クラス名の後ろに具体的な型を明示して使用する¹⁶⁶。

クラステンプレートの定義でも、まず `template` キーワードを記述し、続いて山括弧「< >」の中にテンプレート

¹⁶⁵ここでいう**特殊化**は、後に扱う「明示的特殊化」そのものを意味するものではない。

¹⁶⁶C++17 以降では、コンストラクタに与えた引数などからクラステンプレートの型を推定できる場合もある。これをクラステンプレート引数推論という。

パラメータを記述する。その直後に、通常のクラス定義と同じ形式でクラスを定義する。

書き方： `template <テンプレートパラメータ>`
`class クラス名 { ... };`

テンプレートパラメータとして型を受け取る場合には、関数テンプレートの場合と同様に、

`typename 仮の型名`

の形式で記述する。

クラステンプレートは、「後から具体的な型を与えて完成させるクラスの雛形」と考えることができる。実際には、`RangeValue<int>` や `RangeValue<double>` のような特殊化を使用すると、必要に応じてその特殊化がインスタンス化される。

なお、`template` キーワードで宣言された仮の型名（先の例では `T`）は、その直後に定義されるクラステンプレートの内部でのみ使用できる。従って、別のクラステンプレートを定義する場合には、その直前で改めて

`template <typename T>`

のように記述する必要がある。また、複数のテンプレート定義において、そのたびに同じ名前（例えば `T`）を用いてもよい。

クラステンプレートを使用する際に注意すべき点として、具体的な型を与えて作られたクラスは、それぞれ別の型として扱われることが挙げられる。例えば、同じ `RangeValue` というクラステンプレートから作られたものであっても、`RangeValue<int>` と `RangeValue<double>` は C++ の型としては別々の型である。したがって、これらのオブジェクトを同じ型の値として扱うことはできず、このクラス定義のままでは、そのまま互いに代入することもできない。

さらに、異なる型を持つオブジェクト同士の演算や、メンバ関数テンプレートなどを組み合わせると、設計の自由度は高まる一方で、コードの見通しは悪くなりやすい。このような設計は、数値計算ライブラリやコンテナクラスなどでは有効に使われることがあるが、不用意に行くと、型の関係が分かりにくくなることもある。

そのため、入門段階では、クラステンプレートを「型が異なるが、同じ構造を持つクラスをまとめて定義するための仕組み」として理解することを優先するとよい。クラステンプレートは万能の表現手法ではなく、同じ構造を持ちながら型だけが異なる複数のクラスを、統一的に定義するための仕組みとして捉えるのがよい。標準ライブラリに登場する `vector<int>` や `vector<double>` などを読むための基礎として捉え、必要以上に複雑なテンプレート設計をしない方が安全である。

なお、ここで示した `RangeValue` は、あくまでクラステンプレートの考え方を説明するための単純な例であり、実用的な数値範囲管理クラスとしては、最小値と最大値の大小関係なども確認する必要がある。

■ サンプルプログラム

先に挙げたクラステンプレートの使用例を `test231-2.cpp` に示す。

プログラム：test231-2.cpp

```
1  #include <iostream>
2  using std::cout;
3
4  template <typename T>
5  class RangeValue {
6  private:
7      T value;
8      T min_value;
9      T max_value;
10
11 public:
12     RangeValue(T v, T min_v, T max_v) : min_value(min_v), max_value(max_v) {
13         set(v);        // インバリアントの保証はset関数に任せる
14     }
15
16     void set(T v) {
17         if (v < min_value) {
18             value = min_value;
19         } else if (v > max_value) {
20             value = max_value;
```



```

21         } else {
22             value = v;
23         }
24     }
25
26     T get() const {
27         return value;
28     }
29 };
30
31 int main()
32 {
33     RangeValue<int>    age(20, 0, 120);           // intに基づくクラス
34     RangeValue<double> volume(0.75, 0.0, 1.0);    // doubleに基づくクラス
35     // 出力して確認
36     cout << "age\t= " << age.get() << "\n";
37     cout << "volume\t= " << volume.get() << "\n";
38
39     age.set(200);           // 最大値を超える値
40     volume.set(-0.5);       // 最小値を下回る値
41     // 出力して確認
42     cout << "age\t= " << age.get() << "\n";
43     cout << "volume\t= " << volume.get() << "\n";
44 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test231-2.cpp のコンパイルと実行

```

$ g++ test231-2.cpp -o test231-2  ←コンパイル
$ ./test231-2  ←実行
age      = 20           ←正常範囲にある値は
volume    = 0.75        ←そのまま設定される
age      = 120          ←上限を超える値は上限に
volume    = 0           ←下限を下回る値は下限に設定される

```

16.3 値を抽象化するテンプレート

これまでに見てきた関数テンプレートやクラステンプレートでは、主に「型」を抽象化してきた。例えば、T という仮の型名を用いることで、int 型に対しても double 型に対しても同じ形で使える関数やクラスを定義することができた。

しかし、C++ のテンプレートで抽象化できるのは、型だけではない。整数値などの「値」も、テンプレートパラメータとして受け取ることができる。この仕組みを用いると、型だけでなく、配列の要素数や行列の行数・列数のような値も、クラスや関数の定義の一部として扱うことができる。

例えば、固定された個数の要素を持つ配列のようなクラスを考える。要素の型が int である場合と double である場合では、要素の型が異なる。これは型の違いである。一方、5 個の要素を持つ配列と、10 個の要素を持つ配列では、要素数が異なる。これは値の違いである。

通常の変数で要素数を扱う場合、その値はプログラムの実行時に参照される。これに対して、要素数を受け取るテンプレートパラメータを用いると、その値はクラスの型を構成する情報として扱われる。つまり、C++ では、型だけでなく、要素数のような値も、コンパイル時に決まる情報としてプログラムの構造に組み込むことができる。

このような、型ではなく値を受け取るテンプレートパラメータを**非型テンプレートパラメータ**（non-type template parameter）と呼ぶ。

値をテンプレートパラメータとして受け取る場合には、例えば次のように記述する。

```
template <int N>
```

この記述は、「N という名前で整数値を受け取る」ことを表している。ここで N は通常の変数ではなく、値を受け取るテンプレートパラメータの名前である。

次の例は、要素の型と要素数の両方をテンプレートパラメータとして受け取る、固定サイズ配列の簡単な例である。

例. 要素の型と要素数をテンプレートパラメータとして受け取るクラス

```
template <typename T, int N>
class FixedArray {
private:
    T data[N];
public:
    int size() const { return N; }
    T& at(int i) { return data[i]; }
    const T& at(int i) const { return data[i]; }
};
```

この例では、T が要素の型を表すテンプレートパラメータであり、N が要素数を表す非型テンプレートパラメータである。つまり、FixedArray は、要素の型だけでなく、要素数も後から与えて完成させるクラスの雛形である。

補足) なおこの例では、同じ名前の at メンバ関数が 2 つ定義されている。これは、メンバ関数のオーバーロードである。

1 つ目の at は、通常のオブジェクトに対して呼び出され、要素を書き換えることもできる参照を返す。これに対し、2 つ目の at は、関数名の後ろに const が付いており、const オブジェクトに対して呼び出すことができる。

例えば、「FixedArray<int, 5> a;」のような通常のオブジェクトに対して「a.at(0)」と書くと、T& at(int i) が使用される。一方、「const FixedArray<int, 5> b{};」のような const オブジェクトに対して「b.at(0)」と書くと、「const T& at(int i) const」が使用される。この使い分けは、実行時ではなくコンパイル時に決定される。

このクラステンプレートは、次のように使用できる。

```
FixedArray<int, 5> a;
FixedArray<double, 3> b;
```

この場合、「FixedArray<int, 5>」は、int 型の要素を 5 個持つクラスとして使用される。また、「FixedArray<double, 3>」は、double 型の要素を 3 個持つクラスとして使用される。

ここで重要なのは、5 や 3 が単なる実行時の値ではなく、型を構成する情報として扱われることである。例えば、

```
FixedArray<int, 5>
FixedArray<int, 10>
```

は、どちらも int 型の要素を持つ FixedArray であるが、C++ の型としては別々の型である。要素数がテンプレート引数として与えられているため、要素数の違いも型の違いとして扱われる。

このように、値を抽象化するテンプレートを用いると、サイズや個数のような情報を、クラスの型の一部として表現できる。これは、固定サイズの配列、固定サイズの行列、固定長のバッファなどを扱う場合に有用である。例えば、行列を扱うクラスであれば、行数や列数をテンプレート引数として与えることで「Matrix<double, 2, 3>」のように、数値の型だけでなく、行列の大きさも型として表すことができる。

ただし、値をテンプレート引数として与える場合、その値はコンパイル時に決定できるものでなければならない。例えば、次のように実行時に入力された値をそのままテンプレート引数として使うことはできない。

```
int n;
std::cin >> n;
FixedArray<int, n> a;    // エラー
```

テンプレート引数として与えられる値は、コンパイラがプログラムを翻訳する時点で分かる値でなければならない。

従って、値を抽象化するテンプレートは、実行時に自由に変化する値を扱うための仕組みではない。むしろ、配列の要素数や行列の大きさのように、プログラムの構造として固定しておきたい値を、型の一部として表現するための仕組みである。

標準ライブラリにも、この考え方に基づくクラスが存在する。例えば、「std::array<int, 5>」は、int 型の要素を 5 個持つ固定サイズ配列を表す。ここでは、int が型を表すテンプレート引数であり、5 が値を表すテンプレート引数である。このような記法を理解するためにも、値をテンプレート引数として扱う仕組みを知っておくことは重要である。

なお、配列の要素数を定数として扱うだけであれば、C 言語でも #define によるマクロ定義などを用いることができる。しかし、値をテンプレート引数として与える場合には、単に定数を置き換えているのではなく、その値が型を構成する情報として扱われるという点が重要である。

例えば、「#define N 5」は、ソースコード中の N を 5 に置き換えるためのものである。これに対して、「FixedArray<int,

5>」における 5 は、単なる置換文字列ではなく、「int 型の要素を 5 個持つ配列型」を構成する情報である。そのため、「FixedArray<int, 5>」と「FixedArray<int, 10>」は、C++ の型として別々に扱われる。

16.3.1 型の別名を定義する方法

C++ では、using を用いて型の別名を定義することができる。例えば、

```
using Integer = int;
```

と記述すると、Integer という名前を int 型の別名として使用できる。従って、

```
Integer x = 10;
```

は、

```
int x = 10;
```

と同じ意味になる。これは、C 言語の typedef による型の別名定義と同様の目的で使用できる。例えば、C 言語では、

```
typedef int Integer;
```

のように記述することで、Integer を int 型の別名として使用できる。C++ でも typedef は使用できるが、現代的な C++ では、

```
using 別名 = 型;
```

の形式で記述する using による方法がよく用いられる。この形式では、左側に新しく定義する名前、右側に元になる型を書くため、対応関係が分かりやすい。

この機能は、クラステンプレートと組み合わせると特に有用である。例えば、先に示した FixedArray クラスの中で、public のメンバとして次のように記述することを考える。

```
template <typename T, int N>
class FixedArray {
    (省略)
public:
    using value_type = T;      // この部分
    (省略)
```

これは、value_type という名前を、テンプレートパラメータ T が表す型の別名として定義する例である。

例えば、FixedArray<int, 5> では、T は int であるため、

```
FixedArray<int, 5>::value_type
```

は int 型を表す。同様に、FixedArray<double, 3> では、

```
FixedArray<double, 3>::value_type
```

は double 型を表す。このようにしておくと、そのクラスが扱っている要素の型を、クラス自身に付随する情報として参照できるようになる。

ただし、value_type は実行時に問い合わせる値ではなく、コンパイル時に解釈される型の別名である。例えば、

```
FixedArray<int, 5>::value_type x = 10;
```

は、コンパイラによって、

```
int x = 10;
```

と同じ意味として扱われる。つまり、using による型の別名は、実行時に値を保持する変数ではなく、コンパイル時に型を表すための名前である。

これと同様に、要素数のような値についても、クラスに付随する情報として定義しておくことができる。例えば、次のように記述することができる。

```
static constexpr int static_size = N;
```

これは、テンプレートパラメータ N の値を表す定数を、static_size という名前で、クラスに付随する情報として定義する記述である。

ここで用いている constexpr は、値をコンパイル時に決定できる定数として扱うための指定である。例えば、

```
constexpr int n = 5;
```

と記述すると、n は int 型の名前付き定数として扱われ、その値はコンパイル時に決定される。このため、配列の要

素数や、非型テンプレート引数として値を与える場面など、コンパイル時に値が必要な箇所で使用できる。

一方、`const` は、基本的には「値を書き換えられない」ことを表す指定である。これに対して、`constexpr` は、値をコンパイル時に決定できる定数として扱うための指定である。従って、`const` と `constexpr` は似た場面で使われることもあるが、意味は同じではない。テンプレート引数として値を与える場合には、実行時に変化する変数ではなく、コンパイル時に値が分かる定数を用いる必要がある。

また、`#define` によるマクロ定義とも異なる。`#define` はプリプロセッサによる文字列置換であり、C++ の型システムの外側で処理される。これに対して、`constexpr` は、C++ の型システムとスコープの中で扱われる、型を持った名前付きのコンパイル時定数である。直感的には「型とスコープのある `#define` のようなもの」と考えることもできるが、実際には単なる文字列置換ではなく、C++ の言語機能として扱われる定数である点が重要である。

さらに、`constexpr` で定義された定数は、値として使うだけであれば、通常は実行時の変数としてメモリ上に確保する必要はない。例えば、

```
constexpr int N = 5;
```

と定義し、`FixedArray<int, N>` のように使用する場合、`N` はコンパイル時に 5 と分かる値として扱われる。この意味では、`#define N 5` と記述した場合と似た使い方ができる。

ただし、`constexpr` は C++ の定数であり、必要に応じてオブジェクトとしての実体を持つこともある。例えば、

```
constexpr int n = 5;
const int* p = &n;
```

のように、アドレスを取得する使い方をした場合には、`n` の実体が必要になるので、メモリ上に確保される。

従って、「`constexpr` は必ずメモリ上に確保されない」と考えるのは正確ではなく、通常は定数値として使う限り、実行時の変数として扱う必要はないと理解すべきである。

■ メンバの static 宣言

メンバを `static` 宣言すると、そのメンバは個々のオブジェクトではなく、クラスそのものに属する。従って、

```
static constexpr int static_size = N;
```

と記述すると、`static_size` は各オブジェクトが個別に持つメンバ変数ではなく、クラスに付随するコンパイル時定数として定義される。例えば、「`FixedArray<int, 5>`」に対しては、

```
FixedArray<int, 5>::static_size
```

が 5 を表す定数として使用できる。同様に、「`FixedArray<double, 3>`」に対しては、

```
FixedArray<double, 3>::static_size
```

が 3 を表す定数として使用できる。

つまり、「`using value_type = T;`」は、「このクラスが扱う要素の型」を表す情報であり、「`static constexpr int static_size = N;`」は、「このクラスが持つ要素数」を表す情報である。どちらも、実行時に自由に変化する通常のデータではなく、クラスの型に付随するコンパイル時の情報として扱われる。

応用) 以上のように、`using` による型の別名定義と、`static constexpr` による定数定義を組み合わせると、クラステンプレートが扱う型やサイズに関する情報を、クラス自身に付随する情報として表現できる。これは、標準ライブラリのコンテナクラスなどでも広く用いられる考え方である。

■ サンプルプログラム

先に例示した `FixedArray` を用いたプログラム `test231-3.cpp` を次に示す。

プログラム：test231-3.cpp

```
1 #include <iostream>
2 using std::cout;
3
4 template <typename T, int N>
5 class FixedArray {
6     public:
7         using value_type = T;           // 型の別名の定義
8         static constexpr int static_size = N; // 要素数
9 }
```

```

10 private:
11     T data[N];                                // データ格納用配列
12
13 public:
14     // 要素数を返す関数
15     int size() const {
16         return N;
17     }
18     // インデックス指定した要素の参照を返す関数
19     T& at(int i) {
20         return data[i];
21     }
22     const T& at(int i) const {
23         return data[i];
24     }
25 };
26
27 int main()
28 {
29     FixedArray<int, 5>      ia;
30     FixedArray<double, 3>  da;
31
32     // オブジェクト自身が持つサイズ情報を応用した反復処理
33     for (int i = 0; i < ia.size(); i++) ia.at(i) = (i + 1) * 10;
34     for (int i = 0; i < da.size(); i++) da.at(i) = (i + 1) * 0.5;
35
36     cout << "FixedArray<int, 5>\n";
37     cout << "ia.size() = " << ia.size() << "\n";
38     for (int i = 0; i < ia.size(); i++) {
39         cout << "ia.at(" << i << ") = " << ia.at(i) << "\n";
40     }
41     cout << "-----\n";
42     cout << "FixedArray<double, 3>\n";
43     cout << "da.size() = " << da.size() << "\n";
44     for (int i = 0; i < da.size(); i++) {
45         cout << "da.at(" << i << ") = " << da.at(i) << "\n";
46     }
47 }

```

このプログラムでは、FixedArray<int, 5> と FixedArray<double, 3> という2種類のオブジェクトを作成している。前者は int 型の要素を5個持つ固定サイズ配列であり、後者は double 型の要素を3個持つ固定サイズ配列である。いずれも同じクラステンプレート FixedArray<T, N> から作られているが、テンプレート引数として与えた型と値が異なるため、C++ の型としては別々の型として扱われる。

また、size() メンバ関数は、非型テンプレートパラメータ N の値を返している。そのため、FixedArray<int, 5> のオブジェクトでは size() は 5 を返し、FixedArray<double, 3> のオブジェクトでは size() は 3 を返す。このように、値をテンプレート引数として与えることで、要素数のような情報をクラスの型に組み込むことができる。

なお、C 言語の通常の配列では、配列そのものから要素数を取得するための標準的なメンバ関数は用意されていない。配列を定義した直後であれば、sizeof を用いて要素数を計算できる場合もあるが、配列を関数の引数として渡すと、多くの場合、配列は先頭要素へのポインタに変換されるため、要素数の情報は失われる。そのため、C 言語では、配列を関数に渡す際に、要素数も別の引数として渡すのが一般的である。

これに対して、FixedArray<T, N> では、要素数を表す非型テンプレートパラメータ N がクラスの型を構成するために用いられている。そのため、size() メンバ関数を通じて、オブジェクト自身が自分の要素数を返すことができる。これは、単なる配列ではなく、要素型や要素数に関する情報を、クラスの型に付随する形で保持した配列であると見ることができる。

このプログラムをコンパイルして実行する例を次に示す。

例. test231-3.cpp のコンパイルと実行

```

$ g++ test231-3.cpp -o test231-3  Enter  ←コンパイル
$ ./test231-3  Enter  ←実行

```



```

FixedArray<int, 5>          ←出力
ia.size() = 5
ia.at(0) = 10
ia.at(1) = 20
ia.at(2) = 30
ia.at(3) = 40
ia.at(4) = 50
-----
FixedArray<double, 3>
da.size() = 3
da.at(0) = 0.5
da.at(1) = 1
da.at(2) = 1.5

```

ia.size() が 5 を返し、da.size() が 3 を返していることがわかる。

16.4 変数テンプレート

先に、関数の定義を型について抽象化する**関数テンプレート**や、クラスの定義を型について抽象化する**クラステンプレート**について解説した。C++にはこれらの他に、変数の定義をテンプレートとして記述する**変数テンプレート** (variable template) という機能がある。変数テンプレートを用いると、与えられた型や値に応じて、それぞれに対応した変数をコンパイル時に生成することができる。

例えば、同じ意味を持つ定数を float 型、double 型、long double 型など、複数の型について用意したい場合を考える。このような場合、それぞれについて別々の変数を定義する代わりに、変数テンプレートとして 1 つの形にまとめて定義することができる。これによって、使用する側で指定した型に応じた変数を得ることができる。

関数テンプレートが「型などに応じて関数を生成する仕組み」、クラステンプレートが「型などに応じてクラスを生成する仕組み」であるのに対し、変数テンプレートは「型などに応じて変数を生成する仕組み」と考えることができる。特に、型ごとに異なる定数値や型に関する情報を表す値などを定義する場合に有用である。

標準ライブラリでも変数テンプレートは利用されている。例えば、先の「12.4.5 単純なバイトコピーが可能な型：trivially copyable」(p.244～) で使用した `std::is_trivially_copyable_v<T>` は、型 T がトリビアルにコピー可能であるかどうかを表す値を提供する変数テンプレートの一例である。

変数テンプレートは、`template` 宣言に続けて変数を定義する形で記述する。基本的な書き方を次に示す。

書き方： `template <typename T>`
T 変数名 = 初期値;

ここで T はテンプレートパラメータであり、変数を使用する際に具体的な型を指定する。次のように、変数テンプレートの名前に型を指定して使用する。

書き方： **変数名<型>**

変数テンプレートは C++ の言語機能であり、定義そのものを使用するために特別なヘッダを読み込む必要はない。ただし、標準ライブラリが提供する変数テンプレートを使用する場合は、それぞれの機能が宣言されているヘッダを読み込む必要がある。

次に示すプログラム `test231-4.cpp` は、精度の異なる浮動小数点数の型で円周率を保持する変数を作成する例である。

プログラム： `test231-4.cpp`

```

1  #include    <iostream>
2  #include    <iomanip>
3
4  template <typename T>
5  constexpr T pi = T(3.141592653589793238462643383279502884L);
6
7  int main()
8  {
9      std::cout << std::setprecision(25);
10
11      std::cout << "float:\t\t" << pi<float> << '\n';
12      std::cout << "double:\t\t" << pi<double> << '\n';

```

```

13 |         std::cout << "long double:\t" << pi<long double> << '\n';
14 |     }

```

このプログラムでは、変数テンプレート `pi` を定義し、テンプレートパラメータ `T` に指定された型で円周率の値を保持するようにしている。 `pi<float>`, `pi<double>`, `pi<long double>` のように型を指定することで、それぞれ `float` 型, `double` 型, `long double` 型の変数が別々に生成される。

このプログラムをコンパイルして実行する例を次に示す。

例. test231-4.cpp のコンパイルと実行

```

$ g++ test231-4.cpp -o test231-4  [Enter]    ←コンパイル
$ ./test231-4  [Enter]    ←実行
float:          3.1415927410125732421875    ←型の精度ごとに
double:         3.141592653589793115997963    ←円周率と一致する桁数が
long double:    3.141592653589793238512809    ←異なる

```

注意) 実際の表示の形式、精度は環境ごとに異なる。

この例から、同じ意味を持つ値について、使用する型だけが異なる複数の変数を、1 つの変数テンプレートから生成できることがわかる。また、浮動小数点数では型によって保持できる精度が異なるため、 `pi<float>`, `pi<double>`, `pi<long double>` では、それぞれ円周率の値を保持できる精度も異なる。

上の例では、変数テンプレート `pi` の定義時に `constexpr` を指定しているため、生成される変数はコンパイル時定数として扱われる。これに対して、 `constexpr` や `const` を指定しなければ、通常の変数と同様に、代入によって値を変更することができる。

16.5 型安全な列挙型：enum class

C++ でも従来の `enum`¹⁶⁷ を使用できるが、C++11 以降では、より型安全な列挙型として `enum class` が用意されている。 `enum class` は、名前付きの値を列挙するための仕組みであり、基本的な値の割当て規則は従来の `enum` とよく似ている。すなわち、明示的に値を指定しない場合、先頭の列挙子には 0 が対応し、以降の列挙子には 1 ずつ増加した整数値が対応する。また、途中の列挙子に値を指定した場合には、その後の列挙子には、指定された値から 1 ずつ増加した値が対応する。

例えば、次のように `enum class` を定義できる。

```

enum class Color {
    Red,
    Green,
    Blue
};

```

この場合、 `Color::Red`, `Color::Green`, `Color::Blue` は、それぞれ整数値 0, 1, 2 に対応する。ただし、 `enum class` の列挙子は、従来の `enum` とは異なり、 `Red` のように単独では使用できない。使用する際には、 `Color::Red` のように、列挙型名を付けて指定する。

この性質により、列挙子名の衝突を避けることができる。例えば、色を表す列挙型と、交通信号を表す列挙型の両方で `Red` という列挙子名を使用したい場合でも、 `Color::Red`, `Signal::Red` のように区別して扱うことができる。

`enum class` のもう 1 つの重要な特徴は、列挙値が整数型へ暗黙に変換されないことである。従来の `enum` では、列挙型の値が整数型へ暗黙に変換されるが、 `enum class` では、次のような代入はできない。

```

int n = Color::Red;    ←不可

```

`enum class` の値を整数値として取り出したい場合には、明示的な型変換を行う。

```

int n = static_cast<int>(Color::Red);

```

このように、 `enum class` は内部的には整数値に対応しているが、通常の整数値とは区別された独立した型として扱われる。そのため、整数との混同を避け、列挙型をより安全に使用できる。

また、 `enum class` では、基になる整数型を指定することもできる。例えば、次のように記述できる。

¹⁶⁷ 「9.10 列挙型：名前付き整数の扱い」(p.144) で解説した。

```
enum class Mode : unsigned char {  
    Read, Write, Execute  
};
```

この例では、Mode の基になる整数型として `unsigned char` が用いられる。ただし、特に必要がなければ基になる整数型を明示せずに使用しても良い。

以上のように、`enum class` は、従来の `enum` と同様に名前付きの整数値を定義する仕組みであるが、列挙子名が型名の内側に属し、整数型へ暗黙に変換されない点でより型安全である。現代的な C++ では、単純な整数値ではなく、意味のある値の集合を型として表したい場合に、`enum class` を使用するのが望ましい。

17 ラムダ式

通常の関数を定義する場合、関数名、引数、戻り値の型、関数本体を明示して、独立した関数として記述する。例えば、整数値の 2 乗を求める関数は、次のように定義できる。

```
int square(int x) {  
    return x * x;  
}
```

このような関数定義は、何度も使用する処理に名前を付けて再利用する場合に適している。一方、プログラム中では、その場で一度だけ使う短い処理や、別の関数に引数として渡すための小さな処理を記述したい場合がある。このような場合に、毎回、独立した関数を定義すると、プログラム全体の見通しが悪くなることもある。

C++ では、このような小さな処理をその場で記述するために、**ラムダ式**が用意されている。ラムダ式は、名前を持たない小さな関数のような処理を、式として記述するための機能である。例えば、整数値の 2 乗を求める処理は、ラムダ式を用いて次のように記述できる。

```
[](int x) {  
    return x * x;  
}
```

この記法では、通常関数定義のように関数名を付けず、その場で処理内容を記述している。ラムダ式は、変数に代入して後で呼び出すこともでき、また、STL のアルゴリズムなどに処理内容として直接渡すこともできる。

ラムダ式は見た目には小さな関数のように見えるが、実際には単なる関数定義ではない。ラムダ式は、関数呼び出し演算子 `operator()` を持つ無名の型のオブジェクトを生成する式であり、そのオブジェクトは関数オブジェクトとして働く、と考えることができる。そのため、通常関数とは異なり、外側の変数を取り込んで処理に利用することもできる。この仕組みを**キャプチャ**と呼ぶ。

本章では、まずラムダ式の基本的な書き方と呼び出し方を説明し、次に外側の変数を利用するキャプチャについて説明する。さらに、ラムダ式によって生成されるオブジェクトを関数オブジェクトとして理解し、最後に STL アルゴリズムとの関係について簡単に触れる。

17.1 ラムダ式の基本

ラムダ式は、その場で小さな関数のような処理を記述するための式である。通常関数定義では、関数名を付けて独立した関数を定義するが、ラムダ式では、名前を付けずに処理そのものを式として記述する。ラムダ式の基本的な書き方は次の通りである。

書き方： `[] (引数リスト) { 処理 }`

先頭の `[]` は**キャプチャリスト**と呼ばれる部分である。外側の変数を使用しない場合には、空の `[]` を記述する。丸括弧 `()` の中には、通常関数と同様に仮引数を記述する。波括弧 `{ }` の中には、そのラムダ式が呼び出されたときに実行する処理を記述する。

例えば、整数値の 2 乗を返す処理は、ラムダ式を用いて次のように記述できる。

```
[](int x) {  
    return x * x;  
}
```

このラムダ式は、`int` 型の引数 `x` を受け取り、その 2 乗を返す処理を表している。ただし、このままではラムダ式を記述しただけであり、呼び出すための名前は付いていない。ラムダ式を後から呼び出して使用したい場合には、それを `auto` を用いて変数に代入すると良い。

```
auto square = [](int x) {  
    return x * x;  
};
```

この例では、整数値の 2 乗を求めるラムダ式を `square` という変数に代入している。このようにして得られた `square` は、次のようにして通常関数のように呼び出すことができる。

```
square(5)
```

この呼び出しでは、引数 5 がラムダ式の仮引数 `x` に渡され、`x * x` の値が返される。従って、`square(5)` の値は 25 となる。

ラムダ式には、複数の引数を指定することもできる。例えば、2つの整数値の和を返すラムダ式は、次のように記述できる。

```
auto add = [](int a, int b) {  
    return a + b;  
};
```

この場合、`add(3,4)` の値は 7 となる。このように、ラムダ式は通常の間関数と似た形式で引数を受け取り、値を返す処理を簡潔に記述できる。ただし、ラムダ式は独立した関数定義ではなく、式として記述される点が通常の間関数定義とは異なる。

ラムダ式の型は、ラムダ式ごとにコンパイラが内部的に生成する固有の型である。その型名をプログラム中で直接記述することは通常できないため、ラムダ式を変数に保持する場合には `auto` を用いるのが一般的である。ラムダ式を保持した変数は、関数呼び出し演算子 `operator()` を持つオブジェクトとして扱われ、上記の `square(5)` のように呼び出すことができる。

次に示すプログラム `test232-1.cpp` は、ラムダ式の機能を示す単純な例である。

プログラム：test232-1.cpp

```
1  #include <iostream>  
2  using std::cout;  
3  
4  int main() {  
5      auto square = [](int x) {  
6          return x * x;  
7      };  
8  
9      auto add = [](int a, int b) {  
10         return a + b;  
11     };  
12  
13     cout << "square(5): " << square(5) << "\n";  
14     cout << "add(3,4): " << add(3, 4) << "\n";  
15 }
```

このプログラムは、ラムダ式を変数に代入し、通常の間関数と同じような形式で呼び出す例である。`square` には、整数値を 1 つ受け取り、その 2 乗を返すラムダ式を代入している。また、`add` には、2つの整数値を受け取り、その和を返すラムダ式を代入している。

このプログラムをコンパイルして実行する例を次に示す。

例. test232-1.cpp のコンパイルと実行

```
$ g++ test232-1.cpp -o test232-1  ←コンパイル  
$ ./test232-1  ←実行  
square(5): 25 ←出力  
add(3,4): 7
```

17.1.1 ラムダ式の即時呼び出し

ラムダ式は式として記述されるため、変数に代入せずに、定義した直後に呼び出すこともできる。これを、ここでは**ラムダ式の即時呼び出し**と呼ぶことにする。例えば、整数値の 2 乗を返すラムダ式をその場で呼び出すには、次のように記述できる。

```
[](int x) {  
    return x * x;  
}(5)
```

この例では、ラムダ式を定義した直後に、引数 5 を与えて呼び出している。その結果、仮引数 `x` に 5 が渡され、`x * x` の値である 25 が返される。

この記法を出力式の中で使用すると、次のように書ける。

```
cout << [](int x) {  
    return x * x;  
}(5) << "\n";
```


この場合、ラムダ式の戻り値である 25 が cout に送られる。

ただし、即時呼び出しの形は、処理内容が長くなると読みにくくなることがある。そのため、何度も使う処理や、意味を明確にしたい処理については、ラムダ式を auto で変数に保持してから呼び出す方法や、通常の名前付き関数として定義する方法を検討した方がよい。

17.2 外部の変数の利用：キャプチャ

ラムダ式を囲むスコープで宣言された局所変数を、ラムダ式の内部で使用する場合には、その変数をどのように取り込むかを指定する必要がある。この指定を**キャプチャ**と呼ぶ。キャプチャは、ラムダ式の先頭にある角括弧 [] の中に記述する。外側の変数を使用しない場合には空の [] を記述する。

書き方： [外部の変数のリスト] (引数リスト) { 処理 }

このように記述すると、ラムダ式は「外部の変数のリスト」に列挙された外側の変数を利用できる。変数名だけを記述した場合には値としてキャプチャされ、変数名の前に & を付けた場合には参照としてキャプチャされる。次の例は、外側で宣言された整数値 a を用いて、引数 x に a を加えた値を返すラムダ式である。

```
int a = 10;
auto f = [a](int x) {
    return a + x;
};
```

この例では、ラムダ式のキャプチャリストに a を指定しているため、ラムダ式の内部で外側の変数 a を使用できる。従って、f(5) を呼び出すと、a + x の値、すなわち 15 が返される。

ただし、この場合は外側の変数 a は値としてキャプチャされる。これは、ラムダ式が作られた時点の a の値が、ラムダ式の内部にコピーされることを意味する。そのため、ラムダ式を作成した後で外側の変数 a の値を変更しても、既に作られたラムダ式の中で使用される値は変化しない。これを**値によるキャプチャ**と呼ぶ。例えば、次のような場合を考える。

```
int a = 10;
auto f = [a]() {
    return a;
};
a = 20;
cout << f();
```

この場合、f が作られた時点での a の値 10 がラムダ式の内部にコピーされる。その後、外側の変数 a に 20 を代入しても、f() の戻り値は 10 である。

一方、外側の変数そのものを参照して利用したい場合には、変数名の前に & を付けて**参照**の形にする。例えば、外側の変数 a を参照としてキャプチャするには、次のように記述する。

書き方の例： [&a] (引数リスト) { 処理 }

このように記述すると、ラムダ式の内部から外側の変数 a を参照できる。これを**参照によるキャプチャ**と呼ぶ。参照によるキャプチャでは、ラムダ式の内部で外側の変数を参照し、必要に応じて変更することもできる。

```
int a = 10;
auto g = [&a]() {
    return a;
};
a = 20;
cout << g();
```

この例では、g は外側の変数 a を参照している。そのため、外側の a に 20 を代入した後で g() を呼び出すと、戻り値は 20 となる。

キャプチャの基本的な指定方法を表 44 に示す。

[=] は、ラムダ式の内部で使用する外側の変数を値としてまとめてキャプチャする指定であるが、外側のスコープに存在する全ての変数を無条件にコピーすることを意味しない。すなわち、ラムダ式の本体で使用される変数が必要に応じてキャプチャされる。

同様に、[&] は、ラムダ式の内部で使用する変数を、外側の変数の参照としてまとめてキャプチャする指定である。ただし、どの変数をキャプチャしているかがコード上で見えにくくなるため、[a] や [&a] のように必要な変数を明

表 44: ラムダ式におけるキャプチャの基本

記述例	意 味
[]	外側の変数を使用しない
[a]	変数 a を値としてキャプチャする
[&a]	変数 a を参照としてキャプチャする
[a,b]	変数 a と b を値としてキャプチャする
[&a,&b]	変数 a と b を参照としてキャプチャする
[a,&b]	(混在するケース) 変数 a を値として、変数 b を参照としてキャプチャする
[=]	使用する外側の変数を値としてまとめてキャプチャする
[&]	使用する外側の変数を参照としてまとめてキャプチャする

示する書き方の方が良い場合がある。

キャプチャを用いることで、ラムダ式は外側の状況に応じた処理をその場で作ることができる。ただし、参照によるキャプチャでは、ラムダ式が参照している変数の寿命に注意する必要がある。外側の変数が既に存在しなくなった後でラムダ式を呼び出すと、不正な参照を行う原因になるためである。

17.3 ラムダ式の実体

これまで、ラムダ式を「その場で小さな関数のような処理を記述するための式」として説明してきた。実際、ラムダ式を変数に代入すると、その変数は通常の変数と同じような形式で呼び出すことができる。例えば、次のような記述ができる。

```
auto square = [](int x) {
    return x * x;
};
cout << square(5);
```

この例では、square(5) のように、square を関数のように呼び出している。しかし、ラムダ式は通常の変数定義そのものではない。ラムダ式によって、コンパイラが内部的に生成する無名の型のオブジェクトが作られ、そのオブジェクトが関数オブジェクトとして働く、と考えることができる。

関数オブジェクトとは、関数呼び出し演算子 operator() を持つオブジェクトである。クラスに operator() を定義すると、そのクラスのオブジェクトを関数のように呼び出すことができる¹⁶⁸。ラムダ式も、この仕組みに準えて理解することができる。すなわち、ラムダ式を保持した変数 square に対する呼び出し square(5) は、概念的には次の呼び出しに相当する。

```
square.operator()(5)
```

従って、ラムダ式は「関数のように見えるもの」ではあるが、ラムダ式によって生成されるオブジェクトは operator() を持つ関数オブジェクトである、と考えるとよい。例えば、先の square のラムダ式は、概念的には次のような関数オブジェクトと見ることができる。

```
class Square {
public:
    int operator()(int x) const {
        return x * x;
    }
};
```

もちろん、実際に Square という名前のクラスをプログラマが定義しているわけではない。すなわち、ラムダ式ごとにコンパイラが内部的に固有の型を生成していると考えられる。そのため、ラムダ式の型名をプログラム中で直接記述することは通常できず、ラムダ式を変数に保持する場合には auto で宣言するのが一般的である。

キャプチャを行うラムダ式についても、同様に関数オブジェクトとして理解できる。例えば、次のようなラムダ式を考える。

¹⁶⁸ 「12.8.3 関数のように呼び出せるオブジェクト：operator()」(p.251～) で解説している。

```
int a = 10;
auto f = [a](int x) {
    return a + x;
};
```

この場合、a は値としてキャプチャされるため、ラムダ式が作られた時点の値が内部にコピーされる。これは、概念的には、関数オブジェクトがデータメンバとして a の値を保持していると解釈することができる。

このように、ラムダ式は通常の関数と同じような形式で呼び出すことができるが、ラムダ式によって生成されるオブジェクトは operator() を持つ関数オブジェクトとして働く。このように理解しておく、ラムダ式を STL のアルゴリズム¹⁶⁹ に渡せる理由も理解しやすくなる。STL の多くのアルゴリズムは、条件判定や比較処理として、関数や関数オブジェクトを受け取ることができるためである。

■ サンプルプログラム

次に示すプログラム test232-2.cpp では、add2、dbl、sqr、root という 4 つのラムダ式を定義している。それぞれ、受け取った値に 2 を加える処理、2 倍する処理、2 乗する処理、平方根を求める処理を表している。

プログラム：test232-2.cpp

```
1  #include    <iostream>
2  #include    <cmath>
3  using std::cout;
4
5  // ラムダ式を受け取ってそれを実行する関数
6  template <typename F>
7  double evalLambda(F f) {
8      return f(3.0);
9  }
10
11 int main() {
12     auto add2 = [](double x){ return x + 2.0; }; // 2を足すラムダ式
13     auto dbl = [](double x) { return x * 2.0; }; // 2倍するラムダ式
14     auto sqr = [](double x) { return x * x; };   // 2乗するラムダ式
15     auto root = [](double x) { return std::sqrt(x); }; // 平方根を求めるラムダ式
16     // ラムダ式を「値」として関数に渡す
17     cout << "evalLambda(add2): " << evalLambda(add2) << "\n";
18     cout << "evalLambda(dbl): " << evalLambda(dbl) << "\n";
19     cout << "evalLambda(sqr): " << evalLambda(sqr) << "\n";
20     cout << "evalLambda(root): " << evalLambda(root) << "\n";
21 }
```

関数 evalLambda は、引数として受け取ったラムダ式 f を、値 3.0 に対して実行し、その結果を返す。ラムダ式は関数のように呼び出せるオブジェクトであるため、f(3.0) のように記述できる。この例から、ラムダ式を値として関数に渡し、その関数の内部で実行できることがわかる。

ここで重要なのは、evalLambda の処理内容そのものは変えずに、渡すラムダ式を変えるだけで、異なる計算を実行できるという点である。例えば、add2 を渡せば 3.0 に 2 を加えた値が返され、dbl を渡せば 3 を 2 倍した値が返される。同様に、sqr を渡せば 3 の 2 乗が返され、root を渡せば 3 の平方根が返される。

このように、ラムダ式を利用すると、処理の一部を外部から与える形のプログラムを簡潔に記述できる。このプログラムをコンパイルして実行する例を次に示す。

例. test232-2.cpp のコンパイルと実行

```
$ g++ test232-2.cpp -o test232-2  Enter    ←コンパイル
$ ./test232-2  Enter    ←実行
evalLambda(add2): 5      ← 2 を加える処理
evalLambda(dbl): 6      ← 2 倍する処理
evalLambda(sqr): 9      ← 2 乗する処理
evalLambda(root): 1.73205 ←平方根を求める処理
```

ラムダ式によって生成される関数オブジェクトを、処理を表すデータとして受け渡す考え方は、後に説明する STL アルゴリズムに、条件判定や比較処理を渡す場合にも利用される。

¹⁶⁹ 「20 STL の基礎」(p.384～) で解説する。

18 並行処理の基礎

これまでの解説では、基本的に、記述された処理が一定の順序に従って1つずつ実行されていた。例えば、ある計算を行い、その結果を表示し、続いて次の計算を行うというように、プログラムの各処理は順番に進められる。このような実行方式は**逐次処理** (sequential processing) と呼ばれる。

逐次処理では、原則として、ある処理が終了した後に次の処理が開始される。従って、プログラムの実行順序を比較的容易に把握することができる。これまでに扱ってきた条件分岐や反復、関数呼び出しなども、実行される経路は変化するものの、ある時点でプログラム内で実行されている処理は1つである、という考え方を基本としていた。

これに対して、**並行処理** (concurrent processing) では、複数の処理を別々に進行させることを考える。例えば、時間のかかる計算を行いながら別の処理を実行したり、複数のデータを異なる処理単位で同時期に扱ったりすることができる。

複数の処理が並行して進行する場合、それらが必ずしも物理的に同時に実行されるとは限らない。1つのCPUコア上で複数の処理が短い時間ごとに切り替えられ、時分割で実行される¹⁷⁰ 場合もある。一方、複数のCPUコアを備えた環境では、OSのスケジューラの判断によって、異なる処理が別々のCPUコアに割り当てられ、実際に同時に実行されることもある。このように、複数の処理が同時期に進行することを並行実行といい、複数の処理が物理的に同時に実行されることを**並列実行** (parallel execution) という。

並行処理は、単にプログラムの一部を高速化するための技法ではない。複数の処理が別々に進行するという、これまでは異なる実行モデルを導入するものである。このため、処理が実行される順序が常に一定であるとは限らず、同じプログラムであっても、実行のたびに処理の前後関係が変化することがある。

また、複数の処理が同じデータを読み書きする場合には、処理の実行順序によって結果が変化したり、データが不正な状態になったりすることがある。このような問題を避けるには、複数の処理の間で実行の順序やタイミングを調整し、共有するデータを適切に管理する必要がある。

C++では、`thread` を使用して複数の処理を実行することができる。また、`mutex` や `atomic` など、並行処理を安全に行うための機能も標準ライブラリに用意されている。本節では、これらの機能を用いた基本的な並行処理について説明する。

なお、並行処理には、高度で複雑な設計が必要となる場合も多い。ここでは、並行処理の基本的な考え方を理解することを目的とし、複数のスレッドの実行、終了待ち、共有データの保護など、最も基本的な事柄に限定して扱う。

18.1 並行処理の実行単位：thread

C++では、並行して実行する処理の単位として**スレッド** (thread) を使用する。標準ライブラリには、スレッドを表すクラスとして `std::thread` が用意されている。`thread` を使用するには、ヘッダファイル `thread` を読み込む。

`thread` オブジェクトを生成するときには、新しいスレッド上で実行する関数などをコンストラクタに指定する。

書き方： `std::thread` スレッドオブジェクト名 (呼び出し可能対象, 引数 1, 引数 2, ...);

このコンストラクタの概略的な形は、次のように表すことができる。

```
template<class F, class... Args>
thread(F&& f, Args&&... args);171
```

第1引数 `f` には、新しいスレッドで実行する対象を指定する。これには、通常関数、ラムダ式、関数オブジェクト、クラスのメンバ関数など、関数のように呼び出すことのできるものを指定できる。このような、関数のように呼び出すことのできる対象を、総称して**呼び出し可能対象** (callable) と呼ぶ¹⁷²。

第2引数以降の `args` には、呼び出し可能対象に渡す引数を指定する。引数が不要な場合は、呼び出し可能対象だけを指定する。例えば、次のような引数を取らない関数 `task` が定義されている場合を考える。

```
void task() {
    // 新しいスレッドで実行する処理
}
```

¹⁷⁰ この場合の切り替えのタイミングは OS のスケジューラが判断する。

¹⁷¹ ここで、`F&&` および `Args&&` は、「16.1 関数テンプレート」(p.322～) で解説した**転送参照**である。

¹⁷² 文献によっては**実行可能対象**と呼ぶこともある。

この関数を新しいスレッドで実行するには、次のように `thread` オブジェクトを生成する。

```
std::thread t(task);
```

この文が実行されると、`t` という `thread` オブジェクトが生成されると同時に、新しいスレッドの実行が開始され、そのスレッド上で関数 `task` が呼び出される。すなわち、C++ の `thread` では、オブジェクトを生成した後に、別途、実行開始のためのメンバ関数を呼び出す必要はない。

実行する関数が引数を受け取る場合は、関数名に続けて、その関数に渡す値を指定する。例えば、次のような関数を考える。

```
void task(const std::string& name) {  
    // name を使用する処理  
}
```

この関数に文字列 `"Thread-1"` を渡して実行するには、次のように記述する。

```
std::thread t(task, "Thread-1");
```

この文によって、新しいスレッド上で、実質的に次の関数呼び出しに相当する処理が行われる。

```
task("Thread-1");
```

ただし、この関数呼び出しは、`thread` オブジェクトを生成した元の処理とは別のスレッド上で実行される。通常、関数 `main` を実行しているスレッドは**メインスレッド**と呼ばれる。`thread` オブジェクトを生成すると、メインスレッドとは別に新しいスレッドが開始され、両者は並行して処理を進める。

どちらのスレッドが先に処理を進めるか、また、どの時点で実行が切り替わるかは一定ではない。それらは、実行環境や OS のスケジューラの判断などによって決まる。従って、`thread` を使用したプログラムでは、複数の処理が常に記述した順序で実行されるとは限らない。

18.2 スレッドの終了待ち：join

`thread` オブジェクトを生成すると、新しいスレッドの実行が開始されるが、その処理が終了するまで、メインスレッドが自動的に待機するわけではない。生成したスレッドの終了を待つには、`thread` クラスのメンバ関数 `join()` を使用する。

書き方： スレッドオブジェクト名.`join()`;

例えば、次のように記述する。

```
std::thread t(task);  
t.join();
```

`join()` が呼び出されると、`join()` を呼び出したスレッドは、対象となるスレッドの実行が終了するまで待機する。上の例では、メインスレッドは、関数 `task` を実行しているスレッドの終了を待ち、その後に `join()` より後の処理を再開する。

複数のスレッドを生成した場合は、それぞれの `thread` オブジェクトに対して `join()` を呼び出す。例えば、次の記述では、関数 `task1` と `task2` を別々のスレッドで実行し、両方のスレッドの終了を待つ。

```
std::thread t1(task1);  
std::thread t2(task2);  
t1.join();  
t2.join();
```

`t1.join()` によってメインスレッドが `t1` の終了を待っている間にも、`t2` の処理は並行して進行する。従って、`join()` を呼び出した順序と、各スレッドが実際に終了する順序が一致するとは限らない。例えば、`t2` の処理が先に終了していた場合、その後に `t2.join()` が呼び出されると、対象のスレッドはすでに終了しているため、終了待ちは直ちに完了し、次の処理へ進む。

スレッドとの結び付きを保ったまま、対応する `thread` オブジェクトを破棄してはならない。厳密に言うと、`joinable()` が `true` の状態のまま、対応する `thread` オブジェクトを破棄してはならない。

`joinable()` が `true` である `thread` オブジェクトが破棄されると、`std::terminate()` が呼び出され、プログラムは異常終了する。例えば、次のプログラムは正常には終了しない。


```
int main() {
    std::thread t(task);
}
```

関数 `main` の終了時には、ローカル変数である `t` のデストラクタが呼び出される。この時点で `t.joinable()` が `true` であるため、`thread` のデストラクタから `std::terminate()` が呼び出される。

ここで注意が必要なのは、`joinable()` が `true` であることは、そのスレッドが現在も実行中であることを意味するものではないという点である。スレッド上で実行した関数の処理がすでに終了していても、`join()` などの処理を行っていないければ、`joinable()` は `true` のままである。従って、処理が短時間で終了する場合であっても、基本的には `join()` などを用いて、`thread` オブジェクトとスレッドとの結び付きを適切に解消する必要がある。

`thread` オブジェクトがスレッドとの結び付きを保っているかどうかは、メンバ関数 `joinable()` によって調べることができる。

書き方： スレッドオブジェクト名.`joinable()`

`joinable()` は、その `thread` オブジェクトがスレッドと結び付いている場合に `true` を返し、そうでない場合に `false` を返す。例えば、次のようにして、スレッドとの結び付きを保っている場合にだけ `join()` を呼び出すことができる。

```
if (t.joinable()) {
    t.join();
}
```

`join()` を一度呼び出して終了待ちを完了すると、その `thread` オブジェクトはスレッドとの結び付きを失い、`joinable()` は `false` を返す。同じ `thread` オブジェクトに対して `join()` を繰り返し呼び出すことはできない。

■ サンプルプログラム

次のプログラム `test280-1.cpp` では、関数 `task1` と `task2` を、それぞれ別のスレッド上で実行する。関数 `task1` は 1 秒間隔で「Thread-1」を 8 回表示し、関数 `task2` は 2 秒間隔で「Thread-2」を 3 回表示する。メインスレッドでは、これら 2 つのスレッドを開始した後、`join()` を使用して、それぞれの終了を待つ。

プログラム： `test280-1.cpp`

```
1  #include    <iostream>
2  #include    <string>
3  #include    <chrono>
4  #include    <thread>
5
6  void task1(const std::string& name) {
7      for (int i = 0; i < 8; i++) {
8          std::cout << name << std::endl;
9          std::this_thread::sleep_for(std::chrono::seconds(1));
10     }
11 }
12
13 void task2(const std::string& name) {
14     for (int i = 0; i < 3; i++) {
15         std::cout << "\t" << name << std::endl;
16         std::this_thread::sleep_for(std::chrono::seconds(2));
17     }
18 }
19 int main() {
20     // スレッドの生成と実行開始
21     std::thread t1(task1, "Thread-1");
22     std::thread t2(task2, "Thread-2");
23
24     // スレッドの終了待ち
25     t2.join();
26     std::cout << "\tThread-2 ended" << std::endl;
27
28     t1.join();
29     std::cout << "Thread-1 ended" << std::endl;
30 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test280-1.cpp のコンパイルと実行

```
$ g++ test280-1.cpp -o test280-1  [Enter]    ←コンパイル
$ ./test280-1  [Enter]    ←実行
Thread-1                                ←出力
    Thread-2
Thread-1
    Thread-2
Thread-1
Thread-1
    Thread-2
Thread-1
Thread-1
    Thread-2 ended    ←この時点で task2 が終了
Thread-1
Thread-1
Thread-1 ended    ←この時点で task1 が終了
```

2つのスレッドが並行して処理を進めるため、それぞれのメッセージが入り交じって表示される。なお、「Thread-2 ended」と「Thread-1 ended」は各スレッド自身が表示しているのではなく、join() による終了待ちを終えたメインスレッドが表示している。

どちらのスレッドがどの時点で実行されるかは、OS のスケジューラの判断などによって変化する。従って、実際に表示されるメッセージの順序や間隔は、上に示す実行例と若干異なることがある。

18.3 スレッドの切り離し：detach

thread オブジェクトを生成すると、その処理を行ったスレッドとは別に、新しいスレッドが開始される。例えば、関数 main の中で thread オブジェクトを生成した場合、関数 main を実行しているメインスレッドが、新しいスレッドを生成した側となる。一方、指定された関数を実行する新しいスレッドが、生成された側となる。

前項で説明した join() は、スレッドを生成した側が、生成された側のスレッドの処理が終了するまで待機するためのメンバ関数である。これに対して、生成された側のスレッドの終了を待たず、そのスレッドを対応する thread オブジェクトから切り離す場合には、メンバ関数 detach() を使用する。

書き方： スレッドオブジェクト名.detach();

例えば、メインスレッドから次のように記述した場合を考える。

```
std::thread t(task);
t.detach();
```

最初の文では、メインスレッドが thread オブジェクト t を生成し、新しいスレッド上で関数 task の実行を開始する。続く t.detach() によって、t と、関数 task を実行しているスレッドとの結び付きが解消される。

detach() を呼び出した後、メインスレッドは、関数 task の終了を待つことなく、直ちに次の処理へ進む。一方、切り離された側のスレッドは、thread オブジェクト t とは独立して、関数 task の処理を続ける。従って、その後に t が破棄されても、そのことを理由として std::terminate() が呼び出されることはない。

join() と detach() の違いは、生成されたスレッドとの関係をどのように終了させるかにある。join() では、生成されたスレッドの終了を待ってから結び付きを解消する。これに対して、detach() では、終了を待たずに結び付きを解消し、生成されたスレッドは独立して処理を続ける。

ただし、一度 detach() を呼び出すと、生成した側は、元の thread オブジェクトを通して、生成された側のスレッドを管理できなくなる。その後に join() を呼び出して終了を待つことはできず、スレッドがいつ終了したかを thread オブジェクトから確認することもできない。detach() の実行後、その thread オブジェクトの joinable() は false を返す。

detach() によって切り離されるのは、thread オブジェクトと、元のスレッドとの管理上の結び付きである。生成されたスレッドが別のプロセスへ移されるわけではなく、引き続き同じプロセス内で実行される。このため、メインスレッドが関数 main の実行を終え、プロセス全体が終了すると、切り離されたスレッドも実行を継続できない。

また、切り離されたスレッドは、それを生成した関数が終了した後も実行を続けることがある。例えば、ある関数内でスレッドを生成して detach() を呼び出した場合、生成元の関数が終了してローカル変数が破棄された後も、切り離されたスレッドが動作を続ける可能性がある。このスレッドが、すでに破棄されたローカル変数を参照すると、不

正なメモリアクセスが発生する危険がある。

従って、`detach()` を使用する場合は、生成されたスレッドが使用するデータの寿命を十分に考慮する必要がある。終了を確実に確認する必要がある処理や、生成した側が所有するデータを使用する処理では、通常、`detach()` よりも `join()` を用いて終了を管理の方が安全である。`detach()` は、終了の確認を必要とせず、他のオブジェクトの寿命にも依存しない処理に限定して使用することが望ましい。

18.4 スレッドの休止：`sleep_for`

現在実行中のスレッドを一定時間だけ待機させるには、`std::this_thread::sleep_for()` を使用する。

書き方：`std::this_thread::sleep_for( 時間間隔 );`

引数には `duration` オブジェクト¹⁷³ を指定する。例えば、次の記述では、この文を実行したスレッドを 1 秒間待機させる。

```
std::this_thread::sleep_for(std::chrono::seconds(1));
```

ここで、`std::chrono::seconds(1)` は、1 秒の時間間隔を表す `duration` オブジェクトである。同様に、500 ミリ秒だけ待機させる場合は、次のように記述する。

```
std::this_thread::sleep_for(std::chrono::milliseconds(500));
```

`sleep_for()` によって待機するのは、この関数を呼び出したスレッドだけである。同じプロセス内の他のスレッドは、待機せずに処理を続けることができる。

ただし、指定した時間が経過した直後に必ず実行が再開されるとは限らない。`sleep_for()` は、少なくとも指定した時間だけ、現在のスレッドの実行を休止する。その後、実際に処理を再開する時期は、OS のスケジューラの判断などによって決まる。

■ サンプルプログラム

`thread` オブジェクトを生成した後、`join()` も `detach()` も呼び出さない場合と、`detach()` によってスレッドを切り離した場合の動作を比較する。いずれのプログラムでも、関数 `start_task` の内部で新しいスレッドを生成し、そのスレッド上で関数 `task` を実行する。関数 `task` は、1 秒間隔で「スレッド実行中」と 5 回表示する。

最初のプログラム `test280-2.cpp` では、生成したスレッドに対して `join()` も `detach()` も呼び出さない。このため、関数 `start_task` の終了時に、`joinable()` が `true` のまま `thread` オブジェクトが破棄され、`std::terminate()` が呼び出されてプログラムは異常終了する。

2 つ目のプログラム `test280-3.cpp` では、生成したスレッドに対して `detach()` を呼び出す。これにより、関数 `start_task` が終了して `thread` オブジェクトが破棄された後も、切り離されたスレッドは処理を継続する。ただし、プロセス全体が終了すると切り離されたスレッドも終了するため、動作を確認できるように、メインスレッドを `sleep_for()` で 6 秒間待機させる。なお、複数のスレッドの実行順序は一定ではないため、画面に表示されるメッセージの順序は実行例と異なることがある。

プログラム：`test280-2.cpp`

```
1  #include    <chrono>
2  #include    <iostream>
3  #include    <thread>
4  using std::cout;
5
6  void task() {                                // 新しいスレッドで実行する関数
7      for (int i = 0; i < 5; i++) {
8          cout << "スレッド実行中\n";
9          std::this_thread::sleep_for(std::chrono::seconds(1));
10     }
11 }
12
13 void start_task() {                          // 新しいスレッドを生成して開始する関数
14     std::thread t(task);
15     // join() も detach() も行わない
16 } // t が joinable なまま破棄され、std::terminate() が呼び出される
```

¹⁷³ 「11.14.3 時刻点と時間間隔」(p.230～)で解説した。

```

17 |
18 | int main() {
19 |     start_task();                // join()もdetach()も行われない
20 |     cout << "メイン処理\n";      // ここには到達しない
21 | }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test280-2.cpp のコンパイルと実行

```

$ g++ test280-2.cpp -pthread -o test280-2  [Enter]    ←コンパイル
$ ./test280-2  [Enter]    ←実行
スレッド実行中                ← thread としての task 関数からの出力（これすらも出力されない場合もある）
terminate called without an active exception  ← joinable() な thread オブジェクトが破棄されて異常終了

```

スレッドを使用するプログラムをコンパイルする際、通常は `-pthread` オプションを付ける。（無くてもコンパイルできる環境もある）

プログラム：test280-3.cpp

```

1 | #include <chrono>
2 | #include <iostream>
3 | #include <thread>
4 | using std::cout;
5 |
6 | void task() {                    // 新しいスレッドで実行する関数
7 |     for (int i = 0; i < 5; i++) {
8 |         cout << "スレッド実行中" << std::endl;
9 |         std::this_thread::sleep_for(std::chrono::seconds(1));
10 |     }
11 | }
12 |
13 | void start_task() {              // 新しいスレッドを生成して開始する関数
14 |     std::thread t(task);         // tを生成し、新しいスレッドを開始
15 |     t.detach();                  // tから当該スレッドを切り離す
16 | } // tは破棄されるが、切り離されたスレッドは実行を続ける
17 |
18 | int main() {
19 |     start_task();                // スレッドを開始して切り離す
20 |
21 |     std::cout << "start_task終了後（スレッドは切り離されている）\n";
22 |
23 |     // プロセスが直ちに終了しないよう、動作確認のために待機する
24 |     std::this_thread::sleep_for(std::chrono::seconds(6));
25 |
26 |     std::cout << "メイン処理終了\n";
27 | }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test280-3.cpp のコンパイルと実行

```

$ g++ test280-3.cpp -pthread -o test280-3  [Enter]    ←コンパイル
$ ./test280-3  [Enter]    ←実行
start_task 終了後（スレッドは切り離されている）    ←スレッドの開始と切り離し
スレッド実行中                ← thread としての task 関数からの出力
スレッド実行中
スレッド実行中
スレッド実行中
スレッド実行中
メイン処理終了                ← main 関数が正常に終了

```

18.5 共有データとデータ競合

複数のスレッドを使用するプログラムでは、それぞれのスレッドが独立して処理を進める一方で、同じデータを共有して利用する場合も多い。例えば、共通の計算結果を更新したり、処理の進行状況を記録したり、複数のスレッドから同じオブジェクトへアクセスしたりする処理は、一般的に行われる。

同じプロセス内で動作するスレッドは、グローバル変数や静的変数、ヒープ領域に生成されたオブジェクトなどを共有することができる。このように、複数のスレッドから利用されるデータを**共有データ**という。

共有データを複数のスレッドから読み取るだけであれば、通常は大きな問題にはならない。しかし、複数のスレッドが同じデータを書き換える場合や、あるスレッドが書き換えているデータを別のスレッドが読み取る場合には、処理の実行順序によって、意図しない結果が生じることがある。

複数のスレッドによる共有データへのアクセスが適切に調整されていないと、更新結果の一部が失われたり、データの内容に矛盾が生じたりすることがある。このような問題は、スレッドの実行順序が一定ではないため、実行するたびに異なる形で現れる場合があり、原因の特定が難しくなることも多い。

特に、複数のスレッドが同じ記憶領域へ同時期にアクセスし、そのうち少なくとも1つが書き込みを行い、しかもそれらのアクセスが適切に同期されていない場合には、**データ競合** (data race) が発生する。C++では、データ競合が発生するとプログラムの動作は未定義となるため、共有データへのアクセスを適切に制御しなければならない。

本項では、まず共有データへのアクセスによってデータ競合が発生する例を示し、その後、競合によるデータの不整合を防ぐための基本的な手法について説明する。

18.5.1 データ競合が発生する例

複数のスレッドが同じデータを更新する場合に、どのような問題が生じるかを考える。次のように、整数型のグローバル変数 `shared_value` を定義したものとする。

```
int shared_value = 0;
```

同じプロセス内で動作するスレッドは、同じアドレス空間を共有する。従って、このグローバル変数を参照できる関数を複数のスレッド上で実行した場合、それぞれのスレッドは同じ `shared_value` を読み書きすることになる。

ここでは、次の3つの処理によって、`shared_value` の値を1増加させることを考える。

```
int tmp = shared_value;
tmp++;
shared_value = tmp;
```

最初の文では、共有変数 `shared_value` の値を読み取り、スレッドごとのローカル変数 `tmp` に複製する。次に `tmp` の値を1増加させ、最後にその値を `shared_value` へ書き戻す。1つのスレッドだけで実行する場合は、この処理によって `shared_value` の値は正しく1増加する。

しかし、複数のスレッドが同じ処理を並行して実行すると、読取りから書き込みまでの間に、別のスレッドが同じ共有変数へアクセスすることがある。更新結果が失われる典型的な状況として、`shared_value` の値が10であるときに、2つのスレッド A、B が次の順序で処理を行った場合を考える。

実行順	スレッド	実行する処理
1	A	<code>shared_value</code> の値 10 を <code>tmp</code> に読み取る
2	B	<code>shared_value</code> の値 10 を <code>tmp</code> に読み取る
3	A	<code>tmp</code> を 11 にして <code>shared_value</code> へ書き戻す
4	B	<code>tmp</code> を 11 にして <code>shared_value</code> へ書き戻す

この場合、2つのスレッドがそれぞれ値を1増加させているにもかかわらず、最終的な `shared_value` の値は11となる。本来は12となるはずであるが、後から行われた書き込みによって、一方の更新結果が失われている。

ただし、これはデータ競合によって起こり得る典型的な状況を説明するための概念例であり、C++ではデータ競合が発生した時点で動作は未定義となるため、実際にこの順序や結果になることが保証されているわけではない。

次に示すプログラム `test280-4.cpp` は、複数のスレッドがグローバルの共有変数にアクセスする例であり、10個のスレッドから、共有変数 `shared_value` の値をそれぞれ10回ずつ増加させる。共有変数へのアクセスには排他制御を行っていないため、意図的にデータ競合を含む形となっている。

プログラム：test280-4.cpp

```
1 #include <chrono>
2 #include <iostream>
3 #include <random>
```



```

4  #include    <thread>
5
6  int shared_value = 0;    // 複数のスレッドで共有する変数
7
8  // 共有変数を読み取って1を加え、それを共有変数に戻す処理を10回繰り返す関数
9  void task() {
10     std::mt19937 mt{std::random_device{}()};           // 乱数エンジン
11     std::uniform_int_distribution<int> dist(1, 100);    // 1~100の一樣乱数
12
13     for (int i = 0; i < 10; i++) {
14         // 読取りのタイミングをスレッドごとにずらす
15         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
16
17         int tmp = shared_value;    // 共有変数を読み取り、ローカル変数に複製
18
19         // 読取りから書込みまでの間に他のスレッドが実行されやすくする
20         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
21
22         tmp++;    // ローカル変数に1を加え
23         shared_value = tmp;    // それを共有変数に戻す
24     }
25 }
26
27 int main() {
28     std::thread threads[10];
29
30     // 10個のスレッドを開始
31     for (int i = 0; i < 10; i++) threads[i] = std::thread(task);
32
33     // 全スレッドの終了を待つ
34     for (int i = 0; i < 10; i++) threads[i].join();
35
36     std::cout << "shared_value = " << shared_value << '\n';
37 }

```

このプログラムでは、スレッド間のデータ競合を観察しやすくするため、共有変数を読み取る前と、読み取ってから書き戻すまでの間に、乱数で決めた時間だけスレッドを待機させる。

```
std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
```

待機時間をスレッドごと、また繰り返しごとに変化させることで、各スレッドが共有変数を読み書きするタイミングを意図的にずらしている。ただし、この待機処理自体がデータ競合を発生させるのではない。複数のスレッドが、適切な制御を行わずに同じ共有変数を読み書きしていることが問題である。

また、乱数エンジン `mt` と整数一樣分布 `dist` は、関数 `task` のローカル変数として生成している。従って、各スレッドはそれぞれ独立した乱数エンジンを使用し、乱数生成器自体はスレッド間で共有されない。

関数 `main` では、関数 `task` を実行するスレッドを 10 個生成する。

```
std::thread threads[10];
for (int i = 0; i < 10; i++) threads[i] = std::thread(task);
```

各スレッドによる共有変数の更新が適切に制御され、全ての更新が正しく反映されれば、最終的な `shared_value` の値は、次のように 100 となるはずである。

10 回 × 10 スレッド = 100

しかし、実際には複数のスレッドによる読取りと書込みが重なり、更新結果の一部が失われることがある。

このように、複数のスレッドが同じ記憶領域にある通常の変数へアクセスし、そのうち少なくとも 1 つが書込みを行うにもかかわらず、それらのアクセスが適切に同期されていない状態を**データ競合** (data race) という。ここでいう「通常の変数」とは、本書では、後述する `atomic` 型などによって安全なアクセスが保証されていない変数を指す。C++ では、データ競合を含むプログラムの動作は**未定義動作**となる。従って、単に計算結果が不正になるだけでなく、どのような動作になるかは保証されない。次の実行例では異常終了せず、100 より小さい値が表示されているが、常に同じ形式の結果になるとは限らない。

このプログラムをコンパイルして実行する例を次に示す。

例. test280-4.cpp のコンパイルと実行

```
$ g++ test280-4.cpp -pthread -o test280-4  Enter  ←コンパイル
$ ./test280-4  Enter  ←実行
shared_value = 24  ←期待される 100 とは異なる値
$ ./test280-4  Enter  ←実行
shared_value = 19  ←実行の度に異なる値となる例
```

実行するたびに、各スレッドが共有変数へアクセスする順序が変化するため、shared_value の最終値も変化することがある。ただし、このプログラムにはデータ競合があり、動作は未定義であるため、表示される値やプログラムの動作について一般的な保証はない。また、この場合の未定義動作は、必ずしもプログラムを強制的に終了させるものではなく、一見もっともらしい結果を出力して処理を続けることもある。そのため、問題の存在や原因の特定が困難になる場合がある。

18.5.2 mutex による排他制御

先に示したプログラム test280-4.cpp では、複数のスレッドが同じ共有変数を同時期に読み書きするため、データ競合が発生した。このような問題を防ぐには、共有データへアクセスする処理を、ある時点では 1 つのスレッドだけが実行できるように制限する必要がある。このような制御を**排他制御** (mutual exclusion) という。

C++ の標準ライブラリには、排他制御を行うためのクラスとして `std::mutex` が用意されている。mutex を使用するには、ヘッダファイル `mutex` を読み込む。排他制御に使用する mutex オブジェクトは、例えば次のように定義する。

```
std::mutex mtx;
```

この mutex オブジェクトを、排他制御を行う全てのスレッドで共通に使用する。複数のスレッドがそれぞれ異なる mutex オブジェクトを使用しても、相互の実行を制御することはできない。

mutex オブジェクトをロックするには、メンバ関数 `lock()` を使用する。また、ロックを解除するには、メンバ関数 `unlock()` を使用する。

書き方： `mutex オブジェクト名.lock();`
排他制御する処理
`mutex オブジェクト名.unlock();`

あるスレッドが `lock()` を呼び出して mutex のロックを獲得すると、そのスレッドが `unlock()` を呼び出すまで、他のスレッドは同じ mutex のロックを獲得できない。他のスレッドがその間に `lock()` を呼び出した場合は、先にロックを獲得したスレッドが `unlock()` を呼び出すまで待機する。

従って、複数のスレッドが同じ mutex を使用してこの処理範囲を保護している場合、`lock()` から `unlock()` までの範囲を、ある時点で実行できるスレッドは 1 つに限られる。このように、複数のスレッドによる同時実行を禁止する処理範囲を**クリティカルセクション** (critical section)、または**臨界区間**という。

前項のプログラムでは、共有変数 `shared_value` の値を読み取り、その値を更新して書き戻すまでの処理を、次のように `lock()` と `unlock()` の間に置く。

```
mtx.lock();
int tmp = shared_value;
tmp++;
shared_value = tmp;
mtx.unlock();
```

この場合、あるスレッドが `shared_value` の値を読み取ってから書き戻すまでの間、同じ mutex を使用してこの処理範囲を保護する他のスレッドは、この範囲へ入ることができない。従って、複数のスレッドが同じ値を読み取り、同じ更新結果を重ねて書き戻すことがなくなり、更新結果の消失を防ぐことができる。

次に示すプログラム test280-5.cpp は、前項のプログラムに mutex による排他制御を加えたものである。共有変数 `shared_value` を保護するために、グローバルな mutex オブジェクト `mtx` を定義している。

```
std::mutex mtx;
```

関数 `task` では、共有変数の読取りから書戻しまでの範囲を、`mtx.lock()` と `mtx.unlock()` で囲んでいる。このため、この範囲を複数のスレッドが同時期に実行することはない。

なお、このプログラムでは、排他制御の効果を確認しやすくするため、ロック中にも `sleep_for()` による待機を行っている。ロック中は、他のスレッドが同じ `mutex` の獲得を待たされるため、実際のプログラムでは、ロックを保持する時間をできるだけ短くすることが望ましい。

プログラム：test280-5.cpp

```
1  #include    <chrono>
2  #include    <iostream>
3  #include    <mutex>
4  #include    <random>
5  #include    <thread>
6
7  int shared_value = 0;    // 複数のスレッドで共有する変数
8  std::mutex mtx;        // shared_valueを保護するmutex
9
10 // 共有変数を読み取って1を加え、その結果を書き戻す処理を10回繰り返す
11 void task() {
12     std::mt19937 mt{std::random_device{}()};    // 乱数エンジン
13     std::uniform_int_distribution<int> dist(1, 100);    // 1~100の整数一様乱数
14
15     for (int i = 0; i < 10; i++) {
16         // 処理を開始するタイミングをスレッドごとにずらす
17         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
18
19         mtx.lock();    // 排他制御の開始
20
21         int tmp = shared_value;    // 共有変数を読み取り、ローカル変数に複製
22
23         // ロック中は、他のスレッドはこの処理範囲へ入れない
24         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
25
26         tmp++;    // ローカル変数の値を1増加
27         shared_value = tmp;    // 更新結果を共有変数に書き戻す
28
29         mtx.unlock();    // 排他制御の終了
30     }
31 }
32
33 int main() {
34     std::thread threads[10];
35
36     // 10個のスレッドを開始
37     for (int i = 0; i < 10; i++) threads[i] = std::thread(task);
38
39     // 全スレッドの終了を待つ
40     for (int i = 0; i < 10; i++) threads[i].join();
41
42     std::cout << "shared_value = " << shared_value << '\n';
43 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test280-5.cpp のコンパイルと実行

```
$ g++ test280-5.cpp -pthread -o test280-5  Enter    ←コンパイル
$ ./test280-5  Enter    ←実行
shared_value = 100    ←全ての更新が正しく反映された結果
```

このプログラムでは、10個のスレッドがそれぞれ10回ずつ共有変数を増加させるため、最終的な `shared_value` の値は100となる。`mutex` によって、共有変数の読取りから書戻しまでの一連の処理が排他的に実行され、データ競合が防止されている。

`lock()` と `unlock()` を直接使用する場合には、両者を必ず対応させる必要がある。`lock()` の後で例外が発生したり、途中で関数から抜けるなどして `unlock()` が実行されないと、ロックが解除されず、他のスレッドが待機し続けることがある。このように、他のスレッドがロックの獲得を待ち続け、処理が進まなくなる危険があるため、ロックの解放を確実に行わなければならない。この問題を避けるための方法については、次項で説明する。

18.5.2.1 lock_guard による安全なロック管理

前項では、mutex のメンバ関数 lock() と unlock() を使用して、排他制御の開始と終了を明示的に指定した。しかし、この方法では、lock() を呼び出した後に必ず unlock() を呼び出さなければならない。途中で例外が発生した場合や、return 文などによって関数から抜けた場合には、unlock() が実行されず、ロックが解除されない可能性がある。

このようなロックの解除忘れを防ぐために、C++ の標準ライブラリには std::lock_guard クラスが用意されている。lock_guard は、オブジェクトの生成時に指定された mutex をロックし、そのオブジェクトが破棄されるときにロックを自動的に解除する。基本的な書き方を次に示す。

書き方： std::lock_guard<std::mutex> オブジェクト名 (mutex オブジェクト);

例えば、mutex オブジェクト mtx をロックするには、次のように記述する。

```
std::lock_guard<std::mutex> lock(mtx);
```

この文が実行されると、lock_guard のコンストラクタによって mtx のロックの獲得が試みられる。他のスレッドが既に mtx をロックしている場合は、そのロックが解除されるまで待機し、ロックを獲得した後に lock_guard オブジェクト lock の生成が完了する。ここで使用している lock という名前は任意であり、mutex をロックするメンバ関数 lock() とは別のものである。

lock_guard オブジェクトは、そのオブジェクトが定義されたスコープを抜けると自動的に破棄される。その際に、保持している mutex のロックも自動的に解除される。従って、unlock() を明示的に呼び出す必要はない。(次の例)

```
{
    std::lock_guard<std::mutex> lock(mtx);
} // この範囲では、mtx による排他制御が行われる
```

この例では、lock が生成された時点で mtx がロックされ、閉じ波括弧に到達してスコープを抜けると、lock が破棄されて mtx のロックが解除される。途中で例外が発生した場合や、return 文によってスコープを抜けた場合にも、オブジェクトの破棄に伴ってロックが解除される。

このように、オブジェクトの生成時に必要な資源を取得し、オブジェクトの破棄時にその資源を解放するという、先に説明した **RAII** の考え方がここでも用いられている。lock_guard では、mutex のロックを管理対象の資源として扱うことにより、ロックの解除を確実に行う。

前項のプログラムで、lock() と unlock() を使用していた部分は、次のように書き換えることができる。

```
std::lock_guard<std::mutex> lock(mtx);
int tmp = shared_value;
std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
tmp++;
shared_value = tmp;
```

lock_guard オブジェクト lock は、for 文の本体を表す波括弧内で定義されている。従って、オブジェクトの生成時に mtx がロックされ、その繰返しの処理が終了して波括弧の末尾に到達すると、lock が破棄されてロックが解除される。次の繰返しでは、新しい lock_guard オブジェクトが生成され、再びロックが獲得される。

あるスレッドが lock_guard によって mtx のロックを保持している間は、同じ mtx を使用する他のスレッドはロックを獲得できず、lock_guard の生成時点で待機する。従って、共有変数 shared_value の読取りから書戻しまでの処理は、ある時点では 1 つのスレッドだけによって実行される。

次に示すプログラム test280-6.cpp は、前項のプログラム test280-5.cpp を、lock_guard を使用する形に書き換えたものである。lock() と unlock() を直接呼び出す代わりに、次の文によってロックを管理している。

```
std::lock_guard<std::mutex> lock(mtx);
```

この文以降の処理は、lock が破棄されるまで、mtx による排他制御の対象となる。このプログラムでは、lock のスコープが for 文の本体の末尾まで続くため、共有変数の読取り、待機、更新、書戻しが排他的に実行される。

プログラム： test280-6.cpp

```
1 #include <chrono>
2 #include <iostream>
3 #include <mutex>
4 #include <random>
```



```

5  #include    <thread>
6
7  int shared_value = 0;    // 複数のスレッドで共有する変数
8  std::mutex mtx;        // shared_valueを保護するmutex
9
10 // 共有変数を読み取って1を加え、その結果を書き戻す処理を10回繰り返す
11 void task() {
12     std::mt19937 mt{std::random_device{}()};        // 乱数エンジン
13     std::uniform_int_distribution<int> dist(1, 100);    // 1~100の整数一様乱数
14
15     for (int i = 0; i < 10; i++) {
16         // 処理を開始するタイミングをスレッドごとにずらす
17         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
18
19         // この範囲では、1つのスレッドだけがshared_valueへアクセスできる
20         std::lock_guard<std::mutex> lock(mtx);
21
22         int tmp = shared_value; // 共有変数を読み取り、ローカル変数に複製
23
24         // 排他制御中であることを確認しやすくするための待機
25         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
26
27         tmp++;                // ローカル変数の値を1増加
28         shared_value = tmp;    // 更新結果を共有変数に書き戻す
29     }
30 }
31
32 int main() {
33     std::thread threads[10];
34
35     // 10個のスレッドを開始
36     for (int i = 0; i < 10; i++) threads[i] = std::thread(task);
37
38     // 全スレッドの終了を待つ
39     for (int i = 0; i < 10; i++) threads[i].join();
40
41     std::cout << "shared_value = " << shared_value << '\n';
42 }

```

このプログラムをコンパイルして実行する例を次に示す。

例. test280-6.cpp のコンパイルと実行

```

$ g++ test280-6.cpp -pthread -o test280-6  ←コンパイル
$ ./test280-6  ←実行
shared_value = 100    ←全ての更新が正しく反映された結果

```

このプログラムでも、10個のスレッドによる合計100回の更新が全て正しく反映され、shared_value の最終値は100となる。lock_guard を使用することで、mutex のロックと解除をオブジェクトの寿命に対応させ、解除忘れを防ぎながら安全に排他制御を行うことができる。

18.5.3 atomic による単純な共有値の操作

前項で説明した mutex は、共有データに対する一連の処理を排他的に実行するための汎用性の高い仕組みである。複数の変数をまとめて更新する場合や、構造体、クラスのオブジェクトなどに対する複数の処理を一体として保護する場合には、mutex を使用する方法が適している。

一方、複数のスレッドで共有するデータが整数値や真偽値などの単純な値であり、その値に対して読取り、書込み、増減などの単純な操作だけを行う場合には、mutex を使用しなくても安全に処理できることがある。このような処理には、C++の標準ライブラリに用意されている std::atomic クラステンプレートを使用できる。

atomic は、1個の共有値に対する操作を**原子的** (atomic) に実行するためのクラスである。原子的な操作とは、他のスレッドから見ても途中の状態が観察されず、分割されない1つの操作として扱われる処理を意味する。従って、複数のスレッドが同じ atomic オブジェクトを操作しても、通常の変数を適切な同期なしに共有した場合に生じるデータ競合を防ぐことができる。

例えば、複数のスレッドが共通の整数型のカウンタを増加させるだけであれば、`atomic` を使用することで、`lock()` や `unlock()`、`lock_guard` などを記述せずに安全な更新を行うことができる。このため、単純な共有値に対する処理では、`atomic` を使用する方が簡潔に記述できる場合が多い。

ただし、`atomic` が適しているのは、基本的に1個の値に対して用意された読取り、書込み、増減、交換などの原子的な操作である。複数の値をまとめて更新する場合や、複数の文から成る処理全体の整合性を保つ必要がある場合には、通常は `mutex` を使用する。本項では、整数型の共有値を例として、`atomic` の基本的な使用方法について説明する。

■ `atomic` クラスの基本的な使用方法

`atomic` クラスを使用するには、ヘッダファイル `atomic` を読み込む。`atomic` はクラステンプレートであり、原子的に操作する値の型をテンプレート引数として指定する。基本的な書き方を次に示す。

書き方： `std::atomic<型> オブジェクト名 { 初期値 };`

テンプレート引数の型には、`bool` 型、`int` 型などの整数型、浮動小数点型、ポインタ型などを指定できる。特に、処理の状態を表すフラグ、共有カウンタ、数値、オブジェクトを指すポインタなどを複数のスレッドで共有する場合によく使用される。

また、一定の条件を満たす構造体やクラスの型を指定できる場合もある。ただし、任意の型を使用できるわけではなく、基本的には**トリビアルにコピー可能**¹⁷⁴ (trivially copyable) な型でなければならない。例えば、動的に確保した記憶領域を管理する `std::string` や `std::vector`、独自のコピー処理やデストラクタを持つクラス、仮想関数を持つクラスなどは、典型的には `atomic` の対象には適さない。このような複雑なオブジェクトを複数のスレッドで共有する場合には、通常は `mutex` による排他制御を使用する。

本項では、最も基本的な使用例として、整数型の `atomic` オブジェクトを扱う。

整数型の共有値を保持する `atomic` オブジェクトは、例えば次のように定義できる。

```
std::atomic<int> counter{0};
```

この例では、`int` 型の値を保持する `atomic` オブジェクト `counter` を定義し、初期値として0を設定している。`counter` は複数のスレッドから共有して使用でき、`atomic` に用意された操作によって安全に読み書きできる。

`atomic` オブジェクトに格納されている値を読み取るには、メンバ関数 `load()` を使用する。また、新しい値を書き込むには、メンバ関数 `store()` を使用する。

書き方： `atomic オブジェクト.load()`
`atomic オブジェクト.store(値)`

例えば、次の処理では、`counter` の値を読み取って変数 `n` に格納した後、`counter` に10を書き込んでいる。

```
int n = counter.load();  
counter.store(10);
```

`load()` による読取りと `store()` による書込みは、それぞれ1回の原子的な操作として実行される。従って、複数のスレッドが同じ `atomic` オブジェクトを読み書きしても、通常の変数を同期せずに共有した場合のようなデータ競合は発生しない。

値の読取りと書込みは、通常の変数と同じような記法でも行うことができる。

```
int n = counter;  
counter = 10;
```

これらは、それぞれ `load()` と `store()` に相当する。ただし、原子的な操作であることを明確に示す場合には、`load()` と `store()` を使用する。

整数型の `atomic` オブジェクトでは、値を増減する演算も利用できる。例えば、次の文は、`counter` の値を原子的に1増加させる。

```
counter++;
```

この処理では、現在値の読取り、1の加算、更新結果の書戻しが、一体の原子的な操作として実行される。従って、複数のスレッドが同時期に `counter++` を実行しても、一部の更新結果が失われることはない。同様に、次のような増減

¹⁷⁴ 「12.4.5 単純なバイトコピーが可能な型：trivially copyable」(p.244～)で解説した。

操作も利用できる。

```
counter--;  
counter += 5;  
counter -= 3;
```

値の加算と減算には、メンバ関数 `fetch_add()` および `fetch_sub()` を使用することもできる。

```
counter.fetch_add(5);  
counter.fetch_sub(3);
```

`fetch_add()` は指定した値を加算し、`fetch_sub()` は指定した値を減算する。これらの関数は、更新前の値を戻り値として返す。例えば、次の処理では、`counter` に 5 を加算するとともに、加算前の値を `old` に格納する。

```
int old = counter.fetch_add(5);
```

また、現在の値を別の値に置き換えるには、メンバ関数 `exchange()` を利用できる。

```
int old = counter.exchange(20);
```

この処理では、`counter` の値を 20 に置き換え、置換前の値を `old` に格納する。値の置換と置換前の値の取得は、一体の原子的な操作として実行される。

`atomic` の主な基本操作を表 45 に示す。

表 45: <code>atomic</code> の主な基本操作	
操 作	解 説
<code>load()</code>	現在の値を原子的に読み取る
<code>store(値)</code>	指定した値を原子的に書き込む
<code>++, --</code>	整数値を原子的に 1 増加または 1 減少させる
<code>+=, -=</code>	整数値を原子的に加算または減算する
<code>fetch_add(値)</code>	指定した値を加算し、更新前の値を返す
<code>fetch_sub(値)</code>	指定した値を減算し、更新前の値を返す
<code>exchange(値)</code>	値を置き換え、置換前の値を返す

ただし、`atomic` オブジェクトに対する個々の操作が原子的であっても、複数の文を並べた処理全体が自動的に原子的になるわけではない。例えば、次の処理では、読取り、加算、書込みがそれぞれ別の操作として実行される。

```
int tmp = counter.load();  
tmp++;  
counter.store(tmp);
```

従って、この 3 文を複数のスレッドが並行して実行すると、通常の変数の場合と同じように、更新結果が失われる可能性がある。値を 1 増加させる場合には、これらの処理を分けて記述せず、次のように 1 回の原子的な操作として実行する必要がある。

```
counter++;
```

このように、`atomic` を使用する場合には、目的の処理が 1 回の原子的な操作として用意されているかを確認することが重要である。単純な読取り、書込み、増減などであれば `atomic` を利用できるが、処理が複数の文にまたがる場合や、複数の共有値の整合性を保つ必要がある場合には、通常は `mutex` による排他制御を採用する方が適している。

■ サンプルプログラム

次に示すプログラム `test280-7.cpp` は、先に示した `test280-5.cpp` および `test280-6.cpp` と同様に、10 個のスレッドから共有値 `shared_value` をそれぞれ 10 回ずつ増加させる例である。

プログラム：`test280-7.cpp`

```
1  #include    <atomic>  
2  #include    <chrono>  
3  #include    <iostream>  
4  #include    <random>  
5  #include    <thread>  
6  
7  std::atomic<int> shared_value{0};    // 複数のスレッドで共有する atomic 変数  
8
```

```

9  // 共有変数の値を1増加させる処理を10回繰り返す
10 void task() {
11     std::mt19937 mt{std::random_device{}()};           // 乱数エンジン
12     std::uniform_int_distribution<int> dist(1, 100);    // 1~100の整数一様乱数
13
14     for (int i = 0; i < 10; i++) {
15         // 更新のタイミングをスレッドごとにずらす
16         std::this_thread::sleep_for(std::chrono::milliseconds(dist(mt)));
17         shared_value++; // atomic変数を原子的に1増加
18     }
19 }
20
21 int main() {
22     std::thread threads[10];
23
24     // 10個のスレッドを開始
25     for (int i = 0; i < 10; i++) threads[i] = std::thread(task);
26
27     // 全スレッドの終了を待つ
28     for (int i = 0; i < 10; i++) threads[i].join();
29
30     std::cout << "shared_value = " << shared_value.load() << '\n';
31 }

```

このプログラムでは、共有値を次のように `atomic` オブジェクトとして定義している。

```
std::atomic<int> shared_value{0};
```

関数 `task` では、次の文によって `shared_value` を原子的に 1 増加させる。

```
shared_value++;
```

従って、先の `mutex` を使用した例のように、`lock()`、`unlock()`、`lock_guard` などを記述する必要はない。また、全スレッドの終了後には、`load()` を使用して最終的な値を読み取っている。

```
shared_value.load()
```

このプログラムをコンパイルして実行する例を次に示す。

例. `test280-7.cpp` のコンパイルと実行

```

$ g++ test280-7.cpp -pthread -o test280-7  Enter  ←コンパイル
$ ./test280-7  Enter  ←実行
shared_value = 100  ←出力

```

このプログラムでは、10 個のスレッドによる合計 100 回の増加処理が全て正しく反映され、`shared_value` の最終値は 100 となる。

本章で説明した内容は、C++による並行処理の基礎事項に限られる。C++の並行処理機能には、このほかにもスレッド間の通知、同期処理、非同期処理、メモリ上の操作順序の制御など、多くの機能が用意されている。これらの詳細については、C++標準化委員会の技術資料やC++のリファレンスなど [1, 2, 3, 7] を参照されたい。

19 プログラムの構成と分割

これまでに説明したプログラムでは、主に一つのソースファイルに全ての処理を記述してきた。しかし、プログラムの規模が大きくなると、関数やクラスの定義を一つのファイルにまとめて記述する方法では、プログラム全体の構成を把握しにくくなり、機能の追加や修正も困難になる。このため、実用的なプログラムでは、処理内容や機能ごとに複数のファイルへ分割して記述する方法が一般に用いられる。

C++では、クラス定義の中ではメンバ関数について宣言だけを記述し、その処理内容をクラス定義の外部に記述することができる。さらに、クラス定義をヘッダファイルに記述し、メンバ関数の定義を別のソースファイルに記述することにより、クラスが外部に提供する機能と、その実装とを分離できる。このようにプログラムを複数のファイルに分割した場合、それぞれのソースファイルを個別にコンパイルし、生成されたオブジェクトファイルをリンクして一つの実行プログラムを作成する。

ただし、C++では、通常関数や変数の定義を複数のソースファイルから読み込まれるヘッダファイルに記述すると、同じ定義が重複して現れる場合がある。一方、`inline` 関数、`inline` 変数、クラス定義内で定義されたメンバ関数、テンプレートなどは、ヘッダファイルに定義を記述して利用することが多い。従って、プログラムを適切に分割するためには、宣言と定義の違いや、定義を記述すべきファイルについて理解しておく必要がある。

また、C++のプログラムからC言語で作成された関数を呼び出したり、C言語とC++のソースファイルから生成されたオブジェクトファイルをリンクしたりする場合には、両言語における関数名の扱いやリンク方式の違いを考慮する必要がある。C++では、このような連携のために `extern "C"` が用意されている。

本章では、クラス定義とメンバ関数の定義を分離する方法から始め、クラス定義と実装のファイル分割、複数のソースファイルのコンパイルとリンク、名前空間に属する関数の定義、`inline` 関数や静的データメンバ、テンプレートのファイル分割について説明する。さらに、C言語で記述されたプログラムとC++のプログラムを連携させる方法についても説明する。

19.1 クラスの宣言とメンバ関数の定義

これまでに示したクラスの例では、メンバ関数の処理内容をクラス定義の内部に記述することが多かった。例えば、2次元ベクトルを表すクラスを次のように定義できる。

```
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D(double x0, double y0) : x(x0), y(y0) {}
    double length() const {
        return std::sqrt(x * x + y * y);
    }
};
```

この例では、コンストラクタと `length` メンバ関数の処理内容が、いずれもクラス定義の内部に記述されている。このような記述は、クラスの構成とメンバ関数の処理内容を一度に確認できるため、小さなクラスでは簡便である。しかし、メンバ関数の数が増えたり、それぞれの処理が長くなったりすると、クラス定義全体が大きくなり、クラスが外部に提供する機能を把握しにくくなる。

C++では、クラス定義の内部にはメンバ関数の**宣言**だけを記述し、実際の処理内容を表す**定義**をクラス定義の外部に記述できる。例えば、先の `Vec2D` クラスは、次のように記述することもできる。

```
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D(double x0, double y0);
    double length() const;
};
```

このクラス定義では、コンストラクタと `length` メンバ関数について、名前、引数、戻り値を持つ関数では戻り値の

型、および必要な指定だけが記述されており、処理内容は記述されていない。末尾にセミコロン ; が置かれたこれらの記述は、メンバ関数の宣言である。

メンバ関数の定義は、このクラス定義よりも後の位置に、次のように記述できる。

```
Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
double Vec2D::length() const {
    return std::sqrt(x * x + y * y);
}
```

メンバ関数の定義をクラス定義の外部に記述する場合は、関数名の前に、その関数が属するクラスの名前とスコープ解決演算子 :: を記述する。例えば、Vec2D::length は、Vec2D クラスに属する length メンバ関数を表す。

19.1.1 class 文の外部におけるメンバ関数の定義

クラス定義の外部にメンバ関数を定義する基本的な形式を次に示す。

```
戻り値の型 クラス名::メンバ関数名 ( 仮引数 ) {
    処理
}
```

補足) 仮引数を囲む丸括弧の後に const などの修飾指定を記述することができる。

例えば、Vec2D クラスに、 x 座標と y 座標の値を出力する print メンバ関数を追加する場合、クラス定義内には次の宣言を記述する。

```
void print() const;
```

このメンバ関数の定義をクラス定義の外部に記述する場合は、例えば次のようになる。

```
void Vec2D::print() const {
    std::cout << "(" << x << ", " << y << ")" << std::endl;
}
```

クラス定義の外部に記述されたメンバ関数の本体からも、そのクラスのデータメンバや他のメンバ関数を直接利用できる。従って、この例では、データメンバ x , y の前にオブジェクト名などを付ける必要はない。また、これらが private メンバであっても、同じクラスのメンバ関数から利用できる。

コンストラクタも同様にクラス定義の外部へ記述できる。ただし、コンストラクタには戻り値の型がないため、定義は次の形式となる。

```
クラス名::クラス名 ( 仮引数 ) : メンバ初期化子リスト {
    処理
}
```

補足) メンバ初期化子リストを使用しない記述も可能である。

例えば、先の Vec2D クラスのコンストラクタは、次のように定義されている。

```
Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
```

最初の Vec2D は、このコンストラクタが所属するクラスを表し、:: の後の Vec2D はコンストラクタの名前を表す。デストラクタについても、同じ方法でクラス定義の外部に定義できる。

19.1.2 宣言と定義の一致

クラス定義内に記述するメンバ関数の宣言と、クラス定義の外部に記述する定義では、戻り値の型、引数の型と個数、および const などの指定を一致させる必要がある。例えば、クラス定義内に次の宣言があるとすると、

```
double length() const;
```

この宣言に対応する定義は、次のように記述する。

```
double Vec2D::length() const {
    return std::sqrt(x * x + y * y);
}
```

ここで、定義の末尾にある const を省略して、次のように記述することはできない。

```
double Vec2D::length() {          // 宣言と一致しない
    return std::sqrt(x * x + y * y);
}
```

const が指定されたメンバ関数と、指定されていないメンバ関数は、異なるメンバ関数として扱われる。従って、こ

の定義はクラス内で宣言された `length` メンバ関数の定義とはみなされない。この定義は、クラス内に宣言されていない別のメンバ関数を定義しようとする記述になるため、コンパイル時にエラーとなる。

また、引数の型が異なる場合にも、宣言と定義は対応しない。例えば、

```
void set(double x0, double y0);
```

の宣言に対して、次の定義を記述することはできない。

```
void Vec2D::set(int x0, int y0) {    // 引数の型が一致しない
    x = x0;
    y = y0;
}
```

この定義は、`double` 型の引数を受け取る `set` メンバ関数とは別の関数を定義しようとする記述になる。しかし、`int` 型の引数を受け取る `set` メンバ関数はクラス内に宣言されていないため、エラーとなる。

なお、仮引数の名前は、宣言と定義で一致している必要はない。例えば、次の宣言と定義は正しく対応する。

```
void set(double x0, double y0);
void Vec2D::set(double new_x, double new_y) {
    x = new_x;
    y = new_y;
}
```

メンバ関数をクラス定義の外部に定義する場合には、宣言と定義の対応を明確にするため、仮引数の名前も可能な範囲で統一しておくとうい。

■ サンプルプログラム

次のプログラム `test290-1.cpp` では、`Vec2D` クラスの内部にメンバ関数の宣言だけを記述し、コンストラクタと各メンバ関数の定義を `class` 文の外部に記述している。この段階では、クラス定義とメンバ関数の定義は同じソースファイル内に置かれている。

プログラム：test290-1.cpp

```
1  #include    <iostream>
2  #include    <cmath>
3  using std::cout;
4
5  class Vec2D {
6  private:
7      double x;
8      double y;
9  public:
10     Vec2D(double x0, double y0);
11     void set(double x0, double y0);
12     void print() const;
13     double length() const;
14 };
15
16 Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
17
18 void Vec2D::set(double x0, double y0) {
19     x = x0;
20     y = y0;
21 }
22
23 void Vec2D::print() const {
24     cout << "(" << x << ", " << y << ")\n";
25 }
26
27 double Vec2D::length() const {
28     return std::sqrt(x * x + y * y);
29 }
30
31 int main() {
32     Vec2D v(3.0, 4.0);
33
34     cout << "v = ";
35     v.print();
```

```

36     cout << "length = " << v.length() << "\n";
37
38     v.set(5.0, 12.0);
39
40     cout << "v = ";
41     v.print();
42     cout << "length = " << v.length() << "\n";
43 }

```

この例では、クラスが備えるデータメンバとメンバ関数の一覧を `class` 文の内部で確認でき、それぞれのメンバ関数の処理内容は、クラス定義の後に分けて記述されている。

このプログラムをコンパイルして実行する例を次に示す。

例. test290-1.cpp のコンパイルと実行

```

$ g++ test290-1.cpp -o test290-1   ←コンパイル
$ ./test290-1  ←実行
v = (3, 4) ←出力
length = 5
v = (5, 12)
length = 13

```

次節では、この構成をさらに発展させ、クラス定義とメンバ関数の定義を別々のファイルに分割する方法を説明する。

19.2 クラス定義と実装のファイル分割

前節では、クラス定義の内部にメンバ関数の宣言を記述し、メンバ関数の定義を同じソースファイル内の別の位置に記述した。この方法により、クラスが備えるメンバと、それぞれのメンバ関数の処理内容とを分けて記述できる。さらに、これらを別々のファイルに記述することにより、クラスの構成と実装をファイル単位で分離できる。

一般に、クラス定義はヘッダファイルに記述し、クラス定義の外部に記述するメンバ関数の定義はソースファイルに記述する。また、そのクラスを利用するプログラムは、クラス定義を記述したヘッダファイルを読み込んで使用する。ここでは、前節で示したプログラム test290-1.cpp を、次の三つのファイルに分けて記述することを考える。

ファイル	記述する内容
test290-2-class.h	Vec2D クラスの定義
test290-2-class.cpp	Vec2D クラスのメンバ関数の定義
test290-2-main.cpp	Vec2D クラスを利用する処理

この構成では、test290-2-class.h がクラスの外部に公開する宣言を表し、test290-2-class.cpp がその処理内容を表す。test290-2-main.cpp では、Vec2D クラスのメンバ関数がどのように実装されているかを意識せず、ヘッダファイルに記述された宣言に基づいて利用することができる。

クラス定義を記述するヘッダファイル、メンバ関数を定義するソースファイル、およびクラスを利用するプログラムの記述方法を次に示す。

19.2.1 クラス定義とメンバ関数定義の分離

まず、Vec2D クラスの定義を、ヘッダファイル test290-2-class.h に記述する。

ヘッダファイル：test290-2-class.h

```

1  #ifndef      VEC2D_H
2  #define      VEC2D_H
3
4  class Vec2D {
5  private:
6      double x;
7      double y;
8  public:
9      Vec2D(double x0, double y0);

```

```

10     void set(double x0, double y0);
11     void print() const;
12     double length() const;
13 };
14
15 #endif

```

このヘッダファイルには、データメンバとメンバ関数の宣言を含む Vec2D クラスの定義が記述されているが、メンバ関数の具体的な処理内容は記述されていない。また、ヘッダファイルの多重読み込みを防止するために、`#ifndef`, `#define`, `#endif` による条件コンパイルを使用している。この方法については、先の「8.2 プリプロセッサ」(p.98～)で説明した。

次に、メンバ関数の定義をソースファイル `test290-2-class.cpp` に記述する。

クラスの実装：test290-2-class.cpp

```

1  #include    <iostream>
2  #include    <cmath>
3  #include    "test290-2-class.h"
4  using std::cout;
5
6  Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
7
8  void Vec2D::set(double x0, double y0) {
9      x = x0;
10     y = y0;
11 }
12
13 void Vec2D::print() const {
14     cout << "(" << x << ", " << y << ")\n";
15 }
16
17 double Vec2D::length() const {
18     return std::sqrt(x * x + y * y);
19 }

```

このファイルでは、最初に `test290-2-class.h` を読み込んでいる。これにより、コンパイラは Vec2D クラスの定義を参照した上で、各メンバ関数の定義を処理できる。クラス定義と異なる形式のメンバ関数を定義しようとした場合には、宣言と定義の不一致としてコンパイル時にエラーとなる。

`test290-2-class.cpp` では、標準出力を使用するために `iostream`、平方根を求めるために `cmath` を読み込んでいる。一方、`test290-2-class.h` のクラス定義では、これらの機能を直接使用していないため読み込む必要がない。このように、各ファイルでは、そのファイルの記述に必要なヘッダファイルを読み込む。

最後に、Vec2D クラスを利用する処理を、別のソースファイル `test290-2-main.cpp` に記述する。

プログラム：test290-2-main.cpp

```

1  #include    <iostream>
2  #include    "test290-2-class.h"
3
4  using std::cout;
5
6  int main() {
7      Vec2D v(3.0, 4.0);
8
9      cout << "v = ";
10     v.print();
11     cout << "length = " << v.length() << "\n";
12
13     v.set(5.0, 12.0);
14
15     cout << "v = ";
16     v.print();
17     cout << "length = " << v.length() << "\n";
18 }

```

このファイルでは、Vec2D クラスを使用するために test290-2-class.h を読み込んでいる。一方、メンバ関数の定義が記述された test290-2-class.cpp を #include で読み込む必要はない。test290-2-class.cpp は、test290-2-main.cpp とは別にコンパイルし、生成されたオブジェクトファイルを後でリンクする。

#include でソースファイルを読み込むことも技術的には可能であるが、通常の分割開発では、ソースファイルを別のソースファイルから読み込む方法は取らない。クラスや関数の宣言を記述したヘッダファイルを読み込み、それぞれのソースファイルを個別にコンパイルする。

このプログラムをコンパイルして実行する例を次に示す。

例. test290-2-class.cpp のコンパイル

```
$ g++ test290-2-class.cpp -c  ←コンパイルのみ
```

この処理で、オブジェクトファイル test290-2-class.o ができる。次に、このオブジェクトファイルを使用して test290-2-main.cpp をコンパイル、リンクし、実行可能ファイルを作成する。

例. test290-2-main.cpp のコンパイルと実行

```
$ g++ test290-2-main.cpp test290-2-class.o -o test290-2-main  ←コンパイルとリンク
$ ./test290-2-main  ←実行
v = (3, 4)          ←出力
length = 5
v = (5, 12)
length = 13
```

補足) test290-2-class.cpp, test290-2-main.cpp を一度にコンパイルしてリンクすることも可能である。

複数のソースファイルを個別にコンパイルし、生成されたオブジェクトファイルをリンクする基本的な仕組みは、C 言語の場合と同じである。

19.2.2 ヘッダファイルを記述する際の C++ 固有の注意

C++ のヘッダファイルには、クラス定義、関数の宣言、定数や型の定義などを記述できる。一方、ヘッダファイルは複数のソースファイルから読み込まれるため、記述内容によっては、同じ関数や変数の定義が複数のソースファイルに現れることになる。この問題については、後の節で詳しく説明する。

また、ヘッダファイルでは、using namespace による名前空間全体の省略指定を記述しない方がよい。例えば、ヘッダファイルに次の記述があるとする。

```
using namespace std;
```

このヘッダファイルを読み込んだ全てのソースファイルで、std 名前空間内の名前が修飾なしで参照可能になる。その結果、利用する側のプログラムに同じ名前の関数やクラスがある場合に、名前の衝突が生じる可能性がある。従って、ヘッダファイルでは、必要に応じて、例えば次のようにして名前空間名を明示する。

```
std::string name;
```

特定の名前だけを導入する using std::string; についても、ヘッダファイルを読み込む側へ影響を与えるため、通常は名前空間名を明示して記述の方が安全である。

さらに、ヘッダファイル内で使用する型や関数に必要なヘッダファイルは、そのヘッダファイル自身で読み込んでおく必要がある。例えば、クラス定義のデータメンバとして std::string を使用する場合は、そのヘッダファイル内で string を読み込む。

```
#include <string>
class Person {
private:
    std::string name;
};
```

他のソースファイルで偶然 string が先に読み込まれていることを前提にすると、ヘッダファイルを単独で読み込んだ場合にコンパイルできなくなる。ヘッダファイルは、利用する側で特別な準備をしなくても読み込める状態にしておく必要がある。

19.3 名前空間に属する関数やメンバ関数の定義

独自に定義した関数やクラスを名前空間に所属させることにより、他のプログラムで使用されている名前との衝突を避けることができる。名前空間については、先の「11.5 名前空間」(p.190～)で基本的な使用方法を説明した。本節では、名前空間に属する関数やクラスのメンバ関数について、宣言と定義を分けて記述する方法を説明する。

例えば、名前空間 `mylib` に属する関数 `square` を、次のように宣言できる。

```
namespace mylib {
    double square(double x);
}
```

この関数の定義は、同じ名前空間を再度記述して、次のように表すことができる。

```
namespace mylib {
    double square(double x) {
        return x * x;
    }
}
```

同じ名前空間は、プログラム内の複数の位置に分けて記述できる。この例では、最初の `namespace mylib` に関数の宣言を記述し、後の `namespace mylib` に関数の定義を記述しているが、どちらも同じ名前空間を表す。

名前空間の外部から関数を定義する場合は、関数名の前に名前空間名とスコープ解決演算子 `::` を記述する。従って、先の `square` 関数は、次のように定義することもできる。

```
double mylib::square(double x) {
    return x * x;
}
```

`mylib::square` は、名前空間 `mylib` に属する `square` 関数を表す。この方法で定義する関数は、あらかじめその名前空間内に宣言されている必要がある。例えば、`mylib` 名前空間内に `cube` 関数が宣言されていない状態で、次のように定義することはできない。

```
double mylib::cube(double x) {    // cube は宣言されていない
    return x * x * x;
}
```

名前空間の外部に記述した定義によって、その名前空間に新しい関数を追加することはできない。新しい関数を追加する場合は、名前空間の内部に宣言または定義を記述する。

名前空間に属する関数を呼び出す場合は、次のように名前空間名を指定する。

```
double y = mylib::square(3.0);
```

名前空間名の指定を省略する方法もあるが、名前の所属を明確にするため、必要に応じて名前空間名を明示して記述するとよい。

名前空間内に定義されたクラスについても、メンバ関数の宣言と定義を分離できる。例えば、次の `Vec2D` クラスを名前空間 `mylib` に定義する。

```
namespace mylib {
    class Vec2D {
    private:
        double x;
        double y;
    public:
        Vec2D(double x0, double y0);
        double length() const;
    };
}
```

このクラスのメンバ関数は、同じ名前空間を再度記述し、その内部で定義できる。

```
namespace mylib {
    Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
    double Vec2D::length() const {
        return std::sqrt(x * x + y * y);
    }
}
```

この記述では、定義が `mylib` 名前空間の内部にあるため、クラス名には `mylib::` を付けず、`Vec2D::Vec2D`、`Vec2D::length`

と記述している。ただし、これらが `Vec2D` クラスのメンバ関数であることを表すため、クラス名とスコープ解決演算子`::` は必要である。

メンバ関数の定義を名前空間の外部に記述する場合は、名前空間名とクラス名の両方を指定する。

```
mylib::Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
double mylib::Vec2D::length() const {
    return std::sqrt(x * x + y * y);
}
```

この場合、`mylib::Vec2D::length` は、名前空間 `mylib` に属する `Vec2D` クラスの `length` メンバ関数を表す。

ただし、名前空間に属する関数やクラスのメンバ関数の定義は、その名前空間の内部、またはその名前空間を包含する外側の名前空間スコープに記述する必要がある。例えば、名前空間 `mylib` に属する関数を、無関係な別の名前空間の内部で定義することはできない。通常は、同じ名前空間を再度開いて定義する方法、または名前空間の外部で完全な名前を指定する方法を使用する。

■ サンプルプログラム

次のプログラム `test290-3.cpp` では、名前空間 `mylib` に関数 `square` とクラス `Vec2D` を宣言し、それぞれの定義を名前空間の外部に記述している。

プログラム：test290-3.cpp

```
1  #include    <iostream>
2  #include    <cmath>
3
4  namespace mylib {
5      double square(double x);
6
7      class Vec2D {
8      private:
9          double x;
10         double y;
11     public:
12         Vec2D(double x0, double y0);
13         double length() const;
14     };
15 }
16
17 double mylib::square(double x) {
18     return x * x;
19 }
20
21 mylib::Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
22
23 double mylib::Vec2D::length() const {
24     return std::sqrt(x * x + y * y);
25 }
26
27 int main() {
28     std::cout << "square(3) = " << mylib::square(3.0) << "\n";
29
30     mylib::Vec2D v(3.0, 4.0);
31     std::cout << "length = " << v.length() << "\n";
32 }
```

このプログラムでは、`square` 関数の定義に `mylib::square`、`Vec2D` クラスのメンバ関数の定義に `mylib::Vec2D::` を指定している。また、`main` 関数から関数やクラスを利用する際にも、名前空間名 `mylib::` を明示している。

このプログラムをコンパイルして実行する例を次に示す。

例. `test290-3.cpp` のコンパイルと実行

```
$ g++ test290-3.cpp -o test290-3  Enter    ←コンパイル
$ ./test290-3  Enter    ←実行
square(3) = 9          ←出力
length = 5
```

名前空間に属する関数やクラスをヘッダファイルに記述する場合も、名前空間の内部に宣言やクラス定義を置き、メ

ンバ関数などの定義を別のソースファイルに記述できる。この場合も、宣言と定義について、所属する名前空間やクラス名、戻り値の型、引数、constなどを正しく対応させる必要がある。

19.4 ヘッダファイルに定義を記述する場合

これまでに説明したファイル分割では、クラス定義や関数の宣言をヘッダファイルに記述し、関数やメンバ関数の定義をソースファイルに記述した。ヘッダファイルは複数のソースファイルから読み込まれるため、通常関数や変数の定義をヘッダファイルに記述すると、同じ定義が複数のソースファイルに現れる場合がある。例えば、次の関数定義をヘッダファイル calc.h に記述したとする。

```
double square(double x) {  
    return x * x;  
}
```

このヘッダファイルを二つのソースファイルから読み込むと、square 関数の定義がそれぞれのソースファイルに取り込まれる。その結果、各ソースファイルから生成されたオブジェクトファイルに、同じ square 関数の定義が含まれることになる。これらをリンクすると、通常は関数の定義が重複しているとしてエラーになる。

一方、短い関数を inline 関数として定義する場合や、クラス定義内に記述するメンバ関数、テンプレートなどでは、定義をヘッダファイルに記述する場合がある。C++では、このような定義を複数のソースファイルから安全に利用するために、inline などの仕組みが用意されている。

19.4.1 inline 関数

関数の定義の前に inline を指定すると、その関数を **inline 関数**として定義できる。基本的な形式を次に示す。

```
inline 戻り値の型 関数名 ( 仮引数 ) {  
    処理  
}
```

例えば、先の square 関数は、次のように定義できる。

```
inline double square(double x) {  
    return x * x;  
}
```

この定義をヘッダファイルに記述し、複数のソースファイルから読み込むことができる。ここで、inline の指定は、ソースファイルごとに別々の関数を定義するためのものではない。複数のソースファイルに現れる定義は、全て同じ一つの関数を表すものであり、それぞれの定義内容も同一でなければならない。例えば、あるソースファイルでは $x * x$ を返し、別のソースファイルでは $x * x + 1$ を返すように、同じ名前と形式を持つ inline 関数に異なる処理内容を与えることはできない。

inline という名称は、もともと関数呼び出しの箇所に関数の処理内容を埋め込む**インライン展開**を処理系へ促す指定として導入されたものである。通常関数呼び出しでは、関数を呼び出すための処理を行い、関数の実行後に呼び出し元へ戻る。これに対して、関数の処理内容を呼び出し箇所へ直接展開できれば、関数呼び出しに伴う処理を省略できる場合がある。

ただし、inline を指定しても、実際にインライン展開されるとは限らない。関数を展開するかどうかは、関数の大きさ、最適化の設定、呼び出し方などを考慮してコンパイラが判断する。反対に、inline を指定していない関数が、コンパイラの最適化によってインライン展開される場合もある。従って、現在の C++における inline は、単なる高速化の指定ではなく、主に、**同一の関数定義を複数のソースファイルから利用できるようにする指定**として理解する必要がある。

inline 関数をヘッダファイルに記述する例を次に示す。

```
#ifndef CALC_H  
#define CALC_H  
    inline double square(double x) {  
        return x * x;  
    }  
#endif
```

このヘッダファイルを複数のソースファイルから読み込んでも、square 関数の定義の重複によるリンク時のエラーは

発生しない。どのソースファイルから呼び出した場合も、同じ `square` 関数を呼び出すものとして扱われる。

クラスのメンバ関数についても、クラス定義の外部に記述した定義へ `inline` を指定できる。例えば、次のクラス定義がヘッダファイルに記述されているとする。

```
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D(double x0, double y0);
    double length() const;
};
```

メンバ関数の定義も同じヘッダファイルに置く場合は、次のように `inline` を指定できる。(cmath ヘッダファイルを読み込んだ前提)

```
inline Vec2D::Vec2D(double x0, double y0) : x(x0), y(y0) { }
inline double Vec2D::length() const {
    return std::sqrt(x * x + y * y);
}
```

この場合、Vec2D クラスの定義とメンバ関数の定義を、一つのヘッダファイルとして複数のソースファイルから利用できる。ただし、関数の処理が長い場合や、実装内容を利用側から分離したい場合は、通常どおりメンバ関数の定義をソースファイルに記述する方がよい。inline を使用できるという理由だけで、全てのメンバ関数をヘッダファイルに記述するべきではない。

また、`constexpr` を指定した関数やメンバ関数は、暗黙的に `inline` 関数として扱われる。従って、`constexpr` 関数の定義も、ヘッダファイルに記述して複数のソースファイルから利用できる。

19.4.2 クラス定義内で定義されたメンバ関数

クラス定義の内部に処理内容まで記述したメンバ関数は、`inline` を明示しなくても、暗黙的に `inline` 関数として扱われる。例えば、次の `length` メンバ関数は `inline` 関数である。

```
class Vec2D {
private:
    double x;
    double y;
public:
    Vec2D(double x0, double y0) : x(x0), y(y0) { }
    double length() const {
        return std::sqrt(x * x + y * y);
    }
};
```

このクラス定義をヘッダファイルに記述し、複数のソースファイルから読み込んでも、コンストラクタや `length` メンバ関数の定義が重複したものとして扱われることはない。クラス定義の内部で定義されたこれらのメンバ関数には、次のように `inline` が指定されているものと考えることができる。

```
inline Vec2D(double x0, double y0) : x(x0), y(y0) { }
inline double length() const {
    return std::sqrt(x * x + y * y);
}
```

ただし、この表記は暗黙的な指定を説明するために示したものであり、クラス定義内のメンバ関数へ改めて `inline` を記述する必要はない。明示的に記述することも可能であるが、通常は省略する。

クラス定義内に記述するメンバ関数は、短く、内容を容易に把握できるものに向いている。例えば、単純な値の取得や設定を行うメンバ関数は、クラス定義内に記述されることが多い。

```
double get_x() const { return x; }
double get_y() const { return y; }
```

一方、長い処理や、実装の詳細を外部から隠したい処理は、クラス定義内には宣言だけを置き、定義をソースファイ

ルに分離する方が、クラスの構成を把握しやすい。

19.4.3 inline 変数

通常のグローバル変数をヘッダファイルに定義すると、そのヘッダファイルを読み込んだ複数のソースファイルに、同じ変数の定義が現れる。例えば、ヘッダファイルに次の記述を置くことは適切ではない。

```
int counter = 0;
```

このヘッダファイルを複数のソースファイルから読み込むと、それぞれのオブジェクトファイルに `counter` の定義が含まれ、リンク時に定義の重複として通常はエラーになる。

C++17以降では、変数の定義に `inline` を指定することにより、同一の変数定義を複数のソースファイルから利用できる。このような変数を **inline 変数** という。

```
inline int counter = 0;
```

この定義をヘッダファイルに記述し、複数のソースファイルから読み込むことができる。それぞれのソースファイルから参照される `counter` は、リンク後のプログラムでは一つの変数として扱われる。ソースファイルごとに別々の変数が作られるわけではない。

`inline` 変数は、クラスの静的データメンバにも使用できる。例えば、作成されたオブジェクトの個数を保持する静的データメンバを、次のようにクラス定義内で初期化できる。

```
class Sample {
private:
    inline static int count = 0;
public:
    Sample() { ++count; }
    static int get_count() { return count; }
};
```

このクラス定義をヘッダファイルに置いて複数のソースファイルから読み込んでも、`count` は一つの静的データメンバとして扱われる。従来の形式では、静的データメンバの実体をクラス定義の外部に別途定義する必要がある。静的データメンバの定義については、次の節で詳しく説明する。

■ extern 宣言された変数との違い

分割された複数のプログラムでグローバル変数を共有する方法としては、変数の `extern` 宣言もある。これは、変数の実体を別のソースファイルなどに1つだけ定義し、ヘッダファイルにはその変数を参照するための宣言だけを記述する方法である。これに対して `inline` 変数では、変数の定義そのものをヘッダファイルに記述し、そのヘッダファイルが複数のソースファイルから読み込まれても、一つの変数として扱われるため、記述をより簡潔にできる。

ただし、変数の実体を特定のソースファイル側に置きたい場合には、`extern` 宣言を用いる方法も有効である。

19.4.4 定義の重複に関する注意

C++では、外部から共有される関数や変数などの実体を与える通常の定義は、プログラム全体で原則として一つだけ存在する必要がある。同じ関数や変数の定義が複数のオブジェクトファイルに含まれると、リンク時に定義の重複としてエラーになる場合がある。

一方、クラス定義、`inline` 関数、`inline` 変数、テンプレートなどは、同一の定義を複数のソースファイルから利用できるようにする必要がある。このため、所定の条件を満たす場合には、同じ定義が複数のソースファイルに現れることが許されている。このような定義に関する規則を、**One Definition Rule (ODR)** という。

複数のソースファイルに現れることが許される場合でも、それぞれの定義は同一でなければならない。例えば、あるソースファイルでは次の定義が使用され、

```
inline double square(double x) { return x * x; }
```

別のソースファイルでは、同じ関数について異なる定義が使用される状態にしてはならない。

```
inline double square(double x) { return x * x + 1; }
```

このような状態は、プログラムの定義が一致しておらず、正しいプログラムとはならない。ただし、この種の不一致

は、コンパイラやリンカによって必ず検出されるとは限らないため、特に注意が必要である。

通常は、一つのヘッダファイルに定義をまとめ、全てのソースファイルから同じヘッダファイルを読み込むことで、定義の一致を保つ。ソースファイルごとに同じ `inline` 関数を手作業で重複して記述する方法は避けるべきである。

ヘッダファイルに記述できる主な定義を、簡単にまとめると次のようになる。

記述するもの	ヘッダファイルに定義を置く場合の扱い
通常の間数	原則として定義を置かない
<code>inline</code> 関数	同一の定義を複数のソースファイルから利用できる
クラス定義内で定義したメンバ関数	暗黙的に <code>inline</code> 関数として扱われる
通常グローバル変数	原則として定義を置かない
<code>inline</code> 変数	同一の定義を複数のソースファイルから利用できる
クラス定義	同一の定義を複数のソースファイルから利用できる
テンプレート	一般に宣言と定義をヘッダファイルへ記述する

※ テンプレートについては、「16 型を抽象化するための機能」(p.322～)を参照のこと。

■ サンプルプログラム

次の例では、`inline` 関数と、クラス定義内で定義されたメンバ関数をヘッダファイルに記述し、二つのソースファイルから読み込んで使用する。三つのファイルを同じディレクトリに保存する。

test290-4-func.h
test290-4-sub.cpp
test290-4-main.cpp

ヘッダファイル：test290-4-func.h

```
1  #ifndef      TEST290_4_FUNC_H
2  #define      TEST290_4_FUNC_H
3
4  inline double square(double x) {
5      return x * x;
6  }
7
8  class Value {
9      private:
10         int value;
11     public:
12         Value(int v) : value(v) { }
13         int get() const { return value; }
14 };
15
16 double calc_square(double x);
17
18 #endif
```

このヘッダファイルには、`inline` を指定した `square` 関数と、クラス定義内で定義した `Value` クラスのコンストラクタおよび `get` メンバ関数が記述されている。

関数の実装：test290-4-sub.cpp

```
1  #include      "test290-4-func.h"
2
3  double calc_square(double x) {
4      return square(x);
5  }
```

プログラム：test290-4-main.cpp

```
1  #include      <iostream>
2  #include      "test290-4-func.h"
3
4  int main() {
```



```

5 |     std::cout << "square(3) = " << square(3.0) << "\n";
6 |     std::cout << "square(4) = " << calc_square(4.0) << "\n";
7 |
8 |     Value v(10);
9 |     std::cout << "value = " << v.get() << "\n";
10| }

```

二つのソースファイル test290-4-sub.cpp, test290-4-main.cpp は、どちらも test290-4-func.h を読み込む。square 関数は明示的に、Value クラスのコンストラクタと get メンバ関数は暗黙的に inline 関数として扱われるため、定義の重複によるリンク時のエラーは発生しない。

このプログラムをコンパイルして実行する例を次に示す。

例. test290-4-sub.cpp と test290-4-main.cpp のコンパイルと実行

```

$ g++ test290-4-sub.cpp test290-4-main.cpp -o test290-4-main 
$ ./test290-4-main  ←実行
square(3) = 9          ←出力
square(4) = 16
value = 10

```

このように、ヘッダファイルに関数や変数の定義を記述する場合は、その定義が複数のソースファイルに現れてもよい種類のものであるかを確認する必要がある。

19.5 静的データメンバの定義

通常のデータメンバは、クラスから作成された各オブジェクトが、それぞれ個別に保持する。これに対して、データメンバの宣言に static を指定すると、そのデータメンバは個々のオブジェクトには属さず、クラス全体で一つだけ共有される。このようなデータメンバを静的データメンバという。

例えば、次の Sample クラスでは、静的データメンバ count を宣言している。

```

class Sample {
private:
    static int count;
public:
    Sample();
    static int get_count();
};

```

count は、Sample クラスから作成された各オブジェクトが個別に保持する値ではなく、Sample クラス全体で共有される一つの値である。従って、複数の Sample オブジェクトを作成しても、count がオブジェクトの個数だけ作られるわけではない。

クラス定義内の次の記述は、通常、静的データメンバの宣言であり、その実体を作る定義ではない。

```
static int count;
```

このため、静的データメンバの実体を、クラス定義の外部で別途定義する必要がある。基本的な形式を次に示す。

データ型 クラス名::静的データメンバ名;

先の count は、次のように定義できる。

```
int Sample::count;
```

この定義によって、Sample::count の実体が一つ作られる。静的記憶期間を持つ int 型の変数であるため、初期値を明示しない場合は 0 で初期化される。

初期値を明示する場合は、クラス外の定義に初期値を記述する。

```
int Sample::count = 0;
```

このクラス外の定義では、static を再度記述しない。static は、クラス定義内の宣言において、そのデータメンバが静的データメンバであることを指定する。

静的データメンバを使用するメンバ関数は、例えば次のように定義できる。

```

Sample::Sample() {
    ++count;
}

int Sample::get_count() {
    return count;
}

```

Sample クラスのメンバ関数内では、静的データメンバを `count` と記述して使用できる。クラス外からアクセスが許可されている場合は、次のようにクラス名を指定して参照する。

```
Sample::count
```

ただし、先の例では `count` は `private` に指定されているため、クラス外から直接参照することはできない。代わりに、公開された静的メンバ関数 `get_count` を次のように呼び出す。

```
int n = Sample::get_count();
```

静的メンバ関数も、特定のオブジェクトには属さず、クラス名を指定して呼び出すことができる。静的メンバ関数については、先の「静的メンバ」で説明した。

静的データメンバは、オブジェクトを介して参照することもできる。例えば、静的データメンバが `public` であれば、次のような記述も可能である。

```
Sample s;
std::cout << s.count << "\n";
```

ただし、`count` はオブジェクト `s` が個別に保持する値ではない。クラス全体で共有される値であることを明確にするため、通常は `Sample::count` のようにクラス名を指定して記述する方がよい。

静的データメンバの宣言をヘッダファイルに記述し、その定義をソースファイルに記述する例を次に示す。

ヘッダファイル：sample.h

```

#ifndef SAMPLE_H
#define SAMPLE_H

class Sample {
private:
    static int count;
public:
    Sample();
    static int get_count();
};

#endif

```

ソースファイル：sample.cpp

```

#include "sample.h"

int Sample::count = 0;

Sample::Sample() {
    ++count;
}

int Sample::get_count() {
    return count;
}

```

ヘッダファイルは複数のソースファイルから読み込まれる可能性があるため、従来の形式の静的データメンバについては、次の定義をヘッダファイルへ記述してはならない。

```
int Sample::count = 0;
```

この定義をヘッダファイルに置くと、そのヘッダファイルを読み込んだ複数のソースファイルに、`Sample::count` の定義が現れ、通常はリンク時に定義の重複としてエラーになる。従って、静的データメンバのクラス外定義は、一つのソースファイルにだけ記述する。

前節で説明したように、C++17 以降では、静的データメンバに `inline` を指定することもできる。

```
class Sample {
private:
    inline static int count = 0;
public:
    Sample() { ++count; }
    static int get_count() { return count; }
};
```

`inline static` を指定した静的データメンバは、クラス定義内で初期化できる。この場合、クラス外に次の定義を別途記述する必要はない。

```
int Sample::count = 0;
```

このクラス定義をヘッダファイルに置き、複数のソースファイルから読み込んでも、`count` はプログラム全体で一つの静的データメンバとして扱われる。ソースファイルごとに異なる `count` が作られるわけではない。

なお、`inline static` と `static inline` は、どちらの順序でも記述でき、意味は同じである。本書では、原則として `inline static` の順序で記述する。

また、`constexpr` を指定した静的データメンバは、C++17 以降では暗黙的に `inline` 変数として扱われる。例えば、次のようにクラス定義内で初期化できる。

```
class Circle {
public:
    static constexpr double pi = 3.141592653589793;
};
```

この場合、C++17 以降では、クラス外に `Circle::pi` の定義を別途記述する必要はない。本書では C++17 を前提としているため、以降はこの形式をそのまま使用する。

■ サンプルプログラム

次の例では、`Human` クラスに、コンストラクタが呼び出された回数を保持する静的データメンバ `count` を定義する。クラスの定義と静的データメンバの定義を別のファイルに分け、三つのファイルを同じディレクトリに保存する。

```
test290-5-human.h
test290-5-human.cpp
test290-5-main.cpp
```

ヘッダファイル：test290-5-human.h

```
1  #ifndef      TEST290_5_HUMAN_H
2  #define      TEST290_5_HUMAN_H
3
4  class Human {
5  private:
6      static int count;
7  public:
8      Human();
9      static int get_count();
10 };
11
12 #endif
```

このヘッダファイルでは、`Human` クラスの内部に静的データメンバ `count` を宣言している。この記述だけでは、`count` の実体はまだ作られない。

クラスの実装：test290-5-human.cpp

```
1  #include      "test290-5-human.h"
2
3  int Human::count = 0;
4
5  Human::Human() {
6      ++count;
7  }
8
9  int Human::get_count() {
10     return count;
11 }
```

```
11 }
```

このソースファイルでは、次の記述によって静的データメンバ `count` を定義し、0 で初期化している。

```
int Human::count = 0;
```

また、コンストラクタが呼び出されるたびに `count` を1増やす。

プログラム：test290-5-main.cpp

```
1  #include    <iostream>
2  #include    "test290-5-human.h"
3
4  int main() {
5      std::cout << "count = " << Human::get_count() << "\n";
6
7      Human h1;
8      std::cout << "count = " << Human::get_count() << "\n";
9
10     Human h2;
11     std::cout << "count = " << Human::get_count() << "\n";
12 }
```

このプログラムでは、`Human` オブジェクトを順に作成し、静的メンバ関数 `get_count` によって、それまでにコンストラクタが呼び出された回数を表示する。

このプログラムをコンパイルして実行する例を次に示す。

例. test290-5-human.cpp と test290-5-main.cpp のコンパイルと実行

```
$ g++ test290-5-human.cpp test290-5-main.cpp -o test290-5-main
```

Enter

```
$ ./test290-5-main
```

Enter

←実行

```
count = 0
```

←出力（初期状態）

```
count = 1
```

← `Human` オブジェクトを1つ作成した直後

```
count = 2
```

← 2つ目の `Human` オブジェクトを作成した直後

静的データメンバは、クラス全体に共通する状態や設定を保持する場合に使用できる。例えば、作成されたオブジェクトの個数、共通の設定値、全てのオブジェクトから参照される管理情報などを表す場合に利用される。

19.6 テンプレートの定義とファイル分割

通常関数やクラスでは、宣言をヘッダファイルに記述し、定義をソースファイルに記述することができる。これに対して、関数テンプレートやクラステンプレートでは、一般に、テンプレートの定義も利用側のソースファイルから参照できるようにしておく必要がある。このため、テンプレートを含むプログラムでは、通常関数やクラスとは異なるファイル分割の方法が用いられる。26pt]

19.6.1 テンプレートの定義を利用側から参照できる必要性

例えば、次の関数テンプレート `maximum` を考える。

```
template<class T>
T maximum(T a, T b) {
    return a > b ? a : b;
}
```

この関数テンプレートを使用して、次のように関数を呼び出すことができる。

```
int n = maximum(10, 20);
double x = maximum(1.5, 2.7);
```

コンパイラは、この呼び出しに応じて、例えば `int` 型や `double` 型に対応する関数を生成する。このように、テンプレートから具体的な型に対応する関数やクラスを生成することを**インスタンス化**という。

通常の方法でコンパイラがテンプレートをインスタンス化するためには、テンプレートの宣言だけでなく、その処理内容を記述した定義も参照できなければならない。例えば、ヘッダファイルに次の宣言だけを記述したとする。

ヘッダファイル：maximum.h

```
#ifndef MAXIMUM_H
#define MAXIMUM_H

template<class T>
T maximum(T a, T b);

#endif
```

そして、テンプレートの定義を別のソースファイルに記述したとする。

関数テンプレートの定義ファイル：maximum.cpp

```
#include "maximum.h"

template<class T>
T maximum(T a, T b) {
    return a > b ? a : b;
}
```

この状態で、別のソースファイルから `maximum(10, 20)` を呼び出しても、利用側のソースファイルをコンパイルする際には、`maximum` の定義を参照できない。そのため、コンパイラは `int` 型に対応する `maximum` 関数を生成できず、通常は、リンク時に必要な関数が見つからないというエラーになる。

通常関数では、関数の定義を別のソースファイルでコンパイルし、生成されたオブジェクトファイルをリンクすればよい。しかし、テンプレートは、利用された型が分からなければ、具体的な関数やクラスを生成できない。どの型で使われるかは、テンプレートを利用するソースファイルをコンパイルする際に判明する。このため、通常は、テンプレートの定義を利用側から参照できる場所に置く。

最も一般的な方法は、テンプレートの宣言と定義を、どちらもヘッダファイルに記述する方法である。

ヘッダファイル：maximum.h（改変後）

```
#ifndef MAXIMUM_H
#define MAXIMUM_H

template<class T>
T maximum(T a, T b) {
    return a > b ? a : b;
}

#endif
```

このヘッダファイルを利用側のソースファイルから読み込めば、コンパイラは `maximum` の定義を参照し、必要な型に対応する関数を生成できる。

テンプレートの定義が複数のソースファイルに現れても、それぞれが同一の定義であれば許される。従って、関数テンプレートやクラステンプレートの定義は、一般にヘッダファイルへ記述する。

19.6.2 テンプレート定義の別ファイル化

テンプレートの定義が長い場合、宣言と定義を一つのヘッダファイルにまとめると、ファイルの内容が見通しにくくなることがある。この場合、テンプレートの定義を別のファイルへ分け、宣言を記述したヘッダファイルの末尾から、その定義ファイルを読み込む方法が用いられる。

例えば、クラステンプレート `Box` のクラス定義を、次のヘッダファイルに記述する。

ヘッダファイル：box.h

```
#ifndef BOX_H
#define BOX_H

template<class T>
class Box {
private:
    T value;
public:
    Box(const T& v);
    const T& get() const;
};

#include "box.tpp"

#endif
```


テンプレートのメンバ関数の定義は、次のファイルに記述する。

テンプレート定義ファイル：box.hpp

```
template<class T>
Box<T>::Box(const T& v) : value(v) { }

template<class T>
const T& Box<T>::get() const {
    return value;
}
```

box.h の末尾で box.hpp を読み込むため、box.h を読み込んだ利用側のソースファイルからは、クラステンプレートのクラス定義とメンバ関数の定義の両方を参照できる。従って、コンパイラは必要な型に対応する Box クラスを生成できる。

.hpp は、テンプレートの定義を記述するファイルに用いられることがある拡張子である。このほかに .ipp, .inl などを使用される場合もある。これらは C++ の規格で定められた特別な拡張子ではなく、ファイルの用途を分かりやすくするための慣例である。通常のヘッダファイルと同様に、`#include` によって内容が読み込まれる。

テンプレート定義ファイルを、利用側のソースファイルから直接読み込む必要はない。通常は、宣言を記述したヘッダファイルから定義ファイルを読み込み、利用側では次のようにして宣言側のヘッダファイルだけを読み込む。

```
#include "box.h"
```

また、box.hpp を独立したソースファイルとしてコンパイルするわけではない。その内容は box.h を介して利用側のソースファイルに取り込まれ、そこでコンパイルされる。従って、例えば次のように box.hpp だけを個別にコンパイルするものではない。

```
g++ -c box.hpp
```

テンプレートの宣言と定義を別ファイルに分けても、コンパイラから見れば、利用側で両方を参照できる状態にしておくことが重要である。

関数テンプレートについても、同様に宣言と定義を別ファイルへ分けることができる。例えば、ヘッダファイルの末尾でテンプレート定義ファイルを読み込む。

ヘッダファイル：maximum.h (更に改変)

```
#ifndef MAXIMUM_H
#define MAXIMUM_H

template<class T>
T maximum(T a, T b);

#include "maximum.hpp"
#endif
```

テンプレート定義ファイル：maximum.hpp

```
template<class T>
T maximum(T a, T b) {
    return a > b ? a : b;
}
```

この方法は、宣言と定義を別ファイルに整理するためのものであり、通常関数のように宣言と定義を別々にコンパイルする方法ではない。定義ファイルの内容は、最終的にはヘッダファイルを通して利用側へ取り込まれる。

19.6.3 明示的インスタンス化

使用する型があらかじめ限定されている場合には、テンプレートの定義をソースファイルに置き、特定の型に対応する関数やクラスを、そのソースファイル内であらかじめ生成する方法もある。この操作を**明示的インスタンス化**という。

例えば、関数テンプレート `maximum` を `int` 型と `double` 型だけで使用する場合を考える。ヘッダファイルには、テンプレートの宣言を記述する。

ヘッダファイル：maximum.h（明示的インスタンス化を用いる構成）

```
#ifndef MAXIMUM_H
#define MAXIMUM_H

template<class T>
T maximum(T a, T b);

#endif
```

ソースファイルには、テンプレートの定義と、使用する型に対する明示的インスタンス化を記述する。

ソースファイル：maximum.cpp

```
#include "maximum.h"

template<class T>
T maximum(T a, T b) {
    return a > b ? a : b;
}

template int maximum<int>(int, int);
template double maximum<double>(double, double);
```

最後の二つの記述により、int 型と double 型に対応する maximum 関数が、このソースファイルのコンパイル時に生成される。

利用側のソースファイルでは、テンプレートの定義を参照できなくても、生成済みの関数を呼び出すことができる。

プログラム：main.cpp

```
#include <iostream>
#include "maximum.h"

int main() {
    std::cout << maximum(10, 20) << "\n";
    std::cout << maximum(1.5, 2.7) << "\n";
}
```

これらのファイルは、例えば次のようにコンパイルしてリンクできる。

```
$ g++ maximum.cpp main.cpp -o main
```

ただし、明示的にインスタンス化していない型については、対応する関数が生成されていない。例えば、利用側で次のように long 型を使用しても、maximum<long> の定義は生成されていないため、リンク時にエラーになる。

```
long n = maximum(10L, 20L);
```

この型を使用するには、ソースファイルに次の明示的インスタンス化を追加する必要がある。

```
template long maximum<long>(long, long);
```

従って、このようにテンプレートの定義を利用側から隠す構成では、明示的インスタンス化によって、利用できる型をあらかじめ決めておくことになる。

クラステンプレートについても、クラス全体を明示的にインスタンス化できる。例えば、Box<int> と Box<double> を生成する場合は、テンプレートの定義を記述したソースファイルに次のように記述する。

```
template class Box<int>;
template class Box<double>;
```

この場合も、利用側で利用できるのは、原則として明示的にインスタンス化された型に限られる。

明示的インスタンス化を使用すると、関数テンプレートの定義や、クラステンプレートのメンバ関数などの実装をソースファイル側へ置き、利用側から実装内容を分離できる。また、同じ型に対するインスタンス化を複数のソースファイルで繰り返すことを避けられる場合がある。一方、使用可能な型を追加するたびに、明示的インスタンス化の記述を追加し、ソースファイルを再コンパイルする必要がある。

一般的な汎用テンプレートでは、どの型で使用されるかをあらかじめ限定できないため、宣言と定義をヘッダファイルに置く方法が広く用いられる。明示的インスタンス化は、使用する型が限定されているライブラリや、テンプレートの実装を利用側から分離したい場合などに用いられる。

参考)

テンプレートは、本来、型に依存しない形で処理を記述し、さまざまな型に対して再利用できるようにするための仕組みである。明示的インスタンス化を、テンプレート定義を利用側から隠す構成で用いると、使用できる型をあらかじめ限定することになるため、テンプレートの持つ柔軟性は小さくなる。従って、コンパイル時間や生成されるコード量の抑制、ライブラリとして提供する型の管理などが特に必要な場合を除き、その採用は慎重に検討する必要がある。

■ サンプルプログラム

次の例では、クラステンプレート Box のクラス定義をヘッダファイルに記述し、メンバ関数の定義をテンプレート定義ファイルへ分ける。三つのファイルを同じディレクトリに保存する。

```
test290-6-box.h
test290-6-box.tpp
test290-6-main.cpp
```

ヘッダファイル：test290-6-box.h

```
1  #ifndef      TEST290_6_BOX_H
2  #define      TEST290_6_BOX_H
3
4  template<class T>
5  class Box {
6      private:
7          T value;
8
9      public:
10         Box(const T& v);
11         const T& get() const;
12 };
13
14 #include "test290-6-box.tpp"
15 #endif
```

このヘッダファイルでは、クラステンプレート Box を定義し、末尾で test290-6-box.tpp を読み込んでいる。

テンプレート定義ファイル：test290-6-box.tpp

```
1  template<class T>
2  Box<T>::Box(const T& v) : value(v) { }
3
4  template<class T>
5  const T& Box<T>::get() const {
6      return value;
7  }
```

このファイルには、Box クラスのコンストラクタと get メンバ関数の定義を記述している。このファイルは単独ではコンパイルせず、test290-6-box.h を通して利用側のソースファイルへ読み込まれる。

プログラム：test290-6-main.cpp

```
1  #include      <iostream>
2  #include      "test290-6-box.h"
3
4  int main() {
5      Box<int> b1(10);
6      Box<double> b2(3.14);
7
8      std::cout << b1.get() << "\n";
9      std::cout << b2.get() << "\n";
10 }
```

このプログラムでは、Box<int> と Box<double> のオブジェクトを作成している。コンパイラは、それぞれの型に対応するクラスとメンバ関数を生成する。

このプログラムをコンパイルして実行する例を次に示す。

例. test290-6-main.cpp のコンパイルと実行

```
$ g++ test290-6-main.cpp -o test290-6-main Enter ←コンパイル
$ ./test290-6-main Enter ←実行
10
3.14
```

テンプレートのクラス定義とメンバ関数の定義を別ファイルに分ける場合でも、利用側のソースファイルをコンパイルする時点で、その両方を参照できるようにしておく必要がある。

19.7 C 言語のプログラムとの連携

C++はC言語を基礎として設計されており、C言語で作成された関数やライブラリをC++のプログラムから利用できる。また、C++で定義した関数を、C言語のプログラムから呼び出すこともできる。ただし、CとC++では、関数名をオブジェクトファイルへ記録する方法などに違いがあるため、単に関数の宣言を記述するだけでは、正しくリンクできない場合がある。本節では、CとC++の関数を相互に利用するための基本的な方法を説明する。

19.7.1 CとC++のリンケージの違い

複数のソースファイルから構成されるプログラムでは、あるソースファイルから呼び出される関数と、別のソースファイルに定義された関数とを、リンカが対応させる必要がある。このように、異なる場所に現れる同じ名前が、同一の関数や変数などを表すかどうかを定める規則を、**リンケージ**という。

C言語では、同じ有効範囲に、引数の型や個数だけが異なる同名の関数を複数定義することはできない。例えば、次の二つの関数を、同じ有効範囲に定義することはできない。

```
int calc(int n);
double calc(double x);
```

これに対して、C++では引数の型や個数が異なれば、同じ名前を持つ複数の関数を定義できる。この機能を関数の**オーバーロード**という。

```
int calc(int n);
double calc(double x);
```

C++のコンパイラは、これらの関数を区別するため、関数名だけでなく、引数の型などの情報も組み合わせた名前をオブジェクトファイルへ記録する。このような処理を**名前修飾**、または **name mangling** という。

例えば、ソースコード上では同じ `calc` という名前であっても、オブジェクトファイル内では、`int` 型を受け取る関数と `double` 型を受け取る関数が異なる名前として記録される。具体的な名前修飾の形式は、コンパイラや実行環境によって異なる。

一方、C言語のコンパイラは、C++の関数オーバーロードを前提とした名前修飾を行わない。このため、C言語で定義された関数をC++側で通常のC++関数として宣言すると、C++コンパイラが名前修飾した名前を使用しようとする場合がある。その結果、C側のオブジェクトファイルに記録された関数名と一致せず、リンク時に関数が見つからないというエラーになる。

例えば、C言語のソースファイルに、次の関数を定義したとする。

C言語のソースファイル：calc.c

```
int add(int a, int b) {
    return a + b;
}
```

この関数をC++側で単に次のように宣言すると、**C++言語リンケージ**¹⁷⁵ (C++ language linkage) を持つ関数として扱われる。

```
int add(int a, int b);
```

しかし、実際の関数はC言語のコンパイラで生成されているため、両者のリンケージが一致しない場合がある。C言語で定義された関数をC++から利用する場合は、その関数がC言語リンケージを持つことを、C++側で明示する必要がある。

¹⁷⁵これに対して、C言語の場合は **C言語リンケージ** (C language linkage) という。

19.7.2 extern "C"

C++では、関数の宣言に `extern "C"` を指定することにより、その関数を C 言語リンケージで扱うように指定できる。基本的な形式を次に示す。

`extern "C" 戻り値の型 関数名 ( 仮引数 );`

例えば、先の `add` 関数は、C++側で次のように宣言する。

```
extern "C" int add(int a, int b);
```

この宣言によって、C++コンパイラは、`add` を C 言語リンケージを持つ関数として扱う。従って、C 言語のコンパイラで生成された `add` 関数と正しく対応させることができる。

複数の関数をまとめて C 言語リンケージに指定する場合は、次のように波括弧を使用できる。

```
extern "C" {
    int add(int a, int b);
    int sub(int a, int b);
    double average(double a, double b);
}
```

波括弧の内部に記述した関数宣言は、いずれも C 言語リンケージを持つ。

`extern "C"` は、関数の処理内容を C 言語としてコンパイルする指定ではない。C++のソースファイル内に記述された関数定義は、処理内容としては C++の規則に従ってコンパイルされる。`extern "C"` が指定するのは、主に、外部から関数を識別するためのリンケージである。

例えば、C++のソースファイル内で、次のように関数を定義することもできる。

```
extern "C" int multiply(int a, int b) {
    return a * b;
}
```

この関数は C++コンパイラでコンパイルされるが、C 言語リンケージを持つため、適切な宣言を用意すれば、C 言語のプログラムから呼び出すことができる。

C 言語リンケージを持つ関数では、C++の関数オーバーロードを利用できない。例えば、次のように同じ名前を持つ複数の関数へ `extern "C"` を指定することはできない。

```
extern "C" int calc(int n);
extern "C" double calc(double x);
```

C 言語では引数の型による関数名の区別を行わないため、これらを同じ名前の別関数として扱うことはできない。

また、クラステンプレートや関数テンプレート、通常の非静的メンバ関数を、C 言語からそのまま呼び出すことはできない。C 言語には、クラス、オブジェクト、関数オーバーロード、テンプレートなどの仕組みが存在しないためである。C 言語から C++の機能を利用させる場合は、C 言語から呼び出せる通常の関数を窓口として用意する。

例えば、C++のクラスを内部で利用する処理を、次のような C 言語形式の関数として公開できる。

```
extern "C" int process_value(int n) {
    // C++のクラスやライブラリを内部で利用する
    return n * 2;
}
```

C 言語側から見えるのは `process_value` という通常の関数だけであり、その内部では C++の機能を使用できる。このような関数は、C と C++の間を接続するための窓口として利用される。

19.7.3 C と C++の双方から利用する宣言

C 言語と C++の双方から利用するヘッダファイルでは、`__cplusplus` マクロを使用して記述を切り替えることができる。`__cplusplus` は、C++コンパイラで処理されている場合に自動的に定義されるマクロである。

例えば、C 言語で定義された `add` 関数の宣言を、次のようなヘッダファイルに記述できる。

ヘッダファイル: calc.h

```
#ifndef CALC_H
#define CALC_H

#ifdef __cplusplus
extern "C" {
#endif

int add(int a, int b);

#ifdef __cplusplus
}
#endif

#endif
```

このヘッダファイルをC++のソースファイルから読み込む場合は、__cplusplus が定義されているため、extern "C" { … } が有効になる。従って、add はC言語リンケージを持つ関数として宣言される。

一方、C言語のソースファイルから読み込む場合は、__cplusplus が定義されていないため、extern "C" の部分は除外される。C言語には extern "C" という構文が存在しないため、この条件分岐が必要である。

この形式にしておけば、同じヘッダファイルをC言語とC++の双方から使用できる。C言語用のライブラリで公開されているヘッダファイルにも、同様の記述が用いられていることがある。

宣言する関数が一つだけの場合は、次のように記述することもできる。

```
#ifdef __cplusplus
extern "C"
#endif
int add(int a, int b);
```

ただし、複数の関数を宣言する場合は、波括弧でまとめる形式の方が見通しがよい。

C言語とC++の双方から使用するヘッダファイルには、C++固有の構文を記述しないように注意する必要がある。例えば、次のものはC言語では使用できない。

- ・ クラスや参照型
- ・ 関数オーバーロード
- ・ テンプレート
- ・ 名前空間
- ・ std::string やC++標準ライブラリの型

C言語との境界で使用する関数の引数や戻り値には、int, double, ポインタ, C言語形式の構造体など、両言語から扱える型を用いる。

また、C++の例外をC言語側へ伝えることはできない。C言語から呼び出される関数の内部で例外が発生する可能性がある場合は、C++側で例外を捕捉し、整数のエラーコードなどへ変換して返す必要がある。例えば、次のような形式にできる。

```
extern "C" int run_process() {
    try {
        // C++の処理
        return 0;
    } catch (...) {
        return -1;
    }
}
```

このように、CとC++の境界では、両言語から安全に扱える形式へ変換する必要がある。

19.7.4 CとC++のオブジェクトファイルのリンク

C言語のソースファイルとC++のソースファイルは、それぞれ別々にコンパイルし、生成されたオブジェクトファイルをリンクできる。

例えば、C言語で定義した add 関数を、C++のプログラムから利用する場合を考える。次の三つのファイルを同じディレクトリに保存する。

```
test290-7-cfunc.h
test290-7-cfunc.c
test290-7-main.cpp
```

ヘッダファイル：test290-7-cfunc.h

```
1  #ifndef      TEST290_7_CFUNC_H
2  #define      TEST290_7_CFUNC_H
3
4  #ifdef  __cplusplus
5  extern "C" {
6  #endif
7
8  int add(int a, int b);
9
10 #ifdef  __cplusplus
11 }
12 #endif
13
14 #endif
```

C 言語のプログラム：test290-7-cfunc.c

```
1  #include      "test290-7-cfunc.h"
2
3  int add(int a, int b) {
4      return a + b;
5  }
```

C++のプログラム：test290-7-main.cpp

```
1  #include      <iostream>
2  #include      "test290-7-cfunc.h"
3
4  int main() {
5      std::cout << add(10, 20) << "\n";
6  }
```

まず、C 言語のソースファイルを gcc でコンパイルする。

```
$ gcc -c test290-7-cfunc.c
```

これにより、オブジェクトファイル test290-7-cfunc.o が生成される。次に、C++のソースファイルと C 言語のオブジェクトファイルを、g++ を使用してリンクする。

```
$ g++ test290-7-main.cpp test290-7-cfunc.o -o test290-7-main
```

C++のプログラムを含むため、最終的なリンクには g++ を使用する。g++ を使用することにより、C++標準ライブラリなど、C++のプログラムに必要なライブラリも適切にリンクされる。

このプログラムを実行すると、次の結果が得られる。

```
$ ./test290-7-main 
30                ←出力
```

test290-7-main.cpp は test290-7-cfunc.h を読み込む。C++コンパイラで処理されるため、add 関数は extern "C" によって C 言語リンケージで宣言される。従って、C 言語のコンパイラで生成された add 関数と正しくリンクされる。

次に、C++で定義した関数を C 言語のプログラムから呼び出す例を示す。次の三つのファイルを同じディレクトリに保存する。

```
test290-7-cppfunc.h
test290-7-cppfunc.cpp
test290-7-cmain.c
```

ヘッダファイル：test290-7-cppfunc.h

```
1  #ifndef      TEST290_7_CPPFUNC_H
2  #define      TEST290_7_CPPFUNC_H
3
```

```

4  #ifdef __cplusplus
5  extern "C" {
6  #endif
7
8  int multiply(int a, int b);
9
10 #ifdef __cplusplus
11 }
12 #endif
13
14 #endif

```

このヘッダファイルは、C 言語と C++ の双方から読み込むことができる。C++ コンパイラで読み込まれた場合は、multiply 関数が C 言語のリンケージを持つ関数として宣言される。

C++ のソースファイル：test290-7-cppfunc.cpp

```

1  #include "test290-7-cppfunc.h"
2
3  int multiply(int a, int b) {
4      return a * b;
5  }

```

このソースファイルでは、test290-7-cppfunc.h に記述された宣言に対応する multiply 関数を定義している。ヘッダファイル側ですでに extern "C" を指定しているため、関数定義に extern "C" を改めて記述しなくても、multiply は C 言語リンケージを持つ。

定義側にも C 言語リンケージであることを明示する場合は、次のように記述することもできる。

```

extern "C" int multiply(int a, int b) {
    return a * b;
}

```

ただし、ヘッダファイルに記述された宣言と定義のリンケージは一致していなければならない。通常は、宣言を共通のヘッダファイルにまとめ、定義側ではそのヘッダファイルを読み込む方法が用いられる。

C 言語のプログラム：test290-7-cmain.c

```

1  #include <stdio.h>
2  #include "test290-7-cppfunc.h"
3
4  int main() {
5      printf("%d\n", multiply(6, 7));
6  }

```

このプログラムは C 言語としてコンパイルされるため、ヘッダファイル内の extern "C" の部分は除外される。C 言語側からは、multiply は通常の C 言語の関数として見える。

まず、C++ のソースファイルを g++ でコンパイルする。

```
$ g++ -c test290-7-cppfunc.cpp
```

これにより、オブジェクトファイル test290-7-cppfunc.o が生成される。次に、C 言語のプログラムを gcc でコンパイルする。

```
$ gcc -c test290-7-cmain.c
```

最後に、二つのオブジェクトファイルをリンクする。C++ でコンパイルされたオブジェクトファイルを含むため、最終的なリンクには g++ を使用する。

```
$ g++ test290-7-cmain.o test290-7-cppfunc.o -o test290-7-cmain
```

このプログラムを実行すると、次の結果が得られる。

```
$ ./test290-7-cmain 
42          ←出力
```

この例では、multiply 関数の処理内容は C++ コンパイラによってコンパイルされるが、C 言語リンケージを持つため、C 言語でコンパイルされたプログラムから呼び出すことができる。

C 言語のソースファイル、C++ のソースファイル、それぞれのオブジェクトファイルは、同じプログラム内でリンクできることを示した。ただし、両言語の境界となる関数については、リンケージ、引数と戻り値の型、メモリ管理、例外処理などを、双方から安全に扱える形にそろえる必要がある。C 言語で提供されている既存のライブラリを C++ から利用する場合には、ライブラリ側のヘッダファイルに `extern "C"` のための記述が用意されていることも多い。

【第 2 部のまとめ】

第 2 部では、C++ の基本的な機能と、それらを用いてプログラムを構成するための考え方について解説した。C++ は C 言語を基礎としながら、参照、型推論、例外処理、クラス、関数や演算子のオーバーロード、継承、ポリモーフィズム、テンプレートなど、多くの機能を備えている。これらを利用することで、単に処理手順を記述するだけでなく、データの意味や操作のまとまりを型として表現し、プログラムの構造を明確にできる。

特に、クラスとオブジェクトは、C++ におけるプログラム設計の中心的な概念である。クラスを用いることで、データとそれに対する操作をまとめ、外部に公開するインターフェースと内部の実装を分離できる。また、継承や仮想関数、コンポジションなどを用いて、目的に応じたクラス構成を設計する考え方についても説明した。

さらに、オブジェクトの寿命と所有権、コピーや移動、RAII による資源管理、スマートポインタなど、C++ で安全にオブジェクトや資源を管理するための考え方についても説明した。これらは、動的に確保したメモリやファイルなどの資源を、オブジェクトの寿命と対応させて管理するための重要な仕組みである。

テンプレートを用いることで、型や値を抽象化した関数やクラスを定義できる。これは、`std::vector<T>` や `std::array<T, N>` のような標準ライブラリのクラスを理解するための重要な前提である。また、ラムダ式を用いることで、処理そのものをオブジェクトとして扱い、関数やアルゴリズムへ柔軟に受け渡すことができる。

本部ではさらに、スレッドによる並行処理、共有データに対する排他制御や原子的な操作、プログラムを複数のファイルへ分割する方法についても扱った。宣言と定義、ヘッダファイル、リンケージなどを適切に扱うことは、規模の大きな C++ プログラムを構成する上で重要である。

第 3 部では、ここまで学んだ C++ の文法と設計の考え方を前提として、標準ライブラリや実用的なライブラリの利用方法を扱う。C++ では、言語仕様そのものを理解するだけでなく、豊富なライブラリを組み合わせ、目的に応じて適切に利用することが重要である。

第3部

プログラムライブラリ

20 STLの基礎

C++ には、多くの実用的な機能をまとめた標準ライブラリが用意されている。その中でも、データの集合を扱うためのコンテナ、それらの要素を順にたどるためのイテレータ、データに対して汎用的な処理を行うアルゴリズムなどを中心とする部分は、**STL** (Standard Template Library) と呼ばれることが多い。

STL はその名の通り、テンプレートを活用して設計されたライブラリである。例えば、整数の列を扱う `std::vector<int>`、実数の列を扱う `std::vector<double>`、文字列の列を扱う `std::vector<std::string>` のように、扱う値の型を指定することで、同じ仕組みを様々な型に対して利用できる。このように、STL は「特定の型に依存しない汎用的な部品」を提供するための仕組みである。

STL は、多数の機能から構成されているが、その中心になるのは次のような要素である。

- ・ **コンテナ** : 複数の値をまとめて保持するためのクラス
- ・ **イテレータ** : コンテナ内の要素を順にたどるための仕組み
- ・ **アルゴリズム** : 探索、並べ替え、集計などの汎用的な処理
- ・ **関数オブジェクト** : 関数と同様の構文で実行できるオブジェクト
- ・ **アロケータ** : メモリ確保の方法を制御するための仕組み

STL は非常に大きなライブラリであり、本書では、その全体を詳しく解説することはせず、C++ を学び始めた読者が、実用的なプログラムを書く上で特に重要となる基本的な機能に絞って解説する。具体的には、動的配列を扱う `vector` など、使用頻度の高いコンテナを順に解説し、それに付随する形でイテレータやアルゴリズムについて解説する。

一方、アロケータ、高度な関数オブジェクト、複雑なイテレータの分類、並列実行ポリシー、C++20 で導入された `ranges` ライブラリなどについては本書では扱わない。これらについては、必要に応じて、C++ 標準ライブラリのリファレンスや、より専門的な解説書を参照すること。なお、厳密な仕様は、C++ 標準化委員会¹⁷⁶ によって策定される C++ 規格¹⁷⁷ に定められている。

以下では、まず STL の代表的なコンテナである `vector` を取り上げる。`vector` は、C 言語の配列に似た考え方で利用できる一方で、必要に応じて要素数を変更できるため、STL の基本的な考え方を学ぶ最初の題材として適している。また、`vector` の使い方を通して、コンテナ、イテレータ、アルゴリズムの関係も解説する。

20.1 要素数を変更できる配列：vector

`vector` は、複数の値を順番に並べて保持するためのコンテナである。C 言語の配列と同じように、先頭から順に値を並べ、添字を用いて各要素にアクセスできる。その一方で、通常の配列とは異なり、プログラムの実行中に要素の追加や削除が可能であるため、`vector` は「要素数を変更できる配列」と解釈することができる。

C 言語の配列では、要素数を決めてから配列を作成する。例えば、「`int a[10];`」と宣言すれば、`int` 型の値を 10 個保持する配列が作られる。この配列が作られた後で、その要素数を変更することはできない。より多くの要素を扱う必要が生じた場合には、より大きな領域を別に用意して、既存の要素を移すなどの処理が必要となる。

これに対して、`vector` では、必要に応じて要素を末尾に追加できる。内部では、必要に応じてメモリ領域の確保や再確保が行われるが、通常、プログラマがそれを直接管理する必要はない。そのため、扱うデータの個数があらかじめ明確に決まっていない場合や、実行中にデータを追加していくような処理には、配列よりも `vector` を用いる方がよい。

`vector` は、クラステンプレートとして定義されている。そのため、使用するときには、どの型の値を格納するかを指定する。例えば、`int` 型の値を格納する場合には「`std::vector<int>`」、`double` 型の値を格納する場合には「`std::vector<double>`」のように山括弧で `<型>` を記述する。このように、`vector` は、要素の型を指定することで、様々な型の値に対して同じ仕組みを利用できるコンテナである。`vector` を使用するにはヘッダファイル `vector` をインクルードする。

書き方： `std::vector<要素の型> 変数名;`

¹⁷⁶C++ 標準化委員会：ISO/IEC JTC1/SC22/WG21

¹⁷⁷本書が準拠する C++17 は ISO/IEC 14882:2017。

vector は std 名前空間に属しているため、std::vector のように記述する。

次のプログラム test241-1.cpp は、int 型の値を格納する vector を作り、そこに 3 個の整数値を追加して表示する例である。

プログラム：test241-1.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <cstdint>
4  using std::cout, std::boolalpha;
5
6  int main() {
7      std::vector<int> v;
8      cout << "initial size: " << v.size() << ", ";
9      cout << boolalpha << "emptiness: " << v.empty() << "\n";
10
11     v.push_back(10);
12     v.push_back(20);
13     v.push_back(30);
14
15     cout << "data: ";
16     for(std::size_t i = 0; i < v.size(); i++) {
17         if ( i != 0 ) cout << ", ";
18         cout << v[i];
19     }
20     cout << "\n";
21
22     cout << "current size: " << v.size() << ", ";
23     cout << "emptiness: " << v.empty() << "\n";
24 }
```

このプログラムでは、std::vector<int> 型の変数 v が宣言されている。引数を与えずに生成した vector の要素数は 0 である。その後、メンバ関数 push_back を実行することによって、末尾に要素を順次追加している。

ここで用いている std::size_t は、要素数やメモリサイズなどを表すために用いられる符号なし整数型である。これは、C 言語で用いられる size_t に対応する型であり、C++ では標準名前空間 std に属する名前として std::size_t と記述できる。v.size() の戻り値型は、厳密には std::vector<int>::size_type であり、今回のように標準のアロケータを使用する vector では std::size_t である。そこで、添字を順に変化させる変数 i も std::size_t 型として宣言している。この型を使用するには、ヘッダファイル cstdint をインクルードする。

メンバ関数 empty() は、対象の vector に要素がない場合に true を、そうでない場合に false を返す。

このプログラムをコンパイルして実行する例を次に示す。

例. test241-1.cpp のコンパイルと実行

```
$ g++ test241-1.cpp -o test241-1  [Enter]    ←コンパイル
$ ./test241-1  [Enter]    ←実行
initial size: 0, emptiness: true    ← vector オブジェクトの初期状態
data: 10, 20, 30                    ← 3 要素追加後の内容
current size: 3, emptiness: false   ←この時点での vector オブジェクトの状態
```

20.1.1 生成時の初期化

vector オブジェクトは、生成時に要素数や各要素の初期値を指定することができる。初期化の方法には、丸括弧 () を用いる直接初期化と、波括弧 { } を用いるリスト初期化がある。ただし、vector では、丸括弧と波括弧で意味が異なる場合があるため注意が必要である。

■ 丸括弧による初期化

丸括弧を用いると、要素数を指定して vector オブジェクトを生成できる。

書き方 1: std::vector<型> オブジェクト名 (要素数);

例えば、次のように記述すると、int 型の要素を 10 個持つ vector オブジェクト v が生成される。

```
std::vector<int> v(10);
```

この場合、各要素はその型に応じた既定の方法で初期化される。例えば、`int` 型の要素は 0 に、`double` 型の要素は 0.0 に初期化される。クラス型の要素では、通常、デフォルトコンストラクタによる初期化が行われる。

また、要素数と各要素の初期値を指定することもできる。

書き方 2: `std::vector<型> オブジェクト名 ( 要素数, 初期値 );`

例えば、次のように記述すると、値がすべて 5 である 10 個の要素を持つ `vector` オブジェクトが生成される。

```
std::vector<int> v(10, 5);
```

この場合、`v[0]` から `v[9]` までのすべての要素が 5 に初期化される。要素数を指定する丸括弧の初期化では、第 1 引数が要素数、第 2 引数が各要素に設定する初期値となる。

■ 波括弧による初期化

波括弧を用いると、生成時に格納する要素を順に列挙できる。

書き方: `std::vector<型> オブジェクト名 { 値 1, 値 2, ... };`

例えば、次のように記述すると、10, 20, 30 の 3 つの要素を持つ `vector` オブジェクトが生成される。

```
std::vector<int> v{10, 20, 30};
```

この場合、`v[0]` は 10、`v[1]` は 20、`v[2]` は 30 となる。波括弧の内部に記述した値が、そのまま各要素の初期値として用いられる。

空の波括弧を使用した場合は、要素を持たない空の `vector` オブジェクトが生成される。

```
std::vector<int> v{};
```

ただし、これは `vector` においては、

```
std::vector<int> v;
```

と結果的に同じである。

■ 丸括弧と波括弧の違い

`vector` では、丸括弧と波括弧による初期化で意味が異なる場合がある。例えば、次の 2 つの記述は、見た目はよく似ているが、生成されるオブジェクトの内容は異なる。

```
std::vector<int> v1(10);
std::vector<int> v2{10};
```

`v1` は、要素数が 10 で、各要素が 0 に初期化された `vector` オブジェクトである。これに対して、`v2` は、値 10 を持つ要素が 1 個だけ格納された `vector` オブジェクトである。

同様に、次の 2 つの記述も意味が異なる。

```
std::vector<int> v3(10, 5);
std::vector<int> v4{10, 5};
```

`v3` は、値がすべて 5 である 10 個の要素を持つ。これに対して、`v4` は、10 と 5 の 2 つの要素を持つ。このように、`vector` では、丸括弧を用いると主に要素数や共通の初期値を指定し、波括弧を用いると格納する要素そのものを列挙する。

従って、`vector` オブジェクトの生成時には、必要な要素数を指定するのか、格納する値を列挙するのかを考え、丸括弧と波括弧を適切に使い分ける必要がある。

ここで示した初期化方法の多くは、後述の `list` や `deque` など、他のシーケンスコンテナ¹⁷⁸ でも概ね同様に利用できる。ただし、コンテナの種類によって利用できるコンストラクタや初期化方法が異なる場合があるため、詳細については各コンテナの仕様を確認する必要がある。

20.2 イテレータ

`vector` では、C 言語の配列と同じように、添字を用いて各要素にアクセスできる。例えば `v[0]` は先頭要素、`v[1]` はその次の要素を表す。しかし STL のコンテナでは、添字だけが要素にアクセスする方法ではない。STL では、コ

¹⁷⁸`vector`, `deque`, `list`, `forward_list`, `array` などが代表的である。ただし、`array` は固定長であり、初期化方法は他のコンテナと異なる。

コンテナ内の要素を順にたどるための基本的な仕組みとして **イテレータ**が用意されている。イテレータは、コンテナ内のある位置を指し示し、そこから次の要素へ進むことができるオブジェクトである。

C 言語の配列では、ポインタを用いて配列の要素を順にたどることができる。これと似た考え方で、STL では、イテレータを用いてコンテナ内の要素に順にアクセスすることができる。イテレータはポインタと似た使い方ができるが、必ずしもポインタそのものではない。特に、vector 以外のコンテナの多くは、内部構造が配列とは異なるため、添字やポインタでは自然に扱えない。そのような場合でも、イテレータを用いることで、コンテナの内部構造に強く依存せずに要素に順にアクセスすることができる。

イテレータは、STL のコンテナに対する基本的なアクセス方法である。vector をはじめ、後述の list, map など、内部構造の異なるコンテナであっても、イテレータを用いることで、「先頭から順に要素をたどる」という処理を共通した形で記述できる。また、STL のアルゴリズムは、多くの場合、処理対象の範囲をイテレータで指定する。従って、イテレータを理解することは STL を利用する上で重要である。

対象のコンテナオブジェクトの先頭位置を表すイテレータは、メンバ関数 begin() によって取得し、末尾の次の位置を表すイテレータは、メンバ関数 end() によって取得する。ここで注意すべき点は、end() が末尾要素そのものを指すのではなく、最後の要素のさらに次の位置を表すことである。従って、end() が返すイテレータを介して要素にアクセスすることはできない。このように、begin() から end() の直前までを、コンテナの要素が存在する範囲として扱う。

次に示すプログラム test241-2.cpp は、vector に対するイテレータを用いて、要素に順番にアクセスする例である。

プログラム：test241-2.cpp

```
1  #include <iostream>
2  #include <vector>
3  using std::cout;
4
5  int main() {
6      std::vector<int>    sqr;
7      // 平方数の要素を作成
8      for(int i = 0; i < 10; i++) sqr.push_back(i * i);
9
10     bool first = true;           // 反復処理の初回フラグ
11     std::vector<int>::iterator it; // イテレータ
12     for( it = sqr.begin(); it != sqr.end(); ++it ) {
13         if ( !first ) {
14             cout << ", ";
15         } else {
16             first = false;
17         }
18         cout << *it;           // イテレータが示す要素を出力
19     }
20     cout << "\n";
21 }
```

std::vector<int> 型の変数 sqr に 10 個の平方数を格納し、イテレータを用いてそれらを順番に出力している。

std::vector<int>::iterator は、std::vector<int> オブジェクトの要素をたどるためのイテレータの型である。変数 it に sqr.begin() が返すイテレータを代入し、it != sqr.end() で終端に到達していないことを確認しながら反復している。++it を実行するとイテレータが次の要素へ進み、*it とすると、イテレータが指し示す要素にアクセスできる。

このプログラムをコンパイルして実行する例を次に示す。

例. test241-2.cpp のコンパイルと実行

```
$ g++ test241-2.cpp -o test241-2   ←コンパイル
$ ./test241-2   ←実行
0, 1, 4, 9, 16, 25, 36, 49, 64, 81  ←出力
```

イテレータの変数を宣言する方法は次の通りである。

書き方： コンテナ型::iterator 変数名;

例えば、std::vector<int> のイテレータは、次のように宣言する。


```
std::vector<int>::iterator it;
```

これは、コンテナ型に定義されている `iterator` 型の変数を宣言するものである。実際に先頭位置を表すイテレータを取得するには、対象のコンテナオブジェクトのメンバ関数 `begin()` を実行する。

イテレータが指し示す要素には、アスタリスク「*」を用いてアクセスする。また、イテレータ自体を次の位置へ進めるには、前置インクリメント演算子 `++it` を実行する。

参考) イテレータを次へ進める処理では、`it++` と書くこともできる。ただし、後置インクリメント演算子は変更前の値を返すため、概念上、変更前のイテレータのコピーを必要とする。変更前の値を使用しない場合は、前置インクリメント演算子 `++it` を用いる方が簡潔であり、C++ ではこちらが広く用いられる。本書でも、イテレータの移動には主に前置インクリメント演算子を用いる。

イテレータは、基本的には、コンテナ内の要素を順にたどるための仕組みである。そのため、まずは `begin()` で先頭位置を取得し、`end()` に到達するまで `++it` によって1つずつ進める、という用法が基本である。ただし、イテレータには、現在位置から少し進んだ位置や、少し戻った位置を得るための補助的な機能も用意されている。例えば、ヘッダファイル `iterator` をインクルードすると、表 46 に示すような `std::next()` や `std::prev()` を利用することができる。

表 46: イテレータの移動後の位置を求める関数テンプレート (一部)

関数	解説
<code>std::next(it)</code>	<code>it</code> の1つ次の位置を表すイテレータを返す
<code>std::next(it,n)</code>	<code>it</code> から <code>n</code> 個進んだ位置を表すイテレータを返す
<code>std::prev(it)</code>	<code>it</code> の1つ前の位置を表すイテレータを返す
<code>std::prev(it,n)</code>	<code>it</code> から <code>n</code> 個戻った位置を表すイテレータを返す

例えば、`std::next(it,3)` は、イテレータ `it` から3個先の位置を表すイテレータを返す。また、`std::prev(it)` は、`it` の1つ前の位置を表すイテレータを返す。これらの関数は、元のイテレータそのものを変更するのではなく、移動後の位置を表す別のイテレータを返す。`std::next()` は前方向へ進むことのできるイテレータに使用できる。一方、`std::prev()` は、前後両方向へ移動できるイテレータに対して使用できる¹⁷⁹。

注意) 表 46 の関数は、移動後の位置が有効な範囲内にあるかどうかを自動的にチェックするものではない。そのため、`begin()` より前や `end()` より後を表す位置を求めないように注意する必要がある。移動結果が `end()` となること自体は可能であるが、そのイテレータを介して要素にアクセスすることはできない。

特に `vector` のイテレータは、配列のポインタに近い性質を持っているため、`it+n` や `it-n` のような書き方によって、指定した個数だけ先、または前の位置を表すこともできる。このように、任意の位置へ効率よく移動できるイテレータを**ランダムアクセスイテレータ**と呼ぶ。`vector` や後述の `deque` のイテレータは、この性質を持っている。

ただし、すべてのコンテナのイテレータで `it+n` のような操作ができるわけではない。例えば、`list` のイテレータは前後に1つずつ進むことはできるが、`it+n` のようなランダムアクセスはできない。また、コンテナによっては、前方向にしか進めないイテレータもある。このように、イテレータで利用できる操作は、コンテナの種類によって異なる。

イテレータを用いる際には、そのコンテナのイテレータがどのような移動操作に対応しているかを意識する必要がある。ここでは `vector` を例にして基本的なイテレータの使い方を解説したが、他のコンテナについては、それぞれのコンテナを扱う箇所で、利用できるイテレータの性質を個別に解説する。

20.2.1 要素を書き換えないイテレータ：const_iterator

`vector` などのコンテナでは、通常のイテレータ `iterator` を用いると、イテレータが指し示す要素の値を読み取るだけでなく、その値を書き換えることもできる。例えば、`vector<int>` に対するイテレータ `it` がある要素を指している場合、次のようにしてその要素の値を変更することができる。

```
*it = 100;
```

これに対して、コンテナには、イテレータを介して要素を書き換えることのできない `const_iterator` という型も用意

¹⁷⁹従って、前方向にしか進めない `forward_list`、`unordered_map`、`unordered_set` などのイテレータには使用できない。

されている。const_iterator は、要素の値を読み取ることはできるが、その値を変更することはできないイテレータである。例えば、std::vector<int> に対する const_iterator は、次のように宣言する。

書き方： コンテナ型::const_iterator 変数名；

例. std::vector<int>::const_iterator it；

const_iterator でも、通常のイテレータと同様に、++it によって次の要素へ進んだり、*it によって要素の値を読み取ったりすることができる。ただし、次のように、イテレータを介して要素へ値を代入することはできない。

```
*it = 100;    ←エラー
```

一般に、iterator と const_iterator の重要な違いは、const_iterator ではイテレータを介して要素を書き換えることができない点である。要素を読み取るだけで変更する必要がない場合には、const_iterator を用いることで、誤ってコンテナの内容を書き換えることを防ぐことができる。

const_iterator を明示的に取得するには、メンバ関数 cbegin() と cend() を使用することができる。cbegin() は先頭位置を、kend() は末尾の次の位置を表す const_iterator を返す。例えば、次のように記述する。

```
std::vector<int>::const_iterator it = v.cbegin();
while (it != v.cend()) {
    std::cout << *it << '\n';
    ++it;
}
```

この場合、it 自体は ++it によって次の位置へ進めることができるが、*it を用いて要素の値を書き換えることはできない。

▲注意▲

const_iterator は、const 修飾されたイテレータとは明確に異なる。例えば、

```
const std::vector<int>::iterator it = v.begin();
```

とすると、it 自身が保持する位置を変更することはできないが、*it を介して要素を書き換えることはできる。これに対して const_iterator は、イテレータ自身を次の位置へ移動することはできるが、それを介してコンテナの要素を書き換えることはできない。

■ 通常のイテレータとの関係

const_iterator は、通常の iterator と完全に独立したものではない。通常の iterator は const_iterator に変換することができるため、例えば次のように、begin() が返す通常のイテレータを const_iterator 型の変数に代入することもできる。

```
std::vector<int>::const_iterator it = v.begin();
```

この場合、v.begin() が返す iterator が const_iterator に変換され、以後は it を介して要素を書き換えることはできない。

これに対して、const_iterator を通常の iterator に変換することはできない。例えば、次のような変換の記述はエラーとなる。

```
std::vector<int>::iterator it = v.cbegin();    ←エラー
```

このような変換が許されると、const_iterator によって読み取り専用としてアクセスしている要素を、通常の iterator を介して書き換えることが可能になってしまうためである。

また、対象となるコンテナオブジェクト自体が const として定義されている場合には、通常の begin() や end() を実行しても const_iterator が返される。例えば、

```
const std::vector<int> v{10, 20, 30};
auto it = v.begin();
```

とすると、it は std::vector<int>::const_iterator 型となり、*it を介して要素を書き換えることはできない。

従って、変更可能なコンテナに対して明示的に読み取り専用のイテレータを取得したい場合には cbegin(), cend() を使用し、コンテナ自体が const である場合には begin(), end() を使用しても自動的に const_iterator が得られる。

20.3 要素数が固定された配列：array

array は、複数の値を順番に並べて保持するためのコンテナである。C 言語の配列と同じように、あらかじめ要素数を決めて作成し、添字を用いて各要素にアクセスできる。一方、vector とは異なり、プログラムの実行中に要素数を変更することはできない。そのため、array は「要素数が固定された配列」と解釈することができる。

C 言語では、例えば次のように配列を宣言する。

```
int a[3] = {10, 20, 30};
```

これに対して、C++ の array を用いると、同じような固定長の配列を、標準ライブラリのコンテナとして扱うことができる。例えば、次のように記述する。

```
std::array<int, 3> a{10, 20, 30};
```

array はクラステンプレートであり、そこから生成される std::array<T, N> は、通常の配列に近い性質を持つ**集約型**¹⁸⁰である。使用する際には、要素の型と要素数を指定する。例えば、int 型の値を 3 個格納する場合には「std::array<int, 3>」、double 型の値を 10 個格納する場合には「std::array<double, 10>」のように記述する。array を使用するには、ヘッダファイル array をインクルードする。

書き方： std::array<型, 要素数> 変数名;

array は std 名前空間に属しているため、通常は std::array のように記述する。

array では、要素数が型の一部として指定される。従って、次の 2 つは異なる型である。

```
std::array<int, 3>
std::array<int, 5>
```

前者は int 型の要素を 3 個持ち、後者は int 型の要素を 5 個持つ。このように、array では、要素の型が同じであっても、要素数が異なれば別の型として扱われる。

array オブジェクトは、通常、波括弧を用いて初期化する。例えば、次のように記述すると、3 個の要素がそれぞれ 10, 20, 30 に初期化される。

```
std::array<int, 3> a{10, 20, 30};
```

要素数よりも少ない値を記述した場合、残りの要素はその型に応じた値で初期化される。例えば、次の例では、第 1 要素が 10、第 2 要素と第 3 要素が 0 に初期化される。

```
std::array<int, 3> a{10};
```

また、空の波括弧を用いると、すべての要素がその型に応じた既定の値で初期化される。

```
std::array<int, 3> a{};
```

この例では、3 個の要素がすべて 0 に初期化される。一方、次のように波括弧を付けずにローカルな array オブジェクトを定義した場合、基本型の各要素は初期化されない。

```
std::array<int, 3> a;
```

従って、基本型の要素を持つ array を生成する場合には、必要に応じて波括弧を用いて明示的に初期化するとよい。

array の各要素には、通常の配列や vector と同様に、添字を用いてアクセスできる。

```
a[0] = 10;
cout << a[1];
```

添字は 0 から始まり、要素数が 3 の場合には、使用できる添字は 0, 1, 2 である。また、size() を用いると要素数を取得でき、empty() を用いると要素数が 0 であるかどうかを調べることができる。array の要素数は型に含まれているため、array<T, N> では N が 0 の場合に限り empty() が true を返す。

array は、C 言語の配列と異なり、オブジェクト全体を代入によって複製できる。例えば、次のように記述できる。

```
std::array<int, 3> a{10, 20, 30};
std::array<int, 3> b{};
b = a;
```

この場合、a の 3 個の要素が b に代入される。ただし、要素の型と要素数がともに一致している必要がある。

¹⁸⁰ **集約型** (aggregate type) とは、配列や、一定の条件を満たすクラス型であり、波括弧内に要素やメンバの初期値を順に並べて、全体をまとめて初期化できる。例えば、C 言語の配列や単純な構造体などが集約型に該当する。

array は、要素数がコンパイル時に決まっており、実行中に変化しないデータを扱う場合に適している。例えば、3 次元座標、固定個数の測定値、月ごとのデータなど、要素数があらかじめ明確に決まっている場合には、vector よりも array の方が意図を明確に表現できる。一方、要素数を実行中に増減させる必要がある場合には、vector を使用する。

20.4 双方向リスト：list

先に解説した vector は、C 言語の配列に近い感覚で扱うことができ、添字によって各要素にアクセスできる便利なコンテナである。また、要素を末尾に追加する処理も簡単に記述できるため、実用的なプログラムで頻繁に使用される。しかし、データの扱い方によっては、vector とは異なる性質を持つコンテナが適している場合もある。例えば、大量の要素を保持している状態で、先頭や途中の位置に頻繁に要素を挿入したり、途中の要素の削除を頻繁に行う場合である。

vector は内部的には要素を連続したメモリ領域に並べて保持しているため、途中に要素を挿入すると、その位置より後ろにある要素を後方へ移動する必要がある。また、途中の要素を削除する場合にも、後続の要素を前方へ移動する処理が必要になる。そのような処理では、要素数が大きくなるほど、要素の移動に伴う処理の負担が問題になる。

vector とは異なるコンテナとして list がある。これは、連続したメモリ領域に要素を配置するのではなく、各要素を前後の要素と結び付ける形で管理するコンテナである。このような構造を、**双方向リスト**あるいは**双方向連結リスト**と呼ぶ。list では、各要素が前の要素と次の要素へのつながりを持っている。そのため、ある位置に要素を挿入する場合には、その前後のつながりを付け替えることで処理が実現できる。同様に、ある位置の要素を削除する場合にも、削除する要素の前後のつながりを付け替えることで処理が実現できる。従って、挿入や削除を行う位置がイテレータとして得られている場合には、vector のように後続要素をまとめて移動する必要がない。

このため、list は、全体の要素数が大きく、しかも途中への挿入や削除が頻繁に発生するような処理において有利になることがある。特に、処理の流れの中で現在位置の近くに要素を追加したり、現在位置の要素を削除したりするような場合には、vector よりも効率的に処理を実行できる場合がある。

ただし、list は、いかなる処理においても vector よりも有利であるということを意味しない。連続した要素に順番にアクセスする処理では、メモリ上に要素が連続して配置されている vector の方が効率が高いことも多い。また、挿入や削除を行う位置を先頭から探す必要がある場合には、その探索にも時間がかかる。従って、両者の使い分けについては慎重に検討する必要がある。

list を使用するにはヘッダファイル list をインクルードする。

書き方： std::list<型> 変数名;

list は std 名前空間に属しているため、通常は std::list のように記述する。

重要) list は添字による要素へのアクセスには対応していない。各要素にアクセスするには、基本的にイテレータを用いて先頭または末尾から順にたどる。

以下に示すプログラム test242-1.cpp を例に、list の基本的な使用方法について解説する。このプログラムでは、整数要素の list オブジェクト data を作成し、各種メンバ関数を実行する。

プログラム：test242-1.cpp

```
1  #include    <iostream>
2  #include    <list>
3  using std::cout;
4
5  void show( const std::list<int>& data ) {    // list の要素を全て出力する関数
6      std::list<int>::const_iterator it;      // list のイテレータ
7
8      if ( data.empty() ) {                  // list が空の場合の処理
9          cout << "(empty)\n";
10         return;
11     }
12     // list の全要素の出力
13     bool first = true;
14     for( it = data.begin(); it != data.end(); ++it ) {
15         if ( !first ) {
16             cout << ", ";
```

```

17         } else {
18             first = false;
19         }
20         cout << *it;
21     }
22     cout << "\n";
23 }
24
25 int main() {
26     std::list<int> data;
27
28     cout << "initial data:\t ";
29     show(data);
30
31     // 末尾に要素を追加
32     data.push_back(20);
33     data.push_back(30);
34     data.push_back(40);
35     cout << "after push_back: ";
36     show(data);
37
38     // 先頭に要素を追加
39     data.push_front(10);
40     cout << "after push_front: ";
41     show(data);
42
43     // 30 の位置を探す
44     std::list<int>::iterator it;
45     for(it = data.begin(); it != data.end(); ++it) {
46         if ( *it == 30 ) break;
47     }
48
49     // 30 の直前に 25 を挿入
50     if ( it != data.end() ) {
51         data.insert(it, 25);
52     }
53     cout << "after insert:\t ";
54     show(data);
55
56     // 30 の位置を探す
57     for(it = data.begin(); it != data.end(); ++it) {
58         if ( *it == 30 ) break;
59     }
60
61     // 30 を削除
62     if ( it != data.end() ) {
63         data.erase(it);
64     }
65     cout << "after erase:\t ";
66     show(data);
67 }

```

まず data に push_back メンバ関数で要素を 3 つ追加している。その後、push_front メンバ関数による先頭要素の追加、insert メンバ関数による要素の挿入、erase メンバ関数による要素の削除を実行している。プログラム冒頭で定義している show 関数は data の全要素を出力するもので、各種操作の後で data の状態を表示している。この関数では、引数を const 参照として受け取っているため、要素の走査には const_iterator を使用している。

このプログラムをコンパイルして実行する例を次に示す。

例. test242-1.cpp のコンパイルと実行

\$ g++ test242-1.cpp -o test242-1 ←コンパイル

\$./test242-1 ←実行

initial data:	(empty)	←初期状態
after push_back:	20, 30, 40	←末尾に要素を 3 つ追加した状態
after push_front:	10, 20, 30, 40	←先頭に要素を 1 つ追加した状態
after insert:	10, 20, 25, 30, 40	←途中に要素「25」を追加した状態
after erase:	10, 20, 25, 40	←要素「30」を削除した状態

20.5 両端キューのコンテナ：deque

前節までに、`vector` と `list` について説明した。`vector` は、添字による要素アクセスが高速であり、末尾への要素追加も効率よく行うことができる。そのため、配列に近い用法で多数の要素を管理したい場合に便利なコンテナである。一方、`list` は、途中の要素の挿入・削除に柔軟に対応できるコンテナである。ただし、`list` では添字を用いて要素へアクセスすることはできない。

実際のプログラミングでは、データを先頭側または末尾側で追加したり、取り出したりする処理がしばしば求められる。例えば、データを順に蓄積して後ろから取り出す処理や、新しいデータを末尾に追加しながら古いデータを先頭から取り除く処理などである。このような場合、途中の任意位置に要素を挿入・削除する能力よりも、先頭と末尾の両端に対する追加・削除を効率よく行えることが重要になる。

このような用途に適したコンテナが `deque` である。`deque` は、**double-ended queue** の略であり、日本語では一般に**両端キュー**と呼ばれる。これは、先頭側と末尾側の両方に対して、要素の追加や削除を効率よく行うことができるコンテナである。さらに、`deque` は `vector` と同様に添字を用いた要素アクセスも可能である。すなわち、`d[0]`、`d[1]`、`d[2]` のように、配列に近い用法で各要素を参照することができる。

`deque` は、`vector` のような添字アクセスの使いやすさを持ちながら、`vector` では不利な先頭側への追加・削除にも効率よく対応できるコンテナであると見ることができる。特に、先頭と末尾の両端を使ってデータを出し入れする処理では、`deque` が有用である。ただし、`deque` は途中の要素の挿入・削除においては有利なコンテナではない。途中の要素を頻繁に追加・削除する用途では、`list` などが適している場合がある。また、`deque` の要素は、`vector` のように連続したメモリ領域に配置されることが保証されていない。従って、添字によるアクセスは可能であるが、全要素を C 言語の配列のような連続領域として扱うことはできない。

`deque` を使用するにはヘッダファイル `deque` をインクルードする。

書き方： `std::deque<型>` 変数名;

`deque` は `std` 名前空間に属しているため、通常は `std::deque` のように記述する。

次に示すプログラム `test243-1.cpp` を例に、`deque` の用法の基礎を解説する。このプログラムでは、`deque` オブジェクト `d` を作成し、先頭と末尾に要素を追加する。その後、先頭と末尾の要素を徐々に削除する。

プログラム：`test243-1.cpp`

```
1  #include    <iostream>
2  #include    <deque>
3  using std::cout;
4
5  int main() {
6      std::deque<int> d;
7
8      d.push_back(0);    // 最初に 0 を登録
9
10     for ( int i = 1; i <= 3; i++ ) {
11         d.push_front(-i);    // 先頭側に負の値を追加
12         d.push_back(i);    // 末尾側に正の値を追加
13     }
14     // 添字指定で要素を参照
15     std::deque<int>::size_type i;
16     for ( i = 0; i < d.size(); i++ ) {
17         cout << "d[" << i << "]=" << d[i] << ", ";
18     }
19     cout << "\npop... ";
20
21     // 両端を徐々に削除
22     while ( d.size() > 1 ) {
23         d.pop_front();
24         if ( d.size() > 1 ) {
25             d.pop_back();
26         }
27     }
28     cout << "size:" << d.size() << ", front:" << d.front() << "\n";
29 }
```


このプログラムでは、最初に 0 を deque に登録し、その後 push_front() によって負の値を先頭側へ、push_back() によって正の値を末尾側へ追加する処理を繰り返している。deque オブジェクト d への要素登録が終了した段階で、添字指定で全要素を出力し、その後 pop_front() と pop_back() を用いて、残りの要素数が 1 個になるまで先頭側と末尾側の要素を順に削除している。

このプログラムをコンパイルして実行する例を次に示す。

例. test243-1.cpp のコンパイルと実行

```
$ g++ test243-1.cpp -o test243-1  ←コンパイル
$ ./test243-1  ←実行
d[0]=-3, d[1]=-2, d[2]=-1, d[3]=0, d[4]=1, d[5]=2, d[6]=3, ←要素の登録が終わった状態
pop... size:1, front:0 ←両端の要素の削除を繰り返して、要素が1個だけ残った
```

20.6 キーと値の対応を管理するコンテナ：map

これまでに説明した vector, list, deque は、複数の値を順に並べて保持するためのコンテナであった。これらのコンテナでは、各要素を添字やイテレータによって指定して扱う。一方、実用的なプログラムでは、「名前に対応する得点」、「商品コードに対応する価格」、「学籍番号に対応する氏名」のように、ある値を別の値に対応付けて管理したい場合がある。このような対応関係を扱うためのコンテナが map である。

map は、**キーと値の組（ペア）**を保持するコンテナである。キーは、値を取り出すための手がかりとして用いられる。例えば、学籍番号をキーとし、学生名を値として登録すれば、学籍番号を指定して対応する学生名を取り出すことができる。このように、map は、C 言語の配列のように「何番目の要素か」によって値を扱うのではなく、「どのキーに対応する値か」によってデータを扱う。そのため、map は**連想配列**、あるいは**辞書**のようなコンテナと考えることができる。

重要 map では、同じキーを持つ要素を複数登録することはできず、1つのキーには1つの値が対応する。

C 言語でも、構造体、配列、ポインタ、探索処理などを組み合わせれば、キーと値の対応を管理する仕組みを作ることができる。しかし、キーの探索、データの追加、削除、順序の管理などを安全かつ汎用的に実装しなければならない。C++ の map を用いると、このような対応関係を標準ライブラリのコンテナとして扱うことができる。

map では、登録されたペアは**キーの比較規則に従って自動的に整列された状態**で管理される。比較規則を特に指定しない場合は、キーの昇順に整列される。そのため、map をイテレータで先頭から順にたどると、キーの昇順に要素が現れる。この点は、要素を自動的に整列しない vector や list とは異なる。

map を使用するにはヘッダファイル map をインクルードする。

書き方： std::map<キーの型, 値の型> 変数名;

map は std 名前空間に属しているため、通常は std::map のように記述する。

map オブジェクトに登録された値には、次のようにキーを指定してアクセスできる。

書き方： map オブジェクト[キー]

この書き方は、値の代入にも参照にも使用できる。例えば、既に登録されているキーを指定すれば、そのキーに対応する値を取り出すことができる。また、代入の左辺に用いれば、キーと値の対応を新たに登録したり、既存の値を変更したりできる。ただし、未登録のキーを指定した場合には、値を参照するだけの処理であっても、そのキーに対応する新しいペアが作成され、値はその型に応じた値で初期化される点に注意すること。単純に「そのキーが存在するかどうかを調べる」、または「存在する場合だけ値を参照する」という目的では、find メンバ関数を用いる方が安全である。

書き方： map オブジェクト.find(キー)

find メンバ関数は、「キー」に対応する要素を指すイテレータを返す。「キー」が見つからない場合は、当該 map オブジェクトの end() のイテレータが返される。

■ map オブジェクトに登録されるペアのデータ構造

map に登録される各要素は、キーと値の組、すなわち**ペア**として扱われる。この組は、C++ 標準ライブラリで用意されている std::pair 型のオブジェクトとして表現される。pair は、2つの値を1つのオブジェクトとして保持

するためのクラステンプレートであり、first と second という 2 つのデータメンバ¹⁸¹を持つ。

map の場合、first がキーを表し、second が値を表す。例えば、std::map<std::string, std::string> 型の map では、1 つの要素の型は

```
std::pair<const std::string, std::string>
```

である。ここで、キーに対応する first は変更できない値として扱われる。これは、map がキーの順序に基づいて要素を管理しているためである。

従って、map のイテレータ it がある要素を指しているとき、it->first によってキーを、it->second によって値を参照できる。例えば、it->first が "apple"、it->second が "リンゴ" であれば、その要素は「"apple" というキーに "リンゴ" という値が対応している」ことを表す。

なお、std::pair は単独で使用することもできる¹⁸² が、本節では主に、map の要素を理解するための補足として扱う。

次に示すプログラム test244-1.cpp を例に、map の用法の基礎を解説する。このプログラムでは、map オブジェクト dict を作成し、英単語と日本語の対応（ペア）を登録する。

プログラム：test244-1.cpp

```
1  #include    <iostream>
2  #include    <map>
3  #include    <string>
4  using std::cout;
5
6  int main() {
7      std::map<std::string, std::string> dict;           // mapオブジェクトの作成
8      std::map<std::string, std::string>::iterator it;   // map用イテレータの作成
9
10     // キーと値のペアの登録
11     dict["apple"] = "リンゴ";
12     dict["orange"] = "ミカン";
13     dict["lemon"] = "レモン";
14
15     // キーから値を参照
16     cout << "apple : " << dict["apple"] << "\n";
17     cout << "orange : " << dict["orange"] << "\n";
18     cout << "lemon : " << dict["lemon"] << "\n";
19
20     // キーの存在検査
21     it = dict.find("apple");           // イテレータの形式で結果を取得
22     if ( it != dict.end() ) {         // キーが見つかった場合
23         cout << "\"apple\" is found: " << it->second << "\n";
24     } else {                          // キーが見つからなかった場合
25         cout << "\"apple\" is not found.\n";
26     }
27     it = dict.find("grape");           // 存在しないキーを探すケース
28     if ( it != dict.end() ) {
29         cout << "\"grape\" is found: " << it->second << "\n";
30     } else {
31         cout << "\"grape\" is not found.\n";
32     }
33
34     // 存在しないキーを指定して値を参照するケース
35     cout << "--- 未登録キーを参照する試み ---\n";
36     cout << "size before dict[\"banana\"]: " << dict.size() << "\n";   // 参照前
37     cout << "banana : " << dict["banana"] << "\n";
38     cout << "size after dict[\"banana\"]: " << dict.size() << "\n";   // 参照後
39     // dict["banana"] によって新規登録されたことを検索して確認
40     it = dict.find("banana");
41     if ( it != dict.end() ) {
42         cout << "\"banana\" is found: \"" << it->second << "\"\n";
43     }
44     // 既存のキーに対する値を上書き
45     dict["banana"] = "バナナ";
```

¹⁸¹ クラスや構造体に属する変数と理解するとよい。

¹⁸² その場合は、ヘッダファイル utility をインクルードする。

```

46     cout << "banana : " << dict["banana"] << "\n";
47
48     // 全てのペアを順番にアクセス
49     cout << "--- 全てのペアをイテレータで順番にアクセス ---\n";
50     for ( it = dict.begin(); it != dict.end(); ++it ) {
51         if ( it != dict.begin() ) cout << ", ";
52         cout << it->first << ":" << it->second;
53     }
54     cout << "\n";
55 }

```

このプログラムは、ペアの登録とキーによる参照、キーの存在検査の例を示している。また、`[]` を用いて存在しないキーを指定すると、そのキーが当該 `map` オブジェクトに登録され、値はその型に応じて初期化される。このプログラムでは値の型が `std::string` であるため、空文字列となる。

最後の部分では、イテレータを用いて `map` オブジェクトの全てのペアを出力しているが、登録された順ではなく、キーの比較規則に従った順序でペアが出力される。

このプログラムをコンパイルして実行する例を次に示す。

例. test244-1.cpp のコンパイルと実行

```

$ g++ test244-1.cpp -o test244-1  ←コンパイル
$ ./test244-1  ←実行
apple : リンゴ          ←登録された
orange : ミカン         ←キーに対する
lemon : レモン          ←値を参照
"apple" is found: リンゴ ←キーの存在検査（見つかる場合）
"grape" is not found.    ←キーの存在検査（見つからない場合）
--- 未登録キーを参照する試み ---
size before dict["banana"]: 3 ←参照前のペアの個数
banana :                  ←参照しただけで登録されている
size after dict["banana"]: 4 ←参照後のペアの個数
"banana" is found: ""     ←値は空文字列
banana : バナナ          ←改めて値を上書き登録した後
--- 全てのペアをイテレータで順番にアクセス ---
apple:リンゴ, banana:バナナ, lemon:レモン, orange:ミカン ←登録順ではなく、キーの順に並んでいる

```

20.6.1 順序を管理しない高速な連想コンテナ：unordered_map

前節で説明した `map` は、キーと値の対応を管理するための連想コンテナであった。`map` では、キーの大小関係に基づいて要素が管理されるため、イテレータを用いて全ての要素に順にアクセスすると、キーの順序に従って処理される。

一方、C++ には、キーの順序を管理しない連想コンテナとして `unordered_map` が用意されている。`unordered_map` も、キーと値の対応を管理するためのコンテナであり、基本的な使い方は `map` とよく似ている。例えば、キーを指定して値を登録する処理、キーを指定して値を取り出す処理、キーの存在を検索する処理などは `map` とほぼ同じ方法である。

`unordered_map` を使用するにはヘッダファイル `unordered_map` をインクルードする。

書き方： `std::unordered_map<キーの型, 値の型> 変数名;`

`unordered_map` は `std` 名前空間に属しているため、通常は `std::unordered_map` のように記述する。

`unordered_map` の特徴は、キーの順序を管理しない代わりに、キーから対応する値を高速に探索することを目的としている点である。内部的には、キーから計算されるハッシュ値を利用してデータを管理する。探索、追加、削除は平均的には一定時間で実行できるため、多数のデータから特定のキーに対応する値を取り出す処理では、`map` よりも高速に動作することが期待できる。ただし、ハッシュ値の偏りなどによって処理効率が低下する場合もあり、常に `map` より高速であることが保証されるわけではない。

`unordered_map` では、キーの順序や登録した順序に従って要素が並ぶわけではない。したがって、イテレータによって全ての要素を順に取り出す場合、その走査順が登録順やキーの昇順になることは保証されない。キーの順序に従って要素を扱いたい場合には `map` を使用し、キーによる高速な検索を重視する場合には `unordered_map` を使用する、

と使い分けるのが良い。

次に示すプログラム test245-1.cpp は、イテレータを用いて unordered_map の全てのペアにアクセスする例である。

プログラム：test245-1.cpp

```
1 #include <iostream>
2 #include <unordered_map>
3 #include <string>
4 using std::cout;
5
6 int main() {
7     std::unordered_map<std::string, std::string> dict;
8
9     // キーと値のペアの登録
10    dict["apple"] = "リンゴ";
11    dict["orange"] = "ミカン";
12    dict["lemon"] = "レモン";
13    dict["grape"] = "ブドウ";
14    dict["banana"] = "バナナ";
15
16    // 全てのペアをイテレータで順番にアクセス
17    cout << "--- 全てのペアをイテレータで順番にアクセス ---\n";
18    for (auto it = dict.begin(); it != dict.end(); ++it) {
19        if (it != dict.begin()) cout << ", ";
20        cout << it->first << ":" << it->second;
21    }
22    cout << "\n";
23 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test245-1.cpp のコンパイルと実行

```
$ g++ test245-1.cpp -o test245-1  ←コンパイル
$ ./test245-1  ←実行
--- 全てのペアをイテレータで順番にアクセス --- ←出力
banana:バナナ, lemon:レモン, orange:ミカン, grape:ブドウ, apple:リンゴ
```

この例は、先のプログラム test244-1.cpp とは異なる順序でペアにアクセスする様子を示している。ただし、これはペアがランダムな順に並んでいることを意味するものではない。unordered_map におけるイテレータの走査順は、ハッシュ値、内部の記憶領域の構成、実装上の管理方法などによって決まる。従って、登録順やキーの昇順に従うとは限らない。同じ処理系で同じプログラムを実行した場合には、同じ表示順になることが多いが、この順序は C++ の言語仕様として保証されるものではない。そのため、unordered_map を使用するプログラムでは、イテレータによる走査順に意味を持たせてはならない。

20.7 重複のない値を管理するコンテナ：set

set は、重複のない値を管理するためのコンテナである。vector、list、deque などのコンテナでは、同じ値を複数回格納することができる。これに対して、set では、同じ値を複数回登録しようとしても、その値は 1 つだけ保持される。例えば、整数値を格納する set に、値 10 を何度も登録しようとしても、set の中に保持される 10 は 1 つだけである。従って、set は、重複を避けて値を管理したい場合や、ある値が既に登録されているかどうかを調べたい場合に使用できる。

set では、要素が比較規則に従って自動的に整列された状態で管理される。比較規則を特に指定しない場合は、値の昇順に整列される。そのため、イテレータを用いて全ての要素に順にアクセスすると、値の昇順に要素が現れる。この点は、キーの順序に従って要素が管理される map とよく似ている。ただし、map がキーと値のペアを管理するコンテナであるのに対して、set は値そのものを重複なく管理するコンテナである。

set を使用するには、ヘッダファイル set をインクルードする。

書き方： std::set<要素の型> 変数名;

set は std 名前空間に属しているため、通常は std::set のように記述する。

set に値を登録するには、insert メンバ関数を使用する。

書き方： `set` オブジェクト.`insert( 値 )`

指定した値がまだ登録されていない場合は、新しい要素として登録される。既に同じ値が登録されている場合は、重複する要素は追加されず、`set` の内容は変化しない。

値が要素として登録されているかどうかは、`find` メンバ関数を用いて調べることができる。指定した値が見つかった場合はその要素を指すイテレータが返され、見つからない場合は `end()` のイテレータが返される。

重要) `set` に登録された要素は、イテレータを介して直接変更することはできない。値を変更したい場合は、元の要素を削除してから新しい値を登録する。

次に示すプログラム `test246-1.cpp` は、整数要素の `set` オブジェクト `s` に整数を複数回登録し、重複が起こらない様子を示すものである。

プログラム：`test246-1.cpp`

```
1  #include <iostream>
2  #include <set>
3  using std::cout;
4
5  int main() {
6      std::set<int> s;    // int型の値を管理するsetオブジェクト
7
8      // 0から9までの整数値を登録
9      for (int i = 0; i < 10; ++i) s.insert(i);
10
11     // 全ての要素をイテレータで順にアクセス
12     cout << "--- 0から9までを登録した後 ---\n";
13     cout << "size: " << s.size() << "\n";
14     for (auto it = s.begin(); it != s.end(); ++it) {
15         if (it != s.begin()) cout << ", ";
16         cout << *it;
17     }
18     cout << "\n";
19
20     // 0から14までの整数値を登録
21     for (int i = 0; i < 15; ++i) s.insert(i);
22
23     // 全ての要素をイテレータで順にアクセス
24     cout << "--- 0から14までを登録した後 ---\n";
25     cout << "size: " << s.size() << "\n";
26     for (auto it = s.begin(); it != s.end(); ++it) {
27         if (it != s.begin()) cout << ", ";
28         cout << *it;
29     }
30     cout << "\n";
31 }
```

このプログラムでは、`s` に 0～9 の整数を登録し、更に 0～14 の整数の登録処理を行っている。

このプログラムをコンパイルして実行する例を次に示す。

例. `test246-1.cpp` のコンパイルと実行

`$ g++ test246-1.cpp -o test246-1` ←コンパイル

`$ ./test246-1` ←実行

--- 0 から 9 までを登録した後 ---

size: 10

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

--- 0 から 14 までを登録した後 ---

size: 15

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 ←要素に重複はない

結果として、0～9 の要素は重複登録されず、`s` には 0～14 の値が 1 つずつ保持されていることがわかる。

20.7.1 順序を管理しない高速な集合コンテナ：`unordered_set`

STL には、値の順序を管理しない集合コンテナとして `unordered_set` が用意されている。`unordered_set` も、重複のない値を管理するためのコンテナであり、同じ値を複数回登録しようとしても、その値は 1 つだけ保持される。こ

の点は `set` と同じである。

`unordered_set` の特徴は、値の順序を管理しない代わりに、ある値が登録されているかどうかを高速に調べることが目的としている点である。内部的には、値から計算されるハッシュ値を利用してデータを管理する。探索、追加、削除は平均的には一定時間で実行できるため、多数のデータから特定の値が登録されているかどうかを調べる処理では、`set` よりも高速に動作することが期待できる。ただし、ハッシュ値の偏りなどによって処理効率が低下する場合もあり、常に `set` より高速であることが保証されるわけではない。

`unordered_set` を使用するには、ヘッダファイル `unordered_set` をインクルードする。

書き方： `std::unordered_set<要素の型> 変数名;`

`unordered_set` は `std` 名前空間に属しているため、通常は `std::unordered_set` のように記述する。値の登録には `insert` メンバ関数、値の検索には `find` メンバ関数を使用でき、基本的な用法は `set` とよく似ている。

`unordered_set` では、値の順序や登録した順序に従って要素が並ぶわけではない。従って、イテレータによって全ての要素を順に取り出す場合、その走査順が登録順や値の昇順になることは保証されない。値を順序付きで扱いたい場合には `set` を使用し、順序を必要とせず、値の存在確認を高速に行いたい場合には `unordered_set` を使用する、と使い分けるのが良い。

次に示すプログラム `test247-1.cpp` は、文字列 "programming" から1文字ずつ取り出して `unordered_set` オブジェクト `s` に登録した後、`s` の要素をイテレータを用いて順番に取り出して出力するものである。

プログラム： `test247-1.cpp`

```
1  #include    <iostream>
2  #include    <unordered_set>
3  #include    <string>
4  using std::cout;
5
6  int main() {
7      std::unordered_set<char> s;    // char型の値を管理するオブジェクト
8      std::string word = "programming";
9
10     // 文字列から1文字ずつ取り出して登録
11     for (char c : word) s.insert(c);
12
13     // 全ての要素をイテレータで順にアクセス
14     cout << "word: " << word << ", length:" << word.size() << "\n";
15     cout << "size of s: " << s.size() << "\n";
16     cout << "--- 全ての要素をイテレータで順番にアクセス ---\n";
17
18     for (auto it = s.begin(); it != s.end(); ++it) {
19         if (it != s.begin()) cout << ", ";
20         cout << *it;
21     }
22     cout << "\n";
23 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. `test247-1.cpp` のコンパイルと実行

`$ g++ test247-1.cpp -o test247-1` ←コンパイル

`$ ./test247-1` ←実行

`word: programming, length:11` ←用意された文字列

`size of s: 8` ← `unordered_set` オブジェクトのサイズ

`--- 全ての要素をイテレータで順番にアクセス ---`

`i, m, n, a, g, o, r, p` ←このような要素の順序（重複なし）

文字列 "programming" の長さは11であるが、`unordered_set` オブジェクト `s` の要素数は8である。これは、`r`, `m`, `g` などの重複する文字が1つだけ保持されるためである。この実行例では、イテレータで取り出した順序は、文字列中に現れた順序とも文字コードの昇順とも異なっている。ただし、この走査順は C++ の仕様として保証されるものではなく、処理系などによって異なる場合がある。

20.8 主なコンテナに対して使用できるメンバ関数

STL のコンテナには、種類の異なるコンテナに対して共通して使用できるメンバ関数が多く用意されている。例えば、先頭位置を得る `begin()`、末尾の次の位置を得る `end()`、要素数を調べる `size()`、コンテナが空であるかを調べる `empty()` などは、多くのコンテナで共通して使用できる基本的なメンバ関数である。

一方で、すべての関数がすべてのコンテナで使用できるわけではない。また、同じ名前の関数であっても、コンテナの種類によって意味や効率が異なる場合がある。例えば、`vector` や `deque` の `at()` は添字を指定して要素にアクセスする関数であるが、`map` の `at()` はキーを指定して対応する値にアクセスする関数である。このように、コンテナを使う際には、どの関数が利用できるかだけでなく、その関数がそのコンテナでどのような意味を持つかにも注意する必要がある。

表 47 に、コンテナ操作の関数として使用頻度の高いものを示す。

表 47: コンテナに対する主なメンバ関数（一部）

メンバ関数・操作	戻り値	説明
<code>begin()</code> <code>end()</code>	イテレータ イテレータ	先頭要素の位置を表すイテレータを返す 最後の要素の次の位置を表すイテレータを返す
<code>size()</code> <code>empty()</code>	要素数 bool	現在格納されている要素数を返す 要素数が 0 であるかどうかを返す
<code>at(x)</code> <code>front()</code> <code>back()</code>	要素または値への参照 要素への参照 要素への参照	添字 <code>x</code> の要素、またはキー <code>x</code> に対応する値に検査付きでアクセスする 先頭要素にアクセスする 末尾要素にアクセスする
<code>push_front(x)</code> <code>push_back(x)</code> <code>pop_front()</code> <code>pop_back()</code>	なし なし なし なし	先頭に要素 <code>x</code> を追加する 末尾に要素 <code>x</code> を追加する 先頭要素を削除する 末尾要素を削除する
<code>insert(pos,x)</code> <code>erase(pos)</code> <code>erase(first,last)</code>	イテレータ イテレータ イテレータ	位置 <code>pos</code> を手がかりとして要素 <code>x</code> を挿入する*1 イテレータ <code>pos</code> が示す位置の要素を削除する イテレータが示す範囲 <code>[first,last)</code> の要素を削除する*2
<code>clear()</code> <code>resize(n)</code> <code>resize(n,x)</code> <code>reserve(n)</code> <code>capacity()</code>	なし なし なし なし 容量	すべての要素を削除する 要素数を <code>n</code> 個に変更する 要素数を <code>n</code> 個に変更し、必要に応じて要素 <code>x</code> で埋める 少なくとも <code>n</code> 個の要素を格納できる内部容量を確保する 現在確保されている内部容量を返す

*1 `vector`, `list`, `deque` では `pos` の直前に挿入する。 `map` と `set` では `pos` はヒントであり、実際の位置は比較規則によって決まる。

*2 `first` から `last` の直前までの範囲

注意 1) `vector`, `array`, `deque` の `at()` に範囲外の添字を与えた場合、また、`map` や `unordered_map` の `at()` に未登録のキーを与えた場合には、`std::out_of_range` 例外が送出される。

注意 2) `front()`, `back()`, `pop_front()`, `pop_back()` は、コンテナが空でない場合にのみ使用できる。空である可能性がある場合は、先に `empty()` などを確認する必要がある。

`begin()` と `front()` は、どちらも先頭要素に関係する関数であるが、戻り値の意味が異なる。`begin()` は先頭要素の位置を表すイテレータを返すのに対して、`front()` は先頭要素そのもののへの参照を返す。すなわち、`begin()` は「位置」を返し、`front()` は「要素」を返すと考えよう。

また、`size()` や `capacity()` が返す値は、要素数や容量を表すための符号なし整数型である。厳密には、各コンテナが定義する `size_type` 型であるが、`std::size_t` と同種の整数型であると考えて差し支えない。

表 47 のメンバ関数の `vector`, `array`, `list`, `deque`, `map`, `set` に対する使用可否を表 48 に示す。

表 48 から分かるように、`begin()`, `end()`, `size()`, `empty()`, `clear()`, `insert()`, `erase()` などは、多くのコンテナで共通して使用できる。一方、`push_front()` や `pop_front()` は、先頭への追加や削除を効率よく扱えるコンテナで使用される関数であり、`vector` には用意されていない。また、表 48 で比較した 6 種類のコンテナの中では、`reserve()` と `capacity()` は `vector` だけが持つ。ただし、`unordered_map` と `unordered_set` にも `reserve()` があり、こちらはハッシュテーブルのバケット数や再ハッシュの管理に関係する。

`vector` や `list` などのシーケンスコンテナでは、コンテナ内の位置に従って要素が並ぶ。これに対して、`set` では要素、`map` ではキーが比較規則に従って自動的に整列された状態で保持される。従って、`begin()` から `end()` までイ

表 48: コンテナごとのメンバ関数の使用可否 (一部)

メンバ関数・操作	vector	array	list	deque	map	set
begin()	○	○	○	○	○	○
end()	○	○	○	○	○	○
size()	○	○	○	○	○	○
empty()	○	○	○	○	○	○
at()	○	○	×	○	○	×
front()	○	○	○	○	×	×
back()	○	○	○	○	×	×
push_front()	×	×	○	○	×	×
push_back()	○	×	○	○	×	×
pop_front()	×	×	○	○	×	×
pop_back()	○	×	○	○	×	×
insert()	○	×	○	○	○	○
erase()	○	×	○	○	○	○
clear()	○	×	○	○	○	○
resize()	○	×	○	○	×	×
reserve()	○	×	×	×	×	×
capacity()	○	×	×	×	×	×

イテレータでたどったときの順序が、要素の挿入順になることは保証されない。この点については、set や map を扱う箇所でも解説した。

表 48 では紙幅の都合により省略しているが、unordered_map と unordered_set についても、多くの基本操作はそれぞれ map, set とよく似ている。例えば、begin(), end(), size(), empty(), insert(), erase(), clear() などは共通して使用できる。また、unordered_map ではキーを用いた at() や find(), unordered_set では値を用いた find() などを使用できる。ただし、これらのコンテナでは要素の順序を管理しないため、イテレータで順にたどったときの順序は保証されず、map や set のように比較規則に従った順序になるとは限らない。

このように、STL のコンテナには共通した操作が多く用意されているが、利用できる関数や処理効率はコンテナの内部構造によって異なる。したがって、コンテナを選ぶ際には、「どのようなデータを保持するか」だけでなく、「どのような操作を頻繁に行うか」を考えることが重要である。

20.8.1 STL が提供するアルゴリズム

前項では、size(), insert(), erase() など、各コンテナがメンバ関数として持つ基本的な操作について整理した。一方、STL には、コンテナの外部から、イテレータで指定された範囲に対して処理を行うための汎用的な関数テンプレートも多数用意されている。これらを STL のアルゴリズムと呼ぶ。

STL のアルゴリズムは、特定のコンテナ専用の機能ではなく、主にイテレータで指定された範囲に対して処理を行う。例えば、範囲 [first, last) から特定の値を探す find、条件を満たす要素を探す find_if、要素数を数える count、要素列を整理する sort などがある。

ここで、[first, last) は、イテレータ first が示す位置から、イテレータ last が示す位置の直前までの範囲を表す。この表記は、STL のアルゴリズムを理解する上で重要である。多くの場合、コンテナ全体を対象とするには、begin() と end() を用いて次のように指定する。

`std::アルゴリズム名 ( コンテナ.begin(), コンテナ.end(), ... )`

表 49 に、STL が提供する主なアルゴリズムの一部を示す。

注意) copy(), transform(), 集合演算などの引数 out には、出力先を表すイテレータを与える。出力先には、あらかじめ十分な個数の要素を用意するか、std::back_inserter() や std::inserter() などの挿入イテレータを使用する¹⁸³ 必要がある。

表 49 に示した関数は、コンテナのメンバ関数ではなく、主にヘッダファイル algorithm で提供される関数である。従って、これらを使用するには、通常、ヘッダファイル algorithm をインクルードする必要がある。

¹⁸³ これらの挿入イテレータを使用するには、ヘッダファイル iterator をインクルードする。

表 49: STL の主なアルゴリズム (一部)

分類	主な関数	説明
探索	<code>find(first,last,x)</code>	範囲 <code>[first,last)</code> から値 <code>x</code> を探す
探索	<code>find_if(first,last,p)</code>	条件 <code>p</code> を満たす要素を探す
判定	<code>all_of(first,last,p)</code>	全要素が条件 <code>p</code> を満たすか調べる
判定	<code>any_of(first,last,p)</code>	条件 <code>p</code> を満たす要素があるか調べる
判定	<code>none_of(first,last,p)</code>	条件 <code>p</code> を満たす要素が存在しないか調べる
数え上げ	<code>count(first,last,x)</code>	値 <code>x</code> の個数を数える
数え上げ	<code>count_if(first,last,p)</code>	条件 <code>p</code> を満たす要素の個数を数える
コピー	<code>copy(first,last,out)</code>	範囲の要素を <code>out</code> へコピーする
変換	<code>transform(first,last,out,f)</code>	各要素に関数 <code>f</code> を適用して出力する
置換	<code>replace(first,last,a,b)</code>	値 <code>a</code> を値 <code>b</code> に置き換える
並べ替え	<code>sort(first,last)</code>	範囲の要素を整列する
反転	<code>reverse(first,last)</code>	範囲の要素の順序を反転する
最大・最小	<code>min_element(first,last)</code>	最小要素の位置を求める
最大・最小	<code>max_element(first,last)</code>	最大要素の位置を求める
集合演算	<code>set_union(f1,l1,f2,l2,out)</code>	2つの整列済み範囲の和集合を出力する
集合演算	<code>set_intersection(f1,l1,f2,l2,out)</code>	2つの整列済み範囲の積集合を出力する
集合演算	<code>set_difference(f1,l1,f2,l2,out)</code>	2つの整列済み範囲の差集合を出力する

注意) `count` や `find` などは、コンテナそのものに同名のメンバ関数が用意されている場合もある。その場合は、メンバ関数と STL の汎用アルゴリズムのどちらを使うかを区別する必要がある。

例えば、`vector` に対して値を探す場合には、汎用アルゴリズムの `std::find()` を用いて先頭から順に探索する。一方、`map` や `set` では、キーや要素を探すためのメンバ関数 `find()` が用意されている。特に、`map` の各要素はキーと値のペアであるため、汎用の `std::find()` はペア全体を探索の対象とし、キーだけを指定して探索することはできない。また、汎用アルゴリズムは先頭から順に探索するのに対して、メンバ関数はコンテナの内部構造を利用して効率よく探索できる。従って、`map` や `set` では、通常はメンバ関数 `find()` を使用する。

表 50 に、主なアルゴリズムとコンテナとの関係を示す。

表 50: 主なアルゴリズムとコンテナとの関係

関数	<code>vector</code>	<code>array</code>	<code>list</code>	<code>deque</code>	<code>map</code>	<code>set</code>
<code>find</code>	○	○	○	○	△	△
<code>find_if</code>	○	○	○	○	○	○
<code>all_of</code>	○	○	○	○	○	○
<code>any_of</code>	○	○	○	○	○	○
<code>none_of</code>	○	○	○	○	○	○
<code>count</code>	○	○	○	○	△	△
<code>count_if</code>	○	○	○	○	○	○
<code>copy</code>	○	○	○	○	○	○
<code>transform</code>	○	○	○	○	○	○
<code>replace</code>	○	○	○	○	×	×
<code>sort</code>	○	○	×	○	×	×
<code>reverse</code>	○	○	○	○	×	×
<code>min_element</code>	○	○	○	○	△	△
<code>max_element</code>	○	○	○	○	△	△
<code>set_union</code>	△	△	△	△	△	○
<code>set_intersection</code>	△	△	△	△	△	○
<code>set_difference</code>	△	△	△	△	△	○

表 50 の ○ は比較的そのまま使用しやすいことを、△ は使用できる場合があるが注意が必要であることを、× は通常そのコンテナには適さないことを表す。ただし、この表は厳密な仕様の完全な一覧ではなく、入門段階での使い分けの目安である。

特に、`map` や `set` では、キーや要素を探索するためのメンバ関数 `find()` や `count()` が用意されている。これらはコンテナの内部構造を利用するため、汎用の `std::find()` や `std::count()` で先頭から順に調べるよりも効率がよい。ただし、`map` に対する汎用アルゴリズムでは、キーではなくペア全体が比較の対象となる。なお、C++20 以降では、存在の有無を直接調べる `contains()` メンバ関数も使用できる。

同様に、`map` や `set` は比較規則に従って既に整列されているため、空でなければ、比較規則上の先頭要素は `begin()`、末尾要素は `std::prev(end())` によって得られる。汎用の `std::min_element()` や `std::max_element()` は範囲を先頭から探索するため、通常はこれらを使用する必要はない。ただし、`map` の値の部分などを別の比較規則で調べたい場合には利用できる。

汎用の `std::sort()` 関数はランダムアクセスイテレータを必要とするため、`vector`、`array`、`deque` には使用できるが、`list` には使用できない。`list` には、専用のメンバ関数 `sort()` が用意されている。

集合演算のアルゴリズムは、同じ比較規則に従って整列された 2 つの範囲を対象とする。そのため、常に整列状態を保つ `set` とは相性がよい。一方、`unordered_set` は要素を整列順に管理するコンテナではない。従って、`set_union`、`set_intersection`、`set_difference` などの STL の集合演算アルゴリズムを用いる場合には、必要に応じて `unordered_set` の内容を `set` などの整列されたコンテナに変換してから処理する。既定とは異なる比較規則で整列されている場合には、集合演算にも同じ比較規則を与える必要がある。

注意) 集合演算のアルゴリズムを `vector`、`array`、`list`、`deque` に対して使用する場合には、あらかじめ要素列が整列されている必要がある。

`set` の要素や `map` のキーは、コンテナ内での順序を決めるために使用される。そのため、これらを汎用アルゴリズムで直接置き換えるような処理は通常行わない。`set` の要素を変更したい場合には、いったん削除してから新しい値を挿入する。一方、`map` ではキーを直接変更することはできないが、キーに対応する値の部分は変更できる。

ここでは、各種コンテナのメンバ関数や、STL の汎用アルゴリズムについて紹介した。詳しい仕様については、C++ の標準化に関する公開資料や、標準ライブラリのリファレンスなどを参照すること。資料の主な提供元を巻末の参考文献 (p.588) に挙げる。

20.8.2 コンテナ間の変換処理

STL では、目的に応じて様々なコンテナを使い分ける。例えば、値を末尾に追加した順序で保持したい場合には `vector` を使用し、重複を避けて値を管理したい場合には `set` を使用する。また、値の存在確認を高速に行いたい場合には `unordered_set` を使用し、値を比較規則に従った順序で扱いたい場合には `set` を使用する。このように、コンテナにはそれぞれ異なる特徴があるため、処理の途中で、あるコンテナの内容を別のコンテナに変換したい場合がある。

例えば、`vector` に格納された値から重複を取り除きたい場合には、その内容を `set` に登録する方法が考えられる。また、`unordered_set` に登録された値に対して、整列済み範囲を前提とする STL の集合演算アルゴリズムを使用したい場合には、その内容を `set` に変換してから処理する方法がある。このような処理は、コンテナの特徴を組み合わせるために重要である。

STL のコンテナの多くは、コンストラクタにイテレータの範囲を与えることで、別のコンテナの内容を用いて初期化することができる。コンテナ `c1` の全体を用いて、新しいコンテナ `c2` を作成する基本的な書き方は次のようになる。

書き方： コンテナ型 `c2( c1.begin(), c1.end() )`;

この書き方では、`c1.begin()` から `c1.end()` の直前までの要素が順に取り出され、新しく作成されるコンテナ `c2` に登録される。ただし、変換後の要素の並び方や要素数は、変換先のコンテナの性質によって決まる。例えば、`vector` から `set` に変換すると、重複する値は取り除かれ、要素は比較規則に従って整列される。また、`unordered_set` に変換した場合、イテレータによる走査順は保証されない。

重要) この方法では、変換元の要素を用いて変換先のコンテナが新たに作成されるが、変換元のコンテナ `c1` の内容は変更されない。

変換元から取り出される要素を用いて変換先の要素型を構築できる場合には、この方法によってコンテナ間の変換処理を実現できる。`map` の要素はキーと値のペアであるが、変換先がそのペアを格納できるコンテナであれば、範囲

コンストラクタを利用できる。一方、キーまたは値の一方だけを取り出す場合や、異なる構造の要素に変換する場合には、イテレータによる反復処理で各要素を取り出し、必要な部分を変換しながら変換先のコンテナに登録する。

注意) `array` は要素数が型の一部として決まる集約型であり、この形式の範囲コンストラクタは持たない。ただし、要素がコピー可能な同じ型の `array` であれば、オブジェクト全体を複製できる。例えば、次のように記述する。

```
std::array<int, 3> a1{10, 20, 30};
std::array<int, 3> a2 = a1;
```

この場合、`a1` の全要素が `a2` に複製される。このような複製では、要素の型と要素数がともに一致し、さらに要素がコピー可能である必要がある。

■ サンプルプログラム

次に示すプログラム `test248-1.cpp` は、`unordered_set` オブジェクトを `set` オブジェクトに変換する例である。

プログラム：test248-1.cpp

```
1  #include <iostream>
2  #include <unordered_set>
3  #include <set>
4  #include <string>
5  using std::cout;
6
7  int main() {
8      std::unordered_set<std::string> us;    // 順序を管理しない集合コンテナ
9
10     // 文字列要素の登録
11     us.insert("apple");                    us.insert("banana");
12     us.insert("grape");                    us.insert("lemon");
13     us.insert("orange");                   us.insert("persimmon");
14
15     // unordered_set の全要素をイテレータで順にアクセス
16     cout << "--- unordered_set の要素 ---\n";
17     for (auto it = us.begin(); it != us.end(); ++it) {
18         if (it != us.begin()) cout << ", ";
19         cout << *it;
20     }
21     cout << "\n";
22
23     // unordered_set から set を作成
24     std::set<std::string> s(us.begin(), us.end());
25
26     // set の全要素をイテレータで順にアクセス
27     cout << "--- set に変換した後の要素 ---\n";
28     for (auto it = s.begin(); it != s.end(); ++it) {
29         if (it != s.begin()) cout << ", ";
30         cout << *it;
31     }
32     cout << "\n";
33 }
```

このプログラムでは、`us.begin()` と `us.end()` を範囲コンストラクタに与えて、`set` オブジェクト `s` を作成している。これにより、`us` の全要素が `s` に複製され、`set` の比較規則に従って整列された状態で保持される。変換元の `us` の内容は変更されない。このプログラムをコンパイルして実行する例を次に示す。

例. `test248-1.cpp` のコンパイルと実行

```
$ g++ test248-1.cpp -o test248-1  ←コンパイル
```

```
$ ./test248-1  ←実行
```

```
--- unordered_set の要素 --- ←出力
```

```
persimmon, orange, lemon, banana, grape, apple ←この実行例での unordered_set の走査順
```

```
--- set に変換した後の要素 ---
```

```
apple, banana, grape, lemon, orange, persimmon ←set としての要素の順序
```

この実行例では、`unordered_set` の要素が `persimmon, orange, …` の順に出力されているが、この走査順は C++ の仕様として保証されておらず、処理系などによって異なる場合がある。一方、`set` オブジェクト `s` の要素は、既定の文字列の比較規則に従った順序で出力される。

上の test248-1.cpp は、範囲コンストラクタによって直接変換できるケースであるが、次に示す test248-2.cpp は、map オブジェクトのキーと値を別々の vector として取り出す例である。

プログラム：test248-2.cpp

```
1  #include    <iostream>
2  #include    <map>
3  #include    <vector>
4  #include    <string>
5  using std::cout;
6
7  int main() {
8      std::map<std::string, std::string>  d;  // 英和辞書
9      std::vector<std::string>  k;          // 英単語を格納する vector
10     std::vector<std::string>  v;          // 日本語訳を格納する vector
11
12     // キーと値のペアの登録
13     d["banana"] = "バナナ";               d["lemon"]   = "レモン";
14     d["orange"] = "ミカン";               d["grape"]   = "ブドウ";
15     d["apple"]  = "リンゴ";               d["persimmon"] = "柿";
16
17     // map の全ペアをイテレータで順にアクセス
18     cout << "---- map オブジェクト d の内容 ----\n";
19     for (auto it = d.begin(); it != d.end(); ++it) {
20         if (it != d.begin()) cout << ", ";
21         cout << it->first << ":" << it->second;
22     }
23     cout << "\n";
24
25     // map からキーと値を別々の vector に取り出す (変換処理)
26     for (auto it = d.begin(); it != d.end(); ++it) {
27         k.push_back(it->first);           // 英単語
28         v.push_back(it->second);          // 日本語訳
29     }
30
31     // vector k の内容を表示
32     cout << "---- vector オブジェクト k の内容 ----\n";
33     for (auto it = k.begin(); it != k.end(); ++it) {
34         if (it != k.begin()) cout << ", ";
35         cout << *it;
36     }
37     cout << "\n";
38
39     // vector v の内容を表示
40     cout << "---- vector オブジェクト v の内容 ----\n";
41     for (auto it = v.begin(); it != v.end(); ++it) {
42         if (it != v.begin()) cout << ", ";
43         cout << *it;
44     }
45     cout << "\n";
46 }
```

変換処理では、map のイテレータを用いて全要素をたどり、it->first で得られるキーを k に、it->second で得られる値を v に、それぞれ push.back() メンバ関数で追加している。map の要素はキーの比較規則に従う順序で取り出されるため、k にはキーがその順序で格納され、v には各キーに対応する値が同じ順序で格納される。

このプログラムをコンパイルして実行する例を次に示す。

例. test248-2.cpp のコンパイルと実行

```
$ g++ test248-2.cpp -o test248-2  Enter    ←コンパイル
$ ./test248-2  Enter    ←実行
---- map オブジェクト d の内容 ----
apple:リンゴ, banana:バナナ, grape:ブドウ, lemon:レモン, orange:ミカン, persimmon:柿
---- vector オブジェクト k の内容 ----
apple, banana, grape, lemon, orange, persimmon
---- vector オブジェクト v の内容 ----
リンゴ, バナナ, ブドウ, レモン, ミカン, 柿
```

この結果から、k と v の同じ位置にある要素が、元のキーと値の対応を表していることがわかる。例えば、k[0] の

"apple" には、`v[0]` の "リンゴ" が対応する。ただし、変換後に一方の `vector` だけを並べ替えたり、要素を追加・削除したりすると、この対応関係が崩れるため注意が必要である。

まとめ：

変換元と変換先の要素型がそのまま対応する場合には、`begin()` と `end()` による範囲指定で簡潔に変換できる。一方、要素の一部だけを取り出したい場合や、別の構造に変換したい場合には、イテレータで全要素をたどりながら、必要な値を取り出し、適切な形で変換先に登録する。

21 string による文字列処理

「11.6.4 C++の標準文字列クラス」(p.197～)では、string オブジェクトの基本的な扱いについて説明した。C++ の標準ライブラリには、文字列の検索、部分文字列の取り出し、置換などを行う string のメンバ関数が用意されている。また、文字列ストリームや正規表現など、文字列処理に利用できる機能も用意されている。本章では、これらの機能を用いた、より実用的な文字列処理について説明する。

string は、C 言語の文字配列に比べて豊富な機能を持つが、あらゆる文字列処理を直接行えるわけではない。特に、日本語などのマルチバイト文字を扱う場合、string の要素数は char 型の要素の個数を表し、添字も char 型の要素単位、すなわちバイト単位で指定される。そのため、文字数や文字単位的位置とは一致しないことがある。

本章では、string を実用的に使うための基本的な考え方と、よく使用される処理の書き方を中心に説明する。

21.1 文字と文字列の検索

string オブジェクトに格納された文字列に対して、特定の文字や部分文字列を探す処理は頻繁に必要となる。例えば、文字列の中に特定の単語が含まれているかを調べたり、区切り文字の位置を探したり、末尾側から特定の文字を探したりする場合がある。

string には、このような検索処理を行うためのメンバ関数が用意されている。基本的には、検索対象として「文字列」を指定する場合と、「文字」を指定する場合がある。例えば、文字列 "program" を探す場合と、文字 'g' を探す場合では、次のように引数の指定の仕方が異なる。

```
find("program")    ←文字列を探す場合
find('g')          ←文字を探す場合
```

また、検索の方向として、先頭側から探す方法と、末尾側から探す方法が用意されている。さらに、複数の候補文字のいずれかを探す関数や、逆に、指定した候補文字に含まれない文字を探す関数も用意されている。表 51 に、文字と文字列の検索に関係する主な関数を示す。

表 51: 文字と文字列の検索のための関数 (一部)

関数	探索方向	探す対象
find(t)	前から	文字列 t を探す
find(c)	前から	文字 c を探す
rfind(t)	後ろから	文字列 t を探す
rfind(c)	後ろから	文字 c を探す
find_first_of(cs)	前から	文字集合 cs のいずれかを探す
find_last_of(cs)	後ろから	文字集合 cs のいずれかを探す
find_first_not_of(cs)	前から	文字集合 cs に含まれない文字を探す
find_last_not_of(cs)	後ろから	文字集合 cs に含まれない文字を探す

表 51 において、t は検索対象となる文字列、c は検索対象となる文字、cs は検索対象となる文字の集合を表す文字列である。例えば、find("abc") は文字列 "abc" を探し、find('a') は文字 'a' を探す。また、find_first_of(",.;") は、コンマ、ピリオド、セミコロンのいずれかが最初に現れる位置を探す。

なお、ここで「文字」c として指定できるのは、1 個の char 型の値である。UTF-8 で表現された日本語の文字など、複数バイトからなる文字を検索する場合には、文字列として指定する必要がある。

これらの検索関数は、見つかった位置 (バイト単位的位置) を string::size_type 型の値として返す。文字列を検索した場合には、見つかった部分文字列の先頭位置が返される。検索結果を受け取る変数の型には、string::size_type または auto を用いるとよい。見つからなかった場合には、string::npos が返される。string::npos は、見つからないことを表す特別な値であり、string::size_type 型で表現できる最大値として定義されている。従って、検索結果を使用する場合には、戻り値が string::npos であるかどうかを確認する必要がある。

また、多くの検索関数では、第 2 引数に位置を指定することで、検索を開始する位置を指定できる。例えば、find(t,p) は、位置 p 以降で文字列 t を検索する。一方、rfind(t,p) は、文字列 t の先頭位置が p 以下となる出現箇所のうち、最も後ろにあるものを検索する。

文字列の検索と文字の検索の例を次のプログラム test250-1.cpp で示す。

プログラム：test250-1.cpp

```
1  #include    <iostream>
2  #include    <string>
3  using std::cout;
4
5  int main() {
6      std::string s1 = "programming in C language / C++ language";
7      cout << "0-----1-----2-----3-----4\n" << s1 << "\n";
8
9      // "language" を前方から検索
10     auto p1 = s1.find("language");
11     if (p1 != std::string::npos) {
12         cout << "\"language\" found from front at: " << p1 << "\n";
13     } else {
14         cout << "\"language\" not found from front.\n";
15     }
16
17     // "language" を後方から検索
18     auto p2 = s1.rfind("language");
19     if (p2 != std::string::npos) {
20         cout << "\"language\" found from back at: " << p2 << "\n";
21     } else {
22         cout << "\"language\" not found from back.\n";
23     }
24
25     std::string s2 = "Business,Numbers,Money,People";
26     cout << "0-----1-----2-----3-----4\n" << s2 << "\n";
27
28     // コンマを先頭から順に検索
29     std::string::size_type pos = 0;
30     while (true) {
31         pos = s2.find(',', pos);
32         if (pos == std::string::npos) {
33             break;
34         }
35         cout << "comma found at: " << pos << "\n";
36         ++pos; // 次回は、見つかった位置の次から検索する
37     }
38 }
```

この例では、find と rfind によって、文字列 "language" を検索している。rfind は末尾側から検索を行うが、返される位置は文字列の先頭から数えた位置である。そのため、後方から見つかった "language" の位置も、先頭を 0 とした位置として表示される。

また、後半では、find(',', pos) によって、位置 pos 以降に現れるコンマを検索している。変数 pos は、find() の戻り値に合わせて string::size_type 型としている。コンマが見つからなかった場合は、戻り値が string::npos であることを確認してループを終了する。コンマが見つかった場合だけ pos を 1 つ進めることで、次の検索は見つかったコンマの次の位置から始まる。このように、第 2 引数を更新しながら検索を繰り返すことで、同じ文字を順に探することができる。

このプログラムをコンパイルして実行する例を次に示す。

例. test250-1.cpp のコンパイルと実行

```
$ g++ test250-1.cpp -o test250-1  ←コンパイル
$ ./test250-1  ←実行
0-----1-----2-----3-----4
programming in C language / C++ language
"language" found from front at: 17
"language" found from back at: 32
0-----1-----2-----3-----4
Business,Numbers,Money,People
comma found at: 8
comma found at: 16
comma found at: 22
```


21.2 文字列の分割

文字列の分割とは、1つの文字列を、空白やコンマなどの区切りに基づいて、複数の部分文字列に分ける処理である。例えば、コンマで区切られた文字列 "apple,banana,orange" を分割すると、"apple", "banana", "orange" という複数の文字列が得られる。分割によって得られる部分文字列は複数個になるため、通常は `vector<string>` などのコンテナに格納して扱う。

C++ の `string` には、文字列を分割してコンテナとして返す特別なメンバ関数は用意されていない。そのため、文字列を分割する場合には、目的に応じて文字列ストリームや検索関数を組み合わせて処理を記述する。空白で区切られた文字列を分割する場合には、`istringstream`¹⁸⁴ と入力演算子 `>>` を用いる方法が簡単である。また、コンマのような1文字の区切り文字で分割する場合には、`istringstream` と `getline` を組み合わせる方法が利用できる。

本節では、プログラムの例に沿って文字列を分割する方法について説明する。

次に示すプログラム `test250-2.cpp` は、コンマで区切られた文字列を分割し、分割結果を `vector<string>` オブジェクトに格納する例である。このプログラムでは、文字列から `istringstream` オブジェクトを作成し、`std::getline()` の第3引数に区切り文字 `,` を指定して、コンマまでの部分文字列を順に読み取る。読み取った部分文字列は `push_back()` によって `vector<string>` オブジェクト `words` に追加される。

プログラム：test250-2.cpp

```
1  #include    <iostream>
2  #include    <sstream>
3  #include    <string>
4  #include    <vector>
5  using std::cout;
6
7  int main() {
8      std::string s = "apple,banana,orange,lemon";
9      std::istringstream iss(s);
10     std::vector<std::string> words;
11     std::string word;
12
13     cout << s << "\n";
14
15     // コンマが現れるまでを1つの文字列として読み取る
16     while (std::getline(iss, word, ',')) {
17         words.push_back(word);
18     }
19
20     // 分割結果を表示
21     cout << "--- 分割結果 ---\n";
22     for (auto it = words.begin(); it != words.end(); ++it) {
23         if (it != words.begin()) cout << "/";
24         cout << *it;
25     }
26     cout << "\n";
27 }
```

注) 区切り文字が連続する場合、その間の空文字列も読み取られる。ただし、元の文字列が区切り文字で終わる場合、末尾の区切り文字より後ろの空文字列は、この方法では読み取られない。

このプログラムをコンパイルして実行する例を次に示す。

例. `test250-2.cpp` のコンパイルと実行

```
$ g++ test250-2.cpp -o test250-2  Enter    ←コンパイル
$ ./test250-2  Enter    ←実行
apple,banana,orange,lemon    ←元の文字列
--- 分割結果 ---
apple/banana/orange/lemon    ←分割された文字列を '/' で区切って出力
```

¹⁸⁴`istringstream` を使用するには、ヘッダファイル `sstream` をインクルードする。

21.3 文字の種類の判定と変換

C++ の標準ライブラリには、文字の種類を判定する関数や、英字の大文字／小文字を変換する関数が用意されている。これらの関数は、string オブジェクトの各要素を1つずつ取り出して処理する場合によく使用される。表 52 に、文字の種類判定と変換に用いる主な関数を示す。

表 52: 文字の種類判定と変換に用いる主な関数

関数	戻り値	説明
isspace(c)	整数値	c が空白文字であるかどうかを判定する
isdigit(c)	整数値	c が数字であるかどうかを判定する
isalpha(c)	整数値	c が英字であるかどうかを判定する
isalnum(c)	整数値	c が英数字であるかどうかを判定する
islower(c)	整数値	c が英小文字であるかどうかを判定する
isupper(c)	整数値	c が英大文字であるかどうかを判定する
ispunct(c)	整数値	c が句読点や記号であるかどうかを判定する
tolower(c)	文字コード	c が英大文字であれば小文字に変換する
toupper(c)	文字コード	c が英小文字であれば大文字に変換する

表 52 において、c は判定や変換の対象となる文字を表す。string オブジェクトの要素は char 型であるが、これらの関数は文字コードを表す整数値として引数を受け取る。これらの関数を使用するには、ヘッダファイル ctype をインクルードする。これらの関数は通常、std 名前空間を指定して使用する。

isspace, isdigit, isalpha, isalnum などの判定関数は、bool 型ではなく int 型の値を返す。条件を満たさない場合は 0 を返し、条件を満たす場合は 0 以外の値を返す。従って、これらの関数は if 文の条件式として使用できる。一方、tolower と toupper は、変換後の文字コード (int 型) を返す。対象の文字が変換できない文字である場合には、元の文字に対応する値が返される。

重要) これらの関数に char 型の値を与える場合には、unsigned char 型に変換してから与えるのが安全である。

処理系によっては char 型が負の値を取ることがあり、その値を直接与えると動作が未定義となる。例えば、

```
std::isalpha(static_cast<unsigned char>(c))
```

のように記述する。tolower() などの戻り値を文字として使用する場合には、さらに char 型に変換する。

```
char d = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
```

isspace(c) が判定する空白文字には、通常の空白 ' ' だけでなく、水平タブ '\t', 改行 '\n', 垂直タブ '\v', 改ページ '\f', 復帰 '\r' も含まれる。

なお、これらの関数による判定や変換は、現在設定されているロケールの影響を受ける。通常の初期状態では、主に英数字や ASCII 文字を対象とする処理に用いられる。日本語などのマルチバイト文字を1文字ずつ判定・変換する目的には、そのまま使用できない。

表 52 の関数のうち、文字の種別を判定する関数の働きを確認するプログラム例を test250-3.cpp に、大文字／小文字の変換を確認するプログラム例を test250-4.cpp に示す。

プログラム：test250-3.cpp

```
1 #include <iostream>
2 #include <string>
3 #include <ctype>
4 using std::cout;
5
6 int main() {
7     std::string s;
8     std::string::size_type i;
9
10    // 空白文字の検査
11    s = " \t\n\v\f\rA"; // 末尾のみ非空白
12    cout << "=== space check ===\n";
13    cout << "string: \" \\t\\n\\v\\f\\rA\"\\n";
14    for ( i = 0; i < s.size(); i++ ) {
```

```

15         unsigned char c = static_cast<unsigned char>(s[i]);
16         if ( i != 0 ) cout << ", ";
17         cout << i << ":" << std::isspace(c);
18     }
19     cout << "\n";
20
21     s = "012abcABC.,;:'\""; // 検査対象から成る文字列
22     cout << "=== character type cross check ===\n";
23     cout << "[char]\tisdigit\tisalpha\tisalnum\tislower\tisupper\tispunct\n";
24     for ( i = 0; i < s.size(); i++ ) {
25         unsigned char c = static_cast<unsigned char>(s[i]);
26
27         if ( i%3 == 0 && i < 10 ) {
28             cout << "-----\n";
29         }
30         cout << " " << s[i] << "\t";
31         cout << " " << std::isdigit(c) << "\t";
32         cout << " " << std::isalpha(c) << "\t";
33         cout << " " << std::isalnum(c) << "\t";
34         cout << " " << std::islower(c) << "\t";
35         cout << " " << std::isupper(c) << "\t";
36         cout << " " << std::ispunct(c) << "\t";
37         cout << "\n";
38     }
39 }

```

このプログラムの前半では、文字列 " \t\n\v\f\rA" の要素を1つずつ取り出して、isspace関数を実行し、

インデックス：戻り値

の形式で出力する。後半では、文字列 "012abcABC.,;:'\" の各文字と、各種関数の戻り値のクロス表を表示する。

このプログラムをコンパイルして実行する例を次に示す。

例. test250-3.cpp のコンパイルと実行

\$ g++ test250-3.cpp -o test250-3 ←コンパイル

\$./test250-3 ←実行

=== space check === ←出力

string: " \t\n\v\f\rA"

0:8, 1:8, 2:8, 3:8, 4:8, 5:8, 6:0

=== character type cross check ===

[char]	isdigit	isalpha	isalnum	islower	isupper	ispunct

0	1	0	4	0	0	0
1	1	0	4	0	0	0
2	1	0	4	0	0	0

a	0	2	2	2	0	0
b	0	2	2	2	0	0
c	0	2	2	2	0	0

A	0	1	1	0	1	0
B	0	1	1	0	1	0
C	0	1	1	0	1	0

.	0	0	0	0	0	16
,	0	0	0	0	0	16
;	0	0	0	0	0	16
:	0	0	0	0	0	16
'	0	0	0	0	0	16
"	0	0	0	0	0	16

注意) 各関数が返す 0 以外の具体的な値は処理系によって異なる。

この実行結果から、文字判定関数は、条件を満たす場合に 0 以外の整数値を返すことがわかる。ただし、条件を満たす場合の戻り値は常に 1 であるとは限らず、その具体的な値も処理系によって異なる。従って、これらの関数の戻り値は、具体的な数値そのものではなく、0 であるか 0 以外であるかに注目して扱うのが基本である。

プログラム：test250-4.cpp

```

1 #include <iostream>
2 #include <string>

```

```

3  #include    <cctype>
4  using std::cout;
5
6  int main() {
7      std::string      s = "012abcDEFghiJKL,.;:'$#";
8      std::string::size_type i;
9
10     cout << "string\t\" << s << "\"\ntolower\t\"";
11     // 小文字に変換
12     for ( i = 0; i < s.size(); i++ ) {
13         unsigned char c = static_cast<unsigned char>(s[i]);
14         cout << static_cast<char>(std::tolower(c));
15     }
16     cout << "\"\ntoupper\t\"";
17     // 大文字に変換
18     for ( i = 0; i < s.size(); i++ ) {
19         unsigned char c = static_cast<unsigned char>(s[i]);
20         cout << static_cast<char>(std::toupper(c));
21     }
22     cout << "\"\n";
23 }

```

このプログラムは、文字列 "012abcDEFghiJKL,.;:'\$#" の各要素に対して `tolower()`、`toupper()` を実行する例を示す。各要素は `unsigned char` 型に変換してから関数に与え、戻り値は `char` 型に変換して出力している。このプログラムでは変換結果を直接出力しているため、元の文字列 `s` の内容は変更されない。

このプログラムをコンパイルして実行する例を次に示す。

例. test250-4.cpp のコンパイルと実行

```

$ g++ test250-4.cpp -o test250-4  Enter    ←コンパイル
$ ./test250-4  Enter    ←実行
string  "012abcDEFghiJKL,.;:'$#"    ←元の文字列
tolower "012abcdefgijkl,.;:'$#"    ←すべてのアルファベットを小文字に変換した文字列
toupper "012ABCDEFGHIJKL,.;:'$#"    ←すべてのアルファベットを大文字に変換した文字列

```

この例から、`tolower()` は英大文字を英小文字に変換し、`toupper()` は英小文字を英大文字に変換することがわかる。数字や記号など、対応する大文字または小文字がない文字については、元の文字に対応する値がそのまま返される。

21.4 日本語文字列と文字コードに関する注意

コンピュータ上で文字列を扱う場合、文字は何らかの文字コードによって数値として表現される。英数字や基本的な記号は、古くから ASCII の体系で表現されてきた。ASCII では、英大文字、英小文字、数字、空白、基本的な記号などが 0~127 の値で表現される¹⁸⁵。従って、ASCII の文字だけから成る文字列では、1 文字が 1 個の `char` 要素に対応すると考えても、大きな問題は起こりにくい。

一方、日本語の文字は、ひらがな、カタカナ、漢字、記号などを含むため、ASCII の範囲だけでは表現できない。日本語で扱う文字集合は、ASCII の英数字だけを扱う場合に比べて非常に大きい¹⁸⁶。さらに、多国語対応を考えると、中国語、韓国語、欧州各国語、各種記号、絵文字なども含めて扱う必要があり、Unicode のような非常に大きな文字集合が必要となる。

具体的に、文字を数値やバイト列によって表現する仕組みを、一般にエンコーディングと呼ぶ。日本語の文字を表現するためのエンコーディングとして、Shift_JIS や EUC-JP などが用いられてきた。ただし、現在では、日本語を含む多国語の文字を Unicode によって統一的に扱う方針が一般的になっている。Unicode では、文字を識別するための数値として符号位置 (code point) が定められている¹⁸⁷。また、Unicode の符号位置をバイト列として表現する符号化方式として、現在では UTF-8 が広く用いられている。

¹⁸⁵ASCII は 7 ビットの文字コードであり、通常は各文字が 1 バイトで保持される。「半角英数字」と呼ばれる文字は概ね ASCII 文字のことを指しているが、半角カタカナは ASCII 文字でないことに注意すること。

¹⁸⁶JIS X 0213 の第 1 水準から第 4 水準までを含めると、非漢字を含む定義文字数は 1 万 1 千字を超える。

¹⁸⁷結合文字などでは、見た目の 1 文字が複数の符号位置の組み合わせによって表現される場合もある。

Shift_JIS, EUC-JP, UTF-8 では、ASCII の範囲に相当する英数字などは 1 バイトで表現される。一方、日本語文字など、ASCII の範囲に含まれない文字は、複数のバイトを用いて表現される場合がある。特に UTF-8 では、ASCII の範囲を超える 1 つの符号位置は 2~4 バイトで表現される。そのため、ASCII の範囲に含まれない文字を扱う場合には、文字列の見た目の文字数と、内部で保持される char 要素の個数が一致しないことがある。

C++ の string は、文字列を扱うための標準クラスであるが、日本語の「1 文字」を直接単位として管理するクラスではない。string は、基本的には char の列を保持するクラスである。従って、マルチバイト文字を含む文字列に対して、size(), 添字アクセス, substr() などを用いる場合には注意が必要である。これらを不用意に使用すると、文字数を誤って数えたり、文字の途中のバイトだけを取り出したりして、文字化けを起こしたり、エンコーディングとして不正なバイト列を作ったりする原因にもなる。

また、C++ の string が保持する文字列のエンコーディングは、C++ の仕様として特定の方式に固定されているわけではない。通常の文字列リテラル "... " がどの文字コードのバイト列としてプログラムに組み込まれるかは、ソースファイルのエンコーディング、コンパイラの設定などに依存する。さらに、その文字列が実行時にどのように解釈・表示されるかは、実行環境にも依存する。本節では、現在よく用いられる UTF-8 を例として、日本語文字列を string で扱う際の問題点のいくつかを解説する。

■ 文字列の見た目の文字数とバイトサイズの違い

次に示すプログラム test250-5.cpp では、文字列 "abc 日本語 DEF" を扱っている。この文字列は、見た目には a, b, c, 日, 本, 語, D, E, F の 9 文字である。ここでは、通常の文字列リテラルが UTF-8 のバイト列としてプログラムに組み込まれ、実行環境でも UTF-8 として正しく表示されることを前提としている。

UTF-8 では、ASCII 文字の部分が合計 6 バイト、日本語の 3 文字が合計 9 バイトで表現されるため、size() の戻り値は 15 となる¹⁸⁸。これは、string の size() が見た目の文字数ではなく、保持している char 要素の個数を返すためである。

プログラム：test250-5.cpp

```
1  #include    <iostream>
2  #include    <string>
3  using std::cout;
4
5  int main() {
6      std::string s = "abc日本語DEF";
7
8      cout << "string: s = \"" << s << "\"\n";
9      cout << "見た目の文字数: 9\n";
10     cout << "s.size(): " << s.size() << "\n";
11 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test250-5.cpp のコンパイルと実行

```
$ g++ test250-5.cpp -o test250-5  ←コンパイル
$ ./test250-5  ←実行
string: s = "abc 日本語 DEF"      ←出力
見た目の文字数: 9                ←あらかじめ見た目で数えた値
s.size(): 15                     ← UTF-8 で表現した場合のバイトサイズ
```

■ 部分文字列を取り出す際の問題

string の substr() は、指定した位置から指定した個数の char 要素を取り出す関数である。従って、日本語文字列に対して substr() を用いる場合、引数で指定する位置や長さは、見た目の文字数ではなく、内部で保持されている char 要素の位置と個数として解釈される。

次に示すプログラム test250-6.cpp は、文字列 "これは日本語の文字列です。" に対して substr() を実行する例である。ここでも、通常の文字列リテラルが UTF-8 のバイト列としてプログラムに組み込まれ、実行環境でも UTF-8 として解釈されることを前提としている。UTF-8 では、この文字列に含まれる各日本語文字が 3 バイトで表現され

¹⁸⁸同じ文字列を Shift_JIS のバイト列で表現した場合は、ASCII 文字の部分が 6 バイト、日本語の 3 文字が合計 6 バイトとなるため、全体では 12 バイトとなる。

る。切り出しの開始位置と終了位置がともに文字の境界に合っていれば、正しい UTF-8 の部分文字列が得られる。一方、いずれかの位置が文字を構成するバイト列の途中にある場合には、UTF-8 として不正なバイト列が作られ、正しく表示できないことがある。

プログラム：test250-6.cpp

```
1 #include <iostream>
2 #include <string>
3 using std::cout;
4
5 int main() {
6     std::string s = "これは日本語の文字列です。";
7
8     cout << "string: s = \" " << s << "\"\n";
9     cout << "s.size(): " << s.size() << "\n";
10
11     // 文字の境界に合う位置で部分文字列を取り出す例
12     cout << "--- 文字の境界に合う切り出し ---\n";
13     cout << "s.substr(0, 6): " << s.substr(0, 6) << "\n";
14     cout << "s.substr(6, 9): " << s.substr(6, 9) << "\n";
15
16     // 文字を構成するバイト列の途中で切り出す例
17     cout << "--- 文字の途中で切れる切り出し ---\n";
18     cout << "s.substr(0, 1): " << s.substr(0, 1) << "\n";
19     cout << "s.substr(1, 6): " << s.substr(1, 6) << "\n";
20     cout << "s.substr(0, 7): " << s.substr(0, 7) << "\n";
21 }
```

このプログラムをコンパイルして実行する例を次に示す。

例. test250-6.cpp のコンパイルと実行

\$ g++ test250-6.cpp -o test250-6 ←コンパイル

\$./test250-6 ←実行

string: s = "これは日本語の文字列です。" ←出力

s.size(): 39 ←バイトサイズ

--- 文字の境界に合う切り出し ---

s.substr(0, 6): これ

s.substr(6, 9): は日本

--- 文字の途中で切れる切り出し ---

s.substr(0, 1): <E3> ←文字の境界が不適切なので

s.substr(1, 6): <81><93>れ <E3> ←文字化けを

s.substr(0, 7): これ <E3> ←起こしている

注) 灰色で示した部分は、UTF-8 として不正なバイトを表している。不正なバイト列が実際にどのように表示されるかは実行環境によって異なり、置換文字（◆のような文字）が表示されたり、バイト値に相当する表記が表示されたりする場合がある。

この結果から、substr() の引数は、日本語の文字数ではなく char 要素の位置と個数として解釈されることがわかる。切り出しの開始位置と終了位置が文字の境界に合っていれば正しい部分文字列が得られるが、文字を構成するバイト列の途中で切り出すと、UTF-8 として不正なバイト列が作られる。その表示結果は実行環境によって異なるが、文字化けや置換文字の表示などが起こる可能性がある。従って、日本語などのマルチバイト文字を含む文字列を 1 文字単位で処理する場合には、string の添字アクセスや substr() を不用意に用いないよう注意する必要がある。

21.5 正規表現による文字列処理

正規表現は、文字列のパターンを記述するための記法である。正規表現を用いると、特定の形式を持つ文字列を判定したり、文字列中から条件に合う部分を探したり、条件に合う部分を別の文字列に置き換えたりすることができる。

C++ の標準ライブラリには、正規表現を扱うための機能（表 53）が用意されている。これらを使用するには、ヘッダファイル regex をインクルードする。これらのクラスや関数は std 名前空間に属している。本節では、正規表現そのものの詳しい記法には立ち入らず、C++ における基本的な使用例のみを示す。

表 53 において、s は検索や置換の対象となる文字列、ptn は正規表現のパターンを表す文字列、r は regex オブジェクト、m は一致結果を受け取る smatch オブジェクト、t は置換に用いる文字列を表す。また、it は現在の一致

表 53: 正規表現による文字列処理に用いる主な機能

関数・クラス	説明
<code>regex r(ptn)</code> <code>smatch m</code> <code>sregex_iterator</code> <code>sregex_iterator it(s.cbegin(),s.cend(),r)</code> <code>sregex_iterator end</code>	正規表現パターン <code>ptn</code> を保持するオブジェクト <code>r</code> を作成する 一致結果を受け取るためのオブジェクト <code>m</code> を作成する 正規表現に一致する箇所を順にたどるためのイテレータ型 正規表現 <code>r</code> に最初に一致する箇所を指すイテレータ <code>it</code> を作成する 一致箇所をたどる際の終了位置を表すイテレータ <code>end</code> を作成する
<code>regex_match(s,r)</code> <code>regex_match(s,m,r)</code>	文字列 <code>s</code> 全体が正規表現 <code>r</code> に一致するかを調べる 文字列 <code>s</code> 全体が正規表現 <code>r</code> に一致するかを調べ、結果を <code>m</code> に格納する
<code>regex_search(s,r)</code> <code>regex_search(s,m,r)</code>	文字列 <code>s</code> の中に正規表現 <code>r</code> に一致する部分があるかを調べる 文字列 <code>s</code> の中に正規表現 <code>r</code> に一致する部分があるかを調べ、最初の一致結果を <code>m</code> に格納する
<code>regex_replace(s,r,t)</code>	文字列 <code>s</code> の中で正規表現 <code>r</code> に一致するすべての部分を文字列 <code>t</code> に置き換えた新しい文字列を返す
<code>m.str()</code> <code>m.position()</code> <code>m.length()</code>	一致した文字列を返す 一致した部分の先頭位置を <code>char</code> 要素単位で返す 一致した部分の長さを <code>char</code> 要素単位で返す

箇所を指すイテレータ、`end` は走査の終了位置を表すイテレータである。一致箇所が存在しない場合、作成された `it` は `end` と等しくなる。

`regex` オブジェクトの作成時に正規表現の文法を特に指定しない場合は、ECMAScript に基づく文法が用いられる。正規表現の細かな記法は、他のプログラミング言語や各種ツールで用いられるものと異なる場合がある。

`string` を対象とする正規表現処理も、基本的には `char` 要素単位で行われる。UTF-8 の日本語文字は複数の `char` 要素から構成されるため、`m.position()` や `m.length()` が返す値も `char` 要素単位となり、ここで想定する UTF-8 環境ではバイト単位の値となる。また、`.` などのパターンも、日本語の 1 文字を常に 1 単位として扱えるわけではない。

`regex_match()` は、文字列全体がパターンに一致するかを調べる。一方、`regex_search()` は、文字列の一部にパターンに一致する箇所があるかを調べる。これらの関数は、一致する場合に `true`、一致しない場合に `false` を返す。ただし、`regex_search()` は一致箇所をすべて列挙する関数ではなく、一致結果を `smatch` オブジェクトに格納する形式では、最初の一致結果が格納される。一致箇所を順に取り出したい場合には、`sregex_iterator` を用いる。

次に示すプログラム `test250-7.cpp` は、正規表現関連の関数を用いて、数字の並びを検索する処理と、数字の並びを `"***"` に置換する処理の例である。

プログラム：test250-7.cpp

```

1  #include    <iostream>
2  #include    <string>
3  #include    <regex>
4  using std::cout;
5
6  int main() {
7      std::string s = "ID: A123, score: 95";    // 対象文字列
8      std::regex r("[0-9]+");                  // 数字の並びを意味する表現
9
10     cout << "string:\" " << s << "\"\n";
11     if (std::regex_search(s, r)) {            // 文字列中の数字の並びを検索
12         cout << "number found\n";
13     }
14
15     cout << "replacing number with \"***\"\n";
16     std::string t = std::regex_replace(s, r, "***");
17     cout << "result:\" " << t << "\"\n";
18 }
```

この例では、正規表現 `[0-9]+` によって、文字列中に現れる数字の並びを検索している。`[0-9]` は 0~9 のいずれかの数字を表し、その後ろの `+` は直前のパターンが 1 回以上続くことを表す。従って、`A123` のような部分では、英字 `A` は一致せず、数字部分 `123` が一致対象となる。

`regex_search()` は、数字の並びが少なくとも 1 つ存在するかどうかを調べている。一方、`regex_replace()` は、正規表現に一致するすべての部分を `"***"` に置き換えた新しい文字列を返す。この例では、`123` と `95` の両方が置き換えられ、結果が文字列 `t` に格納される。元の文字列 `s` の内容は変更されない。

このプログラムをコンパイルして実行する例を次に示す。

例. `test250-7.cpp` のコンパイルと実行

```
$ g++ test250-7.cpp -o test250-7  ←コンパイル
$ ./test250-7  ←実行
string:"ID: A123, score: 95"      ←出力
number found
replacing number with "***"
result:"ID: A***, score: ***"
```

21.5.1 正規表現の基本的な要素

使用頻度の高い正規表現の要素を表 54~55 に示す。

表 54: 正規表現の主なパターン (一部)	
パターン	意 味
<code>[c1c2c3...]</code>	文字集合 <code>c1c2c3...</code> のいずれか 1 文字
<code>[^c1c2c3...]</code>	文字集合 <code>c1c2c3...</code> に含まれない 1 文字
<code>[c1-c2]</code>	文字 <code>c1</code> から <code>c2</code> までの範囲に含まれる 1 文字
<code>.</code>	改行文字を除く任意の 1 文字
<code>\d</code>	ASCII の数字 0~9 のいずれか 1 文字
<code>\D</code>	ASCII の数字 0~9 以外の 1 文字
<code>\s</code>	空白文字 1 文字
<code>\S</code>	空白文字以外の 1 文字
<code>\w</code>	ASCII の英字, 数字, アンダースコアのいずれか 1 文字
<code>\W</code>	上記のいずれにも該当しない 1 文字

表 54 の `\d`, `\s`, `\w` などによる文字の分類は、正規表現に設定されたロケール¹⁸⁹の影響を受ける。この表では、通常の初期状態である `"C"` ロケールを前提としている。

表 55 の繰り返し記号は、直前の文字、文字集合、または丸括弧で囲まれたパターンなどに対して適用される。例えば、`[0-9]+` は、ASCII の数字が 1 回以上繰り返されるパターンを表す。

表 55: 正規表現における繰り返しの表記 (一部)			
表記	意 味	表記	意 味
<code>+</code>	1 回以上の繰り返し	<code>*</code>	0 回以上の繰り返し
<code>?</code>	0 回または 1 回	<code>{n}</code>	ちょうど <code>n</code> 回
<code>{m,n}</code>	<code>m</code> 回以上 <code>n</code> 回以下	<code>{m,}</code>	<code>m</code> 回以上

注意) 表 54 に示したものは、正規表現としての表記である。これらを C++ の通常文字列リテラルに記述する場合は、バックスラッシュを `\\` と記述する必要がある。例えば、数字の並びを表す正規表現 `\d+` は、

```
std::regex r("\\d+");
のように記述する。
```

¹⁸⁹ロケールとは、言語や地域に応じた文字の分類、数値や日時の表記などに関する設定である。`"C"` ロケールは、C/C++プログラムの基本的な初期設定であり、英字 `A~Z`, `a~z`, 数字 `0~9`などを基準として文字を分類する。

21.5.2 正規表現に一致する箇所の全検索

文字列中に正規表現に一致する部分が存在するかどうかを調べるために `regex_search` が使用できる。また、`smatch` オブジェクトを組み合わせることで、最初に見つかった一致部分の文字列や位置を取得することもできる。しかし、`regex_search` は、文字列中に現れる一致箇所を全て自動的に列挙する関数ではない。

文字列全体を調べ、正規表現に一致する互いに重ならない箇所を先頭から順にすべて取り出したい¹⁹⁰ 場合には、`sregex_iterator` を使用する。`sregex_iterator` は、`string` オブジェクトに対する正規表現検索の結果を順にたどるためのイテレータである。基本的には、検索対象となる文字列の先頭位置と末尾の次の位置、および正規表現オブジェクトを指定して作成する。

書き方： `std::sregex_iterator it( s.cbegin(), s.cend(), r );`

ここで、`s` は検索対象となる `string` オブジェクト、`r` は検索に用いる `regex` オブジェクトである。この書き方により、`s` の全体から `r` に一致する箇所を順に取り出すためのイテレータ `it` が作成される。一致箇所が存在しない場合、`it` は終了位置を表すイテレータ `end` と等しくなる。

一方、検索結果の終端を表すためには、引数を指定せずに `sregex_iterator` オブジェクトを作成する。

書き方： `std::sregex_iterator end;`

このように、引数を指定せずに作成した `end` は、通常のコンテナにおける `end()` イテレータと同じように、検索結果の終端を表すものとして使用できる。従って、正規表現に一致する箇所を順に処理するには、次のような反復処理を記述できる。

```
for ( ; it != end; ++it ) {  
    } ...
```

反復処理の中で `*it` を参照すると、その時点での一致結果を表す `smatch` オブジェクトとして扱うことができる。このオブジェクトの `str()` を実行すると一致した文字列を取得でき、`position()` を実行すると一致部分の開始位置を取得できる。これらは、`it->str()`、`it->position()` のように記述することもできる。なお、`position()` が返す位置は `char` 要素単位であり、UTF-8 の文字列ではバイト単位の位置となる。

注意) `sregex_iterator` による走査中は、検索対象の文字列 `s` や正規表現オブジェクト `r` の内容を変更してはならない。また、走査が終了するまで、これらのオブジェクトが存在している必要がある。

次に示すプログラム `test250-8.cpp` は、テキストファイル `textCpp.txt` の内容を読み込み、空白を除く ASCII の表示可能文字の並びを正規表現で全検索する例である。

プログラム： `test250-8.cpp`

```
1  #include    <iostream>  
2  #include    <fstream>  
3  #include    <sstream>  
4  #include    <string>  
5  #include    <regex>  
6  using std::cout;  
7  
8  int main() {  
9      std::ifstream ifs("textCpp.txt");  
10     if ( !ifs ) {                                // ファイルがオープンできなかった際の処理  
11         cout << "file open error.\n";  
12         return 1;  
13     }  
14  
15     // ファイル全体を1つの string オブジェクトに読み込む  
16     std::ostringstream oss;  
17     oss << ifs.rdbuf();  
18     std::string text = oss.str();  
19  
20     // 空白を除く ASCII の表示可能文字の並びを表す正規表現  
21     std::regex r("[!~]+");  
22  
23     // 正規表現に一致する箇所を順に取り出す  
24     cout << "---- ASCII 文字列の並び ----\n";
```

¹⁹⁰—一致部分どうしが重なる場合、重なった一致箇所が自動的に列挙されるわけではない。

```

25     std::sregex_iterator it(text.cbegin(), text.cend(), r);
26     std::sregex_iterator end;
27
28     int      c = 0;                // 検出回数
29     for (; it != end; ++it) {
30         if ( c != 0 ) {
31             cout << ",\t";
32             if ( c%5 == 0 ) cout << "\n";
33         }
34         c++;
35         std::smatch m = *it;
36         cout << m.position() << ":\\" << m.str() << "\\";
37     }
38     cout << "\n";
39 }

```

入力ファイル：textCpp.txt （Wikipedia「C++」解説記事から抜粋）

C++（シーplusplus）は、汎用プログラミング言語のひとつである。派生元である C 言語の機能や特徴を継承しつつ、表現力と効率性の向上のために、手続き型プログラミング・データ抽象・オブジェクト指向プログラミング・ジェネリックプログラミングといった複数のプログラミングパラダイムが組み合わされている。C 言語のようにハードウェアを直接扱うような下位層向けの低水準言語としても、複雑なアプリケーションソフトウェアを開発するための上位層向け高水準言語としても使用可能である。アセンブリ言語以外の低水準言語を必要としないこと、使わない機能に時間的・空間的コストを必要としないことが、言語設計の重要な原則となっている。

C++は、1983 年に AT&T ベル研究所の計算機科学者ビャーネ・ストロヴストルップによって公開された。また様々なプラットフォームでその開発環境が導入された。1998 年から ISO と IEC の共同で言語仕様とテンプレートライブラリの標準化が行われるようになり、その後 2003 年、2011 年、2014 年、2017 年、2020 年、2023 年に標準規格が改訂されている。2025 年時点での最新規格は「ISO/IEC 14882:2024」通称「C++23」。

このプログラムでは、ファイル textCpp.txt の内容を 1 つの string オブジェクト text に読み込み、正規表現「[!~]+」に一致する部分を検索している。この正規表現は、空白を除く ASCII の表示可能文字が 1 文字以上連続する部分を表す。従って、C++, AT&T, ISO/IEC, 14882:2024 などの ASCII 文字列をまとめて検出できる。

検出には sregex_iterator を用いており、正規表現に一致する箇所を先頭から順に取り出している。各一致結果について、position() によって検出位置を取得し、str() によって検出された文字列を取得する。これにより、「検出位置:検出文字列」の形式で、ファイル中に現れる ASCII 文字列の並びを表示している。

このプログラムをコンパイルして実行する例を次に示す。

例. test250-8.cpp のコンパイルと実行

\$ g++ test250-8.cpp -o test250-8 ←コンパイル

\$./test250-8 ←実行

--- ASCII 文字列の並び --- ←出力

```

0:"C++",      114:"C",      436:"C",      897:"C++",    906:"1983",
916:"AT&T",    1109:"1998",    1122:"ISO",    1128:"IEC",    1245:"2003",
1255:"2011",   1265:"2014",    1275:"2017",   1285:"2020",   1295:"2023",
1344:"2025",   1381:"ISO/IEC", 1389:"14882:2024", 1411:"C++23"

```

注) 検出位置は char 要素単位であり、ファイルの改行コードや、テキストモードにおける改行の扱いによって異なる場合がある。

22 数値データの処理

C/C++は、システムプログラミングや組込み開発だけでなく、科学技術計算の分野でも広く利用されてきたプログラミング言語である。特に C++では、標準ライブラリの整備が進み、乱数生成、数値列の集計、複素数の扱いなど、理工学系のプログラミングで利用できる基本的な機能が提供されている。

データ処理で広く用いられている各種の言語処理系¹⁹¹では、豊富なライブラリを用いることで、数値計算や統計処理を比較的簡潔に記述することができる。一方、C/C++では、高い実行性能を得ることを重視したプログラムを記述しやすく、計算量の大きな処理や、メモリの扱いを細かく制御したい処理において強みを発揮する。そのため、数値計算やデータ処理の中心部分を C/C++で実装することも多い。

本書では、C++ 標準ライブラリが提供する数値処理機能のうち、基本的なものを取り上げる。具体的には、ヘッダ `random` で提供される乱数生成機能、ヘッダ `numeric` で提供される数値列の集計機能、ヘッダ `complex` で提供される複素数の処理機能について説明する。さらに、C++ 標準ライブラリの機能ではないが、標準的な整数型や浮動小数点型では扱いきれない大きな数や高い精度の計算を行うための多倍長演算ライブラリについても、基本的な使用方法を紹介する。

本書では、数値計算ライブラリを網羅的に扱うことはしない。線形代数、フーリエ変換、微分方程式、最適化、記号数式処理などの本格的な科学技術計算については、専用のライブラリやソフトウェアが存在するが、それらは本書の範囲を超える。ここでは、C++の基本的な数値処理機能を理解し、必要に応じてより高度なライブラリへ進むための足がかりを与えることを目的とする。

22.1 乱数

数値計算やシミュレーションでは、乱数を利用する場面が多い。例えば、確率的な現象を模擬する処理、データをランダムに並べ替える処理、試行を多数回繰り返して結果の傾向を調べる処理などでは、乱数が重要な役割を果たす。

コンピュータ上で利用される乱数の多くは、完全に偶然に得られる値ではなく、決められた計算手順によって生成される値である。このような乱数を**疑似乱数**という。疑似乱数は、見かけ上は不規則に変化する値の列であるが、内部的には一定のアルゴリズムに従って生成される。乱数生成の初期状態を定める値を**種**、あるいは**シード** (seed) という。同じ乱数エンジンに同じ種を与えると、同じ乱数列を再現することができる。

同じ乱数列を再現できることは、実験やデバッグにおいて重要である。プログラムの開発過程では、同じ条件で処理を繰り返し、結果を比較したい場合がある。この場合、種を固定しておけば、同じ乱数列を用いて処理を再実行することができる。

C++標準ライブラリでは、乱数を扱うための機能がヘッダファイル `random` に用意されている。このヘッダファイルでは、乱数列を生成するための**乱数エンジン**と、生成された値を特定の確率分布に従う値として取り出すための**分布オブジェクト**を組み合わせる。すなわち、乱数エンジンが乱数のもとになる値の列を生成し、分布オブジェクトがそれを「指定した範囲の整数」や「正規分布に従う実数」などの形に変換する。例えば、**メルセンヌ・ツイスタ法**¹⁹²に基づく `mt19937` を乱数エンジンとして使用し、整数の一樣分布を表す `uniform_int_distribution` と組み合わせることで、指定した範囲の整数乱数を生成することができる¹⁹³。

本書では、基本的な分布の例として、一樣分布に従う乱数と、正規分布に従う乱数を取り上げる。

22.1.1 乱数エンジン

C++ 標準ライブラリには、方式や出力ビット数などが異なる複数の乱数エンジンが用意されている。乱数エンジンによって、生成される疑似乱数列の周期、統計的な性質、処理速度、内部状態の大きさなどが異なる。主な乱数エンジンを表 56 に示す。

表中の出力幅は、乱数エンジンから値を1回生成したときに得られる値のビット数を表している。これは、分布オブジェクトによって最終的に生成される乱数の範囲を表すものではない。例えば、後で解説する `uniform_int_distribution`

¹⁹¹例えば、Python や GNU R などは、データ処理、統計処理、機械学習などの分野で広く利用されている。

¹⁹²疑似乱数生成アルゴリズムとして広く使われている。

¹⁹³**注意** `mt19937` などの一般的な疑似乱数生成器は、暗号学的な安全性を目的としたものではない。従って、暗号鍵や認証用トークンなどの生成には使用しないこと。

表 56: 主な乱数エンジン

乱数エンジン	方式	出力幅	処理速度	特 徴
mt19937	メルセンヌ・ツイスタ法	32 ビット	速い	周期が非常に長く、処理速度も速い。一般的な数値計算やシミュレーションで広く利用される代表的な乱数エンジンである。
mt19937_64	メルセンヌ・ツイスタ法	64 ビット	速い	1 回の値の生成によって 64 ビット幅の値が得られる。64 ビットの整数値を直接利用したい場合などに適している。
ranlux24	RANLUX	24 ビット	比較的遅い	生成した値の一部を意図的に捨てることで、乱数列に含まれる相関を弱める。生成速度よりも乱数列の統計的な性質を重視した方式である。
ranlux48	RANLUX	48 ビット	比較的遅い	48 ビットの値を生成する RANLUX 方式の乱数エンジンである。乱数列の相関特性を重視するモンテカルロ計算などで選択肢となる。

を用いて 1 から 6 までの整数を生成する場合には、どの乱数エンジンを使用しても、最終的に得られる乱数の範囲は 1 から 6 までとなる。なお、表中の処理速度は一般的な傾向を相対的に示したものであり、実際の速度は使用する環境によって異なる。

本書では、以降の解説において、主として mt19937 を使用する。mt19937 は、周期が非常に長く、処理速度も速いため、一般的な数値計算やシミュレーションに利用しやすい乱数エンジンである。

乱数エンジンのオブジェクトは次のようにして作成する。

書き方： 乱数エンジン名 オブジェクト名 (種);

ここで、種 (seed) とは、疑似乱数列を生成する際の初期状態を決めるために与える値である。例えば、mt19937 型の乱数エンジンを生成するには、次のように記述する。

```
std::mt19937 mt(1234);
```

この例では、整数値 1234 を種として、mt という名前の乱数エンジンを生成している。同じ種類の乱数エンジンに同じ種を与えると、同じ順序の疑似乱数列が生成される。従って、プログラムを実行するたびに同じ計算結果を再現したい場合には、種を一定の値に固定するとよい。

乱数エンジンには、生成する値の型として result_type 型が定義されている。これは、乱数エンジンの出力値を格納できる符号なし整数型であり、単一の整数を種として与えるコンストラクタでも、この型の値が使用される。例えば、mt19937 および ranlux24 では 32 ビット以上、mt19937_64 および ranlux48 では 64 ビット以上の符号なし整数型が使用される。

ただし、乱数エンジンの出力ビット数と、result_type 型そのもののビット数は、必ずしも一致しない。例えば、ranlux24 が 1 回に生成する値の出力幅は 24 ビットであるが、result_type には 32 ビット以上の符号なし整数型が使用される。同様に、ranlux48 の出力幅は 48 ビットであるが、result_type には 64 ビット以上の符号なし整数型が使用される。

種には、通常、0 以上の整数値を指定する。例えば、次のように記述できる。

```
例 1. std::mt19937    mt1(1234);
例 2. std::mt19937_64 mt2(2026);
例 3. std::ranlux24   rl1(100);
例 4. std::ranlux48   rl2(5000);
```

種として素数や奇数などの特別な値を選んでも、それだけで生成される乱数の品質が高くなるわけではない。異なる種を与えると、通常は異なる疑似乱数列が生成される。ただし、種は乱数エンジンの初期状態を決める値であるため、互いに独立した多数の乱数列を必要とするような高度な計算では、種の与え方や初期化の方法についても検討する必要がある。

種として与えた整数値が、そのまま最初の乱数値として出力されるわけではない。乱数エンジンは、種を基にして内部状態を初期化し、その内部状態から疑似乱数列を生成する。また、種を指定せずに、次のようにオブジェクトを生成することもできる。

```
std::mt19937 mt;
```

この場合は、乱数エンジンごとにあらかじめ定められた既定の種が使用される。従って、プログラムを実行するたびに同じ疑似乱数列が生成される。

■ random_device を用いて種を得る方法

プログラムを実行するたびに異なる疑似乱数列を生成したい場合には、`random_device` を用いて実行環境から値を取得し、その値を乱数エンジンの種として使用することができる。例えば、次のように記述する。

```
std::random_device rd;
std::mt19937 mt(rd());
```

この例では、`rd()` によって取得した値を、`mt19937` 型の乱数エンジン `mt` の種として与えている。これにより、通常はプログラムを実行するたびに異なる疑似乱数列が生成される。また、`random_device` オブジェクトを変数として保存する必要がない場合には、次のように簡潔に記述することもできる。

```
std::mt19937 mt(std::random_device{}());
```

この記述では、`std::random_device{}` によって一時的な `random_device` オブジェクトを生成し、続く `()` によって値を取得している。得られた値は、`mt19937` オブジェクト `mt` の種として使用される。

なお、`random_device` が実際にどのような乱数源を利用するかは処理系に依存する。一般には、オペレーティングシステムやハードウェアなどが提供する非決定的な乱数源が利用されるが、そのような乱数源を利用できない処理系では、疑似乱数生成器によって値が生成される場合もある。従って、`random_device` を使用しても、すべての処理系において、プログラムを実行するたびに異なる値が得られることが保証されるわけではない。

■ 乱数の品質

疑似乱数は、あらかじめ定められた計算手順によって確定的に生成されるため、真の乱数ではない。乱数列の周期が短い場合や、生成される値の間に強い規則性や相関がある場合には、計算結果に偏りが生じることがある。このような偏りが少なく、統計的に不規則な性質をよく示す乱数列を、一般に品質の高い疑似乱数列という。

乱数の品質は、単に周期の長さだけで決まるものではない。連続して生成される値の相関、各ビットの現れ方、多数の値を組み合わせたときの分布など、複数の性質によって評価される。また、乱数列の統計的な性質を改善するために複雑な処理を行うと、乱数の生成速度が低下することがある。従って、乱数の品質と処理速度との間には、トレードオフが生じる場合がある。

例えば、RANLUX 方式では、内部で生成した値の一部を使用せずに捨てることで、連続して生成される値の相関を弱める。値を捨てる位置や個数もあらかじめ定められているため、RANLUX 方式で生成される乱数列も確定的な疑似乱数列である。この方式では、乱数列の相関特性を改善できる一方で、`mt19937` などと比べて処理速度は遅くなる。用途に応じて、必要となる乱数の品質と処理速度を考慮し、適切な乱数エンジンを選択する必要がある。

■ 乱数エンジンの出力ビット数

乱数エンジンの出力ビット数は、分布オブジェクトが1個の乱数を生成するために必要とする、乱数エンジンの出力回数に関係する。例えば、64ビットの整数値を広い範囲から生成する場合、32ビットの乱数エンジンでは、複数回の出力を組み合わせる必要がある。これに対して、64ビットの乱数エンジンでは、1回の出力から必要な情報を得られる場合がある。

このため、大きな整数値を扱う乱数を多数生成する場合には、`mt19937_64` などの出力ビット数が大きな乱数エンジンを採用することで、乱数生成処理を効率化できることがある。また、多くの処理系で `double` 型は2進数で53桁程度の精度を持つため、`double` 型の実数乱数を生成する場合にも、出力幅の大きな乱数エンジンが処理効率の面で有利になることがある。ただし、実際に必要となる乱数エンジンの出力回数や処理速度は、使用する分布オブジェクトと標準ライブラリの実装によって異なる。

最終的に生成される乱数の範囲や確率分布は、乱数エンジンではなく、組み合わせる使用する分布オブジェクトによって決まる。また、出力ビット数が大きいことが、そのまま乱数の品質が高いことを意味するわけではない。

22.1.2 一様乱数

一様乱数とは、指定された範囲の値が同じ程度の確率で現れる乱数である。例えば、1 から 6 までの整数が同じ確率で現れる乱数は、サイコロの目を模擬する処理に利用できる。また、0 以上 1 未満の実数が一様に現れる乱数は、確率的なシミュレーションやモンテカルロ法などでよく利用される。

C++ 標準ライブラリでは、ヘッダファイル `random` で、整数の一様乱数を生成するための `uniform_int_distribution`、浮動小数点数の一様乱数を生成するための `uniform_real_distribution` が提供されている。これらの分布オブジェクトは、乱数エンジンと組み合わせて使用する。すなわち、乱数エンジンが疑似乱数の列を生成し、分布オブジェクトがそれを指定された範囲の値として取り出す。

22.1.2.1 整数の一様乱数

整数型 `T` の値を生成する一様分布オブジェクトは、次のように作成する。

書き方： `std::uniform_int_distribution<T>` オブジェクト名 (最小値, 最大値);

ここで、`T` は生成する整数値の型である。例えば、`int` 型の乱数を生成する場合は、`uniform_int_distribution<int>` と記述する。また、コンストラクタに与える最小値 と 最大値 により、生成される整数乱数の範囲を指定する。このとき、最小値は最大値以下でなければならない。

`uniform_int_distribution` では、両端を含む範囲の整数乱数が生成される。例えば、

```
std::uniform_int_distribution<int> dist(1, 6);
```

とすると、1 以上 6 以下の整数乱数を生成する分布オブジェクト `dist` が作成される。

作成した分布オブジェクトから乱数を得るには、乱数エンジンを引数として分布オブジェクトを実行する。

書き方： 分布オブジェクト (乱数エンジン)

例えば、乱数エンジン `std::mt19937 rng;` と、分布オブジェクト `std::uniform_int_distribution<int> dist(1, 6);` が定義されているとき、`dist(rng)` を実行すると、1 以上 6 以下の整数乱数が 1 つ得られる。

これらの機能を用いて実際に 1~6 の範囲の整数乱数を生成するプログラム例 `test260-1.cpp` を次に示す。

プログラム：test260-1.cpp

```
1  #include    <iostream>
2  #include    <random>
3  using std::cout;
4
5  int main()
6  {
7      // 種123で初期化した乱数エンジン
8      std::mt19937    rng(123);
9      // 1~6の整数を生成する分布オブジェクト
10     std::uniform_int_distribution<int>    dist(1, 6);
11
12     for (int i = 0; i < 20; i++) {
13         if ( i != 0 ) cout << ", ";
14         cout << dist(rng);
15     }
16     cout << "\n";
17 }
```

このプログラムでは、種 123 を与えて `mt19937` 型の乱数エンジン `rng` を作成し、1 以上 6 以下の整数を生成する分布オブジェクト `dist` と組み合わせている。`dist(rng)` を 20 回実行することで、20 個の整数乱数を生成している。種を固定しているため、同じ処理系で再度実行すると、同じ乱数列が得られる。

このプログラムをコンパイルして実行する例を次に示す。

例. `test260-1.cpp` のコンパイルと実行

```
$ g++ test260-1.cpp -o test260-1  Enter    ←コンパイル
```

```
$ ./test260-1  Enter    ←実行
```

```
5, 5, 2, 3, 2, 5, 4, 5, 5, 3, 3, 5, 6, 3, 5, 4, 3, 1, 3, 3    ← 1~6 の乱数が 20 個生成された
```

```
$ ./test260-1  Enter    ←再度実行    ↓同じ処理系で同じ種を用いると、同じ乱数列が得られる
```

```
5, 5, 2, 3, 2, 5, 4, 5, 5, 3, 3, 5, 6, 3, 5, 4, 3, 1, 3, 3
```


22.1.2.2 浮動小数点数の一般乱数

浮動小数点数型 `T` の値を生成する一般分布オブジェクトは、次のように作成する。

書き方： `std::uniform_real_distribution<T>` オブジェクト名 (最小値, 最大値);

ここで、`T` は生成する浮動小数点数の型である。例えば、`double` 型の乱数を生成する場合は、

```
uniform_real_distribution<double>
```

と記述する。また、コンストラクタに与える 最小値 と 最大値 により、生成される浮動小数点数乱数の範囲を指定する。このとき、最小値は最大値以下であり、両者の差が `T` 型で表現できる範囲内でなければならない。これにより、指定された範囲内で偏りなく乱数が生成される。

`uniform_real_distribution` では、最小値以上、最大値未満の範囲の浮動小数点数乱数が生成される。例えば、

```
std::uniform_real_distribution<double> dist(0.0, 1.0);
```

とすると、0.0 以上 1.0 未満の `double` 型の乱数を生成する分布オブジェクト `dist` が作成される。

作成した分布オブジェクトから乱数を得る方法は、整数の一般乱数の場合と同じである。すなわち、乱数エンジン
を引数として分布オブジェクトを実行する。

書き方： 分布オブジェクト (乱数エンジン)

例えば、乱数エンジン `std::mt19937 rng;` と、上に示した分布オブジェクト `dist` が定義されているとき、`dist(rng)` を実行すると、0.0 以上 1.0 未満の浮動小数点数 (`double`) の乱数が 1 つ得られる。

これらの機能を用いて実際に 0.0 以上 1.0 未満の浮動小数点数乱数を生成するプログラム例 `test260-2.cpp` を次に示す。

プログラム： `test260-2.cpp`

```
1  #include    <iostream>
2  #include    <iomanip>
3  #include    <random>
4  using std::cout, std::fixed, std::setprecision;
5
6  int main()
7  {
8      // 種123で初期化した乱数エンジン
9      std::mt19937    rng(123);
10     // 0.0以上1.0未満の実数を生成する分布オブジェクト
11     std::uniform_real_distribution<double>    dist(0.0, 1.0);
12
13     cout << fixed << setprecision(6);
14     for (int i = 0; i < 16; i++) {
15         if ( i != 0 ) cout << ", ";
16         cout << dist(rng);
17     }
18     cout << "\n";
19 }
```

このプログラムでは、種 123 を与えて `mt19937` 型の乱数エンジン `rng` を作成し、0.0 以上 1.0 未満の浮動小数点数乱数を生成する分布オブジェクト `dist` と組み合わせている。`dist(rng)` を 16 回実行することで、16 個の乱数を生成している。また、`fixed` と `setprecision(6)` により、小数点以下 6 桁で表示している。表示される値は小数点以下 6 桁に丸められるが、生成された乱数自体の精度が 6 桁に制限されるわけではない。

このプログラムをコンパイルして実行する例を次に示す。

例. `test260-2.cpp` のコンパイルと実行

```
$ g++ test260-2.cpp -o test260-2  [Enter]    ←コンパイル
$ ./test260-2  [Enter]    ←実行              ↓ [0.0,1.0) の乱数が 16 個生成される
0.712955, 0.428471, 0.690885, 0.719150, 0.491119, 0.780028, 0.410924, 0.579694,
0.139951, 0.401018, 0.627317, 0.324151, 0.244759, 0.694755, 0.593902, 0.631792
$ ./test260-2  [Enter]    ←再度実行          ↓ 同じ処理系で同じ種を指定すると、同じ乱数列が再現される
0.712955, 0.428471, 0.690885, 0.719150, 0.491119, 0.780028, 0.410924, 0.579694,
0.139951, 0.401018, 0.627317, 0.324151, 0.244759, 0.694755, 0.593902, 0.631792
```


22.1.3 正規分布に従う乱数

正規分布は、平均値の近くの値が現れやすく、平均値から離れるほど現れにくくなる連続型の確率分布¹⁹⁴である。測定誤差、自然現象のばらつき、統計的なモデル化などでは、正規分布に従う乱数がよく利用される。例えば、多数の測定値に含まれる誤差を模擬する場合や、平均値の周辺にばらつくデータを生成する場合には、正規分布に従う乱数を用いることができる。

C++ 標準ライブラリでは、ヘッダファイル `random` で、正規分布に従う浮動小数点数乱数を生成するための `normal_distribution` が提供されている。`normal_distribution` では、平均と標準偏差を指定して分布オブジェクトを作成する。平均は分布の中心となる値であり、標準偏差は値のばらつきの大きさを表す。

書き方： `std::normal_distribution<T>` オブジェクト名 (平均, 標準偏差);

ここで、`T` は生成する浮動小数点数の型である。このとき、標準偏差には 0 より大きな値を指定しなければならない。例えば、`double` 型の乱数を生成する場合は、`normal_distribution<double>` と記述する。また、コンストラクタに与える 平均と 標準偏差 により、生成される乱数が従う正規分布を指定する。例えば、

```
normal_distribution<double> dist( 0.0, 1.0 )
```

とすると、平均 0.0、標準偏差 1.0 の正規分布に従う `double` 型の乱数を生成する分布オブジェクト `dist` が作成される。

作成した分布オブジェクトから乱数を得る方法は、一様乱数の場合と同じである。すなわち、乱数エンジンを引数として分布オブジェクトを実行する。

書き方： 分布オブジェクト (乱数エンジン)

例えば、乱数エンジン `std::mt19937 rng;` と、上に示した分布オブジェクト `dist` が定義されているとき、`dist(rng)` を実行すると、平均 0.0、標準偏差 1.0 の正規分布に従う浮動小数点数乱数が 1 つ得られる。

これらの機能を用いて実際に平均 0.0、標準偏差 1.0 の正規分布に従う乱数を生成するプログラム例 `test260-3.cpp` を次に示す。

プログラム：test260-3.cpp

```
1  #include    <iostream>
2  #include    <iomanip>
3  #include    <random>
4  using std::cout, std::fixed, std::setprecision, std::setw;
5
6  int main()
7  {
8      // 種123で初期化した乱数エンジン
9      std::mt19937 rng(123);
10     // 平均0.0, 標準偏差1.0の正規分布に従う実数を生成する分布オブジェクト
11     std::normal_distribution<double> dist(0.0, 1.0);
12
13     cout << fixed << setprecision(6);
14     for (int i = 0; i < 14; i++) {
15         if ( i != 0 ) cout << ", ";
16         cout << setw(9) << dist(rng);
17     }
18     cout << "\n";
19 }
```

このプログラムでは、種 123 を与えて `mt19937` 型の乱数エンジン `rng` を作成し、平均 0.0、標準偏差 1.0 の正規分布に従う乱数を生成する分布オブジェクト `dist` と組み合わせている。`dist(rng)` を 14 回実行することで、14 個の乱数を生成している。また、`fixed` と `setprecision(6)` によって小数点以下 6 桁で表示し、`setw(9)` によって各数値の表示幅をそろえている。

このプログラムをコンパイルして実行する例を次に示す。

¹⁹⁴連続型の確率分布では、個々の値そのものの出現確率ではなく、値の現れやすさを表す**確率密度関数**を用いて分布を表す。正規分布の釣鐘型の曲線（ベルカーブ）も、この確率密度関数を描いたものである。

例. test260-3.cpp のコンパイルと実行

```
$ g++ test260-3.cpp -o test260-3 Enter    ←コンパイル
$ ./test260-3 Enter    ←実行                ↓  $N(0,1)$  の乱数が 14 個生成される
-0.569610,  1.695834,  1.110871,  0.967594,  1.521361, -0.048250,  1.595301,
-1.783094, -0.286451, -1.041969, -1.479645,  1.071282,  0.807485, -1.058267
$ ./test260-3 Enter    ←再度実行                ↓ 同じ処理系で再度実行すると、同じ乱数列が再現される
-0.569610,  1.695834,  1.110871,  0.967594,  1.521361, -0.048250,  1.595301,
-1.783094, -0.286451, -1.041969, -1.479645,  1.071282,  0.807485, -1.058267
```

22.1.4 様々な乱数分布

C++ の標準ライブラリで利用できる各種の乱数分布オブジェクトを表 57 に挙げる。

表 57: 様々な乱数分布オブジェクト (一部)

分 布	分布オブジェクトの作成例	解 説
一様整数分布	<code>uniform_int_distribution<T> dist(n, m)</code>	$[n, m]$ の整数の乱数
一様実数分布	<code>uniform_real_distribution<T> dist(a, b)</code>	$[a, b)$ の実数の乱数
ベルヌーイ分布	<code>bernoulli_distribution dist(p)</code>	確率 p で <code>true</code> となる乱数
二項分布	<code>binomial_distribution<T> dist(n, p)</code>	n 回の試行における成功回数
幾何分布	<code>geometric_distribution<T> dist(p)</code>	初めて成功するまでの失敗回数
ポアソン分布	<code>poisson_distribution<T> dist(m)</code>	平均 m の発生回数
指数分布	<code>exponential_distribution<T> dist(lambda)</code>	発生率 λ の待ち時間
正規分布	<code>normal_distribution<T> dist(mu, sigma)</code>	平均 μ , 標準偏差 σ の実数乱数
対数正規分布	<code>lognormal_distribution<T> dist(mu, sigma)</code>	対数の平均が μ , 標準偏差が σ となる正の実数乱数
カイ二乗分布	<code>chi_squared_distribution<T> dist(n)</code>	自由度 n の実数乱数
F 分布	<code>fisher_f_distribution<T> dist(m, n)</code>	自由度 m, n の実数乱数
Student の t 分布	<code>student_t_distribution<T> dist(n)</code>	自由度 n の実数乱数
離散分布	<code>discrete_distribution<T> dist({w0,w1,...})</code>	重み $w_0, w_1, \dots$ に基づく整数乱数

注) 表中では、紙幅の都合により、分布オブジェクトの名前空間 `std::` を省略している。

表中では、乱数分布オブジェクトの名前を `dist` として示す。実際のプログラムでは、目的に応じて任意の変数名を与える。また、`T` は生成される値の型を表す。例えば、整数値を生成する分布では `int`、実数値を生成する分布では `double` などを指定する。ただし、`bernoulli_distribution` は `bool` 型の値を生成するため、テンプレート引数を指定しない。

各分布オブジェクトの詳しい仕様やパラメータの意味については、C++ のリファレンス文献 [6] などを参照のこと。

22.2 数値列の集計：numeric

多数の数値データを扱う場合、それらの総和や平均、分散、標準偏差などの基本的な統計量を求める処理が必要になることが多い。これらの統計量は、数値列の総和などの基本的な集計処理と算術演算を組み合わせることで求められる。また、2つの数値列の対応する要素どうしを掛け合わせて合計する**内積**や、先頭からの**累積和**を求める処理も、数値データの処理ではよく用いられる。C++ の標準ライブラリでは、このような数値列に対する基本的な処理を行うための関数が、ヘッダファイル `numeric` で提供されている。

例えば、数値列の総和を求めるには `accumulate`、2つの数値列の内積を求めるには `inner_product`、累積和を求めるには `partial_sum` を使用することができる。これらの機能は、`vector` などのコンテナに格納された数値列に対して、イテレータで処理範囲を指定して使用する。

ここでは、ヘッダ `numeric` が提供する関数のうち、数値データの集計でよく用いられる基本的なものを取り上げる。

22.2.1 総和と平均

数値列の総和を求めるには、`numeric` ヘッダで提供される関数 `accumulate` を使用する。

書き方： `std::accumulate( 開始位置, 終了位置, 初期値 )`

この関数は、指定された範囲に含まれる値を、初期値に順に加算していき、その結果を返す。ここで、開始位置と終了位置はイテレータで指定する。例えば、`vector` に格納された数値列 `v` の総和を求めるには、次のように記述する。

例. コンテナ `v` の総和を求める

```
double sum = std::accumulate(v.begin(), v.end(), 0.0);
```

この例では、`v.begin()` から `v.end()` の直前までの範囲に含まれる値を合計する。最後の引数 `0.0` は加算を始める際の初期値である。従って、上の式は「`0.0` に `v` の全要素を順に加えていく処理」と考えることができる。

`accumulate` を使用する際には、初期値の型に注意する必要がある。累積値と戻り値の型は、第3引数として与えた初期値の型に基づいて決まる。従って、実数値の総和を求める場合に整数値 `0` を与えると、整数型で加算が進められ、各段階で小数部分が失われる。例えば、次のように記述した場合、戻り値は `int` 型となり、その後で `double` 型の変数 `sum` に変換して格納される。

```
double sum = std::accumulate(v.begin(), v.end(), 0);
```

従って、`double` 型として総和を求める場合には、初期値として `0.0` を指定する。

C++ の標準ライブラリには、数値列の平均値を直接求める関数を用意されていない。平均値を求めるには、`accumulate` で総和を求め、それを要素数で割る。コンテナの要素数は `size()` で求めることができる。ただし、数値列が空の場合、`size()` の値は `0` となる。この場合、平均値を計算することはできない。従って、平均値を求める前には、要素数が `0` でないことを確認する必要がある。

`vector` オブジェクトの要素の総和と平均値を求めるプログラム `test261-1.cpp` を次に示す。

プログラム：test261-1.cpp

```
1  #include    <iostream>
2  #include    <vector>
3  #include    <numeric>
4  #include    <limits>
5  using std::cout, std::vector, std::accumulate, std::numeric_limits;
6
7  template <typename Container>
8  double average( const Container& c )
9  {
10     if ( c.size() == 0 ) {                                // 要素が0個の場合は
11         return numeric_limits<double>::quiet_NaN(); // 非数を返す
12     }
13
14     double sum = accumulate( c.begin(), c.end(), 0.0 );
15     return sum / c.size();
16 }
17
18 int main() {
19     vector<double> v1 = { 12.5, 10.0, 13.5, 9.0, 15.0 };
20 }
```

```

20     vector<double> v2 = {};          // 要素を持たない
21
22     double sum1 = accumulate( v1.begin(), v1.end(), 0.0 );
23     double sum2 = accumulate( v2.begin(), v2.end(), 0.0 );
24     double avg1 = average( v1 );
25     double avg2 = average( v2 );
26
27     cout << "data: ";
28     bool flg = true;
29     for ( double e : v1 ) {
30         if ( flg ) {
31             flg = false;
32         } else {
33             cout << ", ";
34         }
35         cout << e;
36     }
37     cout << "\nsum1 = " << sum1 << "\n";
38     cout << "avg1 = " << avg1 << "\n";
39
40     cout << "empty data:\n";
41     cout << "sum2 = " << sum2 << "\n";
42     cout << "avg2 = " << avg2 << "\n";
43 }

```

注) 非数を表示できる一般的な処理系を前提としている。また、非数の表示形式は処理系によって異なる場合がある。

このプログラムでは、v1 の総和を `accumulate` で求め、`average` 関数で平均値を求めている。`average` 関数では、要素数が 0 の場合に除算を行わず、`numeric_limits<double>::quiet_NaN()` によって非数を返している。また、空の v2 に対して `accumulate` を実行すると、加算する要素が存在しないため、初期値として与えた 0.0 がそのまま返される。このプログラムをコンパイルして実行する例を次に示す。

例. test261-1.cpp のコンパイルと実行

```

$ g++ test261-1.cpp -o test261-1   ←コンパイル
$ ./test261-1  ←実行
data: 12.5, 10, 13.5, 9, 15          ←要素を持つ vector
sum1 = 60                            ←合計値も
avg1 = 12                            ←平均値も得られているが、
empty data:                          ←要素が空の場合は、
sum2 = 0                             ←合計値は得られるが、
avg2 = nan                           ←平均値は非数としている

```

22.2.2 分散と標準偏差

数値列のばらつきを調べるには、**分散**や**標準偏差**を用いる。これらは、データ分析、統計処理、機械学習などの基礎となる重要な量である。平均値がデータ列の中心的な値を表すのに対して、分散や標準偏差は、個々の値が平均値の周囲にどの程度広がっているかを表す。

C++ の標準ライブラリには、数値列の分散や標準偏差を直接求める関数は用意されていない。従って、これらの量を求めるには、平均値を求めた後、各要素と平均値との差を用いて計算する必要がある。

数値列を $x_1, x_2, \dots, x_n$ とし、その平均値を $\bar{x}$ とする。このとき、分散は次の形で定義できる。

$$\frac{1}{n - \text{ddof}} \sum_{i=1}^n (x_i - \bar{x})^2$$

ここで、`ddof` は分母から差し引く値を表す。`ddof` は “Delta Degrees of Freedom” の略であり、自由度の補正量を意味する。`ddof=0` とすると、分母は n となる。これは、データ列そのもののばらつきを表す分散である。一方、`ddof=1` とすると、分母は $n - 1$ となる。これは、母集団から取り出した標本に基づいて母分散を推定するためによく用いられ、**不偏分散**と呼ばれる。

標準偏差は、分散の平方根として定義される。すなわち、分散を V とすると、標準偏差は $\sqrt{V}$ と表される。

分散は偏差の二乗に基づく量であるため、元のデータとは単位が異なる。これに対して、標準偏差は元のデータと同じ単位を持つため、ばらつきの大きさを直感的に理解しやすい。

ただし、分散を求める際には、分母 $n - \text{ddof}$ が 0 以下にならないように注意する必要がある。例えば、 $\text{ddof}=0$ の場合は、要素数が 0 であると分散は定義されない。また、 $\text{ddof}=1$ の場合は、要素数が 1 以下であると分散は定義されない。このような場合、プログラム上では非数 (NaN) を返す設計が考えられる。

次に示すプログラム test261-2.cpp では、数値列の平均値、分散、標準偏差を求める関数を定義し、正規分布に従う乱数列に対して使用している。このプログラムでは、平均 0、標準偏差 2 の正規分布 $N(0, 2^2)$ に従う乱数を 1000 個生成し、それらを vector に格納して、平均値、分散、標準偏差を求める。

プログラム：test261-2.cpp

```
1  #include    <iostream>
2  #include    <vector>
3  #include    <numeric>
4  #include    <random>
5  #include    <limits>
6  #include    <cmath>
7  #include    <iomanip>
8  using std::cout, std::vector, std::accumulate;
9  using std::mt19937, std::normal_distribution;
10 using std::numeric_limits, std::sqrt, std::setprecision;
11
12 template <typename Container>
13 double average( const Container& c )
14 {
15     if ( c.empty() ) {
16         return numeric_limits<double>::quiet_NaN();
17     }
18
19     double sum = accumulate( c.begin(), c.end(), 0.0 );
20     return sum / c.size();
21 }
22
23 template <typename Container>
24 double variance( const Container& c, int ddof = 0 )
25 {
26     // ddofが負の場合は分散を定義できない
27     if ( ddof < 0 ) {
28         return numeric_limits<double>::quiet_NaN();
29     }
30
31     // ddofを要素数と同じ型に変換する
32     using size_type = decltype( c.size() );
33     size_type d = static_cast<size_type>( ddof );
34     size_type n = c.size();
35
36     // 分母 n-ddof が0以下となる場合は分散を定義できない
37     if ( n <= d ) {
38         return numeric_limits<double>::quiet_NaN();
39     }
40
41     double avg = average( c );
42     double sum = 0.0;
43
44     for ( auto e : c ) {
45         double diff = e - avg;
46         sum += diff * diff;
47     }
48
49     return sum / ( n - d );
50 }
51
52 template <typename Container>
53 double stddev( const Container& c, int ddof = 0 )
54 {
55     return sqrt( variance( c, ddof ) );
56 }
57
58 int main()
59 {
60     mt19937 mt( 2026 );
```



```

61     normal_distribution<double> nd( 0.0, 2.0 );
62
63     vector<double> v;
64     for ( int i = 0; i < 1000; i++ ) {
65         v.push_back( nd( mt ) );
66     }
67
68     cout << setprecision( 6 );
69
70     cout << "number of random data on N(0,2^2) : " << v.size() << "\n";
71     cout << "average          : " << average( v ) << "\n";
72     cout << "variance(ddof=0)   : " << variance( v ) << "\n";
73     cout << "variance(ddof=1)   : " << variance( v, 1 ) << "\n";
74     cout << "stddev(ddof=0)      : " << stddev( v ) << "\n";
75     cout << "stddev(ddof=1)      : " << stddev( v, 1 ) << "\n";
76 }

```

関数 `average` は、コンテナを引数として受け取り、その要素の平均値を `double` 型の値として返す。要素数が 0 の場合、平均値は定義されないため、この関数は `numeric_limits<double>::quiet_NaN()` により非数 (NaN) を返す。

関数 `variance` は、コンテナを引数として受け取り、その要素の分散を返す。第 2 引数 `ddof` は、分散を求める際に分母から差し引く値を指定するためのものである。この関数は、`ddof=0` の場合は分母が n となり、データ列そのもののばらつきを表す分散を求める。一方、`ddof=1` の場合は分母が $n-1$ となり、不偏分散を求める。ただし、`ddof` が負の値である場合や、 $n - \text{ddof}$ が 0 以下になる場合は、分散は定義できない。この関数は、そのような場合に NaN を返す仕様としている。

関数 `stddev` は、`variance` で得られた分散の平方根を取り、標準偏差を返す。

このプログラムでは、`mt19937` 型の乱数エンジン `mt` を用意し、`normal_distribution<double> nd(0.0, 2.0);` により、平均 0、標準偏差 2 の正規分布に従う乱数を生成している。理論上、この分布の平均値は 0、分散は 4、標準偏差は 2 である。実際に得られる値は乱数に依存して変動するが、データ数を十分に大きくすると、これらの値に近づくことが期待できる。また、乱数エンジン `mt` に固定した種 2026 を与えているため、同じ処理系で同じプログラムを実行すると、同じ乱数列が生成される。これにより、実行例を再現しやすくしている。

このプログラムをコンパイルして実行する例を次に示す。

例. `test261-2.cpp` のコンパイルと実行

```

$ g++ test261-2.cpp -o test261-2  [Enter]    ←コンパイル
$ ./test261-2  [Enter]             ←実行
number of random data on N(0,2^2) : 1000
average          : -0.00592241      ←平均値はほぼ 0
variance(ddof=0) : 3.94411          ←分散はほぼ 4
variance(ddof=1) : 3.94806          ←不偏分散は若干大きい
stddev(ddof=0)   : 1.98598          ←標準偏差はほぼ 2
stddev(ddof=1)   : 1.98697          ←補正の結果は若干大きい

```

22.2.3 内積と累積和

数値列に対する基本的な計算として、**内積**と**累積和**がある。これらは、ベクトル計算、統計処理、時系列データの処理、機械学習など、様々な場面で利用される重要な計算である。

2 つの数値列を $a_1, a_2, \dots, a_n$, $b_1, b_2, \dots, b_n$ とする。このとき、これらの内積は次のように定義される。

$$a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

すなわち、対応する位置にある要素どうしを掛け合わせ、その総和を求めたものである。内積は、ベクトルどうしの類似度の計算、重み付き和や**加重平均**¹⁹⁵、線形モデルの計算などでよく用いられる。例えば、機械学習では、入力データと重みベクトルの内積を求める処理がしばしば現れる。

一方、数値列 $x_1, x_2, \dots, x_n$ に対する累積和は、次のような数値列として定義される。

$$x_1, \quad x_1 + x_2, \quad x_1 + x_2 + x_3, \quad \dots, \quad x_1 + x_2 + \dots + x_n$$

¹⁹⁵**加重平均**は、各値に重みを掛けて平均を求める方法である。重みの総和が 1 である場合は、重み付き和がそのまま加重平均となる。重みの総和が 1 でない場合は、重み付き和を重みの総和で割ることで加重平均を求める。

例えば、数値列 1,2,3,4,5 の累積和は、1,3,6,10,15 である。累積和は、時系列データの合計値の推移、累積度数、区間和の計算、グラフ描画用データの作成などに利用できる。

C++ の `numeric` ヘッダでは、内積を求める関数 `inner_product` と、累積和を求める関数 `partial_sum` が提供されている。これらを用いると、数値列に対する内積や累積和を簡潔に記述することができる。

書き方： `std::inner_product( 範囲 1 の開始位置, 範囲 1 の終了位置, 範囲 2 の開始位置, 初期値 )`

この関数は、第 1 の範囲（「範囲 1 の開始位置」から「範囲 1 の終了位置」の 1 つ前まで）に含まれる各要素と、第 2 の範囲（「範囲 2 の開始位置」から始まる）に含まれる対応する要素を順に掛け合わせ、その結果を「初期値」に加算していく。第 2 の範囲には、第 1 の範囲と同じ個数以上の要素が存在していなければならない。また、累積値と戻り値の型は初期値の型に基づくため、実数の内積を求める場合には、初期値として 0.0 など指定する。

書き方： `std::partial_sum( 開始位置, 終了位置, 出力先の開始位置 )`

この関数は、指定された範囲（「開始位置」から「終了位置」の 1 つ前まで）に含まれる値を先頭から順に加算し、その途中結果を「出力先の開始位置」から順に格納する。通常のイテレータを出力先として指定する場合には、元の範囲と同じ個数の値を書き込めるだけの領域を、あらかじめ用意しておく必要がある。

次に示すプログラム `test261-3.cpp` は、`inner_product` と `partial_sum` を使用する例である。

プログラム：`test261-3.cpp`

```
1  #include <iostream>
2  #include <vector>
3  #include <numeric>
4  using std::cout, std::vector, std::inner_product, std::partial_sum;
5
6  int main() {
7      vector<double> score = { 80, 90, 70, 60 };
8      vector<double> weight = { 0.2, 0.3, 0.3, 0.2 };
9
10     cout << "score : ";
11     for ( double e : score ) cout << e << " ";
12     cout << "\n";
13
14     cout << "weight: ";
15     for ( double e : weight ) cout << e << " ";
16     cout << "\n";
17
18     double result = inner_product(
19         score.begin(), score.end(), weight.begin(), 0.0 );
20     cout << "weighted average = " << result << "\n";
21
22     vector<int> v = { 1,2,3,4,5,6,7,8,9,10 };
23     vector<int> s( v.size() );
24
25     cout << "[partial sum]\ndata      : ";
26     for ( int e : v ) cout << e << " ";
27     cout << "\n";
28
29     partial_sum( v.begin(), v.end(), s.begin() );
30
31     cout << "partial sum: ";
32     for ( int e : s ) cout << e << " ";
33     cout << "\n";
34 }
```

このプログラムでは、まず、点数を表す `vector` オブジェクト `score` と、重みを表す `vector` オブジェクト `weight` を用意し、それらの内積を求めることで加重平均を計算している。`score` と `weight` の要素は、先頭から順に対応付けられる。従って、この例で計算している値は、次の式で表される。

$$80 \times 0.2 + 90 \times 0.3 + 70 \times 0.3 + 60 \times 0.2$$

これは、それぞれの点数に重みを掛けて加算する計算である。この例では、重みの総和が 1.0 であるため、得られた重み付き和がそのまま加重平均となる。このように、`inner_product` は、対応する要素どうしを掛け合わせ、その総和を求める処理に利用できる。

また、このプログラムでは、整数列 1,2,3,...,10 に対して `partial_sum` を実行し、累積和を求めている。累積和は、

先頭から各位置までの合計値を順に並べたものである。従って、1,2,3,...,10 の累積和は、1,3,6,10,15,21,28,36,45,55 となる。

このプログラムでは、`partial_sum` の計算結果を格納するために、元の数値列 `v` と同じ要素数を持つ `vector` オブジェクト `s` をあらかじめ作成し、その先頭位置 `s.begin()` を出力先として指定している。

このプログラムをコンパイルして実行する例を次に示す。

例. `test261-3.cpp` のコンパイルと実行

```
$ g++ test261-3.cpp -o test261-3  [Enter]    ←コンパイル
$ ./test261-3  [Enter]    ←実行
score : 80 90 70 60                ←この各要素に
weight: 0.2 0.3 0.3 0.2            ←これらの重みをそれぞれかけ合わせ
weighted average = 76              ←得られた結果
[partial sum]
data      : 1 2 3 4 5 6 7 8 9 10    ←これらの
partial sum: 1 3 6 10 15 21 28 36 45 55 ←累積和
```

22.2.4 連番の生成

プログラミングにおいて、1,2,3,... のような連番を持つ数値列が必要になることがある。例えば、データ番号、添字の一覧、簡単なテストデータなどを用意する場合である。このような処理は `for` 文を用いて記述することもできるが、C++ の `numeric` ヘッダでは、連番を生成するための関数 `iota`¹⁹⁶ が提供されている。

書き方: `std::iota( 開始位置, 終了位置, 初期値 )`

この関数は、イテレータで指定された範囲（「開始位置」から「終了位置」の1つ前まで）に、「初期値」から1ずつ増加する値を順に格納する。例えば、初期値として1を指定すると、指定された範囲には1,2,3,... という値が順に格納される。ただし、`iota` はコンテナに新しい要素を追加する関数ではないため、値を格納する要素をあらかじめ用意しておく必要がある。

ただし、`iota` では増分を指定することはできない。常に、直前の値に1を加えた値が次の要素に格納される。従って、2ずつ増える列や、0.5ずつ増える列を作成したい場合は、`for` 文を用いるなどして目的に応じた処理を実装する必要がある。

次に示すプログラム `test261-4.cpp` は、`iota` による連番作成と `for` 文による数列作成を示す例である。

プログラム: `test261-4.cpp`

```
1  #include <iostream>
2  #include <vector>
3  #include <numeric>
4  using std::cout, std::vector, std::iota;
5
6  int main() {
7      vector<int> v1( 10 );
8
9      iota( v1.begin(), v1.end(), 0 );    // 連番作成
10     cout << "iota: ";
11     for ( int e : v1 ) cout << e << " ";    // 出力
12     cout << "\n";
13
14     vector<int> v2( 10 );
15     // 1以外の増分で数列を作成する
16     int x = 10;    // 初期値
17     for ( int& e : v2 ) {    // 自身の要素の「参照」によるfor
18         e = x;    // 「参照」への代入で自身の要素を書き換える手法
19         x += 2;    // 増分
20     }
21     cout << "for: ";
22     for ( int e : v2 ) cout << e << " ";    // 出力
23     cout << "\n";
24 }
```

¹⁹⁶プログラミング言語 APL で連続する整数列を生成するイオタ演算子に由来する。イオタ (*i*) はギリシア文字の1つである。

このプログラムでは、まず `iota` を用いて、`vector` オブジェクト `v1` に 0 から 9 までの連番を格納している。 `iota` は、指定した範囲に初期値から 1 ずつ増加する値を格納する関数である。従って、 `iota( v1.begin(), v1.end(), 0 )` と記述すると、 `v1` には 0, 1, 2, ..., 9 が順に格納される。

一方、 `iota` では増分を指定することはできない。そこで、このプログラムの後半では、 `for` 文を用いて 10 から 28 までの偶数列を作成している。範囲に基づく `for` 文で `int& e` と記述することで、 `v2` の各要素を参照として取り出し、そこに値を代入して更新する手法を取っている。

このプログラムをコンパイルして実行する例を次に示す。

例. test261-4.cpp のコンパイルと実行

```
$ g++ test261-4.cpp -o test261-4  ←コンパイル
$ ./test261-4  ←実行
iota: 0 1 2 3 4 5 6 7 8 9
for: 10 12 14 16 18 20 22 24 26 28
```

22.3 複素数の扱い：complex

C++ の標準ライブラリは、複素数を扱うためのクラステンプレート `complex` を提供している。複素数は、実部と虚部を持つ数値であり、数学では一般に $a + bi$ の形で表される。ここで、 a は実部、 b は虚部であり、 i は $i^2 = -1$ を満たす虚数単位である。

C 言語にも、`complex.h` ヘッダを用いた複素数処理の機能があり、複素数に対して $+$ や $*$ などの演算子を使用できる。一方、C++ の標準ライブラリでは、クラステンプレート `complex` によって複素数が表現されている。コンストラクタや演算子のオーバーロードなどの仕組みにより、通常の数値型に近い形で複素数を扱うことができる。一部の C++ 処理系では、拡張機能によって C 言語形式の複素数処理を利用できる場合もあるが、そのような機能を用いたプログラムには移植性がない可能性がある。従って、C++ で複素数を扱う場合には、C 言語形式の機能ではなく、C++ 標準ライブラリの `std::complex` を使用することを強く推奨する。

`std::complex` を使用するには、ヘッダファイル `complex` を読み込む。

`complex` はクラステンプレートとして定義されており、実部と虚部を表す値の型をテンプレート引数として指定する。通常は、実数計算用に `double` を指定して `complex<double>` として使用する。

書き方： `std::complex<T> 変数名 ( 実部, 虚部 );`

例えば、実部が 3.0、虚部が 4.0 の複素数を作成するには、次のように記述する。

例. 複素数オブジェクトの作成

```
std::complex<double> z(3.0, 4.0);
```

この例で作成される z は、数学的には $3.0 + 4.0i$ に対応する複素数である。また、虚部を省略して実部だけを指定することもできる。

例. 実数値から複素数オブジェクトを作成する例

```
std::complex<double> z(3.0);
```

この場合、 z は $3.0 + 0.0i$ に対応する複素数となる。

`complex` のテンプレート引数には、`float`、`double`、`long double` のいずれかを指定する。C++17 では、これら以外の型を指定した場合の動作は規格上定められていない。従って、`complex<int>` のように整数型を指定する使い方は避ける必要がある。本書では、通常の実数計算では、`double` を指定して `complex<double>` として使用する。

例. C++17 では動作が規定されていない例

```
std::complex<int> z(3, 4);
```

この宣言は、処理系によってはコンパイルできる場合がある。しかし、`float`、`double`、`long double` 以外の型を指定した `complex` の扱いは、C++17 の規格では定められていないため、移植可能な動作を期待することはできない。従って、`complex<int>` のような使い方は避ける必要がある。

■ 複素数の出力形式

`complex` オブジェクトは、そのまま出力ストリームに送出して表示することができる。ただし、その表示形式は数学で用いられる $a + bi$ の形ではなく、実部と虚部を丸括弧の中に並べた形になる。例えば、

```
std::complex<double> z(3.0, 4.0);
std::cout << z << "\n";
```

とすると、出力は次のようになる。

```
(3,4)
```

この出力は、数学的には $3 + 4i$ に対応する複素数を表している。すなわち、`complex` オブジェクトの基本的な出力形式は、(実部, 虚部) である。数学的な $a + bi$ の形で表示したい場合は、後述の `real()` や `imag()` を用いて、実部と虚部を個別に取り出し、出力形式を整える必要がある。

22.3.1 実部、虚部

`complex` オブジェクトの実部と虚部は、メンバ関数 `real()`、`imag()` で取り出すことができる。

書き方: 複素数.real()
書き方: 複素数.imag()

real() は複素数の実部を返し、imag() は虚部を返す。例えば、complex<double> 型の変数 z が $3.0 + 4.0i$ に対応する複素数を保持している場合、z.real() は 3.0 を返し、z.imag() は 4.0 を返す。

例. 複素数の実部と虚部の取得

```
std::complex<double> z(3.0, 4.0);  
double a = z.real();  
double b = z.imag();
```

この例では、 a に実部 3.0 が代入され、 b に虚部 4.0 が代入される。また、標準ライブラリには、同じ処理を行う非メンバ関数 real(), imag() も用意されている。

書き方: real(複素数)
書き方: imag(複素数)

例えば、次のように記述しても、実部と虚部を取り出すことができる。

例. 非メンバ関数による実部と虚部の取得

```
std::complex<double> z(3.0, 4.0);  
double a = std::real(z);  
double b = std::imag(z);
```

この例でも、 a に実部 3.0 が代入され、 b に虚部 4.0 が代入される。後に述べる abs(), arg(), conj() なども、複素数を引数として実行する非メンバ関数である。

一方、実部や虚部を後から変更する場合は、メンバ関数形式の real(), imag() に値を与える。

書き方: 複素数.real(新しい実部)
書き方: 複素数.imag(新しい虚部)

例えば、次のように記述すると、 z の実部と虚部を後から変更できる。

例. 複素数の実部と虚部の変更

```
std::complex<double> z(3.0, 4.0);  
z.real(5.0);  
z.imag(6.0);
```

この場合、 z は $5.0 + 6.0i$ に対応する複素数となる。

22.3.2 基本的な関数

complex ヘッダは、複素数に対して使用できる各種の関数を提供している。複素数 $z = a + bi$ は、複素平面上の点 (a, b) として見ることもできる。このとき、原点からの距離、偏角、共役複素数などを求める関数が用意されている。主な関数を表 58 に示す。

表 58: 複素数に対する基本的な関数

関 数	解 説
abs(z)	複素数 z の絶対値 $ z $ (原点からの距離)
arg(z)	複素数 z の偏角
norm(z)	複素数 z の絶対値の二乗 $ z ^2$ (z の体ノルム)
conj(z)	複素数 z の共役複素数
polar(r , θ)	絶対値 r , 偏角 θ の極座標形式に対応する複素数

複素数 $z = a + bi$ に対して、abs(z) は $\sqrt{a^2 + b^2}$ を返す。これは複素平面上での原点からの距離であり、複素数の絶対値と呼ばれる。また、arg(z) は偏角を返す。偏角は、正の実軸を基準として、複素数 z が表す点の方向を角度で表したものである。角度の単位はラジアンである。なお、数学的には $z = 0$ の偏角は定義されないが、arg() は atan2(imag(z), real(z)) に相当する値を返す。

一方、norm(z) は、複素数の絶対値そのものではなく、絶対値の二乗を返す。すなわち、 $z = a + bi$ のとき、norm(z) は $a^2 + b^2$ を返す。これは $z\bar{z}$ に等しい量であり、代数学では複素数体の体ノルムと呼ばれる。複素平面上での原

点からの距離を求めたい場合は、`norm()` ではなく `abs()` を使用する。

`conj( z )` は、複素数 z の共役複素数を返す。 $z = a + bi$ の場合、共役複素数は $a - bi$ である。すなわち、実部はそのまま、虚部の符号を反転した複素数が得られる。

`polar( r, theta )` は、絶対値が r 、偏角が θ となる複素数を返す関数である。これは、次の複素数を作成することに対応する。

$$r(\cos \theta + i \sin \theta)$$

従って、複素数を実部と虚部で直接指定するのではなく、極形式で指定したい場合に `polar()` を使用する。ただし、`r` には 0 以上で非数 (NaN) でない値を指定し、`theta` には有限値を指定しなければならない。

次のプログラム `test262-1.cpp` は、複素数 $z = 1 + \sqrt{3}i$ を作成し、その絶対値、偏角、絶対値の二乗、共役複素数を求める。さらに、得られた絶対値と偏角を `polar()` に与えることで、元の複素数に相当する値が再構成されることを確認する。

プログラム：test262-1.cpp

```
1 #include <iostream>
2 #include <complex>
3 #include <cmath>
4 using std::cout, std::complex;
5 using std::abs, std::arg, std::norm, std::conj, std::polar, std::sqrt;
6
7 int main() {
8     const double pi = 3.1415926535897932;
9
10    complex<double> z( 1.0, sqrt( 3.0 ) );
11
12    cout << "z = " << z << ", pi/3 = " << pi/3 << "\n";
13    cout << "abs(z) = " << abs( z ) << "\n";
14    cout << "arg(z) = " << arg( z ) << "\n";
15    cout << "norm(z) = " << norm( z ) << "\n";
16    cout << "conj(z) = " << conj( z ) << "\n";
17    cout << "polar(abs(z), arg(z)) = " << polar( abs( z ), arg( z ) ) << "\n";
18 }
```

複素数 z の実部、虚部から、偏角は $\pi/3$ となる。プログラムでは、`arg( z )` の値と比較するために、円周率を表す定数 `pi` を定義し、`pi/3` も併せて出力している。

`abs( z )` は複素数 z の絶対値である 2 を返す。一方、`norm( z )` は絶対値の二乗を返すので、その値は 4 となる。また、`conj( z )` は虚部の符号を反転した共役複素数 $1 - \sqrt{3}i$ を返す。

最後の `polar( abs( z ), arg( z ) )` は、 z から求めた絶対値と偏角を用いて複素数を作成する処理である。理論上は元の複素数 z と同じ値になるが、実際の計算では浮動小数点演算によるわずかな誤差が生じる可能性がある。

このプログラムをコンパイルして実行する例を次に示す。

例. test262-1.cpp のコンパイルと実行

```
$ g++ test262-1.cpp -o test262-1  ←コンパイル
$ ./test262-1  ←実行
z = (1,1.73205), pi/3 = 1.0472
abs(z) = 2                ←絶対値 (原点からの距離)
arg(z) = 1.0472           ←偏角
norm(z) = 4               ←体ノルム
conj(z) = (1,-1.73205)    ←共役複素数
polar(abs(z), arg(z)) = (1,1.73205) ←絶対値と偏角から複素数を構成
```

22.3.3 複素数に対応する数学関数

C++ の標準ライブラリが提供する数学関数の多くは、実数だけでなく複素数にも対応している。例えば、平方根を求める `sqrt`、指数関数 `exp`、対数関数 `log`、三角関数 `sin`、`cos`、`tan` などに、`complex` オブジェクトを引数として与えることができる。

これは、C++ の関数のオーバーロードによって実現されている。すなわち、同じ名前の関数でも、引数として実数値が与えられた場合には実数用の関数が選択され、`complex` オブジェクトが与えられた場合には複素数用の関数が

選択される。従って、複素数に対しても、実数の場合とほぼ同じ書き方で数学関数を使用することができる。

例. 複素数に対する数学関数

```
std::complex<double> z(1.0, 2.0);
std::complex<double> z1 = std::sqrt(z);
std::complex<double> z2 = std::exp(z);
std::complex<double> z3 = std::log(z);
std::complex<double> z4 = std::sin(z);
```

複素数に対応する主な数学関数を表 59 に挙げる。

表 59: 複素数に対応する主な数学関数	
関 数	解 説
<code>sqrt(z)</code>	平方根
<code>exp(z)</code>	指数関数
<code>log(z)</code> , <code>log10(z)</code>	自然対数, 常用対数
<code>pow(z1, z2)</code>	べき乗
<code>sin(z)</code> , <code>cos(z)</code> , <code>tan(z)</code>	三角関数
<code>asin(z)</code> , <code>acos(z)</code> , <code>atan(z)</code>	逆三角関数
<code>sinh(z)</code> , <code>cosh(z)</code> , <code>tanh(z)</code>	双曲線関数
<code>asinh(z)</code> , <code>acosh(z)</code> , <code>atanh(z)</code>	逆双曲線関数

複素数の平方根, 対数, 逆三角関数などは, 数学的には複数の値を持つことがある。C++ の複素数用数学関数は, 規格で定められた範囲や枝に基づいて, そのうちの 1 つの値を返す。この値を**主値**という。

複素数に対応した数学関数を用いると, 実数の範囲では値を持たない計算も扱うことができる。例えば, 実数 -1 の平方根は実数の範囲には存在しないが, 複素数として与えれば, その平方根の主値を求めることができる。

例. 負の数の平方根

```
std::complex<double> z(-1.0, 0.0);
std::cout << std::sqrt(z) << "\n";
```

この実行結果は, 概ね次のようになる。

(0,1)

-1 の平方根には i と $-i$ があるが, `sqrt()` は主値である i に対応する値を返す。

このように, `complex` と複素数用の数学関数を組み合わせることで, 実数の範囲を越えた計算も, 通常の数値計算に近い記法で行うことができる。

22.4 多倍長精度演算

C++ の整数型や浮動小数点数型で扱うことのできる値の範囲と精度には限界がある。例えば、`long long` 型で扱える整数の桁数には上限があり、`double` 型で保持できる有効数字も通常は十進数で 15 桁程度である。このため、非常に大きな整数を正確に扱う処理や、通常の浮動小数点数型を超える精度を必要とする数値計算では、標準の数値型だけでは対応できないことがある。

このような場合には、通常の数値型よりも多くの桁を扱うことのできる**多倍長演算**¹⁹⁷ が用いられる。多倍長演算では、数値を複数の機械語長のデータに分けて保持し、CPU が直接扱える語長の演算を組み合わせることで、加減乗除などの処理をソフトウェアによって実現する。これにより、使用可能なメモリや計算時間の範囲内で、通常の整数型や浮動小数点数型を大きく超える桁数の値を扱うことができる。

ただし、C++ の標準ライブラリには、多倍長整数や任意の精度を持つ浮動小数点数を扱うための本格的な機能は用意されていない。そのため、多倍長精度演算を行う場合には、外部のライブラリや数式処理システムを導入して使用するのが一般的である。代表的なライブラリとして、**GMP**、**MPFR**、**MPC** がある。これらは互いに関連しており、多倍長整数、浮動小数点数、複素数の演算機能を提供する。

GMP は、GNU Multiple Precision Arithmetic Library の略称であり、多倍長の整数、有理数、浮動小数点数に対する演算機能を提供する。GMP は C 言語を中心に実装された移植性の高いライブラリであり、処理環境に応じて最適化された高速な演算機能を提供する。

GMP の原型は Torbjörn Granlund によって開発され、1990 年代初頭から改良が続けられており、多倍長整数演算を必要とする数式処理、暗号処理、整数論、科学技術計算などの基盤として広く利用されている。GMP は GNU プロジェクトの一部として開発、公開されている**自由ソフトウェア**であり、多くの処理環境で使用することができる。

GMP 自体も、`mpf_t` 型によって多倍長浮動小数点数を扱う機能を提供している。この機能は、`double` 型より多くの有効数字を必要とする計算を比較的簡単に行う場合に利用でき、厳密な丸め制御や特殊値の扱いを必要としない用途では有効である。一方、浮動小数点演算の精度や丸め方法をより厳密に管理する必要がある場合には、MPFR が用いられる。

MPFR は、Multiple Precision Floating-Point Reliable Library の略称であり、GMP を基盤として実装された多倍長浮動小数点演算ライブラリである。MPFR は、単に多くの桁を保持するだけでなく、演算結果の丸め方法を明示的に指定し、指定した精度に対して正しく丸められた結果を得ることを重要な目的としている。また、非数や無限大などの特殊な値も扱うことができる。このため、計算結果の再現性や誤差の管理が重要となる科学技術計算、数値解析、数学関数の実装などに適している。

MPFR の開発は、フランスの INRIA や LORIA などの研究機関による支援の下で進められ、現在も継続的に保守、改良されている。

MPC は、GMP と MPFR を基盤として実装された多倍長複素数演算ライブラリである。複素数の実部と虚部に多倍長浮動小数点数を用い、演算の精度や丸め方法を管理することができる。これにより、指定した精度に対して正しく丸められた複素数の演算結果を得ることができる。

MPC は、Andreas Enge, Mickaël Gastineau, Philippe Théveny, Paul Zimmermann らによって開発された。開発にはフランスの国立研究機関である INRIA が深く関わっており、現在の開発作業も INRIA が提供する GitLab 上で管理されている。MPC は GNU プロジェクトの一部として公開されている**自由ソフトウェア**であり、GNU Lesser General Public License (LGPL) の下で利用することができる。

MPC は現在も継続的に保守、改良されており、新しい数学関数の追加、特殊な値に対する演算結果の改善、各種処理環境への対応、不具合の修正などが続けられている。多倍長複素数演算の代表的なライブラリとして、安定した位置を占めている。

GMP, MPFR, MPC の主な役割は、表 60 のように整理できる。

MPFR は内部の多倍長演算に GMP を利用するため、MPFR を使用する処理環境には GMP も必要となる。また、MPC は GMP と MPFR を基盤としているため、MPC を使用する処理環境には、これらのライブラリも必要とな

¹⁹⁷本書では、「多倍長精度演算」とほぼ同じ意味で用いる。

表 60: GMP, MPFR, MPC の主な役割

ライブラリ	主な役割
GMP	多倍長整数, 有理数, および簡便な多倍長浮動小数点数の高速な演算
MPFR	精度と丸め方法を厳密に管理する多倍長浮動小数点演算
MPC	精度と丸め方法を厳密に管理する多倍長複素数演算

る。すなわち、GMP を基礎として MPFR が多倍長浮動小数点演算を提供し、さらに MPC が多倍長複素数演算を提供するという関係にある。

これらのライブラリは、一般の数値計算に利用されるだけでなく、GNU Compiler Collection (GCC) をソースコードから構築する際にも必要となる。GCC は、コンパイル時の定数計算などに高精度な数値演算を必要とするため、GMP, MPFR, MPC を構築時の基盤ライブラリとして利用している。このことから、これらが長い実績と高い信頼性を持つ、多倍長精度演算の代表的なライブラリであることが分かる。

多倍長精度演算や数式処理を行うための処理系には、このほかに **PARI/GP** などもある。PARI/GP は、整数論に関する高速な計算を主な目的として開発された数式処理システムであり、大きな整数の演算、素因数分解、代数的数論、楕円曲線、多項式などを扱う多くの機能を備えている。PARI は C 言語から利用できるライブラリであり、GP はそれらの機能を対話的に利用するためのスクリプト言語および実行環境である。

このように、多倍長精度演算を扱うライブラリや処理系には、目的に応じていくつかの選択肢がある。非常に大きな整数や有理数を正確に扱う場合には、主に GMP を使用する。また、`double` 型より高い精度を比較的簡単に利用したい場合には、GMP の多倍長浮動小数点数を使用することができる。丸め方法の指定、正しく丸められた演算結果、非数や無限大の扱いなどが必要となる場合には MPFR が適している。さらに、多倍長複素数に対して同様の精度管理や丸め制御を必要とする場合には MPC を使用する。

本書では、広く利用されており相互の関係も深い GMP, MPFR, MPC を取り上げる。まず、C 言語形式の API による基本的な使用方法を解説し、C++ 用のインターフェイスが提供されている場合は、C++ から利用する方法についても説明する。

■ GMP/MPFR/MPC のインストール方法について

GMP/MPFR/MPC は、いずれもオープンソースソフトウェア (OSS) であり、公開されているソースコードを入手して、自分でライブラリをビルドすることができる。一方、多くの OS やディストリビューション向けにバイナリパッケージも用意されており、インストーラやパッケージ管理機能を用いて導入することもできる。ただし、処理環境によっては、実行用のライブラリと、ヘッダファイルやリンクに必要なファイルを含む開発用パッケージが分けて提供されていることがある。この場合、C/C++ のプログラムから利用するためには、開発用パッケージもインストールする必要がある。具体的な導入方法については、各ライブラリの配布元や、使用する OS およびパッケージ管理機能の情報を参照されたい。

22.4.1 GMP

GMP は、多倍長整数、有理数、浮動小数点数を扱うための型と関数を提供している。GMP の基本的な機能は C 言語の API として定義されており、扱う数値の種類ごとに、型と関数名の接頭辞が統一されている。

22.4.1.1 GMP の主な型

GMP の主な型を表 61 に示す。

表 61: GMP が提供する主な数値型

数値の種類	型	関数名の接頭辞
多倍長整数	<code>mpz_t</code>	<code>mpz_</code>
多倍長有理数	<code>mpq_t</code>	<code>mpq_</code>
多倍長浮動小数点数	<code>mpf_t</code>	<code>mpf_</code>

型名に含まれる `z`, `q`, `f` は、それぞれ整数、有理数、浮動小数点数を表している。例えば、多倍長整数を扱う型は `mpz_t` であり、この型に対する関数は `mpz_init()`, `mpz_set()`, `mpz_add()` など、`mpz_` で始まる名前を持つ。同様に、有理数に対する関数は `mpq_`、浮動小数点数に対する関数は `mpf_` で始まる。本書では、数値の種類を表す部分をアスタリスク「*」に置き換えて、これらの関数を総称することがある。例えば、`mpz_clear()`, `mpq_clear()`, `mpf_clear()` をまとめて `mp*_clear()` と表記する。

GMP の数値型は、C 言語の `int` や `double` のような単純な基本型ではない。内部では、数値を保持するための記憶領域が必要に応じて動的に確保される。このため、変数を宣言した後に初期化関数を実行し、使用後は解放関数を実行する必要がある。GMP の基本的な処理の流れは、概略として次のようになる。

- 1) 数値型の変数を宣言する
- 2) 初期化関数を実行する
- 3) 値を設定する
- 4) 演算関数を実行する
- 5) 結果を出力する／組み込み数値型や文字列に変換する
- 6) 解放関数を実行する

22.4.1.2 GMP の主な関数

GMP の主な関数の種類を表 62 に示す。

表 62: GMP の主な基本関数			
処 理	整 数	有理数	浮動小数点数
初期化	<code>mpz_init()</code>	<code>mpq_init()</code>	<code>mpf_init()</code> *
解放	<code>mpz_clear()</code>	<code>mpq_clear()</code>	<code>mpf_clear()</code>
値の設定	<code>mpz_set()</code>	<code>mpq_set()</code>	<code>mpf_set()</code>
加算	<code>mpz_add()</code>	<code>mpq_add()</code>	<code>mpf_add()</code>
減算	<code>mpz_sub()</code>	<code>mpq_sub()</code>	<code>mpf_sub()</code>
乗算	<code>mpz_mul()</code>	<code>mpq_mul()</code>	<code>mpf_mul()</code>
除算	丸め方法別の複数の関数	<code>mpq_div()</code>	<code>mpf_div()</code>
比較	<code>mpz_cmp()</code>	<code>mpq_cmp()</code>	<code>mpf_cmp()</code>

* `mpf_t` の初期化と精度の設定については後で解説する

例えば、多倍長整数の和を得る場合は、次のように `mpz_add()` を使用する。

書き方： `mpz_add( 結果, 値 1, 値 2 )`

「値 1」と「値 2」を加算し、「結果」に格納する。このとき、「結果」の変数は初期化済みでなければならない。

例. 多倍長整数 `x`, `y` の加算結果を `z` に得る

`mpz_add( z, x, y );`

GMP の演算関数では、通常、演算結果を格納する変数を第 1 引数に指定し、その後に演算対象となる値を指定する。この例では、`x+y` の結果が `z` に格納される。通常の C/C++ の式である `z=x+y` とは形式が異なるので注意が必要である。

■ 値の設定と変換について

先の表 62 に挙げた値の設定用の関数 (`*_set()`) は、同じ型の変数間で値をコピーするものである。C 言語の数値型の値や文字列を GMP の数値型に変換して設定するには、表 63 に挙げる関数を用いる。

関数名の末尾に付く `si` は signed integer, `ui` は unsigned integer, `d` は double, `str` は string を表す。`si`, `ui` を持つ関数が受け取る整数型は、それぞれ signed long int, unsigned long int である。

`mpq_set_si()` や `mpq_set_ui()` で有理数を設定する場合、分母には 0 以外の値を指定する。また、設定した分子と分母に共通因数がある場合、その有理数は自動的に約分されない。演算に使用する前に `mpq_canonicalize()` を実行し、分子と分母が互いに素となる標準的な形に変換する必要がある。

`mp*_set_str()` は、文字列全体を指定された基数の数値として正しく変換できた場合は 0 を返し、変換できなかった場合は -1 を返す。

表 63: GMP の数値型に値を設定する主な関数

関 数	解 説
<code>mpz_set_si(x, n)</code>	<code>signed long int</code> 型の値 <code>n</code> を多倍長整数 <code>x</code> に設定する
<code>mpz_set_ui(x, n)</code>	<code>unsigned long int</code> 型の値 <code>n</code> を多倍長整数 <code>x</code> に設定する
<code>mpz_set_d(x, d)</code>	<code>double</code> 型の値 <code>d</code> の小数部分を 0 の方向へ切り捨て、 <code>x</code> に設定する
<code>mpz_set_str(x, s, base)</code>	文字列 <code>s</code> を基数 <code>base</code> で整数に変換して <code>x</code> に設定する
<code>mpq_set_si(q, a, b)</code>	<code>signed long int</code> 型の値 <code>a</code> を分子、 <code>unsigned long int</code> 型の値 <code>b</code> を分母として有理数 <code>q</code> に設定する
<code>mpq_set_ui(q, a, b)</code>	<code>unsigned long int</code> 型の値 <code>a</code> , <code>b</code> を分子、分母として有理数 <code>q</code> に設定する
<code>mpq_set_str(q, s, base)</code>	文字列 <code>s</code> を基数 <code>base</code> で有理数に変換して <code>q</code> に設定する
<code>mpf_set_si(f, n)</code>	<code>signed long int</code> 型の値 <code>n</code> を多倍長浮動小数点数 <code>f</code> に設定する
<code>mpf_set_ui(f, n)</code>	<code>unsigned long int</code> 型の値 <code>n</code> を多倍長浮動小数点数 <code>f</code> に設定する
<code>mpf_set_d(f, d)</code>	<code>double</code> 型の値 <code>d</code> を多倍長浮動小数点数 <code>f</code> に設定する
<code>mpf_set_str(f, s, base)</code>	文字列 <code>s</code> を基数 <code>base</code> で浮動小数点数に変換して <code>f</code> に設定する

表 64～66 に、GMP の各数値型から値を取り出す主な関数を示す。

表 64: 多倍長整数から値を取り出す主な関数

関 数	変換先または用途
<code>mpz_get_si(z)</code>	<code>signed long int</code> 型の値を返す
<code>mpz_get_ui(z)</code>	<code>unsigned long int</code> 型の値を返す
<code>mpz_get_d(z)</code>	<code>double</code> 型の値を返す
<code>mpz_get_str(s, base, z)</code>	基数 <code>base</code> の文字列に変換する

表 65: 多倍長有理数から値を取り出す主な関数

関 数	変換先または用途
<code>mpq_get_d(q)</code>	<code>double</code> 型の値を返す
<code>mpq_get_str(s, base, q)</code>	基数 <code>base</code> の分数形式の文字列に変換する
<code>mpq_get_num(num, q)</code>	分子を初期化済みの <code>mpz_t</code> 型変数 <code>num</code> にコピーする
<code>mpq_get_den(den, q)</code>	分母を初期化済みの <code>mpz_t</code> 型変数 <code>den</code> にコピーする

表 66: 多倍長浮動小数点数から値を取り出す主な関数

関 数	変換先または用途
<code>mpf_get_si(f)</code>	<code>signed long int</code> 型の値を返す
<code>mpf_get_ui(f)</code>	<code>unsigned long int</code> 型の値を返す
<code>mpf_get_d(f)</code>	<code>double</code> 型の値を返す
<code>mpf_get_str(s, exp, base, n, f)</code>	仮数部の数字列を取り出し、指数を <code>exp</code> に格納する

これらの表でも、関数名の末尾に付く `si` は `signed integer`, `ui` は `unsigned integer`, `d` は `double`, `str` は `string` を表す。例えば、`mpz_get_si( z )` は、多倍長整数 `z` を `signed long int` 型に変換して返す。

ただし、多倍長の値が変換先の型で表現できる範囲や精度を超えている場合は、元の値を完全には保持できない。`mpz_get_si()` や `mpz_get_ui()` では、変換先に収まらない場合、元の値の下位部分に基づく値が返される。また、`mpz_get_ui()` に負の値を与えた場合は、符号が無視される。一方、`mpf_get_si()` や `mpf_get_ui()` では、変換先の範囲を超えた場合の結果は未定義となる。これらの関数を安全に使用するには、必要に応じて `mpz_fits_slong_p()`, `mpz_fits_ulong_p()`, `mpf_fits_slong_p()` などを用いて、変換先の型に値が収まることを事前に確認する。

また、double 型に変換する関数では、必要に応じて 0 の方向への丸めが行われる。値が double 型で表現できる範囲を超える場合の結果は処理環境に依存し、無限大や 0 が返されることもある。

mp*_get_str() の第 1 引数に NULL を与えると、結果を格納するための記憶領域が GMP によって確保される。第 1 引数に文字配列を与える場合は、結果と文字列終端の '\0' を格納できる十分な大きさの領域を用意する必要がある。また、GMP によって確保された記憶領域は、不要になった後で適切に解放する必要がある。

GMP の異なる型の間で値を変換する主な関数を表 67 に挙げる。これらの関数では、第 1 引数に指定する変換先の変数が初期化済みでなければならない。

表 67: GMP の異なる型の間で値を変換する関数

変 換	関 数	解 説
mpq_t から mpz_t	mpz_set_q(z, q)	小数部分を 0 の方向へ切り捨てる ($z \leftarrow q$)
mpf_t から mpz_t	mpz_set_f(z, f)	小数部分を 0 の方向へ切り捨てる ($z \leftarrow f$)
mpz_t から mpq_t	mpq_set_z(q, z)	整数を分母 1 の有理数に変換する ($q \leftarrow z$)
mpf_t から mpq_t	mpq_set_f(q, f)	浮動小数点数が保持する値を有理数に厳密に変換する ($q \leftarrow f$)
mpz_t から mpf_t	mpf_set_z(f, z)	変換先の精度で浮動小数点数に変換する ($f \leftarrow z$)
mpq_t から mpf_t	mpf_set_q(f, q)	変換先の精度で浮動小数点数に変換する ($f \leftarrow q$)

mpq_set_f() は、mpf_t 型の変数が 2 進浮動小数点数として実際に保持している値を、厳密な有理数に変換する。一方、mpf_set_z() と mpf_set_q() では、変換先の mpf_t 型変数に設定されている精度に従って、値が 0 の方向へ丸められる。このため、元の値を厳密に保持できないことがある。

GMP には、多倍長整数や多倍長浮動小数点数の初期化と値の設定を同時に行う関数も用意されている。主な関数を表 68 に示す。

表 68: 初期化と値の設定を同時に行う主な関数

対象型	関 数	解 説
mpz_t	mpz_init_set(x, y)	x を初期化し、mpz_t 型の値 y をコピーする
mpz_t	mpz_init_set_si(x, n)	x を初期化し、signed long int 型の値 n を設定する
mpz_t	mpz_init_set_ui(x, n)	x を初期化し、unsigned long int 型の値 n を設定する
mpz_t	mpz_init_set_d(x, d)	x を初期化し、d の小数部分を 0 の方向へ切り捨てて設定する
mpz_t	mpz_init_set_str(x, s, base)	x を初期化し、文字列 s を基数 base で整数に変換して設定する
mpf_t	mpf_init_set(x, y)	x を初期化し、mpf_t 型の値 y を設定する
mpf_t	mpf_init_set_si(x, n)	x を初期化し、signed long int 型の値 n を設定する
mpf_t	mpf_init_set_ui(x, n)	x を初期化し、unsigned long int 型の値 n を設定する
mpf_t	mpf_init_set_d(x, d)	x を初期化し、double 型の値 d を設定する
mpf_t	mpf_init_set_str(x, s, base)	x を初期化し、文字列 s を基数 base で浮動小数点数に変換して設定する

mpf_init_set() 系の関数で初期化される変数の精度には、mpf_set_default_prec() で設定された既定の精度が使用される。従って、mpf_init_set(x, y) を実行しても、x が y と同じ精度になるとは限らない。

mpq_t 型には、初期化と値の設定を同時に行う対応関数は用意されていない。そのため、mpq_init() で初期化した後に、mpq_set_ui() などを用いて値を設定する。

表 68 の関数を用いると、初期化関数と値の設定関数を個別に実行する必要がない。例えば、次の 2 つの記述は同じ処理に対応する。

```
mpz_init( x );  mpz_set_si( x, 123 );    ←初期化と値の設定を個別に行う処理
mpz_init_set_si( x, 123 );              ←初期化と値の設定を同時に行う処理
```

注意) 表 68 の関数は、まだ初期化されていない変数に対して使用すること。既に初期化されている変数に対して再び使用してはならない。使用後には、通常の場合と同様に mp*_clear() を実行して、内部で確保された記憶領域を解放する必要がある。

`mpz_init_set_str()` と `mpf_init_set_str()` は、文字列からの変換に成功すると 0、失敗すると -1 を返す。変換に失敗した場合でも変換先の変数は初期化されているため、最後にそれぞれ `mpz_clear()`、`mpf_clear()` を実行する必要がある。

■ 有理数の分子、分母の扱い

`mpq_t` の値から分子、分母を取り出すには、`mpq_get_num`、`mpq_get_den` を使用する。

分子の取り出し： `mpq_get_num( num, q )`

分母の取り出し： `mpq_get_den( den, q )`

これらの関数は、`mpq_t` 型の値 `q` から分子、分母を取り出し、それぞれ `mpz_t` 型の変数 `num`、`den` にコピーする。この場合、`num`、`den` には初期化済みの変数を与える。

逆に、`mpz_t` 型の分子、分母を `mpq_t` 型の値に設定するには、`mpq_set_num`、`mpq_set_den` を使用する。

分子の設定： `mpq_set_num( q, num )`

分母の設定： `mpq_set_den( q, den )`

これらの関数は、`mpz_t` 型の値 `num`、`den` を、それぞれ `mpq_t` 型の値 `q` の分子、分母として設定する。ただし、分母には 0 以外の値を設定しなければならない。また、これらの関数は、設定した有理数を自動的に約分せず、分母の符号も自動的に整理しない。そのため、分子と分母を設定した後、演算に用いる前に、次のように `mpq_canonicalize()` を実行する。

```
mpq_canonicalize( q );
```

これにより、分子と分母が互いに素となるように約分され、分母が正になるように符号が整理される。

22.4.1.3 多倍長浮動小数点数の精度の設定と初期化

`mpf_t` 型の変数では、値を保持するための記憶領域だけでなく、浮動小数点数を何ビットの精度で保持するかも初期化時に設定される。ここでいう精度は、小数点以下の桁数ではなく、数値の仮数部を保持するためのビット数である。例えば、十進数でおおよそ d 桁の精度を得るために必要となるビット数 b は、概略として次の関係から見積もることができる。

$$d \simeq b \log_{10} 2 \simeq 0.301b$$

従って、100 桁程度の精度を得るには約 333 ビットが必要となる。実際の計算では、途中の演算による誤差や余裕を考慮して、これより大きい精度（例えば 400 ビット程度）を指定するとよい。

`mpf_t` 型の主な初期化関数と精度設定関数を表 69 に示す。

表 69: `mpf_t` の初期化と精度設定に関する主な関数

関 数	解 説
<code>mpf_init(x)</code>	既定の精度で <code>x</code> を初期化し、値を 0 にする
<code>mpf_init2(x, prec)</code>	少なくとも <code>prec</code> ビットの精度で <code>x</code> を初期化し、値を 0 にする
<code>mpf_set_default_prec(prec)</code>	以後の初期化で使用する既定の精度を設定する
<code>mpf_get_default_prec()</code>	現在の既定の精度をビット数で返す
<code>mpf_get_prec(x)</code>	<code>x</code> に設定されている精度をビット数で返す
<code>mpf_set_prec(x, prec)</code>	初期化済みの <code>x</code> の精度を、少なくとも <code>prec</code> ビットに変更する

精度を変数ごとに指定するには、`mpf_init2` を使用する。

書き方： `mpf_init2( 変数, 精度のビット数 )`

例えば、次の記述では、`x` を少なくとも 400 ビットの精度で初期化し、初期値を 0 に設定する。

```
mpf_init2( x, 400 );
```

GMP の浮動小数点演算では、演算結果の精度は、結果を格納する変数に設定された精度によって決まる。従って、高い精度で計算する場合には、入力値だけでなく、演算結果を格納する変数も十分な精度で初期化しておく必要がある。

GMP は、内部で数値を `limb` と呼ばれる単位に分けて保持しているため、実際に設定される精度は、指定したビット数を `limb` の整数倍に切り上げた値となる¹⁹⁸。従って、引数 `prec` は厳密な精度ではなく、「少なくとも必要とする

¹⁹⁸`limb` は、GMP が多倍長数値を内部で保持する際の基本単位であり、通常は 32 ビットまたは 64 ビットである。

精度」を指定するものと考えとよい。実際に設定された精度は、`mpf_get_prec` を用いて調べることができる。

```
printf( "precision = %lu bits\n", (unsigned long)mpf_get_prec( x ) );
```

複数の `mpf_t` 型変数を同じ精度で初期化する場合には、あらかじめ `mpf_set_default_prec()` を用いて既定の精度を設定し、その後で `mpf_init()` を実行する。

```
mpf_set_default_prec( 400 );
mpf_init( x );
mpf_init( y );
```

`mpf_set_default_prec()` による設定は、その後に `mpf_init()` や `mpf_init_set()` 系の関数で初期化される変数に適用される。既に初期化済みの変数の精度は変更されない。現在の既定の精度は、`mpf_get_default_prec()` によって調べることができる。

```
mp_bitcnt_t p = mpf_get_default_prec();
```

個々の変数に異なる精度を指定したい場合には `mpf_init2()` を使用する。多数の変数を同じ精度で使用したい場合には、既定の精度を設定してから `mpf_init()` を使用するとよい。ただし、プログラムの途中で既定精度を変更すると、それ以後に初期化される変数の精度が変わるため、変数ごとの精度を明確にしたい場合には `mpf_init2()` の方が分かりやすい。

初期化済みの変数の精度は、`mpf_set_prec` を用いて変更することができる。

書き方： `mpf_set_prec( 変数, 精度のビット数 )`

例えば、次の記述では、`x` の精度を少なくとも 800 ビットに変更する。

```
mpf_set_prec( x, 800 );
```

ただし、精度を変更しても、それ以前に低い精度で計算された値の失われた桁が復元されるわけではない。また、精度を小さくした場合には、現在保持されている値が新しい精度に合わせて切り詰められる。高い精度が必要であることがあらかじめ分かっている場合には、計算を始める前に十分な精度で初期化しておく方がよい。

`mpf_set_prec` は、内部の記憶領域を再確保するため、繰り返しの途中で頻繁に実行する用途には適さない。反復計算の途中で精度を段階的に変更するような特殊な用途を除き、通常は初期化時に精度を決定しておく。

なお、GMP には、記憶領域を再確保せずに一時的に精度を変更する `mpf_set_prec_raw` も用意されている。ただし、使用できる精度に制約があり、解放前には元の精度に戻す必要があるなど、扱いを誤ると正常に動作しない可能性がある。従って、通常のプログラムでは `mpf_init2` または `mpf_set_prec` を使用する。

`mpf_t` 型の変数は、原則として一度だけ初期化する。既に初期化されている変数に対して、再び `mpf_init()` や `mpf_init2()` を実行してはならない。使用後には必ず `mpf_clear()` を実行し、内部で確保された記憶領域を解放する。

```
mpf_clear( x );
```

22.4.1.4 `mpf_t` の数学的な演算のための関数

GMP では、`mpf_t` 型の値に対して、平方根、整数乗、絶対値、整数への丸めなどを行う関数が用意されている。主な関数を表 70 に示す。

表の関数のうち、第 1 引数に `rop` を取る関数では、初期化済みの `mpf_t` 型変数を `rop` に与える。計算結果の精度は、`rop` に設定されている精度によって決まる。なお、`mpf_ceil`, `mpf_floor`, `mpf_trunc` の結果は整数値となるが、結果を格納する変数の型は `mpf_t` のままである。

例. 2 の平方根を少なくとも 400 ビットの精度で求める

```
mpf_t x;
mpf_init2(x, 400);
mpf_sqrt_ui(x, 2);
mpf_clear(x);
```


表 70: mpf_t 型に対する主な数学的演算関数

関 数	解 説
mpf_sqrt(rop, op)	0 以上の op の平方根を rop に格納する
mpf_sqrt_ui(rop, n)	非負整数 n の平方根を rop に格納する
mpf_pow_ui(rop, op, n)	op の非負整数乗を rop に格納する
mpf_abs(rop, op)	op の絶対値を rop に格納する
mpf_neg(rop, op)	op の符号を反転した値を rop に格納する
mpf_ceil(rop, op)	op 以上の最小の整数値を rop に格納する
mpf_floor(rop, op)	op 以下の最大の整数値を rop に格納する
mpf_trunc(rop, op)	op の小数部分を 0 の方向へ切り捨てた値を rop に格納する
mpf_integer_p(op)	op が整数値である場合に 0 以外を返す

mpf_pow_ui の指数は unsigned long int 型の非負整数であり、任意の実数を指数とする累乗を計算する関数ではない。また、GMP には mpf_t 型を対象とする三角関数、指数関数、対数関数などは用意されていない。これらの数学関数を多倍長精度で使用する場合は、MPFR などのライブラリを利用する。

22.4.1.5 比較, 判定のための関数

GMP では、mpz_t, mpq_t, mpf_t 型の値について、符号や大小関係、値の性質などを判定するための関数が用意されている。大小を比較する関数は、一般に、第 1 引数が第 2 引数より大きい場合に正の値、等しい場合に 0、小さい場合に負の値を返す。一方、値の性質を調べる判定関数は、条件を満たす場合に 0 以外を、満たさない場合に 0 を返す。

mpz_t 型の整数に対する主な判定関数を表 71 に示す。

表 71: mpz_t 型の値を判定する主な関数

関 数	解 説
mpz_sgn(x)	x の符号に応じて、正なら 1、ゼロなら 0、負なら -1 を返す
mpz_cmp(x, y)	大小比較。 $x > y \rightarrow$ 正の値, $x = y \rightarrow 0$, $x < y \rightarrow$ 負の値を返す
mpz_cmp_ui(x, y)	x と unsigned long int 型の値 y を比較する (判定は mpz_cmp に準じる)
mpz_cmp_si(x, y)	x と signed long int 型の値 y を比較する (判定は mpz_cmp に準じる)
mpz_even_p(x)	x が偶数であるかを判定する
mpz_odd_p(x)	x が奇数であるかを判定する
mpz_fits_ulong_p(x)	x を unsigned long int 型で表せるかを判定する
mpz_fits_slong_p(x)	x を signed long int 型で表せるかを判定する
mpz_perfect_square_p(x)	x が完全平方数であるかを判定する
mpz_perfect_power_p(x)	x が整数のべき乗 (2 乗以上) として表せるかを判定する
mpz_probab_prime_p(x, reps)	x が素数である可能性を判定する

例えば、mpz_sgn(x) の戻り値を調べると、x が正、ゼロ、負のいずれであるかを判定できる。

```
int s = mpz_sgn(x);
if (s > 0) {
    /* x は正 */
} else if (s < 0) {
    /* x は負 */
} else {
    /* x は 0 */
}
```

また、mpz_get_ui や mpz_get_si によって組込み整数型へ変換する前に、mpz_fits_ulong_p や mpz_fits_slong_p を使用すると、変換先の型で値を表せるかどうかを確認できる。int 型や short 型などについても同様に、mpz_fits_sint_p, mpz_fits_uint_p, mpz_fits_ushort_p が用意されている。

`mpz_probab_prime_p` は、素数判定を行う関数である。戻り値が 0 の場合は合成数、1 の場合は素数である可能性が高い値、2 の場合は素数であることが確定した値を表す。第 2 引数 `reps` には、確率的な判定の強さを指定する。大きな値を指定するほど、合成数を素数であると誤判定する可能性が低くなる。

`mpq_t` 型の有理数に対する主な判定関数を表 72 に示す。

表 72: `mpq_t` 型の値を判定する主な関数

関 数	解 説
<code>mpq_sgn(x)</code>	x の符号に応じて、正なら 1、ゼロなら 0、負なら -1 を返す
<code>mpq_cmp(x, y)</code>	大小比較。 $x > y \rightarrow$ 正の値, $x = y \rightarrow 0$, $x < y \rightarrow$ 負の値を返す
<code>mpq_cmp_z(x, y)</code>	x と <code>mpz_t</code> 型の整数 y を比較する (判定は <code>mpq_cmp</code> に準じる)
<code>mpq_cmp_ui(x, n, d)</code>	x と非負の有理数 n/d を比較する ($d > 0$, 判定は <code>mpq_cmp</code> に準じる)
<code>mpq_cmp_si(x, n, d)</code>	x と有理数 n/d を比較する ($d > 0$, 判定は <code>mpq_cmp</code> に準じる)
<code>mpq_equal(x, y)</code>	x と y が等しいかを判定する

2 つの有理数が等しいかどうかだけを調べる場合は、`mpq_cmp` の戻り値を 0 と比較することもできるが、`mpq_equal` を使用の方が効率がよい。なお、`mpq_t` 型の値は、原則として分子と分母が互いに素で、分母が正である正規形にしてから演算や比較に使用する。必要に応じて、事前に `mpq_canonicalize` を呼び出す。

`mpf_t` 型の浮動小数点数に対する主な判定関数を、表 73 に示す。

表 73: `mpf_t` 型の値を判定する主な関数

関 数	解 説
<code>mpf_sgn(x)</code>	x の符号に応じて、正なら 1、ゼロなら 0、負なら -1 を返す
<code>mpf_cmp(x, y)</code>	大小比較。 $x > y \rightarrow$ 正の値, $x = y \rightarrow 0$, $x < y \rightarrow$ 負の値を返す
<code>mpf_cmp_ui(x, y)</code>	x と <code>unsigned long int</code> 型の値 y を比較する (判定は <code>mpf_cmp</code> に準じる)
<code>mpf_cmp_si(x, y)</code>	x と <code>signed long int</code> 型の値 y を比較する (判定は <code>mpf_cmp</code> に準じる)
<code>mpf_cmp_d(x, y)</code>	x と <code>double</code> 型の値 y を比較する (判定は <code>mpf_cmp</code> に準じる)
<code>mpf_cmp_z(x, y)</code>	x と <code>mpz_t</code> 型の整数 y を比較する (判定は <code>mpf_cmp</code> に準じる)
<code>mpf_integer_p(x)</code>	x が整数値であるかを判定する
<code>mpf_fits_ulong_p(x)</code>	x を 0 の方向へ丸めて <code>unsigned long int</code> 型へ変換できるかを判定する
<code>mpf_fits_slong_p(x)</code>	x を 0 の方向へ丸めて <code>signed long int</code> 型へ変換できるかを判定する

`mpf_cmp_d` の第 2 引数には、`double` 型の正負の無限大を指定できるが、NaN を指定した場合の結果は規定されない。

`mpf_integer_p` は、 x の値が整数である場合に 0 以外を返す。例えば、 x が 3.0 や -5.0 を表している場合には 0 以外を返し、3.5 などの小数部分を持つ値では 0 を返す。ただし、これは数学的な計算結果が本来整数であるかを判定するものではなく、`mpf_t` オブジェクトに実際に格納されている値が整数値であるかを調べる関数である。

`mpf_fits_ulong_p` や `mpf_fits_slong_p` は、小数部分を 0 の方向へ丸めて整数型へ変換した結果が、変換先の型に収まるかを判定する。従って、値に小数部分がないことを判定する関数ではない。小数部分の有無は `mpf_integer_p` を用いて調べる。

`mpf_t` 型では NaN や無限大を扱わないため、MPFR の `mpfr_nan_p` や `mpfr_inf_p` に相当する判定関数は用意されていない。精度や丸め方法を厳密に管理し、特殊な値も含めて判定する必要がある場合は、MPFR を利用する。

なお、`mpz_sgn`, `mpq_sgn`, `mpf_sgn` をはじめ、一部の判定機能はマクロとして実装されている。マクロとして実装された判定機能には、インクリメントや関数の実行など、副作用を持つ式を引数として直接指定しない方がよい。

GMP では、異なる型の値を直接比較する関数も一部用意されている。例えば、`mpq_cmp_z` は `mpq_t` 型の有理数と `mpz_t` 型の整数を、`mpf_cmp_z` は `mpf_t` 型の浮動小数点数と `mpz_t` 型の整数を比較する。一方、`mpq_t` 型と `mpf_t` 型を直接比較する関数は用意されていない。厳密に比較したい場合は、`mpq_set_f` を用いて `mpf_t` 型の値を `mpq_t` 型に

変換してから比較することができる。逆に、有理数を `mpf_t` 型へ変換すると、指定された精度に応じた丸めが行われるため、厳密な比較ではなくなる場合があることに注意する必要がある。

22.4.1.6 `mpf_t` の値の近さの判定

浮動小数点数を用いた計算では、丸め誤差などの影響によって、本来は同じ値になるはずの2つの計算結果が、完全には一致しないことがある。このため、浮動小数点数同士を比較する場合は、単純に値が等しいかどうかを調べるのではなく、2つの値の差が十分に小さいかどうかを判定する方法が用いられる。

2つの値 x, y と、許容する差を表す非負の値 ε に対して、

$$|x - y| \leq \varepsilon$$

が成り立つ場合に、 x と y は近い値であるとみなすことができる。ここでは、この判定を行う関数 `mpfNear` を次のように定義する。

プログラム：mpfNear.c

```
1 #include "mpfNear.h"
2
3 int mpfNear( const mpf_t x, const mpf_t y, const mpf_t tol ) {
4     mp_bitcnt_t prec = mpf_get_prec(x);
5
6     if (mpf_get_prec(y) > prec)     prec = mpf_get_prec(y);
7     if (mpf_get_prec(tol) > prec)   prec = mpf_get_prec(tol);
8     if (mpf_sgn(tol) < 0)           return 0;
9
10    mpf_t diff;
11    mpf_init2(diff, prec);
12
13    mpf_sub(diff, x, y);
14    mpf_abs(diff, diff);
15
16    int result = (mpf_cmp(diff, tol) <= 0);
17
18    mpf_clear(diff);
19    return result;
20 }
```

この関数では、まず x, y, tol の中で最も高い精度を求め、差を格納する `diff` をその精度で初期化している。次に、`mpf_sub` によって $x - y$ を求め、`mpf_abs` によってその絶対値を得る。最後に、`mpf_cmp(diff, tol)` の結果が0以下であれば、差が許容値以下であるため1を返し、それ以外の場合は0を返す。なお、許容値 `tol` が負の場合は、正しい判定条件として使用できないため0を返している。

ただし、`diff` の計算自体も多倍長浮動小数点演算であるため、その精度に応じた丸めが行われる。 x, y, tol の最大の精度を使用することで、不必要に低い精度で計算されることは避けられるが、差が常に厳密に求められることを保証するものではない。特に、差が許容値に極めて近い場合には、丸めの影響を受ける可能性がある。

関数 `mpfNear` を他のプログラムから使用するために、上のプログラム `mpfNear.c` をコンパイルして、関数を含むオブジェクトファイル `mpfNear.o` を作成する。(次の例)

例. `mpfNear.c` をコンパイルして `mpfNear.o` を作成する

```
$ gcc -c mpfNear.c 
```

また、`mpfNear` 関数の宣言を記述したヘッダファイル `mpfNear.h` を次に示す。

ヘッダファイル：mpfNear.h

```
1 #ifndef MPF_NEAR_H
2 #define MPF_NEAR_H
3
4 #include <gmp.h>
5
6 int mpfNear(const mpf_t x, const mpf_t y, const mpf_t tol);
7
8 #endif
```

プログラム：test270-5.c

この例では、`x` と `y` の差が `tol` 以下であることを調べている。このプログラムをコンパイル、リンクして実行する例を次に示す。

```
$ gcc test270-5.c mpfNear.o -lgmp -o test270-5      Enter      ←コンパイルとリンク
$ ./test270-5      Enter      ←実行
x   = 3.1415926535897932384626433832795028842      ←出力
y   = 3.1415926535897932384626433832795028841
tol = 0.00000000000000000000000000000000000010
x と y は近い値です.
```

`mpfNear` は、値が完全に等しいかどうかではなく、指定した許容範囲内にあるかどうかを判定するための関数である。ただし、この方法は差の絶対値だけを用いる**絶対誤差による判定**である。値の大きさが広い範囲にわたる場合には、値の大きさに応じた相対誤差も考慮する必要がある。一方、ゼロに近い値を比較する場合には、**相対誤差**だけでは適切に判定できないことがあるため、絶対誤差も必要となる。実際の処理では、用途や値の範囲に応じて許容誤差の決め方を調整する。

22.4.1.7 書式整形と出力

GMP の数値型は、通常の `printf` 関数に `%d` や `%f`などを指定して出力することはできない。GMP は、多倍長数値を出力するために、標準 C ライブラリの `printf` 系関数を拡張した書式整形関数を提供している。また、数値を指定した基数の文字列としてストリームに直接出力する関数も用意されている。主な出力関数を表 74 に示す。

`gmp_printf` は、標準 C ライブラリの `printf` とほぼ同じ書き方で使用できる。通常の数値型や文字列に対する変換指定も使用でき、それらと GMP の数値型を同じ書式文字列の中に混在させることができる。GMP の数値型に対しては、表 75 に示す型指定子を変換指定子の中に付加する。

例. GMP の数値の書式整形出力

```
gmp_printf( "z = %Zd\n", z );
gmp_printf( "q = %Qd\n", q );
gmp_printf( "f = %.20Ff\n", f );
```

447

表 74: GMP が提供する主な書式整形・出力関数

関 数	解 説
<code>gmp_printf(書式, ...)</code>	標準出力に書式整形して出力する
<code>gmp_fprintf(stream, 書式, ...)</code>	指定したファイルストリームに書式整形して出力する
<code>gmp_sprintf(s, 書式, ...)</code>	書式整形した結果を文字配列 <code>s</code> に格納する
<code>gmp_snprintf(s, n, 書式, ...)</code>	終端文字を含めて最大 <code>n</code> バイトを文字配列 <code>s</code> に格納する
<code>gmp_asprintf(&s, 書式, ...)</code>	必要な記憶領域を動的に確保し、文字列へのポインタを <code>s</code> に設定する
<code>mpz_out_str(stream, base, z)</code>	多倍長整数 <code>z</code> を基数 <code>base</code> で出力する
<code>mpq_out_str(stream, base, q)</code>	多倍長有理数 <code>q</code> を基数 <code>base</code> で出力する
<code>mpf_out_str(stream, base, n, f)</code>	<code>f</code> の仮数部を基数 <code>base</code> , 最大 <code>n</code> 桁で出力する

表 75: GMP の書式整形で使用する主な型指定子

型指定子	対象となる型	記述例
<code>Z</code>	<code>mpz_t</code>	<code>%Zd</code>
<code>Q</code>	<code>mpq_t</code>	<code>%Qd</code>
<code>F</code>	<code>mpf_t</code>	<code>%Ff</code>

数点形式で出力する。

整数および有理数では、変換文字を変更することで、10 進数以外の形式でも出力できる。例えば、`%Zx` は多倍長整数を 16 進数、`%Zo` は 8 進数として出力する。

例. 多倍長整数を異なる進数で出力する例

```
gmp_printf( "decimal      : %Zd\n", z );
gmp_printf( "hexadecimal : %Zx\n", z );
gmp_printf( "octal       : %Zo\n", z );
```

`gmp_fprintf`, `gmp_sprintf`, `gmp_snprintf` も、出力先が異なるだけで、基本的な書式指定の方法は `gmp_printf` と同じである。 `gmp_sprintf` には出力先の大きさを制限する機能がないため、十分な大きさの領域が確保されていることを保証できる場合を除き、領域の大きさを指定できる `gmp_snprintf` を使用の方が安全である。

`gmp_asprintf` は、結果を格納するための記憶領域を動的に確保する。作成された文字列が不要になった後には、その記憶領域を解放する必要がある。GMP のメモリ管理関数を変更していない通常の場合は、標準ライブラリの `free` を用いて解放できる。GMP のメモリ管理関数を変更している場合は、その設定に対応した解放関数を使用する。

一方、`mpz_out_str`, `mpq_out_str`, `mpf_out_str` は、複雑な書式整形を行わず、GMP の数値を指定した基数で直接ストリームに出力する関数である。例えば、次の記述は、多倍長整数 `z` を 10 進数で標準出力に送出する。

例. 多倍長整数の直接出力

```
mpz_out_str( stdout, 10, z );
putchar( '\n' );
```

第 2 引数の `base` には、2 から 62 までの正の基数を指定できる。また、`-2` から `-36` までの値を指定すると、英字を大文字で出力することができる。有理数は 分子/分母の形式で出力され、分母が 1 の場合は分子だけが出力される。

`mpf_out_str` は、仮数部を最大 `n` 桁で出力する。 `n` に 0 を指定した場合は、`f` が保持している精度で正確に表現できる最大桁数が選択される。その出力は、仮数部と指数を組み合わせた形式となり、`gmp_printf` の `%Ff` による固定小数点形式とは異なる。

注意) `mpz_out_str` などのストリーム入出力関数を使用する場合は、`FILE` 型を使用するので、

次のように、ヘッダファイル `stdio.h` を `gmp.h` より先に読み込むこと。

```
#include <stdio.h>
#include <gmp.h>
```


参考)

`gmp_printf` は、標準 C ライブラリの `printf` とほぼ同じ書式指定機能を持ち、通常の C 言語の値と GMP の数値型を、同じ書式文字列の中に混在させて出力することができる。従って、GMP を使用するプログラムでは、`printf` の代わりに `gmp_printf` を用いることができる。

ただし、GMP の数値型に対しては、Z、Q、F などの専用の型指定子を使用する必要があり、一部のフラグや変換指定の扱いは標準の printf と異なる．通常の書式整形出力では gmp_printf を使用し、任意の基数で単純に値を出力したい場合には、mpz_out_str などを使用するとよい．

22.4.1.8 サンプルプログラム (1)：多倍長整数

多倍長整数を扱う例として，100 の階乗 $100!$ を求める．階乗は，1 から指定した整数までの値をすべて乗じたものであり，100 の階乗は次のように定義される．

$$100! = 1 \times 2 \times 3 \times \cdots \times 100$$

100! は 158 桁の非常に大きな整数となるため、通常の整数型で正確に保持することはできない。次のプログラムでは、GMP の多倍長整数型 `mpz_t` を用いて 100! を計算する。

プログラム：test270-1.c

```

1  #include      <stdio.h>
2  #include      <gmp.h>
3
4  int main() {
5      mpz_t f;
6      mpz_init_set_ui( f, 1 );
7
8      for ( unsigned long n=2; n<=100; n++ ) {
9          mpz_mul_ui( f, f, n );
10     }
11
12     gmp_printf( "100! = %Zd\n", f );
13
14     mpz_clear( f );
15 }

```

このプログラムでは、まず `mpz_t` 型の変数 `f` を宣言し、`mpz_init_set_ui` を用いて初期化すると同時に、初期値 1 を設定している。

```
mpz_init_set_ui( f, 1 );
```

次に、for 文を用いて 2 から 100 までの整数を順に `f` に乗じる。この処理には、多倍長整数と `unsigned long int` 型の値を乗算する関数 `mpz_mul_ui` を使用している。プログラム中の次の記述では、`f` と `n` の積を求め、その結果を再び `f` に格納している。

```
mpz_mul_ui( f, f, n );
```

GMP の演算関数では、演算結果の格納先を、演算対象と同じ変数にすることができる。従って、この記述によって、`f` の値は $1, 2, 2 \times 3, 2 \times 3 \times 4, \dots$ のように順に更新され、最終的に $100!$ が得られる。

計算結果は、`gmp_printf` と書式指定 `%zd` を用いて 10 進整数として出力している。最後に `mpz_clear` を実行し、変数 `z` の内部で確保された記憶領域を解放する例を次に示す。

例. test270-1.c のコンパイル, リンクと実行

[illegible]

なお、GMP には、非負整数の階乗を直接計算する `mpz_fac_ui` も用意されている。

22.4.1.9 サンプルプログラム (2) : 多倍長有理数

多倍長有理数を扱う例として、調和数 H_{233} を求める。調和数 H_n は、1 から n までの整数の逆数の和であり、次のように定義される。

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

通常の浮動小数点数型を用いてこの和を計算すると、計算結果は近似値となる。これに対して、GMP の多倍長有理数型 `mpq_t` を用いると、各項を分数として保持し、丸め誤差を生じることなく正確な結果を求めることができる。次のプログラムでは、 H_{233} を既約分数として計算する。

プログラム：test270-2.c

```
1  #include    <stdio.h>
2  #include    <gmp.h>
3
4  int main() {
5      mpq_t sum, term;
6
7      mpq_init( sum );
8      mpq_init( term );
9      mpq_set_ui( sum, 0, 1 );
10
11     for ( unsigned long n=1; n<=233; n++ ) {
12         mpq_set_ui( term, 1, n );
13         mpq_canonicalize( term );
14         mpq_add( sum, sum, term );
15     }
16
17     gmp_printf( "H_233 = %Qd\n", sum );
18     printf( "approximately = %.15f\n", mpq_get_d( sum ) );
19
20     mpq_clear( sum );
21     mpq_clear( term );
22 }
```

このプログラムでは、調和数の和を保持する変数 `sum` と、各項 $1/n$ を保持する変数 `term` を、それぞれ `mpq_t` 型として宣言している。これらを `mpq_init` で初期化した後、`mpq_set_ui` を用いて `sum` に初期値 $0/1$ を設定している。

```
mpq_set_ui( sum, 0, 1 );
```

`mpq_set_ui` の第2引数は分子、第3引数は分母を表す。従って、この記述では有理数 `sum` に 0 を設定している。なお、`mpq_init` による初期化直後の値も $0/1$ であるが、ここでは和の初期値を明示するために、改めて `mpq_set_ui` で 0 を設定している。

次に、`for` 文を用いて、 $1/1$ から $1/233$ までの各項を順に `sum` に加算する。次の記述では、`term` の分子を 1 、分母を n として設定している。

```
mpq_set_ui( term, 1, n );
```

`mpq_set_ui` によって設定された有理数は、一般には自動的に約分されない。そのため、分子と分母に共通因数がある可能性がある場合は、演算に使用する前に `mpq_canonicalize` によって正規化する必要がある。この例の $1/n$ は、分子が 1 で分母が正であるため、設定された時点ですでに正規形となっているが、ここでは正規化の手順を示すために次の処理を実行している。

```
mpq_canonicalize( term );
```

正規化された `term` は、`mpq_add` を用いて `sum` に加算する。プログラム中の次の記述では、`sum+term` の結果を再び `sum` に格納している。

```
mpq_add( sum, sum, term );
```

この処理を 233 回繰り返すことにより、`sum` には H_{233} の正確な値が既約分数として格納される。

計算結果は、`gmp_printf` と書式指定 `%Qd` を用いて分数形式で出力している。また、`mpq_get_d` を用いて `sum` を `double` 型に変換し、通常的小数による近似値も併せて出力している。

最後に、`mpq_clear` を実行し、`sum` と `term` の内部で確保された記憶領域を解放する。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test270-2.c のコンパイル, リンクと実行

```
$ gcc test270-2.c -lgmp -o test270-2  ←コンパイルと GMP のリンク
```

```
$ ./test270-2  ←実行
```

```
H_233 = 386400271638542006532852595456196202619177823114447563864314135283511318069181299
703738492559774996879/6407541246898738977994670450523269572785103006132671829683088161805
724539559635777747445509773488000
approximately = 6.030398506222030
```

22.4.1.10 サンプルプログラム (3) : 多倍長浮動小数点数

多倍長浮動小数点数を扱う例として, $\sqrt{2}$ を高い精度で求める. $\sqrt{2}$ は有限桁の小数や分数では正確に表すことのできない無理数であり, 通常の double 型で保持できる有効数字は十進数で 15 桁程度である. これに対して, GMP の多倍長浮動小数点数型 `mpf_t` では, 計算に使用する精度を指定することができる. 次のプログラムでは, $\sqrt{2}$ を小数点以下 100 桁まで表示する.

プログラム: test270-3.c

```
1  #include    <stdio.h>
2  #include    <gmp.h>
3
4  int main() {
5      mpf_t x;
6
7      mpf_init2( x, 400 );
8      mpf_sqrt_ui( x, 2 );
9
10     gmp_printf( "sqrt(2) = %.100Ff\n", x );
11
12     mpf_clear( x );
13 }
```

このプログラムでは, まず `mpf_t` 型の変数 `x` を宣言し, `mpf_init2` を用いて初期化している.

```
mpf_init2( x, 400 );
```

`mpf_init2` の第 2 引数には, 変数に設定する精度をビット数で指定する. この例では, `x` を少なくとも 400 ビットの精度で初期化している. 400 ビットは十進の有効数字でおよそ 120 桁に相当するため, 小数点以下 100 桁まで表示するための十分な精度を持つ.

次に, `mpf_sqrt_ui` を用いて, 整数 2 の平方根を求め, 結果を `x` に格納する. プログラム中の次の記述では, `unsigned long int` 型の値 2 の平方根を計算している.

```
mpf_sqrt_ui( x, 2 );
```

計算結果は, `gmp_printf` と書式指定 `%.100Ff` を用いて出力している. `F` は `mpf_t` 型の値を表す型指定子であり, `f` は固定小数点形式を表す. また, `.100` は小数点以下の表示桁数を 100 桁に指定している.

```
gmp_printf( "sqrt(2) = %.100Ff\n", x );
```

最後に, `mpf_clear` を実行し, 変数 `x` の内部で確保された記憶領域を解放する.

このプログラムをコンパイル, リンクして実行する例を次に示す.

例. test270-3.c のコンパイル, リンクと実行

```
$ gcc test270-3.c -lgmp -o test270-3  ←コンパイルと GMP のリンク
```

```
$ ./test270-3  ←実行
```

```
sqrt(2) = 1.414213562373095048801688724209698078569671875376948073176679737990732478462
1070388503875343276415727
```

22.4.1.11 `mpz_t` 型の乱数

GMP では, `mpz_t` 型の多倍長整数として一様乱数を生成する関数が用意されている. 乱数の生成には, 使用するアルゴリズムや現在の状態を保持する `gmp_randstate_t` 型のオブジェクトを使用する. 乱数生成に関する主な関数を表 76 に示す.

GMP の乱数生成関数が生成する値は, 決定的なアルゴリズムによって得られる疑似乱数である. 乱数を生成するには, まず `gmp_randstate_t` 型のオブジェクトを宣言し, `gmp_randinit_default` などを用いて初期化する.

表 76: GMP の乱数生成に関する主な関数

関 数	解 説
<code>gmp_randinit_default(state)</code>	乱数生成器 <code>state</code> を既定の方式で初期化する
<code>gmp_randinit_mt(state)</code>	Mersenne Twister 方式で <code>state</code> を初期化する
<code>gmp_randseed_ui(state, seed)</code>	<code>unsigned long int</code> 型の値を乱数の種として設定する
<code>gmp_randseed(state, seed)</code>	<code>mpz_t</code> 型の値を乱数の種として設定する
<code>gmp_randclear(state)</code>	乱数生成器 <code>state</code> が使用している領域を解放する
<code>mpz_urandomb(rop, state, n)</code>	0 以上 $2^n - 1$ 以下の一様乱数を <code>rop</code> に格納する
<code>mpz_urandomm(rop, state, limit)</code>	0 以上 <code>limit-1</code> 以下の一様乱数を <code>rop</code> に格納する

```
gmp_randstate_t state;
gmp_randinit_default(state);
```

`gmp_randinit_default` は、特別な要求がない場合に適した既定のアルゴリズムを選択して乱数生成器を初期化する。乱数生成アルゴリズムとして **Mersenne Twister** を明示する場合は、次のように `gmp_randinit_mt` を使用することもできる。

```
gmp_randinit_mt(state);
```

現在の GMP では、`gmp_randinit_default` でも Mersenne Twister が選択されるが、既定のアルゴリズムは将来の版で変更される可能性がある。通常は、`gmp_randinit_default` を使用するとよい。

乱数生成器を初期化した後、`gmp_randseed_ui` または `gmp_randseed` を用いて、乱数の種を設定する。例えば、現在時刻を種として使用する場合は、次のように記述できる。

```
gmp_randseed_ui(state, (unsigned long int)time(NULL));
```

この処理を行う場合は、`time` を使用するために `time.h` を読み込む必要がある。

```
#include <time.h>
```

同じアルゴリズムに同じ種を設定すると、同じ順序の疑似乱数が生成される。この性質は、計算結果を再現したい場合に利用できる。一方、実行ごとに異なる系列を生成したい場合は、異なる種を設定する必要がある。`time(NULL)` は通常、秒単位の時刻を返すため、短時間に複数回実行すると、同じ種が設定されて同じ系列が生成されることがある。また、現在時刻だけを種とする方法は推測されやすい。

GMP の標準的な乱数生成器は暗号学的な予測困難性を保証するものではないため、暗号鍵の生成などの用途には使用すべきではない。

`mpz_urandomb` は、指定したビット数に基づく非負の一様乱数を生成する関数である。例えば、次の処理では、0 以上 $2^{256} - 1$ 以下の一様乱数を生成し、`x` に格納する。

```
mpz_urandomb(x, state, 256);
```

第 3 引数には、乱数の生成に使用するビット数を指定する。生成される値の範囲は 0 以上 $2^n - 1$ 以下であり、生成された値が常にちょうど n ビットになるわけではない。例えば、先頭側のビットが 0 であれば、実際の値のビット数は n より小さくなる。ちょうど n ビットの正の整数が必要な場合は、生成後に最上位のビットを 1 に設定できる。

```
mpz_urandomb(x, state, n);
mpz_setbit(x, n - 1);
```

ただし、この処理では n が 1 以上でなければならない。

`mpz_urandomm` は、指定した上限未満の非負の一様乱数を生成する関数である。例えば、次の処理では、0 以上 999 以下の一様乱数を生成する。

```
mpz_set_ui(limit, 1000);
mpz_urandomm(x, state, limit);
```

第 3 引数には、正の `mpz_t` 型の上限値を指定する。生成される値は 0 以上であり、上限値そのものは含まれない。従って、 a 以上 b 以下の整数を一様に生成するには、まず 0 以上 $b - a$ 以下の乱数を生成し、その後で a を加える。例え

ば、100 以上 999 以下の乱数は次のように生成できる。

```
mpz_set_ui(limit, 900);
mpz_urandomm(x, state, limit);
mpz_add_ui(x, x, 100);
```

この場合、mpz_urandomm によって 0 以上 899 以下の値が生成され、100 を加えることで 100 以上 999 以下の値となる。

乱数の生成が終了した後は、gmp_randclear を実行して、乱数生成器が使用している領域を解放する。また、乱数を格納する mpz_t 型のオブジェクトも、通常どおり mpz_clear で解放する。

```
gmp_randclear(state);
mpz_clear(x);
mpz_clear(limit);
```

乱数生成処理の基本的な流れを次に例示する。

```
gmp_randstate_t state;
mpz_t x, limit;

gmp_randinit_default(state);
gmp_randseed_ui(state, (unsigned long int)time(NULL));
mpz_inits(x, limit, NULL);

mpz_set_ui(limit, 1000);
mpz_urandomm(x, state, limit);
gmp_printf("%Zd\n", x);

mpz_clears(x, limit, NULL);
gmp_randclear(state);
```

GMP には、2 進表現中に 0 や 1 が長く連続する特殊な値を生成する mpz_rrandomb も用意されている。この関数は、通常の一様乱数を得るためのものではなく、数値計算アルゴリズムなどのテストで、特殊なビットパターンを生成するために使用される。一般的な乱数の生成には、mpz_urandomb または mpz_urandomm を使用するとよい。

次に示すプログラム test270-4.c は、最大 30 桁の整数の一様乱数を 10 個生成して出力するものである。

プログラム：test270-4.c

```
1  #include    <stdio.h>
2  #include    <stdlib.h>
3  #include    <errno.h>
4  #include    <gmp.h>
5
6  int main( int argc, char *argv[] ) {
7      unsigned long seed;
8      char *end;
9
10     gmp_randstate_t state;
11     mpz_t x1, x2, limit;
12
13     if (argc != 2) {
14         fprintf(stderr, "usage: %s Seed\n", argv[0]);
15         return 1;
16     }
17
18     errno = 0;
19     end = NULL;
20     seed = strtoul( argv[1], &end, 10 );
21
22     if (errno == ERANGE || argv[1][0] == '-' || end == argv[1] || *end != '\0') {
23         fprintf(stderr, "Give Seed of unsigned long int!\n");
24         return 1;
25     }
26
27     /* 乱数生成器を初期化し、種を設定する */
28     gmp_randinit_default(state);
29     gmp_randseed_ui(state, seed);
30
31     /* 多倍長整数を初期化する */
32     mpz_inits(x1, x2, limit, NULL);
```



```

33
34     /* limit = 10^30 */
35     mpz_ui_pow_ui(limit, 10, 30);
36
37     /* 乱数を10個生成し, 5行2列で表示する */
38     for (int i = 0; i < 5; ++i) {
39         mpz_urandomm(x1, state, limit);      mpz_urandomm(x2, state, limit);
40         gmp_printf("%030Zd, %030Zd\n", x1, x2);
41     }
42
43     /* 使用した領域を解放する */
44     mpz_clears(x1, x2, limit, NULL);
45     gmp_randclear(state);
46 }

```

このプログラムでは、コマンドライン引数として与えられた値を乱数の種として設定する。まず、コマンドライン引数が1個であることを確認した後、`strtoul` を用いて引数を `unsigned long int` 型の値に変換する。引数が整数として解釈できない場合や、型の範囲を超える場合には、エラーメッセージを出力して処理を終了する。

次に、乱数の上限値として 10^{30} を `limit` に設定し、`mpz_urandomm` を用いて、0 以上 $10^{30} - 1$ 以下の整数乱数を生成する。このとき、乱数の上限値 10^{30} は、非負整数乗を求める `mpz_ui_pow_ui` を用いて算出している。

乱数は2個ずつ生成され、`gmp_printf` の書式 `%030Zd` によって、各値を30桁の0埋め形式として1行に2個ずつ出力する。最後に、多倍長整数のオブジェクトと乱数生成器が使用している領域を解放する。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. `test270-4.c` のコンパイル、リンクと実行

```

$ gcc test270-4.c -lgmp -o test270-4  ←コンパイルと GMP のリンク
$ ./test270-4 1234  ←種を 1234 として実行
490386359359480017971332714487, 328329684707142779870883755138 ←出力
247742890246959191192132975212, 895643232185475316915581106544
923297505956165444405203112352, 686089492733394783382047133727
949954708716640671480540179345, 475678756808114016086445048644
601991976372205148639802717497, 417372965759836575733550631876

```

注) この例と同じ種を引数に与えても、処理環境によっては異なる乱数列が得られる可能性がある。
ただし、同じ環境下では、同じ乱数生成アルゴリズムと種からは同じ乱数列が生成される。

22.4.1.12 C++における GMP の利用

GMP は C 言語を基本として開発されたライブラリであるが、C++ のプログラムからも利用することができる。C++ から GMP を使用方法には、これまでに説明した C 言語用の API をそのまま使用方法と、GMP が提供する C++ 用のクラスインターフェースを使用する方法がある。

C 言語用の API を使用する場合は、C 言語のプログラムと同様に `gmp.h` をインクルードし、`mpz_t`、`mpq_t`、`mpf_t` などの型と、`mpz_add`、`mpq_mul`、`mpf_div` などの関数を使用する。従って、これまでに説明した GMP の型や関数は、C++ のプログラムでも基本的に同じ方法で利用できる。

一方、GMP には、C++ のクラス、コンストラクタ、デストラクタ、演算子のオーバーロードなどの機能を利用した C++ 用のインターフェースも用意されている。このインターフェースでは、多倍長整数、多倍長有理数、多倍長浮動小数点数を、通常の C++ の数値型に近い記法で扱うことができる。また、オブジェクトの初期化と、使用する領域の確保および解放は、コンストラクタとデストラクタによって自動的に行われるため、初期化関数や解放関数を明示的に呼び出す必要がない。

C++ 用のインターフェースを使用する場合は、ヘッダファイル `gmpxx.h` をインクルードする。このヘッダには、多倍長整数を扱う `mpz_class`、多倍長有理数を扱う `mpq_class`、多倍長浮動小数点数を扱う `mpf_class` が定義されている。

また、コンパイル時には、C++ 用のライブラリ `gmpxx` と、その基盤となる GMP 本体のライブラリ `gmp` を、この順序でリンクする。例えば、プログラム `sample.cpp` をコンパイルする場合は、次のように指定する。

```
$ g++ sample.cpp -lgmpxx -lgmp -o sample
```

以降では、C 言語用の API との相違点を中心に、GMP の C++ 用インターフェースについて説明する。

■ GMP の C++ クラスインターフェース

GMP の C++ クラスインターフェースでは、C 言語用の `mpz_t`, `mpq_t`, `mpf_t` に対応して、表 77 に示すクラスが用意されている。これらのクラスのオブジェクトは、C++ の基本的な数値型の変数と同様に宣言し、値を与えて初期化することができる。

表 77: GMP の C 言語用の型と C++ 用のクラス

扱う数値	C 言語用の型	C++ 用のクラス
多倍長整数	<code>mpz_t</code>	<code>mpz_class</code>
多倍長有理数	<code>mpq_t</code>	<code>mpq_class</code>
多倍長浮動小数点数	<code>mpf_t</code>	<code>mpf_class</code>

例えば、これらのクラスのオブジェクトは、次のように作成することができる。

```
mpz_class z = 123456789;
mpq_class q(3, 7);
mpf_class f = 3.14159;
```

`mpz_class` のオブジェクト `z` には整数値が、`mpq_class` のオブジェクト `q` には分子が 3、分母が 7 の有理数が格納される。また、`mpf_class` のオブジェクト `f` には、浮動小数点リテラル 3.14159 を `double` 型の値として評価した結果が格納される。

なお、GMP の C++ クラスインターフェースは、`long`, `unsigned long`, `double` などとの演算に対応しているが、`long long` と `long double` は直接にはサポートしていない。これらの型の範囲や精度が必要な値は、必要に応じて文字列などを介して設定する。

この例の $q = 3/7$ は既に正規形である。分子と分母に共通因数がある値などを指定した場合は、演算に使用する前に、メンバ関数 `canonicalize` を実行して正規化する。

```
mpq_class q(6, 14);
q.canonicalize();
```

`mpf_class` の精度の指定や、高精度の 10 進文字列から値を設定する方法については後に説明する。

C 言語用の API では、オブジェクトを使用する前に `mpz_init`, `mpq_init`, `mpf_init` などを実行して初期化し、使用後は `mpz_clear`, `mpq_clear`, `mpf_clear` などを実行して、使用している領域を解放する必要がある。これに対して、C++ 用のクラスでは、オブジェクトの生成時にコンストラクタが初期化を行い、オブジェクトの寿命が終わるとデストラクタが使用している領域を自動的に解放する。従って、初期化関数や解放関数を明示的に呼び出す必要はない。

また、C++ 用のクラスは値の複製や代入にも対応している。例えば、次のようにオブジェクトを初期化したり、別のオブジェクトへ値を代入したりすることができる。

```
mpz_class a = 100;
mpz_class b = a;
mpz_class c;
c = b;
```

この例では、`b` は `a` の値によって初期化され、その後、`b` の値が `c` に代入される。各オブジェクトはそれぞれ独立して値を保持しており、内部で使用する領域の管理も自動的に行われる。このため、C 言語用の API に比べて、通常の C++ の数値型に近い感覚で多倍長数値を扱うことができる。

このように、オブジェクトの寿命に合わせて内部資源を管理する方法は **RAII** と呼ばれる。GMP の C++ 用クラスでは、この仕組みによって多倍長数値が使用する領域が自動的に管理される。

■ 値の設定と演算子による演算

GMP の C++ 用クラスでは、C++ の基本的な数値型と同様に、代入演算子 `=` を用いて値を設定することができる。また、加減乗除などの演算には、通常の算術演算子を使用することができる。例えば、`mpz_class` のオブジェクトに対する演算は、次のように記述する。

```

mpz_class a = 100;
mpz_class b = 30;
mpz_class c;

c = a + b;      // 加算
c = a - b;      // 減算
c = a * b;      // 乗算
c = a / b;      // 除算
c = a % b;      // 剰余

```

C 言語用の API では、これらの処理に `mpz_add`, `mpz_sub`, `mpz_mul`, `mpz_tdiv_q`, `mpz_tdiv_r` などの関数を使用するが、C++用のクラスでは、通常の整数型と同じ演算子で記述できる。

・除算における注意事項

`mpz_class` のオブジェクト同士に除算演算子 `/` を適用すると、整数除算が行われ、商は 0 の方向へ丸められる。従って、商の小数部分は保持されない。例えば、次の計算結果は 3 となる。

```

mpz_class a = 100, b = 30;
mpz_class c = a / b;

```

除算および剰余を求める場合の除数には、0 以外の値を指定しなければならない。また、`mpq_class` の分子と分母を直接指定する場合も、分母に 0 を指定してはならない。

除算結果を有理数として保持する場合は、除算を行う前に、演算対象の少なくとも一方を `mpq_class` の値として扱う必要がある。例えば、次のように記述すると、 $100/30 = 10/3$ が有理数として得られる。

```

mpq_class q = mpq_class(a) / b;

```

より明示的に、両方の値を `mpq_class` に変換してから除算してもよい。

```

mpq_class q = mpq_class(a) / mpq_class(b);

```

また、分子と分母を直接与えて、次のように有理数を作成することもできる。

```

mpq_class q(a, b);
q.canonicalize();

```

分子と分母を直接与えて作成した値が既約分数の形になっていない場合は、`canonicalize` を実行して正規化する必要がある。この例では、分子と分母の最大公約数によって約分され、`q` の値は $10/3$ となる。

・複合代入、インクリメント、比較

複合代入演算子も使用できる。例えば、次の処理では、`a` の値を順に変更している。

```

a += b;
a -= b;
a *= b;
a /= b;
a %= b;

```

また、`mpz_class` では、整数型と同様にインクリメント演算子 `++` とデクリメント演算子 `--` も使用できる。

```

a++;
--b;

```

比較にも、`==`, `!=`, `<`, `<=`, `>`, `>=` などの演算子を使用できる。従って、例えば次のように条件判定を記述することができる。

```

if (a < b) {
    ...
}

```

C 言語用の API では、値の比較に `mpz_cmp`, `mpq_cmp`, `mpf_cmp` などの関数を使用するが、C++用のクラスでは比較演算子によって、より自然な形で記述できる。

`mpq_class` および `mpf_class` についても、加減乗除や比較には同様の演算子を使用できる。例えば、次のように有理数および多倍長浮動小数点数の演算を記述することができる。

```
mpq_class q1(1, 3), q2(2, 5);
mpq_class q3 = q1 + q2;
mpf_class f1 = 1.25, f2 = 2.5;
mpf_class f3 = f1 * f2;
```

また、GMP の C++ 用クラスのオブジェクトは、C++ の整数型や浮動小数点型の値と組み合わせて演算することもできる。

```
mpz_class z = 100;
z = z * 20 + 3;
mpf_class f = 1.5;
f = f / 2.0;
```

このように、GMP の C++ 用クラスでは、値の設定、算術演算、比較などを、C++ の基本的な数値型に近い記法で表すことができる。ただし、演算結果の型や精度は、使用するクラス、演算対象の型、および結果を格納するオブジェクトの精度などに関係するため、整数、有理数、浮動小数点数のいずれとして計算するかを意識してオブジェクトを選ぶ必要がある。

■ 入出力

GMP の C++ 用クラスでは、C++ の標準ストリームを用いて値を入出力することができる。出力には挿入演算子 << を、入力には抽出演算子 >> を使用する。従って、C 言語用の `gmp_printf` や `gmp_scanf` などを使用する場合に比べて、C++ の基本的な数値型と同様の記法で入出力を行うことができる。例えば、次のように記述する。

```
mpz_class z = 123456789;
mpq_class q(3, 7);
mpf_class f = 1.25;

std::cout << z << std::endl;
std::cout << q << std::endl;
std::cout << f << std::endl;
```

この例では、`z` は整数として、`q` は分子と分母を斜線で区切った有理数として、`f` は浮動小数点数として出力される。

標準入力から値を読み取る場合も、次のように抽出演算子を使用する。

```
mpz_class z;
mpq_class q;
mpf_class f;

std::cin >> z;
std::cin >> q;
std::cin >> f;
```

`mpq_class` の値は、例えば `3/7` のように、分子と分母を `/` で区切って入力することができる。分子と分母の間や、`/` の前後に空白を入れることはできない。また、入力された分数が既約分数の形になっているとは限らないため、その後の演算に使用する場合は、必要に応じて `canonicalize` を実行して正規化する。

```
std::cin >> q;
q.canonicalize();
```

整数の入出力では、C++ の標準的な書式設定を利用して、基数を指定することもできる。例えば、`std::hex`、`std::oct`、`std::dec` を使用すると、16 進数、8 進数、10 進数による入出力を切り替えることができる。

```
mpz_class z = 255;
std::cout << std::dec << z << std::endl;
std::cout << std::hex << z << std::endl;
std::cout << std::oct << z << std::endl;
```

この場合、`z` の値は、順に 10 進数、16 進数、8 進数の形式で出力される。また、`std::showbase` を指定すると、16 進数では `0x`、8 進数では `0` などの接頭辞を付けて出力することができる。

```
std::cout << std::showbase << std::hex << z << std::endl;
```

多倍長浮動小数点数の出力では、`std::fixed`、`std::scientific`、`std::setprecision` などの書式設定を利用でき

る。 `std::setprecision` を使用する場合は、ヘッダファイル `iomanip` をインクルードする。

```
mpf_class f("3.14159265358979323846", 256);
std::cout << std::fixed << std::setprecision(20) << f << std::endl;
```

`std::fixed` または `std::scientific` を指定した場合、`std::setprecision` の引数は小数点以下の桁数を表す。一方、いずれも指定しない通常の形式では、出力する有効数字の桁数として扱われる。

なお、ストリームによる書式設定は、その後の出力にも継続して適用される。例えば、`std::hex` を指定した後で 10 進数による出力へ戻す場合は、`std::dec` を明示的に指定する必要がある。GMP の C++ 用クラスは、このような C++ 標準のストリーム機能と組み合わせて使用できるため、入出力処理を通常の C++ プログラムと同じ形式で記述することができる。

文字列ストリームに対しても、同じ形式で GMP の C++ 用クラスの値を入出力できる。文字列へ変換する場合は `std::ostringstream` を、文字列から値を読み取る場合は `std::istringstream` を使用する。これらを利用する場合は、ヘッダファイル `sstream` をインクルードする。

```
std::ostringstream out;
out << z;
std::string s = out.str();
std::istringstream in("123456789");
in >> z;
```

書式設定についても、標準出力ストリームの場合と同様に、`std::hex`、`std::fixed`、`std::setprecision` などを適用することができる。

■ `mpf_class` オブジェクトの精度の設定

`mpf_class` のオブジェクトは、C 言語用の `mpf_t` 型のオブジェクトと同様に、仮数部の精度をビット数で指定することができる。精度を明示してオブジェクトを作成する場合は、コンストラクタの第 2 引数にビット数を指定する。例えば、次のように記述する。

```
mpf_class x(0, 256);
mpf_class y("3.14159265358979323846", 256);
mpf_class z(1.25, 256);
```

この例では、`x`、`y`、`z` が、それぞれ 256 ビット以上の精度を持つオブジェクトとして作成される。第 1 引数には初期値を指定する。文字列を第 1 引数に指定すると、その文字列が表す値を、指定された精度で読み取ることができる。なお、GMP では内部表現の単位に合わせて精度が調整されるため、実際に設定される精度は、指定したビット数以上の値になることがある。

精度を省略した場合は、初期値の種類に応じて精度が決まる。整数型、浮動小数点型、`mpz_class`、`mpq_class` などの値から作成する場合は、GMP に設定されている既定の精度が使用される。例えば、次のオブジェクトは既定の精度で作成される。

```
mpf_class x = 1.25;
mpf_class y("3.14159265358979323846");
```

既定の精度は、C 言語用の API と同じく、`mpf_set_default_prec` を用いて変更することができる。

```
mpf_set_default_prec(256);
mpf_class x = 1.25;
```

この設定は、その後に精度を明示せずに作成されるオブジェクトに適用される。既に作成されているオブジェクトの精度は変更されない。また、既定精度はプログラム全体に影響するため、個々のオブジェクトについて必要な精度が明確である場合は、コンストラクタで精度を明示の方が分かりやすい。

オブジェクトに設定されている精度は、メンバ関数 `get_prec` によって取得できる。また、`set_prec` を使用すると、作成済みのオブジェクトの精度を変更できる。

```
mpf_class x(0, 256);
std::cout << x.get_prec() << std::endl;
x.set_prec(512);
```


`set_prec` によって精度を変更すると、内部領域の再確保が行われる。精度を小さくした場合は、その時点で格納されている値が新しい精度に合わせて切り詰められる。一方、精度を大きくしても、それ以前の計算で失われた桁が復元されるわけではない。従って、頻繁に繰り返される処理の中で精度を変更することは避け、必要な精度をオブジェクトの作成時に設定しておく方がよい。

`mpf_class` では、コピーによる初期化と、作成済みのオブジェクトへの代入とで、精度の扱いが異なる。例えば、次の例を考える。

```
mpf_class a("3.14159265358979323846", 256);
mpf_class b(a);
mpf_class c(0, 128);
c = a;
```

`b` は `a` の値によって新たに作成されるため、`a` と同じ精度を持つ。一方、代入演算子による `c=a` では、`c` の精度は変更されない。`a` の値が `c` の 128 ビットの精度に合わせて格納される。すなわち、代入は値だけを設定し、代入先の精度を変更しない。

演算結果を既存の `mpf_class` オブジェクトへ代入する場合も、基本的には代入先の精度が使用される。例えば、

```
mpf_class a(1, 256), b(3, 256);
mpf_class r(0, 128);
r = a / b;
```

とすると、除算結果は `r` の 128 ビットの精度で格納される。複合した式の計算で一時的な `mpf_class` の値が必要になる場合も、それらは最終的な代入先の精度に基づいて作成される。従って、高い精度で計算結果を得るには、演算対象だけでなく、結果を格納するオブジェクトにも十分な精度を設定しておく必要がある。

一方、演算式から新しいオブジェクトを直接作成する場合は、式を構成する値の精度に基づいて、新しいオブジェクトの精度が決まる。2 項演算では、基本的に精度の高い方の演算対象に合わせた精度が使用される。必要な精度を確実に指定したい場合は、次のようにコンストラクタの第 2 引数へ明示的にビット数を与えることができる。

```
mpf_class r(a / b, 512);
```

このように、`mpf_class` を使用する場合は、初期値だけでなく、オブジェクト自身が持つ精度と、演算結果を格納する先の精度にも注意する必要がある。

なお、`mpf_class` には、内部領域を再確保せずに現在の精度を変更する `set_prec_raw` も用意されている。この方法では、既に格納されている値は変更されず、その後に格納される値に新しい精度が適用される。ただし、指定できる精度は、初期化時または直前の `set_prec` によって確保された精度以下に限られる。また、オブジェクトを廃棄する前や `set_prec` を実行する前には、`set_prec_raw` を用いて、確保済みの領域に対応する元の精度へ戻す必要がある。誤った使用は予測できない結果を招く可能性があるため、通常の用途では `set_prec` またはコンストラクタによる精度指定を使用する方が安全である。

■ 注意事項

GMP の C++ 用クラスは、必要に応じて C 言語用の API と組み合わせて使用することもできる。ただし、両者を混在させると、プログラムの記述や資源管理が分かりにくくなる場合があるため、通常は C++ 用のクラスインターフェースに統一して使用する方がよい。C 言語用の API との相互利用が必要となる場合の方法については、GMP の公式文書を参照されたい。

また、GMP の C++ 用インターフェースでは、演算式を効率よく処理するために、内部で**式テンプレート**と呼ばれる仕組みが使用されている。このため、次のように GMP のオブジェクトを含む演算式を `auto` で受けると、変数 `x` が `mpz_class` ではなく、式そのものを表す内部的な型となる場合がある。

```
auto x = a + b;
```

このような値は、演算結果そのものではなく、演算に使用するオブジェクトへの参照を内部に保持している場合がある。このため、式を構成するオブジェクトの値が後から変更された場合や、その寿命が終了した場合に、意図しない結果を招く可能性がある。演算結果を保持する場合は、次のように目的のクラスを明示する方が安全である。

```
mpz_class x = a + b;
```

460

mpq_class term(1, n) によって $1/n$ を表す有理数を作成している。この例では、分子が1であり、分母 n も正であるため、作成された値は既に正規形となっている。従って、明示的に canonicalize を実行する必要はない。

調和数を保持する sum は、mpq_class 型のオブジェクトとして作成し、初期値 0 を設定している。各項 term は、複合代入演算子 += を用いて sum に加算する。

```
sum += term;
```

これは、C 言語版の mpq_add(sum, sum, term) に対応する処理である。計算結果は挿入演算子 << を用いて分数形式で出力している。また、sum.get_d() によって double 型の近似値を取得し、std::fixed と std::setprecision(15) を用いて小数点以下 15 桁まで出力している。オブジェクトの記憶領域は自動的に解放されるため、mpq_clear に相当する処理を明示する必要はない。

このプログラムは、次のようにしてコンパイル、リンクして実行する。

例. test270-2.cpp のコンパイル、リンクと実行

```
$ g++ test270-2.cpp -lgmpxx -lgmp -o test270-2 
$ ./test270-2  ←実行
H_233 = 386400271638542006532852595456196202619177823114447563864314135283511318069181299
703738492559774996879/6407541246898738977994670450523269572785103006132671829683088161805
7245395596357777747445509773488000
approximately = 6.030398506222030
```

先に示した test270-3.c (多倍長浮動小数点数による $\sqrt{2}$ の計算) と同じ仕様のプログラムを C++ 用にした test270-3.cpp を次に示す。

プログラム：test270-3.cpp

```
1  #include <iostream>
2  #include <iomanip>
3  #include <gmpxx.h>
4
5  int main() {
6      mpf_class x(2, 400);
7
8      x = sqrt(x);
9
10     std::cout << "sqrt(2) = " << std::fixed << std::setprecision(100)
11                << x << "\n";
12 }
```

mpf_class x(2, 400) によって、初期値を 2、精度を少なくとも 400 ビットとしてオブジェクトを作成している。その後、GMP の C++ 用インターフェースが提供する sqrt によって平方根を求め、結果を同じオブジェクトに格納している。

```
x = sqrt(x);
```

計算結果は、std::fixed と std::setprecision(100) を用いて、小数点以下 100 桁まで出力している。また、x の記憶領域は自動的に解放されるため、mpf_clear に相当する処理を明示する必要はない。

このプログラムは、次のようにしてコンパイル、リンクして実行する。

例. test270-3.cpp のコンパイル、リンクと実行

```
$ g++ test270-3.cpp -lgmpxx -lgmp -o test270-3 
$ ./test270-3  ←実行
sqrt(2) = 1.414213562373095048801688724209698078569671875376948073176679737990732478462
1070388503875343276415727
```

先に示した test270-4.c (多倍長整数による乱数生成) と同じ仕様のプログラムを C++ 用にした test270-4.cpp を次に示す。

プログラム：test270-4.cpp

```
1  #include <iostream>
2  #include <iomanip>
3  #include <string>
```

```

4 #include <stdexcept>
5 #include <gmpxx.h>
6
7 int main( int argc, char *argv[] ) {
8     if (argc != 2) {
9         std::cerr << "usage: " << argv[0] << " Seed\n";
10        return 1;
11    }
12
13    unsigned long seed;
14    std::size_t pos;
15
16    try {
17        std::string s = argv[1];
18
19        if (!s.empty() && s[0] == '-') {
20            throw std::invalid_argument("negative value");
21        }
22
23        seed = std::stoul(s, &pos, 10);
24
25        if (pos != s.size()) {
26            throw std::invalid_argument("invalid character");
27        }
28    }
29    catch (const std::exception&) {
30        std::cerr << "Give Seed of unsigned long int!\n";
31        return 1;
32    }
33
34    // 乱数生成器を初期化し，種を設定する
35    gmp_randclass random(gmp_randinit_default);
36    random.seed(seed);
37
38    // limit = 10^30
39    mpz_class limit;
40    mpz_ui_pow_ui(limit.get_mpz_t(), 10, 30);
41
42    // 乱数を10個生成し，5行2列で表示する
43    std::cout << std::setfill('0');
44
45    for (int i=0; i<5; i++) {
46        mpz_class x1 = random.get_z_range(limit);
47        mpz_class x2 = random.get_z_range(limit);
48
49        std::cout << std::setw(30) << x1 << ", "
50                << std::setw(30) << x2 << "\n";
51    }
52 }

```

このプログラムでは，`std::stoul` を用いてコマンドライン引数を `unsigned long int` 型の乱数の種に変換している．引数が整数として解釈できない場合や，型の範囲を超える場合には例外が発生するため，`try` 文と `catch` 節を用いてエラーを処理している．このように，C++では文字列の変換エラーを例外処理としてまとめて扱うことができる．

乱数生成器には，GMPのC++用インターフェースが提供する `gmp_randclass` を使用している．一方，上限値 10^{30} の計算には，`get_mpz_t` によって `mpz_class` の内部オブジェクトを取得し，C言語用の関数 `mpz_ui_pow_ui` を使用している．

```
mpz_ui_pow_ui(limit.get_mpz_t(), 10, 30);
```

このように，GMPのC++用クラスとC言語用のAPIは，必要に応じて同じプログラムの中で組み合わせて使用することができる．

このプログラムは，次のようにしてコンパイル，リンクして実行する．

例. test270-4.cpp のコンパイル, リンクと実行

```
$ g++ test270-4.cpp -lgmpxx -lgmp -o test270-4 Enter
$ ./test270-4 1234 Enter ←種を 1234 として実行
490386359359480017971332714487, 328329684707142779870883755138 ←出力
247742890246959191192132975212, 895643232185475316915581106544
923297505956165444405203112352, 686089492733394783382047133727
949954708716640671480540179345, 475678756808114016086445048644
601991976372205148639802717497, 417372965759836575733550631876
```

22.4.2 MPFR

GMP は、多倍長浮動小数点数を扱う型として `mpf_t` を提供している。 `mpf_t` は、通常の浮動小数点数型を超える精度で計算を行うための基本的な機能を備えているが、丸め方法を演算ごとに指定することはできず、数値計算における精度と誤差の厳密な管理には限界がある。

これに対して MPFR は、任意精度の浮動小数点計算を、より明確な規則の下で行うために開発されたライブラリである。 MPFR は GMP を基盤として実装されており、GMP の多倍長整数演算機能を内部で利用している。従って、MPFR を使用する処理環境には GMP も必要となる。

MPFR では、数値の仮数部を保持する精度をビット数で指定し、指定したビット数そのものを精度として使用する。これに対して、GMP の `mpf_t` では、指定した精度は最低限必要なビット数として扱われ、実際の精度は `limb` の整数倍に切り上げられる。従って、MPFR では、より明確で処理環境に依存しにくい精度管理を行うことができる。また MPFR では、加減乗除や数学関数の呼び出しごとに丸め方法を指定する。

これは、GMP の `mpf_t` では丸め方法を演算ごとに指定できず、内部で一時的により高い精度を用いる場合があるものの、最終的な結果は格納先の精度に合わせて 0 の方向へ切り詰められるのと異なる。このため MPFR は、計算結果の再現性や誤差の管理が重要となる科学技術計算や数値解析に適している。

GMP の `mpf_t` と比較した場合の、MPFR の主な特徴を次に挙げる。

- 1) 仮数部の精度を、指定したビット数そのもので管理できる
- 2) 演算ごとに丸め方法を指定できる
- 3) 指定した精度と丸め方法に従って、正しく丸められた計算結果を得ることができる
- 4) NaN (非数)、正負の無限大、符号付きゼロを扱うことができる
- 5) 初等関数をはじめとする数学関数が豊富に用意されている

`mpf_t` は、`double` 型よりも高い精度を比較的簡単に利用したい場合や、厳密な丸め制御や特殊値の扱いを必要としない用途で使うことができる。一方、演算ごとの丸め方法、正しく丸められた結果、NaN や無限大などの特殊値、豊富な数学関数などが必要となる場合には、MPFR の `mpfr_t` を使用するのが適している。従って、多倍長浮動小数点計算では、必要とする精度管理や機能に応じて、GMP の `mpf_t` と MPFR の `mpfr_t` を使い分けるとよい。

22.4.2.1 MPFR の主な型

MPFR で使用する主な型を表 78 に示す。

表 78: MPFR で使用する主な型	
型	解 説
<code>mpfr_t</code>	多倍長浮動小数点数を保持する型
<code>mpfr_prec_t</code>	浮動小数点数の精度をビット数で表す整数型
<code>mpfr_rnd_t</code>	演算結果の丸め方法を表す型
<code>mpfr_exp_t</code>	浮動小数点数の指数を表す整数型

MPFR の多倍長浮動小数点数は `mpfr_t` 型の変数に保持する。この型に対する関数は、`mpfr_init2`, `mpfr_set`, `mpfr_add`, `mpfr_sqrt` など、原則として `mpfr_` で始まる名前を持つ。

`mpfr_prec_t` は、変数の精度をビット数で指定したり、設定されている精度を取得したりする際に用いられる。`mpfr_rnd_t` は、演算結果をどの方向へ丸めるかを指定するために用いられる。丸め方法については後に詳しく説明する。

MPFR を使用するプログラム例 test271-1.c を次に示す。このプログラムでは、角度 $a = \pi/3$ に対して、 $x = \cos(a)$ 、 $y = \sin(a)$ を求め、さらに、三角関数の基本公式 $\cos^2(a) + \sin^2(a) = 1$ に基づいて、 $\sqrt{x^2 + y^2}$ の値を計算する。

各変数の精度は 256 ビットに設定し、円周率、三角関数、平方根などを高い精度で計算する。また、すべての演算では、丸め方法として最近接丸めを表す MPFR_RNDN を指定する。計算結果は、小数点以下 70 桁まで表示する。

プログラム：test271-1.c

```
1  #include    <stdio.h>
2  #include    <mpfr.h>
3
4  int main() {
5      mpfr_t a, x, y, x2, y2, s, r;
6
7      /* 各変数を256ビットの精度で初期化する */
8      mpfr_init2( a, 256 );
9      mpfr_init2( x, 256 );      mpfr_init2( y, 256 );
10     mpfr_init2( x2, 256 );      mpfr_init2( y2, 256 );
11     mpfr_init2( s, 256 );      mpfr_init2( r, 256 );
12
13     /* a = pi / 3 */
14     mpfr_const_pi( a, MPFR_RNDN );      // 円周率を
15     mpfr_div_ui( a, a, 3, MPFR_RNDN );  // 3で割る
16
17     /* x = cos(a), y = sin(a) */
18     mpfr_cos( x, a, MPFR_RNDN );        // 余弦関数
19     mpfr_sin( y, a, MPFR_RNDN );        // 正弦関数
20
21     /* r = sqrt(x^2 + y^2) */
22     mpfr_sqr( x2, x, MPFR_RNDN );        // それぞれを
23     mpfr_sqr( y2, y, MPFR_RNDN );        // 2乗して
24     mpfr_add( s, x2, y2, MPFR_RNDN );    // 加算したものの
25     mpfr_sqrt( r, s, MPFR_RNDN );        // 平方根を取る
26
27     /* 小数点以下70桁まで表示する */
28     mpfr_printf( "a = pi / 3 = %.70Rf\n", a );
29     mpfr_printf( "x = cos(a) = %.70Rf\n", x );
30     mpfr_printf( "y = sin(a) = %.70Rf\n", y );
31     mpfr_printf( "sqrt(x^2+y^2) = %.70Rf\n", r );
32
33     mpfr_clear( a );                    // 使用終了後は
34     mpfr_clear( x );      mpfr_clear( y );      // 全ての
35     mpfr_clear( x2 );      mpfr_clear( y2 );    // 変数を
36     mpfr_clear( s );      mpfr_clear( r );      // 解放する
37 }
```

このプログラムでは、角度と計算途中の値を保持する変数 $a, x, y, x2, y2, s, r$ を `mpfr_t` 型として宣言し、`mpfr_init2` によって 256 ビットの精度で初期化している。

円周率の取得には `mpfr_const_pi`、整数による除算には `mpfr_div_ui` を用いて、角度 $a = \pi/3$ を求める。次に、`mpfr_cos` と `mpfr_sin` によって $x = \cos(a)$ 、 $y = \sin(a)$ を計算する。さらに、`mpfr_sqr`、`mpfr_add`、`mpfr_sqrt` を用いて、 $\sqrt{x^2 + y^2}$ を求めている。

MPFR の多くの演算関数では、第 1 引数に計算結果を格納する変数を指定し、最後の引数に丸め方法を指定する。このプログラムでは、すべての演算に最近接丸めを表す `MPFR_RNDN` を指定している。また、入力と出力に同じ変数を使用できる関数では、計算結果を元の変数へ格納することもできる。

計算結果の出力には `mpfr_printf` を使用する。書式指定 `%.70Rf` は、`mpfr_t` 型の値を固定小数点形式で、小数点以下 70 桁まで表示することを表す。最後に、すべての変数に対して `mpfr_clear` を実行し、内部で確保された記憶領域を解放している。

このプログラムをコンパイル、リンクして実行する例を次に示す。

[illegible]

このように MPFR を使用すると，精度と丸め方法を明示しながら，基本演算や数学関数を高い精度で計算することができる．

MPFR の主なオブジェクト管理系関数を表 79 に挙げる。

関 数	解 説
<code>mpfr_init(x)</code>	<code>x</code> を既定の精度で初期化し、値を NaN にする
<code>mpfr_init2(x, prec)</code>	<code>x</code> を <code>prec</code> ビットの精度で初期化し、値を NaN にする
<code>mpfr_clear(x)</code>	<code>x</code> が使用している記憶領域を解放する
<code>mpfr_set_default_prec(prec)</code>	以後の <code>mpfr_init</code> で使用する既定の精度（ビット数）を設定する
<code>mpfr_get_default_prec()</code>	現在の既定の精度をビット数で返す
<code>mpfr_get_prec(x)</code>	<code>x</code> に設定されている精度をビット数で返す
<code>mpfr_set_prec(x, prec)</code>	<code>x</code> の精度を <code>prec</code> ビットに変更し、値を NaN にする
<code>mpfr_prec_round(x, prec, rnd)</code>	<code>x</code> の現在値を保持しながら、精度を <code>prec</code> ビットに変更する

```
mpfr_t x;
mpfr_init2( x, 256 );    ← 256 ビットの精度で初期化
...                      ← 値の設定, 演算, 変換, 出力など
mpfr_clear( x );         ← 記憶領域の解放
```

精度を変数ごとに明示する場合には, `mpfr_init2` を使用する.

```
mpfr_init2( x, 256 );
```

```
mpfr_prec_t p = mpfr_get_prec( x );
```

```
mpfr_set_default_prec( 256 );    ←既定の精度を 256 ビットにする
mpfr_init( x );                  ←この後、この精度が
mpfr_init( y );                  ←適用される
```

465

い。また、現在の既定の精度は `mpfr_get_default_prec` によって調べることができる。

参考) MPFR の既定の精度は、初期状態では 53 ビットに設定されている。

初期化済みの変数の精度を変更するには、`mpfr_set_prec` または `mpfr_prec_round` を使用する。ただし、両者の動作は異なる。

```
mpfr_set_prec( x, 512 );
mpfr_prec_round( x, 512, MPFR_RNDN );
```

`mpfr_set_prec` は、変数 `x` の精度を変更すると同時に、それまで保持していた値を失わせ、値を NaN にする。これに対して `mpfr_prec_round` は、現在値を保持したまま精度を変更する。精度を小さくする場合は、指定した丸め方法に従って値が丸められる。精度を大きくする場合は、値を変えずに精度だけが拡張される。従って、現在値を保持したまま精度を変更したい場合には、`mpfr_prec_round` を使用する。

ただし、低い精度で行われた計算によって既に失われた桁は、後から精度を高くしても復元されない。必要な精度があらかじめ分かっている場合には、計算を始める前に `mpfr_init2` を用いて十分な精度を指定しておくのが基本である。

重要) 一度初期化した変数に対して、初期化関数を重ねて実行してはならない。使用後は `mpfr_clear` で解放する。解放した変数を再び使用する場合には、改めて初期化を行う。

22.4.2.3 値の設定と取得のための関数

`mpfr_t` 型の変数に値を設定する場合には、値の型に応じて `mpfr_set`, `mpfr_set_d` などの関数を使用する。また、`mpfr_t` 型の値を通常の数値型や GMP の数値型へ変換して取り出す関数も用意されている。主な値設定関数を表 80 に示す。

表 80: MPFR に値を設定する主な関数

関 数	解 説
<code>mpfr_set(x, y, rnd)</code>	<code>mpfr_t</code> 型の値 <code>y</code> を <code>x</code> に設定する
<code>mpfr_set_si(x, n, rnd)</code>	<code>signed long int</code> 型の値 <code>n</code> を設定する
<code>mpfr_set_ui(x, n, rnd)</code>	<code>unsigned long int</code> 型の値 <code>n</code> を設定する
<code>mpfr_set_flt(x, d, rnd)</code>	<code>float</code> 型の値 <code>d</code> を設定する
<code>mpfr_set_d(x, d, rnd)</code>	<code>double</code> 型の値 <code>d</code> を設定する
<code>mpfr_set_ld(x, d, rnd)</code>	<code>long double</code> 型の値 <code>d</code> を設定する
<code>mpfr_set_z(x, z, rnd)</code>	<code>mpz_t</code> 型の整数 <code>z</code> を設定する
<code>mpfr_set_q(x, q, rnd)</code>	<code>mpq_t</code> 型の有理数 <code>q</code> を設定する
<code>mpfr_set_f(x, f, rnd)</code>	<code>mpf_t</code> 型の浮動小数点数 <code>f</code> を設定する
<code>mpfr_set_str(x, s, base, rnd)</code>	文字列 <code>s</code> を基数 <code>base</code> で数値に変換して設定する
<code>mpfr_set_zero(x, sign)</code>	符号 <code>sign</code> に応じて正または負の 0 を設定する ⁽¹⁾
<code>mpfr_set_inf(x, sign)</code>	符号 <code>sign</code> に応じて正または負の無限大を設定する ⁽²⁾
<code>mpfr_set_nan(x)</code>	<code>x</code> に NaN を設定する ⁽³⁾

(1)(2)(3) に関しては「22.4.2.14 `mpfr_t` の特殊な値」(p.483) で詳しく解説する。

値を設定する対象の変数 `x` は、あらかじめ `mpfr_init2` などによって初期化されている必要がある。例えば、次の記述では、`x` を 256 ビットの精度で初期化した後、`double` 型の値を設定している。

```
mpfr_t x;
mpfr_init2( x, 256 );
mpfr_set_d( x, 3.14159, MPFR_RNDN );
```

値設定関数の最後の引数 `rnd` には、値を `x` の精度に合わせる際の丸め方法を指定する。通常は、最も近い値への丸めを表す `MPFR_RNDN` を使用するとよい。設定元の値を `x` の精度で正確に表現できない場合には、指定した丸め方法に従って値が丸められる。

上の例で使用している浮動小数点リテラル `3.14159` は、まず `double` 型の値として評価される。一般的な 53 ビット精度の `double` 型では、256 ビットの精度を持つ `x` に、その `double` 型の値を正確に設定できるが、10 進数として

の 3.14159 が厳密に設定されるわけではない。多くの 10 進桁を持つ定数を高い精度で設定する場合は、文字列から値を設定する `mpfr_set_str` を使用する。

書き方： `mpfr_set_str( 変数, 文字列, 基数, 丸め方法 )`

例えば、次の記述では、10 進数の文字列を数値に変換して `x` に設定する。

```
mpfr_set_str( x, "3.14159265358979323846", 10, MPFR_RNDN );
```

`mpfr_set_str` は、変換に成功すると 0 を返し、文字列全体を数値として解釈できない場合には `-1` を返す。変換に失敗した場合には、変換先の値が変更されている可能性がある。外部から与えられた文字列を処理する場合には、次のように戻り値を確認し、変換に失敗したときの値を使用しないようにする。

```
if ( mpfr_set_str( x, s, 10, MPFR_RNDN ) != 0 ) {
    /* 変換に失敗した場合の処理 */
}
```

`mpfr_set_zero`, `mpfr_set_inf` の引数 `sign` には、符号を表す整数を与える。0 以上の値を与えると正の 0 または正の無限大、負の値を与えると負の 0 または負の無限大が設定される。

MPFR の値を通常の数値型や GMP の数値型へ変換して取り出す主な関数を、表 81 に示す。

表 81: MPFR の値を取り出す主な関数

関 数	解 説
<code>mpfr_get_flt(x, rnd)</code>	<code>x</code> を <code>float</code> 型に変換して返す
<code>mpfr_get_d(x, rnd)</code>	<code>x</code> を <code>double</code> 型に変換して返す
<code>mpfr_get_ld(x, rnd)</code>	<code>x</code> を <code>long double</code> 型に変換して返す
<code>mpfr_get_si(x, rnd)</code>	<code>x</code> を <code>signed long int</code> 型に変換して返す
<code>mpfr_get_ui(x, rnd)</code>	<code>x</code> を <code>unsigned long int</code> 型に変換して返す
<code>mpfr_get_z(z, x, rnd)</code>	<code>x</code> を整数に丸め、 <code>mpz_t</code> 型の変数 <code>z</code> に格納する
<code>mpfr_get_q(q, x)</code>	有限な <code>x</code> の値を <code>mpq_t</code> 型の変数 <code>q</code> に正確に変換する
<code>mpfr_get_f(f, x, rnd)</code>	<code>x</code> を <code>mpf_t</code> 型の変数 <code>f</code> に変換して格納する

通常の数値型へ変換する関数は、変換後の値を戻り値として返す。例えば、次の記述では、`x` の値を `double` 型に変換して変数 `d` に格納する。

```
double d = mpfr_get_d( x, MPFR_RNDN );
```

`mpfr_t` 型の値が変換先の型で保持できる精度や範囲を超えている場合には、元の値を完全には保持できない。特に、高い精度で計算した値を `float` 型や `double` 型へ変換すると、多くの桁が失われる可能性がある。

整数型へ変換する場合には、引数 `rnd` で指定した方法に従って整数値へ丸められる。例えば、次の記述では、`x` を最も近い整数へ丸め、`signed long int` 型の値として取り出す。

```
long n = mpfr_get_si( x, MPFR_RNDN );
```

重要) 丸めた結果が変換先の整数型で表現できる範囲を超える場合は、その型の上限値または下限値が返される。変換前に値が範囲内に収まることを確認するには、`mpfr_fits_slong_p` や `mpfr_fits_ulong_p` を使用できる。これらの関数にも、整数への変換に使用する丸め方法を指定する。

非常に大きな整数として取り出す場合には、`mpfr_get_z` を使用する。出力先の `mpz_t` 型の変数は、あらかじめ初期化しておく必要がある。(次の例)

```
mpz_t z;                ← GMP の整数の変数 z の宣言と
mpfr_init( z );         ← その初期化
                        (MPFR の変数 x に値が得られる処理)
mpfr_get_z( z, x, MPFR_RNDN ); ← x の値を z に変換
                        (その他の処理)
mpz_clear( z );         ← z の使用終了後の解放
```

有限な `mpfr_t` 型の値は、2 進数の有限桁で表現された数値であるため、ある有理数として正確に表すことができる。`mpfr_get_q` は、`x` が現在保持している値を正確な有理数に変換し、初期化済みの `mpq_t` 型の変数に格納する。(次

の例)

```
mpq_t q;           ← GMP の有理数の変数 q の宣言と
mpq_init( q );     ← その初期化
                   (MPFR の変数 x に値が得られる処理)
mpfr_get_q( q, x ); ← x の値を q に変換
                   (その他の処理)
mpq_clear( q );     ← q の使用終了後の解放
```

この変換は、計算しようとした数学上の厳密値を復元するものではなく、 x が現在の精度で保持している浮動小数点数を、正確な分数として表現するものである。

`mpfr_get_f` では、 x の値が変換先の `mpf_t` 型変数に設定されている精度と、指定した丸め方法に従って変換される。従って、変換先の精度が低い場合には、元の値を正確に保持できないことがある。

重要) `mpfr_get_z`, `mpfr_get_q`, `mpfr_get_f` の変換先には、それぞれ初期化済みの GMP の数値型変数を指定する。

22.4.2.4 演算のための関数 (1)：基本的な演算

`mpfr_t` 型の値に対する加減乗除などの基本的な演算には、表 82 に示す関数を使用する。これらの関数では、基本的に第 1 引数に演算結果を格納し、最後の引数に丸め方法を指定する。例えば、

```
mpfr_add( r, x, y, MPFR_RNDN );
```

とすると、 $x + y$ を最近接丸めによって丸めた値が r に格納される。演算結果は格納先である r の精度に従って丸められる。

表 82: MPFR の主な基本演算関数

関 数	解 説	関 数	解 説
<code>mpfr_add(r, x, y, rnd)</code>	$x + y$ を求める	<code>mpfr_sub(r, x, y, rnd)</code>	$x - y$ を求める
<code>mpfr_mul(r, x, y, rnd)</code>	xy を求める	<code>mpfr_div(r, x, y, rnd)</code>	x/y を求める
<code>mpfr_neg(r, x, rnd)</code>	$-x$ を求める	<code>mpfr_abs(r, x, rnd)</code>	x の絶対値を求める
<code>mpfr_add_ui(r, x, n, rnd)</code>	$x + n$ を求める	<code>mpfr_sub_ui(r, x, n, rnd)</code>	$x - n$ を求める
<code>mpfr_ui_sub(r, n, x, rnd)</code>	$n - x$ を求める	<code>mpfr_mul_ui(r, x, n, rnd)</code>	xn を求める
<code>mpfr_div_ui(r, x, n, rnd)</code>	x/n を求める	<code>mpfr_ui_div(r, n, x, rnd)</code>	n/x を求める
<code>mpfr_fma(r, x, y, z, rnd)</code>	$xy + z$ を 1 回の丸めで求める	<code>mpfr_fms(r, x, y, z, rnd)</code>	$xy - z$ を 1 回の丸めで求める
<code>mpfr_sqr(r, x, rnd)</code>	x^2 を求める		

表中の x, y, z, r は `mpfr_t` 型の変数であり、 n は `unsigned long int` 型の整数値である。負の整数を含む `long int` 型の値と演算する場合には、`mpfr_add_si`, `mpfr_sub_si`, `mpfr_mul_si`, `mpfr_div_si` など、関数名の末尾が `_si` である関数を使用することができる。また、`double` 型の値との演算には、`mpfr_add_d`, `mpfr_sub_d`, `mpfr_mul_d`, `mpfr_div_d` などを使用できる。

演算結果を格納する変数は、入力に用いる変数と同じでもよい。例えば、

```
mpfr_add(x, x, y, MPFR_RNDN);
```

とすると、 x の値が $x + y$ に更新される。同様に、

```
mpfr_mul(x, x, x, MPFR_RNDN);
```

とすれば、 x の値を 2 乗することができる。ただし、2 乗を求める場合には、その目的が明確に伝わり、一般に効率もよい `mpfr_sqr` を用いて、

```
mpfr_sqr(x, x, MPFR_RNDN);
```

と記述する方がよい。

これらの関数の戻り値は `int` 型である。通常の有限値に対する演算では、演算結果が正確に表現できた場合には 0 となる。丸められた結果が数学的に正確な値より大きい場合には正の値、小さい場合には負の値となる。通常は戻り値を使用せずに演算することもできるが、丸められた結果が数学的に正確な値より大きいのか小さいかは、次のような記述で戻り値を受け取って調べることができる。


```

int inex;
inex = mpfr_div(r, x, y, MPFR_RNDN);
if (inex == 0) {
    printf("The result is exact.\n");
} else if (inex > 0) {
    printf("The result was rounded upward.\n");
} else {
    printf("The result was rounded downward.\n");
}

```

ここでいう上向き、下向きは、指定した丸め方法そのものを表すのではなく、格納された値が数学的に正確な演算結果より大きい小さいかを表す。

`mpfr_fma` は、 $xy + z$ の厳密な値を求めたものとして、最終的な結果を 1 回だけ丸める。従って、

```

mpfr_mul(t, x, y, MPFR_RNDN);
mpfr_add(r, t, z, MPFR_RNDN);

```

とする場合とは、丸め誤差の生じ方が異なる可能性がある。積和演算を行う場合には、中間結果における丸めを避けることができるため、`mpfr_fma` が有効である。

22.4.2.5 比較, 判定のための関数

`mpfr_t` 型の値の大小を比較したり、値の性質を調べたりするには、比較関数と判定関数を使用する。MPFR の主な比較・判定関数を表 83 に示す。

表 83: MPFR の主な比較・判定関数

関 数	解 説
<code>mpfr_cmp(x, y)</code>	x と y の大小を比較する
<code>mpfr_equal_p(x, y)</code>	x と y が等しいかを判定する
<code>mpfr_less_p(x, y)</code>	x が y より小さいかを判定する
<code>mpfr_lessequal_p(x, y)</code>	x が y 以下であることを判定する
<code>mpfr_greater_p(x, y)</code>	x が y より大きいかを判定する
<code>mpfr_greaterequal_p(x, y)</code>	x が y 以上であることを判定する
<code>mpfr_sgn(x)</code>	x の符号に応じて正, 0, 負の値を返す
<code>mpfr_signbit(x)</code>	x の符号ビットが負を表しているかを判定する
<code>mpfr_integer_p(x)</code>	x が整数値であることを判定する
<code>mpfr_nan_p(x)</code>	x が NaN であることを判定する ⁽¹⁾
<code>mpfr_inf_p(x)</code>	x が正または負の無限大であることを判定する ⁽²⁾
<code>mpfr_number_p(x)</code>	x が NaN でも無限大でもないかを判定する ⁽³⁾
<code>mpfr_zero_p(x)</code>	x が 0 であることを判定する ⁽⁴⁾
<code>mpfr_regular_p(x)</code>	x が 0 以外の有限値であることを判定する ⁽⁵⁾
<code>mpfr_unordered_p(x, y)</code>	x と y の少なくとも一方が NaN であることを判定する ⁽⁶⁾

(1)(2)(3)(4)(5)(6) に関しては「22.4.2.14 `mpfr_t` の特殊な値」(p.483) で詳しく解説する。

`mpfr_cmp` は、2 つの値の大小関係に応じて、負, 0, 正のいずれかの値を返す。従って、通常は次のように戻り値を 0 と比較して使用する。

```

if ( mpfr_cmp( x, y ) < 0 ) {
    /* x < y の場合の処理 */
}

```

戻り値そのものが -1 , 0 , 1 に限定されるとは限らないため、具体的な値ではなく、0 より小さいか、0 に等しいか、0 より大きいかを調べる。

大小関係を一つだけ判定する場合には、`mpfr_less_p`, `mpfr_equal_p` などの判定関数を使用することもできる。これらの関数は、条件が成立する場合に 0 以外、成立しない場合に 0 を返す。

```

if ( mpfr_less_p( x, y ) ) {
    /* x < y の場合の処理 */
}

```

`mpfr_cmp` は 3 通りの大小関係をまとめて調べる場合に適しており、`mpfr_less_p` などは特定の条件だけを調べる場合に分かりやすい。

ただし、比較する値の一方または両方が NaN の場合には、通常の大小関係は定義されない。`mpfr_equal_p`, `mpfr_less_p`

などの判定関数は、いずれかの値が NaN である場合には 0 を返す。一方、`mpfr_cmp` も、いずれかの値が NaN の場合には 0 を返し、範囲エラーを示すフラグを設定する。従って、`mpfr_cmp` の戻り値が 0 であっても、NaN を含む場合は 2 つの値が等しいことを意味しない。

NaN を含む可能性がある処理では、下記のようにあらかじめ `mpfr_nan_p` で確認するか、`mpfr_unordered_p` を用いて、2 つの値が比較不能であるかどうかを確認するとよい。

例. 比較の前の NaN の検査 (1)

```
if ( mpfr_nan_p( x ) || mpfr_nan_p( y ) ) {
    /* NaN を含む場合の処理 */
} else if ( mpfr_less_p( x, y ) ) {
    /* x < y の場合の処理 */
}
```

例. 比較の前の NaN の検査 (2)

```
if ( mpfr_unordered_p( x, y ) ) {
    /* NaN を含む場合の処理 */
} else if ( mpfr_less_p( x, y ) ) {
    /* x < y の場合の処理 */
}
```

`mpfr_sgn` は、値が正の場合に正の値、0 の場合に 0、負の場合に負の値を返す。

```
int s = mpfr_sgn( x );
if ( s > 0 ) {
    /* x は正 */
} else if ( s < 0 ) {
    /* x は負 */
} else {
    /* x は 0, または NaN */
}
```

`x` が NaN の場合にも `mpfr_sgn` は 0 を返し、範囲エラーを示すフラグが設定される。従って、NaN の可能性がある場合には、あらかじめ `mpfr_nan_p` で確認する必要がある。

MPFR は正の 0 と負の 0 を区別して保持する。`mpfr_zero_p` は、正負のどちらの 0 に対しても 0 以外を返す。一方、`mpfr_signbit` を用いると、符号ビットを調べることができるため、負の 0 を識別できる。

```
if ( mpfr_zero_p( x ) ) {
    if ( mpfr_signbit( x ) ) {
        /* x は負の 0 */
    } else {
        /* x は正の 0 */
    }
}
```

値の種類を調べる関数は、次のように使い分けることができる。

- `mpfr_nan_p` : NaN であるかを調べる
- `mpfr_inf_p` : 正または負の無限大であるかを調べる
- `mpfr_number_p` : NaN でも無限大でもない値であるかを調べる
- `mpfr_zero_p` : 正または負の 0 であるかを調べる
- `mpfr_regular_p` : 0 以外の有限値であるかを調べる

従って、`mpfr_number_p` は 0 を含む有限値に対して真となるが、`mpfr_regular_p` は 0 に対して偽となる。

`mpfr_integer_p` は、値が整数型で保持されているかを調べるものではなく、`mpfr_t` 型の値が数学的に整数値であるかを判定する関数である。例えば、値が 3.0 であれば真となり、3.5 であれば偽となる。

22.4.2.6 mpfr_t の値の近さの判定

`mpfr_t` 型の値についても、GMP の `mpf_t` 型の場合と同様に、2 つの値の差の絶対値を求め、それが許容値以下であるかを調べることで、値の近さを判定できる。すなわち、2 つの値 x, y と、非負の許容値 ε に対して、

$$|x - y| \leq \varepsilon$$

が成り立つ場合に、 x と y は近い値であるとみなす。ただし、MPFR では NaN や無限大などの特殊な値を扱うことができ、演算ごとに丸め方法を指定するため、これらも考慮して判定処理を作成する必要がある。ここでは、この判定を行う関数 `mpfrNear` を次のように定義する。

プログラム：mpfrNear.c

```
1  #include "mpfrNear.h"
2
3  int mpfrNear(const mpfr_t x, const mpfr_t y, const mpfr_t tol) {
4      mpfr_prec_t prec;
5
6      /* NaNは比較不能であり、許容値には有限の非負値を要求する。*/
7      if (mpfr_nan_p(x) || mpfr_nan_p(y) || mpfr_nan_p(tol)) return 0;
8      if (mpfr_inf_p(tol) || mpfr_sgn(tol) < 0) return 0;
9
10     /* 等しい値は近いとみなす。同符号の無限大や、正負のゼロもここで判定される。*/
11     if (mpfr_equal_p(x, y)) return 1;
12
13     /* 一方だけが無限大の場合は、有限の許容値内には収まらない。*/
14     if (mpfr_inf_p(x) || mpfr_inf_p(y)) return 0;
15
16     /* 入力値の中で最も高い精度に、ガードビットを加える。*/
17     prec = mpfr_get_prec(x);
18
19     if (mpfr_get_prec(y) > prec) prec = mpfr_get_prec(y);
20     if (mpfr_get_prec(tol) > prec) prec = mpfr_get_prec(tol);
21
22     prec += 16;
23
24     mpfr_t diff;
25     mpfr_init2(diff, prec);
26
27     /* 差を0から遠ざかる方向へ丸める。これにより、差を実際より小さく見積もって、
28      * 誤って「近い」と判定することを避ける。mpfr_absは入出力が同じ変数なので、
29      * 新たな丸めは行われない。*/
30     mpfr_sub(diff, x, y, MPFR_RNDA);
31     mpfr_abs(diff, diff, MPFR_RNDN);
32
33     /* 戻り値を0または1に正規化する。*/
34     int result = (mpfr_lessequal_p(diff, tol) != 0);
35
36     mpfr_clear(diff);
37     return result;
38 }
```

この関数では、最初に x , y , tol が NaN でないことを確認している。NaN は他の値との大小関係を持たず、比較不能であるため、いずれかが NaN である場合は 0 を返す。また、許容値 tol には有限の非負値を要求し、負の値または無限大が指定された場合も 0 を返す。

次に、`mpfr_equal_p(x,y)` を使用して、2つの値が等しいかを調べている。等しい有限値の場合は差が 0 となるため、直ちに 1 を返す。符号にかかわらず、正負のゼロも等しい値として扱われる。また、この関数では、同符号の無限大についても「近い」とみなして 1 を返す規則としている。一方、値が等しくなく、 x または y が無限大である場合は、有限の許容値の範囲内には収まらないため 0 を返す。

差を格納する `diff` の精度には、 x , y , tol の中で最も高い精度に 16 ビットを加えた値を使用している。この追加分はガードビットであり、差の計算に伴う丸めの影響を小さくするために設けている。

差の計算には、次の処理を使用している。

```
mpfr_sub(diff, x, y, MPFR_RNDA);
mpfr_abs(diff, diff, MPFR_RNDN);
```

`mpfr_sub` の丸め方法に指定している `MPFR_RNDA` は、0 から遠ざかる方向への丸めを表す。このため、計算された差の絶対値が実際の差より小さくなることを避けることができる。従って、実際には許容範囲外である値を、丸め誤差によって誤って「近い」と判定する可能性を抑えることができる。

続く `mpfr_abs` では、入力と出力に同じ変数 `diff` を指定しているため、符号が調整されるだけで、新たな丸めは行われない。

ただし、差を0から遠ざかる方向へ丸めるため、実際の差が許容値以下であっても、境界に極めて近い場合には「近くない」と判定される可能性がある。16ビットのガードビットは、このような丸めの影響を小さくするためのものである。

最後に、

```
int result = (mpfr_lessequal_p(diff, tol) != 0);
```

によって、差の絶対値が許容値以下であるかを判定している。mpfr_lessequal_p は条件が成立すると0以外を返すため、!= 0によって戻り値を1または0に正規化している。これにより、条件が成立すれば1を返し、成立しなければ0を返す。

関数 mpfrNear を他のプログラムから使用するために、次のようなヘッダファイル mpfrNear.h を作成しておく。

ヘッダファイル：mpfrNear.h

```
1  #ifndef MPFRNEAR_H_INCLUDED
2  #define MPFRNEAR_H_INCLUDED
3  #include <mpfr.h>
4
5  int mpfrNear(const mpfr_t x, const mpfr_t y, const mpfr_t tol);
6
7  #endif
```

また、プログラム mpfrNear.c をコンパイルし、関数 mpfrNear を含むオブジェクトファイル mpfrNear.o を作成する。

例. mpfrNear.c をコンパイルして mpfrNear.o を作成する

```
$ gcc -c mpfrNear.c 
```

次のプログラム test271-4.c は、実際に関数 mpfrNear を使用する例である。

プログラム：test271-4.c

```
1  #include <stdio.h>
2  #include "mpfrNear.h"
3
4  int main() {
5      mpfr_t x, y, tol;
6
7      mpfr_inits2(160, x, y, tol, (mpfr_ptr)0);
8      mpfr_set_str(x,
9          "3.1415926535897932384626433832795028842",
10         10, MPFR_RNDN);
11     mpfr_set_str(y,
12         "3.1415926535897932384626433832795028841",
13         10, MPFR_RNDN);
14     mpfr_set_str(tol,
15         "0.00000000000000000000000000000000000001",
16         10, MPFR_RNDN);
17
18     mpfr_printf("x   = %.37Rf\n", x);
19     mpfr_printf("y   = %.37Rf\n", y);
20     mpfr_printf("tol = %.37Rf\n", tol);
21
22     if (mpfrNear(x, y, tol)) {
23         printf("x と y は近い値です. \n");
24     }
25     else {
26         printf("x と y は近い値ではありません. \n");
27     }
28
29     mpfr_clears(x, y, tol, (mpfr_ptr)0);
30 }
```

このプログラムでは、mpfr_inits2によって、x, y, tol を160ビットの精度で初期化している。また、mpfr_set_strによって文字列から値を設定する際には、丸め方法として MPFR_RNDN を指定している。このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test271-4.c のコンパイル, リンクと実行

```
$ gcc test271-4.c mpfrNear.o -lmpfr -lgmp -o test271-4 Enter
$ ./test271-4 Enter    ←実行
x   = 3.1415926535897932384626433832795028842    ←出力
y   = 3.1415926535897932384626433832795028841
tol = 0.0000000000000000000000000000000000010
x と y は近い値です.
```

mpfrNear は, GMP 用の mpfNear¹⁹⁹ と同じく, 絶対誤差に基づいて 2 つの値の近さを判定する関数である. ただし, MPFR 版では, NaN や無限大などの特殊な値を明示的に扱い, 差の計算における丸め方法と精度も指定している. このため, MPFR の特徴を生かした, より明確な規則に基づく判定処理となっている.

なお, この関数も差の絶対値だけを用いる判定である. 値の大きさが広い範囲にわたる場合には相対誤差も考慮し, ゼロに近い値を比較する場合には絶対誤差を併用するなど, 用途や値の範囲に応じて判定方法を調整する必要がある.

22.4.2.7 整数値への丸めに関する関数

mpfr_t 型の値を整数値へ丸めるには, 表 84 に示す関数を使用する. これらの関数で得られる結果は, C 言語の整数型ではなく, 整数値を保持した mpfr_t 型の値である.

表 84: MPFR の整数値への丸めに関する主な関数

関 数	解 説
mpfr_floor(r, x)	x 以下の最大の整数値を求める
mpfr_ceil(r, x)	x 以上の最小の整数値を求める
mpfr_trunc(r, x)	0 の方向へ丸めて整数値を求める
mpfr_round(r, x)	最も近い整数値に丸め, 中間値は 0 から遠い側へ丸める
mpfr_roundeven(r, x)	最も近い整数値に丸め, 中間値は偶数側へ丸める
mpfr_rint(r, x, rnd)	指定した丸め方法に従って整数値へ丸める
mpfr_frac(r, x, rnd)	x の小数部分を求める
mpfr_modf(i, f, x, rnd)	x の整数部分を i , 小数部分を f に格納する

x の値を 0 の方向へ丸めるには, 次のように mpfr_trunc を使用する.

書き方: mpfr_trunc(結果, 値)

例えば,

```
mpfr_trunc( r, x );
```

において, x が正の場合, mpfr_floor と mpfr_trunc はともに小数部分を取り除く方向へ丸める. 一方, x が負の場合には, 両者の結果が異なることがある. 例えば, $x = -3.7$ の場合には, 次の結果となる.

- mpfr_floor : -4
- mpfr_ceil : -3
- mpfr_trunc : -3

従って, 0 の方向へ小数部分を取り除く場合には mpfr_trunc を使用し, 数直線上で下側または上側の整数を求める場合には, それぞれ mpfr_floor, mpfr_ceil を使用する.

mpfr_round と mpfr_roundeven は, どちらも最も近い整数値へ丸めるが, 値が 2 つの整数のちょうど中間にある場合の処理が異なる. 例えば, $x = 2.5$ の場合, mpfr_round は 0 から遠い側の 3 へ丸めるのに対し, mpfr_roundeven は偶数側の 2 へ丸める. また, $x = 3.5$ は, どちらの関数でも 4 へ丸められる.

mpfr_rint をはじめ, MPFR の多くの関数では, 丸め方法を表す定数を引数 rnd に指定する. 主な丸め方法を表 85 通常の数値計算では, 最も近い値への丸めを行う MPFR_RNDN がよく使用される. MPFR_RNDN では, 丸める値が 2 つの表現可能な値のちょうど中間にある場合に, 丸めた結果の仮数部の最下位ビットが 0 となる側へ丸められる. 整数値への丸めでは, 結果を格納する変数に十分な精度があれば, これは偶数の整数側への丸めとなる.

¹⁹⁹ 「22.4.1.6 mpfr_t の値の近さの判定」(p.446〜)で例示したプログラム.

表 85: MPFR の主な丸め方法

定 数	丸め方法
MPFR_RNDN	最も近い値へ丸め, 中間値は偶数側へ丸める (最近接丸め)
MPFR_RNDZ	0 の方向へ丸める
MPFR_RNDU	正の無限大の方向へ丸める
MPFR_RNDD	負の無限大の方向へ丸める
MPFR_RNDA	0 から遠ざかる方向へ丸める

丸め方法を引数で指定して整数値へ丸めるには, `mpfr rint` を使用する.

書き方: `mpfr_rint( 結果, 値, 丸め方法 )`

例えば, 次の記述では, x を負の無限大の方向へ丸める.

```
mpfr_rint( r, x, MPFR_RNDD );
```

丸め方法を変更することで, 同じ値に対して異なる整数値が得られる. 例えば, $x = -3.7$ の場合には, 次のようになる.

- MPFR_RNDN : -4
- MPFR_RNDZ : -3
- MPFR_RNDU : -3
- MPFR_RNDD : -4
- MPFR_RNDA : -4

これらの丸め方法は, `mpfr_rint` だけでなく, 値の設定, 基本演算, 数学関数など, 丸め方法を引数に取る MPFR の関数で共通して使用される.

ただし, 結果を格納する変数の精度が低く, 数学的に得られる整数値を正確に表現できない場合は, その精度で表現可能な整数値へ丸められる. 従って, 丸めて得られる整数値を正確に保持するには, 結果を格納する変数にも十分な精度を設定しておく必要がある.

`mpfr_floor` から `mpfr_rint` までの整数値への丸め関数の戻り値は, 得られた値が元の値と等しい場合には 0 となる. 得られた値が元の値より大きい場合には正の値, 元の値より小さい場合には負の値を返す. 従って, 必要に応じて値が変化したかどうかや, 丸められた方向を調べることができる.

```
int ret = mpfr_floor( r, x );
if ( ret != 0 ) {
    /* 丸めによって値が変化した */
}
```

整数値へ丸めた後, C 言語の整数型や GMP の `mpz_t` 型として値を取り出す場合には, `mpfr_get_si`, `mpfr_get_ui`, `mpfr_get_z` などを使用する.

`mpfr_frac` は, x の小数部分を求める関数である. この関数で得られる小数部分の符号は x と同じになる. 例えば, $x = -3.7$ の場合には, -0.7 が得られる.

```
mpfr_frac( r, x, MPFR_RNDN );
```

整数部分と小数部分を同時に求めるには, `mpfr_modf` を使用する.

書き方: `mpfr_modf( 整数部分, 小数部分, 値, 丸め方法 )`

例えば,

```
mpfr_modf( i, f, x, MPFR_RNDN );
```

の場合, i と f は, あらかじめ初期化された互いに異なる `mpfr_t` 型の変数でなければならない. x が -3.7 の場合には, i に -3 , f に -0.7 が格納される. このとき, 整数部分と小数部分は, それぞれの格納先の精度と, 指定した丸め方法に従って丸められる.

22.4.2.8 演算のための関数 (2): 累乗・平方根など

`mpfr_t` 型の値の累乗, 平方根, 累乗根などを求める主な関数を, 表 86 に示す. これらの関数では, 基本的に第 1

引数に演算結果を格納し、最後の引数に丸め方法を指定する。

表 86: MPFR の主な累乗・平方根関数

関 数	解 説
<code>mpfr_sqr(r, x, rnd)</code>	x^2 を求める
<code>mpfr_sqrt(r, x, rnd)</code>	$\sqrt{x}$ を求める
<code>mpfr_sqrt_ui(r, n, rnd)</code>	符号なし整数 n の平方根を求める
<code>mpfr_rec_sqrt(r, x, rnd)</code>	$1/\sqrt{x}$ を求める
<code>mpfr_cbrt(r, x, rnd)</code>	$\sqrt[3]{x}$ を求める
<code>mpfr_rootn_ui(r, x, n, rnd)</code>	x の n 乗根を求める
<code>mpfr_rootn_si(r, x, n, rnd)</code>	符号付き整数 n に対する x の n 乗根を求める
<code>mpfr_pow(r, x, y, rnd)</code>	x^y を求める
<code>mpfr_pow_ui(r, x, n, rnd)</code>	符号なし整数 n に対する x^n を求める
<code>mpfr_pow_si(r, x, n, rnd)</code>	符号付き整数 n に対する x^n を求める
<code>mpfr_ui_pow(r, n, x, rnd)</code>	符号なし整数 n に対する n^x を求める
<code>mpfr_ui_pow_ui(r, m, n, rnd)</code>	符号なし整数 m, n に対する m^n を求める
<code>mpfr_hypot(r, x, y, rnd)</code>	$\sqrt{x^2 + y^2}$ を求める

例えば、 x の 2 乗と平方根を求めるには、それぞれ `mpfr_sqr` と `mpfr_sqrt` を使用する。

書き方： `mpfr_sqr( 結果, 値, 丸め方法 )`
`mpfr_sqrt( 結果, 値, 丸め方法 )`

例えば、

```
mpfr_sqr( r, x, MPFR_RNDN );
mpfr_sqrt( r, x, MPFR_RNDN );
```

とすると、それぞれ x^2 と $\sqrt{x}$ を最近接丸めで求め、結果を `r` に格納する。演算結果は、格納先である `r` の精度に従って丸められる。また、多くの関数では、出力先と入力元に同じ変数を指定することもできる。例えば、

```
mpfr_sqrt( x, x, MPFR_RNDN );
```

とすると、 x の平方根を求め、その結果を再び `x` に格納する。

`mpfr_sqrt` は、0 未満の有限値に対して NaN を返す。これに対して、`mpfr_cbrt` は負の値に対しても実数の立方根を求めることができる。例えば、 -8 の立方根は -2 となる。(次の例)

```
mpfr_cbrt( r, x, MPFR_RNDN );
```

一般の累乗根を求めるには、`mpfr_rootn_ui` を使用する。

書き方： `mpfr_rootn_ui( 結果, 値, 指数, 丸め方法 )`

例えば、次の記述では、 x の 5 乗根を求める。

```
mpfr_rootn_ui( r, x, 5, MPFR_RNDN );
```

指数 n が奇数の場合には、負の値に対しても実数の累乗根が得られる。一方、 n が偶数で x が負の場合、結果は NaN となる。また、 n に 0 を指定した場合にも結果は NaN となる。

`mpfr_rootn_si` では、累乗根の指数として符号付き整数を指定できる。正の指数に対する動作は `mpfr_rootn_ui` と同様であり、負の指数を指定すると、対応する累乗根の逆数が得られる。例えば、

```
mpfr_rootn_si( r, x, -3, MPFR_RNDN );
```

とすると、 $1/\sqrt[3]{x}$ を求める。単に $1/\sqrt{x}$ を求める場合には、専用の関数 `mpfr_rec_sqrt` を使用できる。

```
mpfr_rec_sqrt( r, x, MPFR_RNDN );
```

累乗を求めるには、底と指数の型に応じて、`mpfr_pow`, `mpfr_pow_ui`, `mpfr_pow_si` などを使用する。指数が `mpfr_t` 型の場合には、次のように `mpfr_pow` を使用する。

書き方： `mpfr_pow( 結果, 底, 指数, 丸め方法 )`

例えば、

```
mpfr_pow( r, x, y, MPFR_RNDN );
```

の記述では、 x^y を求めて r に格納する。指数が整数であることが分かっている場合には、`mpfr_pow_ui` または `mpfr_pow_si` を使用の方が、値の型と計算の意味が明確になる。

```
mpfr_pow_ui( r, x, 10, MPFR_RNDN );    /* x の 10 乗 */
mpfr_pow_si( r, x, -3, MPFR_RNDN );    /* x の -3 乗 */
```

負の整数を指数として指定すると、例えば $x^{-3} = 1/x^3$ のように、累乗の逆数が求められる。ただし、底が 0 で指数が負の場合には無限大となる。負の 0 を底とし、指数が負の奇数である場合には負の無限大となり、それ以外の場合には正の無限大となる。

`mpfr_pow` では、負の有限値を底とし、整数でない有限値を指数とした場合、実数の範囲では値が定まらないため、結果は NaN となる。例えば、 $(-2)^{1/2}$ は `mpfr_t` 型の実数としては表現されない。複素数の累乗を扱う場合には、複素数用のライブラリを使用する必要がある。

底が符号なし整数で指数が `mpfr_t` 型の場合には、`mpfr_ui_pow` を使用する。また、底と指数がともに符号なし整数の場合には、`mpfr_ui_pow_ui` を使用できる。

```
mpfr_ui_pow( r, 2, x, MPFR_RNDN );    /* 2 の x 乗 */
mpfr_ui_pow_ui( r, 2, 100, MPFR_RNDN ); /* 2 の 100 乗 */
```

$\sqrt{x^2 + y^2}$ を求める場合には、`mpfr_sqr`, `mpfr_add`, `mpfr_sqrt` を順に使用することもできるが、専用の関数 `mpfr_hypot` を使用すると簡潔に記述できる。

書き方： `mpfr_hypot( 結果, 値 1, 値 2, 丸め方法 )`

例. $r \leftarrow \sqrt{x^2 + y^2}$

```
mpfr_hypot( r, x, y, MPFR_RNDN );
```

`mpfr_hypot` は、 x^2 や y^2 を個別の変数へ格納せずに計算できるため、記述が簡潔になる。また、中間結果を個別に丸めてから加算する方法とは異なり、 $\sqrt{x^2 + y^2}$ の厳密な値を、指定した丸め方法に従って丸めた結果が得られる。

22.4.2.9 演算のための関数 (3)：指数関数, 対数関数

`mpfr_t` 型の値に対する指数関数と対数関数を、表 87 に示す。これらの関数では、第 1 引数に演算結果を格納し、最後の引数に丸め方法を指定する。

表 87: MPFR の主な指数・対数関数

関 数	解 説
<code>mpfr_exp(r, x, rnd)</code>	指数関数 e^x を求める
<code>mpfr_exp2(r, x, rnd)</code>	2^x を求める
<code>mpfr_exp10(r, x, rnd)</code>	10^x を求める
<code>mpfr_expm1(r, x, rnd)</code>	$e^x - 1$ を求める
<code>mpfr_log(r, x, rnd)</code>	自然対数 $\log x$ を求める
<code>mpfr_log2(r, x, rnd)</code>	底を 2 とする対数 $\log_2 x$ を求める
<code>mpfr_log10(r, x, rnd)</code>	常用対数 $\log_{10} x$ を求める
<code>mpfr_log1p(r, x, rnd)</code>	$\log(1 + x)$ を求める

自然対数の底 e に対する指数関数を求めるには、`mpfr_exp` を使用する。

書き方： `mpfr_exp( 結果, 値, 丸め方法 )`

例えば、

```
mpfr_exp( r, x, MPFR_RNDN );
```

とすると、 e^x を最近接丸めで求め、結果を r に格納する。底が 2 または 10 の指数関数を求める場合には、それぞれ `mpfr_exp2`, `mpfr_exp10` を使用する。

```
mpfr_exp2( r, x, MPFR_RNDN );    /* 2 の x 乗 */
mpfr_exp10( r, x, MPFR_RNDN );   /* 10 の x 乗 */
```

一般の底に対する累乗は、前項で示した `mpfr_pow` などを用いて求める。

自然対数を求めるには、`mpfr_log` を使用する。

書き方： `mpfr_log( 結果, 値, 丸め方法 )`

例えば、

```
mpfr_log( r, x, MPFR_RNDN );
```

とすると、 $\log x$ を求めて `r` に格納する。ここで $\log x$ は、底を e とする自然対数を表す。底が 2 または 10 の対数を求める場合には、それぞれ `mpfr_log2`, `mpfr_log10` を使用する。

```
mpfr_log2( r, x, MPFR_RNDN );    /* 底を 2 とする対数 */
mpfr_log10( r, x, MPFR_RNDN );   /* 常用対数 */
```

実数の範囲では、対数関数の引数は正でなければならない。`x` が負の場合、`mpfr_log`, `mpfr_log2`, `mpfr_log10` の結果は NaN となる。また、`x` が正または負の 0 の場合には、結果は負の無限大となる。

`mpfr_expm1` は、 $e^x - 1$ を直接求める関数である。同じ値は `mpfr_exp` で e^x を求めてから 1 を引くことでも計算できるが、`x` の絶対値が小さい場合には、 e^x が 1 に近い値となるため、減算によって有効な桁が失われることがある。このような場合には、`mpfr_expm1` を使用するとよい。

```
mpfr_expm1( r, x, MPFR_RNDN );
```

同様に、`mpfr_log1p` は $\log(1+x)$ を直接求める関数である。`x` の絶対値が小さい場合、通常の加算によって $1+x$ を求めると、`x` の情報が丸めによって失われることがある。このような場合には、`mpfr_log` を適用するよりも、`mpfr_log1p` を使用の方が精度を保ちやすい。

```
mpfr_log1p( r, x, MPFR_RNDN );
```

`mpfr_log1p` では、`x` が有限値であり、 $1+x > 0$ 、すなわち $x > -1$ の場合に、数学的には有限な実数値が定まる。 $x = -1$ の場合には負の無限大、 $x < -1$ の場合には NaN となる。また、`x` が正の無限大の場合には、結果も正の無限大となる。

これらの関数でも、演算結果は格納先である `r` の精度に従って丸められる。また、出力先と入力元に同じ変数を指定することができる。例えば、

```
mpfr_log( x, x, MPFR_RNDN );
```

とすると、`x` の自然対数を求め、結果を再び `x` に格納する。

22.4.2.10 演算のための関数 (4)：三角関数

`mpfr_t` 型の値に対する三角関数と逆三角関数を、表 88 に示す。通常の三角関数では、引数の角度をラジアン単位で与える。また、第 1 引数に演算結果を格納し、最後の引数に丸め方法を指定する。

正弦、余弦、正接を求めるには、それぞれ `mpfr_sin`, `mpfr_cos`, `mpfr_tan` を使用する。これらの関数に与える角度はラジアン単位である。

書き方： `mpfr_sin( 結果, 角度, 丸め方法 )`
`mpfr_cos( 結果, 角度, 丸め方法 )`
`mpfr_tan( 結果, 角度, 丸め方法 )`

例えば、

```
mpfr_sin( r, x, MPFR_RNDN );
mpfr_cos( r, x, MPFR_RNDN );
mpfr_tan( r, x, MPFR_RNDN );
```

とすると、それぞれ $\sin x$, $\cos x$, $\tan x$ を最近接丸めで求め、結果を `r` に格納する。演算結果は、格納先である `r` の精度に従って丸められる。また、出力先と入力元に同じ変数を指定することもできる。

```
mpfr_sin( x, x, MPFR_RNDN );
```

正弦と余弦の両方が必要な場合には、`mpfr_sin_cos` を使用するとよい。

書き方： `mpfr_sin_cos( 正弦, 余弦, 角度, 丸め方法 )`

例えば、

```
mpfr_sin_cos( s, c, x, MPFR_RNDN );
```

表 88: MPFR の主な三角関数・逆三角関数

関 数	解 説
<code>mpfr_sin(r, x, rnd)</code>	正弦 $\sin x$ を求める
<code>mpfr_cos(r, x, rnd)</code>	余弦 $\cos x$ を求める
<code>mpfr_tan(r, x, rnd)</code>	正接 $\tan x$ を求める
<code>mpfr_sin_cos(s, c, x, rnd)</code>	$\sin x$ と $\cos x$ を同時に求める
<code>mpfr_sinpi(r, x, rnd)</code>	$\sin(\pi x)$ を求める
<code>mpfr_cospi(r, x, rnd)</code>	$\cos(\pi x)$ を求める
<code>mpfr_tanpi(r, x, rnd)</code>	$\tan(\pi x)$ を求める
<code>mpfr_sec(r, x, rnd)</code>	正割 $1/\cos x$ を求める
<code>mpfr_csc(r, x, rnd)</code>	余割 $1/\sin x$ を求める
<code>mpfr_cot(r, x, rnd)</code>	余接 $1/\tan x$ を求める
<code>mpfr_asin(r, x, rnd)</code>	逆正弦 $\sin^{-1} x$ を求める
<code>mpfr_acos(r, x, rnd)</code>	逆余弦 $\cos^{-1} x$ を求める
<code>mpfr_atan(r, x, rnd)</code>	逆正接 $\tan^{-1} x$ を求める
<code>mpfr_atan2(r, y, x, rnd)</code>	座標 (x, y) の偏角を求める
<code>mpfr_asinpi(r, x, rnd)</code>	$\sin^{-1} x/\pi$ を求める
<code>mpfr_acospi(r, x, rnd)</code>	$\cos^{-1} x/\pi$ を求める
<code>mpfr_atanpi(r, x, rnd)</code>	$\tan^{-1} x/\pi$ を求める
<code>mpfr_atan2pi(r, y, x, rnd)</code>	座標 (x, y) の偏角を π で割った値を求める

逆正弦, 逆余弦, 逆正接はそれぞれ $\arcsin(x)$, $\arccos(x)$, $\arctan(x)$ と表記されることもある。

とすると, $\sin x$ を `s`, $\cos x$ を `c` に格納する. 出力先の `s` と `c` は, あらかじめ初期化された互いに異なる `mpfr_t` 型の変数でなければならない. 正弦と余弦を別々に求めるよりも, 一度の関数呼び出しで両方の値を得ることができる.

角度を π の倍数として与える場合には, `mpfr_sinpi`, `mpfr_cospi`, `mpfr_tanpi` を使用できる. 例えば,

```
mpfr_sinpi( r, x, MPFR_RNDN );
```

とすると, $\sin(\pi x)$ を求める. 従って, `x` に $1/2$ を設定した場合には, $\sin(\pi/2) = 1$ が得られる. これらの関数を使用すると, 円周率を求めて `x` に掛ける処理を別に記述する必要がなく, 中間結果の丸めも避けることができる.

`mpfr_sec`, `mpfr_csc`, `mpfr_cot` は, それぞれ余弦, 正弦, 正接の逆数を求める.

```
mpfr_sec( r, x, MPFR_RNDN );    /* 1/cos(x) */
mpfr_csc( r, x, MPFR_RNDN );    /* 1/sin(x) */
mpfr_cot( r, x, MPFR_RNDN );    /* 1/tan(x) */
```

正弦, 余弦, 正接の値が 0 となる点では, 対応する逆数は定義されず, 結果が無限大となる場合がある.

逆三角関数を求めるには, `mpfr_asin`, `mpfr_acos`, `mpfr_atan` を使用する. これらの関数で得られる角度もラジアン単位である.

```
mpfr_asin( r, x, MPFR_RNDN );
mpfr_acos( r, x, MPFR_RNDN );
mpfr_atan( r, x, MPFR_RNDN );
```

`mpfr_asin` と `mpfr_acos` の実数としての定義域は $-1 \leq x \leq 1$ である. この範囲を外れた有限値を与えた場合には, 結果は NaN となる. 一方, `mpfr_atan` は任意の有限な実数値に対して使用できる.

座標 (x, y) が表す方向の偏角を求めるには, `mpfr_atan2` を使用する.

書き方: `mpfr_atan2( 結果, y 座標, x 座標, 丸め方法 )`

例えば,

```
mpfr_atan2( r, y, x, MPFR_RNDN );
```

とすると, 座標 (x, y) の偏角を求めて `r` に格納する. 引数の順序が `y, x` であることに注意する. `mpfr_atan2` は, y/x

に対して単に `mpfr_atan` を適用する場合と異なり、 x と y の符号を考慮して象限を判定する。また、 $x = 0$ 、 $y \neq 0$ の場合にも偏角を求めることができる。

逆三角関数の結果を π で割った値として求めるには、`mpfr_asinpi`、`mpfr_acospi`、`mpfr_atanpi`、`mpfr_atan2pi` を使用できる。例えば、

```
mpfr_atan2pi( r, y, x, MPFR_RNDN );
```

とすると、座標 (x, y) の偏角を π で割った値が得られる。従って、結果が $1/2$ であれば偏角は $\pi/2$ 、結果が $-1/4$ であれば偏角は $-\pi/4$ である。これらの関数は、角度を π の倍数として扱う場合に便利である。

22.4.2.11 演算のための関数 (5)：双曲線関数

`mpfr_t` 型の値に対する双曲線関数と逆双曲線関数を、表 89 に示す。三角関数とは異なり、双曲線関数の引数は角度ではないため、ラジアンや度といった単位は用いない。これらの関数では、第 1 引数に演算結果を格納し、最後の引数に丸め方法を指定する。

表 89: MPFR の主な双曲線関数・逆双曲線関数

関 数	解 説
<code>mpfr_sinh(r, x, rnd)</code>	双曲線正弦 $\sinh x$ を求める
<code>mpfr_cosh(r, x, rnd)</code>	双曲線余弦 $\cosh x$ を求める
<code>mpfr_tanh(r, x, rnd)</code>	双曲線正接 $\tanh x$ を求める
<code>mpfr_sinh_cosh(s, c, x, rnd)</code>	$\sinh x$ と $\cosh x$ を同時に求める
<code>mpfr_sech(r, x, rnd)</code>	双曲線正割 $1/\cosh x$ を求める
<code>mpfr_csch(r, x, rnd)</code>	双曲線余割 $1/\sinh x$ を求める
<code>mpfr_coth(r, x, rnd)</code>	双曲線余接 $1/\tanh x$ を求める
<code>mpfr_asinh(r, x, rnd)</code>	逆双曲線正弦を求める
<code>mpfr_acosh(r, x, rnd)</code>	逆双曲線余弦を求める
<code>mpfr_atanh(r, x, rnd)</code>	逆双曲線正接を求める

双曲線正弦、双曲線余弦、双曲線正接を求めるには、それぞれ `mpfr_sinh`、`mpfr_cosh`、`mpfr_tanh` を使用する。

書き方： `mpfr_sinh( 結果, 値, 丸め方法 )`
`mpfr_cosh( 結果, 値, 丸め方法 )`
`mpfr_tanh( 結果, 値, 丸め方法 )`

例えば、

```
mpfr_sinh( r, x, MPFR_RNDN );
mpfr_cosh( r, x, MPFR_RNDN );
mpfr_tanh( r, x, MPFR_RNDN );
```

とすると、それぞれ $\sinh x$ 、 $\cosh x$ 、 $\tanh x$ を最近接丸めで求め、結果を r に格納する。演算結果は、格納先である r の精度に従って丸められる。また、出力先と入力元に同じ変数を指定することもできる。

```
mpfr_tanh( x, x, MPFR_RNDN );
```

双曲線正弦と双曲線余弦の両方が必要な場合には、`mpfr_sinh_cosh` を使用するとよい。

書き方： `mpfr_sinh_cosh( 双曲線正弦, 双曲線余弦, 値, 丸め方法 )`

例えば、

```
mpfr_sinh_cosh( s, c, x, MPFR_RNDN );
```

とすると、 $\sinh x$ を s 、 $\cosh x$ を c に格納する。出力先の s と c は、あらかじめ初期化された互いに異なる `mpfr_t` 型の変数でなければならない。

双曲線関数は、指数関数を用いて次のように表すことができる。

$$\sinh x = \frac{e^x - e^{-x}}{2}, \quad \cosh x = \frac{e^x + e^{-x}}{2}, \quad \tanh x = \frac{\sinh x}{\cosh x}$$

これらの式を `mpfr_exp` などを用いて直接計算することもできるが、専用の双曲線関数を使用する方が、記述が簡潔になり、中間結果を個別に丸める処理も避けることができる。

mpfr_sech, mpfr_csch, mpfr_coth は、それぞれ双曲線余弦、双曲線正弦、双曲線正接の逆数を求める。

```
mpfr_sech( r, x, MPFR_RNDN );    /* 1/cosh(x) */
mpfr_csch( r, x, MPFR_RNDN );    /* 1/sinh(x) */
mpfr_coth( r, x, MPFR_RNDN );    /* 1/tanh(x) */
```

$\cosh x$ は実数の範囲で 0 にならないため、mpfr_sech は任意の有限な実数値に対して使用できる。一方、mpfr_csch と mpfr_coth は、 x が 0 の場合に正または負の無限大となる。

逆双曲線関数を求めるには、mpfr_asinh, mpfr_acosh, mpfr_atanh を使用する。

```
mpfr_asinh( r, x, MPFR_RNDN );
mpfr_acosh( r, x, MPFR_RNDN );
mpfr_atanh( r, x, MPFR_RNDN );
```

mpfr_asinh は、任意の有限な実数値に対して使用できる。mpfr_acosh の実数としての定義域は $x \geq 1$ であり、 $x < 1$ の場合には結果は NaN となる。

mpfr_atanh の実数としての定義域は $-1 < x < 1$ である。 $x = 1$ の場合には正の無限大、 $x = -1$ の場合には負の無限大となり、 $|x| > 1$ の場合には結果は NaN となる。

双曲線正弦と双曲線余弦の間には、次の関係が成り立つ。

$$\cosh^2 x - \sinh^2 x = 1$$

これは、三角関数における $\cos^2 x + \sin^2 x = 1$ に対応する双曲線関数の基本的な関係である。

22.4.2.12 演算のための関数 (6)：特殊関数、定数

MPFR では、ガンマ関数、ゼータ関数、誤差関数、Bessel 関数など、多くの特殊関数を多倍長精度で計算することができる。主な特殊関数を表 90 に示す。これらの関数では、基本的に第 1 引数に演算結果を格納し、最後の引数に丸め方法を指定する。

表 90: MPFR の主な特殊関数

関 数	解 説
mpfr_gamma(r, x, rnd)	ガンマ関数 $\Gamma(x)$ を求める
mpfr_gamma_inc(r, x, y, rnd)	上側不完全ガンマ関数を求める
mpfr_lngamma(r, x, rnd)	$\log \Gamma(x)$ を求める
mpfr_lgamma(r, signp, x, rnd)	$\log \Gamma(x) $ を求め、 $\Gamma(x)$ の符号を signp の指す変数に格納する
mpfr_digamma(r, x, rnd)	ディガンマ関数を求める
mpfr_fac_ui(r, n, rnd)	符号なし整数 n の階乗 $n!$ を求める
mpfr_beta(r, x, y, rnd)	ベータ関数を求める
mpfr_zeta(r, x, rnd)	リーマンのゼータ関数を求める
mpfr_zeta_ui(r, n, rnd)	符号なし整数 n に対するゼータ関数を求める
mpfr_erf(r, x, rnd)	誤差関数を求める
mpfr_erfc(r, x, rnd)	相補誤差関数を求める
mpfr_j0(r, x, rnd)	第 1 種 0 次 Bessel 関数を求める
mpfr_j1(r, x, rnd)	第 1 種 1 次 Bessel 関数を求める
mpfr_jn(r, n, x, rnd)	第 1 種 n 次 Bessel 関数を求める
mpfr_y0(r, x, rnd)	第 2 種 0 次 Bessel 関数を求める
mpfr_y1(r, x, rnd)	第 2 種 1 次 Bessel 関数を求める
mpfr_yn(r, n, x, rnd)	第 2 種 n 次 Bessel 関数を求める
mpfr_eint(r, x, rnd)	指数積分を求める
mpfr_li2(r, x, rnd)	二重対数関数の実部を求める
mpfr_agm(r, x, y, rnd)	x と y の算術幾何平均を求める
mpfr_ai(r, x, rnd)	Airy 関数 $\text{Ai}(x)$ を求める

例えば、ガンマ関数を求めるには、`mpfr_gamma` を使用する。

書き方： `mpfr_gamma( 結果, 値, 丸め方法 )`

例えば、

```
mpfr_gamma( r, x, MPFR_RNDN );
```

とすると、 $\Gamma(x)$ を最近接丸めで求め、結果を `r` に格納する。ガンマ関数は、正の整数 n に対して、次の関係を満たす。

$$\Gamma(n) = (n-1)!$$

従って、例えば $\Gamma(6) = 5! = 120$ となる。符号なし整数の階乗そのものを直接求める場合には、`mpfr_fac_ui` を使用することもできる。(次の例：100!)

```
mpfr_fac_ui( r, 100, MPFR_RNDN ); /* 100! */
```

ガンマ関数は値が非常に大きくなることがある。そのような場合には、ガンマ関数そのものではなく、その対数を求める `mpfr_lngamma` または `mpfr_lgamma` が便利である。(次の例)

```
mpfr_lngamma( r, x, MPFR_RNDN );
```

`mpfr_lngamma` は $\log \Gamma(x)$ を求める。一方、`mpfr_lgamma` は $\log |\Gamma(x)|$ を求めると同時に、 $\Gamma(x)$ の符号を整数型の変数へ格納する。例えば、

```
int sign;
mpfr_lgamma( r, &sign, x, MPFR_RNDN );
```

の場合、 $\Gamma(x)$ の符号が定義される場合には、`sign` に 1 または -1 が格納される。ただし、`x` が負の整数、NaN、負の無限大などの場合には、`sign` の値は定義されない。なお、`mpfr_lngamma` は $\Gamma(x)$ が負となる範囲では NaN を返すことがあるため、符号も含めて扱う場合には `mpfr_lgamma` を使用するとよい。

`mpfr_digamma` は、ガンマ関数の対数微分に相当するディガンマ関数 (下記) を求める。

$$\psi(x) = \frac{\Gamma'(x)}{\Gamma(x)}$$

例. `mpfr_digamma( r, x, MPFR_RNDN );`

リーマンのゼータ関数を求めるには、`mpfr_zeta` を使用する。

書き方： `mpfr_zeta( 結果, 値, 丸め方法 )`

例. `mpfr_zeta( r, x, MPFR_RNDN );`

引数が符号なし整数である場合には、`mpfr_zeta_ui` を使用できる。例えば、

```
mpfr_zeta_ui( r, 2, MPFR_RNDN );
```

は、 $\zeta(2) = \pi^2/6$ を求める。ただし、ゼータ関数は $x = 1$ に極を持つため、この点では正の無限大となる。

誤差関数を求めるには、`mpfr_erf` を使用する。また、相補誤差関数 $1 - \operatorname{erf}(x)$ を求めるには、`mpfr_erfc` を使用する。

```
mpfr_erf( r, x, MPFR_RNDN );
mpfr_erfc( r, x, MPFR_RNDN );
```

`mpfr_erfc` は、`mpfr_erf` の結果を 1 から引くことでも計算できるが、 $\operatorname{erf}(x)$ が 1 に近い場合には減算によって有効な桁が失われることがある。このような場合には、`mpfr_erfc` を直接使用するとよい。

Bessel 関数では、次数が 0 または 1 の場合には `mpfr_j0`, `mpfr_j1`, `mpfr_y0`, `mpfr_y1` を使用できる。一般の整数次数 n に対しては、`mpfr_jn` または `mpfr_yn` を使用する。

例. Bessel 関数

```
mpfr_jn( r, n, x, MPFR_RNDN );
mpfr_yn( r, n, x, MPFR_RNDN );
```

これらの関数では、第 2 引数 `n` に整数の次数を指定する。特殊関数は、引数や精度によって計算時間が大きくなる場合があるため、必要以上に高い精度を指定しないことも重要である。

`mpfr_agm` は、2 つの非負の値の算術幾何平均を求める。

例. `mpfr_agm( r, x, y, MPFR_RNDN );`

算術幾何平均は、円周率や楕円積分の高精度計算などに利用される。 `mpfr_eint`, `mpfr_li2`, `mpfr_ai` などは、数値解析や物理学で現れる特殊関数であり、必要に応じて使用する。

MPFR は、代表的な数学定数を直接求める関数も提供している。主な関数を表 91 に示す。

表 91: MPFR が提供する主な数学定数	
関 数	解 説
<code>mpfr_const_pi(r, rnd)</code>	円周率 π を求める
<code>mpfr_const_log2(r, rnd)</code>	$\log 2$ を求める
<code>mpfr_const_euler(r, rnd)</code>	オイラー・マスケローニ定数を求める
<code>mpfr_const_catalan(r, rnd)</code>	カタラン定数を求める

例えば、円周率を求めるには `mpfr_const_pi` を使用する。

書き方： `mpfr_const_pi( 結果, 丸め方法 )`

`mpfr_const_pi( r, MPFR_RNDN );`

この関数では、格納先である `r` の精度に従って円周率が丸められる。同様に、次のようにして他の定数を求めることができる。

```
mpfr_const_log2( r, MPFR_RNDN );
mpfr_const_euler( r, MPFR_RNDN );
mpfr_const_catalan( r, MPFR_RNDN );
```

`mpfr_const_euler` が返す値は、自然対数の底 e ではなく、オイラー・マスケローニ定数であることに注意する。自然対数の底 e を求める場合には、1 を設定した変数に `mpfr_exp` を適用する。

```
mpfr_set_ui( x, 1, MPFR_RNDN );
mpfr_exp( r, x, MPFR_RNDN );
```

数学定数を十進数の文字列として直接記述すると、記述した桁数までの近似値しか得られない。これらの定数関数を使用すると、格納先に指定した精度に応じた値を直接求めることができる。

22.4.2.13 MPFR のキャッシュの解放

`mpfr_const_pi`, `mpfr_const_log2` などの数学定数を求める関数は、同じ定数を再び求める処理を効率化するため、計算した値を MPFR の内部にキャッシュとして保持することがある。これらの数学定数関数は、他の関数の内部から実行される場合もある。また、MPFR は各種の計算を効率化するために、内部で記憶領域のプールを使用することもある。これらの領域は、個々の `mpfr_t` 型のオブジェクトが使用する領域とは別に管理されるため、`mpfr_clear` を実行しても解放されない。

MPFR が内部で使用するキャッシュやプールを解放するには、`mpfr_free_cache` を使用する。

書き方： `mpfr_free_cache()`

この関数は、長時間動作するプログラムにおいて、高精度計算の終了後に MPFR が使用していた領域を回収したい場合や、メモリリーク検査ツールを用いる前に MPFR が保持する領域を解放しておきたい場合などに利用できる。また、MPFR を使用したスレッドを終了する前のキャッシュの解放にも用いられる。

通常の短いプログラムでは、終了時に処理環境によって記憶領域が回収されるため、`mpfr_free_cache` を実行しなくても計算結果には影響しない。ただし、使用した領域を明示的に整理して終了する場合には、すべての計算を終えた後、プログラムの終了前に実行するとよい。

`mpfr_free_cache` は MPFR の処理中でも実行でき、繰り返し実行してもよい。既存の `mpfr_t` 型のオブジェクトが保持する値には影響せず、解放後に必要となったキャッシュは再び作成される。ただし、計算の途中で頻繁に実行すると、キャッシュの再作成によって処理効率が低下する。従って、MPFR を使用する一連の処理が終了した時点で実行するのが基本である。

22.4.2.14 mpfr_t の特殊な値

GMP の `mpfr_t` 型では、非数や無限大などの特殊な値を扱うことはできない。これに対して MPFR の `mpfr_t` 型では、通常の有限値に加えて、NaN（非数）、正負の無限大、および正負のゼロを扱うことができる。これらの特殊な値は、IEEE 754 の浮動小数点数と同様に、演算結果が通常の有限値として表せない場合や、値の符号を区別する必要がある場合などに用いられる。

例えば、0 を 0 で除算した結果や、同符号の無限大同士の差などは NaN となり、0 でない有限値を正または負のゼロで除算した結果は、分子と分母の符号に応じて、正または負の無限大となる。また、MPFR ではゼロにも符号があり、正のゼロと負のゼロを区別して扱うことができる。

特殊な値を明示的に設定するための関数を表 92 に示す。

表 92: `mpfr_t` 型に特殊な値を設定する関数

関 数	解 説
<code>mpfr_set_nan(x)</code>	<code>x</code> に NaN を設定する
<code>mpfr_set_inf(x, sign)</code>	<code>sign</code> の符号に応じて、 <code>x</code> に正または負の無限大を設定する
<code>mpfr_set_zero(x, sign)</code>	<code>sign</code> の符号に応じて、 <code>x</code> に正または負のゼロを設定する

`mpfr_set_inf` と `mpfr_set_zero` では、第 2 引数 `sign` が 0 以上の場合に正の値が設定され、負の場合に負の値が設定される。例えば、次のように記述する。

```
mpfr_set_inf(x, 1);
mpfr_set_inf(y, -1);
mpfr_set_zero(z, -1);
```

この場合、`x` には正の無限大、`y` には負の無限大、`z` には負のゼロが設定される。なお、`mpfr_set_nan` によって設定された NaN の符号ビットは規定されていない。

`mpfr_t` 型の値が特殊な値であるかどうかを判定するための主な関数を、表 93 に示す。これらの関数は、条件を満たす場合に 0 以外を、満たさない場合に 0 を返す。

表 93: `mpfr_t` 型の値を判定する主な関数

関 数	解 説
<code>mpfr_nan_p(x)</code>	<code>x</code> が NaN であるかを判定する
<code>mpfr_inf_p(x)</code>	<code>x</code> が正または負の無限大であるかを判定する
<code>mpfr_number_p(x)</code>	<code>x</code> が NaN でも無限大でもない有限値であるかを判定する
<code>mpfr_zero_p(x)</code>	<code>x</code> が正または負のゼロであるかを判定する
<code>mpfr_regular_p(x)</code>	<code>x</code> が NaN、無限大、ゼロのいずれでもない有限値であるかを判定する
<code>mpfr_signbit(x)</code>	<code>x</code> の符号ビットが設定されているかを判定する
<code>mpfr_unordered_p(x, y)</code>	<code>x</code> または <code>y</code> が NaN であり、両者が比較不能であるかを判定する

`mpfr_number_p` は、NaN と無限大を除く有限値であるかを判定するため、正負のゼロに対しても 0 以外を返す。これに対して `mpfr_regular_p` は、ゼロも除外した通常の非零有限値である場合に 0 以外を返す。従って、有限値であるかどうかだけを調べる場合は `mpfr_number_p` を使用し、さらにゼロでないことも確認する場合は `mpfr_regular_p` を使用する。

また、`mpfr_signbit` は、負の有限値や負の無限大だけでなく、負のゼロに対しても 0 以外を返す。このため、値が 0 であるかを `mpfr_zero_p` で確認した後に `mpfr_signbit` を使用すると、正のゼロと負のゼロを区別することができる。

■ NaN を想定した 2 値の比較の考え方

通常の実数同士を比較する場合、2 つの値 `x`, `y` の関係は、`x < y`, `x = y`, `x > y` のいずれかとなる。しかし、`x` または `y` が NaN である場合は、これらのいずれの関係も成立せず、両者は比較不能（unordered）であるとみなされる。従って、NaN が含まれる可能性のある 2 つの値を比較する場合は、まず `mpfr_unordered_p` を用いて、比較が可能である

かどうかを確認するとよい.

```
if (mpfr_unordered_p(x, y)) {
    /* x または y が NaN であり, 比較不能 */
} else if (mpfr_less_p(x, y)) {
    /* x < y */
} else if (mpfr_greater_p(x, y)) {
    /* x > y */
} else {
    /* x = y */
}
```

`mpfr_unordered_p(x,y)` は, `x` または `y` の少なくとも一方が NaN である場合に 0 以外を返す. NaN を含む比較では, `mpfr_less_p`, `mpfr_equal_p`, `mpfr_greater_p` はいずれも 0 を返す. 従って, これらの判定関数の戻り値を個別に調べるだけでは, 条件が成立しない場合と, NaN のため比較そのものが成立しない場合を区別できない. このため, 比較処理の前に `mpfr_unordered_p` を使用することには意味がある.

機能だけを見れば, 次の条件によっても, 同じく NaN が含まれていることを判定できる.

```
mpfr_nan_p(x) || mpfr_nan_p(y)
```

ただし, この記述は個々の値が NaN であるかを調べるものであるのに対して, `mpfr_unordered_p(x,y)` は, 2 つの値の比較関係が成立するかどうかを直接表す. 従って, 2 値の大小関係を判定する処理では, `mpfr_unordered_p` を使用する方が, プログラムの意図を明確に表すことができる.

22.4.2.15 書式整形や入出力のための関数

MPFR では, `mpfr_t` 型の値を書式整形して出力する関数や, ストリームおよび文字列との間で値を入出力する関数が用意されている. 主な書式整形出力関数を表 94 に示す.

表 94: MPFR の主な書式整形出力関数

関 数	解 説
<code>mpfr_printf(書式, ...)</code>	標準出力に書式整形して出力する
<code>mpfr_fprintf(stream, 書式, ...)</code>	指定したファイルストリームに書式整形して出力する
<code>mpfr_sprintf(s, 書式, ...)</code>	書式整形した結果を文字配列 <code>s</code> に格納する
<code>mpfr_snprintf(s, n, 書式, ...)</code>	最大 <code>n</code> バイトの範囲で文字配列 <code>s</code> に格納する
<code>mpfr_asprintf(&s, 書式, ...)</code>	必要な領域を動的に確保して文字列を作成する

`mpfr_printf` は, 標準 C ライブラリの `printf` と同様の形式で使えるが, `mpfr_t` 型の値を出力する場合には, 書式指定に文字 `R` を付ける.

書き方: `mpfr_printf( 書式文字列, 値, ... )`

例えば, 次の記述では, `x` を固定小数点形式で小数点以下 50 桁を表示する.

```
mpfr_printf( "x = %.50Rf\n", x );
```

`R` の後に指定する主な変換文字を次に示す.

- `Rf` : 固定小数点形式で表示する
- `Re` : 指数形式で表示する
- `Rg` : 値に応じて固定小数点形式または指数形式で表示する
- `Ra` : 16 進浮動小数点形式で表示する
- `Rb` : 2 進数形式で表示する

例えば,

```
mpfr_printf( "%.30Rf\n", x );    /* 固定小数点形式 */
mpfr_printf( "%.30Re\n", x );    /* 指数形式 */
mpfr_printf( "%.30Rg\n", x );    /* 自動選択 */
```

のように使用する. ここで, `.30` は, `Rf` と `Re` では小数点以下の桁数を, `Rg` では有効桁数を指定する. これは表示形式に関する指定であり, `mpfr_t` 型の変数が内部で保持するビット精度を変更するものではない.

書式指定の中で、出力時の丸め方法を指定することもできる。例えば、

```
mpfr_printf( "%.50Rnf\n", x );
mpfr_printf( "%.50Rdf\n", x );
```

では、それぞれ最近接丸め、負の無限大方向への丸めによって表示する。丸め方法を表す主な文字は、N, Z, U, D, Y であり、それぞれ MPFR_RNDN, MPFR_RNDZ, MPFR_RNDU, MPFR_RNDD, MPFR_RNDA に対応する。丸め方法を省略した場合には、最近接丸めが使用される。

ファイルへ書式整形して出力する場合には、mpfr_fprintf を使用する。

```
FILE *fp = fopen( "result.txt", "w" );
mpfr_fprintf( fp, "%.50Rf\n", x );
fclose( fp );
```

mpfr_printf や mpfr_fprintf の戻り値は、正常に出力された文字数であり、エラーが発生した場合には負の値となる。

書式整形した結果を文字配列に格納するには、mpfr_sprintf または mpfr_snprintf を使用する。

```
char s[100];
mpfr_snprintf( s, sizeof(s), "%.40Rf", x );
```

mpfr_sprintf は、格納先の文字配列の大きさを確認しないため、領域不足による問題を避けるには、最大バイト数を指定できる mpfr_snprintf を使用の方が安全である。mpfr_snprintf は、終端のヌル文字を含めて最大 n バイトの範囲に結果を格納する。戻り値は、十分な領域があった場合に書き込まれる文字数であり、終端のヌル文字は含まれない。従って、戻り値が n 以上の場合には、文字列が途中までしか格納されていないことが分かる。

必要な大きさの文字列領域を自動的に確保するには、mpfr_asprintf を使用する。

```
char *s;
if ( mpfr_asprintf( &s, "%.100Rf", x ) >= 0 ) {
    printf( "%s\n", s );
    mpfr_free_str( s );
}
```

注意) mpfr_asprintf によって確保された文字列は、使用後に mpfr_free_str を用いて解放する必要がある。通常の free を使用してはならない。

なお、mpfr_fprintf など、FILE * 型の引数を取る MPFR の入出力関数を使用する場合には、次のように標準入出力用のヘッダファイル stdio.h を mpfr.h より前に読み込む。

```
#include <stdio.h>
#include <mpfr.h>
```

MPFR の値をストリームへ直接出力したり、ストリームから読み込んだりする主な関数を、表 95 に示す。

表 95: MPFR の主なストリーム・文字列入出力関数

関 数	解 説
mpfr_out_str(stream, base, n, x, rnd)	x を指定した基数と有効桁数でストリームに出力する
mpfr_inp_str(x, stream, base, rnd)	ストリームから数値を読み取り、x に格納する
mpfr_get_str(s, exp, base, n, x, rnd)	x の仮数を文字列に変換し、指数を exp に格納する
mpfr_get_str_ndigits(base, prec)	prec ビットの値を文字列から復元するために必要な桁数を求める
mpfr_free_str(s)	MPFR が確保した文字列用の記憶領域を解放する

書式を細かく指定する必要がない場合には、mpfr_out_str を用いて、値をストリームへ直接出力することができる。

書き方： mpfr_out_str(ストリーム, 基数, 桁数, 値, 丸め方法)

例えば、次の記述では、x を 10 進数で有効数字 50 桁に丸めて出力する。

```
mpfr_out_str( stdout, 10, 50, x, MPFR_RNDN );
putchar( '\n' );
```

第 3 引数 n に 0 を指定すると、x の精度に基づいて、出力時と再読込時に最近接丸めを使用した場合に、元の値を復元できるだけの桁数が自動的に選ばれる。

```
mpfr_out_str( stdout, 10, 0, x, MPFR_RNDN );
```

mpfr_out_str の戻り値は、出力した文字数であり、エラーが発生した場合には 0 となる。

通常の有限値は、最上位の数字の直後に小数点を置き、その後に残りの数字と指数を付けた形式で出力される。このとき、指数自体は 10 進数で表される。

ストリームから数値を読み込むには、mpfr_inp_str を使用する。

書き方： mpfr_inp_str(変数, ストリーム, 基数, 丸め方法)

例えば、標準入力から 10 進数を読み込む場合は、次のように記述する。

```
mpfr_inp_str( x, stdin, 10, MPFR_RNDN );
```

読み込んだ値は、x の精度と指定した丸め方法に従って丸められる。戻り値は読み込んだバイト数であり、読み込みに失敗した場合には 0 となる。外部から与えられる入力を扱う場合には、戻り値を確認するとよい。

mpfr_get_str は、mpfr_t 型の値を仮数部の文字列と指数に分けて取り出す関数である。

書き方： mpfr_get_str(文字列, 指数, 基数, 桁数, 値, 丸め方法)

例えば、次のように使用する。

```
char *s;
mpfr_exp_t exp;
s = mpfr_get_str( NULL, &exp, 10, 20, x, MPFR_RNDN );
printf( "s=%s, exp=%ld\n", s, (long)exp );
mpfr_free_str( s );
```

第 1 引数に NULL を指定すると、必要な文字列領域が MPFR によって確保される。使用後には mpfr_free_str で解放する必要がある。

mpfr_get_str が返す文字列には、小数点は含まれない。小数点は先頭の数字の直前にあるものとして解釈し、第 2 引数で得られる指数を用いる。例えば、10 進数の -3.1416 を 5 桁の仮数として取り出した場合には、文字列として `"-31416"`、指数として 1 が得られる。これは、

$$-0.31416 \times 10^1 = -3.1416$$

を表す。

第 4 引数 n に 0 を指定すると、変数の精度に応じた桁数が自動的に選ばれる。また、必要な桁数だけをあらかじめ求める場合には、mpfr_get_str_ndigits を使用する。

```
size_t n = mpfr_get_str_ndigits( 10, mpfr_get_prec(x) );
```

この関数が返す桁数は、指定したビット精度の値を文字列へ変換し、最近接丸めで再び読み込んだときに、元の値を復元するために必要な桁数である。

22.4.2.16 mpfr_t 型の乱数

MPFR では、mpfr_t 型の多倍長浮動小数点数として、一様分布、正規分布、指数分布に従う疑似乱数を生成する関数が用意されている。乱数の生成には、GMP の場合と同様に、gmp_randstate_t 型の乱数生成器を使用する。乱数生成に関する主な関数を表 96 に示す。

表 96: MPFR の乱数生成に関する主な関数

関 数	解 説
mpfr_urandomb(rop, state)	0 以上 1 未満の一様乱数を rop に格納する
mpfr_urandom(rop, state, rnd)	区間 $[0, 1]$ の連続一様分布に基づく乱数を rop に格納する
mpfr_nrandom(rop, state, rnd)	標準正規分布に従う乱数を rop に格納する
mpfr_erandom(rop, state, rnd)	平均 1 の指数分布に従う乱数を rop に格納する

MPFR の乱数生成関数では、乱数生成器の初期化、種の設定、解放に、GMP の乱数生成で使用したのと同じ関数を用いる。例えば、乱数生成器を標準の方式で初期化し、種を設定する処理は次のようになる。

```

gmp_randstate_t state;
gmp_randinit_default(state);
gmp_randseed_ui(state, seed);

```

乱数の生成が終了した後は、`gmp_randclear` を用いて、乱数生成器が使用している領域を解放する。

```

gmp_randclear(state);

```

同じ版の GMP および MPFR を使用し、同じ乱数生成方式に同じ種を設定して、格納先の精度、丸め方法、指数範囲、関数の実行順序などの条件も同じであれば、同じ順序の疑似乱数を再現できる。ただし、格納先の精度などの条件を変更すると、生成される値だけでなく、それ以降の乱数系列も変化する可能性がある。また、GMP および MPFR の標準的な乱数生成器は、暗号学的な予測困難性を保証するものではないため、暗号鍵の生成などの用途には使用しないこと。

`mpfr_urandom` は、区間 $[0, 1]$ の連続一様分布から得られた実数を、指定した丸め方法で `mpfr_t` 型の値に丸めたものとみなせる乱数を生成する関数である。例えば、次の処理では、一様乱数を生成して `x` に格納する。

```

mpfr_urandom(x, state, MPFR_RNDN);

```

第 1 引数には、初期化済みの `mpfr_t` 型の格納先を指定する。生成される値の精度は、`x` に設定されている精度によって決まる。第 2 引数には、初期化と種の設定を済ませた乱数生成器を指定し、第 3 引数には丸め方法を指定する。通常は最近接丸め `MPFR_RNDN` を使用するとよい。

`mpfr_urandomb` も 0 以上 1 未満の一様乱数を生成する関数である。

```

mpfr_urandomb(x, state);

```

この関数は、格納先の精度に対応するランダムな仮数部を生成する方式であり、丸め方法を指定しない。連続一様分布から得られた実数を指定した方法で丸めた値として扱いたい場合は、`mpfr_urandom` を使用すると分かりやすい。一方、`mpfr_urandomb` は、0 以上 1 未満という範囲が明確で、丸め方法を指定せずに一様な浮動小数点乱数を生成したい場合に利用できる。

区間 $[0, 1]$ 以外の一様乱数が必要な場合は、生成した乱数を変換する。 u を区間 $[0, 1]$ の一様乱数とすると、区間 $[a, b]$ に対応する値は、次の式で求められる。

$$x = a + (b - a)u$$

例えば、区間 $[0, 1]$ の乱数が `x` に格納されている場合、区間 $[10, 20]$ に変換するには、次のように記述できる。

```

mpfr_mul_ui(x, x, 10, MPFR_RNDN);
mpfr_add_ui(x, x, 10, MPFR_RNDN);

```

一般の `mpfr_t` 型の値 a , b を使用する場合は、作業用のオブジェクトに $b - a$ を求め、乗算と加算を行う。

`mpfr_nrandom` は、平均 0、分散 1 の標準正規分布に従う乱数を生成する関数である。例えば、次の処理では、標準正規乱数を生成して `x` に格納する。

```

mpfr_nrandom(x, state, MPFR_RNDN);

```

標準正規分布の標準偏差は 1 である。平均を μ 、標準偏差を σ とする正規分布に従う乱数が必要な場合は、標準正規乱数 z を用いて、

$$x = \mu + \sigma z$$

と変換する。例えば、平均 50、標準偏差 10 の正規乱数は、次のように生成できる。

```

mpfr_nrandom(x, state, MPFR_RNDN);
mpfr_mul_ui(x, x, 10, MPFR_RNDN);
mpfr_add_ui(x, x, 50, MPFR_RNDN);

```

より一般に、平均と標準偏差が `mpfr_t` 型の値として与えられる場合は、`mpfr_mul` と `mpfr_add` を使用する。

`mpfr_erandom` は、平均 1 の指数分布に従う乱数を生成する関数である。この関数も、`mpfr_nrandom` と同様に、第 3 引数で丸め方法を指定する。指数分布については、必要に応じて平均値に応じた倍率を掛けて利用できる。

かつて標準正規乱数の生成に使用された `mpfr_grandom` も存在するが、この関数は現在では非推奨であり、将来の版で削除される予定である。標準正規乱数の生成には、処理効率にも優れる `mpfr_nrandom` を使用するとよい。

乱数を格納する `mpfr_t` 型のオブジェクトは、必要な精度を指定して初期化する。例えば、256 ビット精度で乱数を生成する場合は、次のように記述する。

```
mpfr_t x;
mpfr_init2(x, 256);
mpfr_urandom(x, state, MPFR_RNDN);
```

生成された値は、格納先の精度に従って丸められる。多くの有効ビットを持つ乱数が必要な場合は、格納先の `mpfr_t` 型のオブジェクトも、それに応じた精度で初期化する必要がある。使用後は通常どおり `mpfr_clear` を呼び出して領域を解放する。

次に示すプログラム `test271-2.c` は、区間 $[0, 1]$ の一様乱数を 10 個生成し、小数点以下 30 桁の形式で出力するものである。

プログラム：test271-2.c

```
1  #include    <stdio.h>
2  #include    <stdlib.h>
3  #include    <errno.h>
4  #include    <mpfr.h>
5
6  int main( int argc, char *argv[] ) {
7      unsigned long seed;
8      char *end;
9
10     gmp_randstate_t state;
11     mpfr_t x1, x2;
12
13     if (argc != 2) {
14         fprintf(stderr, "usage: %s Seed\n", argv[0]);
15         return 1;
16     }
17
18     errno = 0;
19     end = NULL;
20     seed = strtoul(argv[1], &end, 10);
21
22     if (errno == ERANGE || argv[1][0] == '-' || end == argv[1] || *end != '\0') {
23         fprintf(stderr, "Give Seed of unsigned long int!\n");
24         return 1;
25     }
26
27     /* 乱数生成器を初期化し、種を設定する */
28     gmp_randinit_default(state);
29     gmp_randseed_ui(state, seed);
30
31     /* 乱数を格納するオブジェクトを128ビット精度で初期化する */
32     mpfr_inits2(128, x1, x2, (mpfr_ptr)0);
33
34     /* 一様乱数を10個生成し、5行2列で表示する */
35     for (int i = 0; i < 5; ++i) {
36         mpfr_urandom(x1, state, MPFR_RNDN);
37         mpfr_urandom(x2, state, MPFR_RNDN);
38
39         mpfr_printf("%.30Rf, %.30Rf\n", x1, x2);
40     }
41
42     /* 使用した領域を解放する */
43     mpfr_clears(x1, x2, (mpfr_ptr)0);
44     gmp_randclear(state);
45 }
```

このプログラムでは、コマンドライン引数として与えられた値を乱数の種として設定する。次に、乱数を格納する 2 個の `mpfr_t` 型のオブジェクトを 128 ビット精度で初期化し、`mpfr_urandom` を用いて区間 $[0, 1]$ の一様乱数を生成する。乱数は 2 個ずつ生成され、`mpfr_printf` の書式 `%.30Rf` によって、小数点以下 30 桁の形式で 1 行に 2 個ずつ出力する。最後に、多倍長浮動小数点数のオブジェクトと乱数生成器が使用している領域を解放する。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test271-2.c のコンパイル, リンクと実行

```
$ gcc test271-2.c -lmpfr -lgmp -o test271-2  ←コンパイルと MPFR, GMP のリンク
$ ./test271-2 1234  ←種を 1234 として実行
0.401017234516789330117630646189, 0.181447657629325891109964864568 ←出力
0.635452510623115525795793927258, 0.038966678200145852170079246446
0.158619810164255184979564732449, 0.585788843777034391433573825356
0.125997329210589921167791416233, 0.754962485187126141231063328690
0.487616226934103590378174664445, 0.733069650911138512445524951441
```

注) 生成される乱数の系列は実行環境ごとに異なることがある。

22.4.2.17 10 進桁数による精度の指定

MPFR では, `mpfr_t` 型のオブジェクトの精度を, 仮数部のビット数で指定する. 一方, 高精度計算を行う場合は, 「10 進数で何桁程度の精度が必要か」という形で精度を考えることも多い. このため, 必要な 10 進有効桁数を, MPFR で使用するビット精度へ変換する方法を知っておくと便利である.

10 進有効桁数を表す正の整数を d , これに対応するビット精度を p とする. 10 進数で d 桁分の精度を表すために必要な 2 進数の桁数は, およそ $d \log_2 10$ ビットである. 従って, 両者の関係は次のようになる.

$$p = \lceil d \log_2 10 \rceil$$

ここで, $\lceil x \rceil$ は, x 以上の最小の整数を表す. また, $\log_2 10$ の値は, およそ次のとおりである.

$$\log_2 10 \simeq 3.321928094887$$

例えば, 10 進 20 桁に対応するビット精度は,

$$\lceil 20 \log_2 10 \rceil = 67$$

となる. 同様に, 10 進 40 桁では 133 ビット, 10 進 60 桁では 200 ビットとなる. これらは, 指定した 10 進有効桁数に相当する情報を保持するための, おおよその最低限のビット数である.

ただし, これは 10 進桁数に対応する情報量を保持するためのビット精度であり, 指定した桁数の 10 進小数を 2 進浮動小数点数として厳密に表現できることを意味するものではない.

この換算をプログラム中で行う場合, $\log_2$ などを用いて浮動小数点演算によって計算することもできる. しかし, ビット精度を決めるための計算自体に丸め誤差が入り, 必要なビット数より小さい値が得られることは避けたい. そこで, 後に示す関数 `digits2bits` では, 次の関係を利用する.

$$\log_2 10 < \frac{332193}{100000} = 3.32193$$

右辺は $\log_2 10$ よりわずかに大きい値である. 従って,

$$p = \left\lceil d \frac{332193}{100000} \right\rceil$$

として求めれば, 必要なビット数を下回らない値を, 整数演算だけで得ることができる. `digits2bits` では, 積を単純に計算すると整数型の範囲を超える可能性があるため, 桁数を商と余りに分けて計算し, 乗算や加算のオーバーフローも検査している.

ただし, このようにして得られるビット数は, 指定した 10 進桁数に対応する最低限の目安である. 複数の演算を行う場合は, 途中の丸め誤差や桁落ちによって, 最終結果の有効桁数が減少することがある. このため, 実際の演算では, 換算して得られたビット数よりも少し高い精度を使用することがある.

このように, 必要な精度に余裕を持たせるために追加するビットを, **ガードビット** (guard bits) と呼ぶことがある. 必要なガードビット数は計算内容によって異なり, 一定の値で十分であるとは限らない. 単純な計算では数ビットから数十ビット程度の余裕を加える方法が考えられるが, 多数の演算を繰り返す場合や, 近い値同士の減算によって桁落ちが起こる場合には, より大きな精度が必要になることもある.

次に示すプログラム `test271-3.c` は, 10 進有効桁数を MPFR のビット精度へ変換する関数 `digits2bits` を定義し, 10 進 20 桁, 40 桁, 60 桁の精度で円周率を求めて出力するものである. この例では, 換算して得られたビット数に 16 ビットのガードビットを加え, その値を実際の演算精度として使用する.

プログラム：test271-3.c

```

1  #include    <stdio.h>
2  #include    <stdint.h>
3  #include    <mpfr.h>
4  #define     GUARD_BITS      16
5
6  /*
7   * 10進有効桁数 digits を、MPFR で使用するビット精度に変換する。
8   * 変換に成功した場合は1を返し、結果を*prec に格納する。
9   * 指定が不正な場合や、MPFR の精度の範囲を超える場合は0を返す。
10  */
11 static int digits2bits(mpfr_prec_t *prec, uintmax_t digits) {
12     const uintmax_t numerator    = UINTMAX_C(332193);
13     const uintmax_t denominator = UINTMAX_C(100000);
14
15     uintmax_t quotient, remainder;
16     uintmax_t product, extra, bits;
17
18     if (prec == NULL || digits == 0) return 0;
19
20     /* digits を商と余りに分けて、ceil(digits * numerator / denominator)
21      * をオーバーフローに注意しながら求める。 */
22     quotient = digits / denominator;
23     remainder = digits % denominator;
24
25     if (quotient > UINTMAX_MAX / numerator) return 0;
26
27     bits = quotient * numerator;
28
29     if (remainder > UINTMAX_MAX / numerator) return 0;
30
31     product = remainder * numerator;
32     extra   = product / denominator;
33
34     if (product % denominator != 0) ++extra;
35
36     if (bits > UINTMAX_MAX - extra) return 0;
37
38     bits += extra;
39
40     if (bits < (uintmax_t)MPFR_PREC_MIN) bits = (uintmax_t)MPFR_PREC_MIN;
41
42     if (bits > (uintmax_t)MPFR_PREC_MAX) return 0;
43
44     *prec = (mpfr_prec_t)bits;
45     return 1;
46 }
47
48 int main() {
49     int n;
50     mpfr_prec_t base_prec, work_prec;
51     mpfr_t pi;
52
53     for (n = 20; n <= 60; n += 20) {
54         /* 10進 n 桁に対応する最低限のビット精度を求める */
55         if (!digits2bits(&base_prec, (uintmax_t)n)) {
56             fprintf(stderr, "精度をビット数に変換できません. \n");
57             return 1;
58         }
59         /* 演算途中の丸め誤差に備えて余裕ビットを加える */
60         if (base_prec > MPFR_PREC_MAX - GUARD_BITS) {
61             fprintf(stderr, "演算精度が大きすぎます. \n");
62             return 1;
63         }
64         work_prec = base_prec + GUARD_BITS;    /* 実際に使用する精度 */
65
66         /* 指定したビット精度で円周率を求める */
67         mpfr_init2(pi, work_prec);
68         mpfr_const_pi(pi, MPFR_RNDN);
69
70         mpfr_printf("digits:%d, -> converted bits:%Pd, "

```

```

71         "-> bit-length for prec.:%Pd\n", n, base_prec, work_prec);
72
73     /* 円周率の整数部は1桁なので，小数点以下をn-1桁表示する．*/
74     mpfr_printf("pi = %.*RNf\n", n - 1, pi);
75
76     mpfr_clear(pi);      /* 次回に備えて変数を廃棄 */
77 }
78
79     mpfr_free_cache();
80 }

```

関数 `digits2bits` は，第2引数に与えられた10進有効桁数をビット精度へ変換し，その値を第1引数が指す変数に格納する．変換に成功した場合は1を返し，指定された桁数が不正な場合，整数演算でオーバーフローが生じる場合，または求めた精度がMPFRで使用できる範囲を超える場合は0を返す．

`main` 関数では，変数 `n` の値を20から60まで20ずつ増加させながら，`digits2bits` によって必要なビット数を求める．次に，その値にガードビットとして16ビットを加えた精度で `pi` を初期化し，`mpfr_const_pi` によって円周率を求める．円周率の整数部は1桁であるため，出力時には小数点以下を $n-1$ 桁表示することで，全体を n 桁の10進有効数字としている．

各回の出力後には `mpfr_clear` によって `pi` の領域を解放する．また，`mpfr_const_pi` などが内部で使用するキャッシュを解放するため，プログラムの終了前に `mpfr_free_cache` を実行している．

このプログラムをコンパイル，リンクして実行する例を次に示す．

例. `test271-3.c` のコンパイル，リンクと実行

```

$ gcc test271-3.c -lmpfr -lgmp -o test271-3  [Enter]    ←コンパイルとMPFR, GMPのリンク
$ ./test271-3  [Enter]    ←実行
digits:20, -> converted bits:67, -> bit-length for prec.:83    ←出力
pi = 3.1415926535897932385    ←20桁分のπ
digits:40, -> converted bits:133, -> bit-length for prec.:149
pi = 3.141592653589793238462643383279502884197    ←40桁分のπ
digits:60, -> converted bits:200, -> bit-length for prec.:216
pi = 3.14159265358979323846264338327950288419716939937510582097494    ←60桁分のπ

```

この例では，10進20桁，40桁，60桁に対応する最低限の精度として，それぞれ67ビット，133ビット，200ビットが得られている．実際の演算では，これらに16ビットを加えた83ビット，149ビット，216ビットを使用している．

なお，10進桁数をビット精度へ換算したからといって，あらゆる計算結果について，指定した桁数の正しさが保証されるわけではない．得られる有効桁数は，計算の手順，演算回数，桁落ちの有無，丸め方法などにも依存する．精度が十分であるかを確認する必要がある場合は，異なるビット精度で同じ計算を行い，結果の一致する桁数を比較する方法が有効である．

22.4.3 MPC

MPC (Multiple Precision Complex library) は，多倍長の複素数を扱うためのC言語用ライブラリである．MPCはMPFRを基盤としており，複素数の実部と虚部をそれぞれMPFRの多倍長浮動小数点数として保持する．このため，MPFRと同様に，任意のビット精度を指定して計算を行うことができ，演算結果に対する丸め方法も明確に指定することができる．

MPCは，任意精度の複素数計算に対して，処理系や実行環境に依存しにくい，明確な演算規則を提供することを目的として開発された．通常の固定精度の複素数型では，利用できる精度が処理系によって定まり，MPCのような演算ごとの明示的な丸め指定も一般には行われない．これに対してMPCでは，MPFRが実数の浮動小数点演算に対して実現した考え方を複素数へ拡張し，実部と虚部を，それぞれ指定された精度と丸め方法に従って正しく丸めることを重視している．また，無限大，NaN，符号付きの0を含む複素数についても，明確な演算規則を定めている．高精度での演算を効率良く実行することも，重要な設計目標の一つとされている．

MPCの開発は2000年代初頭に始まり，2012年に公開された版1.0から，GNUプロジェクトのソフトウェアとして配布されている．ただし，MPCはMPFRライブラリの一部として実装されているわけではなく，独立したライブラリとして開発，配布されている．MPFRとは開発者や技術的な基盤を共有しており，両者の開発は密接な関係を持つ．

MPC は、GMP および MPFR の上位に位置するライブラリである。整数や有理数の多倍長演算は GMP が担い、正しく丸められた多倍長実数演算は MPFR が担い、その機能を利用して多倍長複素数演算を実現するのが MPC である。従って、MPC を使用するには GMP と MPFR も必要である。

MPC は、基本的な四則演算だけでなく、平方根、累乗、指数関数、対数関数、三角関数、双曲線関数など、複素数に対する主要な数学関数を備えている。また、実部、虚部、絶対値、偏角、共役複素数などを求める機能や、文字列およびストリームとの間で値を入出力する機能も提供している。このため、数値解析、計算数学、数式処理システムなど、高い精度と再現性が要求される複素数計算の基盤として利用されている。

なお、MPC における精度、丸め方法、特殊な値などの基本的な考え方は MPFR と共通している。本節では、これらの詳細については前節の MPFR の解説を前提とし、MPC に固有の型、複素数の実部と虚部の扱い、複素数用の演算関数などを中心に説明する。

22.4.3.1 MPC の型とオブジェクトの管理

MPC では、多倍長複素数を保持する型として `mpc_t` が定義されている。この型のオブジェクトは、実部と虚部をそれぞれ MPFR の多倍長浮動小数点数として保持する。従って、実部と虚部には同じ精度を設定することも、それぞれ異なる精度を設定することもできる。精度を表す値には、MPFR で定義されている `mpfr_prec_t` 型を使用する。

MPFR の場合と同様に、`mpc_t` 型のオブジェクトを使用する場合は、値を設定する前に初期化を行い、使用後にはオブジェクトが使用しているメモリ領域を解放する必要がある。MPC のオブジェクトの初期化、廃棄、精度の設定および取得に使用する主な関数を表 97 に示す。

表 97: MPC のオブジェクトの管理と精度に関する主な関数

関 数	解 説
<code>mpc_init2(z, prec)</code>	<code>z</code> を初期化し、実部と虚部の精度をともに <code>prec</code> ビットにする
<code>mpc_init3(z, prec_r, prec_i)</code>	<code>z</code> を初期化し、実部と虚部の精度をそれぞれ <code>prec_r</code> , <code>prec_i</code> ビットにする
<code>mpc_clear(z)</code>	<code>z</code> が使用しているメモリ領域を解放する
<code>mpc_set_prec(z, prec)</code>	<code>z</code> の実部と虚部の精度をともに <code>prec</code> ビットに変更する
<code>mpc_get_prec(z)</code>	実部と虚部の精度が同じ場合はその精度を返し、異なる場合は 0 を返す
<code>mpc_get_prec2(&prec_r, &prec_i, z)</code>	<code>z</code> の実部と虚部の精度をそれぞれ取得する

`mpc_init2` は、実部と虚部に同じ精度を設定してオブジェクトを初期化する関数である。例えば、次の記述では、`z` の実部と虚部の精度をともに 256 ビットに設定する。

```
mpc_t z;
mpc_init2(z, 256);
```

一方、`mpc_init3` を使用すると、実部と虚部に異なる精度を設定することができる。次の例では、実部の精度を 256 ビット、虚部の精度を 128 ビットに設定する。

```
mpc_t z;
mpc_init3(z, 256, 128);
```

ただし、一般的な用途では、実部と虚部に同じ精度を設定する `mpc_init2` を使用するとよい。実部と虚部に異なる精度を設定する機能は、両者に対して個別の精度管理を必要とする場合に使用する。

MPFR には、既定の精度をあらかじめ設定し、その精度を用いてオブジェクトを初期化する機能がある。これに対して MPC には、既定の精度を設定する機能はない。精度は `mpc_init2` または `mpc_init3` によって各オブジェクトに個別に設定する。また、MPFR と同様に、丸め方法は各演算関数を呼び出す際に指定する。従って、MPC では、使用する精度と丸め方法がプログラム中に明示される。

`mpc_init2` および `mpc_init3` による初期化直後には、実部と虚部の値として非数 NaN が設定される。従って、初期化後には、後述する値の設定関数を用いるか、演算関数によって値を格納してから、入力値として使用する。

使用を終えたオブジェクトに対しては、次のように `mpc_clear` を呼び出す。

```
mpc_clear(z);
```

これは、`mpc_t` 型のオブジェクトが内部で使用しているメモリ領域を解放するための処理である。初期化した全ての

`mpc_t` 型のオブジェクトに対して、使用後に `mpc_clear` を呼び出す必要がある。基本的な使用の流れは、次のようになる。

```
mpc_t z;
mpc_init2(z, 256);
/* z に値を設定して演算を行う */
mpc_clear(z);
```

同じオブジェクトに対して初期化を繰り返してはならない。再び初期化する必要がある場合は、先に `mpc_clear` を呼び出す。また、反復処理のたびに初期化と解放を繰り返すと効率が低下するため、通常は反復処理の前に初期化し、反復処理の終了後に解放する。

`mpc_set_prec` は、初期化済みのオブジェクトの実部と虚部の精度を変更する関数である。

```
mpc_set_prec(z, 512);
```

この例では、`z` の実部と虚部の精度をともに 512 ビットに変更する。ただし、この関数を呼び出すと、それまでオブジェクトに格納されていた値は失われ、実部と虚部には NaN が設定される。従って、格納されている値を新しい精度に丸めて保持する関数ではない点に注意する必要がある。この処理は、`mpc_clear` でオブジェクトを廃棄してから `mpc_init2` で再び初期化する処理に相当する。ただし、既に確保されているメモリ領域が十分な場合は再割当てを行わないため、この方法の方が効率が良い。

`mpc_get_prec` は、実部と虚部の精度が同じ場合に、その精度を返す。両者の精度が異なる場合は 0 を返す。実部と虚部の精度を個別に取得する場合は、`mpc_get_prec2` を使用する。

```
mpfr_prec_t prec_r, prec_i;
mpc_get_prec2(&prec_r, &prec_i, z);
```

この処理によって、実部の精度が `prec_r` に、虚部の精度が `prec_i` に格納される。

22.4.3.2 値の設定と取得

初期化済みの `mpc_t` 型のオブジェクトには、MPC の複素数、他の多倍長数値型、C 言語の数値型などから値を設定することができる。値を設定する主な関数を表 98 に示す。これらの関数の多くは、第 1 引数に値の設定先を取り、最後の引数に丸め方法²⁰⁰ を取る。

`mpc_set` は、`mpc_t` 型のオブジェクトに格納されている複素数の実部と虚部を、別の `mpc_t` 型のオブジェクトに設定する関数である。

```
mpc_set(z2, z1, MPC_RNDNN);
```

設定される値は、設定先である `z2` の実部と虚部の精度に従って丸められる。従って、`z1` と `z2` の精度が異なる場合には、関数実行後に両者の値が完全には一致しないことがある。`mpc_set` を含む MPC の多くの関数は、設定先のオブジェクトを第 1 引数に取り、最後の引数に丸め方法を取る。

MPC では、複素数の実部と虚部に対する丸め方法を組にして指定する。丸め方法を表す型は `mpc_rnd_t` であり、その値は `MPC_RNDNN` などの定数で指定する。`MPC_RNDNN` の最初の `N` は実部、2 番目の `N` は虚部を最近接丸めとすることを表す。通常の計算では、実部と虚部をともに最近接丸めとする `MPC_RNDNN` を使用すればよい。その他の場合も、実部と虚部のそれぞれについて、MPFR と同じ丸め方法を組み合わせて指定する。

`mpc_set_ui`, `mpc_set_si`, `mpc_set_d` など、1 個の実数値を受け取る関数は、その値を実部に設定し、虚部を正の符号を持つ 0 に設定する。例えば、次の処理では、`z` に複素数 $3.5 + 0i$ が設定される。

```
mpc_set_d(z, 3.5, MPC_RNDNN);
```

同様に、GMP および MPFR の値を設定する場合は、`mpc_set_z`, `mpc_set_q`, `mpc_set_f`, `mpc_set_fr` を使用する。これらの関数でも、設定した値は実部に格納され、虚部は 0 になる。

実部と虚部を同時に設定する場合は、関数名の型を表す部分が 2 個並んだ関数を使用する。例えば、次の処理では、`z` に複素数 $1.25 - 2.5i$ が設定される。

```
mpc_set_d_d(z, 1.25, -2.5, MPC_RNDNN);
```

²⁰⁰後の「22.4.3.3 MPC の丸め方法」(p.495) で解説する。

表 98: MPC の値を設定する主な関数

関 数	解 説
<code>mpc_set(rop, op, rnd)</code>	<code>mpc_t</code> 型の <code>op</code> の実部と虚部を <code>rop</code> に設定する
<code>mpc_set_ui(rop, op, rnd)</code>	<code>unsigned long int</code> 型の値を実部に設定する
<code>mpc_set_si(rop, op, rnd)</code>	<code>long int</code> 型の値を実部に設定する
<code>mpc_set_d(rop, op, rnd)</code>	<code>double</code> 型の値を実部に設定する
<code>mpc_set_ld(rop, op, rnd)</code>	<code>long double</code> 型の値を実部に設定する
<code>mpc_set_z(rop, op, rnd)</code>	<code>mpz_t</code> 型の値を実部に設定する
<code>mpc_set_q(rop, op, rnd)</code>	<code>mpq_t</code> 型の値を実部に設定する
<code>mpc_set_f(rop, op, rnd)</code>	<code>mpf_t</code> 型の値を実部に設定する
<code>mpc_set_fr(rop, op, rnd)</code>	<code>mpfr_t</code> 型の値を実部に設定する
<code>mpc_set_ui_ui(rop, re, im, rnd)</code>	<code>unsigned long int</code> 型の値から実部と虚部を設定する
<code>mpc_set_si_si(rop, re, im, rnd)</code>	<code>long int</code> 型の値から実部と虚部を設定する
<code>mpc_set_d_d(rop, re, im, rnd)</code>	<code>double</code> 型の値から実部と虚部を設定する
<code>mpc_set_ld_ld(rop, re, im, rnd)</code>	<code>long double</code> 型の値から実部と虚部を設定する
<code>mpc_set_z_z(rop, re, im, rnd)</code>	<code>mpz_t</code> 型の値から実部と虚部を設定する
<code>mpc_set_q_q(rop, re, im, rnd)</code>	<code>mpq_t</code> 型の値から実部と虚部を設定する
<code>mpc_set_f_f(rop, re, im, rnd)</code>	<code>mpf_t</code> 型の値から実部と虚部を設定する
<code>mpc_set_fr_fr(rop, re, im, rnd)</code>	<code>mpfr_t</code> 型の値から実部と虚部を設定する
<code>mpc_set_nan(rop)</code>	<code>rop</code> の実部と虚部をともに非数 NaN にする
<code>mpc_swap(op1, op2)</code>	<code>op1</code> と <code>op2</code> の値を効率良く交換する

また、MPFR の 2 個の値を実部と虚部に設定する場合は、次のように `mpc_set_fr_fr` を使用する。

```
mpfr_t re, im;
mpc_t z;
/* re, im, z の初期化と re, im への値の設定 */
mpc_set_fr_fr(z, re, im, MPC_RNDNN);
```

この場合も、`re` と `im` の値は、それぞれ `z` の実部と虚部の精度に従って丸められる。

`mpc_set_d` や `mpc_set_d_d` を使用する場合、引数として与える値は、まず C 言語の `double` 型で表現された値である。例えば、10 進小数の 0.1 は `double` 型では正確に表現できないため、既に近似値となっている。その値をさらに `mpc_t` 型の精度に変換することになるため、10 進表記から直接、多倍長精度の値が生成されるわけではない点に注意する必要がある。高い精度で 10 進小数を設定する場合は、後述する文字列から値を設定する関数を使用するとよい。

`mpc_swap` は、2 個のオブジェクトの内容を効率良く交換する関数である。この関数では値だけでなく、実部と虚部の精度も交換される。従って、両者の精度が異なる場合は、補助変数と `mpc_set` を用いて値だけを交換する処理とは結果が異なる点に注意する必要がある。

MPC の値を C 言語の複素数型として取得する主な関数を表 99 に示す。これらの関数を使用するには、`mpc.h` より前に `complex.h` を読み込む必要がある。

表 99: MPC の値を取得する主な関数

関 数	解 説
<code>mpc_get_dc(op, rnd)</code>	<code>op</code> の値を <code>double complex</code> 型として取得する
<code>mpc_get_ldc(op, rnd)</code>	<code>op</code> の値を <code>long double complex</code> 型として取得する

例えば、次の処理では、`mpc_t` 型の値を `double complex` 型の変数 `w` に変換する。

```
double complex w;
w = mpc_get_dc(z, MPC_RNDNN);
```

この変換では、 z の実部と虚部が、それぞれ `double` 型で表現できる精度と範囲に合わせて変換される。従って、 z が高い精度や広い数値範囲を持っていたとしても、取得した値の実部と虚部は、`double` 型または `long double` 型の精度と範囲に制限される。

なお、実部と虚部を個別に MPFR の値として取得する関数については、次項で説明する。文字列からの値の設定および文字列への変換については、後の「書式整形と入出力」で説明する。

22.4.3.3 MPC の丸め方法

MPC で使用する主な丸め方法を表 100 に示す。定数名の末尾にある 2 文字は、それぞれ実部と虚部に適用する丸め方法を表す。

表 100: MPC の主な丸め方法 (一部)	
定 数	丸め方法
<code>MPC_RNDNN</code>	実部と虚部をともに最も近い値へ丸める
<code>MPC_RNDZZ</code>	実部と虚部をともに 0 の方向へ丸める
<code>MPC_RNDUU</code>	実部と虚部をともに正の無限大の方向へ丸める
<code>MPC_RNDDD</code>	実部と虚部をともに負の無限大の方向へ丸める
<code>MPC_RNDAA</code>	実部と虚部をともに 0 から遠ざかる方向へ丸める

末尾の文字 `N`, `Z`, `U`, `D`, `A` は、それぞれ MPFR の `MPFR_RNDN`, `MPFR_RNDZ`, `MPFR_RNDU`, `MPFR_RNDD`, `MPFR_RNDA` に対応する。MPC には、これら 5 種類を実部と虚部について組み合わせた 25 通りの定数が用意されている。例えば、`MPC_RNDNU` は、実部を最近接丸め、虚部を正の無限大の方向へ丸めることを表す。

多くの場合、実部と虚部をともに最近接丸めとする `MPC_RNDNN` が使用される。最近接丸めでは、実部と虚部がそれぞれ表現可能な最も近い値へ丸められるため、一般に丸め誤差を小さく抑えることができ、誤差も特定の方向へ偏りにくい。従って、丸める方向を特に指定する必要がない場合には、`MPC_RNDNN` を使用するのが基本である。一方、実部と虚部の各成分について計算結果の上限や下限を求める場合など、丸める方向を明確に制御する必要がある場合には、`MPC_RNDUU`, `MPC_RNDDD` などを用途に応じて使用する。

22.4.3.4 実部, 虚部, 絶対値, 偏角

`mpc_t` 型の複素数の実部、虚部、絶対値、偏角などを扱う主な関数とマクロを表 101 に示す。実部、虚部、絶対値、偏角などを求める関数では、演算結果を `mpfr_t` 型のオブジェクトに格納する。また、共役複素数を求める関数では、演算結果を `mpc_t` 型のオブジェクトに格納する。実部と虚部を直接参照するためのマクロも用意されている。

表 101: MPC の複素数の成分や大きさを扱う主な関数とマクロ	
関数・マクロ	解 説
<code>mpc_real(rop, op, rnd)</code>	<code>op</code> の実部を <code>rop</code> に格納する
<code>mpc_imag(rop, op, rnd)</code>	<code>op</code> の虚部を <code>rop</code> に格納する
<code>mpc_abs(rop, op, rnd)</code>	<code>op</code> の絶対値を <code>rop</code> に格納する
<code>mpc_norm(rop, op, rnd)</code>	<code>op</code> の体ノルム、すなわち絶対値の 2 乗を <code>rop</code> に格納する
<code>mpc_arg(rop, op, rnd)</code>	<code>op</code> の偏角をラジアン単位で <code>rop</code> に格納する
<code>mpc_conj(rop, op, rnd)</code>	<code>op</code> の共役複素数を <code>rop</code> に格納する
<code>mpc_realref(op)</code>	<code>op</code> の実部を直接参照する
<code>mpc_imagref(op)</code>	<code>op</code> の虚部を直接参照する

注意) 丸め方法を引数に取る関数では、結果が実数となる場合は MPFR の丸め方法を、結果が複素数となる場合は MPC の丸め方法を指定する点に注意する必要がある。

表 101 のうち、`mpc_real`, `mpc_imag`, `mpc_abs`, `mpc_norm`, `mpc_arg` では、計算結果は実数となるため、第 1 引数には初期化済みの `mpfr_t` 型のオブジェクトを指定する。また、最後の引数には、`MPFR_RNDN` など、MPFR の丸め方法を指定する。

例えば、複素数を格納した `mpc_t` 型のオブジェクト `z` から実部と虚部を取り出す処理は、次のように記述する。

```

mpfr_t re, im;
mpfr_init2(re, 256);
mpfr_init2(im, 256);
mpc_real(re, z, MPFR_RNDN);
mpc_imag(im, z, MPFR_RNDN);

```

この処理では、 z の実部が `re` に、虚部が `im` に格納される。取り出した値は、それぞれ格納先である `re` と `im` の精度に従って丸められる。使用後は、これらのオブジェクトに対して `mpfr_clear` を呼び出す必要がある。

複素数 $z = x + iy$ に対して、`mpc_abs` は絶対値 $|z| = \sqrt{x^2 + y^2}$ を求める。一方、`mpc_norm` は絶対値の 2 乗 $|z|^2 = x^2 + y^2$ を求める。例えば、次の処理では、 z の絶対値とその 2 乗を、それぞれ `a` と `n` に格納する。

```

mpc_abs(a, z, MPFR_RNDN);
mpc_norm(n, z, MPFR_RNDN);

```

平方根を必要としない値で十分な場合は、`mpc_norm` を使用することができる。なお、この関数名に含まれる `norm` は、ここでは絶対値そのものではなく、絶対値の 2 乗を意味する。

`mpc_arg` は、複素数の偏角をラジアン単位で求める関数である。例えば、次の処理では、 z の偏角を `theta` に格納する。

```

mpc_arg(theta, z, MPFR_RNDN);

```

この関数が返す偏角は主値であり、 $-\pi$ から π までの範囲の値となる。ただし、複素数が 0 である場合や実軸上にある場合には、実部と虚部の符号付きの 0 の影響によって、偏角の符号や値が異なることがある。

`mpc_conj` は、共役複素数を求める関数である。複素数 $z = x + iy$ の共役複素数は $\bar{z} = x - iy$ である。`mpc_conj` の計算結果は複素数であるため、第 1 引数には `mpc_t` 型のオブジェクトを指定し、丸め方法には `MPC_RNDNN`などを指定する。例えば、次の処理では、 z の共役複素数を `w` に格納する。

```

mpc_conj(w, z, MPC_RNDNN);

```

この関数では、入力と出力に同じオブジェクトを指定することもできる。従って、次のように記述すると、 z 自身をその共役複素数に置き換えることができる。

```

mpc_conj(z, z, MPC_RNDNN);

```

`mpc_realref` と `mpc_imagref` は、`mpc_t` 型のオブジェクトが内部に保持している実部と虚部を、それぞれ `mpfr_t` 型の値として直接参照するためのマクロである。値のコピーや丸めは行われず、その戻り値を MPFR の関数に直接指定できる。例えば、次のように実部と虚部を個別に設定できる。

```

mpfr_set_ui(mpc_realref(z), 3, MPFR_RNDN);
mpfr_set_si(mpc_imagref(z), -2, MPFR_RNDN);

```

これらのマクロを通して値を変更すると、 z の実部または虚部そのものが変更される。また、参照した実部や虚部に対して、個別に初期化や解放を行ってはならない。

22.4.3.5 基本的な演算

`mpc_t` 型の複素数に対して、加減乗除などの基本的な演算を行う主な関数を表 102 に示す。これらの関数では、基本的に第 1 引数に演算結果の格納先を指定し、最後の引数に実部と虚部の丸め方法を指定する。

例えば、初期化済みの `mpc_t` 型のオブジェクト `z1`, `z2`, `z3` を用いて、四則演算を行う処理は次のように記述する。

```

mpc_add(z3, z1, z2, MPC_RNDNN);
mpc_sub(z3, z1, z2, MPC_RNDNN);
mpc_mul(z3, z1, z2, MPC_RNDNN);
mpc_div(z3, z1, z2, MPC_RNDNN);

```

これらの関数では、演算結果が `z3` の実部と虚部の精度に従って丸められる。従って、最終的に格納される値の精度は、結果の格納先である `z3` に設定された精度によって決まる。ただし、入力値そのものが意図した値をどの程度正確に表しているかは、`z1`, `z2` に設定された精度にも依存する。

MPC の多くの演算関数では、入力と出力に同じオブジェクトを指定することができる。例えば、次の処理では、`z1` に `z2` を加え、結果を再び `z1` に格納する。

```

mpc_add(z1, z1, z2, MPC_RNDNN);

```

表 102: MPC の基本的な演算を行う主な関数

関 数	解 説
<code>mpc_add(rop, op1, op2, rnd)</code>	$op1 + op2$ を <code>rop</code> に格納する
<code>mpc_sub(rop, op1, op2, rnd)</code>	$op1 - op2$ を <code>rop</code> に格納する
<code>mpc_neg(rop, op, rnd)</code>	$-op$ を <code>rop</code> に格納する
<code>mpc_mul(rop, op1, op2, rnd)</code>	$op1 * op2$ を <code>rop</code> に格納する
<code>mpc_sqr(rop, op, rnd)</code>	op の 2 乗を <code>rop</code> に格納する
<code>mpc_div(rop, op1, op2, rnd)</code>	$op1 / op2$ を <code>rop</code> に格納する
<code>mpc_fma(rop, op1, op2, op3, rnd)</code>	$op1 * op2 + op3$ を 1 回の丸めで求める
<code>mpc_add_ui(rop, op1, op2, rnd)</code>	<code>op1</code> と unsigned long int 型の <code>op2</code> の和を求める
<code>mpc_add_fr(rop, op1, op2, rnd)</code>	<code>op1</code> と mpfr_t 型の <code>op2</code> の和を求める
<code>mpc_sub_ui(rop, op1, op2, rnd)</code>	<code>op1</code> から unsigned long int 型の <code>op2</code> を減じる
<code>mpc_sub_fr(rop, op1, op2, rnd)</code>	<code>op1</code> から mpfr_t 型の <code>op2</code> を減じる
<code>mpc_ui_sub(rop, op1, op2, rnd)</code>	unsigned long int 型の <code>op1</code> から <code>op2</code> を減じる
<code>mpc_fr_sub(rop, op1, op2, rnd)</code>	mpfr_t 型の <code>op1</code> から <code>op2</code> を減じる
<code>mpc_mul_si(rop, op1, op2, rnd)</code>	<code>op1</code> と long int 型の <code>op2</code> の積を求める
<code>mpc_mul_fr(rop, op1, op2, rnd)</code>	<code>op1</code> と mpfr_t 型の <code>op2</code> の積を求める
<code>mpc_div_ui(rop, op1, op2, rnd)</code>	<code>op1</code> を unsigned long int 型の <code>op2</code> で除する
<code>mpc_div_fr(rop, op1, op2, rnd)</code>	<code>op1</code> を mpfr_t 型の <code>op2</code> で除する
<code>mpc_ui_div(rop, op1, op2, rnd)</code>	unsigned long int 型の <code>op1</code> を <code>op2</code> で除する
<code>mpc_fr_div(rop, op1, op2, rnd)</code>	mpfr_t 型の <code>op1</code> を <code>op2</code> で除する
<code>mpc_mul_i(rop, op, sgn, rnd)</code>	<code>op</code> に虚数単位 i または $-i$ を乗じる
<code>mpc_mul_2si(rop, op, n, rnd)</code>	<code>op</code> に 2^n を乗じる
<code>mpc_div_2si(rop, op, n, rnd)</code>	<code>op</code> を 2^n で除する

同様に、次の処理では、 z の符号を反転し、結果を z 自身に格納する。

```
mpc_neg(z, z, MPC_RNDNN);
```

このように、演算結果を同じオブジェクトに格納することによって、補助的なオブジェクトを用意せずに値を更新することができる。

複素数の 2 乗を求める場合は、`mpc_mul` で同じ値どうしを乗算することもできるが、専用の `mpc_sqr` を使用する方が効率が良い。

```
mpc_sqr(z2, z1, MPC_RNDNN);
```

この処理では、 $z1$ の 2 乗が $z2$ に格納される。`mpc_sqr` では、入力と出力に同じオブジェクトを指定することもできる。

```
mpc_sqr(z1, z1, MPC_RNDNN);
```

`mpc_fma` は、乗算と加算を組み合わせた積和を求める関数である。

```
mpc_fma(z, z1, z2, z3, MPC_RNDNN);
```

この関数は、 $z1 * z2 + z3$ 全体の厳密な値に対して、最終結果を 1 回だけ丸める。従って、`mpc_mul` の結果を一度丸めてから `mpc_add` を実行する場合とは、丸め誤差が異なることがある。

整数や MPFR の実数値との演算を行う場合は、型に対応した関数を使用することができる。例えば、次の処理では、 z に整数 3 を加え、次に MPFR の値 x を乗じる。

```
mpc_add_ui(z, z, 3, MPC_RNDNN);
mpc_mul_fr(z, z, x, MPC_RNDNN);
```

これらの関数では、整数または MPFR の値は実数として扱われ、その虚部は演算上の正確な 0 とみなされる。なお、減算や除算では被演算数の順序によって結果が異なる。実数値から複素数を減じる場合には `mpc_ui_sub` や `mpc_fr_sub`

を、実数値を複素数で除する場合には `mpc_ui_div` や `mpc_fr_div` を使用する。

`mpc_mul_i` は、複素数に虚数単位 i または $-i$ を乗じる関数である。第 3 引数 `sgn` が 0 以上の場合は i を、負の場合は $-i$ を乗じる。

```
mpc_mul_i(w, z, 1, MPC_RNDNN);
mpc_mul_i(w, z, -1, MPC_RNDNN);
```

複素数を $z = x + iy$ とすると、 $iz = -y + ix$ である。従って、`mpc_mul_i` は、実部と虚部を入れ替えて符号を調整する演算に相当する。

`mpc_mul_2si` および `mpc_div_2si` は、複素数に 2 の整数乗を乗じる、または 2 の整数乗で除する関数である。

```
mpc_mul_2si(w, z, 10, MPC_RNDNN);
mpc_div_2si(w, z, 10, MPC_RNDNN);
```

これらの例では、それぞれ z に 2^{10} を乗じた値と、 z を 2^{10} で除した値を w に格納する。2 の整数乗による乗除は、一般の乗除よりも効率良く実行することができる。

各演算関数の戻り値は、実部と虚部がそれぞれ正確な値に対してどの方向に丸められたかを表す。`MPC_INEX_RE` および `MPC_INEX_IM` を用いると、実部と虚部の情報を個別に取り出すことができる。取り出した値が 0 なら正確に表現されたことを表し、正なら丸めた結果が正確な値より大きく、負なら正確な値より小さいことを表す。

```
int inex = mpc_div(z3, z1, z2, MPC_RNDNN);
int inex_re = MPC_INEX_RE(inex);
int inex_im = MPC_INEX_IM(inex);
```

通常の利用では戻り値を無視してもよいが、丸めの有無や方向を確認する必要がある場合には、これらの値を確認するとい。

22.4.3.6 累乗、指数関数、対数関数

MPC では、`mpc_t` 型の複素数に対する平方根、累乗、指数関数、対数関数が用意されている。これらの演算を行う主な関数を表 103 に示す。各関数では、第 1 引数に演算結果の格納先を指定し、最後の引数に実部と虚部の丸め方法を指定する。

表 103: MPC の累乗、指数関数、対数関数

関 数	解 説
<code>mpc_sqrt(rop, op, rnd)</code>	op の平方根を rop に格納する
<code>mpc_pow(rop, op1, op2, rnd)</code>	$op1$ の $op2$ 乗を rop に格納する
<code>mpc_pow_si(rop, op1, op2, rnd)</code>	$op1$ の long int 型の $op2$ 乗を求める
<code>mpc_pow_ui(rop, op1, op2, rnd)</code>	$op1$ の unsigned long int 型の $op2$ 乗を求める
<code>mpc_pow_fr(rop, op1, op2, rnd)</code>	$op1$ の <code>mpfr_t</code> 型の $op2$ 乗を求める
<code>mpc_exp(rop, op, rnd)</code>	自然指数関数 e^{op} の値を rop に格納する
<code>mpc_exp2(rop, op, rnd)</code>	底を 2 とする指数関数 2^{op} の値を rop に格納する
<code>mpc_exp10(rop, op, rnd)</code>	底を 10 とする指数関数 10^{op} の値を rop に格納する
<code>mpc_log(rop, op, rnd)</code>	op の自然対数を rop に格納する
<code>mpc_log2(rop, op, rnd)</code>	op の底を 2 とする対数を rop に格納する
<code>mpc_log10(rop, op, rnd)</code>	op の常用対数を rop に格納する

`mpc_sqrt` は、複素数の平方根を求める関数である。複素数の平方根は一般に 2 つ存在するが、この関数は主値を返す。結果の実部は 0 以上となり、実部が 0 の場合は虚部も 0 以上となる。例えば、次の処理では、 z の平方根を w に格納する。

```
mpc_sqrt(w, z, MPC_RNDNN);
```

実数の平方根を扱う場合とは異なり、負の実数に対しても複素数として平方根を求めることができる。例えば、 z が $-4 + 0i$ である場合、`mpc_sqrt` によって得られる主値は $0 + 2i$ となる。

`mpc_pow` は、複素数を底および指数とする累乗を求める関数である。例えば、次の処理では、 $z1$ の $z2$ 乗を $z3$ に格納する。


```
mpc_pow(z3, z1, z2, MPC_RNDNN);
```

指数が整数値である場合は、`mpc_pow_si` または `mpc_pow_ui` を使用することができる。

```
mpc_pow_ui(w, z, 10, MPC_RNDNN);
mpc_pow_si(w, z, -3, MPC_RNDNN);
```

最初の処理では z^{10} を、2 番目の処理では z^{-3} を求める。負の整数指数を使用する場合は、符号付き整数を受け取る `mpc_pow_si` を使用する必要がある。また、MPFR の多倍長実数を指数として使用する場合は、`mpc_pow_fr` を使用する。

```
mpc_pow_fr(w, z, x, MPC_RNDNN);
```

この処理では、`mpfr_t` 型の値 x を指数として z^x を求める。`mpc_pow_si`、`mpc_pow_ui`、`mpc_pow_fr` では、指数は虚部を正の 0 とする実数として扱われる。

指数が整数でない場合、複素数の累乗は複素対数と関係し、多価関数となる。MPC では、定められた分岐に基づく主値が返される。従って、負の実軸付近の値や符号付きの 0 を含む値では、入力値が分岐線のどちら側にあるかによって結果が異なることがある。通常の利用では、ライブラリが返す主値をそのまま使用すればよい。

`mpc_exp` は、自然指数関数を求める関数である。複素数 $z = x + iy$ に対して、 $e^z = e^x(\cos y + i \sin y)$ の関係が成り立つ。例えば、次の処理では、 z に対する自然指数関数の値を w に格納する。

```
mpc_exp(w, z, MPC_RNDNN);
```

底を 2 または 10 とする指数関数を求める場合は、それぞれ `mpc_exp2`、`mpc_exp10` を使用する。

```
mpc_exp2(w, z, MPC_RNDNN);
mpc_exp10(w, z, MPC_RNDNN);
```

これらの関数では、入力値も演算結果も複素数として扱われる。

`mpc_log` は、複素数の自然対数を求める関数である。0 でない複素数 z に対して、その主値は $\log z = \log |z| + i \arg z$ と表される。例えば、次の処理では、 z の自然対数を w に格納する。

```
mpc_log(w, z, MPC_RNDNN);
```

MPC の複素対数では主値が返され、負の実軸が分岐線となる。このため、負の実軸上では虚部の符号付きの 0 によって、対数の虚部が正または負の値になることがある。例えば、 $-1 + 0i$ の自然対数の主値は $+\pi i$ となり、 $-1 - 0i$ の場合は $-\pi i$ となる。

底を 2 または 10 とする対数を求める場合は、それぞれ `mpc_log2`、`mpc_log10` を使用する。

```
mpc_log2(w, z, MPC_RNDNN);
mpc_log10(w, z, MPC_RNDNN);
```

これらも複素対数であり、実数の対数関数とは異なり、負の実数に対しても複素数の値を返す。ただし、0 に対する対数や、無限大、非数、符号付きの 0 を含む特殊な値については、通常の有限な複素数とは異なる結果となる点に注意する必要がある。

これらの関数では、演算結果は第 1 引数に指定したオブジェクトの実部と虚部の精度に従って丸められる。また、入力と出力に同じオブジェクトを指定することもできる。例えば、次のように記述すると、 z をその自然指数関数の値に置き換えることができる。

```
mpc_exp(z, z, MPC_RNDNN);
```

累乗、平方根、対数関数では、複素関数の主値や分岐線が結果に影響する。特に負の実軸付近の値を扱う場合は、実部と虚部の符号付きの 0 にも注意する必要がある。

参考) `mpc_exp2`、`mpc_exp10`、`mpc_log2` は MPC 1.4.0 以降で利用できる。

22.4.3.7 三角関数、双曲線関数

MPC では、`mpc_t` 型の複素数に対する三角関数、逆三角関数、双曲線関数、逆双曲線関数が用意されている。これらの演算を行う主な関数を表 104 に示す。多くの関数では、第 1 引数に演算結果の格納先を指定し、最後の引数に実部と虚部の丸め方法を指定する。

表 104: MPC の主な三角関数と双曲線関数

関 数	解 説
<code>mpc_sin(rop, op, rnd)</code>	op の正弦を rop に格納する
<code>mpc_cos(rop, op, rnd)</code>	op の余弦を rop に格納する
<code>mpc_tan(rop, op, rnd)</code>	op の正接を rop に格納する
<code>mpc_sin_cos(rop1, rop2, op, rnd1, rnd2)</code>	op の正弦と余弦を rop1 と rop2 に格納する
<code>mpc_asin(rop, op, rnd)</code>	op の逆正弦の主値を rop に格納する
<code>mpc_acos(rop, op, rnd)</code>	op の逆余弦の主値を rop に格納する
<code>mpc_atan(rop, op, rnd)</code>	op の逆正接の主値を rop に格納する
<code>mpc_sinh(rop, op, rnd)</code>	op の双曲線正弦を rop に格納する
<code>mpc_cosh(rop, op, rnd)</code>	op の双曲線余弦を rop に格納する
<code>mpc_tanh(rop, op, rnd)</code>	op の双曲線正接を rop に格納する
<code>mpc_asinh(rop, op, rnd)</code>	op の逆双曲線正弦の主値を rop に格納する
<code>mpc_acosh(rop, op, rnd)</code>	op の逆双曲線余弦の主値を rop に格納する
<code>mpc_atanh(rop, op, rnd)</code>	op の逆双曲線正接の主値を rop に格納する

`mpc_sin`, `mpc_cos`, `mpc_tan` は、それぞれ複素数の正弦、余弦、正接を求める関数である。引数が実数である場合、角度はラジアン単位で扱われる。例えば、次の処理では、 z の正弦、余弦、正接をそれぞれ w に格納する。

```
mpc_sin(w, z, MPC_RNDNN);
mpc_cos(w, z, MPC_RNDNN);
mpc_tan(w, z, MPC_RNDNN);
```

入力値と演算結果はいずれも複素数である。例えば、複素数 $z = x + iy$ に対して、正弦と余弦は $\sin z = \sin x \cosh y + i \cos x \sinh y$ および $\cos z = \cos x \cosh y - i \sin x \sinh y$ と表される。従って、実数に対する正弦や余弦とは異なり、複素数に対するこれらの関数の値は、実部と虚部がともに大きな値になることがある。

正弦と余弦の両方を同じ入力値に対して求める場合は、`mpc_sin_cos` を使用する。

```
mpc_sin_cos(s, c, z, MPC_RNDNN, MPC_RNDNN);
```

この処理では、 z の正弦が s に、余弦が c に格納される。 s と c には、互いに異なる初期化済みの `mpc_t` 型のオブジェクトを指定する。第 4 引数には正弦の丸め方法を、第 5 引数には余弦の丸め方法を指定する。

`mpc_sin_cos` の戻り値には、正弦と余弦の丸めに関する情報がまとめて格納される。`MPC_INEX1` と `MPC_INEX2` を使用すると、それぞれ第 1 の結果である正弦と、第 2 の結果である余弦の情報を取り出すことができる。

```
int inex = mpc_sin_cos(s, c, z, MPC_RNDNN, MPC_RNDNN);
int inex_s = MPC_INEX1(inex);
int inex_c = MPC_INEX2(inex);
```

取り出した値には、それぞれ複素数の実部と虚部に関する丸め情報が含まれている。`MPC_INEX1` と `MPC_INEX2` のマクロを使用すると、それぞれ第 1 の結果である正弦と、第 2 の結果である余弦の情報を取り出すことができる。例えば、正弦の実部に関する情報は、次のように取得する。

```
int inex_s_re = MPC_INEX_RE(inex_s);
```

取り出した値が 0 なら結果が正確に表現されたことを表し、正なら丸めた結果が正確な値より大きく、負なら正確な値より小さいことを表す。通常の利用では戻り値を無視してもよいが、各結果の丸めの有無や方向を確認する場合に利用できる。

`mpc_asin`, `mpc_acos`, `mpc_atan` は、それぞれ逆正弦、逆余弦、逆正接を求める関数である。例えば、次の処理では、 z の逆正弦の主値を w に格納する。

```
mpc_asin(w, z, MPC_RNDNN);
```

複素数に対する逆三角関数は一般に多価関数であるが、MPC の関数は定められた分岐に基づく主値を返す。従って、分岐線上やその付近の値では、入力値の実部または虚部が持つ符号付きの 0 などによって結果が異なることがある。

`mpc_sinh`, `mpc_cosh`, `mpc_tanh` は、それぞれ双曲線正弦、双曲線余弦、双曲線正接を求める関数である。例えば、次の処理では、 z の双曲線正弦を w に格納する。

```
mpc_sinh(w, z, MPC_RNDNN);
```

複素数 z に対する双曲線正弦と双曲線余弦は、指数関数を用いて

$$\sinh z = \frac{e^z - e^{-z}}{2}, \quad \cosh z = \frac{e^z + e^{-z}}{2}$$

と表される。また、双曲線正接は

$$\tanh z = \frac{\sinh z}{\cosh z}$$

と表される。ただし、実際の計算では、これらの式をプログラムとして直接記述するのではなく、MPC が提供する専用の関数を使用するとよい。

`mpc_asinh`, `mpc_acosh`, `mpc_atanh` は、それぞれ逆双曲線正弦、逆双曲線余弦、逆双曲線正接を求める関数である。例えば、次の処理では、 z の逆双曲線余弦の主値を w に格納する。

```
mpc_acosh(w, z, MPC_RNDNN);
```

逆双曲線関数も一般に多価関数であり、MPC では定められた分岐に基づく主値が返される。分岐線付近の値や、無限大、非数、符号付きの 0 を含む特殊な値については、通常の有限な複素数とは異なる結果となる場合がある。

これらの関数では、演算結果は結果の格納先として指定したオブジェクトの実部と虚部の精度に従って丸められる。また、ここで挙げた単一の結果を返す関数では、入力と出力に同じオブジェクトを指定することもできる。例えば、次のように記述すると、 z をその余弦の値に置き換えることができる。

```
mpc_cos(z, z, MPC_RNDNN);
```

逆三角関数および逆双曲線関数を使用する場合は、複素関数の主値と分岐線が結果に影響する点に注意する必要がある。

22.4.3.8 書式整形と入出力

MPC では、`mpc_t` 型の値と文字列との間で変換を行う関数や、ストリームを介して値を入出力する関数が用意されている。これらの処理に使用する主な関数を表 105 に示す。

表 105: MPC の文字列変換と入出力に関する主な関数

関 数	解 説
<code>mpc_set_str(rop, str, base, rnd)</code>	文字列 <code>str</code> から値を読み取り、 <code>rop</code> に格納する
<code>mpc_strtoc(rop, str, endptr, base, rnd)</code>	文字列の先頭から値を読み取り、読取り後の位置を <code>endptr</code> を介して返す
<code>mpc_get_str(base, n, op, rnd)</code>	<code>op</code> を文字列に変換し、そのアドレスを返す
<code>mpc_free_str(str)</code>	<code>mpc_get_str</code> が確保した文字列を解放する
<code>mpc_out_str(stream, base, n, op, rnd)</code>	<code>op</code> を文字列に変換してストリームに出力する
<code>mpc_inp_str(rop, stream, read, base, rnd)</code>	ストリームから値を読み取り、 <code>rop</code> に格納する

MPC では、複素数を文字列として表す場合、基本的に実部と虚部を空白で区切り、全体を丸括弧で囲んだ形式を使用する。例えば、複素数 $1.25 - 2.5i$ は、次の文字列で表すことができる。

```
"(1.25 -2.5)"
```

虚数単位を表す文字 `i` は記述しない点に注意する必要がある。実数だけを記述した文字列を与えた場合は、その値が実部となり、虚部は正の 0 となる。

`mpc_set_str` は、文字列全体を 1 個の複素数として読み取り、初期化済みの `mpc_t` 型のオブジェクトに格納する関数である。例えば、次の処理では、10 進表記の文字列から複素数 $1.25 - 2.5i$ を読み取り、 z に格納する。

```
int ret;
ret = mpc_set_str(z, "(1.25 -2.5)", 10, MPC_RNDNN);
```

第 3 引数には、文字列で使用されている基数を指定する。通常の 10 進表記を読み取る場合は 10 を指定する。読み取った実部と虚部は、 z のそれぞれの精度に従って丸められる。文字列全体が正しい形式でない場合、この関数は -1 を

返し、`z` の実部と虚部には非数 NaN が設定される。従って、外部から与えられた文字列を読み取る場合は、戻り値を確認するとよい。

```
if (ret == -1) {
    /* 文字列の形式が正しくない場合の処理 */
}
```

`mpc_strtoc` も文字列から複素数を読み取る関数であるが、文字列の先頭にある複素数だけを読み取り、変換されなかった最初の文字を指すポインタを、第 3 引数を介して返す。このため、1 個の複素数の後に別の文字列が続いている場合などに利用できる。文字列全体が 1 個の複素数を表していることを前提とする場合は、`mpc_set_str` の方が扱いやすい。

`mpc_get_str` は、`mpc_t` 型の値を文字列に変換する関数である。例えば、次の処理では、`z` を 10 進表記の文字列に変換する。

```
char *str;
str = mpc_get_str(10, 20, z, MPC_RNDNN);
printf("%s\n", str);
mpc_free_str(str);
```

第 1 引数には基数を指定し、第 2 引数には実部と虚部に出力する有効数字の桁数を指定する。第 2 引数には 0 または 2 以上の値を指定する。0 を指定すると、文字列への変換と再読み込みの両方で最近接丸めを使用し、元の値と同じ精度で再読み込みを行った場合に、元の値を復元できるだけの桁数が自動的に選ばれる。上の例で戻される文字列は、例えば次のように、実部と虚部を空白で区切って丸括弧で囲んだ形式となる。

```
(1.25000000000000000000 -2.50000000000000000000)
```

注) 実際の表記は、`op` に格納されている値、その精度、指定した基数、有効数字の桁数、丸め方法などによって異なる場合がある。

`mpc_get_str` は、結果の文字列を格納するメモリ領域を内部で確保する。このため、使用後には通常の `free` ではなく、必ず `mpc_free_str` を呼び出して解放する必要がある。

`mpc_out_str` は、`mpc_t` 型の値を指定したストリームに出力する関数である。例えば、次の処理では、`z` を 10 進表記で標準出力に出力する。

```
mpc_out_str(stdout, 10, 0, z, MPC_RNDNN);
printf("\n");
```

第 2 引数には基数を、第 3 引数には出力する有効数字の桁数を指定する。桁数に 0 を指定した場合の扱いは、`mpc_get_str` の場合と同様である。この関数は改行を出力しないため、必要な場合は例のように別途改行を出力する。第 1 引数に `NULL` を指定すると、標準出力に出力される。また、戻り値として出力した文字数が返される。

ファイルに出力する場合は、通常のファイルストリームを指定する。

```
FILE *fp;
fp = fopen("complex.txt", "w");
if (fp != NULL) {
    mpc_out_str(fp, 10, 0, z, MPC_RNDNN);
    fprintf(fp, "\n");
    fclose(fp);
}
```

文字列の前後に説明などを付けて出力する場合は、`printf` や `fprintf` と `mpc_out_str` を組み合わせるとよい。

`mpc_inp_str` は、ストリームから複素数を読み取り、初期化済みの `mpc_t` 型のオブジェクトに格納する関数である。例えば、次の処理では、標準入力から 10 進表記の複素数を読み取る。

```
size_t nread;
int ret;
ret = mpc_inp_str(z, stdin, &nread, 10, MPC_RNDNN);
```

第 3 引数には、読み取った文字数を格納する `size_t` 型の変数のアドレスを指定する。読み取った文字数を必要としない場合は `NULL` を指定することもできる。また、第 2 引数に `NULL` を指定した場合も、標準入力から読み取られる。読み取りに失敗した場合は -1 が返され、格納先の実部と虚部には非数 NaN が設定される。

ファイルから読み取る場合は、次のように入力ファイルストリームを指定する。

```
FILE *fp;
fp = fopen("complex.txt", "r");
if (fp != NULL) {
    ret = mpc_inp_str(z, fp, NULL, 10, MPC_RNDNN);
    fclose(fp);
}
```

mpc_inp_str および mpc_out_str を使用する場合は、mpc.h より前に stdio.h を読み込む必要がある。

```
#include <stdio.h>
#include <mpc.h>
```

MPC の文字列入出力では、書式指定子によって表示幅や小数点以下の桁数などを細かく制御するのではなく、基数と有効数字の桁数を指定して、実部と虚部をまとめて入出力する。より細かな書式で表示する必要がある場合は、mpc_real および mpc_imag で実部と虚部を mpfr_t 型のオブジェクトに取り出し、MPFR の書式整形出力関数を使用するとよい。

22.4.3.9 サンプルプログラム：1 の n 乗根

複素数の方程式 $z^n = 1$ は、 n 個の解を持つ。これらの解は、複素平面上の単位円を n 等分する点であり、次のように表される。

$$z_k = e^{2\pi i k/n} = \cos \frac{2\pi k}{n} + i \sin \frac{2\pi k}{n}$$

ここで、 $k = 0, 1, \dots, n-1$ である。次に示すプログラム test272-1.c は、コマンドライン引数として正の整数 n と演算精度を受け取り、角度 $0, \frac{2\pi}{n}, \frac{4\pi}{n}, \dots, 2\pi$ に対応する単位円上の値を順に求めて出力するものである。最後の 2π に対応する値まで出力するため、表示される値は全部で $n+1$ 個となる。

プログラム：test272-1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <errno.h>
4  #include <limits.h>
5  #include <mpc.h>
6
7  int read_positive_ulong( const char *str, unsigned long *value ) {
8      char *end;
9      unsigned long v;
10
11      if (str == NULL || value == NULL || *str == '\0' || *str < '0' || *str > '9') {
12          return 0;
13      }
14
15      errno = 0;
16      end = NULL;
17      v = strtoul(str, &end, 10);
18
19      if (errno == ERANGE || end == str || *end != '\0' || v == 0) return 0;
20
21      *value = v;
22      return 1;
23  }
24
25  int main( int argc, char *argv[] ) {
26      unsigned long n, p, k;
27      mpfr_prec_t prec;
28
29      mpfr_t pi, theta, zero;
30      mpc_t z;
31
32      if (argc != 3) {
33          fprintf(stderr, "usage: %s n Precision\n", argv[0]);
34          return 1;
35      }
36
37      if (!read_positive_ulong(argv[1], &n)) {
38          fprintf(stderr, "Give positive integer n !!\n");
39          return 1;
40      }
41
```



```

42     if (!read_positive_ulong(argv[2], &p) ||
43         p < (unsigned long)MPFR_PREC_MIN ||
44         p > (unsigned long)MPFR_PREC_MAX) {
45         fprintf(stderr, "Give valid precision!\n");
46         return 1;
47     }
48
49     prec = (mpfr_prec_t)p;
50
51     /* MPFRとMPCのオブジェクトを指定された精度で初期化する */
52     mpfr_init2(pi, prec);
53     mpfr_init2(theta, prec);
54     mpfr_init2(zero, prec);
55     mpc_init2(z, prec);
56
57     mpfr_set_zero(zero, 1);
58     mpfr_const_pi(pi, MPFR_RNDN);
59
60     /* theta = 2*pi*k/n として, z = exp(i*theta) を求める */
61     for (k = 0; k <= n; k++) {
62         mpfr_mul_ui(theta, pi, k, MPFR_RNDN);
63         mpfr_mul_2ui(theta, theta, 1, MPFR_RNDN);
64         mpfr_div_ui(theta, theta, n, MPFR_RNDN);
65         /* z = 0 + i*theta */
66         mpc_set_fr_fr(z, zero, theta, MPC_RNDNN);
67         /* z = exp(z) = exp(i*theta) */
68         mpc_exp(z, z, MPC_RNDNN);
69         /* 出力 */
70         printf("k = %lu : ", k);
71         mpc_out_str(stdout, 10, 0, z, MPC_RNDNN);
72         printf("\n");
73     }
74
75     mpc_clear(z);
76     mpfr_clear(pi);
77     mpfr_clear(theta);
78     mpfr_clear(zero);
79
80     mpfr_free_cache();
81 }

```

このプログラムでは、最初のコマンドライン引数を単位円の分割数 n とし、2 番目の引数を MPFR および MPC で使用するビット精度として読み取る。なお、 n に `unsigned long int` 型の最大値を表す `ULONG_MAX` と同じ値を指定すると反復変数がオーバーフローするため、そのような極端に大きな値を与えてはならない。次に、指定された精度で `mpfr_t` 型と `mpc_t` 型のオブジェクトを初期化し、MPFR の関数を用いて円周率 π を求める。

反復処理では、 $k = 0, 1, \dots, n$ に対して角度 $\theta = 2\pi k/n$ を計算し、実部を 0、虚部を θ とする複素数 $i\theta$ を作る。これに `mpc_exp` を適用することで、 $e^{i\theta} = \cos \theta + i \sin \theta$ を求め、単位円上の点として出力する。最後に、使用した MPFR および MPC のオブジェクトを解放する。

このプログラムをコンパイル、リンクして実行する例を次に示す。

例. test272-1.c のコンパイル、リンクと実行 ↓ 必要なライブラリ (MPC, MPFR, GMP) のリンク

```

$ gcc test272-1.c -lmpc -lmpfr -lgmp -o test272-1 Enter
$ ./test272-1 3 80 Enter      ←  $n$  の値 3 と精度 (80 ビット) を与えて実行
k = 0 : (1.00000000000000000000000000000000e0 0)      ←出力
k = 1 : (-4.99999999999999999999999999999999835e-1 8.6602540378443864676372392e-1)
k = 2 : (-5.00000000000000000000000000000000331e-1 -8.6602540378443864676372144e-1)
k = 3 : (1.00000000000000000000000000000000000e0 -9.1647717658286014548183733e-24)

```

この実行例では、 $k = 1$ および $k = 2$ に対応する値も、数学的に正確な値からごくわずかにずれている。例えば、両者の実部は本来 $-1/2$ であるが、有限のビット精度で円周率、角度、複素指数関数を計算しているため、末尾の桁に丸め誤差が現れている。また、角度 0 と 2π に対応する値は数学的にはいずれも $1 + 0i$ となるが、最後の値では虚部にごく小さな値が残っている。このように、多倍長演算でも精度は有限であるため丸め誤差は生じるが、演算精度を高くすることで、その誤差をさらに小さくできる。

第4部

付録

A 定義済みマクロ

A.1 言語モードと規格に関するマクロ

C/C++のソースファイルは、コンパイラに設定された言語モードと規格に基づいてコンパイルされる。同じコンパイラでも、ソースファイルの種類やコンパイラオプションによって、C 言語としてコンパイルされる場合と、C++としてコンパイルされる場合がある。また、選択された言語規格の版によって、使用できる言語機能が異なることもある。

複数の言語モードや規格に対応するプログラムを作成する場合は、現在のコンパイル条件を判定し、それに適したプログラム部分だけをコンパイルする必要が生じることがある。このような場合には、コンパイラによってあらかじめ定義されているマクロと条件コンパイルを組み合わせる。これらのマクロはヘッダファイルに定義されているものではなく、ソースファイルから直接参照することができるものである。

言語モードと規格に関する主な定義済みマクロを表 106 に示す。

表 106: 言語モードと規格に関する主な定義済みマクロ

マクロ	主な対象	意味
<code>__cplusplus</code>	C++	C++としてコンパイルされていることと、C++規格の版を示す
<code>__STDC__</code>	C	ISO C に適合する処理系であることを示す
<code>__STDC_VERSION__</code>	C	C 規格の版を示す

注) 以上は定義済みマクロの一部であり、処理環境ごとに様々なものがある。

これらのマクロは、単純に排他的な関係にあるわけではない。`__cplusplus` は、現在のソースファイルが C++としてコンパイルされる場合に定義される。一方、`__STDC__` と `__STDC_VERSION__` が C++において定義されるかどうかは処理系に依存するため、これらを用いて C 言語と C++を直接判別することはできない。従って、C 言語と C++を区別する場合は、最初に `__cplusplus` が定義されているかどうかを調べる。

厳密には、`__cplusplus` が定義されていないことは、C++としてコンパイルされていないことを意味するだけであり、必ずしも C 言語であることを意味しない。例えば、Objective-C や、プリプロセッサを使用するアセンブリ言語などの可能性もある。

本書では、ソースファイルを C 言語または C++のどちらかとしてコンパイルすることを前提とするため、`__cplusplus` が定義されていない場合は、C 言語としてコンパイルされているものとして扱う。

`__STDC_VERSION__` と `__cplusplus` の主な値を表 107 に示す。

表 107: C/C++規格の版を表す定義済みマクロの値

<code>__STDC_VERSION__</code>		<code>__cplusplus</code>	
C 規格	値	C++規格	値
C99	199901L	C++11	201103L
C11	201112L	C++14	201402L
C17	201710L	C++17	201703L
C23	202311L	C++20	202002L
		C++23	202302L

C90 では `__STDC_VERSION__` は定義されない。

A.2 コンパイラとターゲット環境に関するマクロ

コンパイラは、言語モードや規格に関するものだけでなく、コンパイラの種類や、コンパイル対象となる計算機環境を表すマクロもあらかじめ定義している。具体的には、コンパイラの種類、CPU アーキテクチャ、オペレーティングシステム (OS)、データモデルなどを表す定義済みマクロがある。これらを調べることで、使用しているコンパイラや、生成されるプログラムの実行対象となる環境を、コンパイル時に判定することができる。

ここで、コンパイルによって生成されるプログラムの実行対象となる環境を**ターゲット環境**という。通常は、コンパイラが動作している計算機環境とターゲット環境は同じである。一方、コンパイラが動作している環境とは異なる

CPU や OS に向けてプログラムを生成することもできる。このようなコンパイルを**クロスコンパイル**という。

クロスコンパイルを行う場合、CPU や OS に関する定義済みマクロは、コンパイラが動作している計算機環境ではなく、ターゲット環境を表す。例えば、x86-64 アーキテクチャの計算機上で、GCC や Clang を用いて ARM64 アーキテクチャ向けのプログラムをコンパイルする場合、ターゲットの CPU アーキテクチャを表すマクロとして `__aarch64__` が定義される。MSVC 互換コンパイラでは、同様の目的で `_M_ARM64` が用いられる。従って、これらのマクロを用いることで、ターゲット環境に応じてプログラムの一部を切り替えることができる。

コンパイラとターゲット環境に関する主な定義済みマクロを表 108 に示す。

表 108: コンパイラとターゲット環境に関する主な定義済みマクロ		
分類	マクロ	意味
コンパイラ	<code>__clang__</code>	Clang
	<code>__GNUC__</code>	GCC 互換コンパイラ
	<code>_MSC_VER</code>	MSVC 互換コンパイラ
CPU アーキテクチャ	<code>__x86_64__</code>	x86-64 (GCC, Clang など)
	<code>_M_X64</code>	x86-64 (MSVC 互換コンパイラ)
	<code>__i386__</code>	x86 32 ビット (GCC, Clang など)
	<code>_M_IX86</code>	x86 32 ビット (MSVC 互換コンパイラ)
	<code>__aarch64__</code>	AArch64 (GCC, Clang など)
	<code>_M_ARM64</code>	ARM64 (MSVC 互換コンパイラ)
	<code>__arm__</code>	ARM 32 ビット (GCC, Clang など)
	<code>_M_ARM</code>	ARM 32 ビット (MSVC 互換コンパイラ)
オペレーティングシステム	<code>__linux__</code>	Linux
	<code>__APPLE__</code>	Apple 系 OS (macOS, iOS など)
	<code>_WIN32</code>	Windows (32 ビット, 64 ビット共通)
	<code>_WIN64</code>	64 ビット Windows
データモデル	<code>__LP64__</code>	LP64 (64 ビット Linux, macOS など)
	<code>_WIN64</code>	LLP64 (64 ビット Windows)

注) 以上は定義済みマクロの一部であり、処理環境ごとに様々なものがある。

LP64 は、`int` 型が 32 ビット、`long` 型とポインタが 64 ビットとなるデータモデルである。一方、64 ビット Windows で採用されている LLP64 では、`int` 型と `long` 型が 32 ビット、`long long` 型とポインタが 64 ビットとなる。このため、`_WIN64` は 64 ビット Windows を表すとともに、LLP64 データモデルであることも示す。

表に挙げたマクロの多くは C 言語や C++ の規格で定められたものではなく、コンパイラや処理系が独自に定めたものである。このため、同じ種類のターゲット環境を表す場合でも、コンパイラによって異なる名前のマクロが使用されることがある。

また、これらのマクロは必ずしも排他的に定義されるものではない。例えば、Clang では通常、`__clang__` とともに、GCC との互換性を示す `__GNUC__` も定義される。同様に、64 ビット Windows では `_WIN64` と `_WIN32` の両方が定義される。従って、複数のマクロが同時に定義される可能性を考慮して判定する必要がある。

A.3 ホスト実装とフリースタンディング実装

C 言語と C++ の規格では、処理系を、プログラムの実行時に前提とする環境によって**ホスト実装** (hosted implementation) と**フリースタンディング実装** (freestanding implementation) に分類している。

ホスト実装は、オペレーティングシステム (OS) 上で一般的なアプリケーションを実行する環境を想定したものであり、規格がホスト実装に要求する標準ライブラリの機能が提供される。PC 上で使用する一般的な C/C++ 処理系は、通常、ホスト実装である。

一方、フリースタンディング実装は、OS や一般的な実行環境が存在することを前提としない。この場合、プログラムの開始方法や終了方法などは処理系によって定められ、提供が要求される標準ライブラリの機能もホスト実装より限定される。組込みシステム、OS カーネル、ブートローダなどを開発する場合に、フリースタンディング実装が用いられることがある。

この分類は、クロスコンパイルとは別のものである。クロスコンパイルは、コンパイラが動作している計算機環境とターゲット環境が異なることを表す。これに対して、ホスト実装とフリースタANDING実装の区別は、生成されるプログラムがどのような実行環境を前提とするかを表す。従って、クロスコンパイルによって、別の OS 上で動作するホスト実装向けのプログラムを生成することもできる。

処理系がホスト実装とフリースタANDING実装のどちらであるかは、定義済みマクロ `__STDC_HOSTED__` によって判定できる。このマクロは、名前に `__STDC` を含んでいるが、C 言語だけでなく C++ の規格にも定められている。

C 言語または C++ の規格に適合する処理系では、`__STDC_HOSTED__` が定義され、ホスト実装では値 1、フリースタANDING実装では値 0 となる。従って、このマクロについては、定義されているかどうかではなく、その値を調べる必要がある。

参考) ここに挙げた定義済みマクロには、数値が定義されて比較に用いるものと、定義されているかどうかのみを判定するものがある。例えば、`__STDC_VERSION__`、`__STDC_HOSTED__`、`__GNUC__` などは、その値を判定や比較に用いる。一方、`__linux__` や `_WIN32` などは、`#ifdef` によって定義されているかどうかを調べるのが一般的である。

■ 定義済みマクロを用いたプログラムの例

定義済みマクロを応用して、言語モードと、GCC または Clang のどちらでコンパイルされているかを調べるプログラムの例 `test100.txt` を下に示す。このプログラムは、C 言語、C++ 言語の両方で解釈できる形式のため、ファイルの拡張子を `.txt` としている。このプログラムは、自身をコンパイルしたコンパイラの情報を出力するものである。

プログラム：test100.txt

```
1  #include    <stdio.h>
2
3  int main() {
4  #ifdef __cplusplus
5      printf("Language : C++\n");
6      printf("__cplusplus = %ld\n", (long)__cplusplus);
7  #else
8      printf("Language : C\n");
9  #endif
10
11 #ifdef __STDC__
12     printf("__STDC__ = %d\n", __STDC__);
13 #else
14     printf("__STDC__ : not defined\n");
15 #endif
16
17 #ifdef __STDC_VERSION__
18     printf("__STDC_VERSION__ = %ld\n", (long)__STDC_VERSION__);
19 #else
20     printf("__STDC_VERSION__ : not defined\n");
21 #endif
22
23 #if defined(__clang__)
24     printf("Compiler : Clang\n");
25     printf("Clang version : %d.%d.%d\n",
26           __clang_major__, __clang_minor__, __clang_patchlevel__);
27 #elif defined(__GNUC__)
28     printf("Compiler : GCC\n");
29     printf("GCC version : %d.%d.%d\n",
30           __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
31 #else
32     printf("Compiler : unknown\n");
33 #endif
34 }
```

このプログラムは、コンパイラによってあらかじめ定義されているマクロを調べることで、ソースプログラムが C 言語と C++ のどちらとして処理されているか、また、どのコンパイラによって処理されているかを表示するものである。

まず、マクロ `__cplusplus` が定義されているかどうかを調べている。このマクロは C++ としてコンパイルする場合に定義されるので、これが定義されていれば C++、定義されていなければこの例では C 言語として処理されてい

ると判断できる。また、`__cplusplus` の値から、指定されている C++ 規格のバージョンを知ることができる。例えば C++17 では、この値は通常 201703L となる。

次に、マクロ `__STDC__` と `__STDC_VERSION__` が定義されているかどうかを調べている。`__STDC__` は、処理系が標準 C への準拠を示すために用いるマクロであり、定義されている場合はその値も表示する。また、`__STDC_VERSION__` は、対応する C 言語規格の版を示すためのマクロであり、例えば C17 では 201710L という値になる。これらのマクロが定義されていない場合には、その旨を表示する。

さらに、使用しているコンパイラの種類を判定している。Clang ではマクロ `__clang__` が定義されるので、これを調べることで Clang であることを判定できる。また、`__clang_major__`、`__clang_minor__`、`__clang_patchlevel__` を用いて、Clang のバージョン番号も表示している。

GCC ではマクロ `__GNUC__` が定義される。また、`__GNUC__`、`__GNUC_MINOR__`、`__GNUC_PATCHLEVEL__` を用いると、GCC のバージョン番号を知ることができる。ただし、Clang は GCC との互換性を保つために `__GNUC__` などのマクロも定義する。このため、このプログラムでは先に `__clang__` を調べ、Clang でなかった場合に `__GNUC__` を調べるようにしている。

このように、`__cplusplus`、`__STDC__`、`__STDC_VERSION__` などから言語や規格に関する情報を、`__clang__` や `__GNUC__` などからコンパイラに関する情報を得ることができる。同じソースプログラムを異なる言語モードやコンパイラで処理することで、これらの定義済みマクロの違いを実際に確認することができる。

このプログラムをコンパイルして実行する例を次に示す。

例. GCC の C 言語としてコンパイルして実行する (Windows 上の MSYS2 UCRT64 環境)

```
$ gcc -x c test100.txt; ./a Enter ←コンパイルと実行 (-x c は C 言語モードの強制)
Language : C                ← C 言語である
__STDC__ = 1                ←標準 C への準拠を示す
__STDC_VERSION__ = 202311   ← C 言語の規格 (C23)
Compiler : GCC              ←コンパイラの種類
GCC version : 16.2.0        ←コンパイラのバージョン
```

この例では、コンパイルと実行を 1 行のコマンドラインで行っている。コマンドラインのセミコロン「;」は、Bourne shell 系のシェル環境で 1 行に複数のコマンドを記述するための区切りであり、記述した順序でコマンドが実行される。また、コンパイラに `-o` オプションを指定していない。この場合、GCC が生成する実行形式ファイルの名前は、Unix 系環境では `a.out`、Windows 環境では `a.exe` となる。

例. GCC の C++ 言語としてコンパイルして実行する (Windows 上の MSYS2 UCRT64 環境)

```
$ g++ -x c++ test100.txt; ./a Enter ←コンパイルと実行 (-x c++ は C++ 言語モードの強制)
Language : C++              ← C++ 言語である
__cplusplus = 202002        ← C++ の規格 (C++20)
__STDC__ = 1                ←一応、C++ でも定義されている
__STDC_VERSION__ : not defined ← C++ では定義されていない
Compiler : GCC              ←コンパイラの種類
GCC version : 16.2.0        ←コンパイラのバージョン
```

例. Clang の C 言語としてコンパイルして実行する (Windows 上の MSYS2 CLANG64 環境)

```
$ clang -x c test100.txt; ./a Enter ←コンパイルと実行 (-x c は C 言語モードの強制)
Language : C                ← C 言語である
__STDC__ = 1                ←標準 C への準拠を示す
__STDC_VERSION__ = 201710   ← C 言語の規格 (C17)
Compiler : Clang            ←コンパイラの種類
Clang version : 22.1.8      ←コンパイラのバージョン
```

例. Clang の C++ 言語としてコンパイルして実行する (Windows 上の MSYS2 CLANG64 環境)

```
$ clang++ -x c++ test100.txt; ./a Enter ←コンパイルと実行 (-x c++ は C++ 言語モードの強制)
Language : C++              ← C++ 言語である
__cplusplus = 201703        ← C++ の規格 (C++17)
__STDC__ = 1                ←一応、C++ でも定義されている
__STDC_VERSION__ : not defined ← C++ では定義されていない
Compiler : Clang            ←コンパイラの種類
Clang version : 22.1.8      ←コンパイラのバージョン
```

これらの例では `-std` オプションを指定していないため、表示される言語規格は、各コンパイラのデフォルトの言語モードによって異なっている。GCC や Clang のデフォルトでは、ISO 規格を基礎として **GNU 拡張**を加えた言語モードが選択される。ただし、`__STDC_VERSION__` や `__cplusplus` の値から、GNU 拡張の有無を判別することはできない。

参考) 上の例では、セミコロン「`;`」を用いて、コンパイルと実行を 1 行のコマンドラインで行っている。

セミコロンの代わりに「`&&`」でコマンドを連ねることもでき、その場合は、左側のコマンドが正常に終了した場合に限って、右側のコマンドが実行される。コンパイルに失敗したときに、すでに存在する古い実行形式ファイルを誤って実行することを防げるため、実際の作業では「`&&`」を用いる方が安全である。

B サンプルプログラム

B.1 整数を 2 進数表記の文字列に変換する機能

本書が対象とする C17 の規格では、printf 関数などを用いて、整数を 2 進数表記で出力するための変換指定子は定められていない。使用する処理系やライブラリによっては、2 進数表記のための変換指定子 "%b" が使用できる場合もあるが、C17 以前の規格に準拠するプログラムでは、これが標準の機能ではないことに留意する必要がある。なお、C23 では変換指定子 "%b" が標準化されている。

C17 以前の標準的な処理系でも利用できるように、整数を 2 進数表記の文字列に変換する関数を、次のようなライブラリとして用意しておくことと便利である。

ヘッダファイル：dec2bin.h

```
1  #ifndef DEC2BIN_H
2  #define DEC2BIN_H
3
4  #include <limits.h>
5
6  #define UC2BIN_BUFSIZE  (CHAR_BIT*sizeof(unsigned char)+1)
7  #define U2BIN_BUFSIZE   (CHAR_BIT*sizeof(unsigned int)+1)
8  #define ULL2BIN_BUFSIZE (CHAR_BIT*sizeof(unsigned long long)+1)
9
10 #ifdef __cplusplus
11 extern "C" {
12 #endif
13
14 char *uc2bin( char *buf, unsigned char x );
15 char *u2bin( char *buf, unsigned int x );
16 char *ull2bin( char *buf, unsigned long long x );
17
18 #ifdef __cplusplus
19 }
20 #endif
21
22 #endif
```

注) C 言語と C++ のどちらからも使用できる形で記述されている。

プログラム：dec2bin.c

```
1  #include <limits.h>
2  #include <stddef.h>
3  #include "dec2bin.h"
4
5  // Convert an unsigned integer to a binary string
6  static char *to_binary( char *buf, unsigned long long x, size_t width ) {
7      size_t i = width;
8
9      buf[i] = '\0';
10
11      while ( i > 0 ) {
12          i--;
13          buf[i] = (x & 1ULL) ? '1' : '0';
14          x >>= 1;
15      }
16
17      return buf;
18  }
19
20 // unsigned char to binary string
21 char *uc2bin( char *buf, unsigned char x ) {
22     return to_binary(buf,x,CHAR_BIT*sizeof(x));
23 }
24
25 // unsigned int to binary string
26 char *u2bin( char *buf, unsigned int x ) {
27     return to_binary(buf,x,CHAR_BIT*sizeof(x));
28 }
29
```

```
30 // unsigned long long to binary string
31 char *ull2bin( char *buf, unsigned long long x ) {
32     return to_binary(buf,x,CHAR_BIT*sizeof(x));
33 }
```

このライブラリは、3種類の符号なし整数型の値を2進数表記の文字列に変換するものである。各関数のプロトタイプを次に示す。

プロトタイプ:

```
char *uc2bin( char *buf, unsigned char x );
char *u2bin( char *buf, unsigned int x );
char *ull2bin( char *buf, unsigned long long x );
```

符号なし整数 `x` の値を、その型の幅にそろえた 2 進数表記の文字列に変換し、先頭アドレスを返す。上位の桁には、必要に応じて文字 '0' が補われる。引数には、変換結果を格納するための文字配列 `buf` を渡す。戻り値として、`buf` と同じアドレスを返す。必要な配列の大きさは、`dec2bin.h` で定義された `UC2BIN_BUFSIZE`, `U2BIN_BUFSIZE`, `ULL2BIN_BUFSIZE` によって示される。

dec2bin.c を次のようにしてコンパイルして、オブジェクトファイル dec2bin.o にしておくと便利である。

例. dec2bin.c のコンパイル

```
$ gcc -c dec2bin.c
```

注) dec2bin.c は必ずしもあらかじめオブジェクトファイルにしておく必要はない。これを使用するプログラムと同時にコンパイルしてリンクしてもよい。

このライブラリを応用したプログラムの例 `dec2bin_test1.c` を次に示す.

プログラム：dec2bin_test1.c

```

1 #include <stdio.h>
2 #include "dec2bin.h"
3
4 int main() {
5     unsigned char c = 170U;
6     unsigned int a = 2863311530U;
7     unsigned long long x = 12297829382473034410ULL;
8
9     char buf[ULL2BIN_BUFSIZE];
10
11     printf("uc2bin: %u(dec)\t= %s(bin)\n", (unsigned int)c, uc2bin(buf, c));
12     printf("u2bin: %u(dec)\t= %s(bin)\n", a, u2bin(buf, a));
13     printf("ull2bin: %llu(dec) =\n %s(bin)\n", x, ull2bin(buf, x));
14 }

```

このプログラムをコンパイルして実行する例を次に示す。なお、dec2bin.c は先にコンパイルされて dec2bin.o が作成されていることを前提とする。

例. dec2bin_test1.c をコンパイル, リンクして実行する

[illegible]

この例は、1 バイトが 8 ビット、`unsigned int` 型が 32 ビット、`unsigned long long` 型が 64 ビットの処理系で実行したものである。型の幅が異なる処理系では、出力される 2 進数表記の桁数も異なる。

dec2bin.h は、C++のプログラムからも利用できるように作成されている。同ヘッダファイルでは、関数のプロトタイプを `extern "C"` で囲むことにより、C 言語としてコンパイルされたオブジェクトファイル `dec2bin.o` を C++ のプログラムからリンクできるようにしている。

先の dec2bin_test1.c と同様の処理を行う C++ のプログラム dec2bin_test1.cpp を次に示す.

プログラム: dec2bin_test1.cpp

```
1  #include <iostream>
2  #include "dec2bin.h"
3
4  int main() {
5      unsigned char c = 170U;
6      unsigned int a = 2863311530U;
7      unsigned long long x = 12297829382473034410ULL;
8
9      char buf[ULL2BIN_BUFSIZE];
10
11     std::cout << "uc2bin: "
12               << static_cast<unsigned int>(c) << "(dec)\t= "
13               << uc2bin(buf,c) << "(bin)\n";
14
15     std::cout << "u2bin: "
16               << a << "(dec)\t= "
17               << u2bin(buf,a) << "(bin)\n";
18
19     std::cout << "ull2bin: "
20               << x << "(dec) =\n "
21               << ull2bin(buf,x) << "(bin)\n";
22 }
```

このプログラムは、次のようにしてコンパイルし、C 言語で作成したオブジェクトファイル dec2bin.o とリンクする.

```
$ g++ dec2bin_test1.cpp dec2bin.o -o dec2bin_test1 
```

実行方法は C 言語版の場合と同じであり、同じ出力が得られる.

B.2 高精度の時間計測（C 言語用）

C++では、標準ライブラリの `chrono` に時間計測のための機能が整備されており、`std::chrono::steady_clock` を用いることで、単調増加する時計に基づいた経過時間の計測を行うことができる。一方、C17 の標準ライブラリには、POSIX 環境と Windows 環境の両方で共通して利用できる、同等の時間計測機能は用意されていない。そこで本節では、C 言語のプログラムから利用できる時間計測用ライブラリの実装例を示す。

ここに示すプログラム `dbl_now.c` の `now` 関数は、単調増加する時計の計時値を、秒を単位とする `double` 型の値として返す。POSIX 環境（Linux, macOS）では `clock_gettime`、Windows 環境では `QueryPerformanceCounter` を使用することにより、両方の環境で同じ形式の時間計測を行えるようにしている。

`now` 関数が返す値の基準点は、UNIX エポックではなく、使用する時計によって定められた基準点である。その基準点自体には意味がないため、戻り値は実際の日時を知るためには使用できない。複数回の実行によって得られた値の差から、その間に経過した時間を求めるために使用する。

以下に、`dbl_now.c` の仕様について解説する。

■ 戻り値の意味

`now` 関数は、現在の計時値を、秒を単位とする `double` 型の値として返す。小数部分は 1 秒未満の時間を表す。時計の値を取得できなかった場合は、`NAN` を返す。

戻り値が `NAN` であるかどうかは、`math.h` で定義されている `isnan` を用いて判定する。`NAN` は通常の数値と同じ方法では比較できないため、`t == NAN` などとして判定してはならない。

■ 分解能

POSIX 環境では、`clock_gettime` と `CLOCK_MONOTONIC` を用いる。取得される値は秒とナノ秒の組で表現されるが、ナノ秒単位で値を表現できることは、実際の時計がナノ秒単位の分解能を持つことを意味しない。実際の分解能は、ハードウェアや OS によって異なる。

Windows 環境では、`QueryPerformanceCounter` が提供する高分解能カウンタを使用する。カウンタの周波数は `QueryPerformanceFrequency` によって取得する。実際に測定できる時間の細かさは、ハードウェア、OS、関数の実行に要する時間などによって異なる。

また、取得した値は最終的に `double` 型へ変換されるため、OS が提供する時計の分解能が、戻り値に常にそのまま反映されるとは限らない。ただし、一般的なプログラムの処理時間を測定する用途には十分な分解能が得られる。

■ 基準時刻

`now` 関数が返す値の基準点は、環境によって異なる。POSIX 環境では `CLOCK_MONOTONIC` を用いているため、ある未規定の基準点からの経過時間に相当する、単調に進む計時値が返される。この値は実際の年月日や時刻とは対応しておらず、値の差による経過時間の計測に適している。

Windows 環境でも、高分解能カウンタの基準点自体には意味がなく、得られた値の差を経過時間の計測に使用する。

■ 用途

この関数は、主にプログラムの実行時間や処理時間の測定に使用する。処理の前後で `now` 関数を実行し、得られた 2 つの値の差を取ることで、その間に経過した時間を求めることができる。

■ 使用方法

まず、プログラム `dbl_now.c` をコンパイルして、オブジェクトファイル `dbl_now.o` を作成する。

例. `$ gcc -c dbl_now.c` (GCC によるコンパイルの場合)

`now` 関数を使用するプログラムでは、ヘッダファイル `dbl_now.h` をインクルードし、次のようにして計時値を取得する。

```
double t;  
t = now();
```

時計の値を取得できなかった場合、`now` 関数は `NAN` を返す。エラーを処理する必要がある場合は、`math.h` をインクルードし、次のように判定する。

```

    if ( isnan(t) ) {
        /* エラー処理 */
    }

```

プログラムから実行形式ファイルを作成する際は、dbl_now.c から作成したオブジェクトファイル dbl_now.o をリンクする。

■ 注意

now 関数が返す値は、絶対的な日時（年月日や時刻）を表すものではない。そのため、日時の取得や表示には使用せず、複数回の実行によって得られた値の差から、経過時間を測定する用途に使用する。また、時計の値を取得できなかった場合は NAN が返されるため、必要に応じて isnan を用いて確認する。

ヘッダファイル：dbl_now.h

```

1  #ifndef DBL_NOW_H
2  #define DBL_NOW_H
3
4  /* 単調増加する時計の計時値を秒単位で返す。取得に失敗した場合は NAN を返す。 */
5  double now(void);
6
7  #endif

```

プログラム：dbl_now.c

```

1  #if !defined(_WIN32) && !defined(_POSIX_C_SOURCE)
2  #define _POSIX_C_SOURCE 200809L
3  #endif
4
5  #include <math.h>
6  #include "dbl_now.h"
7
8  #ifdef _WIN32
9  #include <windows.h>
10 #else
11 #include <time.h>
12 #endif
13
14 /* 単調増加する時計の計時値を秒単位で返す */
15 double now(void) {
16     #ifdef _WIN32
17         LARGE_INTEGER freq;
18         LARGE_INTEGER counter;
19
20         if ( !QueryPerformanceFrequency(&freq) ||
21             freq.QuadPart <= 0 ||
22             !QueryPerformanceCounter(&counter) ) {
23             return NAN;
24         }
25
26         return (double)counter.QuadPart / (double)freq.QuadPart;
27     #else
28         struct timespec ts;
29
30         if ( clock_gettime(CLOCK_MONOTONIC,&ts) != 0 ) {
31             return NAN;
32         }
33
34         return (double)ts.tv_sec + (double)ts.tv_nsec*1.0e-9;
35     #endif
36 }
37

```

■ now() を使用するプログラムの例

次のプログラム dbl_now.test02.c は、大きな符号なし整数 n が素数であるかどうかを判定し、その処理に要する時間を、now 関数から得られた 2 つの計時値の差によって測定するものである。素数判定の結果を表示する処理は、時間の計測区間には含めていない。

プログラム：dbl_now_test02.c

```
1  #include    <stdio.h>
2  #include    <math.h>
3  #include    "dbl_now.h"
4
5  /* unsigned long long に対する素数判定 */
6  static int is_prime_ull( unsigned long long n ) {
7      unsigned long long i;
8
9      if ( n < 2 ) return 0;
10     if ( n == 2 ) return 1;
11     if ( n % 2 == 0 ) return 0;
12
13     for ( i=3; i<=n/i; i+=2 ) {
14         if ( n % i == 0 ) return 0;
15     }
16
17     return 1;
18 }
19
20 int main() {
21     unsigned long long n;
22     double t1, t2;
23     int result;
24
25     // n = 4000000000000000007ULL;      // 検査候補(1)
26     n = 4611686018427387847ULL;      // 検査候補(2)
27     // n = 18446744073709551557ULL;    // 検査候補(3)
28
29     t1 = now();
30
31     if ( isnan(t1) ) {
32         fprintf(stderr, "Failed to start time measurement.\n");
33         return 1;
34     }
35
36     result = is_prime_ull(n);
37
38     t2 = now();
39
40     if ( isnan(t2) ) {
41         fprintf(stderr, "Failed to finish time measurement.\n");
42         return 1;
43     }
44
45     if ( result ) {
46         printf("%llu is prime.\n", n);
47     } else {
48         printf("%llu is not prime.\n", n);
49     }
50
51     printf("elapsed = %.6f sec\n", t2-t1);
52 }
```

これをコンパイルして実行する例を次に示す。

例. dbl_now_test02.c をコンパイル, リンクして実行する

```
$ gcc dbl_now_test02.c dbl_now.o -o dbl_now_test02  ←コンパイル, リンク
$ ./dbl_now_test02  ←実行
4611686018427387847 is prime.      ←素数の判定結果
elapsed = 4.442792 sec              ←判定に要した時間
```

表示される計算時間は, 使用する CPU, コンパイラ, 最適化の設定, 実行時の負荷などによって異なる. dbl_now_test02.c の n の値を変更し, 計算時間の変化を確認されたい。

C コンパイラと開発環境

C 言語と C++ で記述されたソースプログラムを実行するには、一般に、コンパイラによってオブジェクトコードへ変換し、必要なライブラリなどとリンクして、実行形式ファイルを作成する。この処理には、コンパイラだけでなく、リンカ、標準ライブラリ、ヘッダファイルなどから構成される開発環境が必要となる。

C/C++ のコンパイラには複数の種類があり、開発主体、対応する計算機環境、提供される機能、コンパイル時に指定できるオプションなどに違いがある。代表的なものとして、GCC、Clang、MSVC などが挙げられる。

これらのコンパイラは、C 言語や C++ の規格そのものではなく、それぞれの規格に基づいてプログラムを処理するためのソフトウェアである。従って、プログラムの開発に使用する計算機環境へ、目的に適したコンパイラと関連ツールをインストールし、利用できる状態にしておく必要がある。

Linux では、コンパイラがあらかじめ導入されている場合もあるが、環境によっては利用者がパッケージ管理システムなどを用いて追加する必要がある。Windows には一般的な C/C++ 開発環境が標準では用意されていないため、MSYS2 や Visual Studio などを導入することで、C/C++ のコンパイラと関連ツールを利用できるようにする。

ここでは、代表的な C/C++ コンパイラである GCC、Clang とその基盤となる LLVM、および MSVC について、それぞれの特徴と位置づけを説明する。また、基本的なコンパイルコマンドの書式と、主なオプションの指定方法についても解説する。さらに、Windows 上でこれらのコンパイラを利用するための開発環境の導入方法を示す。

C.1 GCC

GCC (GNU Compiler Collection) は、GNU プロジェクトによって開発されているコンパイラ群であり、C 言語と C++ をはじめとする複数のプログラミング言語に対応している。また、多くの CPU アーキテクチャやオペレーティングシステム (OS) に対応しており、広く利用されている。

GCC を用いて C 言語と C++ のプログラムをコンパイルする際には、主に `gcc` コマンドと `g++` コマンドを使用する。これらは、プリプロセッサ、コンパイラ、アセンブラ、リンカなどを、指定された入力ファイルやオプションに応じて実行するコンパイラドライバである。ソースファイル名や各種のオプションを引数として与えることで、ソースプログラムのコンパイル、オブジェクトファイルの生成、ライブラリとのリンク、実行形式ファイルの生成などを行うことができる。

`gcc` は主に C 言語のプログラムをコンパイルする際に使用し、`g++` は C++ のプログラムをコンパイルする際に使用する。例えば、C 言語のソースファイル `sample.c` をコンパイルし、実行形式ファイル `sample` を生成するには、次のように実行する。

```
$ gcc sample.c -o sample
```

（「\$」はシェルのプロンプト）

ここで、`-o` オプションは、生成する実行形式ファイルの名前を指定するものである。

C++ のソースファイル `sample.cpp` をコンパイルする場合は、次のように `g++` コマンドを使用する。

```
$ g++ sample.cpp -o sample
```

`gcc` コマンドでも、拡張子が `.cpp` であるファイルは C++ のソースファイルとしてコンパイルされる。ただし、`gcc` コマンドでは C++ 標準ライブラリが自動的にリンクされないため、C++ のプログラムをコンパイル、リンクする場合は、通常 `g++` コマンドを使用する。

C.1.1 基本的なコマンド書式

`gcc` および `g++` の基本的なコマンド書式を次に示す。

```
gcc [オプション] [入力ファイル …] [-o 出力ファイル]
g++ [オプション] [入力ファイル …] [-o 出力ファイル]
```

コマンド名に続けて与える文字列を、コマンドの引数という。入力ファイルとして、コンパイルするソースファイル、リンクするオブジェクトファイル、ライブラリなどを指定できる。また、コンパイルやリンクの方法を指定するためのオプションも引数として与える。

オプションの多くはハイフン `-` から始まる。例えば、`-o` は主な出力ファイルの名前を指定するオプションであり、その直後に出力ファイル名を与える。リンクまで行って実行形式ファイルを生成する場合、`-o` を指定しなければ、標

準の出力ファイル名は Unix 系環境では `a.out`、Windows 環境では `a.exe` となる。

一方、`-c` オプションを指定してリンクを行わない場合は、ソースファイルの拡張子を `.o` に置き換えた名前のオブジェクトファイルが生成される。例えば、`sample.c` からは、標準では `sample.o` が生成される。

複数のソースファイルから実行形式ファイルを生成する場合は、それらのファイル名を空白で区切って並べる。例えば、`main.cpp`、`calc.cpp`、`draw.cpp` をコンパイル、リンクして、実行形式ファイル `sample` を生成するには、次のように実行する。

```
$ g++ main.cpp calc.cpp draw.cpp -o sample
```

多くのオプションやファイル名は、記述する位置をある程度変更できる。ただし、リンク時に指定するオブジェクトファイルやライブラリは、引数として与えられた順序に従って処理される。このため、ライブラリは、通常、そのライブラリを使用するソースファイルやオブジェクトファイルよりも後に記述する。例えば、Unix 系環境で数学ライブラリをリンクする場合は、次のように `-lm` を末尾付近に記述する。

```
$ gcc sample.c -o sample -lm
```

C.1.2 主なコンパイラオプション

`gcc` および `g++` には、多数のコンパイラオプションが用意されている。主なものを表 109 に示す。

表 109: GCC の主なコンパイラオプション

オプション	解 説
<code>-o file</code>	生成する出力ファイルの名前を <code>file</code> にする。
<code>-c</code>	リンクを行わず、ソースファイルごとにオブジェクトファイルを生成する。
<code>-E</code>	プリプロセッサによる処理だけを行い、その結果を標準出力へ出力する。
<code>-S</code>	コンパイル結果をアセンブリ言語のコードとして出力し、アセンブルを行わない。
<code>-x language</code>	後続の入力ファイルを、拡張子にかかわらず指定した言語として処理する。 例えば、 <code>-x c</code> は C 言語、 <code>-x c++</code> は C++ を指定する。
<code>-std=standard</code>	使用する言語規格を指定する。例えば、 <code>-std=c17</code> 、 <code>-std=c++17</code> 。
<code>-Wall</code>	プログラムの問題を発見するための、基本的な警告を有効にする。
<code>-Wextra</code>	<code>-Wall</code> には含まれない追加の警告を有効にする。
<code>-Wpedantic</code>	指定した C/C++ 規格に厳密には適合しない記述について警告を出す。
<code>-Werror</code>	警告をエラーとして扱い、警告が発生した場合は処理を失敗として終了する。
<code>-g</code>	デバッグで使用するための情報を生成する。
<code>-O0</code> , <code>-Og</code> , <code>-O1</code> , <code>-O2</code> , <code>-O3</code>	生成するコードの最適化水準を指定する。
<code>-I dir</code>	ヘッダファイルを検索するディレクトリに <code>dir</code> を追加する。
<code>-L dir</code>	ライブラリを検索するディレクトリに <code>dir</code> を追加する。
<code>-lname</code>	<code>name</code> をライブラリ名として検索し、そのライブラリをリンクする。Unix 系環境における <code>-lm</code> は、数学ライブラリをリンクする指定である。
<code>-Dname</code>	プリプロセッサのマクロ <code>name</code> を、値 <code>1</code> として定義する。
<code>-Dname=value</code>	プリプロセッサのマクロ <code>name</code> を、値 <code>value</code> で定義する。
<code>-pthread</code>	対応する環境で、POSIX スレッドを利用するために必要なコンパイルおよびリンクの設定を有効にする。
<code>-fopenmp</code>	OpenMP による並列処理を有効にし、必要なライブラリをリンクする。
<code>--version</code>	使用している GCC のバージョン情報を表示する。

例えば、C++17 の規格を使用し、主要な警告を有効にして、デバッグ情報を含む実行形式ファイルを生成する場合には、次のように実行する。

```
$ g++ -std=c++17 -Wall -Wextra -g sample.cpp -o sample
```

また、`std::thread` を使用するプログラムをコンパイルする場合には、対応する環境で `-pthread` オプションを指定する。

```
$ g++ -std=c++17 -pthread thread_test.cpp -o thread_test
```

`-pthread` は、単にスレッドライブラリをリンクするだけでなく、処理系によってはプリプロセッサに関する設定にも影響する。従って、分割コンパイルを行う場合には、スレッドを使用するソースファイルのコンパイル時と、最終的なリンク時の両方で `-pthread` を指定する。

C.1.3 最適化水準

GCC では、`-O` から始まるオプションによって、生成する機械語コードの最適化水準を指定することができる。最適化水準を高くすると、生成されたプログラムの実行速度が向上する場合があるが、コンパイル時間や生成されるコードの大きさが増加する場合もある。主な最適化オプションを表 110 に示す。

表 110: GCC の主な最適化オプション

オプション	意味
<code>-O0</code>	ほとんどの最適化を行わない。最適化水準を指定しない場合の標準の設定であり、ソースコードと実行される処理の対応を追いやすい。
<code>-Og</code>	デバッグのしやすさを保ちながら、デバッグを妨げにくい範囲の最適化を行う。
<code>-O1</code>	コンパイル時間を大きく増加させない範囲で、基本的な最適化を行う。
<code>-O2</code>	実行速度とコードサイズの均衡を考慮しながら、広範囲の最適化を行う。一般的な実用プログラムで使用しやすい最適化水準である。
<code>-O3</code>	<code>-O2</code> よりも積極的な最適化を行う。プログラムによっては高速化するが、コードサイズやコンパイル時間が増加し、必ずしも実行速度が向上するとは限らない。
<code>-Os</code>	生成するコードの大きさを抑えることを重視して最適化を行う。
<code>-Ofast</code>	<code>-O3</code> を基礎として、規格への厳密な適合性を緩める最適化も有効にする。浮動小数点演算などの結果が厳密な規格動作と異なる可能性があり、注意を要する。

プログラムをデバッグする場合には、`-O0` または `-Og` を使用する。特に `-Og` は、デバッグのしやすさを考慮しながら一定の最適化を行うために用意された最適化水準である。

通常の実用的なプログラムでは、まず `-O2` を指定して動作と性能を確認するとよい。より高い最適化水準を使用する場合には、実際に実行時間を測定し、その効果を確認する必要がある。最適化水準を高くしても、必ずしも実行速度が向上するとは限らない。

また、プログラムに未定義動作を引き起こす記述が含まれている場合は、最適化の有無や水準によって、見かけ上の動作が変化することがある。`-Ofast` を使用する場合には、浮動小数点演算などの結果が厳密な規格動作と異なる可能性があるため、計算結果についても十分に確認する必要がある。

例えば、C++17 のプログラムを `-O2` で最適化する場合には、次のように実行する。

```
$ g++ -std=c++17 -O2 sample.cpp -o sample
```

C.1.4 文字のエンコーディングの指定

ソースファイルに日本語などの文字を記述する場合には、ソースファイル自体のエンコーディングと、ソースコード中の文字列リテラルをコンパイル後のプログラム内に格納する際のエンコーディングを区別する必要がある。GCC では、これらのエンコーディングをコンパイラオプションによって指定することができる。

■ ソースファイルのエンコーディングの指定

`-finput-charset=charset` オプションを用いると、コンパイラがソースファイルを読み込む際のエンコーディングを指定することができる。例えば、ソースファイルが UTF-8 で記述されている場合には、次のように指定する。

```
-finput-charset=UTF-8
```

GCC では、このオプションが指定されていない場合、ソースファイルのエンコーディングが処理環境のロケールに基づいて判断される。ロケールからエンコーディングを取得できない場合には、UTF-8 が使用される。処理環境による違いを避ける場合や、文字化け、コンパイルエラーが発生する場合には、このオプションを明示的に指定するとよい。

■ 文字列リテラルのエンコーディングの指定

`-fexec-charset=charset` オプションを用いると、通常の文字リテラルや文字列リテラルを実行文字集合へ変換する際のエンコーディングを指定することができる。このオプションを指定しない場合、GCC では UTF-8 が使用される。例えば、文字列リテラルを Windows で広く使用されてきた CP932 のバイト列としてプログラム内に格納する場合には、次のように指定する。

```
-fexec-charset=CP932
```

例えば、UTF-8 で記述されたソースファイルを読み込み、文字列リテラルを CP932 に変換してプログラム内に格納する場合には、次のようにコンパイルする。

```
gcc -finput-charset=UTF-8 -fexec-charset=CP932 sample.c
```

※ C++のプログラムの場合には、`g++` コマンドを使用する。

例えば次のように、ソースファイル中に文字列リテラルが記述されているものとする。

```
const char *s = "日本語";
```

この場合、ソースファイル中の「日本語」は UTF-8 として読み取られ、コンパイル後のプログラム内では CP932 のバイト列として格納される。ただし、`-fexec-charset` オプションが指定するのは、ソースコード中に記述された通常の文字リテラルおよび文字列リテラルのエンコーディングである。`char` 型の配列や C++ の `std::string` 型には、格納されたバイト列がどのエンコーディングによるものであるかを表す情報は保持されない。これらは単なるバイト列を保持するものであり、どのエンコーディングとして扱うかはプログラム側で管理する必要がある。

なお、ワイド文字リテラルやワイド文字列リテラルに用いるエンコーディングは、`-fwide-exec-charset=charset` オプションによって指定する。例えば、`L"日本語"` のようなワイド文字列リテラルが対象となる。既定のエンコーディングは、ターゲット環境における `wchar_t` 型の幅とバイト順序に応じた UTF-16 または UTF-32 である。

また、これらのコンパイラオプションは、ファイルや端末に対する入出力の際に、エンコーディングを自動的に変換するものではない。出力する文字列のエンコーディングと、端末やファイルを読み取る側が想定するエンコーディングが一致していない場合には、文字化けが発生する可能性がある。

GCC で指定できる代表的なエンコーディングを表 111 に示す。これらの名前は、`-finput-charset` オプションおよび `-fexec-charset` オプションの引数として使用できる。ただし、実際に使用できる名前は、GCC が動作する処理環境の文字コード変換機能にも依存する。

表 111: 代表的な文字エンコーディング

エンコーディング	指定例	解 説
UTF-8	UTF-8	Unicode の文字を 1~4 バイトで表現する可変長のエンコーディングである。現在、ソースファイルやテキストファイルに使用するエンコーディングとして広く普及している。ASCII の範囲の文字は、ASCII と同じ 1 バイトで表現される。
CP932	CP932	Microsoft が日本語版 Windows で使用してきたエンコーディングである。Shift_JIS を基礎としているが、独自に追加された文字を含む。Windows 上の従来型のプログラムやデータとの互換性が必要な場合に使用されることがある。
Shift_JIS	SHIFT_JIS	日本語を扱うために広く使用されてきた可変長のエンコーディングである。CP932 とよく似ているが、両者の文字集合と変換規則は完全には同一ではない。古い日本語テキストや既存システムとの互換性のために使用されることがある。
EUC-JP	EUC-JP	UNIX 系の日本語環境で広く使用されてきたエンコーディングである。現在、新しく作成するソースファイルに使用されることは少ないが、古い UNIX 系システムのソースコードやテキストデータを扱う際に必要となることがある。
ISO-2022-JP	ISO-2022-JP	エスケープシーケンスによって使用する文字集合を切り替える 7 ビット系のエンコーディングである。主に従来の電子メールなどで使用されてきた。ソースファイルや実行時の文字列に使用する機会は少ないが、既存のテキストデータを扱う際に必要となることがある。
US-ASCII	US-ASCII	英数字や基本的な記号を 1 バイトで表現する 7 ビットの文字エンコーディングである。日本語は表現できないが、英語だけで記述されたソースファイルや、文字コード上の互換性を確認する場合に使用できる。

C.1.5 実行時のスタックサイズの指定

関数内で宣言されたローカル変数や、関数呼び出しに伴って必要となる情報などは、主にスタック領域に格納される。深い再帰呼び出しを行う場合や、大きな配列をローカル変数として宣言する場合には、スタック領域が不足し、プログラムが異常終了することがある。このような場合には、プログラムの実行時に使用できるスタックサイズを変更することができる。

ただし、スタックサイズの指定方法は、OS や実行ファイルの形式によって異なる。また、実行ファイルにスタックサイズの情報を設定する処理は、厳密にはコンパイラではなくリンカによって行われる。GCC では、`-W1`、オプションを用いることで、リンカにオプションを渡すことができる。

■ Linux におけるスタックサイズの指定

Linux では、プロセスに設定された資源制限によって、使用できるスタックサイズの上限が定められる。Bash で

は、次のコマンドによって、現在設定されているスタックサイズのソフト制限を確認することができる。

```
ulimit -S -s
```

表示される値の単位は KiB である。例えば、スタックサイズのソフト制限を 16MiB に設定する場合には、プログラムを実行する前に次のように指定する。

```
ulimit -S -s 16384
```

ただし、ソフト制限には、ハード制限を超える値を設定することはできない。現在のハード制限は、次のコマンドによって確認できる。

```
ulimit -H -s
```

設定された資源制限は、そのシェルおよびそこから起動されるプログラムに引き継がれる。例えば、次のように設定してからプログラムを実行する。

```
ulimit -S -s 16384
./sample
```

なお、GNU リンカには、ELF 形式の実行ファイルに含まれる PT.GNU_STACK セグメントへサイズ情報を設定する `-z stack-size=` オプションもある。例えば、次の指定は、同セグメントに 16777216 バイトというサイズ情報を記録する。

```
gcc sample.c -Wl,-z,stack-size=16777216
```

ただし、これは ELF 形式の実行ファイルに情報を記録するためのリンカオプションであり、Linux 上でプログラムの主スレッドが実際に使用できるスタックの上限を直接変更する一般的な方法ではない。Linux では、通常、`ulimit -S -s` によって資源制限を設定する。

■ MSYS2 におけるスタックサイズの指定

MSYS2 の UCRT64 環境や MINGW64 環境などで生成される Windows 用の実行ファイルでは、GNU リンカの `--stack` オプションによって、スタックの予約サイズを指定することができる。GCC からこのオプションをリンカに渡す場合には、例えば次のように指定する。

```
gcc sample.c -Wl,--stack,16777216
```

※ C++ のプログラムの場合には、`g++` コマンドを使用する。

この指定では、Windows 用実行ファイルのヘッダに、スタックの予約サイズとして 16777216 バイト、すなわち 16MiB が設定される。`--stack` オプションは Windows 用の PE 形式の実行ファイルを生成するリンカに固有の機能であり、Linux 上の GCC で同じ指定を使用することはできない。

なお、`-Wl,` に続く文字列はコンマで区切られ、それぞれ独立した引数としてリンカに渡される。例えば、

```
-Wl,--stack,16777216
```

という指定では、`--stack` と 16777216 がリンカに渡される。また、`-c` オプションによってリンクを行わず、オブジェクトファイルを生成する場合には、リンカは起動されないため、スタックサイズは設定されない。

スタックサイズを大きくすると、深い再帰呼び出しや大きなローカル配列を使用できる範囲は広がる。しかし、スタックサイズを大きくすることは、無制限な再帰呼び出しや過大なローカル配列を安全にするものではない。大量のデータを扱う場合には、動的に確保したメモリ領域を使用するなど、プログラムの設計を見直すことも重要である。

C.2 Clang / LLVM

Clang は、C 言語や C++ などに対応するコンパイラであり、LLVM プロジェクトを構成する主要なソフトウェアの一つである。LLVM は、プログラムの解析や最適化、各種 CPU に対応した機械語コードの生成などを行うための、モジュール化された再利用可能なコンパイラ基盤である。Clang はソースコードを解析し、LLVM IR と呼ばれる中間表現へ変換する。その後、LLVM の最適化機能とコード生成機能によって、中間表現が最適化され、ターゲットとなる計算機の機械語コードが生成される。

LLVM は、2000 年代の初めに、米国のイリノイ大学アーバナ・シャンペーン校において、Chris Lattner と Vikram Adve らによる研究プロジェクトとして開発が始められた。プログラムの解析、最適化、コード生成などの機能を柔軟に組み合わせることのできるコンパイラ基盤として設計され、その後、LLVM 上で C 言語や C++などを扱うフロントエンドとして Clang が開発された。Clang の初期の開発は米国の Apple 社によって強く推進され、LLVM とともに同社の開発環境へ採用された。現在では、企業、大学、個人の開発者が参加する国際的なオープンソースプロジェクトとして開発が続けられており、GCC と並ぶ代表的な C/C++コンパイラの一つとして広く利用されている。

LLVM の基盤は C/C++に限らず、他のプログラミング言語の処理系や、静的解析ツール、デバッガ、リンカなどの開発、JIT コンパイルの実現にも利用されている。Clang / LLVM の大きな特徴は、コンパイラを構成する各機能が比較的独立した部品として設計されている点にある。このため、新しいプログラミング言語の処理系や、新しい CPU に対応するコード生成機能、ソースコード解析ツールなどを構築するための基盤として利用しやすい。

また、Clang は、コンパイル時のエラーや警告の表示が分かりやすく、コンパイル速度やメモリ使用量にも配慮して設計されている。Clang のコマンド体系は GCC との互換性を重視して設計されており、C 言語のコンパイルには `clang` コマンドを、C++のコンパイルには `clang++` コマンドを使用する。`-o`、`-c`、`-std=c++17`、`-Wall`、`-O2` など、基本的なコンパイラオプションの多くも GCC と共通している。このため、GCC を使用した経験があれば、Clang もほぼ同じ形式で利用することができる。

C.2.1 LLVM IR とコンパイラの構成

一般に、コンパイラの内部では、ソースコードを直接機械語へ変換するのではなく、いくつかの段階を経て処理が行われる。Clang / LLVM では、この処理が主にフロントエンド、中間表現に対する最適化処理、バックエンドに分けられている。Clang / LLVM による処理の概略を次に示す。

ソースコード → Clang → LLVM IR → 最適化 → バックエンド → 機械語コード

Clang は、C 言語や C++のソースコードを読み込み、字句解析、構文解析、型の検査などを行う。このように、特定のプログラミング言語を解析し、コンパイラ内部で扱う形式へ変換する部分を**フロントエンド**という。Clang は、C 言語および C++などに対応するフロントエンドとして動作する。

Clang によって解析されたプログラムは、**LLVM IR** と呼ばれる中間表現へ変換される。IR は *Intermediate Representation* の略であり、ソースコードと機械語コードの間に位置する表現、すなわち**中間表現**である。LLVM IR は、特定のプログラミング言語の文法や、特定の CPU の機械語そのものには直接依存しない中間表現として設計されている。このため、異なるプログラミング言語から生成されたプログラムに、共通の最適化処理を適用することができる。

LLVM IR に対しては、不要な処理の除去、定数式の計算、関数のインライン展開、ループ処理の改善など、さまざまな最適化が行われる。最適化された LLVM IR は、最後に対象となる CPU に対応した機械語へ変換される。この機械語コードの生成を担当する部分を**バックエンド**という。LLVM には、x86 系、ARM 系、RISC-V など、複数の命令体系に対応するバックエンドが用意されている。

この構成では、プログラミング言語を扱うフロントエンドと、CPU ごとの機械語を生成するバックエンドとの間を LLVM IR が仲介する。従って、新しいプログラミング言語の処理系を構築する場合には、その言語から LLVM IR を生成するフロントエンドを開発することで、LLVM が備える最適化機能や各種 CPU 向けのコード生成機能を利用できる。また、新しい CPU に対応する場合にも、LLVM IR を入力とするバックエンドを追加することで、複数のプログラミング言語から利用できる可能性がある。

LLVM IR は、コンパイラの内部で保持されるほか、テキスト形式またはバイナリ形式のファイルとして保存することもできる。人が読むことのできるテキスト形式では、通常、拡張子 `.ll` のファイルとして保存される。また、**LLVM**

ビットコードと呼ばれるバイナリ形式では、拡張子 `.bc` が使用される。Clang を使用して、C 言語のソースファイル `sample.c` からテキスト形式の LLVM IR を生成する例を次に示す。

```
$ clang -S -emit-llvm sample.c -o sample.ll
```

この例では、`-emit-llvm` によって通常の機械語コードではなく LLVM IR を生成し、`-S` によって人が読むことのできるテキスト形式で出力している。`-S` を付けずに `-c` を指定すると、LLVM ビットコードとして出力することができる。

```
$ clang -c -emit-llvm sample.c -o sample.bc
```

通常の C/C++ プログラムの開発では、利用者が LLVM IR を直接扱う必要はない。しかし、LLVM IR を介してフロントエンド、最適化処理、バックエンドが分離されていることは、Clang / LLVM の柔軟性と拡張性を理解するうえで重要である。

C.2.2 clang と clang++ の基本的な使用方法

Clang を用いて C 言語のプログラムをコンパイルする際には `clang` コマンドを使用し、C++ のプログラムをコンパイルする際には `clang++` コマンドを使用する。これらのコマンドの基本的な使用法は、GCC の `gcc` および `g++` とほぼ同じである。例えば、C 言語のソースファイル `sample.c` をコンパイルして、実行形式ファイル `sample` を生成する場合には、次のように実行する。

```
$ clang sample.c -o sample
```

また、C++ のソースファイル `sample.cpp` をコンパイルする場合には、次のように `clang++` コマンドを使用する。

```
$ clang++ sample.cpp -o sample
```

`clang` コマンドでも、拡張子が `.cpp` であるファイルを C++ のソースコードとしてコンパイルすることはできる。ただし、`clang` コマンドを使用してリンクする場合には、C++ 標準ライブラリが自動的に指定されない。このため、通常は C++ のプログラムに `clang++` コマンドを使用する。

`clang` および `clang++` の基本的なコマンド書式を次に示す。

```
clang  [オプション] [入力ファイル …] [-o 出力ファイル]
clang++ [オプション] [入力ファイル …] [-o 出力ファイル]
```

コマンド名に続けて、ソースファイル、オブジェクトファイル、ライブラリなどの入力ファイルと、処理方法を指定するオプションを引数として与える。`-o` は、生成する出力ファイルの名前を指定するためのオプションである。リンクまで行う場合に `-o` を指定しなければ、生成される実行形式ファイルの標準の名前は、Unix 系の環境では `a.out`、Windows 環境では `a.exe` となる。

複数のソースファイルから実行形式ファイルを生成する場合には、それらのファイル名を空白で区切って並べる。例えば、`main.cpp`、`calc.cpp`、`draw.cpp` から実行形式ファイル `sample` を生成する場合には、次のように実行する。

```
$ clang++ main.cpp calc.cpp draw.cpp -o sample
```

また、リンクを行わずにオブジェクトファイルを生成する場合には、`-c` オプションを使用する。

```
$ clang++ -c calc.cpp -o calc.o
```

生成した複数のオブジェクトファイルをリンクして、実行形式ファイルを生成する場合には、次のように実行する。

```
$ clang++ main.o calc.o draw.o -o sample
```

Clang のコマンド体系は GCC との互換性を重視して設計されているため、`-std=c++17`、`-Wall`、`-Wextra`、`-g`、`-O2`、`-I`、`-L`、`-l` など、多くの基本的なオプションを GCC と同じ形式で 사용할 ことができる。ただし、同じ名前のオプションでも、有効になる警告や最適化処理などの詳細は GCC と Clang で異なる場合がある。

例えば、C++17 の規格を使用し、主要な警告を有効にして、デバッグ情報を含む実行形式ファイルを生成する場合には、次のように実行する。

```
$ clang++ -std=c++17 -Wall -Wextra -g sample.cpp -o sample
```

また、最適化水準として `-O2` を指定する場合には、次のように実行する。

```
$ clang++ -std=c++17 -O2 sample.cpp -o sample
```


C.2.3 主なコンパイラオプション

Clang のコマンド体系は GCC との互換性を重視して設計されており、基本的なコンパイラオプションの多くは GCC と共通している。Clang で使用できる主なオプションを表 112 に示す。

表 112: Clang の主なコンパイラオプション

オプション	解 説
-o <i>file</i>	生成する出力ファイルの名前を <i>file</i> にする。
-c	リンクを行わず、ソースファイルごとにオブジェクトファイルを生成する。
-E	プリプロセッサによる処理だけを行う。出力先を指定しない場合は、結果を標準出力へ出力する。
-S	コンパイル結果をアセンブリ言語のコードとして出力し、アセンブルを行わない。
-x <i>language</i>	後続の入力ファイルを、拡張子にかかわらず指定した言語として処理する。
-fsyntax-only	ソースコードの構文解析や意味解析などを行うが、オブジェクトファイルや実行形式ファイルは生成しない。
-emit-llvm	-S または -c と組み合わせ、LLVM IR をテキスト形式またはビットコード形式で出力する。
-std= <i>standard</i>	使用する C/C++ 規格を指定する。例えば、C17 は <code>c17</code> 、C++17 は <code>c++17</code> を指定する。
-Wall	プログラムの問題を発見するための、基本的な警告を有効にする。
-Wextra	-Wall には含まれない追加の警告を有効にする。
-Wpedantic	指定した C/C++ 規格に厳密には適合しない記述について警告を出す。
-Werror	警告をエラーとして扱い、警告が発生した場合にコンパイルを失敗させる。
-Weverything	Clang が備えるすべての警告を有効にする。通常の開発では警告が過剰になる場合があるため、使用には注意が必要である。
-g	デバッガで使用するための情報を生成する。
-O0, -Og, -O1, -O2, -O3	生成するコードの最適化水準を指定する。
-Os, -Oz	生成されるコードの大きさを抑えることを重視して最適化を行う。-Oz は -Os よりもコードサイズの縮小を重視する。
-I <i>dir</i>	ヘッダファイルを検索するディレクトリに <i>dir</i> を追加する。
-L <i>dir</i>	ライブラリを検索するディレクトリに <i>dir</i> を追加する。
-lname	<i>name</i> をライブラリ名として検索し、そのライブラリをリンクする。
-D <i>name</i>	プリプロセッサのマクロ <i>name</i> を値 1 で定義する。
-D <i>name</i> = <i>value</i>	プリプロセッサのマクロ <i>name</i> を値 <i>value</i> で定義する。
-pthread	対応する環境で、POSIX スレッドを利用するために必要なコンパイルおよびリンクの設定を有効にする。
-fopenmp	OpenMP の指示文を解釈し、並列処理のためのコード生成を有効にする。利用には対応する OpenMP ランタイムが必要である。
-fcolor-diagnostics	コンパイル時のエラーや警告を色分けして表示する。
-stdlib= <i>library</i>	C++ で使用する標準ライブラリの実装を指定する。 <i>library</i> には主に <code>libc++</code> または <code>libstdc++</code> を指定する。
--target= <i>triple</i>	コードを生成するターゲット環境を、 <u>ターゲットトリプル</u> によって指定する。
--version	使用している Clang のバージョン情報を表示する。

※「ターゲットトリプル」は、CPU アーキテクチャ、OS、ABI などの組合せによって、ターゲット環境を表す文字列である。

例えば、ソースコードの構文と意味だけを検査し、オブジェクトファイルや実行形式ファイルを生成しない場合には、`-fsyntax-only` オプションを使用する。

```
$ clang++ -std=c++17 -Wall -Wextra -fsyntax-only sample.cpp
```

`-Weverything` は、Clang が備えるすべての警告を有効にするオプションである。ただし、通常は必要性の低い警告や、新しいバージョンで追加された警告も有効になるため、一般的なプログラムの開発では、まず `-Wall` と `-Wextra` を使用するとよい。

Clang の最適化水準には、GCC とおおむね同じ名前のオプションが用意されており、`-O0`、`-Og`、`-O1`、`-O2`、`-O3`、`-Os` などを同様の目的で使うことができる。従って、各最適化水準の基本的な考え方については、前節の GCC に関する解説を参照されたい。Clang には、`-Os` よりもコードサイズの縮小を重視する `-Oz` も用意されている。また、`-Ofast` は非推奨とされているため、同様の最適化が必要な場合には、`-O3 -ffast-math` などを明示的に指定する。

■ 文字のエンコーディングに関する注意

現在の一般的な Clang では、ソースファイルのエンコーディングと、通常の文字リテラルおよび文字列リテラルに用いる実行文字集合のエンコーディングとして、基本的に UTF-8 が使用される。`-finput-charset=UTF-8` および

`-fexec-charset=UTF-8` は指定できるが、GCC のように CP932 などの任意のエンコーディングへ変換する機能は提供されていない。UTF-8 以外で記述されたソースファイルを使用する場合には、あらかじめ UTF-8 へ変換する必要がある。

■ 実行時のスタックサイズの指定

Clang で生成するプログラムのスタックサイズを指定する方法は、使用する OS やリンカによって異なる。スタックサイズの情報を実行形式ファイルに設定する処理は、厳密には Clang ではなくリンカによって行われる。Clang では、`-Wl`, オプションを用いることで、リンカにオプションを渡すことができる。

Linux におけるスタックサイズの設定方法は、GCC の場合と同様である。また、Windows 上の MinGW 系環境で、PE 形式の実行形式ファイルを生成するリンカが `--stack` オプションに対応している場合には、例えば次のように指定する。

```
clang sample.c -Wl,--stack,16777216
```

※ C++ のプログラムの場合には、`clang++` コマンドを使用する。

この指定では、Windows 用実行形式ファイルのヘッダに、スタックの予約サイズとして 16777216 バイト、すなわち 16MiB が設定される。なお、`-Wl`, によって指定される内容はリンカに渡されるため、`-c` オプションによってリンクを行わない場合にはスタックサイズは設定されない。詳しくは、GCC における実行時のスタックサイズの指定に関する説明を参照されたい。

C.3 MSVC (Microsoft Visual C++)

MSVC (Microsoft Visual C++) は、Microsoft が提供するプロプライエタリな C/C++ 開発ツールセットであり、主に Windows 上で動作するソフトウェアの開発に使用される。コンパイラ、リンカ、ライブラリ管理ツール、標準ライブラリなどから構成され、Windows 用の実行形式ファイル、システムライブラリ、Windows SDK などとの連携を前提として設計されている。

Microsoft は、MSVC を利用するための統合開発環境として Visual Studio を提供している。Visual Studio には、ソースコードの編集、プログラムの構築、デバッグ、プロジェクト管理などを行うための高機能な環境が備えられており、その内部で MSVC のコンパイラやリンカが使用される。一方、MSVC のコマンドライン開発ツールは、Visual Studio の統合開発環境を導入せずに、Build Tools for Visual Studio としてインストールすることもできる。

MSVC は、GCC や Clang とは異なる開発の経緯を持つ Microsoft 独自の処理系である。C 言語および C++ の標準規格に対応しているが、コマンドの書式、コンパイラオプション、標準ライブラリの実装、オブジェクトファイルやライブラリの形式などには、GCC や Clang との相違がある。例えば、GCC や Clang では多くのオプションがハイフン - から始まるのに対して、MSVC ではスラッシュ / から始まるオプションが一般的に使用される。従って、他のコンパイラで使用していたコマンドを MSVC で実行する場合には、オプションや周辺ツールの違いに注意する必要がある。

本書では、Visual Studio の統合開発環境 (IDE) の操作方法には立ち入らず、Build Tools for Visual Studio を導入して、Windows の開発者用コマンド環境から MSVC を利用する方法を説明する。具体的には、必要となるツールの導入、コマンド環境の起動、cl コマンドによるコンパイルとリンク、および主なコンパイラオプションを取り上げる。

C.3.1 必要となるツールの導入

MSVC のコマンドライン開発ツールは、Microsoft の Visual Studio 公式ダウンロードページから入手できる。

<https://visualstudio.microsoft.com/ja/downloads/>

このページに掲載されている Build Tools for Visual Studio を選択し、インストーラをダウンロードする²⁰¹。ページ内で該当項目が見つからない場合には、Microsoft のサイト内で「Build Tools for Visual Studio」を検索するとよい。

ダウンロードしたインストーラを起動すると、導入するワークロードや個別のコンポーネントを選択する画面が表示される。C/C++ のコマンドライン開発環境を導入する場合には、C++ によるデスクトップ開発 ワークロードを選択する。これにより、MSVC の C/C++ コンパイラ、リンカ、標準ライブラリ、ヘッダファイル、Windows SDK など、Windows 向け C/C++ プログラムの構築に必要な基本的なツールをインストールできる。

ワークロードの名称や選択画面はバージョンによって異なることがあるが、MSVC のコンパイラと Windows SDK が選択されていることを確認し、その後はインストーラの案内に従って作業を進める。

C.3.2 MSVC を使用するためのコマンド環境の起動

MSVC のコンパイラやリンカを使用するためには、それらの実行ファイル、ヘッダファイル、ライブラリなどの場所を、環境変数に設定する必要がある。これらを手作業で設定することも可能であるが、Build Tools for Visual Studio をインストールすると、必要な環境設定を自動的に行うための専用のコマンド環境が用意される。

Windows のスタートメニューまたは検索画面で、「Developer Command Prompt」または「開発者コマンドプロンプト」を検索すると、インストールされている Build Tools for Visual Studio に対応する起動項目が表示される。また、64 ビット Windows 向けのプログラムを構築する場合には、「x64 Native Tools Command Prompt」に相当する項目を検索して起動するとよい²⁰²。

このコマンド環境を起動すると、MSVC のツールを利用するために必要な環境変数が設定されたコマンドウィンドウが開く。本書では、x64 アーキテクチャの Windows 向けプログラムを構築することを基本とするため、以下では主に x64 Native Tools Command Prompt を使用する。

²⁰¹ 製品名に付けられる年号、ダウンロードページの構成、およびインストーラの画面構成は、Visual Studio の更新に伴って変更される場合がある。

²⁰² 起動項目の名称には、Visual Studio の製品名や年号が付けられることがある。また、表示される名称や配置は、Visual Studio および Windows のバージョンによって異なる。

コマンド環境が正しく設定されていることは、次のコマンドによって確認できる。

```
> cl
```

MSVC のコンパイラが利用可能な場合には、コンパイラのバージョン情報とコマンドの使用方法が表示される。この確認ではソースファイルを指定していないため、コンパイルは行われない。次に、MSVC の具体的な使用方法について解説する。

C.3.3 cl コマンド

MSVC を用いて C 言語および C++ のプログラムをコンパイルする際には、cl コマンドを使用する。GCC や Clang では、C 言語用と C++ 用に異なるコマンドが用意されているが、MSVC では、いずれの言語のソースファイルに対しても基本的に cl コマンドを使用する。ソースファイルの拡張子が .c である場合には C 言語として、.cpp である場合には C++ として処理される。なお、拡張子にかかわらず C 言語として処理する場合には /TC, C++ として処理する場合には /TP オプションを指定する。

例えば、C 言語のソースファイル sample.c をコンパイルする場合には、次のように実行する。

```
> cl sample.c
```

補足) ここでは、Windows のコマンド環境におけるプロンプトを簡略化して「>」で表している。

また、C++ のソースファイル sample.cpp をコンパイルする場合にも、同じ cl コマンドを使用する。

```
> cl sample.cpp
```

これらのコマンドを実行すると、ソースファイルがコンパイルされ、オブジェクトファイルが生成された後、リンカが自動的に起動されて実行形式ファイルが生成される。既定では、sample.c または sample.cpp に対して、オブジェクトファイル sample.obj と実行形式ファイル sample.exe が生成される。

cl コマンドの基本的な書式を次に示す。

cl [オプション] [ソースファイル] [オブジェクトファイル] [ライブラリ] [/link リンカオプション]

コマンド名に続けて、コンパイルするソースファイル、リンクするオブジェクトファイルやライブラリ、コンパイル方法を指定するオプションなどを引数として与える。MSVC のコンパイラオプションは、一般にスラッシュ / から始まり、大文字と小文字が区別される。また、多くのオプションでは、オプション名とそれに続く値との間に空白を入れずに記述する。

生成する実行形式ファイルの名前を指定する場合には、/Fe オプションを使用する。例えば、sample.cpp をコンパイルして、実行形式ファイル test.exe を生成する場合には、次のように実行する。

```
> cl sample.cpp /Fetest.exe
```

/Fe の値として拡張子を省略した場合には、通常、.exe が付加される。例えば、次のコマンドでも実行形式ファイル test.exe が生成される。

```
> cl sample.cpp /Fetest
```

複数のソースファイルから実行形式ファイルを生成する場合には、それらのファイル名を空白で区切って並べる。例えば、main.cpp, calc.cpp, draw.cpp から実行形式ファイル sample.exe を生成する場合には、次のように実行する。

```
> cl main.cpp calc.cpp draw.cpp /Fesample.exe
```

コンパイルだけを行い、リンクを行わずにオブジェクトファイルを生成する場合には、/c オプションを使用する。

```
> cl /c calc.cpp
```

この例では、オブジェクトファイル calc.obj が生成される。出力するオブジェクトファイルの名前を明示的に指定する場合には、/Fo オプションを使用する。

```
> cl /c calc.cpp /Focalc_data.obj
```

生成した複数のオブジェクトファイルをリンクして実行形式ファイルを生成する場合にも、cl コマンドを使用することができる。

```
> cl main.obj calc.obj draw.obj /Fesample.exe
```

cl コマンドは、通常、コンパイルの終了後に MSVC のリンカである link を自動的に起動する。リンカに直接オブ

ションを渡す場合には、コマンドの末尾に `/link` を記述し、その後にリンカ用のオプションを指定する。例えば、リンカの `/OUT` オプションを直接指定する場合には、次のように記述する。

```
> cl sample.cpp /link /OUT:test.exe
```

`/link` より後に記述された引数は、コンパイラではなくリンカへ渡される。`/link` とリンカオプションは、コマンド中の他のファイル名やコンパイラオプションよりも後に記述する必要がある。

C.3.4 主なコンパイラオプション

MSVC には、プログラミング言語の規格、警告の表示、最適化、デバッグ情報の生成、ライブラリの利用などを指定するための、多数のコンパイラオプションが用意されている。主なものを表 113 に示す。

表 113: MSVC の主なコンパイラオプション

オプション	解 説
<code>/Fefile</code>	生成する実行形式ファイルの名前を <i>file</i> にする。
<code>/Fofile</code>	生成するオブジェクトファイルの名前を <i>file</i> にする。
<code>/c</code>	コンパイルのみを行い、リンクを行わずにオブジェクトファイルを生成する。
<code>/E</code>	プリプロセッサによる処理だけを行い、その結果を標準出力へ出力する。
<code>/std:c17</code>	C 言語の規格として C17 を指定する。
<code>/std:c++17</code>	C++ の規格として C++17 を指定する。
<code>/permissive-</code>	一部の Microsoft 固有の非標準動作を抑え、C++ 標準への適合性を高める。
<code>/Zc:__cplusplus</code>	使用する C++ 規格に応じた値を、定義済みマクロ <code>__cplusplus</code> に設定する。(*)
<code>/W0~/W4</code>	コンパイル時に表示する警告の水準を指定する。数値が大きいほど多くの警告が有効になる。
<code>/WX</code>	警告をエラーとして扱い、警告が発生した場合にコンパイルを失敗させる。
<code>/Zi</code>	デバッガで使用するための情報を生成する。
<code>/Od</code>	コードの最適化を行わない。
<code>/O1</code>	生成するコードの大きさを抑えることを重視して最適化を行う。
<code>/O2</code>	実行速度を高めることを重視して最適化を行う。
<code>/I dir</code>	ヘッダファイルを検索するディレクトリに <i>dir</i> を追加する。
<code>/Dname</code>	プリプロセッサのマクロ <i>name</i> を値 1 で定義する。
<code>/Dname=value</code>	プリプロセッサのマクロ <i>name</i> を値 <i>value</i> で定義する。
<code>/EHsc</code>	標準 C++ の例外処理とスタックの巻き戻しを有効にする。また、 <code>extern "C"</code> として宣言された関数は例外を送出しないものとして扱う。
<code>/MD</code>	DLL 版のマルチスレッド対応ランタイムライブラリを使用する。
<code>/MT</code>	静的リンク版のマルチスレッド対応ランタイムライブラリを使用する。
<code>/openmp</code>	OpenMP 2.0 による並列処理を有効にする。
<code>/MP</code>	複数のソースファイルを複数のプロセスによって並列にコンパイルする。
<code>/utf-8</code>	ソース文字集合および実行文字集合として UTF-8 を指定する。
<code>/showIncludes</code>	コンパイル時に読み込まれたヘッダファイルの一覧を表示する。
<code>/nologo</code>	コンパイラの起動時に表示される製品名やバージョン情報を表示しない。
<code>/?</code>	主なコンパイラオプションの説明を表示する。

* `/Zc:__cplusplus` を指定しない場合、`__cplusplus` の値は既定では 199711L となる。`/permissive-` を指定しても有効にならないため、必要な場合は別途指定する。

例えば、C17 の規格を使用し、警告水準を 4 にして `sample.c` をコンパイルする場合には、次のように実行する。

```
> cl /std:c17 /W4 sample.c
```

C++17 の規格を使用し、標準への準拠を強め、例外処理を有効にして `sample.cpp` をコンパイルする場合には、次のように実行する。

```
> cl /std:c++17 /permissive- /W4 /EHsc sample.cpp
```

MSVC における主な最適化オプションは、`/Od`、`/O1`、`/O2` である。`/Od` は最適化を行わず、デバッグの際に使用しやすい。`/O1` は生成するコードの大きさを抑えることを重視し、`/O2` は実行速度を高めることを重視して最適化を行う。通常の実用的なプログラムでは、`/O2` を指定して性能を確認するとよい。

```
> cl /std:c++17 /O2 sample.cpp
```

多数のソースファイルを一度にコンパイルする場合には、`/MP` オプションを指定することで、複数のコンパイラプロセスを起動し、それぞれのソースファイルを並列にコンパイルすることができる。例えば、次のコマンドでは、指

定された複数のソースファイルが可能な範囲で並列にコンパイルされる。

```
> cl /MP /c main.cpp calc.cpp draw.cpp
```

C++標準ライブラリの `std::thread` を使用する場合、GCC や Clang における `-pthread` に相当する特別なコンパイラオプションは通常必要ない。一方、OpenMP による並列処理を使用する場合には、`/openmp` を指定する。

C.3.5 文字のエンコーディングの指定

MSVC では、ソースファイルを読み込む際のエンコーディングと、コンパイル後のプログラム内に通常の文字リテラルや文字列リテラルを格納する際のエンコーディングを、それぞれコンパイラオプションによって指定することができる。

■ ソースファイルのエンコーディングの指定

`/source-charset`: オプションを用いると、コンパイラがソースファイルを読み込む際のエンコーディングを指定することができる。例えば、ソースファイルが UTF-8 で記述されている場合には、次のように指定する。

```
/source-charset:utf-8
```

エンコーディングは、文字セットの名前または Windows のコードページ番号によって指定することができる。コードページ番号を指定する場合には、番号の前にピリオド `.` を付ける。例えば、日本語 Windows で広く使用されてきた CP932 を指定する場合には、次のように記述する。

```
/source-charset:.932
```

MSVC では、`/source-charset`: または `/utf-8` オプションを指定しない場合、ソースファイルの先頭に BOM (byte order mark) が付加されていれば、それに基づいて UTF-8 や UTF-16 などのエンコーディングが判定される。BOM もない場合には、現在のユーザーコードページ、すなわち Windows のユーザー環境で使用される既定のコードページに基づいてソースファイルが読み込まれる。そのため、BOM を持たない UTF-8 のソースファイルを実際に読み込むには、`/source-charset:utf-8` または `/utf-8` を明示的に指定するとよい。

■ 文字列リテラルのエンコーディングの指定

`/execution-charset`: オプションを用いると、通常の文字リテラルや文字列リテラルを、コンパイル後のプログラム内に格納する際のエンコーディングを指定することができる。例えば、文字列リテラルを CP932 のバイト列として格納する場合には、次のように指定する。

```
/execution-charset:.932
```

例えば、UTF-8 で記述されたソースファイルを読み込み、通常の文字列リテラルを CP932 に変換してプログラム内に格納する場合には、次のようにコンパイルする。

```
> cl /source-charset:utf-8 /execution-charset:.932 sample.c
```

例えば次のように、ソースファイル中に文字列リテラルが記述されているものとする。

```
const char *s = "日本語";
```

この場合、ソースファイル中の「日本語」は UTF-8 として読み取られ、コンパイル後のプログラム内では CP932 のバイト列として格納される。ただし、このオプションは、ファイルや端末に対する入出力の際にエンコーディングを自動的に変換するものではない。また、`char` 型の配列や C++ の `std::string` 型には、格納されたバイト列がどのエンコーディングによるものであるかを表す情報は保持されない。

■ ソースと文字列リテラルを UTF-8 にする指定

ソースファイルのエンコーディングと、通常の文字リテラルおよび文字列リテラルに用いる実行文字集合のエンコーディングの両方を UTF-8 にする場合には、`/utf-8` オプションを使用することができる。

```
> cl /utf-8 sample.c
```

この指定は、次の二つのオプションを同時に指定することと同じ意味を持つ。

```
/source-charset:utf-8 /execution-charset:utf-8
```

新しく作成するプログラムで、ソースファイルと、通常の文字リテラルおよび文字列リテラルとともに UTF-8 として

扱う場合には、`/utf-8` オプションを使用すると簡潔である。

なお、`/source-charset:`、`/execution-charset:`、`/utf-8` のいずれかを指定すると、ソースコード中の文字がソース文字セットおよび実行文字セットで正しく表現できるかを検査する `/validate-charset` オプションも、既定で有効になる。

代表的な文字エンコーディングについては、p.520 の表 111 を参照されたい。

C.3.6 実行時のスタックサイズの指定

関数内で宣言されたローカル変数や、関数呼び出しに伴って必要となる情報などは、主にスタック領域に格納される。深い再帰呼び出しを行う場合や、大きな配列をローカル変数として宣言する場合には、スタック領域が不足し、プログラムが異常終了することがある。MSVC では、生成する実行プログラムが使用するスタックサイズを、ビルド時のオプションによって指定することができる。

ただし、スタックサイズの情報を実行形式ファイルに設定する処理は、厳密にはコンパイラではなくリンカによって行われる。MSVC では、コンパイラオプションの `/F` を使用方法と、リンカオプションの `/STACK:` を使用方法がある。

■ `/F` オプションによる指定

`/F` オプションを用いると、生成する実行プログラムのスタックの予約サイズをバイト単位で指定することができる。例えば、スタックの予約サイズを 16MiB に設定するには、次のようにコンパイルする。

```
> cl /F16777216 sample.c
```

`/F` と数値の間には、空白を入れて次のように記述することもできる。

```
> cl /F 16777216 sample.c
```

ここで、16777216 バイトは 16MiB に相当する。数値は 10 進数のほか、`0x` で始まる C 言語形式の 16 進数によって指定することもできる。`/F` オプションが指定されていない場合、x86、x64 および ARM64 向けの実行プログラムでは、通常、スタックの予約サイズとして 1MiB が使用される。

■ リンカオプションによる指定

`/F` オプションによる指定は、リンカの `/STACK:` オプションによる指定に対応する。`cl` コマンドからリンカオプションを直接指定する場合には、`/link` の後に記述する。例えば、スタックの予約サイズを 16MiB に設定する場合には、次のように指定する。

```
> cl sample.c /link /STACK:16777216
```

`/STACK:` オプションの一般的な書式は、次のようになる。

```
/STACK:reserve[,commit]
```

`reserve` には、仮想アドレス空間上に予約するスタック領域全体のサイズを指定する。`commit` には、そのうち最初からコミットするサイズを指定する。どちらも単位はバイトであり、`commit` には `reserve` 以下の値を指定する。例えば、16MiB を予約し、最初に 64KiB をコミットする場合には、次のように指定する。

```
> cl sample.c /link /STACK:16777216,65536
```

通常は予約サイズだけを指定すればよく、コミットサイズを明示的に指定する必要は少ない。コミットサイズを大きくすると、スタック領域を拡張する際の処理回数を減らせる可能性がある一方、初期状態でコミットされるメモリ量が増える。

なお、`/STACK:` オプションは実行形式ファイルを生成する場合に使用されるものであり、DLL を生成する場合には適用されない。また、`/c` オプションによってコンパイルだけを行い、オブジェクトファイルを生成する場合にはリンカが起動されないため、スタックサイズは設定されない。

スタックサイズを大きくすると、深い再帰呼び出しや大きなローカル配列を使用できる範囲は広がる。しかし、スタックサイズの拡大は、無制限な再帰呼び出しや過大なローカル配列を安全にするものではない。大量のデータを扱う場合には、動的メモリ領域を使用するなど、プログラムの設計を見直すことも重要である。

C.3.7 開発ツールのアップデート

MSVC のコンパイラやリンカ、標準ライブラリ、Windows SDK などの開発ツールは、Visual Studio Installer によって管理される。Visual Studio の IDE を導入せず、Build Tools for Visual Studio のみを導入している場合にも、基本的には Visual Studio Installer を使用してアップデートを行う。なお、更新や構成の変更を行う際には、システムの管理者権限を求められる場合がある。

開発ツールをアップデートする場合には、Windows のスタートメニューから「Visual Studio Installer」を起動する。インストール済みの Visual Studio または Build Tools に利用可能なアップデートがある場合には、対象となる製品の欄に「更新」ボタンが表示される。このボタンをクリックすると、その製品にインストールされている開発ツールやコンポーネントが新しいバージョンに更新される。

アップデートの完了後に、Windows の再起動を求められる場合がある。この場合には、作業中のファイルを保存した上で、Windows を再起動する。

また、導入するツールや機能を確認、変更する場合には、Visual Studio Installer で対象となる製品の「変更」を選択する。表示される画面では、ワークロードおよび個別のコンポーネントを選択することができる。C/C++ のコマンドライン開発環境では、主に次のような項目が関係する。

- MSVC の C/C++ コンパイラおよびビルドツール
- Windows SDK
- CMake などの C++ 向けビルド支援ツール

通常のアップデートでは、現在導入されているコンポーネントが更新される。新しい Windows SDK や、特定のバージョンの MSVC ツールセットなどを追加する場合には、「個々のコンポーネント」の画面で必要な項目を選択してから、変更内容を適用する。

アップデート後には、Developer Command Prompt などの MSVC 用コマンド環境を新たに起動し、次のコマンドによってコンパイラのバージョンを確認するとよい。

```
> cl
```

cl コマンドを引数なしで実行すると、コンパイラのバージョンや対象となるアーキテクチャなどが表示される。アップデート前から開いていたコマンドプロンプトでは、古い環境設定が残っている可能性があるため、アップデート後にはコマンド環境をいったん閉じて、新たに起動する方がよい。

なお、cl コマンド自体には、コンパイラをアップデートするための機能はない。MSVC の開発ツールは、Visual Studio Installer によってインストールおよび更新する。多数の計算機を管理する場合や処理を自動化する場合には、コマンドラインから Visual Studio Installer を操作する方法も用意されているが、通常の個人利用では、Visual Studio Installer の画面から更新する方法が分かりやすい。

C.4 MSYS2 による Windows 開発環境

Windows 上に UNIX 系の開発環境を実現する代表的なソフトウェアに Cygwin がある。Cygwin の開発は、1995 年に米国の Cygnus Solutions で始められた。Cygwin は POSIX²⁰³ 互換を実現するためのランタイムライブラリ (cygwin1.dll) の利用によって多くの GNU ソフトウェアを Windows 向けに移植し、実行することを可能にした。しかし、Cygwin 上で構築されたソフトウェアは、通常、POSIX 互換機能を提供する cygwin1.dll に依存する。このため、実行環境への Cygwin ランタイムの導入が必要になるなど、Windows の標準的な実行環境だけで動作するネイティブアプリケーションとは異なる性質を持つ。

Cygwin は、Windows 上に POSIX の API 層を実現するものであるが、これに対して、Windows ネイティブの実行形式ファイルを生成する開発環境である MinGW (Minimalist GNU for Windows)²⁰⁴ が開発された。MinGW を用いて構築されたプログラムは、Cygwin の POSIX 互換ランタイムには依存せず、Windows の API や実行時ライブラリを利用する Windows ネイティブのプログラムとして動作する。

初期の MinGW は、Windows ネイティブのプログラムを生成する GNU ツールチェーンを提供したが、GNU ソフトウェアの構築に使用する UNIX 系のシェルやビルドツールは十分に整備されていなかった。そこで MSYS (Minimal SYStem) が開発された。MSYS は、軽量の POSIX 互換レイヤと、シェルやアプリケーションの構築に必要なツール群を提供し、MinGW との連携を可能にするものであった。

64 ビット環境の普及とともに、従来の MinGW とは別に、64 ビット Windows にも対応する mingw-w64 が開発された。また、MSYS を現代的な Cygwin と mingw-w64 を基礎として再実装した MSYS2 が開発された。MSYS2 は、Windows 上で UNIX 系のコマンド環境を利用しながら、GCC や Clang / LLVM などのツールチェーンによって、Windows ネイティブのソフトウェアを構築するための開発基盤である。

Windows 上で Linux のソフトウェア開発環境を利用する方法としては、WSL2 が広く用いられている。WSL2 では、軽量の仮想化環境の内部で実際の Linux カーネルと Linux ディストリビューションを動作させることができる。ただし、WSL2 の Linux 環境で通常生成されるソフトウェアは Linux 用の ELF 形式であり、WSL2 環境の外で Windows ネイティブのプログラムとして直接実行することはできない。これに対して、MSYS2 の UCRT64 や CLANG64 などの環境で構築されたプログラムは、Windows 用の実行形式ファイルとなり、Windows 上で直接実行できる。

C.4.1 MSYS2 のインストール

MSYS2 のインストールパッケージは公式 Web サイト <https://www.msys2.org/> から入手できる²⁰⁵。

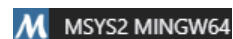
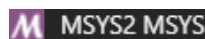
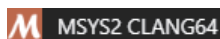
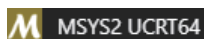


図 12: MSYS2 インストーラのアイコン

このインストーラを起動すると作業用のダイアログが開くので、基本的にはその案内に沿って作業する。インストーラ先には、短く、空白や日本語文字を含まないフォルダを指定することが推奨されており、通常は C:\msys64 を指定する。そのほかの設定項目についても、特別な理由がなければインストーラの推奨設定に従うとよい。

C.4.2 MSYS2 の起動

MSYS2 をインストールすると、スタートメニューに複数の起動項目 (下記) が登録される。



それぞれの起動項目は、使用するコンパイラ、実行時ライブラリ、ライブラリの配置場所などが異なる開発環境に対応している。いずれかの起動項目を選択すると、その開発環境に対応するパスや環境変数が設定されたターミナルウィンドウが開く。主な環境を表 114 に示す。

²⁰³IEEE が標準化した、OS とプログラムの間のインタフェースやコマンド環境などに関する標準規格。

²⁰⁴1990 年代後半に始められた、コミュニティ主導のオープンソースプロジェクトである。

²⁰⁵本書では、x86_64 版を前提とする。

表 114: MSYS2 で利用できる主な開発環境

環境	解 説
UCRT64	GCC, UCRT, libstdc++を使用する 64 ビット Windows 向け開発環境.
CLANG64	Clang / LLVM, UCRT, libc++を使用する 64 ビット Windows 向け開発環境.
MSYS	MSYS2 自体の管理や UNIX 系ツールの実行に使用する POSIX 互換環境.
MINGW64	GCC と旧来の MSVCRT を使用する 64 ビット Windows 向け開発環境. 現在は非推奨化が進められている.

本書では、GCC を利用する場合には主に UCRT64 環境を、Clang / LLVM を利用する場合には CLANG64 環境を使用する。特に理由がない場合には、現在の Visual Studio でも既定で使用されている UCRT を採用した UCRT64 環境を選択するとよい。MSYS 環境は、主として MSYS2 の管理や UNIX 系ツールの利用に用いるものであり、Windows ネイティブの C/C++ プログラムを構築するための中心的な環境ではない。

C.4.3 開発ツールのインストール

MSYS2 のインストール直後は、コンパイラをはじめとする開発ツールはインストールされていない。ここでは、UCRT64 環境および CLANG64 環境で使用する基本的な開発ツールのインストール方法を説明する。

パッケージの依存関係の不整合を避けるため、開発ツールをインストールする前に、既にインストールされているパッケージを、次のコマンドによって最新の状態にしておくことが強く推奨される。

```
$ pacman -Syu
```

基本的なシステムパッケージが更新される場合には、MSYS2 のターミナルを終了するように求められることがある。その場合は、すべての MSYS2 のターミナルを閉じて再度起動し、更新対象がなくなるまで同じコマンドを実行する。

MSYS2 では、パッケージのインストールに `pacman` コマンドを使用する。パッケージをインストールするための基本的な書式を次に示す。

```
pacman -S --needed [パッケージ名]
```

`-S` は、リポジトリから指定したパッケージをインストールするための指定である。この際、指定したパッケージの動作に必要な依存パッケージも自動的にインストールされる。また、`--needed` を指定すると、既にインストールされているパッケージの再インストールを行わない。

MSYS2 のパッケージを作成する場合や、Autotools などの UNIX 系のビルドツールを使用してソフトウェアを構築する場合には、必要に応じて次のパッケージグループもインストールする。

```
$ pacman -S --needed base-devel
```

`base-devel` は、ソフトウェアの構築や MSYS2 のパッケージ作成に使用する基本的なツール群をまとめたパッケージグループである。通常の C/C++ プログラムのコンパイルに必須ではないが、さまざまなソフトウェアをソースコードから構築する場合に役立つ。使用する開発環境に応じた主なパッケージを表 115 に示す。

表 115: 基本的な開発ツールのパッケージ

ツール	UCRT64 環境	CLANG64 環境
コンパイラと周辺ツール	mingw-w64-ucrt-x86_64-toolchain	mingw-w64-clang-x86_64-toolchain
CMake	mingw-w64-ucrt-x86_64-cmake	mingw-w64-clang-x86_64-cmake
Ninja	mingw-w64-ucrt-x86_64-ninja	mingw-w64-clang-x86_64-ninja

注) Ninja は、CMake の依存パッケージとして自動的にインストールされる場合がある。

UCRT64 環境のツールチェーンには、GCC, GDB, binutils, ヘッダファイル, 基本的なライブラリなどが含まれる。これをインストールする場合には、UCRT64 環境のターミナルで次のように実行する。

```
$ pacman -S --needed mingw-w64-ucrt-x86_64-toolchain
```

CLANG64 環境のツールチェーンには、Clang / LLVM, LLD, LLDB, libc++, OpenMP ランタイムなどが含まれる。これをインストールする場合には、CLANG64 環境のターミナルで次のように実行する。


```
$ pacman -S --needed mingw-w64-clang-x86_64-toolchain
```

CMake および Ninja を使用する場合にも、表 115 に示したパッケージ名を同じ書式で指定する。例えば、UCRT64 環境に CMake をインストールする場合には、次のように実行する。

```
$ pacman -S --needed mingw-w64-ucrt-x86_64-cmake
```

CLANG64 環境では、表 115 に示した CLANG64 用のパッケージ名を同様に指定する。

パッケージグループをインストールする際に、インストールするパッケージの選択を求められた場合には、特別な理由がなければ、すべてのパッケージを選択する既定の設定を使用するとよい。

C.4.4 パッケージの管理

MSYS2 では、パッケージの管理に `pacman` を用いる。これは元来 Arch Linux 用に開発されたパッケージ管理ツールであり、MSYS2 では Windows 環境に対応した移植版が使用されている。`pacman` を用いることで、パッケージの検索、インストール、更新、削除などを行うことができる。

重要なコマンドを表 116～117 に示す。

表 116: `pacman` の基本コマンド：リポジトリを利用するもの

コマンド	解 説
<code>pacman -Syu</code>	パッケージデータベースを同期し、インストール済みのパッケージ全体を最新の状態に更新する。
<code>pacman -S パッケージ名</code>	指定したパッケージをインストールする。必要な依存パッケージも併せてインストールする。
<code>pacman -S --needed パッケージ名</code>	上と同じだが、既に最新のものがインストールされている場合は、そのパッケージを再インストールしない。
<code>pacman -Ss キーワード</code>	リポジトリに登録されたパッケージから、名前や説明がキーワードに該当するものを検索する。
<code>pacman -Si パッケージ名</code>	リポジトリに登録されたパッケージの詳細情報を表示する。

表 117: `pacman` の基本コマンド：インストール済みパッケージに関するもの

コマンド	解 説
<code>pacman -Q</code>	インストール済みのすべてのパッケージの一覧を表示する。
<code>pacman -Qe</code>	利用者が明示的にインストールしたパッケージを表示する。依存関係によって自動的に導入されたパッケージは含まれない。
<code>pacman -Qs キーワード</code>	インストール済みのパッケージから、名前や説明がキーワードに該当するものを検索する。
<code>pacman -Qi パッケージ名</code>	インストール済みのパッケージの詳細情報を表示する。
<code>pacman -R パッケージ名</code>	指定したパッケージを削除する。他のパッケージに必要とされている場合は、通常、削除されない。不要になった依存パッケージは自動的に削除しない。
<code>pacman -Rs パッケージ名</code>	指定したパッケージと、依存パッケージとして導入され、その削除によって他のパッケージから必要とされなくなるものを削除する。

D ソフトウェア構築の支援ツール

複数のソースファイルからなるプログラムを作成する場合には、各ソースファイルを個別にコンパイルしてオブジェクトファイルを生成し、それらをリンクして実行形式ファイルを作成する。このように、ソースファイルなどから、オブジェクトファイル、ライブラリ、実行形式ファイルなどを生成する一連の処理を、プログラムの**構築**、または**ビルド**という。

小規模なプログラムであれば、コンパイラのコマンドをその都度入力して構築することも可能である。しかし、ソースファイルの数が増えると、コンパイルやリンクの順序、使用するコンパイラオプション、リンクするライブラリなどを正しく管理することが次第に難しくなる。また、一部のソースファイルだけを変更した場合に、変更とは関係のないファイルまで毎回コンパイルすることは、構築作業として効率がよくない。

例えば、あるソースファイルを変更した場合には、通常はそのファイルから生成されるオブジェクトファイルだけを再コンパイルし、その後、オブジェクトファイルをリンクし直せばよい。これに対して、ヘッダファイルを変更した場合には、そのヘッダファイルを直接または間接に読み込んでいる複数のソースファイルを再コンパイルする必要がある。このように、あるファイルの変更が、どのファイルの再生成に影響するかという関係を、ファイルの**依存関係**という。

複数のファイルからなるプログラムを効率よく構築するためには、これらの依存関係を管理し、変更の影響を受けるファイルだけを必要な順序で再生成する仕組みが必要となる。また、長いコンパイルコマンドを毎回入力するのではなく、構築に必要な手順をあらかじめ記述し、同じ処理を容易に再現できることも重要である。

このような構築作業を支援するために、様々なツールが利用されている。**make** は、ファイル間の依存関係と実行すべきコマンドを記述したファイルに基づいて、必要な処理を自動的に実行するツールである。**CMake** は、使用する処理系や開発環境に応じて、**Makefile** や **Ninja** 用のファイルなど、構築処理に必要なファイルを生成するためのシステムである。また、**Ninja** は、特に高速な構築処理を目的として設計されたビルドツールであり、**CMake** と組み合わせて使用されることも多い。

ここでは、これらのツールの基本的な役割と使用方法について、例を挙げて説明する。ただし、詳細な仕様と使用方法については、各ツールの開発元が公開している情報を参照のこと。

■ サンプルケース

ここで扱うプログラムは、二次元平面上の2点を結ぶ線分について、その長さと x 軸に対する角度を求めて表示するものである。プログラムは、二次元座標を扱う `point.h`、`point.c`、線分を扱う `line.h`、`line.c`、および `main` 関数を含む `geo_main.c` から構成される。また、これらのファイルは1つの**プロジェクト**として、`geo_sample` というディレクトリに配置されている²⁰⁶ とする。

次に示す `point.h` では、二次元座標を表す構造体 `Point` を定義している。`Point` は、 x 座標と y 座標を表す2つの `double` 型のメンバを持つ。また、2つの `Point` 型の値を受け取り、それらの間の距離を返す `point_distance` 関数を宣言している。このヘッダファイルは、座標を扱う他のソースファイルやヘッダファイルから共通して利用される。

ヘッダファイル：`geo_sample/point.h`

```
1  #ifndef      POINT_H
2  #define      POINT_H
3
4  typedef struct {    // 二次元座標を表す構造体
5      double x;
6      double y;
7  } Point;
8
9  // 2点間の距離を求める関数（プロトタイプ宣言）
10 double point_distance(Point p1, Point p2);
11
12 #endif
```

次に示す `point.c` では、`point_distance` 関数を定義している。この関数は、2点の x 座標の差と y 座標の差を求め、それらを用いて2点間の距離を計算する。平方根を求める `sqrt` 関数を使用するために `math.h` を読み込み、`Point`

²⁰⁶一連のファイル群をプロジェクトとして1つのディレクトリ配下に収めるのは、一般的な管理手法である。

型と関数宣言を参照するために point.h を読み込んでいる.

プログラム：geo_sample/point.c

```
1 #include <math.h>
2 #include "point.h"
3
4 // 2点間の距離を求める関数（実装）
5 double point_distance(Point p1, Point p2) {
6     double dx = p2.x - p1.x;
7     double dy = p2.y - p1.y;
8     return sqrt(dx * dx + dy * dy);
9 }
```

次に示す line.h では、線分を表す構造体 Line を定義している。Line は、線分の始点と終点を表す 2 つの Point 型のメンバ begin, end を持つ。従って、Point 型の定義を利用するために、point.h を読み込む必要がある。また、線分の長さを求める line_length 関数と、線分の角度を求める line_angle 関数を宣言している。このように、line.h は point.h に依存している。

ヘッダファイル：geo_sample/line.h

```
1 #ifndef LINE_H
2 #define LINE_H
3
4 #include "point.h"
5
6 typedef struct { // 線分を表す構造体
7     Point begin; // 始点
8     Point end;   // 終点
9 } Line;
10
11 double line_length(Line line); // 線分の長さを求める関数（プロトタイプ宣言）
12 double line_angle(Line line); // 線分の角度を求める関数（プロトタイプ宣言）
13
14 #endif
```

次に示す line.c では、line_length 関数と line_angle 関数を定義している。line_length 関数は、線分の始点と終点を point_distance 関数に渡して、その距離を線分の長さとして返す。従って、実行形式ファイルをリンクする際には、line.o が参照する point_distance 関数の定義を含む point.o も必要となる。

一方、現在のコードでは、line_angle 関数は開発途中の状態であるため、ここでは角度を計算せず、暫定的に 0.0 を返す形にしている。仮引数 line を処理に使用しないため、(void)line; を記述し、未使用の仮引数に関するコンパイラの警告を抑制している。後でこの関数の処理を完成させ、line.c だけを変更した場合に、make がどのファイルを再コンパイルするかを確認する。

プログラム：geo_sample/line.c

```
1 #include "line.h"
2
3 double line_length(Line line) { // 線分の長さを求める関数（実装）
4     return point_distance(line.begin, line.end);
5 }
6
7 double line_angle(Line line) { // 線分の角度を求める関数（開発中につき、未完成）
8     (void)line;
9     return 0.0; // 暫定の戻り値
10 }
```

次に示す geo_main.c には、プログラムの実行開始位置となる main 関数を記述している。始点を (1.0, 2.0)、終点を (4.0, 6.0) とする Line 型の変数 line を生成し、その始点、終点、長さ、角度を順に表示する。Line 型と、線分に対する関数を利用するために line.h を読み込んでいる。また、line.h が point.h を読み込むため、geo_main.c は point.h の内容も間接的に取り込む。

プログラム：geo_sample/geo_main.c

```
1 #include <stdio.h>
```

```

2  #include    "line.h"
3
4  int main( ) {
5      Line line = { {1.0, 2.0}, {4.0, 6.0} };      // 線の構造体
6      printf("begin   : (%.1f, %.1f)\n", line.begin.x, line.begin.y);
7      printf("end     : (%.1f, %.1f)\n", line.end.x, line.end.y);
8      printf("length  : %.2f\n", line_length(line));
9      printf("angle   : %.4f rad\n", line_angle(line));
10 }

```

このプログラムでは、2点間の距離は5となるため、`line_length` 関数による計算結果として 5.00 が表示される。一方、`line_angle` 関数はまだ完成していないため、角度として 0.0000 rad が表示される。

ちなみに、構築支援ツールを使用せずに、上記プログラム全体をコンパイル、リンクして実行する例を次に示す。(geo_sample ディレクトリでの作業)

例. 構築支援ツールを使用せずにプログラム全体をコンパイル、リンクして実行する

```

$ gcc -c point.c      Enter      ←ソースプログラムを
$ gcc -c line.c       Enter      ←それぞれコンパイルして
$ gcc -c geo_main.c   Enter      ←オブジェクトファイルを作り
$ gcc point.o line.o geo_main.o -lm -o geo_sample Enter  ←それらをリンクする
$ ./geo_sample        Enter      ←実行
begin   : (1.0, 2.0)      ←出力
end     : (4.0, 6.0)
length  : 5.00
angle   : 0.0000 rad      ←角度は未実装

```

最初の3つのコマンドでは、各ソースファイルを個別にコンパイルし、`point.o`、`line.o`、`geo_main.o` を生成している。オプション `-c` を指定した場合は、リンクを行わず、オブジェクトファイルだけを生成する。

4つ目のコマンドでは、生成した3つのオブジェクトファイルをリンクし、実行形式ファイル `geo_sample` を生成している。`point.c` では `sqrt` 関数を使用しているため、リンク時には数学ライブラリを指定するオプション `-lm` を付けている²⁰⁷。最後のコマンドで、生成した実行形式ファイルを実行している。

次に、上のソースファイル群を用いて、構築支援ツールについて解説する。

D.1 make

`make` は、複数のファイルからなるプログラムの構築作業を自動化するために、UNIX 系 OS で伝統的に用いられてきたツールである。プログラムを構成するファイル間の依存関係と、それぞれのファイルを生成するためのコマンドを記述しておくことで、必要となるコンパイルやリンクなどの処理を適切な順序で自動的に実行することができる。

近年では、異なる OS や処理系に対応した構築用のファイルを生成する CMake や、高速な構築処理を目的とする Ninja などが広く利用されている。そのため、特に大規模なソフトウェア開発では、開発者が `make` のための構築手順を直接記述する機会は以前より少なくなりつつある。一方で、CMake が生成する構築用のファイルの一つとして Makefile²⁰⁸ が利用されることもあり、UNIX 系 OS 向けの既存のソフトウェアでは、現在でも `make` を用いて構築するものが多い。

また、`make` が扱うターゲット、依存ファイル、更新日時に基づく再構築などの考え方は、他の構築支援ツールを理解する上でも基礎となる。小規模なプログラムの構築を自動化する場合には、`make` だけで十分なことも多い。従って、現在のソフトウェア開発においても、その基本的な仕組みと使用方法を学んでおくことには重要な意味がある。

現在広く使用されている実装としては、GNU プロジェクトが開発・保守する GNU make がある。また、`make` の基本的な仕様は POSIX に規定されており、BSD 系 OS などでは、それぞれ独自の `make` 実装も維持されている。

D.1.1 基本的な使用方法

`make` を用いてプログラムを構築する場合には、通常、プロジェクトの起点となるディレクトリに、Makefile という

²⁰⁷ 環境によっては `-lm` オプションを必要としない場合もある。

²⁰⁸ `make` が使用する設定ファイル。プログラムを構成するファイル間の依存関係や、構築の際に実行するコマンドなどを記述したものである。

名前のファイルを用意する。このファイルには、生成するファイル、その生成に必要なファイル、および生成の際に実行するコマンドなどを記述する。make コマンドを実行すると、make はカレントディレクトリにある Makefile を読み込み、その内容に従って必要な処理を実行する。

補足) 1つのソフトウェアを構成するファイル群（ソース、ヘッダ、Makefile など）は、1つのディレクトリ以下にまとめ、そのディレクトリを**プロジェクト**²⁰⁹の起点として扱うことが一般的である。

Makefile では、基本的に次の形式でファイル間の依存関係と、ターゲットを生成するためのコマンドを記述する。

ターゲット：依存ファイル 1 依存ファイル 2 …
(←タブ→) **ターゲットを生成するためのコマンド**

左辺のターゲットは、通常、生成しようとするファイルの名前である。ただし、後述する clean のように、実行する処理に付けた名前をターゲットとすることもできる。コロン：の右側には、そのターゲットの生成や実行に必要な項目を列挙する。これらを**依存項目**、または**前提条件**といい、ファイルの場合は**依存ファイル**ともいう。次の行には、ターゲットを生成するために実行するコマンドを記述する。このコマンドを記述した部分を**レシピ**という。

レシピを記述する行の先頭には、空白文字ではなくタブ文字を置く必要がある。複数個の空白文字でタブ文字を代用することはできない。

注意) テキストエディタによっては、Tab キーを押した際にタブ文字ではなく複数個の空白文字が挿入されることがあるため注意が必要である。

先に示したプログラムを構築するための Makefile の例を次に示す。このファイルも、point.c、point.h、line.c、line.h、geo_main.c と同じディレクトリ geo_sample に保存する。

ファイル：geo_sample/Makefile

```
1 CC = gcc
2 CFLAGS = -std=c17 -Wall -Wextra
3 LDLIBS = -lm
4
5 TARGET = geo_sample
6 OBJS = point.o line.o geo_main.o
7
8 $(TARGET): $(OBJS)
9     $(CC) $(OBJS) $(LDLIBS) -o $(TARGET)
10
11 point.o: point.c point.h
12     $(CC) $(CFLAGS) -c point.c
13
14 line.o: line.c line.h point.h
15     $(CC) $(CFLAGS) -c line.c
16
17 geo_main.o: geo_main.c line.h point.h
18     $(CC) $(CFLAGS) -c geo_main.c
19
20 clean:
21     rm -f $(TARGET) $(OBJS)
```

Makefile では、イコール = を用いて変数に文字列を設定することができる。この例では、変数 CC に使用するコンパイラのコマンド gcc を設定し、変数 CFLAGS にはコンパイル時に指定するオプションをまとめて設定している。-std=c17 は使用する C 言語規格を C17 に指定するオプションであり、-Wall と -Wextra は、コンパイラによる警告を有効にするためのオプションである。

変数 LDLIBS には、リンクするライブラリに関する指定を設定している。このプログラムでは sqrt 関数を使用するため、数学ライブラリをリンクする -lm を指定している。また、変数 TARGET には生成する実行形式ファイルの名前を、変数 OBJS にはリンクするオブジェクトファイルの一覧を設定している。変数に設定した値は、\$(**変数名**) の形式で参照する。例えば、\$(CC) は gcc に、\$(OBJS) は point.o line.o geo_main.o に置き換えられる。

コンパイラのコマンドやコンパイルオプションをこのように変数としてまとめておくと、それらを変更する場合に、Makefile 内の複数の箇所を個別に書き換える必要がない。例えば、最適化のためのオプションを追加する場合には、変

²⁰⁹実際に、ソフトウェア開発に関するファイル群のセットをプロジェクトと呼ぶことも一般的である。

数 CFLAGS の定義だけを変更すればよい。このように、構築に必要な設定を一箇所にまとめて管理できることも、構築支援ツールを利用する利点の一つである。

次の規則は、実行形式ファイル `geo_sample` を生成するための依存関係とレシピを表している。

```
$(TARGET): $(OBJS)
$(CC) $(OBJS) $(LDLIBS) -o $(TARGET)
```

変数を実際の値に置き換えると、この規則は次の内容を表す。

```
geo_sample: point.o line.o geo_main.o
gcc point.o line.o geo_main.o -lm -o geo_sample
```

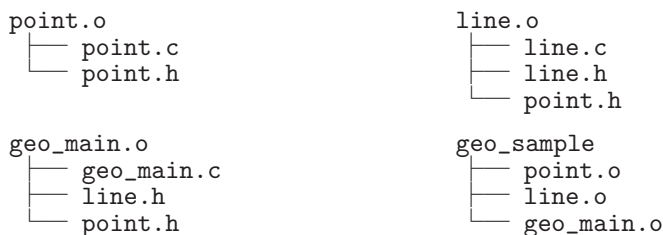
すなわち、実行形式ファイル `geo_sample` は、`point.o`、`line.o`、`geo_main.o` に依存している。これらのオブジェクトファイルがそろった後、レシピに記述されたコマンドによってリンク処理が行われる。

続く 3 つの規則は、各オブジェクトファイルを生成するための依存関係とレシピを表している。例えば、`line.o` に関する規則は次のように記述されている。

```
line.o: line.c line.h point.h
$(CC) $(CFLAGS) -c line.c
```

この規則では、`line.o` が `line.c`、`line.h`、`point.h` に依存することを表している。`line.c` は `line.h` を直接読み込み、`line.h` は `point.h` を読み込むため、`line.o` は `point.h` にも間接的に依存する。これらの依存ファイルのいずれかが変更された場合には、`line.c` が再コンパイルされ、`line.o` が再生成される。

同様に、`geo_main.c` は `line.h` を読み込み、`line.h` が `point.h` を読み込むため、`geo_main.o` は `line.h` だけでなく `point.h` にも依存する。従って、この例における主な依存関係は次のように整理できる。



`make` は、通常、ソースファイルの内容や以前の構築結果を専用の管理ファイルに保存して新旧の状態を比較しているわけではない。ファイルシステムに記録されているターゲットと依存項目の更新日時を比較し、再構築が必要かどうかを判定する。

例えば、`line.c`、`line.h`、`point.h` のいずれかが `line.o` より新しい場合、`line.o` は古い状態にあると判断され、`line.c` が再コンパイルされる。また、再生成された `line.o` が実行形式ファイル `geo_sample` より新しくなると、`geo_sample` を生成するためのリンク処理も再度実行される。このように、前回の構築によって生成されたオブジェクトファイルや実行形式ファイルの実物と、それらの更新日時が、前回の構築状態を示す情報として利用される。

Makefile の最後に記述した `clean` は、構築によって生成されたファイルを削除するためのターゲットである。`clean` という名前のファイルを生成するものではなく、実行形式ファイルとオブジェクトファイルを削除する処理に名前を与えたものである。このようなターゲットについては後で改めて説明する。

初めてプログラムを構築する場合には、まだオブジェクトファイルや実行形式ファイルが存在しない。この状態で、`geo_sample` ディレクトリに移動し、次のように `make` コマンドを実行する。

例. make による最初の構築

```
$ make Enter ←構築作業の開始
gcc -std=c17 -Wall -Wextra -c point.c          ←以下、構築作業の様子が表示される
gcc -std=c17 -Wall -Wextra -c line.c
gcc -std=c17 -Wall -Wextra -c geo_main.c
gcc point.o line.o geo_main.o -lm -o geo_sample
```

`make` は、Makefile に記述された最初の規則のターゲット `geo_sample` を構築の対象として選ぶ。`geo_sample` の生成には 3 つのオブジェクトファイルが必要であるが、初回の構築時にはいずれも存在しない。そのため、`make` は各オブジェクトファイルを生成するためのレシピを順に実行し、最後にそれらをリンクして `geo_sample` を生成する。

構築が完了した後、プログラムを実行すると、次の結果が得られる。

例. できあがったプログラム `geo_sample` の実行

```
$ ./geo_sample  ←プログラムの実行
begin : (1.0, 2.0)
end   : (4.0, 6.0)
length : 5.00
angle  : 0.0000 rad
```

この時点では、`line_angle` 関数はまだ完成していないため、角度として `0.0000 rad` が表示される。

D.1.2 ソースファイルの変更に対する `make` の働き

次に、未完成であった `line_angle` 関数を実装し、プログラムの一部だけを変更した場合に、`make` がどのような処理を行うかを確認する。線分の始点を (x_1, y_1) 、終点を (x_2, y_2) とすると、線分の x 方向と y 方向の変位は、それぞれ次のように求められる。

$$dx = x_2 - x_1, \quad dy = y_2 - y_1$$

線分の始点から終点へ向かう方向が、 x 軸の正の向きに対してなす角度は、`math.h` で宣言されている `atan2` 関数を用いて求めることができる。`atan2(dy,dx)` は、 y 方向の変位 dy と x 方向の変位 dx から、線分の方角をラジアン単位の角度として返す。

`line.c` を次のように変更する。

変更後のプログラム：`geo_sample/line.c`

```
1  #include <math.h>
2  #include "line.h"
3
4  double line_length(Line line) { // 線分の長さを求める関数（実装）
5      return point_distance(line.begin, line.end);
6  }
7
8  double line_angle(Line line) { // 線分の方角を求める関数（実装完了）
9      double dx = line.end.x - line.begin.x;
10     double dy = line.end.y - line.begin.y;
11     return atan2(dy, dx);
12 }
```

変更前の `line.c` では、`line_angle` 関数は暫定的に `0.0` を返していた。変更後のコードでは、始点から終点までの x 方向と y 方向の変位をそれぞれ dx , dy に求め、`atan2(dy,dx)` によって線分の方角を求めている。また、`atan2` 関数を使用するため、`line.c` の先頭で `math.h` を読み込んでいる。

`line.c` を保存した後、再び `make` を実行する。

例. `line.c` の変更後に再構築する

```
$ make 
gcc -std=c17 -Wall -Wextra -c line.c
gcc point.o line.o geo_main.o -lm -o geo_sample
```

このとき、`point.c` と `geo_main.c` をコンパイルするコマンドは実行されない。Makefile では、`line.o` の依存ファイルとして、次の3つを指定している。

```
line.o: line.c line.h point.h
```

`line.c` を変更して保存すると、`line.c` の更新日時は、前回の構築によって生成された `line.o` の更新日時より新しくなる。そのため、`make` は `line.o` が古い状態にあると判断し、`line.c` を再コンパイルして `line.o` を再生成する。

一方、`point.o` の依存ファイルである `point.c` と `point.h`、および `geo_main.o` の依存ファイルである `geo_main.c`、`line.h`、`point.h` には変更がない。これらの依存ファイルは、それぞれのオブジェクトファイルより新しくなっていないため、`point.o` と `geo_main.o` は再生成されない。

`line.o` が再生成されると、その更新日時は実行形式ファイル `geo_sample` より新しくなる。`geo_sample` は `line.o` に依存しているため、`make` は `point.o`、`line.o`、`geo_main.o` を再度リンクし、`geo_sample` を再生成する。すなわち、今回の

再構築で実行される処理は、line.c のコンパイルと、実行形式ファイルのリンクだけである。

再構築されたプログラムを実行すると、次の結果が得られる。

```
$ ./geo_sample 
begin : (1.0, 2.0)
end   : (4.0, 6.0)
length : 5.00
angle  : 0.9273 rad
```

この線分では、始点から終点までの変位は $dx = 3.0$, $dy = 4.0$ であるため、 $\text{atan2}(4.0, 3.0)$ の値として、約 0.9273 rad が得られる。このように、make は各ファイルの更新日時と依存関係を調べ、変更の影響を受ける部分だけを再生成する。ソースファイルの数が多いプログラムでは、変更と無関係なファイルのコンパイルを省略できるため、構築に必要な時間を短縮することができる。

なお、何も変更せずに、さらに make を実行すると、次のように表示される。

```
$ make 
make: 'geo_sample' is up to date.
```

これは、構築対象となるすべてのターゲットが存在し、いずれもその依存ファイルより古くないため、再生成する必要がないことを表している。この場合、コンパイルやリンクのコマンドは実行されない。

D.1.3 擬似ターゲット

ここまでに扱った geo_sample, point.o, line.o, geo_main.o などのターゲットは、いずれも構築によって実際に生成されるファイルである。一方、Makefile には、特定のファイルを生成することを目的とせず、何らかの処理を実行するための名前としてターゲットを定義することもできる。

先に示した Makefile では、構築によって生成されたファイルを削除するために、次の clean ターゲットを記述している。

```
clean:
rm -f $(TARGET) $(OBJS)
```

clean ターゲットには依存ファイルがなく、レシピとして rm コマンドが記述されている。変数を実際の値に置き換えると、実行されるコマンドは次のようになる。

```
rm -f geo_sample point.o line.o geo_main.o
```

このコマンドは、実行形式ファイル geo_sample と、3つのオブジェクトファイルを削除する。rm コマンドに指定した -f オプションは、指定したファイルが存在しない場合にもエラーを表示せずに処理を続けるためのものである。

このような、ファイルの生成を目的とせず、処理に名前を付けるために使用するターゲットを、一般に**擬似ターゲット**という。clean はファイル名ではなく、生成物を削除する処理を指定するための名前である。

ただし、カレントディレクトリに clean という名前のファイルが存在すると、make はそのファイルをターゲットの実体として扱う。依存ファイルのない clean ターゲットでは、clean というファイルが存在すると、ターゲットは既に最新の状態にあると判断され、通常、レシピは実行されない。

そこで、ファイルを生成するためのターゲットではないことを明示するために、.PHONY という特別なターゲットを使用する。Makefile の clean に関する部分を、次のように記述する。

```
.PHONY: clean
clean:
rm -f $(TARGET) $(OBJS)
```

.PHONY の前提条件として clean を指定すると、make は clean を実際に生成されるファイルの名前として扱わず、擬似ターゲットとして扱う。これにより、make clean を実行したときには、clean という名前のファイルが存在するかどうかにかかわらず、clean のレシピが実行される。

以上を反映した Makefile 全体は、次のようになる。

ファイル：geo_sample/Makefile（.PHONY の使用）

```
1 CC = gcc
2 CFLAGS = -std=c17 -Wall -Wextra
3 LDLIBS = -lm
4
5 TARGET = geo_sample
6 OBJS = point.o line.o geo_main.o
7
8 $(TARGET): $(OBJS)
9     $(CC) $(OBJS) $(LDLIBS) -o $(TARGET)
10
11 point.o: point.c point.h
12     $(CC) $(CFLAGS) -c point.c
13
14 line.o: line.c line.h point.h
15     $(CC) $(CFLAGS) -c line.c
16
17 geo_main.o: geo_main.c line.h point.h
18     $(CC) $(CFLAGS) -c geo_main.c
19
20 .PHONY: clean
21 clean:
22     rm -f $(TARGET) $(OBJS)
```

通常、make コマンドを引数なしで実行すると、Makefile に記述された最初の規則のターゲットが構築される。これに対して、特定のターゲット名を引数として指定すると、そのターゲットに対応する処理を実行することができる。clean ターゲットを実行する場合には、次のように入力する。

例. 構築によって生成されたファイルを削除する

```
$ make clean 
rm -f geo_sample point.o line.o geo_main.o
```

make は、指定された clean ターゲットのレシピを実行し、実行形式ファイルとオブジェクトファイルを削除する。point.c、point.h、line.c、line.h、geo_main.c、Makefile などの、構築の元となるファイルは削除されない。

make clean を実行した後に、再び引数なしで make を実行すると、オブジェクトファイルと実行形式ファイルが存在しないため、すべてのソースファイルが再コンパイルされ、リンク処理も再度実行される。

例. clean 後の再構築作業

```
$ make 
gcc -std=c17 -Wall -Wextra -c point.c
gcc -std=c17 -Wall -Wextra -c line.c
gcc -std=c17 -Wall -Wextra -c geo_main.c
gcc point.o line.o geo_main.o -lm -o geo_sample
```

このように、Makefile には、ファイルを生成するための規則だけでなく、生成物の削除、テストの実行、ファイルのインストールなど、開発作業に必要となる様々な処理を擬似ターゲットとして定義することができる。

D.2 CMake

CMake は、ソフトウェアを構築するために必要となるファイルを生成する、オープンソースの構築支援ツールである。make では、ファイル間の依存関係やコンパイル、リンクのためのコマンドを Makefile に直接記述する。これに対して CMake では、ソフトウェアの構成や構築方法を CMake 用の設定ファイルに記述し、その内容に基づいて、実際の構築に使用する Makefile や Ninja 用のファイル、統合開発環境用のプロジェクトファイルなどを生成する。

CMake で利用できるジェネレータ（後述）は、使用する OS や開発環境によって異なる。また、生成される構築用のファイルの形式は、選択したジェネレータによって決まる。例えば、UNIX 系 OS では、make で使用する Makefile や、Ninja で使用するファイルを生成することができる。Windows では、これらに加えて Visual Studio 用のプロジェクトファイルを生成することもできる。このように、CMake では、同じ設定ファイルを基にして、それぞれの環境に適した構築用のファイルを生成することができる。

CMake において、構築用のファイルを生成する仕組みを**ジェネレータ**という。利用するジェネレータを選択することにより、Makefile, Ninja 用のファイル, Visual Studio 用のプロジェクトファイルなど、異なる形式の構築用ファイルを生成することができる。従って、CMake そのものがソースファイルを直接コンパイルするのではなく、選択されたジェネレータによって構築用のファイルを生成し、その後、`make`, `Ninja`, `Visual Studio`などのツールを用いて実際のコンパイルやリンクを行う。

なお、`cmake --build` コマンドを使用すると、CMake を介して、生成された構築用ファイルに対応するビルドツールを起動することもできる。

異なる OS やコンパイラに対応する構築手順を個別に記述すると、同じソフトウェアであっても、環境ごとに異なる Makefile やプロジェクトファイルを維持する必要がある。CMake を利用すると、ソフトウェアの構成や必要なライブラリなどを共通の形式で記述し、環境に応じた構築用のファイルを生成できるため、複数の OS や処理系に対応したソフトウェアを管理しやすくなる。

CMake は、米国のソフトウェア企業である Kitware 社を中心として開発・保守されており、世界各地の開発者からなるコミュニティも開発に参加している。CMake はオープンソースソフトウェアとして公開されており、BSD 3-Clause License の下で利用することができる。

現在では、C や C++ による多くのソフトウェア開発で CMake が利用されており、特に複数の OS やコンパイラに対応するプロジェクトでは、代表的な構築支援ツールの一つとなっている。また、統合開発環境や各種ライブラリ、ソフトウェアの配布環境などでも、CMake への対応が広く行われている。従って、C や C++ による実用的なソフトウェア開発を行う上で、CMake の基本的な仕組みと使用方法を学んでおくことは重要である。

D.2.1 基本的な使用方法

CMake を用いてプログラムを構築する場合には、通常、プロジェクトの起点となるディレクトリに、`CMakeLists.txt` という名前の設定ファイルを用意する。このファイルには、プロジェクトの名前、使用するプログラミング言語、生成する実行形式ファイルやライブラリ、それらを構成するソースファイルなどを記述する。

CMake は、`CMakeLists.txt` を読み込み、使用する OS, コンパイラ, ジェネレータなどに応じて、実際の構築に使用するファイルを生成する。その後、生成された構築用のファイルに基づいて、ソースファイルのコンパイルやリンクが行われる。従って、CMake を使用する場合には、基本的に次の 2 段階の処理を行う。

1. **構成・生成処理**：CMake によって構築に必要な設定を行い、構築用のファイルを生成する
2. **構築処理**：生成されたファイルに基づいてコンパイルとリンクを行う

CMake が `CMakeLists.txt` を読み込み、使用するコンパイラの検出や必要なライブラリの探索などを行う処理を、**構成**、または**コンフィギュレーション**という。その後、選択されたジェネレータによって、実際の構築に使用するファイルが生成される。通常、これらの処理は一連のものとして実行される。

■ 設定ファイルの作成

先に示した `geo_sample` プロジェクトを CMake で構築するための `CMakeLists.txt` を次に示す。このファイルは、`point.c`, `point.h`, `line.c`, `line.h`, `geo_main.c` と同じ `geo_sample` ディレクトリに保存する。

ファイル：`geo_sample/CMakeLists.txt`

```
1 cmake_minimum_required(VERSION 3.21)
2
3 project(geo_sample LANGUAGES C)
4
5 set(CMAKE_C_STANDARD 17)
6 set(CMAKE_C_STANDARD_REQUIRED ON)
7 set(CMAKE_C_EXTENSIONS OFF)
8
9 add_executable(geo_sample geo_main.c point.c line.c)
10
11 if(MSVC)
12     message(STATUS "Compiler: MSVC")
13     target_compile_options(geo_sample PRIVATE /W4)
14 else()
15     message(STATUS "Compiler: non-MSVC")
16     target_compile_options(geo_sample PRIVATE -Wall -Wextra)
```



```
17     target_link_libraries(geo_sample PRIVATE m)
18 endif()
```

CMake の設定ファイルでは、空白文字、タブ文字、改行は、主に命令や引数を区切るために使用される。Makefile のレシピ行とは異なり、行頭にタブ文字を置かなければならないという制約はない。インデントには構文上の意味はないが、条件分岐などの構造を見やすくするために、一般には空白文字やタブを用いて字下げする。

CMake の命令は、基本的に次の形式で記述する。

命令名 (引数 1 引数 2 …)

命令名では大文字と小文字を区別しないが、一般には小文字で記述する。丸括弧内には、その命令に与える引数を空白文字や改行で区切って記述する。また、# から行末まではコメントとして扱われる。

先頭の `cmake_minimum_required` 命令は、この設定ファイルを処理するために必要となる CMake の最低バージョンを指定する。

```
cmake_minimum_required(VERSION 3.21)
```

この例では、CMake 3.21 以降を必要とすることを表している。これより古いバージョンの CMake で構成処理を行おうとすると、エラーとなり処理が停止する。また、この指定は、CMake のバージョンによって動作が異なる機能について、どの規則に基づいて処理するかを決定するためにも用いられる。

ここで指定する CMake のバージョンは、プロジェクトで使用する機能と、対象とする開発環境を考慮して決定する²¹⁰。

次の `project` 命令は、プロジェクトの名前と、プロジェクトで使用するプログラミング言語を指定する。

```
project(geo_sample LANGUAGES C)
```

この例では、プロジェクト名を `geo_sample` とし、使用する言語を C 言語としている。この命令が処理される際に、CMake は使用可能な C コンパイラを探索し、検出したコンパイラに関する情報を取得する。

C++ による開発では C ではなく CXX を指定する。また、

```
project(geo_sample LANGUAGES C CXX)
```

として、C と C++ の両方を使用するプロジェクトを指定することもできる。

続く 3 つの `set` 命令では、C 言語の規格に関する設定を行っている。

```
set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
```

`set` 命令は、CMake の変数に値を設定するために用いる。基本的な記述形式は、次の通りである。

```
set(変数名 値)
```

また、設定した変数の値を参照する場合には、変数名を波括弧で囲み、その前にドル記号 \$ を付けて、`${変数名}` と記述する。例えば、変数 `VALUE` の値は、`$VALUE` として参照する。Makefile では変数を `$(変数名)` と記述することが多い²¹¹ が、CMake では丸括弧ではなく波括弧を使用するため、混同しないように注意が必要である。

`CMAKE_C_STANDARD` に 17 を設定することにより、使用する C 言語規格を C17 とする。`CMAKE_C_STANDARD_REQUIRED` に `ON` を設定すると、指定した規格に対応していない場合に、より古い規格へ自動的に変更することを許可しない。また、`CMAKE_C_EXTENSIONS` に `OFF` を設定すると、処理系独自の拡張機能を含む言語モードではなく、標準規格に準拠した言語モードを使用するように指定する。

`add_executable` 命令は、実行形式ファイルを生成するターゲットを定義する。

```
add_executable(geo_sample geo_main.c point.c line.c)
```

最初の引数 `geo_sample` はターゲットの名前であり、続く `geo_main.c`、`point.c`、`line.c` は、そのターゲットを構成するソースファイルである。これにより、3 つのソースファイルをコンパイルし、それらをリンクして実行形式ファイル

²¹⁰ 使用中の CMake のバージョンは、`cmake --version` コマンドで確認できる。ただし、ここには、その結果をそのまま記述するのではなく、プロジェクトが必要とする最低バージョンを指定すること。

²¹¹ Makefile でも、`${変数名}` という形式で変数を参照することは可能である。

geo_sample を生成するための構築規則が作成される。

ここでは、point.h や line.h を add_executable 命令に列挙していない。ヘッダファイルはそれ自体をコンパイルする対象ではなく、各ソースファイルから読み込まれるためである。ただし、CMake が生成した構築システムでは、ソースファイルとヘッダファイルの依存関係も管理される。従って、ヘッダファイルを変更した場合には、その影響を受けるソースファイルが再コンパイルされる。

target_compile_options 命令は、指定したターゲットをコンパイルする際に使用するオプションを設定する。この例では、使用するコンパイラに応じて警告オプションを切り替えている。

```
if(MSVC)
    message(STATUS "Compiler: MSVC")
    target_compile_options(geo_sample PRIVATE /W4)
else()
    message(STATUS "Compiler: non-MSVC")
    target_compile_options(geo_sample PRIVATE -Wall -Wextra)
    target_link_libraries(geo_sample PRIVATE m)
endif()
```

if 命令で始まる条件分岐は、endif 命令で終了する。また、条件が成立しない場合の処理が不要であれば、else 命令は省略することができる。

この例で使用している MSVC は、Microsoft 社の C/C++コンパイラを使用している場合に真となる CMake の変数である。またこの例では、MSVC と、GCC および Clang を想定している。MSVC を使用する場合には、警告レベルを指定する /W4 を使用し、それ以外の場合には、ここでは GCC や Clang などで使用できる -Wall と -Wextra を指定している。このように、CMake では、使用する処理系に応じて構築時の設定を切り替えることができる。

target_compile_options 命令に指定した PRIVATE は、ここで指定するオプションを、ターゲット geo_sample 自身のコンパイルにだけ使用することを表している。PRIVATE のほかに、ライブラリを利用する側にも設定を伝える PUBLIC や INTERFACE があるが、これらについてはここでは扱わない。

target_link_libraries 命令は、指定したターゲットをリンクする際に使用するライブラリを設定する。

```
target_link_libraries(geo_sample PRIVATE m)
```

このプログラムでは sqrt 関数と atan2 関数を使用するため、本書で対象としている GCC や Clang を使用する場合には、数学ライブラリ m をリンクするように指定している。CMake が GCC 用の Makefileなどを生成した場合には、この指定はリンク時の -lm オプションなどに変換される。

message 命令は、CMake の処理中にメッセージを出力する。基本的な記述形式は、次の通りである。

```
message([メッセージタイプ] "メッセージ")
```

メッセージタイプは省略することもできるが、通常の情報や処理状況を表示する場合には STATUS を指定する。STATUS を指定したメッセージは、先頭に -- が付けられて出力される。

■ 構成・生成処理

CMakeLists.txt を用意した後、CMake による構成・生成処理を行う。ここでは、ソースファイルが置かれているディレクトリと、構築用のファイルや生成物を保存するディレクトリを分離する。このような構築方法をソース外構築という。

geo_sample ディレクトリで、次のコマンドを実行する。

例. CMake による構成・生成処理

```
$ cmake -S . -B build Enter
-- The C compiler identification is GNU ...
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: ... - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Compiler: non-MSVC
-- Configuring done (...)
-- Generating done (...)
-- Build files have been written to: .../geo_sample/build
```

オプション `-S` は、ソースディレクトリ、すなわち `CMakeLists.txt` が置かれているディレクトリを指定する。ここではピリオド `.` を指定しているため、カレントディレクトリがソースディレクトリとなる。オプション `-B` は、ビルドディレクトリ、すなわち構築用のファイルや生成物を保存するディレクトリを指定する。ここでは、`build` という名前のディレクトリを指定している。`build` ディレクトリが存在しない場合には、CMake によって自動的に作成される。

構成・生成処理を行うと、CMake はコンパイラや開発環境を調べ、選択されたジェネレータに対応する構築用ファイルを `build` ディレクトリ内に生成する。例えば、Makefile 用のジェネレータが選択された場合には、`build` ディレクトリ内に Makefile が生成される。この Makefile は CMake によって自動生成されたものであり、通常は利用者が直接編集しない。

■ 構築処理

構成・生成処理が完了した後、次のコマンドでプログラムを構築する。

例. CMake によって生成された構築システムを用いた構築

```
$ cmake --build build 
[ 25%] Building C object .../point.c.o
[ 50%] Building C object .../line.c.o
[ 75%] Building C object .../geo_main.c.o
[100%] Linking C executable geo_sample
[100%] Built target geo_sample
```

補足) 構築時の表示内容は、使用するジェネレータや環境によって異なる。

`cmake --build` コマンドは、構成・生成処理によって生成された構築システムを使用して、コンパイルとリンクを実行する。引数の `build` は、構築用のファイルが保存されているディレクトリである。実際の構築処理には、構成・生成時に選択された `make` や `Ninja` などが使用されるが、利用者はそれらの違いを意識せず、同じ `cmake --build` コマンドで構築を開始することができる。

UNIX 系 OS 上で Unix Makefiles または Ninja ジェネレータを使用した場合、この例では、実行形式ファイル `geo_sample` が `build` ディレクトリ内に生成される²¹²。できあがったプログラムは次のように実行する。

例. CMake を用いて構築したプログラムの実行

```
$ ./build/geo_sample 
begin  : (1.0, 2.0)
end    : (4.0, 6.0)
length : 5.00
angle  : 0.9273 rad
```

この時点では、`line_angle` 関数を実装した変更後の `line.c` を使用しているため、線分の角度として `0.9273 rad` が表示される。

構成・生成処理によって生成されたファイルや、コンパイルによって生成されたオブジェクトファイルなどは、`build` ディレクトリ内にまとめて保存される。そのため、構築によって生成されるファイルを、ソースファイル、ヘッダファイル、`CMakeLists.txt` などと混在させず、`build` ディレクトリ内に分離して管理することができる。不要になった構築結果を削除する場合には、`build` ディレクトリ全体を削除すればよい。

一度構成・生成処理を行った後は、通常、ソースファイルを変更するたびに構成・生成処理を繰り返す必要はない。ソースファイルを変更した場合には、再び次のコマンドを実行すれば、変更の影響を受けるファイルだけが再コンパイルされる。

```
$ cmake --build build 
```

`CMakeLists.txt` を変更した場合には、構築時に CMake が変更を検出し、必要に応じて構成・生成処理を自動的にやり直す。また、使用するジェネレータやコンパイラを変更する場合には、`build` ディレクトリを作り直して、改めて構成・生成処理を行うのが確実である。

²¹²Visual Studio や Ninja Multi-Config などのジェネレータでは、実行形式ファイルが `build/Debug` などのビルド構成別のディレクトリに生成される場合がある。また、Windows では実行形式ファイルに拡張子 `.exe` が付く。

■ 生成物の削除

構築によって生成された実行形式ファイルやオブジェクトファイルなどを削除する場合には、次のように `clean` ターゲットを指定する。

例. 構築によって生成されたファイルの削除

```
$ cmake --build build --target clean 
```

`--target` オプションは、既定のターゲットに代えて、指定したターゲットを処理するために用いる。ここでは、CMake が生成した構築システムに用意されている `clean` ターゲットを指定している。これにより、構築時に生成された実行形式ファイルやオブジェクトファイルなどが削除される。実際の削除処理は、使用しているジェネレータによって生成された Makefile や Ninja 用のファイルなどに基づいて行われる。

この処理を行っても、CMakeCache.txt や、CMake が生成した Makefile などの構築用ファイルは、通常、`build` ディレクトリ内に残る。従って、その後、再び次のコマンドを実行すれば、構成・生成処理をやり直すことなく、プログラムを構築することができる。

```
$ cmake --build build 
```

構成・生成処理によって生成されたファイルも含めて、構築に関係する生成物をすべて削除する場合には、`build` ディレクトリ全体を削除する。ソース外構築では、構築に関係する生成物がこのディレクトリ内にまとめられているため、ソースファイルを残したまま、構築前の状態に戻ることができる。その後、再び構築を行う場合には、次のように構成・生成処理からやり直す。

```
$ cmake -S . -B build 
```

```
$ cmake --build build 
```

なお、生成物を削除した後、続けて構築を行う場合には、次のように `--clean-first` オプションを指定することもできる。

```
$ cmake --build build --clean-first 
```

この場合には、最初に `clean` ターゲットが処理され、その後で通常の構築が行われる。

D.2.2 生成物の格納位置の設定

構築によって生成される実行形式ファイルやライブラリの格納位置は、CMakeLists.txt の中で設定することができる。主な設定用の変数を次に示す。

変数	格納する生成物
CMAKE_RUNTIME_OUTPUT_DIRECTORY	実行形式ファイル, Windows の共有ライブラリ (DLL)
CMAKE_LIBRARY_OUTPUT_DIRECTORY	モジュールライブラリ, Windows 以外の共有ライブラリ
CMAKE_ARCHIVE_OUTPUT_DIRECTORY	静的ライブラリ, Windows のインポートライブラリ

これらの変数には、`set` 命令を用いて格納先のディレクトリを設定する。例えば、実行形式ファイルをビルドディレクトリ内の `bin` ディレクトリに、ライブラリを `lib` ディレクトリに格納する場合には、CMakeLists.txt に次のように記述する。

```
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
```

CMAKE_BINARY_DIR は、構築ツリーの最上位となるディレクトリを表す CMake の変数である。例えば、次のコマンドによって構成・生成処理を行った場合には、CMAKE_BINARY_DIR は `build` ディレクトリの絶対パスを表す。

```
$ cmake -S . -B build 
```

従って、上の設定では、実行形式ファイルは `build/bin` に、ライブラリは `build/lib` に格納される。例えば、`geo_sample` を構築すると、UNIX 系 OS では実行形式ファイルが概ね次の位置に生成される。

```
build/bin/geo_sample
```

この場合、プログラムは次のように実行する。


```
$ ./build/bin/geo_sample Enter
```

これらの変数は、その後に定義されるターゲットの出力先を設定するために用いられる。従って、通常は `add_executable` 命令や `add_library` 命令より前に記述する。

ここで、`-B` オプションと、上に示した出力先変数との違いに注意する必要がある。`-B` オプションは、`Makefile`、`CMake` のキャッシュ、オブジェクトファイルなどを含む構築ツリー全体の格納位置を指定する。これに対して、`CMAKE_RUNTIME_OUTPUT_DIRECTORY` などの変数は、構築によって完成した実行形式ファイルやライブラリの格納位置を指定する。

生成物の格納位置を明示的に設定しない場合には、通常、実行形式ファイルやライブラリは、そのターゲットが定義された位置に対応するビルドディレクトリ内に生成される。例えば、最上位の `CMakeLists.txt` でターゲットを定義し、次のように構成・生成処理を行った場合には、生成物は基本的に `build` ディレクトリ内に格納される。

```
$ cmake -S . -B build Enter
```

また、`-B` オプションを省略すると、カレントディレクトリがビルドディレクトリとして使用される。例えば、ソースファイルが置かれているディレクトリ内で次のように実行すると、`CMake` が生成する構築用のファイルや構築時の生成物も、同じディレクトリ内に作成される。

```
$ cmake -S . Enter
```

この方法は**ソース内構築**となり、ソースファイルと構築による生成物が混在する。そのため、通常は `-B` オプションを用いて、ソースディレクトリとは別のビルドディレクトリを指定する。

なお、複数のディレクトリからなるプロジェクトでは、ターゲットの生成物が、そのターゲットを定義した位置に対応するビルドディレクトリ内に生成される場合がある。また、`Visual Studio`、`Xcode`、`Ninja Multi-Config` など、複数のビルド構成を扱うジェネレータでは、格納先の下に `Debug` や `Release` などのビルド構成別のディレクトリが追加される場合がある。

D.3 Ninja

`Ninja` は、大規模なソフトウェアを高速に構築することを目的として開発された、`make` などの伝統的なツールと比べると比較的新しい構築支援ツールである。構築するファイル間の依存関係と、コンパイルやリンクなどの処理を記述したファイルを読み込み、必要な処理だけを適切な順序で実行する。基本的な役割は `make` と共通しているが、構築処理を高速に開始し、多数の処理を効率よく並行実行できるように、機能を絞った簡潔な設計となっている。

`Ninja` は、`Google Chrome` の基盤となるオープンソースプロジェクト `Chromium` の開発において、大規模なソースコードを高速に構築する必要から、`Evan Martin` によって開発された。`Chromium` では非常に多数のソースファイルが扱われており、従来の構築システムでは、ファイルを変更してから実際のコンパイルが始まるまでに無視できない時間を要することがあった。`Ninja` は、この構築開始時の処理をできるだけ軽量にすることを重視して設計された。

`Ninja` が使用する構築用のファイルは、通常 `build.ninja` という名前である。このファイルを開発者が直接記述することもできるが、`Ninja` の記述形式は、人が手作業で管理することよりも、他のツールによって自動生成されることを想定して設計されている。例えば、`CMake` では、ジェネレータとして `Ninja` を指定することにより、`CMake` の設定ファイルから `build.ninja` を生成することができる。この場合、`CMake` が `Ninja` 用の構築ファイルを生成し、`Ninja` がそれに基づいて実際のコンパイルやリンクを高速に実行する。本書では、`build.ninja` の記述形式そのものには深入りせず、`CMake` によって `build.ninja` を生成し、`Ninja` を用いて構築する基本的な利用方法を説明する。

現在では、`Ninja` は `Google Chrome`、`Android` の一部、`LLVM` など、大規模なソフトウェアの構築に使用されている。また、`CMake` などの構築システム生成ツールと組み合わせ、一般的な `C` および `C++` の開発でも利用されている。ソースコードは `Apache License 2.0` の下で公開されており、現在は `GitHub` 上の `ninja-build` プロジェクトを中心とする開発者コミュニティによって開発・保守されている。

D.3.1 CMake による build.ninja ファイルの生成

ここでは、前節までに使用した `geo_sample` を、CMake と Ninja を組み合わせて構築する。`geo_sample` ディレクトリには、次のファイルが置かれているものとする。

ファイル	内容
<code>geo_main.c</code>	メインプログラム
<code>point.c</code> , <code>point.h</code>	点を扱う関数とその宣言
<code>line.c</code> , <code>line.h</code>	線分を扱う関数とその宣言
<code>CMakeLists.txt</code>	CMake の設定ファイル

ここで使用する `CMakeLists.txt` は、前節の CMake による構築で使用したものと同じであり、Ninja 用に別の設定ファイルを作成する必要はない。CMake は、同じ `CMakeLists.txt` から、指定されたジェネレータに対応する構築用ファイルを生成する。

Ninja を使用するには、Ninja が計算機環境にインストールされ、コマンドとして実行できる状態になっている必要がある。次のコマンドを実行すると、インストールされている Ninja のバージョンを確認できる。

```
$ ninja --version 
```

バージョン番号が表示されれば、Ninja を利用できる。

注意) 一つのビルドディレクトリでは、途中でジェネレータを変更することができない。そのため、ここでは、前節で使用した `build` とは別に、`build_ninja` ディレクトリを使用する。

次に、カレントディレクトリを `geo_sample` ディレクトリとして、次のコマンドを実行する。

```
$ cmake -S . -B build_ninja -G Ninja 
```

`-S .` は、カレントディレクトリをソースディレクトリとして指定する。`-B build_ninja` は、CMake が生成する構築用のファイルを `build_ninja` ディレクトリに格納することを指定する。また、`-G Ninja` は、構築システムを生成するためのジェネレータとして Ninja を選択することを意味する。ジェネレータ名の `Ninja` では、大文字と小文字が区別される。

このコマンドを実行すると、CMake は `CMakeLists.txt` を読み込み、`build_ninja` ディレクトリを作成して、その中に Ninja 用の構築ファイルを生成する。生成された主なファイルは、次のコマンドで確認できる。

```
$ ls build_ninja 
```

Windows のコマンドプロンプトでは、`ls` の代わりに `dir` を使用する。生成されたファイルの中には、Ninja が読み込む `build.ninja` が含まれている。このファイルには、ソースファイルのコンパイルや実行形式ファイルのリンクに必要な規則、入力ファイルと生成物との依存関係などが記述されている。また、C/C++ のヘッダファイルへの依存関係は、コンパイラが出力する情報などを利用して Ninja によって管理される。

`build.ninja` は CMake によって自動生成されたファイルであるため、通常は開発者が直接編集しない。構築方法を変更する場合には、`CMakeLists.txt` を修正し、必要に応じて CMake による構成・生成処理を再度実行する。

D.3.2 Ninja による構築

CMake によって `build.ninja` を生成した後、次のコマンドを実行すると、Ninja による構築が行われる。

```
$ ninja -C build_ninja 
```

`-C` オプションは、指定したディレクトリへ移動してから構築処理を行うことを意味する。従って、上のコマンドでは、Ninja は `build_ninja` ディレクトリ内の `build.ninja` を読み込み、必要なコンパイルとリンクを実行する。

構築が正常に完了すると、実行形式ファイル `geo_sample` が `build_ninja` ディレクトリ内に生成される。UNIX 系 OS では、次のように実行できる。

```
$ ./build_ninja/geo_sample 
```

Windows では、使用する処理系やコマンド環境に応じて、概ね次のように実行する。

```
> build_ninja\geo_sample.exe 
```

Ninja を直接起動する代わりに，次のように CMake を介して構築することもできる．

```
$ cmake --build build_ninja 
```

この場合，CMake は，指定されたビルドディレクトリに記録されているジェネレータの情報に基づいて，対応するビルドツールを起動する．ここではジェネレータとして Ninja を指定しているため，Ninja による構築が行われる．

E 動的ライブラリの構築

これまで本書では、作成した関数やクラスを最終的に一つの実行プログラムへリンクして利用する形を、暗に想定してきた。この方法では、必要な機能がプログラムの構築時に確定し、それらの実装が実行ファイルの一部として組み込まれる。これは、プログラムを複数のソースファイルに分けて開発し、最後に一つの完成品へまとめる場合に適した方法である。

一方、汎用性の高い機能は、個々のアプリケーションに組み込むだけでなく、独立性の高いソフトウェア部品として実装し、多くのプログラムから共有して利用できることが望ましい。このような機能を動的ライブラリとして構築すると、ライブラリの実装を実行ファイルから分離したまま提供できる。利用側のプログラムは、起動時または実行中に動的ライブラリを読み込み、そこに公開されている関数を呼び出す。

動的ライブラリによって共有できるのは、同じプログラミング言語で記述されたプログラム間の機能に限らない。C 言語形式の関数インターフェースは、多くの OS、コンパイラおよび言語処理系で広く利用されている。このため、C または C++ で実装した機能を、対象環境の C ABI (Application Binary Interface) に従う関数として公開すれば、別のプログラミング言語²¹³ や、既存のアプリケーションから利用することも可能になる。すなわち動的ライブラリは、コンパイル済みの汎用機能を、プログラムやプログラミング言語の境界を越えて共有するための手段となる。

C++ では、関数名をバイナリ中のシンボル名へ変換する方法、クラスオブジェクトの配置、例外処理の方式および標準ライブラリのバイナリ表現などが、言語規格によって統一されていない。これらは、使用する OS、コンパイラ、標準ライブラリおよびビルド時の設定などに依存する。このため、C++ の関数やクラスをそのまま動的ライブラリから公開すると、利用側にも同じ処理系、または ABI 互換性のある実行環境が必要になる場合がある。

C ABI にもプラットフォームごとの違いはあるが、各プラットフォームにおける基本的なバイナリインターフェースとして広く利用され、多くの言語処理系から呼び出すことができる。そこで、C++ で実装した機能についても、ここでは主として `extern "C"` を用いて C 言語形式の関数として公開する方法を扱う。ただし、`extern "C"` を指定するだけで任意の C++ の型が他の言語から利用可能になるわけではない。外部に公開する関数の引数と戻り値には、原則として C 言語と互換性のある型を使用する。

動的ライブラリには、保守の単位を分離できるという利点もある。外部に公開する関数の仕様や ABI との互換性が維持されていれば、ライブラリを利用するアプリケーションを再構築することなく、ライブラリだけを修正または更新できる。これにより、不具合の修正や内部実装の改良など、リリース後のメンテナンスにも柔軟に対応できる。

また、動的ライブラリのコードは各実行ファイルへ取り込まれないため、個々の実行ファイルを小さくできる。一つのアプリケーションだけを配布する場合には、動的ライブラリを含めた配布物全体が必ずしも小さくなるとは限らないが、複数のアプリケーションが同じライブラリを利用する場合には、完成したバイナリ群に含まれるコードの重複を減らすことができる。

動的ライブラリは、Windows では主に DLL (Dynamic Link Library)、Linux などの UNIX 系環境では**共有オブジェクト**、macOS では**動的ライブラリ**として提供される。また本書では、これらのプラットフォームごとの名称を総称して、**動的ライブラリ**と呼ぶこととする。

ここでは、これらの基本的な仕組みを説明し、C/C++ で実装した機能を動的ライブラリとして構築する方法を示す。

E.1 動的ライブラリの基本的な仕組み

動的ライブラリには、外部のプログラムから利用する関数が収められている。ただし、ライブラリ内に定義されたすべての関数が、必ず外部から利用できるわけではない。外部から利用する関数は、その名前と所在を識別できる形で公開しておく必要がある。利用側のプログラムは、公開された名前を手掛かりとして目的の関数を見つけ、その処理を呼び出す。

この基本的な考え方は各種のプラットフォームで共通しているが、関数を公開する方法、ライブラリのファイル形式および読み込みに使用する機能などは、OS やコンパイラによって異なる。ここでは、まず動的ライブラリに共通する基本的な仕組みを説明する。

²¹³C 言語形式のインターフェースを持つ動的ライブラリは、Python や C#/.NET などからも広く利用されている。

E.1.1 公開シンボルと利用側プログラムとの結合

コンパイラによって生成されたオブジェクトファイルやライブラリでは、関数や変数などがシンボルと呼ばれる名前によって識別される。動的ライブラリの外部から利用できるようにしたシンボルを、**公開シンボル**または**エクスポートシンボル**と呼ぶ。通常、動的ライブラリでは、外部へ提供する関数を公開シンボルとして登録する。

C または C++ で記述した利用側プログラムから公開関数を通常の関数として呼び出す場合は、関数名、引数の型、戻り値の型などを宣言したヘッダファイルを用いる。利用側のソースプログラムをコンパイルするとき、コンパイラはこの宣言に基づいて関数呼出しのコードを生成する。ただし、動的リンクを行う場合には、関数の実装そのものは実行ファイルへ取り込まれない。実行ファイルには、利用するライブラリや公開シンボルに関する情報が記録され、プログラムの実行時に実際の関数と結び付けられる。

このように、利用側のプログラムに記録された関数の参照と、動的ライブラリ内の公開シンボルとを対応付ける処理を**シンボル解決**と呼ぶ。シンボル解決は主としてシンボル名に基づいて行われ、引数や戻り値の型まで実行時に照合されるわけではない。このため、動的ライブラリと利用側プログラムの間で、関数の宣言と ABI を正しく一致させる必要がある。

Windows では、動的ライブラリの構築時に作成される**インポートライブラリ**を利用してリンクする方法が一般的である。一方、Linux などの UNIX 系環境では、共有オブジェクトをリンク時に直接指定する。具体的なファイル形式や指定方法は異なるが、実行ファイルへ関数の実装を取り込まず、実行時に動的ライブラリ内の公開シンボルと結び付けるという考え方は共通している。

E.1.2 起動時の読み込みと実行中の読み込み

動的ライブラリを利用する方法は、読み込みの時期によって大きく二つに分けられる。一つは、利用する動的ライブラリをリンク時に指定しておき、プログラムの起動時に OS の機能によって読み込ませる方法である。この方法では、プログラムの起動に伴って必要なライブラリがメモリへ配置され、公開シンボルの解決が行われる。利用側では、通常の関数を呼び出す場合とほぼ同じ記述で、動的ライブラリの関数を利用できる。

ただし、必要な動的ライブラリが見つからない場合や、必要な公開シンボルが存在しない場合には、プログラムを正常に起動できないことがある。このため、ライブラリを配置する場所や検索経路を、実行環境に合わせて適切に設定する必要がある。

もう一つは、プログラムの実行中に、利用側のプログラムが OS や実行環境によって提供される動的読み込みの機能を用いて、動的ライブラリを明示的に読み込む方法である。この方法では、まずライブラリを読み込んでその識別情報を取得し、次に関数名を指定して公開関数のアドレスを取得する。取得したアドレスを適切な型の関数ポインタとして扱うことにより、目的の関数を呼び出すことができる。利用を終えたライブラリは、プログラムから明示的に解放することもできる。

実行中に読み込む方法では、実行環境や利用者の設定に応じてライブラリを選択できる。このため、追加機能を後から組み込むプラグイン機構や、利用できる場合にだけ特定の機能を有効にする処理などに適している。その一方で、ライブラリの読み込み、関数アドレスの取得、エラーの確認および解放を利用側のプログラムが管理しなければならない。

一般的なアプリケーションでは、起動時に読み込む方法の方が簡潔であり、通常の関数呼出しに近い形で利用できる。実行中に読み込む方法は、ライブラリを動的に選択する必要がある場合に用いられる。以降では、起動時に読み込む方法を用いて、動的ライブラリを構築し利用する基本的な手順を示す。

E.2 公開インターフェースの設計

動的ライブラリを作成する際には、内部で行う処理だけでなく、利用側のプログラムへの機能を公開するかを明確に設計する必要がある。外部へ公開する関数の名前、引数、戻り値およびデータ型は、動的ライブラリと利用側プログラムの間を結ぶインターフェースとなる。このインターフェースは、ライブラリの実装を変更した後も、可能な限り安定して維持することが望ましい。

特に、C++で実装した機能を広く共有する場合には、クラスやテンプレートなどをそのまま公開するのではなく、外部との境界をC言語形式の関数として設計すると扱いやすい。ライブラリの内部ではC++のクラス、標準ライブラリおよび各種のアルゴリズムを自由に利用し、外部とのデータの受渡しだけをC言語と互換性のある形式に限定することができる。

E.2.1 公開用ヘッダファイルとC ABI

CまたはC++で動的ライブラリを利用する場合は、公開関数の名前、引数および戻り値の型を利用側のコンパイラへ知らせる必要がある。これらの宣言をまとめたものが**公開用ヘッダファイル**である。利用側のプログラムは公開用ヘッダファイルをインクルードし、そこに宣言された関数を呼び出す。一方、関数の具体的な実装は動的ライブラリの内部に置かれる。

公開用ヘッダファイルには、利用側が必要とする宣言だけを記述する。ライブラリ内部でのみ使用する関数、クラスおよび変数などは公開用ヘッダファイルへ記述しない。公開部分と内部実装を分離することにより、利用側へ影響を与えずにライブラリ内部の構成を変更しやすくなる。

CとC++の両方から利用できる公開用ヘッダファイルを作成する場合は、C++固有の型や構文を公開インターフェースへ持ち込まないようにする。関数の引数と戻り値には、基本型、基本型へのポインタ、およびC言語でも定義できる構造体などを使用する。文字列や配列を渡す場合は、その先頭を指すポインタだけでなく、必要に応じて文字数や要素数も併せて渡す。

C++では、同じ名前でも引数の異なる関数を定義するオーバーロードを実現するために、関数名や引数の型などの情報を組み合わせて、バイナリ中のシンボル名を生成する。この変換を**名前修飾** (name mangling) と呼ぶ。名前修飾の方法は処理系によって異なるため、C++の関数をそのまま公開すると、異なるコンパイラや言語処理系からシンボル名を参照することが難しくなる。

C++で定義した関数にC言語リンケージを指定するには、`extern "C"` を用いる。例えば、次の宣言では、`calculate` がC言語リンケージを持つ関数として扱われる。

```
extern "C" int calculate(const double *data, size_t count);
```

`extern "C"` は、C++で記述された関数の内部処理をC言語へ変換する指定ではない。関数のリンケージをC言語形式にし、C++固有の名前修飾を抑えるための指定である。従って、関数の内部ではC++の機能を利用できるが、外部へ公開する引数と戻り値には、原則としてC言語と互換性のある型を使用する必要がある。

ただし、`extern "C"` はC++の構文であり、Cコンパイラでは解釈できない。そこで、C++としてコンパイルされる場合だけ `extern "C"` を有効にするために、処理系が定義する `__cplusplus` マクロを用いる。公開する関数が複数ある場合は、次のようにまとめて指定できる。

```
#ifdef __cplusplus
extern "C" {
#endif

int function1(int value);
double function2(const double *data, size_t count);

#ifdef __cplusplus
}
#endif
```

このヘッダファイルをC++から読み込むと、二つの関数にはC言語リンケージが指定される。Cから読み込む場合は `extern "C"` の部分が取り除かれ、通常のC言語の関数宣言として扱われる。この形にしておくことで、同じ公開用ヘッダファイルをCとC++の双方から利用できる。

注意) `extern "C"` はC言語リンケージを指定するものであり、異なるOSや処理系の間でABI全体の互換性を保証する指定ではない。

E.2.2 公開指定のためのマクロ

C言語リンケージを指定しただけでは、関数が必ず動的ライブラリの外部へ公開されるとは限らない。特にWindowsでDLLを構築する場合は、公開する関数に `__declspec(dllexport)` を指定する。一方、DLLを利用する側では、

一般に`__declspec(dllimport)`を指定する。

しかし、この指定を公開関数ごとに直接記述すると、DLLを構築する場合と利用する場合でヘッダファイルを書き換えなければならない。また、Windows以外の環境では別の扱いが必要になる。そこで、公開方法の違いをプリプロセッサマクロによって吸収する。例えば、`SAMPLE_API`というマクロを次のように定義する。

```
#if defined(_WIN32)
#if defined(SAMPLE_BUILD)
#define SAMPLE_API __declspec(dllexport)
#else
#define SAMPLE_API __declspec(dllimport)
#endif
#else
#define SAMPLE_API
#endif
```

DLL本体を構築するときは`SAMPLE_BUILD`を定義する。これにより、`SAMPLE_API`は`__declspec(dllexport)`に置き換えられる。DLLを利用するプログラムでは`SAMPLE_BUILD`を定義しないため、`SAMPLE_API`は`__declspec(dllimport)`に置き換えられる。

LinuxやmacOSなどでは、通常の構築方法で外部リンケージを持つ関数が公開されるため、上の例では`SAMPLE_API`を空のマクロとしている。GCCやClangには公開するシンボルを限定する機能もあるが、基本的な動的ライブラリを作成する段階では、この簡略な定義で対応できる。

公開関数の宣言では、このマクロを関数の戻り値の型の前へ記述する。

```
SAMPLE_API int calculate(
    const double *data,
    size_t count
);
```

このようなマクロを使用すると、公開関数の宣言を変更することなく、同じヘッダファイルをDLLの構築側、DLLの利用側およびUNIX系環境で共通して使用できる。また、公開指定が必要な関数も明確になる。実際には、ライブラリごとに固有のマクロ名を使用し、他のヘッダファイルで定義されたマクロとの衝突を避ける。

E.3 例) 数値集計ライブラリの実装

ここでは、`double`型の数値データを集計し、データ個数、合計値、最小値、最大値、平均値および分散を求める動的ライブラリを作成する。集計対象となる配列の先頭アドレスとデータ個数を関数へ渡し、計算結果は構造体を通して受け取る仕様とする。

ライブラリの公開インターフェースはC言語形式で定義し、同じ機能をCとC++のそれぞれで実装する例を示す。どちらの実装を使用した場合も、公開される関数の名前、引数、戻り値および計算結果の形式は同じである。このため、利用側のプログラムは、ライブラリの内部がCとC++のどちらで記述されているかを意識せずに使用できる。

次に示す`numstats.h`は、C版とC++版で共通して使用する公開用ヘッダファイルである。

プログラム：numstats.h

```
1  #ifndef NUMSTATS_H
2  #define NUMSTATS_H
3
4  #include <stddef.h>
5
6  // Windowsでは、DLLの構築時と利用時で公開方法を切り替える
7  #if defined(_WIN32)
8  #if defined(NUMSTATS_BUILD)
9  #define NUMSTATS_API __declspec(dllexport)
10 #else
11 #define NUMSTATS_API __declspec(dllimport)
12 #endif
13 #else
14 #define NUMSTATS_API
15 #endif
16
```

```

17 #define NUMSTATS_OK          0          // 正常終了
18 #define NUMSTATS_INVALID_ARG  1          // 引数が不正
19
20 // 処理結果を格納する構造体
21 typedef struct {
22     size_t count;           // データ個数
23     double sum;             // 合計値
24     double minimum;        // 最小値
25     double maximum;        // 最大値
26     double mean;           // 平均値
27     double variance;       // 母分散
28 } numstats_result;
29
30 // C++からコンパイルする場合も、関数をC ABIで公開する
31 #ifdef __cplusplus
32 extern "C" {
33 #endif
34
35 // 配列 dataを集計し、結果をresultへ格納する関数（プロトタイプ宣言）
36 NUMSTATS_API int numstats_summary(
37     const double *data,
38     size_t count,
39     numstats_result *result
40 );
41
42 #ifdef __cplusplus
43 }
44 #endif
45
46 #endif

```

構造体 `numstats_result` には、集計によって得られる六つの値を格納する。関数 `numstats_summary` は、集計対象となる配列を指す `data`、データ個数を表す `count`、および計算結果の格納先を指す `result` を受け取る。

処理が正常に完了すると `NUMSTATS_OK` を返し、引数が不正な場合は `NUMSTATS_INVALID_ARG` を返す。計算結果と処理状態を分けることにより、利用側では、関数が正常に終了したことを確認してから構造体の内容を参照できる。

Windows では、`NUMSTATS_BUILD` が定義されている場合に `NUMSTATS_API` が `__declspec(dllexport)` となり、関数が DLL から公開される。利用側では `NUMSTATS_BUILD` を定義しないため、このマクロは `__declspec(dllimport)` となる。Windows 以外の環境では、ここで使用する `NUMSTATS_API` は空のマクロとなる。

また、C++としてコンパイルする場合には、関数宣言が `extern "C"` の対象となる。これにより、C++版のライブラリでも `numstats_summary` が C 言語リンケージを持つ関数として公開される。

E.3.1 C によるライブラリの実装

次に示す `numstats.c` は、数値集計ライブラリを C で実装したものである。

プログラム：numstats.c

```

1  #include "numstats.h"
2
3  int numstats_summary(const double *data, size_t count, numstats_result *result) {
4      // 配列、データ個数、格納先のいずれかが不正なら処理しない
5      if (data == NULL || count == 0 || result == NULL) {
6          return NUMSTATS_INVALID_ARG;
7      }
8
9      double minimum = data[0];
10     double maximum = data[0];
11     double sum = data[0];
12
13     // 1回目の走査で、最小値、最大値および合計値を求める
14     for (size_t i = 1; i < count; ++i) {
15         if (data[i] < minimum) {
16             minimum = data[i];
17         }
18         if (data[i] > maximum) {

```

```

19         maximum = data[i];
20     }
21     sum += data[i];
22 }
23
24     const double mean = sum / (double)count;    // 平均値
25     double squared_sum = 0.0;
26
27     // 2回目の走査で、平均との差の二乗和を求める
28     for (size_t i = 0; i < count; ++i) {
29         const double difference = data[i] - mean;
30         squared_sum += difference * difference;
31     }
32
33     // すべての計算結果を呼び出し側の構造体へ格納する
34     result->count = count;
35     result->sum = sum;
36     result->minimum = minimum;
37     result->maximum = maximum;
38     result->mean = mean;
39     result->variance = squared_sum / (double)count;    // 分散
40
41     return NUMSTATS_OK;
42 }

```

関数の先頭では、引数として受け取った二つのポインタとデータ個数を検査している。data または result がヌルポインタである場合や、count が 0 である場合には計算を行うことができないため、NUMSTATS_INVALID_ARG を返して処理を終了する。この検査によってデータが 1 個以上存在することが保証されるため、以降の処理では data[0] を安全に参照できる。

最小値、最大値および合計値の初期値には、配列の先頭要素を使用する。その後、2 番目から最後まで要素を順番に調べ、最小値および最大値を必要に応じて更新しながら、すべての値を合計する。先頭要素は初期値として既に合計へ含まれているため、この繰返しは添字 1 から開始する。

平均値は、合計値をデータ個数で割ることによって求める。次に、配列をもう一度先頭から調べ、各要素と平均値との差を求め、その二乗を加算する。得られた二乗和をデータ個数で割った値を分散とする。ここでは、データ個数から 1 を引いた値で割る不偏分散ではなく、データ個数で割る母分散を計算している。

すべての計算が終わると、データ個数、合計値、最小値、最大値、平均値および分散を、result が指す構造体へ格納する。最後に NUMSTATS_OK を返し、処理が正常に完了したことを利用側へ通知する。

E.3.2 C++によるライブラリの実装

次に示す numstats.cpp は、同じ数値集計ライブラリを C++ で実装したものである。

プログラム：numstats.cpp

```

1  #include "numstats.h"
2  #include <algorithm>
3  #include <numeric>
4
5  int numstats_summary(const double *data, size_t count, numstats_result *result) {
6      // 配列、データ個数、格納先のいずれかが不正なら処理しない
7      if (data == nullptr || count == 0 || result == nullptr) {
8          return NUMSTATS_INVALID_ARG;
9      }
10
11     // 標準ライブラリのアルゴリズムで、最小値と最大値を同時に求める
12     const auto minmax = std::minmax_element(data, data + count);
13
14     // accumulate を用いて合計値を求め、そこから平均値を算出する
15     const double sum = std::accumulate(data, data + count, 0.0);
16     const double mean = sum / static_cast<double>(count);
17
18     // 各要素と平均との差の二乗和を求める
19     const double squared_sum = std::accumulate(
20         data,

```

```

21     data + count,
22     0.0,
23     [mean](double total, double value) {
24         const double difference = value - mean;
25         return total + difference * difference;
26     }
27 );
28
29 // すべての計算結果を呼び出し側の構造体へ格納する
30 result->count = count;
31 result->sum = sum;
32 result->minimum = *minmax.first;
33 result->maximum = *minmax.second;
34 result->mean = mean;
35 result->variance = squared_sum / static_cast<double>(count);    // 分散
36
37 return NUMSTATS_OK;
38 }

```

関数の名前、引数および戻り値は C 版と同じである。numstats.h を C++ として読み込むと、numstats.summary の宣言に extern "C" が適用される。このため、関数の内部を C++ で実装しても、外部には C 言語リンケージを持つ関数として公開される。

引数の検査には、C++ でヌルポインタを表す nullptr を使用する。検査する内容は C 版と同じであり、データ配列、データ個数または結果の格納先が不正な場合には、NUMSTATS_INVALID_ARG を返す。

C++ 版では、最小値と最大値を求めるために std::minmax_element を使用し、合計値を求めるために std::accumulate を使用している。これらの関数には、処理範囲の先頭と末尾の次を表す二つのイテレータを渡す。通常の配列を指すポインタもイテレータとして利用できるため、data と data + count を渡すことにより、配列全体を処理範囲として指定できる。この処理範囲は、次の半开区間として表される。

[data, data + count)

data + count は最後の要素ではなく、最後の要素の次の位置を表す。この位置は処理範囲には含まれない。コンテナに対して begin() と end() を指定する場合と同じ考え方である。

std::minmax_element は、最小の要素と最大の要素を指すイテレータの組を返す。この例ではイテレータの実体がポインタであるため、それぞれを間接参照して最小値と最大値を取り出す。また、最初の std::accumulate では初期値に 0.0 を指定し、配列内のすべての要素を double 型の値として加算する。

平均値を求めた後、各要素と平均との差の二乗和も std::accumulate を用いて求める。この処理では、第 3 引数の 0.0 を集約結果の初期値とし、第 4 引数に集約処理を表すラムダ式を指定している。ラムダ式の [mean] は、外側で求めた平均値 mean を値キャプチャする指定である。引数 total にはそれまでの集約結果、value には現在の要素が渡され、平均との差の二乗を total へ加えた値を返す。

このように、std::accumulate は単純な合計だけでなく、第 4 引数に演算を指定することによって、値を順次集約する処理にも利用できる。最後に、データ個数を static_cast<double> によって double 型へ変換して二乗和を割り、得られた分散と他の集計結果を numstats_result 構造体へ格納する。処理が完了すると NUMSTATS_OK を返す。

C++ 版では標準ライブラリの機能を内部で使用しているが、そのことは公開インターフェースに現れない。外部とのデータの受渡しには C 言語と互換性のある関数と構造体だけを使用しているため、同じ公開用ヘッダファイルを C のプログラムから利用できる。

E.3.3 ライブラリを利用するプログラム

次に示す test_numstats.c は、数値集計ライブラリを利用する C のプログラムである。

プログラム：test_numstats.c

```

1  #include <stdio.h>
2  #include "numstats.h"
3
4  int main() {
5      // 集計対象となるサンプルデータ

```

```

6   const double data[] = {
7       -0.9134, -1.5485, -1.1665,  1.6720,  0.8482, -0.7199, -1.9814,
8       0.4779,  0.5387, -0.0354, -0.7361,  0.7249,  1.3724, -0.2171,
9       -0.4946,  0.2202,  1.9446, -0.1286,  1.1341,  0.9049, -0.2402,
10      -0.9546,  0.1199,  0.4314, -1.2430
11  };
12
13  // 配列全体のバイト数を1要素のバイト数で割って、データ個数を求める
14  const size_t count = sizeof(data) / sizeof(data[0]);
15  numstats_result result;           // 集計結果の格納先
16
17  // 動的ライブラリが提供する関数を呼び出す
18  const int status = numstats_summary(data, count, &result);
19
20  // 引数が不正な場合は、メッセージを標準エラー出力へ表示する
21  if (status != NUMSTATS_OK) {
22      fprintf(stderr, "Calculation failed.\n");
23      return 1;
24  }
25
26  // 構造体に格納された集計結果を表示する
27  printf("count      : %zu\n",  result.count);
28  printf("sum        : %.6f\n",  result.sum);
29  printf("minimum     : %.6f\n",  result.minimum);
30  printf("maximum     : %.6f\n",  result.maximum);
31  printf("mean        : %.6f\n",  result.mean);
32  printf("variance    : %.6f\n",  result.variance);
33  }

```

プログラムでは、25 個の `double` 型の値からなる配列 `data` を用意している。配列全体のバイト数を 1 要素のバイト数で割ることにより、データ個数を求めている。この方法は、配列そのものが参照できる場所で使用できる。配列を関数へ渡すと先頭要素を指すポインタとして扱われるため、ライブラリへは配列を指すポインタとデータ個数を別々に渡す。

計算結果を受け取るために、`numstats_result` 型の変数 `result` を用意する。`numstats_summary` には、配列の先頭を表す `data`、データ個数を表す `count`、および `result` のアドレスを渡す。関数から返された処理状態は `status` に格納する。

`status` が `NUMSTATS_OK` でなければ、標準エラー出力へメッセージを表示してプログラムを終了する。正常に終了した場合は、`result` に格納されたデータ個数、合計値、最小値、最大値、平均値および分散を順番に表示する。関数の成否を確認してから構造体を参照することにより、計算に失敗した場合の未確定な値を表示せずに済む。

この利用側プログラムは、C 版と C++ 版のどちらの動的ライブラリに対しても同じソースコードで利用できる。利用側から見えるのは `numstats.h` に記述された公開インターフェースだけであり、集計処理がどのように実装されているかは利用側のプログラムから分離されている。動的ライブラリと利用側プログラムを実際に構築し、実行する方法については、この後でプラットフォームおよびコンパイラごとに示す。

E.4 環境別の動的ライブラリ構築

ここでは、前項で作成した数値集計ライブラリと利用側プログラムを、代表的な OS およびコンパイラを用いて構築する方法を示す。いずれの環境でも、C による実装 `numstats.c` と C++ による実装 `numstats.cpp` のいずれか一方を選び、同じ名前の動的ライブラリを作成する。利用側プログラム `test_numstats.c` の構築方法は、どちらの実装を選んだ場合も同じである。

以下では、`numstats.h`、`numstats.c`、`numstats.cpp` および `test_numstats.c` が同じディレクトリに置かれているものとする。コマンド中の `-O2` および `/O2` は最適化を行うための指定であり、プログラムの動作を確認する段階では省略してもよい。

E.4.1 Linux で GCC を使用する場合

Linux 上で GCC を使用し、C 版の共有オブジェクトを構築するには、次のように実行する。

コマンド： `gcc -std=c17 -O2 -fPIC -shared numstats.c -o libnumstats.so`

C++版を構築する場合は、gcc の代わりに g++ を使用し、C++17 を指定する。

コマンド: `g++ -std=c++17 -O2 -fPIC -shared numstats.cpp -o libnumstats.so`

`-std=c17` と `-std=c++17` は、それぞれ C17 と C++17 の言語規格を指定する。`-fPIC` は、メモリ上の配置位置に依存しない**位置独立コード** (Position Independent Code) を生成するための指定である。共有オブジェクトは実行時にさまざまなアドレスへ配置される可能性があるため、通常はこの指定を付けて構築する。`-shared` は、通常の実行ファイルではなく共有オブジェクトを生成するための指定である。`-o` の後には、生成するファイル名を指定する。Linux の共有オブジェクトには、一般に `lib` で始まり、`.so` で終わる名前を付ける。

利用側プログラムは、次のように構築する。

コマンド: `gcc -std=c17 -O2 test_numstats.c -L. -lnumstats -Wl,-rpath,'$ORIGIN' \`
`-o test_numstats`

`-L.` は、リンクするライブラリの探索先に現在のディレクトリを加える指定である。`-lnumstats` は、基本名が `numstats` であるライブラリをリンクの対象として指定する。この例では、現在のディレクトリにある `libnumstats.so` が使用される。`-l` では、ファイル名先頭の `lib` と末尾の拡張子を除いた名前を記述する。

`-Wl,` は、コンマで区切って記述した後続のオプションを、コンパイラドライバからリンカへ渡すための指定である。`-rpath` によって、実行時に共有オブジェクトを探索する場所を実行ファイルへ記録する。`$ORIGIN` は実行ファイルが置かれているディレクトリを表すため、この例では `libnumstats.so` を `test_numstats` と同じディレクトリに置いて実行できる。シェルによる変数展開を防ぐため、コマンドでは `'$ORIGIN'` のように引用符で囲んでいる。

共有オブジェクトに公開されたシンボルは、次のコマンドで確認できる。

コマンド: `nm -D --defined-only libnumstats.so`

出力に `numstats_summary` が含まれていれば、この関数が外部へ公開されていることを確認できる。また、実行ファイルが参照する共有オブジェクトは、`ldd` コマンドで確認できる。

コマンド: `ldd ./test_numstats`

E.4.2 Linux で Clang を使用する場合

Linux 上で Clang を使用する場合も、基本的なオプションは GCC の場合と同じである。C 版は `clang`、C++版は `clang++` を使用して構築する。

C の場合: `clang -std=c17 -O2 -fPIC -shared numstats.c -o libnumstats.so`

C++の場合: `clang++ -std=c++17 -O2 -fPIC -shared numstats.cpp -o libnumstats.so`

利用側プログラムは、次のように構築する。

コマンド: `clang -std=c17 -O2 test_numstats.c -L. -lnumstats -Wl,-rpath,'$ORIGIN' \`
`-o test_numstats`

Clang のコンパイラドライバは、ここで使用している GCC 形式のオプションを受け付ける。したがって、`-fPIC`、`-shared`、`-L`、`-l` および `-Wl,` の意味は GCC の場合と同じである。生成された共有オブジェクトと実行ファイルは、GCC の場合と同様に `nm` および `ldd` で確認できる。

E.4.3 macOS で Clang を使用する場合

macOS では、標準の開発環境として Clang が使用される。macOS の動的ライブラリには `.dylib` という拡張子を付ける。C 版の動的ライブラリは、次のように構築する。

コマンド: `clang -std=c17 -O2 -dynamiclib numstats.c \`
`-Wl,-install_name,@rpath/libnumstats.dylib -o libnumstats.dylib`

C++版を構築する場合は、`clang++` を使用する。

コマンド: `clang++ -std=c++17 -O2 -dynamiclib numstats.cpp \`
`-Wl,-install_name,@rpath/libnumstats.dylib -o libnumstats.dylib`

`-dynamiclib` は、macOS 用の動的ライブラリを生成するための指定である。`-install_name` は、この動的ライブラリを参照するときに実行ファイルへ記録される名前を指定する。ここでは、実行時の探索経路によって置換される `@rpath/libnumstats.dylib` を指定している。

利用側プログラムは、次のように構築する。

```
コマンド: clang -std=c17 -O2 test_numstats.c -L. -lnumstats -Wl,-rpath,@loader_path \
-o test_numstats
```

-L. と -lnumstats によって、現在のディレクトリにある libnumstats.dylib をリンクする。@loader_path は、その動的ライブラリを参照しているバイナリが置かれているディレクトリを表す。この例では、実行プログラム test_numstats が置かれているディレクトリとなる。これを rpath として記録することにより、動的ライブラリを実行プログラムと同じディレクトリへ置くことができる。

実行ファイルに記録された動的ライブラリの情報は、次のコマンドで確認できる。

```
コマンド: otool -L test_numstats
```

また、動的ライブラリから公開されている外部シンボルは、次のように確認できる。

```
コマンド: nm -gU libnumstats.dylib
```

E.4.4 Windows の MSYS2 UCRT64 環境で GCC を使用する場合

MSYS2 の UCRT64 環境では、MinGW-w64 版の GCC を使用して Windows 用の DLL を構築できる。C 版の DLL は、次のように構築する。

```
コマンド: gcc -std=c17 -O2 -DNUMSTATS_BUILD -shared numstats.c \
-Wl,--out-implib,libnumstats.dll.a -o numstats.dll
```

C++版を構築する場合は、g++ を使用する。

```
コマンド: g++ -std=c++17 -O2 -DNUMSTATS_BUILD -shared numstats.cpp \
-Wl,--out-implib,libnumstats.dll.a -o numstats.dll
```

-DNUMSTATS_BUILD は、プリプロセッサマクロ NUMSTATS_BUILD を定義する指定である。これにより、numstats.h の NUMSTATS_API が __declspec(dllexport) に置き換えられ、numstats_summary が DLL から公開される。-shared は DLL を生成するための指定である。

--out-implib は、DLL と同時にインポートライブラリを生成するリンカオプションである。この例では、DLL 本体として numstats.dll、インポートライブラリとして libnumstats.dll.a が生成される。DLL には関数の実装が格納され、インポートライブラリは利用側プログラムをリンクする際に使用される。

利用側プログラムは、次のように構築する。

```
コマンド: gcc -std=c17 -O2 test_numstats.c -L. -lnumstats -o test_numstats.exe
```

-L. によって現在のディレクトリを探索対象に加え、-lnumstats によって、この例では libnumstats.dll.a がリンクされる。利用側では NUMSTATS_BUILD を定義しないため、NUMSTATS_API は __declspec(dllimport) となる。実行時には numstats.dll が必要になるため、この例では test_numstats.exe と同じディレクトリに置く。

DLL の公開シンボルと実行ファイルの依存関係は、次のように確認できる。

```
コマンド: objdump -p numstats.dll | grep numstats_summary
```

```
コマンド: objdump -p test_numstats.exe | grep "DLL Name"
```

E.4.5 Windows の MSYS2 CLANG64 環境で Clang を使用する場合

MSYS2 の CLANG64 環境では、Clang を使用して Windows 用の DLL を構築できる。使用するコマンド名は異なるが、オプションと生成されるファイルの構成は UCRT64 環境の GCC とほぼ同じである。C 版は次のように構築する。

```
コマンド: clang -std=c17 -O2 -DNUMSTATS_BUILD -shared numstats.c \
-Wl,--out-implib,libnumstats.dll.a -o numstats.dll
```

C++版では clang++ を使用する。

```
コマンド: clang++ -std=c++17 -O2 -DNUMSTATS_BUILD -shared numstats.cpp \
-Wl,--out-implib,libnumstats.dll.a -o numstats.dll
```

利用側プログラムは、次のように構築する。

コマンド: `clang -std=c17 -O2 test_numstats.c -L. -lnumstats -o test_numstats.exe`

-DNUMSTATS_BUILD, -shared, --out-implib, -L および -l の役割は、UCRT64 環境で GCC を使用する場合と同じである。実行時には numstats.dll を test_numstats.exe と同じディレクトリへ置く。公開シンボルと依存する DLL についても、GCC の場合と同様に objdump で確認できる。

E.4.6 Windows で MSVC を使用する場合

MSVC のコマンドラインツールを使用する場合は、Visual Studio の開発者用コマンドプロンプトを起動する。64 ビット版を構築する場合は、x64 Native Tools Command Prompt など、64 ビット用の環境が設定されたコマンドプロンプトを使用する。

C 版の DLL は、次のように構築する。

コマンド: `cl /nologo /std:c17 /utf-8 /O2 /DNUMSTATS_BUILD /LD numstats.c /Fe:numstats.dll`

C++ 版の DLL は、次のように構築する。

コマンド: `cl /nologo /std:c++17 /utf-8 /EHsc /O2 /DNUMSTATS_BUILD /LD numstats.cpp ^
/Fe:numstats.dll`

/std:c17 と /std:c++17 は、使用する言語規格を指定する。MSVC はファイルの拡張子によって C と C++ を判別するため、numstats.c は C として、numstats.cpp は C++ としてコンパイルされる。/utf-8 は、ソースファイルの文字コードと実行文字コードに UTF-8 を使用するための指定である。/EHsc は、C++ の例外処理方式を指定する。

/DNUMSTATS_BUILD は NUMSTATS_BUILD マクロを定義し、NUMSTATS_API を __declspec(dllexport) にする。/LD は実行ファイルではなく DLL を生成するための指定である。/Fe: は生成する DLL の名前を指定する。このコマンドを実行すると、numstats.dll と同時に、利用側プログラムのリンクに使用する numstats.lib が生成される。

利用側プログラムは、生成されたインポートライブラリを指定して構築する。

コマンド: `cl /nologo /std:c17 /utf-8 /O2 test_numstats.c numstats.lib ^
/Fe:test_numstats.exe`

利用側では NUMSTATS_BUILD を定義しないため、公開関数の宣言には __declspec(dllimport) が適用される。numstats.lib はリンク時に使用され、実行時には numstats.dll が必要になる。この例では、DLL を test_numstats.exe と同じディレクトリへ置く。

DLL から公開されているシンボルは、次のように確認できる。

コマンド: `dumpbin /exports numstats.dll`

出力に numstats_summary が含まれていれば、関数が正しく公開されている。また、実行ファイルが依存する DLL は、次のように確認できる。

コマンド: `dumpbin /dependents test_numstats.exe`

■ 実行例: Windows 上の MSYS2 の UCRT64 環境での作業

例. ライブラリ (C 言語版) の構築と、利用側プログラムのコンパイルおよびリンク

```
$ gcc -std=c17 -O2 -shared -DNUMSTATS_BUILD numstats.c \ [Enter]
-Wl,--out-implib,libnumstats.dll.a -o numstats.dll [Enter] ←ライブラリ構築作業
$ gcc -std=c17 -O2 test_numstats.c -L. -lnumstats -o test_numstats.exe [Enter]
↑利用側プログラムのコンパイルとリンク
```

例. test_numstats.exe の実行 (先の例の続き)

```
$ ./test_numstats [Enter]
count      : 25
sum        : 0.009900
minimum    : -1.981400
maximum    : 1.944600
mean       : 0.000396
variance    : 0.999221
```

E.5 公開インターフェース設計上の注意

動的ライブラリでは、公開関数を通して、独立して構築されたライブラリと利用側プログラムとの間でデータを受け渡す。公開インターフェースの設計が不適切であると、ソースコード上では正しく見える場合でも、実行時に誤動作することがある。特に、ライブラリを単独で更新できるようにするためには、ABI の互換性、メモリの管理および C++ 例外の扱いに注意する必要がある。

E.5.1 ABI 互換性の維持

ABI には、関数のシンボル名、引数と戻り値の受渡し方法、データ型の大きさや配置などが含まれる。動的ライブラリを更新した後も利用側プログラムから同じように呼び出すためには、公開インターフェースの ABI を維持しなければならない。

公開済みの関数について、引数や戻り値の型、それらの個数および順序を変更すると、既存の利用側プログラムとの互換性が失われる。関数名を変更した場合も、利用側プログラムが必要なシンボルを見つけられなくなる。特に、シンボル名が同じままでも、引数や戻り値の型の不一致が実行時に検出されるとは限らないため、誤動作につながる可能性がある。このような変更を行った場合は、原則として利用側プログラムも新しいヘッダファイルを用いて再構築する必要がある。

公開する構造体についても注意が必要である。構造体のメンバを追加、削除または並べ替えると、構造体の大きさや各メンバの配置位置が変化する可能性がある。古いヘッダファイルを用いて構築されたプログラムは、変更前の大きさと配置を前提として領域を確保し、構造体へアクセスする。このため、新しいライブラリが変更後の構造に従って書き込むと、値を正しく受け渡せないだけでなく、利用側が用意した領域の外へ書き込む可能性もある。前項の `numstats_result` についても、メンバの型、個数および順序は公開 ABI の仕様の一部となる。

また、C++ のクラス、標準ライブラリのコンテナおよび文字列型などは、コンパイラや標準ライブラリの実装によってバイナリ表現が異なる可能性がある。このため、広く利用する動的ライブラリでは、これらを公開関数の引数や戻り値として直接使用せず、C 言語の基本型、ポインタおよび単純な構造体を用いる方が安全である。

C ABI を採用しても、異なる CPU や OS の間で同じバイナリをそのまま利用できるわけではない。ライブラリと利用側プログラムは、同じ対象環境に合わせて構築する必要がある。C ABI を採用する目的は、各対象環境において、処理系やプログラミング言語の違いを越えて利用しやすい公開インターフェースを作ることである。

E.5.2 メモリの管理とオブジェクトの寿命

ポインタを介してデータを受け渡す場合は、そのポインタが指すメモリを誰が所有し、いつまで有効であるかを明確にする必要がある。ライブラリが、関数内に定義した自動変数へのポインタを利用側へ返してはならない。自動変数は関数の終了時に破棄されるため、返されたポインタは無効になる。

動的に確保したメモリを受け渡す場合は、確保と解放を行う側を決めておく。基本的には、メモリを確保した側が解放する形にするとよい。ライブラリがメモリを確保して利用側へ返す必要がある場合は、そのメモリを解放するための関数も同じライブラリから提供する。これにより、利用側が異なる方法でメモリを解放することによる問題を避けられる。

前項の数値集計ライブラリでは、入力配列と結果格納用構造体の両方を利用側で用意している。ライブラリは、入力配列を参照して計算を行い、受け取った構造体へ結果を書き込むだけである。ライブラリ内では動的なメモリ確保を行わず、入力配列を指すポインタを関数終了後まで保持しない。このような設計は、メモリの所有者と寿命が明確であり、異なる言語からも利用しやすい。

C++ のオブジェクトをライブラリ内部で生成する場合も、その生成と破棄を同じライブラリ内で行うことが望ましい。外部へオブジェクトを直接公開する必要がある場合には、実体を公開せず、識別用の不透明なポインタを受け渡し、生成用関数と破棄用関数を提供する方法が用いられる。ただし、このような設計は必要になった段階で検討すればよく、単純な数値計算ライブラリでは、利用側が用意した領域へ結果を書き込む方式で十分である。

E.5.3 C++例外の扱い

C++で実装した関数では、処理中に例外が発生する可能性がある。しかし、C言語にはC++例外を処理する仕組みがなく、他のプログラミング言語からも、通常はC++例外を直接扱うことができない。また、例外処理のバイナリ上の仕組みは、コンパイラや実行環境に依存する。このため、C ABIとして公開した関数から、C++例外をそのまま外部へ伝播させてはならない。

例外が発生する可能性のある処理を公開関数内で行う場合は、公開関数の内部で例外を捕捉し、エラーコードなど、利用側が解釈できる形式へ変換する。利用側では、関数の戻り値を調べ、処理が成功した場合にだけ計算結果を参照する。この方法であれば、C、Python、C#など、例外処理の仕組みが異なる言語から同じ方法でエラーを確認できる。

公開関数へ `noexcept` を指定するだけでは、例外をエラーコードへ変換したことにはならない。`noexcept` を指定した関数から例外が外へ出ようとする、`std::terminate` が呼び出され、通常はプログラムが終了する。したがって、処理の失敗を利用側へ通知する必要がある場合は、公開関数の内部で例外を捕捉し、定められた戻り値へ変換する必要がある。

動的ライブラリの内部では、C++の例外やオブジェクトの自動的な破棄を通常どおり利用できる。重要なのは、例外をライブラリの公開境界より外へ出さないことである。すなわち、内部ではC++の機能を活用し、外部との間では単純なC言語形式の値とエラーコードを受け渡す構成にするとよい。

F gnuplot

gnuplot は、数式で表された関数や数値データをグラフとして描画するためのソフトウェア

²¹⁴ である。コマンドを対話的に入力して使用できるほか、あらかじめ記述したスクリプトによってグラフを作成することもできる。描画できるグラフは、折れ線グラフ、散布図、棒グラフ、ヒストグラム、3 次元グラフなど多彩である。

gnuplot は、数値計算の結果や実験データを手早く可視化するための、独立したグラフ描画ツールであり、その歴史は長く、1980 年代から開発が続いている。そのため、大学や研究機関、技術系の開発現場などで長く用いられてきた実績がある。

gnuplot は、テキストファイルに保存された数値データを簡単なコマンドで描画できるという手軽さを持っている。また、描画したグラフを、PNG などのビットマップ形式や、SVG、EPS、PDF などのベクター形式の画像ファイルとして保存することもできる。例えば、C や C++ で作成したプログラムから数値データをテキストファイルに出力し、そのファイルを gnuplot で読み込めば、プログラムの計算結果をすぐにグラフとして確認し、必要に応じて文書や発表資料などに利用できる形式で保存することができる。従って、C や C++ によるプログラミングにおいて、数値計算、シミュレーション、データ処理などの結果を可視化するための補助ツールとして、gnuplot は扱いやすい。

本書では、C や C++ のプログラムが出力した数値データを可視化するための道具として、gnuplot の基本的な使い方を解説する。扱う範囲は、データファイルの読み込み、グラフの描画、画像ファイルへの保存を中心とし、gnuplot 独自のスクリプトを用いて数値計算やデータ処理を行う方法については扱わない。

gnuplot の公式情報は、公式 Web サイト

<https://www.gnuplot.info/>

で公開されている。また、gnuplot のソースコードや Windows 用のインストールパッケージも SourceForge の gnuplot プロジェクト

<https://sourceforge.net/projects/gnuplot/>

で配布されている。

Windows の一般的な 64 ビット環境では、最新版のフォルダから、ファイル名に win64 を含む .exe 形式のインストーラをダウンロードして実行する。バージョン番号やファイル名は変わることがあるので、実際に入手する際は、最新版のフォルダ内にある Windows 64 ビット用のインストーラを選択する。

macOS では、Homebrew を利用してインストールする方法が広く用いられている。Homebrew を導入済みであれば、端末で次のように入力してインストールする。

```
$ brew install gnuplot
```

Linux では、使用しているディストリビューションのパッケージ管理システムからインストールするのが一般的である。例えば、Ubuntu や Debian 系の環境では、次のようなコマンド操作でインストールする。

```
$ sudo apt update
```

```
$ sudo apt install gnuplot
```

インストール作業の後は、端末やコマンドプロンプトで「gnuplot」と入力し、gnuplot が起動することを確認すること。gnuplot が起動すると、起動時のメッセージの後に

```
gnuplot>
```

と表示され、コマンド入力待ちの状態になる。終了するには、gnuplot のプロンプトで exit と入力する。

注意) コマンドプロンプトから gnuplot を起動する場合には、インストール時に、gnuplot の実行ファイルが置かれるディレクトリを PATH へ追加する設定²¹⁵を選択しておくといよい。

本書では、Windows 環境での利用を中心に説明するが、基本的な gnuplot のコマンドは macOS や Linux でもほぼ同様に使用できる。ただし、グラフを表示する画面や、画像ファイルとして保存する際に利用できる形式は、OS やインストールされた描画端末の種類によって異なることがある。

²¹⁴ 名称に「gnu」が含まれているが、Free Software Foundation (FSF) や GNU Project の一部ではない。

²¹⁵ コマンドサーチパスへの登録作業

F.1 データファイルの形式

gnuplot では、数値データを記録したテキストファイルを読み込んでグラフを描画することができる。この際に使用するデータファイルは、表計算ソフトウェアで扱う表形式のデータに近いものである。例えば、多くの表計算ソフトウェアでは、複数の列を持つデータをタブ区切りのテキストファイルや CSV ファイル²¹⁶ として保存することができる。gnuplot の場合もこれらと同様に、1 行を 1 件のデータとし、その中に複数の値を列として並べた形式のデータを扱うことができる。

gnuplot で最も基本となるのは、空白またはタブで値を区切ったテキストファイルである。例えば、次のようなデータファイルを考える。

```
0 0
1 1
2 4
3 9
4 16
5 25
```

このデータでは、各行に 2 つの数値が記録されている。第 1 列はグラフの横軸の値、第 2 列はグラフの縦軸の値として扱うことができる。すなわち、このデータは、点 (0,0), (1,1), (2,4), (3,9), (4,16), (5,25) を表している。

C や C++ のプログラムから数値データを出力する場合も、上のように、1 行に 1 件のデータを出力し、複数の値を空白で区切って並べる。例えば、横軸の値と縦軸の値を空白で区切って出力しておけば、gnuplot でそのファイルを読み込んでグラフ化することができる。

なお、gnuplot は、CSV 形式のあらゆるデータを汎用的に処理するための機能を備えているわけではないが、各行に並べた数値を単純にコンマで区切ったデータであれば読み込むことができる。このような CSV ファイルを読み込む場合には、データの区切り文字としてコンマを使用することを gnuplot に指示する必要がある。この方法については、後で説明する。

F.1.1 データファイルの区切り文字の指定

gnuplot では、既定ではスペースやタブなどの空白文字が、データファイルの列の区切りとして扱われる。一方、CSV 形式のように、コンマで値を区切ったデータファイルを読み込む場合は、次のようなコマンドで区切り文字としてコンマを使用することを gnuplot に指示する必要がある。

書き方： `set datafile separator "区切り文字"`

例えば、コンマ区切りの CSV ファイルを読み込む場合は、次のように指定する。

```
set datafile separator ","
```

この設定を行った後は、gnuplot はデータファイル中のコンマを列の区切りとして扱う。

区切り文字の設定を解除し、空白文字を区切りとして扱う既定の状態に戻すには、次のように `whitespace` を指定する。

```
set datafile separator whitespace
```

以下にサンプルプログラムを示して、実際にグラフをプロットする流れを解説する。次に示すプログラム `gnuplot01.c` は $-2\pi \leq x \leq 2\pi$ に対する $\sin(x)$, $\cos(x)$ の値をテキストファイルに出力するものである。

プログラム： `gnuplot01.c`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 int main( int ac, char **av ) {
6     const double PI = 3.141592653589793;
7     const double xmin = -2.0 * PI;
8     const double xmax = 2.0 * PI;
9     int n;
```

²¹⁶CSV は comma-separated values の略であり、コンマで値を区切るテキスト形式のデータである。その一般的な形式は RFC 4180 で説明されている。

```

10
11     if ( ac != 3 ) {
12         fprintf(stderr,"usage: gnuplot01 NumOfPoints OutFileName\n");
13         return 1;
14     } else {
15         n = strtol(av[1],NULL,10);
16         if ( n < 3 || n > 100 ) {
17             fprintf(stderr,"It must be 2 < NumOfPoints <= 100.\n");
18             return 1;
19         }
20     }
21
22     FILE *fp;
23     double x, y1, y2;
24     int i;
25
26     fp = fopen(av[2], "w");
27     if ( fp == NULL ) {
28         fprintf(stderr,"ファイルを作成できませんでした. \n");
29         return 1;
30     }
31
32     for (i = 0; i < n; i++) {
33         x = xmin + (xmax - xmin) * i / (n - 1);
34         y1 = sin(x);
35         y2 = cos(x);
36         fprintf(fp, "%6.3f\t%6.3f\t%6.3f\n", x, y1, y2);
37     }
38
39     fclose(fp);
40 }

```

このプログラムは、実行時の第1引数にデータの個数を、第2引数に出力ファイルの名前を取る。このプログラムをコンパイルして実行する例を次に示す。

例. gnuplot01.c のコンパイルと実行

```

$ gcc gnuplot01.c -o gnuplot01 -lm  ←コンパイル
$ ./gnuplot01 100 gnuplot01.dat  ←データ点の個数を100, 出力ファイル名を"gnuplot01.dat"として実行

```

この処理によって、gnuplot01.dat というデータファイルが作成され、100 個のデータ点が格納される。このファイルには、横軸の値 x と、それに対応する $\sin(x)$, $\cos(x)$ の値が、タブ区切りの3列データとして記録されている。

データファイル：gnuplot01.dat (先頭5行分)

1	-6.283	0.000	1.000
2	-6.156	0.127	0.992
3	-6.029	0.251	0.968
4	-5.902	0.372	0.928
5	-5.775	0.486	0.874

このデータを gnuplot で描画するには、gnuplot を起動し、次のように入力する。

```
plot "gnuplot01.dat" using 1:2 with lines
```

ここで、"gnuplot01.dat" は読み込むデータファイルの名前である。using 1:2 は、第1列を横軸の値、第2列を縦軸の値として使用することを表している。また、with lines は、得られた点を線で結んで描画することを指定する**描画オプション**である。

このコマンドにより、 -2π から 2π までの範囲における $\sin(x)$ のグラフが描画される。実際に、図13のようなウィンドウが表示される。

複数のグラフを同じ座標面にプロットする場合には、次のように複数の描画指定をコンマで区切って列挙する。

```
plot "gnuplot01.dat" using 1:2 with lines, "gnuplot01.dat" using 1:3 with lines
```

この結果、図14のようなウィンドウが表示される。

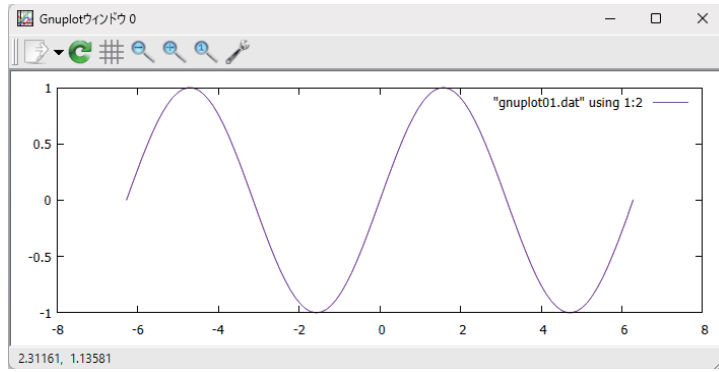


図 13: gnuplot01.dat 内の $\sin(x)$ のデータをプロットした結果

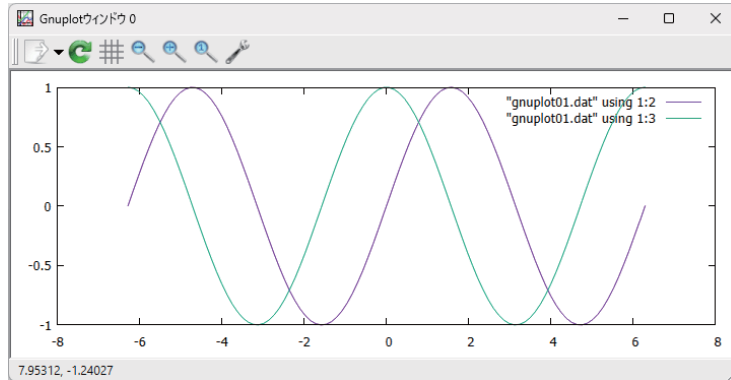


図 14: gnuplot01.dat 内の 2 つの関数をプロットした結果

F.2 描画に関する各種のコマンド

gnuplot では、表 118 に示すようなコマンドによって、作図に関する設定ができる。

表 118: プロット用の各種コマンド (一部)

設定内容	コマンド例	意味
グラフのタイトル	<code>set title "sin(x)"</code>	グラフ上部にタイトルを表示する
横軸のラベル	<code>set xlabel "x"</code>	横軸の説明 (軸ラベル) を表示する
縦軸のラベル	<code>set ylabel "sin(x)"</code>	縦軸の説明 (軸ラベル) を表示する
グリッドの表示	<code>set grid</code>	目盛に合わせた補助線を表示する
x 軸の表示	<code>set xzeroaxis</code>	$y = 0$ を通る横方向の軸線を表示する *1
y 軸の表示	<code>set yzeroaxis</code>	$x = 0$ を通る縦方向の軸線を表示する *1
横軸の範囲	<code>set xrange [-7:7]</code>	横軸の表示範囲を指定する
縦軸の範囲	<code>set yrange [-1.2:1.2]</code>	縦軸の表示範囲を指定する
出力形式	<code>set terminal pngcairo size 800,600</code>	画像ファイルの形式や寸法を指定する
出力ファイル	<code>set output "graph.png"</code>	保存する画像ファイル名を指定する
出力の終了	<code>set output</code>	画像ファイルへの出力を終了する
現在の端末を調べる	<code>show terminal</code>	現在使用中の出力端末を表示する
Qt 画面への表示	<code>set terminal qt</code>	出力端末を画面表示用の Qt 端末に切り替える *2
設定の初期化	<code>reset</code>	端末や出力先などを除く描画設定を既定の状態に戻す

*1 x 軸, y 軸にはグラフ描画時と同様の各種のオプション指定 (太さ, 色, 線種など) が可能

*2 本書の解説に使用した環境では Qt 端末画面が既定

plot コマンドをはじめとするグラフ描画コマンドには、表 119 に示すような**描画オプション**を指定することができる。

表 119: 描画コマンド内で使用する主な描画オプション (一部)

設定内容	描画オプション例	意味
折れ線で描画	<code>with lines</code>	データ点を線で結んで描画する
点で描画	<code>with points</code>	データ点を点として描画する
点と線で描画	<code>with linespoints</code>	データ点を点で示し、線で結んで描画する
線の色	<code>linecolor rgb "blue"</code>	線の色を指定する (省略形: <code>lc</code>)
線の太さ	<code>linewidth 2</code>	線の太さを指定する (省略形: <code>lw</code>)
線の種類	<code>dashtype "-"</code>	破線などの線種を指定する (省略形: <code>dt</code>) *
凡例	<code>title "sin(x)"</code>	グラフの凡例に表示する名前を指定する
凡例を消す	<code>notitle</code>	その描画対象の凡例を表示しない

* `dashtype` では、数値によって定義済みの線種を指定することもできる。

F.2.1 グラフの色の指定

gnuplot では、線や点の色を指定することができる。色の指定には、`"blue"` のような色名を用いる方法と、`"#0000ff"` のような 16 進数による RGB による色コードを用いる方法がある。例えば、次のようにすると、線を青色で描画することができる。

```
plot "gnuplot01.dat" using 1:2 with lines lc rgb "blue"
```

また、次のように 16 進数の RGB 色コードで指定することもできる。

```
plot "gnuplot01.dat" using 1:2 with lines lc rgb "#0000ff"
```

ここで、`lc` は `linecolor` の省略形であり、線の色を指定することを意味する。また、`rgb` は、赤 (red)、緑 (green)、青 (blue) の 3 成分によって色を指定することを表している。16 進数による RGB 色コードは、Web ページの色指定などで広く用いられている形式と同じであり、`"#rrggbb"` の形で指定する。ここで、`rr`、`gg`、`bb` は、それぞれ赤、緑、青の強さを 16 進数 00~ff で表したものである。

gnuplot で使用できる色名は、HTML で用いられる色名の体系²¹⁷ と完全に同じではない。色名を用いる場合は、gnuplot が認識する色名を指定する必要がある。利用可能な色名は、gnuplot のプロンプトで次のように入力して確認できる。

```
show colornames
```

確実に色を指定したい場合は、色名ではなく 16 進数の RGB 色コードを用いるとよい。代表的な色指定の例を表 120 に示す。

表 120: 代表的な色指定

色	色 名	色コード	色	色 名	色コード
赤	<code>"red"</code>	<code>"#ff0000"</code>	シアン	<code>"cyan"</code>	<code>"#00ffff"</code>
緑	<code>"green"</code>	<code>"#00ff00"</code>	マゼンタ	<code>"magenta"</code>	<code>"#ff00ff"</code>
青	<code>"blue"</code>	<code>"#0000ff"</code>	黄	<code>"yellow"</code>	<code>"#ffff00"</code>
黒	<code>"black"</code>	<code>"#000000"</code>	白	<code>"white"</code>	<code>"#ffffff"</code>

■ アルファ値による透過性の指定

画像処理や Web 関連の色指定では、RGB の 3 成分に**アルファ値** (alpha value) を加えて、色の透過性を表すことがある。一般的な RGBA 形式では、アルファ値は**不透明度**を表すものとして扱われ、値が小さいほど透明、大きいほど不透明となる場合が多い。例えば、8 ビットのアルファ値では 00 が完全透明、ff が完全不透明に対応する。

gnuplot でもアルファ値を含む色を指定することができるが、16 進数による色指定では、一般的な RGBA 形式とは異なる規則が用いられるので注意が必要である。通常の RGB 色コードは

```
"#RRGGBB"
```

という形式であるが、gnuplot では、その先頭にアルファ値を表す 2 桁を加えた

²¹⁷いわゆる Web カラー

"#AARRGGBB"

という形式である。ここで、AA がアルファ値、RR、GG、BB がそれぞれ赤、緑、青の成分を表す。

この "#AARRGGBB" 形式では、アルファ値は透過の程度を表しており、00 が完全に不透明、ff が完全に透明となる。すなわち、一般的な RGBA 形式でアルファ値を不透明度として扱う場合とは、値の意味が逆になっている。例えば赤色の場合は次のようになる。

```
"#00ff0000" 完全に不透明な赤
"#80ff0000" 半透明の赤
"#ffff0000" 完全に透明な赤
```

従って、通常の RGB 指定

```
lc rgb "#ff0000"
```

と、アルファ値 00 を付けた

```
lc rgb "#00ff0000"
```

は、ともに不透明な赤を表す。

参考) gnuplot におけるアルファ値の扱いは、指定方法によっても異なる。例えば、`rgbalpha` 指定による画像描画でアルファ値を独立したデータとして与える場合には、0 が完全透明、255 が完全不透明となり、上で説明した "#AARRGGBB" 形式とは逆の規則になる。このため、gnuplot でアルファ値を扱う場合は、単に「アルファ値が大きいほど透明（あるいは不透明）」と理解するのではなく、**どの形式でアルファ値を与えているかを確認**する必要がある。

`rgbalpha` 用いた通常の RGBA の解釈によるアルファ値の指定方法については本書では割愛²¹⁸ する。

アルファ値を含む色指定が実際に有効になるかどうかは、使用している出力端末に依存する。例えば、`pngcairo` や `pdfcairo` などの Cairo 系の出力端末では半透明表示を利用しやすいが、出力形式によっては期待通りに反映されない場合がある²¹⁹。従って、通常は "#RRGGBB" 形式や色名による指定を基本とし、透過性が必要な場合に限って、出力形式の対応状況を確認した上で "#AARRGGBB" 形式などを用いるとよい。

F.2.2 線の太さの指定

線の太さは、`linewidth` によって指定できる。省略形として `lw` を用いることもできる。例えば、次のようにすると、標準より太い線でグラフを描画する。

```
plot "gnuplot01.dat" using 1:2 with lines linewidth 2
```

同じ指定は、省略形を用いて次のように書くこともできる。

```
plot "gnuplot01.dat" using 1:2 with lines lw 2
```

`linewidth` に指定する値は、ポイント数やピクセル数のような絶対的な単位ではない。使用している出力端末における標準的な線幅を基準にした相対的な倍率である。すなわち、`lw 1` が標準的な太さであり、`lw 2` はその 2 倍の太さとして描画される。

ただし、実際の線の見え方は、使用する出力端末や、画像の解像度、表示時の拡大率などによって多少異なることがある。そのため、グラフを画像ファイルとして保存する場合は、目的に合う見た目になるように、`lw 1`、`lw 2`、`lw 3` などを試して調整するとよい。

F.2.3 線種の指定

単独の折れ線グラフを描画する場合は、通常、線は実線として表示される。しかし、複数のグラフを同じ座標平面に重ねて描画する場合は、線の色だけでなく、線種を変えることで各グラフを区別しやすくなる。特に、モノクロで印刷する場合や、色の違いが分かりにくい環境では、実線、破線、点線などを使い分けるとよい。

gnuplot では、線種の指定に `dashstyle` を用いる。省略形として `dt` を用いることもできる。例えば、次のようにすると、破線でグラフを描画する。

```
plot "gnuplot01.dat" using 1:2 with lines dashstyle "-"
```

²¹⁸用法がやや煩雑であり、本書で扱う範囲では必要性も低いため。

²¹⁹SVG など、ベクター形式でも透過性を扱える出力形式がある。

省略形を用いると、次のように書くこともできる。

```
plot "gnuplot01.dat" using 1:2 with lines dt "-"
```

文字による線種指定の例を表 121 に挙げる。

表 121: `dashtype` による線種指定の例

指定	線 種	組み合わせの例	解 説
"."	点線	"-."	破線と点線の組合せ
"-"	破線	".-"	点線と破線の組合せ
"_"	長い破線 (アンダースコア)	"._."	細かい点線
		"._."	点と長い破線の組合せ

`dashtype` には、番号を指定する方法もあるが、その場合は、使用する出力端末によって番号と線種の対応が異なることがある。これに対して、".", "-_", "_" などの文字を組み合わせる方法では、点線、破線、長い破線、およびそれらを組み合わせた線種を明示的に指定できる。なお、実線を指定する場合には `dt solid` と記述するほか、整数の 1 を用いて `dt 1` と記述することもできる。また、`xzeroaxis` や `yzeroaxis` のような軸線を黒い実線として表示したい場合には、`lt -1` を指定する。`lt` は `linetype` の省略形であり、`linetype -1` は、通常、前景色の実線を表す。

実際の線の見え方は、使用する出力端末によって多少異なることがあるので注意すること。

F.3 描画結果の画像ファイルへの保存

`gnuplot` では、描画したグラフをディスプレイに表示するだけでなく、画像ファイルとして保存することができる。そのためには、グラフの出力先を画像ファイル用の出力端末に切り替え、出力ファイル名を指定してから `plot` コマンドを実行する。

画像ファイルへの出力は、主に次の 3 段階で行う。

- (1) 出力形式の切り替え : `set terminal` 出力端末名 オプション
- (2) 出力ファイルの指定 : `set output` "出力ファイル名"
- (3) 出力ファイルのクローズ : `set output`

`set terminal` は、出力形式を指定するコマンドである。例えば、PNG 形式で出力する場合には `pngcairo` 出力端末を指定する。主な指定例を表 122 に挙げる。

表 122: `set terminal` で指定する主な出力形式 (一部)

端末名	主な出力形式	コマンド例
<code>pngcairo</code>	PNG 形式	<code>set terminal pngcairo size 800,600</code>
<code>jpeg</code>	JPEG 形式	<code>set terminal jpeg size 800,600</code>
<code>gif</code>	GIF 形式	<code>set terminal gif size 800,600</code>
<code>svg</code>	SVG 形式	<code>set terminal svg size 800,600</code>
<code>pdfcairo</code>	PDF 形式	<code>set terminal pdfcairo size 5in,3.5in</code>
<code>postscript</code>	EPS 形式	<code>set terminal postscript eps size 5in,3.5in</code>
<code>postscript</code>	PostScript 形式	<code>set terminal postscript size 5in,3.5in</code>
<code>dumb</code>	テキストによる簡易表示	<code>set terminal dumb</code>

`set output` は、出力先となるファイル名を指定するコマンドである。また、ファイルへの出力を終了するには、引数を付けずに `set output` を実行する。

例えば、先に作成したデータファイル `gnuplot01.dat` を読み込み、PNG 形式の画像ファイル `gnuplot01.png` に保存するには、`gnuplot` で次のように入力する。

```
set terminal pngcairo size 800,600
set output "gnuplot01.png"
plot "gnuplot01.dat" using 1:2 with lines lw 2 notitle
set output
```

1行目の `set terminal pngcairo size 800,600` は、PNG 形式の画像を横 800 ピクセル、縦 600 ピクセルの大きさと作成する指定である。2行目の `set output "gnuplot01.png"` は、出力先の画像ファイル名を指定している。その後、`plot` コマンドでグラフを描画すると、画面ではなく、指定した画像ファイルにグラフが保存される。最後の `set output` は、画像ファイルへの出力を終了する指定である。

EPS 形式で保存する場合は、`postscript` 出力端末に `eps` オプションを指定する。例えば、横 3.2 インチ、縦 1.8 インチの EPS ファイルを作成するには、次のように指定する。

```
set terminal postscript eps size 3.2in,1.8in
set output "gnuplot01.eps"
plot "gnuplot01.dat" using 1:2 with lines lw 2 notitle
set output
```

PNG 形式では画像の大きさをピクセル数で指定することが多い。一方、EPS や PDF などの形式では、インチやセンチメートルなどの物理的な寸法で指定することが多い。出力形式によって指定できるオプションや寸法の扱いが異なるため、必要に応じて確認するとよい。

注意) `postscript` 系の出力端末の場合、カラーを有効にするには、`set terminal` コマンドに `color` オプションを与え、
`set terminal postscript eps color size 3.2in,1.8in`
などとする。

なお、画像ファイルへの出力を行った後、続けて画面上にグラフを表示したい場合は、出力端末を画面表示用のものに戻す必要がある。現在使用している出力端末は、次のコマンドで確認できる。

```
show terminal
```

例えば、画面表示用の端末として `qt` が使用できる環境では、次のようにして画面表示用の端末に戻すことができる。

```
set terminal qt
```

ただし、画面表示用の出力端末は環境によって異なる。Windows では `windows`、Linux では `qt` や `x11` などが使われる場合がある。従って、画像ファイルへの出力を行う前に `show terminal` を実行し、それまで使用していた画面表示用の出力端末を確認しておくといよい。

F.4 ファイルに記述されたコマンドの実行

`load` コマンドを使用すると、テキストファイルに記述された一連のコマンドを読み込んで実行することができる。

書き方： `load "ファイルのパス"`

指定したファイルを読み込み、記述されているコマンドを先頭から順に実行する。例えば、`gnuplot` のコマンドを記述したファイル `gnuplot01.plt` を読み込んで実行するには、

```
load "gnuplot01.plt"
```

と入力する。

スクリプト：gnuplot01.plt

```
1  reset
2  # 3.2インチ×1.8インチサイズのEPSに描画する設定
3  set terminal postscript eps color size 3.2in,1.8in
4  set output "gnuplot01_out.eps"           # 出力ファイルの指定
5  set xlabel "x"                           # 横軸ラベルの設定
6  set ylabel "y"                           # 縦軸ラベルの設定
7  set title "y = sin(x), y = cos(x)"        # グラフタイトルの設定
8  set grid                                 # グリッドの表示
9  set xzeroaxis lt -1 lw 1                 # 横軸の表示
10 set yzeroaxis lt -1 lw 1                 # 縦軸の表示
11 set xrange [-6.28:6.28]                  # 横軸の範囲
12 set yrange [-1.2:1.2]                    # 縦軸の範囲
13 # 描画処理の実行
14 plot "gnuplot01.dat" using 1:2 with lines \
15      dt 1 lw 1.5 lc rgb "red" title "sin(x)", \
16      "gnuplot01.dat" using 1:3 with lines \
17      dt 1 lw 1.5 lc rgb "blue" title "cos(x)"
18 set output                               # 出力ファイルを閉じる
```

スクリプトファイルの先頭で `reset` を実行している。これは、それ以前に行われた描画設定の多くを既定の状態に戻し、以前の設定による影響を受けにくくするためである。なお、`reset` では出力端末や出力先などは初期化されないが、このスクリプトでは、それらが続く命令で明示的に設定している。

スクリプトファイルの末尾では、`set terminal qt` を実行し、出力端末を画面表示用の Qt 端末に切り替えている。ただし、この指定を使用できるのは、Qt 端末が利用可能な環境に限られる。この行でエラーが発生する場合には、行頭に `#` を記述して無効にするか、使用している環境に適した画面表示用の出力端末を指定するとよい。

このスクリプトの実行の結果、データファイル `gnuplot01.dat` の内容をプロットしたものが、画像ファイル `gnuplot01_out.eps` (図 15) に出力される。

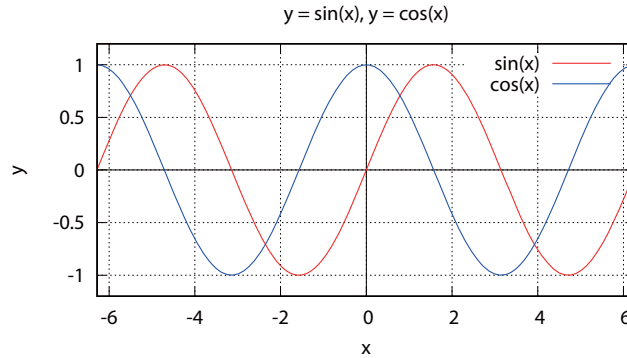


図 15: `gnuplot01.plt` の実行により作成された `gnuplot01_out.eps`

F.5 グラフに日本語を使用する際の注意事項

これまでに示した例では、グラフのタイトル、軸ラベル、凡例などに英数字を用いてきたが、日本語を使用することもできる。ただしその場合は、文字コード、フォント、出力形式などに注意する必要がある。

特に、グラフを画像ファイルとして保存する場合、出力形式によって日本語の扱いやすさが異なる。英数字だけのグラフであれば、EPS 形式や PNG 形式などを比較的容易に利用できるが、日本語を含むグラフでは、PostScript/EPS 形式でフォントの扱いが環境に依存しやすく、日本語が正しく表示されない場合がある。従って、日本語のタイトル、軸ラベル、凡例などを含むグラフをベクター形式で作成する場合には、PDF 形式で出力できる `pdfcairo` 端末が比較的扱いやすい。また、ビットマップ画像として保存する場合には、`pngcairo` 端末が比較的扱いやすい。

日本語を含むグラフを作成する際の基本的な注意点を次に挙げる。

- スクリプトファイルを UTF-8 のエンコーディングで保存する。
- `gnuplot` のスクリプト中で `set encoding utf8` を指定する。
- 日本語を表示できるフォントを指定する。
- 出力端末として `pdfcairo` や `pngcairo` を使用する。
- EPS 形式では、日本語フォントの扱いが環境に依存しやすいことに注意する。

先に作成したデータファイル `gnuplot01.dat` を用いて、日本語のタイトル、軸ラベル、凡例を持つグラフを PDF 形式で出力する例 (`gnuplot01j.plt`) を示す。この例では、出力端末として `pdfcairo` を使用し、日本語フォントとして Windows 環境で一般的に利用できる `Meiryo` を指定している。ただし、利用できるフォントは環境によって異なるため、`Meiryo` が利用できない場合には、その環境にインストールされている日本語フォントの名前に置き換える必要がある。

スクリプト: `gnuplot01j.plt`

```
1  reset
2  set encoding utf8
3  # 3.2インチ×1.8インチサイズのPDFに描画する設定
4  set terminal pdfcairo color size 3.2in,1.8in font "Meiryo,10"
5  set output "gnuplot01_outj.pdf"          # 出力ファイルの指定
6
7  set xlabel "定義域 x"                    # 横軸ラベルの設定
8  set ylabel "値域 y"                      # 縦軸ラベルの設定
9  set title "三角関数 y = sin(x), y = cos(x)" # グラフタイトルの設定
10 set grid                                # グリッドの表示
```

```

11 set xzeroaxis lt -1 lw 1          # 横軸の表示
12 set yzeroaxis lt -1 lw 1          # 縦軸の表示
13 set xrange [-6.28:6.28]           # 横軸の範囲
14 set yrange [-1.2:1.2]             # 縦軸の範囲
15
16 # 描画処理の実行
17 plot "gnuplot01.dat" using 1:2 with lines \
18     dt 1 lw 1.5 lc rgb "red" title "正弦関数 sin(x)", \
19     "gnuplot01.dat" using 1:3 with lines \
20     dt 1 lw 1.5 lc rgb "blue" title "余弦関数 cos(x)"
21 set output                          # 出力ファイルを閉じる
22 set terminal qt                     # 出力を画面表示に戻す設定

```

この結果、データファイル gnuplot01.dat の内容をプロットしたものが、PDF ファイル gnuplot01_outj.pdf (図 16) に出力される。

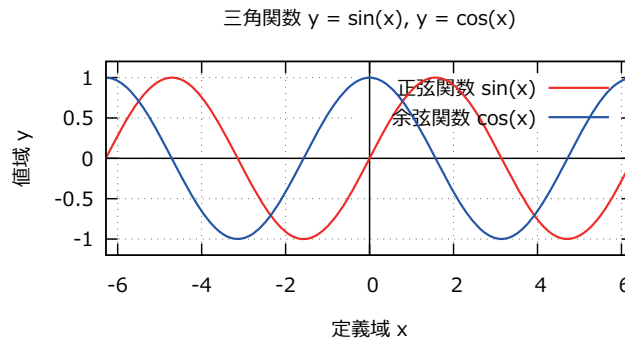


図 16: gnuplot01j.plt の実行により作成された日本語ラベル付きグラフ

F.6 各種のグラフ

gnuplot では、plot コマンドの描画オプションを変更することで、同じ形式のデータファイルから、さまざまな形式のグラフを作成することができる。例えば、with lines を指定するとデータ点を線で結んだ折れ線グラフになり、with points を指定するとデータ点を記号で示す点グラフ（マーカープロット）になる。また、with boxes を指定すると棒グラフを作成することができる。

基本的な考え方は、これまでに説明した折れ線グラフの場合と同じである。すなわち、データファイルの各行を 1 件のデータとして読み込み、using によって横軸と縦軸に対応する列を指定する。その上で、with lines、with points、with boxes などの描画オプションによって、データをどのような形で表示するかを指定する。

F.6.1 マーカープロット（点グラフ）

データ点を線で結ばず、記号だけで表示するグラフを**マーカープロット**、あるいは**点グラフ**と呼ぶことがある。gnuplot では、このようなグラフを描画するために、plot コマンドの描画オプションとして with points を指定する。

例えば、データファイル gnuplot01.dat の第 1 列を横軸、第 2 列を縦軸として、データ点を記号で表示するには、次のように指定する。

```
plot "gnuplot01.dat" using 1:2 with points
```

この場合、データファイルの各行は 1 つの点に対応する。すなわち、折れ線グラフの場合と同じく、using 1:2 によって第 1 列を横軸、第 2 列を縦軸として読み込む。ただし、with lines ではなく with points を指定しているため、データ点は線で結ばれず、各点が記号で表示される。

点の種類は pointtype、または省略形の pt によって指定する。また、点の大きさは pointsize、または省略形の ps によって指定する。ps に指定する値は、既定の点の大きさに対する倍率であり、例えば ps 1.5 は、既定の約 1.5 倍の大きさを指定する。点の種類として pt 7 を用い、点の大きさを ps 1.5 とするには、次のように指定する。

```
plot "gnuplot01.dat" using 1:2 with points pt 7 ps 1.5
```

さらに、点の色を指定する場合は、lc rgb を用いる。例えば、青色の塗りつぶし円でデータ点を表示するには、次のように指定する。

```
plot "gnuplot01.dat" using 1:2 with points pt 7 ps 1.5 lc rgb "blue"
```


主な点の種類の指定例を表 123 に示す。ただし、pt で指定した番号に対して実際に表示される点の形は、使用している出力端末によって多少異なる場合がある。使用中の出力端末で利用できる点の種類は、gnuplot の test コマンドで確認することができる。

表 123: pointtype による点の種類指定の例

指定	点の種類	例	指定	点の種類	例
pt 0	小さな点	•	pt 7	塗りつぶし円	●
pt 1	プラス記号	+	pt 8	三角形	△
pt 2	バツ印	×	pt 9	塗りつぶし三角形	▲
pt 3	アスタリスク	*	pt 10	逆三角形	▽
pt 4	四角形	□	pt 11	塗りつぶし逆三角形	▼
pt 5	塗りつぶし四角形	■	pt 12	ひし形	◇
pt 6	円	○	pt 13	塗りつぶしひし形	◆

なお、pt 0 で表示される小さな点は、ps を指定しても大きさが変わらない場合がある。

先に示したプログラム gnuplot01.c で作成したデータを、マーカープロットとして描画する例を示す。ただし、個々のマーカールを見やすくするため、データ点の個数を 20 として実行する。

例. gnuplot01 (コンパイル済み) で 20 個のデータ点を持つファイルを作成する

\$./gnuplot01 20 gnuplot01_p20.dat ←実行

この処理の結果、20 個の x の値と、それぞれに対応する $\sin(x)$, $\cos(x)$ の値を格納したファイル gnuplot01_p20.dat が作成される。このデータを次に示すスクリプトファイル gnuplot01_p20.plt でプロットし、結果を EPS 形式の画像ファイル gnuplot01_out_p20.eps として出力する。

スクリプト: gnuplot01_p20.plt

```

1  reset
2  # 3.2インチ×1.8インチサイズのEPSに描画する設定
3  set terminal postscript eps color size 3.2in,1.8in
4  set output "gnuplot01_out_p20.eps"          # 出力ファイルの指定
5  set xlabel "x"                             # 横軸ラベルの設定
6  set ylabel "y"                             # 縦軸ラベルの設定
7  set title "y = sin(x), y = cos(x)"          # グラフタイトルの設定
8  set grid                                   # グリッドの表示
9  set xzeroaxis lt -1 lw 1                   # 横軸の表示
10 set yzeroaxis lt -1 lw 1                   # 縦軸の表示
11 set xrange [-6.28:6.28]                   # 横軸の範囲
12 set yrange [-1.2:1.2]                     # 縦軸の範囲
13 # 描画処理の実行
14 plot "gnuplot01_p20.dat" using 1:2 with points \
15      pt 9 ps 1.5 lc rgb "red" title "sin(x)",\
16      "gnuplot01_p20.dat" using 1:3 with points \
17      pt 10 ps 1.5 lc rgb "blue" title "cos(x)"
18 set output                                # 出力ファイルを閉じる
19 set terminal qt                            # 出力を画面表示に戻す設定

```

gnuplot のプロンプトで

load "gnuplot01_p20.plt"

を実行すると、図 17 のような EPS 形式の画像ファイルが作成される。

F.6.2 棒グラフ

これまで、データ点を線で結ぶ折れ線グラフと、データ点を記号で示すマーカープロットについて説明した。棒グラフの場合も、基本的な考え方はこれらと同じである。すなわち、データファイルの各行を 1 つのデータ点として読み込み、using によって横軸と縦軸に対応する列を指定する。その上で、plot コマンドの描画オプションとして with boxes を指定すると、データが棒として描画される。

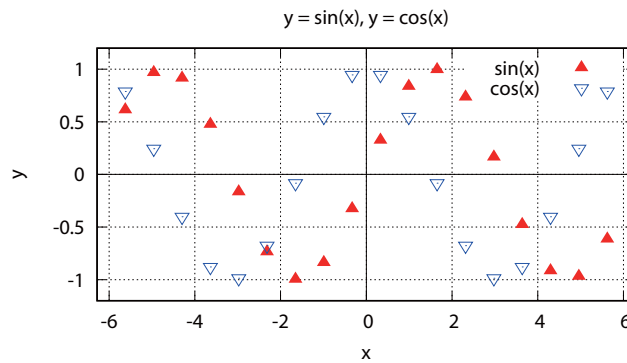


図 17: gnuplot01_p20.plt の実行により作成された gnuplot01_out_p20.eps

例えば、データファイル gnuplot01.dat の第 1 列を横軸、第 2 列を縦軸として、棒グラフを作成するには、次のように指定する。

```
plot "gnuplot01.dat" using 1:2 with boxes
```

この場合、using 1:2 によって第 1 列を横軸、第 2 列を縦軸として読み込む点は、折れ線グラフやマーカープロットの場合と同じである。ただし、with lines や with points ではなく with boxes を指定しているため、各データ点は棒として描画される。通常、各棒は $y = 0$ を基準として縦軸の値まで描かれ、縦軸の値が負の場合には、 $y = 0$ から下向きに伸びる。

棒グラフでは、各棒の幅を boxwidth によって指定することができる。例えば、横軸の座標単位で棒の幅を 0.4 に設定するには、次のように指定する。

```
set boxwidth 0.4
```

これに対して、自動的に求められる棒の幅を基準とした比率で指定する場合には、次のように relative を付ける。

```
set boxwidth 0.4 relative
```

また、棒の塗りつぶし方は set style fill によって指定する。例えば、棒を塗りつぶし、その濃さを 0.7 とし、境界線を黒色で描くには、次のように指定する。

```
set style fill solid 0.7 border lc rgb "black"
```

ここで、solid 0.7 は塗りつぶしの濃さを指定している。値を 1.0 に近づけると指定した色に近い濃さで塗りつぶされ、値を小さくすると背景色に近い薄い色になる。これは透過性を指定するものではない。また、border lc rgb "black" は、棒の境界線を黒色で描く指定である。

棒の色を指定する場合は、折れ線グラフやマーカープロットの場合と同様に、lc rgb を用いる。例えば、赤色の棒で描画するには、次のように指定する。

```
plot "gnuplot01.dat" using 1:2 with boxes lc rgb "red" title "sin(x)"
```

棒グラフでは、横軸上の各位置に棒を立てるため、データ点の個数が多すぎると棒が密集して見づらくなることがある。従って、ここでは先に示したプログラム gnuplot01.c を用い、データ点の個数を少なくして棒グラフを作成する例を示す。

例. gnuplot01 (コンパイル済み) で 17 個のデータ点を持つファイルを作成する

```
$ ./gnuplot01 17 gnuplot01_p17.dat [Enter] ←実行
```

この処理によって、17 個の x の値と、それぞれに対応する $\sin(x)$, $\cos(x)$ の値を格納したファイル gnuplot01_p17.dat が作成される。このデータを次に示すスクリプトファイル gnuplot01_bar.plt でプロットし、結果を EPS 形式の画像ファイル gnuplot01_bar.eps として出力する。

スクリプト: gnuplot01_bar.plt

```
1 reset
2 # 3.2インチ×1.8インチサイズのEPSに描画する設定
3 set terminal postscript eps color size 3.2in,1.8in
4 set output "gnuplot01_bar.eps" # 出力ファイルの指定
5 set xlabel "x" # 横軸ラベルの設定
6 set ylabel "y" # 縦軸ラベルの設定
7 set title "Bar graph of y = sin(x)" # グラフタイトルの設定
```

```

8 | set grid                                # グリッドの表示
9 | set xzeroaxis lt -1 lw 1               # 横軸の表示
10| set yzeroaxis lt -1 lw 1               # 縦軸の表示
11| set xrange [-6.28:6.28]                # 横軸の範囲
12| set yrange [-1.2:1.2]                  # 縦軸の範囲
13| set boxwidth 0.4                       # 棒の幅
14| set style fill solid 0.7 border lc rgb "black" # 棒の塗りつぶしと枠線の設定
15| # 描画処理の実行
16| plot "gnuplot01_p17.dat" using 1:2 with boxes \
17|     lc rgb "red" title "sin(x)"
18| set output                             # 出力ファイルを閉じる
19| set terminal qt                         # 出力を画面表示用の端末に戻す

```

gnuplot のプロンプトで

```
load "gnuplot01_bar.plt"
```

を実行すると、図 18 のような EPS 形式の画像ファイルが作成される。

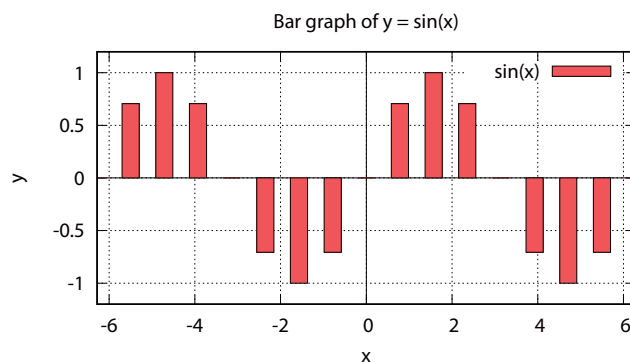


図 18: gnuplot01_bar.plt の実行により作成された gnuplot01_bar.eps

この例では、gnuplot01_p17.dat の第 1 列を横軸、第 2 列を縦軸として読み込み、 $\sin(x)$ の値を棒グラフとして描画している。正の値は横軸から上向きに、負の値は下向きに描かれている。なお、17 個のデータ点のうち、 $\sin(x) = 0$ となる 5 点では棒の高さも 0 となるため、図中では棒として見えない。

また、この例では `set boxwidth 0.4` によって棒の幅を指定し、

```
set style fill solid 0.7 border lc rgb "black"
```

によって、棒を塗りつぶし、境界線を黒色で表示している。棒の幅や塗りつぶしの濃さを変更すると、グラフの見やすさを調整することができる。

F.6.3 散布図

これまでに説明した折れ線グラフ、マーカープロット、棒グラフでは、主に「横軸の値に対して縦軸の値がどのように変化するか」を表示してきた。例えば、関数の値や時系列的なデータを表示する場合には、横軸の値の順序に意味があり、データ点を線で結んだり、棒で表示したりすることで、変化の様子を読み取ることができる。

これに対して、**散布図**は、2 つの数値データの組を平面上の点として表示するグラフである。各組は 1 つの点に対応し、第 1 の値を横軸、第 2 の値を縦軸として配置される。散布図では、点を線で結ぶことを目的とするのではなく、点の広がり方や偏り、あるいは 2 つの値の間に関係があるかどうかを調べることを目的とする。

gnuplot で散布図を作成する方法は、先に説明したマーカープロットとほぼ同じである。すなわち、データファイルの第 1 列を横軸、第 2 列を縦軸として読み込み、`with points` を指定して点として描画する。例えば、データファイル data.dat の第 1 列と第 2 列を用いて散布図を作成するには、次のように指定する。

```
plot "data.dat" using 1:2 with points
```

この指定は、コマンドの形だけを見るとマーカープロットの場合と同じである。しかし、散布図では、各行の 2 つの値を 1 組の観測値として扱い、点群の分布や 2 つの値の関係を読み取る点に特徴がある。従って、散布図は、マーカープロットの描画方法を利用して作成できるが、グラフの読み取り方は異なる。

点の種類や大きさ、色は、マーカープロットの場合と同様に、`pt`, `ps`, `lc rgb` によって指定できる。例えば、青色の小さな点で散布図を描画するには、次のように指定する。

```
plot "data.dat" using 1:2 with points pt 7 ps 0.4 lc rgb "blue"
```

データ点の個数が多い散布図では、点が重なりやすいため、`ps` を小さめに指定すると点群の広がりが見やすくなる。また、点が多数重なる散布図では、アルファ値を含む色指定を用いて半透明の点として描画すると、点が密集している部分が濃く表示される。従って、点群の密度や分布の偏りを視覚的に把握しやすくなる。例えば、半透明の青系の色で点を描画するには、次のように指定する。

```
plot "data.dat" using 1:2 with points pt 7 ps 0.4 lc rgb "#802060ff"
```

ここで、先頭の 80 は約 50 % の透過性を表し、続く 2060ff は青系の色を表す RGB 成分である。このとき、アルファ値を含む色指定が有効になるかどうかは、使用している出力端末に依存する。透過性を利用したい場合は、`pngcairo`、`pdfcairo`、`epscairo`、`svg` などの透過性に対応した出力端末を用いる。

以下に、乱数のプロットを例に挙げて散布図の作成について解説する。次に示すプログラム `gnuplot02.cpp` は、正規分布 $N(0, 2^2)$ および $N(1, 1^2)$ に従う乱数と、0 以上 1 未満の一樣乱数を生成するものである。

プログラム：gnuplot02.cpp

```
1  #include    <iostream>
2  #include    <fstream>
3  #include    <random>
4  #include    <iomanip>
5  #include    <cstdlib>
6
7  int main( int ac, char **av ) {
8      if ( ac != 3 ) {
9          std::cerr << "usage: gnuplot02 NumOfPoints OutFileName" << std::endl;
10         return 1;
11     }
12
13     int n = std::strtol(av[1], NULL, 10);
14     if ( n < 1 ) {
15         std::cerr << "NumOfPoints must be positive." << std::endl;
16         return 1;
17     }
18
19     const unsigned int seed = 12345;          // 乱数の種を固定する
20     std::mt19937 mt(seed);                   // 乱数エンジン
21     // 分布オブジェクト
22     std::normal_distribution<double> dist1(0.0, 2.0); // N(0, 2^2)に従う乱数
23     std::normal_distribution<double> dist2(1.0, 1.0); // N(1, 1^2)に従う乱数
24     std::uniform_real_distribution<double> dist3(0.0, 1.0); // 0以上1未満の一樣乱数
25
26     std::ofstream fout(av[2]);
27     if ( !fout ) {
28         std::cerr << "ファイルを作成できませんでした。" << std::endl;
29         return 1;
30     }
31
32     double x, y, z;
33
34     for ( int i = 0; i < n; i++ ) {
35         x = dist1(mt);      y = dist2(mt);      z = dist3(mt); // 乱数生成
36         fout << std::fixed << std::setprecision(6)
37             << std::setw(12) << x << "\t"
38             << std::setw(12) << y << "\t"
39             << std::setw(12) << z << "\n";
40     }
41
42     fout.close();
43 }
```

このプログラムは、実行時の第 1 引数に生成するデータ点の個数を、第 2 引数に出力ファイルの名前を取る。このプログラムをコンパイルして実行する例を次に示す。

例. `gnuplot02.cpp` のコンパイルと実行

```
$ g++ gnuplot02.cpp -o gnuplot02 Enter ←コンパイル
$ ./gnuplot02 1000 random1000.dat Enter ←データ点の個数を 1000, 出力ファイル名を"random1000.dat"として実行
```

この処理によって、random1000.dat というデータファイルが作成され、1000 個のデータ点が格納される。このファイルの第 1 列には正規分布 $N(0, 2^2)$ に従う乱数、第 2 列には正規分布 $N(1, 1^2)$ に従う乱数、第 3 列には 0 以上 1 未満の一様乱数が記録されており、各列はタブ文字で区切られている。また、このプログラムでは乱数の種を固定しているため、同じ環境で実行すると同じ乱数列が生成される。

このデータを次に示すスクリプトファイル gnuplot02.sct.plt でプロットし、結果を PDF ファイル gnuplot02.sct.pdf として出力する。

スクリプト：gnuplot02.sct.plt

```
1 reset
2 set encoding utf8
3 # PDF形式で散布図を描画する設定
4 set terminal pdfcairo color size 4in,3in
5 set output "gnuplot02.sct.pdf"           # 出力ファイルの指定
6 set xlabel "x"                          # 横軸ラベルの設定
7 set ylabel "y"                          # 縦軸ラベルの設定
8 set title "Scatter plot of random data"  # グラフタイトルの設定
9 set grid                                # グリッドの表示
10 set xzeroaxis lt -1 lw 1                # 横軸の表示
11 set yzeroaxis lt -1 lw 1                # 縦軸の表示
12 # 描画処理の実行
13 plot "random1000.dat" using 1:2 with points \
14      pt 7 ps 0.7 lc rgb "#aa2060ff" notitle
15 set output                               # 出力ファイルを閉じる
16 set terminal qt                           # 出力を画面表示用の端末に戻す
```

gnuplot のプロンプトで

```
load "gnuplot02.sct.plt"
```

を実行すると、図 19 のような散布図が PDF ファイル gnuplot02.sct.pdf として出力される。

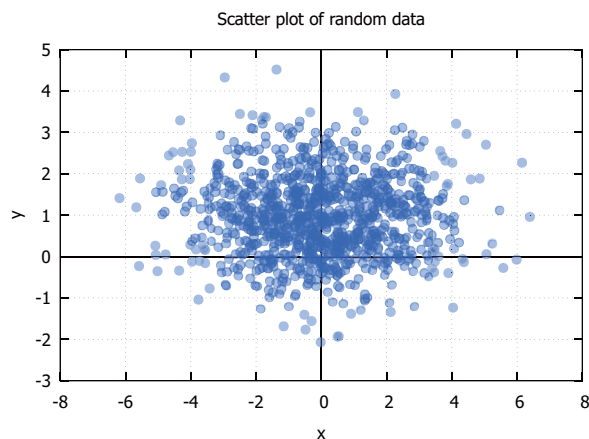


図 19: gnuplot02.sct.plt の実行により作成された散布図

この例では、データファイル random1000.dat の第 1 列を横軸、第 2 列を縦軸として使用している。第 1 列の乱数は標準偏差が 2、第 2 列の乱数は標準偏差が 1 であるため、点群は横方向に広がった形となり、その中心は概ね (0, 1) となる。また、これらの乱数は互いに独立に生成されているため、2 つの値の間に明確な関係は見られない。点を半透明にしているため、多くの点が重なる部分ほど濃く表示され、点群の密度を視覚的に把握しやすい。なお、第 3 列の一様乱数は、この散布図では使用していない。

F.6.4 ヒストグラム

データの分布を調べるためによく用いられるグラフに、ヒストグラムがある。ヒストグラムは、数値データをいくつかの区間に分け、それぞれの区間に含まれるデータの個数を棒の高さとして表示するグラフである。このとき、データを分ける区間を**階級**、各階級に含まれるデータの個数を**度数**と呼ぶ。

先に説明した棒グラフでは、データファイルに与えられた値をそのまま棒の高さとして描画した。これに対して、ヒストグラムでは、個々のデータ値をそのまま棒として描くのではなく、データを階級ごとに分類し、各階級に含まれるデータの個数を集計して棒として描画する。従って、ヒストグラムでは、データの値そのものではなく、データが

どの範囲にどれくらい集中しているかを読み取ることができる。

gnuplot でヒストグラムを作成する方法の一つとして、データを階級ごとに分類する関数を定義し、smooth freq を用いて度数を集計する方法がある。例えば、階級幅を 0.5 とする場合、次のように階級幅を表す変数と、階級を求める関数を定義する。

```
bin_width = 0.5
bin(x) = bin_width * floor(x / bin_width) + bin_width / 2.0
```

ここで、bin_width は階級幅を表す変数である。また、bin(x) は、値 x が属する階級の中央の値を返す関数である。従って、この関数によって、描画時には個々のデータ値が、対応する階級の中央の値に変換される。

例えば、階級幅を 0.5 とすると、階級の境界は $\dots, -1.0, -0.5, 0.0, 0.5, 1.0, \dots$ のように並ぶ。これに対して、棒が描かれる位置は各階級の中央であり、 $\dots, -1.25, -0.75, -0.25, 0.25, 0.75, 1.25, \dots$ となる。このとき、値 0.13 は 0.0 以上 0.5 未満の階級に入り、この階級の中央である 0.25 の位置に集計される。また、値 -0.31 は -0.5 以上 0.0 未満の階級に入り、この階級の中央である -0.25 の位置に集計される。

このようにして、同じ階級に属するデータ値は、描画時に同じ階級の中央の値へ変換される。この例では、その変換結果を smooth freq によって集計し、各階級の度数を求める。従って、ヒストグラムでは、各階級の中央の位置に棒が描かれ、その棒の高さがその階級に含まれるデータの個数を表す。

度数を集計してヒストグラムを描画するには、plot コマンドで次のように指定する。

```
plot "random1000.dat" using (bin($1)):(1.0) smooth freq with boxes
```

この指定では、random1000.dat の第 1 列の値を \$1 として参照し、bin(\$1) によって各データを階級に分類している。また、(1.0) は、各データ点に縦軸の値として 1.0 を与える指定である。smooth freq を指定すると、同じ横軸値を持つデータの縦軸値が合計される。この例では、各データ点に 1.0 を与えているため、その合計が各階級の度数となる。最後の with boxes は、集計された度数を棒として表示するための指定である。なお、freq は frequency の省略形である。

次に、先に作成した乱数データファイル random1000.dat の第 1 列を用いて、正規乱数のヒストグラムを作成する例を示す。このデータの第 1 列は $N(0, 2^2)$ に従う乱数である。次に示すスクリプトファイル gnuplot02_hst.plt は、この第 1 列の値を階級ごとに集計し、ヒストグラムとして描画するものである。

スクリプト：gnuplot02_hst.plt

```
1  reset
2  set encoding utf8
3  # EPS形式でヒストグラムを描画する設定
4  set terminal postscript eps color size 3.2in,1.8in
5  set output "gnuplot02_hst.eps" # 出力ファイルの指定
6  set xlabel "value" # 横軸ラベルの設定
7  set ylabel "frequency" # 縦軸ラベルの設定
8  set title "Histogram of N(0, 2^2)" # グラフタイトルの設定
9  set grid ytics # 横方向のグリッドの表示
10
11 bin_width = 1 # 階級幅
12 bin(x) = bin_width * floor(x / bin_width) + bin_width / 2.0
13
14 set boxwidth bin_width # 棒の幅
15 set style fill solid 0.7 border lc rgb "black" # 棒の塗りつぶしと枠線の設定
16
17 set xrange [-9:9] # 横軸の範囲
18 set yrange [0:]* # 縦軸の範囲
19
20 # 描画処理の実行
21 plot "random1000.dat" using (bin($1)):(1.0) smooth freq with boxes \
22     lc rgb "red" title "N(0,2^2)"
23
24 set output # 出力ファイルを閉じる
25 set terminal qt # 出力を画面表示用の端末に戻す
```

このスクリプトでは、bin_width を 1 として、階級幅を 1 に設定している。また、set boxwidth bin_width * 1 によって棒の幅を階級幅と同じにし、隣り合う階級の棒が接するようにしている。

gnuplot のプロンプトで

```
load "gnuplot02_hst.plt"
```

を実行すると図 20 のようなヒストグラムが作成される。

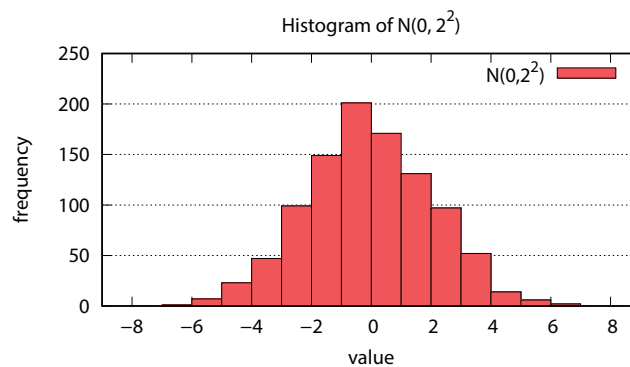


図 20: gnuplot02_hst.plt の実行により作成されたヒストグラム

図 20 では、平均である 0 付近の階級に多くのデータが集まり、そこから左右に離れるほど度数が小さくなっていることが分かる。これは、平均付近の値が多く出現し、平均から離れた値ほど出現しにくいという正規分布の特徴を表している。

なお、ヒストグラムの見え方は、階級幅や表示範囲の取り方によって変化する。階級幅を小さくすると細かい分布の変化が見える一方で、棒の高さのばらつきが大きくなることがある。逆に、階級幅を大きくすると、全体の傾向は見やすくなるが、細かい分布の様子は見えにくくなる。従って、ヒストグラムを作成する際には、データの個数や分布の広がりに応じて、適切な階級幅を選ぶ必要がある。

更に、同じデータファイル random1000.dat の第 3 列を用いて、一様乱数のヒストグラムを作成する例を示す。このデータの第 3 列は、0 以上 1 未満の一様乱数である。次に示すスクリプトファイル gnuplot02_hst2.plt は、この第 3 列の値を階級ごとに集計し、ヒストグラムとして描画するものである。

スクリプト：gnuplot02_hst2.plt

```
1  reset
2  set encoding utf8
3  # EPS形式で一様乱数のヒストグラムを描画する設定
4  set terminal postscript eps color size 3.2in,1.8in
5  set output "gnuplot02_hst2.eps"          # 出力ファイルの指定
6
7  set xlabel "value"                      # 横軸ラベルの設定
8  set ylabel "frequency"                  # 縦軸ラベルの設定
9  set title "Histogram of uniform random numbers" # グラフタイトルの設定
10 set grid ytics                          # 横方向のグリッドの表示
11
12 bin_width = 0.125                       # 階級幅
13 bin(x) = bin_width * floor(x / bin_width) + bin_width / 2.0
14
15 set boxwidth bin_width * 0.9             # 棒の幅
16 set style fill solid 0.7 border lc rgb "black" # 棒の塗りつぶしと枠線の設定
17
18 set xrange [-0.1:1.1]                   # 横軸の範囲
19 set yrange [0:170]                      # 縦軸の範囲
20
21 # 描画処理の実行
22 plot "random1000.dat" using (bin($3)):(1.0) smooth freq with boxes \
23      lc rgb "blue" title "uniform [0,1)"
24
25 set output                               # 出力ファイルを閉じる
26 set terminal qt                           # 出力を画面表示用の端末に戻す
```

このスクリプトでは、階級幅を 0.125 とし、0 以上 1 未満の範囲を 8 つの階級に分けている。また、set boxwidth bin_width * 0.9 によって、棒の幅を階級幅の 90 % に設定し、隣り合う棒の間にわずかな隙間を設けている。

gnuplot のプロンプトで

```
load "gnuplot02_hst2.plt"
```

を実行すると、図 21 のようなヒストグラムが EPS 形式の画像ファイル `gnuplot02_hst2.eps` として出力される。

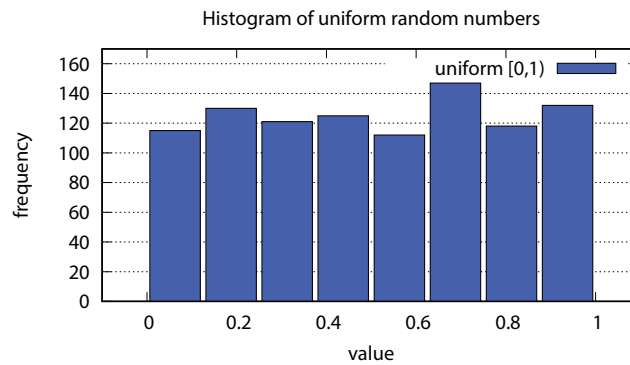


図 21: `gnuplot02_hst2.plt` の実行により作成された一様乱数のヒストグラム

図 21 では、各階級の度数がおおむね同程度になっていることが分かる。これは、一様乱数では、同じ幅を持つ各階級に値が入る確率が等しいためである。ただし、乱数の個数は有限であるため、各階級の度数が完全に等しくなるわけではない。

図 20 の正規乱数のヒストグラムと、図 21 の一様乱数のヒストグラムを比較すると、分布の形状が明確に異なることが分かる。正規乱数では、平均付近の階級に多くのデータが集まり、平均から離れるほど度数が小さくなる。これに対して、一様乱数では、範囲内の各階級におおむね同程度のデータが含まれる。このように、ヒストグラムを用いると、数値データの分布の特徴を視覚的に確認することができる。

F.6.5 3次元プロット（ワイヤーフレーム）

2変数関数の値を視覚的に調べる方法として、3次元プロットがある。その代表的な表示方法の1つが、曲面を線の格子として表示するワイヤーフレームである。ワイヤーフレームでは、 xy 平面上の格子点に対して z の値を計算し、それらの点を線で結ぶことで、関数の概形を立体的に表示する。

ここでは、次の2変数関数を例に考える。

$$z = \cos\left(\sqrt{x^2 + y^2}\right)$$

この関数は、原点からの距離 $\sqrt{x^2 + y^2}$ に応じて z の値が変化する関数である。従って、同じ半径の円周上では同じ値を取り、原点を中心とする同心円状の変化を持つ。このような関数は、3次元の起伏を認識しやすい例である。

gnuplot で3次元データを描画するには、`plot` ではなく `splot` コマンドを用いる。例えば、データファイルの第1列を x 、第2列を y 、第3列を z として3次元的に描画するには、次のように指定する。

```
splot "data.dat" using 1:2:3 with lines
```

ここで、`using 1:2:3` は、第1列、第2列、第3列をそれぞれ x, y, z の値として用いる指定である。また、`with lines` は、隣り合うデータ点を線で結んで表示する指定である。格子状に並んだ3次元データに対してこの指定を用いると、ワイヤーフレームとして曲面を表示することができる。

次に、この関数のデータ点をC言語で生成するプログラムを示す。このプログラムは、 x と y の値を一定間隔で変化させながら、それぞれの格子点に対する $z = \cos(\sqrt{x^2 + y^2})$ の値を計算し、`gnuplot03.dat` に出力する。

プログラム：gnuplot03.c

```
1  #include    <stdio.h>
2  #include    <math.h>
3
4  int main( ) {
5      const double xmin = -4.4;
6      const double xmax =  4.4;
7      const double ymin = -4.4;
8      const double ymax =  4.4;
9      const double h = 0.2;
10
11      FILE      *fp;
12      double    x, y, z;
13
14      fp = fopen("gnuplot03.dat", "w");
```

```

15     if ( fp == NULL ) {
16         fprintf(stderr, "ファイルを作成できませんでした. \n");
17         return 1;
18     }
19
20     for ( y = ymin; y < ymax; y += h ) {
21         for ( x = xmin; x < xmax; x += h ) {
22             z = cos(sqrt(x*x + y*y));
23             fprintf(fp, "%8.4f\t%8.4f\t%8.4f\n", x, y, z);
24         }
25         fprintf(fp, "\n");
26     }
27
28     fclose(fp);
29 }

```

このプログラムでは、 y の値を外側のループで変化させ、その各値に対して x の値を内側のループで変化させている。従って、同じ y の値に対して x を変化させた点列が、ワイヤフレームにおける x 方向の 1 本の線に対応する。

また、内側の x 方向のループが終わるたびに、

```
fprintf(fp, "\n");
```

によって空行を出力している。gnuplot の 3 次元データファイルでは、1 つの空行は点列の区切りとして扱われる。従って、`splot` でデータ点を線で結んで描画する場合、空行の直前と直後にあるデータ点は、同じ点列の続きとしては線で結ばれない。今回の例では、1 つの y の値に対する x 方向の点列を出力し終えたところで空行を入れることで、次の y の値に対する点列を別の点列として扱っている。空行を入れない場合、ある y の値に対する最後の点と、次の y の値に対する最初の点が線で結ばれ、意図しない折り返し線が描かれることがある。

さらに、空行で区切られた各点列に同じ個数の点が含まれている場合、gnuplot はそれらを格子状のデータとして解釈する。このデータを `splot` と `with lines` によって描画すると、各点列のデータ点が順に結ばれるだけでなく、隣り合う点列の対応する点どうしも線で結ばれる。今回のデータでは、同じ y の値に対して x を変化させた点列が x 方向の線として描かれ、さらに、隣り合う y の値に対する点列の同じ位置の点どうしが結ばれることで、 y 方向の線も描かれる。従って、データファイルでは x 方向の点列を順に出力しているが、gnuplot 側ではワイヤフレーム全体が描画される。

このプログラムをコンパイルして実行する例を次に示す。

例. gnuplot03.c のコンパイルと実行

```

$ gcc gnuplot03.c -o gnuplot03 -lm  ←コンパイル220
$ ./gnuplot03  ←実行

```

これにより、3 列のデータファイル `gnuplot03.dat` が作成される。このファイルの第 1 列は x 、第 2 列は y 、第 3 列は z の値である。

次に、このデータファイルをワイヤフレームとして描画するスクリプトファイル `gnuplot03_3w.plt` を示す。

スクリプト: gnuplot03_3w.plt

```

1  reset
2  # EPS形式で3次元ワイヤフレームを描画する設定
3  set terminal postscript eps color size 3.2in,2.0in
4  set output "gnuplot03_3w.eps" # 出力ファイルの指定
5
6  set xlabel "x" # x軸ラベルの設定
7  set ylabel "y" # y軸ラベルの設定
8  set zlabel "z" # z軸ラベルの設定
9  set title "z = cos(sqrt(x^2 + y^2))" # グラフタイトルの設定
10 set grid # グリッドの表示
11 set xrange [-4.4:4.4] # x軸の範囲
12 set yrange [-4.4:4.4] # y軸の範囲
13 set zrange [-1.0:1.0] # z軸の範囲
14 set xyplane at -1.0 # xy平面をz軸の下限に合わせる
15
16 set view 60, 20 # 視点の設定

```

²²⁰環境によってはリンクオプション `-lm` を必要としない場合もある。

```

17 set hidden3d                                # 隠線処理
18 unset key                                    # 凡例を表示しない
19
20 # 描画処理の実行
21 splot "gnuplot03.dat" using 1:2:3 with lines \
22      lt 1 lw 0.5 lc rgb "black"
23 set output                                    # 出力ファイルを閉じる
24 set terminal qt                               # 出力を画面表示用の端末に戻す

```

このスクリプトでは、`set view`によって3次元グラフを見る方向を指定している。また、`set hidden3d`によって隠線処理を有効にしている。これにより、手前の曲面に隠れる線分が表示されなくなり、ワイヤーフレームの形状が見やすくなる。

gnuplotのプロンプトで

```
load "gnuplot03_3w.plt"
```

を実行すると、図 22 のようなワイヤーフレームが EPS 形式の画像ファイル `gnuplot03_3w.eps` として出力される。

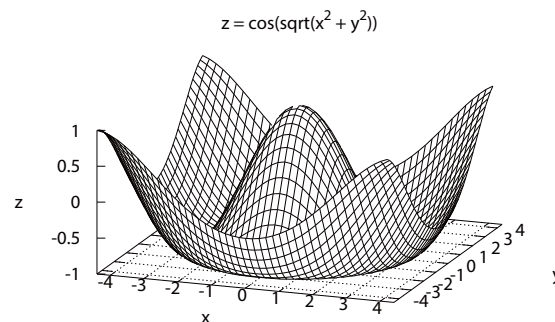


図 22: `gnuplot03_3w.plt` の実行により作成された 3 次元ワイヤーフレーム

図 22 では、原点を中心として、曲面に同心円状の起伏が生じている様子が分かる。これは、この関数の値が $\sqrt{x^2 + y^2}$ ，すなわち原点からの距離によって決まるためである。このように、3 次元ワイヤーフレームを用いると、2 変数関数の概形を立体的に確認することができる。

注意 格子状のデータをワイヤーフレームとして描画する場合は、各点列に含まれるデータ点の個数（上のプログラムでは、同一の y に対する x のデータの個数）をそろえ、それぞれを同じ順序で出力する必要がある。gnuplot は、隣り合う点列の同じ順番の点どうしを対応する点として扱う。点列ごとのデータ点の個数が異なると、点列間を結ぶ線が描画されず、格子状の曲面にならない場合がある。また、点の順序がそろっていない場合には、対応関係が崩れ、意図しない線が描かれることがある。従って、通常は各 y の値に対して、同じ個数の x の値を同じ順序で出力する。

F.6.6 ヒートマップ

前節では、2 変数関数の値を 3 次元ワイヤーフレームとして表示する方法を説明した。3 次元ワイヤーフレームでは、 xy 平面上の格子点に対する z の値を高さとして表示する。これに対して、 z の値を高さではなく色で表し、平面上に表示する可視化手法をヒートマップという。

ヒートマップでは、 xy 平面上のそれぞれの位置における z の値を、色の違いとして表現する。このとき、数値と色との対応関係をカラーマップという。gnuplot では、この対応関係を主に `palette` によって設定する。従って、3 次元の曲面を立体的に表示するのではなく、平面上の色の分布として関数の値の変化を確認できる。これは、温度分布、濃度分布、ポテンシャル分布、画像データなどを表示する際によく用いられる方法である。

ここでも、前節と同じ関数

$$z = \cos\left(\sqrt{x^2 + y^2}\right)$$

を例に用いる。この関数は、原点からの距離 $\sqrt{x^2 + y^2}$ によって値が決まるため、ヒートマップとして表示すると、原点を中心とする同心円状の色の変化が現れる。

gnuplot でヒートマップを作成するには、`splot` コマンドと `pm3d` を用いる。例えば、データファイルの第 1 列を x 、第 2 列を y 、第 3 列を z として、 z の値を色で表示するには、次のように指定する。

```
splot "data.dat" using 1:2:3 with pm3d
```


ここで, using 1:2:3 は, 第1列, 第2列, 第3列をそれぞれ x, y, z の値として用いる指定である. また, with pm3d は, 格子状の3次元データを色付きの面として描画する指定である.

ヒートマップとして表示する場合は, 3次元の斜め方向から見るのではなく, xy 平面を上から見た表示にする. このために, 次の指定を用いる.

```
set view map
```

set view map は, 3次元データを z 軸方向から xy 平面に投影して表示するための指定である. この状態で splot コマンドに with pm3d を指定すると, z の値が色に変換され, 平面上の色の分布として描画される.

さらに, 色と z の値の対応を示すために, カラーバーを表示することができる.

```
set colorbox
```

カラーバーは, グラフ中の色がどの z の値に対応するかを示すものである. また, 色に対応させる z の値の範囲を明示する場合は, 次のように指定する.

```
set cbrange [-1.0:1.0]
```

今回の関数 $z = \cos(\sqrt{x^2 + y^2})$ の値は -1 以上 1 以下であるため, set cbrange [-1.0:1.0] として, 色に対応させる値の範囲をこの関数の値域に合わせている.

次に, 前節で作成したデータファイル gnuplot03.dat を用いて, ヒートマップを作成するスクリプトファイル gnuplot03_cm.plt を示す.

スクリプト: gnuplot03_cm.plt

```
1  reset
2  # EPS形式でカラーマップを描画する設定
3  set terminal postscript eps color size 3.2in,2.8in
4  set output "gnuplot03_cm.eps"          # 出力ファイルの指定
5
6  set xlabel "x"                        # x軸ラベルの設定
7  set ylabel "y"                        # y軸ラベルの設定
8  set title "Heat map of z = cos(sqrt(x^2 + y^2))"  # グラフタイトルの設定
9  set xrange [-4.4:4.4]                 # x軸の範囲
10 set yrange [-4.4:4.4]                 # y軸の範囲
11 set cbrange [-1.0:1.0]                # 色で表す値の範囲
12
13 set view map                           # xy平面を上から見る表示にする
14 set colorbox                           # カラーバーを表示する
15 set size ratio -1                     # x軸とy軸の比率をそろえる
16
17 # カラーマップの色設定
18 set palette                            # 標準のカラーマップ
19 #set palette gray                      # モノクロのカラーマップ
20 # 描画処理の実行
21 splot "gnuplot03.dat" using 1:2:3 with pm3d notitle
22 set output                             # 出力ファイルを閉じる
23 set terminal qt                         # 出力を画面表示用の端末に戻す
```

このスクリプトでは, set size ratio -1 によって, x 軸と y 軸を同じ縮尺で表示するように指定している. これにより, 原点を中心とする円形の色分布が, 縦横に引き伸ばされずに表示される.

gnuplot のプロンプトで

```
load "gnuplot03_cm.plt"
```

を実行すると図 23 の (a) のようなヒートマップが作成される.

図 23 の (a) では, 原点を中心として同心円状に色が変わっていることが分かる. これは, この関数の値が, x と y の組から定まる $\sqrt{x^2 + y^2}$, すなわち原点からの距離だけによって決まるためである. このように, ヒートマップを用いると, 2変数関数の値の変化を平面上の色の分布として確認することができる.

ヒートマップで使用するカラーマップは, gnuplot ではパレットとして扱われ, set palette によって変更できる. 特に指定しない場合は, gnuplot の標準パレットが用いられる. また, 白黒の濃淡 (グレースケール) で表示したい場合は,

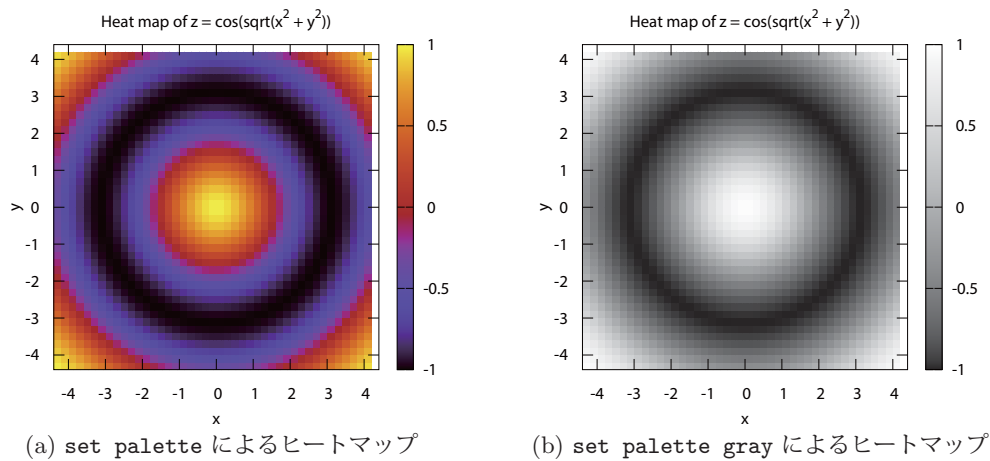


図 23: gnuplot03_cm.plt の実行により作成されたヒートマップ

set palette gray

のように指定する。スクリプト中の set palette の行をコメントにし、続く set palette gray の行を有効にすると、図 23 の (b) のように、 z の値が白黒の濃淡で表される。印刷物や白黒資料で使用する場合には、このようなグレースケール表示が有用である。

この他にも、set palette によって、カラーマップを構成する色の組合せや色の変化の仕方を細かく指定することができる。ただし、パレットの指定方法には多くのバリエーションがあるため、詳しくは gnuplot の公式ドキュメントを参照されたい。

F.6.7 等高線

前節では、2 変数関数の値をヒートマップとして表示する方法を説明した。ヒートマップでは、 xy 平面上の各位置における z の値を色で表した。これに対して、 z の値が同じになる点を線で結ぶ表示方法もある。このような線を等高線と呼ぶ。

等高線は、地図において同じ高さの地点を結ぶ線としてよく知られている。同様に、2 変数関数 $z = f(x, y)$ に対しては、同じ z の値を持つ点を xy 平面上で結ぶことで、関数の値の変化を平面図として表すことができる。等高線を一定の値間隔で描いた場合、等高線どうしの間隔が狭い部分では値が急に変化しており、間隔が広い部分では値の変化が緩やかであると読み取ることができる。

ここでも、前節と同じ関数
$$z = \cos\left(\sqrt{x^2 + y^2}\right)$$

を例に用いる。この関数は、原点からの距離 $\sqrt{x^2 + y^2}$ によって値が決まるため、等高線として表示すると、原点を中心とする同心円状の線が現れる。

gnuplot で等高線を描画するには、splot コマンドに加えて、等高線を表示するための設定を行う。例えば、次のように指定する。

```
set contour base
unset surface
set view map
```

set contour base は、等高線を 3 次元グラフの底面に投影して描くための指定である。unset surface は、3 次元曲面そのものの描画を抑止し、等高線だけを表示するための指定である。また、set view map は、3 次元グラフを z 軸方向から見た平面図として表示するための指定である。

さらに、等高線をどの z の値に対して描くかは、set cntrparam levels によって指定できる。例えば、

```
set cntrparam levels incremental -1.0,0.2,1.0
```

とすると、 -1.0 から 1.0 まで、 0.2 間隔で等高線を描く。今回の関数 $z = \cos(\sqrt{x^2 + y^2})$ の値は -1 以上 1 以下であるため、この指定によって値域全体にわたる等高線を描くことができる。

等高線に色を付け、線の色と z の値を対応させるには、描画時に lc palette を指定する。例えば、

```
splot "data.dat" using 1:2:3 with lines lc palette
```

のように指定すると、等高線の色が z の値に応じて変化する。このとき、色と値の対応を示すために、次のようにカラーバーを表示すると分かりやすい。

```
set colorbox
set cbrange [-1.0:1.0]
```

`set colorbox` は、色と値の対応を表すカラーバーを表示する指定である。また、`set cbrange [-1.0:1.0]` は、色で表す値の範囲を -1.0 から 1.0 までに設定する指定である。これにより、等高線の色がどの z の値に対応するかを読み取ることができる。

次に、前節までと同じデータファイル `gnuplot03.dat` を用いて、等高線を作成するスクリプトファイル `gnuplot03_cnt.plt` を示す。

スクリプト：gnuplot03_cnt.plt

```
1  reset
2  # EPS形式で等高線を描画する設定
3  set terminal postscript eps color size 3.2in,2.8in
4  set output "gnuplot03_cnt.eps"          # 出力ファイルの指定
5
6  set xlabel "x"                          # x軸ラベルの設定
7  set ylabel "y"                          # y軸ラベルの設定
8  set title "Contour map of z = cos(sqrt(x^2 + y^2))" # グラフタイトルの設定
9  set xrange [-4.4:4.4]                  # x軸の範囲
10 set yrange [-4.4:4.4]                  # y軸の範囲
11 set zrange [-1.0:1.0]                  # z軸の範囲
12 set cbrange [-1.0:1.0]                 # 色で表す値の範囲
13 set size ratio -1                      # x軸とy軸の比率をそろえる
14
15 set view map                            # x-y平面を上から見る表示にする
16 set contour base                        # 等高線を底面に描く
17 unset surface                          # 曲面本体を描画しない
18 unset key                              # 凡例を表示しない
19
20 set colorbox                            # カラーバーを表示する
21 set palette                             # 標準のカラーマップ
22 # set palette gray                      # モノクロのカラーマップ
23
24 set cntrparam levels incremental -1.0,0.2,1.0 # 等高線の値を指定
25
26 # 描画処理の実行
27 splot "gnuplot03.dat" using 1:2:3 with lines \
28      lw 1 lc palette notitle
29 set output                              # 出力ファイルを閉じる
30 set terminal qt                          # 出力を画面表示用の端末に戻す
```

このスクリプトでは、`set palette` による標準のカラーマップを用いて等高線を描画している。また、`set palette gray` を指定すると、白黒の濃淡（グレースケール）による等高線図を作成することもできる。印刷物や白黒資料で使用する場合には、このようなグレースケール表示が利用できる。ただし、白に近い等高線は背景と区別しにくくなる場合があるので注意が必要である。

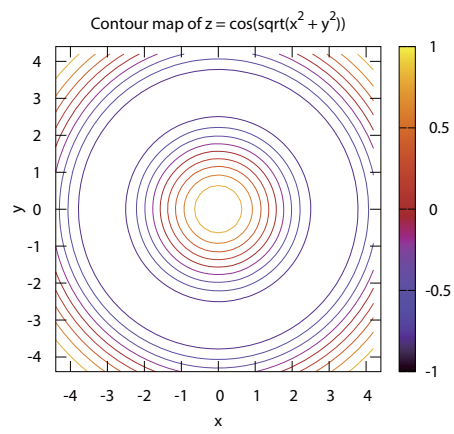
gnuplot のプロンプトで

```
load "gnuplot03_cnt.plt"
```

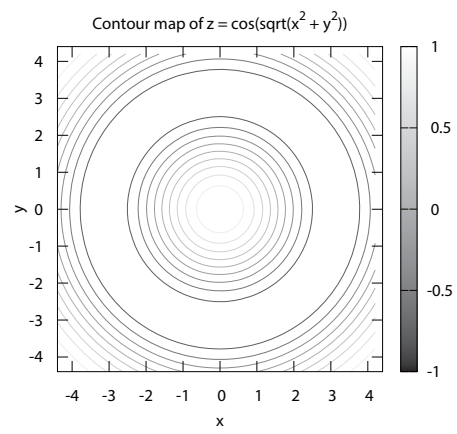
を実行すると図 24 の (a) のような等高線図が作成される。また、`set palette gray` を指定すると、図 24 の (b) のようなモノクロの等高線図が作成される。

図 24 では、原点を中心とする同心円状の等高線が描かれている。これは、この関数の値が、 x と y の組から定まる $\sqrt{x^2 + y^2}$ 、すなわち原点からの距離だけによって決まるためである。また、図 24 の (a) では、等高線の色が z の値に対応しており、右側のカラーバーによって色と値の対応を確認できる。

ワイヤフレームは関数の概形を立体的に確認する方法であり、ヒートマップは値の変化を色の分布として確認する方法である。これに対して、等高線は同じ値を持つ位置のつながりを線として確認する方法である。同じ 3 次元データであっても、表示方法を変えることで、異なる観点からデータの特徴を読み取ることができる。



(a) `set palette` による等高線図



(b) `set palette gray` による等高線図

図 24: `gnuplot03.cnt.plt` の実行により作成された等高線図

参考文献

- [1] ISO/IEC JTC1/SC22/WG21, “ISO/IEC JTC1/SC22/WG21 – The C++ Standards Committee”, <https://www.open-std.org/jtc1/sc22/wg21/>
- [2] Standard C++ Foundation, “Standard C++”, <https://isocpp.org/>
- [3] cppreference.com, “C++ reference”, <https://en.cppreference.com/w/>
- [4] cppreference.com, “Containers library”, <https://en.cppreference.com/w/cpp/container>
- [5] cppreference.com, “Algorithms library”, <https://en.cppreference.com/w/cpp/algorithm>
- [6] cppreference.com, “Pseudo-random number generation”,
<https://en.cppreference.com/w/cpp/numeric/random>
- [7] cppreference.com, “Thread support library”, <https://en.cppreference.com/w/cpp/thread>
- [8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides,
Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Professional, 1994.

索引

*, 23, 58

*/, 5

+, 23

++, 23

-, 23

--, 23

->, 61

., 51

.PHONY, 541

/, 23

/*, 5

//, 5

::, 190, 358

<, 40

<<, 114

<=, 40

=, 13

==, 40

>, 40

>=, 40

>>, 114

#, 98

define, 11, 98

include, 98

%, 23

%A, 35

%LA, 35

%La, 35

%Le, 35

%Lf, 35

%Lg, 35

%X, 35

%a, 35

%b, 36, 511

%c, 35

%d, 35

%e, 35

%f, 35

%g, 35

%hd, 35

%hu, 35

%jd, 35

%ju, 35

%lA, 35

%lX, 35

%la, 35

%ld, 35

%le, 35

%lf, 35

%lg, 35

%llX, 35

%lld, 35

%llo, 35

%llu, 35

%llx, 35

%lo, 35

%lu, 35

%lx, 35

%o, 35

%p, 35

%s, 35

%u, 35

%x, 35

%zd, 35

%zu, 35

&, 57, 114, 204

& &, 40, 99

_Bool, 41

_Complex, 29

_MSC_VER, 507

_M_ARM, 507

_M_ARM64, 507

_M_IX86, 507

_M_X64, 507

_WIN32, 507

_WIN64, 507

__APPLE__, 507

__GNUC_MINOR__, 509

__GNUC_PATCHLEVEL__, 509

__GNUC__, 507, 509

__LP64__, 507

__STDC_HOSTED__, 508

__STDC_VERSION__, 506

__STDC__, 506

__aarch64__, 507

__arm__, 507

__clang__, 507

__clang_major__, 509

__clang_minor__, 509

__clang_patchlevel__, 509

- __cplusplus, 506
- __declspec(dllexport), 553
- __declspec(dllimport), 554
- __i386__, 507
- __linux__, 507
- __x86_64__, 507
- | , 114
- ~, 114, 245
- ^ , 114
- || , 40, 99
- !, 40, 99
- !=, 40
- ¥ ¥, 37
- ¥ ', 37
- ¥ 0, 37
- ¥ n, 37
- ¥ t, 37
- 0B, 9
- 0X, 9
- 0b, 9
- 0x, 9
- 16 進浮動小数点数, 10
- 1 バイトの出力, 67
- 2 進数表記, 10, 511
- 2 の補数, 18, 114
- abort, 168
- abs, 434
- accumulate, 426
- acos, 26, 436
- acosh, 26, 436
- all_of, 402
- any_of, 402
- API, 255
- Arch Linux, 534
- arg, 434
- array, 390
- ASCII, 33, 412
- asin, 26, 436
- asinh, 26, 436
- at, 400
- atan, 26, 436
- atan2, 26
- atanh, 26, 436
- atomic, 353
- auto, 215
- Autotools, 533
- back, 400

- bad, 184
- begin, 387, 400
- bernoulli_distribution, 425
- binomial_distribution, 425
- BOM, 529
- bool, 41, 192
- boolalpha, 193
- boxwidth, 575
- break, 45, 46
- Build Tools for Visual Studio, 526
- build.ninja, 548
- busy waiting, 162
- byte, 196
- C++, 171
- C++言語リンクージ, 377
- cabs, 44
- canonicalize, 455, 456
- capacity, 400
- cbegin, 389
- ccomplex, 174
- ceil, 26
- cend, 389
- cerr, 175
- char, 14
- chi_squared_distribution, 425
- Chromium, 548
- cimag, 29
- cin, 175
- cl, 527
- Clang, 517, 522
- clang, 522
- clang++, 522
- CLANG64, 533
- clear, 185, 400
- clog, 175
- close, 92
- CMake, 533, 535, 542
- CMakeLists.txt, 543
- complex, 29, 433
- complex.h, 174
- conj, 434
- const, 13, 247
- const_cast, 203
- const_iterator, 388
- constexpr, 219, 331
- const メンバ関数, 247
- const 参照, 209

- continue, 47
- copy, 402
- cos, 26, 436
- cosh, 26, 436
- count, 402
- count_if, 402
- cout, 175
- CPU, 2
- CPU アーキテクチャ, 506
- creal, 29
- CSV, 565
- Cygwin, 532
- C 言語リンクージ, 377

- DBL_EPSILON, 20
- DBL_MAX, 20
- DBL_MIN, 20
- DBL_TRUE_MIN, 20
- delete, 273
- deque, 393
- detach, 345
- Developer Command Prompt, 526
- discrete_distribution, 425
- DLL, 551
- do, 48
- double, 14
- double-ended queue, 393

- ECMAScript, 415
- empty, 385, 400
- end, 387, 400
- enum, 144
- enum class, 335
- EOF, 67, 69
- eof, 184
- erase, 392, 400
- EUC-JP, 412
- exchange, 355
- exit, 166
- exp, 26, 436
- expired, 296
- explicit, 243
- exponential_distribution, 425
- extern, 96

- F, 9
- f, 9
- fabs, 26
- fail, 184
- false, 192
- fclose, 74
- FE_DOWNWARD, 26
- FE_TONEAREST, 26
- FE_TOWARDZERO, 26
- FE_UPWARD, 26
- fegetround, 27
- feof, 77
- ferror, 87
- fesetround, 27
- fetch_add, 355
- fetch_sub, 355
- fflush, 76
- fgetc, 77
- fgets, 70
- FILE, 67
- find, 394, 402, 407
- find_first_not_of, 407
- find_first_of, 407
- find_if, 402
- find_last_not_of, 407
- find_last_of, 407
- fisher_f_distribution, 425
- float, 14
- floor, 26
- FLT_EPSILON, 20
- FLT_MAX, 20
- FLT_MIN, 20
- FLT_TRUE_MIN, 20
- fmax, 26
- fmin, 26
- fmod, 26
- fopen, 74
- for, 48, 217
- forward**, 324
- fprintf, 68, 75
- fputc, 75
- fputs, 75
- fread, 79
- friend, 251, 257
- friend クラス, 257
- front, 400
- fscanf, 77
- fseek, 83
- fstream, 179
- fsync, 91
- ftell, 84
- fwrite, 79

- g++, 517
- GCC, 3, 517
- gcc, 6, 517
- geometric_distribution, 425
- get, 186
- get_mpz_t, 462
- get_prec, 458
- getline, 186
- gets, 71
- GMP, 437, 438
- gmp_asprintf, 448
- gmp_fprintf, 448
- gmp_printf, 447, 448
- gmp_randclass, 462
- gmp_randclear, 452
- gmp_randinit_default, 452
- gmp_randinit_mt, 452
- gmp_randseed, 452
- gmp_randseed_ui, 452
- gmp_randstate_t, 451
- gmp_snprintf, 448
- gmp_sprintf, 448
- gmpxx.h, 454
- gmtime, 155
- GNU make, 537
- GNU Project, 3
- gnuplot, 564
- GNU 拡張, 510
- good, 184
- goto, 50

- has-a, 276
- htonl, 125
- htons, 125
- hypot, 26

- I, 29
- IEEE 754, 122
- if, 39
- ifstream, 178
- ignore, 186
- imag, 433
- inf, 22
- INFINITY, 21
- infinity, 195
- inline, 357, 365
- inline 関数, 365
- inline 変数, 367
- inner_product, 430
- insert, 392, 400
- int, 14
- INT16_MAX, 18
- INT16_MIN, 18
- int16_t, 15
- INT32_MAX, 18
- INT32_MIN, 18
- int32_t, 15
- INT64_MAX, 18
- INT64_MIN, 18
- int64_t, 15
- INT8_MAX, 18
- INT8_MIN, 18
- int8_t, 15
- INT_MAX, 18
- INT_MIN, 18
- INTMAX_MAX, 20
- INTMAX_MIN, 20
- intmax_t, 20
- iomanip, 176
- ios::beg, 181
- ios::binary, 179
- ios::cur, 181
- ios::end, 181
- iostream, 175
- iota, 431
- is-a, 276
- is-a 関係, 261
- is_iec559, 196
- isalnum, 410
- isalpha, 410
- isdigit, 410
- isfinite, 195
- isinf, 43, 195
- islower, 410
- isnan, 43, 195, 514
- isnormal, 22
- ispunct, 410
- isspace, 410
- istringstream, 199
- isupper, 410

- jmp_buf, 150
- join, 343

- kill, 152
- Kitware, 543

- L, 9

l, 9	mmpf_set_ui, 440
LDBL_EPSILON, 20	MPC, 437, 491
LDBL_MAX, 20	mpc_abs, 495
LDBL_MIN, 20	mpc_acos, 500
LDBL_TRUE_MIN, 20	mpc_acosh, 500
left, 176	mpc_add, 497
length, 415	mpc_add_fr, 497
limb, 442	mpc_add_ui, 497
list, 391	mpc_arg, 495
LL, 9	mpc_asin, 500
LLP64, 507	mpc_asinh, 500
llrint, 27	mpc_atan, 500
llround, 27	mpc_atanh, 500
LLU, 9	mpc_clear, 492
LLVM, 517, 522	mpc_conj, 495
LLVM IR, 522	mpc_cos, 500
LLVM ビットコード, 523	mpc_cosh, 500
load, 354	mpc_div, 497
localtime, 155	mpc_div_2si, 497
lock, 296	mpc_div_fr, 497
log, 26, 436	mpc_div_ui, 497
log10, 26, 436	mpc_fma, 497
lognormal_distribution, 425	mpc_fr_div, 497
long, 14	mpc_fr_sub, 497
long double, 14	mpc_free_str, 501
long long, 14	mpc_get_dc, 494
longjmp, 150	mpc_get_ldc, 494
lrint, 27	mpc_get_prec, 492
lround, 27	mpc_get_prec2, 492
LU, 9	mpc_get_str, 501
lvalue, 220	mpc_imag, 495
	mpc_imagref, 495
main, 5, 6, 173	MPC_INEX1, 500
make, 535, 537	MPC_INEX2, 500
Makefile, 535	MPC_INEX_IM, 498
malloc, 141	MPC_INEX_RE, 498
map, 394	mpc_init2, 492
max_element, 402	mpc_init3, 492
max_size, 198	mpc_inp_str, 501
memcmp, 138	mpc_mul, 497
memcpy, 136	mpc_mul_2si, 497
memmove, 136	mpc_mul_fr, 497
memset, 135	mpc_mul_i, 497
Mersenne Twister, 452	mpc_mul_si, 497
min_element, 402	mpc_neg, 497
MinGW, 532	mpc_norm, 495
mingw-w64, 532	mpc_out_str, 501
MINGW64, 533	mpc_real, 495

mpc_realref, 495
 mpc_rnd_t, 493
 MPC_RNDAA, 495
 MPC_RNDDD, 495
 MPC_RNDNN, 493, 495
 MPC_RNDNU, 495
 MPC_RNDUU, 495
 MPC_RNDZZ, 495
 mpc_set, 494
 mpc_set_d, 494
 mpc_set_d_d, 494
 mpc_set_f, 494
 mpc_set_f_f, 494
 mpc_set_fr, 494
 mpc_set_fr_fr, 494
 mpc_set_ld, 494
 mpc_set_ld_ld, 494
 mpc_set_nan, 494
 mpc_set_prec, 492
 mpc_set_q, 494
 mpc_set_q_q, 494
 mpc_set_si, 494
 mpc_set_si_si, 494
 mpc_set_str, 501
 mpc_set_ui, 494
 mpc_set_ui_ui, 494
 mpc_set_z, 494
 mpc_set_z_z, 494
 mpc_sin, 500
 mpc_sin_cos, 500
 mpc_sinh, 500
 mpc_sqr, 497
 mpc_strtoc, 501
 mpc_sub, 497
 mpc_sub_fr, 497
 mpc_sub_ui, 497
 mpc_swap, 494
 mpc_t, 493
 mpc_tan, 500
 mpc_tanh, 500
 mpc_ui_div, 497
 mpc_ui_sub, 497
 mpf_abs, 444
 mpf_add, 439
 mpf_ceil, 444
 mpf_class, 454, 455
 mpf_clear, 439
 mpf_cmp, 439, 445
 mpf_cmp_d, 445
 mpf_cmp_si, 445
 mpf_cmp_ui, 445
 mpf_cmp_z, 445
 mpf_div, 439
 mpf_fits_slong_p, 445
 mpf_fits_ulong_p, 445
 mpf_floor, 444
 mpf_get_d, 440
 mpf_get_default_prec, 442
 mpf_get_prec, 442
 mpf_get_si, 440
 mpf_get_str, 440
 mpf_get_ui, 440
 mpf_init, 439, 442
 mpf_init2, 442
 mpf_init_set, 441
 mpf_init_set_d, 441
 mpf_init_set_si, 441
 mpf_init_set_str, 441
 mpf_init_set_ui, 441
 mpf_integer_p, 444, 445
 mpf_mul, 439
 mpf_neg, 444
 mpf_out_str, 448
 mpf_pow_ui, 444
 mpf_set, 439
 mpf_set_d, 440
 mpf_set_default_prec, 442
 mpf_set_prec, 442
 mpf_set_prec_raw, 443
 mpf_set_q, 441
 mpf_set_si, 440
 mpf_set_str, 440
 mpf_set_z, 441
 mpf_sgn, 445
 mpf_sqrt, 444
 mpf_sqrt_ui, 444
 mpf_sub, 439
 mpf_t, 438, 455
 mpf_trunc, 444
 MPFR, 437, 463
 mpfr_abs, 468
 mpfr_acos, 478
 mpfr_acosh, 479
 mpfr_acospi, 478
 mpfr_add, 468
 mpfr_add_ui, 468

mpfr_agm, 480	mpfr_free_cache, 482, 491
mpfr_ai, 480	mpfr_free_str, 485
mpfr_asin, 478	mpfr_gamma, 480
mpfr_asinh, 479	mpfr_gamma_inc, 480
mpfr_asinpi, 478	mpfr_get_d, 467
mpfr_asprintf, 484	mpfr_get_default_prec, 465
mpfr_atan, 478	mpfr_get_f, 467
mpfr_atan2, 478	mpfr_get_fft, 467
mpfr_atan2pi, 478	mpfr_get_ld, 467
mpfr_atanh, 479	mpfr_get_prec, 465
mpfr_atanpi, 478	mpfr_get_q, 467
mpfr_beta, 480	mpfr_get_si, 467
mpfr_cbrt, 475	mpfr_get_str, 485
mpfr_ceil, 473	mpfr_get_str_ndigits, 485
mpfr_clear, 465	mpfr_get_ui, 467
mpfr_cmp, 469	mpfr_get_z, 467
mpfr_const_catalan, 482	mpfr_grandom, 487
mpfr_const_euler, 482	mpfr_greater_p, 469
mpfr_const_log2, 482	mpfr_greaterequal_p, 469
mpfr_const_pi, 482	mpfr_hypot, 475
mpfr_cos, 478	mpfr_inf_p, 469, 483
mpfr_cosh, 479	mpfr_init, 465
mpfr_cosp, 478	mpfr_init2, 465
mpfr_cot, 478	mpfr_inp_str, 485
mpfr_coth, 479	mpfr_integer_p, 469
mpfr_csc, 478	mpfr_j0, 480
mpfr_csch, 479	mpfr_j1, 480
mpfr_digamma, 480	mpfr_jn, 480
mpfr_div, 468	mpfr_less_p, 469
mpfr_div_ui, 468	mpfr_lessequal_p, 469
mpfr_eint, 480	mpfr_lgamma, 480
mpfr_equal_p, 469	mpfr_li2, 480
mpfr_erandom, 486	mpfr_lngamma, 480
mpfr_erf, 480	mpfr_log, 476
mpfr_erfc, 480	mpfr_log10, 476
mpfr_exp, 476	mpfr_log1p, 476
mpfr_exp10, 476	mpfr_log2, 476
mpfr_exp2, 476	mpfr_modf, 473
mpfr_exp_t, 463	mpfr_mul, 468
mpfr_expm1, 476	mpfr_mul_ui, 468
mpfr_fac_ui, 480	mpfr_nan_p, 469, 483
mpfr_fits_slong_p, 467	mpfr_neg, 468
mpfr_fits_ulong_p, 467	mpfr_nrandom, 486
mpfr_floor, 473	mpfr_number_p, 469, 483
mpfr_fma, 468	mpfr_out_str, 485
mpfr_fms, 468	mpfr_pow, 475
mpfr_fprintf, 484	mpfr_pow_si, 475
mpfr_frac, 473	mpfr_pow_ui, 475

mpfr_prec_round, 465	mpfr_tan, 478
mpfr_prec_t, 463	mpfr_tanh, 479
mpfr_printf, 484	mpfr_tanpi, 478
mpfr_rec_sqrt, 475	mpfr_trunc, 473
mpfr_regular_p, 469, 483	mpfr_ui_div, 468
mpfr rint, 473	mpfr_ui_pow, 475
mpfr_rnd_t, 463	mpfr_ui_pow_ui, 475
MPFR_RNDA, 474	mpfr_ui_sub, 468
MPFR_RNDD, 474	mpfr_unordered_p, 469, 483
MPFR_RNDN, 464, 474	mpfr_urandom, 486
MPFR_RNDU, 474	mpfr_urandomb, 486
MPFR_RNDZ, 474	mpfr_y0, 480
mpfr_rootn_si, 475	mpfr_y1, 480
mpfr_rootn_ui, 475	mpfr_yn, 480
mpfr_round, 473	mpfr_zero_p, 469, 483
mpfr_roun-even, 473	mpfr_zeta, 480
mpfr_sec, 478	mpfr_zeta_ui, 480
mpfr_sech, 479	mpq_add, 439
mpfr_set, 466	mpq_canonicalize, 442
mpfr_set_d, 466	mpq_class, 454, 455
mpfr_set_default_prec, 465	mpq_clear, 439
mpfr_set_f, 466	mpq_cmp, 439, 445
mpfr_set_ft, 466	mpq_cmp_si, 445
mpfr_set_inf, 466, 483	mpq_cmp_ui, 445
mpfr_set_ld, 466	mpq_cmp_z, 445
mpfr_set_nan, 466, 483	mpq_div, 439
mpfr_set_prec, 465	mpq_equal, 445
mpfr_set_q, 466	mpq_get_d, 440
mpfr_set_si, 466	mpq_get_den, 440, 442
mpfr_set_str, 466	mpq_get_num, 440, 442
mpfr_set_ui, 466	mpq_get_str, 440
mpfr_set_z, 466	mpq_init, 439
mpfr_set_zero, 466, 483	mpq_mul, 439
mpfr_sgn, 469	mpq_out_str, 448
mpfr_signbit, 469, 483	mpq_set, 439
mpfr_sin, 478	mpq_set_den, 442
mpfr_sin_cos, 478	mpq_set_f, 441
mpfr_sinh, 479	mpq_set_num, 442
mpfr_sinh_cosh, 479	mpq_set_si, 440
mpfr_sinpi, 478	mpq_set_str, 440
mpfr_snprintf, 484	mpq_set_ui, 440
mpfr_sprintf, 484	mpq_set_z, 441
mpfr_sqr, 468, 475	mpq_sgn, 445
mpfr_sqrt, 475	mpq_sub, 439
mpfr_sqrt_ui, 475	mpq_t, 438, 455
mpfr_sub, 468	mpz_add, 439
mpfr_sub_ui, 468	mpz_class, 454, 455
mpfr_t, 463	mpz_clear, 439

- mpz_cmp, 439, 444
- mpz_cmp_si, 444
- mpz_cmp_ui, 444
- mpz_even_p, 444
- mpz_fac_ui, 449
- mpz_fits_slong_p, 444
- mpz_fits_ulong_p, 444
- mpz_get_d, 440
- mpz_get_si, 440
- mpz_get_str, 440
- mpz_get_ui, 440
- mpz_init, 439
- mpz_init_set, 441
- mpz_init_set_d, 441
- mpz_init_set_si, 441
- mpz_init_set_str, 441
- mpz_init_set_ui, 441
- mpz_mul, 439
- mpz_mul_ui, 449
- mpz_odd_p, 444
- mpz_out_str, 448
- mpz_perfect_power_p, 444
- mpz_perfect_square_p, 444
- mpz_probab_prime_p, 444
- mpz_rrandomb, 453
- mpz_set, 439
- mpz_set_d, 440
- mpz_set_f, 441
- mpz_set_q, 441
- mpz_set_si, 440
- mpz_set_str, 440
- mpz_set_ui, 440
- mpz_setbit, 452
- mpz_sgn, 444
- mpz_sub, 439
- mpz_t, 438, 455
- mpz_ui_pow_ui, 454
- mpz_urandomb, 452
- mpz_urandomm, 452
- MSVC, 517, 526
- MSYS, 532, 533
- MSYS2, 532
- mt19937, 419
- mutable, 247
- name mangling, 377
- namespace, 190, 191
- NAN, 21, 514
- NaN, 195
- nan, 22
- nanosleep, 159
- nearbyint, 27
- new, 273
- next, 388
- Ninja, 533, 535, 548
- noboolalpha, 193
- noexcept, 224
- none_of, 402
- norm, 434
- normal_distribution, 424, 425
- npos, 407
- NRVO, 249
- ntohl, 125
- ntohs, 125
- NULL, 65
- nullptr, 189, 201
- nullptr_t, 201
- numeric, 426
- numeric_limits, 195
- O_APPEND, 89
- O_CREAT, 89
- O_RDONLY, 89
- O_RDWR, 89
- O_TRUNC, 89
- O_WRONLY, 89
- ODR, 367
- ofstream, 177
- One Definition Rule, 367
- open, 89, 178, 179
- operator(), 251
- ostringstream, 198
- pacman, 533
- pair, 394
- PARI/GP, 438
- partial_sum, 430
- peek, 186
- pointsize, 573
- pointtype, 573
- poisson_distribution, 425
- polar, 434
- pop_back, 400
- pop_front, 400
- position, 415
- POSIX, 532
- pow, 26, 436

- prev, 388
- printf, 5, 7
- private, 239, 262
- protected, 262, 263
- public, 239, 262
- push_back, 385, 392, 400
- push_front, 392, 400
- putchar, 67
- puts, 67

- RAII, 278, 287, 310, 455
- rand, 27
- random_device, 421
- rdstate, 185
- read, 91, 179
- real, 433
- reference, 204
- regex, 415
- regex_match, 415
- regex_replace, 415
- regex_search, 415
- reinterpret_cast, 180, 203
- replace, 402
- reserve, 400
- reset, 293
- resize, 400
- return, 5
- reverse, 402
- rewind, 85
- rfind, 407
- RGB, 568
- RGBA, 568
- right, 176
- rint, 26, 27
- round, 26, 27
- Rule of Five, 301
- Rule of Three, 299
- Rule of Zero, 305
- rvalue, 220
- RVO, 249

- scanf, 73
- SEEK_CUR, 83
- SEEK_END, 83
- SEEK_SET, 83
- seekg, 181
- seekp, 181
- set, 397
- set style fill, 575
- set_difference, 402
- set_intersection, 402
- set_prec, 458
- set_prec_raw, 459
- set_union, 402
- setfill, 176
- setjmp, 150
- setstate, 185
- setw, 175
- shared_ptr, 293
- Shift_JIS, 412
- short, 14
- sig_atomic_t, 148
- SIGABRT, 152
- sigaction, 153
- SIGALRM, 152
- SIGCONT, 153
- SIGFPE, 152
- SIGINT, 153
- SIGKILL, 153
- signal, 148
- signed, 14
- SIGPIPE, 152
- SIGQUIT, 153
- SIGSEGV, 152
- SIGSTOP, 153
- SIGTERM, 153
- SIGUSR1, 153
- SIGUSR2, 153
- sin, 26, 436
- sinh, 26, 436
- size, 385, 400
- size_t, 15
- size_t, 385
- sizeof, 15, 64
- Sleep, 160
- sleep, 159
- sleep_for, 346
- smatch, 415
- snprintf, 128
- sort, 402
- sprintf, 128
- sqrt, 26, 436
- srand, 27
- sregex_iterator, 415, 417
- sscanf, 130
- static, 16, 97, 255
- 静的データメンバ, 369

- static_cast, 202
- std, 190
- std::bad_alloc, 221
- std::chrono, 191
- std::exception, 221
- std::filesystem, 191
- std::invalid_argument, 221
- std::istream, 199
- std::logic_error, 221
- std::ostream, 198
- std::out_of_range, 221
- std::overflow_error, 221
- std::range_error, 221
- std::ranges, 191
- std::runtime_error, 221
- std::string, 197
- std::swap, 206
- std::this_thread, 191
- std::underflow_error, 221
- stdbool.h, 42
- stderr, 67
- STDERR_FILENO, 89
- stdin, 69
- STDIN_FILENO, 89
- stdio.h, 6
- stdout, 67
- STDOUT_FILENO, 89
- STL, 384
 - アルゴリズム, 401
- store, 354
- str, 415
- strcat, 127
- strchr, 127
- strcmp, 127
- strcpy, 127
- streamoff, 182
- streampos, 182
- strftime, 155
- string, 197
- strlen, 34, 127
- strptime, 156
- strstr, 128
- strtod, 131
- strtof, 131
- strtol, 130
- strtold, 131
- strtoll, 130
- strtoul, 130
- strtoull, 130
- struct, 51
- struct tm, 155
- student_t_distribution, 425
- substr, 413
- swap, 206
- switch, 45
- sync_with_stdio, 188
- tan, 26, 436
- tanh, 26, 436
- tellg, 181
- tellp, 181
- this ポインタ, 246
- thread, 342
- tie, 188
- time, 28, 154
- time_t, 154
- to_integer, 196
- tolower, 410
- toupper, 410
- transform, 402
- trivially copyable, 244
- true, 192
- typedef, 38
- U, 9
- UCRT64, 533
- UINT16_MAX, 18
- uint16_t, 15
- UINT32_MAX, 18
- uint32_t, 15
- UINT64_MAX, 18
- uint64_t, 15
- UINT8_MAX, 18
- uint8_t, 15
- UINT_MAX, 18
- UINTMAX_MAX, 20
- uintmax_t, 20, 57
- uintptr_t, 57
- UL, 9
- ULL, 9
- Unicode, 412
- uniform_int_distribution, 425
- uniform_real_distribution, 425
- union, 121
- unique_ptr, 290
- UNIX エポック, 154
- UNIX 時間, 154

unordered_map, 396
unordered_set, 398
unsigned, 14
use_count, 293
using, 190
using namespace, 190
UTC, 155
UTF-8, 412

va_arg, 110
va_end, 110
va_list, 110
va_start, 110
vector, 384
Visual Studio, 526
void, 6

weak_ptr, 295
while, 47
write, 90, 179
WSL2, 532

アクセス権, 90
アクセス指定子, 239
アスタリスク, 58
アセンブラ, 2
アセンブリ言語, 2
値, 394
値オブジェクト, 308
値のカテゴリ, 220
値渡し, 8
アップデート
 MSVC, 531
アドレス, 35
アドレス演算子, 57
アラインメント, 176
アルゴリズム, 384, 401
アルファ値, 568
アロケータ, 384
アロー演算子, 61
アンダーフロー, 21
アーキテクチャ, 2
忙待ち, 162
依存関係, 535
一時オブジェクト, 209, 279
位置独立コード, 559
一様初期化, 241
一様実数分布, 425
一様整数分布, 425
一様乱数, 27, 422
イテレータ, 384, 386
インクリメント, 23
インクルードガード, 100
インスタンス, 276
インスタンス化, 277, 372
インタプリタ, 3
インデックス, 31
インデント, 5
インバリアント, 253, 307
インライン展開, 365
埋め文字, 176
エスケープシーケンス, 34, 37
F 分布, 425
エンコーディング, 33, 412, 519, 529
演算子のオーバーロード, 248
オブジェクト, 173, 237, 276
オブジェクト指向プログラミング, 171, 268, 276
オブジェクトスライシング, 270
オブジェクトの独立性, 238
階級, 578
カイ二乗分布, 425
開発環境, 517
解放関数, 439
拡張クラス, 259
拡張代入, 14
確率密度関数, 424
加重平均, 429
仮数部, 18
仮想関数, 267
仮想機械, 3
仮想デストラクタ, 272
型, 13
型指定子, 13
型修飾子, 13
型推論, 215
型の別名の定義, 38
型変換, 202
カプセル化, 253
可変長の引数, 109
空文, 5
カラーバー, 584
カラーマップ, 583
仮引数, 5, 8
関数, 5
関数オブジェクト, 252, 384
関数シグネチャ, 211
関数テンプレート, 322

関数テーブル, 113
関数のオーバーロード, 171, 211
関数のポインタ, 111
間接演算子, 58
完全修飾名, 190
完全転送, 325
ガードビット, 489
記憶期間, 15
記憶クラス, 13
記憶クラス指定子, 13
幾何分布, 425
基底クラス, 259, 261
キャスト, 25
キャプチャ, 337, 339
キャプチャリスト, 337
協定世界時, 155
共有オブジェクト, 551
共有所有, 278, 284, 285
共用体, 121
虚数単位, 29
虚部, 29, 433
許容誤差, 43
近似比較, 43
キー, 394
偽, 40
擬似ターゲット, 541
疑似乱数, 27, 29, 419, 451
クラス, 171, 173, 237
クラステンプレート, 326
クラスの定義, 237
クリティカルセクション, 350
クロスコンパイル, 507
グローバル変数, 17
継承, 259
言語規格, 506
言語モード, 506
原子的, 353
公開用ヘッダファイル, 553
高水準言語, 2
構成 (CMake) , 543
構造化プログラミング, 50
構造体, 51
構築, 535
コピーコンストラクタ, 243
コピー初期化, 174
コピー代入演算子, 244
コマンド, 163
コマンドライン引数, 163
コメント, 5
コンストラクタ, 171, 239
コンストラクタのオーバーロード, 241
コンテナ, 384
コンパイラ, 2, 517
コンパイル, 6
コンポジション, 240, 276, 306, 315
コンマ演算子, 49
コードページ, 529
コード領域, 140
再帰関数, 102
再帰的定義, 102
再帰呼び出し, 102
最大幅整数型, 20
最適化, 519
サフィックス, 9
サブクラス, 259
参照, 204
参照の折り畳み, 324
3次元プロット, 581
散布図, 576
シェル, 163
式, 5
識別子, 12
シグナル, 147
シグナルハンドラ, 147
シグナル番号, 148
指示子, 32
指数表現, 18
指数部, 18
指数分布, 425
システムコール, 88
指定子, 32, 52
シャドーイング, 16
終端子, 5
集約型, 390
縮小変換, 174, 242
主値, 436
出力ストリーム, 74
初期化, 13
初期化関数, 439
初期化子, 32, 52
書式指定子, 7
書式整形結果の文字列の作成, 128
書式文字列, 7, 35, 73
真, 40
シンボル, 552
シンボル解決, 552

シーク, 83
シーケンシャルアクセス, 83
シーケンスコンテナ, 386
シード, 419
ジェネレータ, 543
時間計測, 514
字下げ, 5
辞書, 394
実行可能対象, 342
実行形式ファイル, 6
実引数, 5, 8
実部, 29, 433
自動記憶期間, 16
自動変数, 16
寿命, 278
循環参照, 297
純粋仮想関数, 271
自由ソフトウェア, 437
条件演算子, 44
条件コンパイル, 99, 506
条件式, 44
条件判定式, 40
条件分岐, 39
数学的帰納法, 102
スカラ型, 41
スコープ, 15, 279
スコープ解決演算子, 190, 358
スタック, 103
 サイズ, 520, 530
スタックオーバーフロー, 105
スタック領域, 140
Student の t 分布, 425
ストリーム, 67, 175
スマートポインタ, 278, 290
スーパークラス, 259
正規化数, 21
正規表現, 414
正規分布, 424, 425
整数, 18
整数定数式, 46
静的束縛, 268
静的データメンバ, 369
静的変数, 16
静的メンバ, 255
静的領域, 140
精度のビット数, 442
宣言指定子, 13
線種, 569
絶対誤差, 447
前提条件
 make, 538
相対誤差, 447
双方向リスト, 391
双方向連結リスト, 391
添字 (そえじ), 31
添字演算子, 31
ソース外構築, 545
ソース内構築, 548
ソースプログラム, 2, 6
対数正規分布, 425
体ノルム, 434
タイムゾーン, 155
種, 27, 419
多倍長演算, 437
多倍長精度演算, 437
単独所有, 278, 284
ターゲット
 make, 538
ターゲット環境, 506
ターゲットトリプル, 524
代入, 13
ダングリング参照, 280
ダングリングポインタ, 280
逐次処理, 342
中央演算処理装置, 2
中間表現, 522
抽象クラス, 271
直接初期化, 174, 385
追記, 81
追記モード, 81
通常の算術型変換, 25
定義済みマクロ, 506
低水準言語, 2
低水準入出力, 88
定数, 13
テキストファイル, 74
手続き型プログラミング, 171
点グラフ, 573
添字 (てんじ), 31
転送参照, 324
テンプレート, 171, 322
テンプレートパラメータ, 322
テンプレート引数, 323
デクリメント, 23
デザインパターン, 306
デストラクタ, 171, 245

デフォルト引数, 213
データ競合, 348, 349
データメンバ, 395
データモデル, 506
等高線, 585
等値の判定, 42
特殊化, 276
特殊メンバ関数, 298
トリビアルにコピー可能, 244, 354
動的束縛, 267
動的ライブラリ, 551
度数, 578
ドット演算子, 61
内積, 426, 429
内部リンケージ, 97
名前空間, 190
名前修飾, 377, 553
名前の隠蔽, 16, 266
名前の衝突, 190
二項分布, 425
ニモニック, 2
入力ストリーム, 74
ヌルポインタ, 65, 201
ヌル文字, 34, 37
ネットワークバイトオーダー, 125
排他制御, 350
配列, 31, 62
派生クラス, 259, 261
ハッシュ値, 396
範囲に基づく for 文, 217
半角カタカナ, 412
反復, 47
バイエンディアン, 124
バイトオーダー, 124
バイト順, 124
バイト列, 74, 134
バイナリデータ, 74
バイナリファイル, 179
バックエンド, 522
バッファ, 76
バッファリング, 76, 93
パッケージ管理, 534
非型テンプレートパラメータ, 329
引数, 5, 8, 163
非数, 21, 43, 195
非数に関する判定, 43
ヒストグラム, 578
非正規化数, 21

左寄せ, 176
標準エラー出力, 67
標準出力, 67
標準出力ストリーム, 175
標準ストリーム, 175
標準入力, 69
標準入力ストリーム, 175
標準偏差, 427
標準文字列クラス, 197
標準ライブラリ, 172
ヒートマップ, 583
ヒープ領域, 140, 141
ビジーウェイト, 162
ビッグエンディアン, 124
ビット演算, 114
ビットフィールド, 117
ビットマスク, 116
ビルド, 535
ビルドディレクトリ, 546
ファイルスコープ, 17
ファイルディスクリプタ, 88
ファンクタ, 252
フォールスルー, 45
複合代入, 14
複合代入演算子, 249
複合リテラル, 52
複素数, 29, 174, 433
複素数型, 29
符号位置, 412
符号化方式, 412
不定値, 13
浮動小数点数, 18
不変条件, 253
不偏分散, 427
フラグ, 89
フラグメンテーション, 141
フリースタANDING実装, 507
フロントエンド, 522
ブロック, 5
文, 5
分割コンパイル, 94
分散, 427
分布オブジェクト, 419
プリプロセッサ, 6, 98
プリプロセッサディレクティブ, 98
プレフィックス, 9
プログラムの一時停止, 159
プロジェクト, 535

プロセス, 140
プロトタイプ宣言, 7
並行処理, 342
並列実行, 342
ヘッダファイル, 6
変換指定子, 7, 35, 73
変数テンプレート, 334
変数の記憶期間, 15
変数のスコープ, 15
ベルヌーイ分布, 425
ペア, 394
ホスト実装, 507
棒グラフ, 574
ポアソン分布, 425
ポインタ, 35, 57, 58
ポリモーフィズム, 259, 268
マクロ, 12
マニピュレータ, 175
マルチバイト文字, 413
マルチパラダイム言語, 277
マルチプラットフォーム, 3
丸めモード, 26
マーカプロット, 573
右寄せ, 176
未定義動作, 19
無限大, 21, 43, 195
無限大に関する判定, 43
無限ループ, 47
無条件ジャンプ, 50
明示的インスタンス化, 374
命令セット, 2
メインスレッド, 343
メソッドチェーン, 181
メモリ, 57
メモリの初期化, 135
メモリの比較, 138
メモリの複製, 136
メモリリーク, 141, 280
メルセンヌ・ツイスタ法, 419
メンバ, 51
メンバ初期化子リスト, 240
文字, 33
文字定数, 34
文字の検索, 127
文字化け, 413
文字列, 33
文字列からの値の読み取り, 130
文字列の検索, 128
文字列の出力, 67
文字列の長さ, 127
文字列の比較, 127
文字列の複製, 127
文字列の連結, 127
文字列リテラル, 11
戻り値, 5
モード, 74, 90
呼び出し可能対象, 342
読み書きモード, 82
ラベル, 50
ラムダ式, 337
乱数, 27, 419, 451, 486
乱数エンジン, 419
ランダムアクセス, 83
ランダムアクセスイテレータ, 388
離散分布, 425
リスト初期化, 174, 241, 385
リソース管理, 309
リダイレクト, 68, 70
リテラル, 8
リトルエンディアン, 124
両端キュー, 393
臨界区間, 350
リンケージ, 377
累算代入, 14
累積和, 426, 429
例外クラス, 221
例外処理, 221
例外の送出, 222
例外のハンドリング, 222
レシピ
 make, 538
列挙型, 144
連想配列, 394
ロケール, 416
論理型, 192
ローカル時刻, 155
ワイヤフレーム, 581

「C/C++入門」

2026 年 9 月 7 日

第 0.5 版

2026 年 9 月 23 日

第 0.6 版

テキストの最新版と更新情報

本書の最新版と更新情報を，プログラミングに関する情報コミュニティ Qiita で配信しています。



上記 URL の QR コード

本書はフリーソフトウェアです，著作権は保持していますが，印刷と再配布は自由にさせていただいて結構です。（内容を改変せずをお願いします） 内容に関して不備な点がありましたら，是非ご連絡ください。ご意見，ご要望も受け付けています。

● 連絡先

nkatsu2012@gmail.com

中村勝則